unooria®

TECNOLOGIA EDUCACIONAL B2G · PLATAFORMA PÚBLICA INTEGRADA

DOCUMENTAÇÃO TÉCNICA DE ARQUITETURA DE SOFTWARE, ARQUITETURA DE NUVEM E CÓDIGO

Plataforma Unooria

Hub multi-tenant, 48 experiências imersivas e governança por escola

Monorepo de uma plataforma de educação básica para redes públicas de ensino: um *hub* em Next.js que embeda experiências estáticas versionadas em CDN, uma API NestJS com isolamento lógico por escola em *Row-Level Security*, telemetria em dois canais e uma camada de IA generativa com quota por perfil — uma *stack* de nuvem por contrato, cem escolas por *stack*.

DOCUMENTO	**PU-DOC-001 — Documentação Técnica Consolidada**
VERSÃO	**1.4 (Modelagem UML 2.5.1)**
DATA	**09 de setembro de 2026**
AUTOR	**Lucas Salomão**
REPOSITÓRIO	**`plataforma-unooria` (*monorepo*, tronco `main`)**
CLASSIFICAÇÃO	**Documento técnico de projeto — uso interno**

Escopo do documento: contexto de negócio, requisitos, casos de uso, arquitetura de *software* nos quatro níveis do modelo C4, arquitetura de nuvem e implantação em GCP, modelo de *multi-tenancy* e segurança, modelo de dados, contratos de integração, telemetria, camada de IA, catálogo de experiências, especificação da API, *design system*, engenharia de entrega, qualidade, registro de decisões arquiteturais, riscos e evolução. Diagramas vetoriais em TikZ; fontes Mermaid equivalentes no Apêndice A.

Ficha técnica do documento

Identificação

Campo	Conteúdo
Título	Plataforma Unooria — Documentação Técnica Consolidada de Arquitetura de *Software*, Arquitetura de Nuvem e Código
Identificador	PU-DOC-001
Versão	1.4
Data de emissão	27 de agosto de 2026
Autor	Lucas Salomão (<lucas.salomao@unooria.com>)
Repositório	`plataforma-unooria` — *monorepo* pnpm + Turborepo
Tronco	`main` (histórico linear; uma *branch* por experiência)
Linguagem principal	TypeScript 5.7 (*strict*), Node.js 20
Prazo de produção	novembro de 2026
Estado	Em desenvolvimento — M0–M6 concluídos; *stack* de *dev* em GCP operacional
Repositório de infra	Separado (Terraform). Contrato em `docs/INFRA.md`

Escopo e propósito

Este documento consolida, em fonte única, o que estava distribuído entre `AGENTS.md` (as convenções e regras invioláveis do repositório), `README.md`, `docs/ARQUITETURA.md`, `docs/DECISOES.md`, `docs/INFRA.md`, `docs/INFRA-IDENTITY.md` e os comentários de decisão que vivem no próprio código-fonte — que, neste projeto, é onde as decisões mais finas estão registradas.

Acrescenta o material que não existia em forma escrita: especificação de requisitos funcionais e não funcionais rastreáveis, catálogo de casos de uso, modelo C4 completo nos quatro níveis para o *software* e para a nuvem, diagramas de sequência dos contratos de integração, dicionário de dados, especificação da API rota por rota e o registro formal das decisões arquiteturais (ADR) com suas consequências.

> **NOTA**
>
> Este documento é **normativo** para o projeto: quando ele divergir de uma suposição, ele vence. Quando divergir do código, o código vence e o documento está desatualizado — e atualizá-lo faz parte da *Definition of Done* de qualquer tarefa (Capítulo 16).

Histórico de revisões

Versão	Data	Autor	Alterações
0.1	22/08/2026	L. Salomão	Estrutura do *monorepo*; arquitetura e ADRs iniciais em `docs/`.
0.2	25/08/2026	L. Salomão	Migração para o *monorepo* mergeada na `main` (26 *commits*, *fast-forward*); *tenancy* com RLS, exp08ef1b e exp04ef1b.
0.3	26/08/2026	L. Salomão	app-70 (Vila do Lume) e app-91 (bem-estar); decisão de retenção sem prazo declarado.
1.0	**27/08/2026**	**L. Salomão**	**Documentação técnica consolidada: requisitos, casos de uso, modelo C4 de *software* e de nuvem, *multi-tenancy*, modelo de dados, contratos, telemetria particionada, IA, API, *design system*, entrega, qualidade, ADRs e bibliografia em ABNT.**
1.1	01/09/2026	L. Salomão	Capítulo de observabilidade: estado atual por camada, sinais, comparação de ferramentas, SLOs e alertas, privacidade nos sinais, custo por contrato e roteiro em fases; **ADR-57** e **ADR-58** (propostas); `R-05` reclassificado.
1.2	09/09/2026	L. Salomão	Revisão técnica integral e consolidação multi-agente: atualização do modelo de 6 papéis (**ADR-80**), catálogo de 49 tabelas e 30 migrações Prisma, 85 ADRs consolidadas, novo Capítulo 21 de pendências rastreadas, adequação integral da identidade visual e layout ao Manual de Marca oficial (*Brandbook Unooria*).
1.3	09/09/2026	L. Salomão	Terceiro eixo de acesso (*disponibilidade*) e painel de recursos da Unooria: **ADR-87** e **ADR-88**, tabelas `disponibilidades` e `experiencias_versoes`, campo `produto` no manifesto de experiência.

1.4	27/09/2026	L. Salomão	Modelagem UML: os catorze tipos de diagrama da UML 2.5.1 no Capítulo 5 (Seção 5.9), com o perfil da plataforma, conferidos contra o código, e o par Mermaid de cada um (**ADR-122**); casos de uso com os seis papéis; correções que a conferência expôs (grafo do *monorepo*, quota na camada de IA). Acumula as **ADR-89** a **ADR-121**, registradas desde a 1.3 sem revisão de versão.

Documentos de referência do projeto

Artefato	Conteúdo
`AGENTS.md`	Convenções, regras invioláveis de *tenancy* e segurança, contratos de integração e *Definition of Done*. É o documento de governança do repositório.
`README.md`	Instalação, os três modos de execução local e *smoke* visual.
`Manual de Marca`	Manual oficial de identidade visual (*Brandbook Unooria*): paleta canônica BLUNOORIA (Azul Profundo, Lavanda, Azul Claro), proporções e safespace do logotipo e do símbolo infinito orgânico.
`docs/ARQUITETURA.md`	Visão arquitetural resumida por contrato.
`docs/DECISOES.md`	Registro resumido de decisões (base do Capítulo 18).
`docs/INFRA.md`	Contrato entre este repositório e o de Terraform: nomes de recursos, variáveis e o que o CI publica.
`docs/INFRA-IDENTITY.md`	Contrato do gerenciador central de identidade (WSO2 IS), que é único para vários contratos.
`apps/api/prisma/schema.pr`	Modelo de dados anotado, com o porquê de cada decisão de esquema.
`packages/sdk/src/`	Contrato *postMessage* e cliente de telemetria — a fronteira entre hub e experiência.
`.env.example`	Catálogo completo de variáveis de ambiente (Apêndice B).

Sumário

Ficha técnica do documento **i**

Lista de figuras **xxx**

Lista de tabelas **xxxiii**

Lista de siglas e abreviaturas **xxxiv**

1 Introdução **1**

1.1 Propósito deste documento . . . 1

1.2 O problema . . . 1

1.3 A solução em uma página . . . 2

1.4 Como ler este documento . . . 3

1.4.1 Estrutura . . . 3

1.4.2 Convenções tipográficas . . . 4

1.5 O que este documento não cobre . . . 5

2 Contexto de negócio e restrições **6**

2.1 Modelo de negócio . . . 6

2.2 Escala alvo . . . 6

2.3 Restrições de projeto . . . 7

2.4 Os *drivers* arquiteturais . . . 8

2.5 Interessados . . . 9

2.6 As sete regras invioláveis . . . 10

2.7 Fora de escopo, deliberadamente . . . 10

3 Requisitos 12

3.1 Requisitos funcionais 12

3.1.1 Identidade, papéis e acesso 12

3.1.2 Experiências e *embed* 13

3.1.3 Telemetria 14

3.1.4 Inteligência artificial 14

3.1.5 Pedagógico e relatórios 15

3.2 Requisitos não funcionais 17

3.3 Rastreabilidade 18

4 Casos de uso 20

4.1 Atores 20

4.2 Diagrama de casos de uso 21

4.3 Especificação dos casos de uso 22

4.3.1 UC-01 — Autenticar-se 22

4.3.2 UC-03 — Realizar experiência 22

4.3.3 UC-10 — Preparar atividade 24

4.3.4 UC-13 — Ler resultados da turma 24

4.3.5 UC-14 — Gerar plano de aula 25

4.3.6 UC-15 — Fazer *check-in* de bem-estar 26

4.3.7 UC-23 — Publicar manifesto de experiência 27

4.4 Rastreabilidade de casos de uso 27

5 Arquitetura de *software* 30

5.1 Visões arquiteturais 30

5.2 Estilo arquitetural e princípios 31

5.3 C4 nível 1 — Contexto do sistema 32

5.4 C4 nível 2 — Contêineres 32

5.4.1 Catálogo de contêineres e artefatos 33

5.5 C4 nível 3 — Componentes 35

5.5.1 Componentes da API 35

5.5.2 Componentes do hub 39

5.6 C4 nível 4 — Código 41

5.6.1 O contrato da ponte e o *guard* de *tenancy* 41

5.6.2 O gabarito de um módulo de domínio 42

5.7 Estrutura do *monorepo* 43

5.8 Fluxos principais 44

5.8.1 Curadoria de um bloco de *feed* (exp08ef1b) 44

5.8.2 Estados de uma geração assíncrona 44

5.9 Modelagem UML 45

5.9.1 Os catorze diagramas e onde cada um está 46

5.9.2 O perfil da plataforma 48

5.9.3 Visão de cenários: a administração da rede 50

5.9.4 Visão lógica: classes e objetos 52

5.9.5 Visão de desenvolvimento: pacotes, componentes e estrutura composta 56

5.9.6 Visão de processos: atividades, interações e tempo 59

5.9.7 Visão física: a implantação de um contrato 72

6 Arquitetura de nuvem e implantação 73

6.1 Princípios de nuvem 73

6.2 C4 nível 1 de implantação — o contrato 74

6.3 Diagrama de implantação detalhado 74

6.4 Catálogo de recursos por contrato 76

6.5 Caminhos de tráfego 78

6.6 Identidade de carga de trabalho e segredos 80

6.7 Dimensionamento 81

6.7.1 Volumes medidos 81

6.8 Ambientes 82

6.9 O provedor de identidade fora da *stack* 83

6.10 Observabilidade, continuidade e reversão 83

7 *Multi-tenancy* e segurança 85

7.1 Os três níveis de isolamento 85

7.2 Nível 2: *Row-Level Security* no PostgreSQL 85

7.2.1 Como a política é armada 86

7.2.2 A porta única: `comTenancy` 86

7.2.3 Como a escola chega ao contexto 88

7.2.4 Os papéis sem escola fixa: `gestor_contrato`, `admin` e `gestor_regional` 89

7.3 A tabela sem tenant: o cadastro de interesse (**ADR-107**) 89

7.4 A conversa do agente: a posse no Postgres, o conteúdo fora dele 90

7.5 O interruptor da rede: um nível que não é de tenancy 91

7.6 Nível 3: autorização por papel 92

7.6.1 Onde cada papel entra 93

7.6.2 A cadeia de atribuição de papéis 94

7.6.3 A matriz de capacidades 95

7.6.4 As portas que não são papel 99

7.6.5 O desenho por trás 100

7.7 Segurança do *embed* 102

7.8 Segredos e o que nunca vai ao cliente 104

7.9 Privacidade e dados de menores 104

7.9.1 Retenção: uma decisão explícita de não declarar prazo 107

7.9.2 Localidade de processamento 107

7.10 Verificação 108

7.11 Ameaças consideradas 108

8 Modelo de dados 110

8.1 A convenção de nomes 110

8.1.1 A migração de referência, e duas armadilhas registradas 111

8.2 Modelo da plataforma 112

8.3 A tabela que fica fora do modelo: `prospeccao_leads` 115

8.4 A posse das conversas do agente: `agente_professor_conversas` 115

8.5 A rede: escola, lotação e o CPF 115
8.6 Enturmamento e identidade 117
8.6.1 As equipes na missão 118
8.6.2 A evidência pedagógica 119
8.7 Telemetria particionada 120
8.8 Modelo da exp08ef1b (Instanooria) 121
8.9 Modelo das experiências 02, 70, 90 e 91 124
8.10 As cinco tabelas da exp08ef4b 125
8.11 As quatro tabelas da exp05ef1b 126
8.12 As quatro tabelas da exp04ef1b 128
8.13 As quatro tabelas da exp07ef1b 129
8.14 As quatro tabelas da exp06ef2b 130
8.15 As cinco tabelas da exp02em1b 131
8.16 As três tabelas da app-92 132
8.17 Decisões de esquema por experiência 132
8.17.1 app-70 — as decisões que custam informação de propósito 132
8.17.2 app-90 — conteúdo fora do banco 133
8.17.3 app-91 — ausência como decisão de esquema 134
8.18 Migrações 134
8.19 As três tabelas da exp01ef1b 138

9 Contratos de integração 139

9.1 Protocolo `postMessage` hub ↔ experiência 139
9.1.1 Especificação das mensagens (protocolo v1) 139
9.1.2 O *handshake*, passo a passo 141
9.1.3 Regras invioláveis do protocolo 141
9.1.4 Política de evolução 142
9.2 Manifesto de experiências 143
9.2.1 Como o hub resolve a URL do `iframe` 145
9.2.2 Publicação e reversão 146

9.3 Contrato de telemetria 146
9.4 Contrato do *provider* de IA 147
9.5 Contrato do agente de IA do professor 149
9.5.1 O que a api pede ao engine 150
9.5.2 As chaves de estado e as ferramentas 150
9.5.3 O que a api emite ao navegador 151
9.6 Contrato entre duas experiências 152

10 Telemetria 153
10.1 Por que dois canais 153
10.2 O canal analítico, ponta a ponta 155
10.3 A rota de ingestão e sua tranca 155
10.4 Destino: PostgreSQL, não BigQuery 156
10.4.1 `INSERT` multi-linha 156
10.4.2 Partição mensal 157
10.5 Privacidade da telemetria 157
10.6 Comportamento em desenvolvimento 158

11 Camada de inteligência artificial 159
11.1 Onde a IA é usada 159
11.1.1 A diagramação do Jornal da Escola: IA em sala, e só um *enum* 160
11.2 A camada única 162
11.3 Quota por perfil 162
11.4 Credencial: ADC, nunca chave de API 164
11.5 Modelo, região e o custo de escolher 164
11.5.1 Imagem: proporção pedida ao modelo, não recortada depois 164
11.5.2 Voz: a única chamada que não é ao *aiplatform* 165
11.6 Raciocínio conta como saída 166
11.7 *Guardrails* do *check-in* de bem-estar 166
11.8 O segundo *vendor*: o agente de IA do professor 168

11.9 Variáveis de configuração da IA 169

12 Catálogo de experiências 170

12.1 Visão do catálogo 170

12.2 A sala de aula: o que o estudante vê 172

12.3 A sala de aula: o que o professor faz 173

12.4 O guia da aula 173

12.4.1 Blocos vivos: o que não vira texto 174

12.4.2 BNCC Computação 175

12.4.3 Impressão 175

12.4.4 Onde se chega ao guia 175

12.5 exp08ef1b — Instanooria 176

12.6 exp04ef1b — Jornal da Escola: A Edição de Bento 177

12.6.1 O preparo: a estrutura é do servidor, o texto é da IA 179

12.6.2 Ausências deliberadas 179

12.7 backup-exp04ef1b — UnooTube (arquivada) 180

12.8 exp02em1b — unooGPT (Missão Tempestade) 181

12.8.1 A regra de ouro: a Lacuna não cruza a fronteira do `iframe` 182

12.8.2 Três contas que o cliente não faz 182

12.8.3 Ponto cego é ausência, não zero 182

12.8.4 O Relógio da Aula 183

12.8.5 O que o relatório mede do tempo 183

12.8.6 Telas do professor 183

12.8.7 `allow-forms`: o bug que só o *smoke* pega 184

12.8.8 O *smoke* mentiu duas vezes 184

12.8.9 O guia da aula, e a tabela dos vieses 184

12.9 exp01ef1b — O Que a Foto Conta 185

12.10 exp08ef4b — O Manifesto da IA 186

12.10.1 A missão em que não existe resposta certa 186

12.10.2 O portão que a missão existe para instalar 187

12.10.3 A réplica da máquina só existe DEPOIS da pergunta 187

12.10.4 Sem digitação livre, e a ausência é guardrail 187
12.10.5 A IA entra ANTES da aula, e só 188
12.10.6 O Checkpoint A é polling, não WebSocket 188
12.10.7 O único texto livre, e o dado pessoal que é recusado 188
12.10.8 As duas seções do design system, e a fronteira 189
12.10.9 Os áudios ainda não existem 189
12.11 exp05ef1b — Quem mandou isso? (v2.2) 189
12.11.1 O pacote é a porta, e é a atividade 189
12.11.2 A matriz equilibrada, e o formato que não pode ser pista 190
12.11.3 Exatamente três, e três *latches* 190
12.11.4 O gabarito desce UMA vez, e só o das três 191
12.11.5 Três botões, sempre para a frente 191
12.11.6 A régua pedagógica v2: cada habilidade pelo verbo dela 192
12.11.7 Onde a fronteira do disfarce cai — duas vezes 192
12.11.8 Equipes e o Termômetro 192
12.11.9 Sem mídia binária 193
12.12 exp07ef1b — A Seleção 193
12.12.1 O podcast de recapitulação é do professor 194
12.13 exp06ef2b — O Rastro do Boato 195
12.13.1 O portão que a missão existe para instalar 195
12.13.2 As duas aulas, e quem separa uma da outra 196
12.13.3 Encaminhar e calar não dão no mesmo número 196
12.13.4 1.200 nós, e nenhum deles no DOM 196
12.13.5 Zero IA, e a ausência é o desenho 197
12.13.6 Onde a fronteira do disfarce cai 197
12.13.7 O clone é escuro, e o tema claro do PRD cedeu 197
12.14 app-70 — Vila do Lume 198
12.14.1 Duas leituras de uma partida, e elas não se tocam 198
12.14.2 O servidor recalcula; nunca aceita o perfil do cliente 198
12.14.3 `grau` é invariante de API, não regra de tela 198
12.14.4 A bolha é do servidor 199

12.14.5 Telas do professor 199
12.14.6 A exceção de ADR: o jogo em JavaScript clássico 200
12.15 app-90 — Gerador de plano de aula 200
12.16 app-92 — Planejamento de ensino 201
12.16.1 O caminho completo 201
12.16.2 Ela coexiste com a app-90 202
12.16.3 O currículo como dado 203
12.16.4 Componente da rede ≠ componente da Base 203
12.16.5 Divisão de responsabilidade 204
12.16.6 O esqueleto das aulas não vem do modelo 204
12.16.7 Carga horária: hora-aula não é hora-relógio 206
12.16.8 O início do ano letivo é da regência 207
12.16.9 O passo é semanal; o recorte é da entrega 208
12.17 app-91 — Acompanhamento de bem-estar 208
12.18 Como criar uma experiência nova 209

13 Especificação da API 211

13.1 Convenções transversais 211
13.1.1 As três rotas sem `OidcGuard` 212
13.2 Mapa de rotas por área 212
13.3 Superfície que a SDK consome 215
13.4 Grupos de rotas em detalhe 215
13.4.1 Motor de *feed* (exp08ef1b) 215
13.4.2 A sala de aula: a missão dentro da turma 216
13.4.3 As equipes na missão: quem divide um computador 217
13.4.4 A evidência pedagógica: por atividade e rodada 218
13.4.5 Entrada de pessoas: as duas portas, e por que só uma escreve 219
13.4.6 A rede: regionais e escolas 220
13.4.7 Planos de aula (app-90) 221
13.4.8 O agente de IA do professor 221
13.4.9 Planejamento de ensino (app-92) 222

13.4.10 Bem-estar (app-91) 223
13.4.11 Análise 224
13.4.12 O Manifesto da IA (exp08ef4b) 224
13.4.13 Quem mandou isso? (exp05ef1b) 226
13.4.14 Jornal da Escola (exp04ef1b) 227
13.4.15 As rotas que alimentam o guia da aula 230
13.4.16 Observabilidade: os sinais que nascem no navegador 231
13.4.17 `/recursos/*` — o interruptor da rede 232
13.5 Prospecção: a única escrita sem token (**ADR-107**) 234
13.6 Sanidade e prontidão 235
13.7 Documentação executável (OpenAPI) 235

14 *Design system* e camada de interface 237
14.1 Duas famílias no mesmo pacote 237
14.2 A linha que separa as seções é a REVELAÇÃO 239
14.3 A exceção: a peça que imita outra plataforma, dentro da Unooria 239
14.4 Consumo: sempre pela ponte 240
14.5 CSS: uma vez, no ponto de entrada 241
14.5.1 No hub, o CSS não basta: as fontes também 242
14.6 Frames: dois níveis, não empilhar 242
14.7 Tema 243
14.8 Versão: atualizar é ato deliberado 243
14.9 O painel de recursos: três estados, e o rótulo que muda 243
14.10 Lacuna aberta: o `SideBar` traz um item que não é seu 244
14.11 Compensações locais 244
14.11.1 Lacunas abertas, verificadas na 0.1.14 244
14.11.2 Lacunas abertas na 0.1.17 (UnooTube 1.1.0) 245
14.11.3 Lacunas abertas em `unooria` na 0.1.17 (hub, 2026-09-05) 245
14.11.4 Lacuna aberta em `Collapse` na 0.1.19 (app-92, 2026-09-10) 246
14.11.5 Lacunas abertas em `whatsappmobile` na 0.1.17 247

14.11.6 As extensões de `whatsappmobile` na 0.2.4 (exp05ef1b 0.2.0) 248
14.11.7 A segunda leva do `instanooria` e o `AudioPlayer` na 0.2.7 (exp07ef1b 1.0.1) 249
14.11.8 As oito peças da redação na 0.2.8 (exp04ef1b 2.0.0) 250
14.11.9 Lacunas abertas em `chatgpt` na 0.1.19 (exp08ef4b 0.1.0) 251
14.11.10 Avisos que ficaram obsoletos 252
14.12 O chat do agente de IA do professor: as peças da 0.2.10 253
14.13 O menu da conta: o componente que faltava, e a lacuna que ele trouxe 254
14.13.1 Duas lacunas que ficaram sem compensação, por escolha 255
14.14 A barra de aplicação ancora na janela (**ADR-83**) 255
14.15 Cor da plataforma: classe escrita à mão 256
14.16 Rolagem: o `ScrollBar` do DS, em toda área rolável do hub 257
14.16.1 O que sobrou para a compensação local 257
14.16.2 O `autoHide` do pacote é *hover*, não rolagem 258
14.17 O vocabulário das telas administrativas 258
14.18 O idioma da interface: três zonas (**ADR-91**) 259
14.19 Exceção registrada 260

15 Engenharia de entrega 261

15.1 O *monorepo* e o *build* incremental 261
15.2 Integração contínua 262
15.2.1 A trava de vulnerabilidade (Snyk) 263
15.3 Entrega contínua 264
15.3.1 O agente de IA do professor: imagem, `PATCH` e a ordem dos *applies* 264
15.3.2 Sonda, *smoke* e aviso (2026-09-02) 265
15.3.3 O que faz o *deploy* caber no café 266
15.3.4 Nada de nome de recurso no *workflow* 267
15.3.5 Concorrência e reexecução 268
15.4 Ambientes locais 268
15.5 O provedor de identidade sobe à parte 269
15.6 Convenções de *git* 269

16 Qualidade **272**

16.1 A *Definition of Done* 272

16.2 Estratégia de testes 273

16.3 A suíte de ponta a ponta, em detalhe 274

16.3.1 O manifesto: a descrição mora no teste que fotografa 274

16.3.2 Erro provocado não é erro descoberto 275

16.3.3 O roteiro tem de PERDER a corrida para o erro 275

16.3.4 Duas condições para o vermelho significar alguma coisa 276

16.3.5 O que só o *smoke* visual pega 276

16.4 Análise estática 277

16.5 Armadilhas de ferramenta registradas 278

16.6 Cobertura: onde está e onde não está 278

17 Observabilidade **280**

17.1 O que se vê hoje 280

17.2 Princípios 284

17.3 Os sinais, e onde cada um nasce 285

17.3.1 Log estruturado 285

17.3.2 Métricas 286

17.3.3 Rastros distribuídos 289

17.3.4 Erros 290

17.3.5 Experiência do usuário real e sondas sintéticas 291

17.3.6 Banco, mensageria e borda 292

17.3.7 Camada de IA 292

17.3.8 Entrega 293

17.4 Ferramentas: comparação e escolha 294

17.5 SLIs, SLOs e a lista de alertas 296

17.6 Privacidade nos sinais 298

17.7 Custo estimado por contrato 299

17.8 Roteiro de implantação 300

17.9 Estado da implantação (3 de setembro de 2026) 303
17.9.1 As treze políticas que existem 304
17.9.2 Onde a implantação divergiu do desenho, e por quê 306
17.9.3 *Session replay*: instância própria em `dev` 306
17.9.4 Custo do que entrou 307
17.9.5 O que ficou pendente, em ordem de risco 308
17.10 O que fica de fora 308

18 Registro de decisões arquiteturais 309
18.1 Índice 310
18.2 ADR-01 — Experiências como *bundles* estáticos 315
18.3 ADR-02 — *Monorepo* único 316
18.4 ADR-03 — Autenticação por OIDC genérico 317
18.5 ADR-04 — Uma *stack* por contrato; escolas por RLS 317
18.6 ADR-05 — Cloud SQL HA, não AlloyDB 317
18.7 ADR-06 — Pub/Sub para telemetria desde o início 318
18.8 ADR-07 — IA atrás de um *gateway* na própria API 318
18.9 ADR-08 — RLS forçada e uma porta única de banco 319
18.10 ADR-09 — Domínio por experiência na API; gabarito *server-side* 319
18.11 ADR-10 — Bem-estar: instrumento fixo, IA só no entorno, sem gestor 320
18.12 ADR-11 — Telemetria sem PII e vocabulário fechado 320
18.13 ADR-12 — `grau` como invariante de API 321
18.14 ADR-13 — Retenção sem prazo declarado 321
18.15 ADR-14 — Telemetria no PostgreSQL particionado 322
18.16 ADR-15 — O hub assina os *tokens* de experiência 322
18.17 ADR-16 — ADC como credencial; modelo *versus* região 323
18.18 ADR-17 — *Claim* atômico em vez de fila externa 323
18.19 ADR-18 — Exceções registradas 324
18.20 ADR-19 — *Design system* externo como fonte única de interface 324

18.21 ADR-20 — Um banco, um *schema*; prefixo `exp_NN_` 325

18.22 ADR-21 — *Push subscription* para a própria API 325

18.23 ADR-22 — exp04ef1b 1.1.0: *briefing* antes do disfarce 326

18.24 ADR-23 — *Function calling* entra no `AiProvider` 327

18.25 ADR-24 — Teto de custo de IA por SESSÃO 328

18.26 ADR-25 — O Relógio da Aula é tabela da experiência 329

18.27 ADR-26 — Sem prazo de retenção na exp02em1b 330

18.28 ADR-27 — Transcrição é dado do instrumento 331

18.29 ADR-28 — Identidade por pessoa; OneRoster na fronteira 332

18.30 ADR-29 — Senha provisória com troca obrigatória 334

18.31 ADR-30 — A rede: lotação por escola e escola ativa no *token* 335

18.32 ADR-31 — CPF como identidade, guardado só em *hash* 336

18.33 ADR-32 — Transferência: o dado fica, a identidade segue 338

18.34 ADR-33 — O CPF é guardado, e o gestor o vê 339

18.35 ADR-34 — A regional agrupa; quem autoriza é a lotação 340

18.36 ADR-35 — Perfil portátil: um documento assinado, não uma integração 342

18.37 ADR-36 — O OpenAPI é derivado do Zod, não escrito ao lado dele 344

18.38 ADR-37 — O *schema* de resposta é conferido, não só publicado 346

18.39 ADR-38 — O *deploy* paraleliza os *jobs*, não o portão 347

18.40 ADR-39 — Vulnerabilidade de dependência trava o portão, com prazo para os *majors* 348

18.41 ADR-40 — A raiz é a *landing*; a marca vem do DS, não do modelo 349

18.42 ADR-41 — O relatório do estudante sai do aparelho: o iframe expande, não remonta 351

18.43 ADR-42 — "O que o algoritmo escondeu" é a EXPOSIÇÃO, não o domínio 352

18.44 ADR-43 — A resposta do estudante guarda a primeira e a última 353

18.45 ADR-44 — A janela do envolvimento termina com o feed, e o total sai das listagens 354

18.46 ADR-45 — *Agent* e `fetch` vêm do MESMO pacote undici 355

18.47 ADR-46 — O `?hubOrigin=` é entrada não confiável: a experiência confirma quem a embedou 356

18.48 ADR-47 — Nem senha de demonstração nem verificação de TLS moram no fonte . . . 358

18.49 ADR-48 — O teto do CSV é de BYTES e cabe debaixo do limite do ambiente; o modelo do OneRoster vem em pares, e o exemplo leva a escola de verdade . . . 360

18.50 ADR-49 — A atividade do Instanooria declara a TURMA, e o painel dela para de ler a escola inteira . . . 362

18.51 ADR-50 — O perfil importado guarda a PARTIDA e a VERSÃO do mapeamento, e não ganha chave estrangeira . . . 364

18.52 ADR-51 — A suíte de ponta a ponta é organizada por PAPEL, e a descrição de cada tela mora no teste que a fotografa . . . 365

18.53 ADR-52 — Falha do provedor de IA na destilação é 503 com a causa, não 500 mudo . . . 367

18.54 ADR-53 — Um ambiente por *branch*: *environments* do GitHub carregam as `vars`, e a *branch* escolhe a *stack* . . . 368

18.55 ADR-54 — Pessoa nasce por IMPORTAÇÃO: não existe cadastro avulso de usuário . . . 369

18.56 ADR-55 — Instrução de agente: `AGENTS.md` canônico por fronteira, `CLAUDE.md` como ponteiro de uma linha . . . 371

18.57 ADR-56 — O fluxo por ambiente é imposto por *hooks* versionados, não por *branch protection* . . . 372

18.58 ADR-57 — Observabilidade nativa do Google Cloud, com OpenTelemetry como fronteira; nenhuma *stack* auto-hospedada . . . 374

18.59 ADR-58 — Erro de cliente entra pela API; nenhum SDK de terceiro no *bundle* da experiência . . . 377

18.60 ADR-59 — Cadastro unitário existe, e ele DELEGA ao importador; `admin-contrato` tem rota própria . . . 378

18.61 ADR-60 — Microsoft Clarity carrega no hub **e** dentro do *bundle* da experiência . . . 380

18.62 ADR-61 — Trabalho paralelo acontece em *worktree*, com a base escrita à mão . . . 382

18.63 ADR-62 — O WAF não inspeciona o corpo de rota autenticada nem de texto livre de estudante . . . 384

18.64 ADR-63 — O CI não publica a mídia pesada, mas se recusa a publicar sem ela . . . 385

18.65 ADR-64 — O marco da exp02em1b carrega o relógio do instante; a faixa da decisão é gravada, não recomputada . . . 386

18.66 ADR-65 — O guia da aula é dado renderizado por um componente único, com blocos vivos para o que tem fonte de verdade em outro lugar . . . 388

18.67 ADR-66 — A exp07ef1b escreve com IA na tela do estudante, e o gabarito continua fora do modelo . . . 390

18.68 ADR-67 — A mídia da exp07ef1b é gerada por IA, preparada pelo professor e cacheada por contrato . . . 391

18.69 ADR-68 — O clone de WhatsApp da exp07ef1b é escuro, e as quatro peças que faltam são compensação marcada . . . 393

18.70 ADR-69 — A missão vive DENTRO da turma: a sala de aula por liberação . . . 394

18.71 ADR-70 — As páginas legais descrevem o sistema que existe, e a rede é a controladora 396

18.72 ADR-71 — O trabalho paralelo se reparte por fatia vertical, e o runtime continua singleton . . . 398

18.73 ADR-72 — A fronteira do disfarce é a revelação, e a Etapa 4 da exp06ef2b fica fora dele 400

18.74 ADR-73 — O planejamento de ensino é dirigido pela BNCC extraída do PDF homologado, e o componente da rede não é o componente da Base . . . 402

18.75 ADR-74 — `generateJson` informa o que o vendor cobrou por um *callback*, e não pelo retorno . . . 405

18.76 ADR-75 — A rolagem do hub é o `ScrollBar` do *design system* . . . 406

18.77 ADR-76 — O planejamento é semanal, e a quinzena é recorte de exportação; o `iframe` ganha `allow-downloads` por declaração . . . 408

18.78 ADR-77 — A trilha é do *layout* da área, nunca da página . . . 409

18.79 ADR-78 — O *logoff* mora no menu do avatar, não na barra . . . 411

18.80 ADR-79 — Toda escola pertence a uma regional, e a rede nunca fica sem nenhuma . . . 412

18.81 ADR-80 — Seis papéis, com cadeia de atribuição, e o regional que não mexe na RLS . . . 414

18.82 ADR-81 — O panorama da diretoria SOMA, e isso não custou RLS . . . 417

18.83 ADR-82 — O relatório da diretoria atravessa escolas com um LAÇO, não afrouxando a RLS . . . 418

18.84 ADR-83 — O header ancora na JANELA, não na coluna de conteúdo . . . 420

18.85 ADR-84 — Sinal do navegador **sem sessão**, e o aparelho como rótulo fechado . . . 422

18.86 ADR-85 — Trocar de escola **remonta** o miolo da área . . . 423

18.87 ADR-86 — O guia da aula **mostra** o conteúdo da missão, não o descreve . . . 425

18.88 ADR-87 — Disponibilidade: um terceiro eixo, com nome próprio, e o produto como teto 428

18.89 ADR-88 — O painel da Unooria é só do `admin`: a primeira assimetria com o `gestor_contrato` 430

18.90 ADR-89 — O histórico é tabela, não log — e a REDE lê o dela 431

18.91 ADR-90 — A suíte da API roda em UM *worker* 433

18.92 ADR-91 — O idioma da interface tem TRÊS zonas, e o clone não se traduz 435

18.92.1 Emenda de 2026-09-10: acessível não é só *traduzido* 437

18.93 ADR-92 — Voltar um momento REABRE a sessão; a trava é a revelação, não o relatório 437

18.94 ADR-93 — Sincronização de sala por *polling* HTTP curto, não por WebSocket 440

18.95 ADR-94 — O latch de turma anda por POLLING curto, nunca por canal persistente 441

18.96 ADR-95 — A sessão do hub dura 2h de INATIVIDADE, e vale igual para todo papel 443

18.97 ADR-96 — Ficar na missão CONTA como uso, e a sessão que morre tem tela 445

18.98 ADR-97 — A tela diz POR QUE não entrou, e a suíte espera o desfecho 447

18.99 ADR-98 — A plataforma assume UM fuso, numa constante só 449

18.100 ADR-99 — O fuso vem de uma variável de ambiente, em três superfícies 451

18.101 ADR-100 — Da sala para a missão há UM caminho, e ele leva à ficha 454

18.102 ADR-101 — Quatro ambientes, e a frente nasce e volta em `dev-local` 456

18.103 ADR-102 — A doutrina é uma fonte só, lida por quatro agentes 458

18.104 ADR-103 — A frente mergeada se enterra sozinha 460

18.105 ADR-104 — A base se realinha sozinha depois da promoção 462

18.106 ADR-105 — A quota de IA é DADO, editável pela Unooria, e não variável de ambiente 463

18.107 ADR-106 — O `_template` nasce com a fiação inteira e sem componente de interface 465

18.108 ADR-107 — O site público entra no hub como PORTE, e o cadastro de interesse passa pela api 467

18.109 ADR-108 — A missão de sala nasce VAZIA, e a revelação é do relógio da turma 469

18.110 ADR-109 — O rótulo da tabela da BNCC de Computação é CENTRADO, e a atribuição é uma PARTIÇÃO 472

18.111 ADR-110 — O código da experiência diz onde ela mora na trilha 474

18.112 ADR-111 — Equipe é quem divide um computador, e o painel conta por computador 477

18.113 ADR-112 — A evidência é por atividade e rodada, e a melhor rodada vale por habilidade 481

18.114 ADR-113 — A Seleção substitui o Eco no Grupo, e a limpeza vem antes 485

18.115 ADR-114 — Quem mandou isso? v1 é retirada; a v2.2 entra com o mesmo código 487

18.116 ADR-115 — A Seleção: o par do Mural é do computador, e a IA gradua a legenda 490

18.117 ADR-116 — A experiência que perde o código é arquivada, não apagada 491

18.118 ADR-117 — Jornal da Escola: a redação é da Unooria, o professor abre o resultado e a revisão, e a IA só escolhe o layout 494

18.119 ADR-118 — Quem mandou isso? v2.2: atividade é o pacote, rodada é a reaplicação, e a régua mede pela seleção e pelas perguntas do PO 496

18.120 ADR-119 — O podcast da A Seleção é do professor, e a IA reescreve só as três falas que dependem do tema 500

18.121 ADR-120 — A régua v2 de Quem mandou isso?: cada habilidade pelo verbo dela, e refazer o mesmo pacote não mede memória 502

18.122 ADR-121 — O CI verifica só os PRs para dev, hml e main 504

18.123 ADR-122 — A modelagem UML entra no Capítulo 5, com os catorze tipos da UML 2.5.1 e par Mermaid mesmo onde o Mermaid não tem o tipo 505

18.124 ADR-123 — O plano anual tem uma estrutura só, definida pelo *prompt*, e o template é exemplo de formato 507

18.125 ADR-124 — O `CLAUDE.md` admite o bloco que uma ferramenta escreve, e só ele 509

18.126 ADR-125 — O grafo do código nasce copiado em cada *worktree* e se atualiza sozinho no *pull*, e fica fora do *git* 510

18.127 ADR-126 — O agente de IA do professor roda no Vertex AI Agent Engine, atrás da api 511

18.128 ADR-127 — Só *data store*, inclusive para as habilidades da BNCC 514

18.129 ADR-128 — SSE só no chat do agente 516

18.130 ADR-129 — O *markdown* do agente vira HTML no navegador 518

18.131 ADR-130 — A atividade é do professor, e a equipe é da atividade 519

19 Riscos e evolução 521

19.1 Registro de riscos 521

19.2 Estado de implementação 528

19.3 Evolução planejada 528

19.3.1 Antes de novembro de 2026 (bloqueante) 529

19.3.2 Depois, por ordem de retorno 529

19.4 O que continuará fora de escopo 530

20 Conclusão 531

20.1 O que foi construído 531

20.2 As decisões que mais definiram o resultado 531

20.3 O que este projeto ensinou sobre documentação 532

20.4 Estado de prontidão para produção 532

20.5 Encerramento 533

21 Pendências identificadas e roadmap de evolução 534

21.1 Quadro resumo de pendências 534

21.2 Detalhamento técnico das pendências 535

21.2.1 PEND-01 — Transição do emissor de tokens OIDC para WSO2 IS 535

21.2.2 PEND-02 — Testes E2E de autorização regional e troca de contexto 535

21.2.3 PEND-03 — Monitoramento de quotas de IA e telemetria de custos 536

21.2.4 PEND-04 — Portão de validação de integridade do OpenAPI no CI 536

21.2.5 PEND-05 — Verificação estrutural de Row-Level Security em migrações 536

21.3 Cronograma de mitigação e alinhamento com lançamento 537

A Fontes Mermaid dos diagramas 538

A.1 Figura 1.1 — Visão geral da plataforma 538

A.2 Figura 4.1 — Casos de uso 539

A.3 Figura 5.1 — C4 nível 1: contexto 540

A.4 Figura 5.2 — C4 nível 2: contêineres 540

A.5 Figura 5.3 — C4 nível 3: componentes da API 541

A.6 Figura 5.4 — C4 nível 3: componentes do hub 543

A.7 Figura 5.5 — C4 nível 4: tipos que carregam a regra 543

A.8 Figura 5.6 — Grafo de dependências do *monorepo* 544

A.9 Figura 5.7 — Curadoria de um bloco de *feed* 544

A.10 Figura 5.8 — Estados de uma geração assíncrona 545

A.11 Figura 5.9 — Os catorze tipos de diagrama da UML 2.5.1 545

A.12 Figura 5.10 — Perfil UML da plataforma 547

A.13 Figura 5.11 — Casos de uso da administração da rede 548

A.14 Figura 5.12 — Classes: o modelo de domínio da plataforma 549

A.15 Figura 5.13 — Classes: a porta da IA 551

A.16 Figura 5.14 — Objetos: um instantâneo da disponibilidade 553

A.17 Figura 5.15 — Pacotes: quem pode importar quem 554

A.18 Figura 5.16 — Componentes e interfaces 555

A.19 Figura 5.17 — Estrutura composta: a página da missão 556

A.20 Figura 5.18 — Atividades: o resolvedor de disponibilidade 557

A.21 Figura 5.19 — Sequência: a entrada do estudante na missão 558

A.22 Figura 5.20 — Sequência: uma chamada de IA com quota 559

A.23 Figura 5.21 — Sequência: um turno do agente de IA do professor 560

A.24 Figura 5.22 — Comunicação: o caminho de um lote de telemetria 561

A.25 Figura 5.23 — Tempo: *token*, renovação e sessão 561

A.26 Figura 5.24 — Máquina de estados de protocolo: a ponte 562

A.27 Figura 5.25 — Visão geral de interação: a jornada do estudante 563

A.28 Figura 5.26 — Implantação: que artefato roda em que nó 564

A.29 Figura 6.1 — Implantação de um contrato 565

A.30 Figura 6.2 — Implantação detalhada 566

A.31 Figura 6.3 — Identidade de carga de trabalho 567

A.32 Figura 7.1 — Os três níveis de isolamento 567

A.33 Figura 7.2 — Propagação da escola 567

A.34 Figura 8.1 — Modelo de dados da plataforma 568

A.35 Figura 8.2 — Enturmamento e sala de aula 569

A.36 Figuras 8.3 e 8.4 — Modelo da exp08ef1b 570

A.37 Figuras 8.5 e 8.6 — Demais experiências 571

A.38 Figura 9.1 — *Handshake* hub-experiência 572

A.39 Figura 9.2 — Resolução da URL do *embed* 573

A.40 Figura 9.3 — Contrato entre duas experiências 573

A.41 Figura 10.1 — Canal analítico 573

A.42 Figura 11.1 — Camada de IA 574

A.43 Figura 11.2 — *Guardrails* do bem-estar 574

A.44 Figura 12.1 — Os quatro graus de medida 575

A.45 Figura 12.2 — app-92: o caminho do planejamento 575

A.46 Figura 12.3 — app-92: como uma semana nasce 576

A.47 Figuras 13.1 e 13.2 — Mapa de rotas da API 577

A.48 Figura 14.1 — A linha da revelação 578

A.49 Figura 15.1 — *Pipeline* de entrega 579

A.50 Figura 16.1 — Estratégia de testes 579

A.51 Figura 17.1 — Cadeia de uma requisição e o que cada elo revela 580

A.52 Figura 17.2 — Erro de cliente entra pela API 580

A.53 Figura 19.1 — Matriz de riscos 580

B Variáveis de ambiente 582

B.1 API — `apps/api/.env` 582

B.1.1 Banco e identidade 582

B.1.2 Inteligência artificial 583

B.1.3 Agente de IA do professor (**ADR-126**) 584

B.1.4 Armazenamento 585

B.1.5 Telemetria e chaves de serviço 585

B.1.6 Fuso da plataforma (**ADR-99**) 586

B.1.7 Observabilidade (Capítulo 17) 587

B.2 Hub — `apps/hub/.env.local` 588

B.3 Experiências — variáveis de *build* do Vite 589

B.4 Agente de IA do professor — `apps/agente-professor` 590

B.5 `vars` do repositório (CI/CD) 591

B.6 Configuração mínima para subir sem GCP 592

C Catálogo de *endpoints* 593

C.1 a-selecao 593

C.2 agente-professor 605

C.3 ai 607

C.4 analise 607

C.5 atividades 608

C.6 auth 613

C.7 bemestar 614

C.8 equipes 615

C.9 escolas 619

C.10 estudantes 623

C.11 evidencias 624

C.12 experiencias 627

C.13 foto-conta 628

C.14 healthz 632

C.15 jornal-escola 633

C.16 lotacoes 646

C.17 manifesto-ia 646

C.18 matriculas 654

C.19 observabilidade 654

C.20 perfis 656

C.21 perfis-recebidos 656

C.22 periodos-letivos 657

C.23 planejamento 658

C.24 planos 661

C.25 posts 663

C.26 progresso 664

C.27 prospeccao 664
C.28 quem-mandou-isso 666
C.29 rastro-boato 675
C.30 readyz 679
C.31 recursos 679
C.32 rostering 683
C.33 runs 685
C.34 salas 688
C.35 telemetry 689
C.36 turmas 690
C.37 unoogpt 691
C.38 usuarios 695
C.39 vila 697

D *Payloads* e esquemas de validação 701

D.1 Protocolo `postMessage` (v1) 701
D.2 *Token* de experiência 702
D.3 Telemetria — canal analítico 703
D.4 Sinais de observabilidade do navegador 704
D.5 app-70 — o *payload* de uma partida 705
D.5.1 As decisões que cada campo carrega 706
D.5.2 As duas travas estruturais 707
D.5.3 O cap de eventos, e por que ele caiu de 2000 para 200 708
D.6 Agente de IA do professor — o *stream* e o engine 708
D.7 Manifesto de experiência 710
D.8 Validação de variáveis de ambiente 711
D.9 Importação de usuários por CSV 711

E Dicionário de dados 713

E.1 Índice de tabelas 713

E.2 Tabelas da plataforma 716
E.2.1 escolas 716
E.2.2 perfis_portateis 717
E.2.3 periodos_letivos (catálogo) 718
E.2.4 turmas 719
E.2.5 matriculas 719
E.2.6 liberacoes 720
E.2.7 equipes_montagens 720
E.2.8 equipes 721
E.2.9 equipes_membros 722
E.2.10 evidencias_marcas 722
E.2.11 evidencias 723
E.2.12 evidencias_eventos 724
E.2.13 users 725
E.2.14 experiencias (catálogo) 727
E.2.15 experiencias_versoes (catálogo) 728
E.2.16 experiencias_arquivadas (catálogo) 728
E.2.17 disponibilidades (catálogo) 729
E.2.18 recursos_auditoria (catálogo) 730
E.2.19 ia_quotas (catálogo) 732
E.2.20 agente_professor_conversas 732
E.2.21 progresso 733
E.2.22 ia_uso_diario 733
E.2.23 telemetria_eventos (particionada) 733
E.2.24 prospeccao_leads (sem tenant) 734
E.3 Tabelas da exp08ef1b 735
E.3.1 exp08ef1b_activities 735
E.3.2 exp08ef1b_posts 736
E.3.3 exp08ef1b_runs 737
E.3.4 exp08ef1b_events — o canal operacional 737
E.3.5 exp08ef1b_app_70_perfis_importados 738

E.4 Tabelas das demais experiências 738
E.4.1 backup_exp04ef1b_sessoes 738
E.4.2 exp04ef1b_sessoes 739
E.5 Tabelas da exp02em1b 740
E.5.1 exp02em1b_sessoes 740
E.5.2 exp02em1b_mensagens 741
E.5.3 exp02em1b_marcos 741
E.5.4 exp02em1b_simulacoes 742
E.5.5 exp02em1b_relogio_turma 742
E.5.6 app_70_sessoes 743
E.5.7 app_70_perfis 744
E.5.8 app_70_bolhas 744
E.5.9 app_90_planos e app_90_referencias 745
E.5.10 app_92_configuracoes 745
E.5.11 app_92_planos_anuais 746
E.5.12 app_92_planos_de_periodo 747
E.5.13 exp08ef4b_conteudo_turma 748
E.5.14 exp08ef4b_sessoes 749
E.5.15 exp08ef4b_auditorias 750
E.5.16 exp08ef4b_respostas_percurso 750
E.5.17 exp08ef4b_gates_turma 751
E.5.18 exp05ef1b_pacotes 752
E.5.19 exp05ef1b_gates_turma 752
E.5.20 exp05ef1b_sessoes 753
E.5.21 exp05ef1b_reflexoes 754
E.5.22 app_91_ciclos e app_91_checkins 754
E.6 O que o Prisma não modela 755
F Operação 756
F.1 Comandos de desenvolvimento 756
F.2 Testes de ponta a ponta 757

F.3 *Design system* . . . 759
F.4 Publicação e reversão . . . 759
F.5 Provisionamento de um contrato novo . . . 760
F.6 Diagnóstico: sintoma → causa . . . 760
F.7 *Checklist* de operação . . . 780
F.8 Compilar esta documentação . . . 781

Referências **782**

Lista de figuras

1.1 Visão geral da plataforma . . . 3

4.1 Casos de uso da plataforma . . . 21

5.1 C4 nível 1 — contexto . . . 32

5.2 C4 nível 2 — contêineres . . . 33

5.3 C4 nível 3 — componentes da API . . . 36

5.4 C4 nível 3 — componentes do hub . . . 39

5.5 C4 nível 4 — tipos que carregam a regra . . . 41

5.6 Grafo de dependências do monorepo . . . 43

5.7 Sequência: curadoria de um bloco de feed . . . 44

5.8 Estados de uma geração assíncrona . . . 45

5.9 Os catorze tipos de diagrama da UML 2.5.1 . . . 47

5.10 Perfil UML da plataforma . . . 49

5.11 Casos de uso da administração da rede . . . 51

5.12 Classes: o modelo de domínio da plataforma . . . 53

5.13 Classes: a porta da IA . . . 54

5.14 Objetos: um instantâneo da disponibilidade . . . 55

5.15 Pacotes: quem pode importar quem . . . 57

5.16 Componentes e interfaces . . . 58

5.17 Estrutura composta: a página da missão . . . 59

5.18 Atividades: o resolvedor de disponibilidade . . . 60

5.19 Sequência: a entrada do estudante na missão . . . 62

5.20 Sequência: uma chamada de IA com quota . . . 64

5.21 Sequência: um turno do agente de IA do professor . . . 66

5.22 Comunicação: o caminho de um lote de telemetria . . . 67
5.23 Tempo: *token*, renovação e sessão . . . 69
5.24 Máquina de estados de protocolo: a ponte . . . 70
5.25 Visão geral de interação: a jornada do estudante . . . 71
5.26 Implantação: que artefato roda em que nó . . . 72

6.1 Implantação de um contrato . . . 74
6.2 Implantação detalhada de um contrato . . . 75
6.3 Identidade de carga de trabalho . . . 80

7.1 Os três níveis de isolamento . . . 85
7.2 Propagação da escola pela requisição . . . 88

8.1 Modelo de dados da plataforma . . . 112
8.2 Enturmamento . . . 117
8.3 Modelo de dados da exp08ef1b: atividade e conteúdo . . . 121
8.4 Modelo de dados da exp08ef1b: sessão do estudante . . . 122
8.5 Modelo de dados das exps 02 e 70 . . . 124
8.6 Modelo de dados das exps 90 e 91 . . . 125

9.1 Sequência do handshake hub-experiência . . . 141
9.2 Resolução da URL do embed . . . 145
9.3 Contrato entre duas experiências . . . 152

10.1 Canal analítico de telemetria . . . 155

11.1 Camada de IA . . . 162
11.2 Guardrails do check-in de bem-estar . . . 167

12.1 Os quatro graus de medida . . . 198
12.2 app-92: o caminho do planejamento de ensino . . . 202
12.3 app-92: como uma semana nasce . . . 205

13.1 Mapa de rotas da API — plataforma e exp08ef1b . . . 213

13.2 Mapa de rotas da API — demais experiências 214

14.1 A linha da revelação 239

15.1 Pipeline de entrega 264

16.1 Estratégia de testes 273

17.1 Cadeia de uma requisição e o que cada elo revela hoje 284

17.2 Erro de cliente entra pela API 290

19.1 Matriz de riscos 527

21.1 Cronograma de resolução das pendências identificadas até a entrada em produção. 537

Lista de tabelas

5.4 Os catorze diagramas UML e a pergunta que cada um responde aqui 47

5.5 O que o resolvedor decide no instantâneo da Figura 5.14 55

5.6 As mensagens da Figura 5.22 67

Lista de siglas e abreviaturas

Siglas técnicas

Sigla	Significado
ADC	*Application Default Credentials* — cadeia de resolução de credencial do Google Cloud
ADK	*Agent Development Kit* — o *framework* de agentes do Google, em Python (o agente de IA do professor)
ADR	*Architecture Decision Record* — registro de decisão arquitetural
API	*Application Programming Interface*
ANPD	Autoridade Nacional de Proteção de Dados
BFF	*Backend for Frontend* — camada de *proxy* do hub para a API
BNCC	Base Nacional Comum Curricular
C4	Modelo de visualização arquitetural em quatro níveis (Contexto, Contêiner, Componente, Código)
CDN	*Content Delivery Network*
CI/CD	*Continuous Integration / Continuous Delivery*
CORS	*Cross-Origin Resource Sharing*
CSP	*Content Security Policy*
CSV	*Comma-Separated Values*
DASS-21	*Depression, Anxiety and Stress Scales* — versão de 21 itens
DoD	*Definition of Done* — critério de conclusão de tarefa
DPA	*Data Processing Agreement* — acordo de tratamento de dados
DS	*Design System*
EF	Ensino Fundamental (séries `1EF`–`9EF`)
EM	Ensino Médio (séries `1EM`–`3EM`)
ER	Entidade-Relacionamento (modelo de dados)
GCP	*Google Cloud Platform*
HA	*High Availability* — alta disponibilidade
HTTP(S)	*HyperText Transfer Protocol* (*Secure*)

Sigla	Significado
IaC	*Infrastructure as Code* — infraestrutura como código
IdP	*Identity Provider* — provedor de identidade
JSON	*JavaScript Object Notation*
JWK/JWKS	*JSON Web Key* / *JSON Web Key Set*
JWT	*JSON Web Token*
LB	*Load Balancer*
LGPD	Lei Geral de Proteção de Dados Pessoais (Lei 13.709/2018)
LLM	*Large Language Model* — modelo de linguagem de grande escala
NDJSON	*Newline-Delimited JSON* — um objeto JSON por linha
OIDC	*OpenID Connect*
OTel/OTLP	*OpenTelemetry* / *OpenTelemetry Protocol* — padrão aberto de instrumentação (rastros, métricas, logs) e seu protocolo de exportação
ORM	*Object-Relational Mapping*
PII	*Personally Identifiable Information* — dado pessoal identificável
PITR	*Point-in-Time Recovery* — recuperação a um instante no tempo
PK/FK	*Primary Key* / *Foreign Key*
REST	*Representational State Transfer*
RF	Requisito Funcional
RLS	*Row-Level Security* — segurança em nível de linha do PostgreSQL
RNF	Requisito Não Funcional
RS256	RSASSA-PKCS1-v1_5 com SHA-256 — algoritmo de assinatura de JWT
RUM	*Real User Monitoring* — medida de desempenho no navegador do usuário real (Web Vitals)
SA	*Service Account* — conta de serviço do GCP
SDK	*Software Development Kit*
SLA/SLO	*Service Level Agreement* / *Objective*
SLI	*Service Level Indicator* — a medida sobre a qual um SLO promete um valor
SQL	*Structured Query Language*
SSE	*Server-Sent Events* — fluxo de eventos do servidor ao navegador numa resposta HTTP aberta
SSR	*Server-Side Rendering*
TLS	*Transport Layer Security*
TTL	*Time To Live* — tempo de validade
UC	Caso de Uso (*Use Case*)

Sigla	Significado
UML	*Unified Modeling Language*
VPC	*Virtual Private Cloud*
WIF	*Workload Identity Federation* — federação de identidade de carga de trabalho
WSO2 IS	WSO2 *Identity Server* — o gerenciador central de identidade do produto
xAPI	*Experience API* — especificação de telemetria de aprendizagem

Termos do domínio

Os nomes de domínio deste projeto são em português brasileiro por convenção explícita (`AGENTS.md`, seção “Convenções de código”); termos técnicos permanecem em inglês. O glossário abaixo fixa o significado de cada um para todo o documento.

Termo	Significado neste projeto
`contrato`	**Tenant de nível 1**: uma rede pública (estado ou município) que contratou a plataforma. Um contrato equivale a uma *stack* de infraestrutura GCP inteira e separada.
`escola`	**Tenant de nível 2**: uma unidade escolar dentro de um contrato. Cem ou mais escolas compartilham a mesma *stack*, isoladas logicamente por `escola_id` + RLS.
`turma`	Agrupamento de estudantes de uma mesma série dentro de uma escola. Chave de balanceamento de vários instrumentos (bolhas, distribuição de lados).
`estudante`	Usuário-estudante. É o único papel cujos dados são de menores, o que governa a maior parte das decisões de retenção e de log.
`professor`	Usuário que prepara e conduz a aula; consome relatórios de turma e usa as ferramentas de professor (app-90, app-91).
`gestor`	Usuário administrativo da escola: importa usuários, redefine senhas. Não vê dado de bem-estar de professor.
`admin-contrato`	Único papel autorizado a atravessar escolas, e apenas em rotas explicitamente marcadas. Publica manifestos de experiência.
`experiência`	Aplicação imersiva independente (*bundle* Vite estático) que o hub embeda em `iframe`. Nunca é um serviço.
`hub`	Aplicação Next.js que autentica, roteia por papel, embeda as experiências e desenha todo o *chrome* institucional.
`aparelho`	Metáfora de interface: a casca de dispositivo (`DispositivoFrame`) que o hub desenha em volta do `iframe` quando a experiência declara `viewport`.

Termo	Significado neste projeto
`clone`	Superfície de interface que imita um aplicativo real (Instagram, YouTube, WhatsApp, ChatGPT, TikTok) para que a experiência possa medir comportamento em contexto reconhecível.
`revelação`	Momento pedagógico em que a experiência sai da ficção do clone e volta à linguagem institucional. É a linha que separa as seções do *design system*.
`missão`	Unidade pedagógica: uma experiência com objetivo, guia do professor e leitura de resultado.
`manifesto`	Linha da tabela `experiencias` (ou o `public/experiencia.json` do *bundle*) que descreve id, nome, versão ativa, séries, *capabilities*, `viewport`, público e permissões.
`versaoAtiva`	Versão do *bundle* que o hub serve. *Rollback* é a troca deste campo — não um novo *deploy*.
`bolha`	Nível de dificuldade do instrumento da app-70 (A, B ou C), atribuído pelo servidor e estável por estudante.
`grau`	Quanto uma partida sustenta a medida de um valor: `scored`, `indicative`, `zero` ou `none`. A diferença entre `zero` e `none` é invariante de API.
`run`	Sessão de um estudante numa atividade da exp08ef1b: carrega o estado do motor de feed adaptativo.
`comTenancy`	Porta única de acesso ao banco na API: abre transação e arma a RLS com a escola do *token*.

1

Introdução

1.1 Propósito deste documento

Este documento descreve a arquitetura, o escopo e a implementação da **Plataforma Unooria**: uma plataforma de educação básica destinada a redes públicas de ensino, composta por um *hub* web multi-tenant e um catálogo alvo de 48 experiências imersivas que cobrem do 1º ano do Ensino Fundamental ao 3º ano do Ensino Médio.

A descrição arquitetural segue a norma ISO/IEC/IEEE 42010 (International Organization for Standardization, 2022): identifica os interessados, enuncia os *drivers* que restringem o desenho, apresenta visões complementares do sistema e registra as decisões com suas consequências. A notação de visualização é o modelo C4 (BROWN, 2026; BROWN, 2024), aplicado *duas vezes* — uma para a arquitetura de *software* (Capítulo 5) e outra para a arquitetura de nuvem e implantação (Capítulo 6) — porque neste projeto os dois recortes não coincidem: há contêineres lógicos que não são serviços em execução, e há serviços em execução que não são contêineres lógicos novos.

> **NOTA**
>
> Todo diagrama deste documento é vetorial, desenhado nativamente em TikZ. Não há imagem rasterizada de diagrama, nem dependência de ferramenta externa de renderização. Para quem precisa editar, revisar ou versionar o diagrama fora do LaTeX, o Apêndice A traz a fonte Mermaid equivalente de cada figura, referenciando o mesmo número.

1.2 O problema

Uma plataforma educacional para rede pública tem três exigências que, juntas, determinam quase toda a arquitetura — e que raramente aparecem juntas em produto educacional comercial:

1. **Isolamento contratual forte, isolamento escolar barato.** Cada contrato é uma relação jurídica com um ente federativo. Vazamento de dado entre *contratos* é inaceitável; por isso

cada contrato recebe uma *stack* de infraestrutura própria. Já as cem escolas de um mesmo contrato não podem custar cem infraestruturas: elas compartilham a *stack* e são isoladas logicamente.

2. **Pico de carga previsível e brutal.** A rede toda entra na plataforma às oito da manhã. O alvo de projeto é de aproximadamente dez mil estudantes por contrato com dois a quatro mil simultâneos nesse pico — uma curva de carga que não se parece com a de um SaaS de escritório.
3. **Dados de menores.** A maior parte dos titulares dos dados tratados é criança ou adolescente. Isso não é um detalhe de conformidade tratado no fim: é um *driver* arquitetural que decide o que se registra, onde se registra, quem lê e o que o relatório está *proibido* de afirmar (Brasil, 2018; Autoridade Nacional de Proteção de Dados, 2024).

A quarta restrição não é do domínio, é do projeto, e é a mais severa: **uma pessoa desenvolve, com prazo de produção em novembro de 2026**. Isso transforma "simplicidade" de preferência estética em requisito de viabilidade. Cada peça de infraestrutura nova é uma peça que uma pessoa vai operar de plantão; cada abstração a mais é código que ninguém revisa em par.

> **DECISÃO DE PROJETO**
>
> A regra de projeto que decorre disso está escrita no documento de governança do repositório e é aplicada em todo este documento: **simplicidade** > **sofisticação**. Nenhum serviço, fila, cache ou biblioteca de estado entra sem *gargalo medido*. As duas exceções são decisões tomadas de antemão e justificadas: o Pub/Sub da telemetria (**ADR-06**) e o *provider* plugável de IA (**ADR-07**).

1.3 A solução em uma página

A plataforma é um *monorepo* com quatro tipos de artefato, e a distinção entre eles é a decisão arquitetural mais consequente do projeto:

Artefato	Tecnologia	Como é entregue
`apps/hub`	Next.js (App Router)	Contêiner no Cloud Run
`apps/api`	NestJS + PostgreSQL	Contêiner no Cloud Run
`apps/experiencias/*`	Vite + React + `@edu/sdk`	***Bundle* estático** em Cloud Storage atrás de CDN
`packages/*`	TypeScript puro	Consumido por *workspace*; nunca implantado sozinho

> **REGRA INVIOLÁVEL**
>
> **Experiências nunca viram serviços.** Uma experiência é um conjunto de arquivos estáticos em `gs://{contrato}-exp/{expId}/{versao}/`. Não há processo, não há porta, não há *scaling*. Isso vale para as 48: o custo marginal de publicar a quadragésima oi-

> tava experiência é o custo de copiar arquivos para um *bucket*. *Rollback* é o hub passar a apontar para a versão anterior — uma linha de `UPDATE`, não um *deploy*.

A Figura 1.1 resume o caminho de uma sessão de estudante, do *login* à coleta de telemetria.

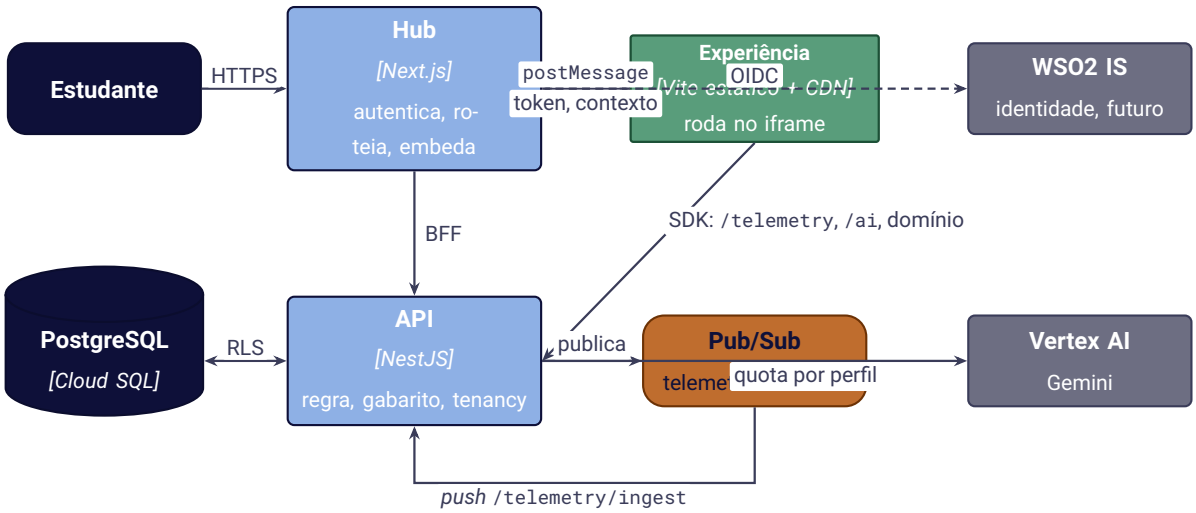


Figura 1.1 – Visão geral: o hub é a única porta do usuário; a experiência é estática e recebe o contexto por `postMessage`; a API concentra regra, gabarito e *tenancy*; a telemetria analítica sai pelo Pub/Sub e volta à própria API por *push*.

Legenda: pessoa · contêiner da plataforma · artefato estático · banco de dados · fila · sistema externo

1.4 Como ler este documento

1.4.1 Estrutura

Cap.	Título	Responde a
1–2	Introdução, contexto de negócio	Por que a plataforma existe e sob que restrições
3–4	Requisitos, casos de uso	O que ela deve fazer, e para quem
5	Arquitetura de *software* (C4)	Como o *software* está organizado
6	Arquitetura de nuvem (C4 + implantação)	Onde ele roda e quanto custa operá-lo
7	*Multi-tenancy* e segurança	Como um dado de escola não vaza para outra
8	Modelo de dados	Que dados existem e por quê
9–11	Contratos, telemetria, IA	As três fronteiras que mais custam se quebrarem
12–14	Experiências, API, *design system*	O catálogo, a superfície HTTP e a interface

15–16	Entrega e qualidade	Como o código chega à escola e o que o impede de chegar errado
17	Observabilidade	Como se sabe que ela está funcionando na escola, e quem é avisado quando não está
18–20	ADRs, riscos, conclusão	As decisões, o que pode dar errado e o estado atual
A–F	Apêndices	Mermaid, variáveis, *endpoints*, *payloads*, dicionário de dados, operação

1.4.2 Convenções tipográficas

Marca	Significado
`identificador`	Identificador de código: campo, função, tipo, coluna, valor.
`caminho/arquivo.ts`	Caminho de arquivo ou diretório no repositório.
`GET /rota/:id`	*Endpoint* HTTP da API.
`VARIAVEL`	Variável de ambiente.
RF-01, **RNF-01**	Requisito funcional / não funcional (Capítulo 3).
UC-01	Caso de uso (Capítulo 4).
ADR-01	Decisão arquitetural registrada (Capítulo 18).
DR-01	*Driver* arquitetural (Seção 2.4).

Os blocos destacados têm significado fixo ao longo do documento:

> **NOTA**
>
> Informação de contexto que ajuda a entender uma escolha, mas cuja ausência não quebra nada.

> **DECISÃO DE PROJETO**
>
> Decisão de projeto tomada deliberadamente, com a alternativa descartada e o motivo. Toda decisão deste tipo tem uma entrada correspondente no Capítulo 18.

> **REGRA INVIOLÁVEL**
>
> Invariante que não pode ser violada por nenhuma implementação futura. As sete regras invioláveis da plataforma estão consolidadas na Seção 2.6.

> **ATENÇÃO**
>
> Armadilha conhecida: algo que já falhou, ou que falha silenciosamente. Quase todos estes blocos registram um erro efetivamente cometido e medido no projeto.

LIMITAÇÃO CONHECIDA

Limitação assumida do estado atual, com o caminho de saída. Não é bug: é escopo que ficou de fora, de propósito.

1.5 O que este documento não cobre

- **O código Terraform.** Ele vive em repositório separado. O *contrato* entre os dois repositórios — nomes de recursos esperados, variáveis consumidas, artefatos publicados — está no Capítulo 6 e em `docs/INFRA.md`.
- **O conteúdo pedagógico.** O catálogo de missões, os textos e a trilha por ano são produto, não arquitetura. O Capítulo 12 descreve a *mecânica* de cada experiência apenas até onde ela impõe exigências ao *software*.
- **O *design system*.** Os componentes vivem em repositório próprio (`storybook-unooria`) e chegam aqui como pacote npm publicado. O Capítulo 14 documenta as *regras de consumo*, não o catálogo de componentes.
- **As 43 experiências ainda não escritas.** O Capítulo 12 documenta as cinco existentes e o *gabarito* que as próximas devem seguir.

2

Contexto de negócio e restrições

2.1 Modelo de negócio

A Plataforma Unooria é vendida **B2G**: o cliente é um ente federativo — uma secretaria estadual ou municipal de educação — e não a escola nem a família. A consequência arquitetural imediata é que a *unidade comercial* e a *unidade de uso* são diferentes, e o desenho precisa acomodar as duas.

Nível	Entidade	Isolamento
1	`contrato` (rede)	**Físico**: *stack* GCP inteira e separada, provisionada por um *module* Terraform por contrato. Projeto, *buckets*, banco, *Load Balancer* e contas de serviço próprios.
2	`escola`	**Lógico**: `escola_id` obrigatório em toda tabela de dado de estudante, mais *Row-Level Security* no PostgreSQL. Cem ou mais escolas por *stack*.
3	`turma` / `estudante`	**Autorização**: *claims* do JWT (`turma_id`, `perfil`) verificados na API por `RolesGuard` e por filtro explícito nas *queries*.

> **DECISÃO DE PROJETO**
>
> **Uma *stack* por contrato, escolas por RLS** (ADR-04). A alternativa "uma *stack* por escola" daria isolamento físico perfeito e transformaria a empresa em operadora de cem infraestruturas por contrato — inviável para uma pessoa. A alternativa oposta, uma *stack* única para todos os contratos, tornaria um incidente em um município um incidente em todos, e não sobreviveria à primeira análise jurídica de um edital. O ponto escolhido separa fisicamente o que é contratual e logicamente o que é operacional.

2.2 Escala alvo

Grandeza	Alvo por contrato	Consequência

Estudantes cadastrados	~10.000	Cabe folgadamente numa instância pequena de Cloud SQL.
Simultâneos no pico (8h)	2.000–4.000	`min-instances` no Cloud Run para eliminar *cold start*; *pooler* de conexões obrigatório.
Escolas por *stack*	100+	RLS e `escola_id` em todo lugar; nenhuma *query* cruza escolas fora de `admin-contrato`.
Experiências no catálogo alvo	48	Custo marginal de publicação *deve* ser próximo de zero ⇒ *bundle* estático.
Faixa etária	1º EF ao 3º EM	Instrumentos e interface variam por série; regras de instrumento mudam nos anos iniciais.

O perfil de carga é o dado mais importante desta tabela e merece ser dito por extenso: **o tráfego não é distribuído, é uma parede**. Entre 7h50 e 8h10 de um dia letivo, uma fração grande da rede faz *login* e abre uma experiência quase ao mesmo tempo. O desenho responde a isso com três medidas independentes:

1. O grosso dos *bytes* do pico — os *bundles* das experiências e as imagens geradas — não passa pela aplicação. Sai da CDN, com o caminho versionado e, portanto, imutável e cacheável indefinidamente (ADR-01).
2. Os serviços têm instâncias mínimas provisionadas, então o pico não paga inicialização de contêiner nem *warm-up* de *pool* do Prisma.
3. A telemetria de alto volume não é síncrona: vai para o Pub/Sub e é gravada em lote, fora do caminho da requisição do estudante (Capítulo 10).

2.3 Restrições de projeto

ID	Restrição	Origem e consequência
RE-01	Uma pessoa desenvolve e opera	Restrição de recurso. Nenhuma peça de infraestrutura sem gargalo medido; preferência sistemática por absorver capacidade na API existente em vez de criar serviço novo.
RE-02	Produção em novembro de 2026	Restrição de prazo. Ordem de implementação por milestones (M0–M6); o que não bloqueia a primeira rede fica documentado como evolução (Capítulo 19).
RE-03	Identidade centralizada em WSO2 IS	Exigência do cliente e da estratégia multi-contrato. A aplicação nunca conhece o WSO2: conhece OIDC (ADR-03).
RE-04	Google Cloud Platform	Decisão de plataforma. Serviços gerenciados preferidos a orquestração própria; Cloud Run em vez de GKE.

RE-05	Interface exclusivamente do *design system*	Decisão de produto e de marca. Nenhum componente de UI nasce neste repositório; o que falta nasce no `storybook-unooria` (Capítulo 14).
RE-06	Dados de menores	LGPD art. 14 e orientação da ANPD (Brasil, 2018; Autoridade Nacional de Proteção de Dados, 2024). Governa telemetria, log, retenção e *o que o relatório pode afirmar*.
RE-07	Dado de saúde de professor	LGPD art. 11 (dado sensível). Determina a ausência deliberada de visão de gestor na app-91.

2.4 Os *drivers* arquiteturais

Um *driver* arquitetural é uma exigência que, se ignorada, invalida a solução — não a piora, invalida. Foram sete.

ID	Driver	Consequência arquitetural
DR-01	Isolamento verificável entre escolas	`escola_id NOT NULL` + `FORCE ROW LEVEL SECURITY` em toda tabela de dado de estudante; acesso ao banco por uma *única* porta (`PrismaService.comTenancy`) que arma a política antes de qualquer trabalho. *Teste* de *tenancy* com *role* sem *ownership* (`tests/tenancy.spec.ts`).
DR-02	Custo marginal quase nulo por experiência	Experiência é *bundle* estático versionado em *bucket*, nunca um serviço. *Rollback* por ponteiro (`versaoAtiva`), não por *deploy* (**ADR-01**).
DR-03	Correção e pontuação não podem estar no cliente	Gabarito, correção e qualquer nota são *server-side*, num módulo de domínio por experiência na API. O *bundle* nunca recebe a resposta certa (**ADR-09**).
DR-04	Troca de provedor de identidade sem tocar código	Validação de *token* por OIDC genérico: `OIDC_ISSUER` e `OIDC_JWKS_URI`. Hoje o emissor é o próprio hub; amanhã é o WSO2, e o `OidcGuard` não muda (**ADR-03**).
DR-05	IA com custo previsível e sem segredo no cliente	Toda chamada de IA passa por `apps/api/src/ai`, com quota diária persistida *por perfil* verificada antes da chamada. Nenhuma chave, URL interna ou *prompt* de sistema em *bundle* (**ADR-07**).
DR-06	O pico da manhã não pode derrubar o transacional	Dois canais de telemetria com garantias diferentes: o operacional é síncrono e transacional; o analítico é assíncrono via Pub/Sub, com *insert* multi-linha e tabela particionada por mês (**ADR-06**, **ADR-14**).
DR-07	O relatório não pode produzir rótulo sobre uma criança	Invariantes *de tipo*, não de tela: valor com uma única fonte de medida não recebe número; "não teve oportunidade" nunca é lido como "escolheu não"; listagem de turma não exibe valor psicométrico (**ADR-12**).

> **NOTA**
>
> **DR-07** é incomum como *driver* arquitetural, porque parece uma questão de produto. Ele está aqui porque foi implementado como *estrutura*: a API devolve uma união discriminada em que só a variante `scored` tem `numero: number`; as outras têm `null` *por tipo*. O hub fica estruturalmente incapaz de imprimir "Poder 100" a partir de um único sinal — ele nunca recebe esse número, e o compilador nem deixa procurá-lo. Prosa num comentário não é controle; um tipo é.

2.5 Interessados

Interessado	O que precisa	Onde é atendido
Estudante	Entrar, escolher a missão, jogar sem atrito, ver o próprio resultado	Hub do estudante (`/estudante`), `/exp/*`, dossiê da experiência
Professor	Preparar a aula, conduzir o momento coletivo, ler a turma, gerar plano de aula, cuidar do próprio bem-estar	`/professor/*`, app-90, app-91, relatórios por turma
Gestor escolar	Cadastrar e manter usuários da escola	`/admin/usuarios`, importação CSV
Secretaria (rede)	Visão agregada da rede, sem identificar criança	`/analise/*` (papel `admin-contrato`)
Encarregado de dados (DPO)	Saber o que é coletado, por quanto tempo, quem lê, e onde o dado é processado	Capítulos 7 e 10; **ADR-11**, **ADR-13**
Desenvolvedor	Adicionar experiência sem reaprender a plataforma	SDK, `_template`, *scaffolding* (`pnpm new:exp`), Capítulo 12
Operação	Subir, observar e reverter sem adivinhar	CI/CD (Capítulo 15), Apêndice F

2.6 As sete regras invioláveis

Estas regras estão registradas em `AGENTS.md` e são normativas: uma implementação que as viole está errada, independentemente de passar nos testes. Cada uma tem o capítulo onde é detalhada e verificada.

#	Regra	Detalhe em
1	Toda *query* SQL é escopada por `escola_id` (e `estudante_id` quando aplicável). Tabelas de dado de estudante têm `escola_id NOT NULL` + RLS. Nenhuma *query* atravessa escolas, exceto em rotas `admin-contrato`.	Cap. 7
2	Nenhum segredo em *bundle* de cliente: sem chave de API, sem *prompt* de sistema, sem URL interna. IA só pela API.	Cap. 11
3	Resposta correta, correção e pontuação que vale nota são *server-side*. O cliente envia a resposta do estudante; a API corrige.	Cap. 12
4	O *token* é obtido pelo hub via OIDC e repassado à experiência por `postMessage` com `targetOrigin` explícito — nunca `"*"`. A experiência mantém o *token só em memória*.	Cap. 9
5	IA tem quota diária por usuário *e por perfil*, e *rate limit*. Nenhuma chamada de IA sem passar pelo `QuotaService`: `temSaldo` antes, `consumir` depois.	Cap. 11
6	Dados de menores: nenhuma PII em log em texto claro. A telemetria carrega `estudante_id` opaco, nunca nome ou *e-mail*.	Cap. 10
7	Nenhuma migração remove `escola_id` ou afrouxa RLS. Renomeações usam `RENAME`, nunca `DROP`/`CREATE`.	Cap. 8

2.7 Fora de escopo, deliberadamente

O que *não* entra é parte do desenho. As exclusões abaixo são decisões, não omissões, e cada uma tem um gatilho explícito de reavaliação (Capítulo 19).

Descartado	Motivo e gatilho de reavaliação
Kubernetes (GKE)	Cloud Run entrega *autoscaling*, TLS e revisões sem *control plane* para operar. Gatilho: necessidade de *workload* de longa duração ou topologia de rede que o Cloud Run não expresse.
AlloyDB	A robustez vem de HA + PITR + *pooler*, não do *engine*. Ambos são PostgreSQL; migrar depois é trivial (**ADR-05**).
Microsserviço por experiência	Seriam 48 serviços para operar, cada um com *deploy*, log e escala próprios — para servir arquivos estáticos (**ADR-01**).
Redis como *buffer* de telemetria	Quem amortece o pico já é o Pub/Sub, e ele é *durável*: um *buffer* em memória perde os eventos do processo que morre antes do *flush*. O gargalo medido era o laço de `INSERT`, e ele foi corrigido (**ADR-14**).

BigQuery para o canal analítico	Nenhum relatório atual lê `telemetria_eventos`; seria uma segunda plataforma de dados para operar sem resolver problema existente (**ADR-14**).
Multi-região	Uma rede é geograficamente concentrada. Gatilho: exigência contratual de continuidade regional.
Worker dedicado de telemetria	Substituído por *push subscription* para a própria API — uma peça a menos para operar. Gatilho: volume que justifique isolar o consumo.

3

Requisitos

Este capítulo especifica os requisitos da plataforma de forma rastreável: cada requisito recebe identificador estável, prioridade, estado e o ponto do sistema onde é verificado. Requisito sem ponto de verificação é intenção, não requisito.

A prioridade segue MoSCoW: **M** (*must*) bloqueia a entrada em produção; **S** (*should*) é esperado na primeira rede; **C** (*could*) é evolução planejada. O estado reflete o repositório em 27 de agosto de 2026.

3.1 Requisitos funcionais

3.1.1 Identidade, papéis e acesso

ID	Requisito	Pri	Estado	Verificação
RF-01	Autenticar usuário por credencial da rede e emitir sessão.	M	Implementado	`apps/hub/lib/auth.ts`; `POST /auth/login`
RF-02	Emitir *token* de experiência assinado em RS256, com TTL de 15 minutos e *claims* `{sub, escola_id, turma_id?, perfil}`, audiência `plataforma-api`.	M	Implementado	`lib/chave-token.ts`; `GET /.well-known/jwks.json`
RF-03	Validar *token* na API por JWKS do emissor configurado, sem conhecer o produto de identidade.	M	Implementado	`src/auth/oidc.guard.ts`
RF-04	Restringir rota por papel (`estudante`, `professor`, `gestor`, `admin-contrato`).	M	Implementado	`src/auth/roles.ts`

ID	Requisito	Pri	Estado	Verificação
RF-05	Redirecionar usuário que acessa área de outro papel para a home do próprio papel, sem expor 403.	S	Implementado	`apps/hub/middleware.ts`; `lib/papeis.test.ts`
RF-06	Importar usuários em massa por CSV (`nome`, `usuario`, `senha`, `papel`, `tu`	M	Implementado	`POST /usuarios/importar-csv`; `tests/usuarios.spec.ts`
RF-07	Redefinir senha de usuário da escola.	S	Implementado	`POST /usuarios/:id/reset-senha`
RF-08	Trocar o emissor de identidade para o WSO2 IS alterando apenas configuração.	M	Preparado	`OIDC_ISSUER`, `OIDC_JWKS_URI`; `wso2/`

3.1.2 Experiências e *embed*

ID	Requisito	Pri	Estado	Verificação
RF-10	Manter manifesto de versões por experiência e servir a versão ativa.	M	Implementado	Tabela `experiencias`; `PUT /experiencias/:expId`
RF-11	Reverter versão de experiência trocando `versaoAtiva`, sem *build* nem *deploy*.	M	Implementado	`scripts/publicar-manifesto.mjs`
RF-12	Embutir a experiência em `iframe` com *sandbox* mínimo, concedendo capacidade extra apenas a quem a declara no manifesto.	M	Implementado	`components/ExpFrame.tsx`; `scripts/probe-pointerlock.mjs`
RF-13	Entregar *token* e contexto à experiência por `postMessage` com `targetOrigin` explícito.	M	Implementado	`packages/sdk/src/bridge.ts`; `bridge.test.ts`
RF-14	Renovar *token* da experiência sob demanda, sem recarregar o `iframe`.	M	Implementado	`exp:token-expirando` → `GET /api/exp-token` → `hub:token`
RF-15	Pausar e retomar a experiência quando a página do hub perde ou recupera visibilidade/foco.	S	Implementado	`hub:pause` / `hub:resume`
RF-16	Desenhar casca de aparelho quando a experiência declara `viewport`, lendo o manifesto *ou* o `experiencia.json` do *bundle*.	S	Implementado	`lib/embed-experiencia.ts`; `components/DispositivoFrame.tsx`

ID	Requisito	Pri	Estado	Verificação
RF-17	Persistir e recuperar progresso genérico por (`escola`, `estudante`, `experiência`).	M	Implementado	`GET/POST /progresso/:expId`
RF-18	Expor uma bancada local de *embed* para desenvolvimento (`/dev-embed?src=`).	C	Implementado	`app/dev-embed`

3.1.3 Telemetria

ID	Requisito	Pri	Estado	Verificação
RF-20	Coletar evento analítico no padrão xAPI simplificado, em lote de até 50 eventos com *flush* a cada 10 segundos.	M	Implementado	`packages/sdk/src/telemetry.ts`
RF-21	Enviar o último lote no `pagehide` por `sendBeacon`, com o *token* no corpo.	M	Implementado	`POST /telemetry/beacon`
RF-22	Derivar identidade do evento do *token*, nunca do corpo enviado pelo cliente.	M	Implementado	`src/telemetry/telemetry.controller.ts`
RF-23	Gravar o canal analítico de forma idempotente sob entrega *at least once*.	M	Implementado	`ON CONFLICT (event_id, ts) DO NOTHING`
RF-24	Autenticar o consumo do *push* do Pub/Sub, falhando fechado na ausência de configuração.	M	Implementado	`src/telemetry/pubsub-push.guard.ts`; `tests/telemetria-push.spec.ts`
RF-25	Manter o canal operacional (motor de feed) síncrono e transacional, fora do Pub/Sub.	M	Implementado	`POST /runs/:id/eventos`, `GET /runs/:id/feed`

3.1.4 Inteligência artificial

ID	Requisito	Pri	Estado	Verificação
RF-30	Concentrar toda chamada a provedor de IA num único módulo com interface neutra de *vendor*.	M	Implementado	`src/ai/provider.ts`

ID	Requisito	Pri	Estado	Verificação
RF-31	Verificar quota diária por usuário *e por perfil* antes da chamada, e registrar consumo depois.	M	Implementado	`src/ai/quota.service.ts`; tabela `ia_uso_diario`
RF-32	Autenticar no provedor por ADC, sem chave de API em variável de ambiente.	M	Implementado	`src/ai/vertex.provider.ts`; `tests/vertex-provider.spec.ts`
RF-33	Oferecer *provider* de teste (`mock`) que permita subir sem GCP.	M	Implementado	`AI_PROVIDER=mock`
RF-34	Detectar corte de saída do modelo (`finishReason` = `MAX_TOKENS`) e tratá-lo como erro, não como conteúdo.	M	Implementado	`src/planos/planos.service.ts`
RF-35	Gerar plano de aula anual de forma assíncrona, com *claim* atômico e consulta por *polling*.	M	Implementado	`POST /planos` (202), `GET /planos/:id`
RF-36	Guardar conteúdo autoral do professor em armazenamento privado, fora do *bucket* público.	M	Implementado	`{contrato}-planos`; `src/object-storage`
RF-37	Validar todo texto gerado por IA que chegue ao professor no contexto de bem-estar (sem vocabulário clínico, ordem, promessa ou *link*), com *fallback* fixo.	M	Implementado	`src/bemestar/regras.ts`; `tests/bemestar-regras.spec.ts`

3.1.5 Pedagógico e relatórios

ID	Requisito	Pri	Estado	Verificação
RF-40	Corrigir e pontuar no servidor; nunca enviar gabarito ao *bundle*.	M	Implementado	`src/unootube/`, `src/motor/`, `src/vila/`
RF-41	Curar o feed adaptativo por blocos, de forma determinística dado um gerador de aleatoriedade.	M	Implementado	`src/motor/curadoria.ts`; `tests/motor-pesos.spec.ts`
RF-42	Serializar a curadoria por `run` com *lock* consultivo, para que dois pedidos simultâneos não descartem um bloco.	M	Implementado	`pg_advisory_xact_lock`; `tests/motor-concorrencia.spec.ts`

ID	Requisito	Pri	Estado	Verificação
RF-43	Atribuir bolha (A/B/C) balanceada dentro da turma, no servidor, e mantê-la estável entre partidas.	M	Implementado	`app_70_bolhas`; `tests/vila-regras.spec.ts`
RF-44	Não servir a bolha B nos anos iniciais (1EF–5EF).	M	Implementado	`bolhasElegiveis`
RF-45	Recalcular o perfil de valores no servidor a partir das medidas brutas, recusando *payload* que traga o perfil.	M	Implementado	`CHAVES_DE_PERFIL` em `src/vila/contrato.ts`
RF-46	Permitir reprocessar o histórico quando a versão do mapeamento subir.	M	Implementado	`app_70_sessoes.payload`; `versaoMapeamento`
RF-47	Devolver leitura em que só a variante `scored` tem número; as outras têm `null` *por tipo*.	M	Implementado	`src/vila/leitura.ts`
RF-48	Abrir o relatório individual pelo *nível* do instrumento antes do desempenho.	M	Implementado	`/professor/entre-redes/vila-do-lume/est`
RF-49	Distinguir “cartões servidos” de “cartões que apareceram”, e nunca afirmar omissão a partir de ausência.	M	Implementado	`cartoesServidos`
RF-50	Zerar os dados da turma de uma experiência, apagando também a atribuição de bolha, e *sem* tocar dados de outra experiência.	S	Implementado	`DELETE /vila/professor/turmas/:turmaId/dados`
RF-51	Importar o perfil da Vila como largada do Instanooria, preservando redistribuição manual do professor.	S	Implementado	`POST /atividades/:id/perfis/da-vila`; `tests/vila-ato1.spec.ts`
RF-52	Exportar análise por turma e por rede em CSV.	S	Implementado	`GET /atividades/:id/analise/csv`, `GET /analise/csv`
RF-53	Não expor visão de gestor nem agregado para o dado de bem-estar do professor.	M	Implementado (por ausência)	`src/bemestar/bemestar.controller.ts`
RF-54	Permitir ao professor apagar integralmente os próprios dados de bem-estar.	M	Implementado	`DELETE /bemestar/dados`

ID	Requisito	Pri	Estado	Verificação
RF-55	Conduzir a turma por etapas, com a etapa servindo de **piso** para quem ficou para trás e de **teto** para quem se adiantou: ninguém passa do ponto de parada antes de a aula seguinte ser liberada.	M	Implementado	`POST /rastro-boato/turmas/:turmaId/gate`; `regras/passos.ts`

3.2 Requisitos não funcionais

Organizados pelas características de qualidade de produto da ISO/IEC 25010 (International Organization for Standardization, 2023).

ID	Característica	Requisito e critério	Estado
RNF-01	Desempenho — capacidade	Suportar 2.000–4.000 sessões simultâneas por contrato no pico, sem *cold start*: `min-instances` $\geq$ 1 na API e no hub, concorrência 80 na API.	Configurado; teste de carga pendente
RNF-02	Desempenho — vazão de ingestão	O caminho de gravação da telemetria analítica deve custar $O(\text{lotes})$ e não $O(\text{eventos})$ em *round-trips* ao banco.	Implementado (`INSERT` multi-linha)
RNF-03	Desempenho — borda	*Bundles* e mídia servidos por CDN com caminho imutável; só o `index.html` tem *cache* curto (60 s).	Implementado
RNF-04	Confiabilidade	Banco com HA e PITR; *pooler* de conexões obrigatório; *deploy* por revisões reversíveis.	Provisionado
RNF-05	Confiabilidade — entrega	Perda de evento analítico tolerada apenas quando o processo morre antes do *flush*; a partir do Pub/Sub, a entrega é durável e *at least once* com gravação idempotente.	Implementado
RNF-06	Segurança — isolamento	Nenhuma *query* de dado de estudante pode retornar linha de outra escola, mesmo com erro de aplicação. Verificado por RLS forçada e por teste com *role* sem *ownership*.	Implementado
RNF-07	Segurança — segredo	Zero segredo em artefato de cliente. Segredos em Secret Manager; credencial de IA por ADC.	Implementado
RNF-08	Segurança — superfície	*Sandbox* de `iframe` mínimo; `targetOrigin` explícito; toda mensagem validada por esquema nos dois lados; CORS por lista explícita.	Implementado
RNF-09	Segurança — *fail closed*	Rota cuja proteção depende de configuração responde 401/503 na ausência dela, nunca fica aberta.	Implementado

ID	Característica	Requisito e critério	Estado
RNF-10	Privacidade	Telemetria sem PII; texto livre sensível nunca vai a log nem à telemetria; IA não recebe nome, escola, pontuação nem faixa clínica.	Implementado
RNF-11	Manutenibilidade	TypeScript *strict* sem `any` (uma exceção registrada em ADR); validação de todo *input* externo por Zod, inclusive variáveis de ambiente, no *boot*.	Implementado
RNF-12	Manutenibilidade — acoplamento	Nenhum módulo fora de `src/ai` conhece o *vendor* de IA; nenhum componente fora de `packages/ui` importa o *design system* diretamente.	Implementado
RNF-13	Portabilidade	Provedor de identidade e provedor de IA substituíveis por configuração ou por arquivo novo, sem alterar chamadores.	Implementado
RNF-14	Testabilidade	A plataforma sobe inteira sem GCP (`AI_PROVIDER=mock`, `ASSETS_DRIVER=local`, `PLANOS_DRIVER=local`, `TELEMETRIA_TOPIC` vazio).	Implementado
RNF-15	Observabilidade	*Smoke* visual obrigatório para mudança de tela — *typecheck* não detecta ícone virando texto nem `iframe` vazio. Em produção: log estruturado com correlação, alerta por sintoma (SLO) e erro de cliente registrado pela API (Capítulo 17).	Parcial — só o *smoke*
RNF-16	Custo	Custo marginal de experiência publicada ≈ armazenamento + tráfego de CDN. Nenhum serviço por experiência.	Implementado
RNF-17	Conformidade	Localidade de processamento de IA declarada e auditável; ausência de prazo de retenção é decisão registrada, não omissão.	Documentado

LIMITAÇÃO CONHECIDA

RNF-01 está *configurado* mas não *verificado*: o teste de carga com 4.000 *logins* e aberturas de experiência em dez minutos é obrigatório antes de novembro de 2026 e ainda não foi executado. É o maior risco técnico aberto do projeto (Capítulo 19, `R-01`).

3.3 Rastreabilidade

A matriz abaixo liga *drivers*, requisitos-chave e as decisões que os realizam. Ela é a resposta à pergunta “por que o sistema é assim” em uma única página.

Driver	Requisitos	Decisões (ADR)

DR-01	**RNF-06**, **RF-04**	**ADR-04** (*stack* por contrato), **ADR-08** (RLS forçada e porta única)
DR-02	**RF-10**, **RF-11**, **RNF-03**, **RNF-16**	**ADR-01** (*bundle* estático), **ADR-02** (*monorepo*)
DR-03	**RF-40**, **RF-45**	**ADR-09** (domínio por experiência na API)
DR-04	**RF-03**, **RF-08**, **RNF-13**	**ADR-03** (OIDC genérico), **ADR-15** (hub como emissor provisório)
DR-05	**RF-30**–**RF-34**, **RNF-07**	**ADR-07** (*gateway* de IA na API), **ADR-16** (ADC e modelo *versus* região)
DR-06	**RF-20**–**RF-25**, **RNF-02**	**ADR-06** (Pub/Sub), **ADR-14** (Postgres particionado, sem Redis nem BigQuery)
DR-07	**RF-47**–**RF-49**, **RF-53**	**ADR-12** (`grau` como invariante de API), **ADR-10** (bem-estar sem visão de gestor)

4

Casos de uso

Os casos de uso descrevem o sistema pela ótica de quem o usa. Cada um recebe identificador estável, ator primário, pré-condição, fluxo principal, fluxos alternativos e a regra de negócio que o governa. A rastreabilidade para requisitos está na Seção 4.4.

4.1 Atores

Ator	Tipo	Descrição
Estudante	Primário	Estudante da rede. Escolhe a missão no hub próprio e entra na experiência. É titular de dado de menor.
Professor	Primário	Prepara e conduz a aula, lê a turma, gera plano de aula e faz o próprio *check-in* de bem-estar.
Gestor de escola	Primário	`gestor_escola`. Importa e mantém as pessoas, as turmas e o ano letivo da escola.
Gestor regional	Primário	`gestor_regional`. Tudo o que o gestor de escola faz, em uma escola da regional por vez; lê o panorama e o relatório da diretoria.
Gestor do contrato	Primário	`gestor_contrato`. Tudo o que o regional faz, com a rede inteira como escopo; mantém escolas e regionais, lê o agregado da rede e o histórico de recursos.
Admin (Unooria)	Primário	`admin`. Tudo o que o gestor do contrato faz, e mais o painel da Unooria: disponibilidade, quota de IA e cadastros de interesse (**ADR-88**).
Visitante do site	Primário	Quem preenche o cadastro de interesse do site público, sem conta (**ADR-107**).
Provedor de identidade	Secundário	WSO2 Identity Server (futuro). Hoje o papel de emissor é exercido pelo próprio hub.
Vertex AI (Gemini)	Secundário	Geração de texto e de imagem, sempre intermediada pela API.
Pub/Sub	Secundário	Transporte durável do canal analítico de telemetria.

CI/CD (GitHub Actions)	Secundário	Publica *bundles* e faz *upsert* do manifesto. É um ator: age no sistema por API autenticada.

4.2 Diagrama de casos de uso

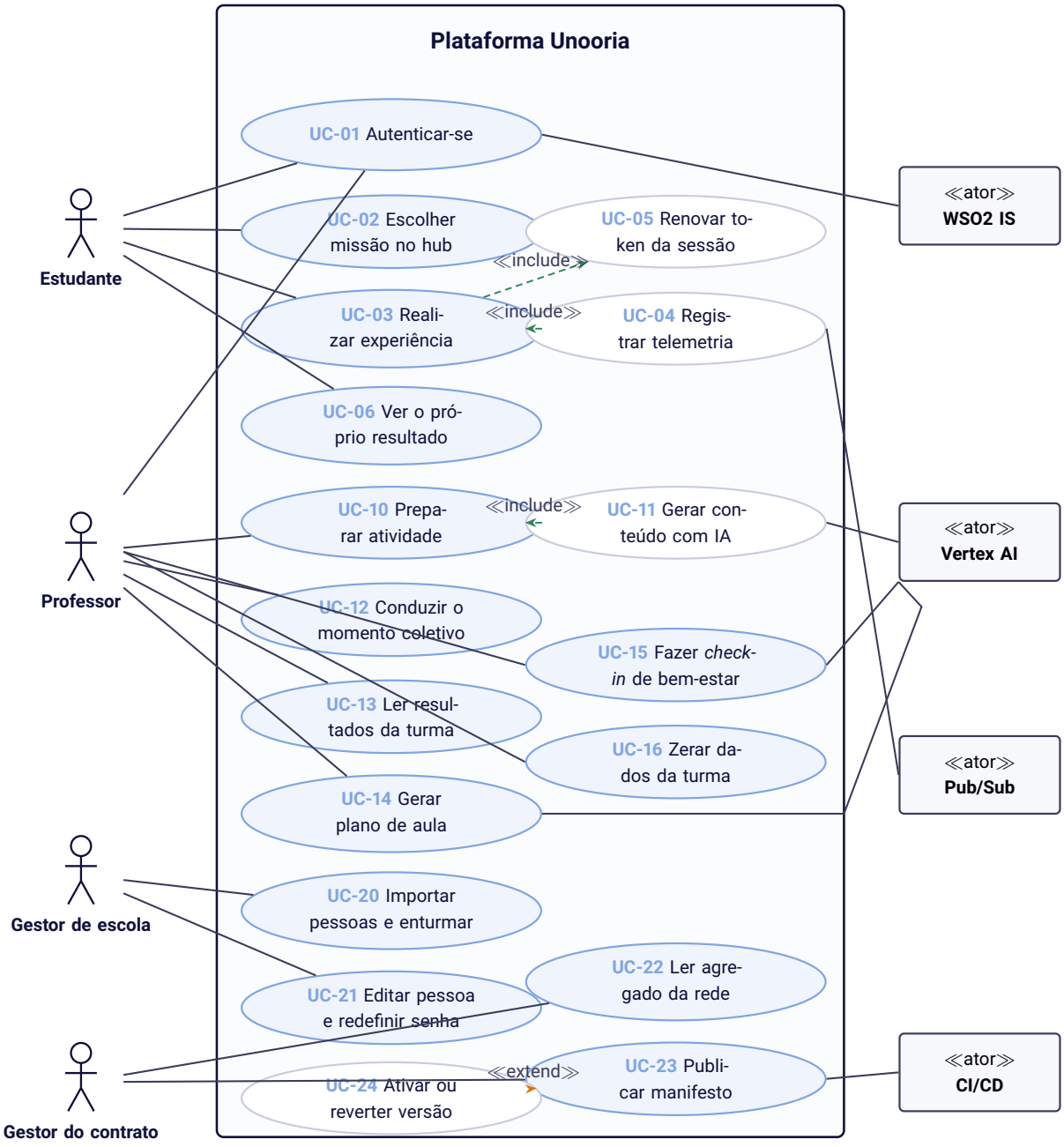


Figura 4.1 – Casos de uso da plataforma. As elipses em tom claro são casos de uso de *suporte*, incluídos por outros e nunca iniciados diretamente por um ator humano. Cada papel tem a sua área, e o *middleware* do hub redireciona quem entra na de outro. Os seis papéis, e a generalização entre os quatro de gestão, estão na Figura 5.11.

Legenda: caso de uso primário · caso de uso de suporte · - - - ≪include≫ · - - - ≪extend≫

4.3 Especificação dos casos de uso

4.3.1 UC-01 — Autenticar-se

> **UC-01 Autenticar-se**
>
> | | |
> |---|---|
> | **Ator primário** | Qualquer usuário |
> | **Pré-condição** | Usuário existe na escola (criado por UC-20) |
> | **Pós-condição** | Sessão do hub estabelecida; usuário na *home* do próprio papel |
> | **Requisitos** | **RF-01**, **RF-02**, **RF-05** |

Fluxo principal

1. O usuário informa credencial em `/login`.
2. O hub valida a credencial e cria a sessão, com `papel`, `escolaId`, `turmaId` e `serie`.
3. O *middleware* redireciona para a *home* do papel: `/estudante`, `/professor` ou `/admin/usuarios`.

Fluxo do PRIMEIRO acesso. Se a senha veio de uma importação ou de um reajuste (`senha_provisoria`), o passo 3 vira `/trocar-senha`, e não há saída: a mesma marca faz o BFF recusar dados e o `/api/exp-token` recusar *token* de experiência — só com o *middleware*, bastaria digitar `/exp/...` para pular. Trocada a senha, a sessão é **refeita** com a nova, porque o *cookie* antigo ainda diz que falta trocar (**ADR-29**).

Fluxos alternativos

- **A1 — credencial inválida:** o formulário permanece com mensagem; nenhuma sessão é criada.
- **A2 — usuário já autenticado acessa área de outro papel:** redirecionado para a *home* do próprio papel. *Não* recebe 403 — um estudante que digita `/professor` volta ao lugar dele sem mensagem de erro, o que é decisão de produto, não descuido de segurança (a autorização real é verificada de novo na API).
- **A3 — WSO2 como emissor:** o passo 2 passa a ser um *redirect* OIDC. Nada muda nos passos 1 e 3, nem na API.

4.3.2 UC-03 — Realizar experiência

Este é o caso de uso central da plataforma e o que mais determina a arquitetura.

> **UC-03 Realizar experiência**
>
> | | |
> |---|---|
> | **Ator primário** | Estudante (ou Professor, nas ferramentas de professor) |

Pré-condição	Sessão válida; experiência disponível para o papel
Pós-condição	Progresso e telemetria registrados; conclusão notificada ao hub
Inclui	UC-04 (telemetria), UC-05 (renovação de *token*)
Requisitos	**RF-12**–**RF-17**, **RF-40**

Fluxo principal

1. O estudante escolhe a missão em `/estudante` e o hub navega para `/exp/{expId}`.
2. O hub monta o *embed*: assina o *token* de experiência (RS256, 15 min), consulta o manifesto e resolve a URL versionada do *bundle*.
3. Se o manifesto declara `viewport`, o hub desenha a casca de aparelho em volta do `iframe`.
4. O *bundle* carrega, a SDK emite `exp:ready` e o hub responde `hub:init` com *token*, dados do estudante, `apiUrl` e `config`.
5. A experiência opera: chama rotas de domínio da API, registra telemetria em lote e salva progresso.
6. Ao terminar, emite `exp:concluir`; o hub registra progresso e libera a tela seguinte.

Fluxos alternativos

- **A1 — *token* perto de expirar:** a experiência emite `exp:token-expirando`; o hub busca `GET /api/exp-token` e responde `hub:token`. O `iframe` não recarrega e o estado não se perde.
- **A2 — versão de protocolo incompatível:** os dois lados verificam a versão *maior*. Sem `hub:init`, o *handshake* expira e a experiência mostra erro. Falha alta, nunca silenciosa.
- **A3 — sem manifesto (ambiente novo ou desenvolvimento):** o hub lê o `experiencia.json` servido ao lado do `index.html` do *bundle* para obter `viewport` e `permissoes`.
- **A4 — página do hub perde foco:** o hub emite `hub:pause`; a experiência congela cronômetros. Ao voltar, `hub:resume`.
- **A5 — experiência de professor acessada por `/exp/*`:** a rota responde 404. Ferramenta de professor não é missão de estudante.

> **ATENÇÃO**
>
> O passo 4 falhava de forma intermitente numa versão anterior da SDK: com os *assets* em *cache* (um F5), a experiência ficava pronta *antes* de o hub hidratar o *listener*, e o único `exp:ready` se perdia — o *handshake* expirava sem culpa de ninguém. A correção foi *re-enviar* `exp:ready` a cada 400 ms até o hub responder, e tratar `hub:init` repetido como evento normal, não como ruído.

4.3.3 UC-10 — Preparar atividade

UC-10 Preparar atividade (exp08ef1b)

Ator primário	Professor
Pré-condição	Sessão de professor válida
Pós-condição	Atividade `ativa`, com *feed* gerado e revisado
Inclui	UC-11 (geração com IA)
Requisitos	**RF-30**, **RF-31**, **RF-41**

Fluxo principal

1. O professor cria a atividade com tema, recorte e contexto de aula.
2. A API destila o antagonismo (dois lados) e propõe a pergunta-chave; o professor *aprova ou reescreve* — a origem fica registrada em `pergunta_origem`.
3. O professor dispara a geração. O servidor planeja os lotes (o *mix* é decisão do servidor; a IA apenas preenche) e o *worker* executa em segundo plano.
4. O professor acompanha o progresso por *polling*, revisa os posts, reprova ou regenera itens e ativa a atividade.

Fluxos alternativos

- **A1 — sem saldo de quota:** a geração é recusada *antes* da chamada ao provedor. Gastar *token* e depois recusar seria pagar pelo que não se entrega.
- **A2 — geração órfã (instância morta no meio):** o *polling* do painel detecta a fila órfã e redispara; itens `rodando` há tempo demais voltam à fila por `updatedAt`.
- **A3 — falha de geração de imagem:** o post recebe arte procedural de *fallback* e `image_status = "fallback"`. A atividade não trava.

4.3.4 UC-13 — Ler resultados da turma

UC-13 Ler resultados da turma

Ator primário	Professor
Pré-condição	Turma com pelo menos uma sessão registrada
Pós-condição	Nenhuma — é leitura
Requisitos	**RF-47**–**RF-49**, **RF-52**

Regras de negócio que governam este caso de uso — e que são a parte mais importante dele:

1. **A listagem da turma não mostra valor psicométrico.** Listagem é ordenável, e uma coluna "Poder" viraria ranking de crianças. Os valores vivem na tela individual, uma criança por vez.
2. **O relatório individual abre pelo nível do instrumento.** "Caiu em 2 de 3" sem dizer que era o nível C faz o adulto concluir que a criança é ingênua — exatamente o rótulo que o projeto existe para não produzir.
3. **Ausência de oportunidade nunca é lida como omissão.** Zero cartões aparecidos produz "as mensagens não chegaram nesta partida", nunca "ela ignorou tudo".
4. **Valor com uma única fonte de medida não recebe número.** A API não devolve o número; o hub não pode imprimi-lo.
5. **O agregado é de gestor; a linha de uma criança, não.** São *controllers* separados, e não um `@Roles` misto com exceção por método.

4.3.5 UC-14 — Gerar plano de aula

> **UC-14 Gerar plano de aula (app-90)**
>
> | | |
> |---|---|
> | **Ator primário** | Professor |
> | **Pré-condição** | Referências curriculares do contrato importadas |
> | **Pós-condição** | Plano `pronto` em armazenamento privado, editável |
> | **Requisitos** | **RF-35**, **RF-36**, **RF-34** |

Fluxo principal

1. O professor escolhe disciplina e série; a experiência chama `POST /planos`, que responde **202** imediatamente.
2. A API faz *claim* atômico da linha (`pendente` → `gerando`) e chama o modelo de saída longa com o material real — currículo da rede e as quatro unidades do ano, algo em torno de 190.000 *tokens* só de *prompt*.
3. O Markdown gerado é gravado imutável em `planos/{escolaId}/{id}/gerado.md`; a conversão para HTML alimenta o editor.
4. O professor edita e salva; o HTML é sanitizado por *allowlist* e gravado em `atual.html`. O versionamento de objetos do *bucket* dá histórico de cada salvamento sem uma linha de código.

Fluxos alternativos

- **A1 — saída cortada:** `finishReason = MAX_TOKENS` é *erro*, não conteúdo. O plano vai para `erro` com mensagem, em vez de entregar meio plano.
- **A2 — *restart* no meio da geração:** a linha fica `gerando` sem *worker*; após 12 minutos é considerada órfã e retomada, no máximo duas vezes.

- **A3 — restaurar o original:** o professor descarta a edição e volta ao `gerado.md`, que nunca foi sobrescrito.

4.3.6 UC-15 — Fazer *check-in* de bem-estar

> **UC-15 *Check-in* de bem-estar (app-91)**
>
> | | |
> |---|---|
> | **Ator primário** | Professor (e somente ele) |
> | **Pré-condição** | Sessão de professor válida |
> | **Pós-condição** | *Check-in* do dia registrado; ao sétimo, ciclo pontuado e classificado |
> | **Requisitos** | **RF-37**, **RF-53**, **RF-54** |

Fluxo principal

1. `GET /bemestar/hoje` abre o *check-in* do dia com três itens do DASS-21 — *verbatim*, na versão brasileira validada — e os textos de entorno gerados pela IA.
2. O professor responde os três itens (0–3) e, se quiser, escreve texto livre.
3. `POST /bemestar/checkins/:id/respostas` grava. No sétimo *check-in* a API soma por subescala ($\times 2$) e classifica pelas faixas publicadas.

Fluxos alternativos

- **A1 — texto de IA barrado, sem saldo ou sem *provider*:** entra o texto fixo do catálogo. O *check-in* nunca deixa de abrir.
- **A2 — sinal de apoio** (item 17 ou 21 ≥ 2, ou termo de risco no texto livre): mensagem *fixa* de apoio com CVV 188, SAMU 192 e CAPS/UBS, e a IA *não é chamada*.
- **A3 — ciclo expirado:** 21 dias sem fechar encerra o ciclo como `expirado`, sem pontuação.
- **A4 — apagar tudo:** `DELETE /bemestar/dados` remove ciclos e *check-ins* do professor.

> **REGRA INVIOLÁVEL**
>
> O instrumento é **fixo e *verbatim***. A IA nunca reescreve um item do DASS-21: reformular quebra a validação psicométrica e invalida as faixas de corte. O que a IA gera é o entorno — abertura do dia, pergunta aberta, acolhimento — e cada texto passa por validação antes de chegar ao professor. A IA nunca recebe nome, escola, pontuação ou faixa.

4.3.7 UC-23 — Publicar manifesto de experiência

> **UC-23 Publicar manifesto**
>
> | | |
> |---|---|
> | **Ator primário** | CI/CD (em nome de `admin-contrato`) |
> | **Pré-condição** | *Bundle* publicado em `gs://{contrato}-exp/{expId}/{versao}/` |
> | **Pós-condição** | Linha da tabela `experiencias` atualizada; hub passa a servir a nova versão |
> | **Estendido por** | UC-24 (reverter versão) |
> | **Requisitos** | **RF-10**, **RF-11** |

Fluxo principal

1. O CI sincroniza `dist/` para o caminho versionado do *bucket* e marca o `index.html` com `Cache-Control: max-age=60`.
2. O CI assina um *token* de cinco minutos com a chave de assinatura e chama `PUT /experiencias/{expId}` com o conteúdo do `public/experiencia.json`.
3. A API valida o *token* contra o JWKS que o *hub* publica e faz o *upsert*.

> **NOTA**
>
> A ordem importa e está codificada no *workflow*: as experiências são publicadas *depois* do hub, porque o *token* do passo 2 é validado contra o JWKS servido pelo hub. Hub fora do ar significa manifesto rejeitado — e um manifesto rejeitado é preferível a um manifesto aceito apontando para um hub que não subiu.

UC-24 — **Reverter versão:** o mesmo *script* com `--versao <anterior>`. Não há *build*, não há imagem, não há *deploy*: é um `UPDATE` de `versao_ativa`, e o próximo estudante que abrir a experiência recebe o *bundle* anterior. É a operação de emergência mais rápida da plataforma, e existe por desenho (**ADR-01**).

4.4 Rastreabilidade de casos de uso

UC	Requisitos	Superfície	Módulo da API
UC-01	**RF-01**, **RF-02**, **RF-05**	`/login`	`auth`
UC-02	**RF-10**	`/estudante`	`experiencias`
UC-03	**RF-12**–**RF-17**	`/exp/:expId`	`motor`, `unootube`, `vila`, `progress`
UC-04	**RF-20**–**RF-25**	SDK	`telemetry`
UC-05	**RF-14**	`/api/exp-token`	`auth`
UC-06	**RF-40**	dossiê da experiência	`dossie`

UC-10	**RF-41**	`/professor/atividades/:i`	`atividades`, `geracao`
UC-11	**RF-30–RF-33**	—	`ai`, `geracao`
UC-12	**RF-41**	painel de etapas	`atividades`, `motor`
UC-13	**RF-47–RF-52**	`/professor/entre-re`	`analise`, `vila`, `unootube`
UC-14	**RF-34–RF-36**	`/professor/plano-de-aula`	`planos`
UC-15	**RF-37**, **RF-53**	`/professor/bem-esta`	`bemestar`
UC-16	**RF-50**	botão de zerar turma	`vila`
UC-20	**RF-06**	`/admin/importar`, `/admin/turmas`	`rostering`
UC-21	**RF-07**	`/admin/usuarios` (editar e redefinir senha)	`usuarios`
UC-25	**RF-01**	`/trocar-senha`	`auth`
UC-26	**RF-06**	`/professor/turmas`	`rostering`
UC-27	**RF-06**	`/admin/escolas` (lotação em lote), `/admin/minhas-escol`	`rostering`
UC-28	**RF-07**	`/admin/turmas/:id` (perfil portátil)	`portabilidade`
UC-22	**RF-52**	`/analise`	`analise`
UC-23	**RF-10**	CI/CD	`experiencias`
UC-24	**RF-11**	*script* e `/unooria/recursos`	`experiencias`, `recursos`
UC-29	—	`/admin/usuarios` (balcão)	`usuarios`, `rostering`
UC-30	—	incluído por UC-29 e UC-31	`usuarios` (`regras.ts`)
UC-31	—	`/admin/usuarios` (papel sem escola)	`usuarios`
UC-32	—	seletor do cabeçalho (`/api/escola-ativa`)	`rostering`
UC-33	—	`/admin/minhas-escolas`	`rostering`
UC-34	—	`/admin/relatorios`	`rostering`
UC-35	—	`/admin/escolas`, `/admin/regionais`	`rostering`
UC-36	—	`/admin/recursos`	`recursos`

UC-37	—	`/unooria/recursos`	`recursos`
UC-38	—	`/unooria/recursos`	`recursos`
UC-39	**RF-31**	`/unooria/uso-de-ia`	`recursos`, `ai`
UC-40	—	`/unooria/prospeccao`	`prospeccao`
UC-41	—	site público (`/`)	`prospeccao`
UC-42	—	`/professor/turmas/[id]`	`salas`
UC-43	—	incluído por UC-42	`comum`, `recursos`
UC-44	—	`/professor/entre-redes/[ano]/[unidade]`	`equipes`

NOTA

Os casos UC-29 a UC-44 nasceram da modelagem UML (Figura 5.11, Seção 5.9.3) e, com exceção do UC-39, ainda não têm requisito funcional no Capítulo 3: disponibilidade, liberação por turma, equipes, panorama e prospecção entraram no produto por ADR, sem RF. Escrever esses RFs é pendência, e o travessão na coluna diz exatamente isso.

5

Arquitetura de *software*

Este capítulo descreve a organização interna do *software*: que peças existem, o que cada uma faz, como conversam e por que a fronteira está onde está. A arquitetura de *nuvem* — onde essas peças rodam — é o Capítulo 6.

5.1 Visões arquiteturais

O sistema é descrito por cinco visões complementares, no espírito do modelo 4+1 (KRUCHTEN, 1995). Nenhuma delas é suficiente isoladamente.

Visão	Notação	Onde está
Lógica	C4 níveis 1–3; UML: classes, objetos e o perfil da plataforma	Seções 5.3 a 5.6; 5.9.2 e 5.9.4
Processos	UML: sequência, estados, atividades, comunicação, tempo e visão geral de interação	Seções 5.8 e 5.9.6; Cap. 9
Desenvolvimento	Estrutura do *monorepo*; UML: pacotes, componentes e estrutura composta	Seções 5.7 e 5.9.5
Física / implantação	Diagramas de implantação e de rede; UML: implantação com artefatos	Cap. 6; Seção 5.9.7
Cenários	Casos de uso	Cap. 4; Seção 5.9.3

O C4 é a notação da arquitetura e continua sendo. A Seção 5.9 acrescenta os catorze tipos de diagrama da UML 2.5.1, cada um onde diz algo que o C4 não tem notação para dizer — tipo, estado, prazo, instância, interface, artefato (**ADR-122**).

5.2 Estilo arquitetural e princípios

A plataforma é, deliberadamente, um **monólito modular** de *backend* (uma API NestJS com módulos de domínio bem separados) servindo uma **constelação de artefatos estáticos** (as experiências). Não é microsserviços, e não é por acaso.

> **DECISÃO DE PROJETO**
>
> **Um serviço de *backend*, muitos artefatos de *frontend*** (**ADR-01**, **ADR-02**). Microsserviços resolvem escala organizacional — times independentes que precisam implantar sem coordenação. Aqui há uma pessoa. O que precisa escalar independentemente é a *entrega* das experiências, e isso foi resolvido tirando-as do plano de execução: *bundles* estáticos numa CDN escalam sem que exista um serviço. A modularidade que os microsserviços dariam por processo é obtida por *módulo NestJS* e por convenção de prefixo de tabela — com a vantagem de um `JOIN` continuar sendo um `JOIN`, e não uma chamada de rede.

Os princípios que atravessam todo o código:

Validação na fronteira, sempre.
Todo *input* externo passa por Zod (MCDONNELL, 2026): corpo, *query*, *postMessage* recebido e as variáveis de ambiente — estas no *boot*, de modo que configuração errada derruba o processo na subida, não no meio de uma requisição de estudante.

Uma porta por recurso crítico.
O banco só é acessado por `PrismaService.comTenancy`; a IA só por `src/ai`; o *design system* só pela ponte `packages/ui`; a entrada de pessoas e turmas só por `src/rostering` — que é também o único lugar da API que conhece o OneRoster, do mesmo jeito que só o *provider* conhece o fornecedor de IA (**ADR-28**). Recurso com duas portas tem duas chances de esquecer a tranca.

Regra de domínio pura, casca fina.
Cada módulo de experiência separa `catalogo.ts` (dados), `regras.ts` (funções puras, testáveis sem banco) e um *service* que persiste. O *controller* é só a casca HTTP.

Falhar alto, ou falhar fechado.
Incompatibilidade de protocolo falha alto e visível; rota cuja proteção depende de configuração ausente responde 401, nunca fica aberta.

O tipo carrega a regra.
Quando uma regra pode ser expressa no sistema de tipos, ela é — porque comentário não é controle (Seção 5.6).

5.3 C4 nível 1 — Contexto do sistema

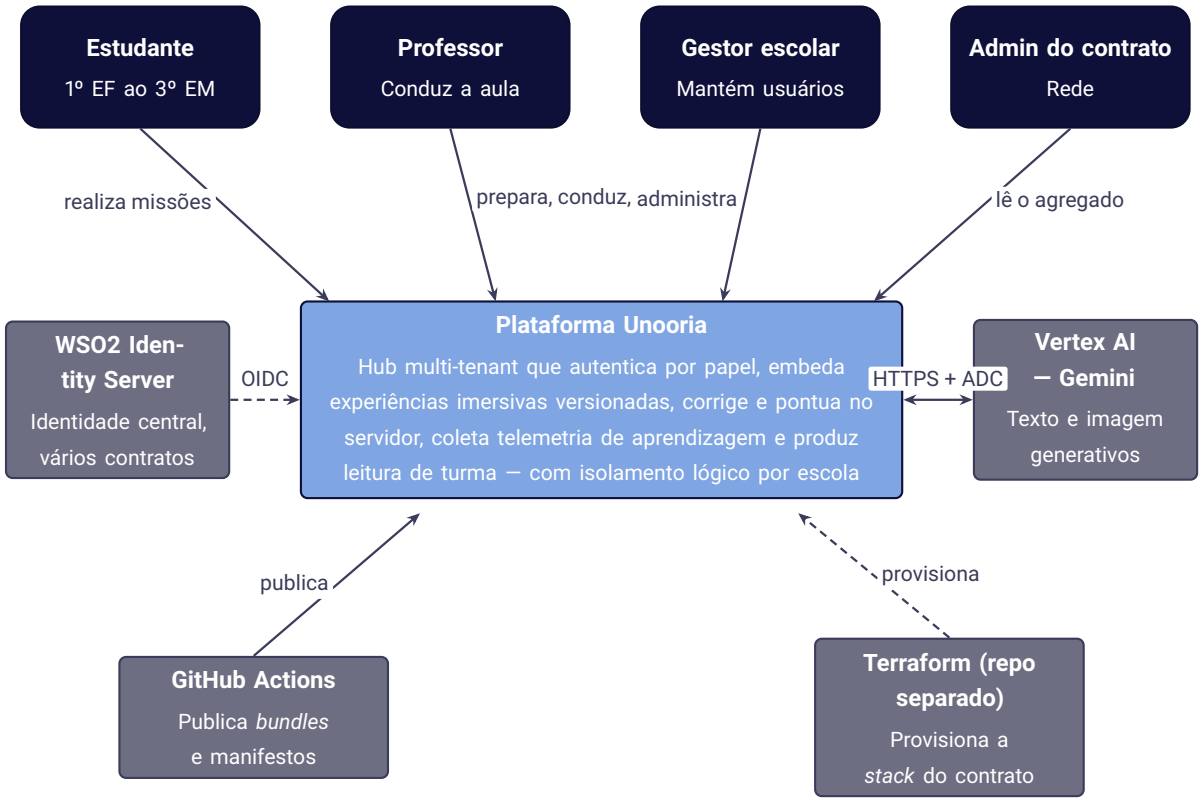


Figura 5.1 – C4 nível 1: contexto. Quatro papéis humanos mutuamente exclusivos, um provedor de identidade compartilhado entre contratos, um único serviço externo na dependência de IA, e dois atores de automação. O Terraform aparece porque a fronteira entre os dois repositórios é um contrato explícito, não um detalhe de operação.

> **NOTA**
>
> A dependência externa *não substituível em tempo de execução* é o Vertex AI, e mesmo assim sua indisponibilidade não derruba a plataforma: as experiências que não usam IA (exp04ef1b, app-70) seguem íntegras, o *feed* da exp08ef1b continua sendo curado a partir de conteúdo já gerado, e as ferramentas de professor degradam para texto fixo (app-91) ou recusam a geração com mensagem clara (app-90). Essa contenção de acoplamento é deliberada (**RNF-12**).

5.4 C4 nível 2 — Contêineres

O nível de contêineres é onde a assimetria da arquitetura aparece. Há *cinco* contêineres de execução e um número crescente de *artefatos* que não são contêineres — e tratá-los como se fossem seria o erro que este desenho existe para evitar. O quinto, desde 2026-09-30, é o agente de IA do professor no Vertex AI Agent Engine (**ADR-126**): o único contêiner que **só a API chama**, e o único escrito em Python.

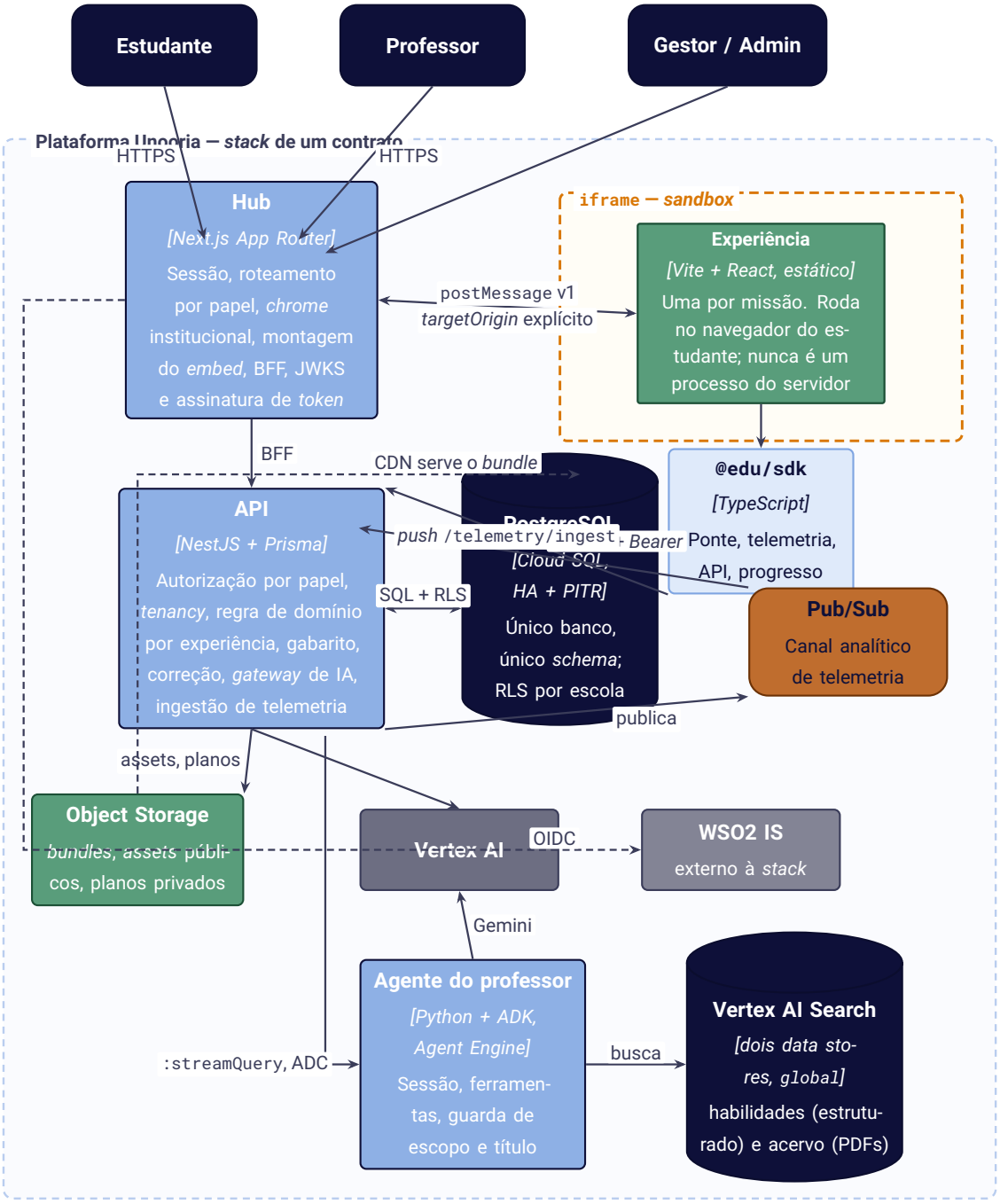


Figura 5.2 – C4 nível 2: contêineres. Só o hub, a API, o banco, o Pub/Sub e o agente de IA do professor são peças em execução; o agente roda no Vertex AI Agent Engine e só a API o chama — o navegador nunca fala com ele nem com os *data stores* (ADR-126). A experiência é um *artefato* servido pela CDN e executado no navegador; a SDK é biblioteca dentro dele. A linha tracejada laranja é a fronteira do *sandbox* do `iframe` — a fronteira de segurança mais importante do desenho de *frontend*.

Legenda: contêiner em execução · artefato estático · biblioteca · banco · fila · armazenamento de objetos · externo

5.4.1 Catálogo de contêineres e artefatos

Peça	Tipo	Tecnologia	Responsabilidade e limite
`apps/hub`	Contêiner	Next.js 16, App Router	Autenticar, roteirizar por papel, desenhar *todo* o *chrome* institucional, montar o *embed*, assinar *token* de experiência e publicar JWKS. **Não** contém regra de domínio de experiência.
`apps/api`	Contêiner	NestJS, Prisma	Autorizar, garantir *tenancy*, aplicar regra de domínio, guardar gabarito, corrigir, intermediar IA, ingerir telemetria. É o único que fala com o banco.
`apps/experiencias/*`	Artefato	Vite, React	Interface e interação de uma missão. **Nunca** guarda gabarito, nem calcula nota que valha, nem persiste *token*.
`packages/sdk`	Biblioteca	TypeScript puro, Zod	Contrato `postMessage`, cliente HTTP autenticado, telemetria em lote, progresso. É a *única* forma legítima de uma experiência falar com a plataforma.
`packages/ui`	Biblioteca	TypeScript, React	Ponte para o *design system*: marca `'use client'`, desfaz colisão de nomes entre seções e preserva *tree-shaking*.
PostgreSQL	Contêiner gerenciado	Cloud SQL	Único banco, único *schema* lógico. Guarda tudo, com RLS forçada.
Pub/Sub	Contêiner gerenciado	Pub/Sub	Amortecedor *durável* do pico de telemetria. Entrega *at least once* por *push* autenticado.
Object Storage	Artefato	Cloud Storage	Três *buckets* com regimes diferentes: *bundles* (público, imutável), *assets* (público, CDN), planos (**privado**, versionado).
`apps/agente-professor`	Contêiner gerenciado	Python 3.12, Google ADK, Vertex AI Agent Engine	O agente de IA do professor: instrução, ferramentas, guarda de escopo, título e a sessão (histórico). **Só a API o chama**; não conhece papel, escola nem quota — a posse da conversa é do Postgres (**ADR-126**).
Data stores	Contêiner gerenciado	Vertex AI Search, `global`	`agente-professor-habilidades` (estruturado, uma linha por habilidade) e `agente-professor-acervo` (PDFs, *chunking* por *layout*), alimentados do *bucket* `<projeto>-agente-professor` por `ingestao/` (**ADR-127**).

REGRA INVIOLÁVEL

Um banco, um *schema*. Nem banco por experiência — elas não são isoladas (o perfil da Vila alimenta a largada do Instanooria, e o PostgreSQL não faz `JOIN` entre bancos; além disso, um *pool* do Prisma por experiência estouraria `max_connections`) — nem *schema* por experiência: a única coisa que um *schema* daria além do prefixo é fronteira de `GRANT`, e

existe *uma role* de aplicação. A separação é por **prefixo de tabela**: o código da experiência (`exp08ef1b_`, `app_70_` — **ADR-110**) para tabela de experiência, sem prefixo para tabela de plataforma (Capítulo 8).

5.5 C4 nível 3 — Componentes

5.5.1 Componentes da API

A API é organizada em módulos NestJS. A Figura 5.3 separa-os em quatro faixas, e a faixa a que um módulo pertence determina o que ele pode conhecer. Desde 2026-09-02 a faixa transversal tem um quinto componente que a figura não desenha, `observabilidade/` (Capítulo 17): o *middleware* que abre o contexto da requisição antes dos *guards* (o `TenancyInterceptor` passou a *preencher* a escola nesse mesmo objeto, em vez de abri-lo), o *logger* pino com *redaction*, o `FiltroDeExcecao` global, o *sampler* por rota e o `otel.ts` — que é a **primeira importação** do `main.ts`, porque a instrumentação automática do OpenTelemetry precisa alterar `http`, `express`, `undici` e o Prisma antes de o Nest os carregar.

Há um segundo componente que a figura não desenha, de 2026-09-30: `agente-professor/`, o módulo do agente de IA do professor (**ADR-126**). Ele é de **plataforma**, não de experiência — o agente é um produto sem experiência, trancado por `@Produto("agente-professor")` —, e usa três portas: `comTenancy` para a posse da conversa (`agente_professor_conversas`), o `QuotaService` de `ai/` para o saldo e o consumo, e a `AgenteProfessorPort`, a segunda porta de `ai/`, que só `ai/agent-engine.client.ts` implementa. De `curriculo/` ele lê `habilidadePorCodigo`, como `evidencias/`, para conferir os códigos que o agente cita (**ADR-127**). O par Mermaid do Apêndice A traz as relações dele.

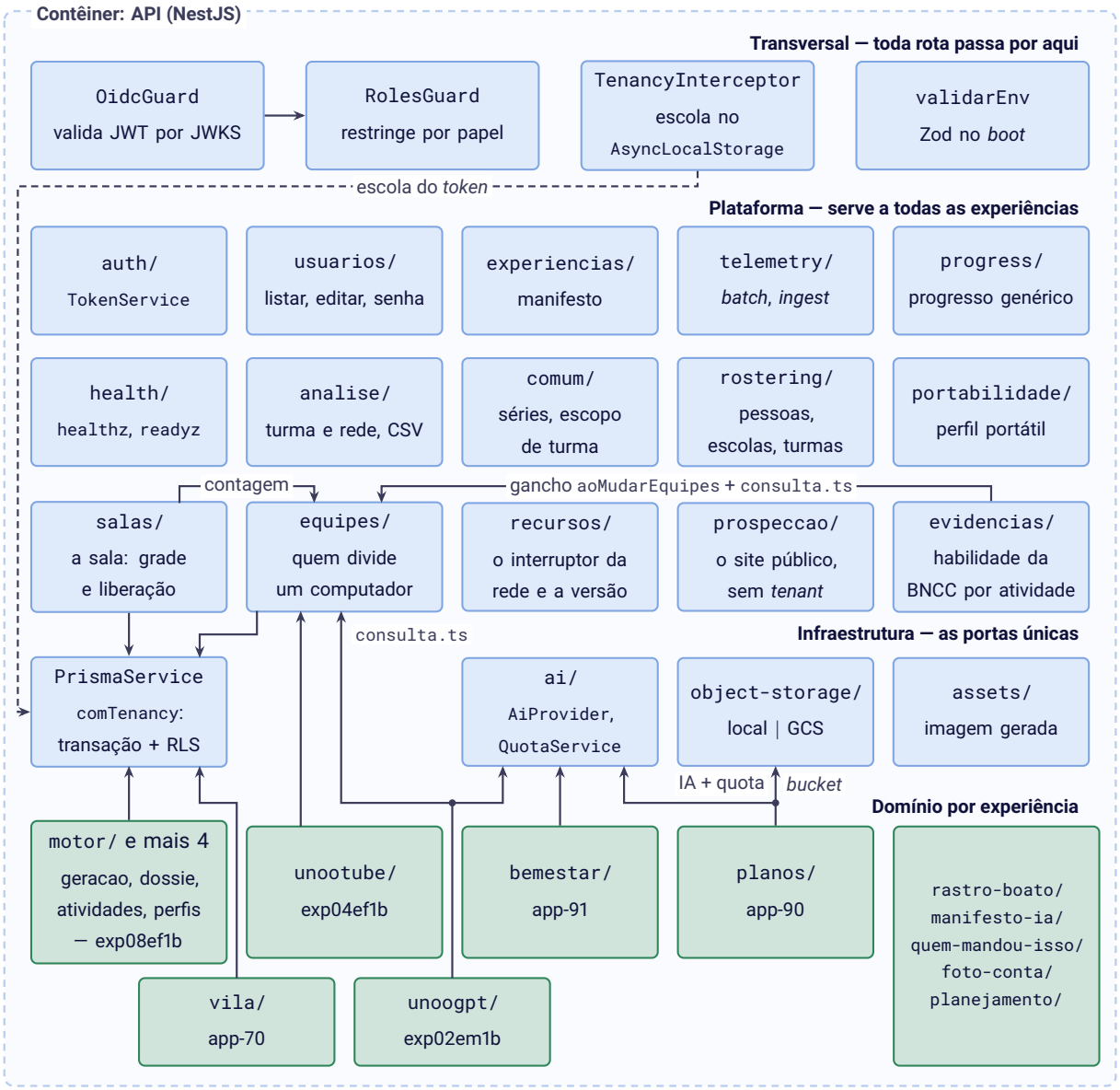


Figura 5.3 – C4 nível 3: componentes da API. A faixa transversal roda em toda rota; a de plataforma serve a todas as experiências; a de infraestrutura contém as portas únicas (banco, IA, armazenamento); a de domínio tem um módulo por experiência — `catalogo.ts` + `regras.ts` + *service* + *controllers* por papel —, e nenhum módulo dessa faixa conhece outro. `equipes/` (ADR-111) é lido por `salas/` (ADR-69), que mostra a contagem na sala, e pelos painéis de `unootube/` e `unoogpt/`, pela porta `equipes/consulta.ts`; `evidencias/` (ADR-112) lê a mesma porta e se registra no gancho `aoMudarEquipes`, e as missões o chamam pela porta `marcar` — hoje só `quem-mandou-isso/` (ADR-118), que também lê `equipes/consulta.ts` e usa IA com quota no preparo. A caixa da direita reúne, sem setas, os módulos da exp06ef2b, da exp08ef4b, da exp05ef1b, da exp01ef1b e da app-92, na ordem em que aparecem; as setas da exp05ef1b estão no par Mermaid, para não cruzarem a faixa de plataforma. Todo módulo com dado passa por `comTenancy`: as setas até o `PrismaService` são uma amostra, e a fonte Mermaid do Apêndice A lista as relações por módulo.

REGRA INVIOLÁVEL

> **NOTA**
>
> `unoogpt/` é o módulo de domínio mais fundo da API: 3.285 linhas de motor probabilístico, agente e regras puras, com subpastas (`modelo/`, `regras/`, `agente/`) em vez do par `catalogo.ts` + `regras.ts`. A granularidade veio do projeto de origem e foi **mantida de propósito**: cada arquivo tem teste próprio, e colar arquivos bem fatorados produziria colisão de nomes e falha mais difícil de localizar. O padrão do Capítulo 12 continua valendo para experiência nova; este é um porte.

`recursos/` é o terceiro eixo, e ele não é de *tenancy* (ADR-87). O catálogo (`experiencias/`) diz que forma a experiência tem; `salas/` diz se ela está aberta naquela turma; `recursos/` diz se ela EXISTE naquela rede — e é pré-condição dos outros dois. Ele expõe a leitura para qualquer autenticado e a ESCRITA só ao `admin` (ADR-88), em *controllers* separados; a guarda vive em `comum/`, ao lado da de liberação, porque é de dentro de transações de outros módulos que ela é chamada. A superfície dele no hub é a área `/unooria/*`, a única restrita a um papel que o `gestor_contrato` não tem.

A porta da sala é a exceção que confirma a regra. `salas/` é módulo de PLATAFORMA: ele não conhece missão nenhuma pelo nome — sabe turma, série e catálogo. E a guarda que as experiências chamam (`exigirMissaoLiberada`) NÃO mora nele, mora em `comum/liberacao.ts`. Fosse um *provider* do `salas/`, cada módulo de experiência teria de importá-lo, e uma dependência esquecida viraria uma porta aberta em silêncio (ADR-69).

`equipes/` é o segundo módulo de plataforma no mesmo molde (ADR-111). Ele diz QUEM DIVIDE UM COMPUTADOR em cada missão de cada turma, e a única coisa que sabe de missão é a lista `EXPERIENCIAS_COM_EQUIPE`. O que cada missão FAZ com a equipe — o painel contando por computador — é assunto do módulo dela: `unootube/` e `unoogpt/` leem `equipes/consulta.ts` (funções livres que recebem o `tx`, como `comum/liberacao.ts`) e aplicam `colapsarEmUnidades`, de `equipes/regras.ts`, sem injetar provedor nenhum. O sentido inverso — a missão dizendo ao sorteio o lado de cada criança — é um REGISTRO: o módulo da missão chama `EquipesService.registrarFonteDeGrupo` na subida, e `equipes/` nunca importa nada dele. No hub, a seção mora na ficha da missão (`components/equipes/`), e as chamadas passam pelo BFF (`/api/plataforma/equipes/*`), do lado do cliente porque o professor monta as equipes na frente da sala.

`evidencias/` é o terceiro, e o primeiro que as missões CHAMAM (ADR-112). Ele guarda que habilidades da BNCC cada estudante demonstrou numa atividade de uma missão, rodada a rodada, e não conhece missão nenhuma pelo nome: o que cada uma sabe de si entra por três pontos. A **ficha** é dado estático do módulo (`evidencias/fichas/<expId>.ts`, no registro `FICHAS`, que tem uma só: a da exp05ef1b, ADR-118), porque o gabarito não pode ir para o *bundle*. O **contexto** — em que turma, atividade e rodada o estudante está agora — é um REGISTRO, como a fonte de lado das equipes: o mó-

dulo da missão chama `EvidenciasService.registrarContexto(expId, resolver)` na subida. E a **marca** é uma chamada, `EvidenciasService.marcar(tx, …)`, feita DA transação da missão, sem abrir `comTenancy` — a marca cai junto se a jogada cair. Por isso o módulo exporta o *service*, que é a porta das missões, e a dependência vai num sentido só: a missão conhece a evidência, e `evidencias/` nunca importa módulo de missão. Com `equipes/` o arranjo é o inverso: `evidencias/` importa o `EquipesModule`, lê `equipes/consulta.ts` (`mapaDeEquipes`, `computadoresDaMontagem`) e registra o recálculo das cópias no gancho `EquipesService.aoMudarEquipes`, que o `salvar` e o `desfazer` chamam dentro da própria transação; `equipes/` não sabe que a evidência existe. As regras moram em `evidencias/regras.ts`, puras, e os códigos da BNCC vêm de `curriculo/` (`habilidadePorCodigo`). No hub, o painel "Habilidades da BNCC" é genérico, desenhado só quando a resposta diz `temFicha: true`: entra na página da atividade do Instanooria (que ainda não tem ficha) e nos resultados da exp05ef1b, onde a atividade é o pacote aprovado — é lá que ele aparece hoje. O perfil do estudante é uma página nova, aberta pela aba Estudantes da turma.

Módulo de domínio não conhece módulo de domínio. `vila/` não importa `unootube/`. Quando duas experiências precisam se comunicar — o perfil da Vila alimentando a largada do Instanooria — a comunicação é por *tabela de contrato* explícita (`exp08ef1b_app_70_perfis_importados`, que carrega os dois números no nome) e por um *endpoint* de fronteira (`POST /atividades/:id/perfis/da-vila`). Chamar a tabela por um número só mentiria sobre quem depende dela.

5.5.2 Componentes do hub

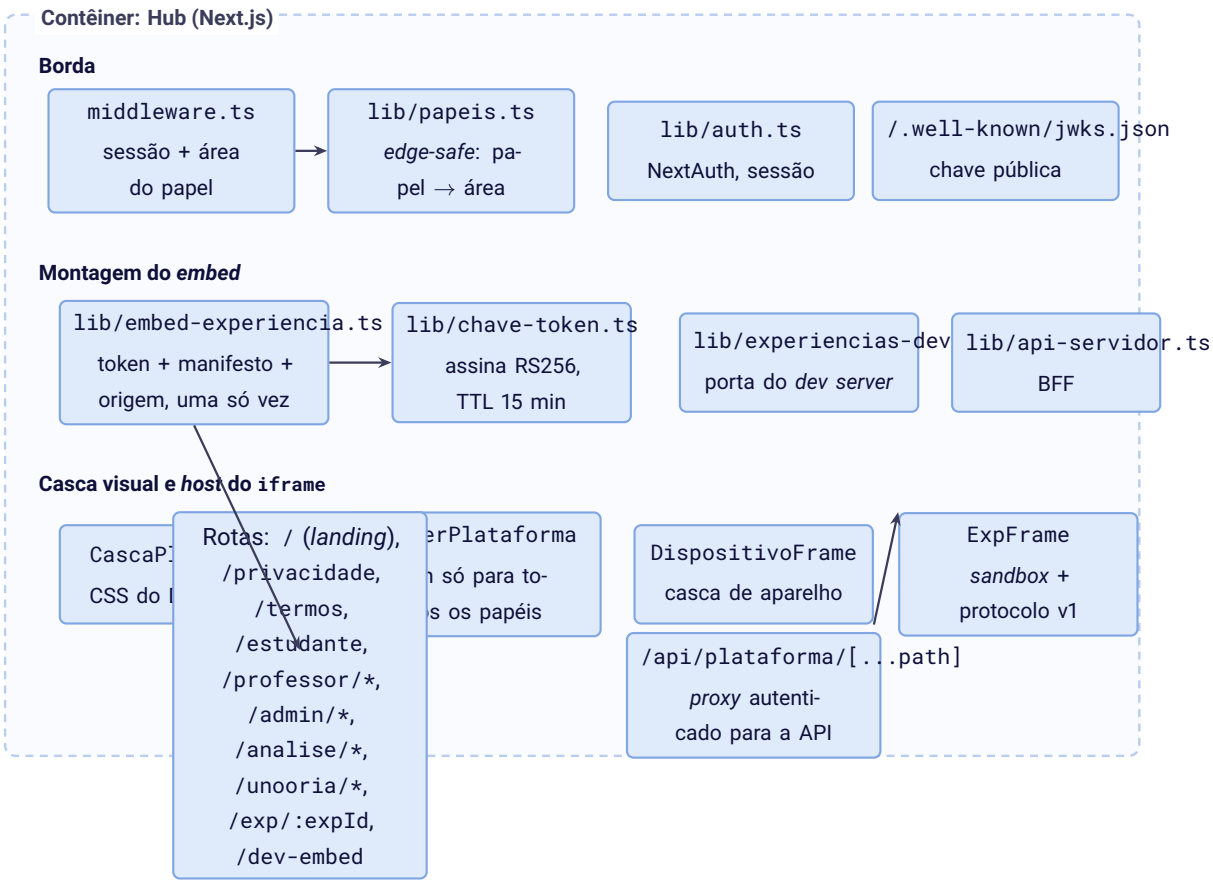


Figura 5.4 – C4 nível 3: componentes do hub. A montagem do *embed* é *uma só função* usada pelas quatro superfícies que embedam (`/exp/[expId]`, `/dev-embed`, `/professor/plano-de-aula` e `/professor/bem-estar`) — duplicá-la era o caminho mais curto para uma delas passar a assinar o *token* com *claims* diferentes.

DECISÃO DE PROJETO

Um único *header* para todos os papéis, e uma única casca visual. Antes, cada área copiava o próprio *header* e a experiência não tinha nenhum. Hoje `HeaderPlataforma` serve `/estudante`, `/professor`, `/admin`, `/analise` e `/exp/*`, e a casca visual saiu do *layout* do grupo de rotas para `CascaPlataforma`, porque `/exp/*` e `/dev-embed` estão fora do grupo e também precisam dela. A regra de ouro passou a valer um nível abaixo: o *header* é *chrome* do hub, desenhado *acima* do aparelho — dentro do `iframe` nada da marca institucional entra.

DECISÃO DE PROJETO

O *layout* de cada área é onde moram as regras que valem para a área inteira. São três hoje, e nenhuma pertence à página: a **trilha** (ADR-77), a **largura da coluna** (ADR-83) e a **fronteira de remontagem por escola ativa** (ADR-85, o `EscopoDaEscola` que envolve o miolo). As três chegaram ali pelo mesmo caminho: eram responsabilidade de cada página,

e responsabilidade de página se esquece — 15 telas sem *breadcrumb*, quatro larguras de coluna sem razão registrada e quatro telas que não reagiam à troca de escola. **Área nova precisa de *layout* novo**, e é o *layout* que faz as três coisas; sem ele a área nasce com os três defeitos, como `/admin` nasceu.

DECISÃO DE PROJETO

O chat do agente de IA do professor é do *layout* de `/professor`, não de uma página (**ADR-126**). O botão flutuante (`ChatLauncher` do DS) e o painel não modal (`ChatPanel`, compacto ou expandido) moram em `components/agente-professor/`, montados pelo *layout* da área só quando o produto `agente-professor` está ligado para a escola — o produto não tem *card* na *home*: a superfície é o botão. Duas peças fogem do desenho do resto do hub. A **rota de *stream*** (`/api/plataforma/agente-professor/…`) é própria, fora do BFF genérico, porque ele tem teto de 180 s e devolve o corpo sem os cabeçalhos de *stream* (**ADR-128**); a sessão e o *token* curto são os mesmos (`lib/bff.ts`). Listar, criar, ler e apagar seguem pelo BFF. E o ***markdown* vira HTML no navegador**, com `react-markdown` sem HTML cru, porque o texto chega em pedaços (**ADR-129**). O painel leva `data-clarity-mask`.

NOTA

A raiz é a *landing* pública, e continua roteando quem tem sessão (**ADR-40**). `/` sem sessão renderiza `components/landing/` — porte da tela desenhada no Stitch, com a estrutura, a copy e o movimento do modelo (navegação fixa em vidro, *shader* WebGL de fundo, revelação ao rolar) e a cor, a tipografia, os ícones, o logo e os botões do DS. Com sessão, `/` redireciona para a *home* do papel (ou para `/trocar-senha`): o `login-form` e a troca de senha fazem `router.replace("/")` e dependem desse roteamento para cair na área certa sem o *client* conhecer o papel. A *landing* está fora do `matcher` do *middleware* e não lê nada da sessão. Os dois pedaços com ciclo de vida (`FundoAnimado`, `RevelarAoRolar`) são *client components*; o resto é servidor.

NOTA

As páginas legais são a mesma superfície pública da *landing* (**ADR-70**). `/privacidade` e `/termos` completam o conjunto público do hub — com `/`, `/login` e o JWKS, é tudo que existe sem sessão. Elas são alcançadas pelo rodapé da *landing*, que até então oferecia os dois links com `href="#"`, e por isso reaproveitam a casca dela: `components/legal/` importa `landing.css` e monta o mesmo `FundoAnimado`, a mesma navegação em vidro e a mesma escala tipográfica. Duplicar aquilo criaria a segunda cópia, que diverge na primeira mudança de marca.

O conteúdo entra como *dado* (uma lista de seções com `id`, `titulo` e `conteudo`), não como marcação livre: é o que mantém o sumário lateral em sincronia com o texto — numeração, âncora e ordem saem da mesma lista que renderiza as seções, então não existe o estado

em que o índice promete um item que o corpo não tem. O único *client component* é o `SumarioAtivo`, que acende o item em leitura.

5.6 C4 nível 4 — Código

O nível 4 do C4 raramente vale a pena desenhar por inteiro. Vale aqui para *três* lugares, porque neles o desenho de tipos *é* a arquitetura.

5.6.1 O contrato da ponte e o *guard* de *tenancy*

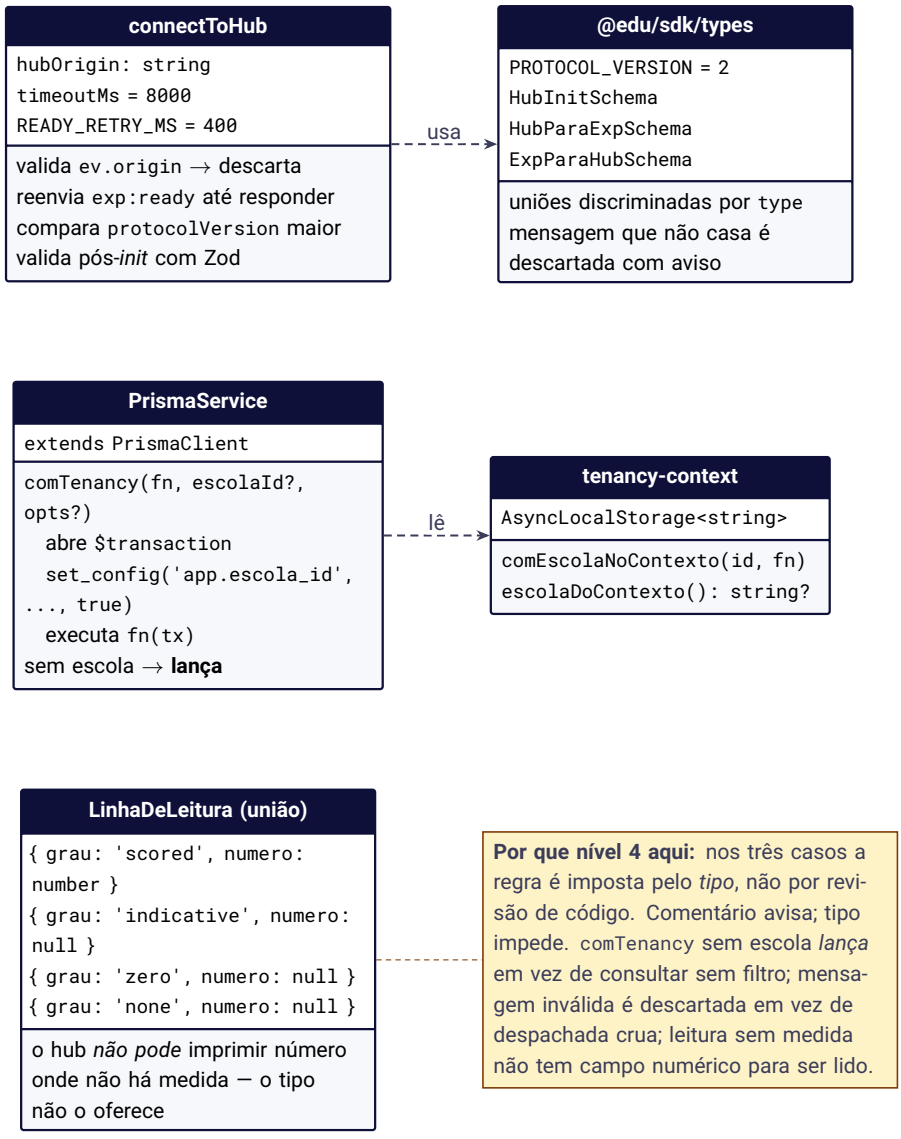


Figura 5.5 – C4 nível 4: os três pontos em que a regra arquitetural está no sistema de tipos. `comTenancy` é a única porta do banco e falha alto sem escola; o contrato da ponte descarta o que não valida; a leitura de valores não oferece número onde não há medida.

5.6.2 O gabarito de um módulo de domínio

Todo módulo de experiência segue a mesma forma. Não é convenção estética: é o que permite testar a regra sem banco e trocar a persistência sem tocar a regra.

Listing 5.1 – Forma canônica de um módulo de domínio de experiência

```
apps/api/src/<slug>/
  catalogo.ts     // DADOS: itens, textos, gabarito. Constantes puras.
  regras.ts       // REGRA: funcoes puras. Sem Prisma, sem fetch, sem Date.
  <slug>.service.ts   // PERSISTENCIA: le com lock, aplica a regra, grava.
                      // Toda query dentro de comTenancy e filtrando escola_id.
  <slug>.controller.ts        // casca HTTP do papel "estudante"
  <slug>-professor.controller.ts  // casca HTTP do papel "professor"
  <slug>.module.ts

// A separacao de controllers POR PAPEL e deliberada: um @Roles misto com
// excecao por metodo e o caminho mais curto para um GET de professor herdar
// silenciosamente a permissao de um POST de estudante.
```

NOTA

A consequência prática dessa forma aparece nos testes: `vila-regras.spec.ts`, `unootube-regras.spec.ts` e `bemestar-regras.spec.ts` rodam *milissegundos* sem PostgreSQL, porque exercitam `regras.ts` puro. Os testes que precisam de banco (`*-postgres.spec.ts`, `tenancy.spec.ts`) são separados e cobrem exatamente o que não dá para afirmar sem ele: *tenancy*, *lock* e concorrência.

5.7 Estrutura do *monorepo*

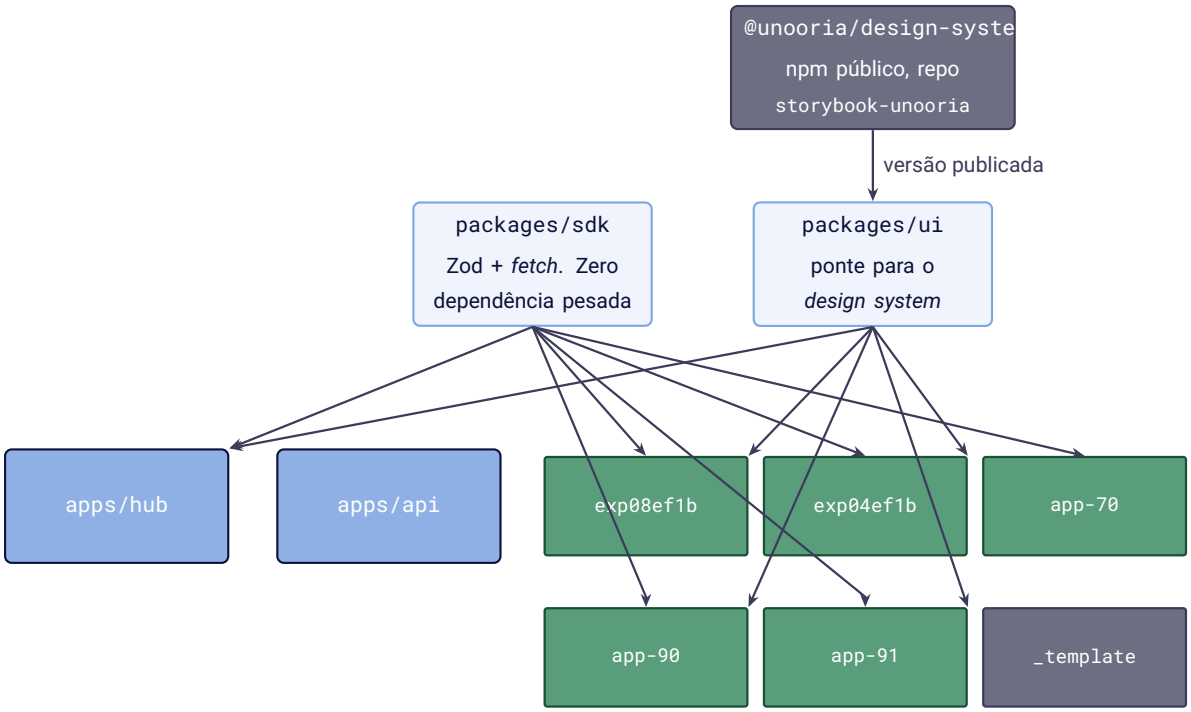


Figura 5.6 – Grafo de dependências do *monorepo*, com uma amostra das experiências. A SDK é consumida pelo hub e por toda experiência — é por isso que uma mudança nela precisa testar todas no mesmo *pull request*, e por isso o *monorepo* (**ADR-02**). A API não importa pacote nenhum do *workspace*: valida o que recebe com os próprios *schemas* (Figura 5.15). A app-70 não depende de `packages/ui`: é jogo em *canvas* 3D, sem DOM de interface, e é a exceção registrada em ADR à regra do *design system*.

Diretório	Conteúdo
`apps/hub`	Aplicação Next.js: rotas por papel, *embed*, BFF, JWKS.
`apps/api`	Aplicação NestJS: 34 módulos, Prisma, 40 migrações (contagem de 2026-09-25).
`apps/experiencias/*`	Uma pasta por experiência, mais `_template`.
`packages/sdk`	Contrato da ponte e telemetria.
`packages/ui`	Ponte do *design system* e CSS da plataforma.
`docs/`	Contratos com infraestrutura e registro de decisões.
`scripts/`	*Scaffolding*, chave de assinatura, importação de referências, a suíte de ponta a ponta (`scripts/e2e/`), *probe* de *pointer lock*.
`wso2/`	*Compose* próprio do provedor de identidade, com ciclo de vida separado.
`documentacao-latex/`	Este documento.

A orquestração é Turborepo (Vercel, 2026b), com *pipeline* declarado em `turbo.json`: `build` depende do `build` das dependências; `typecheck` e `test` também. É isso que permite ao CI rodar apenas o que mudou:

Listing 5.2 – O comando que o CI executa

```
pnpm turbo typecheck lint test build --filter='...[HEAD^]'
```

5.8 Fluxos principais

5.8.1 Curadoria de um bloco de *feed* (exp08ef1b)

Este é o fluxo com mais concorrência da plataforma e o que mais exige do banco.

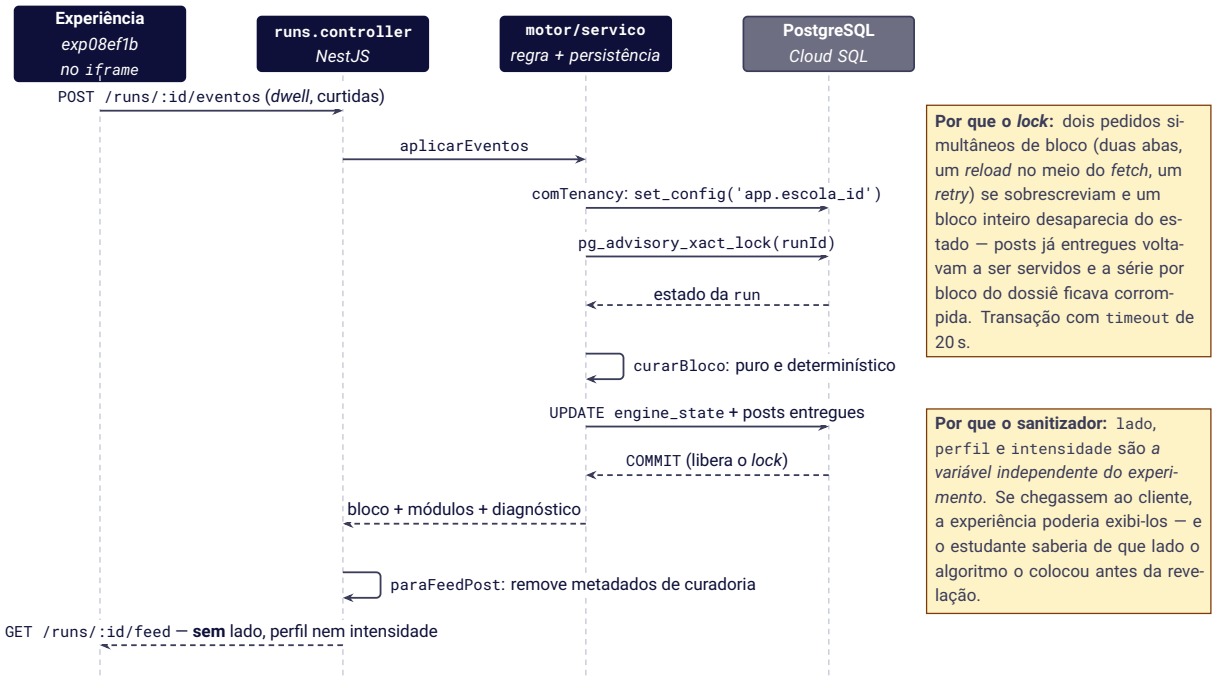


Figura 5.7 – Sequência da curadoria de um bloco de *feed*. A regra (`curarBloco`) é pura; o *service* lê com *lock*, aplica e grava; o *controller* sanitiza. As três responsabilidades ficam em arquivos diferentes de propósito.

5.8.2 Estados de uma geração assíncrona

Duas rotinas da plataforma são assíncronas e longas: a geração de *feed* da exp08ef1b e a geração de plano de aula da app-90. Ambas seguem a mesma máquina de estados, com *claim* atômico em vez de fila externa.

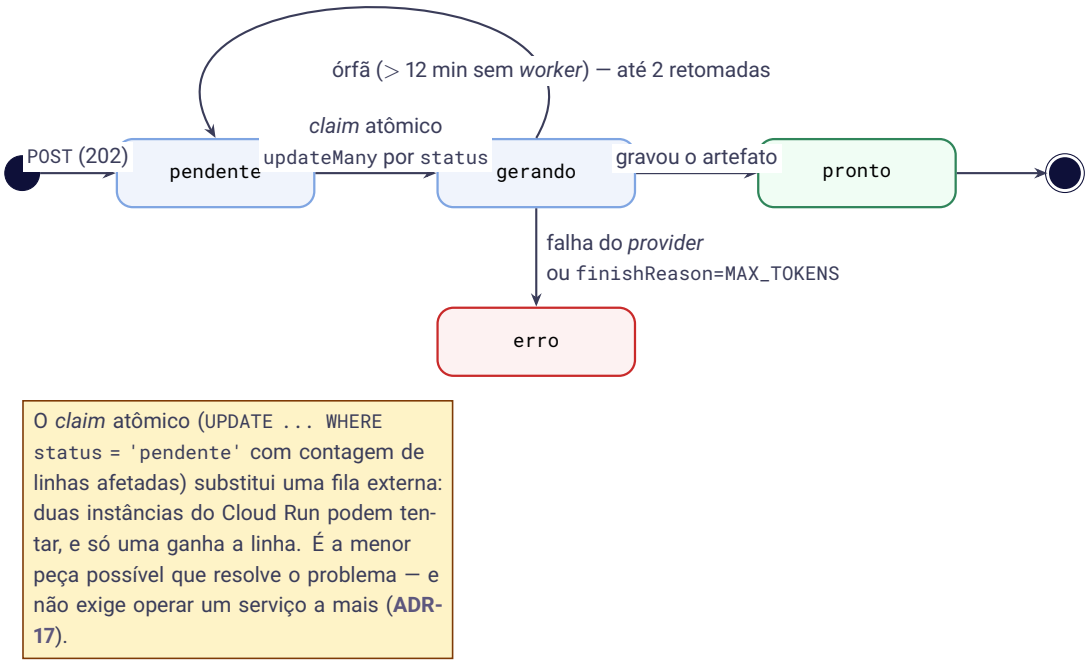


Figura 5.8 – Máquina de estados das gerações assíncronas (exp08ef1b e app-90). O *claim* atômico dispensa fila externa; a retomada de órfã é disparada pelo *polling* do próprio painel, não por um *scheduler*.

LIMITAÇÃO CONHECIDA

Não há retomada no *boot*. Se todas as instâncias morrerem e ninguém abrir o painel, uma geração órfã fica parada até que alguém abra. É aceito porque o *polling* do painel é justamente o momento em que alguém se importa com aquela geração — e porque a alternativa é um *scheduler*, que é uma peça nova para operar. O gatilho de reavaliação é a primeira reclamação real de geração parada.

5.9 Modelagem UML

O C4 das seções anteriores responde *que peças existem e quem fala com quem*. Ele não tem notação — por escolha de quem o desenhou (BROWN, 2026) — para o que uma peça *é* (tipo, multiplicidade, herança, interface), para o que ela *faz ao longo do tempo* (estado, prazo, ordem de mensagens) nem para *uma situação concreta* (instâncias com valores). É o que a UML cobre. Esta seção desenha a plataforma nos catorze tipos de diagrama da UML 2.5.1 (Object Management Group, 2017), distribuídos pelas cinco visões do modelo 4+1 (KRUCHTEN, 1995) que abrem este capítulo (**ADR-122**).

Nem todo tipo rende o mesmo, e a seção não finge que rende. Onde o documento já tinha um diagrama daquele tipo — casos de uso, sequência, máquina de estados, atividades —, a Tabela 5.4 aponta para ele e a seção acrescenta só o que faltava. Onde não tinha, cada diagrama foi posto no lugar em que o tipo mostra algo que nenhum outro mostra. O de tempo, por exem-

plo, é o único em que a janela de 401 entre o *token* de 15 minutos e o pedido de renovação a cada 10 aparece sem conta de cabeça (Figura 5.23).

> **REGRA INVIOLÁVEL**
>
> **Diagrama que diverge do código é pior do que diagrama nenhum**, porque quem lê acredita nele. Cada figura desta seção cita os arquivos de onde saiu, e todas foram conferidas contra o código em 2026-09-27. Quem muda a peça muda o diagrama e o par Mermaid do Apêndice A no mesmo *commit*. Os nomes são os do código (`QuotaService`, `exigirMissaoLiberada`), porque é por eles que se procura; o texto em volta é em português.

5.9.1 Os catorze diagramas e onde cada um está

A UML separa os diagramas em dois ramos. Os **de estrutura** descrevem o que existe e como se compõe, sem tempo. Os **de comportamento** descrevem o que acontece, e quatro deles — os **de interação** — descrevem o que acontece *entre* participantes, por mensagens. A Figura 5.9 é a própria taxonomia desenhada como diagrama de classes, com o número das figuras deste documento em cada folha.

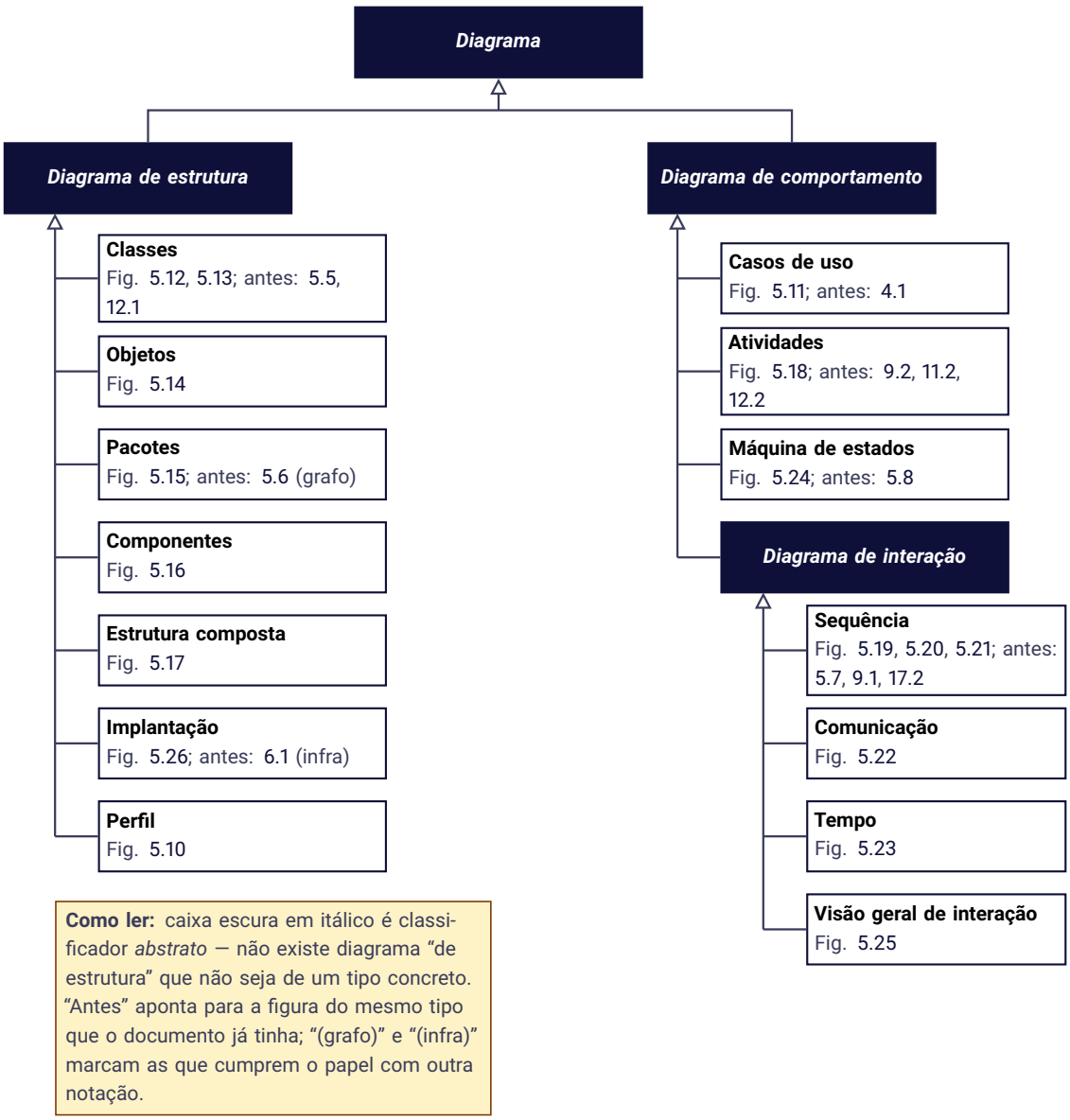


Figura 5.9 – Os catorze tipos de diagrama da UML 2.5.1, desenhados como a especificação os organiza: uma hierarquia de generalização com três classificadores abstratos. Cada folha traz o número das figuras deste documento daquele tipo.

A Tabela 5.4 diz, para cada tipo, a pergunta que ele responde *nesta* plataforma — que é o critério pelo qual cada exemplar foi escolhido.

Tabela 5.4 – Os catorze diagramas UML e a pergunta que cada um responde aqui

Diagrama	Visão 4+1	A pergunta que ele responde aqui	Figuras
Estrutura			
Perfil	—	Que estereótipos a plataforma acrescenta à UML, e que restrição cada um carrega?	5.10

Diagrama	Visão 4+1	A pergunta que ele responde aqui	Figuras
Classes	Lógica	Que conceitos existem, com que multiplicidade — e onde a regra é restrição do modelo e não só do código? A porta da IA: que interface, que implementações, quem decora quem?	5.12, 5.13
Objetos	Lógica	Numa rede concreta, com três linhas em `disponibilidades`, o que fica ligado e por quê?	5.14
Pacotes	Desenvolvimento	Quem pode importar quem no *monorepo* e dentro da API?	5.15
Componentes	Desenvolvimento	Que interface cada peça *fornece* e qual *requer* — quais delas são HTTP e qual é `postMessage`?	5.16
Estrutura composta	Desenvolvimento	Por dentro da página da missão: que partes, que portas, e por onde o *token* atravessa o *sandbox*?	5.17
Implantação	Física	Que artefato roda em que nó, e por que caminho de comunicação?	5.26
Comportamento			
Casos de uso	Cenários	Quem administra a rede, e o que cada papel da cadeia pode fazer que o de baixo não pode?	5.11, 4.1
Atividades	Processos	Qual é o algoritmo que decide se um recurso está ligado, e que parte dele é pura?	5.18
Máquina de estados	Processos	Em que estados a ponte pode estar, do lado da experiência, e o que a tira de cada um?	5.24, 5.8
Sequência	Processos	Em que ordem as duas travas da entrada na missão disparam? E a quota da IA: o que roda antes e o que roda depois do *provider*?	5.19, 5.20, 5.21
Comunicação	Processos	Que objetos um lote de telemetria atravessa, em que ordem numerada, até virar linha no banco?	5.22
Tempo	Processos	Quando o *token*, a sessão e o relógio da SDK vencem, um em relação ao outro?	5.23
Visão geral de interação	Processos	Como as interações anteriores se encadeiam numa jornada do estudante, do *login* à missão?	5.25

5.9.2 O perfil da plataforma

Um **perfil** é o mecanismo que a UML oferece para uma plataforma estender a linguagem sem alterá-la: cada *estereótipo* se aplica a uma metaclasse da UML (`Class`, `Component`…) e carrega *valores etiquetados* e *restrições*. Ele vem primeiro porque os diagramas seguintes usam os estereótipos dele, e porque escrever as regras invioláveis do `AGENTS.md` como restrição de estereótipo mostra a que elemento cada uma se aplica. As restrições são as do código: `escola_id NOT NULL` e RLS forçada nas tabelas com dado de estudante (Seção 5.6 e Capítulo 7), a guarda

de liberação no funil das rotas de estudante (ADR-69), a quota antes e depois da IA (ADR-105) e o caminho imutável do *bundle*.

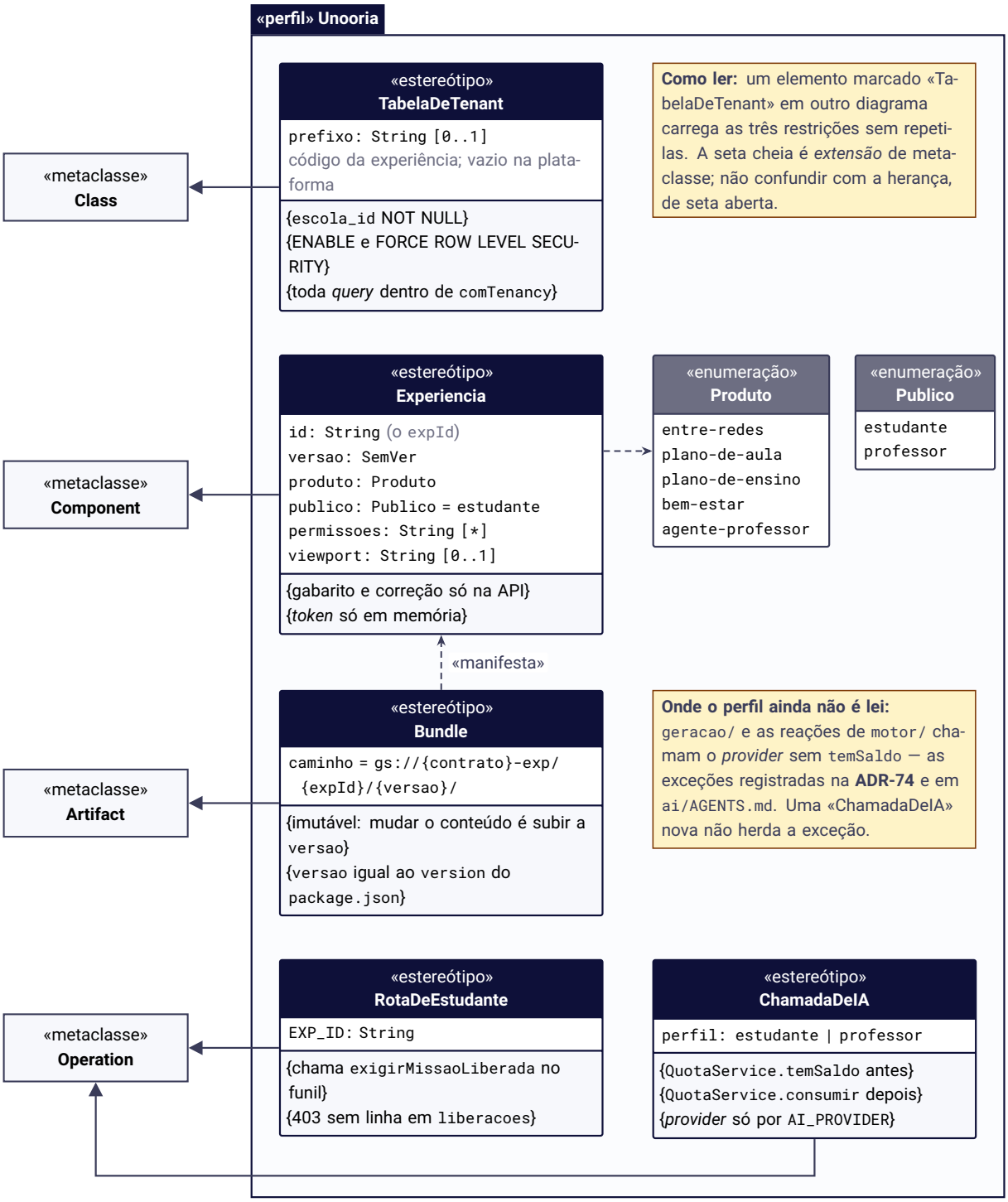


Figura 5.10 – O perfil UML da plataforma: cinco estereótipos sobre quatro metaclasses. As restrições entre chaves são as regras invioláveis do `AGENTS.md` ditas no lugar a que se aplicam. Fontes: `recursos/catalogo.ts` (produtos), `comum/liberacao.ts`, `ai/quota.service.ts`, `prisma/prisma.service.ts` e `public/experiencia.json` de cada experiência.

5.9.3 Visão de cenários: a administração da rede

O diagrama de casos de uso do Capítulo 4 (Figura 4.1) cobre a sala de aula e foi desenhado antes do modelo de seis papéis (**ADR-80**). A Figura 5.11 cobre o que ele não cobre: quem administra a rede. A notação que carrega a regra é a **generalização de atores**. Um ator filho faz tudo o que o pai faz, e o código confirma a cadeia inteira: toda rota `GESTORES` abre para os quatro papéis de gestão, toda rota `GESTORES_DO_CONTRATO` inclui o `admin`, e as únicas rotas que separam um papel do de cima são `@Roles("admin")` (o painel da Unooria, **ADR-88**) e o relatório da diretoria (`gestor_regional` em diante, **ADR-82**). O que muda de um papel para o outro, fora essas rotas, é o *escopo* e não a capacidade: o `gestor_regional` trabalha uma escola da regional por vez, e o `gestor_contrato` escolhe a escola por `?escolaId=` porque o *token* dele não tem escola.

A cadeia de *atribuição* é outra relação e não se confunde com a generalização. Ela diz quem cria quem (`ATRIBUI_DIRETAMENTE`, em `usuarios/regras.ts`), e entra no diagrama como o caso de suporte **UC-30**, incluído pelos dois caminhos de cadastro. O **UC-29** (balcão) não escreve em `users`: monta uma linha de planilha e inclui o **UC-20**, o importador (**ADR-54**, **ADR-59**).

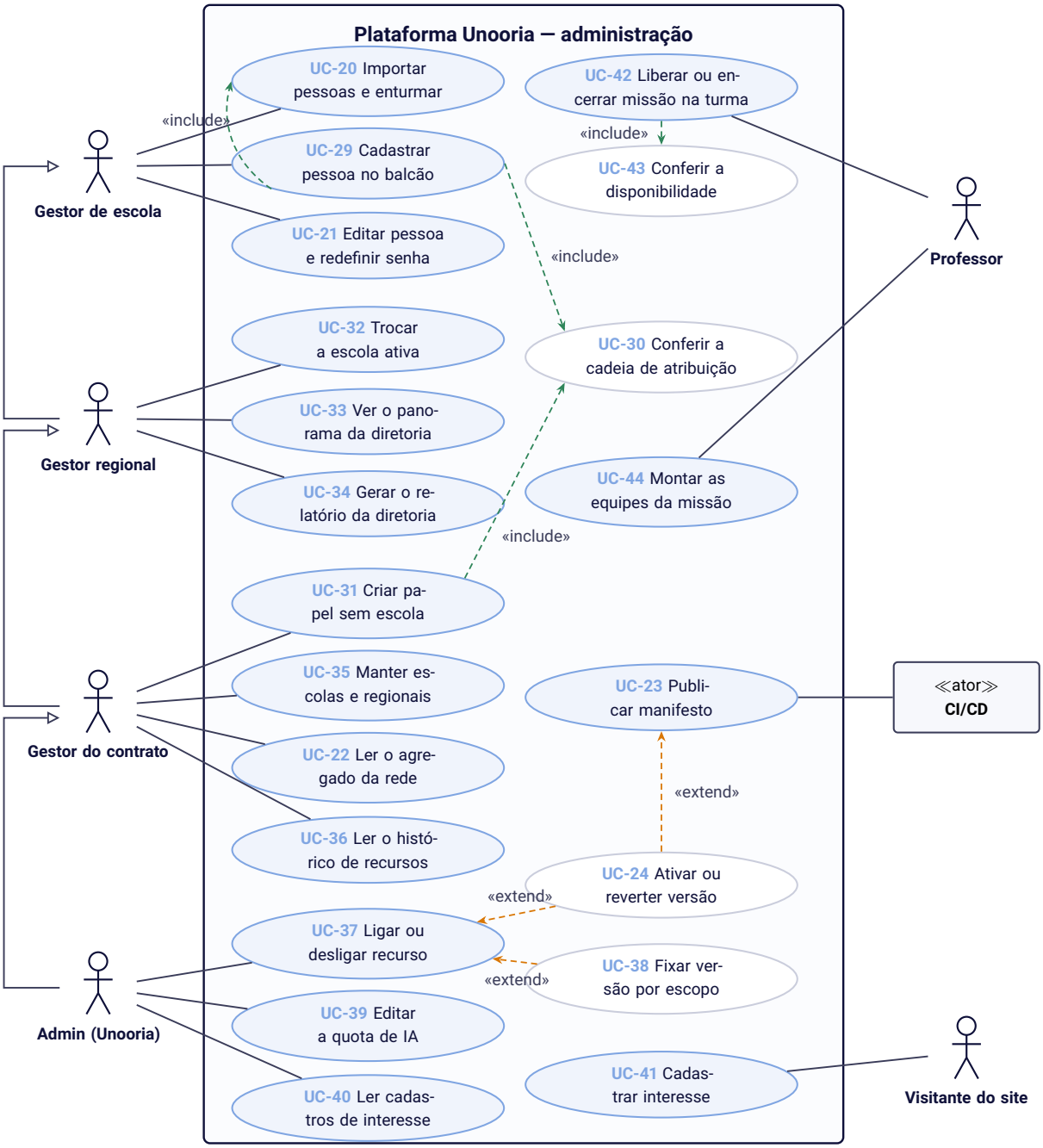


Figura 5.11 – Casos de uso da administração da rede, com os seis papéis da **ADR-80**. A seta de generalização entre atores é a regra: o *admin* faz tudo o que o gestor do contrato faz, e mais o painel da Unooria (**UC-37** a **UC-40**, **ADR-88**); o gestor do contrato faz tudo o que o regional faz, com a rede inteira como escopo. Fontes: `auth/claims.ts` (`GESTORES`, `GESTORES_DO_CONTRATO`), `usuarios/regras.ts` (`ATRIBUI_DIRETAMENTE`) e os `@Roles` de cada *controller*.

Legenda: ◯ caso de uso primário ◯ caso de uso de suporte (incluído ou extensão) seta de triângulo aberto entre atores: generalização

> **NOTA**
>
> **Três ausências deliberadas.** O professor não herda de gestor nenhum: a gestão dele é da turma em que leciona (**UC-42** exige matrícula de professor ativa, e o gestor só lê a sala).

> "Ver o consumo de IA" não existe como caso de uso: a tela de UC-39 mostra os tetos, e o contador `ia_uso_diario` não sai por rota nenhuma. E o UC-24 estende dois casos: pelo painel (UC-37, o caminho preferido) e pelo `publicar-manifesto.mjs --versao` (UC-23), que continua valendo para quem opera pelo terminal.

5.9.4 Visão lógica: classes e objetos

5.9.4.1 O modelo de domínio da plataforma

O modelo entidade-relacionamento do Capítulo 8 diz como as tabelas se ligam. O diagrama de classes da Figura 5.12 diz outra coisa: que *conceitos* a plataforma tem, e quais regras são restrição do modelo em vez de detalhe de consulta. Três notações carregam o que o ER não carrega. O **atributo derivado** (barra antes do nome) marca os dois *caches* de `users`: `/escola` é a lotação principal e `/turma` é a matrícula principal, e quem escreve os dois é `matricularEm`, nunca a tela. A **classe abstrata** `Recurso` é a da **ADR-87**: produto e experiência são recursos, e o *produto* é o teto das experiências que declara. E as **restrições entre chaves** dizem, no próprio elemento, o que é invariável. Duas delas são o contraste da regra 8 do `AGENTS.md`: ausência de linha é *bloqueada* em `Liberacao` e *ligada* em `Disponibilidade`.

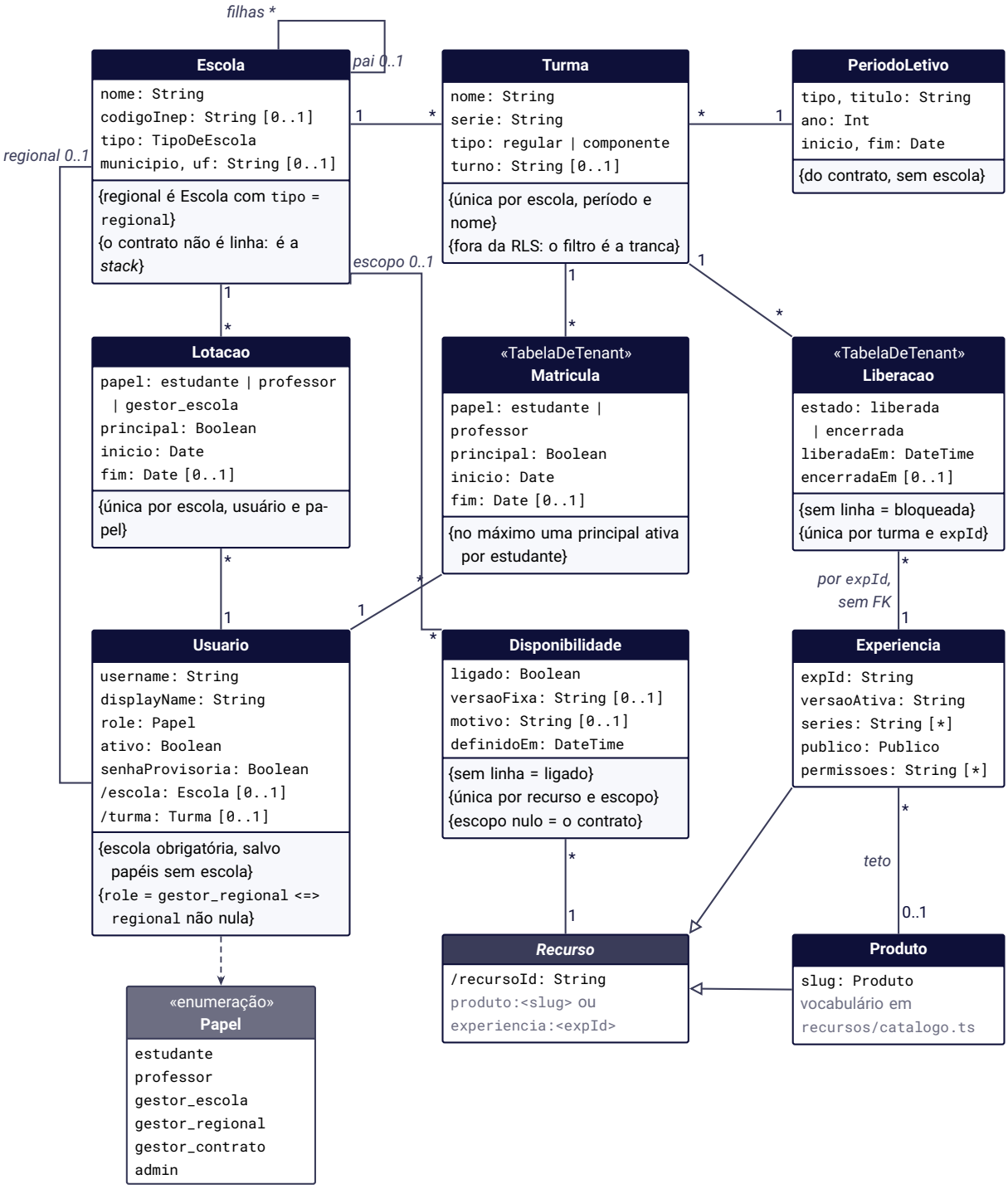


Figura 5.12 – O modelo de domínio da plataforma. «TabelaDeTenant» marca só as duas tabelas com RLS forçada; `users`, `turmas` e `lotacoes` têm `escola_id` mas ficam fora da RLS, porque o *login* as lê antes de saber a escola — nelas o filtro da aplicação é a única tranca (Capítulo 7). Fontes: `apps/api/prisma/schema.prisma`, os CHECKs das migrações `modelo_de_papeis`, `nivel_regional` e `liberacoes_por_turma`, `rostering/matricula.ts` e `recursos/regras.ts`.

5.9.4.2 A porta da IA: estratégia, decorador e quota

A regra 5 do `AGENTS.md` — IA só por `src/ai`, quota antes e depois — tem uma forma de projeto, e é ela que a Figura 5.13 mostra. São dois padrões conhecidos. A **estratégia** é a interface `AiProvider` com duas implementações trocáveis por *env* (`AI_PROVIDER`: `vertex` ou `mock`). O

decorador é o `MedidorDeProvider`, que implementa a mesma interface e embrulha a implementação real para medir latência, *tokens* e `finishReason` (Capítulo 17). Quem monta os dois é a fábrica do `AiModule`, e nenhum consumidor sabe qual veio: todos injetam o *token* de injeção `AI_PROVIDER`. Não existe `AiService` de fachada — a busca no código não o encontra, e o diagrama não o inventa. Cada consumidor chama o *provider* e o `QuotaService` por conta própria, e é por isso que a quota é regra de revisão e não de tipo: nada impede um *service* novo de injetar o *provider* e esquecer o `temSaldo`.

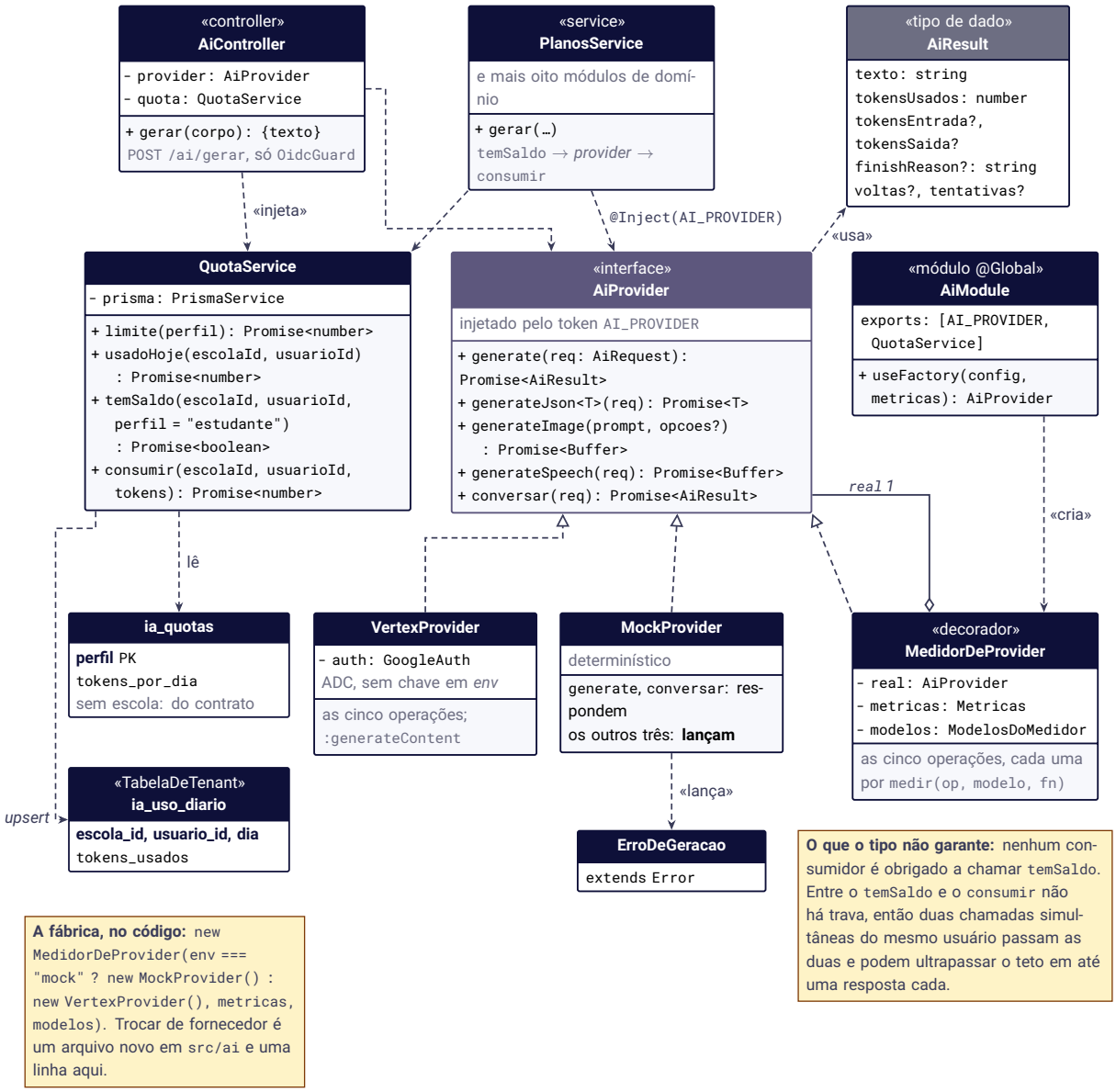


Figura 5.13 – A porta da IA como diagrama de classes de projeto: estratégia (`AiProvider` com duas implementações) e decorador (`MedidorDeProvider`), montados pela fábrica do `AiModule` e injetados pelo *token* `AI_PROVIDER`. A quota é um serviço à parte, que cada consumidor chama. Fontes: `ai/provider.ts`, `ai/ai.module.ts`, `ai/medidor.provider.ts`, `ai/quota.service.ts`, `ai/ai.controller.ts`.

5.9.4.3 Um instantâneo da disponibilidade

As três regras da disponibilidade — o escopo mais específico ganha, o produto é teto, ausência de linha é ligado — são fáceis de enunciar e fáceis de errar quando se combinam. O diagrama de objetos da Figura 5.14 congela uma rede com uma regional, duas escolas e quatro linhas em `disponibilidades`, e a Tabela 5.5 diz o que `exigirRecursoDisponivel` decide em cada combinação. Os valores são de exemplo; os campos, os códigos de experiência e os *slugs* de produto são os reais.

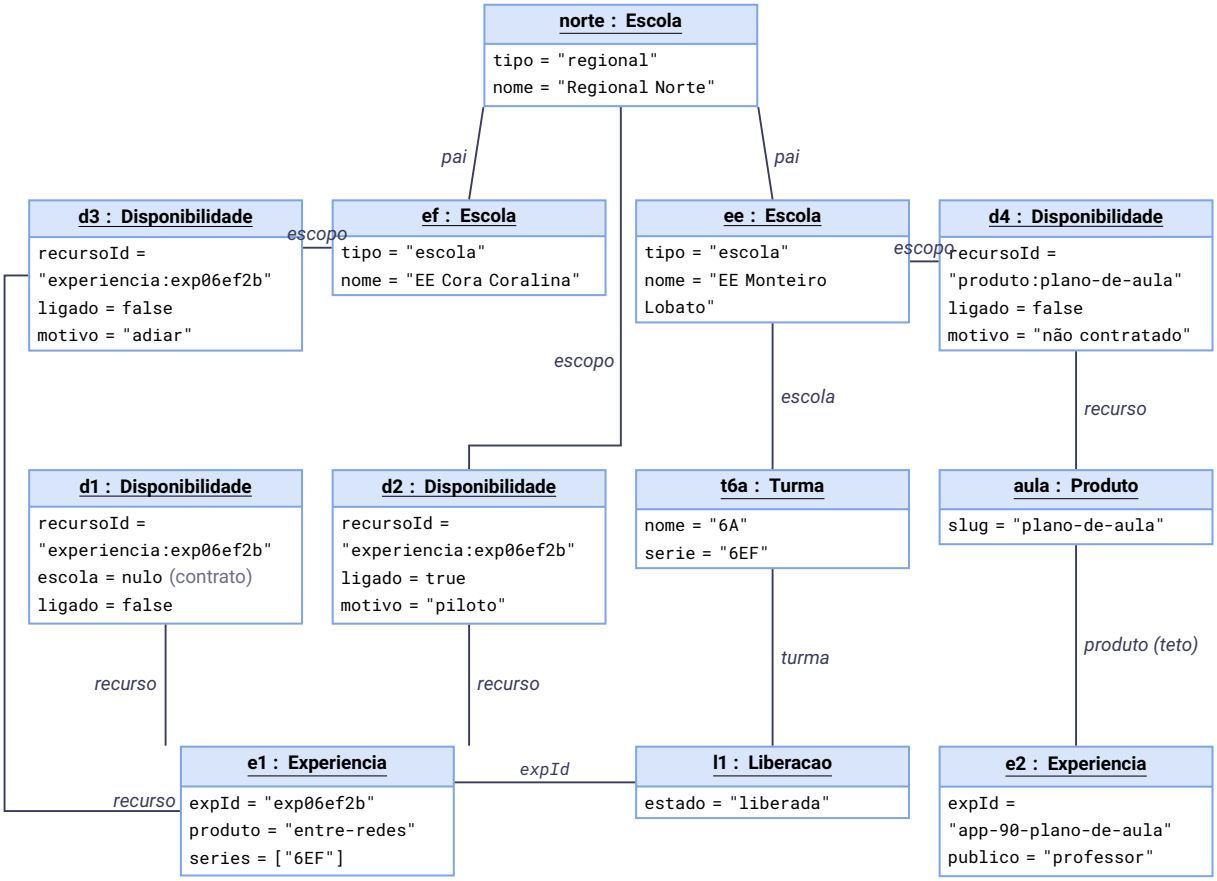


Figura 5.14 – Diagrama de objetos: uma rede com uma regional, duas escolas e quatro linhas de disponibilidade. Cada caixa é uma instância (nome sublinhado) e cada linha é um *link*, instância de uma associação da Figura 5.12. `d1` não tem *link* de escopo: escola nula é o contrato inteiro.

Tabela 5.5 – O que o resolvedor decide no instantâneo da Figura 5.14

Recurso, na escola	Decisão	Por quê (`recursos/regras.ts`)
`exp06ef2b` em `ee`	ligada	`ee` não tem linha; a regional tem `d2`, que vence a do contrato (`d1`).
`exp06ef2b` em `ef`	desligada	`d3` é da própria escola e vence a regional: o mais específico ganha, nos dois sentidos.

Recurso, na escola	Decisão	Por quê
`exp06ef2b` numa escola de outra regional	desligada	Nem escola nem regional têm linha; vale `d1`, do contrato. Um estudante com a missão liberada recebe 403 `recurso-indisponivel` — a liberação `l1` não muda nada, porque disponibilidade é pré-condição.
`app-90-plano-de-aula` em `ee`	desligada	`d4` desliga o *produto* na escola, e o produto é teto: nenhuma linha de experiência religaria. O professor lê "Este recurso não está disponível para a sua rede."
`app-90-plano-de-aula` em `ef`	ligada	Nenhuma linha em nenhum escopo, nem do produto nem da experiência.
`exp05ef1b` em qualquer escola	ligada	Ausência de linha é *ligado*: o interruptor é mecanismo de exceção.

5.9.5 Visão de desenvolvimento: pacotes, componentes e estrutura composta

5.9.5.1 Quem pode importar quem

O diagrama de pacotes da Figura 5.15 tem uma pergunta só: que dependência de código é permitida. As setas saem das declarações reais — os `package.json` do *workspace* e os `import` da API — e duas ausências valem tanto quanto as setas. **A API não importa pacote nenhum do *workspace***: ela não depende de `@edu/sdk`, e o que a SDK envia (o lote de telemetria, por exemplo) é validado de novo do lado da API, com o Zod dela. E **hub e API não se importam**: falam por HTTP, pelo BFF, e um tipo compartilhado entre os dois seria o primeiro passo para o hub assumir regra de domínio.

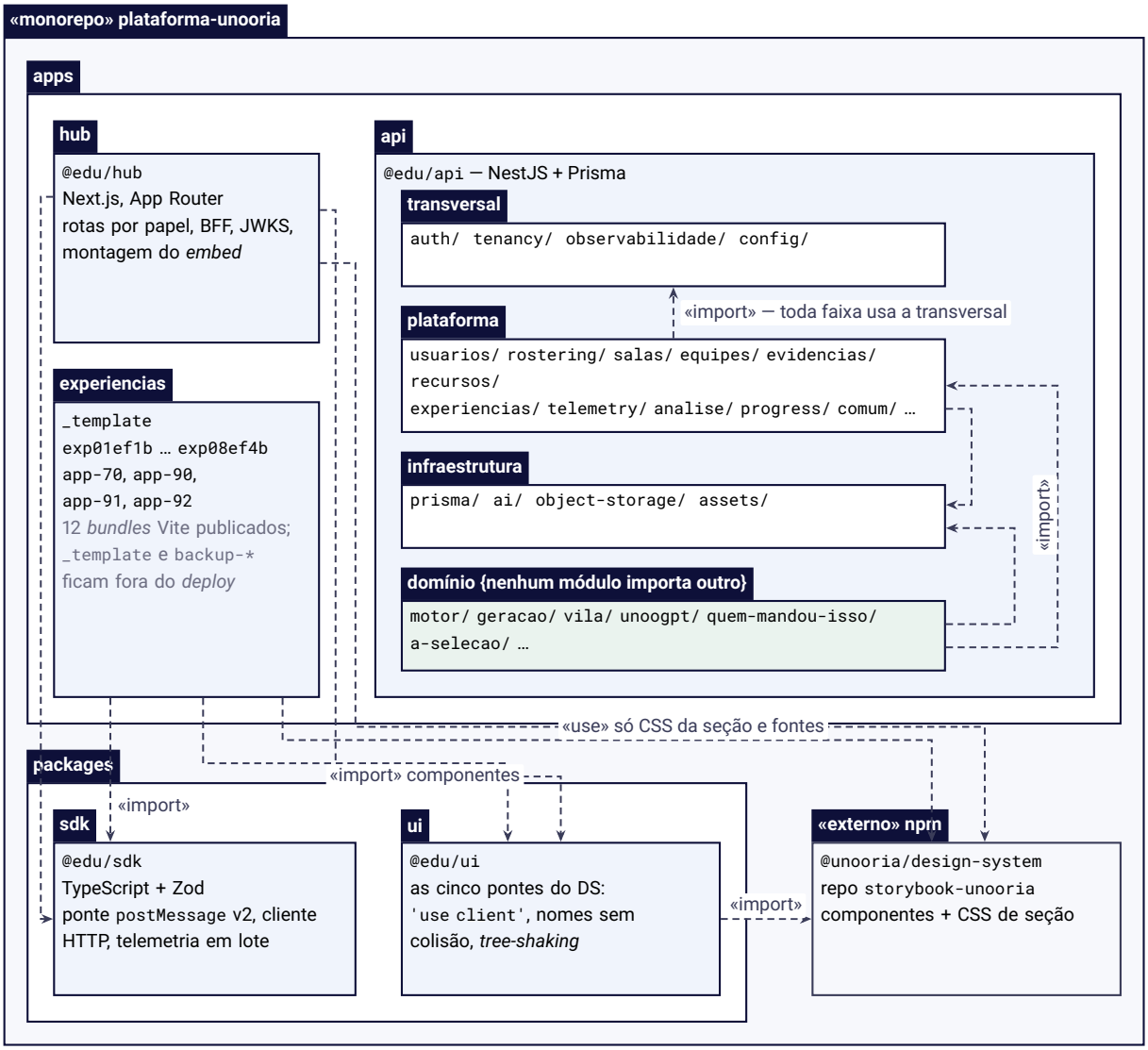


Figura 5.15 – Diagrama de pacotes do *monorepo*. As setas tracejadas são dependências de código declaradas: o hub e as experiências importam a SDK e os componentes pela ponte do DS, e o CSS da seção direto do pacote publicado; a API não importa nada do *workspace*. A `app-70` importa só a SDK: é jogo em *canvas*, sem DOM de interface. No domínio, duas experiências conversam por tabela de contrato ou por *endpoint* de fronteira, nunca por `import`. Fontes: os `package.json` do *workspace* e a faixa de cada módulo na Figura 5.3.

5.9.5.2 Quem fornece e quem requer

O C4 nível 2 (Figura 5.2) desenha as mesmas peças e diz quem chama quem. O diagrama de componentes da Figura 5.16 diz a *direção da dependência de interface*: o pirulito é a interface que o componente **fornece**, o soquete é a que ele **requer**, e o encaixe dos dois é um conector de montagem. Duas direções surpreendem quem conhece só o C4. **O JWKS é fornecido pelo hub e requerido pela API**: a API depende do hub para validar qualquer *token*, e é por isso que trocar para o WSO2 é trocar `OIDC_JWKS_URI` e `OIDC_ISSUER` sem tocar o *guard*. E **a ingestão da telemetria é fornecida pela API e requerida pelo Pub/Sub**: não há *worker*; é a assinatura *push* que chama `POST /telemetry/ingest`, autenticada por ID *token* do Google.

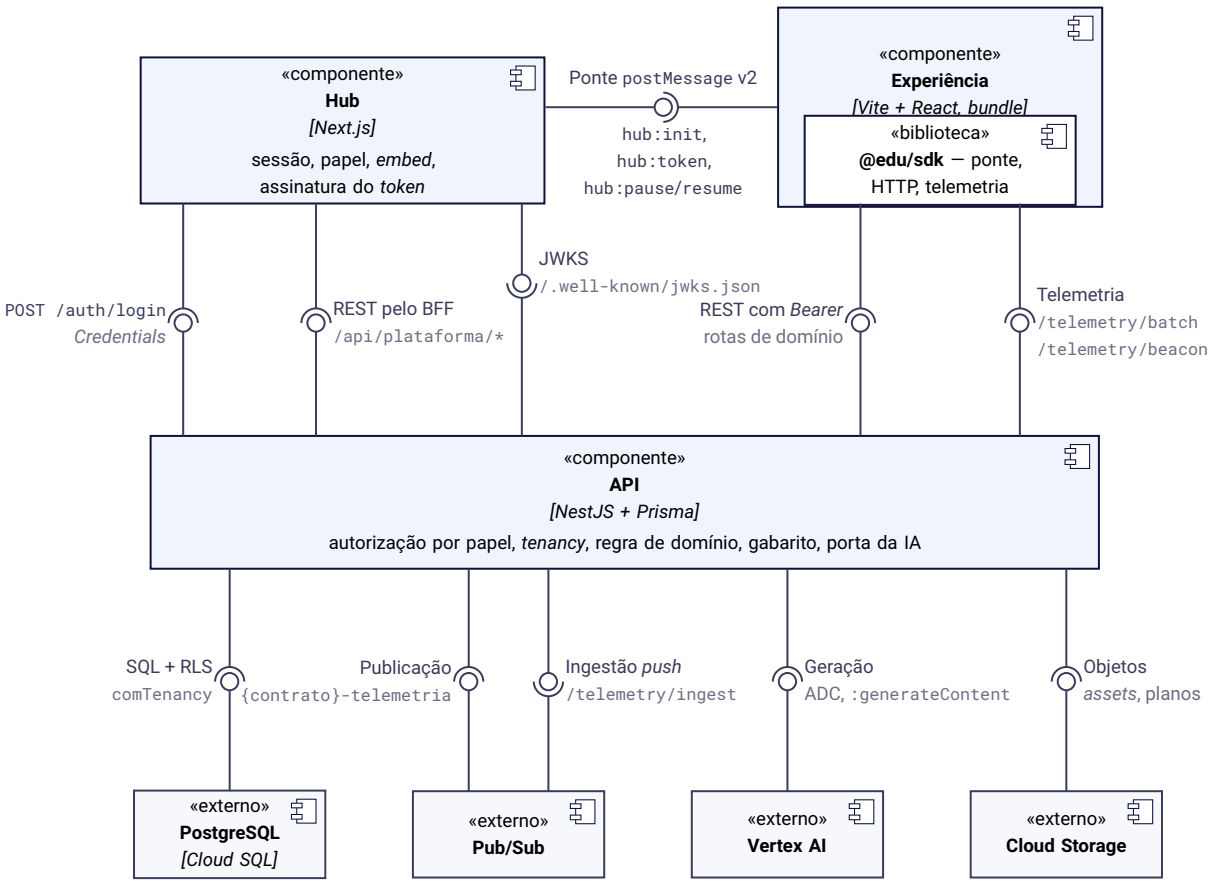


Figura 5.16 – Diagrama de componentes: bola é interface fornecida, soquete é interface requerida, e o encaixe é o conector de montagem. A experiência requer a ponte do hub e, pela SDK, o REST e a telemetria da API; a API requer o JWKS do hub. O Cloud Storage também serve o *bundle* ao navegador pela CDN, o que é artefato e não interface — está na Figura 5.26. Fontes: `hub/components/ExpFrame.tsx`, `hub/app/.well-known/jwks.json/route.ts`, `api/src/auth/token.service.ts`, `api/src/telemetry/telemetry.controller.ts` e `packages/sdk/src/types.ts`.

5.9.5.3 Por dentro da página da missão

O diagrama de estrutura composta abre um classificador e mostra as *partes* que ele contém, as *portas* por onde ele fala com o exterior e os *conectores* entre elas. A Figura 5.17 abre o `ExpHost`, o componente cliente que a página `/exp/[expId]` renderiza, e mostra por onde cada coisa atravessa a fronteira do *sandbox* (a linha laranja). O *token* entra uma vez, como propriedade vinda do servidor (`montarEmbed`), e é renovado pela porta `token` do `ExpFrame`, nunca pela experiência: ela só *pede*, pela ponte, e recebe `hub:token` de volta. O *bundle* entra pela porta `src`, direto da CDN. Entre o hub e a experiência, é só isso que atravessa. O que a experiência fala com a API não passa pelo hub: sai direto do `iframe`, com o *token* da memória (Figura 5.16).

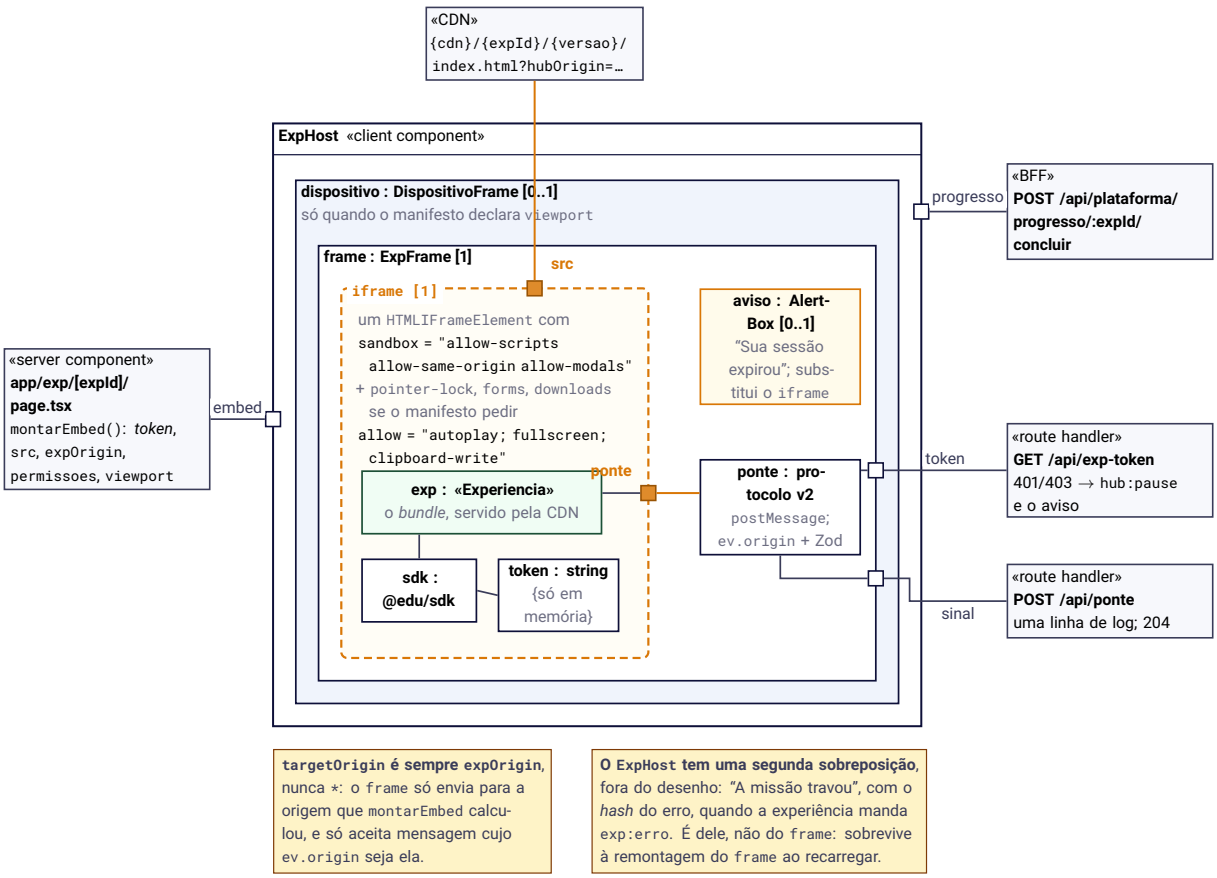


Figura 5.17 – Diagrama de estrutura composta do `ExpHost`. Partes são instâncias dentro do classificador (com multiplicidade); quadrados na borda são portas; a linha laranja tracejada é a fronteira do *sandbox*, e as portas laranja são as duas únicas ligações entre o hub e o que roda dentro dela; as chamadas da experiência à API saem direto do `iframe`. Fontes: `hub/components/ExpHost.tsx`, `hub/components/ExpFrame.tsx`, `hub/components/DispositivoFrame.tsx`, `hub/lib/embed-experiencia.ts` e `hub/app/api/exp-token/route.ts`.

5.9.6 Visão de processos: atividades, interações e tempo

5.9.6.1 O algoritmo da disponibilidade

O diagrama de atividades da Figura 5.18 é o `exigirRecursoDisponivel`, a primeira das duas travas de toda entrada em experiência. As duas *partições* são os dois arquivos, e é a divisão que importa: a da esquerda lê o banco dentro da transação de quem chama; a da direita é função pura, testada em milissegundos sem PostgreSQL. As regras da Tabela 5.5 moram inteiras na partição da direita.

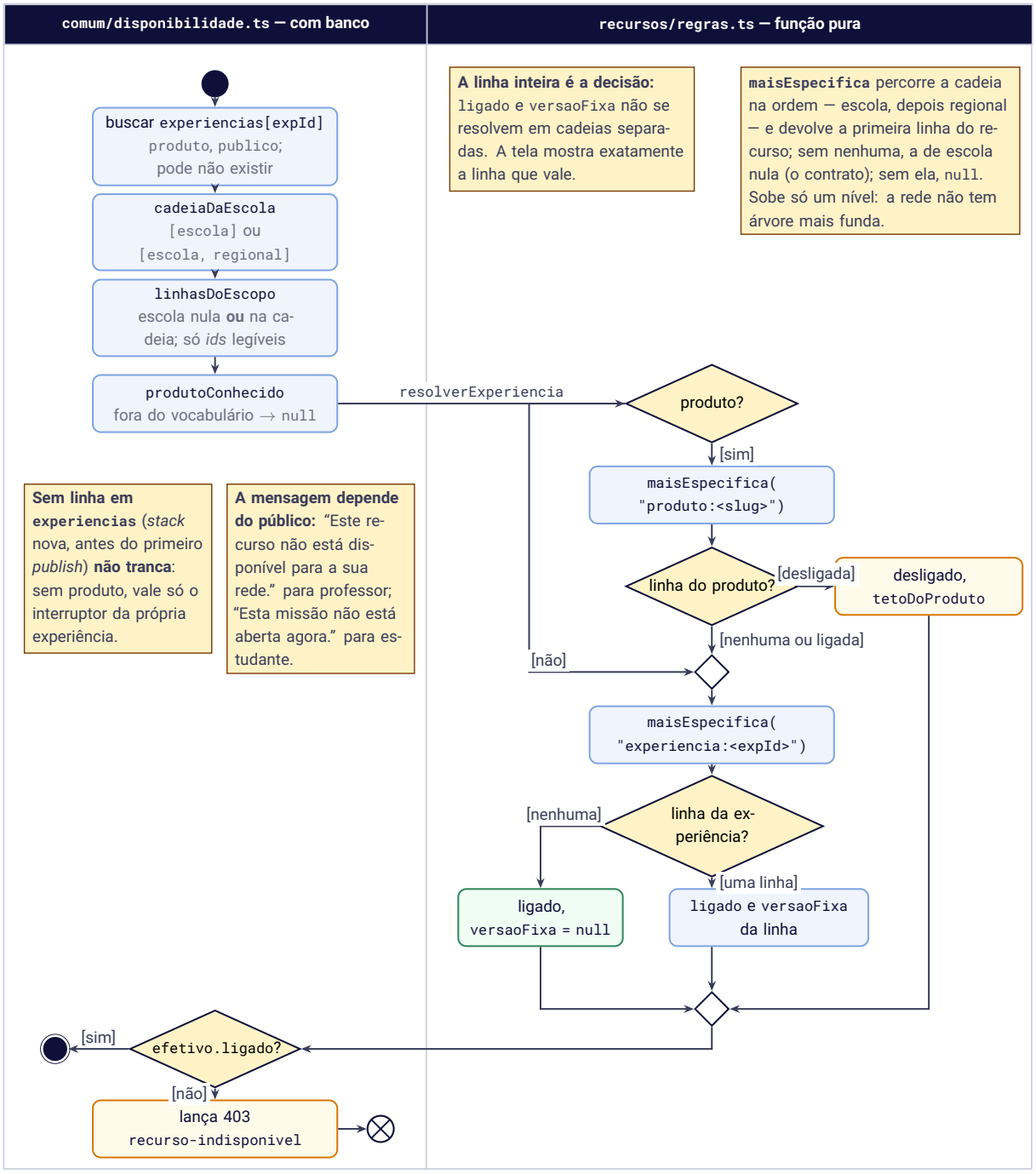


Figura 5.18 – Diagrama de atividades do `exigirRecursoDisponivel`, com uma partição por arquivo. A da direita é `resolverExperiencia`, pura; os losangos vazios são nós de junção. A terminação em *X* é o fim de fluxo pela exceção: a transação de quem chamou desfaz o que tiver feito. Fontes: `comum/disponibilidade.ts` e `recursos/regras.ts`.

> **NOTA**
>
> **O produto sem experiência entra por um atalho do mesmo algoritmo.** O agente de IA do professor não tem linha em `experiencias`, então a classe declara `@Produto("agente-professor")` em vez de `@Recurso(expId)`, e o `DisponibilidadeGuard` chama `exigirProdutoDisponivel`: só o ramo do TETO do diagrama acima (`disponibilidadeDoProduto` → `resolverProduto`, escola →

> regional → contrato → padrão, ausência de linha = ligado), sem a etapa da experiência. É o MESMO resolvedor de propósito: uma segunda leitura da tabela com regra própria seria o painel dizendo "desligado" e a porta abrindo. Declarar os dois decoradores na mesma classe lança, como declarar nenhum.

5.9.6.2 A entrada na missão: onde cada trava dispara

A regra 7 do `AGENTS.md` diz que filtrar o *card* no hub é UX e que a trava é da API. O diagrama de sequência da Figura 5.19 mostra por quê, na ordem em que as coisas acontecem. O hub consulta a grade e devolve 404 para missão que não está liberada — mas é a mesma grade que o estudante vê, e o *bundle* é público: quem digita `/exp/:expId` de uma missão fechada, ou chama a API direto, só encontra a trava de verdade na primeira rota de estudante, no funil `garantirSessao`/`estudanteDaEscola`. Ali as duas travas disparam em série, disponibilidade antes de liberação.

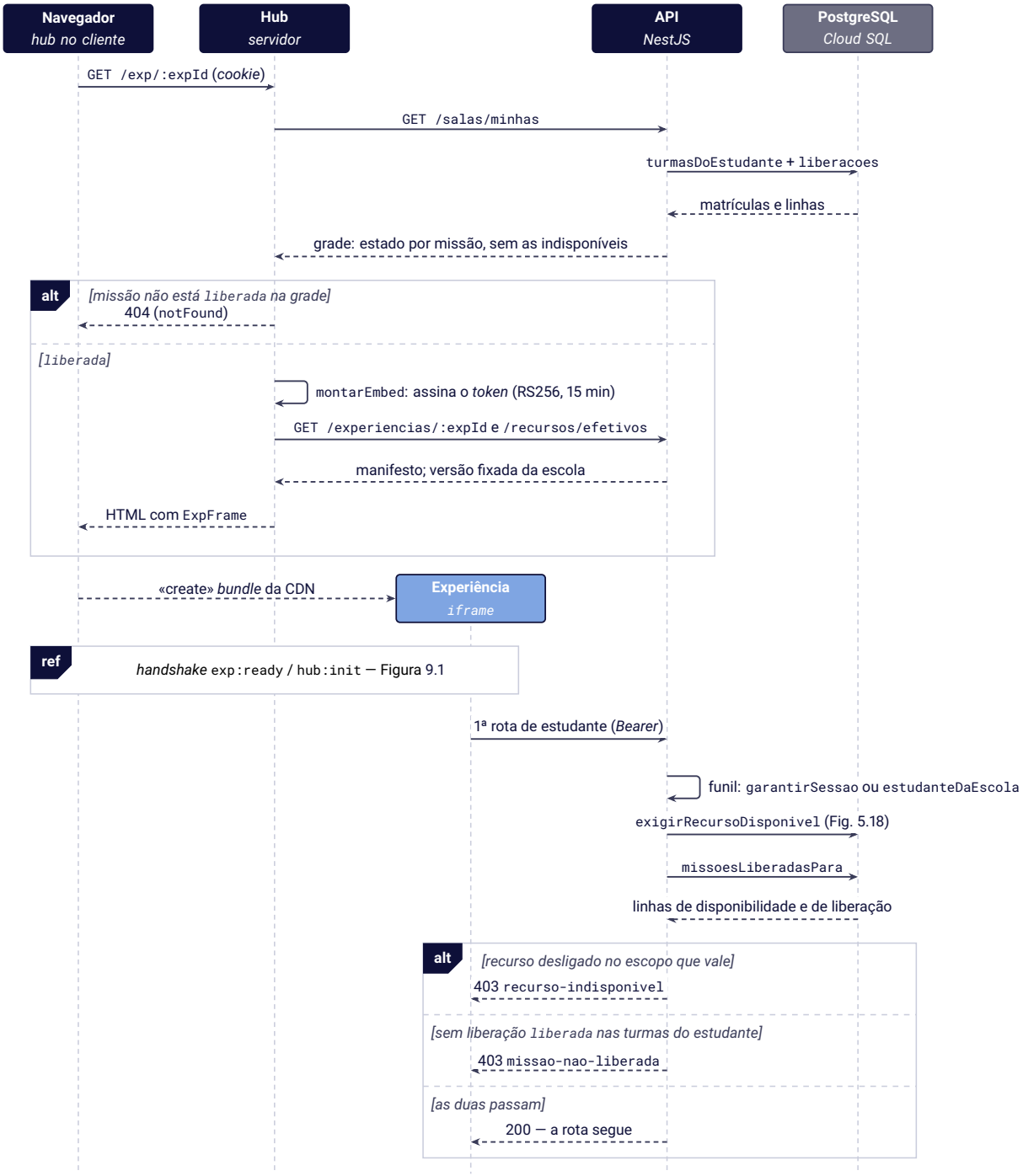


Figura 5.19 – Sequência da entrada do estudante na missão. O filtro do hub (o primeiro `alt`) é UX; as travas estão no segundo, na API, e disparam em série. A linha de vida da experiência começa na mensagem «create»: antes dela, o `iframe` não existe. Fontes: `hub/app/exp/[expId]/page.tsx`, `hub/lib/embed-experiencia.ts`, `api/src/comum/liberacao.ts`, `api/src/comum/disponibilidade.ts` e `api/src/salas/salas.service.ts`.

NOTA

Os dois 403 chegam iguais à experiência. A SDK não repassa o corpo do erro: `call` lança `Error("API GET … → 403")`, e cada experiência tira o *status* da mensagem. A exp05ef1b e a exp04ef1b mostram o aviso dentro do disfarce (“Este grupo ainda não abriu

> para a sua turma”); o hub não mostra nada, porque não existe mensagem de ponte para isso. Se um dia a diferença entre “não liberada” e “indisponível na rede” precisar chegar ao estudante, o caminho é a SDK expor o `codigo`, não a experiência adivinhar.

5.9.6.3 Uma chamada de IA: o que roda antes e depois do *provider*

A Figura 5.20 é a única porta de IA aberta ao cliente, `POST /ai/gerar`, na ordem do código. Os dois fragmentos `break` são as saídas antecipadas: quando a guarda do `break` vale, o operando roda e o resto da interação *não* acontece. O que o diagrama deixa ver e o texto esconde: a leitura do teto (`ia_quotas`) roda *fora* do `comTenancy`, porque a linha é do contrato e não tem escola; a do consumo (`ia_uso_diario`) roda dentro, porque tem. E o prompt de sistema é escolhido e montado dentro do *controller*, com a série resolvida da turma do *token* — nunca vinda do corpo.

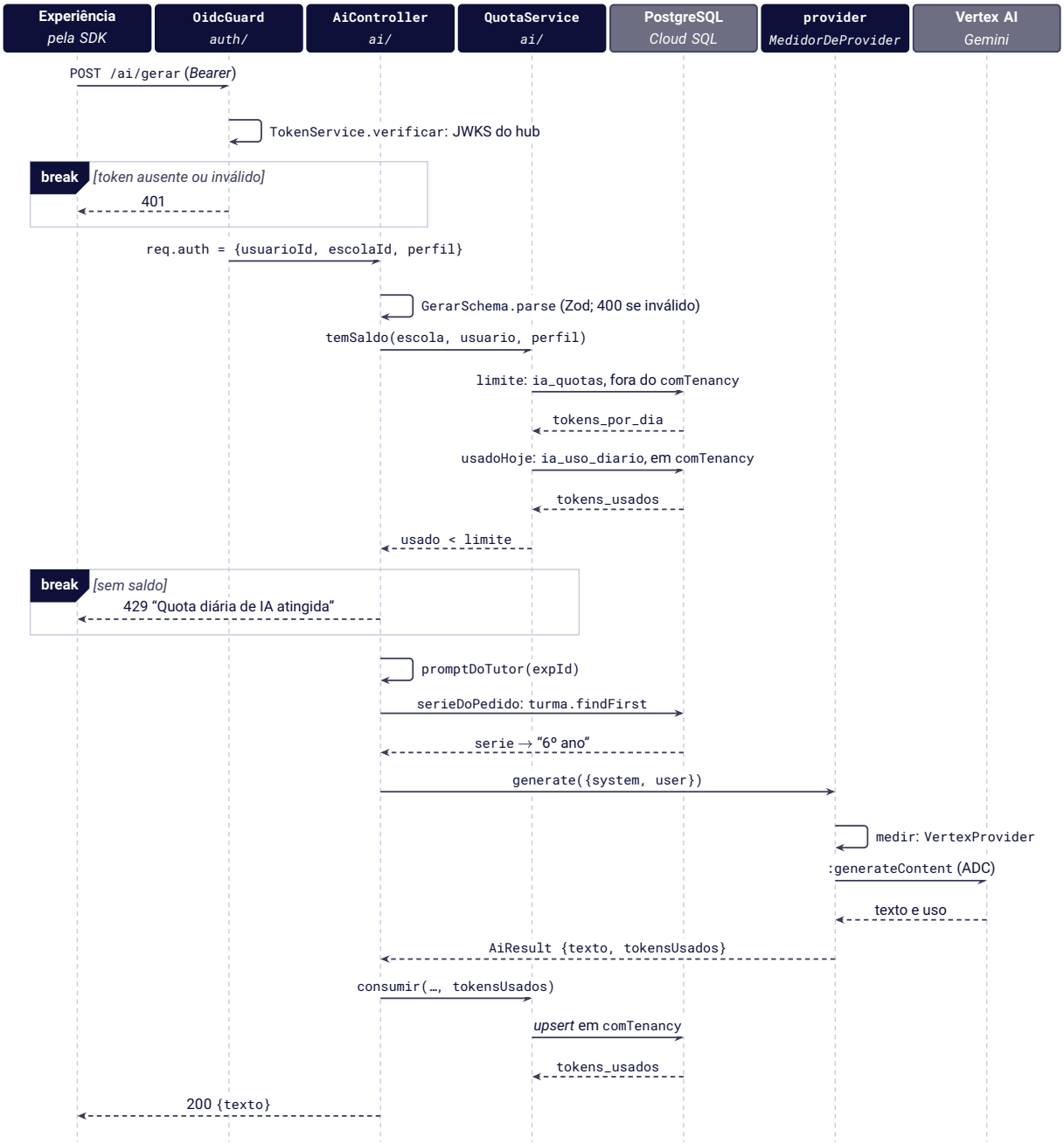


Figura 5.20 – Sequência de `POST /ai/gerar`. `break` é a saída antecipada: 401 sem *token* válido, 429 sem saldo. O consumo é gravado *depois* da resposta do modelo, com os *tokens* que ele declarou; entre o `temSaldo` e o `consumir` não há trava. Fontes: `ai/ai.controller.ts`, `ai/prompts.ts`, `ai/quota.service.ts`, `ai/medidor.provider.ts`, `auth/oidc.guard.ts`.

LIMITAÇÃO CONHECIDA

`/ai/gerar` não tem *rate limit*. A quota limita o custo do dia; a rajada dentro do dia só é contida nas rotas que chamam `rateLimit` — `planos`, `bemestar`, `manifesto-ia`, `vila` e as reações do `motor`. O `unoogpt` chama `rateLimit` e ignora o retorno, o que faz o limite de 12 por minuto que o contrato da rota promete não valer. E o `Map` do `rateLimit` é por instância: com `min-instances` acima de um, o limite efetivo é multiplicado. Os três

> pontos estão fora desta frente; ficam registrados aqui para quem desenhar a próxima rota de IA.

5.9.6.4 Um turno do agente de IA do professor: a quota antes e depois do *stream*

A Figura 5.21 é `POST /agente-professor/conversas/:id/mensagens`, a única rota da plataforma que responde `text/event-stream` (**ADR-128**), na ordem do código. Ela é a Figura 5.20 com três diferenças que só o diagrama deixa ver juntas. **Toda recusa sai antes do *stream***, em JSON: depois do 200 só existe o evento `erro`. **O consumo da quota mora num `finally`**, e soma os *tokens* dos eventos NÃO parciais — o professor pode apertar "Parar" ou fechar a aba no meio, e o que o modelo gerou foi pago. E **a posse é conferida no Postgres**, porque o Agent Engine não confere o usuário numa sessão pedida por nome (**ADR-126**). O laço de ferramentas dentro do agente aparece com uma ferramenta só; num turno real o modelo pode chamar `buscar_habilidades` e `buscar_acervo` mais de uma vez.

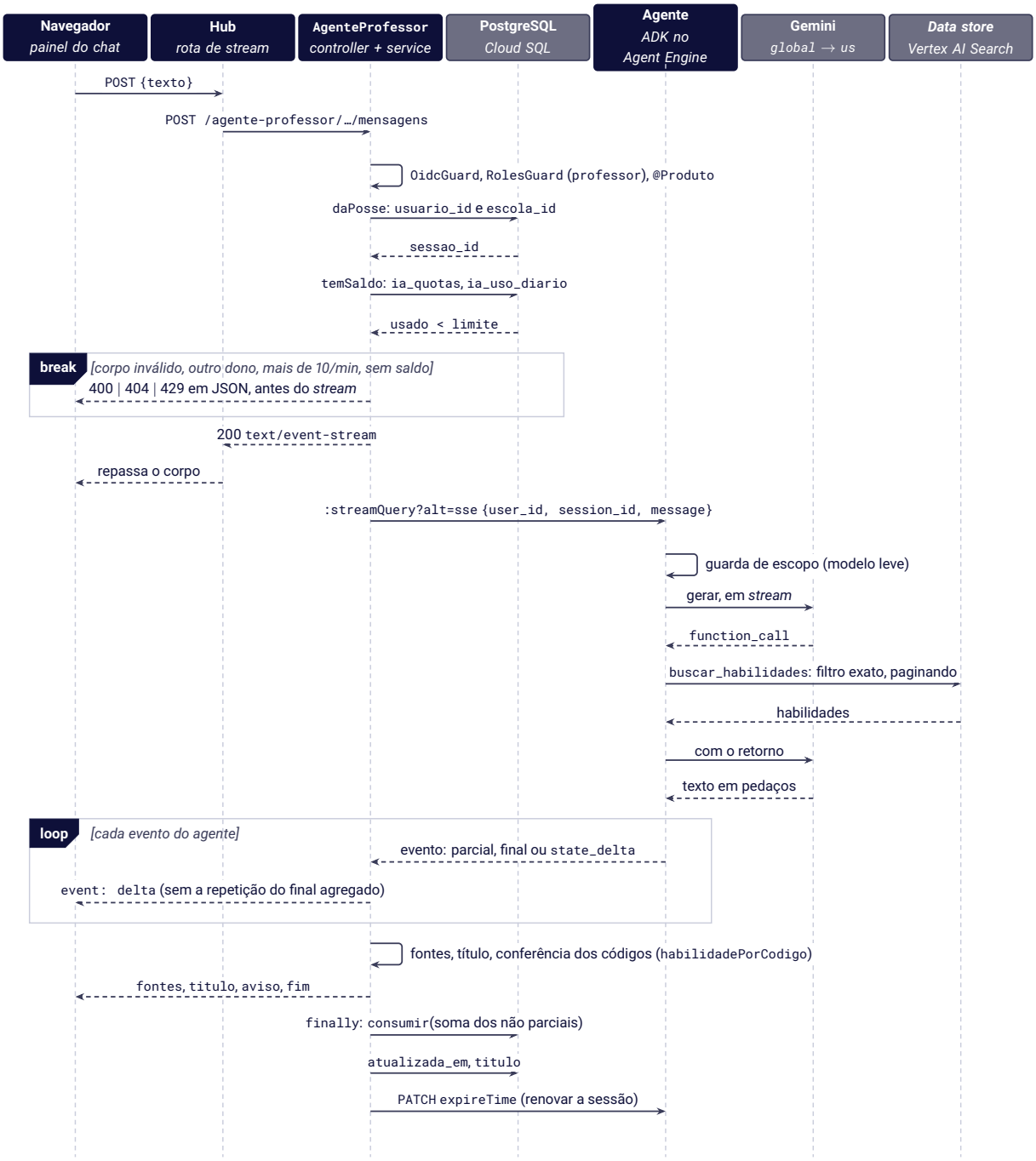


Figura 5.21 – Sequência de `POST /agente-professor/conversas/:id/mensagens`. `break` é a saída antecipada, sempre em JSON e antes do *stream*. Durante o `loop` a api escreve `: ping` a cada 15 s; "Parar" (ou fechar a aba) aborta cada salto até o Vertex — fechar o *painel* não aborta, o turno segue e acende o selo do botão —, e o teto de 240 s vira `event: erro` `TEMPO_ESGOTADO`. O consumo roda no `finally` também no abandono e no erro; a renovação só depois de `fim`, e a falha dela é só log. Nenhuma transação `comTenancy` fica aberta durante o *stream*. Fontes: `agente-professor/agente-professor.controller.ts`, `agente-professor/agente-professor.service.ts`, `agente-professor/regras.ts`, `ai/agent-engine.client.ts`, `apps/agente-professor/agente_professor/plugins.py`.

5.9.6.5 O caminho de um lote de telemetria

O diagrama de comunicação mostra os mesmos fatos de um diagrama de sequência, arrumados de outro jeito: os objetos ficam onde a topologia os põe, as linhas são os *links* que existem entre eles, e a ordem vai no número de cada mensagem (`1.1.2` acontece dentro de `1.1`, que acontece dentro de `1`). Para a telemetria, é a forma certa: o interessante não é o tempo, é *quem fala com quem*, e três gatilhos independentes (o relógio da SDK, o `pagehide` e a entrega do Pub/Sub) atravessam os mesmos objetos. A Figura 5.22 corrige uma leitura comum da tabela de arquitetura: **não existe *worker***. Quem consome o tópico é a própria API, pela assinatura *push*, e o destino é o PostgreSQL, numa tabela particionada por mês.

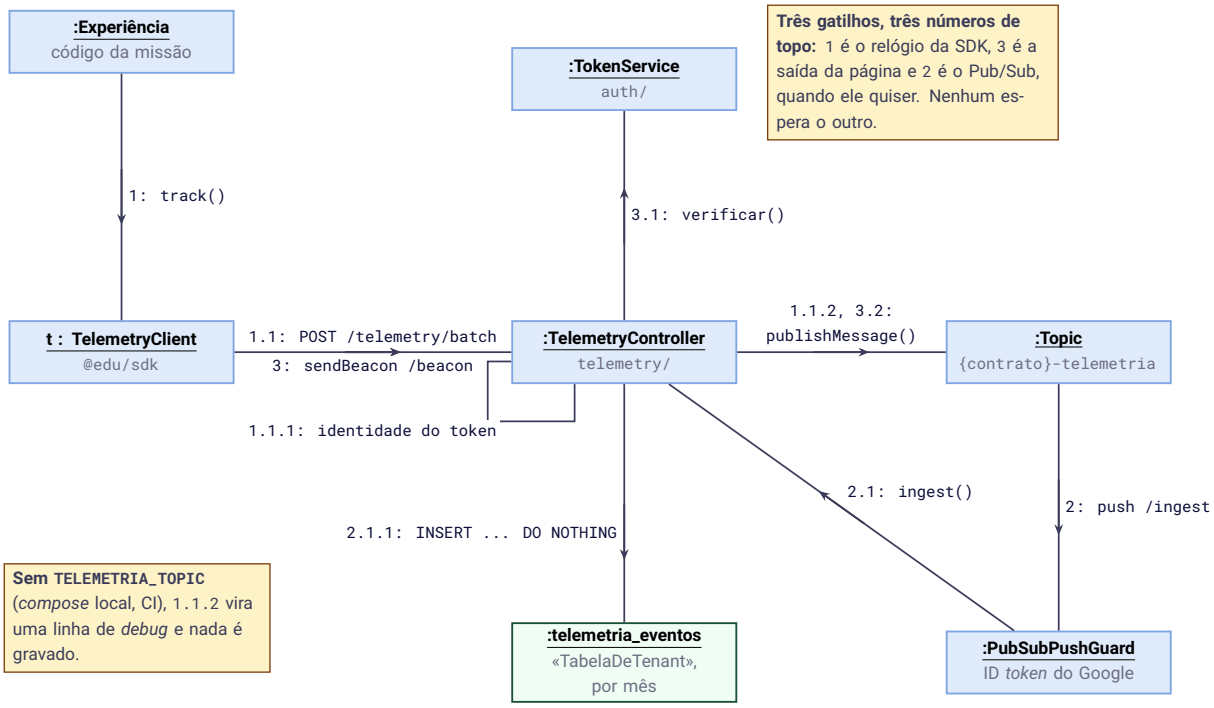


Figura 5.22 – Diagrama de comunicação da telemetria. Objetos com nome sublinhado; *links* como linhas; mensagens numeradas pela aninhação. O `ON CONFLICT (event_id, ts) DO NOTHING` torna a reentrega do Pub/Sub inofensiva, e a gravação é agrupada por escola, uma transação `comTenancy` por grupo. Fontes: `packages/sdk/src/telemetry.ts`, `api/src/telemetry/telemetry.controller.ts`, `api/src/telemetry/pubsub-push.guard.ts` e `docs/INFRA.md`.

Tabela 5.6 – As mensagens da Figura 5.22

Nº	Gatilho ou guarda	O que acontece
1	a missão registra um evento	Entra no *buffer*; acima de 500 eventos, sai o mais antigo.
1.1	a cada 10 s, ou ao chegar a 50	`POST /telemetry/batch` com *Bearer* e `keepalive`; um lote em voo por vez, e a falha devolve o lote ao *buffer* com *backoff* de 1 a 60 s.
1.1.1	—	O Zod descarta `estudanteId` e `escolaId` do corpo; o *controller* põe os do *token*. O `estudante_id` é o `sub`: id interno, não nome nem e-mail — e também não é *hash*.

Nº	Gatilho ou guarda	O que acontece
1.1.2	TELEMETRIA_TOPIC definida	publishMessage({json: {eventos}}). A resposta {ok, recebidos} conta o que foi *publicado*, não o que foi gravado.
2	entrega do Pub/Sub, *at least once*	*Push* para POST /telemetry/ingest; o *guard* confere emissor, audiência e o e-mail da conta de serviço do *push*. Sem as *envs*, 401.
2.1	o *guard* passou	Decodifica message.data (base64) e valida com Zod.
2.1.1	—	Agrupa por escola e grava um INSERT de várias linhas por grupo, dentro de comTenancy, com ON CONFLICT (event_id, ts) DO NOTHING: reentrega é rotina.
3	pagehide ou aba oculta	sendBeacon para /telemetry/beacon, com o *token* no corpo — o *beacon* não leva cabeçalho. Retorno false mantém o lote no *buffer*.
3.1	—	TokenService.verificar no *token* do corpo; se falha, descarta com métrica.
3.2	—	O mesmo publishMessage de 1.1.2. A rota responde 204 sempre.

5.9.6.6 Três relógios: *token*, renovação e sessão

O diagrama de tempo põe o tempo no eixo horizontal e o estado de cada participante numa faixa. É o único tipo da UML em que *prazo* é notação de primeira classe (as restrições entre chaves medem distância no eixo), e é por isso que ele mostra o que as outras figuras escondem. São três relógios independentes: o *token* de experiência vive 15 minutos (hub/lib/chave-token.ts); a SDK pede um novo a cada 10, por relógio fixo, sem ler o exp do JWT e sem reagir a 401 (packages/sdk/src/index.ts); e a sessão do hub morre 2 horas depois da última renovação (**ADR-95**), que o próprio pedido de *token* provoca (**ADR-96**). A Figura 5.23 desenha o caso em que o pedido dos 20 minutos falha por rede.

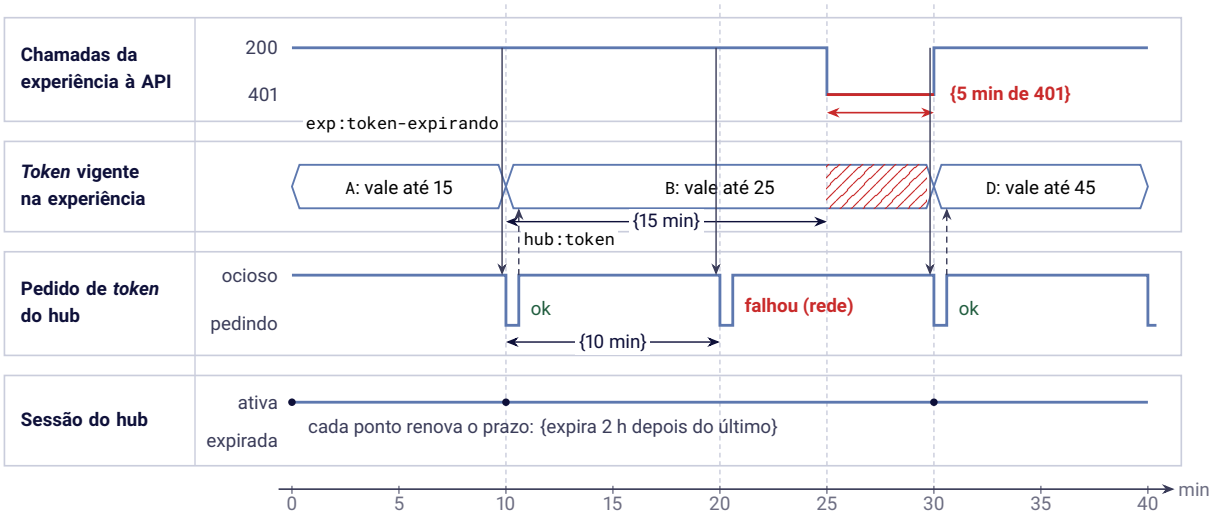


Figura 5.23 – Diagrama de tempo de uma missão aberta, no cenário em que o pedido de *token* dos 20 minutos falha por rede. O *token* `B` (emitido aos 10) vence aos 25; o próximo pedido é aos 30; entre os dois, toda chamada da experiência à API recebe 401. As restrições entre chaves são de duração. Fontes: `packages/sdk/src/index.ts`, `hub/components/ExpFrame.tsx`, `hub/lib/chave-token.ts`, `hub/app/api/exp-token/route.ts` e `hub/lib/sessao.ts`.

LIMITAÇÃO CONHECIDA

A falha transitória da renovação custa 5 minutos de 401, não zero. O comentário do `ExpFrame` diz que "o *token* atual ainda vale alguns minutos e a SDK pede de novo em 10" — as duas metades são verdade, mas os números não fecham: sobram 5 minutos de validade para 10 de espera. O efeito na sala é uma missão que falha por 5 minutos e volta sozinha, sem aviso, e a experiência vê só "API … → 401". Consertos possíveis, todos fora desta frente: a SDK reagir a 401 pedindo *token* na hora, o `ExpFrame` tentar de novo em segundos em vez de esperar o próximo tique, ou o relógio da SDK ler o `exp` do JWT. O inverso também vale e é a consequência aceita na **ADR-96**: enquanto a aba estiver aberta e a rede responder, a sessão nunca expira, porque cada tique a renova.

5.9.6.7 A ponte como máquina de estados de protocolo

A sequência do *handshake* (Figura 9.1) mostra *uma* conversa que dá certo. A máquina de estados de protocolo da Figura 5.24 mostra *todas* as conversas permitidas, do lado da experiência: em que estado a SDK pode estar, que mensagem é aceita em cada um e o que acontece com o resto. Não há `enum` de estado no código — `packages/sdk/src/bridge.ts` guarda a máquina em uma *flag* (`conectado`) e dois temporizadores —, e é por isso que desenhá-la vale: é o único lugar em que os quatro estados aparecem juntos.

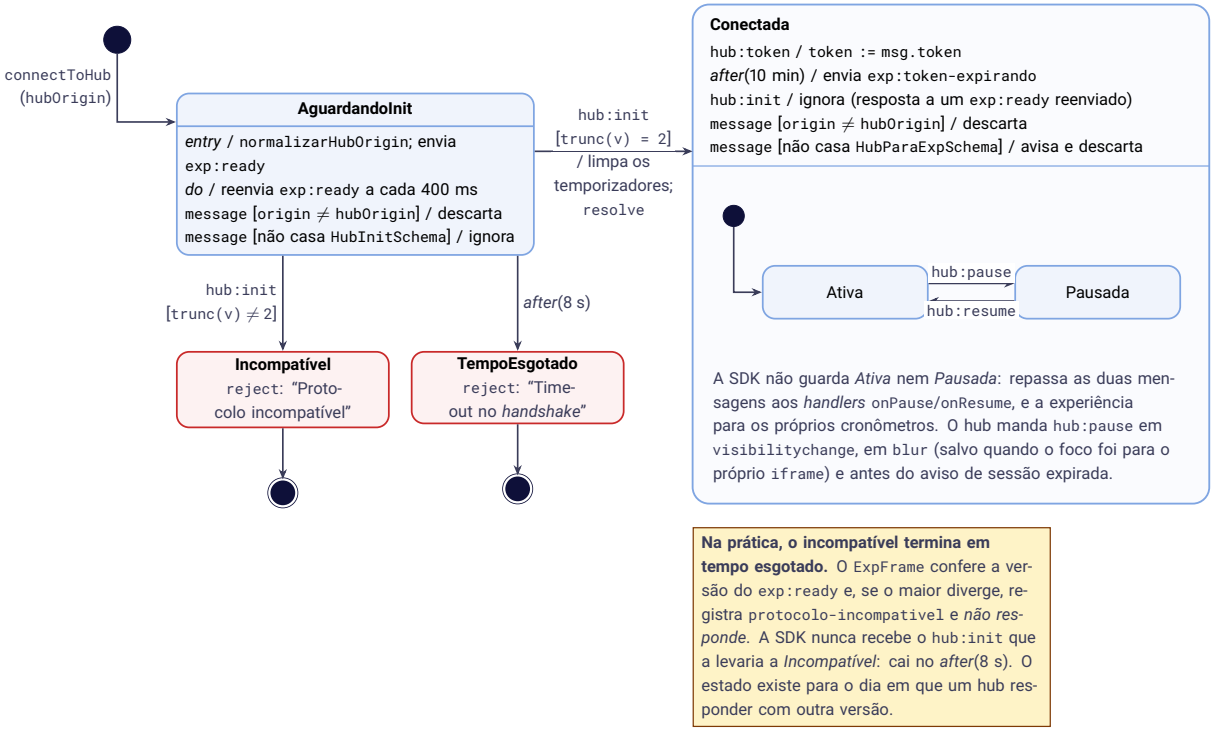


Figura 5.24 – Máquina de estados de protocolo da ponte, do lado da experiência (`connectToHub` e `createExperiencia`). Os compartimentos listam as transições internas — as que não mudam de estado — no formato *evento* [*guarda*] / *efeito*. Fontes: `packages/sdk/src/bridge.ts`, `packages/sdk/src/index.ts`, `packages/sdk/src/types.ts` (`PROTOCOL_VERSION = 2`) e `hub/components/ExpFrame.tsx`.

5.9.6.8 A jornada do estudante, do *login* à missão

O diagrama de visão geral de interação é um diagrama de atividades em que cada nó é uma *interação*: um quadro `ref` aponta para uma sequência desenhada em outro lugar, e o fluxo entre eles tem as decisões, as bifurcações e os fins de um diagrama de atividades. É o índice das interações das seções anteriores. A Figura 5.25 costura a jornada de um estudante numa manhã de aula: cada quadro diz em que figura a conversa está desenhada por inteiro.

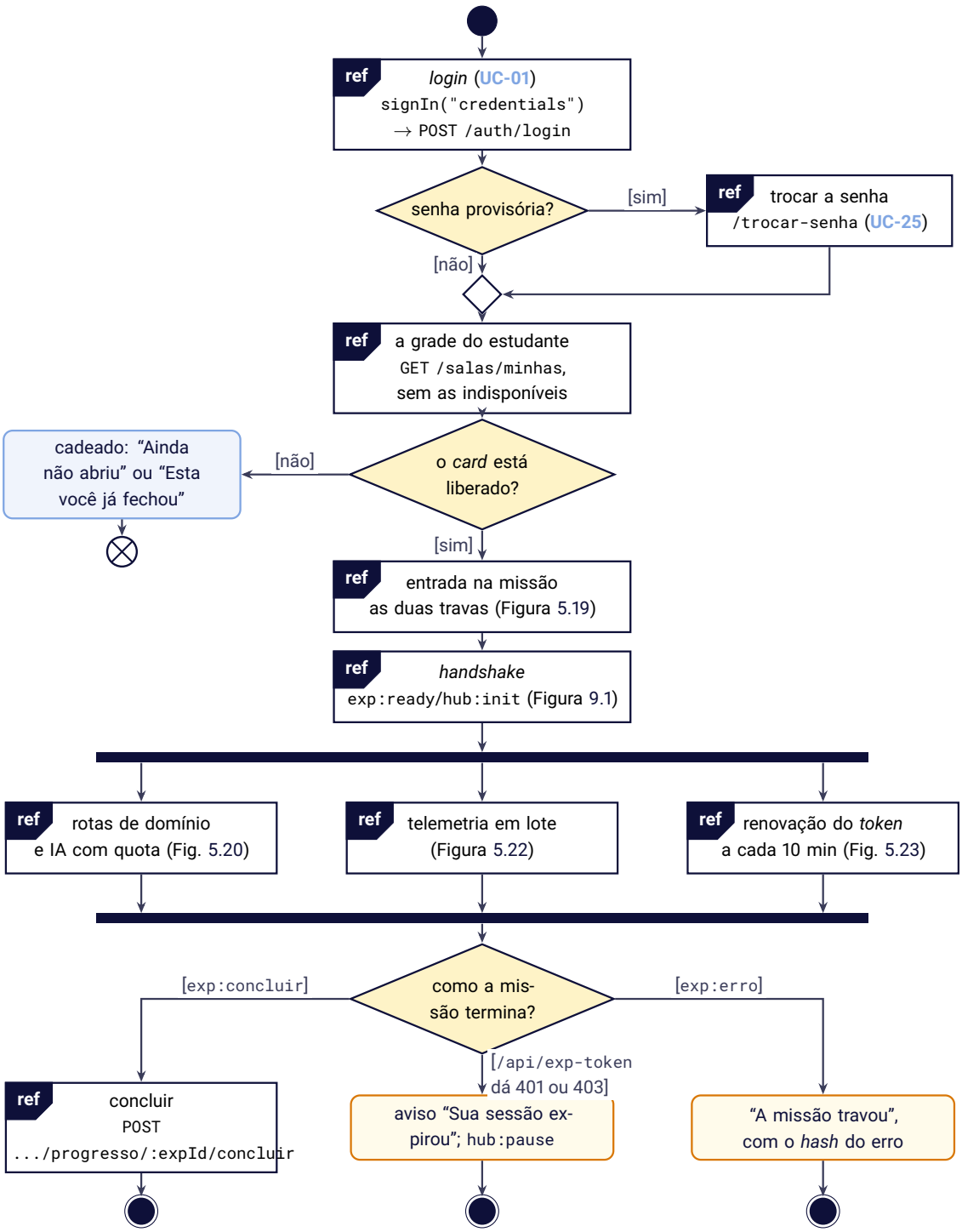


Figura 5.25 – Visão geral de interação da jornada do estudante. Cada quadro `ref` é um uso de interação desenhada por inteiro na figura indicada; as barras são bifurcação e junção — as três interações do meio correm juntas enquanto a missão está aberta. A saída pelo cadeado termina em *X* (fim de fluxo): o estudante continua na *home*. Fontes: `hub/app/(plataforma)/login/login-form.tsx`, `hub/middleware.ts`, `hub/components/cartao-missao-estudante.tsx`, `hub/components/ExpHost.tsx` e `hub/components/ExpFrame.tsx`.

5.9.7 Visão física: a implantação de um contrato

O Capítulo 6 desenha a implantação com a notação de infraestrutura: rede, sub-rede, serviço gerenciado, a seta pública e a privada. O diagrama de implantação da UML (Figura 5.26) responde outra pergunta: *que artefato roda em que nó*. Um nó é um recurso computacional (caixa em perspectiva); um ambiente de execução é um nó que hospeda software; um **artefato** é o arquivo que o CI produz e alguém executa, e é ele que liga o diagrama ao *pipeline* do Capítulo 15. Três leituras saem daqui. A imagem da API é implantada *duas* vezes: no serviço e no *job* de migração, que roda `prisma migrate deploy` na VPC antes de todo *deploy* da API. O *bundle* é armazenado num nó e executado em outro: vive no *bucket*, roda no navegador, e a versão que vale não está no *bucket*, está na linha de `experiencias` na API. E nenhum artefato é implantado à mão: cada seta «implanta» é um passo do `deploy.yml` — inclusive a do agente de IA do professor, cuja imagem não roda num Cloud Run, e sim no Vertex AI Agent Engine, chamado só pela API.

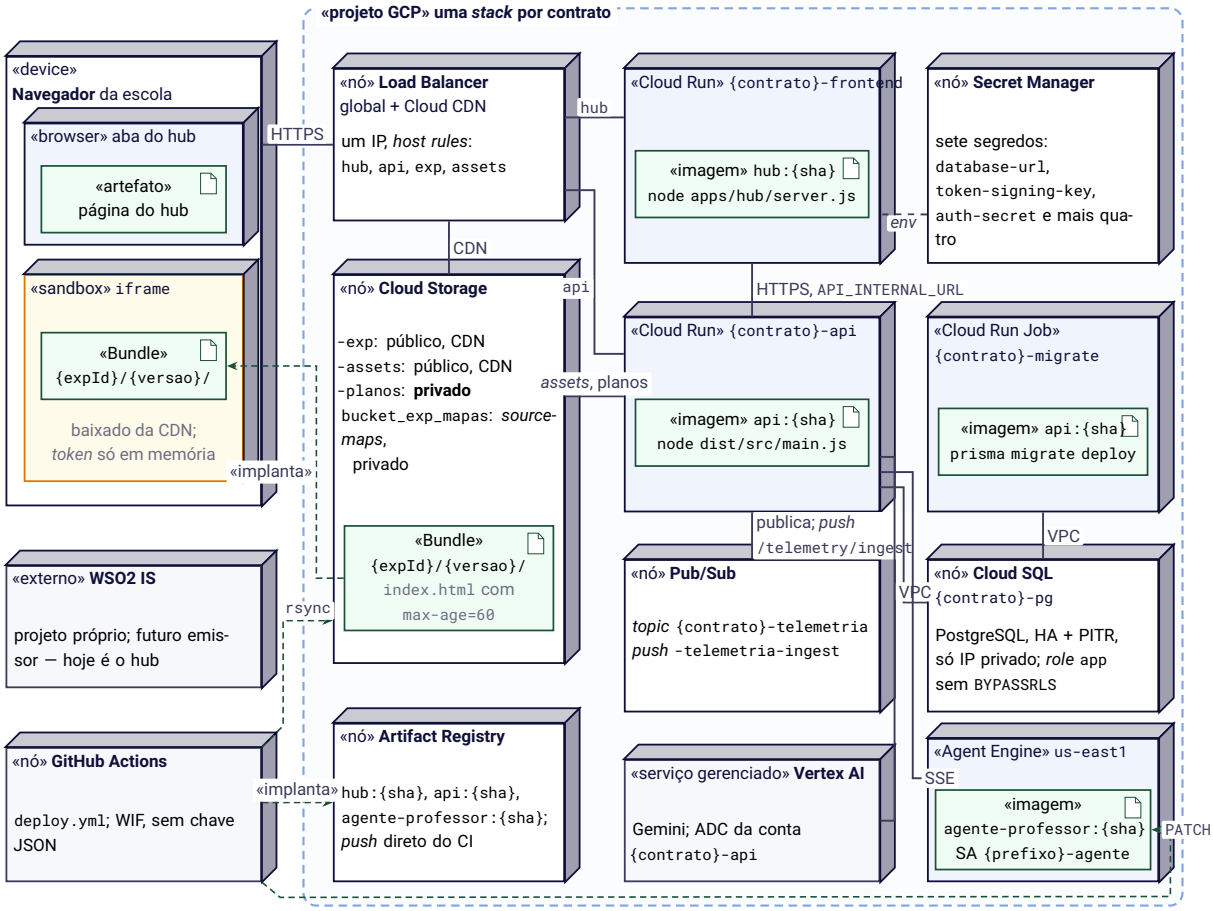


Figura 5.26 – Diagrama de implantação de um contrato. Caixas em perspectiva são nós; as azuis são ambientes de execução; as verdes com a folha dobrada são artefatos. A mesma imagem `api:{sha}` está implantada em dois nós, e o *bundle* está armazenado num e executado em outro. O CI ainda executa o *job* de migração e publica o manifesto (`PUT /experiencias/:expId`), o que o diagrama deixa no texto para não cruzar a figura. O nó do Agent Engine só existe depois do segundo *apply* do Terraform; daí em diante o CI troca a imagem dele por `PATCH` da `imageUri` (**ADR-126**), e os dois *data stores* e o Gemini que ele chama ficam em `global`. Escala, rede e certificados são do Terraform, em outro repositório (Capítulo 6). Fontes: `apps/api/Dockerfile`, `apps/hub/Dockerfile`, `.github/workflows/deploy.yml` e `docs/INFRA.md`.

6

Arquitetura de nuvem e implantação

Este capítulo descreve *onde* o *software* do Capítulo 5 roda: quais recursos gerenciados existem, como o tráfego atravessa a plataforma, como a identidade de carga de trabalho substitui chaves e onde estão as fronteiras de rede.

A infraestrutura é provisionada por Terraform (BRIKMAN, 2022) num **repositório separado**. Este documento descreve o *contrato* entre os dois repositórios — nomes de recursos esperados, variáveis consumidas, artefatos publicados — e não o código Terraform.

> **REGRA INVIOLÁVEL**
>
> **Este repositório nunca cria recurso GCP imperativamente.** Nenhum *script* de `scripts/` chama `gcloud ... create`. O CI apenas *usa* recursos que já existem: envia imagem para um Artifact Registry existente, executa um Cloud Run Job existente, sincroniza um *bucket* existente. Recurso criado por *script* é recurso que o Terraform não sabe que existe — e que ninguém destrói quando o contrato termina.

6.1 Princípios de nuvem

Uma *stack* por contrato.
Todos os nomes de recurso derivam de `{contrato} = {app_name}-{env}`. No piloto, `plataforma-unooria-dev`. Replicar um contrato é instanciar o *module* Terraform com outro prefixo.

Serviço gerenciado antes de orquestração.
Cloud Run em vez de GKE; Cloud SQL em vez de PostgreSQL autogerido; Pub/Sub em vez de *broker* próprio. O critério não é moda: é quantas peças uma pessoa consegue operar de plantão às oito da manhã.

Sem chave de longa duração.
O CI autentica por Workload Identity Federation; a API autentica no Vertex por ADC. Nenhum arquivo JSON de conta de serviço existe em nenhum lugar do fluxo.

Reversão por ponteiro.
Três níveis: revisão do Cloud Run (hub e API), `versaoAtiva` no manifesto (experiências) e versionamento de objetos (planos de aula). Nenhum deles exige *build*.

Estático na borda, dinâmico no mínimo.
O que pode ser servido pela CDN é servido pela CDN. O que sobra para o Cloud Run é regra de negócio.

6.2 C4 nível 1 de implantação — o contrato

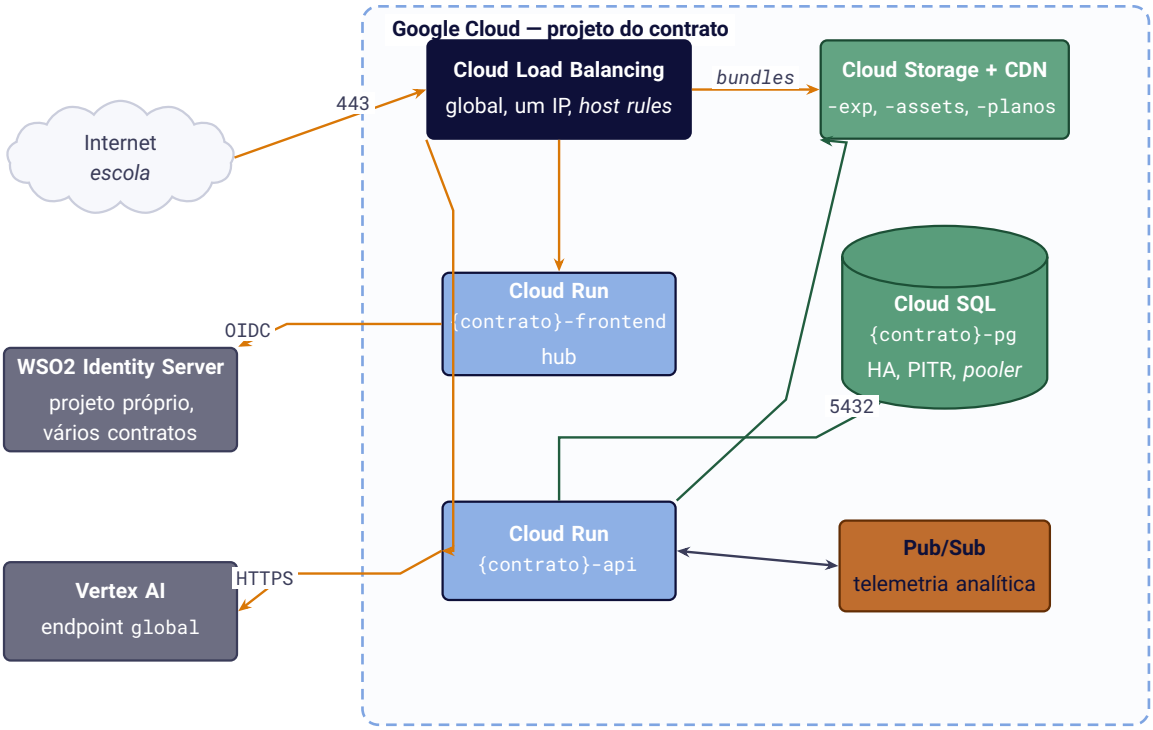


Figura 6.1 – Implantação de um contrato. Um único *Load Balancer* global recebe todo o tráfego HTTPS e distribui por *host rules*: hub, API, *bundles* e *assets* respondem em domínios diferentes com o *mesmo* IP. O Cloud SQL só tem IP privado — daí a existência do Cloud Run Job de migração dentro da VPC.

Legenda: tráfego público (TLS) tráfego privado (VPC) serviço de borda serviço de aplicação banco mensageria fora da *stack*

6.3 Diagrama de implantação detalhado

A Figura 6.2 abre o mesmo desenho no nível em que ele é operado: com os *buckets* separados por regime de acesso, os segredos, o registro de imagens e os dois caminhos de identidade de máquina.

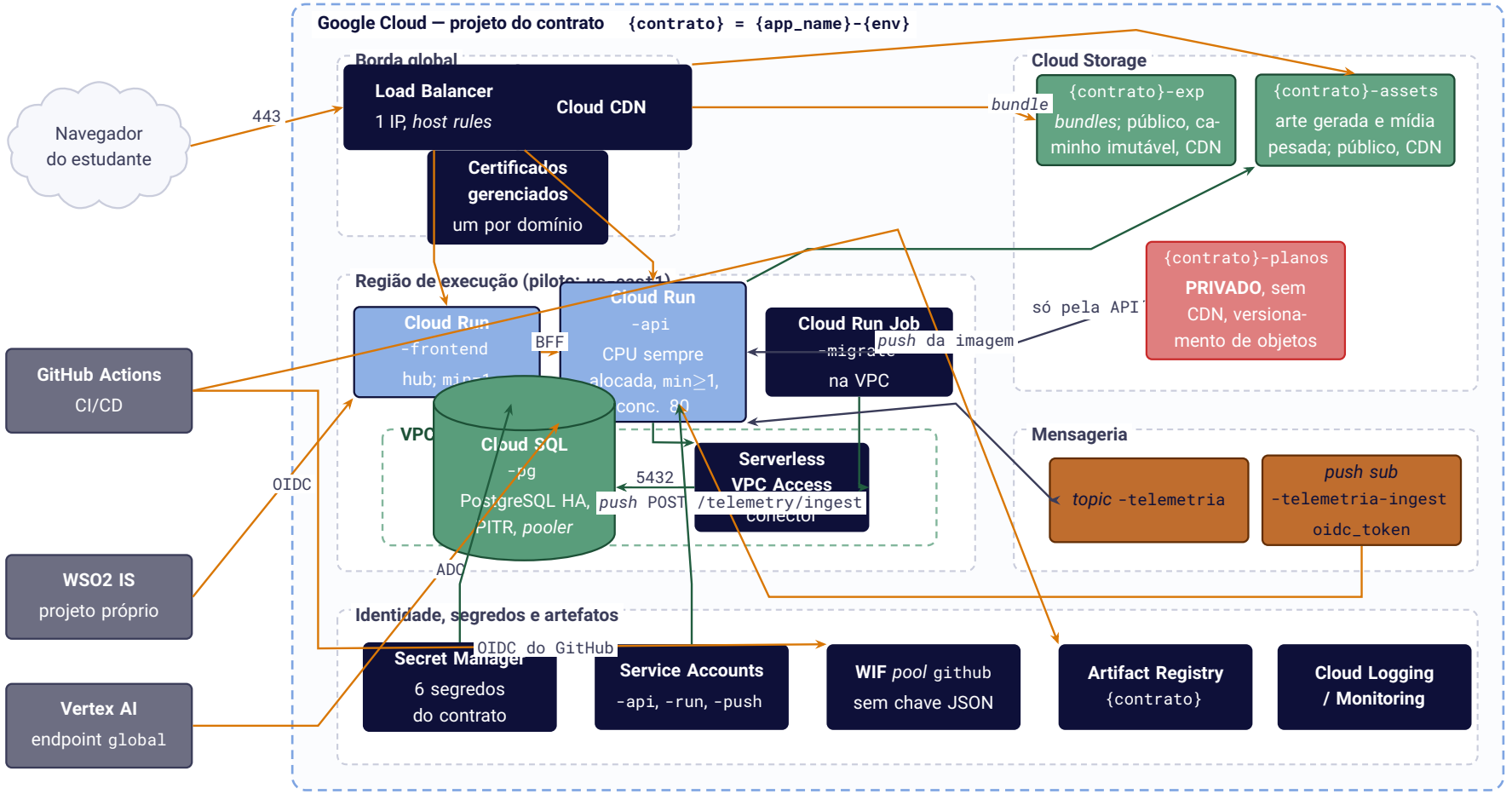


Figura 6.2 – Implantação detalhada. Os três *buckets* têm regimes de acesso deliberadamente diferentes: o de *bundles* e o de *assets* são públicos atrás da CDN (caminho versionado, portanto imutável); o de planos de aula é **privado**, sem CDN, com versionamento de objetos, e só é lido pela API depois de verificar papel e posse. O Cloud SQL não tem IP público: é isso que obriga a migração a rodar como *Job* dentro da VPC.

6.4 Catálogo de recursos por contrato

Este é o contrato formal com o repositório de Terraform. Nome divergente significa CI quebrado.

Recurso	Nome esperado	Consumido por
Cloud Run (serviço)	`{contrato}-frontend` (hub), `{contrato}-api`	`deploy.yml`
Cloud Run (*job*)	`{contrato}-migrate` — na VPC, roda `prisma migrate deploy`	`deploy.yml`
Artifact Registry	`{contrato}` (mesma região do Cloud Run)	`deploy.yml`
Bucket de experiências	`{contrato}-exp` + Cloud CDN	`deploy.yml`; hub (URL do `iframe`)
Bucket de *assets*	`{contrato}-assets` + Cloud CDN, leitura pública	API (`ASSETS_BUCKET`); mídia pesada das experiências
Bucket de planos	`{contrato}-planos` — **privado**, sem CDN, versionamento de objetos ligado	API (`PLANOS_BUCKET`)
Cloud SQL	`{contrato}-pg` — HA, PITR, *pooler* ativo	API (`DATABASE_URL`)
Pub/Sub	*topic* `{contrato}-telemetria` + *push sub* `{contrato}-telemetria-in` → `POST {api}/telemetry/ingest`	API
Secret Manager	`{contrato}-database-url`, `{contrato}-db-password`, `{contrato}-token-signing-key`, `{contrato}-auth-secret`, `{contrato}-perfil-import-token`, `{contrato}-planos-import-token`	*env* do Cloud Run
Service Accounts	`{contrato}-api` (Vertex, *publisher* do Pub/Sub, `objectAdmin` nos *buckets* de *assets* e de planos), `{contrato}-run` (hub), `{contrato}-push` (assina o *push* da telemetria)	Cloud Run, Pub/Sub

Recurso	Nome esperado	Consumido por
Load Balancer	Um só, global: hub em `{hub_domain}`, API em `{api_domain}`, *bundles* em `{exp_domain}`, *assets* em `{assets_domain}` — mesmo IP, *host rules*	DNS
WIF para GitHub Actions	*pool* `github`	`deploy.yml` (sem chave JSON)
Bucket do agente	`<projeto>-agente-professor` — privado, versionado; prefixos `bncc/`, `livros/`, `habilidades/`, `metadados/`, `import-erros/`	`apps/agente-professor/ingestao` (fora do Terraform); os *data stores* leem dele
Data stores (Vertex AI Search, `global`)	`agente-professor-habilid` (estruturado, esquema próprio) e `agente-professor-acervo` (*parser* de *layout*, *chunking* de 500 *tokens* com títulos ancestrais)	o agente (`discoveryengine.viewer`)
Agent Engine	`agente-professor` em `us-east1` (dev e hml; produção a decidir). **Nasce só no segundo *apply***, quando `agente_professor_imagem` aponta para uma imagem publicada	API (`AGENTE_PROFESSOR_ENGINE`); `deploy.yml` (`PATCH` da imagem)
SA do agente	`{prefixo}-agente` (o id de SA tem teto de 30 caracteres): `aiplatform.user`, `discoveryengine.viewer`, log, rastro, métrica e leitura do *bucket*	o Agent Engine
Papéis custom no engine	`agenteProfessorInvocador` (SA da API: `reasoningEngines.get/query`, `sessions.get/update`) e `agenteProfessorPublicador` (CI: `reasoningEngines.get/update`)	API; `deploy.yml`

NOTA

Os recursos do agente de IA do professor (ADR-126, ADR-127) vivem em `agente_professor.tf` no repositório de fundação, que subiu o *provider* `google` para `~> 7.46` (o `google_vertex_ai_reasoning_engine` com `container_spec` e o IAM do engine não existem na 6.x). Três coisas ficam FORA do Terraform: a importação dos documentos (`documents:import`, pelo `ingestao/` do *monorepo* — dado não é infraestrutura), os *service agents* do Discovery Engine e do Agent Engine (criados uma vez por projeto com `gcloud beta services identity create`; o recurso que faria isso só existe no *provider* beta), e o **aumento de quota do Agent Engine** (90 `streamQuery` e 300 eventos de sessão por minuto, por projeto e região), que se pede no console antes do *go-live*. O *chunking* do acervo nasce certo ou não nasce: a configuração de processamento é imutável, e trocá-la é recriar o *data store* e reimportar tudo. Apagar o engine apaga o histórico de todos os professores; o `deletion_policy` padrão recusa engine com sessões, e não se troca para `FORCE`.

DECISÃO DE PROJETO

Uma *role* de banco, não duas. O desenho original previa duas *roles*: uma dona do *schema* para as migrações e uma `app_runtime` sem *ownership* para a aplicação. Na prática ficou **uma**, porque as tabelas usam `FORCE ROW LEVEL SECURITY` — que sujeita o próprio dono às políticas e torna a separação desnecessária. O que *continua* obrigatório é a *role* não ter `BYPASSRLS` nem `SUPERUSER`; há SQL de verificação no *runbook* e um teste local (`tests/tenancy.spec.ts`) que cobre o mesmo contrato com uma *role* sem *ownership*.

6.5 Caminhos de tráfego

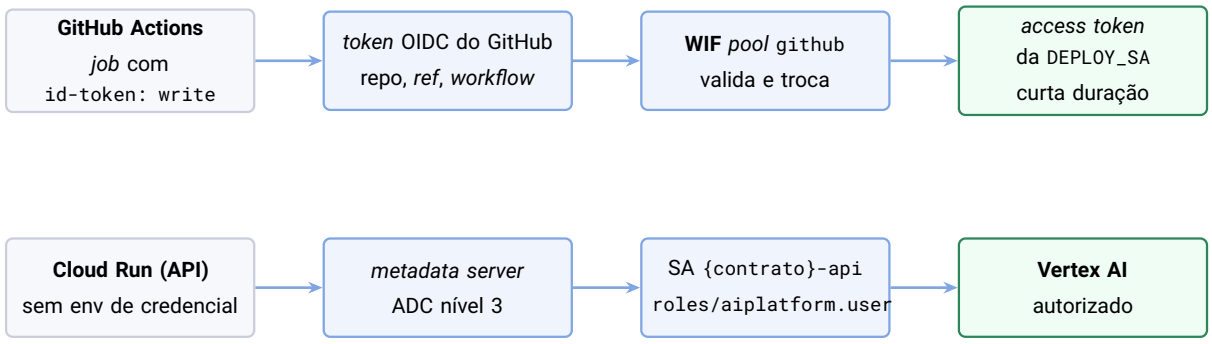
Caminho	Rede	Observação
Navegador → LB → hub	Pública, TLS gerenciado	Sessão do hub; SSR e BFF.
Navegador → LB → CDN → *bucket* de *bundles*	Pública, TLS	O grosso dos *bytes* do pico. Caminho versionado, portanto imutável; só o `index.html` tem `max-age=60`.
Navegador (dentro do `iframe`) → LB → API	Pública, TLS + *Bearer*	CORS por lista explícita (`CORS_ORIGINS`).
hub → API	Pública, TLS	`API_INTERNAL_URL`; o hub repassa o *token* do usuário.
API → Cloud SQL	**Privada**, VPC	O banco não tem IP público.
Cloud Run Job → Cloud SQL	**Privada**, VPC	Único caminho para rodar migração.
API → Vertex AI	Pública, TLS + ADC	Endpoint `global`.

Navegador → LB → hub → API → Agent Engine	Pública, TLS; SSE em cada salto	O chat do agente (**ADR-128**). A API chama `{região}-aiplatform.googleapis.com` com o ADC dela; teto de 240 s na API, 250 s na rota de *stream* do hub, 300 s no Cloud Run do hub, 900 s no da API e 60 minutos fixos no LB com NEG *serverless*.
Agent Engine → Gemini e *data stores*	Pública, TLS + SA do agente	Modelo em `global` com reserva em `us`; *data stores* em `global`. O navegador nunca chega aqui.
API → Pub/Sub *topic*	Pública, TLS + SA	Publicação do lote analítico.
Pub/Sub → API (`/telemetry/ingest`)	Pública, TLS + *ID token*	Verificado por `PubSubPushGuard`: emissor Google, `aud` e `email` da SA de *push*.
hub → WSO2 IS	Pública, TLS	OIDC. Fora da *stack* do contrato.

ATENÇÃO

O *Load Balancer* exige **dois** certificados gerenciados quando o hub e a API estão em domínios distintos — um por domínio. Provisionar apenas o do hub faz a API responder com erro de TLS que *parece* problema de aplicação: o navegador não completa o *handshake* e a experiência mostra falha de rede sem nenhuma linha no log da API.

6.6 Identidade de carga de trabalho e segredos



A mesma cadeia nos três ambientes. A API resolve a credencial com `google-auth-library`, que tenta, na ordem: (1) `GOOGLE_APPLICATION_CREDENTIALS`; (2) o arquivo de `gcloud auth application-default login`; (3) o *metadata server*. Antes disso havia *só* o nível 3 — e por isso nenhuma chamada de IA funcionava fora do Cloud Run: nem no *host*, nem no contêiner, onde `metadata.google.internal` não resolve. Em desenvolvimento a API ficava presa em `AI_PROVIDER=mock` e a app-90 devolvia um plano falso.

Nunca chave de API. Não há campo de env para chave de IA em lugar nenhum do projeto — nem em `.env.example`. Os dois *compose* de desenvolvimento montam o ADC do *host* em `/adc` somente leitura; em Cloud Run a SA do serviço *é* a credencial, e o que falta é apenas o papel `roles/aiplatform.user`. Sem ele, toda geração falha com 403 do Vertex — e o 403 vem do Vertex, não da API, o que é um sintoma confusamente parecido com quota esgotada.

Figura 6.3 – Os dois caminhos de identidade de máquina, ambos sem chave de longa duração: WIF para o CI e ADC para a aplicação.

Segredo / variável	Onde vive	Por quê
`{contrato}-database-url`, `{contrato}-db-password`	Secret Manager	Credencial de banco.
`{contrato}-token-signing-k`	Secret Manager	Chave privada RS256 que assina o *token* de experiência. Também usada pelo CI para assinar o *token* de cinco minutos do *upsert* de manifesto.
`{contrato}-auth-secret`	Secret Manager	Segredo de sessão do NextAuth.
`{contrato}-perfil-import-` `{contrato}-planos-import-`	Secret Manager	Chaves de serviço de duas rotas de importação. **Ausentes = rota desligada** (503), nunca aberta.
`OIDC_ISSUER`, `OIDC_JWKS_URI`	*env* em texto plano	**Não são segredo**: são URLs públicas do hub. Ficam em texto plano derivadas de `hub_domain` — é isso que garante que o `iss` do *token* e o esperado pela API *não podem* divergir.
`TELEMETRIA_PUSH_SA`, `TELEMETRIA_PUSH_AUDIENCE`	*env* em texto plano	Identificam quem pode chamar `/telemetry/ingest`. Ausentes = rota 401.

`AGENTE_PROFESSOR_MODO`, `AGENTE_PROFESSOR_ENGINE`	*env* em texto plano	`desligado` enquanto o engine não existe; `agent-engine` e o *resource name* completo depois do segundo *apply*. Não são segredo: quem autoriza é o IAM do engine.

6.7 Dimensionamento

Recurso	Configuração	Motivo
Cloud Run — API	CPU sempre alocada, `min-instances` ≥ 1, concorrência 80	Há trabalho *in-process* fora do ciclo de requisição: o *worker* de geração de conteúdo da exp08ef1b e a geração de plano de aula da app-90. Com CPU liberada entre requisições, a geração morreria no meio.
Cloud Run — hub	`min-instances` ≥ 1	Elimina *cold start* no pico de *login*.
Cloud SQL	Instância pequena, HA, PITR, *pooler*	10.000 estudantes e o volume medido de telemetria cabem folgadamente. O *pooler* é obrigatório porque o *autoscaling* do Cloud Run multiplica *pools* do Prisma e estoura `max_connections`.
CDN	*cache* longo no caminho versionado; `max-age=60` no `index.html`	O caminho carrega a versão, logo o conteúdo é imutável. O `index.html` é o ponteiro: *cache* longo nele atrasaria um *rollback*.
Pub/Sub	*Push subscription* com `oidc_token`	Amortecedor durável do pico, e uma peça a menos que um *worker* dedicado.

6.7.1 Volumes medidos

Grandeza	Valor medido	Como se chegou a ele
Payload de uma partida da app-70	1,1 kB de JSON (1,5 kB em JSONB)	Medido em 2026-08-26. O jogo tem sete pontos de log, todos atos discretos com guarda própria: 8 Chamas, 4 aldeões, 1 Borda + 3 Destaques, 4 cartões, 1 primeira conversa — teto real de ~22 eventos por partida. Nada por quadro, nada por metro andado.
Armazenamento da app-70 por contrato	**~30** MB	10.000 estudantes jogando uma vez. A estimativa *anterior* — centenas de MB a 2,4 GB — partia do teto do contrato (2000 eventos) e errava por duas ordens de grandeza. O teto do contrato caiu de 2000 para 200, que é o que de fato limita a linha se um cliente adulterado mandar lixo.

Custo de gravação da telemetria	~16 `INSERT`/s no pico (era ~800)	Um lote de 50 eventos custava 50 *round-trips*; em `INSERT` multi-linha custa um. No pico de ~4000 simultâneos a diferença é entre saturar e não notar.
Prompt de um plano de aula anual	~190.000 *tokens*	Currículo da rede + as quatro unidades do ano. É o que justifica a quota separada de professor (2.000.000/dia ≈ 6 a 9 planos).
Mídia da exp04ef1b	52 MB (21 arquivos MP4)	Fora do *git* e fora do *bundle*: vai para o *bucket* de *assets*, referenciada por `VITE_VIDEOS_BASE_URL`.

6.8 Ambientes

Ambiente	Como sobe	Diferenças relevantes
Host (desenvolvimento)	`docker compose up -d pg` + `pnpm dev`	Tudo no *host*; só o banco em contêiner. `AI_PROVIDER` pode ser `mock`; *drivers* de armazenamento locais; sem Pub/Sub.
Contêiner (desenvolvimento)	`pnpm docker:dev`	A *stack* inteira em contêiner — banco, API, hub e as cinco experiências — com o código do *host* por *bind mount*: salvar recompila lá dentro. Mesmas portas do `pnpm dev`, então é um *ou* o outro.
Imagens congeladas	`docker compose up -d --build`	Aproxima-se da produção: imagens construídas, migração e *seed* automáticos no primeiro `up`.
`dev` em GCP	CI no *push* para `main`	*Stack* real do piloto (`plataforma-unooria-dev`), com todos os recursos da Figura 6.2.
Produção	Mesmo CI, outras `vars` de repositório	O *workflow* não conhece o nome de nenhum recurso: tudo entra por `vars`, cujos valores saem de `terraform output`.

NOTA

Nos ambientes de contêiner, a leitura *server-side* do `experiencia.json` exige `EXP_INTERNAL_HOST` — que troca apenas o *host* e **preserva a porta**. `EXP_INTERNAL_URL`, que é um endereço fixo, só serve quando existe *uma* origem de *bundles* (a CDN, em produção): em desenvolvimento, cada experiência tem um *dev server* numa porta diferente, e um endereço fixo faria todas lerem o manifesto de uma só. O sintoma dessa configuração errada é específico e memorável: o UnooTube, que não declara `viewport`, aparece dentro da casca de celular da Instanooria.

ATENÇÃO

Os *dev servers* das experiências declaram `allowedHosts: ["host.docker.internal"]`. O Vite 5.4+ recusa cabeçalho `Host` que não conheça, e sem essa declaração o hub em

contêiner não lê o manifesto — e a experiência aparece esticada na janela em vez de dentro do aparelho.

6.9 O provedor de identidade fora da *stack*

O WSO2 Identity Server foge do padrão `{contrato}-*` por desenho: é **único para vários contratos** e tem ciclo de vida próprio. Vive em projeto GCP próprio e, em desenvolvimento, num *compose* separado (`wso2/docker-compose.yml`, IS em `:9443` e banco próprio em `:5433`).

DECISÃO DE PROJETO

O hub assina os *tokens* de experiência enquanto o IdP central não entra (ADR-15). O hub emite RS256 com TTL de 15 minutos e publica o JWKS em `/.well-known/jwks.json`; a API valida por `OIDC_ISSUER` e `OIDC_JWKS_URI`. Trocar para o WSO2 é trocar essas duas variáveis — o `OidcGuard` não muda uma linha. A alternativa (esperar o IdP para começar a integrar) faria toda a plataforma depender de um servidor Java em *bootstrap* para que qualquer experiência pudesse ser testada.

ATENÇÃO

O `issuer` do WSO2 é *tenant-qualified* (`.../t/{contrato}/oauth2/token`) e deve ser copiado **literalmente** do *discovery*, nunca montado por concatenação. Um `iss` montado à mão que difere por uma barra faz toda validação falhar com "*issuer* inválido" — e a mensagem não diz qual dos dois lados está errado.

6.10 Observabilidade, continuidade e reversão

Aspecto	Como é atendido hoje
Sanidade de processo	`GET /healthz` (processo vivo) e `GET /readyz` (dependências prontas). São rotas sem *guard*, de propósito.
Log	Cloud Logging pelo *stdout* do Cloud Run. **Nenhuma PII em texto claro**; texto livre de bem-estar nunca vai a log (só contagens).
Métricas	Métricas nativas do Cloud Run e do Cloud SQL. Não há métrica de aplicação customizada — é lacuna conhecida (Cap. 19).
Reversão do hub e da API	Revisões do Cloud Run. `gcloud run services update-traffic` devolve o tráfego à revisão anterior.
Reversão de experiência	`node apps/hub/scripts/publicar-manifesto.mjs <exp> --versao <anterior>`. Sem *build*, sem imagem.

Reversão de conteúdo de professor	Versionamento de objetos do *bucket* de planos: cada “Salvar” é uma versão, sem uma linha de código de histórico.
Recuperação do banco	HA + PITR do Cloud SQL.
Migração de *schema*	Cloud Run Job dentro da VPC, executado **antes** do *deploy* do serviço, com `--wait` para que a falha interrompa o *pipeline*.

LIMITAÇÃO CONHECIDA

Não há *alerting* configurado, nem painel. Há log e métrica nativa, e há sanidade por `/readyz` — o que basta para diagnosticar, mas não para ser avisado; e `/readyz` não é chamada por ninguém, porque o *deploy* não configura sonda. O estudo completo — o que se vê hoje, elo por elo, os sinais a acrescentar, a comparação de ferramentas e o roteiro em fases com custo por contrato — é o Capítulo 17. A fase 0 dele (canais, *uptime checks*, políticas, SLOs e painel, tudo em Terraform) é o que fecha o risco `R-05` antes de novembro de 2026.

7

Multi-tenancy e segurança

Este é o capítulo que, se estiver errado, invalida o produto. Uma plataforma que serve cem escolas de uma rede não pode, sob nenhuma circunstância — inclusive sob erro de programação — devolver o dado de uma escola a outra.

7.1 Os três níveis de isolamento

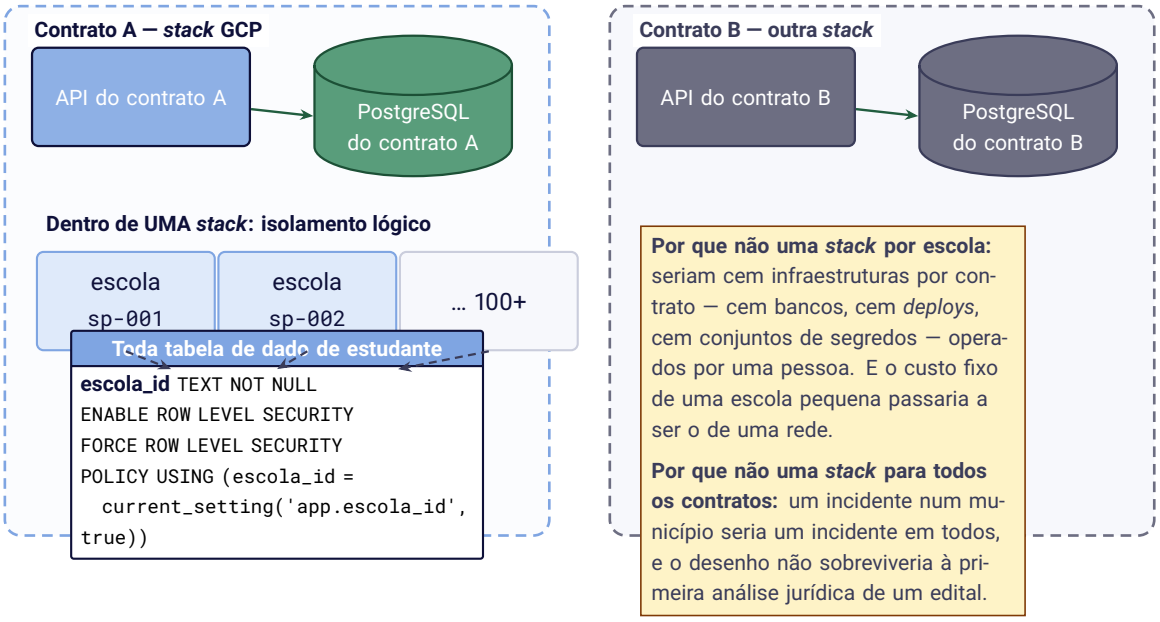


Figura 7.1 – Isolamento em três níveis: físico entre contratos, lógico entre escolas de um contrato, e por autorização entre turmas e estudantes de uma escola.

7.2 Nível 2: *Row-Level Security* no PostgreSQL

7.2.1 Como a política é armada

Toda tabela de dado de estudante tem `escola_id TEXT NOT NULL` desnormalizado e uma política de RLS (The PostgreSQL Global Development Group, 2026a). A migração inicial aplica isso em laço sobre a lista de tabelas:

Listing 7.1 – Aplicação da RLS na migração inicial

```
DO $$
DECLARE t text;
BEGIN
  FOREACH t IN ARRAY ARRAY[ /* ... tabelas de dado de estudante ... */ ] LOOP
    EXECUTE format('ALTER TABLE %I ENABLE ROW LEVEL SECURITY', t);
    EXECUTE format('ALTER TABLE %I FORCE ROW LEVEL SECURITY', t);
    EXECUTE format(
      'CREATE POLICY %I ON %I USING (escola_id = current_setting(''app.escola_id'', true))',
      t || '_por_escola', t
    );
  END LOOP;
END $$;
```

> **DECISÃO DE PROJETO**
>
> **`FORCE ROW LEVEL SECURITY`, não apenas `ENABLE`.** `ENABLE` isenta o *dono* da tabela das políticas. Com `FORCE`, nem o dono escapa — e é isso que dispensou a segunda *role* de banco que o desenho original previa. O que continua obrigatório é a *role* da aplicação não ter `BYPASSRLS` nem `SUPERUSER`, porque esses atributos derrotam até o `FORCE`.

7.2.2 A porta única: `comTenancy`

Listing 7.2 – A única porta de acesso ao banco (`src/prisma/prisma.service.ts`)

```
async comTenancy<T>(
  fn: (tx: Prisma.TransactionClient) => Promise<T>,
  escolaId?: string,
  opts?: { timeout?: number },
): Promise<T> {
  const escola = escolaId ?? escolaDoContexto();
  if (!escola) {
    // Falha ALTA. A alternativa — consultar sem filtro — e exatamente o
    // acidente que este metodo existe para tornar impossivel.
    throw new Error(
      "comTenancy sem escola: rota sem OidcGuard/TenancyInterceptor ou " +
      "chamada fora de requisicao sem escolaId explicito",
    );
  }
  return this.$transaction(async (tx) => {
    // `true` = local a TRANSACAO. Um SET de sessao vazaria a escola para a
    // proxima requisicao que reusasse a conexao do pool.
    await tx.$executeRaw`SELECT set_config('app.escola_id', ${escola}, true)`;
    return fn(tx);
  }, opts);
}
```

Três detalhes desse método carregam a segurança inteira:

1. **O terceiro argumento de `set_config` é `true`**, ou seja, o valor é local à transação. Um `SET` de sessão sobreviveria ao `COMMIT` e a conexão voltaria ao *pool contaminada* — a próxima requisição, de outra escola, herdaria a política errada. Esse é o bug clássico de RLS com *pool* de conexões, e é evitado por um booleano.
2. **Sem escola, lança.** Não consulta "sem filtro", não devolve lista vazia: lança. Rota nova que esqueceu o `OidcGuard` quebra no primeiro teste, em vez de funcionar devolvendo tudo.
3. **A escola vem do contexto, não do parâmetro.** O parâmetro explícito existe só para código fora de requisição — *seed*, *worker*, testes.

REGRA INVIOLÁVEL

O `comTenancy` arma a RLS; ele NÃO filtra a *query*. Ele executa um `set_config` que as POLÍTICAS leem — e tabela sem política não é alcançada por ele. `users`, `turmas` e `lotacoes` estão deliberadamente fora da RLS (são lidas no *login*, antes de se saber a escola), então nelas o `escola_id` no `where` não é defesa em profundidade: **é a única tranca**.
Isso já custou um vazamento real. `distribuicaoDaTurma`, que monta o painel de perfis do Instanooria, rodava `user.findMany({ where: { role: 'estudante' } })` — dentro de um `comTenancy`, o que dava a impressão de estar protegido. O painel listava nome e login de crianças de TODAS as escolas do contrato, e os contadores que decidem quantos estudantes o botão "Redistribuir a turma" move eram calculados sobre esse universo. Ver **ADR-49**; a regressão é coberta por `tests/perfis-distribuicao-tenancy.spec.ts`, que roda como superusuário justamente para que a RLS não mascare a ausência do `where`.

ATENÇÃO

A armadilha inversa: tabela COM RLS lida fora do `comTenancy` não falha — devolve vazio. As três tabelas das equipes na missão (`equipes_montagens`, `equipes` e `equipes_membros`, **ADR-111**) entram na política como toda tabela de escola: `escola_id NOT NULL`, `ENABLE` e `FORCE ROW LEVEL SECURITY` e uma política por escola em cada uma — `equipes_membros` com mais razão, por ser a única das três com id de criança. O risco delas é outro: quem as lê são OUTROS módulos (o painel do unooGPT, a grade da sala), por funções livres que recebem o `tx` (`equipes/consulta.ts`), no molde de `comum/liberacao.ts`. Chamadas fora da transação de tenancy, elas devolvem um mapa vazio, e o painel volta a contar por criança sem que ninguém saiba por quê. E como `users` está fora da RLS, a lista de estudantes que a montagem aceita sai de `users.turma_id` com o `escolaId` no `where` — a regra acima, de novo.

ATENÇÃO

Na evidência pedagógica, a mesma política falha ALTO na escrita. As três tabelas do módulo `evidencias/` (**ADR-112**) — `evidencias_marcas`, `evidencias` e `evidencias_eventos`, todas com id de criança — têm `escola_id NOT NULL`, `ENABLE` e `FORCE ROW LEVEL SECURITY` e uma política por escola. Quem ESCREVE a marca é outro módulo: a missão chama `EvidenciasService.marcar` com o `tx` DELA, e o serviço não abre `comTenancy` de propósito, para a marca cair junto se a jogada cair. Com um cliente sem a escola armada, a leitura devolve vazio, como nas equipes, mas o `INSERT` é recusado: a política sem `WITH CHECK` vale também para a linha nova, e `current_setting('app.escola_id', true)` nulo não casa com escola nenhuma. O erro de política no log é o sintoma de uma missão marcando fora da transação de *tenancy*. O histórico tem uma tranca a mais, no molde de `recursos_auditoria`: a *role* de *runtime* recebe só `SELECT, INSERT` em `evidencias_eventos`, e um *trigger* recusa `UPDATE` para qualquer *role*, inclusive a dona da tabela. E, de novo, a lista de estudantes da turma sai de `users.turma_id` com o `escolaId` no `where`.

7.2.3 Como a escola chega ao contexto

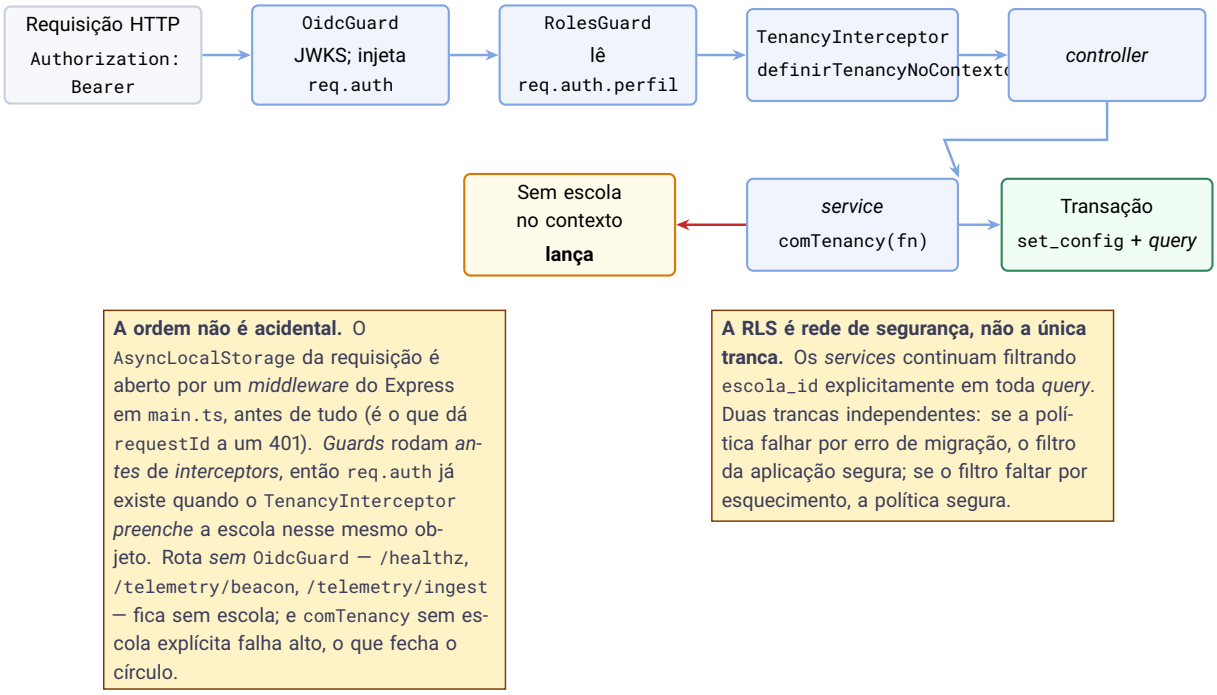


Figura 7.2 – Como a escola do *token* chega à política de RLS. Duas trancas independentes protegem o mesmo invariante.

7.2.4 Os papéis sem escola fixa: `gestor_contrato`, `admin` e `gestor_regional`

> **REGRA INVIOLÁVEL**
>
> `gestor_contrato` e `admin` (**ADR-80**) são os papéis autorizados a atravessar escolas na gestão da rede. Para ambos, `escolaId` nas *claims* é vazio. O `gestor_regional` tampouco tem escola fixa: cobre a sua regional e seleciona uma escola ativa por sessão. O *CHECK* da migração (`20260908150000_modelo_de_papeis`) restringe `escola_id NULL` em `users` estritamente a `'gestor_contrato'`, `'admin'` e `'gestor_regional'`. Nenhuma outra combinação é aceita pelo banco.

As rotas cross-escola vivem em *controllers* separados (`src/analise/analise.controller.ts`, o *controller* de `/analise/*`) e não num `@Roles` misto com exceção por método — porque um `@Roles` misto é o caminho mais curto para um `GET` novo herdar silenciosamente a permissão do vizinho.

7.3 A tabela sem tenant: o cadastro de interesse (ADR-107)

> **REGRA INVIOLÁVEL**
>
> **`prospeccao_leads` não tem `escola_id` e não tem RLS**, e é a ÚNICA tabela de dados pessoais do banco nessa situação. A exceção está escrita aqui porque a regra inviolável nº 1 não admite esquecimento: toda outra tabela de dados de pessoa carrega `escola_id NOT NULL` mais política de RLS.

Quem escreve nessa tabela é um **visitante do site público**, meses antes de existir contrato: não há escola para isolar, não há pessoa da rede para identificar e não há sessão para exigir. Uma política por `escola_id` sobre uma tabela sem a coluna esconderia a tabela de todos. A consequência prática, registrada em `apps/api/src/prospeccao/AGENTS.md`: as *queries* desse módulo rodam FORA de `comTenancy` — de propósito, e só ali.

O que protege cada lado:

- **A leitura** é `@Roles("admin")` (**ADR-88**), como todo o painel da Unooria. O `gestor_contrato` é o cliente, e a lista de quem procurou a fornecedora é sobre ele, não para ele.
- **A escrita é pública por desenho**, e é a única rota de dados da api sem *guard*. A defesa dela é o Zod, a cota por IP (5/hora por endereço, 200/hora na instância), o campo-armadilha e o *dedupe* de uma hora. O IP é chave de cota e não é gravado em lugar nenhum — ele identificaria a pessoa sem acrescentar nada ao que o cadastro já diz.
- **Dois *controllers*, não um**, pelo mesmo motivo dos *controllers* cross-escola acima: um `@UseGuards` esquecido num método some na revisão; um arquivo inteiro sem *guard* é decisão que se lê no topo.

A base legal é o CONSENTIMENTO (LGPD art. 7º, I), ao contrário do dado de estudante, cuja base é a execução de política pública. Aqui ele é válido porque a relação é voluntária: ninguém é obrigado a pedir uma apresentação comercial. Por isso a coluna guarda a FRASE aceita (`consentimento_texto`) e não um booleano — um booleano prova que alguém clicou, não a que consentiu (art. 8º, §2º) — e por isso a *role* de *runtime* tem `DELETE`, que `ia_quotas` não tem: o titular pode revogar e pedir a eliminação (art. 18, VI), e tabela sem `DELETE` torna o pedido impossível de atender. **Não há rota de exclusão**; o pedido é atendido por acesso direto ao banco, e a tela de Prospecção diz isso em vez de prometer um botão inexistente.

7.4 A conversa do agente: a posse no Postgres, o conteúdo fora dele

O agente de IA do professor (**ADR-126**) é o primeiro produto em que o dado de uma pessoa **não mora no Postgres**: o conteúdo da conversa é a sessão do Vertex AI Agent Engine. O que o Postgres guarda é **de quem ela é**, e é essa linha que faz o isolamento valer.

- **Por que a posse não pode ficar no engine.** Os *endpoints* de sessão pedidos por nome de recurso não conferem o usuário, e a SA da api enxerga *todas* as sessões do engine. Sem a linha, quem soubesse um `sessao_id` leria a conversa de outro professor — de outra escola, inclusive. A checagem do ADK ("does not belong to user") é defesa em profundidade.
- **`agente_professor_conversas` tem `escola_id NOT NULL`, `ENABLE` e `FORCE ROW LEVEL SECURITY`**, com a mesma política das outras tabelas, porque o título que o agente gera sai da pergunta — e a pergunta pode trazer o nome de um estudante. A RLS tranca a escola; o `daPosse` acrescenta o `usuario_id`, e **toda rota passa por ele antes de tocar a sessão**. Conversa de outro professor, ou de outra escola, é **404, nunca 403** (403 confirmaria que o id existe); id que não é UUID também é 404. A conversa pertence a (professor, escola ativa): trocar de escola troca a lista.
- **Identidade só do *token*.** O `user_id` no engine é o `usuarioId` do *token*, e o `nome_professor` vem de `users.display_name`; nada vem do corpo.
- **Só `professor`.** A classe tem `@Roles("professor")`, e o chat só é montado no *layout* de `/professor`. A coordenação ficou fora por decisão do dono do produto, e a ausência é estrutural: não há rota.
- **Nenhuma transação `comTenancy` fica aberta durante o *stream***: posse antes, atualização depois, cada uma curta. Um turno dura dezenas de segundos, e uma transação pendurada nesse tempo seria uma conexão do *pool* presa por professor.

> **ATENÇÃO**
>
> **Dado de professor — e de estudante citado por engano — fora do Postgres.** O texto da conversa fica na sessão do engine (`us-east1` em `dev` e `hml`), fora do PITR, das exportações e do Capítulo 8; o modelo processa em `global`. Três consequências. **Apagar exige o engine no ar**: com ele fora, o `DELETE` responde 503 e a linha fica, porque apagar só a

> linha deixaria a conversa guardada sem dono alcançável. **O histórico não tem prazo**: a sessão nasce com expiração distante e é renovada a cada mensagem, e quem apaga é o professor. E **nenhum registro leva texto** — nem a pergunta, nem a resposta, nem o título, nem o corpo de erro do engine, na api e no agente: só ids, contagens, códigos, duração e *tokens*.

- **O painel INTEIRO leva `data-clarity-mask="true"`**: o Microsoft Clarity grava a tela do hub (ADR-60), e o professor pode escrever o nome de um estudante na pergunta. Junto do campo fica o aviso fixo "Não escreva nomes de estudantes." — aviso, não trava. E o `localStorage` guarda só o tamanho do painel; conversa, título e texto nunca vão para lá.
- **O *markdown* da resposta vira HTML no navegador sem HTML cru** e com links só `http`, `https` e `mailto` absolutos (ADR-129): é o que impede o modelo, ou um trecho de PDF, de injetar elemento na página.
- **Cloud Armor: a 921150 sai, as outras ficam.** A assinatura de injeção de cabeçalho (CR/LF) casa qualquer corpo com quebra de linha, que o professor produz ao colar texto ou usar Shift+Enter. A regra 1106 a exclui em `/agente-professor/` (*host* da api) e `/api/plataforma/agente-professor/` (*host* do hub) — os dois saltos passam pelo LB. Os conjuntos de SQLi, XSS, LFI, RFI, RCE e *scan* **continuam valendo**: o professor que colar um trecho de SQL ou de HTML na pergunta é barrado no LB, e o sintoma é uma falha de rede sem linha no log da api (Apêndice F).

7.5 O interruptor da rede: um nível que não é de tenancy

A ADR-87 acrescentou um eixo que decide se algo abre e que **não é autorização**: a *disponibilidade*. Ele responde "este recurso existe nesta rede?", e é PRÉ-CONDIÇÃO da liberação por turma (ADR-69), que por sua vez é pré-condição dos relógios e *gates* das experiências.

Eixo	Pergunta	Quem decide
catálogo	que forma a experiência tem?	o manifesto / o CI
disponibilidade (ADR-87)	existe NESTA REDE?	a Unooria (`admin`)
liberação (ADR-69)	está aberta NESTA TURMA?	o professor

A tabela `disponibilidades` fica FORA da RLS, contra o hábito da regra inviolável nº 1, e a exceção é declarada: é configuração comercial e operacional da rede, não dado de estudante, e a linha de escopo-contrato tem `escola_id` NULO por desenho — uma política por `escola_id` esconderia justamente a resposta que a tabela existe para dar. É o mesmo motivo de `users`, `turmas` e `lotacoes`. A ausência é provada por teste, para que a próxima migração não a "conserte": ligar RLS ali faz TODO produto parecer ligado.

Onde a guarda mora, e por que num ponto só em cada caso:

- *Missões de estudante* — dentro de `exigirMissaoLiberada`, ANTES da checagem de liberação. As seis experiências já passam por aquele funil, e um ponto único torna impossível existir liberação valendo num recurso que a rede não tem. A ordem importa para o diagnóstico: o `codigo` do 403 diz em qual dos dois eixos procurar.
- *Ferramentas de professor* (app-90, app-91, app-92) — não têm liberação por turma e somam **28 rotas**. A guarda é um `DisponibilidadeGuard` de CLASSE, com `@Recurso("app-90-plano-de-aula")` no `@Controller`: rota nova nasce coberta. Anotar rota por rota é o desenho em que a 29ª nasce sem guarda, sem erro e sem sintoma — o produto simplesmente deixa de poder ser desligado.
- *O escape é declarado* (`@SemDisponibilidade()`) e vale **só para apagar os próprios dados**. Desligar um produto fecha tudo; a exceção existe porque "não consigo apagar meus dados porque o produto foi desligado" é indefensável, e mais ainda no *check-in* de bem-estar, que é DASS-21.
- *Produto sem experiência* — o agente de IA do professor — declara `@Produto("agente-professor")` no lugar de `@Recurso`: não há linha em `experiencias` a resolver, e a pergunta é a do TETO, pelo mesmo resolvedor (`exigirProdutoDisponivel`). O escape é o mesmo: só o `DELETE` da conversa tem `@SemDisponibilidade`.
- *Guard montado sem `@Recurso` nem `@Produto` LANÇA*, em vez de liberar — liberar em silêncio é exatamente a falha que ele existe para impedir. Os dois juntos também lançam: o teto do produto já está dentro da resolução da experiência.

A mensagem que chega ao estudante é a MESMA de `missao-nao-liberada`: quem lê é uma criança dentro de um app disfarçado, e "este produto foi desativado para a sua rede" quebra a ficção que a experiência está medindo. Quem distingue os dois casos é o `codigo` (`recurso-indisponivel`), que é o que o log e a tela leem. Na ferramenta de professor não há disfarce a proteger, e ali a mensagem diz o que aconteceu.

7.6 Nível 3: autorização por papel

> **REGRA INVIOLÁVEL**
>
> **`admin` e `gestor_contrato` deixaram de ter as MESMAS capacidades** (**ADR-88**, revendo a afirmação da **ADR-80**). A primeira e única exceção é a *disponibilidade* de recursos: o painel vive em `/unooria/*` e é restrito ao `admin`, porque ligar e desligar produto é decisão da fornecedora, não de quem administra o contrato do lado do cliente — um `gestor_contrato` que pode LIGAR passa a ter acesso a produto que a rede não contratou. Para todo o resto a igualdade continua valendo.
>
> **Uma rota volta atrás dessa linha, e só uma**: `GET /recursos/auditoria` (**ADR-89**), o histórico do painel. Ler por que o produto da própria rede mudou de estado é informação do cliente; decidir o estado não é. `GET /recursos/painel` e as escritas seguem em 403 para o `gestor_contrato`.

A segunda capacidade só do `admin` é a quota de IA (`/recursos/quotas-de-ia/*`, **ADR-105**), pela mesma razão: quanto de IA uma rede pode gastar por dia é o custo variável da fornecedora. Aqui nem a leitura volta atrás da linha — o `gestor_contrato` recebe 403 no GET e no PUT, e a tela `/unooria/uso-de-ia` o manda embora pelo *middleware*.
E a lista das experiências arquivadas (`GET /recursos/arquivadas`, **ADR-116**) também é só dele: a rota não conta nada em tempo de uso — os números de turmas e estudantes guardados são a fotografia que a migração tirou ao arquivar —, e por isso não atravessa a RLS. O que a fecha no `admin` é o mesmo motivo das outras: o catálogo do que a Unooria já tirou do produto é da fornecedora.

Papel	Escopo de dados	Pode
`estudante`	Própria matrícula e progresso	Entrar nas missões liberadas para as suas turmas, salvar progresso e interagir no *feed*. **Nunca** vê gabarito, nem dado de colega.
`professor`	A própria escola; as turmas em que leciona	Liberar missões, controlar relógio de aula, emitir pareceres e gerar planos de aula e ensino. **Não** vê bem-estar de outro professor, nem turma em que não leciona.
`gestor_escola`	A própria escola (*tenant*)	Matricular estudantes, enturmar, gerir professores da unidade e consultar relatórios locais. **Não** vê a linha de uma criança nos instrumentos de valores, nem bem-estar docente.
`gestor_regional`	As escolas da regional vinculada	Alternar entre escolas da regional, visualizar panorama unificado (**ADR-81**) e relatório pedagógico por laço de oito escolas (**ADR-82**).
`gestor_contrato`	Toda a rede (cross-escola)	Criar escolas, importar CSVs de enturmamento, provisionar supervisores regionais e emitir consolidados.
`admin`	Toda a rede (conta Unooria)	Mesmas capacidades operacionais de `gestor_contrato`, preservando a autoria na cadeia de atribuição transitiva descendente (**ADR-80**).

7.6.1 Onde cada papel entra

O *middleware* do hub (`apps/hub/middleware.ts`) não conhece telas: ele conhece **áreas**, e `apps/hub/lib/papeis.ts` é a única fonte do mapa papel → área. Quem digita a URL de outra área não recebe 403 — é redirecionado para a própria casa, porque estudante que tropeça em `/professor` não precisa de uma mensagem de erro, precisa voltar.

Papel	Casa	O que existe na área

`estudante`	`/estudante`	O *hub* de missões e o aparelho da experiência (`/exp/:expId`). São a MESMA área: o hub escolhe, o `iframe` joga.
`professor`	`/professor`	Atividades (exp08ef1b), turmas, guia, plano de aula (app-90), bem-estar (app-91) e "entre redes" — os resultados por unidade.
`gestor`	`/admin/usuarios`	Usuários, turmas, importar, ano letivo. E "minhas escolas", que só aparece para quem tem mais de uma lotação.
`gestor_regional`	`/admin/usuarios`	O mesmo do `gestor_escola`, e uma escola por vez — o que muda é de onde sai a LISTA de escolas que ele pode ativar: da regional dele (`users.regional_id`), não da lotação (**ADR-80**).
`gestor_contrato`	`/admin/usuarios`	O mesmo, mais `/admin/regionais` e `/admin/escolas` (organizar a rede e abrir unidade) e `/analise` (o nível de produto, na rede inteira).
`admin` (Unooria)	`/admin/usuarios`	As MESMAS capacidades de `gestor_contrato`. O que distingue é de quem é a conta, não o que ela pode — e o único acréscimo real é a ponta da cadeia: é ele que atribui `gestor_contrato`.

ATENÇÃO

Os QUATRO papéis de gestão moram na mesma área do *middleware* — `areaDoPapel` devolve `/admin/usuarios` para os dois. A distinção entre eles **não** é feita ali: é feita item a item na navegação (`SO_ADMIN_DO_CONTRATO` em `components/nav-admin.tsx`), por `redirect` nas páginas que não admitem gestor (`/admin/escolas` e `/admin/regionais`) e, definitivamente, pelo `RolesGuard` da API. Um gestor que digite `/analise` vê a tela abrir e ficar **vazia**: a API responde 403, o BFF converte em `null` e não há o que renderizar. Nenhum dado atravessa — mas a tranca que segurou foi a segunda, não a primeira.

7.6.2 A cadeia de atribuição de papéis

Quem CONCEDE cada nível de acesso é uma pergunta diferente de quem pode chamar cada rota, e ela tem resposta própria desde a **ADR-80**:

$$\texttt{admin} \rightarrow \texttt{gestor_contrato} \rightarrow \texttt{gestor_regional} \rightarrow \texttt{gestor_escola} \rightarrow \texttt{professor},\ \texttt{estudante}$$

Cada nível atribui o de baixo, e o alcance real é o **fechamento transitivo** dessa tabela (`papeisQueCria`, em `usuarios/regras.ts`) — não uma lista escrita à mão por papel, que teria quatro cópias da mesma regra. **Para cima, nunca**: nem o próprio nível. Um papel que se autoatribui não tem quem o conceda, e é por isso que `gestor_contrato` deixou de criar outro `gestor_contrato` e `gestor_escola` deixou de criar outra coordenação de escola — as duas coisas valiam antes desta ADR.

A tranca é `podeCriarPapel`, e não o `@Roles`: o *guard* protege a ROTA, não o campo `papel` do corpo. A lista que a tela oferece sai da mesma cadeia (espelhada em `hub/lib/papeis.ts`) apenas para não oferecer o que a API recusaria — seletor que promete o que dá 403 faz a pessoa preencher o formulário inteiro para receber o erro no fim.

7.6.3 A matriz de capacidades

As duas tabelas a seguir são o **contrato de autorização** lido do código: cada linha existe porque há um `@Roles` que a sustenta. ✓ é "pode"; – é "não existe rota que faça isso para este papel" — a ausência é estrutural, não disciplinar.

Identidade, enturmamento e rede	Rota	Est.	Prof.	Gest.	Adm.
Criar pessoa — em lote, por IMPORTAÇÃO	`POST /rostering/importar`, `/rostering/oneroster/:arquivo`	–	–	✓	✓
Criar UMA pessoa — o formulário que delega ao importador	`POST /usuarios`	–	–	✓	✓
Listar as pessoas da escola	`GET /usuarios`	–	–	✓	✓
Editar pessoa (nome, CPF, e-mail, papel, `ativo`)	`PATCH /usuarios/:id`	–	–	✓	✓
Redefinir senha (temporária, mostrada uma vez)	`POST /usuarios/:id/reset-senha`	–	–	✓	✓
Diagnosticar contas sem matrícula da rede	`GET /rostering/contas-de-assento`	–	–	✓	✓
Criar e editar turma	`POST`, `PATCH /turmas`	–	–	✓	✓
Matricular, ajustar e encerrar matrícula	`/turmas/:id/matriculas`, `/matriculas/:id`	–	–	✓	✓
Ler as turmas em que atua	`GET /turmas/minhas`	–	✓	✓	✓
Abrir uma turma e ver quem está nela	`GET /turmas/:id`	–	✓*	✓	✓
Abrir o ano letivo	`POST /periodos-letivos`	–	–	✓	✓
Ler os períodos letivos	`GET /periodos-letivos`	–	✓	✓	✓
Abrir ou editar ESCOLA ou REGIONAL	`POST`, `PATCH /escolas`	–	–	–	✓
Excluir uma REGIONAL (escola não se exclui)	`DELETE /escolas/:id`	–	–	–	✓
Buscar professor/coordenação na REDE (sem escola de origem)	`GET /escolas/profissionais`	–	–	–	✓

Identidade, enturmamento e rede	Rota	Est.	Prof.	Gest.	Adm.
Atribuir papel SEM escola (`gestor_contrato`, `gestor_regional`)	`POST /usuarios/sem-escola`	–	–	–	✓**
Relatório da DIRETORIA (consolidado e por escola)	`GET /escolas/minhas/relatorio`	–	–	✓***	✓
Conceder lotações em lote (nível da rede)	`POST /escolas/lotacoes-em-lote`	–	–	–	✓
Lotar alguém numa escola, encerrar lotação	`POST /escolas/:id/lotacoes`, `DELETE /lotacoes/:id`	–	–	✓	✓
Transferir estudante entre escolas do contrato	`POST /estudantes/:id/transferir`	–	–	✓	✓
Ler o histórico de quem foi transferido	`GET /estudantes/:id/historico`	–	✓	✓	✓
Emitir, importar e remover perfil portátil	`/estudantes/:id/perfil-portatil`, `/perfis-recebidos`	–	–	✓	✓
Ver as escolas em que tem lotação	`GET /escolas/minhas`	✓	✓	✓	✓
Publicar ou atualizar experiência no catálogo	`PUT /experiencias/:expId`	–	–	–	✓
Ler o catálogo de experiências	`GET /experiencias`	✓	✓	✓	✓

* O professor só abre a turma **em que tem matrícula** (`professorLeciona`); gestor e admin enxergam a escola inteira. É a diferença entre "dá aula nesta turma" e "trabalha nesta escola" (**ADR-28**).

*** Só o `gestor_regional`, e só a PRÓPRIA regional: o `?regionalId=` dele é ignorado. O `gestor_escola` não alcança — para quem tem uma escola o relatório seria a própria escola com um título errado (**ADR-82**).

** A rota aceita os dois perfis de contrato, mas QUAL papel cada um pode atribuir sai da cadeia, não do `@Roles`: `gestor_contrato` atribui `gestor_regional`; `gestor_contrato` pedindo outro `gestor_contrato` recebe 403, porque esse é atribuído pela Unooria (**ADR-80**).

Vida pedagógica e leitura de dados	Rota	Est.	Prof.	Gest.	Adm.
Realizar experiência e salvar progresso	`/progresso/:expId`, `/runs/*`	✓	–	–	–
Preparar e conduzir atividade (exp08ef1b)	`/atividades/*`	–	✓	–	–
Destilar, gerar e regenerar o *feed* com IA	`/atividades/:id/destilar`, `/gerar`, `/posts/:id/regenerar`	–	✓	–	–

Vida pedagógica e leitura de dados	Rota	Est.	Prof.	Gest.	Adm.
Monitor da turma e análise da atividade (+CSV)	`GET /atividades/:id/monitor`, `/analise`	–	✓	–	–
Reiniciar o *run* de um estudante	`POST /runs/:id/reset`	–	✓	–	–
Ler o dossiê de um *run*	`GET /runs/:id/dossie`	✓†	✓	–	–
A sala: a grade da turma	`GET /salas/turma/:id`	–	✓‡	✓	✓
Liberar/encerrar missão na turma	`PUT /salas/turma/:id/missoes/:expId`	–	✓‡	–	–
As salas do estudante (a grade dele)	`GET /salas/minhas`	✓	–	–	–
Ler as equipes de uma missão na turma (ADR-111)	`GET /equipes/turma/:turmaId/missao/:expId`	–	✓‡	✓	✓
Propor equipes (sorteio, N vazias, a atividade anterior) — nunca grava	`POST .../sugestao`, `GET .../ultima`	–	✓‡	–	–
Salvar ou desfazer as equipes (sempre editáveis)	`PUT`, `DELETE /equipes/turma/:turmaId/missao/:expId`	–	✓‡	–	–
A própria equipe na missão (só primeiros nomes)	`GET /equipes/minha`	✓	–	–	–
O painel da evidência de uma atividade (ADR-112)	`GET /evidencias/turma/:turmaId/missao/:expId/atividade/:atividadeId`	–	✓‡	✓	✓
Confirmar para a turma a cópia da projeção	`POST .../rodadas/:rodada/confirmar`	–	✓‡	–	–
Ajustar a evidência (um ou vários, mesmo com a missão encerrada)	`PATCH .../rodadas/:rodada/ajustes`	–	✓‡	–	–
O perfil de evidência de um estudante (sem origem)	`GET /evidencias/estudante/:estudanteId`	–	✓‡	✓	✓

Vida pedagógica e leitura de dados	Rota	Est.	Prof.	Gest.	Adm.
A pergunta de múltipla escolha, sem o gabarito	`GET /evidencias/:expId/perguntas/:perguntaId`	✓	–	–	–
A resposta — que devolve só "enviada", nunca o resultado	`POST .../resposta`	✓	–	–	–
Vila do Lume: agregado da turma	`/vila/agregado/*`	–	✓	✓	–
Vila do Lume: a linha de uma criança	`/vila/professor/*`	–	✓‡	–	–
Jornal da Escola: a própria missão (ADR-117)	`/jornal-escola/*`	✓	–	–	–
Jornal da Escola: preparar, conduzir e ler o painel (pacote, imagens, os dois portões, zerar), e o material do guia	`/jornal-escola/turmas/*`, `/professor/material`	–	✓‡	✓	✓
unooGPT: a própria partida	`/unoogpt/*`	✓	–	–	–
unooGPT: turma e transcrição individual	`/unoogpt/professor/*`	–	✓‡	–	✓
unooGPT: agregado da escola (sem `estudanteId`)	`/unoogpt/agregado/escola`	–	–	✓	✓
Bem-estar: o próprio *check-in*, histórico, apagar	`/bemestar/*`	–	✓	–	–
Plano de aula: criar, versionar, restaurar	`/planos/*`	–	✓	–	–
Conversar com o agente de IA do professor — só as PRÓPRIAS conversas, na escola ativa (ADR-126)	`/agente-professor/*`	–	✓	–	–
Análise de produto, na rede (+CSV)	`/analise/*`	–	–	–	✓
Chamar a IA (com quota diária POR PERFIL)	`POST /ai/gerar`	✓	✓	✓	✓
Alterar o teto diário de IA de um perfil (só `admin`, ADR-105)	`/recursos/quotas-de`	–	–	–	–
Listar as experiências arquivadas (só `admin`, ADR-116)	`GET /recursos/arquivadas`	–	–	–	–
Enviar telemetria	`POST /telemetry/batch`	✓	✓	✓	✓

† O estudante lê o dossiê do **próprio** *run*; a posse é verificada no *service*, não pelo papel.

‡ "Sim" aqui é do PAPEL; o escopo é a **matrícula**. A turma, por `professorLeciona`; a criança, por `professorEnsinaOEstudante`. Vale também para o que APAGA ("zerar turma") e para o relógio da aula do unooGPT — as três coisas que uma prova de "mesma escola" deixava passar.

REGRA INVIOLÁVEL

As quatro ausências mais importantes desta tabela. *Bem-estar* não tem uma única rota de gestor nem de agregado — o instrumento é do professor, sobre o professor. No *unoOGPT*, o controller do gestor **não tem rota que aceite** `estudanteId`: não é que ele não deva ver a linha de uma criança, é que não existe endereço onde ela caiba. E `admin-contrato` não alcança a Vila nem o dossiê: ele lê a rede em agregado, nunca a sala. Na *evidência pedagógica*, o estudante **não tem rota que leia o próprio resultado** (ADR-112): as duas dele são a pergunta, sem gabarito, e a resposta, que devolve `{ enviada: true }` na primeira e na segunda vez.

DECISÃO DE PROJETO

Pessoa nasce por importação — e o cadastro unitário DELEGA a ela. Existe `POST /usuarios`, e ele não escreve em `users`: monta a MESMA linha da planilha e chama o MESMO importador (ADR-59, acionando o gatilho da ADR-54). Quem grava continua sendo `POST /rostering/importar` — ou `/rostering/oneroster/users`, para a rede que já exporta o padrão — e por isso as regras de CPF, série, turno, login gerado, lotação e matrícula são as mesmas nas duas telas.
A diferença é de SEMÂNTICA: a importação é idempotente pela matrícula da rede (e pelo CPF), então reimportar não duplica nem reseta a senha de quem já entrou; o formulário, que se chama "Novo usuário", **recusa com 409** quem já existe, em vez de atualizar em silêncio — e o 409 de um CPF achado em outra escola sai sem id, sem nome e sem dizer qual unidade.
A exceção são as contas sem escola (`gestor_contrato` e `gestor_regional`, ADR-80): elas não têm `escola_id`, lotação nem turma, não cabem numa planilha DA ESCOLA, e por isso têm rota própria, `POST /usuarios/sem-escola`. A cadeia de atribuição transitiva impede escalada de privilégio: `admin` cria `gestor_contrato`, e este cria `gestor_regional`. Gestor escolar recebe 403 ali — conceder o alcance da rede inteira ou de uma diretoria é decisão de governança do contrato, nunca de uma unidade isolada.

7.6.4 As portas que não são papel

Cinco grupos de rotas não passam pelo `RolesGuard` porque não há papel para checar — e cada uma prova outra coisa no lugar. Elas são a superfície que um leitor da matriz acima poderia concluir, erradamente, que não existe.

Rota	Prova	Por quê
`POST /auth/login`, `POST /auth/trocar-senha`	A credencial	Quem chega ainda não tem *token*. Trocar a senha provisória exige a ATUAL — mesmo nível de prova do login. 8 tentativas por minuto por usuário.
`GET /healthz`, `/readyz`	Nada	*Probe* do Cloud Run. Não tocam dado de ninguém.

`POST /telemetry/beacon`	O *token* no CORPO	`navigator.sendBeacon` não manda cabeçalho. Responde 204 sempre: a página já foi descartada e ninguém leria o erro.
`POST /telemetry/ingest`	`PubSubPushGuard`	O *push* do Pub/Sub, autenticado pelo OIDC da conta de serviço — não por pessoa.
`POST /observabilidade/publico/*`	**Nenhuma**	A única rota sem guard nenhum (**ADR-84**). Existe porque a tela de *login* não tem sessão, e o 401 do BFF descartava justamente o sinal de que ninguém estava conseguindo entrar. A contenção é estrutural, não de identidade: Zod `.strict()` com corpo pequeno, `expId` forçado a `hub`, *redaction*, cota por IP mais teto global de instância, e nenhuma linha de log por requisição recusada. Não lê nem escreve dado de ninguém — só emite log e conta métrica.
`POST /perfis/importar`, `/planos/referencias`	Chave de serviço (`x-perfil-token`, `x-import-token`)	Máquina falando com máquina no *provisionamento*. Nenhuma delas é alcançável a partir do navegador de um usuário.

7.6.5 O desenho por trás

DECISÃO DE PROJETO

Agregado e linha individual são *controllers* diferentes. Em `src/vila/` há `VilaProfessorController` com dois `@Controller` distintos: `/vila/agregado/*` abre para `professor` e `gestor`; `/vila/professor/*`, que devolve a linha de uma criança, só para `professor`. A separação física é o que garante que a próxima rota adicionada ao arquivo herde a permissão *certa*.

DECISÃO DE PROJETO

Dentro da escola, o escopo do professor é a MATRÍCULA. Até a migração do enturmamento não havia vínculo professor↔turma no modelo: os seletores listavam todas as turmas da escola e a única prova exigida para abrir os resultados de uma delas era "mesma escola do *token*" — que é a regra de *tenancy*, não a de quem dá aula ali. O professor do 8A abria bem-estar e Vila do 9B.

Agora `matriculas` responde por isso: `ondeAsTurmasSaoVisiveis` filtra as listas e `professorLeciona` vira 403 nas telas de uma turma só. Gestor e admin-contrato seguem enxergando a escola inteira, de propósito — é a mesma distinção que o OneRoster faz entre `roles`, no nível da organização, e `enrollments`, no nível da turma (**ADR-28**).

A camada de cima é a **sala de aula** (**ADR-69**): quem tem matrícula de professor na turma libera a missão nela, e é essa liberação que abre a porta para o estudante. Gestor lê a sala e não decide a aula — a mesma linha que separa administrar a escola de dar a aula.

Seis rotas ficaram de fora dessa porta, e foram fechadas depois: o "zerar turma" da Vila e do unooGPT, os cartões da Vila, os dois relatórios individuais e — a mais grave — o **relógio da aula** do unooGPT, em que um professor da escola dava a largada ou encerrava a aula de uma turma que não é dele, no meio dela. As duas telas individuais carregam leitura de perfil e transcrição de texto livre de menor de idade; para elas a pergunta não é "esta turma é minha?", e sim "esta criança é de alguma turma minha?" — `professorEnsinaOEstudante`, que aceita a matrícula ativa *ou* o cache `users.turma_id`. As duas fontes porque as listagens destes módulos partem do cache: olhando só a matrícula, um cache adiantado faria o nome aparecer na folha e a tela dele responder "não encontrado".

DECISÃO DE PROJETO

Numa REDE, a pessoa atravessa escolas; o *token* não. O professor lotado em duas unidades escolhe em qual está trabalhando, e o *token* leva **essa** — então a política de RLS e as tabelas com `escola_id` continuam exatamente como estavam (**ADR-30**). A troca de escola ativa é validada contra as lotações *pela API*, nunca contra a lista que o *cookie* carrega: *cookie* é dado do cliente.

E a fronteira precisa se mover na TELA junto com o *token* (**ADR-85**). Trocar de escola reemite o *cookie*, mas uma tela já aberta continua com o que buscou antes — e uma experiência embutida continua com o *token* anterior em memória, porque ele viaja uma vez só, no *handshake*. Aí não é leitura desatualizada: é escrita na escola de onde a pessoa acabou de sair, com um *token* legítimo que a RLS aceita sem reclamar. Por isso o layout de cada área do hub remonta o miolo quando a escola ativa muda.

Acima da escola existe um nível — a **regional** (Diretoria de Ensino, polo, secretaria municipal) —, que **agrupa unidades sem criar um tenant unificado**. Nenhuma política de RLS foi flexibilizada para acomodar regionais (**ADR-79**, **ADR-80**, **ADR-81**): o `gestor_regional` opera uma escola ativa por vez, selecionada entre as escolas cujo `escolas.pai_id` coincide com seu `users.regional_id`. Quando a diretoria precisa de relatório consolidado (**ADR-82**), a API itera sobre as escolas autorizadas em lotes de oito, armando o `set_config` isolado para cada unidade, sem quebrar o isolamento de dados no banco.

E na transferência de um estudante, o dado **fica onde foi produzido**. O que atravessa é uma leitura única e estreita (`GET /estudantes/:id/historico`), restrita a quem tem lotação na escola onde a criança está hoje, devolvendo participações e nunca o conteúdo dos instrumentos (**ADR-32**).

DECISÃO DE PROJETO

Sair da REDE é outra coisa, e sai como DOCUMENTO. Transferência entre escolas do contrato é lotação. Já a criança que vai para outra rede sai da instalação — outro banco, nenhum *JOIN* —, e não há garantia de que o destino seja cliente. Por isso a saída é um arquivo assinado pelo hub (a mesma chave dos *tokens*, com `aud` próprio), emitido a pedido do titular (LGPD art. 18, V), e o que chega de fora vive numa tabela só dele: nunca vira

> matrícula, progresso nem perfil, e nunca entra em média. Assinatura inválida é RECUSA; emissor inalcançável é um rótulo visível na tela (**ADR-35**).

> **DECISÃO DE PROJETO**
>
> **A senha do primeiro acesso é travada em três lugares.** Enquanto `senha_provisoria` for verdadeira, o *middleware* prende as páginas em `/trocar-senha`, o BFF recusa dados e `/api/exp-token` não emite *token* de experiência. Só com o *middleware*, bastaria digitar `/exp/...` para pular a troca — a experiência abriria por baixo, com o *token* que o hub continuaria assinando (**ADR-29**).

7.7 Segurança do *embed*

A fronteira mais exposta da plataforma não é a API: é o `iframe`. Dentro dele roda código que, em produção, vem de outra *origin* (a CDN) e que recebe um *token* do usuário.

Medida	Implementação e por quê
`targetOrigin` explícito	Nunca `"*"`, nos dois sentidos. O hub usa a *origin* derivada da *mesma* URL do `iframe`; a SDK usa o `hubOrigin` recebido por *query string*. `"*"` entregaria o *token* a qualquer documento que conseguisse virar o pai do *frame* (Mozilla Developer Network, 2026b).
`hubOrigin` normalizado	A *query string* é entrada NÃO CONFIÁVEL — o *bundle* é público, e quem embedar a experiência escolhe aquele parâmetro. `normalizarHubOrigin()` o reduz a uma origem canônica, recusa esquema fora de http(s), credencial e origem opaca, exige hub em https quando a experiência está em https, e — a trava que fecha o caminho — confere contra `location.ancestorOrigins[0]`, a origem REAL de quem embedou, que vence a *query string* (**ADR-46**).
Validação de `ev.origin`	Os dois lados descartam mensagem cuja *origin* não bate, *antes* de olhar o conteúdo.
Validação por esquema	Toda mensagem pós-*init* passa por Zod nos dois lados; o que não casa é descartado com aviso, **nunca** despachado cru aos *listeners*.
Verificação de versão maior	Hub e experiência comparam `protocolVersion`. Incompatível falha alto: sem `hub:init`, o *handshake* expira e a experiência mostra erro.
Token só em memória	A experiência nunca escreve o *token* em `localStorage` nem em *cookie*. TTL de 15 minutos, renovado por mensagem.
Sandbox mínimo	`allow-scripts allow-same-origin allow-modals`, mais `allow-pointer-lock`, `allow-forms` e `allow-downloads` **apenas** para quem declara `pointer-lock`, `forms` ou `downloads` no manifesto (Mozilla Developer Network, 2026a).

`allow` do *frame*	`autoplay; fullscreen; clipboard-write`. A *Permissions-Policy* tem *default* `'self'`: sem delegar `autoplay`, um `<video muted autoplay>` cross-origin rejeita `play()` em silêncio.

NOTA

Cada token do *sandbox* está lá por um motivo medido: `allow-same-origin` porque, sem ele, a *origin* do `iframe` cross-origin vira `"null"` e a validação de *origin* quebra dos dois lados; `allow-modals` porque, sem ele, o navegador *ignora* `confirm()` do `iframe` silenciosamente (devolve `false`), e a app-90 confirma exclusão assim; `allow-pointer-lock` porque, num `iframe` com *sandbox*, `requestPointerLock()` morre com `SecurityError` sem ele — verificado em `scripts/probe-pointerlock.mjs`, que precisa do Chrome do sistema porque o Chromium do Playwright não concede *pointer lock* nem no topo da página. E `allow-downloads` porque, sem ele, o navegador **bloqueia todo *download*** iniciado de dentro do `iframe`: o professor clicava em "Baixar .doc" nas ferramentas de planejamento e **nada acontecia**, sem erro na tela e sem linha no *console* — o Chrome registra esse bloqueio no painel *Issues*, e o `a.click()` do *bundle* volta como se tivesse funcionado (**ADR-76**; MEDIDO com dois `iframe` iguais, um com o token e outro sem).

ATENÇÃO

A permissão do manifesto só vale se a rota do hub a repassar. As três páginas que embedam ferramenta de professor (`/professor/plano-de-aula`, `/plano-de-ensino`, `/bem-estar`) renderizam o `ExpFrame` direto e **não passavam `permissoes`** — só `/exp/[expId]` e `/dev-embed` passavam. O efeito é silencioso: o *frame* monta o *sandbox* sem o token e a capacidade declarada simplesmente não existe naquela tela.

DECISÃO DE PROJETO

Permissão de `iframe` é por experiência, não *default* do *frame*. Prender o ponteiro do mouse de um estudante é capacidade de quem a declarou — e concedê-la a todas as experiências significaria que qualquer *bundle* publicado pode capturar o mouse. O mesmo vale para `allow-downloads`: entregar arquivo ao computador de quem está usando é capacidade de quem declarou, e só as duas ferramentas de professor exportam documento. Já `fullscreen` *não* entra no manifesto: o `allow` do *frame* já o delega a todas, e campo de manifesto que não muda nada é pior que campo nenhum, porque quem lê acredita ter declarado algo.

7.8 Segredos e o que nunca vai ao cliente

REGRA INVIOLÁVEL

Nenhum segredo em *bundle* de cliente. Sem chave de API, sem *prompt* de sistema, sem URL interna. Consequências concretas:

- Chamada de IA só existe como `POST /ai/gerar` na API. O *prompt* de sistema mora em `apps/api/src/ai/prompts.ts` e nunca é devolvido.
- O gabarito do UnooTube arquivado (`factual`, hoje em `backup-unootube`) não era persistido no estado do estudante nem devolvido em nenhuma rota de estudante.
- O gabarito do Jornal da Escola (se cada notícia é publicável e o pilar que falta), a consequência e o conjunto certo da Chave moram no pacote, no servidor. A notícia do estudante é uma *allowlist*, e cada parte desce num portão: a pista quando ela a abre, a reação com o resultado liberado, o pilar da vaga depois da Chave — e o conjunto da Chave, nunca (ADR-117).
- Os textos dos cartões da app-70 chegam ao jogo **sem gabarito**, e apenas os da bolha daquela criança.
- `lado`, `perfil` e `intensidade` dos posts da exp08ef1b são removidos pelo sanitizador antes de o *feed* sair da API — são a variável independente do experimento.

7.9 Privacidade e dados de menores

ATENÇÃO

O CPF é guardado, e o que o protege é a CONTENÇÃO. Ele é a identidade imutável na rede — o que torna possível reconhecer a mesma criança numa transferência — e o gestor precisa vê-lo, para conferir identidade contra o sistema da rede e atender a família no balcão (ADR-33). CPF é dado pessoal comum, não do art. 11.
O que o cerca: `users.cpf` é o **único** lugar do banco onde o número existe; ele nunca vai a *log* (nem em mensagem de erro), nunca a telemetria, JSONB, *claim* de *token* ou *bundle* de experiência; e só gestor e admin-contrato alcançam a rota que o devolve. O teste estrutural cobra os dois lados — que só exista essa coluna, e que nenhum JSONB de telemetria ou progresso carregue onze dígitos seguidos, que é o caminho mais fácil de ele escapar.

Prática	Como é garantida
Telemetria sem PII	O evento carrega `estudante_id` opaco (UUID), nunca nome ou *e-mail*. A identidade é derivada do *token* no servidor, não do corpo enviado pelo cliente — o que é, ao mesmo tempo, anti-*spoof* e garantia de que o cliente não *pode* enviar PII no lugar do id.

Sem canal de texto arbitrário	Em `app_70_sessoes.payload`, o campo `eventos[].dados` aceita apenas número, booleano ou *string* de um **vocabulário fechado**. O campo `texto` livre que existia era o único canal de texto arbitrário do *payload* — e o caminho por onde o nome de uma criança chegaria a um JSONB de longa duração. É trava estrutural, não lista de proibidos.
Texto sensível fora de log	O texto livre do *check-in* de bem-estar nunca vai a log nem à telemetria; só contagens.
IA sem contexto identificável	O *prompt* de bem-estar recebe apenas dia da semana, mês, número do *check-in* e o texto livre *delimitado como conteúdo*. Nunca nome, escola, pontuação ou faixa.
Ranking sem menor real	O ranking do UnooTube (hoje `backup-exp04ef1b`) usava colegas **fictícios**, com distribuição desenhada para a lição. Não há pontuação de criança real exposta a outra criança — nem no Jornal da Escola, em que o painel só mostra as verificadas num histograma anônimo e a criança não vê a edição de ninguém.
Dado de saúde com posse	`app_91_ciclos` e `app_91_checkins` têm RLS *e* filtro por `professor_id` em toda *query*, e não existe rota de gestor nem agregado. O próprio titular apaga tudo.
Texto de menor com contenção	`exp06ef2b_reflexoes` guarda o "Meu percurso": 280 caracteres que **a professora da turma lê**, porque ali o texto *é* o instrumento. `exp08ef4b_respostas_percurso` guarda a justificativa da cláusula: 800 caracteres, mesma finalidade e mesma fronteira — e com uma trava que o outro não tem, o **filtro de dado pessoal que recusa antes de gravar**. Os dois ficam sob RLS, fora de log e fora da telemetria. Ver as decisões logo abaixo. Na A Seleção (**ADR-115**) a **legenda** dos Stories vai, com o primeiro nome, ao colega pareado no Mural — por isso passa por um sanitizador (link, telefone, e-mail, menção, palavrão) antes de gravar —, e a `exp07ef1b_reflexoes` só ao professor da turma. O Jornal da Escola (**ADR-117**) é o oposto, e de propósito: **não tem texto livre de menor**. Toda resposta é vocabulário fechado (notícia, tipo de pista, bloco da Chave, motivo, alternativa), e por isso não há recorte de gestor a aplicar — o gestor lê o mesmo painel que o professor.

> **DECISÃO DE PROJETO**
>
> **O texto livre de menor foi reaberto pelo Eco no Grupo, e a mudança de postura sobreviveu à missão.** A **ADR-66**, que a registrou, ficou superada com a retirada (**ADR-113**). Até o Eco, o produto tinha exatamente um campo aberto, e ele era de *professor* (o *check-in* de bem-estar); as experiências de estudante trabalhavam com vocabulário fechado, e a linha acima sobre `app_70_sessoes` descreve a trava estrutural que existe justamente por isso. A conversa que abriu o precedente saiu com a missão — a tabela dela caiu na migração `20260925180000_remover_eco_no_grupo` —, e o que resta são os campos das missões seguintes, cada um com a contenção da tabela acima.

DECISÃO DE PROJETO

A exp06ef2b leva o texto da criança ao painel do professor, e isso está certo. O que decide é a **finalidade**, não o grau de cuidado: o Eco no Grupo mantinha fora do painel uma *conversa* com personagem, porque levá-la à tela do adulto mudaria o que a criança escreve.

Na exp06ef2b o campo é o *registro do percurso*: 280 caracteres em que a criança nomeia o gatilho que reconheceu e o que fará antes de encaminhar da próxima vez. Ali o texto **é** o instrumento formativo — um painel que mostrasse só "entregou / não entregou" guardaria a redação para ninguém. Quem lê é o professor *da turma dela*: nenhuma rota de gestor o alcança, e o campo não existe em nenhum *payload* de telemetria.

Consequência aceita: a coluna é de longa duração e visível a um adulto por desenho. Gatilho de reavaliação: se um painel de gestor, de rede ou de análise pedir `exp06ef2b_reflexoes.texto`, a decisão volta à mesa — a fronteira é o professor da turma, e ela não se move por conveniência de relatório.

DECISÃO DE PROJETO

A exp08ef4b RECUSA o dado pessoal em vez de guardá-lo com *flag*, e essa é a primeira vez que o produto trata a caixa livre de menor de forma ativa, não só contida.

A questão estava aberta no insumo: quando a criança escreve um telefone ou um CPF na justificativa da cláusula, o servidor deve responder **422 com dica corretiva** e não gravar, ou **ofuscar os termos e salvar** com marca de moderação para o professor? As duas alternativas foram consideradas e descartadas. Gravar com *flag* deixaria o telefone da criança dentro da coluna, à espera de alguém que decidisse o que fazer com ele — e a regra de dados de menores prefere não guardar: o que não foi gravado não vaza em *backup*, em *dump* de diagnóstico nem em *log*. Ofuscar-e-salvar devolveria ao professor um parágrafo mutilado, com a argumentação cortada no meio, sem que ninguém soubesse o que estava ali.

Recusar também ensina, e é o que fecha a escolha: a dica diz o que tirar no exato momento em que a criança ia entregar o dado, dentro de uma missão cujo tema é privacidade. É a única superfície do produto em que a regra vira experiência vivida.

O filtro é conservador de propósito, e não adivinha nome próprio. Ele casa padrões estruturados — telefone, CPF, e-mail, CEP, cartão, `@perfil` — e nada mais. Um filtro de sobrenome recusaria "meu tio Silva foi reprovado por um sistema desses", que é exatamente o tipo de argumento situado que a etapa mede, e a criança não teria como saber o que fazer com a mensagem de erro. Nome próprio é risco **aceito e registrado**: o texto é lido só pelo professor da turma dela, nunca vai a *log* e nunca ao canal analítico. E nenhum padrão casa número de quatro dígitos solto — o falso positivo mais provável é o ANO ("desde 2019 a escola usa esse sistema").

O que vai ao *log* quando a recusa acontece: o **nome** do padrão que casou (`padroes: ["telefone"]`) e o **tamanho** do texto, nunca o trecho. Procurar o conteúdo no log seria procurar o que a regra existe para não guardar.

Consequência aceita: a criança pode receber a recusa sem entender de imediato qual parte do texto a causou, porque a mensagem não repete o trecho. Gatilho de reavaliação: se o filtro produzir falso positivo em uso real com frequência que atrapalhe a aula, o caminho é estreitar o padrão específico — não passar a gravar.

7.9.1 Retenção: uma decisão explícita de não declarar prazo

DECISÃO DE PROJETO

Não há prazo de retenção declarado para `app_70_sessoes` nem para `telemetria_eventos` (decisão de 2026-08-26, **ADR-13**).
Havia uma coluna `expires_at` de 90 dias no *schema* e *nenhuma rotina que a cumprisse*. Coluna de prazo que ninguém lê é pior que prazo nenhum, porque quem lê o *schema* acredita nela — é a mesma armadilha de um campo `consentimento_em` que existia no aplicativo de origem: a regra saiu do código, a coluna ficou, e todo mundo continuou acreditando. A coluna foi removida na migração `20260826170000_vila_sem_prazo`.
O que se ganha em troca é o **reprocessamento**, que é requisito real deste instrumento: `payload` é a única fonte para recomputar o perfil quando a versão do mapeamento subir, e o mapeamento é hipótese de *design* que vai mudar. 90 dias matavam exatamente a comparação entre duas gerações do mapeamento.
Se um contrato exigir prazo, ele volta **junto com a rotina que o cumpre** — nunca só a coluna. Com a tabela particionada por mês, expurgo passou a ser `DROP TABLE particoes.<mês>` em vez de um `DELETE` que trava a tabela.

LIMITAÇÃO CONHECIDA

A pergunta "qual é o prazo de retenção do dado comportamental de menor" está **aberta** e é decisão de produto e de contrato, não de módulo: ela vale para a plataforma inteira, porque `telemetria_eventos` guarda evento por `estudante_id` de *todas* as experiências. Enquanto não for respondida, o que existe é esta seção — que é melhor que uma coluna mentindo.

7.9.2 Localidade de processamento

ATENÇÃO

O modelo de IA em uso (`gemini-3.6-flash`) só é publicado no endpoint `global`: medido no projeto do contrato, ele responde 404 em `southamerica-east1`, `us-east1` e `us-central1`. A consequência é explícita e foi aceita: **o tráfego de IA sai de São Paulo, inclusive o do estudante**, e isso precisa constar do contrato e do DPA da rede. Voltar à região é possível a qualquer momento, mas exige trocar *também* o modelo pelo que a

> região publica (lá, a família 2.5) — as duas variáveis mudam juntas ou a geração para de funcionar.
> O agente de IA do professor segue a mesma regra por outro caminho: o modelo dele está em `global` (com reserva em `us`), os dois *data stores* em `global` e a sessão — o histórico das conversas — no Agent Engine em `us-east1`. Nenhum dos três fica no Brasil.

7.10 Verificação

Segurança não documentada por teste é intenção. O que é efetivamente verificado:

Teste	O que afirma
`tests/tenancy.spec.ts`	Com uma *role* sem *ownership* e a política ativa, uma *query* da escola A não retorna linha da escola B — nem quando o filtro da aplicação é omitido.
`tests/planos-tenancy.s`	Um professor não lê nem edita plano de outro professor, mesmo na mesma escola (RLS + posse).
`tests/telemetria-push.spec.ts`	`POST /telemetry/ingest` rejeita *token* sem a `aud` correta, sem a SA correta, e **rejeita tudo** quando as variáveis estão ausentes.
`packages/sdk/src/bridg`	Mensagem de *origin* errada é ignorada; mensagem que não valida é descartada; versão maior divergente rejeita a conexão.
`tests/motor-concorrencia.spec.ts`	Dois pedidos simultâneos de bloco não descartam um bloco (*lock* consultivo).
`tests/vila-regras.spec`	*Payload* que traga chave de perfil é recusado; bolha B não é servida nos anos iniciais; `zero` e `none` não se confundem.
`scripts/probe-pointerlo`	Sem o token de *sandbox*, `requestPointerLock()` falha; com ele, funciona.

7.11 Ameaças consideradas

Análise no espírito do OWASP Top 10 (OWASP Foundation, 2021), restrita ao que é específico desta plataforma.

Ameaça	Categoria	Mitigação
Estudante lê dado de outra escola	*Broken access control*	RLS forçada + filtro explícito + *claims* do *token*. Duas trancas independentes, uma delas no banco.
Estudante lê o gabarito	Exposição de dado sensível ao negócio	Correção *server-side*; gabarito não persistido no estado do estudante nem devolvido; sanitizador remove metadados de curadoria.
Cliente adulterado envia perfil pronto	Integridade de medida	Servidor *recalcula* e recusa *payload* que traga chave de perfil; cap de eventos por partida.

Injeção de telemetria falsa	Autenticação ausente em *endpoint* de serviço	`PubSubPushGuard` verifica emissor, audiência e SA; *fail closed* sem configuração.
Roubo de *token* do `iframe`	Exposição de credencial	TTL de 15 min, escopo mínimo, só em memória, `targetOrigin` explícito, HTTPS. E a correção é *server-side* de qualquer forma, então o *token* não compra nota.
Esgotamento de quota de IA por abuso	Consumo de recurso	Quota diária persistida por (escola, usuário, dia), com incremento atômico, verificada *antes* da chamada; *rate limit* por usuário.
Vazamento de PII por log	Privacidade	Proibição explícita; telemetria com id opaco; vocabulário fechado no *payload*; texto sensível nunca logado.
Chave de serviço vazada	Gestão de segredo	Não existe chave de longa duração: WIF no CI, ADC na aplicação.
Rota nova esquecendo a *tenancy*	Erro de programação	`comTenancy` *lança* sem escola; não há caminho alternativo ao banco.

LIMITAÇÃO CONHECIDA

Não foi feita revisão de segurança externa nem teste de intrusão. Para uma plataforma que trata dado de menor numa rede pública, isso é item de contratação — e está registrado como risco `R-06` (Capítulo 19).

8

Modelo de dados

Um banco, um *schema* lógico, 77 tabelas em `public` e um *schema* auxiliar (`particoes`) fora do alcance do ORM. O número é uma contagem, e envelhece a cada migração: são os `model` do *schema* do Prisma (`apps/api/prisma/schema.prisma`) em 2026-09-27, todos com `@@map`, sem a `_prisma_migrations` do próprio Prisma e sem nenhuma *view*, que o banco não tem. Este capítulo explica a convenção de nomes, o modelo por área e as decisões de esquema que não são óbvias — que, neste projeto, são a maioria.

O dicionário de dados campo a campo está no Apêndice E.

8.1 A convenção de nomes

REGRA INVIOLÁVEL

Tabela de experiência leva o CÓDIGO dela (ADR-110): `exp08ef1b_` para missão, `app_70_` para o resto. Assim: `app_70_sessoes`, `exp02em1b_sessoes`, `exp08ef4b_sessoes`, `app_91_checkins`. A tabela de uma experiência **arquivada** ganha `backup_` antes do código (**ADR-116**): `backup_exp04ef1b_sessoes` é a do UnooTube, e as `exp04ef1b_*` são do Jornal da Escola, que tomou o código (**ADR-117**) — as duas famílias convivem sem colidir.

Tabela da plataforma — que serve a todas — fica sem prefixo: `escolas`, `turmas`, `users`, `experiencias`, `experiencias_versoes`, `experiencias_arquivadas`, `disponibilidades`, `ia_quotas`, `recursos_auditoria`, `progresso`, `telemetria_eventos`, `ia_uso_diario`.

Tabela que é contrato entre duas experiências carrega os dois números: `exp08ef1b_app_70_perfis_importados` — a app-70 escreve, a exp08ef1b lê. Chamá-la de uma só mentiria sobre quem depende dela.

DECISÃO DE PROJETO

Um banco, um *schema* — nem banco por experiência, nem *schema* por experiência. Banco por experiência não funciona porque *as experiências não são isoladas*: o perfil de valores medido na Vila do Lume alimenta a largada do Instanooria, e o PostgreSQL não faz `JOIN` entre bancos. Além disso, um *pool* do Prisma por experiência estouraria `max_connections` com o *autoscaling* do Cloud Run.
Schema por experiência não funciona porque a única coisa que um *schema* daria além do prefixo é fronteira de `GRANT` — e existe **uma** *role* de aplicação. Seriam 48 *schemas* para obter exatamente o que um prefixo de nome já dá.

8.1.1 A migração de referência, e duas armadilhas registradas

A renomeação para o prefixo por experiência está em `20260827090000_tabelas_por_experiencia`; a troca do número pelo código (**ADR-110**) está em `20260923120000_codigo_das_experiencias`, que resolve as mesmas duas armadilhas abaixo pelo OID do objeto em vez do nome. A primeira é a migração de referência para qualquer renomeação futura: `RENAME`, nunca `DROP`/`CREATE`, e renomeando *constraints*, índices e políticas de RLS junto.

ATENÇÃO

Duas armadilhas registradas naquela migração, ambas produzindo o mesmo sintoma — nomes duplicados como `exp08ef1b_exp08ef1b_..._pkey`:

1. `FOR ... IN SELECT` sobre o catálogo do sistema **relê o que o próprio laço alterou**. A lista de tabelas a renomear tem de ser materializada antes do laço.
2. `RENAME CONSTRAINT` renomeia **também** o índice que a sustenta. Renomear os dois em sequência aplica o prefixo duas vezes.

8.2 Modelo da plataforma

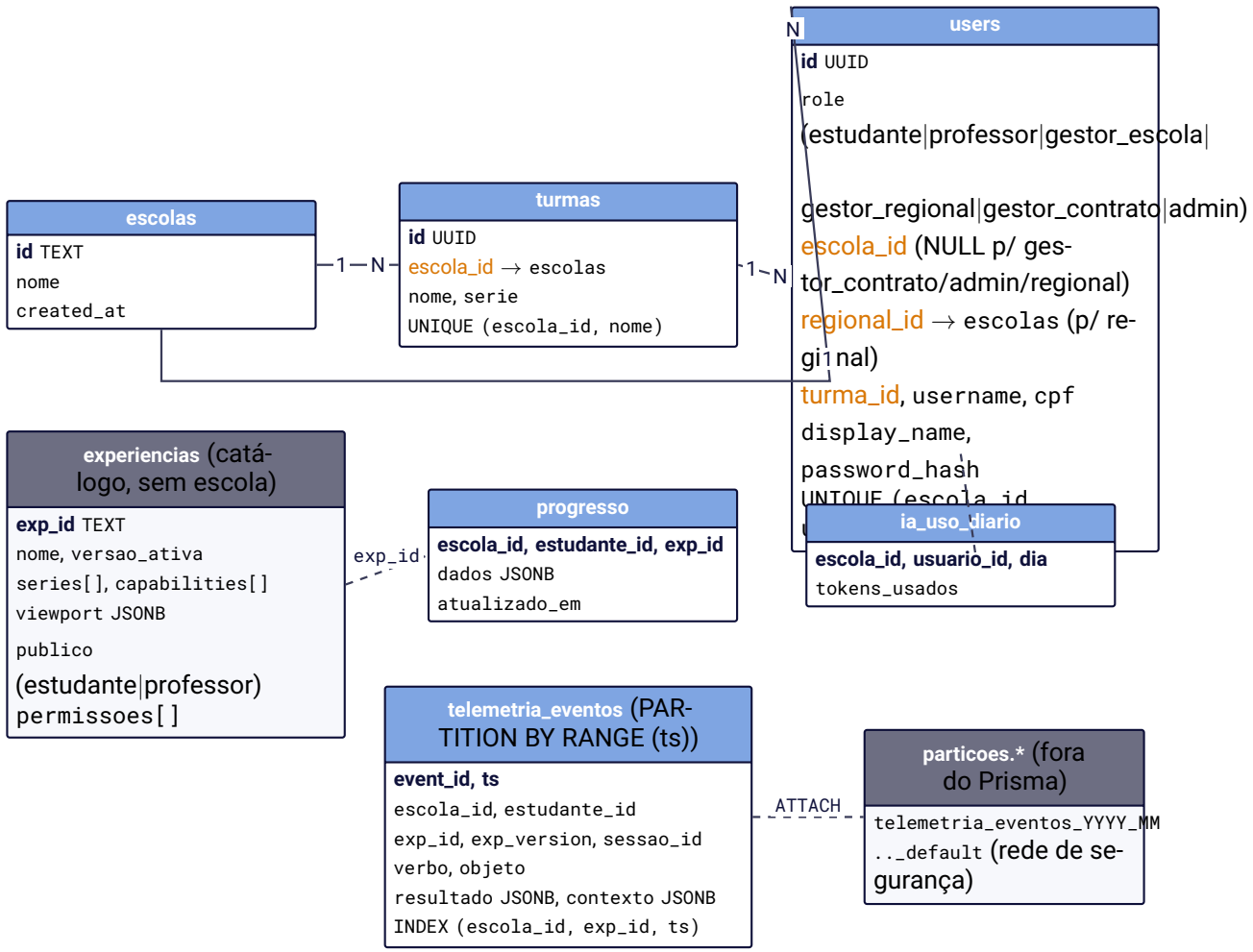


Figura 8.1 – Modelo de dados da plataforma. As caixas em azul são tabelas de *tenant* (têm `escola_id` e RLS); as cinzas são **catálogo do contrato** — `experiencias` e as partições de telemetria não pertencem a uma escola e ficam fora da RLS de propósito.

Tabela	Regime	Nota de esquema
`escolas`	*Tenant* raiz	O `id` é **texto**, e é o *mesmo* valor que circula no *claim* `escola_id` do JWT e no `set_config` da RLS. Um id legível (`sp-001`) vale mais que um UUID quando o valor aparece em três lugares diferentes de um diagnóstico. A tabela guarda TRÊS níveis (`tipo`: `rede` → `regional` → `escola`), mas só a `escola` é *tenant*: regional e rede não têm turma, estudante nem lotação.
`turmas`	*Tenant*	`UNIQUE (escola_id, periodo_letivo_id, nome)`: duas escolas podem ter “8A”, e a mesma escola tem o “8A” de 2026 e o de 2027 — são turmas **diferentes**, e é essa diferença que preserva o histórico na virada do ano.

`users`	*Tenant*	`username` é único **por escola**, não globalmente — uma rede tem muitos `joao.silva`. *CHECK*: `escola_id NOT NULL` exceto quando `role` é `'gestor_contrato'`, `'admin'` ou `'gestor_regional'` (**ADR-80**). O vínculo funcional de quem atua na escola vive em `lotacoes`; `users.escola_id` é apenas o *cache* da escola de primeiro acesso e nunca o critério de listagem.
`periodos_letivos`	Catálogo	Ano letivo e seus recortes. Sem `escola_id` e sem RLS: o calendário é do contrato, e duplicá-lo por escola só criaria cem linhas divergentes.
`lotacoes`	Identidade	O vínculo PESSOA × ESCOLA. Professor **acumula** escolas; estudante tem uma ativa (índice único parcial no banco). Fora da RLS pelo mesmo motivo de `users`: é lida no *login*, antes de se saber a escola (**ADR-30**).
`matriculas`	*Tenant*	O enturmamento. Ver Seção 8.6.
`experiencias`	Catálogo	Sem `escola_id`: o manifesto é do contrato, não da escola. Escrita só por `admin-contrato`.
`experiencias_versoes`	Catálogo	Uma linha por versão publicada, com o MANIFESTO INTEIRO — é a lista do *rollback* pela tela, e é o que faz voltar a versão voltar também `viewport` e `permissoes` (**ADR-87**).
`experiencias_arquiv`	Catálogo	A experiência **arquivada** (**ADR-116**): a linha SAI de `experiencias` e vem para cá, com a fotografia do arquivamento (turmas que tinham liberado, estudantes com progresso). Tabela à parte, e não coluna, para nenhum dos leitores de `experiencias` precisar lembrar de filtrar. *CHECK*: o id é `backup-` mais o código de origem.
`recursos_auditoria`	Catálogo	**Append-only** (**ADR-89**): quem ligou, desligou ou trocou versão, quando e por quê. Existe porque a linha de `disponibilidades` é SOBRESCRITA. `UPDATE` recusado por *trigger*; `autor_nome` congelado na linha, sem FK para `users`.
`disponibilidades`	Catálogo	**O interruptor da rede** (**ADR-87**): uma linha responde "este recurso existe neste escopo?". `escola_id` NULO = o contrato inteiro, e preenchido pode ser uma escola OU uma regional. Fora da RLS pela mesma razão de `experiencias`, mais uma própria: a política por `escola_id` esconderia justamente a linha de escopo-contrato.
`progresso`	*Tenant*	PK composta `(escola_id, estudante_id, exp_id)`. É o progresso *genérico* da SDK; estado rico de experiência vai para a tabela da experiência.
`ia_uso_diario`	*Tenant*	Contador de quota, uma linha por (escola, usuário, dia). Incremento atômico por `ON CONFLICT DO UPDATE ... RETURNING`.

`ia_quotas`	Catálogo	**O teto diário de IA por perfil** (**ADR-105**): uma linha por perfil (`estudante`, `professor`), escopo do contrato. Semeada pela *migration* com o que valia por *env*; sobrescrita pelo `admin`, com o histórico em `recursos_auditoria`. Fora da RLS pela razão de `disponibilidades`.
`telemetria_eventos`	*Tenant*, particionada	Ver Seção 8.7.
`perfis_portateis`	*Tenant*	O documento assinado que veio de OUTRA instalação (**ADR-35**). Nunca se mistura com o dado local: não vira matrícula, progresso nem perfil, e não entra em média. É a única exceção à contenção do CPF — o número é o conteúdo do documento, e alterá-lo destruiria a assinatura.
`equipes_montagens`	*Tenant*	A decisão da aula por turma × missão × ATIVIDADE (**ADR-111**, **ADR-130**): `modo` (`sorteio`, `manual` ou `individual`), `computadores` (nulo só no `individual`), `ausentes` e a `versao` da concorrência otimista. PK composta `(escola_id, turma_id, exp_id, atividade_id)`. Ver Seção 8.6.1.
`equipes`	*Tenant*	Um computador da montagem: `nome` (1 a 30 caracteres depois do *trim*) e `ordem`. Nome único dentro da atividade — duas "Equipe 3" no mesmo painel são a dúvida que ninguém resolve no meio da aula.
`equipes_membros`	*Tenant*	Quem senta diante de cada computador, com `turma_id`, `exp_id` e `atividade_id` repetidos da equipe para o `UNIQUE` de **uma equipe por criança por atividade**. A única das três com id de criança.
`evidencias_marcas`	*Tenant*	O DADO BRUTO da evidência pedagógica (**ADR-112**): o que a missão viu o estudante fazer, uma linha por marcador da ficha × atividade × rodada. Só a missão escreve, pela porta `marcar`. Ver Seção 8.6.2.
`evidencias`	*Tenant*	O nível de UM estudante numa habilidade da BNCC, numa atividade, numa rodada, com `origem` e `situacao` — é o que o painel lê. **N/A é a ausência de linha.**
`evidencias_eventos`	*Tenant*	O histórico da evidência, com de/para, quem e quando. **Append-only no banco**: *trigger* contra `UPDATE`, e a *role* de *runtime* só com `SELECT, INSERT`.
`agente_professor_conversas`	*Tenant*	A POSSE das conversas do agente de IA do professor (**ADR-126**): de quem é cada sessão do Agent Engine. O conteúdo não mora aqui. Ver Seção 8.4.

8.3 A tabela que fica fora do modelo: `prospeccao_leads`

ADR-107. Cadastro de interesse do site público: nome, e-mail, telefone, organização, cargo, a frase consentida e a data. **Sem `escola_id`, sem RLS e sem relação com `users`** — não há tenant a que pertencer, porque a linha nasce meses antes de existir contrato, escrita por um visitante numa rota sem sessão. É a única tabela de dado pessoal do banco nessa condição, e o porquê está no Capítulo 7 ("A tabela sem tenant") e no verbete do Apêndice E.

Duas colunas merecem nota no modelo: `email` é guardado em minúsculas por *CHECK* (é a chave do *dedupe* de uma hora) e o aceite é `consentimento_texto`, a FRASE marcada — não um booleano, que provaria o clique e não o conteúdo do consentimento.

8.4 A posse das conversas do agente: `agente_professor_conversas`

ADR-126. A conversa do agente de IA do professor é a sessão do Vertex AI Agent Engine — eventos, estado, expiração —, e o banco **não a replica**. Esta tabela guarda só o que o engine não garante: **de quem a sessão é**. Os *endpoints* de sessão do engine pedidos por nome não conferem o usuário, e a SA da api enxerga todas as sessões; sem a linha, um `sessao_id` conhecido abriria a conversa de outro professor (Capítulo 7).

- **Tabela de PLATAFORMA, sem prefixo de experiência**: o agente é o produto `agente-professor`, não uma experiência (**ADR-110**).
- `escola_id NOT NULL`, RLS `ENABLE` + `FORCE` com a política por escola, FKs para `escolas` e `users` com `ON DELETE CASCADE`, e o `GRANT` à *role* de *runtime*, guardado.
- `sessao_id` **único**: uma sessão tem um dono só. *CHECK* de 1 a 200 caracteres.
- `titulo` é **CACHE** do `titulo_conversa` que o AGENTE escreve no estado da sessão depois do primeiro turno: nulo até lá, e um *CHECK* de 1 a 120 caracteres impede que um estado inesperado vire uma linha de lista enorme (espelha `TAMANHO_MAXIMO_DO_TITULO`).
- O índice `(escola_id, usuario_id, atualizada_em DESC)` é a lista do painel: as do professor na escola ativa, a mais recente primeiro.
- **Sessão que sumiu do engine apaga a linha** no `GET` seguinte; a lista nunca carrega uma conversa que não abre.

A mesma migração acrescenta `agente-professor` ao *CHECK* `ck_experiencias_produto`. Nenhuma experiência aponta para ele — é o primeiro produto SEM experiência —, mas o *CHECK* espelha a tupla `PRODUTOS` inteira, e um teste compara os dois textos.

8.5 A rede: escola, lotação e o CPF

Uma rede estadual tem milhares de unidades, professores que dão aula em mais de uma e crianças que se transferem entre elas. Três colunas e uma tabela sustentam isso, e a decisão que

as torna baratas é o que **não** muda: o *token* continua carregando UMA escola, então nenhuma tabela com `escola_id` muda, e a política de RLS forçada das tabelas de tenant fica intocada (**ADR-30**).

Peça	O que resolve, e o que ela não é
`lotacoes`	Pessoa × escola, datado. Professor e gestor **acumulam** — lotação nova não encerra a anterior. Estudante tem UMA ativa, e é o banco que garante: "estudante em duas escolas" é indistinguível de uma transferência que ficou pela metade.
Escola ativa	Estado de SESSÃO, não coluna. Quem tem mais de uma escolhe no cabeçalho, e a troca é validada contra as lotações *pela API* — nunca contra a lista que o *cookie* carrega.
`users.cpf`	A identidade IMUTÁVEL do estudante. Nome, *login* e matrícula mudam; o CPF não. É o que transforma "esse CPF já existe na escola B" em transferência reconhecida, em vez de duas crianças com o histórico partido. Guardado em 11 dígitos, único por contrato, e **exibido ao gestor** — é como ele confere identidade contra o sistema da rede. O que sustenta a decisão é a contenção: esta é a única coluna onde o número existe (**ADR-33**).
`escolas.codigo_inep` `municipio`, `uf`	Identificar a unidade numa rede de milhares. Sem município, a lista de escolas é impossível de navegar.
`escolas.tipo` `escolas.pai_id` `escolas.regional_padrao`	O nível REGIONAL — Diretoria de Ensino, polo, secretaria municipal. `tipo` tem vocabulário fechado (`escola`, `regional`, `rede`) e um *trigger* recusa `pai_id` que aponte para uma `escola` ou para a própria linha. **Agrupa, não autoriza**: serve para MONTAR a lista de unidades de uma diretoria — o insumo da lotação em lote — e nenhuma consulta de autorização a lê (**ADR-34**). **Desde a ADR-79 o vínculo é OBRIGATÓRIO**: `pai_id` não é nulo em `tipo = 'escola'` (*CHECK*) e o pai tem de ser `regional`. `regional_padrao` marca a regional que a plataforma cria para a rede sem divisão administrativa — no máximo uma, por índice único parcial.

ATENÇÃO

Coluna que ninguém lê é dívida com cara de recurso. `tipo` e `pai_id` existiram por um tempo sem nenhuma consulta que as usasse — a mesma armadilha do `expires_at` sem rotina e do `turmas.consentimento_em` do aplicativo de origem. Ou a coluna nasce com o consumidor, ou não nasce.

E coluna OPCIONAL que devia ser obrigatória é a mesma dívida, mais discreta. `pai_id` nulável deixava a unidade órfã, e o custo só ficou visível na tela: o filtro "Regional" não PARTICIONAVA a lista, porque havia escolas fora de toda regional, e a lotação em lote — o consumidor do agrupamento — cobria menos do que parecia. A **ADR-79** fechou isso com *CHECK*, *trigger* e a regional padrão.

DECISÃO DE PROJETO

Na transferência, o dado FICA e a identidade SEGUE (ADR-32). Tudo o que a criança produziu continua com o `escola_id` da escola onde produziu — verdade histórica que um *UPDATE* em massa apagaria. O que atravessa é a leitura: `GET /estudantes/:id/historico`, uma porta só, restrita a quem tem lotação na escola onde ela está hoje, devolvendo participações e nunca o conteúdo dos instrumentos.
É também a única operação que troca o `app.escola_id` *no meio de uma transação*: sem isso o *update* na escola de origem não veria linha nenhuma, a RLS o esconderia sem erro, e a criança ficaria matriculada nas duas.

8.6 Enturmamento e identidade

Duas tabelas respondem "quem é a pessoa" e "em que turma ela está, desde quando". São isomórficas ao OneRoster 1.2 (`academicSessions` e `enrollments`) por decisão registrada em ADR-28, com nomes em pt-BR — o padrão em si só aparece na fronteira de importação.

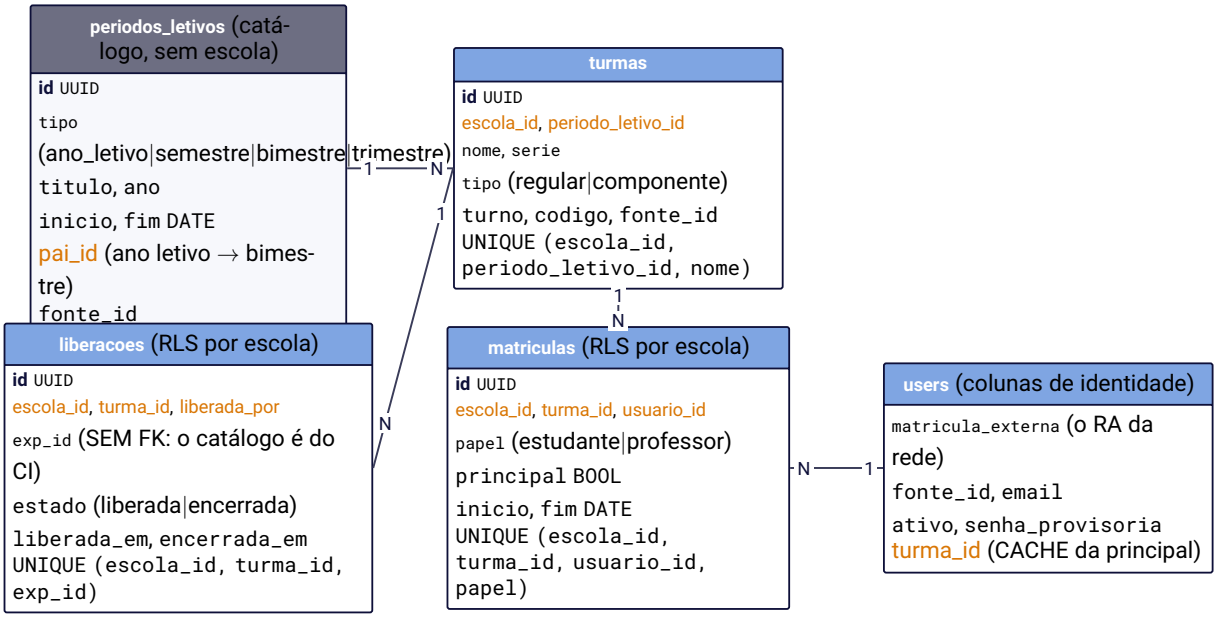


Figura 8.2 – Enturmamento e sala de aula. A verdade é `matriculas`; `users.turma_id` permanece como **cache** da matrícula principal, porque o *login* e o *claim* `turma_id` do *token* o leem antes de haver escola no contexto de tenancy. `liberacoes` é a camada que faltava entre a turma e o catálogo de experiências (ADR-69): sem linha, a missão não abre para aquela sala.

Decisão de esquema	Por quê
`papel` só admite `estudante` e `professor`	Gestor é papel no **nível da escola** (o `roles.csv` do padrão), não matrícula de turma — e é por isso que ele continua enxergando a escola inteira enquanto o professor passa a ver só as turmas dele.

`fim` em vez de `DELETE`	Há dado de estudante pendurado em oito tabelas. Apagar a matrícula deixaria esse histórico órfão de turma; encerrar preserva "esteve no 8A até março".
`matriculas` tem RLS, `turmas` e `users` não	A exceção de identidade existe porque o *login* precisa achar a pessoa **antes** de saber a escola — e a matrícula nunca é lida nesse momento. *Armadilha*: consulta fora de `comTenancy` devolve lista **vazia**, sem erro.
`liberacoes.exp_id` **sem** chave estrangeira	O catálogo `experiencias` é escrito pelo CI ao publicar o *bundle*, e numa stack de desenvolvimento ele está vazio — com FK, liberar dependeria de um *deploy* que não aconteceu. Mesma escolha de `progresso.exp_id` (**ADR-69**).
Não existe estado `bloqueada`	Bloqueio é a **ausência** de linha. Guardá-lo obrigaria a criar uma linha por turma × experiência só para o mundo nascer fechado — e ele nasce fechado de graça. *CHECK* garante o par `estado`/`encerrada_em`: data sem regra que a cumpra é a armadilha do `expires_at`.
Índice único **parcial** em `matricula_externa`	Duas pessoas sem RA não colidem entre si (`NULL` não é valor), mas o mesmo RA não se repete dentro de uma escola. O Prisma não modela índice parcial: ele vive na *migration*, como os *CHECKs* e a RLS.

8.6.1 As equipes na missão

ADR-111. Equipe é quem DIVIDE UM COMPUTADOR numa missão: a resposta física a uma sala com menos máquinas que crianças. São três tabelas de plataforma, na migração `20260925120000_equipes_na_missao`, e nenhuma mexe na porta da sala — a liberação continua por turma, e o dado da experiência continua por criança. O que muda é a LEITURA: o painel passa a poder contar por computador.

Decisão de esquema	Por quê
Três tabelas, e não uma coluna `equipe`	A montagem tem estado próprio — quantos computadores, sorteio ou manual, quem faltou, a `versao` — que não pertence a criança nenhuma. Uma coluna na tabela da experiência exigiria a mesma coluna em cada missão com equipe, e cada painel reinventaria a regra.
`turma_id`, `exp_id` e `atividade_id` repetidos em `equipes_membros`	É o que torna possível `UNIQUE (escola_id, turma_id, exp_id, atividade_id, estudante_id)`: uma criança, uma equipe por atividade (**ADR-130**). Sem as colunas ali, a regra só existiria no código, e dois `PUT` em paralelo poriam a mesma criança em dois computadores.
`versao` começa em 1, com *CHECK*	O cliente manda `0` quando não havia montagem. Versão 0 gravada tornaria o 409 incapaz de distinguir "não existe" de "existe e ninguém mexeu".
Troca INTEIRA, com cascata	`equipes` tem FK composta para a montagem, e `equipes_membros` para a equipe, as duas com `ON DELETE CASCADE`: salvar apaga as equipes e grava as novas; desfazer apaga a montagem, e o resto cai junto. Por isso nada que precise sobreviver a uma edição pode pender de `equipes` — o lado da equipe no Instanooria é resolvido na largada do *run*, e não guardado ao lado dela.

`exp_id` e `atualizado_por` **sem** FK	O primeiro pela razão de `liberacoes.exp_id`: o catálogo é do CI e está vazio em desenvolvimento, e quem restringe as missões é `EXPERIENCIAS_COM_EQUIPE`, no código. O segundo porque é carimbo: uma FK para `users` faria a saída de um professor da rede apagar ou travar a montagem da turma inteira.
Membro que mudou de turma continua na linha	A linha de `equipes_membros` não some quando a criança muda de turma — ela não foi apagada, só se mudou. A LEITURA filtra por `users.turma_id`, a mesma fonte dos painéis; a FK de `estudante_id` para `users`, com cascata, cobre só a exclusão.
CHECK do nome com `btrim`	Cinco espaços passariam por um `NOT NULL` e apareceriam no painel como uma equipe sem nome. O teto de 30 é `NOME_MAX` no código, e `tests/equipes-postgres.spec.ts` compara os dois vocabulários, como o de `modo`.

8.6.2 A evidência pedagógica

ADR-112. Que habilidades da BNCC cada estudante DEMONSTROU numa atividade de uma missão, rodada a rodada. São três tabelas de plataforma, na migração `20260925150000_evidencia_pedagogica`, e elas separam o que a missão VIU (`evidencias_marcas`) do nível que o professor LÊ e pode corrigir (`evidencias`), com toda mudança no histórico (`evidencias_eventos`). A primeira missão a escrever nelas é a exp05ef1b (**ADR-118**): a marca da EF05CO08 na transação do Mural, e as da EF05CO03 e EF05CO04 pelas perguntas.

Decisão de esquema	Por quê
A marca e a evidência em tabelas separadas	A evidência PRÓPRIA é derivada (a melhor marca da habilidade na rodada), mas a cópia da equipe e o ajuste do professor não são marca nenhuma. Guardar só a evidência perderia o que a missão viu — qual marcador, com que nível, sob que versão da ficha; guardar só as marcas não teria onde pôr o ajuste, e o painel refaria a precedência a cada leitura.
PK da marca **sem** `turma_id`	A chave é `(escola_id, exp_id, atividade_id, rodada, estudante_id, marcador)`: a marca é da criança naquela atividade e rodada, e a turma é só o contexto em que ela foi registrada. É o `INSERT ... ON CONFLICT DO NOTHING` dessa chave que faz valer a PRIMEIRA resposta de uma pergunta, mesmo com duas chegando juntas.
Uma evidência por habilidade por rodada	`UNIQUE (escola_id, exp_id, atividade_id, rodada, estudante_id, habilidade)`. O nome do índice é o que o Prisma gera, truncado em 63 caracteres: trocá-lo faria o `migrate diff` acusar um índice a renomear.
N/A é AUSÊNCIA de linha	"Não avaliado" não se grava: sem linha, a habilidade não conta em lugar nenhum, e nunca como `nao_atingiu`. O N/A EXPLÍCITO, `nao_participou`, é uma linha — e só do professor (*CHECK* `evidencias_nao_participou_check`), porque existe para barrar a cópia da equipe.

CHECKs de COERÊNCIA entre colunas	`a_revisar` só na origem `projecao`; `fonte_estudante_id` se e só se for cópia (`equipe/projecao`); `editado_por` se e só se for ajuste (`professor`). A linha que o código montar errada é recusada pelo banco, em vez de chegar ao painel como um estado que ele não sabe mostrar.
O vocabulário em dois lugares	Os *CHECKs* de `status`, `origem`, `situacao`, `tipo` e o *regex* de `atividade_id` repetem as constantes de `evidencias/regras.ts`, e `tests/evidencias-postgres.spec.ts` compara os dois — o par que já custou um *deploy* em `permissoes`. A marca só aceita os três níveis: "não participou" nunca é o que a missão viu.
`estudante_id` COM FK, em cascata — no histórico também	É a trava da identidade única (`tests/identidade-unica.spec.ts`), e a evidência é dado da criança: sai com ela. No histórico, cascata e não `SET NULL`: anular a coluna seria um `UPDATE`, que o *trigger* recusa, e a exclusão da conta travaria.
`equipe_id`, `fonte_estudante_id`, `editado_por` e `autor_id` **sem** FK	As equipes são trocadas INTEIRAS a cada `PUT` (**ADR-111**): uma FK para `equipes` apagaria ou travaria a cópia junto — e a cópia de uma rodada passada existe justamente para guardar a equipe que teve. Os outros três são carimbo: não podem travar a saída de ninguém da rede. `exp_id` também fica sem FK, pela razão de `liberacoes.exp_id`.
Histórico append-only NO BANCO	*Trigger* `BEFORE UPDATE` que lança para qualquer *role*, a dona da tabela inclusive, no desenho de `recursos_auditoria`. O `DELETE` fica fora, pela retenção futura, pelo isolamento de teste e pela cascata da turma; quem não apaga é a *role* de *runtime*, que só recebe `SELECT, INSERT`.
Índice por `registrada_em`	A RODADA EM ANDAMENTO de uma turma × missão é a da marca mais recente, e é só ela que o recálculo das equipes toca: `(escola_id, turma_id, exp_id, registrada_em)` a encontra. Os outros índices são as duas leituras — o painel, por `(escola_id, turma_id, exp_id, atividade_id, rodada)`, e o perfil, por `(escola_id, estudante_id)`.

8.7 Telemetria particionada

> **DECISÃO DE PROJETO**
>
> `telemetria_eventos` é **particionada por mês** (`PARTITION BY RANGE (ts)`), na migração `20260827100000_telemetria_particionada` (The PostgreSQL Global Development Group, 2026b). As partições vivem no *schema* `particoes`, **fora do alcance do Prisma** — em `public` cada partição viraria *drift* de *schema* a cada `prisma migrate dev`.

Três consequências do particionamento merecem registro:

1. **A chave primária é composta: `(event_id, ts)`.** O PostgreSQL exige a chave de partição em todo índice único. A *dedupe* do Pub/Sub segue correta, porque a reentrega da mesma mensagem traz o *mesmo* `ts` — então `ON CONFLICT (event_id, ts) DO NOTHING` continua sendo idempotência real, não aproximação.

2. **Há uma partição `DEFAULT`, como rede de segurança.** Sem ela, um evento com `ts` fora das faixas seria **recusado** — e telemetria perdida em silêncio é pior que telemetria mal guardada. As faixas declaradas vão até 2028-12.
3. **Expurgo passou a ser `DROP TABLE`.** Se um contrato exigir prazo de retenção, ele é cumprido por `DROP TABLE particoes.<mês>` — instantâneo — em vez de um `DELETE` que trava a tabela.

> **ATENÇÃO**
>
> A partição `DEFAULT` encher é **sinal de que faltou criar partição**, não funcionamento normal. Não há *job* agendado nesta plataforma — de propósito, porque seria a única peça de *scheduling* do sistema — e as faixas vão até 2028-12. Verificar a contagem da partição `DEFAULT` é item do *checklist* de operação (Apêndice F).

8.8 Modelo da exp08ef1b (Instanooria)

A exp08ef1b é a experiência mais rica em dados: um motor de *feed* adaptativo com geração de conteúdo por IA, sessão por estudante e dossiê de fechamento. São doze tabelas `exp08ef1b_` mais a tabela de contrato com a app-70.

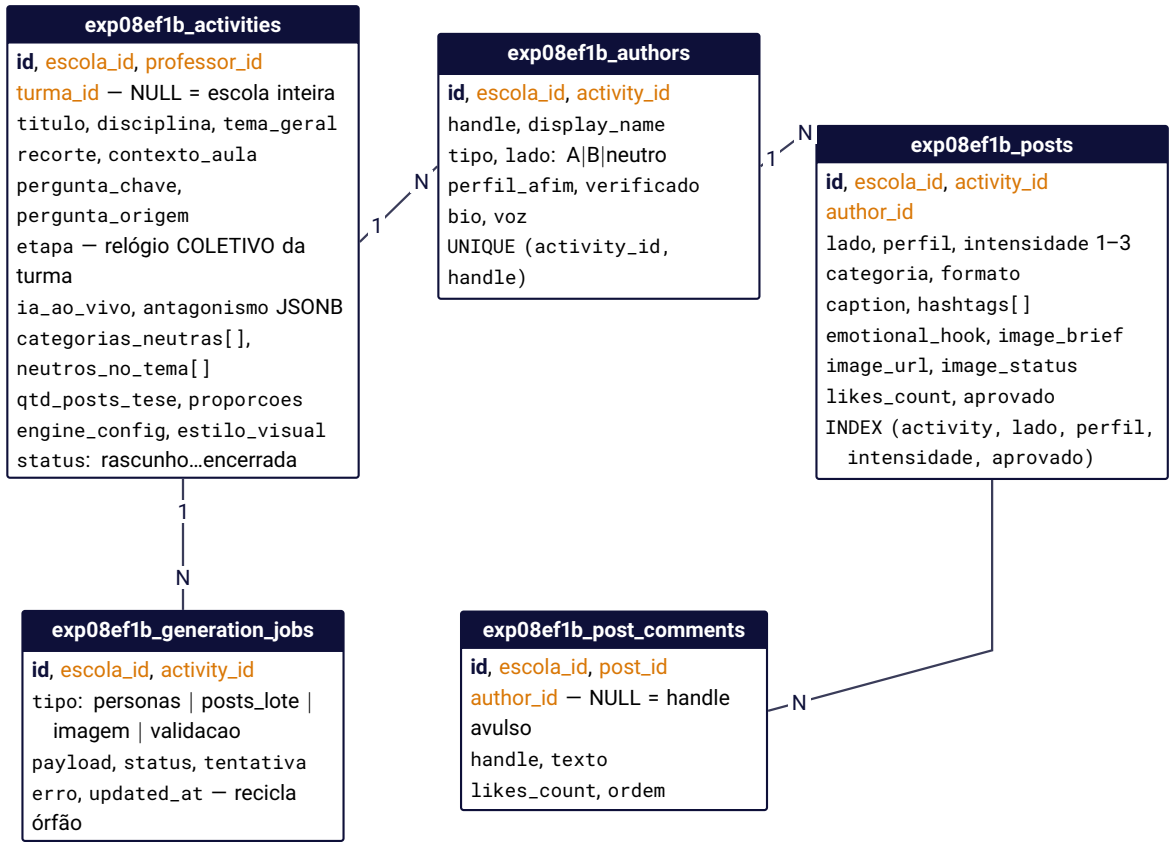


Figura 8.3 – Modelo de dados da exp08ef1b, parte 1: a atividade preparada pelo professor e o conteúdo gerado. `lado`, `perfil` e `intensidade` em `exp08ef1b_posts` são a variável independente do experimento — estão no banco, e o sanitizador da API os remove antes de o *feed* sair (Seção 5.8).

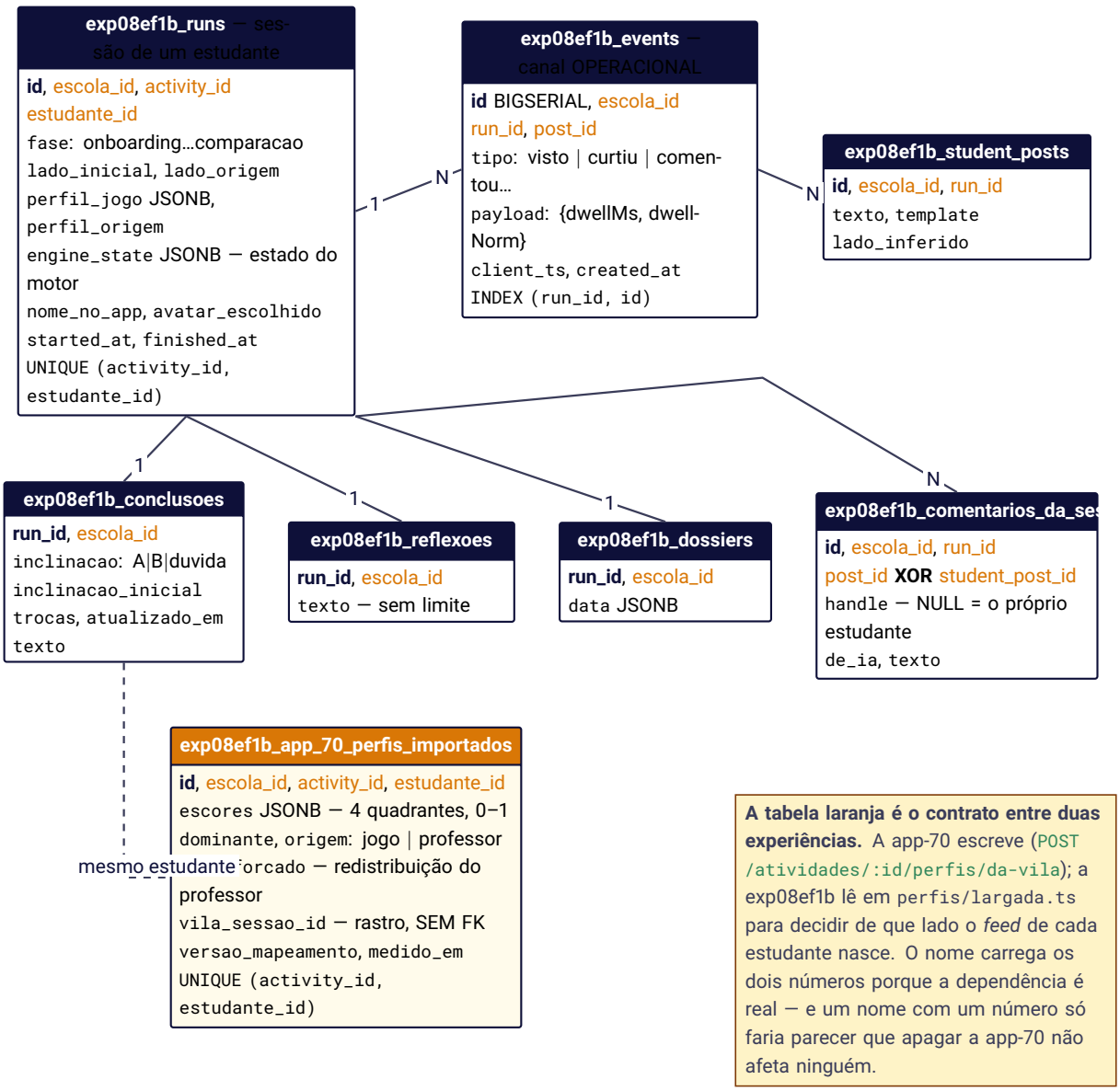


Figura 8.4 – Modelo de dados da exp08ef1b, parte 2: a sessão do estudante. A `run` é o centro — carrega o estado do motor em JSONB e é a unidade de *lock* da curadoria. `exp08ef1b_events` é o canal *operacional* de telemetria, síncrono e transacional, e não deve ser confundido com `telemetria_eventos`, o canal analítico (Capítulo 10).

Decisão de esquema	Por quê
`engine_state` em JSONB, não em tabela	O estado do motor é um agregado lido e escrito *inteiro*, sempre dentro do mesmo *lock*. Normalizá-lo daria *joins* sem nenhum ganho de consulta.
`nome_no_app` e `avatar_escolhido` na `run`, não em `users`	A conta é **do aparelho**: outra turma reusa o mesmo *login*. O nome que a criança digitou pertence àquela sessão.
`pergunta_origem`	Distingue pergunta gerada por IA de pergunta escrita ou editada pelo professor. É auditoria de autoria pedagógica, e custa uma coluna.

`lado_origem` na `run`	`perfil`, `sorteio` ou `redistribuido`: auditoria de *por que* aquele estudante caiu naquele lado. Sem isso, uma reclamação de professor não tem resposta.
`post_id` **XOR** `student_post_id` em `comentarios_da_sessao`	O comentário ancora num post do banco *ou* no post que o estudante publicou. Nunca nos dois — e o *CHECK* está na migração.
`de_ia`	Permite ao professor distinguir, na auditoria, o comentário escrito pela persona (modelo) do escrito pelo estudante.

8.9 Modelo das experiências 02, 70, 90 e 91

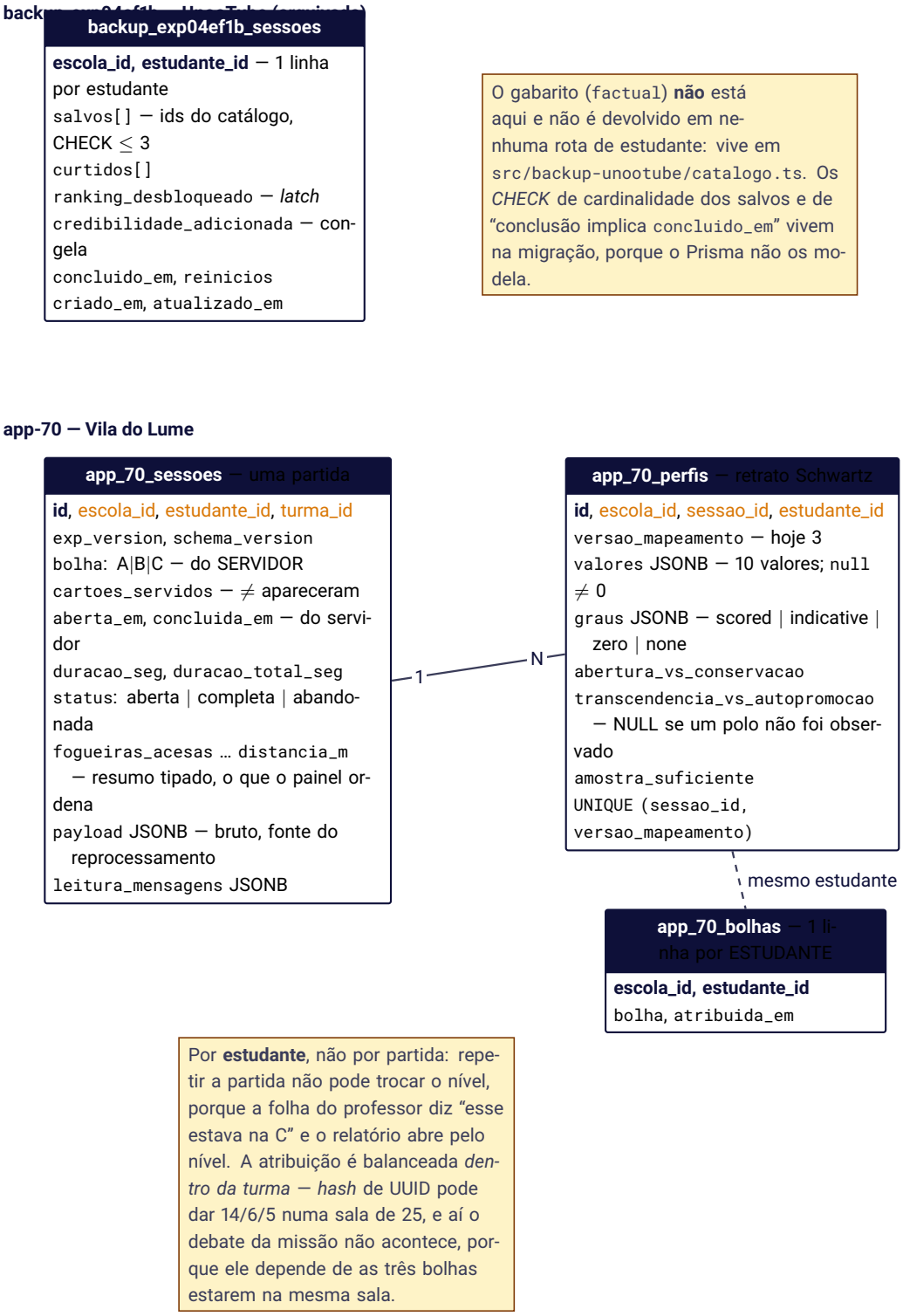


Figura 8.5 – Modelo de dados do UnooTube arquivado (`backup-exp04ef1b`, **ADR-116**) e da app-70. Em `app_70_perfis`, `null` **não é zero**: é valor que não recebeu número. `graus` diz por quê, e é essa distinção que impede um relatório de afirmar recusa onde houve ausência de oportunidade.

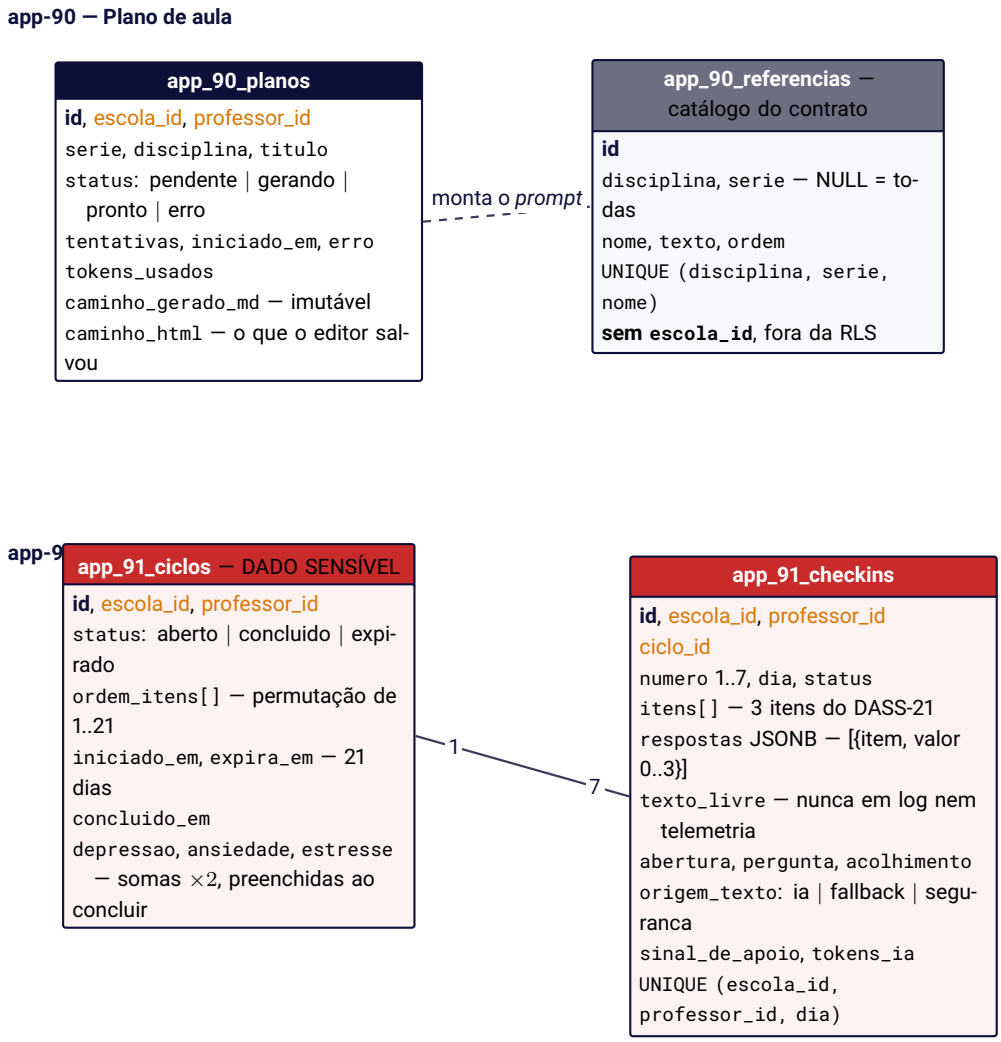


Figura 8.6 – Modelo de dados da app-90 e da app-91. As caixas vermelhas são dado *sensível* de saúde (LGPD art. 11): RLS e posse por `professor_id` em toda *query*, sem nenhuma rota de gestor ou agregado. `app_90_referencias` é catálogo do contrato e fica **fora** da RLS, como `experiencias`.

8.10 As cinco tabelas da exp08ef4b

Tabela	O que guarda
`exp08ef4b_conteudo_turma`	O **pacote curado** que o professor homologa antes da aula — é a tabela que não existe nas outras experiências. Uma linha por turma, com o contexto, os 3 questionamentos (com as réplicas) e as 4 cláusulas (com os argumentos de mediação) em `JSONB`. O booleano `aprovado_por_professor` é o portão da exigência 8.2: sem ele verdadeiro, a experiência do estudante lê o banco curado da Unooria, e rascunho de IA **não conta**. **Sobrevive ao "Zerar turma"**: a curadoria custou tempo do professor.

`exp08ef4b_sessoes`	O estado da missão, uma linha por estudante. Chave `(escola_id, estudante_id)`. O que é **1:1** com a sessão é *coluna* dela: a `clausula_id` fixada, a `latencia_clausula_seg`, o `confirmou_provocacao` e o `manifesto_markdown` compilado. **Não há coluna `score`**, e a ausência é o desenho: a missão não pontua.
`exp08ef4b_auditorias`	Cada questionamento disparado contra a máquina — **1:N** de propósito. A cota é sobre questionamentos *distintos*, e um contador perderia *quais* a criança escolheu fazer, que é o dado da hipótese pedagógica (a contradição só aparece no do preconceito histórico). Um `UNIQUE (escola_id, estudante_id, prompt_id)` faz a dedupe valer mesmo se um caminho novo esquecer de checar.
`exp08ef4b_respostas_perc`	O "Meu Percurso": duas palavras-chave e um parágrafo de 20 a 800 caracteres escrito por menor de idade. Fora de *log*, fora da telemetria analítica, alcançável só pelo professor da turma dela. **Sem *flag* de moderação**: o texto com dado pessoal é recusado com **422** e não chega a esta tabela — não há registro moderado a marcar.
`exp08ef4b_gates_turma`	O **Checkpoint A**, por turma — `missao | debate`. Chave `(escola_id, turma_id)`. É esta linha que o *polling* de 5 segundos lê, porque não há WebSocket de aplicação no Cloud Run (ADR-93).

REGRA INVIOLÁVEL

Os *CHECK* desta migração amarram três invariantes que as regras puras já aplicam. `(clausula_id IS NULL) = (manifesto_markdown IS NULL)`: o documento existe se e somente se a cláusula foi fixada — um manifesto sem cláusula teria o artigo em branco, e uma cláusula sem documento seria a Etapa 4 sem o artefato que ela promulga. No percurso, `cardinality(palavras_chave_tags) = 2` **e** `tags[1] <> tags[2]`: marcar a mesma palavra duas vezes deixaria um *slot* logicamente vazio dentro de um *array* cheio, e a nuvem agregada da turma contaria a mesma criança duas vezes na mesma palavra. E o pacote da turma tem `jsonb_array_length` fixo em 3 e 4: um pacote com duas cláusulas faria a tela do pico renderizar meio *grid*, e o sintoma apareceria na aula, não no *deploy*.

ATENÇÃO

`exp08ef4b_auditorias` tem id `BIGSERIAL` e exige `GRANT USAGE` na **sequência**, além do *grant* de tabela — a quarta aparição da mesma armadilha. Sem ele o `INSERT` falha para a *role* de runtime, e só em produção, que é onde ela existe.

8.11 As quatro tabelas da exp05ef1b

A v2.2 de Quem mandou isso? (ADR-118), na migração `20260926130000_exp05ef1b_quem_mandou_isso_v2`. As quatro têm `escola_id NOT NULL`, RLS `ENABLE` + `FORCE` com a política por

escola, e nenhuma sequência: as chaves são `uuid` ou compostas, e o `GRANT` à *role* de *runtime* é só de tabela.

Tabela	O que guarda
`exp05ef1b_pacotes`	O **pacote de uma turma**: o recorte do preparo (componente curricular, tema, fatos, contexto) e as oito mensagens **com** o gabarito em `JSONB` — lido e aprovado sempre inteiro, então não há tabela-filha. `ativo` marca o que a turma usa, `rodada_atual` é a reaplicação, `aprovado_em` nulo é rascunho. **É a ATIVIDADE da evidência**: o `id` é o `atividade_id` das marcas.
`exp05ef1b_gates_turma`	O **relógio da turma** — `triagem`, `revelacao`, `explicar`, `reflexao` —, que só anda para a frente e volta a `triagem` ao aprovar um pacote e ao zerar. Chave `(escola_id, turma_id)`; é a linha que o *polling* de 3 s lê (**ADR-93**).
`exp05ef1b_sessoes`	A sessão do estudante numa **rodada** de um pacote, chave `(escola_id, estudante_id, pacote_id, rodada)`: nasce no "Entrar no grupo", e zerar a turma abre outra linha em vez de apagar esta. Guarda a seleção (0 a 3) e os carimbos dos *latches* — `confirmado_em`, `votou_em`, `reflexao_em`, `concluido_em` —, o motivo votado, o `tempo_feed_segundos` e a `versao_instrumento`. **Não tem coluna de passo** nem de pontuação.
`exp05ef1b_reflexoes`	A regra Se/Então, 1:1 com a sessão: a condição (`condicional_se`) e o "Então" (`acao_entao`, 15 a 240 caracteres) — **texto livre escrito por menor de idade**. Tabela à parte de propósito: não vai a *log* (`acao_entao`, `acaoEntao` e `entao` estão na redação), não vai à telemetria, e o painel só a preenche para quem leciona na turma.

REGRA INVIOLÁVEL

Os *latches* estão em ordem no banco. Cada carimbo exige o anterior: confirmar o Mural exige **exatamente três** na seleção, votar exige ter confirmado (e motivo e carimbo andam juntos), a regra exige o voto, e concluir exige a regra. O *CHECK* vale por *statement*, então o *service* grava carimbo e dado no MESMO `UPDATE` — a seleção com o `confirmado_em`, o motivo com o `votou_em` —, senão o estado intermediário reprova. A seleção fica dentro dos oito ids (`<@ ARRAY['msg_1' …'msg_8']`) e com no máximo três; o `tempo_feed_segundos` existe exatamente quando o Mural fechou.

Os vocabulários estão em DOIS lugares, e de propósito: componente curricular, origem, fase, motivo e condição são *CHECK* aqui e constante em `catalogo.ts` e `regras/passos.ts`, e a suíte de Postgres compara os dois textos. É o par que já custou um *deploy* em `permissoes`.

A matriz 2×2 NÃO entra no banco. O *CHECK* cobra que `mensagens_json` é um *array* de oito; contar os quadrantes e o equilíbrio de formato exigiria desmontar o `JSONB` dentro do *CHECK*, e o gabarito tem uma gramática só — a de `regras/pacote.ts#validarPacote`.

Decisão de esquema	Por quê

Dois índices únicos **parciais** por turma	Um `WHERE ativo` e um `WHERE aprovado_em IS NULL`: um pacote ativo e um rascunho por turma. O `@@unique` do Prisma não os expressa, e sem eles dois cliques em "Aprovar" deixariam a turma com dois pacotes ativos — e o estudante com o que o `findFirst` achasse primeiro.
CHECKs de aprovação	`ativo` só com `aprovado_em`; `aprovado_em` e `aprovado_por` nulos juntos; rascunho sempre na rodada 1. `aprovado_por` é carimbo sem FK: a saída de um professor da rede não pode travar o pacote da turma.
O tema com teto de caracteres E de palavras	`btrim` entre 3 e 60 e até 8 palavras: o tema entra na fala da Nina, e cinco espaços não são um tema.
FK composta `(escola_id, pacote_id)`	A sessão de uma escola não pode apontar para o pacote de outra — o alvo é o `@@unique([escolaId, id])` do pacote.
O "Então" com `btrim`	`char_length(btrim(acao_entao)) BETWEEN 15 AND 240`: quinze espaços não são uma regra, e contariam como entregue no painel.
Retenção	A da vigência contratual, sem rotina de expurgo, como nas outras missões (**ADR-13**). Declarar um prazo aqui seria escrever o que nenhum *scheduler* cumpre.

8.12 As quatro tabelas da exp04ef1b

O Jornal da Escola (**ADR-117**), na migração `20260927120000_exp04ef1b_jornal_da_escola`. As quatro têm `escola_id NOT NULL`, RLS `ENABLE` + `FORCE` com a política por escola, e nenhuma sequência: as chaves são `uuid` ou compostas, e o `GRANT` à *role* de *runtime* é só de tabela, guardado. O prefixo não colide com `backup_exp04ef1b_sessoes`, a do UnooTube arquivado.

Tabela	O que guarda
`exp04ef1b_pacotes`	A **atividade** da evidência: a disciplina, o tema, os tópicos e as oito notícias **com** o gabarito (publicável, pilar em falta, explicação) e a consequência, em `JSONB`; a `midia` (as URLs geradas, por notícia), a `origem` (`padrao` | `ia`), o carimbo de aprovação e os contadores da telemetria do professor (`geracoes`, `tokens_usados`, `painel_vistas`). `ativo` marca o que a turma usa.
`exp04ef1b_gates_turma`	O **relógio da turma**: `exploracao` | `resultado` | `revisao`, o pacote em andamento, a **rodada** e os carimbos dos dois portões. Chave `(escola_id, turma_id)`; é a linha que o *polling* de 5 s lê (**ADR-93**). Zerar e homologar voltam a etapa e limpam os carimbos no mesmo `UPDATE`.
`exp04ef1b_sessoes`	Uma linha por estudante, pacote e rodada (`UNIQUE` de cinco colunas, com o nome dado à mão). O passo gravado, as pistas abertas (`JSONB`, uma entrada por par e fase), as vagas, o layout, a origem do layout e as verificadas de cada edição, o motivo, as tentativas da Chave e os carimbos dos *latches*. **Nenhum campo de texto livre.**

`exp04ef1b_diagramacoes`	O *cache* da diagramação por IA: pacote e as três vagas NA ORDEM (a `chave`, como `n1,n5,n3`) → layout, com os *tokens* gastos. É o que limita o custo às combinações usadas. Fica sob RLS também: a chave diz quais três notícias uma turma escolheu.

REGRA INVIOLÁVEL

Os *latches* são EQUIVALÊNCIAS com o passo gravado, e não só implicações: `publicada1_em` existe se e só se o passo saiu de `desafio` e `mesa`; o motivo, se e só se saiu também de `motivo`; a Chave formada, se e só se o passo está de `revisao` em diante; `publicada2_em`, de `comparativo` em diante; e `fim`, se e só se há `concluida_em`. Cada edição publicada exige as três vagas DIFERENTES dentro dos oito ids, o layout, a origem e as verificadas no mesmo gesto. Os passos `resultado` e `chave` **nunca** são gravados — a etapa os impõe a quem está em `publicada` —, e um *CHECK* recusa os dois: gravados, fariam o F5 de uma rodada zerada abrir a Chave sem a revisão liberada. O *CHECK* vale por *statement*, então o *service* grava carimbo e passo no MESMO `UPDATE`.

Decisão de esquema	Por quê
Um índice único **parcial** por turma	`exp04ef1b_pacotes_um_ativo`, `WHERE ativo`: sem ele, duas homologações simultâneas — ou duas primeiras entradas criando o banco padrão — deixariam a turma com duas atividades em andamento. O Prisma não o enxerga, e ele não aparece no *schema*.
O pacote de IA exige quem o aprovou	*CHECK* `origem = 'padrao' OR aprovado_por IS NOT NULL`: o pacote `ia` sem aprovação é um rascunho que escapou do preparo: nada gerado chega à turma sem a curadoria do professor. O `padrao` pode nascer sozinho, na primeira entrada da turma.
Oito notícias, e a régua fora do banco	O *CHECK* cobra um *array* de oito; meios, composição e réguas de palavras são o Zod da homologação (`invariantes.ts`). Contá-los no banco exigiria desmontar o `JSONB` dentro do *CHECK* — duas gramáticas para a mesma regra.
Os portões andam com a etapa	`resultado_liberado_em` existe se e só se a etapa é `resultado` ou `revisao`, e `revisao_liberada_em` se e só se é `revisao`.
Os vocabulários em DOIS lugares	Etapa, passo, meio, layout, motivo e origem são *CHECK* aqui e constante em `vocabulario.ts` e `regras/passos.ts`, e a suíte de Postgres compara os dois textos.
Retenção	A da vigência contratual, sem rotina de expurgo, como nas outras missões (**ADR-13**).

8.13 As quatro tabelas da exp07ef1b

Tabela	O que guarda
`exp07ef1b_coberturas`	A **atividade** da evidência: o tema que a professora homologou, os 12 posts **com** o gabarito (enquadramento, fonte ouvida, ângulo) em `JSONB`, as URLs da mídia gerada e a `origem` (`padrao` | `ia`). Um índice único parcial deixa só uma cobertura `ativa` por turma; *CHECK* cobra os 12 posts e o `aprovada_por` da cobertura de IA. `podcast` guarda as três falas do Bloco 2 do podcast de recapitulação no tema dela (nulo = as do roteiro aprovado, **ADR-119**).
`exp07ef1b_gates_turma`	O relógio da turma: `exploracao` | `mural` | `fichas` | `explicacao`, mais a cobertura e a **rodada**. Zerar incrementa a rodada, não apaga.
`exp07ef1b_sessoes`	Uma linha por estudante, cobertura e rodada. A tríade (três ids distintos do feed), a legenda (10 a 140 *code points*), a matriz (as três lentes somando 3), o par (`par_sessao_id` sem FK ou `par_virtual`, e o rótulo congelado). Os *CHECK* amarram o *latch*: publicado equivale a tríade, legenda e matriz.
`exp07ef1b_reflexoes`	A reflexão: 100 a 500 caracteres de **texto livre de menor**, com `chars = char_length(texto)`. Fora de *log* e da telemetria, e só sai para quem leciona a turma (**ADR-108**).

8.14 As quatro tabelas da exp06ef2b

Tabela	O que guarda
`exp06ef2b_sessoes`	O estado da missão, uma linha por estudante. Chave `(escola_id, estudante_id)`. O que é **1:1** com a sessão é *coluna* dela: a `opcao` da intervenção, a `latencia_seg`, a `ordem_blocos` do ciclo e o `coerente`. As tabelas `exp06ef2b_decisoes` e `exp06ef2b_modelagens` que o insumo pedia dariam uma linha por estudante com uma *FK* cada, e o *JOIN* existiria para nada.
`exp06ef2b_auditorias`	Cada abertura do painel de dados de envio — **1:N** de propósito. A cota é sobre mensagens *distintas*, e um contador perderia *quais* o estudante escolheu auditar, que é o dado pedagógico. Um `UNIQUE (escola_id, estudante_id, mensagem_id)` faz a dedupe valer mesmo se um caminho novo esquecer de checar antes.
`exp06ef2b_reflexoes`	O "Meu percurso": 280 caracteres de texto livre escrito por menor de idade — **diferente do Eco no Grupo** (retirado, **ADR-113**), que tinha fechado essa porta. Aqui o insumo pede a redação como instrumento metacognitivo, e ela vem com o mesmo isolamento: fora de log, fora da telemetria analítica, e alcançável só pelo professor da turma dela.
`exp06ef2b_gates_turma`	A etapa da aula, por **turma** — `aula1 | aula2`. Chave `(escola_id, turma_id)`.

REGRA INVIOLÁVEL

Três *CHECK* que são a rede das regras puras. O `score` só aceita **0, 70 ou 100** — os três valores que `regras/pontuacao.ts` produz; qualquer outro seria pontuação vinda do cliente (regra inviolável nº 3). `cardinality(ordem_blocos) IN (0, 4)` amarra o modelo do ciclo: 1, 2 ou 3 blocos é slot vazio, e um quarto *CHECK* exige que `coerente` seja nulo exatamente quando a ordem está vazia. E `passo = 'fim'` está amarrado a `concluido_em` não nulo: uma tela dizendo "missão cumprida" sobre uma linha sem carimbo é um relatório que mente para o professor.

ATENÇÃO

`exp06ef2b_auditorias` tem id `BIGSERIAL` e exige `GRANT USAGE` na **sequência**, além do *grant* de tabela — a terceira aparição da mesma armadilha. Sem ele o `INSERT` falha para a *role* de runtime, e só em produção, que é onde ela existe.

8.15 As cinco tabelas da exp02em1b

Tabela	O que guarda
`exp02em1b_sessoes`	O estado da partida, uma linha por estudante. Chave `(escola_id, estudante_id)` — **não há id de sessão**: a identidade vem do *token*, e foi isso que dispensou a guarda de posse do projeto de origem, onde a sessão era um *uuid* na URL. Inclui `tokens_gastos` e `chamadas_ia` (ADR-24).
`exp02em1b_mensagens`	A **Transcrição**: a conversa entre o estudante e o agente. `texto` é caixa livre escrita por menor de idade — o dado mais sensível da experiência (ADR-27).
`exp02em1b_marcos`	A trilha da decisão: o que a criança fez, em que minuto da cidade — e com **que cronômetro na tela** (`segundos_restantes`, `motivo_do_fim`), recalculado do relógio da turma no instante da gravação (ADR-64). Seis tipos: `fontes`, `leitura`, `alocacao`, `envio`, `inicio` (entrou na Investigação) e `recomeco` (o clique em Recomeçar, na tentativa que fecha). `dados` traz **só dado público** — nada de viés, Lacuna ou parâmetro oculto, porque este JSON chega ao navegador.
`exp02em1b_simulacoes`	O desfecho de cada tentativa, com `versao_prompt` gravada junto: sem ela não se sabe qual *prompt* produziu o que está em produção. E a **faixa da decisão** (`esperado`), gravada no confirmar em vez de recomputada a cada leitura (ADR-64).
`exp02em1b_relogio_turma`	O Relógio da Aula, por **turma** (ADR-25). Chave `(escola_id, turma_id)`.

> **REGRA INVIOLÁVEL**
>
> Os filhos apontam para o **par** `(escola_id, estudante_id)`, não para um id de sessão. Cada linha filha carrega `escola_id` por construção — que é o que a RLS precisa — e nenhuma consulta nova pode esquecê-lo.

Cinco colunas do projeto de origem não vieram, porque estavam mortas lá, e foram verificadas uma a uma antes do descarte: `simulacoes_restantes` e `custo_cents` (zero leituras), `camadas_reveladas` e `pergunta_id` (só escritas vazias) e `evidencias` (só escrita, e o que ela guardava já está em `rodadas`). Centavo virou *token*: preço depende de tabela do *vendor*, que muda sem aviso e não é conhecida pela aplicação.

Nenhuma coluna de prazo (ADR-26). E 20 *CHECKs* de estado impossível na migração, entre eles: justificativa sem destino, encerrar relógio que nunca começou, duração de relógio igual a zero (dividiria por zero no minuto narrativo) e `debitou` numa fala que não é do estudante.

> **ATENÇÃO**
>
> As duas tabelas com `BIGSERIAL` (`exp02em1b_mensagens` e `exp02em1b_marcos`) exigem `GRANT USAGE` na **sequência**, além do *grant* de tabela. Sem ele o `INSERT` falha para a *role* de *runtime* mesmo com todos os privilégios de tabela concedidos.

8.16 As três tabelas da app-92

A app-92 (Planejamento de Ensino dirigido pela BNCC, ADR-73, ADR-76) persiste o plano curricular do docente em três tabelas:

Tabela	O que guarda
`app_92_configuracoes`	Regência anual do professor (ano letivo, série, componente, divisão do ano, aulas semanais e grade de horários em JSON). Chave única em `(escola_id, professor_id, ano_letivo, serie, componente)`.
`app_92_planos_anuais`	Plano consolidado gerado com IA (objetivos gerais, cronograma de unidades temáticas e competências BNCC), com controle de versão, *prompt* imutável e `tokens_gastos`.
`app_92_planos_de_periodo`	O detalhamento do plano em granularidade **estritamente semanal** (ADR-76), amarrado à configuração e ao plano anual, contendo semanas letivas, habilidades BNCC trabalhadas, estratégias didáticas e instrumentos de avaliação.

8.17 Decisões de esquema por experiência

8.17.1 app-70 — as decisões que custam informação de propósito

Decisão	Por quê
`turma_id` desnormalizado na sessão	O estudante troca de turma entre anos letivos, e o resultado precisa continuar *na turma em que foi medido*.
Datas cravadas pelo **servidor**	O aplicativo de origem derivava do relógio do cliente, e documentava que a hora numa máquina de escola costuma estar errada.
O `id` da sessão é emitido pela API	No aplicativo de origem o cliente escolhia a própria chave de idempotência, e duas crianças na mesma máquina colidiam — a segunda partida voltava "duplicada" e desaparecia.
`valores` com `null`, e `graus` separados	`null` **não é zero**: é valor que não recebeu número, porque tem uma fonte de medida só ou porque a oportunidade não chegou. `graus` diz *por quê*, e é essa distinção que impede um relatório de afirmar recusa onde houve ausência.
Eixos do circumplexo anuláveis	Eixo com polo não observado não é zero: é *indefinido*. Colocar zero produziria um ponto no meio do gráfico, que se lê como "equilibrado".
`amostra_suficiente` como coluna	Falso ⇒ o painel abre avisando que **não sabe**. Não é rigor estatístico: é impedir que uma partida sem escolha vire uma frase sobre quem a criança é.
Nova linha por `versao_mapeamento`	O mapeamento é hipótese de *design* e vai mudar. Guardar só o resultado tornaria todo o passado incomparável no dia da revisão. `UNIQUE (sessao_id, versao_mapeamento)`.
Sem tabela de evento fino	O bruto vive em `payload` (fonte do reprocessamento) e o canal analítico da SDK já grava em `telemetria_eventos`. Duas tabelas de log seriam duas histórias de retenção para o mesmo dado.

8.17.2 app-90 — conteúdo fora do banco

> **DECISÃO DE PROJETO**
>
> O **conteúdo** do plano de aula não fica no PostgreSQL: fica em armazenamento de objetos *privado* (`planos/{escolaId}/{planoId}/gerado.md` imutável + `atual.html` do editor). A tabela guarda metadados, estado da geração e *caminhos*.
> O ganho não é performance: é que o versionamento de objetos do *bucket* dá histórico de cada "Salvar" **sem uma linha de código de histórico**. Um professor que sobrescreveu o plano por engano tem recuperação, e ninguém escreveu uma tabela de versões.

8.17.3 app-91 — ausência como decisão de esquema

> **DECISÃO DE PROJETO**
>
> Não existe tabela de agregado de bem-estar, e não existe *view* por escola. Isso não é lacuna: é a decisão. Dado de saúde (LGPD art. 11) tem RLS *e* posse por `professor_id` em toda *query*; o titular apaga tudo com `DELETE /bemestar/dados`. Uma visão agregada e anonimizada para a rede é decisão *futura*, com DPA e *k*-anonimato — e não sai de um `GROUP BY`.

8.18 Migrações

Migração	Conteúdo
`20260824145301_init`	Esquema inicial: *tenancy* (`escolas`, `turmas`, `users`), tabelas da exp08ef1b, tabelas de plataforma, e o laço que aplica `ENABLE/FORCE ROW LEVEL SECURITY` + política por escola.
`20260825011950_unootube_sessoe`	Estado *server-side* da exp04ef1b, com *CHECK* de cardinalidade dos salvos.
`20260825120000_planos_de_aula`	Planos e referências curriculares.
`20260825225016_bem_estar`	Ciclos e *check-ins* do DASS-21, com RLS e `GRANT`.
`20260826120000_permissoes_de_if`	Coluna `permissoes` no manifesto, com *CHECK* do vocabulário aceito.
`20260826140000_vila_do_lume`	Sessões, perfis e bolhas da app-70.
`20260826170000_vila_sem_prazo`	Remoção da coluna `expires_at` (**ADR-13**).
`20260827090000_tabelas_por_exp`	Renomeação para o prefixo por experiência (então `exp_NN_`): `RENAME` de tabelas, *constraints*, índices e políticas. É a migração de referência.
`20260827100000_telemetria_particionada`	Particionamento mensal de `telemetria_eventos` no *schema* `particoes`, com partição `DEFAULT` e faixas até 2028-12.
`20260827184956_exp_03_unoogpt`	Sessões e marcos de conversa da exp02em1b (UnooGPT), com isolamento por escola e RLS.
`20260828093000_enturmamento_e_identidade`	`periodos_letivos`, `matriculas` (com RLS), colunas de identidade em `users` e o *backfill* de `users.turma_id` para matrícula principal.
`20260828140000_rede_lotacoes_e`	`lotacoes` e o índice único parcial que impede estudante ativo em duas escolas.
`20260828170000_cpf_em_claro`	Troca de `cpf_hash` por `users.cpf` (11 dígitos, *CHECK* e único parcial) — **ADR-33**.

`20260828200000_nivel_regional`	Vocabulário `escola` | `regional` | `rede` em `escolas.tipo` (`distrito` renomeado), o *trigger* que exige que o pai AGRUPE, e índices em `pai_id`/`tipo` — **ADR-34**.
`20260828210000_permissao_forms`	Adição do token `forms` ao vocabulário de permissões do iframe (**ADR-26**).
`20260828230000_perfil_portatil`	`perfis_portateis`, com RLS, *CHECK* do vocabulário de `verificacao` e único por (escola, estudante, emissor, emissão) — **ADR-35**.
`20260829120000_conclusao_primeira_coleta`	Registro de inclinação inicial, trocas de rota e convergência no término da exp08ef1b.
`20260830090000_atividade_por_t`	Adição da coluna `turma_id` nas atividades da exp08ef1b para agregação analítica docente.
`20260830100000_perfil_importado_vinculo_app70`	Vínculo de proveniência entre `exp08ef1b_app_70_perfis_importados` e a sessão da Vila do Lume.
`20260902120000_exp_03_trilha_c`	Rastreamento temporal nos marcos do diálogo com o bot (segundos restantes e causa de encerramento).
`20260903120000_exp_07_o_eco_no_grupo`	Estado *server-side* da exp07ef1b (WhatsApp / Desinformação), com RLS e validações de progresso. As quatro tabelas caíram na `20260925180000_remover_eco_no_grupo`.
`20260905120000_liberacoes_por_`	Tabela de controle de abertura de missões por turma (`liberacoes`), governança docente com RLS (**ADR-69**).
`20260905160000_experiencia_declaracao_unidade`	Adição da coluna `unidade` (1 a 4) ao catálogo curricular de experiências.
`20260906120000_exp_92_plano_de`	Tabelas da app-92: `app_92_configuracoes`, `app_92_planos_anuais` e `app_92_planos_de_periodo` (**ADR-73**).
`20260906140000_exp_92_inicio_do_ano_letivo`	Início do ano letivo parametrizado na configuração curricular por escola/ano (**ADR-73**).
`20260906170000_exp_06_rastro_b`	Tabelas de sessão e simulação de rede da exp06ef2b (Rastro do Boato) com tema escuro e topologia restrita (**ADR-72**).
`20260907120000_exp_92_planejamento_estruturado`	Reestruturação do período de planejamento com cardinalidade estritamente semanal de 40 semanas (**ADR-76**).
`20260907130000_permissao_downl`	Liberação do token `allow-downloads` no sandbox de iframes de experiências para exportação de planos (**ADR-76**).
`20260908120000_regional_padrao_escolas_regional_padrao`	Regional padrão com índice único parcial, o *CHECK* que exige `pai_id` em `tipo = 'escola'`, o *trigger* que passa a exigir pai `regional` e a adotar pela padrão, a criação da regional padrão e o *backfill* das órfãs — **ADR-79**.

`20260908150000_modelo_de_papei`	O vocabulário de papéis de quatro nomes para seis (`gestor` → `gestor_escola`, `admin-contrato` → `gestor_contrato`, mais `gestor_regional` e `admin`), `users.regional_id` com FK, *CHECK* bicondicional e *trigger*, e o `lotacoes_papel_check` acompanhando — **ADR-80**.
`20260923120000_codigo_das_experiencias`	O código da experiência (**ADR-110**): `RENAME` por OID de tabela, *constraint*, índice, política e *sequence* (`exp_01_` → `exp08ef1b_`, `exp_70_` → `app_70_`) e o `UPDATE` do `exp_id` em sete tabelas, com o `FORCE ROW LEVEL SECURITY` e o *trigger append-only* afrouxados só dentro da transação.
`20260925120000_equipes_na_missao`	As equipes na missão (**ADR-111**): `equipes_montagens`, `equipes` e `equipes_membros`, as três com RLS forçada; o `UNIQUE` de uma equipe por criança por missão, os *CHECKs* de `modo`, `computadores`, `versao`, nome e ordem, e o `GRANT` à *role* de *runtime*.
`20260925150000_evidencia_pedagogica`	A evidência pedagógica (**ADR-112**): `evidencias_marcas`, `evidencias` e `evidencias_eventos`, as três com RLS forçada e FK de `turma_id` e `estudante_id` em cascata; os *CHECKs* de vocabulário e de coerência, o *trigger* que recusa `UPDATE` no histórico e o `GRANT` à *role* de *runtime* (só `SELECT, INSERT` no histórico).
`20260925180000_remover_eco_no_grupo`	A retirada do Eco no Grupo (**ADR-113**): `DROP` das quatro tabelas `exp07ef1b_*` e `DELETE` das linhas `exp07ef1b` em `experiencias`, `experiencias_versoes`, `liberacoes`, `progresso` e `disponibilidades`, porque o código vai significar outra missão. `liberacoes` e `progresso` perdem o `FORCE` só durante o `DELETE` e a asserção — o molde da `20260923120000`. Ficam `telemetria_eventos` (o `exp_version` separa as duas missões) e `recursos_auditoria`, que é só de inserção.
`20260925190000_retira_exp05ef1b_v1`	A retirada da primeira versão de Quem mandou isso? (**ADR-114**): `DROP` das cinco tabelas `exp05ef1b_*` e `DELETE` das linhas `exp05ef1b` em `experiencias`, `experiencias_versoes`, `liberacoes`, `progresso` e `disponibilidades`, porque o código volta com a v2.2. `liberacoes` e `progresso` perdem o `FORCE` só durante o `DELETE` e a asserção — o molde da `20260923120000`. Ficam `telemetria_eventos` (o `exp_version` separa as duas versões) e `recursos_auditoria`, que é só de inserção.

`20260926120000_arquivar_unootube`	O UnooTube ARQUIVADO, e não apagado (**ADR-116**): cria `experiencias_arquivadas` e move para lá a linha de `experiencias`, com a fotografia de turmas e estudantes; `RENAME` de `exp04ef1b_sessoes` para `backup_exp04ef1b_sessoes` (*constraints*, índices e política junto, por OID); e o código como valor vira `backup-exp04ef1b` em versões, liberações, progresso, equipes, evidência, disponibilidades e auditoria. `equipes_membros` precisa do próprio `UPDATE`: a FK dela é para `equipes(id)`, e o código ali é cópia. Fica `telemetria_eventos`. Asserção no fim.
`20260926130000_exp05ef1b_quem_mandou_isso_v2`	A v2.2 de Quem mandou isso? (**ADR-118**): `exp05ef1b_pacotes`, `_gates_turma`, `_sessoes` e `_reflexoes`, as quatro com RLS forçada; os índices únicos parciais (um pacote ativo e um rascunho por turma), os *CHECKs* de vocabulário e de ordem dos *latches*, o "Então" de 15 a 240 depois do `btrim`, e o `GRANT` à *role* de *runtime*, guardado. O DDL saiu do `prisma migrate diff` contra um banco sombra, só com os comandos da missão.
`20260926140000_exp07ef1b_a_selecao`	As quatro tabelas da A Seleção (**ADR-115**), com `escola_id NOT NULL`, RLS `FORCE`, *policy* por escola e *grant* guardado para `app_runtime`.
`20260927090000_exp07ef1b_podcast`	A coluna `podcast` em `exp07ef1b_coberturas` (**ADR-119**), `JSONB` nulo com *CHECK* de objeto; sem *backfill*, porque nulo é a leitura das falas aprovadas, e sem RLS nova — a tabela já tem a dela.
`20260927120000_exp04ef1b_jornal_da_escola`	As quatro tabelas do Jornal da Escola (**ADR-117**): `exp04ef1b_pacotes`, `_gates_turma`, `_sessoes` e `_diagramacoes`, com `escola_id NOT NULL`, RLS `FORCE` e política por escola; o índice único parcial de um pacote ativo por turma, os *CHECKs* de vocabulário, de vagas e de *latch* (equivalências com o passo), e o `GRANT` à *role* de *runtime*, guardado. Os nomes de índice e de `UNIQUE` são dados à mão: o que o Prisma inventaria para cinco colunas passa de 63 caracteres, e o Postgres o cortaria em silêncio.
`20260930120000_agente_professor`	A posse das conversas do agente de IA do professor (**ADR-126**): `agente_professor_conversas` com `escola_id NOT NULL`, RLS `FORCE`, `sessao_id` único, os *CHECKs* de título (1 a 120) e de `sessao_id`, o índice da lista e o `GRANT`, guardado. E `agente-professor` no *CHECK* `ck_experiencias_produto`.

REGRA INVIOLÁVEL

Nenhuma migração remove `escola_id` ou afrouxa RLS. Renomeação usa `RENAME`; nada de `DROP`/`CREATE` em tabela com dado. A exceção é a retirada de uma missão inteira (**ADR-113**, **ADR-114**), que apaga as tabelas dela de propósito, numa migração própria. Em produção, a migração roda como Cloud Run Job dentro da VPC **antes** do *deploy* do serviço,

com `--wait` — porque o *schema* novo tem de estar no banco antes de a revisão nova receber tráfego.

ATENÇÃO

O que o Prisma **não** modela e por isso vive na migração: políticas de RLS, *CHECK constraints*, partições, `GRANT`. Ler apenas `schema.prisma` dá uma visão incompleta do banco — é por isso que os campos correspondentes são `TEXT`/`SMALLINT` no *schema* e a validação de aplicação é feita com Zod.

8.19 As três tabelas da exp01ef1b

Tabela	O que guarda
`exp01ef1b_turma_config`	A aula, por **turma**: tema, nível de leitura, o pacote APROVADO (JSONB validado pelo mesmo *schema* da geração; nulo = banco curado padrão), o `latch_liberado` da Roda e a `tentativa` corrente. Chave `(escola_id, turma_id)`.
`exp01ef1b_runs`	A execução de um estudante numa **tentativa**: etapa (*CHECK* de vocabulário fechado), marca, o par foto+legenda escolhido, a latência de decisão (teto de 2h no *CHECK*: acima é aba esquecida) e a reflexão — que só aceita `'lupa'`, porque o Globo é *retry* e vive no contador. O reset da turma INCREMENTA a tentativa em vez de apagar: as *runs* antigas ficam imutáveis na tentativa anterior (DAD-06). Única por `(escola_id, turma_id, estudante_id, tentativa)`.
`exp01ef1b_posts_turma`	A publicação no mural coletivo, assinada pela **marca** (coruja/estrela/sol) — nome de criança não existe aqui. 1:1 com a *run* (`UNIQUE run_id`): toque duplo e *retry* de rede não criam segundo post.

REGRA INVIOLÁVEL

O congelamento da Etapa 5 NÃO é tabela própria: é o par `latch_liberado` (config da turma) × etapa `roda` (da *run*). A liberação é um `UPDATE` simples e idempotente por erro (`409 latch-ja-liberado`), sem *lock* de linha — quem "recebe o *broadcast*" é o *polling* dos tablets (**ADR-94**).

9

Contratos de integração

Há quatro fronteiras nesta plataforma cuja quebra é caríssima, porque atinge código que já está publicado e não pode ser atualizado em conjunto: o protocolo `postMessage` entre hub e experiência, o formato do evento de telemetria, o manifesto de experiências e a interface do *provider* de IA. Este capítulo especifica os quatro e a política de versionamento de cada um — e, desde 2026-09-30, um quinto, que tem quatro pontas em vez de duas: o do agente de IA do professor (Seção 9.5).

> **REGRA INVIOLÁVEL**
>
> **Mudança em contrato exige cuidado extremo e atualização deste documento.** Um *bundle* publicado há três meses no *bucket* de um contrato continua sendo servido a estudantes: ele não é reconstruído quando a API muda. Toda alteração de contrato é, por definição, uma alteração *retrocompatível* ou um *bump* de versão maior com suporte à anterior.

9.1 Protocolo `postMessage` hub ↔ experiência

9.1.1 Especificação das mensagens (protocolo v1)

Sentido	Mensagem	Carga e semântica
exp → hub	`exp:ready`	`{ protocolVersion }`. Anúncio de que o *bundle* carregou. É **reenviado** a cada 400 ms até o hub responder.

hub → exp	`hub:init`	`{ protocolVersion, token, estudante{id,nome,serie,turmaId,escolaId}, apiUrl, config }`. Resposta ao `exp:ready`; só a primeira conta. `config` é o saco aditivo do protocolo: hoje carrega `expId`, `clarityProjectId` (**ADR-60**), `obs.traceRatio` (a fração de sessões com OpenTelemetry Web, Capítulo 17) e `fuso` (o fuso da plataforma, **ADR-99**) — campo novo aqui não muda `PROTOCOL_VERSION`, porque a experiência que não o conhece o ignora. Os três são valores de *runtime* do hub, e nenhum poderia ser assado no *bundle*: ele é construído uma vez e não conhece contrato.
exp → hub	`exp:concluir`	`{ payload: { score?, dados? } }`. Conclusão da missão. O hub registra progresso.
exp → hub	`exp:token-expirando`	Sem carga. Pedido de renovação; o hub responde `hub:token`.
hub → exp	`hub:token`	`{ token }`. Novo *token*, sem recarregar o `iframe`.
exp → hub	`exp:estado`	`{ estado: {…} }`. Estado de alto nível para o *chrome* opcional do hub. **Aditivo** ao v1; o hub *não* interpreta domínio, só repassa a quem plugou um *callback*.
exp → hub	`exp:erro`	`{ mensagem, hash }`: a experiência quebrou de um jeito que ela não trata (`window.onerror`, `unhandledrejection`). Só para o hub TROCAR A TELA (“A missão travou”, recarregar); o registro do erro vai da SDK direto à API (`POST /observabilidade/erros-cliente`, **ADR-58**). Aditivo, v2 continua; hub antigo descarta com o aviso de *schema*.
hub → exp	`hub:pause` / `hub:resume`	Sem carga. Emitidos por visibilidade e foco da página do hub.

NOTA — o que o hub faz com o `exp:estado`

O `ExpHost` guarda o último estado publicado e reconhece dois campos, ambos da exp08ef1b: `acao: "sair"` (a sessão é do HUB, em *cookie* `httpOnly` fora do alcance do *iframe*, então a experiência pede e o hub executa) e `fase`, que acende a moldura dos posts perdidos e — com `passo: "relatorio"` — solta o aparelho para o relatório ocupar a área inteira (ADR-41, 18.42).

Duas consequências de o hub SUBSTITUIR o estado inteiro a cada mensagem: publicar só o `passo` apagaria a `fase`, então quem publica leva as duas; e é a própria troca de fase seguinte que desfaz o estado anterior, sem mensagem de “desligar”. As outras experiências publicam estado (`{ tela }` na exp04ef1b, `{ passo, tentativa }` na exp02em1b) e o hub ignora — o canal é genérico de propósito.

9.1.2 O *handshake*, passo a passo

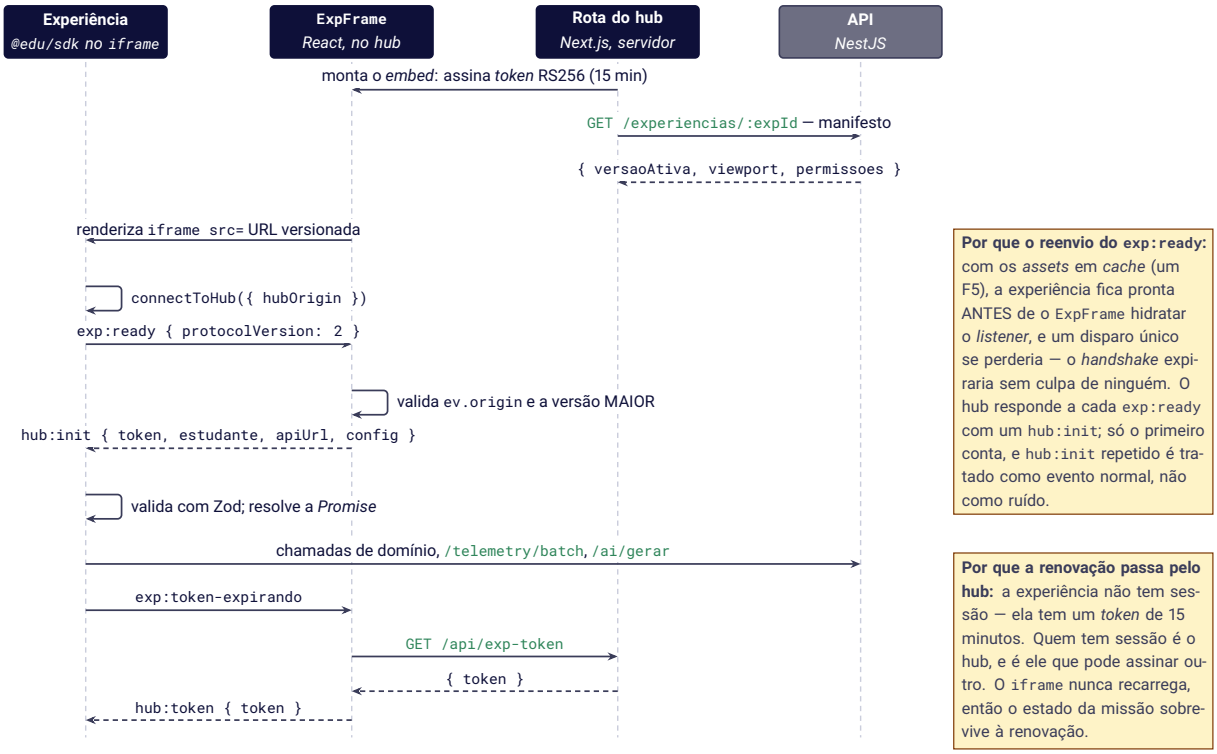


Figura 9.1 – Sequência do *handshake* e da renovação de *token*. O `targetOrigin` é explícito nos dois sentidos, e as duas pontas descartam mensagem de *origin* divergente antes de olhar o conteúdo.

9.1.3 Regras invioláveis do protocolo

REGRA INVIOLÁVEL

1. **`targetOrigin` explícito, nunca `"*"`.** O hub usa a *origin* derivada da *mesma* URL que colocou no `iframe`; a SDK usa o `hubOrigin` que recebeu por *query string*. Com `"*"`, o *token* iria para qualquer documento que conseguisse virar o pai do *frame*.
2. **Aquela *query string* é entrada NÃO CONFIÁVEL, e passa por `normalizarHubOrigin()`.** O *bundle* é público: quem embeda a experiência num `iframe` próprio escreve o parâmetro, e com ele escolheria de quem se aceita o `hub:init` e para onde vai tudo o que sai depois. A função reduz o valor a uma origem canônica, recusa esquema fora de http(s), credencial e origem opaca, exige hub em https quando a experiência está em https, e confere contra `location.ancestorOrigins[0]` — a origem REAL de quem embedou, que VENCE o parâmetro (ADR-46).
3. **Toda mensagem pós-*init* é validada com Zod nos DOIS lados.** Mensagem inválida é descartada com aviso, nunca despachada crua aos *listeners*.
4. **Versão MAIOR verificada nos dois lados; incompatível falha alto.** O hub não envia `hub:init` e registra erro no console; a experiência expira o *handshake* e mostra a própria mensagem de erro.

5. **O *token* vive só em memória** na experiência. Nunca `localStorage`, nunca *cookie*.
6. **Nunca quebrar compatibilidade sem *bump* de versão maior e suporte à anterior.**

Listing 9.1 – O núcleo da validação, em `packages/sdk/src/bridge.ts`

```
function handler(ev: MessageEvent) {
  if (ev.origin !== hubOrigin) return;           // regra 1: valida origin

  if (!conectado) {
    const init = HubInitSchema.safeParse(ev.data);
    if (!init.success) return;                   // antes do init, tudo mais e ruido
    // Versao MAIOR precisa bater — hub e experiencia de geracoes diferentes
    // falham ALTO, nao silenciosamente.
    if (Math.trunc(init.data.protocolVersion) !== PROTOCOL_VERSION) {
      reject(new Error(`Protocolo incompativel: hub v${init.data.protocolVersion}, ` +
                       `experiencia v${PROTOCOL_VERSION}`));
      return;
    }
    conectado = true;
    resolve({ init: init.data, send, onMessage });
    return;
  }

  // hub:init repetido (resposta a um exp:ready reenviado) nao e ruido nem erro.
  if ((ev.data as { type?: unknown })?.type === "hub:init") return;

  const msg = HubParaExpSchema.safeParse(ev.data);
  if (!msg.success) {
    console.warn("[sdk] mensagem do hub descartada (schema):", ev.data?.type);
    return;                                      // regra 2: descarta, nao despacha
  }
  listeners.forEach((cb) => cb(msg.data));
}
```

9.1.4 Política de evolução

Tipo de mudança	Versão	Regra
Nova mensagem opcional (como `exp:estado`)	v1, aditivo	Permitido sem *bump*: hub antigo ignora mensagem que não conhece, e experiência antiga nunca recebe a nova sem pedir.
Novo campo opcional em `hub:init`	v1, aditivo	Permitido. O esquema Zod tem `.default()` ou `.optional()`, e a experiência antiga simplesmente não o lê.
Campo existente muda de tipo ou semântica	**v2**	Exige *bump* de versão maior *e* que o hub continue respondendo v1 enquanto houver *bundle* v1 publicado em algum contrato.
Remoção de mensagem ou de campo	**v2**	Mesma regra. A remoção só é efetiva quando o último *bundle* v1 tiver sido republicado.

ATENÇÃO

`hub:pause` e `hub:resume` existem porque **esconder um `iframe` não dispara `visibilitychange` nele**. Sem essas mensagens, o cronômetro de *dwell* da experiência correria com o estudante de costas — e *dwell* é a medida central do motor de *feed* da exp08ef1b. O hub escuta `visibilitychange` e `blur`/`focus` da janela, porque trocar de janela ou de aplicativo não muda `visibilityState`.
Uma exceção, e ela custou um bug: clicar em qualquer lugar **dentro** do `iframe` move o foco para a experiência, e o navegador dispara `blur` no documento de cima. Isso *não* é o estudante saindo — e como o `focus` correspondente não volta enquanto ele continua ali, um `hub:pause` nesse caso é definitivo. Na exp04ef1b o vídeo parava no primeiro toque em "Salvar" e não voltava mais. Quando o foco foi para a nossa experiência, `document.activeElement` é o próprio `iframe`, e `ExpFrame.tsx` não emite nada.

9.2 Manifesto de experiências

O manifesto é o contrato entre o CI, a API e o hub. Ele existe em duas cópias que devem concordar: o arquivo `public/experiencia.json` dentro do *bundle*, e a linha da tabela `experiencias` na API — que o CI escreve lendo o arquivo.

Listing 9.2 – `apps/experiencias/app-70-vila-do-lume/public/experiencia.json`

```
{
  "id": "app-70-vila-do-lume",
  "nome": "Vila do Lume — A Noite Longa",
  "versao": "1.1.3",
  "produto": "entre-redes",
  "series": [],
  "capabilities": ["telemetria"],
  "permissoes": ["pointer-lock"],
  "verbos": {
    "attempted":  "entrou na vila (partida aberta)",
    "interacted": "acendeu fogueira, gastou tocha, gravou na Pedra dos Nomes ou respondeu a uma mensagem",
    "progressed": "acendeu mais uma das 8 fogueiras",
    "completed":  "acendeu as 8 fogueiras e reacendeu o Fogo Sagrado"
  }
}
```

Campo	Tipo	Semântica
`id`	*string*	Slug da experiência; é a chave primária da tabela e o primeiro segmento do caminho no *bucket*.
`nome`	*string*	Nome de exibição.
`versao`	*semver*	Vem do `package.json` da experiência. É o segundo segmento do caminho no *bucket*, o que torna o *bundle* imutável.

`series`	*string[]*	Séries em que a experiência se aplica. **Vazio significa TODAS as séries**, não nenhuma — ver o alerta abaixo.
`unidade`	*1–4?*	A unidade do ano a que a missão pertence. Ausente na transversal e na ferramenta de professor: elas não moram numa unidade. É o que ORDENA a grade da sala (**ADR-69**).
`produto`	*slug*	A que PRODUTO a experiência pertence: `entre-redes`, `plano-de-aula`, `plano-de-ensino` ou `bem-estar`. É o que a põe no painel da Unooria e o que lhe dá o TETO do produto (**ADR-87**) — e é por ele que a composição produto → experiências é DERIVADA do manifesto, em vez de ser uma lista mantida à mão que ninguém atualiza quando a experiência seguinte nascer. Cobrado por `tests/manifestos-declaram-produto.spec.ts`, e **não** pelo Zod da rota: um 400 no `PUT /experiencias/:expId` mataria a ativação DEPOIS de os *bundles* já terem subido.
`capabilities`	*string[]*	`telemetria`, `progresso`, `ia`, `avaliacao`, `tempo-real`. Declara o que a experiência usa da plataforma.
`viewport`	`{largura, altura}?`	Presente ⇒ o hub embeda dentro do `DispositivoFrame` (casca de aparelho com escala). A exp08ef1b e a exp06ef2b são 375×812; a exp04ef1b *não* tem `viewport`, porque foi desenhada para *desktop*. Quem desenha o aparelho é o hub — a experiência **não** empilha uma casca própria.
`publico`	`estudante|professor`	`professor` ⇒ ferramenta do professor, e `/exp/*` responde **404** para ela.
`permissoes`	*string[]*	Permissões que o *iframe* daquela experiência precisa: `pointer-lock` (app-70), `forms` (exp02em1b, exp08ef4b, exp05ef1b) e `downloads` (app-90, app-92), traduzidos pelo `ExpFrame` nos tokens `allow-pointer-lock`, `allow-forms` e `allow-downloads`. Vazio (o *default*) = nenhuma. **O vocabulário é fechado em três lugares** — o Zod da api, o CHECK de `experiencias.permissoes` e o `ExpFrame` —, e `tests/permissoes-de-iframe.spec.ts` falha se os dois primeiros divergirem: a divergência só aparece em produção, como 500 no `PUT /experiencias/:expId`.
`verbos`	*objeto*	Documentação dos verbos de telemetria que a experiência emite. Não é lido por código; é contrato de leitura para quem analisa os dados.

REGRA INVIOLÁVEL

`series: []` significa **todas as séries**, não nenhuma: é assim que uma experiência *transversal* se declara (é o caso da app-70, que não pertence a ano nenhum). Desde a sala de aula (**ADR-69**) este campo é **lido**: é ele que decide se a missão entra na grade de uma turma, e vazio continuar querendo dizer "todas" deixou de ser hipótese e virou o que sus-

tenta a transversal aparecer em toda sala. O contrário faz a experiência desaparecer da tela sozinha, sem erro em lugar nenhum.

Campo novo no `experiencia.json` tem de ser acrescentado ao `publicar-manifesto.mjs` também — foi assim com `permissoes`, com `unidade` e com `produto`. Esquecer é INVISÍVEL em desenvolvimento (onde o hub lê o arquivo do *bundle*) e quebra em produção, onde ele lê a coluna. E a cadeia tem CINCO pontos, não dois: o Zod da rota, a coluna, o `publicar-manifesto.mjs`, a *interface* `Manifesto` do *seed* (`prisma/catalogo-e-liberacoes.ts`) e o `_template` — este último porque `pnpm new:exp` copia o arquivo dele, e um *template* incompleto faz TODA experiência nova nascer errada.

9.2.1 Como o hub resolve a URL do `iframe`

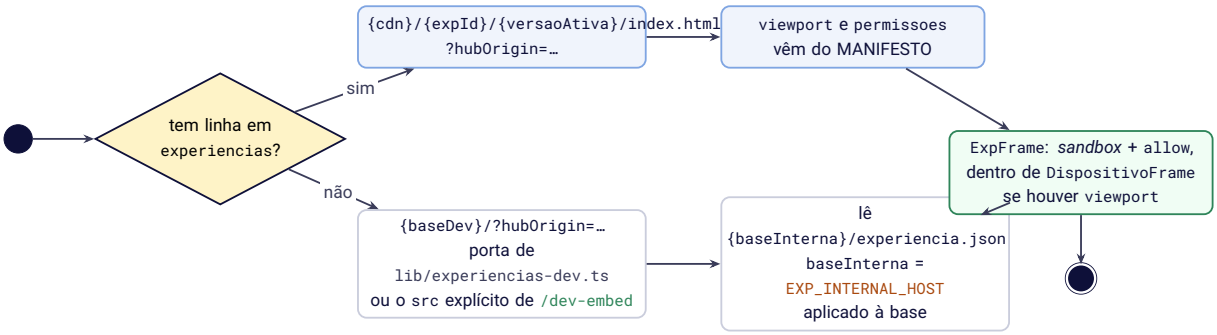


Figura 9.2 – Resolução da URL do *embed* e da casca de aparelho. Os dois caminhos convergem: mesmo sem manifesto, o hub obtém `viewport` e `permissoes` lendo o `experiencia.json` servido ao lado do `index.html` do *bundle*.

ATENÇÃO

A leitura do `experiencia.json` do *bundle* **não é só de desenvolvimento**. Antes era, e uma *stack* com a tabela `experiencias` vazia — *compose* local, ou ambiente novo antes do primeiro *publish* do CI — servia a exp08ef1b *esticada na janela* em vez de dentro do aparelho de 375×812. Pelo mesmo motivo, a app-70 abriria sem `allow-pointer-lock`: o jogo monta, a câmera não anda, e nada no console diz por quê.

NOTA

Essa leitura é *server-side*, e `NEXT_PUBLIC_CDN_EXP_URL` é o endereço do **navegador** — de dentro de um contêiner, `localhost:5173` é o próprio contêiner. Daí as duas variáveis: `EXP_INTERNAL_HOST` troca só o *host* e **preserva a porta** (é o que serve em desenvolvimento, onde cada experiência tem um *dev server* numa porta diferente), e `EXP_INTERNAL_URL` é um endereço fixo, que só serve quando há *uma* origem de *bundles* — a CDN, em produção.

9.2.2 Publicação e reversão

Operação	Comando
Publicar (feito pelo CI)	`gcloud storage rsync -r {dir}/dist gs://{bucket}/{exp}/{ver}/` + `node apps/hub/scripts/publicar-manifesto.mjs {dir}`
Reverter	`node apps/hub/scripts/publicar-manifesto.mjs <exp> --versao x.y.z`

O *script* assina um *token* de cinco minutos com a chave de assinatura e chama `PUT /experiencias/:expId` como `admin-contrato`. A API valida esse *token* contra o JWKS que o *hub* publica — o que é a razão de o *workflow* publicar as experiências *depois* do hub.

9.3 Contrato de telemetria

O formato do evento é inspirado em xAPI (Advanced Distributed Learning Initiative, 2023) e simplificado ao que a plataforma efetivamente usa.

Listing 9.3 – Evento de telemetria (canal analítico)

```
{
  "eventId":    "uuid",          // gerado pela SDK; chave de dedupe
  "ts":         "ISO-8601",      // relogio do cliente
  "estudanteId":    "...",           // SOBRESCRITO pelo servidor, a partir do token
  "escolaId":   "...",           // SOBRESCRITO pelo servidor
  "expId":      "exp08ef1b",
  "expVersion": "1.2.0",
  "sessaoId":   "uuid",
  "verbo":      "completed | attempted | interacted | scored | progressed | ...",
  "objeto":     "fase-3",
  "resultado":  { },             // livre, por experiencia
  "contexto":   { }              // livre, por experiencia
}
```

Cláusula	Regra
Quem preenche o quê	A SDK preenche tudo **exceto** `verbo`, `objeto` e `resultado` — esses são a única coisa que a experiência informa.
Identidade	`estudanteId` e `escolaId` enviados pelo cliente são **descartados**: o servidor os deriva do *token*. É anti-*spoof* e, ao mesmo tempo, garantia de que o cliente não *pode* colocar PII no lugar do id.
Lote	Até 50 eventos por requisição, *flush* a cada 10 s ou quando o *buffer* enche.
Fechamento de página	No `pagehide`, `POST /telemetry/beacon` com o *token* **no corpo** — `sendBeacon` não envia cabeçalho.
Idempotência	`ON CONFLICT (event_id, ts) DO NOTHING`. O Pub/Sub entrega *at least once*, então reentrega é rotina, não exceção.

Documentação dos verbos	Cada experiência declara seus `verbos` no manifesto e no próprio README. Verbo não documentado é dado que ninguém consegue interpretar seis meses depois.

O detalhamento dos dois canais — por que são dois, e o que acontece em cada um — é o Capítulo 10.

9.4 Contrato do *provider* de IA

Listing 9.4 – `apps/api/src/ai/provider.ts` — a interface neutra de *vendor*

```ts
export interface AiRequest {
  system: string;
  user: string;
  maxTokens?: number;
  /** Familia de modelo, NEUTRA de vendor. `padrao` e o modelo do dia a dia;
   *  `longo` e o de saida grande (planos anuais). Quem chama nao sabe qual e. */
  modelo?: "padrao" | "longo";
  temperatura?: number;
}

export interface AiResult {
  texto: string;
  tokensUsados: number;
  /** `STOP` (natural) ou outro motivo do vendor (`MAX_TOKENS`, `SAFETY`...).
   *  Aditivo: quem nao liga, ignora. Quem gera texto longo TEM de olhar. */
  finishReason?: string;
}

export interface AiFerramenta {
  nome: string;
  descricao: string;
  /** Reusa o AiJsonSchema neutro. `description` por PROPRIEDADE nao e
   *  comentario: e por ela que o modelo descobre o que pode mandar. */
  parametros: AiJsonSchema;
}

export interface AiConversaRequest {
  system: string;
  historico: Array<{ papel: "estudante" | "agente"; texto: string }>;
  ferramentas?: {
    declaracoes: AiFerramenta[];
    executar: (nome: string, args: Record<string, unknown>) => Promise<unknown>;
  };
  /** Motivos para RECUSAR a fala. Motivo derruba a tentativa e o modelo tenta
   *  de novo sabendo por que. As regras sao do dominio; o laco e do provider. */
  validar?: (texto: string) => string[];
  socorro?: string;              // o que pedir na ULTIMA tentativa
  maxVoltas?: number;            // teto de idas e voltas de ferramenta (6)
  tentativas?: number;           // default 3
}

export interface AiSpeechRequest {
  texto?: string;                                  // monologo
  turnos?: Array<{ falante: string; texto: string }>; // dialogo
  /** Timbre por falante, NEUTRO de vendor: `jovem-feminino`, `idoso-masculino`...
```

```ts
   *  Quem chama nunca escolhe a voz do catalogo do fornecedor. */
  falantes: AiFalante[];
  direcao?: string;              // atuacao: "tom assustado, voz tremula"
  idioma?: string;               // BCP-47, default pt-BR
}

export interface AiJsonRequest<T> {
  prompt: string;                // NAO `user`: o pedido estruturado tem nomes proprios
  system?: string;
  responseSchema: AiJsonSchema;  // NAO `schema`
  /** Valida e normaliza; lancar faz REGERAR com o motivo no prompt. */
  validar: (bruto: unknown) => T;
  temperatura?: number;
  tentativas?: number;
  /** Teto de saida. Ausente = o teto do modelo. Uma semana de aulas passa
   *  de 5k tokens, e JSON truncado nao chega como MAX_TOKENS legivel. */
  maxTokens?: number;
  modelo?: "padrao" | "longo";   // mesma familia neutra de AiRequest
  /** Quanto o vendor cobrou, UMA VEZ POR TENTATIVA -- inclusive as que
   *  `validar` rejeitou: foram pagas igual. Quem chama `temSaldo` TEM de
   *  passar isto; sem ele nao ha de onde tirar o numero (ADR-74). */
  aoUsarTokens?: (tokens: number) => void;
}

export interface AiProvider {
  generate(req: AiRequest): Promise<AiResult>;
  /** Saida JSON estruturada, com retry e reforco do erro anterior no prompt. */
  generateJson<T>(req: AiJsonRequest<T>): Promise<T>;
  /** Bytes originais (jpeg/png). Falha => quem chama cai no fallback procedural.
   *  A proporcao e PEDIDA ao modelo (`opcoes.proporcao`), nunca recortada depois. */
  generateImage(prompt: string, opcoes?: AiImagemOpcoes): Promise<Buffer>;
  /** MP3 pronto para <audio src>. Falha => quem chama cai na transcricao. */
  generateSpeech(req: AiSpeechRequest): Promise<Buffer>;
  /** Conversa multi-turno com ferramentas (ADR-23). `tokensUsados` soma TODAS
   *  as voltas e TODAS as tentativas, inclusive as rejeitadas: e o que foi pago. */
  conversar(req: AiConversaRequest): Promise<AiResult>;
}

export const AI_PROVIDER = "AI_PROVIDER";   // token de injecao do Nest
```

REGRA INVIOLÁVEL

Nenhum módulo fora de `src/ai` conhece o *vendor*. Trocar de provedor é criar um arquivo novo (`src/ai/<vendor>.provider.ts`) e registrá-lo no módulo por `AI_PROVIDER`. Implementações atuais: `vertex.provider.ts` (*default*) e `mock.provider.ts` (testes e desenvolvimento sem GCP).

O corolário aparece em `AiTimbre` e `AiProporcao`: quem chama pede “voz de senhora idosa” e “retrato 4:5”, e é o *provider* que traduz para o nome de voz e o `imageConfig` do fornecedor do dia. Um id de voz do catálogo do Google atravessando o domínio seria *vendor* vazando para dentro da missão.

NOTA

`AiJsonSchema` é um subconjunto deliberadamente pequeno de JSON Schema (`object`, `array`, `string`, `number`, `boolean`, `enum`), e cada *provider* o traduz para o formato do seu modelo. O campo `validar` do `AiJsonRequest` recebe a saída bruta e *lança* quando ela não serve — o que faz a tentativa ser regerada com o erro anterior reforçado no *prompt*. Validação como parte do contrato, não como verificação posterior.

REGRA INVIOLÁVEL

`generate` e `generateJson` não compartilham os nomes dos campos, e confundi-los não é erro de digitação: `prompt`/`responseSchema` contra `user`/`schema`. Um `as never` sobre o objeto literal cala o compilador, e o pedido chega ao *provider* sem `responseSchema` — a montagem do corpo estoura *antes* da requisição, as três tentativas queimam em segundos e o professor vê "Falhou" com zero token gasto. Foi o que aconteceu na app-92 (plano de aulas). **Nunca use `as never` para satisfazer este contrato**: é justamente ele que o typecheck existe para conferir.
O segundo ensinamento é do laço de *retry*: falha de **transporte** não é falha de **conteúdo**. Reescrever o *prompt* não devolve permissão nem conserta projeto errado, então `4xx` (menos `429`) falha na primeira tentativa e a mensagem preserva o *status*; `429` e `5xx` são passageiros e seguem no laço. Sem essa distinção o corpo de erro do *vendor* — que não tem `candidates` — virava "resposta vazia", e a causa real morria atrás de uma mensagem genérica.

9.5 Contrato do agente de IA do professor

O agente (**ADR-126**) é o único contrato da plataforma com **quatro pontas**: o agente em Python (`apps/agente-professor`), a api (`src/agente-professor` + `src/ai/agent-engine.client.ts`), o hub (o chat flutuante e a rota de *stream*) e a infraestrutura (o repositório de fundação). A fonte normativa é `docs/regras/agente-professor.md`; esta seção o resume para quem lê a arquitetura, e os *payloads* estão no Apêndice D.

Listing 9.5 – `apps/api/src/ai/agente-professor.port.ts` — a segunda porta de `src/ai` (abreviado)

```
export interface EventoAgente {        // um Event do ADK, NORMALIZADO
  autor: string;                        // "user" ou o nome do agente
  parcial: boolean;                     // partial=true do ADK em modo SSE
  textos: string[];                     // sem as partes de raciocinio
  chamadasDeFerramenta: Array<{ nome: string; args: Record<string, unknown> }>;
  respostasDeFerramenta: Array<{ nome: string; resposta: unknown }>;
  estadoDelta?: Record<string, unknown>;   // por aqui chega titulo_conversa
  tokens?: number;                      // usage_metadata; so vale nos NAO parciais
  erro?: { codigo: string; mensagem?: string };
}

export interface AgenteProfessorPort {
  readonly modo: "agent-engine" | "local" | "mock" | "desligado";
```

```
  criarSessao(usuarioId: string, estado: Record<string, unknown>,
              expiraEm?: Date): Promise<{ sessaoId: string }>;
  lerSessao(usuarioId: string, sessaoId: string): Promise<SessaoAgente>;
  apagarSessao(usuarioId: string, sessaoId: string): Promise<void>; // idempotente
  renovarSessao?(sessaoId: string, expiraEm: Date): Promise<void>;  // so agent-engine
  enviar(usuarioId: string, sessaoId: string, texto: string,
         signal: AbortSignal): AsyncIterable<EventoAgente>;
}
```

REGRA INVIOLÁVEL

Só `agent-engine.client.ts` conhece o engine, pela mesma regra do `AiProvider`. É porta própria, e não um método a mais no `AiProvider`, porque modelo, ferramentas e instrução moram no agente: a api só conduz a sessão. **Um cliente, dois alvos**: `agent-engine` (`:query` e `:streamQuery?alt=sse` do Agent Engine, com o ADC) e `local` (o *shim* de `apps/agente-professor`, que expõe O MESMO contrato em `/api/reasoning_engine` e `/api/stream_reasoning_engine`, sem *token*). `mock` é o da suíte e2e; `desligado` é o padrão.

9.5.1 O que a api pede ao engine

Chamada	Uso
`async_create_session` (`:query`)	Cria a sessão com `user_id = users.id`, o estado `{nome_professor}` e `expire_time` distante pelos `**kwargs`. Se o engine recusar o campo, a criação repete sem ele (TTL padrão de 365 dias).
`async_get_session` (`:query`)	Monta o histórico da tela (só o texto do professor e o texto FINAL do agente) e lê o título quando o *stream* terminou sem ele. "Não existe" vira 404 e apaga a linha.
`async_delete_session` (`:query`)	Apagar. Idempotente: sessão que já não existe não é erro.
`async_stream_query` (`:streamQuery?alt=sse`)	O turno, com `run_config.streaming_mode = "sse"`. A resposta vem em linhas `data:` *ou* NDJSON — o cliente decide por linha, e linha que não é JSON é contada e vai a log, sem derrubar o turno.
`PATCH v1beta1/…/sessions/{id}?updateMask=expireTime`	A renovação depois de cada turno entregue. É a API NATIVA: o `:query` do ADK não tem método de atualizar sessão, e a SA da api precisa de `sessions.update`. Falha é só log (`agente-professor.renovacao-falhou`).

9.5.2 As chaves de estado e as ferramentas

Chave	Quem escreve	Quem lê

`nome_professor`	a api, na criação (primeiro nome de `users.display_name`)	a instrução do agente
`titulo_conversa`	**o agente**, com o próprio LLM, depois do primeiro turno	a api (no `actions.state_delta` do *stream*, ou numa leitura da sessão) e o hub

As duas ferramentas são de *data store* (**ADR-127**). `buscar_habilidades` consulta o estruturado com filtro exato e paginação; `buscar_acervo` devolve trechos dos PDFs, e as chaves de cada trecho — `titulo`, `pagina_inicio`, `pagina_fim`, `trecho`, `fonte_id` — **são lidas pela api** para o evento `fontes`. Renomear uma delas sem mudar a api junto faz as fontes sumirem da tela, sem erro.

9.5.3 O que a api emite ao navegador

`POST /agente-professor/conversas/:id/mensagens` `{ texto }` (1 a 4.000 caracteres) responde `text/event-stream` (**ADR-128**):

`delta`	`{ texto }` — dos eventos parciais, e do final agregado SÓ no que os parciais não cobriram (a recusa de escopo chega num evento final sem parcial antes, com `custom_metadata.fora_do_escopo = true`).
`fontes`	`{ fontes: [{ titulo, paginaInicio, paginaFim }] }`, dos retornos de `buscar_acervo`, sem repetição.
`titulo`	`{ titulo }`, só quando muda.
`aviso`	`{ codigo: "habilidade-inexistente", mensagem }`, um por código citado que `habilidadePorCodigo` não conhece.
`fim`	`{}`.
`erro`	`{ codigo, mensagem }`, com `codigo` em `QUOTA`, `RATE_LIMIT`, `AGENTE_INDISPONIVEL` ou `TEMPO_ESGOTADO`.

Mais : `ping` a cada 15 s. O *stream* termina em `fim` **ou** em `erro`; fontes, título, avisos e `fim` saem depois do último evento do agente, nessa ordem. Tudo o que recusa responde JSON **antes** de abrir o *stream*.

> **ATENÇÃO**
>
> **Três armadilhas medidas, que nenhum tipo pega.**
>
> 1. **O ADK em SSE emite os parciais E um evento final agregado que REPETE o texto inteiro.** Repassar os dois duplica a resposta na tela; o `DeduplicadorDeTexto` deixa passar do final só o que faltou, por segmento (uma chamada ao modelo).

2. `:streamQuery?alt=sse` **responde ora** `data:`**, ora NDJSON** (*issue* aberta no `python-aiplatform`). O *parser* aceita os dois, por linha.
3. ***Tokens* só dos eventos NÃO parciais**, somados: um turno com ferramenta faz mais de uma chamada ao modelo, e os parciais repetem o `usage_metadata`. A guarda de escopo e o título somam os deles ao primeiro evento não parcial seguinte.

Política de evolução. O `ContratoInterceptor` não confere o SSE: quem confere cada `data:` é o teste, contra `DADOS_POR_EVENTO` de `contrato-de-rotas.ts`, e **evento novo entra lá antes de sair da api**. O ADK está fixado no `uv.lock` (`~=2.10.0`); subir de versão é refazer as medições de `tests/test_e2e.py` do agente, porque é delas que saem as três armadilhas acima. Chave de estado, nome de ferramenta e chave de trecho mudam nas quatro pontas no mesmo PR.

9.6 Contrato entre duas experiências

Quando uma experiência precisa do resultado de outra, a comunicação nunca é direta — módulos de domínio não se conhecem (Seção 5.5). O contrato é uma tabela nomeada com os dois números e um *endpoint* de fronteira.

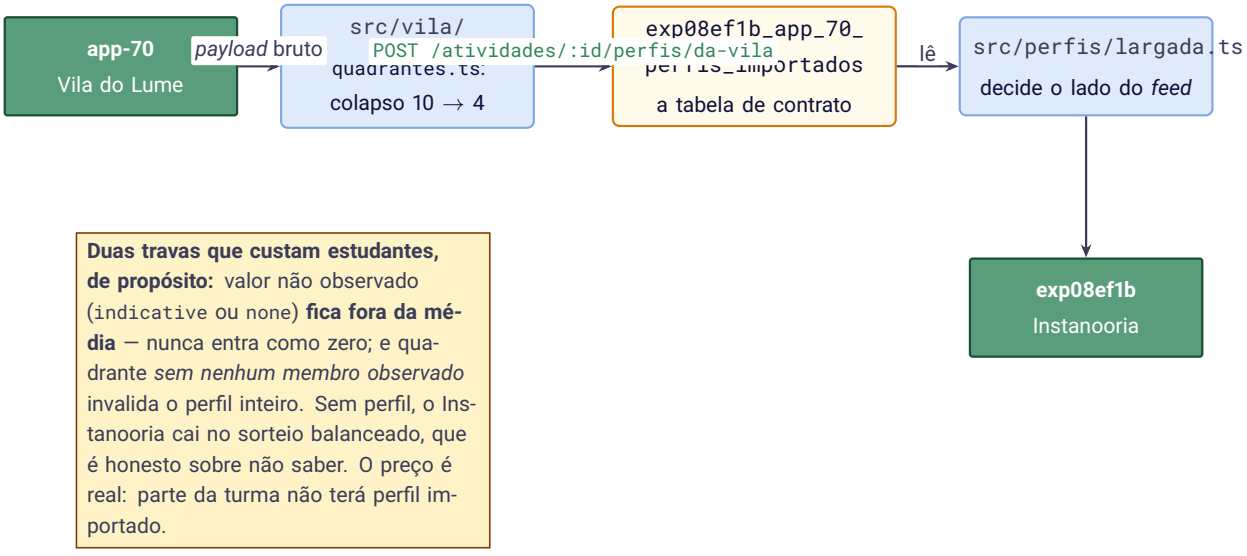


Figura 9.3 – O único acoplamento entre experiências, e como ele é explícito: uma tabela cujo nome carrega os dois números, escrita por um *endpoint* de fronteira acionado pelo professor.

REGRA INVIOLÁVEL

O disparo é do **professor**, e `lado_forcado` é **preservado** no *upsert*. Importar sozinho — ou levar o objeto inteiro no `update` — desfaria em silêncio uma redistribuição de turma que o professor fez antes da aula. `PERFIL_IMPORT_TOKEN` e `POST /perfis/importar` continuam vivos apenas como compatibilidade com a ponte externa original.

10

Telemetria

DECISÃO DE PROJETO

Dois canais de telemetria, não um. A plataforma tem dois fluxos de evento com garantias, latências e destinos diferentes, e confundi-los quebra um dos dois:

- o canal **operacional** (motor), *síncrono e transacional*, lido em tempo quase real para curar o próximo bloco de *feed*;
- o canal **analítico** (SDK), *assíncrono e em lote*, para análise posterior.

10.1 Por que dois canais

Aspecto	Operacional (motor)	Analítico (SDK)
Rota	`POST /runs/:id/eventos`, `GET /runs/:id/feed`	`POST /telemetry/batch`, `/beacon`, `/ingest`
Transporte	HTTP direto, dentro da transação	HTTP → Pub/Sub → *push* → HTTP
Latência	Milissegundos; a próxima tela depende dela	Segundos a minutos; ninguém espera
Ordenação	**Garantida** pela transação e pelo *lock* da `run`	Sem garantia
Destino	`exp08ef1b_events`	`telemetria_eventos` (particionada)
Entrega	Exatamente uma vez (transacional)	*At least once*; *dedupe* por `(event_id, ts)`

Quem lê	O motor de curadoria, o painel do professor, o dossiê	Ninguém, hoje — é dado de análise futura

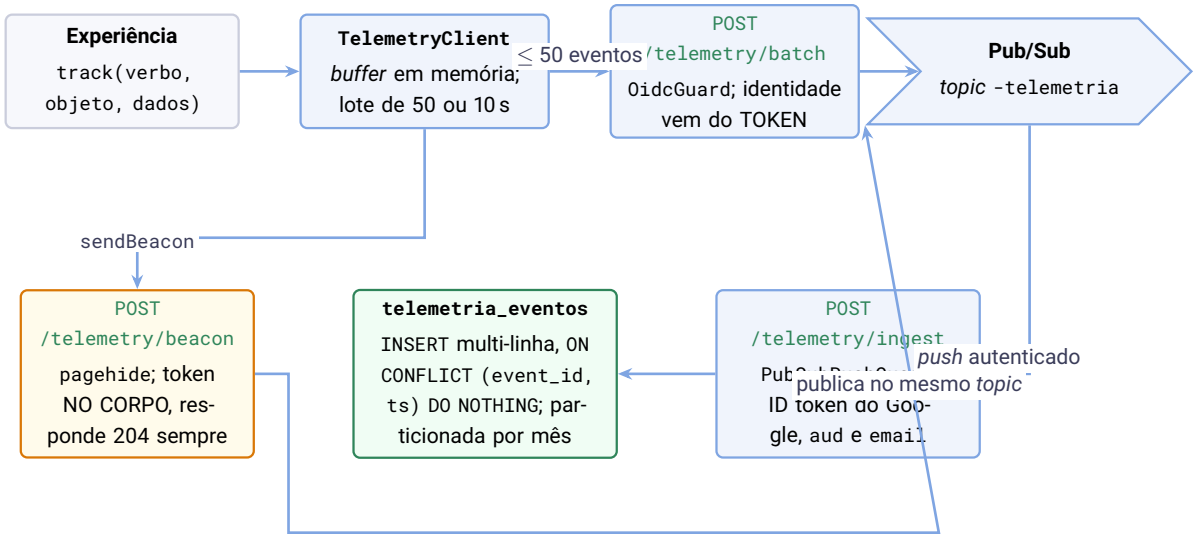
REGRA INVIOLÁVEL

***Dwell* NUNCA passa pelo Pub/Sub.** A latência e a falta de ordenação quebrariam a adaptação do *feed* e o *lock* por `run`: o motor precisa saber, no instante em que cura o próximo bloco, exatamente quais posts o estudante viu e por quanto tempo. Um evento que chega dois minutos depois, fora de ordem, produziria uma curadoria baseada em passado errado — e o estudante não teria como perceber.

NOTA

A recíproca também vale, e é o que justifica o segundo canal: mandar *tudo* pelo caminho transacional colocaria, no pico de quatro mil simultâneos, um `INSERT` sincronizado por interação de estudante dentro da mesma transação que serve a tela. O canal analítico existe para absorver volume sem competir com o que o estudante está esperando.

10.2 O canal analítico, ponta a ponta



Onde há perda possível, e só ali: entre o `track()` e o *flush*. Se o processo do navegador morrer com o *buffer* cheio — aba fechada à força, queda de energia — aqueles eventos somem. O `sendBeacon` do `pagehide` cobre o fechamento normal. A partir do Pub/Sub, a entrega é **durável**: é exatamente por isso que um *buffer* em Redis não substituiria o Pub/Sub — ele reintroduziria a perda que o Pub/Sub eliminou.

Por que o *beacon* responde 204 sempre: o navegador já descartou a página e ninguém lê o corpo. Erro aqui só serviria para poluir o log do cliente. Falha de verificação de *token* vira descarte silencioso — com aviso no servidor, não no cliente. O *token* vai no **corpo** porque `sendBeacon` não envia cabeçalho, e isso é limitação da API do navegador, não escolha.

Figura 10.1 – O canal analítico ponta a ponta. Os únicos eventos que podem se perder são os que estão no *buffer* em memória do navegador no instante em que o processo morre; do Pub/Sub em diante a entrega é durável.

10.3 A rota de ingestão e sua tranca

`POST /telemetry/ingest` é a rota mais delicada da API: ela grava dado de estudante, o `escolaId` vem no corpo (já enriquecido pelo `/batch`) e ela **não passa pelo `OidcGuard`** — porque quem chama é o Pub/Sub, não um usuário, e não existe *token* do hub nessa chamada.

REGRA INVIOLÁVEL

Sem `PubSubPushGuard`, a rota seria um *endpoint* público que escreve telemetria falsa em qualquer escola. O *guard* verifica **três** coisas no ID token que o Pub/Sub assina:

1. **emissor Google** (`accounts.google.com`), pelo JWKS público;
2. `aud` igual a `TELEMETRIA_PUSH_AUDIENCE`;
3. `email` igual a `TELEMETRIA_PUSH_SA`.

As três juntas é que fazem a prova. A assinatura sozinha só diz “algum projeto GCP”, que não é ninguém em particular.
Falha fechada: sem as duas variáveis, a rota rejeita tudo com 401 — nunca fica aberta. O Terraform casa os dois lados: a *subscription* é criada com `oidc_token` dessa mesma conta de serviço e audiência.

DECISÃO DE PROJETO

***Push subscription* para a própria API, e não um *worker* dedicado.** O desenho original previa `{contrato}-telemetry-worker` como serviço próprio. A *push subscription* entrega o mesmo resultado com uma peça a menos para operar — um serviço a menos com *deploy*, log, escala e *alerting* próprios. O *worker* separado volta se o volume justificar isolar o consumo do tráfego de estudante.

10.4 Destino: PostgreSQL, não BigQuery

DECISÃO DE PROJETO

O canal analítico grava no mesmo PostgreSQL (decisão de 2026-08-27, **ADR-14**).
Avaliou-se mandar o canal analítico para o BigQuery por *subscription* nativa do Pub/Sub. Não foi, por dois motivos concretos: (a) os relatórios do professor **não leem** `telemetria_eventos` — eles leem `exp08ef1b_events`, o canal síncrono do motor —, então o BigQuery não resolvia problema nenhum *hoje*; e (b) criava uma segunda plataforma de dados para operar, com IAM, custo e ferramentas próprios.
Ficando no mesmo banco, se um relatório precisar desse dado um dia, é um `JOIN` local com `turmas` e `users` — e é uma plataforma de dados a menos.

O que sustenta essa escolha na escala alvo são duas medidas, e nenhuma delas é um serviço novo:

10.4.1 `INSERT` multi-linha

Listing 10.1 – O gargalo real, e a correção (`src/telemetry/telemetry.controller.ts`)

```
// UM INSERT multi-linha, nao um por evento. E o gargalo real deste caminho:
// um lote de 50 eventos custava 50 round-trips ao banco, e no pico da manha
// (~4k estudantes simultaneos) isso e ~800 INSERTs/s numa instancia pequena.
// Em lote vira ~16/s, que e trivial.
//
// Nao e caso para Redis: quem amortece o pico ja e o Pub/Sub, que ainda por
// cima e DURAVEL — um buffer em memoria perderia eventos no processo que
// morre antes do flush.
const linhas = doGrupo.map(
  (e) => Prisma.sql`(
    ${e.eventId}::uuid, ${new Date(e.ts)}, ${e.escolaId}, ${e.estudanteId}, ${e.expId},
    ${e.expVersion}, ${e.sessaoId}::uuid, ${e.verbo}, ${e.objeto},
```

```
    ${JSON.stringify(e.resultado ?? null)}::jsonb,
    ${JSON.stringify(e.contexto  ?? null)}::jsonb)`,
);
await tx.$executeRaw(Prisma.sql`
  INSERT INTO telemetria_eventos
    (event_id, ts, escola_id, estudante_id, exp_id, exp_version, sessao_id,
     verbo, objeto, resultado, contexto)
  VALUES ${Prisma.join(linhas)}
  ON CONFLICT (event_id, ts) DO NOTHING`);
```

NOTA

O lote é **agrupado por escola** antes da gravação, e cada grupo grava na sua própria *tenancy* (`comTenancy(fn, escolaId)`). Um lote do Pub/Sub pode conter eventos de escolas diferentes, e uma única transação não poderia ter duas políticas de RLS armadas ao mesmo tempo.

10.4.2 Partição mensal

Detalhada na Seção 8.7. Em resumo: `PARTITION BY RANGE (ts)`, partições no *schema* `particoes` (fora do alcance do Prisma), chave primária composta `(event_id, ts)` porque o PostgreSQL exige a chave de partição em todo índice único, partição `DEFAULT` como rede de segurança, e faixas declaradas até 2028-12.

DECISÃO DE PROJETO

Nada de Redis como *buffer* (ADR-14). Quem amortece o pico já é o Pub/Sub — e ele é *durável*. Um *buffer* em memória perde os eventos do processo que morre antes do *flush*, ou seja, reintroduz exatamente a perda que o Pub/Sub eliminou. Redis só entra com gargalo medido, e o gargalo medido aqui era o laço de `INSERT`.

10.5 Privacidade da telemetria

REGRA INVIOLÁVEL

- O evento carrega `estudante_id` **opaco** (UUID), nunca nome ou *e-mail*.
- A identidade é **derivada do *token*** no servidor. O que o cliente enviou nesses campos é descartado — então o cliente não *pode* colocar PII ali.
- Nenhuma PII em log em texto claro.
- Texto livre sensível (o *check-in* de bem-estar) **nunca** vai à telemetria; só contagens.
- Em `app_70_sessoes.payload`, `eventos[].dados` aceita apenas número, booleano ou *string* de um **vocabulário fechado**. O campo de texto livre que existia era o único canal de texto arbitrário do *payload* — e o caminho por onde o nome de uma criança chegaria a um JSONB de longa duração. Trava estrutural, não lista de proibidos.

LIMITAÇÃO CONHECIDA

Retenção sem prazo declarado também aqui. `telemetria_eventos` guarda evento por `estudante_id` de *todas* as experiências e não tem prazo — pela mesma razão registrada na Seção 8.7 e em **ADR-13**: prazo só existe junto com a rotina que o cumpre. A diferença é que, particionada, essa rotina passou a ser trivial de escrever (`DROP TABLE particoes.<mês>`), o que remove o último argumento técnico para não ter prazo. A pergunta que sobra é de produto e de contrato, e é da plataforma inteira.

10.6 Comportamento em desenvolvimento

Configuração	Comportamento
`TELEMETRIA_TOPIC` vazio	Os eventos são apenas *logados* no console (`[telemetria/dev] N eventos`). Não há Pub/Sub em desenvolvimento e nada chega a `/ingest`.
`TELEMETRIA_PUSH_SA` / `..._AUDIENCE` vazias	`/telemetry/ingest` responde 401 a tudo. É o comportamento correto: em desenvolvimento ninguém deveria chamá-la.

NOTA

Essa combinação é o que permite rodar a plataforma inteira, com telemetria funcionando de ponta a ponta do lado do cliente, sem nenhuma credencial de GCP — e é o que faz **RNF-14** (testabilidade sem nuvem) ser verdade e não aspiração.

11

Camada de inteligência artificial

A IA generativa é usada em vários lugares da plataforma, com propósitos e riscos muito diferentes. Este capítulo descreve a camada única por onde todos passam, a economia de *tokens* que a sustenta e os *guardrails* que impedem cada uso de produzir dano.

11.1 Onde a IA é usada

Uso	Onde	O que gera, e o que NÃO gera
Geração de *feed*	exp08ef1b, `src/geracao/`	Personas fictícias, *captions*, *hashtags* e arte dos posts. O *mix* (quantos posts de cada lado, perfil e intensidade) é decidido pelo **servidor**; a IA só preenche.
Reação ao vivo	exp08ef1b, `src/motor/`	Comentários de persona reagindo ao que o estudante publicou. Desligável pelo professor no meio da aula, sem *deploy* (`ia_ao_vivo`).
Plano de aula	app-90, `src/planos/`	Um plano anual completo em Markdown, a partir do currículo da rede e das unidades do ano. **Não** decide nota nem avalia estudante.
Entorno do *check-in*	app-91, `src/bemestar/`	Abertura do dia, pergunta aberta e acolhimento. **Nunca** reescreve um item do DASS-21, nunca vê pontuação, nome ou escola.
Preparo da cobertura	exp07ef1b, `src/a-selecao/`	Um lote de 12 posts (4:4:4 por perfil) sobre o tema da professora, e as imagens a partir da descrição de cada um. **Nada** chega à turma sem a homologação dela; sem IA, roda o banco curado.
Leitura da legenda	exp07ef1b, `src/a-selecao/`	Um nível de EF07CO11 para a legenda dos Stories (o Canal A), que só o professor vê e corrige. Falha, timeout ou saldo zerado não marcam nada (**ADR-115**).

Preparo do pacote	exp04ef1b, `src/jornal-escola/`	O texto das oito notícias sobre o tema do professor, num **molde sorteado no servidor** (que notícia é de que meio e que papel tem no gabarito), e as imagens a partir da descrição de cada uma. **Nada** chega à turma sem a homologação; sem saldo (429) ou sem provedor (503), o hub oferece o banco padrão.
Diagramação da página	exp04ef1b, `src/jornal-escola/`	**Em sala**, com a quota do *estudante*: UM valor de um *enum* de três layouts, e nenhum texto. Não escolhe notícia, não reordena as vagas, não vê o gabarito. Qualquer falha cai no modelo automático (ADR-117).
Agente de IA do professor	produto `agente-professor`, `apps/agente-professor` + `src/agente-professor/`	Respostas sobre a BNCC e o acervo de livros da Unooria, com as fontes (título e página). **Fora** da api: o agente roda no Vertex AI Agent Engine (ADR-126). Recusa o que está fora do escopo; não planeja aula nem avalia estudante, e o código de habilidade que cita é conferido pela api (ADR-127).

REGRA INVIOLÁVEL

Correção, pontuação e gabarito nunca são gerados por IA. A IA produz *conteúdo* e *texto de entorno*. Tudo que vale nota, classifica ou mede é código determinístico e testável — e roda no servidor (Capítulo 12). **Uma exceção, declarada:** na A Seleção a IA *sugere* o nível de EF07CO11 lendo a legenda, com temperatura 0, uma tentativa, *schema* fechado e rubrica fixa no servidor. A sugestão não é nota, a criança não a vê, e o professor a confere e ajusta (ADR-115).

11.1.1 A diagramação do Jornal da Escola: IA em sala, e só um *enum*

É a segunda chamada de IA com a turma diante da tela, depois da leitura da legenda da A Seleção, e segue as mesmas travas (ADR-117). A criança monta as três vagas da primeira página, e a tela pede a diagramação uma vez por combinação. `diagramacao.service.ts` roda, nesta ordem:

1. o ***cache*** (`exp04ef1b_diagramacoes`): a mesma página — o pacote e as três vagas na ordem — montada por outra criança não paga outra chamada, e o custo fica limitado às combinações usadas;
2. o ***rate limit*** por estudante, em memória: doze diagramações a cada dez minutos, contra o toque nervoso;
3. a **quota do perfil `estudante`**, com `temSaldo` antes (regra inviolável nº 5);
4. o `generateJson`, com temperatura 0, **uma** tentativa e teto de 32 *tokens* de saída; o *schema* de resposta só tem o *enum* dos três layouts — não há campo de texto onde a IA escreva —, e um `validar` Zod confere a saída;
5. o **teto de tempo** de cerca de 6 s por `Promise.race`, porque o *provider* não tem *timeout*;

6. o débito em `aoUsarTokens`, que cobra também a chamada que perdeu a corrida para o teto (**ADR-74**).

A IA recebe das três notícias só a manchete, o número de palavras do resumo e se há imagem, como DADO delimitado — nunca o gabarito, a pista nem a consequência. **Falhou qualquer passo, o modelo automático diagrama** (`regras/layout.ts`): imagem na manchete principal dá `hero-image`, três manchetes curtas dão `columns`, e o resto `classic`. É determinístico e decide pela FORMA, nunca pelo gabarito, e o resultado do modelo não vai ao *cache*. A tela não fica sabendo de onde o layout veio, e o layout não vale nada na correção. Em *dev* e na suíte de ponta a ponta a IA é o *mock*, e a página é sempre do modelo: a diagramação por IA só se prova com o Vertex.

> **NOTA**
>
> **O preparo é do professor, e a estrutura é do servidor.** Pedir a um modelo "oito notícias, de três a quatro publicáveis, duas ou três antigas, e nenhum meio carregando a resposta" devolve, com frequência, o conjunto que a régua recusa — um modelo de linguagem conta mal. Por isso o molde é DADO: `sortearMolde` sorteia a cada geração o meio e o papel de cada id, a IA escreve o conteúdo, e o `validar` recusa a saída que mexeu no molde ou fere `invariantes.ts`, o que faz a tentativa ser regerada. A quota é a do perfil de quem prepara, com o que o *vendor* cobrou debitado por tentativa, inclusive as recusadas.

11.2 A camada única

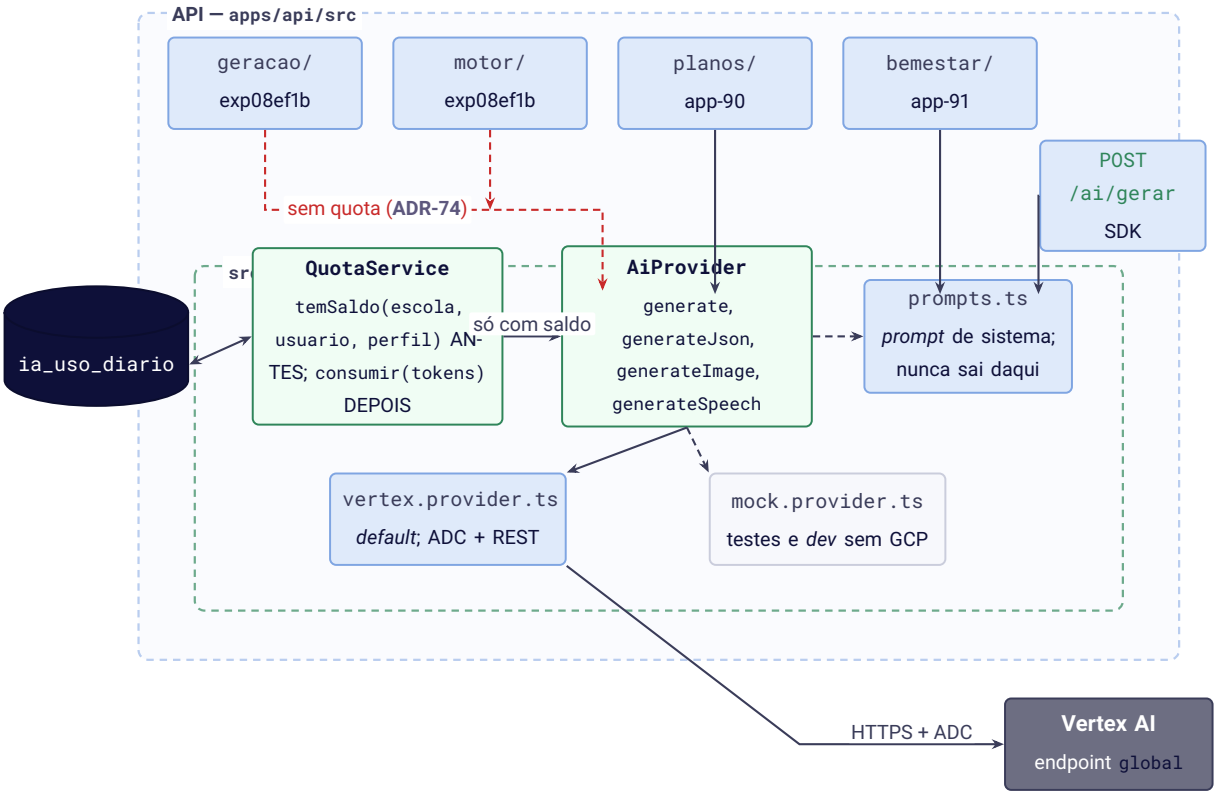


Figura 11.1 – A camada de IA, com os quatro primeiros módulos de domínio e a rota da SDK — hoje são onze módulos que chamam o *provider*, além da rota (Figura 5.13). `planos/`, `bemestar/` e a rota passam por `QuotaService` antes do *provider*; `geracao/` e as reações de `motor/` chamam o *provider* direto, sem quota — a exceção registrada na **ADR-74** e em `ai/AGENTS.md`. Nenhum módulo fora da fronteira verde conhece o *vendor* do modelo.

11.3 Quota por perfil

REGRA INVIOLÁVEL

Nunca exponha chamada de IA sem passar pelo `QuotaService`: `temSaldo(escola, usuario, perfil)` **antes**, `consumir` **depois**.

Em `generateJson`, o "depois" exige `aoUsarTokens` (**ADR-74**): ele devolve `T`, não `AiResult`, e sem o *callback* não há número para debitar. Um `temSaldo` sem `aoUsarTokens` é meia regra — barra quem já estourou o teto e faz o teto nunca ser alcançado. Foi o que aconteceu no plano de aulas da app-92, a operação mais repetida da ferramenta.

Listing 11.1 – Incremento atômico da quota (`src/ai/quota.service.ts`)

```
/** Incremento atomico; devolve o total do dia depois do consumo. */
async consumir(escolaId: string, usuarioId: string, tokens: number): Promise<number> {
  const dia = this.hoje();
  return this.prisma.comTenancy(async (tx) => {
    const linhas = await tx.$queryRaw<Array<{ tokens_usados: number }>>`
```

```
      INSERT INTO ia_uso_diario (escola_id, usuario_id, dia, tokens_usados)
      VALUES (${escolaId}, ${usuarioId}, ${dia}, ${tokens})
      ON CONFLICT (escola_id, usuario_id, dia)
        DO UPDATE SET tokens_usados = ia_uso_diario.tokens_usados + ${tokens}
      RETURNING tokens_usados`;
    return Number(linhas[0]?.tokens_usados ?? tokens);
  }, escolaId);
}
```

DECISÃO DE PROJETO

A quota é persistida, não um `Map` em memória. O `Map` que existia antes tinha dois furos: zerava a cada *restart* e era *por instância* — com `min-instances: 3` no Cloud Run, a mesma criança tinha três quotas. Uma linha por (escola, usuário, dia), com incremento atômico, resolve os dois.

DECISÃO DE PROJETO

O limite depende do PERFIL. O estudante interage com o tutor da exp08ef1b e com os turnos da exp02em1b (Missão Tempestade), onde uma partida completa gasta cerca de 77.000 *tokens* medidos: o teto de estudante é calibrado em 300.000 *tokens*/dia, o que cabe três partidas completas. O valor inicial de 20.000 estourava no quarto turno com erro de sala fechada. O professor gera planos anuais em que só o *prompt* — currículo da rede mais material didático — passa de 200.000 *tokens*: o teto dele é outro (2.000.000/dia, o que dá seis a nove planos). Um teto único obrigaria a escolher entre "estudante com quota absurda" e "professor que não consegue gerar um plano".

DECISÃO DE PROJETO

O limite é DADO, não variável de ambiente (ADR-105). Uma linha por perfil em `ia_quotas` (`estudante`, `professor`; vocabulário fechado por CHECK), escopo do contrato inteiro, semeada pela *migration* com os valores acima. O `QuotaService` a lê a cada `temSaldo` — sem *cache*, para que a alteração valha na chamada seguinte em todas as instâncias — e todo perfil que não é `estudante` cai na linha `professor`. Quem altera é só o `admin`, em `/unooria/uso-de-ia`, com motivo obrigatório; a mudança entra em `recursos_auditoria` com o `{de, para}`. As *envs* `AI_DAILY_TOKEN_QUOTA_PER_*` **deixaram de existir**: mantê-las como *fallback* seria a segunda fonte de verdade. Sem linha (uma *stack* antes da *migration*), vale a constante do código e o registro acusa `ia-quota-sem-linha`.

NOTA

A verificação acontece **antes** de chamar o *provider*. Gastar *token* e depois recusar seria pagar pelo que não se entrega. Há também *rate limit* por usuário (`src/comum/rate-limit.ts`), que é uma proteção diferente: a quota limita custo diário, o *rate limit* limita rajada.

11.4 Credencial: ADC, nunca chave de API

REGRA INVIOLÁVEL

O *provider* obtém o *token* com `google-auth-library`, que resolve na mesma ordem nos três ambientes:

1. `GOOGLE_APPLICATION_CREDENTIALS`;
2. o arquivo de `gcloud auth application-default login`;
3. o *metadata server* (Cloud Run).

Não existe variável de ambiente para chave de API em nenhum lugar do projeto — nem em `.env.example`.

ATENÇÃO

Antes dessa mudança havia **só** o *metadata server*, e por isso *nenhuma* chamada de IA funcionava fora do Cloud Run — nem no *host*, nem no contêiner, onde `metadata.google.internal` não resolve. Em desenvolvimento a API ficava presa em `AI_PROVIDER=mock` e a app-90 devolvia um plano **falso**, o que é pior que falhar: parecia funcionar. Os dois *compose* montam o ADC do *host* em `/adc` somente leitura; em Cloud Run a conta de serviço do serviço já *é* a credencial, e ela precisa de `roles/aiplatform.user`. Sem o papel, toda geração falha com 403 do Vertex.

11.5 Modelo, região e o custo de escolher

DECISÃO DE PROJETO

Modelo `gemini-3.6-flash`, localização `global` — para texto e para o plano de aula (decisão de 2026-08, **ADR-16**).
Medido no projeto do contrato: o `3.6-flash` dá **404** em `southamerica-east1`, `us-east1` e `us-central1`. Só o endpoint `global` (sem *host* regional) o serve.
A consequência é explícita e foi aceita: **o tráfego de IA sai de São Paulo, inclusive o do estudante**, e isso precisa constar do contrato e do DPA da rede. Voltar para `VERTEX_LOCATION=southamerica-east1` é possível a qualquer momento, mas exige trocar **também** o modelo pelo que a região publica — a família 2.5, já que a 2.0 saiu do catálogo e a 3.x nunca esteve lá. As duas variáveis mudam juntas, ou a geração para de funcionar.

11.5.1 Imagem: proporção pedida ao modelo, não recortada depois

Fato	Consequência

O *default* anterior era `gemini-2.0-flash-exp`	Esse modelo **não existe mais** em lugar nenhum (404 na região *e* em `global`), e por isso *toda* geração de imagem caía no *fallback* procedural — sem erro visível para o professor.
Modelo atual: `gemini-3.1-flash-image`	Responde em `global`. Alternativas verificadas: `gemini-2.5-flash-image` e `gemini-3-pro-image`.
A proporção `4:5` é PEDIDA no `imageConfig`	Sem isso, o 3.1 devolve 1408×768, e o `cover` do `AssetsService` (1080×1350) descarta mais da metade da largura.
O modelo aceita 1:1, 2:3, 3:2, 3:4, 4:3, 4:5, 5:4, 9:16, 16:9 e 21:9 pelo MESMO custo em *tokens*	Experiência com outro formato deve **pedir** o formato, não recortar depois.

11.5.2 Voz: a única chamada que não é ao *aiplatform*

`generateSpeech` chegou ao `AiProvider` com o Eco no Grupo (**ADR-67**) e ficou no contrato depois que a missão saiu (**ADR-113**): hoje nenhuma experiência sintetiza voz. É a exceção do capítulo: as outras três operações batem em `{loc}-aiplatform.googleapis.com`, e a síntese bate em `texttospeech.googleapis.com/v1/text:synthesize` — mesma credencial ADC, mesma família de modelos (`VERTEX_TTS_MODEL`, `gemini-2.5-flash-tts`), outro serviço, que precisa estar **habilitado** no projeto.

Decisão	Por quê
`audioEncoding: "MP3"`	O sistema não tem *ffmpeg*, e um WAV de três minutos passaria de 10 MB. O encoding vem pronto do serviço, e o `<audio>` do navegador toca direto.
`input.prompt` carrega a direção de atuação	"tom assustado, voz trêmula" é instrução ao modelo, não texto falado. Os dois têm teto de 4.000 *bytes*, conferido no domínio antes do pedido — estourar devolve 400 sem dizer qual dos dois campos.
`multiSpeakerMarkup` para diálogo	Duas vozes numa chamada só, cada turno com seu `speakerId`. Monólogo usa `voice.name`; diálogo usa `multiSpeakerVoiceConfig`. **Mandar os dois** devolve 400.

> **ATENÇÃO**
>
> **Com ADC de *usuário*, a síntese exige `x-goog-user-project`.** O sintoma engana: a geração de *imagem* funciona e só a voz responde `403 PERMISSION_DENIED`, porque o `aiplatform` aceita o projeto do *endpoint* e o Text-to-Speech cobra um projeto de cota explícito. O *provider* manda o cabeçalho quando `VERTEX_PROJECT_ID` está definida. Em produção, com *service account*, o cabeçalho é inócuo.

11.6 Raciocínio conta como saída

ATENÇÃO

Nos modelos das famílias 2.5 e 3.x, os *tokens de pensamento* saem do `maxOutputTokens`. Com pensamento livre, o plano anual voltava **vazio**, com `finishReason: MAX_TOKENS` — o modelo gastava todo o orçamento pensando e não sobrava saída. `VERTEX_THINKING_BUDGET` (*default* `0`, desligado) mantém a paridade com o aplicativo de origem, que usava um modelo sem essa etapa; `-1` volta ao dinâmico.

REGRA INVIOLÁVEL

Quem gera texto longo checa `AiResult.finishReason`. `MAX_TOKENS` significa *conteúdo cortado*, e isso é **erro** — não resultado parcial aceitável. Entregar meio plano de aula a um professor é pior que dizer que falhou.

Parâmetro	Valor e motivo
`MAX_TOKENS_SAIDA` do plano	**64k**, não os 32k do aplicativo de origem (que eram o limite do modelo usado lá). Medido: com o material real (currículo da rede + as 4 unidades do ano) o plano é cortado em 32k e o professor recebe "o plano ficou grande demais" sem plano nenhum. A família 2.5 entrega até 65k; 64k dá a folga sem chegar ao limite duro.
`AiRequest.modelo`	`"padrao"` ou `"longo"`. O `longo` mapeia para `VERTEX_MODEL_LONGO`. Quem chama **não sabe** qual modelo é — a família é neutra de *vendor*.

11.7 *Guardrails* do *check-in* de bem-estar

Este é o uso de IA com maior risco de dano na plataforma, e por isso o mais cercado. O instrumento é o DASS-21 (LOVIBOND; LOVIBOND, 1995), na versão brasileira validada (VIGNOLA; TUCCI, 2014), em domínio público.

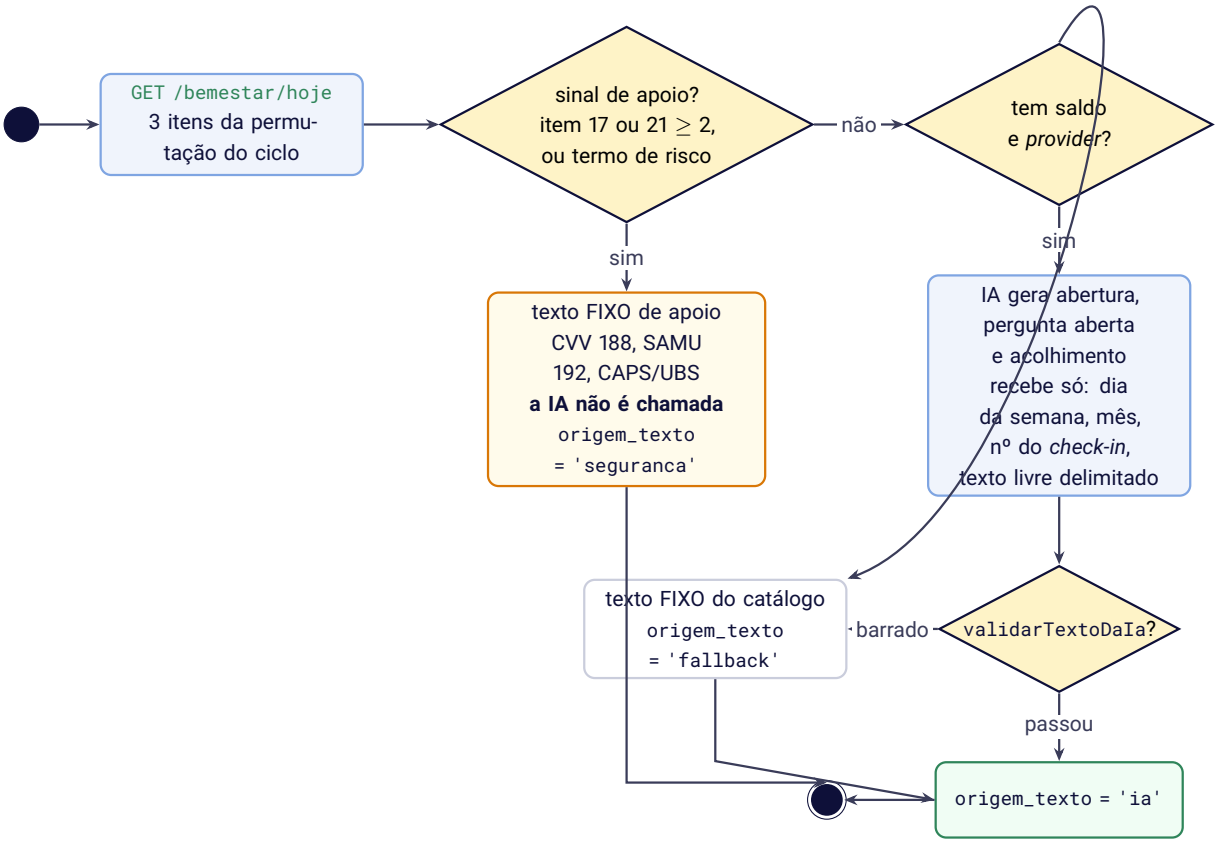


Figura 11.2 – Os *guardrails* do *check-in*. Três caminhos levam a texto seguro, e em nenhum deles o *check-in* deixa de abrir: se a IA falha, é barrada ou não há saldo, entra o texto fixo do catálogo; se há sinal de apoio, a IA nem é chamada.

REGRA INVIOLÁVEL

1. **Os 21 itens são fixos e *verbatim*** (`catalogo.ts`). A IA **nunca** reescreve um item: reformular quebra a validação psicométrica e invalida as faixas de corte publicadas.
2. **O que a IA gera é o entorno**, e cada texto passa por `regras.validarTextoDaIa`: sem vocabulário clínico, sem ordens, sem promessas, sem linguagem de gestão, sem *links*, com limite de tamanho.
3. **Barrado, sem saldo ou sem *provider* ⇒ texto fixo do catálogo.** O *check-in* nunca deixa de abrir.
4. **A IA nunca recebe** nome, escola, pontuação ou faixa — só dia da semana, mês, número do *check-in* e o texto livre, *delimitado como conteúdo*.
5. **Sinal de apoio bypassa a IA**: item 17 ou 21 ≥ 2, ou termo de risco no texto ⇒ mensagem **fixa** de apoio, e a IA não é chamada.
6. **A correção é *server-side***: ao sétimo *check-in* a API soma por subescala ($\times 2$) e classifica pelas faixas publicadas (`pontuar` e `faixaDe` em `regras.ts`).

> **NOTA**
>
> O texto livre do professor *vai* ao Vertex (endpoint `global`), delimitado como conteúdo e sem nenhum outro contexto identificável. Isso consta do DPA junto com o resto do uso de IA — e é a razão de a Seção 7 tratar localidade de processamento como item de conformidade, não de arquitetura.

11.8 O segundo *vendor*: o agente de IA do professor

O agente de IA do professor é a **única** IA da plataforma que não passa pelo `AiProvider` (**ADR-126**). O modelo, a instrução e as ferramentas moram num agente Google ADK em Python (`apps/agente-professor`), servido pelo Vertex AI Agent Engine; a api chama o engine por uma porta própria de `src/ai`, `AgenteProfessorPort`, que só `agent-engine.client.ts` implementa (Seção 9.5). O que a regra da camada única protege continua na api: papel, disponibilidade, *rate limit*, quota e a posse da conversa.

Peça do agente	O que faz
`LlmAgent` `agente_professor`	A conversa: `AGENTE_MODELO` (`gemini-3.7-flash`), temperatura 0,3, com as duas ferramentas.
Guarda de escopo	*Plugin* `before_agent`: o modelo leve (`AGENTE_MODELO_ESCOPO`, `gemini-3.5-flash-lite`) decide se a pergunta é de BNCC ou do acervo; fora disso, recusa pelo nome **sem chamar o modelo principal**, num evento com `custom_metadata.fora_do_escopo`.
Título	*Plugin* `after_agent`: depois do primeiro turno, o mesmo modelo leve escreve `state.titulo_conversa`. Turno recusado ou falha do modelo de título: não há título, e o turno seguinte tenta de novo.
Poda de contexto	Até 6.000 caracteres por *string* e 24.000 por resposta de ferramenta, **exceto** `buscar_habilidades` (120.000: a Língua Portuguesa do 8º ano inteira tem cerca de 58.000); retorno de ferramenta de turno anterior vira resumo com os códigos.
Modelo com reserva	`global`, e `us` quando `global` responde 404, 429 ou 5xx — só ANTES do primeiro pedaço da resposta; depois, trocar duplicaria o texto na tela.

> **ATENÇÃO**
>
> **`GOOGLE_CLOUD_LOCATION` é reservada no Agent Engine** e vale a região do *deploy* (`us-east1`). Configurar o modelo por ela o mandaria para a região do engine, onde ele pode nem existir — um 404 que só aparece em produção. A location do modelo se configura em `modelos.py`, pelo `client_kwargs` do `Gemini` (`AGENTE_MODELO_LOCATION`).

A quota é a mesma, contada de outro jeito. O turno usa o perfil `professor` de `ia_quotas`: `temSaldo` antes de abrir o *stream*, e `consumir` num `finally` com a **soma dos `usage_metadata` dos eventos NÃO parciais** — os parciais repetem o uso, e um turno com ferramenta faz mais de

uma chamada ao modelo. Os *tokens* da guarda e do título chegam somados ao evento seguinte. O escape conhecido: quando o modelo de título responde mas o saneamento zera o texto, não há evento, e cerca de 40 *tokens* ficam fora da quota. O `MedidorDeProvider` não vê o agente; a métrica dele é a linha de log `agente-professor.turno` (Capítulo 17).

A avaliação mora no agente: 16 casos em `avaliacao/agente_professor.evalset.json` (escopo, código exato, a lista inteira de um ano, tema, acervo com citação, nome de estudante), rodados por `adk eval` contra o `dev`; e o relatório de recall das habilidades (`ingestao/avaliar_habilidades`) é o gatilho da **ADR-127**.

11.9 Variáveis de configuração da IA

Variável	Default	Papel
`AI_PROVIDER`	`vertex`	`vertex` ou `mock`.
`VERTEX_PROJECT_ID`	—	**Obrigatória** com `AI_PROVIDER=vertex`: o *boot* recusa sem ela. Sem projeto, a URL viraria `.../projects/undefined/...` e o erro só apareceria na primeira geração.
`VERTEX_LOCATION`	`global`	Ver **ADR-16**.
`VERTEX_MODEL`	`gemini-3.6-fl`	Modelo do dia a dia.
`VERTEX_MODEL_LONGO`	`gemini-3.6-flash`	Modelo de saída longa. A variável separada existe para trocar um sem tocar no outro.
`VERTEX_IMAGE_MODEL`	`gemini-3.1-fl`	Quem trocar precisa conferir a proporção devolvida.
`VERTEX_TTS_MODEL`	`gemini-2.5-flash-tts`	Voz. Serviço à parte (Text-to-Speech), habilitado no projeto. Nenhuma missão sintetiza voz hoje.
`VERTEX_THINKING_BUDGET`	`0`	`0` desligado, `-1` dinâmico, > 0 teto.
`AI_DAILY_TOKEN_QUOTA_PER_*`	**removidas**	A quota diária por perfil é a tabela `ia_quotas` desde a **ADR-105**, editada pelo `admin`; não há *env* de *fallback*.
`AGENTE_PROFESSOR_MODO`	`desligado`	`agent-engine` | `local` | `mock` | `desligado`. Ausente = `desligado` em qualquer `NODE_ENV` — ao contrário do `AI_PROVIDER`; `mock` e `local` são recusados em produção. Ver Apêndice B.

> **DECISÃO DE PROJETO**
>
> **Validação de *env* no *boot*, com Zod.** `VERTEX_PROJECT_ID` ausente com `AI_PROVIDER=vertex` derruba o processo na subida, com mensagem que diz o que fazer ("em *dev*, use `AI_PROVIDER=mock` para subir sem GCP"). A alternativa — descobrir na primeira geração de um professor real — é um incidente em vez de um erro de configuração.

12

Catálogo de experiências

O catálogo alvo é de 48 experiências. Cinco existem, e são elas que definiram o gabarito que as próximas seguem. Este capítulo documenta cada uma até onde a mecânica impõe exigência ao *software* — e não como material pedagógico.

12.1 Visão do catálogo

Experiência	Séries	Público	Capabilities	Módulo na API
`exp08ef1b`	`8EF`	estudante	telemetria, progresso, ia	`motor`, `geracao`, `atividades`, `perfis`, `dossie`
`exp04ef1b`	`4EF`	estudant	telemetria, progresso, ia, avaliacao	`jornal-escola`, `evidencias`
`backup-exp04ef1b`	`4EF`	**arquivada**	telemetria, progresso	`backup-unootube` (sem rotas)
`exp02em1b`	`2EM`	estudant	telemetria, ia	`unoogpt`
`exp01ef1b`	`1EF`	estudante	telemetria, progresso	`foto-conta`
`exp05ef1b`	`5EF`	estudant	telemetria, progresso, avaliacao	`quem-mandou-isso`
`exp06ef2b`	`6EF`	estudante	telemetria, progresso, avaliacao	`rastro-boato`
`exp07ef1b`	`7EF`	estudant	telemetria, progresso, ia, avaliacao	`a-selecao`, `evidencias`
`exp08ef4b`	`8EF`	estudante	telemetria, progresso, avaliacao	`manifesto-ia`

app-70-vila-do-l	**transversa**	estudant	telemetria	vila
app-90-plano-de-au	la	**professor**	telemetria, ia	planos
app-91-bem-estar	—	**professo**	telemetria, ia	bemestar
app-92-plano-de-en	**1EF–3EM**	**professor**	telemetria, ia	planejamento, curriculo
_template	—	—	telemetria, progresso	—

A exp05ef1b é a v2.2 de Quem mandou isso?, que voltou com o mesmo código e a partir da 0.2.0 depois de a primeira versão sair (**ADR-114**, **ADR-118**). Ela, A Seleção (exp07ef1b) e o Jornal da Escola (exp04ef1b) são as três missões com ficha pedagógica: as perguntas finais passam pelas rotas genéricas de evidencias, e não pelo módulo de cada uma.

A unidade 1 do 4º ano é o **Jornal da Escola: A Edição de Bento** (**ADR-117**), que tomou o código exp04ef1b e a porta 5174 do UnooTube e nasce na 2.0.0. O UnooTube foi **arquivado**, e não apagado (**ADR-116**) — a pasta, o módulo, a tabela e as linhas de plataforma dele ganharam o prefixo backup-, e ele saiu do catálogo, da grade e do *deploy*. A 2.0.0 existe por causa dele: o caminho exp04ef1b/ do *bucket* pode guardar o UnooTube 1.2.8, e uma versão 1.x do Jornal cairia num caminho tratado como imutável.

A unidade 1 do 7º ano é A Seleção, que reocupou o código exp07ef1b e a porta 5179 do Eco no Grupo (**ADR-113**, **ADR-115**).

Experiência	Porta em *dev*	Onde aparece no hub
exp08ef1b	5173	/exp/exp08ef1b
exp04ef1b	5174	/exp/exp04ef1b (o Jornal da Escola; o UnooTube arquivado não sobe)
app-90-plano-de-aula	5175	/professor/plano-de-aula
app-91-bem-estar	5176	/professor/bem-estar
app-70-vila-do-lume	5177	/exp/app-70-vila-do-lume
exp02em1b	5178	/exp/exp02em1b
exp07ef1b	5179	/exp/exp07ef1b
exp06ef2b	5181	/exp/exp06ef2b
app-92-plano-de-ensino	5180	/professor/plano-de-ensino
exp08ef4b	5182	/exp/exp08ef4b
exp01ef1b	5183	/exp/exp01ef1b
exp05ef1b	5184	/exp/exp05ef1b

> **NOTA**
>
> As portas vivem em `apps/hub/lib/experiencias-dev.ts` — uma por experiência. É esse mapa que o hub usa **fora de produção**, e é por isso que `EXP_INTERNAL_HOST` tem de *preservar a porta* (Seção 9.2).
> A escolha era "sem manifesto na tabela, use o mapa" — e isso quebrou quando o *seed* passou a popular `experiencias` (**ADR-69**, que precisa do catálogo para montar a grade da turma). Sem CDN configurado, a URL versionada cai no *default* `localhost:5173`, o *dev server* da exp08ef1b — que responde a QUALQUER caminho com a própria SPA. Toda missão abria o Instanooria, com o *handshake* completando e o `boot` dando 403 na experiência errada. **De onde vem o *bundle* é pergunta do AMBIENTE; o manifesto responde o que a experiência declara.**

12.2 A sala de aula: o que o estudante vê

O hub do estudante (`/estudante`) abre pela **turma**, não pelo catálogo. `GET /salas/minhas` devolve as turmas em que ele tem matrícula e, em cada uma, a grade do ano dela: as experiências cuja `series` contém a série da turma, mais as transversais (`series: []`).

Estado	O que a criança vê
`liberada`	O cartão é um *link* e diz "Começar".
`bloqueada`	**CADEADO** e "Ainda não abriu", sem *link*. Não some da tela: a grade do ano é a promessa do que vem depois, e uma lista que muda de tamanho sozinha não deixa ninguém entender que a próxima unidade existe. Não é estado do banco — é a ausência de linha em `liberacoes`.
`encerrada`	Continua visível, com o rótulo. O que ela fez não pode evaporar da tela quando o professor fecha a aula.

Missão de OUTRA SÉRIE não aparece — e isso é diferente de bloqueada: a do 4º ano não pertence à grade do 8º, não está esperando liberação. Missão sem *copy* de estudante também não entra, mesmo liberada: sem título e sem chamada o cartão seria um retângulo com um *slug* dentro.

> **REGRA INVIOLÁVEL**
>
> **Esconder o cartão é UX.** `/exp/:expId` confere a liberação antes de montar o *embed* e responde **404** sem ela, e a API recusa a entrada com **403** `missao-nao-liberada`. O *bundle* é público e o endereço é adivinhável: antes disto, qualquer estudante logado abria qualquer missão digitando a URL.

12.3 A sala de aula: o que o professor faz

`/professor/turmas/:id` deixou de ser a lista nominal e virou a SALA, com duas abas:

- **Missões** — a grade daquela sala, com o estado de cada uma, quem a abriu e quando, e o botão *Liberar*/*Encerrar*. Daqui sai também o caminho para conduzir a missão **com a turma já escolhida**.
- **Estudantes** — a lista nominal que já existia, com o caminho para o relatório de cada criança.

A aba vive na URL (`?aba=`), como a turma nos resultados: o *link* fica compartilhável e o botão voltar funciona.

> **DECISÃO DE PROJETO**
>
> **O caminho para conduzir a missão sai da SALA.** Antes era: voltar ao hub, entrar no Entre Redes, achar o ano, abrir resultados e escolher a turma num `<select>` — e a página do ano era a MESMA para as duas turmas da série. Era assim que 2EM-A e 2EM-B viravam uma coisa só na cabeça de quem usava.
> A trilha do Entre Redes continua existindo e continua sendo por ano: ela é o CATÁLOGO, o lugar de conhecer a missão. Uma faixa no topo dela diz quem abre a missão para a turma, porque sem isso o professor lê a grade inteira como se tudo estivesse no ar para os estudantes dele.

> **NOTA**
>
> **O filtro por série das telas de resultado ficou; o *fallback* saiu.** Elas ofereciam "mostrando todas" quando o professor não tinha turma da série da missão — e aí ele escolhia uma sala que nunca vai jogar aquilo. Sem turma da série, agora a tela diz isso e aponta para *Minhas turmas*.

> **REGRA INVIOLÁVEL**
>
> **Professor com uma turma tem o mesmo fluxo de quem tem quatro:** ele entra na sala clicando. Redirecionar direto de `/professor` para a única sala foi considerado e recusado — dois fluxos para o mesmo papel é o tipo de atalho que faz a tela de lista parecer quebrada para metade da rede.

12.4 O guia da aula

Toda missão publicada tem um **guia da aula** em `/professor/.../guia` — e a Vila do Lume, transversal, em `/professor/entre-redes/vila-do-lume/guia`. É o material com que o professor conduz a missão: objetivos e habilidades da BNCC Computação, preparo técnico e pedagógico, roteiro cronometrado, banco de perguntas de debate, mediação, folha de observação, avaliação, cuidados, adaptações e dúvidas frequentes.

> **DECISÃO DE PROJETO**
>
> **O guia é DADO, não página.** Ele nasceu como uma página só, do Instanooria: 447 linhas de JSX em que conteúdo, ordem e marcação eram a mesma coisa, com `notFound()` para qualquer outro `expId`. Replicar aquilo por missão daria cinco arquivos que ninguém revisa juntos — e a seção que alguém esquecesse de copiar simplesmente não existiria para aquela turma, sem nenhum sinal.
> Hoje o conteúdo mora em `apps/hub/lib/entre-redes/guias/<exp>.ts` e a ordem é de um renderizador único (`apps/hub/components/guia/`). Seção que falta é campo que falta: o tipo cobra de quem escreve um guia, e `guias/index.test.ts` cobra de quem publica uma missão e não escreve nenhum. Ver **ADR-65**.

12.4.1 Blocos vivos: o que não vira texto

O que tem fonte de verdade em outro lugar **não** é copiado para o guia. O renderizador aceita um `ReactNode` ancorado ao fim de uma seção, e a página da missão injeta o que só ela sabe montar:

- **exp08ef1b** — a tabela dos momentos do painel vem de `CONTROLE_DO_PROFESSOR`, e as sete telas da revelação de `lib/insta/copy-da-revelacao.ts` — **cópia gêmea** do arquivo homônimo da exp08ef1b, não a mesma fonte: as duas mudam juntas, e nada no *build* cobra isso.
- **exp04ef1b** (o Jornal da Escola) — `GET /jornal-escola/professor/material` devolve, numa chamada só, três blocos. No roteiro, os dois portões com o que dizer à turma ao apertar cada um (a MESMA frase que o painel da aula mostra ao lado do botão); no debate, as três perguntas da mediação, verbatim; e no preparo, as oito notícias do banco padrão com o gabarito e a reação do leitor, e os seis blocos da Chave com as iscas marcadas. O componente que os desenha é `guia/material-do-jornal.tsx`.
- **backup-exp04ef1b** (o UnooTube, arquivado na **ADR-116**; o bloco saiu do guia com ele) — a tabela dos 21 vídeos, com o `factual` de cada um, vinha de `GET /unootube/professor/videos`. A criança de nove anos cita o vídeo pelo apelido do canal, e sem a lista o professor não sabe de qual ela está falando.
- **exp02em1b** — a tabela dos oito vieses vem de `GET /unoogpt/professor/fontes`, derivada da própria `instrucao` que o agente recebe.
- **exp06ef2b** — as três intervenções, com desfecho e reação, e o ciclo na ordem certa vêm de `GET /rastro-boato/professor/material`. Lado a lado aparece o que sustenta a missão: A e C produzem a **mesma** reação do grupo.
- **app-70** — os cartões das três bolhas vêm de `GET /vila/professor/cartoes`, **por turma**: nos anos iniciais a bolha B não é servida, e projetá-la mandaria o professor esperar a reação de um grupo que não existe na sala dele.

Descrever também é copiar (**ADR-86**). As missões do UnooTube (hoje `backup-exp04ef1b`), da exp06ef2b e do Eco no Grupo (retirado, **ADR-113**) não guardavam o conteúdo: guardavam a

paráfrase dele — e uma varredura pelo texto exato do catálogo dentro de cada guia devolveu **zero** ocorrências nas três. A do UnooTube dizia *"metade cita fonte verificável … a outra metade é sensacionalista"* quando a divisão real é **11 e 10**, porque 21 é ímpar.

> **REGRA INVIOLÁVEL**
>
> Uma cópia desses textos no hub diverge na primeira revisão de *copy*, e o resultado é o professor projetando para a turma uma mensagem que ninguém recebeu, ou contando um viés que o cenário já não produz.

12.4.2 BNCC Computação

Cada guia lista habilidades do *Complemento de Computação à BNCC* (MEC, 2022) com o **texto oficial verbatim** e uma coluna nossa dizendo onde, na missão, a habilidade acontece. A tela avisa que o recorte é proposta da plataforma e pede conferência com a coordenação.

> **ATENÇÃO**
>
> A numeração do complemento **não** é a que se supõe por analogia com as outras áreas: as habilidades de Computação são **por ano** (`EF04CO08`, `EF07CO08`, `EF08CO11`), mais dois blocos plurianuais (`EF15CO*`, do 1º ao 5º, e `EF69CO*`, do 6º ao 9º) e `EM13CO*` no Médio. `EF67CO` e `EF89CO` não existem — e já foram escritos errados uma vez. Um teste guarda a forma do código.

12.4.3 Impressão

O professor conduz a aula com o guia na mão enquanto o computador projeta a tela da turma. `components/guia/guia.css` tem a folha de impressão: fora o cabeçalho da plataforma, o índice lateral e todo botão; cada linha de tabela e cada item de lista inteiros na mesma folha; borda de 1pt onde a sombra do cartão não imprime. As regras miram classes próprias (`.guia-bloco`), nunca as do `@unooria/design-system` — nome interno de componente muda por versão, e ninguém imprime para conferir depois de subir o DS.

12.4.4 Onde se chega ao guia

- na ficha da missão, botão "Abrir guia da aula" (as quatro missões de ano e a transversal, com o mesmo rótulo);
- na **sala de aula** (`/professor/turmas/:id`, aba Missões), um *link* por missão — é o caminho de toda turma × toda missão, e sai de `lib/rotas-da-missao.ts`, que é quem sabe se a missão tem guia;
- nas telas de **resultados**, que é onde o professor está durante a aula.

> **NOTA**
>
> `/professor/guia`, endereço de quando a plataforma tinha uma atividade só, continua redirecionando — agora para o guia da missão atual, com a ficha dela como *fallback* se um dia ela perder o guia.

12.5 exp08ef1b — Instanooria

Clone de rede social de imagens. O estudante navega um *feed* curado por um motor adaptativo que o empurra progressivamente para um dos dois lados de uma controvérsia; na virada da missão, o dossiê mostra o que ele não viu.

Exigência ao *software*	Como é atendida
Feed adaptativo por blocos, determinístico	`motor/curadoria.ts`: função pura que recebe um gerador de aleatoriedade. É isso que permite ao teste afirmar "só engaja A ⇒ domínio > 0,75 até o bloco 4".
Serialização por sessão	`pg_advisory_xact_lock(runId)` dentro da transação, com `timeout` de 20 s.
Gabarito de curadoria fora do cliente	`motor/sanitizador.ts` remove `lado`, `perfil` e `intensidade` antes de o *feed* sair.
Geração de conteúdo em lote, assíncrona	`geracao/`: fila em `exp08ef1b_generation_jobs`, *worker in-process*, reciclagem de órfão por `updated_at`.
Momento coletivo controlado pelo professor	`activities.etapa` é um relógio **coletivo**: o professor abre e fecha, e a tela do estudante trava conforme.
Reação ao vivo, desligável sem *deploy*	`activities.ia_ao_vivo`, alterável por `PATCH /atividades/:id/ia`.
Largada por perfil, sorteio ou redistribuição	`perfis/largada.ts`, com precedência explícita e `lado_origem` gravado para auditoria.
Casca de aparelho de 375×812	`viewport` no manifesto ⇒ `DispositivoFrame`.

Listing 12.1 – Precedência da largada (`src/perfis/largada.ts`)

```
// 1. `lado_forcado` — o professor redistribuiu antes da aula. Essa decisao NAO
//    pode ser desfeita por um recalculo de afinidade no login.
// 2. Afinidade do perfil — o mecanismo normal.
// 3. Sorteio balanceado — sem perfil, ou perfil `misto`. Escolhe o lado com
//    menos runs, para a turma ainda sair perto de 50/50.
//
// Existe como modulo proprio porque DUAS rotas precisam da MESMA regra: o
// primeiro inicio e o reset da sessao. Quando o reset tinha regra propria, ele
// mantinha o lado antigo — e reaproveitar o aparelho repetia a semente sem
// ninguem perceber.
```

Verbo	Significado documentado
`attempted`	entrou no aparelho
`progressed`	mudou de fase
`completed`	concluiu a missão (dossiê visto)

12.6 exp04ef1b — Jornal da Escola: A Edição de Bento

4º ano · Unidade 1 · redação própria da Unooria, sem imitar aplicativo · `unooria`, com o post em `instanooria` e a mensagem em `whatsappmobile` · 2.0.0

O Bento cuida do jornal da escola e pede ajuda com a edição de hoje. A mesa tem oito notícias de oito meios — jornal, rádio, telejornal, blog, portal, rede social, mensagem encaminhada e mural — e a primeira página tem três vagas. A criança abre, se quiser, as pistas de cada notícia (de onde veio, quando aconteceu, quem assina), põe três na capa e publica, sem volta. Logo depois diz como escolheu — pelas fotos e pela urgência, ou pela checagem — e espera. Quando o professor libera o resultado, os leitores da escola reagem às três notícias publicadas, e a tela diz quantas estavam verificadas (0, 33, 67 ou 100 %: a descrição da SELEÇÃO, nunca da criança). Depois da conversa com a turma, o professor libera a revisão: a Chave do Editor junta fonte, data e autoria, a mesa volta com um marcador em cada vaga, e a criança publica a 2ª edição e compara as duas. O Desafio Final, individual e fora da redação, fecha a missão com duas perguntas que não mostram acerto. As decisões estão na **ADR-117**; o PDF da proposta revisada vence o PRD da esteira quando os dois divergem.

Decisão	Como está implementada
A redação e as peças	Mesa, gaveta, página, Bento, leitores, Chave e o Desafio Final são da seção `unooria`, com as oito peças da 0.2.8 (Seção 14.11.8). Na gaveta, a matéria aparece no formato original: o post de rede social é o `PostCard` do `instanooria`, a mensagem encaminhada é o `MessageBubble` do `whatsappmobile`, e os outros seis meios são o `NewsArticle`. A peça imitada vem SEM o cromo do aplicativo, sem curtidas e sem horário: a data é uma pista, e o carimbo a entregaria antes da hora. Cada família mora num arquivo (`src/app/pecas/`), que só importa a sua ponte.
Atividade e rodada	A ATIVIDADE é o **pacote** das oito notícias, e quem a cria é o professor (**ADR-130**): a plataforma não cria nada, e sem atividade em andamento a missão responde 403 `atividade-nao-preparada` — a porta fechada que a experiência já mostra. O banco da Unooria (“Feira de Ciências e Notícias da Cidade”) é ponto de partida da criação. Criar não põe no ar: o pacote nasce pronto, e o professor escolhe o em andamento na lista; pô-lo no ar abre a rodada seguinte à última. Zerar sobe a rodada sem apagar nada; criar o mesmo conteúdo do mais recente é idempotente (**ADR-112**).

Dois portões, um botão	`exploracao` → `resultado` → `revisao`, uma etapa por vez, sem pular nem voltar (409 `transicao-de-etapa-invalida`). A etapa só conduz quem publicou e respondeu o motivo: quem ainda monta a página continua montando (a publicação atrasada é caminho previsto), e da Chave em diante cada um anda sozinho. `passoEfetivo` (`regras/passos.ts`) reconcilia os dois relógios; os passos `resultado` e `chave` nunca são gravados, e um *CHECK* recusa. O estudante lê `GET /jornal-escola/gate` por *polling* de 5 s, e só nas esperas (**ADR-93**).
O motivo depois do *latch*	A pergunta vem logo depois da 1ª publicação e antes do resultado, uma vez só e com qualquer etapa. Feita no modal de publicar, anteciparia a descoberta; feita depois do resultado, sairia contaminada pela consequência. O painel mostra só o agregado.
O gabarito desce em portões	A notícia do estudante é uma *allowlist* (`noticiaPublica`): sem gabarito, sem pistas e sem consequência. O texto da pista desce no `POST /jornal-escola/pista`, que é o registro da EF15CO04; as reações, no `GET /jornal-escola/resultado` com a etapa `resultado` aberta, e só das três escolhidas; o pilar em falta de cada vaga, no `GET /jornal-escola/revisao`, depois da Chave; o conjunto certo da Chave, nunca. As verificadas são calculadas no *latch*, NO SERVIDOR. Um *grep* no `dist/` prova que o *bundle* não tem gabarito, prompt nem a lista das iscas.
A Chave do Editor	Seis blocos — os três pilares e três ISCAS (“foto”, “urgência”, “compartilhada”) —, na ordem embaralhada por sessão e estável num F5 (FNV-1a e *mulberry32* sobre o id da sessão). Cada envio é uma tentativa, e a isca volta à bandeja e conta: sem isca, a EF15CO02 daria “atingiu” para toda criança que tocasse três blocos. O número da tentativa vem da tela, e o reenvio da rede não soma.
A diagramação em sala	A IA escolhe UM de três layouts (`classic`, `hero-image`, `columns` — os `NEWSPAPER_LAYOUTS` do DS) e nada mais (Capítulo 11). Falhou qualquer coisa, o modelo automático (`regras/layout.ts`) diagrama pela forma — a imagem da manchete e o tamanho dos títulos —, nunca pelo gabarito, e a tela não fica sabendo. O layout é cosmético: não vale na correção.
Equipes	A missão nasce com equipes (**ADR-111**): o painel, o histograma e o radar contam por COMPUTADOR (`colapsarEmUnidades`), e o “X de Y publicaram” tira o Y da montagem. A linha do painel é só progresso — nenhum campo de `PROIBIDO_NA_LINHA_DO_PAINEL`, com teste —; as verificadas vivem no histograma anônimo de 0 a 3, e o radar é do OBJETO editorial: em quantas capas cada notícia foi.
Evidência	A ficha `evidencias/fichas/exp04ef1b.ts`, a terceira da plataforma: EF15CO04 pelas pistas abertas antes de publicar (os três tipos em duas notícias é `atingiu`); EF15CO02 pela Chave (uma tentativa, `atingiu`; duas ou três, `em_processo`); EF04CO08 pela 2ª edição (os pares notícia × tipo abertos nas duas fases e as verificadas) e pelas perguntas `p1` e `p2`, valendo o melhor — o que afrouxa o “E” do PRD. Na rodada nova do mesmo pacote, quem já foi avaliada não é marcada de novo (**ADR-120**), e as três ações são copiadas aos colegas de computador. A síntese final é UMA só: o recibo das perguntas é `{ enviada: true }` para qualquer alternativa.

Mídia	As imagens são do professor, no preparo: 640 por 360, duas por chamada (o BFF corta em 30 s), com o nome igual ao *hash* da descrição, em `experiencias/exp04ef1b/jornal/` — fora de `exp04ef1b/videos/`, que o UnooTube deixou. Sem imagem, o cartão se desenha pelo meio, e a missão roda igual.
Sem texto livre	Toda resposta da criança é vocabulário fechado: ids de notícia, tipo de pista, bloco da Chave, motivo e alternativa. Não há campo aberto a conter, e por isso o gestor lê o mesmo painel que o professor (Capítulo 7).

12.6.1 O preparo: a estrutura é do servidor, o texto é da IA

O professor prepara a turma em "A aula e os resultados", com três caminhos: seguir no banco padrão (que já está em sala), "Usar Banco Padrão" em um toque, ou gerar um pacote com IA sobre a disciplina, o tema e os tópicos dele. Na geração, o **molde** — que notícia é de que meio e que papel tem no gabarito (publicável, antiga, sem fonte ou sem autoria) — é SORTEADO no servidor a cada chamada (`regras/molde.ts`), e a IA só escreve o conteúdo de cada vaga. O sorteio existe porque os ids são a posição na mesa: com o papel amarrado à posição, a turma que já jogou um pacote saberia que "a `n2` é a antiga" em todos os outros. O `validar` do `generateJson` recusa a saída que mexeu no molde ou fere `invariantes.ts`: pelo menos três publicáveis, duas ou três antigas, duas ou três sem confirmação; meios tradicionais e digitais dos dois lados do gabarito, e imagem também; pelo menos quatro notícias com imagem, e sempre na TV e na rede social; as réguas de palavras; e nenhum termo de carimbo ("falsa", "fake news", "você errou") no que a criança lê.

O preparo não esconde a falha. Sem saldo, `POST .../gerar-conteudo` responde 429; com o provedor fora, 503 — e o hub oferece o banco padrão em um toque. "Regenerar notícia" mantém o id, o meio e o papel da que sai; se a IA falhar, devolve 200 com `usouIa: false` e a notícia como estava. Nada chega à turma sem "Aprovar pacote", que recebe as oito notícias INTEIRAS: curadoria é editar, e editar uma notícia edita também o gabarito e a consequência dela. A homologação passa pela mesma régua, e o pacote `ia` sem quem o aprovou é recusado pelo banco (Seção 8.12).

12.6.2 Ausências deliberadas

- **Sem arrastar**: "Adicionar ao Jornal" e "Trocar Matéria" são o caminho; o arraste é atalho, fora desta versão.
- **Sem reordenar as vagas**: trocar a manchete é esvaziar a primeira vaga e escolher outra.
- **O Bento não tem áudio** até haver gravação.
- **O balão do leitor não diz de qual notícia é**, e a reação é da própria notícia — nunca uma frase sobre quem escolheu.
- **Sem ranking da turma** nem exposição de quem escolheu o quê: a criança não vê a edição de ninguém.

> **ATENÇÃO**
>
> O manifesto declara `viewport` 375 por 812, `permissoes: ["forms"]` (o `RadioGroup` do motivo e do Desafio Final) e as capacidades `telemetria`, `progresso`, `ia` e `avaliacao`. A porta em *dev* é a 5174, com `strictPort`. Em *dev* e na suíte de ponta a ponta a IA é o *mock*: a diagramação cai sempre no modelo automático, e as imagens não são geradas — os dois são comportamento declarado (**ADR-117**), não defeito.

A suíte `scripts/e2e/estudante/12-jornal-da-escola.mjs` percorre a missão com a professora e dois estudantes da mesma turma: a peça imitada na gaveta, a pista que desce do servidor, o motivo depois do *latch*, os dois portões pelo *polling* com um F5 no meio, a isca que volta à bandeja, o antes e o depois, e o painel sem nota por criança. Ela **roda verde** desde 2026-09-27. Rodá-la a partir de uma worktree exige a api, o hub e o Vite da missão nas portas alternativas (3011, 3010 e 5174) sobre um banco descartável, sem subir o *compose*.

12.7 backup-exp04ef1b — UnooTube (arquivada)

Arquivada em 2026-09-25 (**ADR-116**): o Jornal da Escola tomou o código `exp04ef1b`. O que segue descreve a missão como ela era; o código continua no monorepo com o prefixo `backup-`, compilando e testando, mas não é servido, publicado nem listado — só aparece na área `/unooria/arquivados`.

Clone de plataforma de vídeo, para o 4º ano. O estudante salva até três vídeos; ao terceiro, o ranking desbloqueia; "adicionar credibilidade" reordena o ranking por fontes e é a virada pedagógica.

Fluxo (1.1.0). Entrada (a tela de entrada da missão) → *feed* → três favoritos na bandeja → "Concluir missão" → diálogo → *ranking* → "Adicionar credibilidade" (a virada). Até a 1.0.0 o estudante caía direto no *feed* sem saber o que estava fazendo, e o terceiro salvo abria o diálogo sozinho — a palavra "missão" só aparecia depois de a medida já ter sido tomada.

Decisão	Motivo
Gabarito (`factual`) fora do *bundle*	Vive em `src/backup-unootube/catalogo.ts`. A API corrige e devolve inscritos e fontes — nunca a resposta (regra 3).
Ranking com colegas **fictícios**	Distribuição desenhada para a lição. Não há pontuação de menor real exposta a outro menor (LGPD).
URLs das fontes como **texto**	O *sandbox* do `iframe` bloqueia *popups*, e o estudante não sai da plataforma.
Salvos **congelados** depois de "adicionar credibilidade"	O *score* da experiência já foi reportado — nota e estado não podem divergir. Curtir segue livre; reiniciar descongela.
`ranking_desbloqueado` é *latch*	Remover um salvo depois não tranca o ranking de novo. Estado que anda para trás confunde a criança e o professor.
Sem `viewport`	Foi desenhada para *desktop*: não entra em casca de aparelho.

Tema escolhido pelo estudante	Perfil → APARÊNCIA, lembrado em `localStorage` — que a regra 4 proíbe para *token*, não para preferência de tela.
Entrada só com a missão **zerada**	`src/lib/missao.ts`. Recarregar no meio cai direto no *feed*: repetir a tela de entrada a cada `F5` faria o estudante pular a leitura justamente na vez em que ele leria. Reiniciar pelo perfil volta à tela de entrada.
Conclusão pelo **botão**, não pelo 3º salvo	O terceiro salvo habilita "Concluir missão"; quem fecha é o estudante. A distância entre `missao-3-salvos` e `concluir-missao` é quanto tempo ele passou revendo a escolha. Nada muda na API: `ranking_desbloqueado` já vem do servidor no 3º salvo.
Limite de 3 vira **diálogo**, não *toast*	O *toast* sumia em 5,2 s e não dava saída. O diálogo põe a troca na tela, com os três favoritos e um "Remover" em cada — e não repreende. Usa o `Modal` do **clone**: quem fala é o aplicativo sobre a própria mecânica.
Métricas de popularidade **visíveis**	`visualizacoesRotulo` e `curtidasRotulo` no catálogo, sem nenhuma marcação de verdadeiro ou falso. São cosméticas — o gabarito continua sendo `factual` — mas não são aleatórias: o vídeo *fake* é o que tem os números grandes, e há teste que garante isso.

NOTA

O UnooTube foi o caso canônico da **linha da revelação** (Capítulo 14): *feed*, cromo, *playlists* e canal usam a seção do clone; ranking, os dois diálogos da missão, a tela de entrada e o botão "Concluir missão" usam a seção da plataforma. O *toast* e o diálogo de limite **não** cruzam a linha — os dois falam de dentro da ficção ("Vídeo salvo!", "sua *playlist* tem três lugares"). O critério nunca foi "é depois no tempo?", e sim "quem está falando?" — ver **ADR-22**.

REGRA INVIOLÁVEL

O *badge* `+N` de ganho de inscritos, ao lado do chip do canal, torna legível **durante** a missão o padrão *fake* $= +1.000$ contra confiável $= +50$. Foi aceito de propósito (**ADR-22**): o total já se movia e o delta sempre foi derivável de dois quadros seguidos. É o primeiro suspeito se a distribuição de `fontesConfiaveis` da turma subir de forma anômala.

12.8 exp02em1b — unooGPT (Missão Tempestade)

Porte de um projeto autônomo (`gpturia`), publicado na trilha do Entre Redes em **2ª série do Ensino Médio, unidade 1**. Um clone de assistente de IA em que o estudante decide, sob incerteza e com o relógio correndo, onde alocar duas equipes de resgate entre quatro bairros de uma cidade fictícia antes de uma tempestade.

> O propósito **não é acertar a alocação**. É experimentar que perguntas melhores e dados melhores produzem respostas melhores — e que nenhuma delas produz a resposta certa.

12.8.1 A regra de ouro: a Lacuna não cruza a fronteira do `iframe`

Cada um dos oito conjuntos de dados distorce a cidade de um jeito: o satélite vê telhado e não vê morador, e infla o bairro industrial em 1,6 vez; o cadastro de moradores está velho onde a cidade cresceu. Essa distorção — a **Lacuna** — é o **material do debate do professor**, e a experiência nunca a revela.

A consequência é arquitetural: **não há revelação no software**. A exp04ef1b troca da seção do *design system* do clone para a da plataforma na virada da missão; aqui não existe virada, e por isso a exp02em1b é 100% da seção `chatgpt` e não importa `unooria.css` nenhum. A marca institucional entra só como *chrome* do hub, desenhada acima do `iframe`.

O que isso proíbe no *bundle*, verificado por `grep` no `dist/` como parte da conferência de entrega: `instrucao` (o viés de cada fonte), `pSevero`, `expostosSevero`, `expostosBrando` e `fracaoVulneravel` (os parâmetros ocultos do mundo), `CAPACIDADE_DA_EQUIPE`, `JANELA_CONFORTAVEL`, `EFICIENCIA_MINIMA` e `TETO_DE_COBERTURA` (a calibragem), e o *prompt* de sistema.

12.8.2 Três contas que o cliente não faz

Três cálculos que o projeto de origem fazia no navegador subiram para a API no porte, e o motivo é o mesmo nos três: um *bundle* capaz de computá-los deixaria o estudante antecipar o desfecho antes de enviar.

Conta	Por que subiu, e como chega ao cliente
Minuto da cidade	Depende do **pivô** da extensão de tempo (**ADR-25**), uma conta sutil que existia nos dois lados — duas cópias, duas chances de divergir. Chega pronta em `EstadoDoRelogio`; o cliente só faz o número descer entre dois *pollings*.
Projeção das rotas	Usa `chegaATempo` e a capacidade da equipe. Chega em `Projecao`, e é ela que produz o aviso "chega com N de sobra" ao lado do botão que manda.
Alcance por bairro	Idem, com o `aperto` ("chega, mas raspando") já resolvido como booleano — era a constante de calibragem `JANELA_CONFORTAVEL` viajando no *bundle*. Chega em `BairroPublico.alcance`, com o mesmo portão do `tempos`: só existe com a malha viária carregada.

12.8.3 Ponto cego é ausência, não zero

> **REGRA INVIOLÁVEL**
>
> `BairroPublico.estimativa === null` significa "nenhuma fonte carregada cobre este bairro". A tela **nunca** pode escrever "0 pessoas em risco" aí — a verdade é "não sabemos".

É a mesma distinção que separa `zero` de `none` no relatório do professor, e era um **defeito no projeto de origem**: `montarMapa` devolvia uma estimativa para todo bairro, inclusive os sem cobertura, com todos os números zerados — e ela atravessava até a tela. O comentário do

próprio módulo do modelo já dizia a intenção (“precisa aparecer como ausência, não como número ruim”) e o tipo do sanitizador já prometia `null`; só o código não cumpria. A correção está em `regras/camadas.ts`, e o teste de camadas fixa o invariante.

12.8.4 O Relógio da Aula

O relógio é da **turma**, não do estudante: quem dá a largada é o professor, e ela vale para a sala ao mesmo tempo. Detalhes e alternativas em **ADR-25**. O que o professor controla é *quando dá para falar* com o assistente — nunca o Passo de ninguém.

12.8.5 O que o relatório mede do tempo

Tudo o que é *quando* entra pela trilha (`exp02em1b_marcos`), nunca pela telemetria: o canal analítico é preenchido pelo cliente, assíncrono e sem leitor, e o que o professor lê não pode vir dele. Cada marco carrega o relógio da turma **recalculado no instante** da gravação (`segundos_restantes`, `motivo_do_fim`) — não o `minutos_gastos` da sessão, que só o turno do assistente atualizava e atrasava até um turno inteiro nas rotas diretas. A correção mexeu também no jogo: quem alocava pelo mapa e enviava sem falar de novo saía no minuto de um turno atrás (**ADR-64**). Dois marcos novos, `inicio` e `recomeco`, marcam a entrada na Investigação e o clique em Recomeçar; a fala do estudante é datada pela hora de *envio*, não pelo fim do turno. O que se **deriva** na leitura, sem coluna: mudanças de destino (pares consecutivos distintos, não cliques — o cartão de duas opções manda um POST por equipe, e justificar passa pela mesma rota), tempo até a primeira fala (da largada do professor e do clique em Começar, os dois), % do risco coberto e posição na faixa. Não existe “liberar recomeço”: o professor controla o relógio, e é o relógio que fica registrado ao lado do clique.

12.8.6 Telas do professor

- `/professor/entre-redes/2-serie/unidade-1` — a ficha, que **abre com a faixa da largada**: sem ela a aula não começa, e “não abre” é o suporte que a faixa evita.
- `.../unidade-1/resultados` — o painel do relógio (largada, +2 min, encerrar, modo livre, redefinir) e o **retrato de conduta** da turma. Não há coluna de resgatados: listagem é ordenável, e ordenar crianças por acerto viraria *ranking* (**ADR-27**). O *smoke* visual falha se uma aparecer.
- `.../unidade-1/estudantes/:estudanteId` — a linha de uma criança: os graus, **como usou o tempo** (até a primeira fala, cada marco datado pelo cronômetro que ela via, mudanças de destino antes de confirmar, com quanto relógio confirmou ou se estourou, e qual botão de recomeço apertou), o desfecho de cada tentativa (resgatados, em risco, % do risco coberto, posição na faixa da decisão, justificativas) e a **transcrição** com a hora de cada fala. Só o professor da escola chega aqui, e o número de resgate só aparece aqui — uma criança por vez (**ADR-64**).

- "Zerar a turma" fica no fim da página, longe da largada: ação destrutiva ao lado do botão mais apertado é acidente esperando acontecer.

12.8.7 `allow-forms`: o bug que só o *smoke* pega

> **ATENÇÃO**
>
> O manifesto declara `"permissoes": ["forms"]`, e **sem isso a missão não funciona**. O `Composer` do *design system* é um `<form>`, e num `iframe` com *sandbox* sem `allow-forms` o navegador bloqueia o *submit*: o estudante digita a pergunta, aperta Enter e nada acontece.

O sintoma engana de propósito — a conversa parece viva, porque a fala de abertura do agente já está na tela, e o único rastro é um aviso de console sobre "*form's frame is sandboxed*". Passou pelo *typecheck*, pelo *lint* e por duas versões do *smoke* antes de ser encontrado. A correção usa o mecanismo que a plataforma já tinha, `permissoes` no manifesto, como o `pointer-lock` da app-70: por experiência, porque permitir *submit* de formulário é capacidade de quem declarou.

12.8.8 O *smoke* mentiu duas vezes

O roteiro de `scripts/e2e/estudante/04-unoogpt.mjs` usa **dois contextos** de navegador — o professor dá a largada, o estudante joga — porque um contexto só passaria com o relógio quebrado. Duas asserções foram descartadas no caminho, e as duas passavam com a funcionalidade quebrada:

1. *esperar por texto do assistente*: a fala de abertura dele já está na tela desde o começo;
2. *esperar pela fala do estudante*: o cliente a mostra otimistamente e só a retira cerca de 30 segundos depois, quando a API falha.

A asserção honesta é a conversa **sobreviver a um F5**, porque aí ela veio de `exp02em1b_mensagens`. Teste que passa com a funcionalidade quebrada é pior que teste nenhum.

12.8.9 O guia da aula, e a tabela dos vieses

O guia do professor do projeto de origem estava **vazio** — o material de condução do debate nunca havia sido escrito, e é no debate que a lição acontece, porque a missão deliberadamente não revela nada por software. Ele existe agora em `/professor/entre-redes/2-serie/unidade-1/guia` (Seção 12.4), com roteiro, banco de perguntas em quatro níveis, mediação e rubrica.

> **DECISÃO DE PROJETO**
>
> **A tabela dos oito vieses vem da api, não de uma cópia no hub.** `GET /unoogpt/professor/fontes` (professor-only) devolve, por fonte, o que ela mostra,

> o período, os bairros que cobre e **em que ela engana** — e o viés é derivado da própria `instrucao` que o agente recebe, cortada na marca `VIÉS:`. Se a marca sumir numa recalibragem, a função lança e o *build* quebra: melhor isso do que servir ao professor a instrução inteira como se fosse o viés, e ele contar aquilo à turma.
> O guia diz, em destaque, para não projetar essa tabela nem entregá-la impressa: quem sabe de antemão qual fonte engana em quê decora um quadro em vez de descobrir a discordância cruzando dados.

> **LIMITAÇÃO CONHECIDA**
>
> A ficha do catálogo e o guia foram redigidos a partir das ADRs, dos dois *playtests* documentados e do próprio cenário. **Nenhum dos dois passou por playtest em sala** — estão marcados no código como ponto de partida para revisão pedagógica, não palavra final.

12.9 exp01ef1b — O Que a Foto Conta

1º ano · Unidade 1 · clone do Instagram · `instanooria`

A porta de entrada do Entre Redes, para crianças em alfabetização inicial. No mural da turma (`@turma_mural`), duas fotos do MESMO pátio contam histórias opostas: o plano geral com as peças espalhadas e o plano detalhe do brinquedo montado. O estudante escolhe a *marca* (Coruja, Estrela ou Sol), explora um *feed* fechado de 3 posts, publica UMA foto com UMA legenda — e Leo e Sofia, colegas simulados, comentam em cadência fixa (1,5s e 3,0s) exatamente o que a escolha deixou ver e o que escondeu. Depois do eco, **todos os tablets congelam** numa tela sem nenhum botão (a Roda de Conversa) e só o professor os solta, pelo `LIBERAR REFLEXÃO FINAL` da tela de resultados. A síntese final é formativa: o Globo ("a foto mostra tudo") vira retry gentil de Nina, e a Lupa conclui com a insígnia "Olhar de Detetive".

O que define a missão, tecnicamente:

- **O latch é do servidor e o canal é polling de 3s** na rota `GET /foto-conta/latch` (**ADR-94**) — sem WebSocket e sem SSE, como as demais superfícies síncronas da plataforma.
- **A matriz causal do Aha nunca entra no bundle**: o boot serve o pacote SEM `comentarios` (`Omit` estrutural no contrato), e o par escolhido volta na resposta de `POST /foto-conta/publicar`, uma linha por vez — o `factual` do UnooTube desta missão.
- **Acessibilidade de 1º EF é requisito**: CAIXA ALTA, máx. 8 palavras por texto, pictograma e locução em toda instrução (redundância tripla). O áudio TOLERA ausência: as locuções profissionais são pendência de produção e a tela degrada para texto+ícone.
- **As duas famílias do DS convivem no mesmo bundle**: E1/E5/E6 são casca Unooria (`@edu/ui/ds`); E2–E4 são o clone (`@edu/ui/ds-insta`, importando do subcaminho do pacote).

- **O pacote da aula é curado pelo professor**: a IA (Vertex, com quota) escreve legendas e comentários adaptados ao tema; a aprovação revalida com o MESMO *schema*; sem aprovação, o banco curado padrão ("O Mistério das Peças no Pátio") assume sozinho.
- **Zerar a turma ARQUIVA por tentativa** (`tentativa` na config e nas *runs*): nada é apagado, e a reaplicação recomenda trocar o tema (PED-13).
- **100% formativa**: sem *score*, sem *ranking*; a lista de *hesitação de leitura* (>60s na decisão) nomeia crianças SÓ na rota autenticada do professor, para mediação — o modelo e a telemetria só conhecem o id opaco (DAD-07).

12.10 exp08ef4b — O Manifesto da IA

8º ano · Unidade 4 · clone de assistente generativo · `chatgpt` + `unooria`

A direção está homologando a compra de um sistema que prevê notas e monitora presença, e o assistente da escola já está *online* afirmando que algoritmos são neutros e objetivos. O estudante questiona a origem dos dados históricos, ouve a máquina admitir que assume as punições antigas como "probabilidade estatística válida" — e então fixa **uma** cláusula de governança que o código passa a obedecer. O sistema compila o *Manifesto de Direitos Digitais dos Estudantes* com a cláusula dele em destaque.

12.10.1 A missão em que não existe resposta certa

É a diferença estrutural desta experiência para todas as outras do catálogo, e ela atravessa o desenho inteiro: as **quatro cláusulas são igualmente válidas** sob governança de direitos — transparência dos dados, auditoria de viés, veto humano e anonimização. Não há gabarito a esconder do *bundle*, porque não há gabarito.

O que isso muda, ponto por ponto:

Onde apareceria	O que existe no lugar
Correção da resposta	Nada a corrigir. O servidor cobra a *cota* de auditoria, trava a escolha (*latch*) e **compila** o manifesto — as três coisas que ele decide.
Nota da missão	Nenhuma. O `exp:concluir` da ponte tem o campo `score` e esta experiência **não o manda**: um número ali deixaria a tela pontuar uma escolha que a missão não pontua. O progresso da plataforma lê o `concluido_em`.
Taxa de acerto no painel	A distribuição das quatro cláusulas como **contagem**, com a frase dizendo o que ela é. Com quatro opções e uma "mais votada", calcular percentual de acerto é o erro natural de quem chegar depois — e `PROIBIDO_NO_PAINEL` tem o teste que cobra a ausência de `acerto` e `taxa` entre as colunas.

12.10.2 O portão que a missão existe para instalar

Fixar a regra do sistema sem tê-lo **contraditado duas vezes** é **recusado** (`auditoria-insuficiente`, 409). O número dois é a hipótese pedagógica, não uma folga: o primeiro questionamento a máquina responde com estatística; o segundo — sobre os preconceitos anteriores ao algoritmo — a obriga a admitir que herda o passado. Chegar ao pico sem ter visto a contradição é decidir sobre o que a aula ainda não mostrou.

Distintos é a palavra que faz o trabalho, e aqui ela se comporta DIFERENTE da exp06ef2b: reenviar o mesmo questionamento responde **409** (`prompt-ja-enviado`), e não 200 com `novo: false`. A razão é o que a tela faz: lá reabrir um balão é releitura legítima, e um erro transformaria o comportamento desejado em falha; aqui o cartão já disparado fica desabilitado no seletor, então o segundo disparo do mesmo id só chega por clique duplo ou aba repetida — e a conversa não pode ganhar a mesma pergunta duas vezes.

12.10.3 A réplica da máquina só existe DEPOIS da pergunta

`GET /manifesto-ia/sessao` devolve os três questionamentos **sem** a `replicaIa` (o tipo do servidor é `Omit<PromptDeAuditoria, "replicaIa">`), e as quatro cláusulas **sem** a `justificativaSugerida`. Ver a defesa antes de fazer a pergunta desmontaria a cena da auditoria; ver o argumento pronto de cada cláusula transformaria a autoria da Etapa 5 em cópia — ele é material de **mediação**, e aparece na rota *professor-only* do pacote da turma.

12.10.4 Sem digitação livre, e a ausência é guardrail

O `Composer` do clone é `readOnly`: o estudante escolhe um dos três cartões de questionamento, e o texto dele preenche o campo. Prompt aberto num clone de assistente é a porta para *jailbreak*, alucinação e conteúdo fora da faixa numa aula de 50 minutos com adolescentes — e o objeto da aula não é conversar livremente, é auditar um argumento específico. Os alvos de anexo e de voz também saem (`hideLeadingActions`): alvo que não faz nada ensina o estudante a desconfiar da tela.

> **ATENÇÃO**
>
> O `Composer` é um `<form>`, e por isso o manifesto declara `permissoes: ["forms"]`. Sem o *token* de *sandbox* o navegador bloqueia o *submit* em silêncio — a armadilha que custou um defeito inteiro na exp02em1b. E `clipboard-write` **não** entra ali, apesar de o botão de cópia precisar da API: o vocabulário de `permissoes` é fechado (`pointer-lock | forms | downloads`) e o `allow` do `ExpFrame` já concede *clipboard* a todas as experiências. Declará-lo derrubaria o `PUT /experiencias/:expId` do *deploy* com 400 — depois de os *bundles* já terem subido.

12.10.5 A IA entra ANTES da aula, e só

O motor de sala é 100% determinístico: as réplicas, as cláusulas e o manifesto saem do pacote da turma ou do catálogo curado. A única chamada ao provedor está na **preparação docente** (`POST /manifesto-ia/turmas/:id/gerar-conteudo`), com *structured output* e *schema* estrito, e ela devolve **rascunho**: o que publica é o clique do professor em "Aprovar Conteúdo da Turma", que grava `aprovado_por_professor`. Sem esse clique a missão lê o banco curado da Unooria — que é homologado, não remendo.

A consequência é a que a banca registrou como ponto forte do insumo: **zero tokens durante a aula**, e nenhuma turma de 35 estudantes às oito da manhã depende de o Vertex responder. A validação da saída rejeita balão acima de 60 palavras e termo proibido em tela, e rejeitar faz o modelo *regerar* — vale mais que aceitar e contar com a revisão de um texto que o professor talvez só passe o olho.

12.10.6 O Checkpoint A é polling, não WebSocket

O congelamento da turma para o debate oral de dez minutos é uma linha em `exp08ef4b_gates_turma` que a tela lê a cada 5 segundos. O *handoff* pedia *broadcast* `hub:pause` por WebSocket, e não há WebSocket de aplicação no Cloud Run: a decisão inteira, com as quatro alternativas descartadas, está na **ADR-93**.

Duas propriedades desse gate que o separam do da exp06ef2b: ele é **opcional** (a missão corre inteira sem clique nenhum, porque a especificação admite a sessão individual assíncrona) e mexe só no **teto** do passo, nunca no piso — pausar não empurra para a frente quem ainda estava auditando a máquina, e são justamente essas crianças que o debate tenta alcançar.

12.10.7 O único texto livre, e o dado pessoal que é recusado

A Etapa 5 pede duas palavras-chave e um parágrafo autoral de 20 a 800 caracteres. É caixa escrita por menor de idade: fora de *log* (`justificativaTexto` entrou em `CHAVES_REDIGIDAS`), fora da telemetria analítica, e alcançável só pelo professor da turma dela.

Dado pessoal detectado é recusado com 422 e NÃO gravado. A pergunta estava aberta no insumo — recusar com dica, ou gravar com *flag* de moderação — e as duas alternativas foram descartadas: gravar com *flag* deixaria o telefone da criança na coluna à espera de alguém que decidisse o que fazer com ele, e ofuscar-e-salvar devolveria ao professor um parágrafo mutilado sem que ninguém soubesse o que estava ali. Recusar também **ensina**, que é o ponto: a dica diz o que tirar no exato momento em que a criança ia entregar o dado.

> **NOTA**
>
> O filtro é deliberadamente conservador nos padrões estruturados (telefone, CPF, e-mail, CEP, cartão, `@perfil`) e **não tenta adivinhar nome próprio**. Recusar "meu tio Silva foi

> reprovado por um sistema desses" cortaria justamente o argumento situado que a etapa mede, e a criança não teria como saber o que fazer com a mensagem. Nome próprio é risco **aceito e registrado**. E nenhum padrão casa número de quatro dígitos solto: o falso positivo mais provável é o ANO ("desde 2019 a escola usa esse sistema").

12.10.8 As duas seções do design system, e a fronteira

A linha é a de sempre — **quem está falando** —, mas aqui ela **não é uma revelação**: o estudante sempre soube que era uma aula, e o clone é o ambiente da tarefa, não uma armadilha. `unooria` no briefing de Amara, no congelamento, no Meu Percurso e no fechamento; `chatgpt` da abertura da conversa até o manifesto promulgado.

O que a fronteira protege é o **tom**: a máquina precisa soar corporativa para a tese de neutralidade dela ser derrubável, e um `Button` institucional dentro da conversa a faria soar amigável no exato momento em que ela defende a vigilância.

12.10.9 Os áudios ainda não existem

O *brief* pede `e1-alerta-amara.mp3` (~650 KB) e `e6-podcast-manifesto.mp3` (~3,8 MB, no *bucket* de mídia escolar). Nenhum foi gravado. A tela mantém o **texto sempre visível** — acessibilidade de sala sem fone, não *fallback* de mídia ausente — e mostra o botão de ouvir **desabilitado com o motivo à vista**, em vez de um *player* que não toca nada. Quando os arquivos chegarem, entram ao lado do texto.

12.11 exp05ef1b — Quem mandou isso? (v2.2)

5º ano · Unidade 1 · clone de mensageiro móvel · `whatsappmobile` · 0.2.1

A Nina pede ajuda: "rolou uma confusão sobre" o tema que a professora escreveu no preparo. No grupo "Turma 5ºB" chegam oito mensagens de colegas — duas em cada quadrante da matriz verdadeira ou falsa × emocional ou neutra —, e a criança encaminha **exatamente três** ao "Mural da Turma", confirma e diz, numa enquete, o que pesou na escolha. Quando a professora libera a revelação, cada mensagem do Mural ganha **dois** crachás, e a descoberta é que um eixo não prediz o outro: a mensagem séria podia ser falsa, e a alarmada, verdadeira. Depois vêm a síntese, a regra "Se a mensagem for…, então eu…" e duas perguntas rápidas da Nina. A primeira versão saiu na **ADR-114**; esta volta com o mesmo código, e as decisões dela estão na **ADR-118**.

12.11.1 O pacote é a porta, e é a atividade

A missão nasce vazia (o padrão da **ADR-108**, retomado): sem pacote aprovado pela professora daquela turma, a lista de conversas não tem a Nina — tem uma linha de espera, e quem tira a

criança dali é o *polling*, sozinho, quando o pacote é aprovado. Não há queda para o banco curado: a fala da Nina leva o *tema* do pacote (“Rolou uma confusão sobre {tema}”), e um grupo sobre outro assunto seria uma conversa sobre um fato que não houve na escola dela. Por isso o tema herda as réguas do clone: até 60 caracteres, até 8 palavras, e sem “atividade”, “gabarito”, “nota” nem “exercício”.

O preparo é da professora, em “A aula e os resultados”: componente curricular, tema, de 3 a 6 fatos da aula e o contexto opcional. “Gerar Atividade com IA” escreve as oito mensagens **pré-aula**, pelo `QuotaService` (`temSaldo` antes, `consumir` depois); sem saldo ou com o provedor fora, a api devolve o banco curado (“O Mistério do Passeio ao Museu”) como rascunho, e a tela diz por quê. O rascunho é gravado — o preparo custou saldo, e um F5 não pode jogá-lo fora. A curadoria mostra os quatro quadrantes com os selos que só ela vê; ela edita qualquer TEXTO, mas não o gabarito nem o formato de cada posição. Nada chega à turma sem “Aprovar Pacote”.

O pacote aprovado é a ATIVIDADE da evidência pedagógica (**ADR-112**): o identificador do pacote é o `atividadeId` das marcas. “Zerar Turma para Nova Aplicação” abre a RODADA seguinte do mesmo pacote e não apaga nada; aprovar um pacote diferente abre atividade nova, e aprovar o mesmo é idempotente. Os dois voltam a turma à triagem — senão a turma nova nasceria com a revelação aberta, e o gabarito desceria na entrada.

12.11.2 A matriz equilibrada, e o formato que não pode ser pista

A régua do pacote (`regras/pacote.ts#validarPacote`) vale igual para o gerado e para o curado, e é a mesma que a aprovação cobra: **duas mensagens por quadrante**, quatro textos, duas imagens e dois links, balão até 45 palavras e legenda até 25 — e **imagem e link com uma verdadeira e uma falsa cada**. Sem a última, o formato viraria a resposta. A IA não decide a estrutura: `sortearMolde` sorteia o tipo e o quadrante de cada posição (a posição não pode virar pista entre turmas), o modelo só escreve, e a saída passa pela mesma régua da curadoria.

> **NOTA**
>
> Nos artefatos do documento de requisitos, os dois links do pacote curado eram falsos. A régua os reprova de propósito: no curado, a `msg_5` (verdadeira) passou a link e a `msg_7` (falsa) a texto, sem mudar uma palavra, e a versão original reprova num teste que prende isso. É *default* até o dono do produto responder (**ADR-118**).

12.11.3 Exatamente três, e três *latches*

O passo do estudante **não é coluna**: é `passoEfetivo(latches, fase)`, em `regras/passos.ts`. Os *latches* são carimbos da sessão, cada um com *CHECK* exigindo o anterior (Seção 8.11):

Gesto	Rota	O que fecha

A seta de encaminhar	`PUT /selecao`	Nada: é o rascunho da triagem, de 0 a 3, guardado para o F5. Quatro é 400 `selecao-acima-do-limite`.
“Confirmar envio”	`POST /mural`	O **latch 1**: exatamente três (2 é 400 `selecao-incompleta`), com a marca da EF05CO08 na MESMA transação. O corpo leva a seleção que a criança está VENDO, e não a última que o `PUT` gravou. Outra seleção depois é 409 `mural-ja-enviado`.
O voto	`POST /enquete`	O **latch 2**: um dos quatro motivos. Quem votou espera a turma.
“Salvar e Enviar Reflexão”	`POST /reflexao`	O **latch 3**, só com a reflexão liberada: a condição e o “Então” de 15 a 240 caracteres DEPOIS do `trim`, recusado também quando é texto sem sentido (`entao-sem-sentido`).
Depois da P2	`POST /concluir`	O fim, sem exigir as respostas: quem sai antes delas fica “Não avaliado”.

A quarta seta **fala**: com três escolhidas, as outras esmaecem mas continuam respondendo com o aviso de limite — o estado `exhausted` do `ForwardButton` (DS 0.2.6), com `aria-disabled`, e não `disabled`, que engoliria o toque (Seção 14.11.6).

12.11.4 O gabarito desce UMA vez, e só o das três

As mensagens saem por `mensagensPublicas` (um `Omit` de `veracidade` e `tom`), e o *schema* de saída não tem lugar para eles. O bloco `revelacao` da sessão é `null` até o estudante **votar** E a turma estar na revelação — e traz só os dois eixos das TRÊS que ele encaminhou, mais o *callback* do motivo votado e as duas barras da turma. O teste da api varre as CHAVES, não o texto: a fala da Nina antes da P1 contém “verdadeira”. Na experiência, a varredura é por CONTEÚDO — o banco curado, as alternativas, o par id e eixo —, no `src/` pelo `vitest` e no `dist/` por `grep`: as chaves `veracidade` e os rótulos dos crachás existem no *bundle*, e está certo.

12.11.5 Três botões, sempre para a frente

`exp05ef1b_gates_turma` guarda a fase da turma — `triagem`, `revelacao`, `explicar`, `reflexao` —, e a professora a move por três botões, cada um com confirmação e nenhum com inverso: “Liberar Revelação”, “Apresentar Síntese” (que pode ser pulada) e “Liberar Reflexão”. O estudante lê `GET /gate` a cada 3 s (**ADR-93**) e relê a sessão quando muda o passo, o preparo, a fase OU a atividade — a fase porque quem ainda está no grupo quando a revelação abre continua no grupo, e a atividade porque um “Zerar” pode coincidir com o mesmo passo.

A revelação não empurra a triagem, e é a diferença para a v1: quem não tinha confirmado continua escolhendo, confirma, vota e só então recebe a própria revelação — junto com o passo

da turma, que pode já ser a reflexão. Nesse caso a experiência mostra o Mural revelado com "Continuar" antes da regra; o servidor não guarda "já viu", e um F5 mostra de novo.

12.11.6 A régua pedagógica v2: cada habilidade pelo verbo dela

É a primeira missão com ficha (`evidencias/fichas/exp05ef1b.ts`, versão 2, **ADR-120**). A **EF05CO08** sai das três do Mural: só três verdadeiras são `atingiu`; duas e uma falsa, de qualquer tipo, `em_processo`; uma ou nenhuma verdadeira, `nao_atingiu` (`regras/evidencia.ts`, conferida nas 56 combinações). Ela é marcada uma vez por criança e pacote: quem já foi avaliada numa rodada anterior da mesma atividade fecha o Mural sem marca nova (`EvidenciasService.jaAvalia`). A **EF05CO03** vem da P1, que pede a frase certa com "não", "e" e "ou" sobre os dois selos; a **EF05CO04**, da P2, que pede para seguir uma regra no áudio do tio, e da rubrica com que o professor classifica o Se/Então na folha da turma — um ajuste da evidência, que vale sobre a pergunta. As perguntas passam pelas rotas genéricas `/evidencias/exp05ef1b/perguntas/*`. O resolvedor que a missão registra (`registrarContexto`) diz a turma, o pacote ativo e a rodada, e só responde no passo das perguntas: uma P1 por `POST` direto antes do Mural é 409 e não mede nada. As alternativas chegam embaralhadas a cada leitura, e a tela responde pelo `id` e carrega cada pergunta uma vez por aba.

12.11.7 Onde a fronteira do disfarce cai — duas vezes

O critério é o de sempre, **quem está falando** (**ADR-72**), e aqui ele produz um vaivém. Do convite até a síntese é o mensageiro, revelação inclusive: os crachás são `MessageTag`, e o termômetro e a lâmina da síntese entram como IMAGEM enviada no chat — mensageiro não tem gráfico, mas recebe foto de gráfico. A regra Se/Então sai para a **casca** (`@edu/ui/ds`), porque é sobre o grupo da FAMÍLIA do tio, fora da ficção que a revelação desmontou. As perguntas P1 e P2 **voltam ao clone**: são a Nina perguntando, com a mesma enquete (`Poll`) da Etapa 3. E "Missão Concluída" é a casca de novo, a escola guardando a regra. Cada família mora num arquivo, e cada arquivo só importa a sua ponte.

12.11.8 Equipes e o Termômetro

A missão nasce com equipes (**ADR-111**): o painel e o Termômetro contam por COMPUTADOR, com `mapaDeEquipes` dentro do `comTenancy` e `colapsarEmUnidades`. O Termômetro é `null` antes da revelação e não tem lugar para nome: a porcentagem das unidades que encaminharam ao menos uma falsa neutra, a das verdadeiras emocionais que ficaram de fora ($(2N - \Sigma VE)/2N$) e o cruzamento com o motivo votado. Com zero unidades, a tela escreve "sem dados", nunca 0%. "Projetar o Termômetro" abre uma janela em tela cheia só com a conta; a folha nominal, com a regra de cada um, é para a tela da professora.

REGRA INVIOLÁVEL

O "Então" é texto livre de menor. Mora numa tabela confinada (`exp05ef1b_reflexoes`), está na redação do *log* (`entao`, `acaoEntao`, `acao_entao`), não vai à telemetria nem a `reportarErro`, e no painel só vem preenchido para quem LECIONA na turma. A distinção mora no *service* (`professorId ? texto : null`), e não no `@Roles`: `professorLeciona(undefined)` é verdadeiro para o gestor.

12.11.9 Sem mídia binária

Nenhum arquivo existe nesta missão. O comunicado é o texto do pacote desenhado como papel (`MessageBubble.Media` sem `src`, com o texto no `alt`); o link é a prévia montada da legenda, no formato "Nome do site: manchete"; tocar em qualquer um abre a mídia em tela cheia com a transcrição. Não há áudio assistivo nesta versão (entra quando houver gravação) e não há botão de ouvir.

ATENÇÃO

O `TextArea` e o `Selection` da casca são campos de formulário, e o manifesto declara `permissoes: ["forms"]` — sem o *token* de *sandbox*, o envio morre em silêncio (a lição da exp02em1b). O `viewport` é 375 por 812, e a porta em *dev* é a 5184, com `strictPort`.

LIMITAÇÃO CONHECIDA

A IA vive no preparo, e só. O `capabilities` do manifesto não declara `ia`: a única chamada ao provedor é a geração do pacote, pela professora, antes da aula. Curadoria, *latches*, régua, Termômetro e síntese são determinísticos — uma turma de trinta às oito da manhã não depende do Vertex responder.

12.12 exp07ef1b — A Seleção

7º ano · Unidade 1 · clone de rede social de fotos · `instanooria`

A Amara pede ajuda para cobrir um evento do bairro nos Stories do grêmio. O feed tem 12 posts de três perfis — o oficial, o comunitário e o de números, 4 de cada, intercalados — e todos contam fatos verdadeiros do mesmo acontecimento. O estudante salva o que acha informativo, poda os salvos até EXATAMENTE três, escreve uma legenda e publica, sem volta. No Mural, a tríade dele aparece ao lado da de um colega que escolheu outra coisa; quando a professora revela as fichas técnicas (fonte ouvida, ângulo da imagem e o equilíbrio das três lentes), a sala descobre que ninguém mentiu, e que cada seleção deixou dois terços de fora. A explicação vem em três cards com trava de 8 s; a reflexão, na voz da escola; e as duas enquetes da Amara, na DM, fecham a missão (**ADR-115**).

Decisão	Como está implementada
Atividade e rodada	A ATIVIDADE é a **cobertura**: o tema que a professora cria (partindo do banco curado da Praça Central ou de um lote gerado por IA). A plataforma não cria nada (**ADR-130**): sem cobertura em andamento, 403 `atividade-nao-preparada`. Criar não põe no ar — a professora escolhe a em andamento na lista, e pô-la no ar abre a rodada seguinte à última. Zerar a turma abre a rodada seguinte e guarda a anterior. É o que a evidência pedagógica exige (**ADR-112**).
Um botão só	A professora avança `exploracao` → `mural` → `fichas` → `explicacao`, uma etapa por vez e com o destino explícito — clique duplo não pula etapa. O estudante lê `GET /a-selecao/gate` por *polling* de 5 s (**ADR-93**).
O par do Mural	É do COMPUTADOR (**ADR-111**): `colapsarEmUnidades` sobre as equipes, distância máxima entre as matrizes, nunca dentro da mesma unidade. O par é carimbado por sessão, com o rótulo congelado (o primeiro nome ou o nome da equipe) — nunca pelo id da equipe, que o `PUT` recria. Sobra de sala ímpar, quem publica depois e a aula de um computador só recebem um colega virtual (Tomás ou Malu).
O que o bundle não sabe	A lente de cada post, as tags e a matriz vêm da api, e as fichas só depois da etapa `fichas`. A cópia que nomeia as lentes carrega sob demanda, na explicação. Um *grep* no `dist/` prova que não há gabarito, tag nem trecho de prompt no *bundle*.
Legenda e reflexão	Contam *code points* dos dois lados (o Zod e o *CHECK* do banco). A legenda passa por um sanitizador (link, telefone, e-mail, menção, palavrão) porque outro estudante a lê no Mural; a reflexão é só do professor da turma (**ADR-108**).
Evidência	A ficha `evidencias/fichas/exp07ef1b.ts`: EF07CO11 com a leitura da legenda por IA (o Canal A) e a enquete `p1`; EF07CO08 com a enquete `p2`. As alternativas são embaralhadas a cada exibição. Sem IA, o Canal A não marca nada, e fica "Não avaliado".
Mídia	A professora gera as imagens no preparo (2 por chamada, porque o BFF corta em 30 s), com cache no *bucket* por contrato em `experiencias/exp07ef1b/selecao/`. Post sem foto vira um card de texto com a MESMA forma para os três perfis, e o bloco de números vira gráfico desenhado a partir dos dados.
O que o DS cobre	Desde a 1.0.1 (DS 0.2.7), **nenhuma compensação local**: a faixa de dica e o aviso da poda são o `InlineHint`, o gesto de rolar é o `GestureHint`, a tarja de conexão é o `Snackbar` do clone, a etiqueta longa da ficha é o `Badge` com `wrap`, e o contador da legenda do `StoryComposer` conta *code points*, como a api.

12.12.1 O podcast de recapitulação é do professor

O documento de requisitos pede um episódio de cerca de 3m45s em que Lucas e Clara recapitulam a missão. O texto fala com o estudante, mas o podcast é do **professor**, que o toca à turma

depois da aula se quiser (**ADR-119**): ele mora em "A aula e os resultados", com o `AudioPlayer` do DS e a transcrição sempre na tela, e nenhuma rota de estudante o serve.

Decisão	Como está implementada
O roteiro	O do sheet aprovado, cinco blocos, em `a-selecao/podcast.ts`. Só **três falas** mudam com o tema — as do Bloco 2 que dizem o que a Folha Oficial, a Voz Coletiva e a Métrica Urbana mostraram.
O tema da IA	As três falas saem do MESMO lote do preparo, sob a régua da aprovação (12 a 60 palavras, até 400 bytes, sem link, e-mail nem "@"), e a professora as revisa junto com os posts. Ficam em `exp07ef1b_coberturas.podcast`; nulo lê as do sheet.
A voz	O TTS do Gemini pelo *provider* da api (`generateSpeech`): Lucas e Clara numa chamada de diálogo por bloco, com a direção de atuação do *brief*. Um trecho por chamada, porque o episódio não cabe nos 30 s do BFF.
O episódio	Com os cinco trechos, a mesma chamada os emenda num MP3 (`regras/mp3.ts`), tirando de cada um as etiquetas e o quadro de duração. Cada trecho é o *hash* do texto e do pedido de voz: os blocos fixos saem uma vez por contrato, e um tema novo paga só o Bloco 2.

A suíte `scripts/e2e/estudante/11-a-selecao.mjs` percorre a missão com a professora e dois estudantes de escolhas divergentes, e termina no podcast: as três falas no preparo, a transcrição e o ramo do áudio que o ambiente permitir (pronto com a IA de verdade; a transcrição sozinha com o *mock*).

12.13 exp06ef2b — O Rastro do Boato

6º ano · Unidade 2 · clone de mensageiro móvel · `whatsappmobile`

Um aviso de que a água da escola está contaminada corre quatro grupos em dez minutos — 480 encaminhamentos — e a nota da direção com o laudo do laboratório sai do lugar 12 vezes. O estudante abre os **dados de envio** de cada mensagem (quantos saltos anônimos, quem assinou, quando ela apareceu pela primeira vez), descobre que o alerta veio de seis encaminhamentos sem autor — e decide o que fazer quando chega um áudio convocando um protesto. Depois vê a própria decisão desenhada como topologia de rede.

12.13.1 O portão que a missão existe para instalar

Decidir sem ter aberto os dados de **três mensagens distintas** é **recusado** (`auditoria-insuficiente`, 409). Distintas é a palavra que faz o trabalho: reabrir o mesmo balão é releitura — devolve 200 com `novo: false` e não conta de novo, porque reler é o comportamento que a missão quer. A dedupe é do servidor (`regras/auditoria.ts`) e tem um `UNIQUE` no banco como rede: um contador incrementado no cliente seria um número que o estudante controla.

As três fontes que sustentam a descoberta ficam em **conversas diferentes** de propósito — o boato no grêmio, a amplificação anônima na turma e o laudo no grupo do bairro. Empilhá-las num chat só faria a auditoria caber numa rolagem, e a Etapa 2 perderia a navegação que ela mede.

12.13.2 As duas aulas, e quem separa uma da outra

`exp06ef2b_gates_turma` guarda uma etapa por turma (`aula1 | aula2`), e a professora a move por **um** botão no painel de resultados — e o inverso dele. São só duas etapas: o debate desta missão acontece *depois* do mapa, com a turma toda já na Aula 2, e não há o que congelar no meio. Gate que ninguém aperta enche o painel de controle morto.

Etapa	Piso	Teto — e por quê
`aula1`	`contexto`	`breakpoint`: quem decide cedo veria o mapa sozinho, e o Aha! foi desenhado para a turma ler os nós junta
`aula2`	`resultado`	`fim`

> **NOTA**
>
> **O latch é do estudante, não da turma.** "Pausar simulação" volta a etapa para `aula1`, mas quem já entregou o percurso continua em `fim`: uma volta de etapa não pode desfazer um registro que a criança fechou.

12.13.3 Encaminhar e calar não dão no mesmo número

O mapa da Etapa 4 desenha o alcance real de cada caminho: **1.200** conexões para quem encaminhou, **850** para quem apagou para si, e a corrente cortada no nó 3 para quem denunciou e fixou o laudo. A diferença entre 1.200 e 850 é o ponto da missão — se os dois empatassem, a tela ensinaria que a inação é neutra, e o que a missão descobre é o contrário.

Na interrupção os anéis evitados **continuam na tela, apagados**: o estudante precisa ver o que *não* aconteceu, e um grafo que simplesmente acaba no nó 3 não mostra o que foi evitado.

12.13.4 1.200 nós, e nenhum deles no DOM

O cálculo da malha é do servidor (`regras/topologia.ts`), por duas razões, e a segunda dita a forma:

1. **Segredo de servidor.** A matriz de propagação é parte do gabarito: com ela no bundle, o estudante lê no DevTools que a opção B corta a corrente *antes* de escolher, e a decisão que a missão mede deixa de ser uma decisão (regra inviolável nº 3).
2. **Performance no aparelho da escola.** 1.200 elementos SVG numa thread de Chromebook de entrada engasgam. A resposta não é desenhar menos e mentir no número: cada anel

traz `nos` (o total REAL, que a legenda escreve por extenso) e `desenhar` (quantos círculos pôr no DOM, teto de 36). A mancha comunica a escala; o texto dá o número. A suíte de ponta a ponta conta os círculos — 133, não 1.200.

12.13.5 Zero IA, e a ausência é o desenho

Nenhuma chamada a `apps/api/src/ai`, nenhum token de quota (`RNF-CUST-001`). Reações do grupo, topologia e validação do ciclo saem do catálogo e das regras puras. Uma aula de 35 minutos não pode depender de o Vertex estar de pé, e esta missão não precisa de linguagem gerada: o que ela mede é leitura de metadados e uma escolha.

12.13.6 Onde a fronteira do disfarce cai

O disfarce vale **até o fim da Etapa 3**. Do breakpoint em diante quem fala é a escola: a espera entre as aulas, o gráfico das curvas, o mapa da rede, o construtor do ciclo e o "Meu percurso".

Os wireframes põem a Etapa 4 dentro do clone, como "painel de métricas do grupo". O aplicativo real **não tem** visualizador de topologia de rede — inventar essa tela dentro do disfarce quebraria o teste de fidelidade que o próprio QA de design aplicou, e é o oposto da verossimilhança seletiva, que admite ausências e não invenções. É a mesma decisão que o ranking da exp04ef1b já tinha tomado: *a linha é a revelação, não o arquivo* (**ADR-72**).

12.13.7 O clone é escuro, e o tema claro do PRD cedeu

A seção `whatsappmobile` publica exclusivamente o tema escuro. O PRD desta missão crava "tema claro forçado" e descreve balões verdes sobre fundo branco; derivar uma paleta clara aqui é proibido (é a mesma regra do `unitube`), e o contraste do escuro passa o `RNF-ACES-001` com folga (**ADR-68**).

> **LIMITAÇÃO CONHECIDA**
>
> **Nenhum áudio foi gravado.** Os três áudios diegéticos e o episódio de recapitulação existem apenas como transcrição — que está *sempre* na tela, por acessibilidade (`RNF-ACES-002`), não como substituto. Quando a locução chegar, ela entra ao lado do texto, nunca no lugar dele. A missão roda inteira assim, e a suíte de ponta a ponta a percorre sem áudio nenhum.

> **LIMITAÇÃO CONHECIDA**
>
> **Duas pendências de governança pedagógica** seguem abertas no insumo: os nomes formais do autor e do revisor da matriz do gabarito, e a confirmação do batimento da unidade na grade anual da rede parceira. Nenhuma delas impede a codificação; as duas impedem o status "aprovado" do PRD.

12.14 app-70 — Vila do Lume

Jogo 3D em primeira pessoa, **transversal** (`series: []`): não pertence a ano nenhum. É o "Ato 1" que alimenta a largada da exp08ef1b.

12.14.1 Duas leituras de uma partida, e elas não se tocam

REGRA INVIOLÁVEL

O retrato Schwartz (`mapeamento.ts`: 10 valores + 2 eixos) lê *escolhas com custo* dentro do jogo. A missão "A Noite Longa" (`mensagens.ts`) lê *julgamento sobre 5 mensagens*. A separação é **de medida**: um golpe que custasse fagulha faria a criança que caiu aparecer como *menos generosa* no retrato, e nada no relatório mostraria isso.

12.14.2 O servidor recalcula; nunca aceita o perfil do cliente

REGRA INVIOLÁVEL

O jogo sabe computar o retrato e **deliberadamente não o envia**: `CHAVES_DE_PERFIL` em `src/vila/contrato.ts` *recusa payload* que traga o perfil. É isso que permite reprocessar o histórico quando `VERSAO_MAPEAMENTO` subir (hoje 3, herdada do piloto para os dados serem comparáveis).

12.14.3 `grau` é invariante de API, não regra de tela

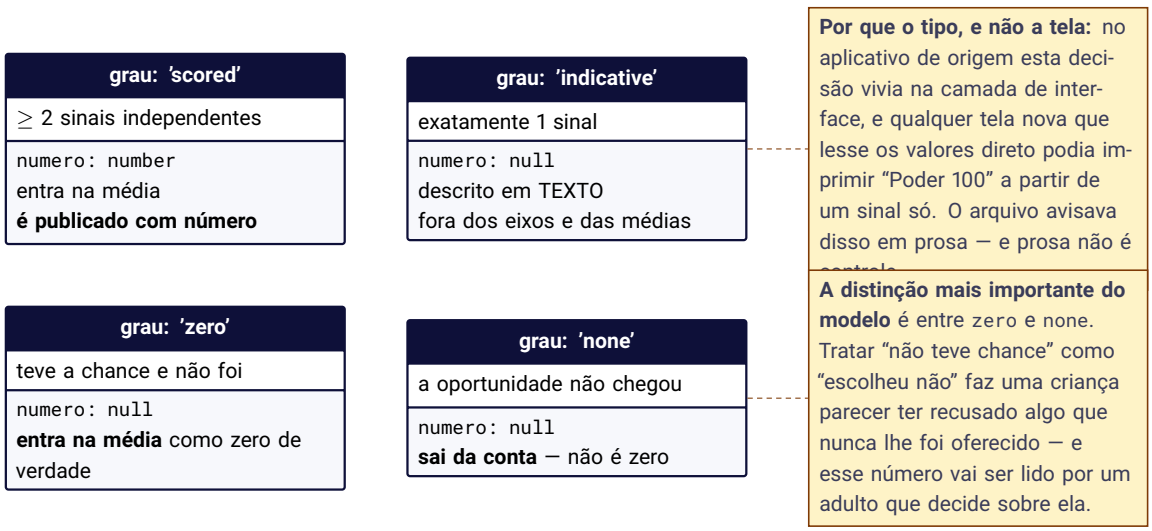


Figura 12.1 – Os quatro graus de medida da app-70. A API devolve uma união discriminada em que **só** a variante `scored` tem `numero: number`; as outras têm `null` *por tipo*. O hub fica estruturalmente incapaz de imprimir número onde não há medida.

12.14.4 A bolha é do servidor

REGRA INVIOLÁVEL

`POST /vila/sessoes` atribui a bolha A/B/C **balanceada dentro da turma**, persiste em `app_70_bolhas` (estável entre partidas) e devolve os cartões — **só os desta criança e sem gabarito**.
Antes era `estudanteSlot % 3` no navegador, o que punha no *bundle* a condição do próprio instrumento e deixava as três versões legíveis antes da aula.
Nos anos iniciais (1EF–5EF) a bolha B **não é servida** ("suporte" exige entender o que é suporte) — regra que só a API pode aplicar, porque só ela sabe a série.
O *lock* consultivo por turma não é zelo: a sala clica junto, e sem ele os 25 caem todos na bolha A.

ATENÇÃO

`cartoesServidos` ≠ "apareceram". O primeiro é quantos a API entregou; o segundo, quantos chegaram a aparecer na partida. Zero aparecidos ⇒ a tela diz "as mensagens não chegaram nesta partida", **nunca** "ela ignorou tudo" — é a mesma distinção entre `zero` e `none`.

12.14.5 Telas do professor

Rota	Conteúdo e regra
`.../vila-do-lume`	Ficha: para que serve, quem consome o perfil, passo a passo.
`.../vila-do-lume/guia`	O debate. Projeta as mensagens das **três** bolhas lado a lado — é a revelação da missão, e a primeira vez que a turma vê as outras versões.
`.../resultados?turmaId=`	Listagem da turma, com o botão de zerar.
`.../estudantes/:id`	A leitura individual, que abre pelo **nível**.

REGRA INVIOLÁVEL

1. **O `/guia` precisa saber a TURMA.** Nos anos iniciais a API não serve a bolha B, e uma tela que projetasse as três mandaria o professor esperar a reação de um grupo que a sala dele não tem. `GET /vila/professor/cartoes?turmaId=` filtra pela *mesma* regra de `abrir` (`bolhasElegiveis`).
2. **Os textos vêm da API, nunca de cópia no hub.** Os textos *são* o instrumento, e uma vírgula divergente entre o que o jogo serviu e o que o professor projeta põe a sala discutindo uma mensagem que ninguém recebeu.
3. **A listagem da turma não mostra valor de Schwartz nenhum.** Listagem é ordenável, e uma coluna "Poder" viraria ranking de crianças. Os valores vivem na tela individual, uma criança por vez, com o grau ao lado.

4. **O agregado abre para gestor; a linha de uma criança, não.** São *controllers* separados, e não um `@Roles` misto com exceção por método.
5. **Zerar a turma apaga as BOLHAS junto.** Rodar de novo com a mesma bolha serviria à criança o mesmo golpe: ela reconhece, e a segunda rodada não mede nada. O que **não** é tocado é `exp08ef1b_app_70_perfis_importados` — aquilo pertence à atividade da exp08ef1b, e apagar de lá trocaria os lados de uma aula em andamento.

12.14.6 A exceção de ADR: o jogo em JavaScript clássico

DECISÃO DE PROJETO

`apps/experiencias/app-70-vila-do-lume/public/jogo/` — cerca de dez mil linhas de JavaScript clássico com variáveis globais compartilhadas e `three.min.js` vendorizado — **fica como está**, e está nos `ignores` do ESLint (**ADR-18**).
É um instrumento com distâncias e tempos calibrados, e o compilador não tem como saber que uma refatoração mudou a *medida*. O *shim* (`src/`, TS *strict*) é a única porta: `window.VILA`. A rede que substitui o `tsc` ali é o *smoke* visual.

12.15 app-90 — Gerador de plano de aula

Ferramenta do professor. Detalhada nas Seções 11 e 4; o que resta registrar é a estrutura do plano e a divisão de responsabilidade de armazenamento.

O plano gerado tem uma estrutura só, definida pelo *prompt* (**ADR-123**): cabeçalho, fundamentação, objetivos gerais, o percurso do ano numa tabela e, em cada bimestre, questão norteadora, conteúdos, habilidades, estratégias, formas de avaliação e produto de síntese; fecham metodologia geral, avaliação geral e referências. O template que acompanha os documentos do professor é apenas um exemplo de formato, de outra disciplina, e `tests/planos-estrutura.spec.ts` impede que *prompt*, exemplo e *mock* divirjam e confere cada habilidade do exemplo contra a BNCC do repositório.

Artefato	Onde vive, e por quê
`gerado.md`	*Bucket* privado, **imutável**. É o original da IA: restaurar é voltar a ele.
`atual.html`	*Bucket* privado. O que o editor salvou, **sanitizado por *allowlist***. Versionamento de objetos dá histórico de cada "Salvar".
Referências curriculares	PostgreSQL (`app_90_referencias`), como **texto**. Catálogo do contrato, sem `escola_id`, fora da RLS. Importadas por `scripts/importar-referencias-plano.mjs` com `PLANOS_IMPORT_TOKEN`.
Metadados e estado	PostgreSQL (`app_90_planos`).

> **ATENÇÃO**
>
> `PLANOS_IMPORT_TOKEN` ausente ⇒ a rota de importação responde **503**, não fica aberta. O mesmo vale para `PERFIL_IMPORT_TOKEN`.

12.16 app-92 — Planejamento de ensino

Ferramenta do professor, e o primeiro produto da plataforma cujo insumo é o **currículo estruturado** em vez de texto livre. A decisão está na **ADR-73**; o que resta registrar é a arquitetura do dado e a fronteira com a app-90.

12.16.1 O caminho completo

Quatro passos, na ordem em que um plano de ensino se constrói numa escola. Nenhum pode ser pulado, e todos podem ser revisitados: em agosto o professor volta à distribuição, remaneja o que não coube e regera o plano do 3º bimestre — um planejador que só anda para a frente não sobrevive ao primeiro mês de aula real.

A **IA entra em exatamente dois pontos**, os dois marcados na Figura 12.2. Todo o resto — a Base como dado, a distribuição, o calendário, a costura e a exportação — é decisão humana ou conta determinística, e é o que faz o documento ser auditável.

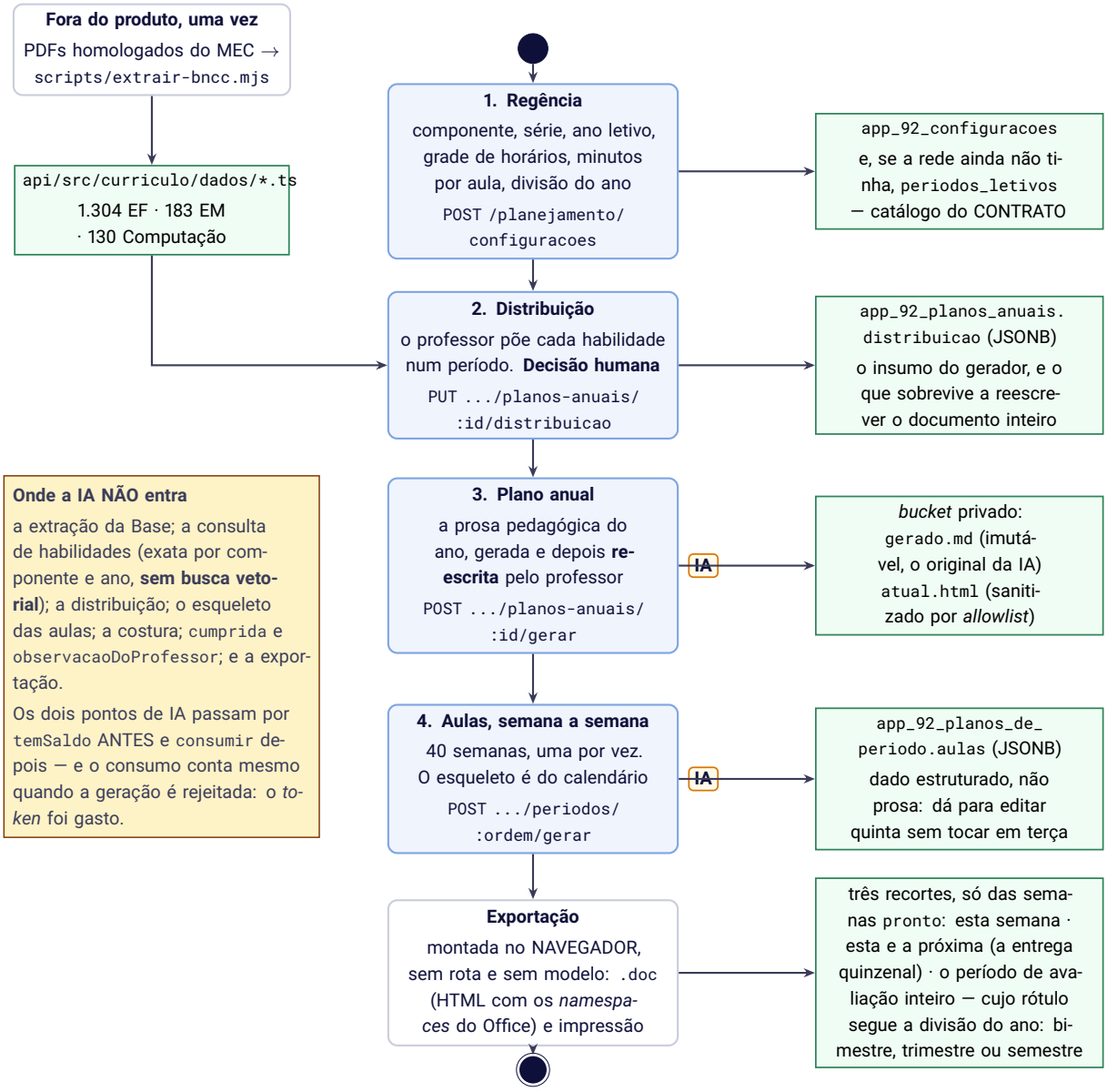


Figura 12.2 – O planejamento de ensino de ponta a ponta, e os dois únicos pontos em que um modelo é chamado. A Base chega como dado extraído fora do produto; a distribuição das habilidades pelos períodos é decisão do professor e é o insumo das duas gerações; a exportação não passa pelo servidor.

12.16.2 Ela coexiste com a app-90

Não são versões da mesma coisa: são ferramentas para redes diferentes.

	app-90 — Plano de aula	app-92 — Plano de ensino
Insumo	Currículo da rede e material didático, importados como **texto**	**BNCC estruturada**, extraída do PDF homologado
Cobertura	As disciplinas com material importado	**Todos** os componentes, `1EF` a `3EM`
Granularidade	Plano anual	Plano anual **e** plano de aulas, **semana a semana**

Serve a	Rede com documento curricular próprio	Rede sem currículo próprio, ou que queira o planejamento aula a aula

O único código compartilhado é a *allowlist* de HTML do editor (`planos/sanitizar.ts`), reusada de propósito: duas *allowlists* divergentes seriam duas superfícies de XSS armazenado.

12.16.3 O currículo como dado

`apps/api/src/curriculo` guarda o extrato da Base, gerado por `scripts/extrair-bncc.mjs` a partir dos PDFs oficiais. Os arquivos são `.ts`, e não `.json`, porque o `nest build` não copia arquivo não-TS para o `dist` sem configurar `assets` — o mesmo motivo pelo qual `prompt-plano-de-aula.ts` guarda um *prompt* como *string*.

Documento	Habilidades	Cobertura da extração
BNCC Ensino Fundamental	**1.304**	99,8 % com unidade temática; 98 % com objeto de conhecimento
BNCC Ensino Médio	**183**	agrupadas por competência específica de área (a Base do Médio não tem objeto de conhecimento)
BNCC Computação	**130**	objeto de conhecimento em TODAS as 104 que o têm (as 26 do Médio usam competência específica); **81 % com a explicação** e 45 % com os exemplos do anexo oficial

> **NOTA**
>
> As colunas “Explicação da habilidade” e “Exemplos” do anexo do Parecer CNE/CEB nº 2/2022 são texto pedagógico oficial que **nenhum *dataset* público da Base empacota**. Elas entram no *prompt* do plano, e é o que faz a Computação aparecer como prática de sala de aula em vez de repetição do texto legal.

12.16.4 Componente da rede ≠ componente da Base

É a distinção que faz o produto funcionar no Ensino Médio, e a que mais quebra implementação ingênua.

Componente da matriz	Onde a Base guarda	Código
Língua Portuguesa	o próprio componente	`EM13LP__`
Matemática	o próprio componente	`EM13MAT___`

Biologia, Física, Química	**a área**	`EM13CNT___`
Filosofia, Geografia, História, Sociologia	**a área**	`EM13CHS___`
Arte, Educação Física, Língua Inglesa	**a área**	`EM13LGG___`

No Fundamental os dois coincidem. No Médio não: a resposta da API traz `habilidadesPorArea: true`, e a tela **avisa** o professor — sem isso, quem leciona Biologia no 2º ano recebe "nenhuma habilidade encontrada".

12.16.5 Divisão de responsabilidade

Artefato	Onde vive, e por quê
A **distribuição** de habilidades por período	PostgreSQL (`app_92_planos_anuais.distribuicao`, JSONB). É a decisão HUMANA do professor e o insumo do gerador: ele pode reescrever o documento inteiro sem que o sistema perca qual habilidade cai em que bimestre.
`gerado.md`	*Bucket* privado, **imutável** — o original da IA.
`atual.html`	*Bucket* privado, sanitizado por *allowlist*.
As **aulas** do plano de período	PostgreSQL (`app_92_planos_de_periodo.aulas`, JSONB) como **dado estruturado**, não prosa. É o que permite editar a aula de quinta sem tocar na de terça, e o que faz o documento impresso nascer do dado.
Metadados e estado	PostgreSQL (`app_92_configuracoes` e as duas acima).

12.16.6 O esqueleto das aulas não vem do modelo

Número, data, dia da semana e horário saem de `planejamento/calendario.ts`, calculados da grade real do professor; o modelo preenche apenas o conteúdo pedagógico das aulas que já existem, e `planejamento/costura.ts` descarta sobra e completa falta. A Figura 12.3 mostra a conciliação inteira, com as três regras da costura e as três barreiras contra um código de habilidade inventado.

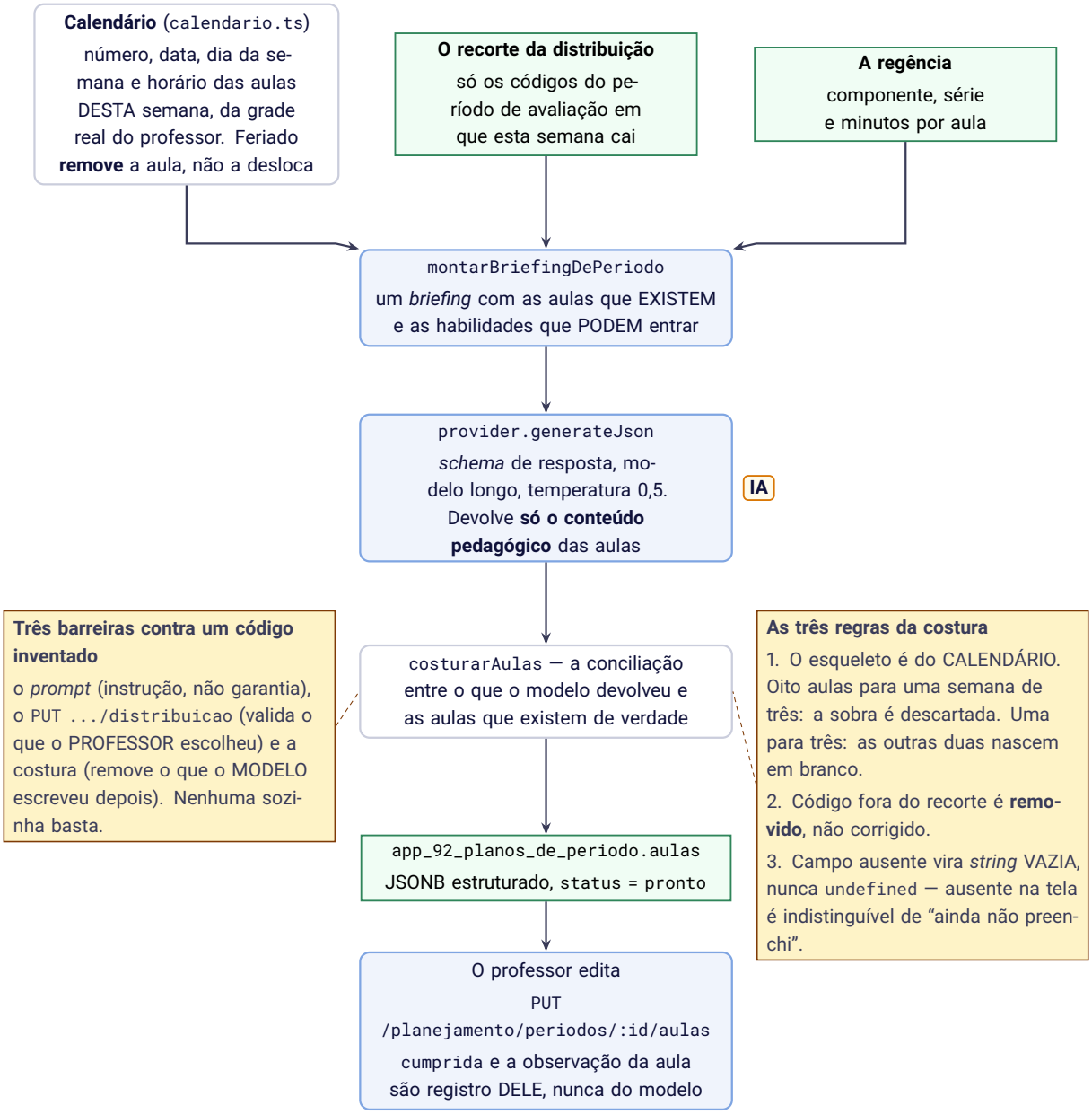


Figura 12.3 – Como uma semana de aulas nasce. O modelo recebe as aulas que existem e as habilidades que podem entrar, e devolve apenas conteúdo pedagógico; a costura concilia. Uma resposta malformada degrada para "planejar à mão", nunca para aulas em dias que não existem.

ATENÇÃO

Deixar o modelo decidir quantas aulas cabem na semana produzia **cinco** — o número redondo — para um professor de Arte que tem **uma**. Pelo mesmo motivo, `cumprida` e `observacaoDoProfessor` nunca vêm da IA: são o registro do que aconteceu na aula, e um plano que nasce "dado" deixa de medir qualquer coisa.

12.16.7 Carga horária: hora-aula não é hora-relógio

As **800 h** do Fundamental e as **1.000 h** do Médio (LDB art. 24, I, redação da Lei 14.945/2024 — não são mais 800 h) são horas de **60 minutos**. O Parecer CNE/CEB nº 8/2004 é literal: "não se pode considerar uma aula de 45 minutos igual a uma hora". Toda conversão passa por `curriculo/carga-horaria.ts`.

Aulas/semana	Horas-relógio/ano	
25 (de 50 min)	**833 h**	logo acima do mínimo do Fundamental
30 (de 50 min)	**1.000 h**	**exatamente** o mínimo do Médio — é a âncora que explica por que quase toda rede pública converge para 30 aulas semanais no Médio
30 (de 45 min)	900 h	**não** cumpre o mínimo do Médio

A matriz de referência do produto é a **mediana de dez redes** (SEDUC-SP, SME-SP, SEE-MG, SEDUC-RS, SEEDUC-RJ, SEED-PR, SEDUC-GO, SEE-PE, SEDUC-CE, SME-Rio), e a tela diz que é sugestão: **a BNCC não define grade horária** — quem define é a rede (LDB art. 23 e 26; Res. CNE/CP nº 2/2017, art. 8º, II).

> **DECISÃO DE PROJETO**
>
> Não existe "a" matriz média: a jornada do Fundamental é bimodal O total semanal do Ensino Fundamental **não se distribui em torno de um centro**. Ele tem duas famílias, e nada no meio:

Total	Horas/ano	Redes
25 aulas	**833 h** (≈ o mínimo de 800 h)	SEE-MG, SEDUC-RS, SME-Rio, SEED-PR, SEDUC-GO, SEE-PE
28–30 aulas	**890–1.000 h**	SEDUC-SP, SME-SP, SEEDUC-RJ, SEEDF

Uma “média” de 27,5 aulas produziria uma matriz que **nenhuma rede real usa**. A escolha entre 833 h e 1.000 h não é variação estatística — é decisão de política de jornada, e as duas maiores redes do país estão na família de 1.000 h.
Por isso o produto pré-preenche o **subtotal da base comum** (26 aulas, nos anos iniciais e nos finais) e não um total de semana: quem fica em 25 trima exatamente uma aula do bloco — na prática Arte (MG, GO) ou o Ensino Religioso (SME-Rio, PE) —, e quem vai a 30 acrescenta parte diversificada. É o professor que sabe em qual das duas famílias a rede dele está.
No **Ensino Médio** a convergência é real: nove das dez redes fazem 24 de Formação Geral Básica + 6 de itinerário = 30 aulas = 1.000 h. A exceção (São Paulo) só redistribui entre as séries, mantendo 2.400 h + 600 h no triênio.

NOTA

Nos **anos iniciais** a matriz do produto soma 24, não 26: Língua Inglesa fica fora porque a LDB art. 26 §5º só a exige “a partir do sexto ano” e a BNCC não tem habilidade dela antes disso. A mediana observada é de 2 aulas (quatro das seis redes que informam), mas são aulas **sem currículo comum a que se ancorar** — e componente sem habilidade da Base não é planejável num produto dirigido pela Base.

12.16.8 O início do ano letivo é da regência

ATENÇÃO

`periodos_letivos` é catálogo do CONTRATO, e o *seed* o preenche como ano **civil** (01/01–31/12). Como o calendário alinha cada período na segunda-feira, o professor que declarava 02/02/2026 recebia um 1º bimestre começando em **29/12/2025**. A coluna `app_92_configuracoes.inicio_do_ano_letivo` é o que vale; o catálogo é apenas sugestão do primeiro preenchimento, e um “ano letivo” que abre em 1º de janeiro é descartado como ano civil.

Datas de calendário escolar trafegam como **data civil** (`YYYY-MM-DD`) e são convertidas ao **meio-dia UTC**: ao meio-dia nenhum fuso do Brasil (UTC−2 a UTC−5) atravessa a meia-noite, então o dia da semana não escorrega — e uma aula de segunda que vira domingo é bug que passa no *typecheck* e sobrevive até a impressão.

Feriado **remove** a aula, não a desloca: deslocar empurraria o plano inteiro e desalinharia as 40 semanas seguintes. Repor é decisão pedagógica do professor, não conta do sistema.

12.16.9 O passo é semanal; o recorte é da entrega

O planejamento granular tem **uma** unidade: a **semana** letiva. São 40 no ano, e o professor detalha uma por vez — planeja a 1ª, depois a 2ª, e assim por diante. Não existe "plano quinzenal" como objeto do produto.

Houve uma `periodicidade` configurável (semanal ou quinzenal) e ela foi retirada na ADR-76. O que a quinzena resolvia era a **entrega** — a escola que recolhe planejamento de quinze em quinze dias —, e isso é recorte de **exportação**: o mesmo plano semanal sai em três formatos, sem nada mudar no que foi planejado.

Botão	O que sai no documento
`Baixar esta semana`	a semana aberta, com as aulas datadas
`Esta e a próxima (quinzena)`	duas semanas, uma por página — a entrega quinzenal. Só aparece com a semana seguinte **já planejada**: sem ela o documento sairia com uma semana só, e o nome mentiria
`Baixar o <período> inteiro`	todas as semanas `pronto` daquele período de avaliação. **O rótulo não é fixo**: ele é o do período, e segue a divisão do ano escolhida na regência — "3º Bimestre", "2º Trimestre" ou "1º Semestre", conforme `DIVISOES_DO_ANO`. Só aparece com **mais de uma** semana planejada no período

Os dois documentos de conjunto saem do mesmo `documentoDeSemanas`: uma semana por página, e o cabeçalho conta as semanas e as aulas.

As semanas vêm de `GET /planejamento/planos-anuais/:id/periodos`, que devolve **só as `pronto`**: um documento entregue à coordenação não leva semana que falhou na geração nem semana que ninguém planejou.

> **ATENÇÃO**
>
> Como passo de planejamento, a quinzena custava caro: `app_92_planos_de_periodo` é endereçada por `(escola, plano anual, ordem)`, e a **ordem muda de significado** com a periodicidade — a 3ª quinzena não é a 3ª semana. Um plano gravado em modo quinzenal e lido em modo semanal casa pela ordem e devolve as datas de um recorte com as aulas de outro.

12.17 app-91 — Acompanhamento de bem-estar

Ferramenta do professor, detalhada na Seção 11. O que resta registrar é a mecânica de micro-doses e a decisão de escopo de leitura.

Mecânica	Implementação

21 itens em 7 *check-ins*	`ordem_itens` é a permutação de 1..21 sorteada na abertura do ciclo; cada *check-in* leva 3 itens, um de cada subescala.
Um *check-in* por dia	`UNIQUE (escola_id, professor_id, dia)`, com `dia` no fuso `America/Sao_Paulo`.
Ciclo expira em 21 dias	`expira_em`; ciclo não fechado vira `expirado`, sem pontuação.
Resumo leve para o *card* do hub	`GET /bemestar/resumo`, sem IA. É o que a home do professor lê — e a home não pode depender de uma chamada de modelo.
Apagar tudo	`DELETE /bemestar/dados`.

DECISÃO DE PROJETO

Nenhuma rota de gestor nem agregado (ADR-10). Dado de saúde (LGPD art. 11) com RLS e posse por `professor_id` em toda *query*; só o professor lê. Uma visão agregada e anonimizada para a rede é decisão *futura*, com DPA e *k*-anonimato — e não sai de um `GROUP BY`.

12.18 Como criar uma experiência nova

Listing 12.2 – O caminho completo

```
# 0. Uma frente por experiencia, com o CODIGO dela (ADR-110):
#    exp + ano + etapa + unidade + b -- exp06ef3b = 6o ano, unidade 3.
#    O de missao sai da serie e da unidade; o que nao e missao e app-NN-nome.
pnpm worktree criar exp06ef3b --fatia exp/exp06ef3b

# 1. Scaffolding do _template (copia, ajusta nomes, preenche series/unidade)
pnpm new:exp exp06ef3b

# 2. Implementar em src/App.tsx+
#    - auth, api e telemetria: SOMENTE pela SDK
#    - interface: SOMENTE o @unooria/design-system, pela ponte packages/ui,
#      na secao do clone que a experiencia imita
#    Falta componente? Ele nasce no repo storybook-unooria e chega por versao
#    nova — nunca como arquivo local.

# 3. Declarar capabilities e verbos no public/experiencia.json

# 4. Documentar verbo/objeto de telemetria no README da experiencia

# 5. Build gera dist/ 100% estatico. Testar embedado:
pnpm --filter @edu/exp06ef3b dev          # porta nova em lib/experiencias-dev.ts
# e abrir a bancada do hub:
#   http://localhost:3000/dev-embed?src=http://localhost:5180
```

Checklist	Verificação

Módulo de domínio na API	Necessário sempre que houver gabarito, estado de missão ou visão de professor. Prefixo minúsculo igual ao slug (`src/<slug>/`), com `catalogo.ts` + `regras.ts` + *service* com `comTenancy` + *controllers* por papel.
Tabelas	Prefixo = o código (`exp08ef1b_`, `app_70_`), `escola_id NOT NULL` + RLS.
Porta de *dev*	Registrada em `apps/hub/lib/experiencias-dev.ts`.
`allowedHosts` do Vite	Deve incluir `host.docker.internal`, senão o hub em contêiner não lê o manifesto.
`build.sourcemap` do Vite	`"hidden"` em todas (o `_template` inclusive): o `.map` é gerado sem o comentário `sourceMappingURL`; o CI o exclui do *bucket* público e o sobe num *bucket* PRIVADO (`{project}-exp-mapas`). O *bucket* dos *bundles* tem acesso uniforme, então qualquer objeto lá é público, e o mapa é o fonte — na exp08ef1b, a revelação legível antes da hora. Nunca `true`, nunca `inline`.
`viewport`	Só se a experiência exigir casca de aparelho.
`permissoes`	Só o que o `iframe` realmente precisa.
Mídia $>$ 2 MB	Vai para o *bucket* de *assets*, referenciada por variável de *build*. Nunca no *git*, nunca no *bundle*.
Ponta a ponta	Uma suíte em `scripts/e2e/<papel>/` se a experiência tem tela nova no hub.

> **NOTA**
>
> O `_template` continua **cru de interface, de propósito**: o que ele não traz, o *design system* traz. Um *template* rico em componentes locais seria a primeira violação da regra de interface, replicada 48 vezes. A **fiação**, porém, ele traz inteira desde a **ADR-106**: manifesto completo (`produto`, `unidade`, `publico`, `permissoes`), *handshake* em `bootstrap.tsx`, `lib/{api,telemetria,tipos,textos}.ts`, folhas da seção `unooria`, `versao.test.ts` e `vitest` — a experiência nova nasce compilando, testando e conversando com o hub.

13

Especificação da API

A API é um serviço NestJS (NestJS, 2026) com 34 módulos (os `imports` de `app.module.ts` em 2026-09-25). Este capítulo especifica a superfície HTTP: as convenções transversais, os grupos de rotas e a autorização de cada um. O catálogo completo, rota por rota, está no Apêndice C.

13.1 Convenções transversais

Aspecto	Convenção
Autenticação	`Authorization: Bearer <JWT>`, validado por `OidcGuard` contra o JWKS de `OIDC_ISSUER`. Três rotas fogem disso, e cada uma tem substituto explícito (abaixo).
Autorização	`@Roles(...)` + `RolesGuard`, *sempre* depois do `OidcGuard` na lista de *guards*. Sem `@Roles`, a rota aceita qualquer papel autenticado.
Tenancy	`TenancyInterceptor` global abre o contexto com a escola do *token*; toda *query* passa por `comTenancy` e filtra `escola_id` explicitamente.
Validação de entrada	Zod em todo corpo e *query*. Não há `class-validator` nem *pipes* de DTO: o esquema Zod é a fonte única do tipo e da validação.
Corpo	JSON com teto de 2 MB; `text/csv` cru com 30 MB, para a importação de pessoas — o teto do *bodyParser* e o da rota saem da MESMA constante (`TETO_DO_ARQUIVO`, em `rostering.service.ts`). O número cabe debaixo dos 32 MiB que o Cloud Run recusa por requisição HTTP/1, para o 413 do ambiente nunca ser o que o gestor vê (**ADR-48**).
CORS	Lista explícita em `CORS_ORIGINS` — hub mais os *dev servers* das experiências. Nunca `*`.
Erros	*Exceptions* do Nest (`BadRequestException`, `ForbiddenException`, `NotFoundException`). Domínio com vocabulário próprio de erro usa um `CodigoErro` tipado (ex.: `src/unoogpt/erros.ts`).
Assincronia longa	**202 + *polling***. Nenhuma rota fica aberta esperando modelo de linguagem.

13.1.1 As três rotas sem `OidcGuard`

Rota	Substituto	Por quê
`GET /healthz`, `GET /readyz`	Nenhum — públicas	Sonda de saúde. Exigir *token* tornaria a sonda dependente do IdP.
`POST /telemetry/beacon`	Verificação manual do *token*, pelo `TokenService`	`sendBeacon` não envia cabeçalho, então o *token* vem no corpo. O rigor da verificação é o mesmo; só o transporte muda.
`POST /telemetry/ingest`	`PubSubPushGuard`	Quem chama é o Pub/Sub, não um usuário. Verifica emissor Google, `aud` e `email` da conta de serviço. **Falha fechada** sem configuração.

ATENÇÃO

Há ainda duas rotas de *serviço* protegidas por chave em cabeçalho, e não por JWT: `POST /perfis/importar` (`x-perfil-token`) e `POST /planos/referencias` (`x-import-token`). Ambas são chamadas por *script* local ou por integração externa, e ambas respondem **503** quando a variável correspondente está ausente — desligadas, não abertas.

13.2 Mapa de rotas por área

O mapa vem em duas figuras: a plataforma e a exp08ef1b na Figura 13.1, e as demais experiências, uma linha por experiência, na Figura 13.2. O catálogo rota por rota, com método e esquema, é o Apêndice C.

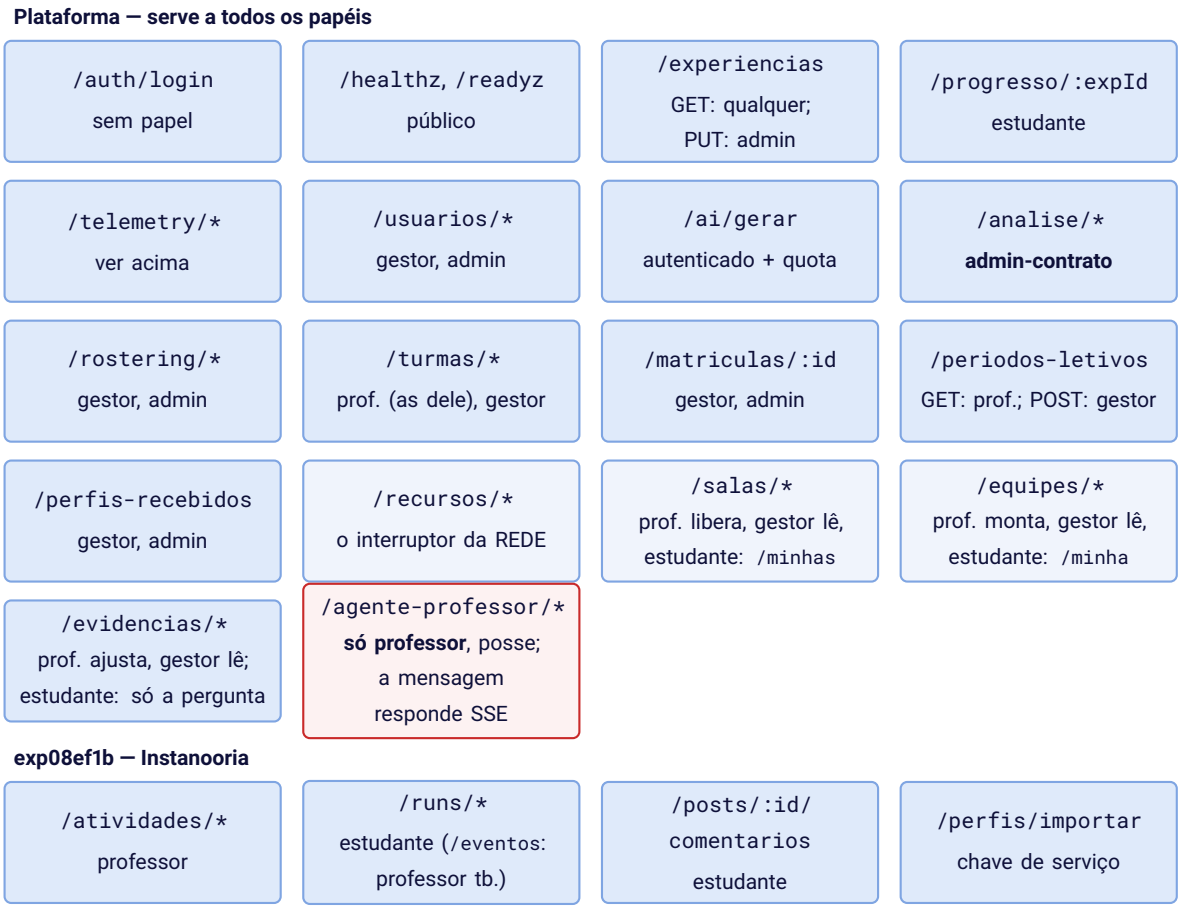


Figura 13.1 – Mapa das rotas da plataforma e da exp08ef1b, por papel autorizado; as das demais experiências estão na Figura 13.2. Em `/turmas/*` a autorização do professor é por **matrícula**, não por escola: é a diferença entre "dá aula nesta turma" e "trabalha nesta escola" (ADR-28). Em destaque, os três eixos que nenhuma experiência conhece pelo nome: `/recursos/*` diz se ela existe NESTA REDE (ADR-87), `/salas/*` se está aberta NESTA TURMA (ADR-69) e `/equipes/*` quem divide um computador nela (ADR-111). Nas duas últimas quem escreve é só quem leciona; o gestor só lê — como em `/evidencias/*` (ADR-112), em que o estudante alcança só a pergunta e a resposta, nunca o próprio resultado. `/agente-professor/*` é só do professor, e cada um alcança só as próprias conversas na escola ativa (ADR-126).

	o estudante	o professor e a gestão	o guia, o agregado e o serviço
backup-exp04ef1b UnooTube (arquivada)	`---` sem rotas: fora do `app.module` (ADR-116)	`---` era `/unootube/turmas/*`	`---` era `/unootube/professor/*`
app-70 Vila do Lume	`/vila/sessoes` estudante	`/vila/professor/*` **professor**	`/vila/agregado/*` prof., gestor
app-90 Plano de aula		`/planos/*` professor	`/planos/referencias` chave de serviço
app-91 Bem-estar		`/bemestar/*` **só professor**, posse	
exp02em1b unooGPT	`/unoogpt/*` estudante	`/unoogpt/professor/*` **só professor**, transcrição	`/unoogpt/agregado/*` gestor, admin — sem `estudanteId`
exp06ef2b O Rastro do Boato	`/rastro-boato/*` estudante	`/rastro-boato/turmas/*` prof., gestor, admin — o *gate* da aula	`/rastro-boato/professor/*` prof., gestor — material do guia
exp08ef4b O Manifesto da IA	`/manifesto-ia/*` estudante	`/manifesto-ia/turmas/*` prof., gestor — o debate E o conteúdo da turma	
exp01ef1b O Que a Foto Conta	`/foto-conta/*` estudante	`/foto-conta/turmas/*` prof., gestor — o *latch* da Roda	
exp04ef1b Jornal da Escola	`/jornal-escola/*` estudante — e as perguntas em `/evidencias`	`/jornal-escola/turmas/*` prof., gestor — o pacote, os dois portões e o painel	`/jornal-escola/professor/*` prof., gestor — material do guia
exp05ef1b Quem mandou isso?	`/quem-mandou-isso/*` estudante — e as perguntas em `/evidencias`	`/quem-mandou-isso/turmas/*` prof., gestor — o pacote E o relógio da turma	`/quem-mandou-isso/professor/*` prof., gestor — material do guia

Figura 13.2 – Mapa das rotas das demais experiências: uma linha por experiência, uma coluna por quem chama. Duas áreas estão destacadas por **ausência deliberada**: bem-estar não tem nenhuma rota de gestor nem de agregado; e no unooGPT a transcrição do estudante só existe no controller do professor — o do gestor **não tem uma rota que aceite `estudanteId`**, o que torna estrutural, e não disciplinar, o limite do que ele alcança (ADR-27). Em `/rastro-boato/turmas/*` mora o *gate* da aula: é a professora quem move a turma de etapa, e a rota é **idempotente** de propósito — é o clique que se dá duas vezes. As `/<slug>/professor/*` servem o **material do guia da aula**: catálogo com gabarito, sem `escolaId` e sem *tenancy*, em que o que a guarda protege é só o papel (ADR-86). A exp08ef4b **não** tem uma dessas, e a ausência é o desenho: o material de mediação dela é o pacote CURADO por turma, então ele mora em `/manifesto-ia/turmas/:id/conteudo` — com *tenancy*, porque o conteúdo não é igual para todo contrato. A exp05ef1b tem **as duas coisas**, e elas respondem perguntas diferentes: `/quem-mandou-isso/professor/material` é o banco curado da Unooria, igual em todo contrato, e `/quem-mandou-isso/turmas/:id/pacote` é o pacote que AQUELA professora aprovou — o único que a turma dela vai ler. O Jornal da Escola repete o desenho (ADR-117).

13.3 Superfície que a SDK consome

Uma experiência nunca chama a API “à mão”: chama pela SDK, e a SDK oferece exatamente quatro capacidades.

Listing 13.1 – `ExperienciaHandle` — tudo que a experiência pode fazer

```ts
export interface ExperienciaHandle {
  estudante: Estudante;
  config: Record<string, unknown>;

  /** Cliente HTTP autenticado. Sem retry fora do GET — repetir um POST
   *  de dominio pode duplicar efeito. */
  api: {
    get<T>(path: string): Promise<T>;
    post<T>(path: string, body?: unknown): Promise<T>;
    put<T>(path: string, body?: unknown): Promise<T>;
    patch<T>(path: string, body?: unknown): Promise<T>;
    delete<T>(path: string): Promise<T>;
  };

  /** Unica porta para IA. A experiencia NAO conhece modelo nem prompt. */
  ai: { gerar(input: { atividade: string; mensagem: string }): Promise<{ texto: string }> };

  /** Telemetria em lote. A experiencia informa SO verbo, objeto e dados. */
  track(verbo: TelemetryEvent["verbo"], objeto: string,
        dados?: Omit<TelemetryEvent, "verbo" | "objeto">): void;

  progresso: {
    salvar(dados: Record<string, unknown>): Promise<void>;
    carregar<T>(): Promise<T | null>;
  };

  concluir(payload?: { score?: number; dados?: Record<string, unknown> }): void;
  on(evento: "pause" | "resume", cb: () => void): void;

  /** Estado de alto nivel para o chrome opcional do hub (exp:estado). */
  enviarEstado(estado: Record<string, unknown>): void;
}
```

> **NOTA**
>
> Não há método de “corrigir” nem de “pontuar” na SDK, e isso é intencional: correção é rota de domínio da experiência na API (`/unoogpt/*`, `/vila/*`, `/runs/*`), não capacidade genérica. Uma API genérica de avaliação convidaria a mandar a resposta certa junto.

13.4 Grupos de rotas em detalhe

13.4.1 Motor de *feed* (exp08ef1b)

O grupo com mais rotas da plataforma, todas escopadas por `run` e todas de papel `estudante`, com duas exceções marcadas.

Rota	Papel	Semântica
`POST /runs`	estudante	Cria ou recupera a sessão; resolve a largada (perfil, sorteio ou redistribuição).
`GET /runs/boot`	estudante	Estado inicial que a experiência precisa para montar a primeira tela.
`POST /runs/:id/entrar`	estudante	Nome no aplicativo e avatar escolhidos.
`GET /runs/:id/feed`	estudante	**Próximo bloco curado.** Abre transação, toma o *lock* da `run`, aplica eventos pendentes, cura, persiste e sanitiza.
`POST /runs/:id/eventos`	**estudante, professor**	Canal operacional de telemetria (*dwell*, curtidas, salvos). Síncrono e transacional.
`GET /runs/:id/fase`	estudante	Fase atual, para travar a tela conforme o relógio coletivo da turma.
`POST /runs/:id/conclusao`	estudante	Inclinação escolhida (A, B ou dúvida).
`POST /runs/:id/reflexao`, `GET .../reflexao`	estudante	Texto livre de fechamento, sem limite de caracteres.
`POST /runs/:id/student-post`	estudante	Post publicado pelo estudante.
`POST /runs/:id/reacoes`	estudante	Dispara reações de persona ao vivo (se `ia_ao_vivo`).
`GET /runs/:id/dossie`	**estudante, professor**	O dossiê de fechamento — a única rota do grupo que o professor também lê.
`GET /runs/:id/perdidos`	estudante	Os posts que o estudante *não* viu. É a virada pedagógica da missão.
`POST /runs/:id/reset`	**professor**	Zera a sessão. Recalcula a largada pela *mesma* regra do primeiro início.

13.4.2 A sala de aula: a missão dentro da turma

Três rotas, e elas são a camada entre `turmas` e `experiencias` (**ADR-69**):

- `GET /salas/minhas` (estudante) — as turmas dele e, em cada uma, a grade do ano. Devolve a grade INTEIRA, não só o que está aberto: missão bloqueada aparece com **cadeado**, encerrada aparece com o rótulo.
- `GET /salas/turma/:turmaId` (professor, gestor, admin) — a mesma grade com quem liberou e quando. `?todas=1` acrescenta as missões de outras séries, marcadas: turma multisseriada e reforço existem, mas trazê-las por padrão afogaria a grade do ano.

- `PUT /salas/turma/:turmaId/missoes/:expId` (**só professor com matrícula na turma**) — libera ou encerra. `PUT` porque o efeito é o estado final: é o clique que um professor nervoso dá duas vezes, na frente da sala.

> **REGRA INVIOLÁVEL**
>
> **Esconder o *card* no hub é UX; a porta é do servidor.** E desde a **ADR-87** essa porta tem DUAS trancas em série: antes de perguntar se o professor liberou, ela pergunta se o recurso EXISTE nesta rede. Toda entrada de estudante em experiência — `POST /runs`, `GET /runs/boot` e o funil de sessão do UnooTube, do unooGPT, da Vila e das missões seguintes — chama `exigirMissaoLiberada` e responde **403** `missao-nao-liberada` sem liberação. O *bundle* é público e `/exp/:expId` é endereço adivinhável: antes disto, qualquer estudante logado abria qualquer missão digitando a URL.
> A guarda é PRÉ-CONDIÇÃO do relógio da exp02em1b e dos *gates* da exp06ef2b, não substituta — e fica na CRIAÇÃO da sessão, não em toda rota: quem já entrou fica, porque encerrar fecha a entrada e não expulsa a criança no meio da aula.

13.4.3 As equipes na missão: quem divide um computador

Seis rotas no módulo `equipes/` (**ADR-111**): cinco de professor, todas sob `/equipes/turma/:turmaId/missao/:expId`, e uma de estudante, a literal `/equipes/minha`. Os prefixos são distintos de propósito: assim a ordem de registro dos *controllers* não decide quem responde, e o do estudante é outro arquivo, sem `@Roles` misto.

- `GET /equipes/turma/:turmaId/missao/:expId` (professor que leciona, gestor, admin) — a montagem, as equipes com os membros e `semEquipe` (os ausentes inclusive). Missão que não aceita equipe responde **200** com `permitida: false`, e não 400: é por esta resposta que o hub decide se desenha o bloco.
- `POST .../sugestao` (só professor) — uma proposta que **nunca grava**, e por isso responde 200: `sorteio` distribui os presentes por tamanhos que diferem em no máximo 1; `manual` devolve N equipes vazias e todos em `semEquipe`. O `agrupamento` (lados, neutros, equipes mistas) só vem quando a missão registrou uma fonte de lado; hoje, nenhuma registrou.
- `GET .../ultima` (só professor) — a montagem mais recente da MESMA turma em OUTRA missão com equipe, reprojetada sobre a turma de hoje: quem saiu cai e é contado em `saidas`, quem chegou fica em `semEquipe`, e os ausentes de lá não vêm. Também não grava.
- `PUT .../` (**só professor com matrícula na turma**) — grava a montagem INTEIRA. O corpo leva a `versao` em que a tela se baseou (`0` sem montagem); versão velha é **409** `equipes-desatualizadas` com `versaoAtual`. Montagem impossível é **400** `montagem-invalida` com `erros`: a lista INTEIRA, em frases para o professor e sem nome de criança. Missão fora da lista é 400 `missao-sem-equipes`; fora do catálogo, 404; ferramenta de professor ou recurso desligado na rede, 403.

- `DELETE .../` (só professor) — desfaz. **Idempotente**, e sem checagem de catálogo nem de disponibilidade: missão desligada na rede não pode deixar equipes presas que ninguém consegue apagar.
- `GET /equipes/minha?expId=` (estudante) — **sempre 200**: `equipe: null`, ou o nome dela e `colegas` com SÓ o primeiro nome dos outros membros. Um 404 viraria `api-recusou` no log do hub a cada abertura de missão.

Turma que o professor não leciona responde **404** em todas as rotas de professor, nunca 403 — é o desenho da sala. E três respostas de outros módulos mudaram de forma para contar por COMPUTADOR:

Rota	O que mudou
`GET /salas/turma/:turmaId`	`missoes[].equipes`: quantas equipes a missão tem na turma, num `groupBy` para a grade inteira. SÓ no *schema* do professor, e não no `MissaoNaGradeSchema`, que também responde `/salas/minhas`: a contagem da sala não é assunto do estudante.
`GET /unoogpt/professor/turmas/:turmaId`	O mesmo na folha da Missão Tempestade: `linhas[].equipe`, e o `retrato` conta as linhas, com `porComputador`. "Abrir" leva ao representante.

13.4.4 A evidência pedagógica: por atividade e rodada

Seis rotas no módulo `evidencias/` (**ADR-112**), em dois *controllers*: quatro do lado de quem dá aula — o painel, a confirmação e o ajuste de uma atividade, e o perfil de um estudante — e duas do estudante, a pergunta e a resposta. Nenhum caminho de um casa com o do outro — o número de segmentos e as literais diferem —, então a ordem de registro não decide quem responde; e o do estudante é outro arquivo, sem `@Roles` misto. As duas escritas do professor devolvem o PAINEL já atualizado, para a tela não precisar reler.

- `GET /evidencias/turma/:turmaId/missao/:expId/atividade/:atividadeId` (professor que leciona, gestores, `admin`) — o painel: as habilidades da ficha com o texto da BNCC, as rodadas que têm evidência, e para cada estudante da turma a equipe dele na missão, `porRodada` (cada habilidade com `status`, `origem` e `situacao`) e o `resultado` — a melhor rodada por habilidade, só com o que vale. Habilidade ausente é N/A. `pendentes` conta, por rodada, as cópias da projeção a revisar. Missão SEM ficha responde **200** com `temFicha: false` e tudo vazio, e não 400: é por esta resposta que o hub decide se desenha o bloco. A ordem é a alfabética, e nenhum campo soma níveis.
- `POST .../rodadas/:rodada/confirmar` (só professor) — "Confirmar para a turma": as cópias `a_revisar` da rodada passam a `valida`, com UM evento de turma no histórico. Responde 200, e não 201 (`@HttpCode(200)`), porque é ação sobre o que já existe; e repetir não muda nada.

- `PATCH .../rodadas/:rodada/ajustes` (**só professor com matrícula na turma**) — de 1 a 200 ajustes `{ estudanteId, habilidade, status }`, a qualquer momento, mesmo com a missão encerrada. O ajuste é a origem `professor`, que nada sobrescreve depois; `nao_participou` é o N/A explícito. Estudante fora da turma, habilidade fora da ficha ou o mesmo par duas vezes: **400** `ajuste-invalido` com `erros`, a lista INTEIRA em frases. Rodada depois da última que a atividade teve: **400** `rodada-inexistente` (a rodada 1 sempre existe, para a atividade feita fora da missão). Ajuste que repete o que já estava é ignorado, sem evento.
- `GET /evidencias/estudante/:estudanteId` (professor, gestores, `admin`) — o perfil: em cada missão (com o nome do catálogo, ou o `expId` quando ela não está nele), em cada atividade, o nível de cada habilidade — a melhor rodada, só com o que vale: sem a projeção a revisar, sem N/A e sem `nao_participou`. Sem origem, e sem juntar missões. O professor só abre quem ele ensina (`professorEnsinaOEstudante`).
- `GET /evidencias/:expId/perguntas/:perguntaId` (estudante) — o enunciado e as alternativas, só `id` e `texto`: o nível de cada uma é o gabarito, e fica no servidor. As alternativas saem **embaralhadas a cada leitura** (ADR-118): a ficha as guarda na ordem da régua, e "a primeira é a certa" viraria a pista — a tela responde pelo `id` e carrega a pergunta uma vez. Pergunta fora da ficha: **404** `pergunta-nao-encontrada`.
- `POST /evidencias/:expId/perguntas/:perguntaId/resposta` (estudante) — o corpo leva SÓ `alternativaId`; a atividade e a rodada saem do resolvedor da missão. Responde **sempre** `{ enviada: true }`, na primeira e na segunda vez — vale a primeira. Sem atividade em andamento (ou sem resolvedor registrado): **409** `sem-atividade-em-andamento`; alternativa que não é da pergunta: **400** `alternativa-desconhecida`.

As duas do estudante passam por `exigirMissaoLiberada`: missão não liberada na turma dele é **403** `missao-nao-liberada`, e recurso desligado na rede, `recurso-indisponivel`. Turma que o professor não leciona, e estudante que ele não ensina, respondem **404**, nunca 403 — o desenho da sala. O gestor, que tem a escola do *token* vazia quando é `gestor_contrato`, diz a escola em `?escolaId=`. E o estudante **não tem rota que leia o próprio resultado**: a ausência é o contrato.

13.4.5 Entrada de pessoas: as duas portas, e por que só uma escreve

Criar pessoa tem duas superfícies e **um** caminho de escrita. Em lote é `POST /rostering/importar` (ou o *binding* OneRoster); uma por vez é `POST /usuarios`, o formulário do balcão — que não escreve em `users`: serializa os campos como uma linha da MESMA planilha e chama o MESMO importador (ADR-59). É o que mantém CPF com dígito verificador, série, turno, login gerado, turma, lotação e matrícula com uma definição só.

Rota	Papel	Semântica

`POST /rostering/importar`	gestor, admin	O lote. Idempotente pela matrícula da rede e pelo CPF: reimportar atualiza, não duplica, e não reseta a senha de quem já entrou.
`POST /usuarios`	gestor, admin	Uma pessoa. **Criar é criar**: CPF, matrícula ou login já existentes devolvem 409 e nada é escrito. A senha sai uma vez no corpo, sempre provisória.
`POST /usuarios/admin-contrato`	**admin-contrato**	O único papel que não passa pelo importador: sem `escola_id`, sem lotação, sem turma. O login dele é único na REDE, não na escola.

ATENÇÃO

O 409 de um CPF encontrado em **outra escola** da rede não devolve id, nome nem o nome da unidade — é dado de outro *tenant*. A mensagem diz o caminho legítimo (transferência é pela importação, que move a matrícula junto) e nada além disso.

13.4.6 A rede: regionais e escolas

Só `admin-contrato` chega a estas rotas — escola é *tenant*, e abrir um inquilino com RLS e dado de menor atrás não é operação de quem administra uma unidade.

Rota	Código	Semântica
`POST /escolas`	400	`tipo: escola` EXIGE `paiId` de uma regional existente (**ADR-79**). O 400 carrega o texto do produto — "Selecione uma regional para cadastrar a escola." — e a tela o mostra sem traduzir. `tipo: regional` entra sem pai.
`PATCH /escolas/:id`	400	`id` e `tipo` não se editam: os dois circulam em *token* e em RLS. `paiId` vazio numa `escola` é 400; TROCAR de regional é permitido — é a unidade mudando de diretoria. É por aqui que a regional padrão ganha o nome da rede.
`DELETE /escolas/:id`	409	**Só regional.** Escola não se exclui por rota nenhuma. Dois conflitos nomeados: `regional-unica` ("A rede precisa ter pelo menos uma regional.") e `regional-com-escolas`, que devolve `escolas` e `destinos` no corpo — é com esses dois números que a tela decide entre abrir o diálogo de transferência e explicar que não há destino. `?transferirPara=` move as escolas e exclui na MESMA transação.

ATENÇÃO

Regional e rede não recebem lotação, e a checagem estava em UM lugar só. `POST /escolas/lotacoes-em-lote` filtrava `tipo = 'escola'` desde o início; `POST`

> `/escolas/:id/lotacoes` não filtrava nada — por ela dava para lotar alguém numa regional e produzir no seletor do cabeçalho uma escola sem turma, sem estudante e sem dado. Corrigido com a **ADR-79**: a regra agora vale nas duas portas.

13.4.7 Planos de aula (app-90)

Rota	Código	Semântica
`POST /planos`	**202**	Enfileira a geração e devolve o id. *Não* espera o modelo.
`GET /planos/:id`	200	Estado (`pendente`, `gerando`, `pronto`, `erro`) e, quando pronto, o conteúdo. É o alvo do *polling*.
`PUT /planos/:id/conteudo`	200	Salva o HTML do editor, sanitizado por *allowlist*.
`POST /planos/:id/restaurar`	200	Volta ao `gerado.md`, que nunca foi sobrescrito.
`PATCH /planos/:id`	200	Renomeia.
`DELETE /planos/:id`	200	Exclui.
`GET /planos/referencias/disponiveis`	200	Que disciplinas e séries têm material importado — é o que popula o formulário.

13.4.8 O agente de IA do professor

ADR-126. Classe com `@Roles("professor")`, `DisponibilidadeGuard` e `@Produto("agente-professor")`: rota nova nasce coberta pelo interruptor, e o único escape é o `DELETE`. O id de todas as rotas é o da linha de `agente_professor_conversas`, nunca o `sessao_id` do engine; id que não é UUID responde como inexistente.

Rota	Código	Semântica
`GET /agente-professor/conversas`	200	As do professor na escola ativa, a mais recente primeiro. `titulo` é `null` até o fim do primeiro turno. Sai do Postgres; não toca o engine.
`POST /agente-professor/conversas`	**201**	Cria a sessão no engine (com `nome_professor` e expiração distante) e a linha da posse. Não chama o modelo: não consome quota. 503 `AGENTE_INDISPONIVEL` com o agente fora do ar ou `desligado`.
`GET /agente-professor/conversas/:id`	200	Título e mensagens, montados da sessão: o texto do professor e o texto FINAL do agente, com as fontes de cada turno. Sessão que sumiu do engine apaga a linha e responde 404.

`POST /agente-professor/conversas/:id/mensagens`	200 SSE	O turno (Seção 9.5): `text/event-stream` com `delta`, `fontes`, `titulo`, `aviso`, `fim` e `erro`. Antes de abrir o *stream*, em JSON: 400 (corpo), 404 (posse), 429 `RATE_LIMIT` (10 por minuto) e 429 `QUOTA`.
`DELETE /agente-professor/conversas/:id`	**204**	Apaga a sessão e a linha. `@SemDisponibilidade`: com o produto desligado, o professor continua apagando o que é dele. Engine fora do ar: 503, e a linha FICA.

REGRA INVIOLÁVEL

Outro dono é 404, nunca 403 — 403 confirmaria que o id existe. E os *rate limits* são por professor: 10 mensagens, 10 conversas novas, 30 exclusões e 120 leituras por minuto. O `Map` do `rateLimit` é por instância, como nas outras rotas.

ATENÇÃO

O `ContratoInterceptor` não confere a rota de mensagens: ela escreve na resposta crua (`@Res()`, `tipoDaResposta: text/event-stream`). Quem confere cada `data:` é o teste, contra `DADOS_POR_EVENTO` de `contrato-de-rotas.ts`; no OpenAPI a resposta aparece como `string`. E é a rota que infla o p95 da api (**ADR-128**).

13.4.9 Planejamento de ensino (app-92)

Duas famílias: a **regência** (o que o professor leciona) e o que pende dela — o plano anual e as 40 semanas. Identidade sempre do *token*: um `professorId` vindo do corpo deixaria qualquer professor escrever no caderno de outro, e a posse é conferida em toda rota com id.

Rota	Código	Semântica
`GET /planejamento/configuracoes`	200	As regências DESTE professor, com o dimensionamento (aulas no ano, horas-relógio) e o aviso `gradeCoerente: false`.
`POST /planejamento/configuracoes`	200	Cria ou atualiza. Idempotente na chave (professor, ano, série, componente).
`GET .../configuracoes/:id/calendario`	200	Os períodos de avaliação e as **40 semanas**, cada uma com quantas aulas tem de fato e o plano já gerado, se houver.
`DELETE /planejamento/configuracoes/:`	200	Apaga o planejamento **inteiro**: a regência, o plano anual e todas as semanas. O banco faz a cascata; os prefixos do *object storage* são varridos ANTES do `delete`.
`POST .../configuracoes/:id/plano-anual`	200	Cria o plano anual, com a distribuição **vazia** — que habilidade cai em que período é decisão do professor.

`PUT .../planos-anuais/:id/distrib`	200	A decisão pedagógica central. Recusa código fora do recorte componente × ano.
`POST .../planos-anuais/:id/gerar`	**202**	Um plano anual leva minutos. *Claim* atômico; quota conferida antes e consumida depois.
`GET /planejamento/planos-anuais/:`	200	O plano com o conteúdo. `html` é `null` enquanto o status não é `pronto` — é assim que a tela sabe continuar o *polling*.
`POST .../periodos/:ordem/gerar`	**202**	Gera as aulas de UMA semana. O esqueleto (quantas, quando) vem do calendário, não do modelo.
`GET .../planos-anuais/:id/periodo`	200	As semanas **já planejadas**, para exportar várias juntas. Filtro opcional por `periodoDeAvaliacao`.
`GET /planejamento/periodos/:id`	200	Uma semana, aula a aula.
`PUT .../periodos/:id/aulas`	200	As aulas editadas — e `cumprida`/`observacaoDoProfessor`, que são preenchidos DEPOIS da aula e não vêm da IA.

> **NOTA**
>
> `GET .../planos-anuais/:id/periodos` devolve **só as `pronto`**, em ordem: é a fonte do documento de mais de uma semana (a entrega quinzenal e a do período de avaliação inteiro — ver a Seção 12.16.9), e um documento entregue à coordenação não leva semana que falhou na geração. O filtro é no **banco**: baixar o 1º bimestre não tem por que carregar as aulas dos outros três.

13.4.10 Bem-estar (app-91)

Rota	Semântica
`GET /bemestar/resumo`	**Leve, sem IA.** É o que o *card* do hub lê — a home do professor não pode depender de uma chamada de modelo.
`GET /bemestar/hoje`	Abre o *check-in* do dia: 3 itens *verbatim* + textos de entorno (IA ou *fallback*).
`POST /bemestar/checkins/:id/respostas`	Grava. No sétimo, pontua e classifica *server-side*.
`GET /bemestar/historico`	Série do próprio professor.
`DELETE /bemestar/dados`	Apaga tudo do próprio professor.

REGRA INVIOLÁVEL

Todas as cinco são `@Roles("professor")` e filtram `professor_id = req.auth.usuarioId` em toda *query*. RLS por escola não basta aqui: dois professores da mesma escola não podem se ler.

13.4.11 Análise

Rota	Papel	Escopo
`GET /atividades/:id/analise`	professor	Uma atividade da própria escola.
`GET /atividades/:id/analise/c`	professor	Exportação da mesma visão.
`GET /analise/resumo`	**admin-contrato**	Toda a rede.
`GET /analise/atividades/:id`	**admin-contrato**	Uma atividade, cross-escola.
`GET /analise/csv`	**admin-contrato**	Exportação da rede.

DECISÃO DE PROJETO

São **dois *controllers* no mesmo arquivo**, com `@Roles` diferentes: um em `/atividades` para o professor, outro em `/analise` para o `admin-contrato`. A separação física é o que garante que a próxima rota adicionada herde a permissão certa — e é a mesma decisão tomada em `src/vila/`.

13.4.12 O Manifesto da IA (exp08ef4b)

Rota	Papel	O que faz
`GET /manifesto-ia/sessao`	estudante	O estado, o pacote da turma e a conversa **reconstituída** — é o que faz um F5 devolver a tela exata. Os questionamentos saem **sem** a réplica da máquina e as cláusulas **sem** o argumento de mediação.
`GET /manifesto-ia/gate`	estudante	O *polling* de 5 s do Checkpoint A (**ADR-93**). Três campos, sem catálogo: é a rota mais chamada da experiência, e por isso ela não faz *JOIN*.
`POST /manifesto-ia/disparar`	estudante	Envia um questionamento e devolve a defesa da máquina. **A réplica só existe nesta resposta.** Quando o segundo questionamento distinto entra, o passo anda para a decisão **no servidor**.

`POST /manifesto-ia/clausula`	estudante	O PICO, e *latch*. Cobra a cota de auditoria (409 `auditoria-insuficiente`) e **compila** o manifesto. Nada é corrigido: as quatro cláusulas são válidas.
`POST /manifesto-ia/percurso`	estudante	As duas palavras-chave e o parágrafo autoral. Dado pessoal é recusado com **422** e não gravado; durante o debate responde 409 `debate-em-andamento`.
`POST /manifesto-ia/concluir`	estudante	O *latch* final. **Não recebe `score`** — a missão não pontua.
`GET /manifesto-ia/turmas/:id/painel`	prof., gestor	Progresso e escolha por estudante, mais a distribuição das quatro cláusulas e a nuvem de palavras-chave. **Sem nota, sem nível e sem taxa de acerto.** O parágrafo autoral sai aqui e só aqui.
`GET /manifesto-ia/turmas/:id/conteu`	prof., gestor	O pacote da turma **com** o material de mediação: a réplica de cada questionamento e o argumento de cada cláusula.
`POST .../gerar-conteudo`	prof., gestor	A única chamada de IA da experiência, **pré-aula**. Devolve **rascunho** — não persiste nada.
`POST .../homologar`	prof., gestor	O clique em "Aprovar Conteúdo da Turma": o único lugar que liga `aprovado_por_professor`.
`POST .../gate`	prof., gestor	"Pausar para Debate" / "Retomar Missão". Idempotente: é o clique que se dá duas vezes.
`DELETE /manifesto-ia/turmas/:id`	prof., gestor	Zerar a turma. Apaga a execução e **preserva** `exp08ef4b_conteudo_turma`.

REGRA INVIOLÁVEL

Duas rotas de escrita e a de leitura do pacote são por TURMA, com *tenancy* — diferente das quatro `/<slug>/professor/*` de catálogo. A razão é que o conteúdo desta missão é parametrizável: o caso da escola, as réplicas e as cláusulas podem ter sido reescritos pelo professor daquela turma, e servir um catálogo global faria o guia projetar uma fala que ninguém recebeu.

ATENÇÃO

`POST .../gerar-conteudo` nunca falha para fora. Sem saldo de quota, com o provedor fora do ar ou com a saída reprovada pela validação, ela devolve o **banco curado** com `usouIa: false`, e a tela diz isso. Bloquear o professor seria o único jeito de esta rota estragar a aula — ele precisa poder aprovar o pacote curado com o Vertex indisponível.

13.4.13 Quem mandou isso? (exp05ef1b)

Quinze rotas em três *controllers* (**ADR-118**): oito do estudante (`@Roles("estudante")` na classe, e `exigirMissaoLiberada` no funil de TODAS), seis por turma e uma de catálogo. As duas perguntas de evidência não são deste módulo: passam por `/evidencias/exp05ef1b/perguntas/*` (Seção 13.4.4).

Rota	Papel	O que faz
`GET /quem-mandou-isso/sessao`	estudante	O estado, as 8 mensagens **sem** o gabarito, as falas que dependem do servidor (a da Nina leva o tema), as opções e os limites — é o que faz um F5 cair na tela certa. `pacote` é `null` enquanto não há pacote aprovado; `revelacao` é `null` até o voto E a revelação aberta.
`GET /quem-mandou-isso/gate`	estudante	O *polling* de 3 s (**ADR-93**): a fase da turma, o passo, se há pacote, a atividade e o `agora` do SERVIDOR. Sem catálogo: é a rota mais chamada.
`POST /quem-mandou-isso/entrar`	estudante	"Entrar no grupo": abre a sessão da rodada. Idempotente.
`PUT /quem-mandou-isso/selecao`	estudante	O rascunho da triagem, de 0 a 3. Quatro é 400 `selecao-acima-do-limite`; depois do Mural, 409 `mural-ja-enviado`.
`POST /quem-mandou-isso/mural`	estudante	**O latch 1**: exatamente 3 (menos é 400 `selecao-incompleta`), com a marca da EF05CO08 na mesma transação. A mesma seleção de novo é idempotente; outra, 409.
`POST /quem-mandou-isso/enquete`	estudante	**O latch 2**: o motivo. O mesmo voto de novo é idempotente; outro, 409 `enquete-ja-votada`.
`POST /quem-mandou-isso/reflexao`	estudante	**O latch 3**: a regra Se/Então, só com a reflexão liberada (antes, 409 `reflexao-nao-liberada`). Três recusas com código próprio: `entao-curto`, `entao-longo` e `entao-sem-sentido`. **Não recebe nota**: a regra não é corrigida.
`POST /quem-mandou-isso/concluir`	estudante	O fim, depois das perguntas — sem exigi-las. Idempotente.
`GET /quem-mandou-isso/turmas/:id/pacote`	prof., gestor	O ativo, o rascunho, o banco curado e os componentes curriculares — **com** o gabarito.

`POST .../gerar`	**só professor**	A única chamada de IA da missão, **pré-aula**, pelo `QuotaService`; grava o rascunho. Sem saldo ou com o provedor fora, devolve o banco curado com `usouIa: false` e o `motivo` — nunca falha para fora. A turma é conferida (404) antes de gastar *token*.
`POST .../aprovar`	**só professor**	"Aprovar Pacote", com o pacote INTEIRO: a régua de `validarPacote` (400 `pacote-invalido` com a lista de `erros`). Pacote diferente abre atividade nova; o mesmo é idempotente.
`POST .../gate`	**só professor**	Os três botões (`liberar_revelacao`, `apresentar_sintese`, `liberar_reflexao`). Só para a frente: o clique repetido ou fora de ordem não recua a fase.
`GET .../painel`	prof., gestor	O progresso, o Termômetro (`null` antes da revelação), a folha por computador e o roteiro. O "Então" vem `null` para quem não leciona na turma.
`POST .../nova-rodada`	**só professor**	"Zerar Turma para Nova Aplicação": a rodada que o professor VIU no corpo torna o duplo clique idempotente; rodada velha é 409 `rodada-desatualizada`.
`GET /quem-mandou-isso/professor/material`	prof., gestor	O guia da aula: o banco curado com gabarito, o roteiro, as falas e as perguntas de evidência COM o nível. Catálogo, sem *tenancy* (**ADR-86**).

REGRA INVIOLÁVEL

O gabarito sai em quatro respostas de PROFESSOR — o pacote, o rascunho da geração, o painel (pela folha, que nomeia o quadrante de cada escolha) e o material do guia — **e em UMA de estudante**: o bloco `revelacao` da sessão, só com os eixos das três que ele encaminhou, depois do voto e da revelação aberta pela turma. A turma do estudante é `users.turma_id`, a mesma fonte do painel e das equipes — e não o *claim* do *token*.

13.4.14 Jornal da Escola (exp04ef1b)

Vinte e três rotas em três *controllers* (**ADR-117**): treze do estudante (`@Roles("estudante")` na classe, e `exigirMissaoLiberada` no funil `garantirSessao` de TODAS, com a disponibilidade junto), nove por turma, para professor e gestores, e uma de catálogo. A turma do estudante sai do *token*, nunca do corpo. As duas perguntas do Desafio Final não são deste módulo: passam por `/evidencias/exp04ef1b/perguntas/*` (Seção 13.4.4).

Rota	Papel	O que faz
`GET /jornal-escola/sessao`	estudante	O estado e as 8 notícias na ordem da mesa, **sem** gabarito, pistas nem consequência; o passo já reconciliado com a etapa; as pistas já abertas, com o texto (o F5); e os seis blocos da Chave na ordem da bandeja, `null` até a revisão abrir. Cria o banco padrão na primeira entrada da turma.
`GET /jornal-escola/gate`	estudante	O *polling* de 5 s (**ADR-93**): etapa, rodada, pacote, passo e o `agora` do SERVIDOR. Sem notícias e sem JOIN.
`POST /jornal-escola/passo`	estudante	Os dois passos que a criança pede: `mesa` e `transferencia`. A escada é de sentido único.
`POST /jornal-escola/pista`	estudante	Abre fonte, data ou autoria de uma notícia e devolve o texto — é o registro da EF15CO04. Idempotente por par na mesma fase.
`POST /jornal-escola/diagramar`	estudante	O layout da página com as três vagas de agora. **Nunca falha por causa da IA**: *cache*, IA com a quota do estudante e teto de cerca de 6 s, e o modelo automático (Capítulo 11).
`POST /jornal-escola/publicar`	estudante	**O latch da 1ª edição**: três ids, as verificadas calculadas AQUI, e a marca da EF15CO04 na mesma transação. A resposta não traz o resultado. Outra publicação, 409 `edicao-ja-publicada`; o professor zerou no meio, 409 `rodada-mudou`.
`POST /jornal-escola/motivo`	estudante	A pergunta de depois do latch, uma vez só (`visual` | `checagem`).
`GET /jornal-escola/resultado`	estudante	As verificadas e uma reação de leitor por vaga, só com o motivo respondido e a etapa `resultado` aberta (antes, 409 `resultado-nao-liberado`).
`POST /jornal-escola/chave`	estudante	Uma tentativa da Chave: três blocos em qualquer ordem, corrigidos no servidor; `devolvidos` são as iscas. Na formação, a marca da EF15CO02. O número da tentativa descarta o reenvio da rede.
`GET /jornal-escola/revisao`	estudante	O pilar em falta de cada vaga da 1ª edição, depois da Chave.
`POST /jornal-escola/republicar`	estudante	**O latch da 2ª edição**, com a marca da EF04CO08 (`edicao-2`). Pode repetir as três vagas da 1ª.
`GET /jornal-escola/comparativo`	estudante	As duas edições, com o que cada uma causou e o pilar que faltava em cada notícia.

`POST /jornal-escola/concluir`	estudante	O fim, sem exigir as perguntas. Idempotente.
`GET /jornal-escola/turmas`	prof., gestor	O seletor, por matrícula para o professor.
`GET .../turmas/:id/painel`	prof., gestor	O botão da próxima etapa e o progresso POR COMPUTADOR; o histograma anônimo, o motivo agregado e o radar das notícias. Nenhum nível nem verificada por linha. A leitura do professor conta no EVT-P04; a do gestor, não.
`POST .../gate`	prof., gestor	O botão único: `exploracao` → `resultado` → `revisao`. Destino igual ao atual é aceito e não muda nada; fora de ordem, 409.
`POST .../zerar`	prof., gestor	A próxima rodada do mesmo pacote, sem apagar nada.
`GET .../pacotes`	prof., gestor	O pacote em andamento **com** o gabarito, e o histórico das atividades da turma (o `?pacote=` do painel de habilidades).
`POST .../gerar-conteudo`	prof., gestor	O rascunho das 8 notícias por IA, **pré-aula**, no molde sorteado. Não persiste. Sem saldo, 429; sem provedor, 503.
`POST .../regenerar-noticia`	prof., gestor	Uma notícia nova com o mesmo id, meio e papel. Falha da IA é 200 com `usouIa: false` e a notícia como estava.
`POST .../homologar`	prof., gestor	"Aprovar Pacote", com as oito notícias INTEIRAS e a régua de `invariantes.ts` (400 `pacote-invalido` com a lista de `erros`). Pacote novo é atividade nova; o mesmo conteúdo é idempotente.
`POST .../gerar-midia`	prof., gestor	Até duas imagens novas por chamada, com custo estimado na quota do professor; repetir até `completo`.
`GET /jornal-escola/professor/material`	prof., gestor	O banco padrão com gabarito, os blocos da Chave com os pilares marcados e o roteiro da mediação. Catálogo, sem *tenancy* (**ADR-86**).

REGRA INVIOLÁVEL

O gestor alcança as MESMAS rotas de turma que o professor, e isso é deliberado: a missão não tem texto livre de menor, então não há campo a recortar. O que separa os dois é a turma — `turmaVisivel` devolve 404, e não 403, para a turma que o professor não leciona —, e `gestor_contrato` e `admin` dizem `?escolaId=` (`comum/escola-alvo.ts`), levado explícito ao `comTenancy`. As rotas que gastam IA do professor têm, além da quota, um teto de requisição por minuto.

13.4.15 As rotas que alimentam o guia da aula

Duas rotas existem só para o **guia da aula** do professor (Seção 12.4), e as duas pelo mesmo motivo: o texto que o professor projeta ou lê à turma **é** o instrumento, e uma cópia dele no hub divergiria na primeira revisão de *copy*.

Rota	Papel	O que devolve
`GET /vila/professor/cartoes`	professor	As mensagens das três bolhas, **filtradas pela turma** (`?turmaId=`): nos anos iniciais a bolha B não é servida.
`GET /unoogpt/professor/fontes`	professor	As 8 fontes do cenário com o viés de cada uma, o período e os bairros que cobrem.
`GET /a-selecao/professor/material`	prof., gestor	O roteiro da Etapa 4 (as três perguntas do debate), os gatilhos dos três momentos, os 12 posts do banco curado com o gabarito e as seleções dos colegas virtuais.
`GET /rastro-boato/professor/mate`	prof., gestor	As 3 intervenções com `desfecho` e a reação do grupo, e o ciclo na **ordem certa** — o contrário do que a tela do estudante recebe.
`GET /foto-conta/turmas/:id/projecao`	prof., gestor	A projeção da Roda da exp01ef1b: o par de fotos, os rótulos, as 3 perguntas do debate e o termômetro do *latch* — o material vem da API para a lousa e os tablets nunca discordarem.
`GET /quem-mandou-isso/professor/material`	prof., gestor	O banco curado da exp05ef1b com o gabarito, o roteiro da roda, as falas da Nina e as perguntas de evidência com o nível de cada alternativa.
`GET /jornal-escola/professor/material`	prof., gestor	O banco padrão do Jornal da Escola com o gabarito e a reação de cada notícia, os seis blocos da Chave com os pilares marcados, e o roteiro: o gatilho do resultado, as três perguntas do debate e o gatilho da revisão. É também o "Usar Banco Padrão" do preparo.

REGRA INVIOLÁVEL

As de `/<slug>/professor/*` são catálogo; a da Vila não. Em `/unoogpt/professor/fontes`, `/rastro-boato/professor/material`, `/quem-mandou-isso/professor/material` e `/jornal-escola/professor/material` não há `escolaId` nem *tenancy* a aplicar — o conteúdo é igual para todo contrato, e o que a guarda protege é o PAPEL. Por isso vivem em *controller* separado do de turmas. `/vila/professor/cartoes` é a exceção: ela recebe `?turmaId=` porque a bolha servida depende da série da turma.

As cinco devolvem exatamente os campos que o servidor retira antes de a opção chegar ao estudante (regra inviolável nº 3): o que muda entre as duas superfícies é o **leitor** (**ADR-86**). No unooGPT isso é mais extremo — a Missão Tempestade não revela nada por *software*, de propósito, e esta é a única superfície em que o viés é dito com todas as letras. Quem sabe de antemão qual fonte engana em quê decora um quadro em vez de cruzar dados.

NOTA

O viés não é um campo escrito à mão ao lado da instrução do agente: é **derivado** dela, cortado na marca `VIÉS:`. Dois textos seriam duas verdades, e a recalibragem do cenário mexeria num só.

13.4.16 Observabilidade: os sinais que nascem no navegador

Três rotas em `/observabilidade`, com `OidcGuard`, todas `204`, todas com *schema* Zod `.strict()` e cota por (escola, usuário) em memória — porque quem as chama é um *bundle* público, e o que passa por elas vira log, métrica ou *span* num projeto do Google (**ADR-58**; Capítulo 17).

Duas delas têm um par **público** em `/observabilidade/publico/*` — `erros-cliente` e `vitals` —, sem `OidcGuard`, para a *landing*, o *login* e a troca de senha, onde ainda não há sessão (**ADR-84**). Quem escolhe entre o par autenticado e o público é o **BFF do hub**, não o cliente: com sessão o log ganha o `usuarioId`; sem sessão, a linha sai marcada `autenticado: false` e sem identidade nenhuma. A cota do par público é por IP, somada a um teto global de instância.

- `POST /observabilidade/erros-cliente`: um erro não capturado (`window.onerror`, `unhandledrejectio` da experiência ou do hub. Vira **uma** linha `ERROR` com `@type ReportedErrorEvent`, `serviceContext = {expId, versao}` e `stack_trace` — o que o Error Reporting agrupa e chama de "novo na versão X". A *redaction* da API passa por `mensagem`, `stack` e `url`; o identificador do estudante é o do *token*, opaco. Cota: 5 por minuto; acima, `429` e um contador, sem linha de log.
- `POST /observabilidade/vitals`: um Web Vital (`LCP`, `INP`, `CLS`, `TTFB`) amostrado a 10% no cliente. Vira ponto num histograma OpenTelemetry por `superficie`, `rota` **normalizada** (`normalizar-rota.ts`: `/professor/:secao`, `exp04ef1b`, senão `outra`) e `perfil` do *token*. Nunca vira log.
- `POST /observabilidade/v1/traces`: o lote OTLP/JSON que o OpenTelemetry Web da experiência exporta. A API valida o subconjunto ($\leq$ 64 *spans*, atributos por *allowlist*, URL sem *query*), **substitui** o `resource` pelo `service.name`/`service.version` validados e reexporta para a Telemetry API do Google com a credencial da própria API (`otlp-proxy.service.ts`). O navegador nunca fala com o Cloud Trace. Sem `GCP_PROJECT_ID` o lote é contado e descartado.

13.4.17 `/recursos/*` — o interruptor da rede

UMA rota de leitura, e ela é a base dos *cards* do hub e do 404 das rotas de produto:

- `GET /recursos/efetivos` (**qualquer autenticado**) — resolve a cadeia escola → regional → contrato, aplica o TETO do produto e devolve `{produtos, experiencias}`. `produtos` traz SEMPRE todos os do vocabulário (hoje cinco), ligados ou não: um mapa que só lista o ligado obriga quem lê a saber a lista completa de cor — e o hub passaria a ter a segunda fonte do vocabulário. O quinto, `agente-professor`, é o primeiro produto SEM experiência: no painel da Unooria ele aparece com `experiencias: []` (a composição é derivada de `experiencias.produto`, e nenhuma aponta para ele), e é por `produtos["agente-professor"].ligado` que o hub decide montar o botão flutuante do professor (**ADR-126**).

Três decisões dentro dessa rota. O escopo sai do **token** (o claim `escola_id`) e nunca de *query*, para não haver "me diga o que está ligado na escola do vizinho". Perfil de contrato não tem escola no token, então a resposta é a do CONTRATO — a cadeia fica vazia e só as linhas de escopo-contrato valem. E ela **não devolve `motivo` nem quem desligou**: isso é conversa entre a Unooria e a rede, e num *payload* que qualquer autenticado lê viraria a explicação errada na tela errada.

A ESCRITA é do painel, restrita ao `admin` (**ADR-88**) — a primeira capacidade que ele tem e o `gestor_contrato` não:

- `GET /recursos/painel` — por recurso, o que o interruptor DAQUELE escopo mostra (`estadoAqui`, três estados) e o que efetivamente vale (`efetivo`, com a origem). Sem `?escolaId=`, o escopo é o contrato.
- `PUT /recursos/{recursoId}` — liga ou desliga. `escolaId` vem no CORPO e `null` explícito é o contrato: numa *querystring*, ausente e vazio são indistinguíveis, e desligar a rede inteira não pode acontecer porque alguém montou a URL sem o parâmetro. **Desligar exige `motivo`** — Zod aqui, CHECK no banco.
- `DELETE /recursos/{recursoId}` — remove a exceção do escopo. É o terceiro estado (*herda*); sem esta rota, exceção criada por engano seria permanente. Remover o que não existe devolve `removida: false` e não é erro.

O *upsert* é SQL cru, e a razão é estrutural: o `upsert` do Prisma não consegue endereçar a linha de escopo-contrato, porque o único composto `(recurso_id, escola_id)` tipa `escolaId` como não-nulo — um único composto do PostgreSQL não casa `NULL`, que é a mesma limitação que obrigou o índice PARCIAL a existir. `ON CONFLICT (recurso_id) WHERE escola_id IS NULL` infere aquele índice, e são dois `ON CONFLICT` distintos porque são dois índices distintos.

Toda escrita registra `codigo: recurso-alterado` com recurso, escopo, estado e motivo — a linha é sobrescrita a cada mudança, então o registro é a única resposta a "quem desligou isto, e quando?".

A VERSÃO tem três rotas, e duas alavancas que nunca aparecem juntas:

- `GET /recursos/experiencias/{expId}/versoes` — o histórico de `experiencias_versoes`, com `emRollback: true` quando a ativa não é a última publicada. É o estado que a tela tem de dizer com todas as letras, porque o próximo *deploy* que tocar a experiência sobrescreve o *rollback*.
- `POST /recursos/experiencias/{expId}/ativar` — troca a `versao_ativa` do CONTRATO e **restaura o manifesto daquela versão**. É o conserto de um defeito que ninguém havia notado: `publicar-manifesto.mjs --versao x.y.z` manda o número antigo com o `experiencia.json` ATUAL do repositório, então voltar a versão nunca voltava `viewport` nem `permissoes` — a app-70 reabria sem `allow-pointer-lock`, a criança entrava na Vila e a câmera não girava, sem erro em lugar nenhum.
- `PUT /recursos/experiencias/{expId}/versao` — fixa (ou solta) `versao_fixa` num escopo. É o canário: uma escola roda a versão nova enquanto a rede segue na anterior. `versao: null` solta a fixação e **preserva** o `ligado` da linha, que é decisão à parte.

Ativar só no escopo-contrato; fixar só fora dele, e a tela oferece uma alavanca por escopo. Fixar no contrato seria um segundo jeito, mais fraco, de fazer o que ativar faz — trocaria a versão do *bundle* sem restaurar o manifesto, reabrindo exatamente o defeito acima.

As duas escritas conferem no CDN com um HEAD no `index.html` da versão (`CDN_EXP_URL`), e a resposta carrega `conferida`: sem a env (*dev*) a conferência é pulada, e a tela DIZ isso — "ativei" e "ativei e conferi que o *bundle* está lá" são garantias diferentes.

O HISTÓRICO é a única rota de `/recursos/*` que a REDE alcança (ADR-89): `GET /recursos/auditoria`, aberta a `admin` e `gestor_contrato`. Ela lê `recursos_auditoria`, escrita por toda mudança do painel — e o `autor_nome` vem CONGELADO da linha, nunca de um *JOIN*: o histórico tem de dizer quem era aquela pessoa naquele dia, e não o nome de hoje. Sem `?escolaId=` traz o contrato inteiro; com ele, traz as mudanças daquela escola **e** as de escopo-contrato, porque foram elas que decidiram o que vale ali. As escritas e o `GET /recursos/painel` continuam respondendo 403 ao `gestor_contrato`.

E o hub HONRA a versão fixada: `montarEmbed` lê `GET /recursos/efetivos` em paralelo com o manifesto e monta a URL do `iframe` com a versão do escopo. Sem essa leitura, a coluna seria gravada, o painel a mostraria e nada mudaria — campo que não muda nada é pior que campo nenhum.

A QUOTA DE IA mora no mesmo prefixo (ADR-105), porque é o mesmo painel, a mesma guarda e o mesmo histórico — e só o `admin` a alcança, nem na leitura:

- `GET /recursos/quotas-de-ia` — uma linha por perfil de `ia_quotas` (`estudante`, `professor`), sempre as duas, com quem definiu (nome resolvido) e o motivo. O contador do dia (`ia_uso_diario`) NÃO sai aqui: é dado por pessoa.
- `PUT /recursos/quotas-de-ia/{perfil}` — sobrescreve o teto. **Motivo obrigatório sempre** (Zod): aqui toda mudança é exceção ao padrão. Zero e negativo recusados (Zod e CHECK), teto de sanidade de 100.000.000. Valor igual ao atual devolve `alterada: false` e não grava nem historia. Registra `codigo: ia-quota-alterada` e a linha de `recursos_auditoria`

(ação `quota-alterada`, `detalhe = {de, para}`). Vale para a rede inteira na próxima chamada de IA: o `QuotaService` lê a linha a cada `temSaldo`, sem *cache*.

- `GET /recursos/arquivadas` (**ADR-116**) — as experiências arquivadas, tiradas do produto sem serem apagadas, lidas de `experiencias_arquivadas`: nome, código de origem, onde ficava na grade, última versão, motivo, data e a FOTOGRAFIA de turmas e estudantes que a migração tirou ao arquivar. Não reconta nada em tempo de uso — contar `liberacoes` e `progresso` de todas as escolas atravessaria a RLS.

O `GET /recursos/auditoria` ganhou `?prefixo=` (`quota:` recorta as mudanças de quota) e a coluna `detalhe`, só preenchida nessas linhas.

13.5 Prospecção: a única escrita sem token (ADR-107)

O formulário do site público (`/`) grava pela api, e não num banco próprio. São três rotas, e o prefixo separa o que tem *guard* do que não tem:

- `POST /prospeccao/publico/cadastros` — **sem *guard***, e é a única rota de dados da api nessa condição. Recebe nome, e-mail, telefone, organização, cargo e o aceite; grava em `prospeccao_leads` com a FRASE consentida. **Três situações respondem a MESMA coisa** (`201 {recebido: true}`): gravou, o e-mail já se cadastrara na última hora, ou o campo-armadilha veio preenchido. Dizer "este e-mail já está cadastrado" transformaria a rota num verificador de quem procurou a Unooria; recusar a armadilha com erro ensinaria o robô a tentar sem o campo. Cota de 5/hora por IP e 200/hora na instância; acima, 429.
- `GET /prospeccao/cadastros` — `admin`. Do mais novo para o mais antigo, teto de 5.000 linhas, com o `total` real ao lado: se os dois divergirem, a tela avisa em vez de afirmar que aquilo é tudo. A data sai em ISO 8601 e quem formata é a tela (**ADR-98**/**ADR-99**).
- `GET /prospeccao/cadastros/exportacao` — `admin`. O mesmo recorte em CSV, com separador `;` e BOM UTF-8 (o par que faz o Excel em português abrir colunas separadas e acentos certos), todo campo entre aspas e aspas internas dobradas — nome de secretaria leva `;` com frequência, e um separador solto dentro do valor desloca a linha inteira sem nada acusar. O caminho **não** termina em `.csv`, pela mesma razão de `/analise/exportar/{nivel}`: segmento de rota com ponto quebra a validação de tipos do App Router no hub. O nome do arquivo sai no `content-disposition`.

A escrita não passa pelo BFF. O `/api/plataforma/*` do hub exige sessão, e ampliar a exceção dos sinais de navegador (**ADR-84**) para uma rota que ESCREVE dado pessoal tornaria "o BFF exige sessão" uma regra com lista de exceções. Ela tem rota própria no hub (`POST /api/cadastro-de-interesse`), que também traduz o código de recusa na frase que o visitante lê. O que essa rota **precisa** repassar é o `x-forwarded-for`: sem ele todos os visitantes chegam à api como um cliente só, o primeiro robô gasta a cota de todos e o formulário passa a recusar gente de verdade.

13.6 Sanidade e prontidão

Rota	Semântica	Uso
`GET /healthz`	O processo está vivo	*Liveness* do Cloud Run.
`GET /readyz`	As dependências estão prontas	*Readiness*: verifica o banco. É o que impede uma revisão nova de receber tráfego antes de conseguir consultar.

> **NOTA**
>
> As duas são públicas de propósito. Uma sonda de saúde que precise de *token* depende do provedor de identidade para dizer se a API está viva — o que inverte a relação de diagnóstico exatamente quando ela mais importa.

13.7 Documentação executável (OpenAPI)

A API publica um documento OpenAPI 3.0 em `/docs` (Swagger UI navegável) e `/docs-json`. Ele é **derivado** dos *schemas* Zod que as próprias rotas usam para validar — não escrito ao lado deles (ADR-36).

Peça	Papel
`openapi/contrato.ts`	O decorator `@Contrato`: resumo, prosa, corpo, *query*, resposta e notas. Só grava metadata.
`openapi/rotas.ts`	Percorre os *controllers* pelo `DiscoveryService` e devolve verbo, caminho, `@HttpCode`, `@Roles`, *guards* e contrato de cada rota. Fonte única dos três consumidores.
`openapi/zod-para-openapi.ts`	Zod → JSON Schema no dialeto do OpenAPI 3.0.
`openapi/documento.t`	Monta cada operação. O esqueleto de caminhos vem do roteador do Nest; o conteúdo, do `@Contrato`.
`openapi/porta-do-docs.`	A porta de senha (*Basic*), com comparação de tempo constante.
`openapi/contrato.in`	Confere a resposta contra o *schema* declarado, fora de produção (ADR-37).

> **NOTA**
>
> **Os papéis no documento vêm do `@Roles` real**, não de um campo repetido no `@Contrato`. Repetir abriria espaço para o documento dizer `gestor` enquanto o *guard* exige `admin-contrato` — e uma documentação de autorização errada é pior que nenhuma.

A trava. `tests/openapi-cobertura.spec.ts` percorre o roteador — não uma lista escrita à mão — e cobra `@Contrato` de toda rota, salvo as nomeadas numa lista explícita de pendências que só encolhe. Ela confere também que o acesso declarado bate com os *guards* que a rota realmente tem (rota que se diz pública e carrega `OidcGuard`, ou o contrário), e que a lista de pendências não guarda rota que já não existe. Rota nova sem contrato fica vermelha.

O documento e o Apêndice C são GERADOS. O `docs/openapi.json` é versionado no repositório — é o artefato que um cliente ou um edital consome sem precisar subir a plataforma — e o catálogo de *endpoints* sai dele, não mais de prosa escrita à mão.

Listing 13.2 – Regerar o documento e o apêndice

```
npx ts-node --transpile-only apps/api/scripts/gerar-openapi.ts
node scripts/gerar-endpoints-tex.mjs
cd documentacao-latex && ./compilar.sh
```

> **NOTA**
>
> A coluna de autorização do apêndice vem das extensões `x-papeis` e `x-acesso` que o documento carrega, e não da prosa da descrição. Um gerador que fizesse *parsing* do próprio texto procurando "**Papéis:**" quebraria na primeira vez que alguém reescrevesse a frase — e quebraria em silêncio, produzindo uma tabela de autorização errada, que é o pior defeito possível num catálogo de rotas.

14

Design system e camada de interface

> **REGRA INVIOLÁVEL**
>
> **Todo elemento de interface do hub e das experiências vem do `@unooria/design-system`, sempre na última versão publicada.** Componente de interface novo se escreve *no repositório do design system* e chega aqui por versão — não em `packages/ui`, não dentro de uma experiência. Antes de criar um botão, um *card* ou um campo: procure na seção correspondente do pacote, ela provavelmente já tem.

O pacote é npm **público** no escopo `@unooria` — sem *registry* privado, sem `.npmrc` no repositório. A fonte é o repositório `storybook-unooria` (Storybook em `:6007`, publicado por *tag* `v*` no GitHub Actions).

14.1 Duas famílias no mesmo pacote

O pacote publica **seções** independentes, cada uma com *namespace* de token e prefixo de classe próprios. Elas nunca se cruzam.

Seção	Import	Tokens	Onde vale
`unooria` (49, oito delas da 0.2.8)	raiz do pacote	`--unooria-*`, classe `.unooria-theme-*`	Hub: *landing* (`/`), *login*, `/professor`, `/admin`, `/analise`; *header* de todos os papéis; telas **pós-revelação** do estudante; e a redação INTEIRA do Jornal da Escola (exp04ef1b), que não é clone de nada
`instanooria` (21 + 7 extensões)	`/instanooria`	`--insta-*`, `insta-`	exp08ef1b, exp07ef1b; e só o `PostCard` na gaveta do Jornal da Escola (Seção 14.3)
`unitube` (30)	`/unitube`	`--uni-*`, `uni-` — **só escuro**	backup-exp04ef1b (o UnooTube arquivado)

`whatsappmobile` (39)	`/whatsappmobi`	`--wa-*` — só escuro	**exp05ef1b e exp06ef2b** — as duas missões de mensageiro. Na exp05ef1b, do convite da Nina à síntese, e de novo nas perguntas finais (a regra Se/Então e o fechamento são da plataforma). E só o `MessageBubble` na gaveta e no Desafio Final do Jornal da Escola
`chatgpt` (25)	`/chatgpt`	`--gpt-*`, `data-gpt-theme`	**exp02em1b inteira** — ela não tem revelação no software, então nunca troca para a seção da plataforma
`tiktooria` (35)	`/tiktooria`	`--tik-*`	— (dois *contextos* de superfície: cromo claro de lista e perfil, imersivo preto de *feed* e câmera — não dois temas)

NOTA — "pós-revelação" virou uma tela de verdade

Por muito tempo a linha "telas pós-revelação do estudante" descrevia um único componente: o `DonutChart` do relatório, num *chunk* tardio. O resto do relatório era Tailwind do clone sobre `bg-night`.
Com a ADR-41 (18.42) o relatório sai do aparelho e passa a ser claro e largo, e a autorização deixa de ser teórica. A fronteira não mudou: quem a marca é a REVELAÇÃO — sete telas, fundo `night`, sem nada da plataforma. Antes dela, o disfarce; depois, a língua da escola.
A trava técnica continua a mesma e não pode cair: o CSS do DS entra por `React.lazy`, no *chunk* do relatório, e nunca no *bundle* do feed — lá ele seria legível no DevTools antes da hora.

ATENÇÃO

A seção `unitube` é **só escura**: o Figma não publica variante clara, e **derivar uma é proibido**. Uma paleta clara inventada não é o clone — é uma aproximação, e a experiência mede reconhecimento.

14.2 A linha que separa as seções é a REVELAÇÃO

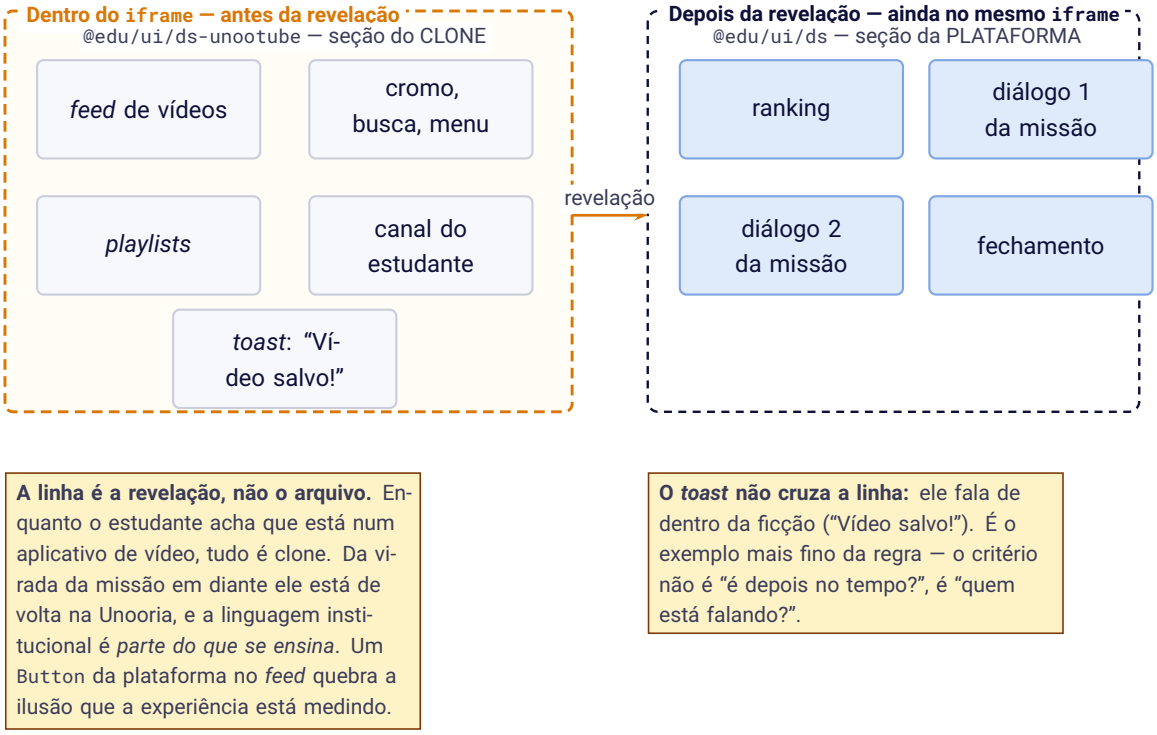


Figura 14.1 – A linha da revelação, no UnooTube (hoje `backup-exp04ef1b`). As duas seções do *design system* coexistem no mesmo *bundle* e a fronteira entre elas é pedagógica, não técnica.

> **NOTA**
>
> **A linha tem duas pontas, não uma.** A versão 1.1.0 do UnooTube abriu com uma **tela de entrada** que explica a missão, e ela usa a seção da **plataforma** — antes do *feed*, ou seja, *antes* da revelação. Não é exceção à regra: é a regra aplicada pelo critério que ela mesma enuncia, "quem está falando?". No *briefing* quem fala é a escola, e não há disfarce a proteger — a criança ainda não viu um *Short*. O disfarce começa no toque em "Começar missão". Ver **ADR-22**.

14.3 A exceção: a peça que imita outra plataforma, dentro da Unooria

REGRA INVIOLÁVEL

A exceção, e só ela (ADR-117): conteúdo de uma plataforma DENTRO de uma tela da Unooria usa a seção da plataforma que ele imita. O Jornal da Escola (exp04ef1b) não é clone de nada: mesa, gaveta, página, Bento, leitores, Chave e Desafio Final são `unooria`. Mas a notícia que veio de rede social aparece na gaveta como `PostCard` do `instanooria`, e a mensagem encaminhada como `MessageBubble` do `whatsappmobile`: a peça imitada fala na voz de onde veio, porque um post que não parece post não ensina a reconhecer post.

A regra é por **PEÇA**, nunca por tela. O cromo em volta continua sendo da Unooria, e a peça do clone não traz a barra, as abas nem o fundo do aplicativo — o balão do WhatsApp vai num contêiner pintado com `--wa-color-surface-base`, e não no `ChatWallpaper`, que é o FUNDO do app. O post vem sem `Actions`, `Likes` e `Timestamp`, e o balão sem hora nem confirmação de entrega: a data é uma pista que a criança revela à parte, e o número de curtidas viraria pista falsa de confiabilidade. É o INVERSO do que continua proibido — a plataforma dentro do clone — e vale só para conteúdo que a criança deve reconhecer como vindo daquela plataforma. Cada família mora num arquivo (`src/app/pecas/`), que só importa a sua ponte.

NOTA

Três seções na mesma tela, e nenhuma vaza. A gaveta é a primeira tela do repositório com três seções ao mesmo tempo, e a verificação foi feita na 0.2.8, nas folhas agregadas e nas de cada componente: todo seletor tem o prefixo da seção (`.unooria-*`, `.insta-*`, `.wa-*`), nenhuma folha reseta `body`, `html`, `*` ou elemento solto, os blocos `:root` só definem variáveis do próprio *namespace* e as famílias das folhas de fonte não colidem. O tema também não vaza: o `PostCard` com `theme="light"` põe `data-insta-theme="light"` no próprio cartão, e o `instanooria` não escurece com o sistema em modo escuro. Nada exigiu mexer no DS nem nas pontes.

14.4 Consumo: sempre pela ponte

REGRA INVIOLÁVEL

Consumir sempre pela ponte em `packages/ui`, nunca importando o pacote direto num componente:

`@edu/ui/ds`	`unooria`	plataforma, e a redação do Jornal da Escola (com as oito peças da 0.2.8: `NewsCard`, `NewsArticle`, `RevealClue`, `BottomSheet`, `NewspaperPage`, `SpeechBubble`, `PuzzleBlock` e `PuzzleTrack`, mais `NEWS_MEDIUMS` e `NEWSPAPER_LAYOUTS`)

`@edu/ui/ds-insta` `instanooria`	exp08ef1b, exp07ef1b (com as extensões da 0.2.5: `DirectThread`, `MessageBubble`, `DmPoll`, `ConfirmDialog`, `BottomSheet`, `FloatingButton`, `StoryComposer`; e as da 0.2.7: `InlineHint`, `GestureHint` e o `Snackbar` do clone, como `SnackbarInsta`); e o `PostCard` da gaveta do Jornal
`@edu/ui/ds-unootube` `unootube`	backup-exp04ef1b (o UnooTube arquivado, **ADR-116**: não roda mais)
`@edu/ui/ds-chatgpt` `chatgpt`	exp02em1b
`@edu/ui/ds-whatsapp` `whatsappmobile`	exp05ef1b, exp06ef2b; e o `MessageBubble` da gaveta do Jornal

A ponte existe por dois motivos, e os dois estão registrados nos arquivos:

1. **Marcar `'use client'` de uma vez**, para o App Router do Next.js.
2. **Desfazer a colisão de nomes.** `Icon`, `Avatar`, `Badge`, `Button`, `Logo` e `TextField` existem nas *duas* famílias. Os clones só saem pelo *namespace* (`InstanooriaUI.Icon`), e **destruturar na ponte** é o que mantém o *tree-shaking* e evita o erro `TS2742` causado pelo `@types/react` interno do pacote.

> **NOTA**
>
> Clone novo em uso = arquivo novo `packages/ui/src/ds-<clone>.ts` + `exports` no `packages/ui/package.json`. Mesmo padrão, sempre.

14.5 CSS: uma vez, no ponto de entrada

Aplicação	Onde o CSS entra
Hub	`components/casca-plataforma.tsx` — e é *por isso* que `/exp/*` e `/dev-embed`, que estão fora do grupo de rotas `(plataforma)`, também têm o *design system*.
Experiência	O próprio `src/main.tsx`, com o CSS da *sua* seção.

> **REGRA INVIOLÁVEL**
>
> **Nunca importe o consolidado `/styles.css` numa experiência**: ele arrasta as seis seções e a tipografia da plataforma para dentro do clone.

14.5.1 No hub, o CSS não basta: as fontes também

ATENÇÃO

O `unooria.css` declara `Open Sans` e `Material Symbols Rounded` por `@import` de URL remota — e o **Next remove `@import` externo** de CSS importado por JS, *silenciosamente*: a regra continua lá, o `getComputedStyle` responde "Open Sans" e o navegador desenha com a fonte do sistema.
Como `Material Symbols Rounded` é fonte de **ligadura**, o sintoma é o **ícone virando texto**: onde deveria haver um ícone de adicionar, aparece "addInvestigação".
Por isso `components/casca-plataforma.tsx` também monta `FontesDaPlataforma` (`@edu/ui/fontes`). Numa experiência Vite o `@import` sobrevive e isso não é necessário.

ATENÇÃO

Segundo caminho para o mesmo sintoma: `icon` como *string*. Componentes de navegação do pacote (`SideBar`, `Tab`) tipam `icon` como `React.ReactNode`, não como nome de ícone. Passando a *string*, o item renderiza o **texto cru**: a barra lateral da área administrativa escrevia "group", "school" e "upload_file" na lateral — com a mesma cara de fonte que não carregou, e com a fonte perfeitamente carregada.
A forma certa é `icon={<Icon name="group" />}`. Quem pegou foi o *smoke* visual: *typecheck* não reclama, porque `string` é um `ReactNode` válido.

REGRA INVIOLÁVEL

Quem importa, declara. Todo aplicativo que escreve `@unooria/design-system` no código tem a dependência no seu `package.json` — hub incluído, porque ele importa o `unooria.css`. Depender do *hoisting* do pnpm via `packages/ui` funcionava *por acaso*: bastava a ponte trocar de versão (ou o `node_modules` ser reinstalado com outro *layout*) para o CSS do hub sumir sem ninguém ter mexido nele.

14.6 Frames: dois níveis, não empilhar

Componente	Onde vale
`PhoneFrame`, `DesktopFrame`	Cascas de aparelho **do clone**, publicadas pelo pacote. Servem ao Storybook e a telas soltas.
`DispositivoFrame` (hub)	**Dentro da plataforma, quem desenha o aparelho é o hub**, dirigido pelo `viewport` do manifesto. A experiência embedada preenche o `iframe` e não traz casca própria.

14.7 Tema

Seção	Como recebe o tema
Clone (`unitube`)	Lê `data-uni-theme` na raiz do documento.
Plataforma (`unooria`)	Recebe `theme` por **prop**.

> **ATENÇÃO**
>
> Quem troca o tema escreve nos **dois** (`backup-exp04ef1b/src/lib/tema.ts`), senão o estudante escolhe escuro no *feed* e o ranking volta branco — porque o ranking é da seção da plataforma, que não lê o atributo da raiz. No UnooTube a escolha é do estudante (Perfil → APARÊNCIA) e é lembrada em `localStorage`, que a regra 4 proíbe para *token*, não para preferência de tela.

14.8 Versão: atualizar é ato deliberado

> **ATENÇÃO**
>
> `^0.1.11` **não** pega `0.2.0`: *caret* em `0.x` só sobe *patch*. "Última versão" é, portanto, ativo:

```
npm view @unooria/design-system version     # qual e a ultima publicada
pnpm up -r @unooria/design-system --latest  # sobe em ui/hub/exps de uma vez
# commitando o lockfile E os package.json juntos
```

14.9 O painel de recursos: três estados, e o rótulo que muda

Duas regras de interface nasceram com o painel da Unooria (**ADR-87**), e as duas são de SIGNIFICADO, não de estilo:

Interruptor de TRÊS estados, nunca dois. `ligado | desligado | herda`. Sem o terceiro, "ligado nesta escola" fica idêntico a "herdado ligado", e desfazer uma exceção criada por engano é impossível — um controle de dois estados não tem como expressar "apague a minha linha e volte a seguir o escopo de cima". Por isso o controle é um `Selection` de três opções e não um `Toggle`.

O rótulo de *herda* depende do escopo. No contrato não existe "de cima": ali é "Padrão (ligado)". Numa escola é "Herda de cima". Chamar as duas coisas de "Herda" faria o operador do escopo-contrato procurar um nível que não existe.

E uma terceira, que vale para todo controle desta plataforma: **controle sem efeito fica DESABILITADO com o motivo ao lado, nunca escondido.** A experiência sob um produto desligado mostra o interruptor inerte e a frase "o produto está desligado neste escopo"; ativo, ele prometeria uma mudança que o resolvedor ignora, e escondido pareceria defeito.

14.10 Lacuna aberta: o `SideBar` traz um item que não é seu

O `SideBar` da seção `unooria` renderiza SEMPRE, como primeiro item, um botão "Iniciativas" fixo no componente, que alterna um painel de `appsItems`. **Nenhuma área deste hub passa `appsItems`**, então o clique não faz nada — e isso vale para `/admin` desde que a barra existe, e agora para `/unooria`.

Item de menu que não leva a nada é pior que item nenhum: ele consome a primeira posição da barra, que é onde a mão vai. É **lacuna do pacote**, com pedido no `storybook-unooria` — o que falta é o item ser condicional a haver `appsItems`. Não há compensação local aqui de propósito: esconder o botão por CSS seria reimplementar decisão de componente do lado de fora, e a regra de ouro é o contrário disso.

14.11 Compensações locais

> **REGRA INVIOLÁVEL**
>
> **Componente que falta no pacote não nasce na experiência.** Vira *compensação local* marcada no CSS (comentário `(b) COMPENSAÇÃO`) e um pedido ao `storybook-unooria`. A compensação é **exceção controlada e decrescente**: cada bloco compensa uma lacuna real, anotada no comentário, e sai quando o pacote publicar a correção.

14.11.1 Lacunas abertas, verificadas na 0.1.14

> **NOTA**
>
> As quatro primeiras da tabela abaixo foram **atendidas na 0.1.17** e as compensações correspondentes foram apagadas do UnooTube (hoje `backup-exp04ef1b`). Duas lacunas novas apareceram na 1.1.0 da experiência e estão registradas logo depois.

Lacuna	Consequência
`unitube` não publica *toast*, diálogo, tabela nem estado vazio	Quatro compensações locais no UnooTube.
`ChannelHeader` não tem `showSubscribe`	O botão de inscrever-se é *sempre* desenhado, e o canal do UnooTube é o do **próprio estudante**.

`.uni-channel-header__cover` usa a primitiva `--uni-color-neutral-50`, não um token semântico	No escuro vira uma laje branca.
`styles-unootube.css` repete o `@import` da Roboto no **meio** do arquivo	O Vite descarta com aviso; o do topo continua valendo, então a fonte carrega. É ruído, não bug.
Não existe componente de seleção (só `TextField`, `TextArea`, `PasswordInput`, `SearchInput`)	`<select>` do hub usa `.unooria-select`, com os tokens do pacote — mesma altura, raio e cores do `TextField`.

14.11.2 Lacunas abertas na 0.1.17 (UnooTube 1.1.0)

Lacuna	Consequência
`unootube` não publica `Tag` nem `Badge`	Nenhum dos dois arquivos de origem desenha um. A etiqueta “LIMITE ATINGIDO” do diálogo de limite é compensação local — e sai **neutra**, não âmbar como o Figma pede: a seção não tem token de aviso, e derivar um seria inventar valor.
O véu do `ShortsPlayer` tem altura **fixa** de 194 px	Ele existe para a legenda ficar legível. Numa miniatura de 88 px sem legenda (`showChannel false`), o véu é mais alto que a miniatura inteira e só a faixa mais escura do gradiente aparece: o sintoma é **retângulo preto no lugar do vídeo**, com o vídeo carregado por baixo. Compensado com `display: none` nas miniaturas da bandeja e do diálogo. Deveria ser proporcional, ou preso ao `showChannel`.
`Button` do clone não tem variante de ênfase	“Entendi!” e “Salvar” recebem preenchimento por ligação local, com os tokens da própria marca.

14.11.3 Lacunas abertas em `unooria` na 0.1.17 (hub, 2026-09-05)

As duas lacunas abaixo apareceram nas telas de turma do professor, onde a listagem tem uma linha por estudante — 40 numa sala cheia. As duas são compensadas em `packages/ui/styles/plataforma.` e viraram pedido no `storybook-unooria`.

Lacuna	Consequência

`Table` não tem cabeçalho fixo	`TableProps` não publica `stickyHeader` nem `maxHeight`, e o `.unooria-table-container` só rola na horizontal. Numa turma de 40 estudantes o cabeçalho sai da tela junto com a página, e o professor lê "sim · — · 3" sem saber de qual coluna. Compensado pela classe `.tabela-rolavel`, que dá altura máxima ao contêiner (tela útil menos o cromo, piso de 20 rem) e prende o `<th>` com `position: sticky`. Vale para as duas famílias de tabela do hub: a `<table>` crua e a do DS (`className="tabela-rolavel"`).
`Button` não tem variante de perigo discreta	`ButtonVariant` é `primary | secondary | outline | ghost | danger`: as três primeiras saem no navy da marca, que não diz "isto apaga", e a `danger` é sólida — peso de ação primária, errado para "Zerar a turma" ao lado da leitura da aula. Compensado por `.acao-destrutiva`, que troca o navy do `outline` pelo token de perigo e mantém o fundo transparente.

ATENÇÃO

Variante que não existe não é erro de compilação — é botão sem cor. O `Button` monta a classe por interpolação (`unooria-button--variant-${variant}`), então um `variant="tertiary"` produz uma classe que nenhuma folha define: o botão perde fundo, cor e borda e fica com os estilos de base. Era o que acontecia com "Zerar a turma" do UnooGPT e do Eco no Grupo (retirado, **ADR-113**), e com o "confirmar" do diálogo (`variant="destructive"`). O *typecheck* não pega. Ao escrever um `variant`, confira contra `ButtonVariant`.

14.11.4 Lacuna aberta em `Collapse` na 0.1.19 (app-92, 2026-09-10)

O `Collapse` esconde o corpo fechado com `grid-template-rows: 0fr`, `opacity: 0` e `aria-hidden="true"`, e o `__body-inner` com `overflow: hidden`. **Nenhuma dessas propriedades remove do FOCO** — nem `inert`, nem `visibility`, nem `display: none`. A árvore de acessibilidade está correta (o `aria-hidden` é do pacote), e é justamente isso que torna o defeito pior: **descendente focável dentro de `aria-hidden="true"` é foco fantasma** — o `Tab` entra no controle e o leitor de tela não anuncia nada, porque para ele aquele elemento não existe.

Medido na tela de aulas da app-92, que abre o ano em quatro bimestres e só deixa o primeiro aberto (`defaultOpen`): os três fechados deixavam **30 botões de semana** recebendo foco do teclado. Um professor que navega por `Tab` atravessava trinta controles invisíveis antes do próximo elemento visível.

Compensação, em `apps/experiencias/app-92-plano-de-ensino/src/estilos/globals.css`: `visibility: hidden` no corpo com `aria-hidden="true"` — é a única propriedade CSS que tira do foco sem tirar do fluxo — com `transition` de `visibility` atrasada pela duração do collapse, para que o fechamento continue animado. Ela não muda aparência nenhuma. **Morre quando o pacote publicar `inert` no corpo fechado**, e o pedido está aberto no `storybook-unooria`.

> **REGRA INVIOLÁVEL**
>
> **A compensação é guardada por teste.** A suíte `professor/09-plano-de-ensino` conta quantos controles de bimestre fechado recebem foco e reprova se voltarem a ser mais de zero. Sem essa rede, remover a compensação passa no *typecheck*, passa no *build* e o defeito só reaparece para quem navega por teclado — que é exatamente quem não vai abrir uma *issue*.

O segundo custo do corpo fechado no DOM é de automação: um `locator(...).filter(...)` acha conteúdo de bimestre fechado, o Playwright o considera visível e o clique morre interceptado pelo contêiner colapsado. Foi o que quebrou aquela suíte no dia em que as dez semanas do primeiro bimestre ficaram todas planejadas. Quem automatiza a tela abre os bimestres antes de procurar.

14.11.5 Lacunas abertas em `whatsappmobile` na 0.1.17

A seção tem 29 componentes e cobre o cromo inteiro do mensageiro — barra de status, cabeçalho de conversa, balão com mídia e documento, campo de escrever, diálogo, folha inferior. As quatro peças abaixo foram levantadas pelo Eco no Grupo (exp07ef1b 0.1.0, retirado pela ADR-113); **três saem de composição**, e só uma é compensação de verdade (ADR-68). O selo de encaminhada e o balão de voz continuam em uso na exp05ef1b.

Lacuna	Como está resolvido
`ForwardedBadge` — o selo “Encaminhada com frequência”	Composição: `Icon name="forward"` (o pacote tem o glifo) com os tokens de texto secundário. Nenhuma cor inventada.
`PinnedMessageBanner` — a barra do alfinete	Composição sobre a superfície de *appbar*. O pacote tem o *tone* `pin`, com a cor certa, mas **nenhum glifo de alfinete** — ele é desenhado localmente, junto de `play` e `pause`, que também faltam.
`ReplyQuoteSelectionSheet` — o menu das 3 respostas	Composição com `Radio` e um card de citação. **Não use `ListItem`**: o título dele tem `white-space: nowrap` — certo numa lista de conversas, e errado aqui, porque truncar as opções faria a criança escolher sem ler.
`ChatAudioBubbleWithTra` — o balão de voz	**A única compensação real.** `MessageBubble` aceita texto, `.Media` (imagem) e `.Document`; não há player de áudio na seção. O balão local traz play, onda em *divs* (sem Canvas, por causa do *framerate* em Chromebook escolar) e a transcrição sempre visível.

> **ATENÇÃO**
>
> **A seção publica só o tema escuro** (`WaTheme = 'dark'`; o CSS traz `:root, [data-wa-theme=dark]` e nada mais). Os wireframes das missões de mensageiro descrevem telas claras; derivar a paleta é proibido, pela mesma regra do `unitube`. O contraste do escuro (`#D3DAE0` sobre `#101D25`, cerca de 11:1) passa o AA com folga — ver ADR-68.

> E o `PhoneFrame` do pacote **não entra** na experiência: quem desenha o aparelho é o hub, pelo `viewport` do manifesto. Empilhar os dois daria um celular dentro de outro.

14.11.6 As extensões de `whatsappmobile` na 0.2.4 (exp05ef1b 0.2.0)

A v2.2 de Quem mandou isso? pediu ao DS as peças de que o grupo da turma precisava, e a 0.2.4 publicou **dez extensões** em `whatsappmobile` — peças sem frame no arquivo de origem, derivadas dos tokens e das formas que ele já tinha. A exp05ef1b é o primeiro consumidor; a seção foi de 29 para 39 componentes, todos exportados pela ponte `@edu/ui/ds-whatsapp`, e o repositório está em `^0.2.7`: a 0.2.5 trouxe as extensões do `instanooria` por cima sem mudar a API desta, a 0.2.6 fechou as duas lacunas da tabela abaixo, e a 0.2.7 trouxe a segunda leva do `instanooria` e o `AudioPlayer` da plataforma (Seção 14.11.7).

Peça	Onde a exp05ef1b a usa
`Button` (`filled` | `outline`)	"Ir para o Mural da Turma", "Confirmar envio", "Revisar seleção no grupo". Colide no NOME com o `Button` da plataforma: numa tela com as duas seções, o *alias* é obrigatório.
`ForwardButton`	A seta de encaminhar de cada balão, com o `tick` quando escolhida.
`BottomSheet`	"Encaminhar para…", com um destino só.
`MediaViewer`	O comunicado e a prévia em tela cheia, com a transcrição.
`ChatBanner`	A faixa "Encaminhe 3 mensagens…" com o contador N/3, tocável; e o apoio da Nina aos 90 s.
`AdminOnlyNotice`	A tarja "Apenas administradores podem enviar mensagens" no lugar do teclado.
`SystemMessage`	A entrada no grupo, a espera, o motivo declarado e a trava da síntese.
`Poll`	A enquete de motivo (Etapa 3) **e** as duas perguntas da Nina — o mesmo componente, controlado de ponta a ponta, dentro de um `MessageBubble`.
`MessageTag`	Os crachás da revelação, pictograma e rótulo, sem variante de cor: quem distingue é a palavra e a forma.
`Toast`	Os avisos rápidos (limite de 3, faltam mensagens, falha de envio). Sobe para 148 px da base, para não cobrir os botões do rodapé.

E o `MessageBubble` ganhou `sender` e `forwardedLabel` (o remetente no grupo e o selo "Encaminhada" no Mural), `.LinkPreview`, `.Invite` (o convite da Nina) e o `.Media` **sem** `src`: a mídia DESENHADA, com `children` e o texto no `alt`, que é como o comunicado, o termômetro e a lâmina da síntese existem sem arquivo nenhum. Três das compensações da exp07ef1b já têm peça no pacote (o selo "Encaminhada", a faixa fixa e a folha genérica), mas a exp07ef1b e a exp06ef2b **não foram migradas** — é outra frente.

Lacuna medida na 0.2.4	Compensação na exp05ef1b (até a 0.2.0)
`ForwardButton` esmaece só por `:disabled`	E `<button disabled>` ENGOLE o toque — mas a *copy* pede que a quarta seta FALE (“Limite de 3 mensagens atingido…”). A seta fica acionável com `aria-disabled`, e a opacidade é `.encaminhar--esgotado`. A suíte de ponta a ponta clica essa seta com `force`, porque o Playwright recusa `aria-disabled`.
`MessageBubble.Media` não publica `onClick`	O toque que abre o comunicado é um `<button>` sem aparência em volta (`.toque-na-midia`); o nome acessível é o do `alt`. A `.LinkPreview` publica `onClick`.

> **NOTA**
>
> As duas viraram pedido ao `storybook-unooria` e **fecharam na 0.2.6**: o `ForwardButton` ganhou `exhausted` (esmaecido, com `aria-disabled`, e acionável) e o `.Media` passou a aceitar `onClick` e `actionLabel`, virando `<button>` com o mesmo desenho. As compensações saíram da exp05ef1b na 0.2.1. O clique forçado da suíte de ponta a ponta continua: quem recusa `aria-disabled` é o Playwright, não o DS. **Não são pedido** o desenho do comunicado, da lâmina e do termômetro: são CONTEÚDO dentro da moldura que o DS já dá, não controle.

14.11.7 A segunda leva do `instanooria` e o `AudioPlayer` na 0.2.7 (exp07ef1b 1.0.1)

A A Seleção saiu na 1.0.0 com quatro compensações locais e três pedidos ao DS sem compensação. A 0.2.7 fechou os sete, e a 1.0.1 da missão não tem mais nenhum bloco “(b) COMPENSAÇÃO” no CSS dela. A mesma versão trouxe o primeiro player de áudio da plataforma, para o podcast de recapitulação (**ADR-119**).

Peça	O que resolve
`InlineHint`	A faixa sob a barra, em tom neutro (orientação) ou de atrito (a cor de alerta na borda, o texto sempre primário), com `announce` para o aviso que nasce de um toque. É a dica do feed e o aviso da poda.
`GestureHint`	A marca de toque que demonstra rolar, deslizar ou tocar uma vez; decorativa, e desligada com movimento reduzido.
`Snackbar` (clone)	O aviso passageiro em cores invertidas, fora do fluxo e sempre montado como região de status. Colide com o da plataforma e sai pela ponte como `SnackbarInsta`.
`Badge wrap`	A etiqueta longa quebra linha; o raio desce para o dos cartões.
`StoryComposer`	Contador, mínimo e máximo em *code points*, sem os atributos nativos que medem UTF-16 — o emoji valia dois.
`PostCard.Media`	`errorDescription`: com a imagem quebrada, o `alt` continua na tela sob a mensagem.

`TextField` e `TextArea`	O `label` passou a nomear o campo (`htmlFor` e `id`); antes aparecia e não dava nome nenhum.
`AudioPlayer`	Ouvir e pausar com rótulo visível, a barra de posição nativa com o tempo dito por extenso ao leitor de tela, e o estado indisponível que mostra o motivo em vez de sumir. Design próprio da seção `unooria`; o glifo de tocar é o `play_arrow` da mesma biblioteca-base, que o quadro de ícones não publica.

14.11.8 As oito peças da redação na 0.2.8 (exp04ef1b 2.0.0)

O Jornal da Escola é a primeira missão cuja superfície é da seção `unooria`, e não de um clone — e a seção não tinha nada de jornal: nem quadro no Figma, nem peça. A 0.2.8 publicou **oito**, todas de design próprio, montadas sobre as medidas do `Card`, da `Tag` e do `Button` e reexportadas pela ponte `@edu/ui/ds`. A regra que as orienta é a mesma da missão: nenhuma peça sabe o gabarito, porque ele mora no servidor.

Peça	O que é
`NewsCard`	O cartão da mesa: faixa do meio, imagem 16:9 (ou o desenho do meio), manchete e resumo. O cartão inteiro é UM botão, com a manchete como nome; "na capa" vai no nome, e não em `aria-pressed`. O vocabulário dos oito meios (`NEWS_MEDIUMS`, rótulos e glifos) mora ao lado dele.
`NewsArticle`	A matéria no formato do meio de origem: o recorte de jornal, o boletim de rádio TRANSCRITO (a atividade não depende de som), o quadro de TV com a tarja, o blog, o portal e o aviso do mural. O jornal se reconhece pelo recorte e pelas colunas, e não por uma serifa inventada.
`RevealClue`	A pista que se revela: um botão de 64 px com o painel logo abaixo, no padrão *disclosure* (`aria-expanded`); fechado, o painel leva `hidden`.
`BottomSheet`	A gaveta: a folha que sobe de baixo, com o comportamento inteiro do `Modal` (Esc, trava de rolagem, prisão do Tab, devolução do foco). Sem gesto de arrastar. Não é o `BottomSheet` do `instanooria`, que tem o mesmo nome e mora na outra ponte.
`NewspaperPage`	A primeira página: cabeçalho e EXATAMENTE três vagas, nos três `NEWSPAPER_LAYOUTS` (`classic`, `hero-image`, `columns`) — a lista que a api valida com `z.enum`. O layout é por CONTÊINER, e a mesma página serve no aparelho, na tela do professor e na miniatura do comparativo. O marcador de cada vaga é o que a Chave põe na revisão.
`SpeechBubble`	O balão do Bento (`guide`) e dos leitores (`reader`), com humor por cor E ícone com nome — nunca só cor. É uma `<figure>` com `<blockquote>`, e a figura se nomeia pelo nome de quem fala.
`PuzzleBlock`	A peça da Chave do Editor, tocada para encaixar (sem arraste). Não decide se é certa: desenha o `state` que recebe (`placed`, `returned`, `locked`); `locked` é `aria-disabled`, para seguir no Tab.

`PuzzleTrack`	A calha: as vagas numeradas, que se juntam num bloco só quando a Chave se forma. A região `aria-live` existe sempre, até vazia, para que a formação seja anunciada.

> **ATENÇÃO**
>
> **Quatro *props* que enganam na 0.2.8**, registradas no `AGENTS.md` da missão: o `AlertBox` nasce `role="alertdialog"` com `aria-modal`, e dentro da gaveta (que já é o modal) calaria o resto para o leitor de tela — ali ele vai com `role="status"`; as `Tabs` geram ids GLOBAIS (`tab-${value}`), então os valores de aba têm de ser únicos na tela; o `RadioGroup` com `value={undefined}` vira não controlado, e sem seleção recebe `""`; e o `BottomSheet` devolve o foco a `finalFocusRef`, que a mesa aponta para o `NewsCard` que a abriu, porque o Safari não foca botão no clique.

Lacuna aberta na 0.2.8	Compensação na exp04ef1b 2.0.0
Alvo de toque infantil no `Button`, no `Tab` e no `RadioButton`	O `Button` para em `lg` (52 px), as abas em 56 px e a opção do `RadioGroup` tem o tamanho do texto; a missão pede 64 px, que o `RevealClue`, o `PuzzleBlock` e o `Snackbar` já dão. A compensação é só altura e recuo, marcada `(b) COMPENSAÇÃO` em `estilos/globals.css`, e virou pedido ao `storybook-unooria`: morre quando o tamanho de toque infantil for publicado.

14.11.9 Lacunas abertas em `chatgpt` na 0.1.19 (exp08ef4b 0.1.0)

A seção tem 26 componentes e cobre o cromo do cliente *web* — barra lateral, campo de escrever com a doca, balão de fala, cartão de agente, diálogo, etiqueta. A exp08ef4b pediu **nove** peças que ela não publica, e a maior delas é estrutural para o não-negociável 4 da missão.

Lacuna	Como está resolvido
`CodeBlock` — o bloco de *markdown* com cabeçalho e ação de cópia	**A maior, e a única que mexe com o mecanismo da missão.** O não-negociável 4 exige o manifesto dentro do bloco nativo do assistente: é essa moldura que faz o documento parecer **emitido pelo sistema** em vez de escrito pela escola. Compensação: a moldura, o cabeçalho "markdown", a barra e a fonte monoespaçada, com os tokens do pacote — e o botão de copiar **é** o `Button` da seção.
`SelectableCard` — cartão de escolha exclusiva com corpo longo	O *grid* 2×2 do pico. `GPTCard` é para grade de agentes e `Tag` é etiqueta curta; nenhum tem estado de seleção com texto de artigo. O elemento continua sendo `<input type="radio">` **real** dentro de `<label>`: a exclusividade, a navegação por seta e o anúncio "2 de 4" do leitor de tela vêm do navegador, não de JavaScript.
`StreamingCursor`	O cursor do efeito de digitação. `ChatBubble` desenha a fala pronta; não há estado de *streaming* na seção. O caractere é um `::after`, e a animação respeita `prefers-reduced-motion`.

`Spinner`	O indicador de 16px do *delay* de 2000 ms. Ele **para** sob `prefers-reduced-motion` em vez de sumir: sem nenhum sinal, a tela pareceria travada por dois segundos.
`ContextBanner`	A faixa dos três princípios em disputa, fixa abaixo do cabeçalho. `Badge` é etiqueta *inline* e `Divider` é um fio.
`InlineNotice`	O aviso de bloqueio por cliques rápidos — 15 palavras, que não cabem num `Badge`.
`Toast`	A notificação de canto de Amara e a de cópia. O `Snackbar` existe, mas na seção `unooria`, e a fala é **dentro** da conversa.
`BlockingOverlay` (na `unooria`)	O congelamento para o debate. **Esta tela não fecha**, e é por isso que ela não é um `Modal`: o diálogo do pacote traz `closeOnEscape` e botão de fechar, que dariam à criança o jeito de sair do congelamento que ele existe para impedir. O bloqueio real é o atributo `inert` no conteúdo de baixo.
`StampBadge` (na `unooria`)	O selo de protocolo da certidão. `Badge` e `Tag` são etiquetas curtas; a *spec* chama esta peça de "certidão".

ATENÇÃO

Duas props do pacote diferem entre as seções, e as duas custaram erro de *typecheck* nesta experiência: o `AlertBox` da `unooria` usa `variant` (não `type`), e o `Modal` da `unooria` usa `isOpen` enquanto o da `chatgpt` usa `open`. O compilador pegou as duas — mas o `Button` da `unooria` **aceita prop inventada em silêncio**, então nem sempre pega.

NOTA

`inert`, e não `pointer-events: none`, no congelamento: o segundo pararia o clique e deixaria o *Tab* alcançar os botões atrás do véu — a mesma classe de defeito que o `Collapse` da app-92 produziu, com 30 alvos focáveis dentro de um `aria-hidden`.

14.11.10 Avisos que ficaram obsoletos

NOTA

Dois avisos herdados do repositório de origem estão **obsoletos na 0.1.11**, medidos no pacote:

- o `MetricCard` **não inventa mais tendência** — a prop é `tagText?` e a documentação diz "sem ela a etiqueta não é renderizada";
- o `PositiveNegativeBarChart` tem escala **fixa** em ± 100, que é exatamente o que os eixos do circumplexo pedem — então a barra divergente *não* é mais escrita à mão.

O terceiro segue valendo: `AlertBox` é **diálogo** de 436 px; para faixa de contexto, o componente é `FeedbackAlert` (e a prop é `type`, não `variant`).

ATENÇÃO

O `MetricCard` recebe número ou rótulo curto — nunca frase. O valor é `white-space: nowrap` com `font-size: clamp(26px, 4vw, 36px)`, dentro de um cartão de `height: 128px` e `max-width: 301px` — cerca de 269 px úteis. Valor maior **vaza para fora do cartão**, por cima do que está ao lado, e nada acusa: não é defeito de CSS, é o componente usado para o que ele não é.

Quem mostrou isso foi o cartão "Ao confirmar" do relatório individual do Instanooria, que derramava "estourou — a chuva chegou" sobre a página. Os outros 40 `MetricCard` do hub são número ou `n/m`. A saída não foi compensar no CSS, e sim separar as duas leituras em `lib/unoogpt-leitura.ts`: `envioNoCartao` (`1:40`, "esgotado", "sem relógio") para o cartão e `envioEmTexto` ("tempo esgotado — a chuva chegou") para o texto corrido. O porquê de o tempo ter acabado não cabe num KPI e não precisa caber: ele já está por extenso na linha do tempo logo abaixo.

O recorte é por pixel, não por número de caracteres, e medir é barato: "tempo esgotado" parecia caber e mede 291 px contra 267 px úteis. A medida sai de um `.unooria-metric-card` montado no DOM de qualquer rota da plataforma — o CSS do DS é global no *bundle* do hub — lendo o `getBoundingClientRect().width` do valor depois de `document.fonts.ready`.

ATENÇÃO

Sobrescrever classe do pacote exige DOIS seletores (`.app-shell > .x`): o CSS do pacote é injetado pelo módulo do componente, ou seja, *depois* do CSS da experiência no grafo de importação. Com peso igual, quem ganha é sempre o pacote.

14.12 O chat do agente de IA do professor: as peças da 0.2.10

O agente de IA do professor (**ADR-126**) é a primeira superfície de conversa da seção `unooria` — os chats que já existiam eram de clone (`chatgpt`, `unoochat`, `whatsappmobile`) e imitam outra plataforma, o que aqui seria a fronteira errada: quem fala é a Unooria. A 0.2.10 (PR #9 do `storybook-unooria`) publicou as peças na seção `unooria`, e o hub as consome pela ponte, como todo o resto. Em 2026-09-30 a versão publicada no npm ainda era a 0.2.9.

Peça	O que é
`AgentAvatar`	O rosto do agente: o mascote, **embutido no pacote** (não há `src`), nos tamanhos do `Avatar`. Sem iniciais, status nem selo — o agente não é uma pessoa. `decorative` o tira da árvore acessível quando o nome já está escrito ao lado.
`ChatLauncher`	O botão flutuante que abre o painel, com `aria-controls` apontando para ele.

`ChatPanel`	O painel **NÃO modal**: sem véu e sem prisão de foco, a página continua usável ao lado. Dois tamanhos, `compacto` (até 400 × 640 px, acima do botão) e `expandido` (até 960 px ou 90% da largura, 85% da altura); abaixo de 600 px, os dois viram quase tela cheia. Fechado, continua montado com `hidden`: o rascunho e a rolagem sobrevivem. O corpo rola dentro do `ScrollBar`; Esc fecha.
`ChatPanelAction`	Os botões do cabeçalho (expandir, histórico, fechar).
`ChatMessage`	Uma fala, do professor ou do agente, com lugar para as fontes.
`ChatComposer`	O campo de escrever e enviar.
`TypingIndicator`	O "escrevendo" enquanto o primeiro `delta` não chega.
`RichContent`, `RichContentTable`	A tipografia do texto do agente (títulos, listas, tabelas de habilidades), aplicada ao HTML que o `react-markdown` produz no navegador (**ADR-129**).
`ChatHistory`	A coluna das conversas guardadas, com o título de cada uma e a confirmação de apagar (que trata o próprio Esc, para o painel não fechar junto).

NOTA

O painel leva `data-clarity-mask` (Capítulo 7): o professor pode escrever o nome de um estudante, e o Clarity grava a tela. A máscara é do hub, não do DS — o componente não sabe que existe Clarity.

14.13 O menu da conta: o componente que faltava, e a lacuna que ele trouxe

O `HeaderMenu` chegou na **0.1.19** para tirar o "Sair" da barra e pô-lo dentro do avatar (**ADR-78**). Ele é o caso de manual do procedimento de três passos: a busca no pacote foi feita ANTES de escrever qualquer coisa e mostrou que o DS 0.1.17 não tinha nada equivalente — sem `Dropdown`, `Menu` ou `Popover` na seção, o `Avatar` com `onClick` mas sem prop de menu, e o único menu suspenso do pacote (`TableDropdownMenu`) com gatilho fixo em três pontos e sem aceitar *children*. Não achou ⇒ pedido no `storybook-unooria`, e o componente nasceu LÁ.

ATENÇÃO

A tira do `HeaderMenu` COBRE o que está à esquerda do gatilho, em vez de empurrar. Ela é um *overlay* ancorado à esquerda do avatar, e o componente assume que ali há espaço vazio — verdade num *header* que termina no avatar, FALSA no da plataforma, onde ali ficam o nome e a linha de apoio. Integrado ingenuamente, o item "Sair" abre desenhado EM CIMA das letras. Passa no *typecheck*, passa no *build*, e só aparece abrindo o menu na tela.

A compensação vive em `apps/hub/components/menu-da-conta.tsx`, que junta identidade e menu na mesma peça e esconde o bloco enquanto a tira está aberta — com

> `invisible`, não desmontado, senão o gatilho escorrega embaixo do cursor. Ela **não** tem entrada em `plataforma.css` de propósito: é comportamento de página, e uma classe faria o próximo a integrar o componente achar que era decoração.

14.13.1 Duas lacunas que ficaram sem compensação, por escolha

Compensar sairia pior que esperar o pacote, nos dois casos:

- **`Selection` não aceita rótulo acessível sem rótulo visível.** O `...rest` dele aterra no `<div>` de fora, e o `aria-labelledby` do `combobox` só existe se a prop `label` for passada — e ela renderiza uma linha acima do controle, que não cabe num *header* de 64px. É o que impede o `<select>` cru do seletor de escola de migrar: trocar um controle que tem `aria-label` por um sem nome acessível é regressão, não modernização. O resto da API casa 1:1 (`options`, `value`, `onChange`, `size="sm"` = 40px, `error`), então a migração é de minutos no dia em que a prop existir. O `Breadcrumb` e o `Avatar` já publicam `ariaLabel`: é inconsistência do pacote, não desenho.
- **No `Breadcrumb`, `isCurrent` e `as` se contradizem.** Com `as`, o DS renderiza o componente polimórfico mesmo no item marcado como atual, e a página vira link para si mesma. A ponte da trilha compensa omitindo `href` e `as` no último item.

> **NOTA**
>
> **O aviso do `<select>` sem componente no DS estava OBSOLETO e ninguém tinha percebido.** O comentário de `plataforma.css` afirmava que o pacote não publica componente de seleção. Foi verdade e deixou de ser: o `Selection` existe, a ponte o exporta, e a área `/admin` já havia migrado. O comentário sobreviveu à lacuna e virou a desculpa que manteve **24** `<select>` crus no hub depois de o pacote já resolver o problema — a mesma armadilha do `expires_at` que este documento registra em cinco ADRs: a regra sai do código, o registro fica, e quem lê acredita nele. Ele agora nomeia os cinco arquivos que faltam migrar e diz que o último apaga o bloco.

14.14 A barra de aplicação ancora na janela (ADR-83)

O `HeaderPlataforma` é o único *chrome* do hub: mesma altura, mesmos elementos, da tela do estudante à área do contrato. Ele nasceu, porém, com um parâmetro `largura` que casava o miolo com o `max-width` do `<main>` de cada área — e como o miolo é centralizado, a coluna da área passou a decidir a POSIÇÃO da marca. Medido, a marca ocupava quatro X diferentes: 20px em `/admin` e `/exp`, 164px em `/analise`, 228px em `/professor` e `/estudante` num viewport de 1440px — 448px de salto em 1920px.

ATENÇÃO

Barra de aplicação não se alinha ao conteúdo: ela se alinha à janela. O miolo do header é *full-bleed* (`px-6`, sem `max-width` e sem `mx-auto`), e o mesmo `px-6` é o gutter da coluna de cada área. Escrever os dois valores separados é o que produziu, em `/analise`, uma marca 4px à esquerda do título da própria página — e quatro pixels ninguém nomeia olhando.

Isto **não** é compensação de lacuna do DS: é decisão de layout do hub, e por isso mora no código do header (a constante `GUTTER`) e não em `plataforma.css`. Vira lacuna no dia em que o pacote publicar um componente de barra de aplicação — é lá que o gutter deveria morar, e é o gatilho de reavaliação da **ADR-83**.

A superfície PÚBLICA (`/`, `/privacidade`, `/termos`) ficou fora por decisão: `.landing-nav__miolo` mantém 1440px centralizado e `Logo size="md"`, porque é outra superfície com outro público.

14.15 Cor da plataforma: classe escrita à mão

ATENÇÃO

A cor das superfícies da plataforma vem de classes escritas à mão em `packages/ui/styles/plataforma.css`, e o tema do Tailwind está **vazio** (`theme.extend: {}`) — de propósito, para que nada da marca institucional vaze para a superfície do estudante, nem por nome de classe.
A consequência é uma armadilha real: **classe de cor que não existe ali não é erro em lugar nenhum**. Em texto, ela herda a cor e passa desapercebida; em **fundo**, o elemento fica branco no branco. Foi o que aconteceu com `bg-unooria-neutral-900`: a letra da bolha ficou invisível. O *smoke* da app-70 checa o `backgroundColor` **computado** do círculo por isso — `toBeVisible()` não pega.
Existem hoje apenas: `bg-unooria-{navy-800, neutral-50, neutral-100, neutral-200}`, `border-unooria-neutral-200` e `text-unooria-{danger, navy-900, neutral-500, neutral-700, neutral-800, neutral-950}`.

NOTA

O *preflight* do Tailwind fica **ligado**. Desligá-lo parecia inofensivo — o pacote traz o próprio CSS — mas o pacote estiliza **componentes**, não elementos: sem o *reset*, todo `<a>` volta a ser sublinhado e todo `<ul>` volta a ter marcador. Foi o que o *smoke* visual pegou nos *cards* de produto.

14.16 Rolagem: o `ScrollBar` do DS, em toda área rolável do hub

Nenhuma tela do hub declara `overflow` para rolar. Área que rola é um `<ScrollBar>` do `@edu/ui/ds`: as 24 tabelas de `/admin`, `/analise` e `/professor`, o sumário de `/privacidade` e `/termos`, e as 5 tabelas largas do guia da aula. Dele vêm o `overflow`, a barra, o `autoHide` e o *fade* (**ADR-75**).

Até 2026-09-07 nada disso usava o componente: as tabelas rolavam em `<div>` com `overflow` escrito à mão, e o `ScrollBar` — publicado pelo pacote desde a 0.1.x, com `orientation`, `thickness`, `theme` e `autoHide` — não era referenciado em lugar nenhum do repositório. O componente existia e ninguém o procurou; é o caso que motivou o procedimento de três passos da regra de ouro (procurar, usar, e só então pedir ao `storybook-unooria`).

Tabela nova nasce assim:

Listing 14.1 – A forma canônica de uma tabela que rola

```
<ScrollBar orientation="both" autoHide classNam="tabela-rolavel">
  <Table … />        {/* ou a <table> crua */}
</ScrollBar>
```

> **ATENÇÃO**
>
> **`orientation="both"` não é enfeite.** O *default* é `vertical`, que traz `overflow-x: hidden` — e cortaria a coluna da direita das tabelas largas, que são a maioria aqui. No guia, onde a rolagem é só lateral, vale `orientation="horizontal"`.

14.16.1 O que sobrou para a compensação local

A `.tabela-rolavel` não desapareceu: ela **encolheu** para o que o pacote ainda não publica — a `max-height` que provoca a rolagem e o `<th>` preso (a lacuna de `stickyHeader`, já registrada). O `overflow` saiu dela; devolvê-lo criaria dois contêineres de rolagem, um dentro do outro.

Duas armadilhas medidas ao fazer a migração:

- **o cabeçalho preso para de grudar dentro do `ScrollBar`.** A `Table` do DS traz o próprio `.unooria-table-container` com `overflow-x: auto`, e é NELE que o `<th>` *sticky* se prende — um contêiner que não rola na vertical. A regra `.tabela-rolavel .unooria-table-container { overflow: visible }` devolve o papel ao `ScrollBar`, que cuida dos dois eixos;
- **no papel, o `ScrollBar` recorta.** O guia é impresso e levado para a sala; `components/guia/guia.css` devolve `overflow: visible` e `max-height: none` em `@media print`, senão a tabela sai cortada na largura da coluna.

14.16.2 O `autoHide` do pacote é *hover*, não rolagem

O `ScrollBar.types.ts` documenta "oculta a barra quando ocioso e a exibe suavemente com *fade-in* durante o *hover* ou rolagem", mas o CSS publicado só tem regras `:hover`. Na prática: a barra aparece ao passar o mouse por cima da área e continua invisível quando a rolagem vem da roda do mouse ou do teclado. É menos do que se queria, e é o preço aceito de manter a fonte no pacote — com pedido aberto no `storybook-unooria` e a prop já no lugar para o dia em que ele publicar o gatilho por rolagem.

Fora do hub nada mudou. As nove experiências seguem com a barra do sistema operacional: o `ScrollBar` é da seção `unooria`, e a regra das duas famílias proíbe que ele entre num clone.

14.17 O vocabulário das telas administrativas

O *design system* resolve o componente; ele não resolve a PALAVRA. E a palavra divergiu: o mesmo perfil se chamava "gestor" num diálogo e "Coordenação" no filtro ao lado; a mesma coisa era "unidade" numa tela e "escola" na outra; "regional" e "diretoria" alternavam na mesma frase. Nada disso quebra teste, e tudo isso faz quem chega concluir que são coisas diferentes.

A regra vale para TEXTO DE TELA — rótulo, título, descrição, *placeholder*, mensagem — e não para nome de campo de API:

Use	Nunca
escola	"unidade"
regional	"diretoria" (nem alternando com ela)
vínculo, **vincular**	"lotação", "lotar" — `lotacoes` segue sendo o nome da TABELA, porque é o termo que a rede pública usa para o registro
Coordenação (da escola, regional)	"Gestor" na interface. O identificador continua `gestor_escola`: é contrato replicado em quatro fontes
Código interno nos FORMULÁRIOS	"Id da escola". Na LISTAGEM o id aparece sem rótulo, de propósito: é a chave que a secretaria usa nas planilhas de importação, e um "Código interno:" por linha dobraria a altura da célula

> **ATENÇÃO**
>
> **Pessoa e vínculo não são a mesma contagem, e a tela tem de dizer qual é qual.** A coluna "Professores" conta PESSOAS naquela escola (índice único por escola × pessoa × papel); o total do resumo conta VÍNCULOS na rede, porque quem dá aula em duas escolas entra duas vezes. Por isso o resumo diz "vínculos de professores" e leva a frase "Um professor pode ter vínculo com mais de uma escola". Essa confusão já produziu um *bug* de divergência entre indicador e listagem (apêndice F) e duas conversas de esclarecimento.

Uma divergência fica REGISTRADA em vez de corrigida: o campo `unidades` da resposta de `GET /escolas` conta as escolas de uma regional, e o produto chama isso de "escolas". Ele não foi renomeado porque é contrato publicado (OpenAPI e apêndice C), e trocá-lo por pureza de vocabulário é *churn* sem ganho para quem usa a tela.

14.18 O idioma da interface: três zonas (ADR-91)

Todo texto que uma pessoa lê é pt-BR — rótulo, botão, título, `aria-label`, *placeholder*, mensagem de erro, guia do professor, página legal. Mas a regra não é única, porque a plataforma tem três superfícies com exigências opostas:

Zona	Regra
Plataforma (hub, experiências do professor)	pt-BR sem exceção. É a Unooria falando com professor, gestor e estudante, e numa rede pública não se pode pressupor repertório de inglês.
Superfície disfarçada (dentro do clone)	O nome que o aplicativo REAL usa em português. `Shorts`, `Stories`, `Feed`, `Playlists` e `Ranking` ficam: traduzi-los quebra o disfarce que a experiência está medindo.
O professor falando SOBRE o clone (guia da aula, catálogo, painéis)	Depende de o termo nomear objeto na tela ou ser jargão vazado — abaixo.

A fronteira é onde se erra. O termo **nomeia um objeto que está na tela** (`feed`, `Shorts`, `ranking`)? Fica — traduzir faria o guia dizer uma palavra e a tela mostrar outra, e o professor perde o referente no meio da aula. É **jargão de implementação vazado** (`card de pausa`, `o modal`, `num popup`)? Traduz: não nomeia nada que a criança veja, e existe palavra portuguesa exata — `tela de pausa`, `quadro de sistematização`, `janela`.

Identificador não é texto e fica como está: chave de objeto, rota, coluna, verbo/objeto de telemetria, nome de glifo do Material Symbols (`checklist`, `psychology`, `schedule`), variante de componente (`warning`, `success`, `danger`), severidade de log, método HTTP.

> **REGRA INVIOLÁVEL**
>
> **Anglicismo que fica por decisão, não por esquecimento:** `fake news`, `cyberbullying`, `print`, `chat` e `on-line`. Os dois primeiros são o vocabulário da BNCC e da Lei 13.185/2015 que o próprio guia cita; `print` é a palavra que a criança usa, e a missão da exp06ef2b gira em torno desse objeto. A ADR-91 tem a razão de cada um. **Não "corrija" nenhum deles.**

Duas armadilhas que a troca de terminologia cobrou. A suíte e2e casa por texto EXATO (`getByText`, `getByRole({name})`): mudar copy sem atualizar a suíte a derruba com *timeout*, e o *timeout* reporta o seletor que faltou, nunca a causa. E rótulo visível com espelho na api muda nos DOIS

lados — `ROTULO_PASSO` da exp02em1b espelha `apps/api/src/unoogpt/regras/passos.ts`, e o *typecheck* não vê a divergência.

14.19 Exceção registrada

DECISÃO DE PROJETO

O `AgentAvatar` carrega um *bitmap* dentro do pacote, contra a regra de que imagem não entra no *design system* (vai para o *bucket* de *assets*). O mascote do agente é desenho, não ícone vetorial, e é identidade da marca como o `Logo`: embutido em WebP, ele aparece igual em toda superfície sem depender de URL, CDN ou variável de ambiente, e não há `src` para errar. A exceção é dele só: imagem de conteúdo continua indo para o *bucket*, e o peso do WebP entra no pacote de quem importar a peça.

DECISÃO DE PROJETO

O site público `/` está fora da regra do *design system* (**ADR-107**): ele é porte 1:1 de um material de marketing aprovado fora deste repositório, com folha (`components/site/site.css`), tipografia (Manrope) e movimento próprios. **A fronteira do DS é a porta do login**: dali para dentro, a regra de ouro continua valendo sem exceção. A landing anterior (**ADR-40**), essa sim feita de DS, continua viva em `/landing-anterior` com `noindex`, porque `/privacidade` e `/termos` ainda montam o fundo e a folha dela.

DECISÃO DE PROJETO

A app-70 está fora da regra do *design system*, deliberadamente: é jogo em *canvas* 3D, sem DOM de interface (**ADR-18**). A pendência do UnooTube (hoje `backup-exp04ef1b`) — que tinha CSS próprio em `src/estilos` — foi **resolvida**: ela consome a seção `unitube` pela ponte `@edu/ui/ds-unootube`, com a aparência escolhida pelo estudante.

15

Engenharia de entrega

15.1 O *monorepo* e o *build* incremental

> **DECISÃO DE PROJETO**
>
> ***Monorepo* único, com pnpm *workspaces* + Turborepo** (ADR-02). A SDK é consumida por 48 aplicações: uma mudança nela precisa testar tudo no *mesmo pull request*, ou a incompatibilidade aparece num *bundle* publicado semanas depois. Repositórios separados só se houver estúdio externo produzindo experiência.

Listing 15.1 – `turbo.json` — o *pipeline* inteiro

```
{
  "tasks": {
    "build":     { "dependsOn": ["^build"],
                   "outputs": ["dist/**", ".next/**", "!.next/cache/**"] },
    "dev":       { "cache": false, "persistent": true },
    "lint":      { },
    "typecheck": { "dependsOn": ["^build"] },
    "test":      { "dependsOn": ["^build"] }
  }
}
```

O `dependsOn: ["^build"]` é o que faz o grafo funcionar: `typecheck` de uma experiência exige o `build` da SDK, então o *filtro* por mudança arrasta as dependências corretas.

Comando	Uso
`pnpm turbo build --filter=@edu/exp08ef1b`	Builda uma experiência mais as dependências dela.
`pnpm turbo build --filter='...[HEAD^]'`	Só o que mudou. É o que o CI usa.
`pnpm turbo typecheck lint test`	O portão local da *Definition of Done*.

15.2 Integração contínua

Listing 15.2 – `.github/workflows/ci.yml` (abreviado)

```yaml
name: ci
on:
  # So os PRs para dev, hml e main. PR para dev-local nao roda: ali o
  # portao e a verificacao local da Definition of Done (ADR-121).
  pull_request:
    branches: [dev, hml, main]
  # Chamado pelo deploy.yml no push para dev/hml. NAO tem `push` proprio:
  # rodaria duas vezes no mesmo commit, e o deploy precisa ESPERAR por ele —
  # dois workflows em paralelo deixariam um teste vermelho subir para escola.
  workflow_call:
    inputs:
      completo:   # `forcar` no deploy builda tudo; o portao cobre o mesmo
        type: boolean

jobs:
  verify:
    runs-on: ubuntu-latest
    services:
      postgres:   # 9 das 23 suites da api sao de INTEGRACAO (RLS, locks)
        image: postgres:16
    steps:
      - uses: actions/checkout@v4
        with: { fetch-depth: 2 }
      - uses: pnpm/action-setup@v4
      - uses: actions/setup-node@v4
        with: { node-version: 24, cache: pnpm }
      - run: pnpm install --frozen-lockfile
      - uses: astral-sh/setup-uv@v6   # apps/agente-professor e Python (ADR-126)
        with: { version: "0.12.7", enable-cache: true }
      - run: pnpm --filter @edu/api migrate:deploy
      - run: |
          if [ "$COMPLETO" = "true" ]; then
            pnpm turbo typecheck lint test build
          else
            pnpm turbo typecheck lint test build --filter='...[HEAD^]'
          fi
      # COMENTADO no arquivo real desde 07d9ec0 -- ver a advertencia abaixo
      - name: Snyk (trava em high)   # high/critical em dependencia = vermelho
        env: { SNYK_TOKEN: "${{ secrets.SNYK_TOKEN }}" }
        run: |   # o --exclude e obrigatorio: o passo roda DEPOIS do build
          npm install -g snyk
          snyk test --all-projects --dev --severity-threshold=high \
            --exclude=.next,.next-prod,dist,build,storage,storage-privado,jogo
```

DECISÃO DE PROJETO

O CI é chamado pelo *deploy* por `workflow_call`, e **não** tem gatilho próprio em `push`. Com gatilho próprio, ele rodaria duas vezes no mesmo *commit* — e, pior, em *paralelo* com o *deploy*, o que deixaria um teste vermelho subir para uma escola. Assim o *deploy* **espera** pelo portão. O gatilho de `pull_request` filtra a *branch* de destino (**ADR-121**): roda nos PRs para `dev`, `hml` e `main`, e não no de *branch* de trabalho para `dev-local`, onde o portão

> é o `pnpm turbo typecheck lint test` da *Definition of Done*. A promoção `dev-local` → `dev` verifica tudo o que acumulou.

15.2.1 A trava de vulnerabilidade (Snyk)

> **ATENÇÃO**
>
> **O passo está COMENTADO no `ci.yml` desde o *commit* `07d9ec0`** (“*chore: disable Snyk vulnerability scan in CI workflow*”), situação de 2026-09-05. O parágrafo abaixo descreve o portão como ele foi desenhado e como volta a funcionar quando o passo for reativado — **hoje ele não trava nada**. O bloco de YAML acima mostra o passo na forma ativa, que é a forma comentada no arquivo. Ver a advertência da **ADR-39**, inclusive para o efeito colateral: o prazo do `apps/api/.snyk` era cobrado por este passo e agora vence em silêncio.

O portão termina com `snyk test --all-projects --dev --severity-threshold=high`: vulnerabilidade `high` ou `critical` em qualquer dependência do *workspace* deixa o CI vermelho — e, como o *deploy* espera pelo portão, nada sobe (**ADR-39**). O passo roda sempre no *workspace* inteiro, sem o filtro por mudança do resto do portão: o *lockfile* é um só, e a vulnerabilidade nova chega pelo banco de dados do Snyk, não por um *commit* — “dependência que não mudou” não existe como categoria.

O `--dev` não é detalhe: sem ele o Snyk pula as `devDependencies`, e foi por aí que um *critical* no `vitest` passou verde na primeira semana da trava (achado pelo `pnpm audit`, que olha tudo). Ferramenta de desenvolvimento não embarca no *bundle*, mas roda na máquina de quem desenvolve e no *runner* do CI — e é lá que um *dev server* furado vaza código.

> **NOTA**
>
> **O que exige *major* de *framework* não bloqueia — expira.** O `apps/api/.snyk` ignora, com **prazo e motivo**, as vulnerabilidades cujo *fix* é o Nest 11 (`@nestjs/core`, e o `file-type` que é ESM-only e quebraria o `require` do Nest 10) ou um *bump* que só o Prisma pode fazer (`deepmerge-ts`). Quando o prazo vence sem o *upgrade*, o próprio passo volta a falhar — é o prazo **com** a rotina que o cumpre, a resposta à armadilha do `expires_at`. O resto foi consertado na hora: o hub subiu para o **Next 16** (as *criticals* do 15 não tinham *patch* na linha 15.x), e `pnpm.overrides` no `package.json` da raiz cobre o transitivo (`next` 16.3.3 por baixo do *design system*, `multer` 2.2.0, `js-yaml` 4.3.1, `lodash` 4.18.1).

O segredo `SNYK_TOKEN` vem dos *secrets* do repositório (`snyk config get api | gh secret set SNYK_TOKEN`). Ausente, o passo **falha alto** — nunca pula, pela mesma regra do *guard* do Pub/Sub: env ausente é porta fechada, não porta aberta.

15.3 Entrega contínua

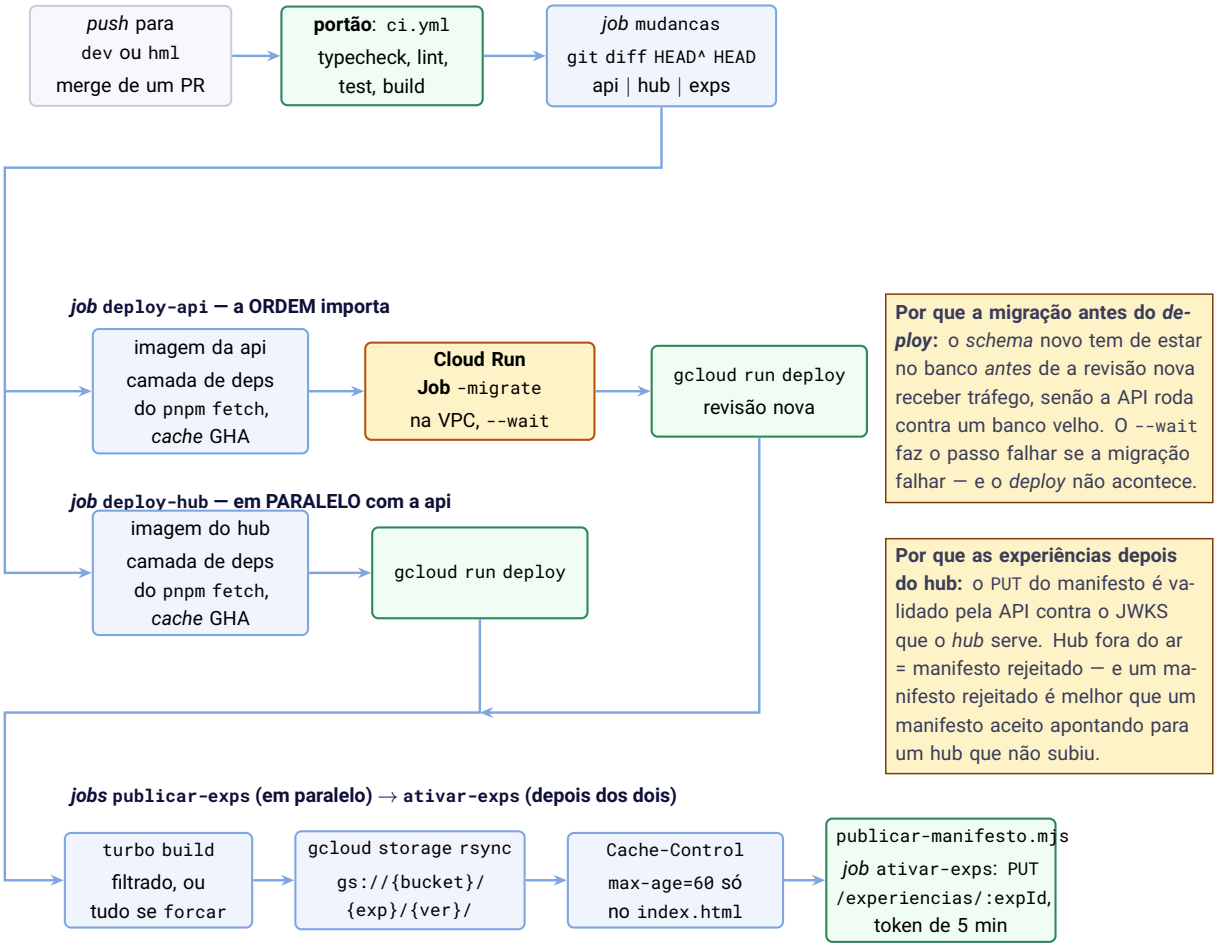


Figura 15.1 – O *pipeline* de entrega. O portão e o *job* `mudancas` decidem o que roda; api e hub sobem em *jobs* paralelos (cada um autentica por WIF, sem chave JSON); as experiências publicam por último, depois dos dois.

15.3.1 O agente de IA do professor: imagem, `PATCH` e a ordem dos *applies*

O agente (**ADR-126**) não é um Cloud Run, e a figura acima não o desenha: é um quarto *job*, `deploy-agente-professor`, em paralelo com os da api e do hub, disparado quando o *job* `mudancas` vê `apps/agente-professor` no *diff* (o pacote é Python e não lê nada de `packages/` nem do *lockfile* do pnpm). Ele constrói a imagem pelo `Dockerfile` do pacote (`uv sync --locked`), publica `{região}-docker.pkg.dev/…/agente-professor:{sha}` no mesmo Artifact Registry e, se o engine já existe, faz `PATCH` do engine com `updateMask=spec.containerSpec.imageUri` e espera a operação longa terminar — falha se o engine recusar a imagem, ou se a operação passar de 15 minutos. O resto da `spec` (variáveis, métodos, SA) é do Terraform, que ignora a `imageUri` (`ignore_changes`); sem isso, cada *apply* desfaria o *deploy*.

O engine só nasce depois da primeira imagem, e isso dá uma ordem que o CI não resolve sozinho:

1. **Uma vez por projeto**, os *service agents* do Discovery Engine e do Agent Engine (`gcloud beta services identity create`); e o `terraform init -upgrade` do *provider* `google` 7.46.
2. ***Apply* 1**, com `agente_professor_imagem = ""`: *bucket*, *data stores*, esquema das habilidades, SA, papéis e a exclusão do Cloud Armor. A api ganha `AGENTE_PROFESSOR_MODO=desligado`.
3. **Ingestão**, fora do Terraform: `importar esquema`, `exportar_habilidades`, `importar habilidades`, os PDFs no *bucket*, `dividir_pdfs`, `gerar_metadados_acervo`, `importar acervo` e o relatório `avaliar_habilidades`.
4. **O CI publica a primeira imagem.** Com `vars.AGENTE_PROFESSOR_ENGINE` vazia, o *job* só publica e deixa um aviso com a referência a pôr no `tfvars`.
5. ***Apply* 2**, com `agente_professor_imagem` preenchida: o engine (minutos; o *timeout* do *provider* é 20), o IAM nele e a api com `AGENTE_PROFESSOR_MODO=agent-engine` e o *resource name*. A `var AGENTE_PROFESSOR_ENGINE` do ambiente no GitHub recebe o mesmo *resource name*.
6. **Daí em diante, quem troca a imagem é o CI.**

> **ATENÇÃO**
>
> **Até o *apply* 2, o chat responde "agente indisponível"** — é o `desligado` fazendo o que deve, não uma falha. E o *runner* do CI precisa do `uv`: sem ele, o `pnpm turbo` da raiz **falha** com a instrução de instalação, para que um pacote que some da verificação não vire um verde falso (`AGENTE_PROFESSOR_PULAR_SEM_UV=1` pula com aviso).

15.3.2 Sonda, *smoke* e aviso (2026-09-02)

Três passos que o *pipeline* não tinha, todos da fase 0 do Capítulo 17 — a parte dela que é deste repositório:

- **Sonda pós-*deploy*** nos *jobs* da api e do hub. O `gcloud run deploy` volta quando a revisão *recebe tráfego*, não quando ela responde; o passo pergunta, pelo domínio público, até 12 vezes em 2 minutos: `/readyz` = `{"ok":true}` na api (processo + banco), e `/api/healthz` **e** `/.well-known/jwks.json` com `"keys"` no hub — sem a chave carregada nenhum *token* de experiência é validável, e o `ativar-exps` falharia com um 401 que não aponta para a causa. Vermelho = o `ativar-exps` não ativa nada.
- ***Smoke***: o *job* `smoke` roda a suíte da *landing* (`publico/01`) de `scripts/e2e` contra o ambiente recém-publicado, depois de tudo — só leitura e sem *login*, então sem senha de demonstração no CI; o relatório sobe como artefato em falha. Falha de *workflow* avisa por e-mail quem fez o *push* (comportamento padrão do GitHub); o alerta de disponibilidade em regime é dos *uptime checks* do Cloud Monitoring (Terraform), por e-mail. A suíte agendada como monitor sintético (a "sonda" da Seção 17.4) foi deixada para uma onda posterior, por decisão de 2026-09-02.

E um quarto passo, dos *sourcemaps*: o `rsync` das experiências ganhou `--exclude='.*\.map$'` (vale para origem e destino), e os `.map` sobem para o *bucket* **privado** `EXP_MAPAS_BUCKET`, no mesmo caminho versionado. `scripts/mapear-stack.mjs` traduz a *stack* de um erro de cliente com o mapa da versão exata.

15.3.3 O que faz o *deploy* caber no café

> **DECISÃO DE PROJETO**
>
> **As imagens de api e hub sobem em *jobs* paralelos, e a camada de dependências dos *Dockerfiles* é cacheável por si** (ADR-38). Medido nos *runs* de 2026-08-28: as duas imagens eram 80% do tempo de *deploy* — 4m53s a api num *push* comum, mais 6m30s o hub em série num `forcar` — porque `COPY . .` vinha *antes* do `pnpm install`, e qualquer mudança de código invalidava a camada inteira. O *cache* GHA que o *workflow* sempre teve só acertava apt e corepack.

Os dois *Dockerfiles* passaram a abrir com uma camada que depende **só do *lockfile***:

Listing 15.3 – A camada de dependências (api e hub)

```
# Camada de dependencias: so invalida quando o lockfile muda.
COPY pnpm-lock.yaml pnpm-workspace.yaml ./
RUN pnpm fetch
# Codigo por cima; o install resolve do store local, sem rede.
COPY . .
RUN pnpm install --frozen-lockfile --offline
```

`pnpm fetch` existe para isto: baixa o *store* inteiro lendo apenas o *lockfile*, sem precisar dos `package.json`. Num *commit* que só toca código, a camada cara vem do *cache* e sobra o que é daquele *app*: `nest build` (ou `next build`), `pnpm deploy --prod` e o `prisma generate`.

E como as duas imagens não dependem uma da outra, cada uma virou um *job* — `deploy-api` e `deploy-hub` — atrás do mesmo portão, com o *diff* calculado uma única vez num *job* `mudancas`. A ordem que **importa** continua explícita no grafo, agora em duas partes: SUBIR o *bundle* não depende de nada além do portão, porque o caminho carrega a versão (`{expId}/{versao}/`) e ninguém pede por ele até o manifesto mudar — o *upload* é inerte. ATIVAR depende dos dois *deploys*, e isso não é preferência: a api valida o *PUT* contra o JWKS que o hub serve. Separar os dois encurta a janela em que o hub novo conviveria com o *bundle* antigo: de um `turbo build` inteiro para a duração de um *PUT*. E dentro de `deploy-api` a migração continua antes da revisão nova. A única ordem que se perdeu foi acidental: o hub não espera mais a api — os dois conversam por HTTP e o *rollout* do Cloud Run nunca garantiu revisão casada mesmo.

15.3.4 Nada de nome de recurso no *workflow*

DECISÃO DE PROJETO

Tudo que muda de um ambiente para outro entra por `vars` de ***environment*** do GitHub (`dev`, `hml` e, no futuro, `prd`) — o *workflow* **não sabe** o nome de nenhum recurso. Cada *job* de *deploy* declara `environment:` com o nome da *branch* que disparou o *push* (`main` mapeia para `prd`), e é esse vínculo que resolve as `vars` da *stack* certa. Os valores saem de `terraform output` do *workspace* correspondente, e há um *script* que os grava no *environment* certo (`scripts/configurar-github-ci.sh`) — ver a **ADR-53**.
É isso que faz o mesmo *workflow* servir a `dev`, a `hml` e à produção, e ao próximo contrato, sem *fork*.

ATENÇÃO

As `vars` de *environment* **só resolvem dentro de um *job* que declara `environment:`** — um bloco `env:` no nível do *workflow* as receberia vazias, sem erro. Por isso o `deploy.yml` não tem `env:` global: cada *job* de *deploy* declara o *environment* e as próprias variáveis. E referenciar um *environment* que não existe o **cria em silêncio**, com todas as `vars` vazias — o *job* `mudancas` tem um passo de guarda que derruba um `workflow_dispatch` disparado de *branch* que não é de ambiente.

	Conteúdo
`GCP_PROJECT`, `REGION`	Projeto e região.
`AR_REPO`	Artifact Registry.
`API_SERVICE`, `HUB_SERVICE`, `MIGRATE_JOB`	Nomes dos serviços e do *job*.
`EXP_BUCKET`, `ASSETS_BASE_URL`	*Buckets* e base pública de *assets*.
`API_URL`, `HUB_ORIGIN`	Endereços públicos.
`WIF_PROVIDER`, `DEPLOY_SA`	Identidade federada do CI.
`CLARITY_PROJECT_ID`	Id do projeto Microsoft Clarity (**ADR-60**). Única `vars` que **não** sai de `terraform output`: é criada à mão em cada *environment*.

ATENÇÃO

`CLARITY_PROJECT_ID` entra na *imagem*, não no serviço. É a única variável de todo o *deploy* que chega ao *bundle* do **cliente**, e `NEXT_PUBLIC_*` é substituída pelo literal durante o `next build` — defini-la no Cloud Run depois não altera um *bundle* já compilado. Por isso o passo *Imagem do hub* a passa como `build-arg`, e o `apps/hub/Dockerfile` a recebe

num `ARG` antes do `turbo build`. As duas consequências: trocar o id exige *rebuild* (não basta reiniciar o serviço), e uma imagem passa a ser específica do ambiente que a construiu — promover `hml` para produção reconstrói, não repromove a mesma *tag*. Ausente, o `ARG` cai no *default* vazio e o hub sobe com o Clarity desligado, sem erro.

15.3.5 Concorrência e reexecução

Listing 15.4 – A política de concorrência do *deploy*

```
# Dois deploys ao mesmo tempo brigam pela mesma revisao do Cloud Run e pelo
# mesmo banco. `cancel-in-progress: false` de proposito: abortar no meio de
# uma migration deixa schema e imagem fora de sincronia — melhor enfileirar.
concurrency:
  group: deploy-${{ github.ref }}
  cancel-in-progress: false
```

NOTA

Há um `workflow_dispatch` com *input* `forcar`, para publicar tudo ignorando o *diff* — primeiro *deploy* de um contrato, ou reexecução depois de falha. Com `forcar`, o *build* também tem de ser de **todas** as experiências, senão o laço de publicação encontra pasta sem `dist/`.

NOTA

O laço de publicação é tolerante por desenho: pasta de experiência sem `package.json`, sem `public/experiencia.json` ou sem `dist/` é **pulada com aviso**, não é erro. Experiência incompleta é trabalho em andamento, e trabalho em andamento não deve quebrar o *deploy* do resto.

15.4 Ambientes locais

Modo	Comando	Quando usar
Banco em contêiner, resto no *host*	`docker compose up -d pg` + `pnpm dev`	Iteração rápida com *debugger* do editor. O caminho mais direto.
Tudo em contêiner, sincronizado	`pnpm docker:dev`	**Recomendado.** Banco, API, hub e as seis experiências, cada um com o próprio *watcher*, com o código do *host* por *bind mount*: salvar um arquivo recompila lá dentro (API ∼1 s, hub e Vite na hora), sem `build` e sem `up` de novo.

Imagens congeladas	`docker compose up -d --build`	Aproximar-se da produção. Migrações e *seed* rodam sozinhos no primeiro `up`.

> **ATENÇÃO**
>
> As portas são as mesmas nos três modos (hub `:3000`, API `:3001`, experiências `:5173`–`:5177`), então **não dá para rodar `pnpm docker:dev` e `pnpm dev` ao mesmo tempo** — é um *ou* o outro.

Situação	O que fazer
Mexeu em dependência (`package.json` ou *lockfile*)	`pnpm docker:dev:deps` e reiniciar. O `node_modules` de cada contêiner é **volume próprio** — binário de macOS não roda no Linux da imagem — então instalar no *host* não basta.
Salvou um arquivo e nada recompilou	`DEV_POLLING=1 pnpm docker:dev`. O *inotify* pode não atravessar o *bind mount* em alguns *setups*.

15.5 O provedor de identidade sobe à parte

Listing 15.5 – O WSO2 tem ciclo de vida próprio

```
docker compose -f wso2/docker-compose.yml up -d     # IS :9443, banco dele :5433
docker compose -f wso2/docker-compose.yml watch     # olha conf/ e bootstrap/
```

> **NOTA**
>
> O `watch` é o equivalente do *hot reload* para um servidor Java: `conf/` já entra por volume (nada de *rebuild*), e mudar o `deployment.toml` **reinicia** o servidor — que é o ciclo mais curto possível ali, porque o *entrypoint* só copia o volume de configuração para o *server home* no *boot* (∼2 min). Mudar `bootstrap/` reexecuta o *bootstrap* (idempotente); só mudar o `Dockerfile` (o *driver* JDBC) refaz imagem.

15.6 Convenções de *git*

Convenção	Regra
Fluxo	**Quatro ambientes** (ADR-101): *branch* de trabalho → PR para `dev-local` → PR para `dev` → PR para `hml` → (futuro) PR para `main`. Quem testa muda a cada degrau: `dev-local` é o Docker da máquina e quem olha é quem desenvolve; `dev` é do time; `hml`, do cliente. **Ninguém commita direto em *branch* de ambiente** — em `dev`, `hml` e `main` o *merge* do PR dispara o *deploy* da *stack*; em `dev-local` ele integra a frente ao ambiente local, que não tem *deploy* e por isso não está no `deploy.yml`.

Worktree	**Toda tarefa começa num**, paralela ou não (**ADR-61**, **ADR-101**): `pnpm worktree criar <branch> --fatia <caminhos>`, sobre `origin/dev-local` ESCRITO. A cópia principal é onde a *stack* roda, e editar nela mistura o que subiu com o que está sendo escrito. `--fatia` é obrigatória (**ADR-71**).
Merge	*Merge commit* ou *squash*; **rebase fica desabilitado** nas opções do repositório. O *job* `mudancas` compara `HEAD^` (o primeiro pai), que num *merge commit* é a ponta antiga da *branch* de ambiente — um *rebase-merge* faria o *diff* enxergar só o último *commit* do PR.
Branches	**Uma por experiência**, com o código dela (`exp06ef3b` — **ADR-110**). Trabalho que não é experiência usa o prefixo do tipo do *commit*: `fix/quota-por-perfil`, `chore/deps`, `ci/deploy`.
Commits	`feat(exp06ef3b): ...`, `fix(api): ...`, `chore(sdk): ...` — o escopo é o código da experiência ou o aplicativo tocado. Validado pelo *hook* `.githooks/commit-msg`.
Promoção	O PR de `hml` para `main` leva **tag semver** (`v1.4.0`) e *release*, criadas depois do *merge*. MAJOR só em quebra de contrato externo (`PROTOCOL_VERSION` da ponte, manifesto, telemetria, contrato do Terraform); MINOR é o caso comum. A *tag* é o nome do que está em produção — sem ela, "voltar para a anterior" vira caça a *SHA* no meio de um incidente.
Hotfix	Sai de `main`, entra por PR em `main`, leva *tag* de PATCH — e desce a escada INTEIRA por **retro-merge** no mesmo dia: `hml`, `dev` e `dev-local`. Parar em `dev` deixa a base local sem a correção, e a frente seguinte nasce sem ela.
Imposição	`branch protection` e `rulesets` respondem **403** (repositório privado, plano *Free*). As regras vivem em `.githooks/` — `pre-push` recusa *push* direto em *branch* de ambiente e nome fora do padrão — instaladas pelo `prepare` do `pnpm install` (`core.hooksPath`). Ver **ADR-56**.

NOTA

O fluxo completo, com os comandos de cada ponte (promoção, *hotfix*, *retro-merge*, *tag*, *rollback* por camada), está em `CONTRIBUTING.md`, na raiz do repositório.

ATENÇÃO

Nada de *build* no *git*. O `.gitignore` cobre `dist/`, `.next/`, `.next-prod/`, `.turbo/`, `*.tsbuildinfo`, `storage/`, `storage-privado/` e os *prints* das suítes de ponta a ponta. `apps/hub/.next-prod/` já entrou uma vez por um `git add -A` distraído: 358 arquivos e 250 MB de *cache* de *webpack* no histórico, que só saíram reconstruindo a *branch* inteira. **Stage por caminho, nunca `-A` sem olhar o `git status`.**

> **NOTA**
>
> `apps/hub/next-env.d.ts` **é** versionado (o Next espera isso), mas alterna entre `./.next` e `./.next-prod` conforme o último *build*. Modificado depois de um `build:isolado` = descartar, não *commitar*.

16

Qualidade

16.1 A *Definition of Done*

REGRA INVIOLÁVEL

Nenhuma tarefa está concluída sem estes seis itens. É o portão que substitui a revisão em par que uma pessoa não tem.

1. ***Typecheck* + *lint* + testes passando** (`pnpm turbo typecheck lint test`).
2. **Se mexeu em tela: a suíte de ponta a ponta do papel afetado** (`node scripts/e2e/rodar.mjs <papel>`) — *typecheck* verde não pega ícone virando texto, *link* sublinhado nem `iframe` vazio. Tela nova é passo novo na suíte, com `titulo`/`mostra`/`prova` preenchidos: são eles que viram slide e legenda de vídeo.
3. ***Input* externo validado com Zod.**
4. ***Queries* novas escopadas por *tenancy*** (revisar mentalmente a regra 1).
5. **Sem segredo nem URL interna em código de cliente.**
6. **Se tocou contrato** (ponte, telemetria, *provider*, manifesto): **documentação atualizada** — em `AGENTS.md` e neste documento.

16.2 Estratégia de testes

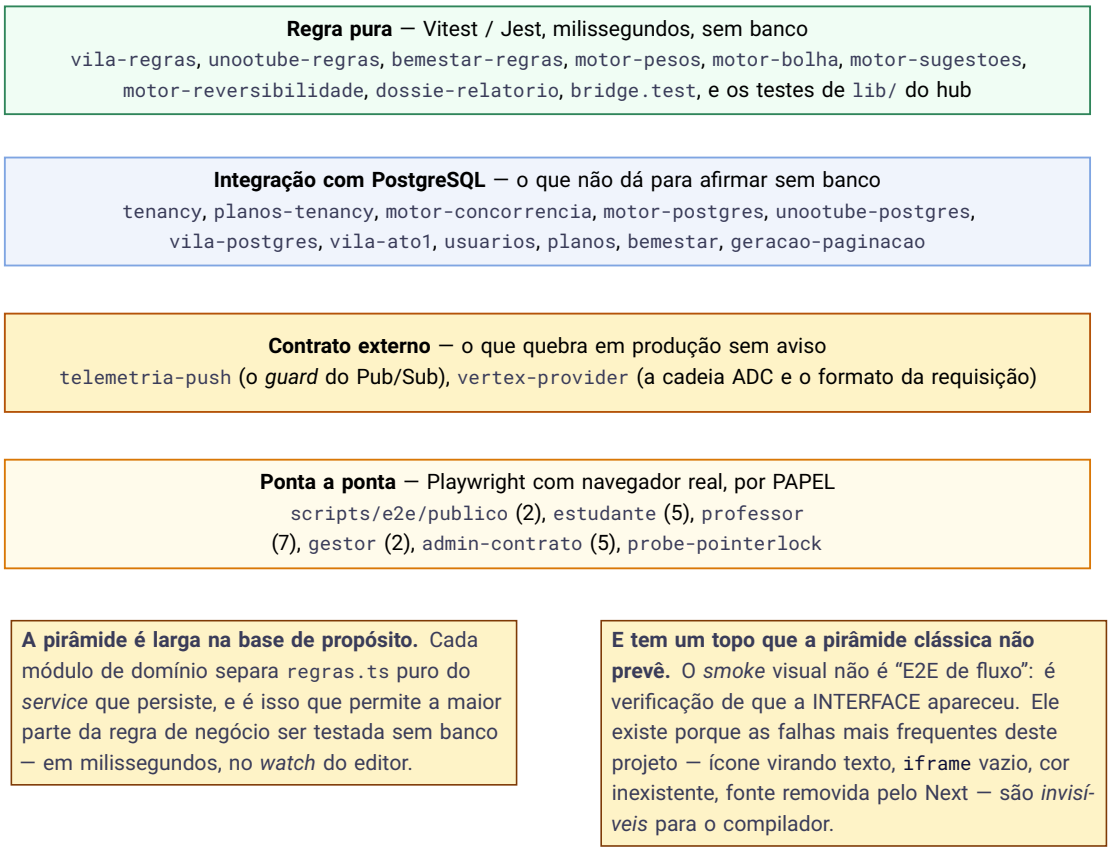


Figura 16.1 – As quatro camadas de verificação. A quarta é incomum e é a mais importante deste projeto: ela pega a classe de falha que o *typecheck* estruturalmente não vê.

> **NOTA**
>
> **Caso registrado (2026-08-27).** O *smoke* da exp04ef1b foi o único verificador a pegar o `hub:pause` espúrio do `ExpFrame.tsx` (**ADR-22**): 31 tarefas de `typecheck`, `lint` e teste passavam, e o que denunciou foi uma linha do relatório — "autoplay BLOQUEADO". O defeito já existia antes da mudança que o expôs; nenhuma camada abaixo tinha como vê-lo, porque ele mora no encontro entre foco de janela, *postMessage* e política de mídia do navegador.

Camada	Ferramenta	Mínimo obrigatório
Regra pura	Vitest (SDK, experiências) / Jest (NestJS)	Contratos: ponte, telemetria, *guards* de *tenancy*. Toda função de `regras.ts`.
Integração	Jest + PostgreSQL de teste	*Tenancy*, *lock*, concorrência, idempotência.

Contrato externo	Jest com *fake* de rede	*Guard* do Pub/Sub e formato da requisição ao Vertex.
Smoke visual	Playwright, navegador real	Toda mudança de tela.

16.3 A suíte de ponta a ponta, em detalhe

Ela vive em `scripts/e2e/`, organizada por PAPEL — `publico` (3 suítes), `estudante` (10), `professor` (11), `gestor_escola` (4), `gestor_contrato` (6) e `admin` (3), contadas em 2026-09-25. Por papel, e não pelo nome da experiência, porque é o crachá que decide o que a pessoa alcança, e porque a organização antiga escondia as telas da PLATAFORMA, que são a maioria (ADR-51, Seção 18.52).

Listing 16.1 – A suíte de ponta a ponta

```
pnpm exec playwright install chromium    # uma vez por maquina. NAO `npx playwright install`:
                                         # o npx baixa a build de navegador de OUTRA versao
node scripts/e2e/rodar.mjs               # tudo: publico, estudante, professor, gestor, admin
node scripts/e2e/rodar.mjs professor     # so um papel
node scripts/e2e/rodar.mjs estudante/02  # so uma suite
node scripts/e2e/rodar.mjs --deck        # tudo + deck (.pptx) + videos (.webm)
node scripts/probe-pointerlock.mjs
```

ATENÇÃO

Duas coisas exigem o **Chrome do sistema**, não o Chromium do Playwright: a suíte da app-70 e o *probe* de *pointer lock*. O Chromium empacotado **não concede *pointer lock* nem no topo da página**, então usá-lo produziria um falso negativo — o teste diria que a permissão de *sandbox* não funciona quando o problema é o navegador.

NOTA

As suítes precisam da plataforma no ar (contêineres ou `pnpm dev`) e gravam *prints* na pasta indicada — que está no `.gitignore`. A senha de todos os usuários de demonstração é `unooria123`; os *logins* do *seed* são `gestor.demo`, `prof.demo` e `estudante01`–`estudante40`. Nenhum roteiro fixa *login* no código: eles saem de `credenciais-demo.csv`, que o *seed* escreve — fixá-los fazia qualquer mudança no *seed* quebrar seis scripts com “timeout esperando o feed”, que não diz que a conta mudou de nome.

16.3.1 O manifesto: a descrição mora no teste que fotografa

Cada passo é registrado com `r.registrar(pagina, nome, {titulo, mostra, prova, detalhe})`, e isso vira um `passos.json` por suíte, ao lado dos *prints*. O *deck* de evidências

(`.pptx`) e os vídeos (`.webm`, um por papel e um por missão do estudante) são gerados DELE — os geradores não conhecem o produto, só desenham.

Antes existia um roteiro de 66 KB mantido à mão, separado dos scripts, e ele divergiu: descrevia telas que já não eram fotografadas e omitia passos novos. É a mesma armadilha do `expires_at` — o registro fica, a regra sai, e quem lê acredita nele.

16.3.2 Erro provocado não é erro descoberto

Quando um roteiro exercita uma trava de propósito — ativar uma atividade com conteúdo de menos (422), subir um perfil portátil adulterado (400), abrir a análise de produto com o papel errado (403) — o erro que aparece na tela é a PROVA de que a trava existe. Ele é declarado com `r.provocado(padrão, motivo)` e sai numa seção própria do relatório.

Sem isso, o vermelho aparecia justamente na corrida que demonstrava o comportamento correto — e vermelho que não significa nada treina a pessoa a ignorar o vermelho, que é o único jeito de um *smoke* deixar de servir.

16.3.3 O roteiro tem de PERDER a corrida para o erro

Peças compartilhadas em `scripts/e2e/lib/` — `roteiro.mjs` (a casca: navegador, *login* com portão de hidratação, *print* com manifesto, asserções e relatório) e `smoke.mjs` (as esperas que correm contra o erro). Antes cada roteiro carregava sua própria cópia, e as doze cópias divergiram: uma tirava `fullPage` e outra não, três reprovavam por erro de console e nove não.

Medido em 2026-08-29: ao gerar um plano de aula, a API falhou em **306 ms** e o roteiro ficou **180 s** esperando uma `<table>` que nunca viria — 588 vezes o tempo que a resposta levou. O erro estava escrito em vermelho na tela desde o primeiro segundo, e o que saiu no console foi `Timeout waiting for locator('table')`: um sintoma que não aponta para a causa, dentro do próprio instrumento que existe para encontrá-la.

Os 3 minutos são o menor dos problemas. **Nenhum roteiro olhava estado de erro** — nenhuma ocorrência de `FeedbackAlert` em `scripts/`. Uma falha que ainda deixasse uma `<table>` na tela (plano cortado por `MAX_TOKENS` caindo num *fallback*) passaria **verde**.

Peça	O que faz, e onde vale
`esperarOuFalharCedo`	Corrida entre o sinal de sucesso e o alerta de erro; sobe a mensagem que o professor LEU, não o nome do seletor que faltou. O *timeout* continua alto (a geração real leva de 30 a 60 s) — o que muda é que o caminho de falha não o paga.
`ALERTA_DE_ERRO`	`.unooria-feedback-alert--error`. É a CLASSE, não `[role="alert"]`: o *role* é o mesmo para *info*, *warning* e *success*, e esperar por ele faria um aviso benigno derrubar o roteiro. Vale para quem fala pela PLATAFORMA (app-90, app-91).

`vigiarFalhasDaApi`	Anota respostas 4xx/5xx para o relatório, com rótulo do contexto. É o sinal das experiências DISFARÇADAS (exp08ef1b, exp02em1b), que mostram erro em `<p>` avulso, com classe diferente em cada tela, e por isso não têm gancho estável. **Não derruba o roteiro**: um 404 de sondagem é esperado, e derrubar por *status* inventaria falha. Para 5xx e 422 ele anexa o CORPO — "500 POST /destilar" manda procurar no *log* da API; "422 … 12 posts aprovados, o recomendado é 300" resolve na hora.
`esperarOuFalharPorHttp`	O irmão do primeiro para quem não tem alerta no DOM: corre entre o sinal de sucesso e a RESPOSTA de erro da rota, e põe o corpo dela na mensagem. Foi ele que trocou "180 s esperando *Perguntas propostas pela IA*" por "9 s, HTTP 500 em /destilar".

> **ATENÇÃO**
>
> Os dois mecanismos são complementares, e o caso da app-90 mostra por quê: o `POST /planos` devolve **202**, e o erro só aparece no CORPO do *polling*. Um vigia de *status* HTTP não o pegaria — ali o sinal é o alerta no DOM. Na exp02em1b é o contrário: a rota devolve **503** e não há classe estável para esperar.

16.3.4 Duas condições para o vermelho significar alguma coisa

Idempotência. O roteiro do estudante cobre *login* → **onboarding** → *feed*, e o *onboarding* só acontece uma vez: na segunda execução a criança cai direto no *feed*, a espera pelo texto da tela de entrada estoura em 30 s, e o relatório diz *iframe* vazio quando o *iframe* está perfeito, só adiantado. Ele passou a zerar a turma no começo, como o da explicabilidade e o da exp02em1b já faziam. ***Smoke* que só passa uma vez por banco não é *smoke*** — e o sintoma dessa falha é indistinguível de um defeito de produto.

Ruído de dev não pode pintar de vermelho. O `performance.measure(routeChangeToRender, …)` de `next/dist/client/index.js` pode receber *timestamp* negativo numa navegação cliente e lança `Failed to execute 'measure' on 'Performance'`. Aparece em 11 dos 12 roteiros; como o da exp04ef1b e o da exp02em1b reprovam por QUALQUER erro de console, os dois viviam vermelhos por um defeito que não é nosso — e vermelho permanente treina a pessoa a ignorar o vermelho, que é o único jeito de um *smoke* deixar de servir. A lista de ruído (`ehRuidoDeDev`) é explícita, curta, e quem filtra **conta e diz quantos ignorou**: silenciar por padrão devolveria o problema em outra forma.

16.3.5 O que só o *smoke* visual pega

Falha	Por que o *typecheck* não vê

Ícone virando texto	O Next remove `@import` externo de CSS importado por JS, *silenciosamente*. `Material Symbols Rounded` é fonte de ligadura, então o sintoma é "addInvestigação" no lugar do ícone. O `getComputedStyle` *responde* "Open Sans" — a regra continua no CSS.
Cor inexistente em fundo	Classe de cor que não existe em `plataforma.css` não é erro em lugar nenhum. Em texto herda a cor; em **fundo** o elemento fica branco no branco. O *smoke* da app-70 checa o `backgroundColor` **computado** — `toBeVisible()` não pega.
`iframe` vazio	*Handshake* que expira, *dev server* fora do ar, manifesto ausente: o componente monta, o `iframe` existe, e não há nada dentro.
Experiência esticada na janela	`viewport` não lido (manifesto vazio + `EXP_INTERNAL_HOST` errado). Nenhum tipo muda.
Link sublinhado, marcador de lista de volta	*Preflight* do Tailwind desligado. O pacote estiliza *componentes*, não elementos.
Câmera do jogo que não anda	`allow-pointer-lock` ausente: `requestPointerLock()` morre com `SecurityError` e nada no console diz por quê.

> **ATENÇÃO**
>
> **Testes de navegador precisam de portão de hidratação.** Em desenvolvimento, o SSR chega *antes* do *bundle*, e o `fill` do Playwright não move o estado do React — o campo mostra o texto e o `onChange` nunca rodou. Toda suíte espera um sinal de hidratação antes de interagir.

16.4 Análise estática

Regra	Aplicação
TypeScript *strict* em tudo	`tsconfig.base.json`, herdado por todos os *workspaces*. Inclui `noUncheckedIndexedAccess` — é por isso que o código de curadoria usa `!` em índices provadamente válidos, com comentário.
Sem `any`	Use `unknown` + *narrow*. Uma exceção, registrada em ADR: `app-70/public/jogo/**` está nos `ignores` do ESLint.
Zod em todo *input* externo	Corpo, *query*, `postMessage` recebido, variáveis de ambiente.
ESLint	`eslint.config.mjs` na raiz, com `typescript-eslint`, `eslint-plugin-react-hooks` e `@next/eslint-plugin-next`.

> **ATENÇÃO**
>
> O ESLint **ignora a pasta `storage`**. Um módulo com esse nome passaria no *lint* sem ser lido — foi por isso que o módulo de armazenamento de objetos se chama `object-storage`, e não `storage`.

16.5 Armadilhas de ferramenta registradas

Cada linha abaixo é um erro efetivamente cometido, com o sintoma que ele produz. Estão aqui porque o sintoma nunca aponta para a causa.

Armadilha	Sintoma, e o que fazer
Build isolado apagando o `.next` do *dev*	Sintoma: "tela branca" ou "CSS bugado" depois de validar um *build* de produção. Validar sempre com `pnpm build:isolado` (`HUB_DIST_DIR=.next-prod`).
Standalone do Next não copia `static` nem `public`	Sintoma: CSS 404 e página sem estilo. Distinguir do caso acima pelo **código** do erro, não pela aparência.
Substituição de texto sem asserção	Edição por *replace* que não casa passa em **silêncio**, e o `tsc` não acusa. Sempre afirmar o casamento.
Arquivo em `public/` escrito em *runtime*	Dá 404: `public/` é do *build*. E `Cache-Control` de rota dinâmica só cola via `next.config`.
Probe de *hot reload* como comentário	O *esbuild* remove comentários — o falso negativo faz parecer que o *inotify* não atravessa o *bind mount*.
`sanitize-html` recente é ESM-only	Fixar `~2.13.1` na API (CJS, Node 20).
Design system quebrando o *dev server* do Vite	ESM importando diretório: o *build* passa, o *dev* morre. Sintoma: `iframe` vazio ou `ERR_CONNECTION_REFUSED`.
Vite pulando de porta	Uma segunda instância da mesma experiência ocupa a porta seguinte — e o hub passa a ler o manifesto da experiência errada.

16.6 Cobertura: onde está e onde não está

Área	Cobertura	Observação
Tenancy e RLS	✓ Forte	Teste com *role* sem *ownership*; o caso mais crítico do sistema.
Regra de domínio	✓ Forte	`regras.ts` puro em todos os módulos.
Concorrência do motor	✓ Forte	*Lock* consultivo, reversibilidade, paginação da geração.

Contrato da ponte	✓ Forte	*Origin*, esquema, versão maior.
Guard do Pub/Sub	✓ Forte	Inclui o caso *fail closed*.
Provider de IA	✓ Parcial	Cadeia ADC e formato da requisição. Não há teste contra o Vertex real — e não deveria haver.
Telas	✓ Ponta a ponta	Trinta e sete suítes, seis papéis; não é cobertura de interação, é verificação de que apareceu.
Carga	**– Ausente**	O teste de 4.000 *logins* em dez minutos é obrigatório antes de novembro de 2026 e não foi executado. Risco `R-01`.
Segurança externa	– Ausente	Sem revisão externa nem teste de intrusão. Risco `R-06`.
Acessibilidade	– Ausente	Nenhuma verificação automatizada de contraste, foco ou leitor de tela. Risco `R-07`.

LIMITAÇÃO CONHECIDA

Não há métrica de cobertura de linha configurada, e não há meta numérica. A escolha é deliberada: com uma pessoa desenvolvendo, uma meta de percentual incentiva testar o que é fácil em vez do que é caro se falhar. O critério em uso é o da *Definition of Done* — contratos, *tenancy* e regra pura — e ele é verificável por inspeção da lista de arquivos de teste.

17

Observabilidade

Este capítulo é um estudo com recomendação: levanta o que a plataforma revela de si mesma hoje, define o que ela precisa revelar para ser operada por uma pessoa em novembro de 2026, compara as ferramentas que fazem isso — com preferência declarada por Google Cloud e por software livre — e fecha num roteiro em fases, com custo estimado por contrato. As duas decisões que ele recomenda estão registradas como **ADR-57** e **ADR-58**, no estado *proposta*, e o risco `R-05` passa a apontar para aqui.

> **DECISÃO DE PROJETO**
>
> **Telemetria não é observabilidade.** O Capítulo 10 descreve o instrumento *pedagógico*: eventos xAPI do estudante, gravados para análise do que ele fez na experiência. Este capítulo trata do instrumento *operacional*: o que a plataforma emite sobre *si mesma* — latência, erro, saturação, custo de IA, fila — para quem a opera. Os dois usam a palavra "evento" e os dois passam pela API, e é justamente por isso que a confusão é frequente (MAJORS; FONG-JONES; MIRANDA, 2022). Um mede a criança; o outro mede o software. Nenhum substitui o outro, e a regra 6 (nada de PII em log) vale igual para os dois.

17.1 O que se vê hoje

> **NOTA**
>
> Este levantamento retrata 1º de setembro de 2026 e **já não descreve o ambiente**: entre 2 e 3 de setembro entraram o log estruturado, os rastros, as sondas, doze políticas de alerta, dois SLOs, a fila morta da telemetria e o painel de operação. O estado corrente está na Seção 17.9, ao fim do capítulo. O texto abaixo é mantido como registro do ponto de partida — é o que justifica cada decisão que veio depois.

O levantamento foi feito sobre o código em `dev` em 1º de setembro de 2026, com busca por biblioteca, por padrão de código e por configuração de implantação. O resultado é o mesmo em todas as camadas: **não há nenhuma biblioteca de observabilidade instalada em nenhum pacote do *monorepo*** — nem OpenTelemetry, nem `pino`/`winston`, nem `prom-client`, nem Sentry,

nem `web-vitals`. O que existe é o que a infraestrutura dá de graça e o que a suíte de ponta a ponta prova quando alguém a roda.

Camada	O que existe	O que falta, e onde se vê
API	`ConsoleLogger` padrão do Nest em **texto**, quatro `Logger` nomeados e 19 `console.*` com prefixo manual. `GET /healthz` (processo) e `GET /readyz` (`SELECT 1`). Disciplina manual da regra 6, com comentário em cada ponto sensível.	Sem JSON ⇒ cada linha vira `textPayload` sem severidade. Sem *request-id* nem `traceparent` (`tenancy-context.ts` carrega só `escolaId`). Sem log de requisição na aplicação. Sem `ExceptionFilter`: erro não previsto vira 500 genérico com *stack* no *stdout*. Sem `LOG_LEVEL`. `new PrismaClient()` sem `log` nem métrica. Um desvio da regra 6: `auth.controller.ts` loga o `username` em claro.
Cloud Run	Log de requisição e métricas nativas (contagem, latência, instâncias, CPU, memória) chegam ao Cloud Logging e ao Cloud Monitoring sem configuração.	O `gcloud run deploy` do `deploy.yml` não configura sonda: vale a *startup probe* TCP padrão, e `/readyz` **não é usada por nada**. O `docs/INFRA-IDENTITY.md` manda "nunca usar probe TCP" para o WSO2; a API não segue o próprio conselho.
Hub	`console.*` esparso (`[hub]`, `[chave-token]`, `[sessao]`, `[login]` — `console` e não o `pino` de `lib/log.ts` nos dois últimos, porque os módulos que os emitem são importados pelo *middleware*, que roda no EDGE); o único log diagnóstico real é `avisarManifestoIlegivel`, escrito depois de um incidente. Nas telas, falha vira `FeedbackAlert` de erro.	Sem `error.tsx`, `global-error.tsx`, `instrumentation.ts`, `reportWebVitals` ou rota de saúde. Sem `HEALTHCHECK` no Dockerfile nem no *compose*. `apiDoServidor` **engole qualquer falha em `null`**, sem log: "API caiu" e "não há dados" são indistinguíveis. Os cinco *route handlers* do BFF não registram nada, inclusive o 404 de rota fora da *allowlist*. **Dos `fetch` do hub, só o do *login* tem *timeout*** — 8s, e o prazo estourado chega à tela como "não foi possível entrar agora" (**ADR-97**); todos os outros esperam indefinidamente.
Experiências e SDK	O protocolo da ponte tem `exp:ready`, `exp:concluir`, `exp:estado`, `exp:token-expirando`. O `TelemetryClient` faz lote de 50 a cada 10 s e `sendBeacon` no `pagehide`.	**Não existe `exp:erro`**, nem `window.onerror`, nem `unhandledrejection` em lugar nenhum: um *crash* na app-70 durante a aula não deixa rastro fora do console do estudante. Falha ao renovar *token* ou ao registrar conclusão é `console.error` no navegador. Sem *sourcemap* nos *bundles*. Fila de telemetria só em memória, sem *backoff*, sem teto, sem `visibilitychange` (a exp08ef1b registra os três eventos e comenta por quê; a SDK não aplicou a lição).

Camada	O que existe	O que falta, e onde se vê
Pipeline de telemetria	Guarda do *push* falha fechada; `INSERT` multi-linha; *dedupe* por `(event_id, ts)`.	Quatro caminhos de descarte silencioso no *controller*, só com `console.warn`. `recebidos` conta o publicado, não o gravado. Nenhum sinal de atraso da *subscription*, de *dead letter* ou de lote perdido no cliente.
IA	`ia_uso_diario` guarda *tokens* por (escola, usuário, dia) e o `AiResult` já carrega `finishReason`, `voltas` e `tentativas`.	O `VertexProvider` (438 linhas) **não mede tempo e não loga**. Ninguém consome `voltas`/`tentativas`. Custo não é calculado. Degradação deliberada (503 amigável, texto fixo, arte procedural) é boa para o estudante e **invisível** para quem opera: a falha "dá certo" e não conta em lugar nenhum.
Trabalhos em segundo plano	Fila de geração e de planos persistida em tabela, com `status`, `erro` e `tentativas` consultáveis.	A visibilidade é `SELECT` numa tabela: sem alerta, sem taxa de erro, sem *dead letter*. Não há *job* agendado — por decisão — e portanto nenhuma verificação recorrente roda sozinha.
CI/CD	`deploy.yml` separa subir de ativar, roda a migração com `--wait` e reprova o *pipeline* se ela falhar.	**Sem *smoke* pós-*deploy*** e **sem notificação em falha**: o único sinal é o *exit code* do `gcloud`, que volta assim que a revisão recebe tráfego, não quando ela responde. Snyk comentado.
Infra (Terraform)	`docs/INFRA-IDENTITY.md` desenha *uptime checks*, métrica de log, dez alertas com severidade e painel — **só para o WSO2**.	`docs/INFRA.md` não tem **uma linha** sobre alerta, sonda, SLO, retenção de log ou painel para hub, API, CDN, Cloud SQL ou Pub/Sub.
Suíte e2e	`scripts/e2e`: 19 suítes por papel, `esperarOuFalharCedo`, `vigiarFalhasDaApi`, classificação de ruído/pendência/defeito, relatório estruturado e *exit code* honesto.	Só roda com operador. Não corre depois de um *deploy*, não roda em horário, não emite métrica: o melhor detector de defeito da plataforma depende de alguém lembrar de ligá-lo.

LIMITAÇÃO CONHECIDA

O resumo honesto é: **a plataforma é diagnosticável, mas não é observável.** Quem sabe onde olhar reconstrói uma falha a partir do log do Cloud Run e de um `SELECT`; ninguém é avisado de que ela aconteceu, e uma classe inteira de falha — a que acontece dentro do `iframe`, no navegador do estudante — não deixa rastro em lugar nenhum. Para o piloto em `dev`, com uma pessoa olhando, isso bastou. Para cem escolas às oito da manhã, não basta.

ATENÇÃO

Medição de 2026-09-01, na *stack* de `dev`: num dia inteiro de operação, o serviço da API produziu **exclusivamente** entradas no *log* `run.googleapis.com/requests`. Nenhuma linha em `varlog/stdout` ou `stderr` depois do *banner* de subida do contêiner — ou seja, os quatro `Logger` nomeados e os 19 `console.*` da tabela acima **não aparecem no Cloud Logging** como sinal consultável.
A consequência é concreta e já foi encontrada em diagnóstico: quando o Vertex recusa uma geração, o `unoogpt.service` escreve `turno sem resposta: <motivo>` com o corpo da resposta do provedor — a única pista de *qual* foi o erro (modelo inexistente, papel de IAM faltando, região errada), já que o estudante vê a mesma frase amigável nos três casos. Esse texto não está em lugar nenhum que se possa consultar depois. O efeito colateral perverso é que a **ausência** de erro no *log* não prova ausência de erro no serviço, e foi preciso recorrer ao *log* do *load balancer* — que enxerga o *status* HTTP e a latência, não a razão — para separar "não falhou" de "falhou em silêncio".
Isso reforça a prioridade do log estruturado em JSON (Seção 17.3.1 adiante): o problema não é o formato do texto, é o sinal não existir.

NOTA

O que mudou na API em 2 de setembro de 2026 (fase 1). O `ConsoleLogger` saiu: o Nest loga pelo `nestjs-pino` (`observabilidade/logger.ts`), em JSON com `severity`, `message`, `time` RFC 3339, `serviceContext` e os campos `logging.googleapis.com/trace`, `spanId` e `trace_sampled` preenchidos a partir do contexto da requisição. Esse contexto (`tenancy/tenancy-context.ts`) é aberto por um **middleware do Express** antes dos *guards* — um 401 sai com `x-request-id` como qualquer 200 — e o `TenancyInterceptor` só o preenche com escola e perfil. O `requestId` é o `traceId` que o *load balancer* mandou (`traceparent` ou `X-Cloud-Trace-Context`), então o código que o professor lê no 500 é o que quem opera cola no Logs Explorer. A *redaction* é mecânica (`observabilidade/redaction.ts`): por nome de chave em qualquer profundidade e por padrão na mensagem (e-mail, CPF, `Bearer`, *query string*); o `username` do `auth.controller.ts` deixou de existir no log. O `FiltroDeExcecao` global responde 500 `{codigo, requestId}` com UMA linha `ERROR` e `stack_trace`, e converteu em 400 um caso que era 500 mudo: o `ZodError` que escapava de um `parse(body)` no *handler*, devolvido como `path: code`, sem o valor recebido. Os 19 `console.*` viraram `logger.*` com um campo `codigo` estável (`telemetria.descartado`, `planos.geracao-com-erro`…) — é por ele que se filtra e que se deriva métrica de log. Os descartes do `/telemetry/beacon`, rota sem *guard*, saem em `debug`: uma linha por lote recusado seria vetor de inundação; o sinal contável entra com as métricas da fase 2. O que ainda não existe: *traces*, métricas da aplicação, e tudo do hub e da SDK.

A Figura 17.1 percorre a cadeia de uma requisição e marca, elo por elo, o que se enxerga hoje e o que está cego.

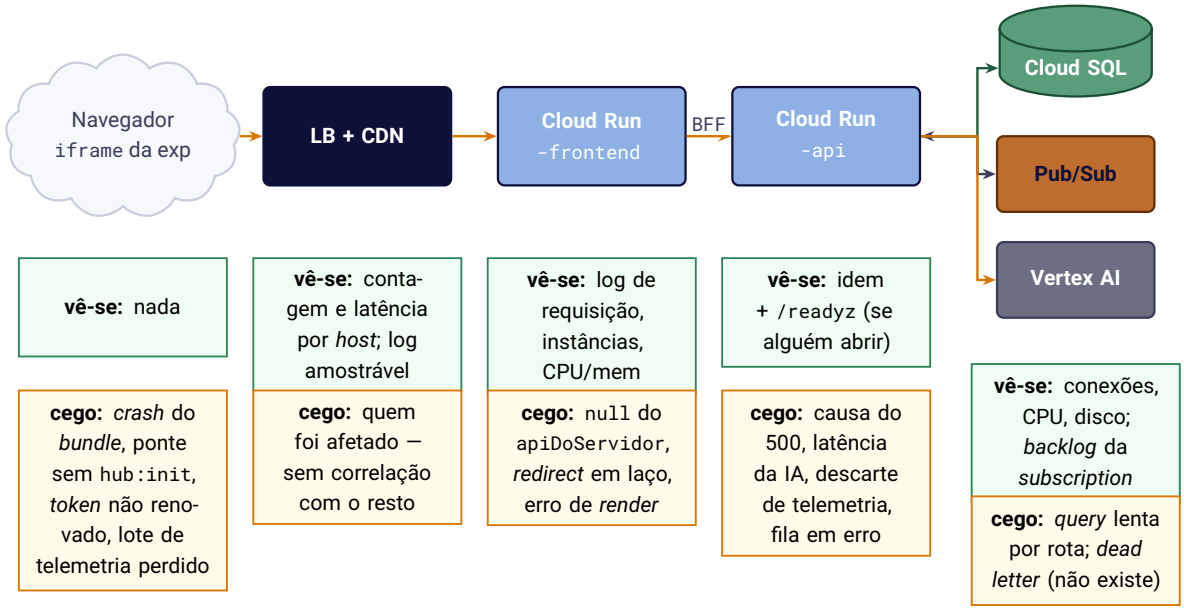


Figura 17.1 – A cadeia de uma requisição e o que cada elo revela hoje. O verde é o que a infraestrutura entrega sem configuração; o laranja é onde a plataforma está cega. O primeiro elo — o navegador, onde a experiência roda — é o mais cego e o que mais importa: é lá que a criança está.

17.2 Princípios

As escolhas deste capítulo derivam de sete princípios, na ordem em que decidem um empate.

1. **Alerta é sintoma, não causa.** Quem é avisado é avisado de que o estudante não consegue abrir a experiência — não de que a CPU passou de 80%. Causa vai para painel e para log; sintoma vai para o telefone (BEYER et al., 2016). A consequência prática é que a lista de alertas é **curta**: uma pessoa que recebe vinte alertas por dia para de lê-los na primeira semana, e a suíte e2e já pagou para aprender isso com o ruído de console (Capítulo 16).
2. **Uma pessoa opera.** Dois níveis de urgência e só dois: *acorda* (indisponibilidade, erro em massa, dado em risco) e *amanhã* (saturação que cresce, custo, fila parada). Nada de escalonamento, rodízio ou ferramenta de plantão enquanto o time é de um.
3. **Dado de menor não entra em sinal operacional.** Log, métrica, *trace* e relato de erro carregam identificadores opacos (`estudante_id`, `escola_id`, `sessao_id`), nunca nome, CPF, *username*, e-mail ou texto livre. É a regra 6 estendida da telemetria para tudo o que sai do processo — e ela precisa ser **mecânica** (*redaction* no *logger*), não convenção.
4. **Nativo do Google Cloud antes de peça nova.** O contrato com o Terraform (Capítulo 6) já proíbe serviço sem necessidade demonstrada. Cloud Logging, Cloud Monitoring, Cloud Trace e Error Reporting já recebem o que o Cloud Run emite; a franquia gratuita cobre um contrato; e um contrato a mais não é uma instância a mais para operar. Uma *stack* auto-

hospedada só entra quando algo que ela faz e o nativo não faz for demonstrado, e isso não aconteceu.

5. **OpenTelemetry é a fronteira.** A instrumentação da API e do hub fala OpenTelemetry (OpenTelemetry Authors, 2026); o destino é o Cloud Trace/Monitoring *hoje*. Trocar de destino (uma rede que exija ferramenta própria, um SIEM, um Grafana) é trocar o exportador, não a instrumentação — o mesmo raciocínio do *provider* de IA (**ADR-07**) e do OIDC genérico (**ADR-03**).
6. **Por contrato, e visão única para quem opera.** Log e métrica ficam no projeto do contrato (isolamento da rede, mesmo argumento da *stack* por contrato). A pessoa que opera vê todos os contratos por um *metrics scope* do Cloud Monitoring num projeto de observação sem recurso nenhum — métrica atravessa projeto de graça; log **não** atravessa, de propósito.
7. **Custo previsível e declarado.** Cada sinal tem volume estimado e franquia ao lado (Seção 17.7). Sinal cujo custo cresce com o tráfego sem teto (log de requisição a 100%, *trace* sem amostragem) é amostrado *na origem*, não cortado na fatura.

17.3 Os sinais, e onde cada um nasce

17.3.1 Log estruturado

O *stdout* do Cloud Run já é o transporte: uma linha que seja JSON válido é lida pelo agente de log como `jsonPayload`, e três campos com nome especial viram metadado de primeira classe (Google Cloud, 2026b): `severity` (o nível), `logging.googleapis.com/trace` (o identificador que junta esta linha ao *trace* e ao log de requisição do próprio Cloud Run) e `logging.googleapis.com/spanId`. Um erro com *stack trace* e `severity: ERROR` é agrupado pelo Error Reporting sem mais nada.

> **O que muda na API**
>
> - `nestjs-pino` substitui o `ConsoleLogger`: `pino` em JSON, com `level` mapeado para `severity` (`warn` → `WARNING`, `error` → `ERROR`, `fatal` → `CRITICAL`) e `messageKey: "message"`. Em desenvolvimento, `pino-pretty`; em produção, JSON cru. Uma dependência de produção e uma de desenvolvimento.
> - **Contexto por requisição** no mesmo `AsyncLocalStorage` da *tenancy* (`tenancy-context.ts`): `escolaId`, `perfil`, `requestId` e `trace`. O *logger* filho lê dali, e toda linha emitida dentro de um *handler* sai com os quatro campos sem que quem escreve o log precise lembrar deles.
> - **Correlação**: o `trace` vem do cabeçalho `traceparent` (W3C (W3C, 2021)) quando existe, e do `X-Cloud-Trace-Context` que o *load balancer* injeta quando não. O valor gravado é `projects/{id}/traces/{traceId}`, que exige o identificador do projeto: entra como `GCP_PROJECT_ID`, injetada pelo Terraform (Apêndice B).

- ***Redaction* mecânica**: a opção `redact` do `pino` apaga, por caminho, `authorization`, `cookie`, `token`, `senha`, `cpf`, `username`, `email`, `nome` e qualquer campo de texto livre (`texto`, `mensagem`, `transcricao`, `observacao`) em qualquer profundidade. É a regra 6 saindo da convenção e entrando na ferramenta; o `username` do `auth.controller.ts` deixa de vazar por construção.
- **Um `ExceptionFilter` global**: `HttpException` mantém o corpo de hoje; erro não previsto (inclusive `ZodError` que escapou) vira 500 com `{ codigo: "erro-interno", requestId }` no corpo e **uma** linha `ERROR` com *stack* e contexto no log. O `requestId` no corpo é o que o professor lê na tela e o que quem opera cola no Logs Explorer — é a ponte entre o relato e o registro.
- **Log de requisição da aplicação: não.** O Cloud Run já grava um por requisição, com método, rota, *status*, latência e `trace`. Duplicá-lo dobraria o volume do sinal mais caro da lista para acrescentar zero informação. O que a aplicação loga é o que o Cloud Run não sabe: o `escolaId`, o motivo de um 4xx, a decisão de degradar.
- **Prisma**: `log: [{ emit: "event", level: "query" }]` ligado só quando `LOG_LEVEL=debug`, e um *listener* que emite `WARNING` para *query* acima de 500 ms em qualquer nível. A *preview feature* `metrics` do Prisma (contadores de pool e histograma de *query*) alimenta a Seção 17.3.2.

No hub, o mesmo `pino` (sem o módulo do Nest) em `lib/log.ts`, usado por `apiDoServidor`, pelos *route handlers* e pelo *middleware*. `apiDoServidor` **continua devolvendo `null`** para não quebrar as páginas que dependem disso, mas passa a registrar `WARNING` com rota, *status* e `requestId` antes de devolver — e ganha `AbortSignal.timeout(10000)`, porque uma API pendurada não pode pendurar o hub junto. O 404 silencioso da *allowlist* do BFF vira `WARNING` com o prefixo recusado: é o sintoma de uma tela nova que esqueceu de pedir passagem.

REGRA INVIOLÁVEL

Exclusão na origem, não na fatura. Dois filtros de exclusão no *sink* `_Default` de cada contrato, criados pelo Terraform: (1) o log de requisição do Cloud Run para `/healthz`, `/readyz` e os *user agents* dos *uptime checks*; (2) amostragem de 20% (`sample(insertId, 0.2)`) do log de requisição com *status* 2xx e latência abaixo de 1 s. Requisição com erro, ou lenta, fica em 100%. Log excluído não é ingerido e não é cobrado, e é o que mantém o sinal mais volumoso dentro da franquia (Seção 17.7).

17.3.2 Métricas

Três origens, do mais barato para o mais informativo:

Origem	O que dá	Custo e esforço

Nativas do Google Cloud	Cloud Run: `request_count` por *status*, `request_latencies` (distribuição), `instance_count`, CPU, memória, latência de *startup*. Cloud SQL: conexões, CPU, disco, *replication lag*, *failover*. Pub/Sub: `oldest_unacked_message_age`, `num_undelivered_messages`, `dead_letter_message_count`. LB: contagem e latência por *host* e classe de *status*; CDN: taxa de acerto.	**Grátis, zero código.** É a base de todos os alertas da fase 0.
Métricas de log	Um contador ou distribuição derivado de um filtro do Cloud Logging: "linhas com `codigo=recurso-auditoria-nao-gravada`" — a mudança do painel valeu mas o histórico dela não foi escrito (**ADR-89**: o registro nunca derruba a operação que o chamou, e é por isso que este contador existe); "linhas com `codigo=recurso-alterado`" — toda vez que a Unooria liga ou desliga um produto numa rede (**ADR-87**), com recurso, escopo, estado e motivo; "linhas com `codigo=ia-quota-alterada`" — o teto diário de IA de um perfil mudou (**ADR-105**), com `de`, `para` e motivo, e `ia-quota-sem-linha` quando a api está aplicando a constante do código porque a linha de `ia_quotas` não existe; "linhas com `codigo=telemetria-descartada`", "`severity=ERROR` da revisão X", "execução do *job* `-migrate` com falha".	As *de sistema* são grátis; as *definidas pelo usuário* contam na franquia de métrica. Zero código: é um filtro no Terraform.

Métricas da aplicação (OpenTelemetry)	O que só o processo sabe: **IA** — latência por modelo e rota (histograma), *tokens* de entrada/saída, `finishReason`, `voltas`, `tentativas`, recusas de quota; **telemetria** — lotes recebidos, eventos publicados, gravados e descartados por motivo; **fila** — geração e planos por `status`, idade do mais antigo em `gerando`; **pool do Prisma** — conexões em uso e espera; **ponte** — *handshakes* concluídos e expirados (via hub).	Exportadas a cada 60 s pelo SDK OpenTelemetry — **por OTLP/JSON direto para a Telemetry API do Google** (`exportador-de-metricas-otlp.ts` não pelo `@google-cloud/opentelemetry-clo` medido em 2026-09-02, ele arrasta o pacote `googleapis` inteiro, 202 MB em `node_modules`, para chamar um método. Cerca de 60 séries por contrato: cabe na franquia de 150 MiB com folga. Duas coisas medidas em `dev` em 2026-09-02: **o `resource` precisa carregar `gcp.project_id`** (`observabilidade/recurso.ts`) — sem ele a Telemetry API recusa *todo* lote com `HTTP 400`, a cada 60 s, com log e *trace* funcionando, e o `gcpDetector` não o preenche; e **a ingestão OTLP grava no domínio `prometheus.googleapis.com`**, não em `workload.googleapis.com` (onde o exportador legado gravava). A série sai como `prometheus.googleapis.com/unooria.ia.latencia/histogram`, no recurso monitorado `prometheus_target` — `job` = `service.name`, `instance` = a instância do Cloud Run, `location` = a região, tudo preenchido pelo

ATENÇÃO

Cardinalidade. `escola_id` **não** é rótulo de métrica: cem escolas multiplicam cada série por cem, e a franquia acaba na primeira semana. Escola vai para o log (que se filtra) e para a telemetria (que se agrega em SQL). Rótulos de métrica são conjuntos **fechados** e pequenos: rota, perfil, modelo, `finishReason`, motivo de descarte, *status* de fila. A mesma regra que a telemetria aplica ao vocabulário do *payload* (**ADR-11**).

17.3.3 Rastros distribuídos

Um *trace* responde à pergunta que nem log nem métrica respondem: *onde* os 1.800 ms de uma abertura de missão foram gastos — no hub, no BFF, na API, no *lock* do *feed*, no Vertex. A instrumentação automática do OpenTelemetry para Node cobre HTTP de entrada e saída (`undici`, portanto o `fetch` do Vertex e o do hub), Express (o Nest), `pg` (o Prisma) e Pub/Sub, sem tocar no código dos módulos. No hub, o `instrumentation.ts` do Next (Vercel, 2026a) registra o mesmo SDK, e o `traceparent` atravessa do navegador ao Cloud SQL.

DECISÃO DE PROJETO

Exportador direto, sem *sidecar* de coletor. O Google recomenda um OpenTelemetry Collector como segundo contêiner do serviço Cloud Run (Google Cloud, 2026a). Ele faz sentido quando há vários destinos, ou quando se quer *tail sampling* (guardar só os *traces* com erro, decidido no fim). Nenhuma das duas condições existe hoje, e o *sidecar* é mais um contêiner com CPU, memória, versão e falha própria em cada serviço de cada contrato. A API e o hub exportam direto, com ADC (a mesma credencial de *service account* que já autentica o Vertex), pelos exportadores `@google-cloud/opentelemetry-cloud-trace-exporter` e `-cloud-monitoring-exporter`. O *sidecar* é o gatilho de reavaliação registrado na **ADR-57**.

Atualização de 2026-09-02: os dois exportadores citados acima foram DEPRECIADOS pelo Google, com arquivamento marcado para 30 de outubro de 2026 — antes da entrada em produção. O que a plataforma usa hoje são os exportadores *oficiais* do OpenTelemetry sobre OTLP (`exporter-trace-otlp-http` e `-metrics-otlp-http`), apontados para a Telemetry API (`telemetry.googleapis.com`), com a credencial ADC entregue por uma **função assíncrona** em `headers` — o *token* expira em uma hora, e cabeçalho estático pararia de exportar sozinho. **O destino não mudou**: Cloud Trace e Cloud Monitoring. Mudaram o transporte e o domínio em que a métrica aparece.

Amostragem: `ParentBased` sobre `TraceIdRatioBased(0.02)` — 2% das requisições, com uma diferença deliberada em relação ao *default* do SDK (implementada em 2026-09-02, `observabilidade/sample`
o pai remoto não decide. O `traceparent` chega da internet — da SDK no navegador, do hub, de um `curl` — e honrar `sampled=01` deixaria qualquer cliente forçar 100% de amostragem, que é custo. Como a decisão por razão é determinística no `traceId`, hub e API com o mesmo *ratio* tomam a *mesma* decisão sem confiar um no outro; o pai *local* (span filho no mesmo processo)

segue o pai, como sempre. O identificador de *trace* existe em **100%** das requisições (ele vem do *load balancer*, e com o SDK ligado o span de servidor existe mesmo não amostrado), então todo log de erro carrega um `trace` para pesquisar mesmo quando os *spans* não foram guardados; o que a amostragem decide é só quanta *linha do tempo* se armazena. Rotas de IA (`/ai`, `/unoogpt`, `/planos`, `/bemestar`) e `/runs/:id/feed` usam dez vezes o *ratio*, até 100%: são as em que a latência é o produto. `/healthz` e `/readyz` não viram *span*: o *uptime check* bate 4.320 vezes por dia.

No hub a instrumentação é **menor de propósito**: o Next já emite os *spans* de servidor e já extrai o `traceparent` de entrada assim que existe um `TracerProvider` global (medido: um `traceparent` forjado na requisição sai como `traceId` de todos os *spans* do *render*); instalar o `instrumentation-http` ali duplicaria o *span* raiz. Só o `undici` entra, para o `fetch` de saída à API levar o cabeçalho. O gate é o mesmo nos dois processos: `OBS_TRACE_RATIO` $>$ 0 **e** (`GCP_PROJECT_ID` ou `OBS_DEBUG=1`, que imprime os *spans* no console sem GCP). Desligado, nenhum objeto do SDK é criado — é o que mantém o Jest e o `nest start --watch` sem GCP intocados.

17.3.4 Erros

Erros de servidor chegam ao Error Reporting pelo log estruturado, sem biblioteca: linha com `severity: ERROR` e *stack trace* é agrupada por assinatura, com contagem, primeira e última ocorrência e a revisão do Cloud Run em que apareceu. “Erro novo nesta revisão” é o alerta de *deploy* mais barato que existe.

Erros de **cliente** são o problema que a Figura 17.1 mostra em laranja no primeiro elo, e a escolha aqui é a **ADR-58**: a experiência **não** carrega SDK de terceiro. Um *bundle* público que carrega um *DSN* do Sentry é um *bundle* que manda o *user agent*, a URL, a *stack* e o que mais o erro arrastar para um processador fora do contrato e do DPA da rede — para um erro que aconteceu na tela de uma criança. O caminho é a API, que já tem o *token*, a escola e a regra 6 (Figura 17.2).

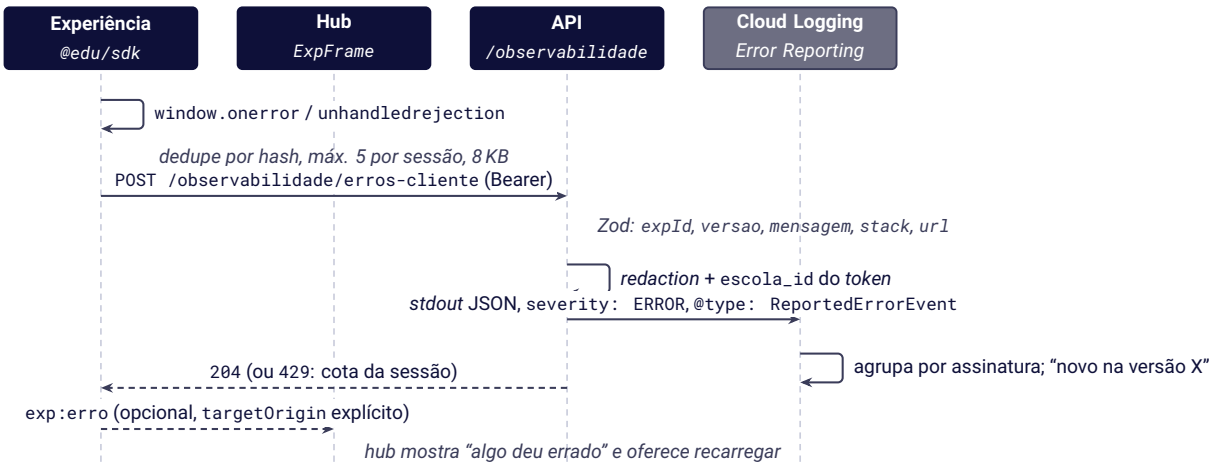


Figura 17.2 – Um erro não capturado dentro do `iframe` chega ao Error Reporting pela API, nunca por um serviço de terceiros: a API já tem o *token*, a escola e a *redaction*. A mensagem `exp:erro` da ponte é opcional e só serve para o hub trocar a tela; o registro não depende dela.

Três detalhes que decidem se isso funciona ou vira ruído. **Cota por sessão**: um laço de erro numa experiência gera milhares de eventos por segundo; a SDK manda no máximo cinco por sessão, com *dedupe* por *hash* de mensagem e *stack*, e a API responde 429 acima disso, por `(escola, usuario)`. ***Sourcemaps* privados**: o *build* das experiências gera `build.sourcemap: "hidden"` (o *bundle* continua sem o comentário que aponta para eles), e o CI sobe os `.map` para um *bucket* **separado e privado**, `gs://{project}-exp-mapas/{expId}/{versao}/`. Não é a "pasta `mapas/` com ACL privada no mesmo *bucket*" que a primeira versão deste texto propôs: o *bucket* dos *bundles* tem acesso uniforme (*uniform bucket-level access*, medido no Terraform), e nesse modo ACL por objeto não existe — qualquer objeto ali é público. A *stack* minificada é legível com o mapa da versão exata (`scripts/mapear-stack.mjs`), e a versão já está no caminho. **Nada de texto do estudante**: o *payload* é um *schema* fechado; `mensagem` e `stack` passam pela *redaction* da API e têm tamanho máximo.

No hub, `error.tsx` e `global-error.tsx` fazem o equivalente para erro de *render*: registram pelo `pino` do servidor (o `global-error` do cliente posta na mesma rota) e mostram a tela de erro do *design system* com o `requestId`.

17.3.5 Experiência do usuário real e sondas sintéticas

Web Vitals. O `useReportWebVitals` do Next envia LCP, INP e CLS das telas do hub — amostrados a 10%, sem identificador de pessoa, com `perfil` e rota — para a mesma rota da API, que os emite como distribuição OpenTelemetry. Nas experiências, o mesmo pacote `web-vitals` (4 KB) entra pela SDK. É o único sinal de que "a plataforma está lenta na escola X" que não depende de alguém reclamar — e rede pública tem escola com conectividade ruim, o que é exatamente onde o problema aparece e o *trace* do servidor não vê.

Uptime checks do Cloud Monitoring, de três regiões, a cada 60 s, criados pelo Terraform, com validação de **conteúdo** e não só de *status*:

Alvo	O que um único *probe* prova
`GET {hub}/.well-known/jwks.json` contém `"keys"`	DNS, TLS, LB, hub de pé, chave de assinatura carregada. É o *probe* de maior valor: sem ele nenhum *token* de experiência é validável.
`GET {api}/readyz` = `{"ok":true}`	API de pé **e** Cloud SQL respondendo, pela VPC. Hoje ninguém chama esta rota; passa a ser chamada 4.320 vezes por dia.
`GET {hub}/api/healthz`	Rota nova do hub, sem dependência: processo vivo. Separa "hub caiu" de "API caiu" no alerta.
`GET {exp}/exp08ef1b/{versao}/index.html`	CDN e *bucket* servindo a versão ativa. A versão vem do manifesto, então o *check* é regerado no passo *ativar* do *deploy*.
`GET {hub}/` contém o texto da *landing*	O *render* do Next funciona, não só o processo.

A suíte e2e vira monitor sintético. É o ativo mais valioso do levantamento e o que menos custa aproveitar: `rodar.mjs publico` já é "o canário mais barato que existe" (nas palavras do próprio arquivo), e cada suíte já sai com *exit code* honesto e relatório classificado. Faltam três coisas, todas fora do código da suíte: (1) um *workflow* do GitHub Actions com `schedule` (a cada 30 minutos em horário escolar, a cada 2 horas fora dele) rodando `publico` e `estudante` contra `hml` com contas de sonda; (2) o mesmo *workflow* chamado como *smoke* ao fim do `deploy.yml`, com `if: failure()` notificando; (3) o resultado convertido em métrica: `gcloud monitoring` escrevendo uma série `e2e/suite_ok` por suíte, para que um alerta do Cloud Monitoring trate a suíte como qualquer outro sinal. O GitHub Actions dá 2.000 minutos por mês no plano *Free*; a suíte pública leva menos de dois minutos, o que cabe.

> **NOTA**
>
> Os *synthetic monitors* do Cloud Monitoring (Playwright rodando numa Cloud Function) fariam o mesmo dentro do projeto, mas custam US$ 1,20 por mil execuções com só cem grátis — e a suíte já existe, com a lógica de espera que corre contra o erro. Reescrevê-la para outra plataforma seria pagar duas vezes pela mesma prova.

17.3.6 Banco, mensageria e borda

Cloud SQL: *Query Insights* ligado no Terraform (grátis) dá a lista das *queries* mais caras por *hash*, com plano; a extensão `pg_stat_statements` é o mesmo dado por SQL, para o *checklist* do Apêndice F. A sonda de *tenancy* entra aqui também: uma métrica de log que conta `WARNING` de *query* fora de `comTenancy` (a RLS devolve zero linhas em silêncio, e o Apêndice F já registra dois incidentes disso).

Pub/Sub: a *subscription* de telemetria ganha `dead_letter_policy` para um tópico `{contrato}-telemetria-dlq` depois de cinco tentativas. Hoje, um lote que a API recusa (401 por *env* ausente, 400 por envelope inválido) é reentregue para sempre — e é invisível. Com a *dead letter*, ele vira uma contagem, e a contagem vira alerta.

Load balancer e CDN: log do LB ligado com amostragem de 10% (suficiente para taxa de erro por *host*), e uma métrica de acerto de CDN por *bucket*. Uma queda na taxa de acerto do *bucket* de *bundles* depois de um *deploy* é o sintoma de *index.html* sem `Cache-Control` ou de caminho publicado sem subir a versão — as duas armadilhas que o `AGENTS.md` da raiz já descreve.

17.3.7 Camada de IA

A IA é o custo variável da plataforma e o sinal de que menos se sabe hoje. O `AiProvider` ganha um invólucro de medida (`ai/medidor.provider.ts`, entregue em 2026-09-02) — **dentro** de `apps/api/src/ai`, o único lugar onde uma chamada de IA acontece, envolvendo o *provider* real na *factory* do módulo, de modo que nenhum consumidor sabe que ele existe — que emite, por chamada: latência (histograma por modelo e rota), *tokens* de entrada e saída, `finishReason`, `voltas`, `tentativas` e o resultado (`ok`, `erro-provider`, `quota`). O `QuotaService` emite as recu-

sas por perfil. Custo é métrica *derivada*: uma tabela de preço por modelo em `catalogo-modelos.ts`, multiplicada no painel, não no processo — preço muda sem *deploy*, e a métrica de *tokens* é a que vale.

Três alertas saem daqui: erro do *provider* acima de 10% em 5 minutos (*acorda*, se em horário escolar: é a exp02em1b inteira), latência p95 acima de 30 s (*amanhã*: é o `R-13` materializado), e consumo do dia acima de duas vezes a média móvel de 7 dias (*amanhã*: é custo ou laço). O `R-14` mediu uma partida em 71.397 *tokens* à mão; com isto, a medida é contínua.

O agente de IA do professor não passa pelo medidor (**ADR-126**): o modelo é chamado de dentro do Agent Engine. O sinal dele é **uma linha de log por turno**, emitida no `finally` do *service*, inclusive no abandono:

Campo de	Conteúdo
`resultado`	`fim` | `erro` | `abortado` (o professor apertou "Parar", trocou de conversa ou fechou a aba) | `tempo` (o teto de 240 s).
`falha`	No `erro`, só o *status* HTTP do engine (`http-503`), o código do evento de erro do ADK, `sessao-inexistente` ou o NOME do tipo de erro — nunca o corpo da resposta, que pode ecoar a pergunta.
`conversaId`, `modo`	O id da linha da posse e o modo do cliente (`agent-engine`, `local`, `mock`).
`duracaoMs`, `tokens`, `eventos`	Duração do turno, soma dos *tokens* dos eventos não parciais (o que a quota consumiu) e quantos eventos o engine mandou.

Ao lado dela, os códigos de exceção: `agente-professor.indisponivel` (o engine não respondeu), `.sessao-sumiu`, `.renovacao-falhou` (a SA sem `sessions.update`, ou o engine fora), `.linhas-descartadas` (linha do *stream* que não era JSON, só a contagem), `.quota-nao-consumida`, `.conversa-nao-atualizada` e `.turno-com-erro`. **O que NUNCA vai a log**, na api e no agente: a pergunta, a resposta, o título, o `texto` da busca e o corpo de erro do engine — só ids, contagens, códigos, duração e *tokens*. O professor pode escrever o nome de um estudante; o `test_plugins.py` do agente confere o lado Python.

17.3.8 Entrega

O `deploy.yml` ganha três passos que hoje não existem: um *smoke* pós-*deploy* (o *workflow* da suíte, acima), notificação de falha (`if: failure()` para o mesmo canal dos alertas) e uma **anotação de *deploy*** — uma linha de log `codigo=deploy`, com SHA, revisão e versão de cada experiência ativada, que vira marcador vertical nos painéis. "O erro começou às 14h07" ao lado de "a revisão 00042 recebeu tráfego às 14h06" é metade de um diagnóstico.

17.4 Ferramentas: comparação e escolha

A comparação é por função, não por fornecedor. Em cada linha, a escolha recomendada vem primeiro; as alternativas trazem a condição em que passariam a valer.

Função	Escolha	Custo	Alternativas e quando
Log	**Cloud Logging**, via *stdout* JSON do Cloud Run	50 GiB/projeto/mês grátis; US$ 0,50/GiB além; 30 dias inclusos	*Grafana Loki* auto-hospedado: uma VM, retenção e *backup* por conta própria, e o log sairia do projeto do contrato. *Grafana Cloud Free* (50 GB, 14 dias): idem quanto à saída do projeto. *OpenObserve* ou *SigNoz*: só se o nativo faltar em consulta (não falta: Logs Explorer e Log Analytics em SQL).
Métricas e painéis	**Cloud Monitoring**; painel por contrato em JSON no Terraform; *metrics scope* central	Métricas do Google grátis; custom: 150 MiB grátis, US$ 0,258/MiB além	*Prometheus + Grafana* auto-hospedados: um coletor por serviço (*scrape* não atravessa Cloud Run sem *sidecar*) e uma instância central para N contratos. *Grafana Cloud Free* (10 k séries) como **painel** sobre a fonte de dados Cloud Monitoring, se a pessoa preferir a ergonomia do Grafana — é leitura, não muda a coleta. Entra quando houver mais de um operador com opinião sobre painel.
Rastros	**Cloud Trace** pelo SDK OpenTelemetry + exportador Google	2,5 M *spans*/mês grátis; US$ 0,20/M além	*Grafana Tempo* ou *Jaeger*: exigem coletor e armazenamento. Entra se uma rede exigir retenção de *trace* acima de 30 dias (Cloud Trace guarda 30).
Erros	**Error Reporting** pelo log; erro de cliente **pela API** (ADR-58)	Grátis	*Sentry SaaS* (Developer: 5 k erros/mês, 1 usuário; Team: US$ 26/mês): *session replay* e *breadcrumbs* são melhores, mas o dado sai do contrato — só com DPA que o cubra. *GlitchTip* ou *Sentry* auto-hospedados: fala o protocolo do Sentry dentro do projeto; entra se o agrupamento do Error Reporting se mostrar insuficiente para 48 experiências.

Função	Escolha	Custo	Alternativas e quando
Analytics de produto	**Microsoft Clarity**, no hub e dentro do *bundle* (**ADR-60**)	Grátis (teto de 100.000 gravações/dia por projeto)	*PostHog* ou *Matomo* auto-hospedados: ficariam dentro do projeto do contrato, e é o caminho se o *replay* em processador de terceiro não passar no DPA de uma rede. *Hotjar*, *FullStory*: pagos, mesmo problema de residência. **Não substituem** a telemetria própria — ela é que corrige, pontua e vira relatório de professor; o Clarity só mostra a tela.
Sondas	**Uptime checks** do Cloud Monitoring + **suíte e2e** agendada no GitHub Actions	1 M execu-ções/mês grátis; Actions: 2.000 min/mês grátis	*Uptime Kuma* auto-hospedado: bonito, mas uma VM a mais para fazer o que a franquia já faz. *Synthetic monitors* do Cloud Monitoring: US$ 1,20/mil execuções; reescreveria a suíte. *Checkly*, *Grafana Synthetic*: pagos, só com vários operadores.
Roteamento de alerta	**Canais nativos**: e-mail + *push* do app Cloud Console (acorda) e Google Chat/Slack (amanhã)	Grátis; a cobrança anunciada de política (Seção 17.7)	*ntfy* auto-hospedado ou o plano grátis do *Pushover*: *push* sem depender do app do Google. *Grafana OnCall* (grátis até 3 usuários) ou *PagerDuty*: escalonamento e rodízio — **só faz sentido com o segundo operador**.
Teste de carga	**k6** (livre), da máquina de quem opera, contra `hml`	Grátis	*Grafana Cloud k6*: 500 VUh/mês grátis, executores distribuídos — útil se 4.000 usuários virtuais não saírem de um só *laptop*.
APM completo	—	—	*Datadog*, *New Relic*, *Dynatrace*: por *host* ou por GB, na casa de centenas de dólares/mês por contrato. Entram quando houver vários contratos em várias nuvens ou um time de operação com SLA formal — e a instrumentação OpenTelemetry já estará pronta para eles.

> **Onde o pago se justifica hoje**: em nada. **Onde se justifica no primeiro contrato em produção**: num canal de *push* confiável para o alerta *acorda* — o app do Cloud Console serve, e um *Pushover* custa US$ 5 uma vez. **Onde se justificará com o segundo operador**: rodízio de plantão (Grafana OnCall grátis; PagerDuty se houver SLA contratual). **Onde nunca, por desenho**: SDK de **erro** de terceiro dentro de um *bundle* de experiência.
> **E onde a regra acima abriu exceção**: *analytics de produto*. A **ADR-60** pôs o Microsoft Clarity no hub e dentro do *bundle*, porque o `iframe` é cross-origin e gravar só o hub deixaria de fora exatamente a tela em que o estudante passa a sessão. A distinção que

> sobra é estreita e vale dizê-la: erro de cliente continua entrando pela API, com *redaction* e `escola_id` do *token*; o que sai para terceiro é gravação de sessão, com identificador opaco e sem a resposta do estudante.

17.5 SLIs, SLOs e a lista de alertas

Um SLO é uma promessa numérica sobre um sintoma, e o alerta é a taxa com que a promessa está sendo consumida (BEYER et al., 2018). Os SLOs abaixo são por contrato, com janela de 30 dias, medidos no Cloud Monitoring (que tem *service monitoring* nativo para Cloud Run, com alerta de *burn rate* pronto). A meta de 99,5% dá 3 h 36 min de orçamento de erro por mês — honesto para uma pessoa e um Cloud Run com `min-instances=1`; 99,9% (43 minutos) só com o segundo operador.

SLI	Definição	SLO
Disponibilidade do hub	Requisições ao `-frontend` sem 5xx, exceto sondas	99,5%
Disponibilidade da API	Requisições ao `-api` sem 5xx, exceto `/ai/*` e `/unoogpt/*` (medidas à parte)	99,5%
Latência da API	p95 de `/runs/:id/feed`, `/experiencias/:id`, `/auth/*` abaixo de 800 ms	99%
Turno de IA	Chamadas ao *provider* concluídas (não `erro-provider`)	98%
Ingestão de telemetria	`oldest_unacked_message_age` abaixo de 5 minutos	99%
Ponte	*Handshakes* `exp:ready` → `hub:init` concluídos em 8 s (via métrica do hub)	99%

A lista de alertas cabe numa página, e isso é deliberado. Cada linha tem um dono (a pessoa), uma severidade e uma ação de primeira resposta que aponta para o Apêndice F.

Sev.	Condição	Por que este e não outro	Primeira resposta
A	*Uptime check* do JWKS ou do `/readyz` falha em 2 de 3 regiões por 2 minutos	Ninguém entra, ou nenhuma experiência valida *token*. É **o** sintoma.	Revisão anterior (`update-traffic`); estado do Cloud SQL
A	*Burn rate* do SLO de disponibilidade (hub ou API) $>$ 14,4 em 1 h (consome 2% do orçamento mensal numa hora)	Erro em massa, independentemente da causa. Substitui "5xx $>$ 1%": a mesma taxa às 8h e às 23h significa coisas diferentes.	Error Reporting: o erro novo da revisão

Sev.	Condição	Por que este e não outro	Primeira resposta
A	Erro do *provider* de IA > 10% em 5 min, entre 7h e 19h	A exp02em1b inteira depende disso, e a sala espera.	Quota do Vertex; catálogo do modelo (`R-12`); `error.cause`
A	`instance_count` da API = `max-instances` por 5 min	Saturação: a próxima requisição espera na fila do Cloud Run.	Subir `max`; olhar p95 e pool do Prisma
A	Execução do *job* `-migrate` com falha (métrica de log)	O *deploy* parou no meio, com *schema* velho e código novo prontos para divergir.	Não reexecutar às cegas: ler o log do *job*
B	*Burn rate* > 6 em 6 h	Degradação lenta que vai estourar o mês.	Painel; Query Insights
B	Conexões do Cloud SQL > 80% do `max_connections` por 10 min	O pool do Prisma é por instância; escala de instância multiplica conexão.	`connection_limit` na `DATABASE_URL`; *pooler*
B	`oldest_unacked_message_age` > 10 min, ou `dead_letter_message_count` > 0	A telemetria está parando ou sendo recusada — hoje ninguém saberia.	`/telemetry/ingest` no log: 401 é *env*, 400 é envelope
B	Latência p95 da IA > 30 s por 15 min	É o `R-13` acontecendo.	`streamGenerateContent` é a saída conhecida
B	Descartes de telemetria > 1% dos eventos em 1 h (métrica da aplicação)	Uma experiência publicou verbo fora do vocabulário, ou *payload* acima do teto.	Motivo do descarte no rótulo
B	Fila de geração ou de planos com item em `gerando` há mais de 15 min, ou taxa de `erro` > 20% em 1 h	O *worker* vive na API; se ela reciclou, o *job* ficou órfão.	`SELECT` do *checklist*; reciclagem em 5 min é automática
B	Erros de cliente de uma `(expId, versao)` acima de 50 em 10 min, ou erro **novo** nas 2 h após ativação	Uma versão quebrou no navegador da escola.	`publicar-manifesto.mjs --versao anterior`: reversão sem *build*
B	Certificado gerenciado a menos de 30 dias; disco do Cloud SQL > 80%; consumo de IA do dia > 2× a média de 7 dias	Três “amanhãs” clássicos.	Terraform; disco; quem consumiu

Sev.	Condição	Por que este e não outro	Primeira resposta
B	Suíte e2e agendada em vermelho duas vezes seguidas	Uma única falha é ruído de rede; duas é defeito.	Relatório da suíte diz qual tela

Quatorze políticas, seis de severidade A. Os "A" vão para e-mail e *push* (app do Cloud Console); os "B" vão para e-mail e um canal de Google Chat ou Slack, e são lidos de manhã. Nenhum alerta de CPU, memória ou latência de *startup*: são causas, e ficam no painel.

17.6 Privacidade nos sinais

Os sinais operacionais carregam dado de menor com mais facilidade que a telemetria, porque nascem em pontos que ninguém desenhou para isso: uma mensagem de exceção que embute o valor recebido, uma URL com *query string*, um *stack trace* de cliente com o estado da tela. As regras:

Ponto	Regra
Log	*Redaction* mecânica por caminho no `pino` (lista acima), nos dois processos. Mensagem de `ZodError` nunca vai inteira ao log — só `path: code`, como o `ContratoInterceptor` já faz. Identificador de pessoa em log é sempre o UUID opaco.
Trace	Atributos automáticos incluem a URL: o *hook* `ignoreIncomingRequestHook` exclui sondas e o `urlFilter` remove *query string*. Nenhum atributo com corpo de requisição.
Erro de cliente	*Schema* fechado, tamanho máximo, *redaction* na API, sem *cookie*, sem `localStorage`, sem *DOM*. O *stack* é o único texto livre, e ele é código, não dado.
Web Vitals	Sem identificador de pessoa; `perfil` e rota apenas.
Session replay	O único sinal que sai do projeto do contrato (**ADR-60**). `identify` manda o UUID opaco — nunca nome, nunca e-mail, nunca o `friendlyName` da API do Clarity. Escola e turma vão por `id`. Do `track()` sai só `verbo:objeto`: `resultado` e `contexto` carregam resposta de estudante e ficam na telemetria própria. O mascaramento de campo de entrada, número e e-mail é o *default* do Clarity e não foi afrouxado.
Retenção	Log do `_Default`: 30 dias (padrão, sem custo). Um *bucket* de auditoria separado, alimentado por *sink* das linhas `codigo=auth.*`, `codigo=importacao.*`, `codigo=deploy` e dos *Admin Activity audit logs*: 400 dias. É o que uma rede pública pergunta num incidente, e é barato porque é pequeno.

Residência	O *bucket* de log de cada contrato é criado na região do contrato (`us-east1` no piloto), não em `global`. Log não sai do projeto: o *metrics scope* central vê métrica, não log.
Acesso	Quem lê log de produção lê dado operacional de uma rede pública: IAM por pessoa, `roles/logging.viewer` por projeto, e os *Data Access audit logs* do próprio Cloud Logging ligados — quem leu o log também fica em log.

17.7 Custo estimado por contrato

Volumes estimados para um contrato em regime: dez mil estudantes, vinte dias letivos, cerca de 40 milhões de requisições por mês entre hub e API (pico de 4.000 simultâneos às 8h), e os preços de referência do Google Cloud Observability em setembro de 2026 (Google Cloud, 2026c). Os preços mudam; as **proporções** são o que importa.

Sinal	Volume/mês	Franquia	Custo estimado
Log de requisição do Cloud Run, **sem** exclusão (≈1 KB por linha)	≈40 GiB	50 GiB	US$ 0 — mas a 80% da franquia, sem folga para o log da aplicação nem para o do LB
Idem, **com** exclusão de sondas e amostragem de 20% dos 2xx rápidos	≈12 GiB	idem	US$ 0, com 38 GiB de folga
Log da aplicação (`WARNING`+, eventos de negócio, ≈0,5 KB)	≈1–2 GiB	idem	US$ 0
Log do LB a 10%	≈4 GiB	idem	US$ 0
Traces a 2% (20% em IA e *feed*), ≈8 *spans* cada	≈8 M *spans*	2,5 M	≈US$ 1,10
Métricas da aplicação, ≈60 séries a 1 ponto/min	≈20 MiB	150 MiB	US$ 0
Métricas de log definidas pelo usuário (6)	≈2 MiB	idem	US$ 0
Uptime checks: 5 alvos × 3 regiões × 60 s	≈657 k execuções	1 M	US$ 0
Políticas de alerta: 14, ≈16 referências de métrica	—	—	≈US$ 5,60 quando a cobrança anunciada (US$ 0,35 por referência por mês) entrar em vigor; US$ 0 até lá
Retenção estendida do *bucket* de auditoria (≈0,5 GiB × 13 meses)	—	—	≈US$ 0,07

Error Reporting, Query Insights, painéis, *metrics scope*	—	—	US$ 0
Suíte e2e agendada no GitHub Actions (≈1.400 min)	—	2.000 min	US$ 0 (plano *Free*, repositório privado)
Total por contrato			**≈US$ 1 hoje; ≈US$ 7 com a cobrança de alerta**

NOTA

Para comparar: uma VM `e2-small` para um Grafana + Loki + Prometheus auto-hospedados custa cerca de US$ 15 por mês antes do disco e do *backup* — **por instância**, e ela teria de ser central (senão é uma por contrato) e receber dados de N projetos por rede e IAM próprios. O nativo custa menos que isso *no pior caso*, e a diferença não é o dinheiro: é a peça que não existe para ser atualizada às 2h da manhã. Os dois riscos de custo do nativo são conhecidos e têm teto: log de requisição sem exclusão (por isso a exclusão é do Terraform, não de quem lembra) e cardinalidade de métrica (por isso `escola_id` não é rótulo).

17.8 Roteiro de implantação

Quatro fases. A fase 0 é a que fecha o `R-05` antes de novembro de 2026 e **não toca em código de aplicação**: é Terraform e um *workflow*. As fases seguintes são incrementais e cada uma tem uma prova de pronto que a suíte ou o painel exibe.

Fase	Entrega	Onde	Prova de pronto
0 antes de nov./2026	Canais de notificação; 5 *uptime checks*; as 14 políticas (as que dependem de métrica da aplicação entram na fase 2); SLOs de disponibilidade; painel por contrato; filtros de exclusão; *bucket* de auditoria e Data Access audit logs do Logging; *dead letter* da telemetria; Query Insights; sondas HTTP (`startup` em `/healthz`, `liveness` em `/healthz`) no recurso Cloud Run; *metrics scope* central. *Smoke* pós-*deploy* e `if: failure()` no `deploy.yml`; suíte `publico`+`estudante` agendada contra `hml`.	Repositório de Terraform (`docs/INFRA.md` ganha a seção); `.github/workflows/` (*a parte deste repositório entregue em 2026-09-02*, `ci/obs-sonda-e-mapas`: sonda pós-*deploy*, *job* `smoke` da *landing*, mapas no *bucket* privado; no Terraform, APIs, papéis, *envs*, *bucket* de mapas, sondas HTTP no módulo e os *uptime checks* com alerta por e-mail — escritos e validados, aguardando `apply`. A suíte agendada como monitor sintético ficou para uma onda posterior, por decisão de 2026-09-02)	Derrubar a API em `hml` e receber o *push* em menos de 3 minutos. *Deploy* com migração quebrada notifica.
1 1–2 semanas	**API** (*entregue em 2026-09-02*, `feat/obs-api-log`): `nestjs-pino` com *redaction*, contexto de requisição no ALS, `ExceptionFilter` global, `LOG_LEVEL`, `GCP_PROJECT_ID`, Prisma com aviso de *query* lenta. **Hub** (*entregue em 2026-09-02*, `feat/obs-hub-log`): `lib/log.ts`, `apiDoServidor` logando e com *timeout*, `/api/healthz`, `error.tsx` e `global-error.tsx`, `onRequestError` com o `digest` da tela (o `HEALTHCHECK` do Dockerfile ficou de fora: o Cloud Run o ignora). **SDK**: `visibilitychange`, *backoff* e teto de 500 eventos no *buffer*, retorno do `sendBeacon` verificado.	`apps/api`, `apps/hub`, `packages/sdk`	Um 500 provocado em `hml` aparece no Error Reporting agrupado, com `requestId` igual ao da tela, em menos de um minuto. Teste unitário prova que `username` e `cpf` saem redigidos.

Fase	Entrega	Onde	Prova de pronto
2 2–3 semanas	OpenTelemetry na API e no hub (`instrumentation.ts`), exportador Google, amostragem 2%/20%, `traceparent` do navegador ao banco (*traces entregues em 2026-09-02*, `feat/obs-traces`: `otel.ts` como primeira importação do `main.ts`, `@prisma/instrumentation` no lugar do `pg`, *sampler* por rota que ignora a *flag* do pai remoto). *Métricas da API e as três rotas de `/observabilidade` entregues em 2026-09-02*, `feat/obs-metricas`: `Metricas` injetável, `MedidorDeProvider`, contadores do pipeline de telemetria, proxy OTLP dos *spans* do navegador. *Lado do cliente entregue em 2026-09-02*, `feat/obs-sdk`: OpenTelemetry Web na SDK (`document-load` + `fetch`, exportador OTLP/JSON próprio para a API, *ratio* pelo `config.obs` do `hub:init`, ≈17 KB gz em *chunks* só carregados com *ratio* $>$ 0), captura de erros com dedupe e teto, Web Vitals a 10%, `exp:erro` na ponte, telemetria com teto e *backoff*, *sourcemaps* `hidden` nas sete experiências; no hub, tela “A missão travou”, `/api/ponte` (métrica de log dos *handshakes*), erros, vitais e OpenTelemetry Web do próprio hub pelo BFF (o *ratio* chega por um `<meta>` escrito pelo *layout* a partir de `OBS_TRACE_RATIO_CLIENTE`). Métricas da aplicação: IA, telemetria, filas, pool, ponte. Rota `POST /observabilidade/erros-cliente` e `/observabilidade/vitals`	`apps/api/src/observabilidade/`, `apps/api/src/ai`, `packages/sdk`, `deploy.yml`	Um `throw` provocado na exp04ef1b em `hml` vira erro agrupado por `(expId, versao)` com *stack* mapeada. Um *trace* de abertura de missão mostra hub → API → Postgres → Vertex numa só linha do tempo.

Fase	Entrega	Onde	Prova de pronto
3 condicional	Grafana (Cloud Free ou local) como painel de leitura sobre o Cloud Monitoring; coletor *sidecar* com *tail sampling*; Grafana OnCall; Sentry/GlitTip auto-hospedado; k6 distribuído.	—	Cada item tem gatilho na **ADR-57**. Nenhum começa por conta própria.

> **ATENÇÃO**
>
> A fase 0 depende do repositório de Terraform, que é separado e tem seu próprio ciclo. O que cabe a este repositório é **o contrato**: a lista de recursos que `docs/INFRA.md` passa a esperar (canais, políticas, *checks*, painel, exclusões, *bucket* de auditoria, *dead letter*, sondas do serviço, `GCP_PROJECT_ID`) e o *workflow* da suíte. Sem esse contrato escrito, a fase 0 vira "clicar no console" — que funciona em `dev` e desaparece no segundo contrato.

17.9 Estado da implantação (3 de setembro de 2026)

O que segue não é plano: é o que responde hoje no projeto `entre-redes-dev`, verificado consultando o Cloud Monitoring e o Cloud Logging, e não apenas o código que os declara. A fase 0 do roteiro está fechada em `dev`; `hml` e `prd` recebem o mesmo código pelo mesmo *tfvars*, e esperam apenas o `apply`.

Sinal	Estado	Onde vive, e como se confere
Log estruturado da API	**no ar**	`jsonPayload.codigo` com `requestId`; `GCP_PROJECT_ID` liga a linha ao rastro. Conferido pela sonda `codigo=api-iniciada`
Rastros distribuídos	**no ar**	OpenTelemetry no Express: há *span* por *middleware* e por rota — `GET /unoogpt/professor/turmas/:turmaId` aparece como `RPC_SERVER` no Cloud Trace
Sondas sintéticas	**no ar**	Quatro *uptime checks* a cada 60 s, de três regiões, validando **conteúdo**: `/readyz` da API, `/api/healthz` e o JWKS do hub, e o painel do OpenReplay
Alertas	**no ar**	Doze políticas, todas habilitadas, num canal de e-mail. Detalhe na tabela seguinte
SLOs	**no ar**	Dois *custom services* no Cloud Monitoring, API e hub, com $99{,}5\%$ de requisições sem 5xx em janela rolante de 30 dias
Fila morta da telemetria	**no ar**	`dead_letter_policy` com cinco tentativas na *subscription* de *push*, mais um tópico e uma *subscription* de retenção por 7 dias para inspeção manual

Sinal	Estado	Onde vive, e como se confere
Painel de operação	**no ar**	Um *dashboard* com 5xx, erro interno, p95, instâncias, CPU e disco do banco, fila morta e um cartão por sonda — na ordem em que se investiga
Higiene do log	**no ar**	Exclusão no *sink* `_Default` das sondas e das rotas de saúde, restrita a `severity < WARNING`: erro nessas rotas continua registrado
Error Reporting	**parcial**	API habilitada, mas **zero grupos** em uma semana. Ninguém provou ainda que a linha de erro sai no formato que ele agrupa — as poucas linhas `ERROR` do período foram falha de conexão do próprio Cloud Run, sem *stack*
Erro de cliente	**corrigido em 08/09**	O código existia desde 02/09 e **nunca registrou nada**: o `github.sha` de 40 caracteres estourava o `max(32)` do campo `versao` e a API devolvia 400 a todo erro do hub. O mesmo valor descartava o lote OTLP do hub em `identidadeDoLote` — 425 *spans* em `hml` entre 5 e 7/09, com a rota respondendo **204**. Corrigido pela **ADR-84**, que também abriu o par **público** das rotas (a tela de login não tinha sessão, e o 401 do BFF descartava o sinal de que ninguém conseguia entrar) e passou a CONTAR todo caminho de recusa — a ausência desse contador é a razão de seis dias sem ninguém notar
Session replay	**parcial**	Instância própria de OpenReplay no ar, sem nenhum *tracker* instrumentado — nenhuma sessão gravada
Sondas do Cloud Run	**ausente**	O módulo já aceita `startup_probe_path` e `liveness_probe_path`, e as três rotas de saúde respondem; nenhum serviço as usa ainda

17.9.1 As treze políticas que existem

A lista prevista na Seção anterior tinha quatorze linhas e nasceu de um desenho; doze nasceram do que a métrica nativa permite medir *sem tocar na aplicação* — porque o repositório de infraestrutura evolui sozinho, enquanto o *monorepo* está em obra. A décima terceira, aplicada em 9 de setembro de 2026, é a primeira que lê uma métrica DA APLICAÇÃO, e nasceu de um caso concreto: ver a **ADR-84**.

Sev.	Política	Condição
A	`api-readyz`, `hub-healthz`, `hub-jwks` e `openreplay` fora do ar	Sonda falha em 2 de 3 regiões por 2 min
A	Disco do banco enchendo	Acima de 85%: é o único item da lista que piora com o tempo e não volta sozinho
B	API e hub respondendo 5xx	Mais de cinco respostas 5xx em 5 min, por serviço

Sev.	Política	Condição
B	Erro interno na API	Mais de cinco `codigo=erro-interno` em 5 min — lê o *log da aplicação*, então aqui existe *stack*
B	Telemetria na fila morta	Qualquer mensagem parada: o limite é zero, e não um percentual, porque cada uma é evento de estudante perdido
B	API e hub lentos	p95 acima de 10 s na API e de 2 s no hub, por 10 min
B	CPU do banco alta	Acima de 80% por 10 min
B	*Spans* do navegador descartados	Qualquer um em 15 min com `motivo` em `sem-identidade`, `sem-projeto` ou `upstream`. O limite é ZERO: os três são falha de configuração ou de contrato, e a rota responde **204** mesmo descartando — o cliente nunca sabe. `cota` fica de fora de propósito: cota estourada é o sistema funcionando

Cada política carrega no campo `documentation` a sua própria primeira resposta em *markdown*, com os comandos já preenchidos — o e-mail do alerta chega com o passo a passo dentro, e não com um *link* para o Apêndice F.

> **ATENÇÃO**
>
> **Duas políticas ficaram escritas e DESLIGADAS, e a razão é uma ordem que não se pode inverter.** `sinal do navegador recusado` (`erros_recusados` com `motivo=invalido`) e `erro de cliente em massa` (`erros_cliente`) leem métricas que a aplicação precisa ter emitido ao menos uma vez: o Cloud Monitoring **recusa** criar política sobre métrica inexistente, com `404 Cannot find metric(s) that match type`, e um contador OpenTelemetry só cria a série quando incrementa pela primeira vez. Medido aplicando em `dev` em 9 de setembro de 2026. A sequência obrigatória é: *deploy* da plataforma → a primeira recusa (ou o primeiro erro) acontecer → ligar `obs_alerta_erro_de_cliente`. A variável existe para que essa dependência esteja escrita, em vez de ser descoberta num `apply` vermelho.

> **ATENÇÃO**
>
> **Esta política encontrou um defeito NOSSO no primeiro tráfego de verdade, e a primeira leitura dele estava errada.** Uma versão anterior deste capítulo dizia que os *spans* `motivo=sem-projeto` de `hml` (292 entre 5 e 7 de setembro) eram falta de `GCP_PROJECT_ID`, e que "o conserto é do Terraform, não da aplicação". **Não era.** A *env* já estava no serviço desde 3 de setembro, nos dois ambientes; o que faltava era o `ClienteOtlpGoogle` chegar ao construtor do `ProxyOtlpService`, que o importava com `import type` — o Nest perdia o *token* e o `@Optional()` injetava `undefined` em silêncio, com a rota respondendo **204**. Corrigido em 2026-09-14; o sintoma e a verificação de um comando estão no Apêndice F. Duas lições sobrevivem ao *bug*: alerta sobre métrica DA APLICAÇÃO encontra defeito de código, e não só de configuração; e o primeiro palpite

> sobre a causa — aqui, justamente a *env* que esta documentação já esperava que faltasse — tem de ser conferido contra o serviço antes de virar tarefa para outro repositório.

> **NOTA**
>
> O outro motivo que `hml` produziu no mesmo dia, `sem-identidade`, era ambiente ATRASADO e não defeito: até `cf9489f` (8 de setembro) a api validava `service.version` com `{1,32}` enquanto o CI injetava `NEXT_PUBLIC_HUB_VERSAO` com o `github.sha` inteiro, de 40 caracteres — todo lote do hub caía, também com **204**. O conserto de um ambiente atrasado é a promoção, não código.

17.9.2 Onde a implantação divergiu do desenho, e por quê

O desenho pedia	O que se fez, e a razão
Alerta por *burn rate* do SLO	Ficou de fora. *Burn rate* precisa de histórico para calibrar, e não havia nenhum. Os SLOs foram criados e passam a acumular: o alerta entra depois, com número medido em vez de arbitrado
5xx como **taxa** (> 1%)	Virou **contagem absoluta**. Com o tráfego de `dev` — uma pessoa — um erro em duas requisições é 50% e alertaria todo dia. A taxa volta a fazer sentido em `prd`
p95 por rota, isentando `/ai/*` e `/unoogpt/*`	Impossível sem tocar a aplicação: `run.googleapis.com/request_latencies` não tem rótulo de rota. O limite da API ficou folgado (10 s) justamente porque as rotas de IA moram nesse serviço; o corte fino exige métrica própria da aplicação. **Desde o agente de IA do professor, nem o folgado basta**: um *stream* de 1 a 3 minutos por mensagem (**ADR-128**) empurra o p95 da API acima de 10 s assim que o chat tiver uso. A proposta — métrica de log de distribuição sobre o *request log* do Cloud Run da API, com `NOT httpRequest.requestUrl:"/agente-professor/"` no filtro e `httpRequest.latency` como valor, e o alerta passando a ler essa métrica — está no RUNBOOK do repositório de fundação e **não foi aplicada**; antes, conferir em `dev` que a métrica recebe ponto (extrator que não casa deixa o alerta mudo, pior que barulhento)
Push para severidade A	Só e-mail. O canal ainda não foi verificado — vale disparar um alerta de teste antes de confiar nele, porque alerta que não chega é pior que alerta que não existe

17.9.3 *Session replay*: instância própria em `dev`

A recomendação de instrumentar erro de cliente ganhou uma peça de infraestrutura: uma instância **self-hosted** de OpenReplay, em VM própria com `k3s`, fora do *Load Balancer* — o *chart* traz o próprio `ingress-nginx` e emite o próprio certificado, e o Cloud Armor barraria os *payloads* binários do *ingest*. As gravações não ficam no disco da máquina: o MinIO embutido está

desligado e o *storage* aponta para quatro *buckets* GCS pela API S3 de interoperabilidade, com chave HMAC.

DECISÃO DE PROJETO

Por que *self-hosted* e não SaaS. A gravação mostra a tela de estudante menor de idade. *Self-hosted* mantém o dado no mesmo projeto GCP do resto da plataforma, sob o mesmo contrato — e é o que torna a conversa com a escola possível. O preço é operar mais uma VM, com *snapshot* diário do disco porque o Postgres interno guarda as contas e o índice das sessões.

ATENÇÃO

O *iframe* cross-domain exige `crossdomain.enabled` nos dois lados. A documentação do OpenReplay mostra apenas `parentDomain` no *tracker* de dentro do *iframe*; no código-fonte, a decisão é `crossdomainMode = inIframe() && options.crossdomain?.enabled`. Sem `enabled: true` *dentro* do *iframe* da experiência, ele abre sessão própria em vez de entrar na sessão do hub — e o replay chega partido em duas. Com a configuração certa, o *tracker* do *iframe* não fala com o *ingest*: ele envia por `postMessage` ao hub, que é o único a exportar. Exige *tracker* $\geq$ 18.

LIMITAÇÃO CONHECIDA

Em modo *cross-domain* o *iframe* não coleta `Viewport`, `Connection`, `Performance` nem `Tabs` — *web vitals* da experiência **não** entram no replay. DOM, ponteiro, rolagem, console e exceção entram. E se a experiência renderizar em `canvas` ou WebGL, o replay padrão mostra retângulo vazio: aí é preciso *canvas recording*, que muda a ordem de grandeza do consumo de disco.

17.9.4 Custo do que entrou

Item	Custo/mês	Observação
VM do OpenReplay (`e2-standard-4`)	US$ 98	Quatro *vCPU* e 16 GB: a v1.27 sobe ClickHouse ao lado de Postgres e Redis, e com 8 GB o nó mata *pod* por memória
Disco de 100 GB, IP estático e *snapshots*	US$ 19	As gravações ficam no GCS, cobradas por volume, com expiração em 90 dias
Doze políticas de alerta	US$ 4,20	Cobrança anunciada de US$ 0,35 por referência, não antes de setembro de 2026
Quatro sondas e o painel	US$ 0	4 sondas $\times$ 3 regiões $\times$ 60 s $\approx$ 520 mil execuções, dentro da franquia de 1 milhão; *dashboard* não tem cobrança

17.9.5 O que ficou pendente, em ordem de risco

1. **Erro de JavaScript no cliente.** Instrumentar o *tracker* do OpenReplay no hub e no *bundle* da experiência. A infraestrutura está paga e de pé; falta código no *monorepo*.
2. **Sondas `startup` e `liveness`** na API e no hub. As rotas respondem e o módulo aceita; aplicar cria revisão nova, então convém fazer junto de um *deploy* — e conferir no *plan* que a imagem do CI é preservada.
3. **Alerta de *burn rate***, quando os SLOs tiverem um mês de histórico.
4. **Orçamento de faturamento** (`billing_budget`): não existe nenhum, e a conta acabou de ganhar uma VM permanente.
5. **Verificar o canal de e-mail** com um alerta de teste.

17.10 O que fica de fora

Fora	Condição para reavaliar
Stack de observabilidade auto-hospedada (Grafana LGTM, SigNoz, OpenObserve)	Uma rede exigir que os sinais fiquem fora do Google Cloud, ou o nativo faltar numa consulta demonstrada. Ver **ADR-57**.
APM comercial (Datadog, New Relic, Dynatrace)	Vários contratos em várias nuvens, ou time de operação com SLA formal. A instrumentação OpenTelemetry já estará pronta para eles.
SDK de erro de terceiro no *bundle* da experiência	Nunca, por desenho (**ADR-58**). A versão auto-hospedada dentro do projeto é o caminho, se o Error Reporting não bastar. **Não confundir** com o *analytics* de produto da **ADR-60**, que está DENTRO do *bundle*: são ferramentas e decisões diferentes.
Ferramenta de plantão (PagerDuty, Opsgenie, Grafana OnCall)	O segundo operador.
Log de requisição da aplicação a 100%	Nunca: o Cloud Run já grava um.
`escola_id` como rótulo de métrica	Nunca: cardinalidade. Escola se filtra em log e se agrega em SQL.
BigQuery como destino de log	Um relatório de auditoria que o Log Analytics (SQL sobre o *bucket*, incluso) não consiga responder.

18

Registro de decisões arquiteturais

Cada decisão abaixo segue a forma de um ADR (NYGARD, 2011): contexto, decisão, alternativas descartadas, consequências — inclusive as ruins — e o gatilho que faria reavaliá-la. Decisão sem alternativa descartada e sem gatilho de reavaliação não é decisão registrada: é justificativa.

São **130 decisões**, numeradas até ADR-130, **sem lacunas**. A 117 (o Jornal da Escola) entrou depois da 121: o número tinha sido reservado entre as sessões antes de qualquer uma escrever. Quando eram cem, as cinco últimas nasceram no MESMO dia, em três frentes paralelas, e não colidiram porque os números foram **reservados por quem repartia o trabalho antes de qualquer uma escrever** — que é a lição que a colisão de 2026-09-06 e a de 2026-09-07 deixaram. A ADR-65 ficou vaga por um tempo: o número se perdeu numa colisão entre duas frentes paralelas que reservaram o mesmo identificador — o risco que a **ADR-61** registra — e a lacuna foi ocupada pelo **guia da aula** (**ADR-65**), em vez de a numeração seguir de 71 para 72.

A colisão voltou a acontecer no mesmo dia, e em dobro: **três** frentes paralelas fecharam decisão em 2026-09-06, e duas escreveram ADR-65 sem se ver. Prevaleceu a mais antiga; as outras seguiram para **ADR-72** e **ADR-73**. É a razão de a **ADR-61** mandar conferir `origin/dev` na hora de escrever o número — e a prova de que conferir só a branch integrada não basta: o número precisa ser reservado contra as frentes ABERTAS.

Renumerar as seis ADRs entre 66 e 71 para fechar a lacuna, na época, foi considerado e recusado, e a razão não é o trabalho: dentro deste repositório a troca é mecânica, porque os `\label` são nominais e o `\adrr` é formatação, não referência resolvida. O problema é o que sobra depois. Renumerar não deixa referência QUEBRADA — deixa referência **errada**: "ADR-69" continuaria resolvendo, para outra decisão, em silêncio, em onze mensagens de *commit* imutáveis, nos PRs já mergeados e em qualquer cópia deste PDF que já tenha circulado. É a forma exata do `expires_at` que cinco ADRs deste capítulo registram como armadilha. Um identificador vale por nunca mudar de significado; a ordem cronológica é o que se abre mão — e é por isso que a **ADR-65** não é, em data, a mais antiga das que a cercam.

18.1 Índice

ID	Decisão	Data	Estado
ADR-01	Experiências como *bundles* estáticos, não serviços	2026-08	Aceita
ADR-02	*Monorepo* único (pnpm + Turborepo)	2026-08	Aceita
ADR-03	Autenticação por OIDC genérico	2026-08	Aceita
ADR-04	Uma *stack* por contrato; escolas por RLS	2026-08	Aceita
ADR-05	Cloud SQL HA, não AlloyDB	2026-08	Aceita
ADR-06	Pub/Sub para telemetria desde o início	2026-08	Aceita
ADR-07	IA atrás de um *gateway* na própria API	2026-08	Aceita; revista para o agente do professor (ADR-126)
ADR-08	RLS forçada e uma porta única de banco	2026-08-24	Aceita
ADR-09	Domínio por experiência na API; gabarito *server-side*	2026-08-24	Aceita
ADR-10	Bem-estar: DASS-21 fixo, IA só no entorno, sem visão de gestor	2026-08-25	Aceita
ADR-11	Telemetria sem PII e vocabulário fechado no *payload*	2026-08-26	Aceita
ADR-12	`grau` como invariante de API, não regra de tela	2026-08-26	Aceita
ADR-13	Retenção sem prazo declarado, em troca de reprocessamento	2026-08-26	Aceita
ADR-14	Telemetria no PostgreSQL particionado; sem BigQuery, sem Redis	2026-08-27	Aceita
ADR-15	O hub assina os *tokens* enquanto o IdP central não entra	2026-08	Aceita, transitória
ADR-16	Credencial por ADC; modelo `3.6-flash` em `global`	2026-08	Aceita, com custo
ADR-17	*Claim* atômico em vez de fila externa	2026-08	Aceita
ADR-18	Exceções registradas: jogo em JS clássico, app-70 fora do DS	2026-08-26	Aceita
ADR-19	*Design system* externo como fonte única de interface	2026-08	Aceita
ADR-20	Um banco, um *schema*; prefixo `exp_NN_`	2026-08-27	Aceita; prefixo revisto na ADR-110
ADR-21	*Push subscription* para a própria API, sem *worker*	2026-08	Aceita

ADR-22	exp04ef1b 1.1.0: *briefing* na voz da escola, conclusão por botão	2026-08-27	Aceita, com custo
ADR-23	*Function calling* entra no `AiProvider`, não na experiência	2026-08-27	Aceita
ADR-24	Teto de custo de IA por SESSÃO; a quota diária é fusível	2026-08-27	Aceita, com número a medir
ADR-25	O Relógio da Aula é tabela da experiência, não coluna em `turmas`	2026-08-27	Aceita
ADR-26	Sem prazo de retenção na exp02em1b; exclusão por “zerar a turma”	2026-08-27	Aceita
ADR-27	Transcrição do estudante é dado do instrumento: professor lê, gestor não	2026-08-27	Aceita
ADR-28	O *feed* do professor consome a API do estudante	2026-08-27	Aceita
ADR-29	A app-70 não emite nota, emite perfil	2026-08-27	Aceita
ADR-30	Vínculo pessoa × escola datado (`lotacoes`)	2026-08-28	Aceita
ADR-31	CPF como identificador civil, sem senha no CSV	2026-08-28	Aceita
ADR-32	Sessão por *cookie* HttpOnly com rotação	2026-08-28	Aceita
ADR-33	Rotação de chave a frio por variável de ambiente dupla	2026-08-28	Aceita
ADR-34	Uma escola ativa por sessão; sem agregação de rede	2026-08-28	Aceita
ADR-35	Portabilidade de dados por JWS assinado	2026-08-28	Aceita
ADR-36	Perfil portátil é chave/valor com esquema fechado	2026-08-28	Aceita
ADR-37	Validação de entrada exclusivamente via Zod	2026-08-29	Aceita
ADR-38	Documentação viva em LaTeX compilada no CI	2026-08-29	Aceita
ADR-39	Dependências sob varredura de vulnerabilidade (Snyk)	2026-08-29	Aceita
ADR-40	Landing institucional: estática, sem sessão, 1440 px	2026-08-30	Aceita
ADR-41	Dispositivo expansível na pós-revelação da exp08ef1b	2026-08-30	Aceita
ADR-42	Exposição no dossiê mede o *feed*, não o domínio	2026-08-30	Aceita
ADR-43	Conclusão do estudante: guarda primeira e última inclinação	2026-08-30	Aceita
ADR-44	Janela de envolvimento fecha no último post visto	2026-08-30	Aceita
ADR-45	Geração de planos com *streaming* SSE e retenção de 30 dias	2026-08-31	Aceita
ADR-46	Embed via iframe restrito: sandbox, allow e canal postMessage	2026-08-31	Aceita
ADR-47	Rate limit por IP em rotas públicas, por usuário em autenticadas	2026-08-31	Aceita
ADR-48	Importação de CSV síncrona com limite de 30 MB	2026-08-31	Aceita

ADR-49	Atividade vinculada à turma; clone para cada turma nova	2026-09-01	Aceita
ADR-50	Perfil da Vila importado na exp08ef1b sem FK rígida	2026-09-01	Aceita
ADR-51	Testes E2E com Playwright contra ambiente real	2026-09-01	Aceita
ADR-52	Mock de IA determinístico para testes e homologação	2026-09-01	Aceita
ADR-53	GitHub Actions com três ambientes (`dev`, `hml`, `main`)	2026-09-02	Aceita
ADR-54	Balcão de atendimento: cadastro individual delega ao importador	2026-09-02	Aceita
ADR-55	Reset de senha pelo professor gera credencial temporária	2026-09-02	Aceita
ADR-56	Soft delete com `desativado_em` em usuários	2026-09-02	Aceita
ADR-57	Observabilidade: traces OpenTelemetry com proporção configurável	2026-09-02	Aceita
ADR-58	Sourcemaps enviados a bucket privado, nunca na CDN	2026-09-02	Aceita
ADR-59	Quem escreve usuário é o importador; balcão gera linha CSV	2026-09-03	Aceita
ADR-60	Microsoft Clarity via `@edu/sdk`, desligado se sem ID	2026-09-03	Aceita
ADR-61	Frentes paralelas com git worktree sobre `origin/dev`	2026-09-03	Aceita
ADR-62	DLQ no Pub/Sub para telemetria rejeitada	2026-09-04	Aceita
ADR-63	Ingestão de telemetria desacoplada do processamento analítico	2026-09-04	Aceita
ADR-64	Relógio da Tempestade: tempo no marco, não em background	2026-09-05	Aceita
ADR-65	Guia da aula é gerado do catálogo, sem tela dedicada	2026-09-06	Aceita
ADR-66	exp07ef1b: IA no runtime para conversas, gabarito server-side	2026-09-06	Superada (missão retirada, **ADR-113**)
ADR-67	Mídias da exp07ef1b: imagens e áudios pré-gerados no bucket	2026-09-06	Aceita como molde; o Eco saiu (**ADR-113**)
ADR-68	exp07ef1b e exp06ef2b: clone de mensageria só no tema escuro	2026-09-06	Aceita (vale para a exp06ef2b; o Eco saiu, **ADR-113**)
ADR-69	Missão só abre na turma se o professor liberou (`liberacoes`)	2026-09-06	Aceita
ADR-70	Página de transparência algorítmica aberta ao público	2026-09-06	Aceita
ADR-71	Fatias verticais por módulo em frentes paralelas	2026-09-06	Aceita

ADR-72	Fronteira do disfarce da exp06ef2b: mapa da rede fala pela escola	2026-09-06	Aceita
ADR-73	Planejamento curricular dirigido pela BNCC (app-92)	2026-09-06	Aceita; recortada para o agente do professor (**ADR-127**)
ADR-74	Contabilidade de tokens de IA com callback `aoUsarTokens`	2026-09-06	Aceita
ADR-75	Rolagem unificada via `ScrollBar` do Design System	2026-09-07	Aceita
ADR-76	Planejamento semanal estruturado com suporte a downloads	2026-09-07	Aceita
ADR-77	Trilha de navegação (*breadcrumb*) no layout, não na página	2026-09-07	Aceita
ADR-78	Logoff no menu da conta com limpeza completa de cookies	2026-09-07	Aceita
ADR-79	Vínculo obrigatório escola × regional na governança da rede	2026-09-07	Aceita
ADR-80	Modelo de seis papéis e cadeia de atribuição transitiva	2026-09-08	Aceita
ADR-81	Panorama da diretoria de ensino consolidado por regional	2026-09-08	Aceita
ADR-82	Relatório pedagógico da regional com varredura em lote	2026-09-08	Aceita
ADR-83	Header institucional ancorado na janela (full-bleed)	2026-09-08	Aceita
ADR-84	Observabilidade de cliente sem sessão com sanitização	2026-09-08	Aceita
ADR-85	Trocar de escola remonta o miolo da área (`<EscopoDaEscola>`)	2026-09-09	Aceita
ADR-86	O guia da aula mostra o conteúdo da missão, não o descreve	2026-09-09	Aceita
ADR-87	Disponibilidade: terceiro eixo, e o produto como teto	2026-09-09	Aceita
ADR-88	O painel da Unooria é só do `admin`	2026-09-09	Aceita
ADR-89	Histórico do painel é tabela append-only, e a rede lê o dela	2026-09-10	Aceita
ADR-90	A suíte da API roda em UM *worker*	2026-09-10	Aceita
ADR-91	O idioma da interface tem três zonas, e o clone não se traduz	2026-09-10	Aceita
ADR-92	Voltar um momento reabre a sessão; a trava é a revelação	2026-09-10	Aceita
ADR-93	Sincronização de sala por *polling* HTTP, não WebSocket	2026-09-10	Aceita
ADR-94	O latch de turma anda por *polling* curto, nunca por canal persistente	2026-09-10	Aceita
ADR-95	A sessão do hub dura 2h de inatividade, igual para todo papel	2026-09-10	Aceita
ADR-96	Ficar na missão conta como uso, e a sessão que morre tem tela	2026-09-10	Aceita

ADR-97	A tela diz POR QUE não entrou, e a suíte espera o desfecho	2026-09-10	Aceita
ADR-98	A plataforma assume UM fuso, numa constante só	2026-09-11	Aceita
ADR-99	O fuso vem de uma variável de ambiente, em três superfícies	2026-09-11	Aceita
ADR-100	Da sala para a missão há UM caminho, e ele leva à ficha	2026-09-15	Aceita
ADR-101	Quatro ambientes, e a frente nasce e volta em `dev-local`	2026-09-15	Aceita
ADR-102	A doutrina é uma fonte só, lida por quatro agentes	2026-09-15	Aceita
ADR-103	A frente mergeada se enterra sozinha	2026-09-15	Aceita
ADR-104	A base se realinha sozinha depois da promoção	2026-09-15	Aceita
ADR-105	A quota de IA é dado, editável pela Unooria, e não variável de ambiente	2026-09-15	Aceita
ADR-106	O `_template` nasce com a fiação inteira e sem componente de interface	2026-09-16	Aceita
ADR-107	O site público entra no hub como porte, e o cadastro de interesse passa pela api	2026-09-16	Aceita
ADR-108	A missão de sala nasce vazia, e a revelação é do relógio da turma	2026-09-17	Aceita; a v1 saiu (**ADR-114**) e a v2.2 a retoma (**ADR-118**)
ADR-109	O rótulo da tabela da BNCC de Computação é CENTRADO, e a atribuição é uma partição	2026-09-17	Aceita
ADR-110	O código da experiência diz onde ela mora na trilha	2026-09-23	Aceita
ADR-111	Equipe é quem divide um computador, e o painel conta por computador	2026-09-25	Aceita
ADR-112	A evidência é por atividade e rodada, e a melhor rodada vale por habilidade	2026-09-25	Aceita
ADR-113	A Seleção substitui o Eco no Grupo, e a limpeza vem antes	2026-09-25	Aceita
ADR-114	Quem mandou isso? v1 é retirada; a v2.2 entra com o mesmo código	2026-09-25	Aceita
ADR-115	A Seleção: o par do Mural é do computador, e a IA gradua a legenda	2026-09-25	Aceita
ADR-116	A experiência que perde o código é arquivada, não apagada	2026-09-25	Aceita
ADR-117	Jornal da Escola: a redação é da Unooria, o professor abre o resultado e a revisão, e a IA só escolhe o layout	2026-09-27	Aceita

ADR-118	Quem mandou isso? v2.2: atividade é o pacote, rodada é a reaplicação, e a régua mede pela seleção e pelas perguntas do PO	2026-09-25	Aceita; a régua foi substituída pela **ADR-120**
ADR-119	O podcast da A Seleção é do professor, e a IA reescreve só as três falas que dependem do tema	2026-09-26	Aceita
ADR-120	A régua v2 de Quem mandou isso?: cada habilidade pelo verbo dela, e refazer o mesmo pacote não mede memória	2026-09-26	Aceita
ADR-121	O CI verifica só os PRs para `dev`, `hml` e `main`	2026-09-26	Aceita
ADR-122	A modelagem UML entra no Capítulo 5, com os catorze tipos da UML 2.5.1 e par Mermaid mesmo onde o Mermaid não tem o tipo	2026-09-27	Aceita
ADR-123	O plano anual tem uma estrutura só, definida pelo prompt, e o template é exemplo de formato	2026-09-30	Aceita
ADR-124	O `CLAUDE.md` admite o bloco que uma ferramenta escreve, e só ele	2026-09-30	Aceita; emenda a **ADR-55** e a **ADR-102**
ADR-125	O grafo do código nasce copiado em cada *worktree* e se atualiza sozinho no *pull*, e fica fora do *git*	2026-09-30	Aceita; completa a **ADR-124**
ADR-126	O agente de IA do professor roda no Vertex AI Agent Engine, atrás da api	2026-09-30	Aceita; revisa a **ADR-07**
ADR-127	Só *data store*, inclusive para as habilidades da BNCC	2026-09-30	Aceita, em avaliação; recorta a **ADR-73**
ADR-128	SSE só no chat do agente	2026-09-30	Aceita
ADR-129	O *markdown* do agente vira HTML no navegador	2026-09-30	Aceita
ADR-130	A atividade é do professor, e a equipe é da atividade	2026-10-01	Aceita

18.2 ADR-01 — Experiências como *bundles* estáticos

> **Contexto**
>
> O catálogo alvo é de 48 experiências, cada uma uma aplicação React independente. O pico de carga é uma parede às oito da manhã, e o grosso dos *bytes* é o código e a mídia dessas aplicações. Uma pessoa opera tudo.

Decisão. Cada experiência é um *bundle* Vite 100% estático, publicado em `gs://{contrato}-exp/{expId}/{v` e servido pela CDN. Não há processo, porta, nem *scaling* por experiência. O hub monta a URL do `iframe` com `versaoAtiva` do manifesto.

Alternativas descartadas.

- *Microsserviço por experiência*: 48 serviços com *deploy*, log, escala e *alerting* próprios — para servir arquivos estáticos.
- *Rotas do hub monolítico*: cada experiência entraria no *bundle* do hub, e o hub cresceria sem limite. Uma mudança em qualquer experiência exigiria *deploy* do hub inteiro.

Consequências.

- Custo marginal de publicação ≈ armazenamento + tráfego de CDN.
- O caminho carrega a versão ⇒ o *bundle* é imutável e pode ficar um ano na borda.
- ***Rollback* é um `UPDATE`**: trocar `versaoAtiva`. Sem *build*, sem imagem, sem *deploy*. É a operação de emergência mais rápida da plataforma.
- *Ruim*: a experiência não tem *backend* próprio, então toda lógica de domínio dela vive na API — o que concentra responsabilidade num serviço só (mitigado por **ADR-09**).
- *Ruim*: um *bundle* publicado não é reconstruído quando a API muda, o que torna os contratos do Capítulo 9 rígidos.

Gatilho de reavaliação. Uma experiência que precise de execução no servidor por natureza — multiplayer em tempo real com autoridade de servidor, ou processamento pesado por sessão.

18.3 ADR-02 — *Monorepo* único

Contexto. A SDK é consumida por 48 aplicações. O *design system* é externo, mas a ponte é interna.

Decisão. Um *monorepo* com pnpm *workspaces* e Turborepo (Vercel, 2026b). `apps/*`, `apps/experiencias/*`, `packages/*`.

Alternativa descartada. Repositório por experiência, com a SDK publicada em npm. Uma mudança na SDK só seria testada contra as 48 aplicações depois de publicada — e a incompatibilidade apareceria num *bundle* servido a estudantes semanas depois.

Consequências. Mudança na SDK testa tudo no mesmo *pull request*; o CI builda só o que mudou (`--filter='...[HEAD^]'`). *Ruim*: o repositório cresce, e `pnpm install` completo é lento.

Gatilho. Estúdio externo produzindo experiência — aí a experiência sai para repositório próprio e passa a consumir a SDK publicada.

18.4 ADR-03 — Autenticação por OIDC genérico

Contexto. O WSO2 Identity Server é exigência do cliente e da estratégia multi-contrato: é único para várias redes.

Decisão. A aplicação **não conhece o WSO2**: conhece OIDC. A API valida *token* por `OIDC_ISSUER`, `OIDC_JWKS_URI` e `OIDC_AUDIENCE`. Trocar de provedor de identidade é trocar essas variáveis.

Alternativa descartada. SDK do WSO2 na aplicação. Amarraria o código a um produto que pode mudar por decisão comercial, e tornaria o teste local dependente de um servidor Java em *bootstrap*.

Consequências. `OIDC_ISSUER` e `OIDC_JWKS_URI` vão como *env* em texto plano (não são segredo: são URLs públicas), derivadas de `hub_domain` — e é isso que garante que o `iss` do *token* e o esperado pela API **não podem** divergir.

Gatilho. Nenhum previsto. É a decisão mais estável do conjunto.

18.5 ADR-04 — Uma *stack* por contrato; escolas por RLS

Contexto. Cada contrato é uma relação jurídica com um ente federativo; cada contrato tem 100+ escolas.

Decisão. Isolamento **físico** entre contratos (*stack* GCP inteira, por *module* Terraform) e **lógico** entre escolas (`escola_id` + RLS).

Alternativas descartadas.

- *Uma stack por escola*: cem infraestruturas por contrato, operadas por uma pessoa. E o custo fixo de uma escola pequena passaria a ser o de uma rede.
- *Uma stack para todos os contratos*: um incidente num município seria um incidente em todos, e o desenho não sobreviveria à primeira análise jurídica de um edital.

Consequências. `escola_id` em toda tabela de dado de estudante, RLS forçada, e `admin-contrato` como única exceção cross-escola. *Ruim*: cada contrato novo é uma *stack* nova para provisionar, monitorar e faturar.

Gatilho. Exigência contratual de isolamento físico *por escola* — o que mudaria o modelo comercial, não só a arquitetura.

18.6 ADR-05 — Cloud SQL HA, não AlloyDB

Decisão. PostgreSQL gerenciado (Cloud SQL) com HA, PITR e *pooler* de conexões obrigatório.

Alternativa descartada. AlloyDB. **Motivo:** a robustez que este sistema precisa vem de HA + PITR + *pooler*, não do *engine*. Ambos são PostgreSQL, então migrar depois é trivial se a carga justificar.

Consequências. O *pooler* é **obrigatório**: o *autoscaling* do Cloud Run multiplica *pools* do Prisma e estouraria `max_connections`. E o banco só tem IP privado — o que obriga a migração a rodar como Cloud Run Job dentro da VPC.

Gatilho. Carga medida que sature uma instância de Cloud SQL com *pooler* bem dimensionado.

18.7 ADR-06 — Pub/Sub para telemetria desde o início

Contexto. No pico, quatro mil estudantes simultâneos gerando evento a cada poucos segundos.

Decisão. O canal *analítico* de telemetria vai para o Pub/Sub. **O resto da aplicação fala PostgreSQL direto** — inclusive o canal *operacional* do motor.

Alternativa descartada. Tudo síncrono no PostgreSQL. Colocaria um `INSERT` sincronizado por interação de estudante dentro da mesma transação que serve a tela.

Consequências. Uma peça de infraestrutura a mais — e é a única exceção declarada à regra "nada de fila sem gargalo medido", junto com **ADR-07**. Em troca: o pico é amortecido por um *buffer* **durável** (ver **ADR-14**).

Gatilho. Nenhum. É decisão tomada de antemão e mantida.

18.8 ADR-07 — IA atrás de um *gateway* na própria API

Decisão. Toda chamada de IA passa por `apps/api/src/ai`, com interface neutra de *vendor*, *prompt* de sistema no servidor, quota diária persistida por perfil e *rate limit*. Pré-gerar em lote sempre que a pedagogia permitir.

Alternativas descartadas.

- *Chamada direta do bundle*: exporia chave e *prompt* de sistema, e tornaria a quota incontrolável.
- *Serviço de IA separado*: um Cloud Run a mais para operar, sem ganho — a API já tem CPU sempre alocada.

Consequências. Nenhum módulo fora de `src/ai` conhece o *vendor*; trocar de provedor é criar um arquivo. *Ruim*: a API carrega o custo de latência das gerações longas, o que obriga o padrão 202 + *polling*.

Gatilho. Volume de IA que justifique isolar o consumo, ou requisito de GPU própria.

Revisão. O agente de IA do professor roda fora da api, no Vertex AI Agent Engine, por decisão do dono do produto e sem que nenhum dos dois gatilhos tenha disparado (**ADR-126**). A api continua sendo a única porta — papel, disponibilidade, *rate limit*, quota e posse da conversa —, e o resto da IA segue esta decisão.

18.9 ADR-08 — RLS forçada e uma porta única de banco

Decisão. `ENABLE` e `FORCE ROW LEVEL SECURITY` em toda tabela de dado de estudante; acesso ao banco **exclusivamente** por `PrismaService.comTenancy`, que abre transação e executa `set_config('app.` `..., true)` antes do trabalho.

Alternativas descartadas.

- *Apenas filtro na aplicação*: uma *query* nova que esqueça o filtro devolve dado de outra escola, e ninguém percebe.
- *Apenas RLS*: um erro de migração desliga a proteção inteira.
- *Duas roles* (dona para migração, sem *ownership* para a aplicação): desnecessário com `FORCE`, que sujeita o próprio dono às políticas.

Consequências. Duas trancas independentes. O terceiro argumento `true` do `set_config` é **crítico**: local à transação, então a conexão volta limpa ao *pool*. `comTenancy` sem escola **lança** — não consulta sem filtro. *Ruim*: toda leitura, inclusive trivial, abre transação.

Gatilho. Nenhum. É a decisão que sustenta o produto.

18.10 ADR-09 — Domínio por experiência na API; gabarito *server-side*

Contexto. A exp04ef1b (2026-08-24) foi a primeira experiência com gabarito.

Decisão. Não há módulo genérico de avaliação. Cada experiência com lógica de domínio ganha um módulo próprio na API (`src/<slug>/`), com `catalogo.ts` + `regras.ts` + *service* + *controllers* por papel. O gabarito nunca vai ao *bundle*; a API corrige.

Alternativa descartada. Um módulo genérico de avaliação com "itens" e "respostas corretas". Uma API genérica de correção convida a mandar a resposta certa junto — e cada experiência tem uma mecânica que não cabe num formato comum.

Consequências. Decisões concretas herdadas desta ADR: ranking com colegas **fictícios** (sem pontuação de menor real exposta a outro menor); URLs das fontes como **texto** (o *sandbox* bloqueia *popups*, e o estudante não sai da plataforma); salvos **congelados** depois da conclusão (nota e estado não podem divergir); estudante roteado pela **série**. *Ruim*: um módulo novo por experiência — 48 módulos no limite.

Gatilho. Se três experiências consecutivas tiverem mecânica idêntica, extrair o comum. Nunca antes.

18.11 ADR-10 — Bem-estar: instrumento fixo, IA só no entorno, sem gestor

Contexto. 2026-08-25. Um *check-in* de bem-estar *do professor*, com o DASS-21.

Decisão. O instrumento entra ***verbatim*** (versão brasileira de Vignola & Tucci (VIGNOLA; TUCCI, 2014)) em microdoses de três itens por dia. A IA gera abertura, pergunta aberta e acolhimento, atrás de `regras.validarTextoDaIa`, com *fallback* fixo, e **nunca vê** pontuação, nome ou escola. Correção *server-side* ao sétimo *check-in*. Item sentinela (17 ou 21 ≥ 2) ou termo de risco $\Rightarrow$ mensagem fixa de apoio, sem IA.

Por ser dado de saúde (LGPD art. 11): não há visão de gestor nem agregado por escola. RLS + posse por `professor_id`, e o próprio professor apaga tudo.

Alternativas descartadas.

- *IA reescrevendo os itens* para "soar mais natural": quebra a validação psicométrica e invalida as faixas de corte.
- *Questionário completo de uma vez*: engajamento vem de custar menos de um minuto, não de ser curto uma vez.
- *Painel agregado para a rede*: sai de um `GROUP BY` e é exatamente o que não se deve fazer com dado de saúde de uma equipe pequena, onde *n* baixo reidentifica.

Consequências. O *check-in* nunca deixa de abrir. *Ruim*: a rede não tem nenhuma visão de bem-estar dos professores — o que é uma perda real de informação de gestão, aceita.

Gatilho. Uma visão agregada e anonimizada é decisão futura, com DPA e *k*-anonimato explícito.

18.12 ADR-11 — Telemetria sem PII e vocabulário fechado

Decisão. A telemetria carrega `estudante_id` **opaco**, e a identidade é derivada do *token* no servidor — o que o cliente enviou nesses campos é descartado. Em `app_70_sessoes.payload`, `eventos[].dados` aceita apenas número, booleano ou *string* de um **vocabulário fechado**.

Alternativa descartada. Campo `texto` livre no *payload* (que existia). Era o único canal de texto arbitrário — e o caminho por onde o nome de uma criança chegaria a um JSONB de longa duração.

Consequências. Trava **estrutural**, não lista de proibidos: não há como enviar texto arbitrário, então não há como revisar mal. *Ruim*: um evento novo que precise de rótulo textual exige acrescentar o rótulo ao vocabulário — ou seja, exige uma decisão explícita.

Gatilho. Nenhum.

18.13 ADR-12 — `grau` como invariante de API

Contexto. O mapeamento comportamento → valor de Schwartz é **hipótese de *design*, não instrumento validado**: Schwartz se valida por questionário (PVQ para adultos, PBVS-C para crianças (SCHWARTZ, 2012; DöRING et al., 2010)), não por telemetria de jogo.

Decisão. A API devolve uma **união discriminada** em que só a variante `scored` tem `numero: number`; as outras têm `null` **por tipo**. E a distinção entre `zero` ("teve a chance e não foi", entra na média) e `none` ("a oportunidade não chegou", sai da conta) é preservada em todo o caminho.

Alternativa descartada. Manter a decisão na camada de interface, com um comentário avisando. Foi assim no aplicativo de origem, e qualquer tela nova podia imprimir "Poder 100" a partir de um sinal só. **Prosa não é controle; um tipo é.**

Consequências. O hub fica estruturalmente incapaz de desenhar o número errado — ele nunca o recebe, e o compilador nem deixa procurar. Decisões herdadas: o relatório abre pelo **nível** do instrumento antes do desempenho; a listagem de turma **não** mostra valor psicométrico (listagem é ordenável, e uma coluna "Poder" viraria ranking de crianças); `cartoesServidos` ≠ "apareceram".

Gatilho. Se um instrumento validado por questionário for adotado, a união ganha uma variante — não perde a distinção.

18.14 ADR-13 — Retenção sem prazo declarado

Contexto. 2026-08-26. Havia `expires_at` de 90 dias no *schema* da app-70 e **nenhuma rotina que o cumprisse**.

Decisão. A coluna **saiu** (`20260826170000_vila_sem_prazo`). Não há prazo de retenção declarado para `app_70_sessoes` nem para `telemetria_eventos`.

Alternativa descartada. Manter a coluna. **Coluna de prazo que ninguém lê é pior que prazo nenhum**, porque quem lê o *schema* acredita nela — é a mesma armadilha de um campo `consentimento_em` que existia no aplicativo de origem: a regra saiu do código, a coluna ficou, e todo mundo continuou acreditando.

Consequências. Ganha-se o **reprocessamento**, que é requisito real do instrumento: `payload` é a única fonte para recomputar o perfil quando `VERSAO_MAPEAMENTO` subir, e 90 dias matavam exatamente a comparação entre duas gerações do mapeamento. *Ruim, e é explícito*: dado comportamental de menor fica armazenado indefinidamente, e isso é uma exposição de conformidade em aberto.

Gatilho. Exigência contratual de prazo. Quando vier, ele volta **junto com a rotina que o cumpre** — e, com a tabela particionada (**ADR-14**), essa rotina é `DROP TABLE particoes.<mês>`. A pergunta é da plataforma inteira, não de um módulo.

18.15 ADR-14 — Telemetria no PostgreSQL particionado

Contexto. 2026-08-27. Avaliou-se mandar o canal analítico para o BigQuery por *subscription* nativa do Pub/Sub. Mediu-se também o custo real do caminho de gravação.

Decisão. O canal analítico grava no **mesmo PostgreSQL**, com `INSERT` multi-linha e `telemetria_eventos` particionada por mês. **Sem BigQuery. Sem Redis.**

Alternativas descartadas.

- *BigQuery*: os relatórios do professor **não leem** `telemetria_eventos` (leem `exp08ef1b_events`), então não resolvia problema nenhum hoje — e criava uma segunda plataforma de dados para operar.
- *Redis como buffer*: quem amortece o pico já é o Pub/Sub, e ele é **durável**. Um *buffer* em memória perde os eventos do processo que morre antes do *flush*, ou seja, reintroduz a perda que o Pub/Sub eliminou. **O gargalo medido era o laço de `INSERT`** — um lote de 50 custava 50 *round-trips*, o que no pico dava ∼800 `INSERT`/s; em lote, ∼16/s.

Consequências. A chave primária é composta `(event_id, ts)`, porque o PostgreSQL exige a chave de partição em todo índice único — e a *dedupe* segue correta, porque a reentrega traz o mesmo `ts`. As partições vivem no *schema* `particoes`, fora do alcance do Prisma. Há partição `DEFAULT` como rede de segurança, porque evento recusado é telemetria perdida em silêncio. *Ruim*: as faixas vão até 2028-12 e **não há *job* agendado** — criar partição é item de operação.

Gatilho. Volume que sature o PostgreSQL, ou um relatório que efetivamente precise de análise sobre `telemetria_eventos` em escala.

18.16 ADR-15 — O hub assina os *tokens* de experiência

Contexto. O IdP central (WSO2) tem ciclo de vida próprio e não estava pronto quando as experiências precisaram de *token*.

Decisão. O **hub** assina os *tokens* de experiência (RS256, TTL 15 min, *claims* `{sub, escola_id, turma_id?, perfil}`, `aud plataforma-api`) e serve o JWKS em `/.well-known/jwks.json`. A API valida por `OIDC_ISSUER` e `OIDC_JWKS_URI`.

Alternativa descartada. Esperar o WSO2. Faria toda a plataforma depender de um servidor Java em *bootstrap* para que qualquer experiência pudesse ser testada.

Consequências. Trocar para o WSO2 é trocar duas variáveis — o `OidcGuard` não muda uma linha. **Efeito colateral útil:** o CI usa a mesma chave para assinar o *token* de cinco minutos

do *upsert* de manifesto, o que é o motivo de as experiências serem publicadas *depois* do hub. *Ruim*: o hub acumula responsabilidade de emissor, e `TOKEN_SIGNING_KEY_PEM` passa a ser obrigatória em produção.

Gatilho. WSO2 pronto e configurado por contrato. A ADR é **transitória por desenho**.

18.17 ADR-16 — ADC como credencial; modelo *versus* região

Decisão, parte 1 — credencial. O *provider* de IA obtém o *token* com `google-auth-library`, que resolve `GOOGLE_APPLICATION_CREDENTIALS` → arquivo de `gcloud auth application-default login` → *metadata server*. **Nunca chave de API** — não existe variável para ela no projeto.

Contexto. Antes havia *só* o *metadata server*, e por isso *nenhuma* chamada de IA funcionava fora do Cloud Run. Em desenvolvimento a API ficava presa em `mock` e a app-90 devolvia um plano **falso** — o que é pior que falhar, porque parecia funcionar.

Decisão, parte 2 — modelo e região. Modelo `gemini-3.6-flash`, localização `global`, para texto *e* para plano de aula.

Medição. O `3.6-flash` dá **404** em `southamerica-east1`, `us-east1` e `us-central1`. Só o endpoint `global` o serve.

Consequência aceita, e ela é séria. O tráfego de IA sai de São Paulo, inclusive o do estudante, e isso precisa constar do contrato e do DPA da rede.

Gatilho. Exigência de residência de dados. Voltar à região é trocar `VERTEX_LOCATION` **e** o modelo, juntos — a região publica a família 2.5, não a 3.x.

Decisões relacionadas. `VERTEX_THINKING_BUDGET = 0`: nos modelos 2.5 e 3.x os *tokens* de pensamento saem do `maxOutputTokens`, e com pensamento livre o plano anual voltava **vazio** com `finishReason: MAX_TOKENS`. Teto de saída do plano em **64k**, não 32k. Modelo de imagem `gemini-3.1-flash-image`, com proporção `4:5` **pedida** ao modelo — o *default* anterior não existe mais, e sem pedir a proporção o `cover` descarta metade da largura.

18.18 ADR-17 — *Claim* atômico em vez de fila externa

Contexto. Duas rotinas longas e assíncronas: geração de *feed* (exp08ef1b) e geração de plano de aula (app-90). Duas ou três instâncias do Cloud Run rodando ao mesmo tempo.

Decisão. *Claim* atômico na própria linha (`UPDATE ... WHERE status = 'pendente'`, com contagem de linhas afetadas) e *worker in-process*. Para o motor de *feed*, *lock* consultivo por `run` (`pg_advisory_xact_lock`).

Alternativas descartadas.

- *Fila externa* (Pub/Sub, Cloud Tasks): peça a mais para operar, para um problema que um `UPDATE` condicionado resolve.
- *Sem lock no motor*: dois pedidos simultâneos de bloco (duas abas, *reload* no meio do *fetch*, *retry*) se sobrescreviam e um bloco inteiro desaparecia do estado — posts entregues voltavam a ser servidos e a série por bloco do dossiê ficava corrompida.

Consequências. A API precisa de **CPU sempre alocada** e `min-instances` ≥ 1, porque há trabalho fora do ciclo de requisição. Retomada de órfã é disparada pelo *polling* do painel, não por *scheduler*. *Ruim*: se todas as instâncias morrerem e ninguém abrir o painel, uma geração órfã fica parada até que alguém abra.

Gatilho. Primeira reclamação real de geração parada.

18.19 ADR-18 — Exceções registradas

Contexto. Duas regras da plataforma têm exceção, e exceção não registrada vira precedente.

Exceção 1 — `any` e ESLint. `apps/experiencias/app-70-vila-do-lume/public/jogo/` — cerca de dez mil linhas de JavaScript clássico com globais compartilhadas e `three.min.js` vendorizado — fica como está, e está nos `ignores` do ESLint. **Motivo:** é um instrumento com distâncias e tempos calibrados, e o compilador não tem como saber que uma refatoração mudou a *medida*. O *shim* (`src/`, TS *strict*) é a única porta: `window.VILA`. A rede que substitui o `tsc` ali é o *smoke* visual.

Exceção 2 — *design system*. A app-70 não usa o `@unooria/design-system`. **Motivo:** é jogo em *canvas* 3D, sem DOM de interface.

Consequências. As duas exceções são **nominais** — valem para uma pasta e uma experiência, não para uma categoria. Qualquer outra experiência que peça a mesma exceção precisa de uma ADR nova.

18.20 ADR-19 — *Design system* externo como fonte única de interface

Decisão. Todo elemento de interface vem do `@unooria/design-system`, consumido pela ponte `packages/ui`. Componente que falta **nasce no repositório do *design system*** e chega por versão nova — não como arquivo local.

Alternativa descartada. Componentes locais em `packages/ui`. Com 48 experiências, um componente local viraria 48 divergências — e a fidelidade ao clone é o que a experiência *mede*.

Consequências. Duas famílias no mesmo pacote que **nunca se cruzam**; a linha entre elas é a **revelação**, não o arquivo. Compensação local só como exceção marcada no CSS, com pedido correspondente ao repositório do *design system*. *Ruim*: uma lacuna do pacote bloqueia a experiência até a próxima publicação — e `^0.1.x` não sobe *minor*, então atualizar é ato deliberado.

Gatilho. Nenhum. É decisão de produto e de marca, não de engenharia.

18.21 ADR-20 — Um banco, um *schema*; prefixo `exp_NN_`

Revista pela ADR-110 no prefixo: desde 2026-09-23 ele é o CÓDIGO da experiência (`exp08ef1b_`, `app_70_`), não o número de criação. O resto desta decisão continua valendo como está.

Decisão. Um banco, um *schema* lógico. Tabela de experiência leva `exp_NN_`; tabela de plataforma fica sem prefixo; tabela que é contrato entre duas experiências carrega **os dois números** (`exp08ef1b_app_70_perfis_importados`).

Alternativas descartadas.

- *Banco por experiência*: elas **não** são isoladas — o perfil da Vila alimenta a largada do Instanooria, e o PostgreSQL não faz `JOIN` entre bancos. E um *pool* do Prisma por experiência estouraria `max_connections`.
- *Schema por experiência*: a única coisa que daria além do prefixo é fronteira de `GRANT`, e existe **uma** *role* de aplicação.

Consequências. Migração de referência `20260827090000_tabelas_por_experiencia`: `RENAME`, nunca `DROP`/`CREATE`, renomeando *constraints*, índices e políticas junto. Duas armadilhas registradas lá: `FOR ... IN SELECT` sobre o catálogo **relê o que o próprio laço alterou**, e `RENAME CONSTRAINT` renomeia **também** o índice que a sustenta — as duas produzem `exp08ef1b_exp08ef1b_..._pkey`

Gatilho. Necessidade de fronteira de `GRANT` por experiência — o que só faria sentido com mais de uma *role* de aplicação.

18.22 ADR-21 — *Push subscription* para a própria API

Decisão. O canal analítico é consumido por *push subscription* direto num *endpoint* da própria API (`POST /telemetry/ingest`), e não por um `{contrato}-telemetry-worker` separado.

Alternativa descartada. *Worker* dedicado, como previa o desenho original de infraestrutura. Seria um Cloud Run a mais com *deploy*, log, escala e *alerting* próprios.

Consequências. A rota grava dado de estudante e **não** passa pelo `OidcGuard`, então precisa de tranca própria: `PubSubPushGuard` verifica emissor Google, `aud` e `email` da conta de serviço de *push*, e **falha fechada** sem configuração. O Terraform casa os dois lados. Gravação idempotente por `ON CONFLICT`, porque a entrega é *at least once*.

Gatilho. Volume que justifique isolar o consumo do tráfego de estudante.

18.23 ADR-22 — exp04ef1b 1.1.0: *briefing* antes do disfarce

Contexto. Até a 1.0.0 o estudante da exp04ef1b caía direto no *feed* de *Shorts* sem nenhum enquadramento: a primeira vez que a palavra "missão" aparecia para ele era no diálogo que abria sozinho depois do terceiro salvo — isto é, *depois* de a medida já ter sido tomada. Não havia contador visível de favoritos, não havia bandeja, e o teto de três se manifestava como um *toast* de 5,2 s. O Figma de redesenho (arquivo `oX1NGR8sPyduAS52hDaJFX`, seis telas de 1440×900, público de nove anos) acrescentou o enquadramento antes da ação e o retorno durante ela.

Decisão. Três mudanças, tomadas juntas:

1. **Tela de entrada na voz da escola.** O *briefing* usa a seção da **plataforma** (`@edu/ui/ds`) — a primeira vez que ela aparece *antes* da revelação. O critério da linha da revelação (Capítulo 14) continua sendo "quem está falando?": no *briefing* quem fala é a escola, e não há disfarce a proteger, porque a criança ainda não viu um *Short*.
2. **Conclusão pelo botão.** O terceiro salvo apenas habilita "Concluir missão" na bandeja; o diálogo abre no clique.
3. **Ganho de inscritos visível.** Um *badge* `+N` ao lado do chip do canal, por 1,4 s, quando o número sobe.

Alternativas descartadas. (a) Reproduzir o *wireframe* ao pé da letra, com o azul e o tema claro fixo dele — forçaria a paleta da plataforma para dentro da ficção e quebraria o seletor claro/escuro do estudante; o Figma é *wireframe*, e vale como especificação de estrutura, texto e estados, não de cor. (b) Reduzir o catálogo a oito vídeos, como o *wireframe* ilustra — mexeria no instrumento (o balanço 10 *fake* / 11 factual) e na comparabilidade com dados já coletados. (c) Manter a abertura automática do diálogo no terceiro salvo, com o botão como caminho alternativo — duas portas para a mesma coisa, e a primeira empurra a criança para a virada no instante do toque. (d) Animar só o total, sem o *badge* de delta.

Consequências ruins, aceitas. O *badge* torna legível **durante** a missão que um vídeo vale +1.000 inscritos e outro +50, ou seja, expõe o padrão que o *ranking* existe para revelar. Foi aceito porque o total já se movia — o delta sempre foi derivável de dois quadros seguidos — e porque a nota da etapa 3 do Figma decide isso explicitamente: o ganho varia, e o motivo **não** é explicado ali. A tela de entrada também gasta um toque a mais antes do *feed*, e um estudante apressado a atravessa sem ler.

O que não mudou. As curtidas continuam fora do cálculo; o gabarito (`factual`) continua no servidor e fora do *wire* — há teste que enumera os campos públicos do catálogo; o *score* continua saindo de `POST /unootube/credibilidade`. Nenhuma rota nova, nenhuma migração.

Efeito colateral corrigido no hub. O *feed* passou a montar depois de um clique *dentro* do `iframe`, e isso revelou um defeito que já existia: clicar dentro do `iframe` tira o foco da janela do hub, o `ExpFrame.tsx` traduzia esse `blur` em `hub:pause`, e o `focus` correspondente nunca voltava enquanto o estudante continuasse ali — o vídeo parava no primeiro toque em "Salvar" e não

voltava mais, em qualquer versão. A correção é uma linha: quando `document.activeElement` é o próprio `iframe`, o foco foi para a experiência e não há perda de visibilidade nenhuma.

Gatilho de reavaliação. Distribuição de `fontesConfiaveis` da turma subindo de forma anômala — o *badge* é o primeiro suspeito. Ou o professor relatando que a turma atravessa o *briefing* sem ler, o que pediria que ele fosse do professor e não da tela.

18.24 ADR-23 — *Function calling* entra no `AiProvider`

> **Contexto**
>
> A exp02em1b (Missão Tempestade) tem um agente conversacional em que **nenhum número pode sair do modelo de linguagem**: as estimativas vêm de um motor probabilístico determinístico, e o agente apenas narra o que a ferramenta devolveu. O projeto de origem implementava isso com um laço de *function calling* do SDK do vendor — até seis idas e voltas por turno, com três tentativas que reaproveitam o motivo da recusa anterior. A interface `AiProvider` desta plataforma tinha `generate`, `generateJson` e `generateImage`: nenhuma delas conversa.

Decisão. A interface ganha um quarto método, `conversar`, com tipos neutros de vendor (`AiFerramenta` reusa o `AiJsonSchema` que já existia). O laço de ferramentas, a preservação da assinatura de raciocínio e o *retry* que aprende ficam dentro de `vertex.provider.ts`; as regras de recusa ficam no domínio, passadas como *callback* `validar`. `tokensUsados` é a soma de todas as voltas e de todas as tentativas, **inclusive as rejeitadas** — é o que foi pago.

Alternativas descartadas.

- *A experiência falar com o vendor*: violaria a regra de que só `src/ai` conhece o provedor, e poria a chave e o protocolo do vendor no caminho de uma experiência.
- *Módulo da experiência chamar o SDK do vendor diretamente*: mesmo problema um nível acima — o dia de trocar de provedor passaria a exigir mexer em cada experiência que conversa.
- *Resposta estruturada em vez de ferramentas* (`generateJson` devolvendo uma lista de ações): funcionaria para um turno simples, mas o agente encadeia *carregar fonte* → *cruzar dados* → *narrar* decidindo cada passo com o resultado do anterior. Uma lista de ações declarada de antemão não tem essa realimentação.

Consequências.

- Qualquer experiência futura ganha agente com ferramentas de graça, e a troca de provedor continua sendo um arquivo novo em `src/ai`.
- *Ruim*: `conversar` é obrigatório na interface, então as três implementações existentes tiveram de ganhá-lo — inclusive os *fakes* dos testes de outros módulos.

- *Ruim*: o `MockProvider` não conhece os *slugs* do domínio, então chama a primeira ferramenta declarada com argumento vazio. Em desenvolvimento o mapa não enche pela conversa; enche pelo painel.
- Três defeitos do provedor ficaram cobertos por teste com `fetch` dublado: a assinatura de raciocínio reenviada *verbatim* (sem ela o Vertex recusa a continuação com 400 e a narrativa nunca chega), o descarte das partes de pensamento (senão o raciocínio do modelo aparece na tela do estudante) e o teto de voltas (modelo preso pedindo ferramenta vira recusa, não bolha vazia).

Gatilho de reavaliação. Um provedor cujo protocolo de ferramentas não caiba na assinatura de `conversar` — por exemplo, ferramentas paralelas na mesma volta. Ou latência de turno acima do que a sala tolera, que empurraria para `streamGenerateContent` e mudaria a forma do retorno.

18.25 ADR-24 — Teto de custo de IA por SESSÃO

> **Contexto**
>
> O `QuotaService` limita tokens por usuário e por dia, com teto por perfil (`AI_DAILY_TOKEN_QUOTA_PER_ESTUDANTE` valia 20.000). Esse número foi dimensionado para o tutor da exp08ef1b: uma conversa curta. O agente da exp02em1b tem *prompt* de sistema de cerca de 11.500 caracteres, reenviado inteiro a cada volta de ferramenta e a cada tentativa; uma partida de sete turnos fica na ordem de 100 a 200 mil tokens — duas ordens de grandeza acima.

Decisão. O controle que importa passa a ser **por sessão**, no módulo da experiência: `tokens_gastos` e `chamadas_ia` na linha da partida, com tetos em `regras/defaults.ts`. A quota diária continua como **fusível da plataforma** e sobe para acomodar duas partidas. A checagem acontece antes de chamar o provedor, e o consumo é registrado no `finally` — inclusive quando o turno falha, porque o token foi gasto de todo modo.

Alternativas descartadas.

- *Quota por experiência no `QuotaService`*: um limite por `expId` em variável de ambiente. Não mexeria no teto do tutor, mas complicaria um serviço que hoje é simples e por perfil — e continuaria sendo por dia, quando o que tem começo e fim é a sessão.
- *Encolher o instrumento para caber em 20.000*: reduzir o *prompt*, as voltas de ferramenta e o histórico. Cabe na quota atual e mexe justamente no que foi calibrado em *playtest*.
- *Teto em centavos*, como no projeto de origem: preço depende de tabela do vendor, que muda sem aviso e não é conhecida pela aplicação. Token é o que o provedor devolve.

Medição — 2026-08-27, `gemini-3.6-flash` no endpoint `global`

Uma **partida completa** (8 turnos, 7 chamadas de IA, do briefing ao desfecho) custou **71.397 tokens** em 40 segundos.
Por turno: 7,5k a 8k na maioria, com dois picos (20,4k e 16,4k) nos turnos de mais voltas. O turno de conversa pura — "não entendi, explica de novo" — custou 3.846 e **não debitou pergunta**, que é a regra funcionando.
E o que domina o custo **não é o tamanho do contexto: é o número de voltas** de ferramenta, porque cada volta reenvia o histórico inteiro. Um turno de duas voltas custou 11.481; o mesmo turno com quatro fontes já carregadas e seis falas de histórico, mas uma volta, custou 8.463 — *menos*, com mais contexto.

Os números, com a aritmética. `TETO_TOKENS_SESSAO` = 150.000: o dobro de uma partida completa medida, o que cabe a missão inteira com folga e ainda corta antes de o custo virar problema. `TETO_CHAMADAS_SESSAO` = 24: ao custo medido o teto de *tokens* morde primeiro, por volta do 19º turno — este é a rede contra o laço, não o controle de custo.

ATENÇÃO

A medição encontrou um defeito que a estimativa não teria encontrado: com `AI_DAILY_TOKEN_QUOTA_PER_ESTUDANTE` em 20.000 — o valor dimensionado para o tutor da exp08ef1b, e que estava **fixo no *compose* de produção** — a missão recusava o **quarto turno** com "a sala de operações fechou por hoje". A quota subiu para 300.000 (cerca de quatro partidas, porque "Recomeçar" abre partida nova) em quatro lugares: os dois *composes*, o *default* da validação de ambiente e o `.env.example`.

Consequências. O custo fica limitado onde ele acontece, e a mensagem de estouro é uma frase de jogo ("a sala de operações fechou por hoje") em vez de vocabulário de infraestrutura — o estudante não sabe o que é quota. *Ruim*: são dois tetos no mesmo caminho, e um estouro exige olhar os dois para saber qual disparou.

Gatilho de reavaliação. A primeira medição real. E a terceira experiência com agente conversacional, que tornaria o teto por sessão um padrão a subir para a plataforma.

18.26 ADR-25 — O Relógio da Aula é tabela da experiência

Contexto

A Missão Tempestade tem uma restrição de tempo que é parte do instrumento: cinco minutos de relógio de parede valem a hora do cenário. No projeto de origem esse cronômetro armava sozinho quando o estudante entrava na investigação — e numa sala isso é uma corrida com largadas diferentes: quem chegou primeiro terminava antes de o professor acabar de explicar. As colunas do relógio moravam em `turmas`.

Decisão. O relógio é da **turma**, e vive em `exp02em1b_relogio_turma` — tabela *da experiência*, com `escola_id` e RLS como as outras. Quem dá a largada é o professor, no painel, e ela vale para a sala inteira ao mesmo tempo. Enquanto `iniciado_em` é nulo o estudante vê o cronômetro parado e o campo de fala desabilitado.

Alternativa descartada. *Colunas em `turmas`*, como no origem. `turmas` serve a plataforma inteira: uma coluna de cronômetro de uma missão faria toda experiência futura herdar o vocabulário dela, e a próxima que precisasse de tempo teria de escolher entre reusar colunas com nome errado ou adicionar as suas. É o mesmo raciocínio do prefixo `exp_NN_` (**ADR-20**).

Consequências.

- A sala começa junto, e a espera passa a ser a aula — não uma tela travada. A ficha do professor abre com essa faixa, porque "não abre" é o suporte que ela evita.
- A extensão de tempo usa um **pivô**: sem ele, esticar de cinco para sete minutos jogaria a cidade do minuto 48 para o 34, e a chuva andaria para trás na cara da turma.
- A largada é **idempotente** — é exatamente o clique que um professor nervoso dá duas vezes.
- Estudante sem turma vê "aguardando", que é a verdade. O *fallback* do origem armava um cronômetro individual, e com ele um estudante solto jogava sem largada.
- *Ruim*: o professor passa a ter uma ação sem a qual a aula não acontece. É dependência operacional nova, e o suporte precisa saber dela.

Gatilho de reavaliação. Uma segunda experiência com tempo de sala — aí o relógio da turma vira serviço da plataforma, não tabela de uma missão.

18.27 ADR-26 — Sem prazo de retenção na exp02em1b

> **Contexto**
>
> O projeto de origem tinha `expires_at` de 30 dias em `messages` e `marcos`, com uma rota de *cron* que o cumpria. Esta plataforma não tem *job* agendado, por decisão — e já registrou duas vezes o custo de uma coluna de prazo sem rotina: o `expires_at` da app-70 e o `turmas.consentimento_em` do aplicativo de origem, onde a regra saiu do código, a coluna ficou, e quem leu o *schema* acreditou nela.

Decisão. Nenhuma das cinco tabelas da exp02em1b tem coluna de prazo. No lugar entra **um** caminho de exclusão que existe de fato: "zerar a turma" (`DELETE /unoogpt/professor/turmas/:turmaId`), acionado por quem tem a posse pedagógica, apagando as partidas *daquela turma* e devolvendo o relógio ao início.

Alternativas descartadas.

- *Portar o `expires_at` de 30 dias*: prazo que nenhuma rotina cumpre é pior que prazo nenhum, porque cria a crença de que há retenção.

- *Trazer o prazo E o job*: seria o primeiro agendador da plataforma, com o custo operacional que a decisão de não ter agendador evitou. Se um contrato exigir prazo, ele volta **junto com a rotina**, nunca só a coluna.
- *`DELETE /unoogpt/dados` no estilo do bem-estar*: lá o professor é dono das linhas; aqui o dado é do estudante, e um botão "apagar tudo" no professor seria posse que ele não tem.

Consequências. O relógio é apagado junto, e o diálogo diz por quê: rodar de novo com a aula "já começada" faria a segunda rodada nascer no meio da tempestade. *Ruim*: sem prazo, a transcrição de um menor de idade fica no banco até alguém apagar — e a pergunta de retenção continua aberta para `telemetria_eventos` e para a plataforma inteira, não só para este módulo.

Gatilho de reavaliação. Exigência contratual de prazo, ou a primeira rotina agendada da plataforma — a partir dela o custo marginal de cumprir um prazo cai muito.

18.28 ADR-27 — Transcrição é dado do instrumento

> **Contexto**
>
> A missão inteira acontece numa conversa em texto livre, escrita por menor de idade. O professor precisa ler essa conversa: é nela que aparece *como* a criança perguntou, e o relatório sem ela é uma lista de contagens. Ao mesmo tempo, o `AGENTS.md` registra a trava oposta na app-70 — o campo de texto livre foi removido do *payload* porque era o caminho por onde o nome de uma criança chegaria a um JSONB de 90 dias.

Decisão. A transcrição vive em coluna dedicada (`exp02em1b_mensagens.texto`), com RLS por escola, e é servida **apenas ao professor**: `UnoogptTurmasController` tem `@Roles("professor", "admin-contrato")` — e `admin-contrato` é a exceção cross-escola de sempre, não o gestor —, enquanto o agregado do gestor está em `UnoogptAgregadoController`, que **não tem nenhuma rota que aceite `estudanteId`**. Ela nunca vai a log nem à telemetria.

Alternativa descartada. Um controller só, com `@Roles` incluindo gestor e uma exceção dentro do método que devolve a criança. É a forma que vaza: basta um método novo esquecer a exceção. Com dois controllers, nenhum descuido futuro leva o gestor à conversa de uma criança, porque não existe rota para isso no controller dele.

Consequências.

- O relatório individual abre pelo **nível** — graus, com `zero` ("teve a chance e não foi") distinto de `none` ("a oportunidade não chegou") — antes do que a criança fez. É a regra da app-70, e existe para o adulto não concluir sobre a criança o que o instrumento não mediu.
- A **listagem** da turma não tem coluna de resgatados: listagem é ordenável, e ordenar crianças por acerto viraria *ranking*. O *smoke* visual falha se uma aparecer.
- A telemetria analítica carrega contagem, nunca o texto.

- *Ruim*: o dado mais sensível da experiência não tem prazo (**ADR-26**), e a única forma de apagá-lo é uma ação do professor.

Gatilho de reavaliação. Pedido de visão de rede sobre conduta — aí o caminho é agregado sem identidade no controller do gestor, nunca abrir a linha da criança.

18.29 ADR-28 — Identidade por pessoa; OneRoster na fronteira

> **Contexto**
>
> As experiências foram portadas de projetos onde **cada atividade tinha o próprio cadastro de usuário**. No monorepo a unificação técnica já existia — toda experiência recebe `estudante.id` pelo `hub:init` e grava `estudante_id` com chave estrangeira para `users` —, mas três resíduos impediam que "acompanhar por estudante" fosse verdade:
>
> 1. A conta era de **assento**, não de pessoa. Está escrito no *schema* desde o porte: `exp08ef1b_runs.nome_no_app` existe "porque a conta é do APARELHO: outra turma reusa o mesmo login". O nome real da criança vivia dentro de UMA experiência, e nenhuma das outras o enxergava.
> 2. Não existia **enturmamento**. O vínculo era a coluna `users.turma_id`: um estudante cabia em uma turma só, o PROFESSOR não tinha turma nenhuma — e por isso todo professor enxergava todas as turmas da escola — e não havia ano letivo. Em fevereiro o 8A vira 9A e todo o acumulado do ano anterior passaria a ser lido pela turma nova, em silêncio.
> 3. A entrada de pessoas fazia *upsert* por `usuario` e exigia **senha em texto claro no arquivo**. Corrigir um login criava um segundo estudante, com o histórico preso no primeiro.

Decisão. O modelo interno passa a ser **isomórfico ao OneRoster 1.2** (1EdTech): entram `periodos_letivos` (`academicSessions`) e `matriculas` (`enrollments`, a tripla usuário × turma × escola com papel e datas), mais as colunas de identidade em `users`, `turmas` e `escolas` (`matricula_externa`, `fonte_id`, `codigo_inep`).

O padrão vive **só na fronteira** `apps/api/src/rostering/` — a mesma disciplina de `src/ai`, onde apenas o *provider* conhece o fornecedor. Nada do domínio conhece `sourcedId`, o contrato do `hub:init` não muda, e os nomes seguem em pt-BR.

Alternativas descartadas.

- *Modelar do zero, sem padrão*: chegaríamos a quase a mesma tabela. O padrão não é o motivo — é a conferência de que o desenho está certo, o vocabulário de graça e um argumento de edital.

- *LTI 1.3 / NRPS*: responde "quem chegou agora", em tempo real, e pressupõe estar embutido num LMS. Não provisiona quem ainda não entrou, que é exatamente o caso da matrícula da rede antes do ano letivo.
- *Ed-Fi*: padrão de *data warehouse* de rede; escopo muito maior que o problema, para uma equipe de uma pessoa.
- *Esperar o padrão nacional*: o MEC publicou a Portaria 269, de 24/03/2026, criando a Infraestrutura Nacional de Dados da Educação (EducaDados) e o Padrão Nacional de Interoperabilidade, com implantação gradual e especificação técnica ainda não publicada. Adotar o *modelo* agora e manter o *formato* atrás de adaptadores é o que serve aos dois futuros.
- *Adotar também os bindings do padrão*: o CSV do OneRoster são sete arquivos com identificadores globais; nenhum secretário de escola os preenche. A tela oferece uma **planilha simples**, de uma linha por pessoa, e o mesmo núcleo aplica o lote.
- `demographics` *do OneRoster*: raça e etnia no vocabulário do censo dos Estados Unidos, além de nascimento e sexo. Dado sensível (LGPD art. 11) que a plataforma não usa para nada — não entra nem como coluna.

Consequências.

- `users.turma_id` **sobrevive**, com novo significado: cache da matrícula principal. O *login* e o *claim* `turma_id` do *token* (contrato congelado) o leem antes de haver escola no contexto, onde `matriculas` — que tem RLS — não é alcançável.
- "Coordenação" continua sendo `gestor`: `PAPEIS` é contrato congelado entre o hub e a API, e um quinto papel atravessaria *middleware*, *guards* e RLS. A planilha aceita sinônimos (`coordenacao`, `diretor`, `principal`) e normaliza.
- *Ruim*: quatro tabelas e um módulo a mais, e uma migração nas duas tabelas mais centrais três meses antes de produção.
- *Armadilha registrada*: consulta que leia `matriculas` fora de `comTenancy` volta **vazia**, sem erro. O sintoma é lista de pessoas "sem turma".

Nota de implementação — o binding CSV, arquivo por arquivo. O padrão empacota os sete arquivos num zip com `manifest.csv`. Aqui cada um é enviado em uma chamada (`POST /rostering/oneroster` na ordem de dependência: empacotamento é embalagem, não semântica, e aplicar um por vez dá o mesmo resultado sem dependência de descompactação e sem lote pela metade guardado em lugar nenhum. Fora de ordem também não corrompe: a linha que aponta para um `sourcedId` inexistente volta no relatório **dizendo qual arquivo falta**, em vez de criar meia entidade. Quatro decisões dentro dessa leitura:

- `orgs.csv` só **aprende** o código INEP da escola desta importação. Escola é *tenant*: criar um a partir de arquivo enviado por *upload* seria abrir um inquilino com *stack*, RLS e dados de menores por trás de um clique.
- Na 1.2 o papel saiu de `users.csv` e foi para `roles.csv`. A coluna `role` é aceita quando vem (provedores seguem emitindo a da 1.1); sem ela, a pessoa nasce como **estudante** — o menor privilégio — e o relatório diz quantas linhas dependem do `roles.csv`. Falhar fechado: um

professor que entra como estudante perde acesso e reclama; um estudante que entrasse como professor leria a turma inteira.

- A coluna `password`, que o padrão permite, é **ignorada** (ADR-29) — e a contagem de linhas que a traziam vai ao relatório, para não sumir em silêncio.
- Em `bulk`, ausência de linha **não** desativa ninguém: é contada e listada. Lote completo que chegou pela metade não pode apagar uma rede por omissão.

Gatilho de reavaliação. Publicação da especificação técnica do EducaDados — em especial se ela adotar o CPF como identificador do estudante, que entra em coluna de reconciliação de acesso restrito e **nunca** em telemetria, JSONB ou *claim* de *token*.

18.30 ADR-29 — Senha provisória com troca obrigatória

> **Contexto**
>
> Antes da integração com o WSO2, alguém precisa criar as contas e entregar uma credencial a 10 mil pessoas que nunca acessaram a plataforma. O formato anterior resolvia isso com uma coluna `senha` na planilha — arquivo que circula por e-mail, WhatsApp e *pen drive*, e que fica no computador da secretaria.

Decisão. A planilha **não tem coluna de senha**. A senha inicial é escolhida na tela de importação, em dois modos: uma para o lote inteiro, ou uma por pessoa (recomendado para estudantes). Quem nasce da importação recebe `senha_provisoria = true`, e o primeiro acesso cai em `/trocar-senha` sem saída. Reimportar **não** redefine a senha de quem já entrou.

Alternativas descartadas.

- *Manter a senha na planilha*: é o vazamento por construção, e a reimportação passaria a redefinir a senha de quem já trocou.
- *Delegar tudo ao WSO2 agora*: o piloto precisa de contas antes do provedor de identidade existir, e a plataforma continuaria sem caminho para a escola sem federação.
- *Link mágico por e-mail*: a maioria dos estudantes da rede pública não tem e-mail institucional em uso, e a plataforma não envia e-mail hoje.

Consequências.

- A trava é **tripla**: *middleware* (páginas), BFF (dados) e `/api/exp-token`. Só com o *middleware*, bastaria digitar `/exp/...` para pular a troca.
- Ao trocar, a sessão é **refeita** com a senha nova, não remendada: o *cookie* antigo ainda diz "precisa trocar", e sem refazê-lo o *middleware* devolveria a pessoa à mesma tela — agora com a senha anterior já inválida. É o beco sem saída mais fácil de criar neste fluxo.

- As senhas geradas são devolvidas **uma vez**, para impressão. Não ficam em coluna nem em *log*; depois disso o caminho é o reajuste de senha pelo gestor, que remarca a conta como provisória.
- *Ruim, e dito na própria tela*: com uma senha só para o lote, qualquer estudante entra na conta de outro até o primeiro acesso dele. Por isso o modo "uma por pessoa" existe e é o recomendado para turma grande.
- O *seed* de desenvolvimento nasce com `senha_provisoria = false`: prender os usuários de demonstração na troca pararia os seis suítes de ponta a ponta na mesma tela.

Gatilho de reavaliação. Entrada do WSO2 em produção — a partir daí a senha local vira caminho de exceção (escola sem federação), não o principal.

18.31 ADR-30 — A rede: lotação por escola e escola ativa no *token*

> **Contexto**
>
> Vender a uma rede estadual não é vender a uma escola grande. São milhares de unidades, professores que dão aula em mais de uma, e crianças que se transferem entre elas. O modelo tinha `users.escola_id` **único**, e ele é a chave de *tenancy*: circula no *claim* `escola_id` do JWT e no `set_config` da RLS. Um professor em duas escolas não caberia nele.
> O tamanho do problema, medido: **38 tabelas** carregam `escola_id` e **23 arquivos** da API derivam a escola do *token*. (Números de 2026-09-05; eram 31 e 19 quando a decisão foi tomada. A diferença é o custo que a alternativa descartada teria hoje — ele só cresce, e é por isso que a medida é refeita em vez de citada.)

Decisão. O *token* continua carregando **UMA** escola. Nenhuma das 38 tabelas muda, nenhum dos 23 arquivos muda, a RLS continua sendo `escola_id = current_setting('app.escola_id')` e a regra inviolável nº 1 continua literalmente verdadeira. O que entra é a camada *acima*:

- `lotacoes` — o vínculo PESSOA × ESCOLA, datado. "Lotação" é o termo que a rede pública usa para a alocação de um servidor a uma unidade. Professor e gestor **acumulam**: lotação nova não encerra a anterior. Estudante tem **uma ativa**, garantida por índice único parcial no banco.
- **Escola ativa na sessão.** Quem tem mais de uma escolhe no cabeçalho; a troca passa pelo servidor (`POST /api/escola-ativa`), que valida contra `GET /escolas/minhas` — **nunca** contra a lista que o *cookie* carrega. *Cookie* é dado do cliente, e aceitar a escola que ele pedir deixaria qualquer professor ler qualquer unidade da rede.
- `users.escola_id` passa a ser **cache** da lotação principal — o mesmo papel que `turma_id` já tinha em relação a `matriculas`.

Alternativas descartadas.

- *Token com várias escolas e RLS em `= ANY(...)`*: tocaria as 38 tabelas e trocaria a garantia "esta conexão vê UMA escola" por "vê um conjunto". A propriedade que sustenta o isolamento de um contrato B2G é justamente a primeira.
- *Um usuário por escola (dois cadastros para o mesmo professor)*: é o que acontece hoje sem esta camada, e é o problema — duas senhas, dois logins, e nada ligando as duas metades da mesma pessoa.
- *`lotacoes` com RLS*: ela é lida no LOGIN, antes de se saber a escola. Uma política por `escola_id` esconderia justamente a resposta ("quais escolas são suas"), e o sintoma seria o professor de duas unidades não conseguindo trocar para a segunda. Fica fora, como `users`/`turmas`, com filtro explícito nos *services*.

Consequências.

- Criar escola é `admin-contrato` e ninguém mais — inclusive no `orgs.csv`. Escola é *tenant*: abrir inquilino com RLS, dados de menores e *stack* por trás não é operação de quem administra uma unidade.
- *Ruim*: a escola ativa é estado de SESSÃO. Duas abas do mesmo professor em escolas diferentes brigam pelo mesmo *cookie* — a última troca vence, e a outra aba passa a mostrar a escola nova sem avisar.
- *Ruim*: o campo extra que eu pus dentro do item da `SideBar` para filtrar o menu virou atributo no DOM (o tipo do DS tem índice aberto e ele espalha o que não conhece). A restrição foi para fora do item; fica registrado porque é o tipo de erro que o *typecheck* não pega.

Gatilho de reavaliação. Um contrato em que a MESMA pessoa precise ver duas escolas ao mesmo tempo na mesma tela — aí a conversa volta para o *token* com conjunto de escolas, e com ela o custo nas 38 tabelas.

18.32 ADR-31 — CPF como identidade, guardado só em *hash*

> **ATENÇÃO**
>
> **SUPERADA pela ADR-33.** O *hash* entregava o que a *plataforma* precisa do CPF e não o que o *gestor* precisa: ver o número. Fica registrada porque o raciocínio continua valendo para qualquer dado que a plataforma só precise *comparar* — e porque a lição de processo é a parte mais útil dela.

> **Contexto**
>
> Numa rede, a criança troca de escola e o dado dela precisa continuar sendo dela. Nome muda (casamento dos pais, correção de registro), *login* muda, matrícula muda a cada rede — e a escola de origem e a de destino não compartilham nenhuma dessas chaves.

> Sem um identificador imutável, cada transferência cria uma segunda pessoa e o histórico nasce partido em duas.
> O identificador que o Brasil usa é o CPF, e é o que a Portaria MEC 269/2026 adota como chave do estudante no EducaDados. É também o dado pessoal mais sensível que esta plataforma tocaria, de menores de idade, num banco que já guarda perfil de valores e *check-in* de saúde.

Decisão. O CPF é a chave de identidade e **nunca** é guardado. O banco tem `users.cpf_hash` = `HMAC-SHA256(cpf, CPF_PEPPER)` e `users.cpf_final`, os três últimos dígitos em claro para confirmação humana na tela. O *hash* é **determinístico** de propósito: é o que permite procurar (basta *hashear* o que o gestor digitou) e reconhecer a mesma pessoa. Único por **contrato**, não por escola — é a identidade da pessoa na rede inteira.

Alternativas descartadas.

- *Guardar o número em claro*: dá tudo o que o *hash* dá e mais um acervo de CPF de menores, que a plataforma não precisa para nada — ela não emite documento nem presta contas ao fisco. O SIS da rede já tem o número.
- *Hash com sal por linha* (`bcrypt`): inutilizaria o campo como identidade. Sem determinismo não há busca nem reconhecimento, que é a única razão de o CPF estar aqui.
- *Hash sem pimenta*: CPF tem 11 dígitos e espaço de busca trivial — um SHA-256 puro é reversível por força bruta em minutos. A pimenta é o que torna o *hash* inútil fora desta instalação.
- *Cifrar em vez de hashear*: exigiria guardar a chave capaz de devolver o número, e o número é exatamente o que não se quer poder devolver.

Consequências.

- `CPF_PEPPER` **não muda**. Trocá-la invalida todos os *hashes* e a plataforma deixa de reconhecer quem já cadastrou — o sintoma é toda criança transferida virando cadastro novo. Ausente, a coluna `cpf` da importação é **recusada** com mensagem explícita, nunca aceita e guardada de outro jeito.
- O dígito verificador é conferido na leitura da planilha. Não é preciosismo: CPF digitado errado numa planilha de 10 mil linhas cria uma identidade fantasma que nunca vai casar com a da rede, e o erro só aparece meses depois, na primeira transferência.
- O CPF é **opcional**. Rede sem o dado continua funcionando com `matricula_externa` como chave — e perde só o reconhecimento automático na transferência.
- *Ruim*: três dígitos em claro são identificáveis em combinação com o nome. É o preço de o gestor conseguir confirmar que é a pessoa certa antes de transferir uma criança.
- O teste estrutural `identidade-unica.spec.ts` falha se aparecer qualquer coluna com `cpf` no nome fora de `users.cpf_hash` e `users.cpf_final`.

Gatilho de reavaliação. A especificação técnica do EducaDados exigir *enviar* o CPF de volta à rede — aí ele volta a entrar, e a decisão passa a ser onde a chave de cifra mora, não se o número existe.

18.33 ADR-32 — Transferência: o dado fica, a identidade segue

> **Contexto**
>
> "Os dados do estudante seguem o estudante, independente da escola" é requisito de produto. E colide de frente com a regra inviolável nº 1: tudo o que a criança produziu tem `escola_id` da escola onde produziu, com RLS por cima. Depois de uma transferência, a escola nova não alcança nada disso — e a antiga continua alcançando tudo.

Decisão. O **dado fica onde foi produzido**; o que segue o estudante é a **identidade**. A transferência encerra a lotação e as matrículas na origem, abre na destino e move o *cache* `users.escola_id` — e mais nada. Para a leitura atravessada existe **uma** porta: `GET /estudantes/:id/historico`, que devolve **participações** (em que escola, em que turma, em que missão, quando), nunca o conteúdo dos instrumentos.

Alternativas descartadas.

- *Reescrever `escola_id` nas 38 tabelas na transferência*: apaga de onde o fato veio. "Esta criança respondeu isso na escola A, em 2026" deixaria de ser recuperável, e um *UPDATE* em massa por transferência é a operação mais fácil de deixar pela metade.
- *Afrouxar a RLS para o `estudante_id` atravessar*: transformaria a exceção numa regra geral. A porta única é auditável; uma política mais larga não é.
- *O histórico devolver o instrumento completo*: abriria, por uma tela de secretaria, o retrato de valores da Vila e a transcrição do unooGPT — o que **ADR-12**, **ADR-27** e **ADR-10** fecharam. Quem lê o instrumento é quem tem a posse pedagógica na escola onde ele aconteceu.

Consequências.

- O `historico` só abre para quem tem **lotação na escola onde o estudante está hoje** — é a unidade responsável por ele — ou para `admin-contrato`.
- A transferência é a única operação que troca o `app.escola_id` **no meio de uma transação**: ela encerra na origem e abre no destino, e sem trocar o contexto o *update* na origem não veria linha nenhuma (a RLS o esconderia, sem erro, e a criança ficaria matriculada nas duas escolas). O `set_config(..., true)` é local à transação, então a conexão volta limpa ao *pool*.
- Quem recebe a criança é a escola de **destino**: um gestor só transfere PARA a própria escola, e o caminho normal é importar a planilha com o CPF — o importador reconhece e transfere.
- *Ruim*: o professor da escola nova vê que a criança participou de uma missão na escola anterior, mas não o que ela respondeu. Para continuar um trabalho pedagógico iniciado em

outra unidade, isso é pouco — e é deliberado até haver uma regra de consentimento que diga o contrário.

Gatilho de reavaliação. Pedido pedagógico formal de continuidade entre escolas da mesma rede — aí a conversa é sobre consentimento e sobre QUAL instrumento atravessa, um a um, nunca sobre abrir a fronteira inteira.

18.34 ADR-33 — O CPF é guardado, e o gestor o vê

> **Contexto**
>
> **ADR-31** decidiu guardar apenas `HMAC-SHA256(cpf, CPF_PEPPER)` e os três últimos dígitos. A decisão pesou o risco de o banco acumular CPF de menores e **não perguntou quem usa o dado**.
> Quem usa é o gestor da escola, e ele precisa **ver o número**: confere identidade contra o sistema de gestão da rede, preenche documento de transferência e atende a família no balcão com o documento na mão. *Hash* é de mão única — não existe configuração que o exiba, porque não existe chave. Não era uma limitação a ajustar; era a funcionalidade ausente por construção.

Decisão. O CPF é guardado: `users.cpf`, os 11 dígitos sem pontuação (a formatação é da tela, e um *CHECK* no banco impede que "529.982.247-25" e "52998224725" coexistam como duas pessoas). Único por **contrato**, não por escola. Exibido formatado na lista do gestor, buscável com ou sem pontuação, e corrigível na edição — o balcão erra, e um dígito trocado numa planilha de dez mil linhas só aparece meses depois, na primeira transferência.

Sobre a base legal. CPF é dado pessoal **comum**, não dado sensível do art. 11 da LGPD — que trata de saúde, raça, religião, filiação e biometria. O tratamento tem base clara num contrato de execução de política pública de educação, e é o que todo sistema de gestão escolar brasileiro guarda. O cuidado que a plataforma tem com `app_91_checkins` (saúde) não é o mesmo caso.

Alternativas descartadas.

- *Manter o hash e exibir os três dígitos*: é o que havia, e não atende — três dígitos não servem para conferir documento nem para preencher formulário.
- *Hash para identidade + coluna cifrada para exibição*: duas representações do mesmo fato, uma chave a operar e a rotacionar, e um vazamento de *dump* + chave devolve o número de todo modo. O ganho real sobre a cifra de disco que o Cloud SQL já aplica é pequeno; a complexidade é permanente.
- *Exibir mascarado na lista e completo só no cadastro*: foi considerado e recusado pelo dono do produto — o gestor confere pela lista, e obrigá-lo a abrir cada pessoa transforma uma conferência de turma em trinta cliques.

Consequências.

- **O que sustenta a decisão não é a coluna, é a contenção.** `users.cpf` é o ÚNICO lugar onde o número existe: nunca em *log* (nem em mensagem de erro), nunca em telemetria, JSONB, *claim* de *token* ou *bundle* de experiência. O teste estrutural cobra pelos dois lados — que só exista essa coluna, e que nenhum JSONB de `telemetria_eventos` ou `progresso` carregue onze dígitos seguidos, que é o caminho mais fácil de o número escapar.
- O índice único é da **rede**, não da escola. Duas importações simultâneas com o mesmo CPF colidem no banco — é o comportamento correto, e o efeito colateral apareceu primeiro nos testes, onde duas suítes usavam o mesmo número.
- `CPF_PEPPER` deixou de existir.
- *Ruim*: o banco passa a ser um acervo de CPF de menores de idade. A proteção é de acesso (só gestor e admin-contrato alcançam a rota) e de contenção, não de criptografia de aplicação.
- *Lição de processo, registrada porque custou retrabalho*: uma escolha **irreversível** — *hash* não volta atrás — foi tomada sem o requisito de quem usa o dado estar escrito. A pergunta "quem precisa LER isso, e como?" devia ter vindo antes da primeira migração.

Gatilho de reavaliação. Exigência contratual de cifra em nível de aplicação, ou um incidente de acesso indevido — aí a conversa é sobre onde a chave mora e quem a usa, não sobre se o número existe.

18.35 ADR-34 — A regional agrupa; quem autoriza é a lotação

> **Contexto**
>
> Uma rede estadual não chega organizada em escolas soltas. Em São Paulo são **Diretorias de Ensino**; em outras redes, polos ou secretarias municipais. Entre a escola e o contrato existe um nível intermediário real, com gente que trabalha nele: o supervisor que acompanha as cinquenta unidades da sua diretoria.
> A **ADR-30** deixou a plataforma pronta para a pessoa em várias escolas — a lotação existe, o *token* carrega uma escola por vez e o cabeçalho troca a escola ativa. O que faltava era o *agrupamento*: sem ele, pôr um supervisor nas cinquenta unidades é abrir cinquenta telas. As colunas `escolas.tipo` e `escolas.pai_id` tinham sido acrescentadas antes, "porque custam zero", e **nenhuma consulta as lia**. Era exatamente a armadilha que este projeto já documentou duas vezes — o `expires_at` sem rotina e o `turmas.consentimento_em` do aplicativo de origem: a regra sai do código, a coluna fica, e quem lê o *schema* acredita nela.

Decisão. O nível regional passa a ser **real e lido**, com um vocabulário fechado de três valores em `escolas.tipo` — `escola` (a unidade, onde há turma, estudante e dado), `regional` (o que agrupa unidades) e `rede` (a raiz). `pai_id` liga unidade a regional, e um *trigger* no banco recusa `pai_id` apontando para uma `escola` ou para si mesma.

E a decisão que dá sentido a tudo: **a hierarquia agrupa e nunca autoriza**. Estar numa diretoria não dá acesso a nada. Quem dá acesso é a lotação, uma por unidade. A regional é o *insumo* da lotação em lote (`POST /escolas/lotacoes-em-lote`) — ela monta a lista de escolas que o admin do contrato confirma antes de conceder.

Alternativas descartadas.

- *Um papel novo, `supervisor`*: `PAPEIS` é contrato congelado entre o *middleware* do *hub*, os *guards* da api e a RLS; um quinto papel toca os três. E não resolveria nada que a lotação já não resolva — o supervisor faz o que um gestor faz, em mais de uma escola.
- *Autorizar por hierarquia* ("quem está na regional lê as unidades dela"): é uma segunda fonte de verdade sobre acesso, ao lado da lotação, e as duas divergem no dia em que alguém corrige um `pai_id`. Pior: transformaria uma coluna *organizacional* em coluna de segurança, e a RLS deixaria de ser legível numa linha.
- *`regional_id` no token, com a api somando as escolas da diretoria*: quebraria a regra 1 em todas as 38 tabelas com `escola_id` e daria ao supervisor uma tela que mistura duas escolas — que é precisamente o que o isolamento lógico existe para impedir.
- *Aceitar o critério no lugar da lista* ("lote todos da regional X"): o servidor expandiria o critério e o humano confirmaria um número que não viu. A rota exige a **lista** de `escolaIds`, no máximo mil.

Consequências.

- **Nada mudou na tenancy.** O *token* continua carregando uma escola, a política de RLS continua sendo a mesma linha, e as 38 tabelas com `escola_id` não foram tocadas. O nível regional inteiro vive acima delas.
- A regional **não recebe lotação**: `lotarEmLote` ignora o que não for `tipo = 'escola'` e devolve a lista de ignoradas. Lotar alguém numa regional daria uma escola sem turma nem estudante no seletor do cabeçalho.
- O panorama (`GET /escolas/minhas/resumo`, tela `/admin/minhas-escolas`) **lista, não soma**: uma linha por unidade, com turmas, estudantes e quantos ainda não fizeram o primeiro acesso. Somar as escolas numa métrica só seria a tela que a decisão acima recusa.
- O `orgs.csv` do OneRoster passa a criar regionais a partir de `district/local` e a ligar unidades por `parentSourcedId`. Pai desconhecido vira *aviso* e a escola entra sem regional — perder a escola por causa do agrupamento seria trocar o essencial pelo acessório.
- *Ruim*: a lotação em lote é uma operação de poucos cliques que concede acesso a muitas escolas. A contenção é ser exclusiva de `admin-contrato`, exigir a lista explícita e mostrar os nomes antes de confirmar.
- *Lição de processo*: coluna acrescentada "porque custa zero" e nunca lida é dívida com aparência de recurso. Ou nasce com o consumidor, ou não nasce.

Gatilho de reavaliação. Uma rede que exija leitura agregada de fato — "o desempenho da diretoria inteira" — muda a pergunta: aí é relatório com escopo próprio (como `/vila/agregado/*` faz para o gestor), nunca alargamento do *token*.

18.36 ADR-35 — Perfil portátil: um documento assinado, não uma integração

> **Contexto**
>
> Um contrato é uma rede e uma *stack*. A **ADR-32** resolveu a criança que troca de escola DENTRO da rede: o dado fica onde foi produzido e a leitura atravessa por uma porta só. Fica de fora o caso que o dono do produto levantou: a criança que sai da rede estadual para a municipal — outro contrato, outro banco, nenhum *JOIN* possível.
> E há a restrição que decide o desenho: **não há garantia nenhuma de que a rede de destino comprou o produto**. Qualquer solução que exija duas instalações desta plataforma conversando resolve só o caso raro em que as duas existem.

Decisão. A saída é um **documento assinado**, e não uma integração. Um arquivo JSON `{ perfil, assinatura }` que a escola de origem emite a pedido do titular (LGPD art. 18, V — portabilidade) e que a escola de destino importa como um ato dela. Para uma rede que nunca vai instalar esta plataforma, o mesmo arquivo continua sendo um documento legível.

Quatro decisões dentro dessa:

- **O hub assina; a api verifica.** Assinar exige a chave privada, que só o hub tem — é a MESMA que assina os *tokens* de experiência e é publicada em `/.well-known/jwks.json`, então nenhum material de chave novo entrou. `aud: "perfil-portatil"` separa os dois usos. Verificar acontece na api porque é o passo que **não pode confiar em quem chama**: se a api recebesse o veredito pronto do hub, bastaria forjar o campo.
- **O que vem de fora nunca se mistura.** O documento vive em `perfis_portateis` e em lugar nenhum mais: não vira matrícula, progresso nem perfil da Vila, e não entra em média. É a afirmação de OUTRA instalação sobre a criança, possivelmente de outra `versao_mapeamento`. Separado, o professor lê os dois e sabe qual é qual.
- **Assinatura inválida é recusa; emissor inalcançável é rótulo.** São coisas diferentes e a tela diz qual foi. Não conseguir falar com o emissor é limitação de rede; assinatura que não confere é adulteração, e essa não entra.
- **O documento se liga a alguém que já está aqui.** Importar não cria estudante: a matrícula é ato da escola (planilha ou OneRoster), e só depois o perfil se anexa — pelo CPF, que é a única chave que significa alguma coisa numa base onde nenhum id da origem existe.

Alternativas descartadas.

- *API entre as duas instalações*: exige que as duas existam, que se conheçam e que uma libere acesso à outra. Não cobre o caso comum, que é a rede de destino não ser cliente.
- *Um banco só para várias redes* (`rede_id` em tudo): foi medido e descartado antes — e não resolveria este caso, porque a rede de destino continuaria fora da instalação.
- *Exportar CSV sem assinatura*: quem recebe não tem como saber se o arquivo foi editado no caminho, e um histórico escolar sem procedência não serve para decidir nada.
- *Assinar o documento como payload do JWS*: o arquivo viraria uma *string* opaca. Ele é entregue à FAMÍLIA, que tem direito de lê-lo; por isso o JWS cobre o *hash* canônico e o documento fica legível ao lado.
- *Buscar o JWKS de qualquer emissor*: o `emissor.hub` vem de dentro do arquivo, ou seja, de um desconhecido — seria SSRF. A allowlist `PORTABILIDADE_EMISSORES` é a decisão de confiança: o documento **seleciona** entre origens que o operador escreveu, e nunca introduz uma URL.

Consequências.

- **O CPF passa a existir num segundo lugar**, e é a única exceção à contenção da **ADR-33**: `perfis_portateis.documento`. O CPF é o CONTEÚDO do documento — é o que reconhece a criança lá fora — e é guardado *verbatim* porque alterá-lo destruiria a assinatura. A exceção está numa lista explícita dentro do teste estrutural, para que a próxima seja uma decisão e não um vazamento que ninguém viu.
- **Bem-estar (app-91) nunca entra no documento.** DASS-21 é dado de saúde, a posse é do professor e não existe rota de gestor para ele em lugar nenhum. Um documento que o gestor emite não pode carregar o que nem o gestor lê.
- `PORTABILIDADE_EMISSORES` vazia (o default, e o caso de dev) faz TODO documento entrar como `nao-verificada`. Degradar visivelmente é melhor que recusar o direito do titular — e a tela sai em âmbar, não em verde.
- **A entrada da allowlist pode apontar o JWKS** (`origem=urlDoJwks`), e é isso que torna o caminho criptográfico exercitável fora de produção. O `emissor.hub` do documento é o endereço do **navegador** — a mesma distinção de `EXP_INTERNAL_HOST`, e pela mesma razão: de dentro do contêiner da api, `localhost:3000` é a própria api. A regra de canal passou a valer sobre a URL do JWKS: `https` sempre, exceto **fora de produção**, onde `http` é aceito e avisado no *log* (quem escreveu a entrada foi o operador, no servidor, e o tráfego não sai da máquina). Em produção, `http` é recusado — a chave pública viria por um canal que qualquer um reescreve.
- Sem isso, a garantia mais importante do recurso — documento adulterado é RECUSADO — não teria prova no ambiente onde se trabalha todo dia, e o *smoke* visual registrava o caminho degradado como se fosse o normal.
- O caminho criptográfico é coberto por teste com as chaves injetadas, e foi esse teste que pegou o pior erro possível deste recurso: a classificação do erro olhava o TEXTO da exceção, e a palavra "failed" de "signature verification failed" caía na regra de erro de rede. Documento FORJADO entraria marcado "não deu para conferir" em vez de recusado. Hoje a classificação é pelo `code` da biblioteca.

- *Ruim*: reemitir gera outro documento, e a escola de destino pode acabar com dois do mesmo estudante. A lista mostra data *e hora* e permite apagar; unificar por conteúdo esconderia que foram duas emissões.

Gatilho de reavaliação. A especificação técnica do EducaDados (Portaria MEC 269/2026) definir um formato nacional de transferência de histórico: aí este documento vira um *binding* daquele formato, e o que sobrevive é a decisão de ser documento assinado em vez de integração.

18.37 ADR-36 — O OpenAPI é derivado do Zod, não escrito ao lado dele

> **Contexto**
>
> A api tinha 146 rotas em 27 *controllers* e nenhuma documentação executável (hoje são 170 operações em 32 *controllers* — o inventário cresce, a decisão não muda): quem quisesse saber o que `POST /rostering/importar` aceita lia o *controller*. A única descrição existente é o Apêndice C deste documento — prosa escrita à mão, que nada obriga a concordar com o código.
> A complicação está em como a api valida. Não há DTO de classe: são 42 *schemas* Zod declarados como `const` dentro dos *controllers* e aplicados por `.parse()`/`.safeParse()` no corpo do método, sem *pipe* compartilhado. O `@nestjs/swagger` infere esquema de classe anotada com `class-validator`; sobre Zod ele não infere nada.

Decisão. O documento é **derivado** do *schema* que a rota já usa para validar. Um decorator único, `@Contrato`, declara resumo, prosa, corpo, *query* e **resposta**; `zod-to-json-schema` traduz para o dialeto do OpenAPI 3.0; e `openapi/documento.ts` monta cada operação.

Três decisões dentro dessa:

- **O `@Contrato` não chama `@ApiOperation`/`@ApiResponse`.** Ele só grava metadata. Um decorator não enxerga o verbo da rota — o `@Get`/`@Post` pode ser aplicado depois dele, e a ordem de avaliação é de baixo para cima — então teria de ADIVINHAR o código de sucesso, e também não enxerga o `@Roles`. Quem monta a operação é o construtor do documento, que percorre as rotas já registradas e tem o verbo, o `@HttpCode` e o `@Roles` à mão.
- **Os papéis vêm do `@Roles` real**, nunca de um campo repetido no `@Contrato`: repetir abriria espaço para o documento dizer `gestor` enquanto o *guard* exige `admin-contrato`.
- **O esqueleto de caminhos vem do roteador do Nest** (`SwaggerModule.createDocument`), que é a única fonte incapaz de inventar uma rota que não existe. As duas travessias — a do roteador e a do `DiscoveryService` — precisam concordar, e divergência de um caminho sequer é erro alto na subida.

Alternativas descartadas.

- *Anotar `@ApiBody`/`@ApiResponse` à mão nas 146 rotas*: criaria uma segunda descrição do contrato ao lado da primeira, sem nada que as obrigue a concordar. É literalmente a armadilha do `expires_at` que ficou no *schema* depois que a rotina saiu do código.
- *Migrar a validação para DTOs com `class-validator`*: reescreveria a validação das 146 rotas para ganhar inferência automática, e trocaria o Zod — que a convenção da casa usa em TODO input externo, *env* incluído — por um segundo validador só nas rotas.
- *`nestjs-zod`*: exige DTO de classe (`createZodDto`) e *pipe* global. É a mesma reescrita, com uma dependência a mais.
- *`@asteasolutions/zod-to-openapi`*: monta o documento a partir de um registro próprio, fora do roteador do Nest — ou seja, pode listar rota que não existe e omitir rota que existe, que é exatamente o defeito que se quer evitar.

Consequências.

- **`.refine()` some da conversão, em silêncio.** JSON Schema não expressa função JavaScript, e a biblioteca descarta. O `AjustarMatriculaSchema` exige "`turmaDestinoId` OU `principal`" e isso não aparece como esquema. Por isso o `@Contrato` tem o campo `notas`: restrição que o Zod checa e o JSON Schema não sabe dizer vira texto na descrição. A validação não muda — o que as notas consertam é o documento não mentir por omissão.
- **Esquemas saem embutidos, sem `$ref`** (`$refStrategy: "none"`). É seguro porque não há `z.lazy` em lugar nenhum da api. Custa repetição no JSON e ganha um documento que não depende de um registro de nomes estáveis.
- *Ruim*: a assinatura publicada do `zodToJsonSchema` estoura o compilador com **TS2589** ("type instantiation is excessively deep") — o tipo de retorno instancia `JsonSchema7Type`, uma união recursiva grande, mesmo quando `target: "openApi3"` resolve para `object`. O módulo fixa localmente a assinatura que usa. Some quando a biblioteca simplificar o retorno.
- **A entrega foi em etapas, com lista que só encolhe.** Documentar 146 rotas num commit seria um *diff* que ninguém revisa, e uma resposta mal descrita passaria batida. `tests/openapi-cobertura.spe` percorre o roteador e cobra `@Contrato` de toda rota, salvo as nomeadas numa lista explícita — o mesmo molde do `SEM_CHAVE_POR_DECISAO` de `identidade-unica.spec.ts`, com a diferença de que aquelas exceções são permanentes e estas eram dívida. **A lista está vazia**: todas as rotas declaram contrato (170 operações em 2026-09-05), e rota nova sem ele é teste vermelho.
- **O Apêndice C deixou de ser escrito à mão.** Ele é gerado do `docs/openapi.json`, que é gerado do `@Contrato`. Eram 547 linhas de prosa descrevendo 146 rotas sem que nada obrigasse essa prosa a concordar com o código — exatamente o defeito que esta ADR existe para fechar, e que teria sobrevivido dentro do próprio documento que a registra.

Gatilho de reavaliação. Subir para Zod 4, que traz `z.toJSONSchema` nativo: aí a dependência de conversão sai, e o TS2589 provavelmente com ela.

18.38 ADR-37 — O *schema* de resposta é conferido, não só publicado

> **Contexto**
>
> O *schema* de ENTRADA não tem como envelhecer: a api o usa para validar em toda requisição, então errar nele quebra a rota na hora. O de SAÍDA não tinha nenhum — os *services* devolvem entidade do Prisma direto, *spreads* e formas ad-hoc — e passa a existir só para o documento.
> Afirmação que nada exercita é a mesma coisa que a coluna de prazo sem rotina que a cumpra: quem lê acredita, e ninguém percebe quando deixa de ser verdade.

Decisão. Três camadas, em ordem de força.

1. **Teste**, com `.parse()` estrito, nas suítes que já exercitam rota de verdade. É a conferência que dói.
2. **Runtime fora de produção**: o `ContratoInterceptor` roda `safeParse` na resposta e avisa no *log*. Nunca lança.
3. **Produção**: o interceptor é *no-op*.

Alternativas descartadas.

- *Validar a resposta também em produção*: um *schema* estrito demais transformaria rota que funciona em 500. O documento não vale uma indisponibilidade.
- *Só publicar, sem conferir nada*: é o `expires_at` de novo, agora em forma de *schema*.

Consequências.

- **O interceptor serializa antes de conferir** (`JSON.parse(JSON.stringify(...))`). Ele roda ANTES da serialização, onde ainda há `Date` do Prisma; sem essa volta, todo campo de data acusaria divergência falsa contra um `z.string().datetime()` correto. É também parte do motivo de ele não rodar em produção.
- **O aviso carrega só caminho e código do problema, nunca o valor.** A mensagem do Zod embute o recebido em alguns casos (`invalid_enum_value`), e a resposta desta api carrega nome de criança e CPF — regra de não registrar PII em *log*, nem dentro de mensagem de erro.
- Funcionou na primeira rota documentada: o *schema* do `POST /auth/login` declarava `escolaId` como `uuid`, e `Escola.id` é `String @id` sem *default* — o id vem da rede. O aviso apareceu no primeiro *login* real.

Gatilho de reavaliação. A conferência aparecer no perfil de latência de desenvolvimento a ponto de atrapalhar — aí ela passa a valer só em teste.

18.39 ADR-38 — O *deploy* paraleliza os *jobs*, não o portão

> **Contexto**
>
> Medido nos *runs* de 2026-08-28, um *push* comum levava ~9,6 min até a escola e um `forcar`, ~26 min — e 80% disso eram as duas imagens Docker, por duas causas que se somavam: `COPY . .` antes do `pnpm install` invalidava a camada de dependências em todo *commit* (o *cache* GHA só acertava apt e corepack), e api e hub buildavam em série no mesmo *job* sem depender uma da outra.

Decisão. Duas mudanças, nenhuma nova peça de infra: (1) os *Dockerfiles* abrem com `COPY pnpm-lock.yaml pnpm-workspace.yaml` + `pnpm fetch` — que baixa o *store* lendo só o *lockfile* — e o `pnpm install --offline` roda depois do `COPY . .`; (2) o *workflow* vira quatro *jobs* — `mudancas` (o *diff*, calculado uma vez), `deploy-api` e `deploy-hub` em paralelo atrás do portão, e `publicar-exps` depois dos dois, porque o `PUT` do manifesto é validado contra o JWKS que o hub serve.

Alternativas descartadas.

- *Buildar as imagens em paralelo com o portão*, deixando só `gcloud run deploy` e a migração esperarem o verde: ganharia 2–6 min de caminho crítico, mas gastaria *runner* em todo *commit* vermelho, e o pipeline passaria a ter dois regimes de *gating* para raciocinar. O portão continua na frente de tudo.
- *Cache remoto do Turborepo* ligando `verificar`, o *build* do *runner* e os *builds* dentro das imagens: é a otimização certa quando o `tsc`/`next build` virarem o gargalo — hoje o gargalo era o `pnpm install` repetido, e um serviço de *cache* é uma peça a mais para operar.
- *Copiar o `dist/` do runner para dentro da imagem* em vez de buildar lá dentro: acopla o *Dockerfile* ao *workflow* (um `docker build` local deixaria de funcionar sozinho) para economizar um passo que, com a camada de dependências cacheada, custa segundos.

Consequências.

- O hub não espera mais o *deploy* da api. A ordem era acidental — os dois conversam por HTTP e o *rollout* do Cloud Run nunca garantiu revisão casada — mas fica registrado: se um dia um contrato exigir hub-depois-da-api, é um `needs` de uma linha.
- `publicar-exps` usa `!cancelled()` com `result != 'failure'` nos `needs`: os *deploys* podem ter sido PULADOS (um *push* que só toca experiência), e pulado não pode bloquear a publicação.
- Cada *job* que autentica no GCP repete o *setup* de ~1 min (WIF, gcloud, buildx). É o preço do paralelismo, pago em paralelo.
- O `pnpm fetch` depende só do *lockfile*: mudou dependência, a camada refaz inteira. É o comportamento certo — e é raro na frequência de *commit* deste repo.

Gatilho de reavaliação. O `nest build`/`next build` dentro das imagens passar de ~2 min com a camada de dependências já em *cache* — aí o cache remoto do Turborepo entra na conversa.

18.40 ADR-39 — Vulnerabilidade de dependência trava o portão, com prazo para os *majors*

> **ATENÇÃO**
>
> **O portão descrito acima NÃO está ativo (situação em 2026-09-05).** O passo `Snyk` do `ci.yml` está **comentado** desde o *commit* `07d9ec0` (“*chore: disable Snyk vulnerability scan in CI workflow*”). Enquanto ele estiver assim, `high` e `critical` não deixam o CI vermelho, e nada impede que uma dependência vulnerável suba.
> A consequência que dói mais é a de segunda ordem: o prazo de 2026-10-31 do `apps/api/.snyk` era cobrado *pelo próprio CI* — vencido sem o *upgrade*, o passo voltaria a falhar. Com o passo comentado, o prazo vence **em silêncio**, e o `.snyk` passa a ser exatamente o que a alternativa descartada abaixo chama de armadilha: um registro em que todo mundo acredita e que nada cumpre — a mesma forma do `expires_at` sem rotina.
> Isto está registrado aqui porque a ADR descrevia uma proteção que deixou de existir, e ADR que descreve o que não está no código é pior que ADR nenhuma. Reativar o passo é decisão de quem cuida do CI; o `AGENTS.md` da raiz proíbe que um agente o faça por conta própria, e proíbe também rodar a varredura na sessão (**ADR-55**). O que muda com esta nota é que a ausência do portão deixou de ser invisível.

> **Contexto**
>
> A plataforma guarda dado de menor e roda em rede pública; dependência com vulnerabilidade conhecida é passivo que cresce em silêncio. Na primeira varredura (Snyk, 2026-08-29) havia *criticals* reais: o Next 15 do hub e o `next@16.3.0` que o *design system* arrasta para dentro de todo consumidor, mais *highs* transitivas na api (`multer`, `js-yaml`, `lodash`) e três cujo *fix* exige *major* de *framework* (Nest 11) ou *bump* de terceiro (Prisma).

Decisão. O portão (`ci.yml`) ganha `snyk test --all-projects --severity-threshold=high`: `high` ou `critical` = CI vermelho = nada sobe. O estoque existente foi tratado em três movimentos — o hub subiu para o **Next 16** (a linha 15.x não tinha *patch* para as *criticals*), o transitivo consertável entrou em `pnpm.overrides` na raiz, e o que exige Nest 11/Prisma está em `apps/api/.snyk` com **motivo e prazo** (2026-10-31): vencido o prazo sem o *upgrade*, o próprio passo volta a falhar. `SNYK_TOKEN` ausente = passo vermelho, nunca pulado (a regra do *guard* do Pub/Sub).

Alternativas descartadas.

- *Só relatório, sem travar* (`continue-on-error`): vira o `expires_at` — todo mundo acredita que há uma verificação, e ninguém lê o relatório de um passo que nunca fica vermelho.

- *Bloquear até consertar TUDO, sem* `.snyk`: com *fix* que é *major* de *framework*, o pipeline ficaria vermelho por semanas e o *deploy* pararia — e um portão que todo mundo precisa contornar ensina a contornar portões.
- *Rodar só no push para* `main`: o PR mergearia vulnerável e o autor descobriria depois do merge, no pior momento para reverter.

Consequências.

- Cada execução do portão testa os 12 projetos do *workspace* contra a API do Snyk — em plano gratuito há cota mensal de testes. Se a cota virar o problema, a válvula é mover a trava para o *push* em `main` + um agendado diário, nunca desligar.
- O `.snyk` expira em 2026-10-31: o *upgrade* do Nest 10→11 tem prazo de verdade, cumprido pelo próprio CI.
- `pnpm.overrides` força versão por baixo de quem não atualizou (`@unooria/design-system>next`, `multer`, `js-yaml`, `lodash`); cada entrada deve SUMIR quando o pai atualizar — *override* esquecido é *pin* que ninguém lembra por quê.
- O *fix* definitivo do `next` transitivo é do repo `storybook-unooria`: o DS não deveria arrastar `next` para dentro de experiência Vite que nunca o executa.

Gatilho de reavaliação. A cota do Snyk barrar o portão, ou o `.snyk` vencer: os dois têm a mesma resposta — resolver a causa (plano ou *upgrade*), não afrouxar a trava.

18.41 ADR-40 — A raiz é a *landing*; a marca vem do DS, não do modelo

> **Contexto**
>
> A raiz do hub (`/`) só roteava: sem sessão, `/login`; com sessão, a *home* do papel. Quem chegava ao endereço da plataforma via um formulário de usuário e senha e nada sobre o produto — não havia *landing*. A tela foi desenhada no Stitch (projeto `16689192725057015920`, "Unooria — Educação Digital Infinita"), e o pedido era implementá-la fiel ao modelo.
> O HTML exportado pelo Stitch traz duas coisas que não são da marca: o tema Material que a ferramenta gera sozinha (primária `#396093`, Nunito Sans, Inter, Material Symbols *Outlined*, Tailwind por CDN) e o logo como PNG hospedado no Google. E o `login-form` e a troca de senha fazem `router.replace("/")` contando com o roteamento da raiz — é o que leva a pessoa à área certa sem o *client* conhecer o papel.

Decisão. Três partes.

1. **`/` sem sessão é a *landing*; com sessão, continua roteando** para a *home* do papel (ou para `/trocar-senha`). Nada muda no `login-form`, na troca de senha nem no *middleware* (a raiz segue fora do `matcher`). Quem já está dentro e quer ver a *landing* sai da conta.

2. **Fiel ao modelo em estrutura, copy, seções e movimento** — navegação fixa em vidro, *hero* sobre o *shader* WebGL do modelo (portado do `<script>` inline para um *client component* com ciclo de vida), cards flutuantes, revelação ao rolar, rodapé — **e do DS em cor, tipografia, ícones, logo e botões**: tokens `--unooria-*` (a primária do modelo vira `brandnavy-900`; os roxos, `brandpurple-600/700`; o vermelho, `danger-600`), Open Sans, `Icon` (Material Symbols *Rounded*, os mesmos nomes de ícone), `Logo`/`LogoSymbol` vetoriais no lugar dos PNG e `Button` nas duas chamadas. O painel "conceito" de Nossa Resposta, que no modelo era um PNG do símbolo sob um gradiente, é o `--unooria-color-gradient-1` com o `LogoSymbol` — mesma imagem, sem *raster*.
3. **O CSS da *landing* vive ao lado do componente** (`components/landing/landing.css`), não em `plataforma.css`: é composição de página, não compensação de lacuna do DS. Toda cor ali é *token*; translucidez sai de `color-mix()` sobre o *token*, não de literal `rgba`.

Alternativas descartadas.

- *Landing sempre visível, com uma rota nova (`/inicio`) para o roteamento por sessão*: exigiria trocar o destino do `login-form` e da troca de senha e criar uma segunda porta que só redireciona. Mais peças para o mesmo resultado — e quem está logado raramente quer a página de venda.
- *Landing fora do hub, como HTML estático no bucket*: a página teria de duplicar o CSS e as fontes do DS, e o botão "Começar Agora" atravessaria origens. O hub já serve a casca do DS (`CascaPlataforma`) — a *landing* é a quinta superfície a usá-la.
- *Reproduzir a paleta e as fontes do Stitch*: seria a primeira superfície da plataforma fora da marca, e por definição a mais vista. A regra do DS (Capítulo 14) existe exatamente para isso.
- *Trazer o Tailwind por CDN e o tema do modelo*: o hub já tem Tailwind só para utilitárias de *layout*, com o tema VAZIO de propósito — cor por nome de classe é o que a decisão de `plataforma.css` proíbe.

Consequências.

- **O botão primário sai azul-céu, não navy** (`#0d0d35` no modelo): é o `Button` do DS. Se a marca quiser o CTA em navy, o lugar é o pacote, não uma classe local.
- **O cleanup do *shader* NÃO chama `loseContext()`.** O Strict Mode do React (dev) monta, desmonta e remonta o efeito no MESMO canvas, e um canvas só tem um contexto WebGL — perdido de propósito na primeira desmontagem, volta perdido na segunda montagem e o fundo fica em branco. Foi o primeiro sintoma do *smoke* (`contextLost: true`). Com `prefers-reduced-motion` desenha um quadro e para; na aba oculta, para até voltar; sem WebGL, o gradiente do DS declarado no CSS ocupa o lugar.
- **A revelação ao rolar só esconde depois que o JS montou** (`.landing--animada`). Esconder no CSS e revelar no JS deixaria a página em branco para quem o *bundle* não alcança.
- **Cor de ícone vai pela *prop* `color` do `Icon`**, não por classe: a folha do pacote vem depois no grafo de importação e, com peso igual, vence — os ícones saíam pretos.

- Os *links* de Privacidade, Termos e Contato do rodapé apontavam para `#`, como no modelo: as páginas não existiam. **Isso mudou em 2026-09-05** — Privacidade e Termos apontam para `/privacidade` e `/termos` (**ADR-70**); só o Contato segue em `#`, e continua sendo pendência de produto.
- Suíte própria (`scripts/e2e/publico/01-landing.mjs`): fontes do DS carregadas (Open Sans e a de LIGADURA dos ícones), canvas com contexto, cor de fundo COMPUTADA do card de vidro, todo `.landing-revelar` visível ao fim, nada transbordando em celular, e o CTA levando ao `/login`. Só precisa do hub no ar.

Gatilho de reavaliação. A plataforma ganhar uma área pública maior (institucional, ajuda, termos): aí a *landing* vira um grupo de rotas `(publico)` com *layout* próprio, e os *links* do rodapé deixam de ser `#`. **Disparou pela metade em 2026-09-05**: as duas páginas legais nasceram (**ADR-70**) e os *links* delas deixaram de ser `#`, mas como rotas próprias — o grupo `(publico)` com *layout* compartilhado ainda não existe, e nasce quando a terceira página pública aparecer.

18.42 ADR-41 — O relatório do estudante sai do aparelho: o iframe expande, não remonta

> **Contexto**
>
> A explicabilidade da exp08ef1b é a peça mais densa do produto e rodava inteira dentro do aparelho de 375×812: seis blocos, um gráfico só e três dos cinco posts de maior *dwell* — quando o *payload* já carregava mais. As barras por bloco e a linha da escalada existiam em `graficos.tsx` e não tinham consumidor de tela; a "escalada" virara uma frase por falta de espaço.
> Ela acontece DEPOIS da revelação, e nesse ponto não há mais disfarce a proteger: a regra da ponte (`packages/ui/src/ds.ts`) já autorizava o DS da plataforma nas "telas pós-revelação do estudante".

Decisão. No passo do relatório, a experiência publica `exp:estado { fase: "explicabilidade", passo: "relatorio" }` e o `ExpHost` liga `expandido` no `DispositivoFrame`, que troca APENAS o CSS do próprio `<div>` interno. O relatório vira claro, responsivo, com os três gráficos e o *top* 5. A revelação continua no aparelho, escura: ela é a fronteira, e não estava comprimida.

Alternativas descartadas.

- *Um `Modal size="full"` do hub buscando `/runs/:id/dossie`*: o BFF já permitiria (a rota autoriza o estudante a ler o próprio) e havia precedente em `modal-estudante.tsx`. Descartado porque duplicaria a renderização do relatório em dois apps — ou obrigaria a experiência a perder os *exports* em texto e PNG, que são dela.
- *Uma rota própria do hub* (`/estudante/relatorio/:runId`): tira o estudante do aparelho no meio de uma aula conduzida por botão.

- *`requestFullscreen()` de dentro do iframe*: o `allow="…fullscreen"` já está delegado a todas as experiências e ninguém usa. O hub perderia o controle da moldura, e a largura lógica continuaria a mesma — ficaria maior, não menos comprimido.

Consequências.

- **Expandir é ESTILO, nunca árvore.** Tirar o `children` do `DispositivoFrame` para renderizá-lo fora desmontaria o `<iframe>` e o estudante voltaria à revelação com a sessão recarregada. É por isso que o modo é uma *prop*, e não um `if` trocando o *wrapper*.
- **A condição é POSITIVA, e o *default* é contraído.** Qualquer outro estado — inclusive mensagem perdida — cai no aparelho de 375px. O `IntersectionObserver` do *dwell* se refere ao *iframe*: um FEED numa área larga inflaria o sinal central do motor.
- **O layout largo é `md:`, o estreito é o *default*.** Se a expansão não acontecer, o estudante vê exatamente o que via.
- **Nenhuma mensagem hub→exp para o layout**: o *iframe* muda de largura de verdade, e as *media queries* de dentro respondem sozinhas. Os dois gráficos a mais são renderizados e escondidos por CSS — perguntar a largura em JS custaria um `matchMedia` e um render a mais na hidratação.
- A volta usa o caminho que já existia: o professor avança, o *polling* de 5s vê a fase nova, a experiência publica `{ fase: "perdidos" }` sem `passo`, e o aparelho volta. Até 5s de atraso, que é a latência que a troca de tela já tinha.
- **Paridade**: o relatório do professor (`modal-estudante.tsx`) ganhou a seção *"por que o feed dele começou desse lado"*, que ele não lia — e é a frase que a criança acabou de ler.

Gatilho de reavaliação. Uma segunda experiência precisar do mesmo gesto: aí `expandido` deixa de ser lido de dois campos conhecidos no `ExpHost` e vira mensagem própria no `ExpParaHubSchema`, com *bump* de `protocolVersion`.

18.43 ADR-42 — "O que o algoritmo escondeu" é a EXPOSIÇÃO, não o domínio

> **Contexto**
>
> Havia duas contas para a mesma frase. A tela do estudante (`/runs/:id/perdidos`) decidia qual era "o lado dele" CONTANDO os posts servidos (`consumo.A >= consumo.B`); o relatório do professor (`dossie/computar.ts`) decidia pelo `dominioDe()`, o *score* do motor — *dwell*, sinais, escada de intensidade.
> Enquanto as duas apontam o mesmo lado, os números batem. Elas divergem exatamente quando o estudante VIRA O JOGO, porque virar o jogo *é* a definição de a atenção ter terminado do lado oposto ao que o feed empurrou. Ou seja: as duas telas se contradiziam justamente sobre os estudantes que o professor mais quer discutir.

> O caso está registrado numa auditoria de UX: sobre o MESMO estudante, o painel dizia "terminou dominante em Processo Lento e Gradual, virou o jogo" e a tela dele dizia "32 posts que nunca chegaram até você, todos de Processo Lento e Gradual". Duas frases sobre a mesma criança, nomeando lados opostos. O relatório do professor ("escondeu 30") contava os posts do lado que o algoritmo estava EMPURRANDO nele.

Decisão. Uma conta só, em `apps/api/src/dossie/exposicao.ts`, consumida pelas duas rotas: "o lado dele" é o que o feed mais SERVIU, e o número é quantos aprovados do outro lado nunca foram servidos. `ladoDominante` continua existindo e continua no relatório — ele responde "para onde a atenção dele foi", que é outra pergunta, e é a que sustenta o "virou o jogo".

Alternativa descartada. *Padronizar no domínio.* É a leitura mais fiel ao que a atenção do estudante fez, mas "o algoritmo escondeu" é uma afirmação sobre o que o feed ENTREGOU: medir pelo domínio credita à máquina o resultado do comportamento da criança. E com `dominio` $\leq$ `0,5` o resultado era `indefinido`, que virava "escondeu 0 posts".

Consequências.

- **O número do professor MUDA** em atividades já rodadas — `VERSAO_DO_RELATORIO` sobe para 4 e os dossiês congelados são recomputados na leitura. É o preço de ter contado errado.
- **Sem post de tese servido, não se nomeia lado nenhum.** A rota devolve `ladoPerdido: null` e a tela diz que a sessão terminou antes de o feed abrir. "0 posts que nunca chegaram até você" afirmaria o contrário do que houve — é a mesma distinção entre `zero` e `none` da Vila do Lume.
- **Post desaprovado não conta como escondido**: nunca teve chance de aparecer para ninguém, e culpar o algoritmo por uma decisão editorial do professor seria outra mentira pequena.
- A trava é `apps/api/tests/exposicao-lado-oposto.spec.ts`, e o teste do meio monta exatamente o estudante que virou o jogo.

Gatilho de reavaliação. Alguém precisar de "o oposto do lado dominante" para outra tela: é uma terceira grandeza, e ela nasce com nome próprio — nunca reaproveitando esta.

18.44 ADR-43 — A resposta do estudante guarda a primeira e a última

> **Contexto**
>
> A tela da resposta não tem botão de confirmar: tocar na pílula já registra, e o quadro diz "pode trocar de ideia enquanto espera — vale a última". O estudante oscila de propósito enquanto os colegas terminam.
> O `upsert` sobrescrevia `inclinacao` e a primeira escolha desaparecia. O `created_at` ficava congelado no primeiro envio enquanto a inclinação mudava, ou seja, passava a

> mentir sobre a própria linha. E a correlação "lado em que o feed nasceu × resposta dada" — o dado mais forte que esta atividade produz — não tinha como dizer se estava lendo a reação imediata ao feed ou o que sobrou depois de dois minutos de espera e de olhar a tela do colega.

Decisão. Migração aditiva em `exp08ef1b_conclusoes`: `inclinacao_inicial` (gravada uma vez, nunca mais tocada), `trocas` e `atualizado_em`. A correlação continua usando a ÚLTIMA — é a que o estudante assumiu e é a que ele vai defender no debate — e a análise da turma passa a DIZER isso, com quantos mexeram na resposta e quantos terminaram noutro lado.

Alternativas descartadas.

- *Emitir evento de telemetria a cada toque*: o canal analítico tem retenção indefinida e `estudante_id`; guardar a oscilação de opinião de uma criança num JSONB de eventos é mais dado do que a pergunta pede.
- *Trocar a correlação para a primeira escolha*: seria decidir pelo professor qual das duas conta. A decisão é dele, e agora ele tem os dois números.

Consequências. `inclinacao_inicial` é NULL nas linhas anteriores à migração, e é assim que se lê "esta resposta é anterior à mudança": preenchê-la com a inclinação atual inventaria uma primeira escolha que ninguém observou. Reenviar a mesma opção (toque duplo, rede lenta) não conta como troca. As duas colunas novas vão ao CSV da análise.

Gatilho de reavaliação. Aparecer demanda por ver a oscilação completa (não só a primeira e a última): aí é tabela de histórico, não mais colunas.

18.45 ADR-44 — A janela do envolvimento termina com o feed, e o total sai das listagens

> **Contexto**
>
> Duas coisas, medidas na mesma auditoria.
> A primeira: o painel ao vivo mostrava a imersão da turma DERRETENDO conforme a aula avançava ($52{,}7 \rightarrow 42{,}8 \rightarrow 38{,}7 \rightarrow 35{,}1$) e isso lia como adolescente desengajando. Não era. O *buffer* de eventos da experiência vive enquanto a *run* vive: durante o debate, a explicação e a reflexão — mais de metade do roteiro de 100 minutos — cada `hub:pause` ainda registra `perdeu_foco`. Com a conta fechando no último evento da SESSÃO, `feedMs` crescia depois de o feed fechar, cada vão virava `ociosoMs`, e `ativoMs / feedMs` caía sozinha.
> A segunda: o total 0–100 aparecia como número numa COLUNA de tabela, por criança, no monitor e na análise — ordenável, comparável. O próprio código dizia, em comentário, que "o total existe para ordenar uma lista, não para ser exibido sozinho", e a tela o exibia

> sozinho. E ele é somável justamente onde menos se sustenta: *Gestos* e *Exploração* ficam em 0/20 na maioria das sessões, então o teto real não é 100.

Decisão. `fimDoFeedDe(eventos)` — o último `visto`, que é o fim do feed por definição — corta a janela em `calcularTempo` E no desconto por perda de foco de `calcularImersao`. E o total sai das duas listagens por estudante, ficando as cinco barras (com o eixo no `title`); o número segue no popup do estudante, que é uma criança por vez, e nos agregados de turma. Um nome só: *Envolvimento*, que estava "Imersão" em quatro telas e "Envolvimento" numa.

Alternativas descartadas.

- *Cortar por `run.finishedAt`*: `clientTs` é o relógio do APARELHO e `finishedAt` é o do servidor. Comparar os dois numa escola com relógio desregulado traria o mesmo erro de volta por outro caminho.
- *Parar de registrar eventos fora do feed, no cliente*: corrigiria daqui para a frente e não tocaria em nenhum dado já coletado — e depende de a experiência estar atualizada.
- *Recalibrar os tetos de Gestos e Exploração*: exige medição em turmas reais. Fica em aberto; mexer nos tetos por intuição trocaria um número errado por outro.

Consequências. `VERSAO_DO_RELATORIO` sobe para 4 e recomputa o que está congelado. Um teste do `dossie-relatorio.spec.ts` precisou de um `visto` para delimitar o feed: ele montava trinta `perdeu_foco` e nenhum post visto, ou seja, descrevia perdas de foco fora de qualquer feed — uma distinção que antes não existia. O professor perde a leitura de um número por estudante na tabela; ganha o padrão dos cinco eixos, que é o que a conversa com ele usa.

Gatilho de reavaliação. Dados de turmas reais mostrarem que *Gestos* e *Exploração* pontuam: aí os tetos são recalibrados e a pergunta de exibir o total volta a fazer sentido.

18.46 ADR-45 — *Agent* e `fetch` vêm do MESMO pacote undici

> **Contexto**
>
> O provider do Vertex usa um *dispatcher* próprio: `generateContent` não-*streaming* só devolve os *headers* quando a geração termina, e o teto de 300 s do `fetch` do Node estourava no meio de um plano anual. A solução era um `Agent` do pacote `undici` com 15 minutos, passado como opção `dispatcher` ao `fetch` GLOBAL.
> Isso acopla duas coisas que ninguém obriga a concordar. O `fetch` global usa o undici EMBUTIDO na *runtime* (`process.versions.undici`), que é outro pacote, com outra versão. Com os `Dockerfile` fixados em **Node 26.7** (undici **8.x**) e a API declarando `"undici": "^6"`, o `Agent` da 6 deixou de satisfazer a interface de *handler* da 8.
> O sintoma não aponta para nada: `fetch failed`. A razão real (`invalid onError method`) vive só em `error.cause`, e o `catch` da geração logava apenas `error.message`. **TODA**

> chamada de IA morria — plano de aula, *unooGPT*, arte — e por dias isso pareceu falha de credencial, porque havia de fato um ADC vencido por cima.

Decisão. Importar o `fetch` do PRÓPRIO undici (`import { Agent, fetch } from "undici"`). Vindos do mesmo pacote, os dois concordam por construção, e a versão embutida na *runtime* deixa de ser uma dependência invisível. O *log* da geração passa a imprimir o `cause` junto — só no *log*: a mensagem que vai ao banco é a que o professor lê.

Alternativas descartadas.

- *Subir `undici` para `^8`*: funciona hoje e quebra no próximo *bump* de Node, porque a condição continua sendo "duas versões coincidirem" — e nada no CI verifica isso. Seria a mesma armadilha, adiada.
- *Largar o dispatcher e usar o `fetch` global puro*: devolveria o teto de 300 s que motivou o *dispatcher*, e o plano anual voltaria a morrer no meio.
- *Fixar o `undici` pela versão da runtime*: exigiria ler `process.versions.undici` e casar no `package.json`. Duas fontes de verdade sobre a mesma versão, uma delas fora do controle do repo.

Consequências. O teste `ai-conversar.spec.ts` dublava `global.fetch` e passou a não interceptar nada — voltava *array* vazio. Ele agora mocka o módulo `undici` encaminhando para `global.fetch`, o que mantém a dublagem em uma linha e o que se testa segue sendo a lógica do provider. Fica registrado que o `dispatcher` é a razão de o `fetch` não poder ser o global: quem apagar a importação "porque o Node já tem `fetch`" reabre exatamente este defeito.

Gatilho de reavaliação. O Vertex passar a ser consumido por `streamGenerateContent?alt=sse`: com *streaming* os *headers* chegam de imediato, o teto de 300 s deixa de incomodar, e o *dispatcher* — junto com todo este acoplamento — pode simplesmente sair.

18.47 ADR-46 — O `?hubOrigin=` é entrada não confiável: a experiência confirma quem a embedou

> **Contexto**
>
> A regra 4 diz que o *token* passa do hub para a experiência por `postMessage` com `targetOrigin` explícito, nunca `"*"`. A bridge cumpria a letra da regra e não o espírito: o valor do `targetOrigin` — e, o que é pior, o valor comparado em `ev.origin !== hubOrigin` para decidir de quem se aceita o `hub:init` — vinha CRU da *query string* (`new URLSearchParams(location.search).get("hubOrigin")`).
> O *bundle* da experiência é público por construção (ADR-01: ele mora num *bucket* servido por CDN). Quem abrisse `https://cdn/exp08ef1b/1.0.0/index.html?hubOrigin=https://evil.tld` den-

> tro de um *iframe* próprio escrevia, com aquele parâmetro, a política de confiança da experiência: passava a poder entregar um `hub:init` forjado — com `estudante`, `apiUrl` e `token` escolhidos por ele — e a receber **tudo** o que a experiência emitisse depois (`exp:concluir`, `exp:estado`). O Snyk classificou como *code injection* no ponto em que o parâmetro entra; a leitura é discutível, o caminho não.

Decisão. Todo `hubOrigin` passa por `normalizarHubOrigin()` (`packages/sdk/src/bridge.ts`), chamado tanto onde o parâmetro é lido quanto dentro do `connectToHub` — a função é idempotente, e a segunda chamada existe porque é ali que o valor vira `targetOrigin`. São três travas:

1. `new URL(...).origin`: o valor vira uma ORIGEM canônica, nunca uma *string* com caminho, credencial ou *query*. Não é higiene: `postMessage` compara só a origem, mas a igualdade do *handler* nunca casaria, e o sintoma seria *timeout* de *handshake* em vez de "o parâmetro está errado". Esquema fora de http(s) e origem opaca (`"null"`) são recusados.
2. Experiência servida por https exige hub em https — a mesma regra do JWKS da portabilidade. Em desenvolvimento tudo é http e a trava não pesa.
3. `location.ancestorOrigins[0]`, quando o navegador publica: é a origem REAL de quem embedou, e ela VENCE a *query string*. É a trava que fecha o caminho de fato — o atacante controla o parâmetro, não o endereço do documento que abriu o *iframe*.

Alternativas descartadas.

- *Lista de origens permitidas embutida no bundle*: a experiência é a mesma em todos os contratos, e cada contrato tem o seu domínio de hub. A lista viraria *env* de *build*, ou seja, um *bundle* por contrato — exatamente o que a ADR-01 evita.
- *Deduzir a origem de `document.referrer`*: depende da *referrer policy* do hub e some com `no-referrer`. O `ancestorOrigins` não é forjável pela página.
- *Deixar como estava, porque o token forjado é do próprio atacante*: verdadeiro para o roubo direto e irrelevante para o resto — o que o atacante ganha é uma experiência que fala com a API dele e devolve a ele o que a criança produziu.

Consequências. O Firefox não implementa `ancestorOrigins`; lá sobram as duas primeiras travas. `hubOrigin` malformado deixa de morrer por *timeout* de *handshake* e passa a lançar com a razão escrita — diagnóstico melhor, mas é uma falha NOVA para quem embedar a experiência fora do hub. `HUB_ORIGIN` mal configurado no hub, que antes só travava o *handshake*, agora recusa explicitamente.

> **NOTA**
>
> **O apontamento do Snyk que originou esta ADR NÃO some com ela, e está certo assim.** A regra *Code Injection* (CWE-94) desenha o fluxo `location.search` → `setInterval` e afirma que o valor "é executado como JavaScript". O *sink* apontado é o `setInterval(anunciar, READY_RETRY_MS)` do *retry* do `exp:ready`, e `setInterval`

> só AVALIA código quando o primeiro argumento é uma *string*: ali ele recebe uma referência de FUNÇÃO, e o `hubOrigin` entra por captura de *closure* para terminar como `targetOrigin` de um `postMessage`, que não executa nada. O valor tem três usos no pacote — dois `postMessage` e uma comparação com `ev.origin` —, e nenhum é *sink* de execução.
> O que aquele fluxo tinha de verdadeiro era a origem de confiança vindo da URL, e é isso que esta ADR fecha. Fica registrado para que ninguém CONTORÇA o código para agradar o *scanner*: esconder a captura do *closure* deixaria o *retry* ilegível sem tirar risco nenhum. A nota vive junto da chamada, em `packages/sdk/src/index.ts`.

Gatilho de reavaliação. O dia em que uma experiência precisar ser embedada por um hub de terceiros (LTI, por exemplo): a trava 3 passa a exigir uma lista de embedadores conhecidos, e ela terá de vir do manifesto — não do parâmetro.

18.48 ADR-47 — Nem senha de demonstração nem verificação de TLS moram no fonte

> **Contexto**
>
> Três apontamentos do Snyk com a mesma raiz: credencial e política de segurança escritas como constante no código, onde não há como diferenciar desenvolvimento de produção.
>
> - `apps/api/prisma/seed.ts` trazia `SENHA_PADRAO = "unooria123"`. O seed é idempotente e roda contra o `DATABASE_URL` que estiver no ambiente: apontado por engano para o banco de um contrato, ele criaria 61 contas — inclusive o `admin-contrato`, o único papel que atravessa escolas — com uma senha publicada no `README`.
> - `wso2/bootstrap/bootstrap.mjs` abria com `process.env.NODE_TLS_REJECT_UNAUTHORIZED = "0"`, porque o IS local usa o certificado *self-signed* da distribuição (CN=`localhost`, host `identity-server`). Aquilo desliga a verificação do PROCESSO INTEIRO, para toda origem: o *script* que carrega a senha do *admin* do *tenant* passava a aceitar qualquer certificado de qualquer servidor.
> - `apps/api/src/auth/auth.controller.ts` calculava o *hash* "dummy" anti-*timing* sobre uma *string* literal.

Decisão. O princípio é um só: o que é credencial vem do AMBIENTE, e o que é confiança se PROVA em vez de ser dispensada.

- `SEED_SENHA` é obrigatória e não tem *default* no fonte — ausente, o seed falha alto. O *default* de desenvolvimento vive nos dois `docker-compose`, exatamente como o `docs/unooria-docs-local` do Swagger (ADR-36). O seed também deixou de imprimir a senha.

- O *bootstrap* do WSO2 PINA o certificado: lê o certificado do IS no primeiro contato (o único `rejectUnauthorized: false` que resta, e ele não autentica nada — serve para DESCOBRIR a chave), grava o PEM que api e hub já consumiam por `NODE_EXTRA_CA_CERTS` e o usa como a ÚNICA CA aceita nas chamadas seguintes. Só a conferência de NOME do host é dispensada, com `checkServerIdentity`, porque o CN da distribuição não bate com o nome do serviço — a identidade da chave continua obrigatória.
- O *hash* dummy passa a ser calculado sobre `randomBytes(32)`, sorteado a cada subida e descartado no mesmo instante.

Alternativas descartadas.

- *Manter o literal do seed e ignorar o apontamento no `.snyk`*: o `.snyk` deste repo é para vulnerabilidade cujo *fix* exige *major* de *framework*, com prazo cobrado pelo CI. Aqui o *fix* custa dez linhas.
- *Sortear a senha do seed quando a env falta*: nada ficaria no fonte, mas as suítes de ponta a ponta passariam a depender de ler `stdout`, e o `README` não teria o que documentar.
- *Passar a CA do IS por `NODE_EXTRA_CA_CERTS` no bootstrap*: a variável é lida na SUBIDA do processo, e o certificado só existe depois que o *script* o busca. Exigiria dois processos.
- *Aceitar também o nome do host, reescrevendo o certificado do IS*: trocar a distribuição por um certificado gerado no *up* é mais infra local para operar, num serviço que já leva dois minutos para subir.

Consequências. `pnpm --filter @edu/api seed` no *host* exige `SEED_SENHA` — no `apps/api/.env` (que o Prisma carrega) ou na linha de comando. Nos dois *compose* nada muda. O *bootstrap* do WSO2 trocou o `fetch` global pelo `node:https`, porque o `fetch` do Node não aceita agente e é por agente que se passa uma CA; são vinte linhas a mais, e é o preço de não desligar o TLS do processo. O *bootstrap* passou a FALHAR se não conseguir ler o certificado, onde antes seguia com um aviso: sem CA não há como falar com o IS sem voltar a aceitar qualquer certificado.

Gatilho de reavaliação. O WSO2 deixar de ser *self-signed* em desenvolvimento (certificado de uma CA local no *up*): a pinagem e o `checkServerIdentity` saem juntos, e o `chamar` pode voltar ao `fetch`.

18.49 ADR-48 — O teto do CSV é de BYTES e cabe debaixo do limite do ambiente; o modelo do OneRoster vem em pares, e o exemplo leva a escola de verdade

> **Contexto**
>
> A tela de importação tem duas abas. A da *planilha da escola* oferece dois arquivos para *download* — o modelo em branco e um exemplo preenchido — e os dois são GERADOS da mesma constante que o *parser* consome. A do **OneRoster 1.2** lista os sete arquivos do *binding* CSV e não oferecia nenhum: quem não tem o *export* pronto do sistema de gestão — e é a maioria antes da primeira integração — tinha de montar o cabeçalho lendo a especificação da 1EdTech.
> O teto do corpo era 10 MB, dimensionado para a planilha de UMA escola, e o `MAX_LINHAS` de 50 mil, que existe do lado da planilha, não vale para o caminho OneRoster. Medido com o formato real do padrão (~130 B por linha de `enrollments.csv`), o pior caso realista de UM contrato — ~10 mil estudantes, cinco componentes cada — dá ~7 MB. Os 10 MB apertavam exatamente ali.
> Acima da aplicação há um teto que não é nosso: o Cloud Run recusa requisição maior que 32 MiB em HTTP/1, no *frontend* do Google e ANTES do contêiner. Não é quota que se peça para aumentar — é como a camada de *serving* funciona, porque ela bufferiza o corpo inteiro antes de repassar.

Decisão.

- O teto de BYTES sobe para 30 MB e passa a viver numa constante só, `TETO_DO_ARQUIVO` em `rostering.service.ts`, que o *bodyParser* de `main.ts` também lê. O número é escolhido para ficar DEBAIXO dos 32 MiB do ambiente: a aplicação não promete o que o Cloud Run não entrega, e a mensagem que o gestor vê é sempre a nossa — a que diz para dividir o arquivo. Ainda são ~4× o pior caso medido.
- A ORDEM entre os dois tetos importa: com o *parser* menor que a rota, o corpo morre no *middleware* e o gestor recebe um `PayloadTooLarge` do Express, sem o texto que diz o que fazer. O `mb` do *parser* é MEBIbyte (`30mb` = 31,4 MB), então ele fica ~5% ACIMA da rota — e é essa folga que faz a nossa mensagem ser a que aparece.
- O `MAX_LINHAS` de 50 mil NÃO sobe: ele é da planilha de uma escola, e 50 mil pessoas numa unidade é um arquivo errado, não um arquivo grande.
- Cada arquivo do padrão ganha o mesmo par da planilha: `GET /rostering/modelos/oneroster/:arquivo` (só o cabeçalho) e `.../:arquivo/exemplo` (preenchido). As colunas do modelo são um SUBCONJUNTO declarado de `COLUNAS_POR_ARQUIVO`, a constante que o leitor confere, e `tests/oneroster-modelos.spec.ts` cobra as DUAS direções — obrigatória que falte no modelo, e coluna do modelo que o *parser* não conheça (o dado entraria e sumiria).
- O exemplo é gerado com a escola de VERDADE no `sourcedId` do `orgs.csv` e no `schoolSourcedId` das turmas e matrículas, e com o ano corrente no calendário.

- Os sete exemplos formam um lote COERENTE — uma escola, um ano letivo, duas turmas, quatro pessoas e o enturmamento delas. O teste aplica os sete contra o banco, na ordem da tela, e afirma turma, papéis e matrículas.

Alternativas descartadas.

- *Escrever os sete arquivos à mão, como texto fixo*: é a armadilha que o modelo da planilha já evitou uma vez. Modelo escrito à mão diverge do *parser* no primeiro campo novo, e o sintoma chega como "baixei o modelo de vocês e deu erro de cabeçalho".
- *Publicar o modelo com TODAS as colunas que a api sabe ler*: `users.csv` tem 24. Modelo com 24 colunas não é modelo, é a especificação — e a escola preencheria `sms`, `phone` e `agentSourcedIds` sem que nada os leia.
- *Um `esc.exemplo` fixo no `orgs.csv`*: para o gestor, o lote inteiro voltaria como "escola não é a desta importação", e o arquivo que existe para conferir o caminho provaria o contrário. Para o `admin-contrato`, que PODE criar escola por CSV (ADR-34), o exemplo abriria um *tenant* de mentira — com RLS e dado de menor atrás dele.
- *Um único arquivo por item, sem o modelo em branco*: é o exemplo que o gestor abre para entender, e o modelo que ele preenche. Linha de exemplo dentro do modelo vira pessoa fantasma importada no primeiro uso — a mesma razão pela qual `planilha.ts` mantém os dois separados.
- *Empacotar os sete num zip*: empacotamento é embalagem, não semântica. A rota de importação já é um arquivo por chamada, e um *zip* para baixar exigiria dependência nova para produzir algo que o gestor descompactaria antes de usar.
- 100 MB — foi o primeiro número escolhido, e ele é PIOR que 30 MB: entre 33,5 MB e 100 MB a plataforma prometeria algo que o ambiente recusa, e o gestor receberia um 413 que não é nosso, sem mensagem útil e sem linha nenhuma no log da api. Teto que promete mais do que o ambiente entrega não é folga, é o erro que ninguém diagnostica.
- *HTTP/2 ponta a ponta (h2c) para tirar o limite do Cloud Run*: é a saída documentada, porque em HTTP/2 a requisição é *streamada* em vez de bufferizada. Custa trocar o servidor HTTP do Nest e fazer o hub repassar em HTTP/2 — um modo de transporte novo para operar, por um problema que os números acima dizem que não existe.
- *Streaming do corpo em vez de teto maior*: resolveria a memória, mas a leitura inteira do CSV é o que permite recusar o lote com o número da linha e o nome do campo. Trocar isso por um *parser* incremental é reescrever as duas fronteiras para um caso que ainda não apareceu.

Consequências. O corpo do CSV é lido inteiro para a memória em três lugares — o *route handler* do BFF do hub, o *bodyParser* da api e o `parseCsv` —, então 30 MB de arquivo custam alguns múltiplos disso em pico de *heap* por importação simultânea. O *rate limit* de 5 por minuto por usuário é o que segura isso hoje, e ele ficou ~3× mais barato do que ficaria com 100 MB.

A rede que precisar de mais que 30 MB num arquivo só NÃO tem saída pela tela: ela divide o `enrollments.csv` por escola ou por diretoria, que é uma operação que o formato já suporta

(um arquivo por chamada). Os sintomas dos três tetos — o nosso, o do *parser* e o do ambiente — estão no Apêndice F.

Gatilho de reavaliação. Uma rede chegar com *export* que não caiba em 30 MB depois de dividido: aí a conversa é sobre o CAMINHO — *upload* direto para o *bucket* com URL assinada e importação assíncrona a partir dele —, não sobre subir o número. Aquele caminho tira o limite do ambiente e o pico de *heap* de uma vez.

18.50 ADR-49 — A atividade do Instanooria declara a TURMA, e o painel dela para de ler a escola inteira

ATENÇÃO

REVISTA em 2026-09-05 pela ADR-69: a turma virou OBRIGATÓRIA. A alternativa descartada aqui (*uma atividade por turma, obrigatória*) foi adotada — `POST /atividades` exige `turmaId` e o "Todas as turmas da escola" saiu do formulário. O que faltava em 2026-08-30 apareceu: a plataforma ganhou o modelo de **sala de aula**, em que a turma é o contexto de tudo o que o professor faz, e a atividade sem turma era a última peça que ainda falava de um lugar que não é uma sala — era ela que fazia 2EM-A e 2EM-B aparecerem como uma lista só no painel de perfis, no monitor e na análise.
O custo previsto aqui é real e foi aceito: quem dá a mesma aula em quatro salas cria quatro atividades. A tabela de junção continua sendo a resposta certa se isso doer.
O resto desta ADR segue valendo, e a COLUNA continua anulável e sem *backfill*: as atividades criadas antes desta data seguem escolares, com a faixa que já as explica. Fechou a porta de criar mais uma assim, não o passado.

Contexto

A `exp08ef1b_activities` tinha `escola_id` e `professor_id`, e nenhum vínculo com `turmas` — embora o próprio esquema já falasse como se tivesse: o comentário de `etapa` chama o campo de "relógio COLETIVO da turma", e a função que monta o painel de perfis se chama `distribuicaoDaTurma`.
Ao investigar por que a importação dos perfis da Vila do Lume (`POST /atividades/:id/perfis/da-vila`) varria a escola, apareceu uma coisa pior: aquela função rodava `user.findMany({ where: { role: 'estudante' } })` — **sem `escola_id`**. E `users` está FORA da RLS: a lista de tabelas com política em `20260824145301_init` não a inclui, e o `comTenancy` apenas arma `app.escola_id` para o Postgres, sem tocar no `where` do Prisma. O painel do professor listava nome e login de crianças de TODAS as escolas do contrato, e `contagem`, `desequilibrio` e `precisaRedistribuir` — que decidem quantos estudantes o botão "Redistribuir a turma" move — eram calculados sobre esse universo.

> O `DaVilaSchema` já aceitava `turmaId` opcional desde que a rota nasceu, e a tela nunca o enviou: mandava `{}`. O relatório dizia "N ainda não jogaram" sobre crianças de salas que o professor não tem.

Decisão. Duas coisas, e a primeira vale sozinha.

1. **`escola_id` explícito** em `distribuicaoDaTurma`. É a regra inviolável nº 1 — toda *query* filtra por escola ALÉM da RLS — e aqui a RLS nem existia como rede.
2. **`exp08ef1b_activities.turma_id`**, ANULÁVEL e sem *backfill* (`20260830090000_atividade_por_turma`). Com turma, o painel, a listagem de perfis, a importação da Vila e a entrada do estudante em `POST /runs` falam todos da MESMA sala. Sem turma — que é o caso de toda atividade anterior à migração — vale a escola inteira, o comportamento de antes. A turma é conferida contra a MATRÍCULA do professor (`professorLeciona`), nunca contra a escola do *token*.

Alternativas descartadas.

- *Só um seletor de turma na tela, sem coluna*: o professor escolheria a sala a cada visita e nada registraria para qual turma a atividade foi feita. Pior: `POST /runs` não teria como saber, e o estudante do 8B continuaria entrando na atividade do 8A.
- *`turma_id` NOT NULL, com backfill*: escolher uma turma para as atividades que já existem afirmaria uma decisão que nenhum professor tomou. É a mesma regra do `inclinacao_inicial` em `20260829120000_conclusao_primeira_e_ultima`: NULL quer dizer "anterior à mudança", e isso é informação.
- *Confiar só na RLS para o vazamento*: `users` não tem política, e não deve ter — ela é lida no *login*, antes de se saber a escola. A tranca correta é o `where`.
- *Uma atividade por turma, obrigatória*: transformaria a criação de atividade num laço para o professor que dá a mesma aula em quatro salas, sem que nada no produto peça isso hoje.

Consequências. Atividades antigas seguem escolares, e o painel agora DIZ isso em uma faixa — antes ele mostrava a lista sem explicar de quem era. O `POST /runs` ganhou uma leitura a mais (`users.turma_id`, o cache da matrícula principal) antes de escolher a atividade; a ordenação põe a atividade DA SALA à frente da geral quando as duas estão ativas. O professor que criar atividade sem escolher turma continua no comportamento antigo, sem aviso de erro: é opção, não omissão.

Gatilho de reavaliação. Uma rede rodar o Instanooria em várias turmas simultâneas com a MESMA pergunta: aí a pergunta é se a atividade deve ter várias turmas (tabela de junção) em vez de uma, e a resposta muda o painel inteiro.

Este gatilho disparou em 2026-09-05, pelo outro lado: ver a advertência no topo desta ADR.

18.51 ADR-50 — O perfil importado guarda a PARTIDA e a VERSÃO do mapeamento, e não ganha chave estrangeira

> **Contexto**
>
> `app_70_sessoes` (chamada `vila_sessoes` até a **ADR-20**) não tem prazo de retenção por uma decisão registrada em `20260826170000_vila_sem_prazo`: o bruto do jogo é a ÚNICA fonte para recalcular o perfil quando `VERSAO_MAPEAMENTO` subir, e o mapeamento é hipótese de design que vai mudar.
> Só que a cópia que alimenta o Instanooria (`exp08ef1b_app_70_perfis_importados`) guardava `escores`, `dominante`, `origem`, `lado_forcado` e `medido_em` — e nem a sessão de onde veio, nem a versão sob a qual foi medida. Ao subir a constante, os `app_70_perfis` seriam recalculados e as cópias ficariam na geração anterior, decidindo o lado do feed, sem nada em lugar nenhum dizendo isso. É exatamente a armadilha do `expires_at`: o registro fica, a regra sai, e quem lê acredita.

Decisão. Duas colunas anuláveis, sem *backfill* (`20260830100000_perfil_importado_sabe_a_partida`): `vila_sessao_id` e `versao_mapeamento`. Gravadas no `create` E no `update` do *upsert* — se só o `create` gravasse, reimportar não tiraria a linha do estado "desatualizada" e o aviso da tela nunca sumiria. O painel do professor passa a dizer quantos perfis vieram de uma versão anterior, como AVISO: perfil de geração antiga ainda é melhor informação que o sorteio.

SEM chave estrangeira para `vila_sessoes`, e isso é o centro da decisão. Zerar a turma na Vila apaga sessões e bolhas; com FK, isso levaria em cascata — ou travaria com `RESTRICT` — o perfil que está decidindo o lado de uma aula em andamento. Hoje zerar a Vila NÃO toca `perfis_importados`, de propósito, e a coluna não pode mudar isso: ela é RASTRO da medida, não vínculo de integridade. Sessão apagada vira um id que não resolve, que é exatamente a verdade do que aconteceu.

Alternativas descartadas.

- *Um valor novo em `origem`* (por exemplo `jogo-v3`): `perfis_importados_origem_check` e `runs_perfil_origem_check` congelam o vocabulário em dois lugares, e o contrato Zod num terceiro. `vila_sessao_id IS NULL` já responde "não veio da Vila desta instalação" sem mexer em nenhum deles.
- *Preencher a versão das linhas existentes com a atual*: afirmaria uma medida que ninguém fez. NULL é a leitura honesta de "anterior a esta migração, ou preenchido à mão pelo professor".
- *Reimportar sozinho quando a versão sobe*: desfaria em silêncio a redistribuição que o professor fez antes da aula. O disparo continua sendo dele (ADR do Ato 1), e o que muda é que agora a tela AVISA que vale reimportar.
- *Uma rotina de reprocessamento agora*: ela não existe nem para a própria Vila. Escrever a coluna sem o consumidor seria a armadilha de novo, ao contrário — por isso a coluna nasce COM o aviso na tela que a lê.

Consequências. A capacidade de reprocessar continua sendo capacidade, e não rotina: nada recalcula sozinho. O que existe agora é a informação para decidir — o painel sabe dizer que a medida envelheceu. Um perfil cuja sessão foi apagada mantém o id órfão, e isso é aceito.

Gatilho de reavaliação. A primeira subida real de `VERSAO_MAPEAMENTO` com dado de contrato em produção: aí se descobre se o aviso basta ou se a reimportação precisa virar rotina assistida.

18.52 ADR-51 — A suíte de ponta a ponta é organizada por PAPEL, e a descrição de cada tela mora no teste que a fotografa

> **Contexto**
>
> A validação visual eram doze arquivos soltos em `scripts/`, nomeados pela EXPERIÊNCIA que cobriam (`walkthrough-estudante`, `walkthrough-rede`, `walkthrough-ato1-vila`). Três problemas mediram-se sozinhos.
> O primeiro é de cobertura: telas inteiras não tinham dono. Criar uma atividade do zero, os períodos letivos, a análise de produto, o binding do OneRoster, o compositor de post do Instanooria — nada disso era exercitado, porque nenhum arquivo se chamava por elas.
> O segundo é de divergência. Cada roteiro carregava sua própria cópia de `registrar`, de `entrar`, do vigia de console e do relatório final, e as doze cópias divergiram: uma tirava `fullPage` e outra não, três reprovavam por erro de console e nove não. Divergência em código de teste é pior que em código de produção — quem lê o resultado não sabe qual regra valeu.
> O terceiro é o pior. O deck de evidências nascia de um segundo arquivo (`testes/.gerador-ppt/roteiro.mjs`, 66 KB) mantido À MÃO, com a descrição de cada slide escrita longe do teste que tirava o *print*. Ele divergiu do que os scripts faziam: descrevia telas que já não eram fotografadas e omitia passos novos. É a mesma armadilha do `expires_at` e do `turmas.consentimento_em` — o registro fica, a regra sai, e quem lê acredita nele.

Decisão. A suíte passa a viver em `scripts/e2e/<papel>/`, com uma casca compartilhada em `scripts/e2e/lib/roteiro.mjs` e um *runner* único (`scripts/e2e/rodar.mjs`).

Por PAPEL, não por experiência. Quem usa o produto é uma pessoa com um crachá, e é o crachá que decide o que ela alcança. Organizar por papel faz a LACUNA aparecer: "o gestor não tem suíte de períodos letivos" é uma frase que alguém diz em voz alta; "não existe `walkthrough-periodos`" não é. A contrapartida está registrada abaixo.

O manifesto é a fonte. Cada `registrar()` grava, junto com o *print*, o TÍTULO da tela, o que ela MOSTRA e a regra que ela PROVA, num `passos.json` por suíte. O deck (`gerar-deck.mjs`) e os vídeos (`gerar-videos.mjs`) só desenham — não conhecem o produto. A descrição passou a morar onde o passo mora, e sair de sincronia deixou de ser possível.

Alternativas descartadas.

- *Manter a organização por experiência*: é a que casa com o `apps/experiencias/`, e para a app-70 e a exp02em1b seria mais direta. Mas ela esconde as telas da PLATAFORMA, que são a maioria e não pertencem a experiência nenhuma — e foram justamente elas que ficaram sem cobertura por dois anos de projeto.
- *Um processo só para todas as suítes*, mais rápido por reaproveitar o navegador: uma suíte que trava levaria as outras vinte junto, e o código de saída deixaria de ser por suíte. O *runner* paga o custo do `spawn` de propósito.
- *Playwright Test* (`@playwright/test`) no lugar da casca própria: traz *runner*, *fixtures*, *retries* e relatório HTML de graça. O que ele NÃO traz é o manifesto descritivo, que é o objetivo aqui — a saída dele é um relatório de teste, não um documento de evidência com uma frase de produto por tela. Adotá-lo significaria manter o roteiro do deck à mão de novo, que é o problema que esta ADR resolve.
- *Manter o roteiro do deck à mão, só arrumando o que divergiu*: arruma a divergência de hoje e garante a de amanhã.
- *Deck em PDF ou HTML*: o entregável pedido é `.pptx` — é o que a rede abre, edita e projeta.

Consequências. As suítes de dois contextos (o professor conduz o relógio, o estudante joga) ficam no papel do PROTAGONISTA, e não em ambos: a missão da exp08ef1b mora em `estudante/`, ainda que metade dos *prints* sejam da tela do docente. O passo carrega de qual contexto veio, e o deck e o vídeo dizem isso na tarja.

A ORDEM de execução passa a importar e está fixada no *runner*: o estudante produz o estado que o professor lê, e a administração vem por último porque importa gente e troca senha. Rodar uma suíte isolada continua possível; rodar o professor antes do estudante devolve telas de resultado vazias.

E há um custo aceito: uma experiência nova não tem mais um lugar óbvio. Ela entra em `estudante/` se for missão, em `professor/` se for ferramenta — e a `AGENTS.md` da pasta `apps/experiencias/` passa a dizer isso.

Gatilho de reavaliação. Um papel novo no produto (o *responsável*, por exemplo, se o acesso da família for adiante), ou a suíte passar de 40 minutos de execução — aí a paralelização por papel deixa de ser opcional e o *runner* precisa crescer.

18.53 ADR-52 — Falha do provedor de IA na destilação é 503 com a causa, não 500 mudo

> **Contexto**
>
> Quatro lugares chamam `generateJson`, e três decidem o que fazer quando ele falha: as reações ao vivo caem em lista vazia (o feed fica quieto, como um app em que ninguém respondeu), e a geração de posts e de personas vira *job* falho que o painel de progresso mostra. O quarto, `geracao.controller.ts` (`POST /atividades/:id/destilar`), deixava a exceção subir.
> O Nest a convertia em `{"statusCode":500,"message":"Internal server error"}`. E o *log* não salvava: o `ErroDeGeracao` guarda a causa real em `detalhe`, e o `ExceptionsHandler` imprime apenas a `message` — que no `VertexProvider` é a genérica "Falha ao gerar conteúdo estruturado.".
> Com `AI_PROVIDER=mock` isso deixa de ser um caso raro e vira certeza: `MockProvider.generateJson` lança SEMPRE, por decisão (o mock não sabe satisfazer schema arbitrário). Ou seja, no modo que existe para subir sem GCP — o recomendado em desenvolvimento, e o *default* dos dois *compose* — **criar atividade era impossível, e nada em lugar nenhum dizia por quê**. Custou uma tarde de investigação: a destilação reproduzida fora da api funcionava, o `UPDATE` do Prisma funcionava, e a resposta só apareceu num `ps eww` do processo, onde se via o `AI_PROVIDER=mock` que o `.env` não declarava.

Decisão. A chamada entra num `try/catch` e a falha vira **503** com `mensagem` legível, acrescida da causa quando existe. A causa é lida como `detalhe ?? message`: o `VertexProvider` enche o `detalhe` e deixa a `message` genérica; o `MockProvider` faz o contrário. Ler só um dos dois deixaria sem diagnóstico exatamente o modo em que a falha é garantida.

503 e não 500 porque a requisição está correta e repetir pode funcionar: o que falhou é uma dependência. É a mesma distinção que a api já faz em `/ativar` com o 422.

Alternativas descartadas.

- *Um `ExceptionFilter` global para `ErroDeGeracao`*: pareceria mais econômico e seria pior — transformaria em 503 também os três chamadores que hoje DEGRADAM de propósito, e a lista vazia das reações ao vivo viraria erro na cara da criança no meio da aula. A decisão de degradar é de cada chamador, e por isso o tratamento fica em cada um.
- *Fazer o `MockProvider` devolver um antagonismo plausível*: seria o mock passando a conhecer o domínio, e um dia divergiria dele em silêncio. Pior: esconderia o caminho de falha em vez de torná-lo legível — em produção o provedor também cai, e é esse caminho que precisa estar exercido.
- *Deixar o 500 e melhorar só o log*: resolveria para quem tem acesso ao *log* da api. O professor continuaria vendo "Internal server error", e a suíte de ponta a ponta continuaria reportando um sintoma que não aponta para a causa.

Consequências. `tests/destilar-ia-indisponivel.spec.ts` trava o comportamento em cinco asserções, incluindo que a atividade NÃO é movida pela metade (nem `status`, nem `pergunta_chave`). O roteiro `scripts/e2e/professor/02-atividade-do-zero.mjs` passou a falhar em 9 s com a causa impressa, no lugar dos 180 s esperando "Perguntas propostas pela IA" — é a mesma lição do `esperarOuFalharCedo`, agora do lado do servidor.

Ruim: a mensagem expõe texto do provedor ao cliente. Aceito porque a rota é de professor autenticado e a causa é operacional ("sem implementação", "resposta vazia"), nunca dado de estudante — e o corte em 200 caracteres impede que um *stack* inteiro vaze para a tela.

Gatilho de reavaliação. Um quarto consumidor de `generateJson` sem fallback, ou o dia em que a destilação ganhar um — por exemplo, um antagonismo escrito à mão pelo professor sem passar pela IA, que hoje ainda depende dela para avaliar o equilíbrio.

18.54 ADR-53 — Um ambiente por *branch*: *environments* do GitHub carregam as `vars`, e a *branch* escolhe a *stack*

> **Contexto**
>
> Até agosto de 2026 havia uma *stack* GCP só (`dev`) e o *deploy* disparava no *push* para `main`, lendo doze `vars` do nível do **repositório**. Com a *stack* de `hml` provisionada (projeto GCP próprio, *workspace* Terraform próprio, WIF *pool* e *deployer* SA próprios), esse desenho não comporta dois ambientes: `vars` de repositório são um conjunto único, e `main` não pode significar "dev" e "hml" ao mesmo tempo.
> O fluxo de trabalho também mudou: ninguém mais commita direto em `main`. *Branch* de trabalho entra por PR em `dev`; `dev` promove por PR para `hml`; e, quando produção existir, `hml` promoverá por PR para `main`.

Decisão. Um *environment* do GitHub por ambiente (`dev`, `hml`, futuramente `prd`), cada um com o conjunto completo de `vars` e o *secret* `TOKEN_SIGNING_KEY_PEM` escopados nele. O `deploy.yml` dispara em `push` para `dev` e `hml`, e cada *job* de *deploy* declara `environment: ${{ github.ref_name == 'main' && 'prd' || github.ref_name }}` — a *branch* **é** o seletor de *stack*, com `main` como a única exceção de nome. Cada *environment* tem *deployment branch policy* restrita à própria *branch*: o de `hml` só aceita *job* vindo de `hml`, então nem um `workflow_dispatch` de outra *branch* consegue assumir a identidade WIF do ambiente errado. O `scripts/configurar-github-ci.sh` deriva o *environment* de `terraform workspace show` — a mesma fonte dos valores — e cria *environment*, *policy*, `vars` e *secret* de uma vez.

Alternativas descartadas.

- *Prefixar as `vars` de repositório* (`DEV_API_SERVICE`, `HML_API_SERVICE`) e escolher no *shell*: dobra os nomes, espalha `if` por todos os *jobs* e não dá nenhuma trava — qualquer *branch* leria as `vars` de qualquer ambiente. O *environment* dá o escopo E a *branch policy* de graça.

- *Um `deploy-dev.yml` e um `deploy-hml.yml`*: os dois divergiriam em silêncio — é a mesma armadilha do roteiro de 66 KB da suíte de ponta a ponta, mantido à mão até descrever telas que não existiam. Um *workflow* só, parametrizado pelo *environment*, não tem como divergir de si mesmo.
- *Deploy por tag* (`v1.2.3-hml`): desacopla o histórico da promoção, mas cria um segundo vocabulário (qual *commit* está em `hml`? o da *tag*, não o da ponta de nenhuma *branch*) e dispensa o PR de promoção — que é justamente onde o *diff* `dev..hml` fica visível e revisável.

Consequências. O bloco `env:` global do `deploy.yml` desceu para dentro dos *jobs*: `vars` de *environment* só resolvem em *job* que declara `environment:`, e no nível do *workflow* chegariam vazias, sem erro. Referenciar um *environment* inexistente o cria em silêncio (com `vars` vazias), então o *job* `mudancas` ganhou um passo de guarda que derruba *dispatch* de *branch* que não é de ambiente. O *cache* do Docker ganhou o ambiente no escopo (`scope=api-dev`, `scope=api-hml`): `dev` e `hml` não invalidam o *cache* um do outro a cada *push* alternado. E o critério de *diff* por `HEAD^` passou a depender do método de *merge*: *merge commit* e *squash* preservam o primeiro pai como a ponta antiga da *branch* de ambiente; *rebase* o quebraria, e fica desabilitado nas opções do repositório.

Ruim: são $12 \times N$ `vars` para manter (N = ambientes), e a UI do GitHub não compara *environments* lado a lado — um valor desatualizado só aparece como falha no meio do *deploy*. Mitigado porque nenhum valor é digitado: o *script* os copia de `terraform output`, e reexecutá-lo é idempotente.

Gatilho de reavaliação. A chegada de `main`/produção (que testará o mapeamento `main` → `prd` e pedirá *required reviewers* no *environment*); ou o segundo contrato — *environments* escalam por ambiente, não por contrato, e aí a conversa vira um repositório de *deploy* por contrato ou uma matriz.

18.55 ADR-54 — Pessoa nasce por IMPORTAÇÃO: não existe cadastro avulso de usuário

Estado: vigente, com o gatilho acionado (2026-09-01). O cadastro unitário passou a existir — e existe do jeito que o gatilho abaixo prescreve: um formulário que monta a MESMA linha de CSV e chama o MESMO importador, nunca um segundo caminho de escrita em `users`. Ver **ADR-59**, que também abriu a rota separada de `admin-contrato`. Tudo o mais nesta ADR continua valendo, inclusive as três alternativas descartadas.

> **Contexto**
>
> Quem pergunta "como eu crio um usuário?" procura, naturalmente, um botão "Novo" na tela `/admin/usuarios`. Ele não existe, e a ausência nunca esteve escrita em lugar nenhum — estava só implícita no fato de a API não ter `POST /usuarios`.

> O provisionamento desta plataforma não é auto-serviço: a lista de pessoas vem da REDE (a secretaria, o sistema de matrícula), e a identidade que importa é a **matrícula da rede** — com o CPF como chave que promove uma linha repetida a transferência entre unidades (**ADR-28**, **ADR-32**). Uma pessoa não é só um `users`: é lotação, matrícula em turma, período letivo e senha provisória, nascidos juntos na mesma transação.

Decisão. As **únicas** duas chamadas que criam linha em `users` são `POST /rostering/importar` (a planilha simples, uma linha por pessoa) e `POST /rostering/oneroster/users` (o *binding* do padrão, para a rede que já o exporta). *Desde a **ADR-59**, some-se a elas `POST /usuarios/admin-contrato` — o único papel que não cabe numa planilha da escola; `POST /usuarios` não é uma terceira, porque delega à primeira.* As duas exigem `gestor` — na própria escola — ou `admin-contrato`, com `?escolaId=`. Criar UMA pessoa é importar uma planilha de uma linha: a operação é idempotente pela matrícula (e pelo CPF), então repetir não duplica nem reseta a senha de quem já entrou.

O que existe depois da criação são as operações de **balcão**, e essas sim são unitárias: `PATCH /usuarios/:id` corrige nome, CPF, e-mail, papel e `ativo`; `POST /usuarios/:id/reset-senha` devolve uma senha temporária uma única vez.

Alternativas descartadas.

- *Um `POST /usuarios` com formulário na tela*: é uma segunda porta de escrita sobre o mesmo modelo, e portas de escrita divergem. Na primeira regra nova — o CPF que vira transferência, a matrícula que precisa de período letivo aberto, o papel `coordenacao` que normaliza para `gestor` — uma das duas a implementaria e a outra não, em silêncio. É exatamente o motivo pelo qual `POST /usuarios/importar-csv` sobreviveu apenas como *delegação* para o importador, e não como um segundo importador.
- *Auto-cadastro pelo próprio usuário*: são menores. Não há prova de vínculo com a escola que uma pessoa possa apresentar sozinha, e a consequência de aceitar uma seria uma criança dentro do *tenant* errado.
- *Provisionamento just-in-time no primeiro login* (criar a pessoa a partir do *token* do IdP): tentador quando o WSO2 entrar, e recusado desde já — o *token* do IdP não carrega turma nem série, e a pessoa nasceria sem enturmamento, invisível para o professor e fora de qualquer conferência da rede. Quando o WSO2 chegar, ele autentica; quem provisiona continua sendo a importação.

Consequências. *Ruim*: o gestor que percebe "esqueci de incluir a Ana" precisa montar um arquivo para uma linha — atrito real de balcão, e a tela hoje só aceita *upload*, sem campo de colar. Mitigado em parte porque o balcão de verdade (nome errado, CPF trocado, senha perdida, saída da rede) é editar, não criar, e essas rotas existem.

Gatilho de reavaliação. Se a operação pedir cadastro unitário com frequência, a saída é um formulário que monta a MESMA linha de CSV e chama o MESMO importador — nunca um segundo caminho de escrita em `users`.

18.56 ADR-55 — Instrução de agente: `AGENTS.md` canônico por fronteira, `CLAUDE.md` como ponteiro de uma linha

> **Contexto**
>
> Uma pessoa desenvolve esta plataforma, e boa parte do trabalho passa por agentes de codificação. O que eles sabem sobre o projeto vem dos arquivos de instrução que a ferramenta carrega sozinha ao tocar uma pasta — e esses arquivos viraram um padrão *de fato*, nunca escrito: `AGENTS.md` por domínio, com um `CLAUDE.md` de uma linha ao lado. Padrão que não está escrito não se verifica, e os desvios apareceram. A doutrina da Vila do Lume — as regras que protegem a Camada 2 do instrumento — viveu meses em `app-70-vila-do-lume/docs/CLAUDE.md`: o carregamento é por caminho, então quem editava o jogo em `public/jogo/` nunca a recebia. Ela ainda mandava rodar dois `scripts/walkthrough-*.mjs` que deixaram de existir quando a suíte passou a ser organizada por papel (**ADR-51**) — instrução desatualizada, que é pior que instrução nenhuma, porque o agente obedece.

Decisão. Três arquivos, três papéis que não se misturam. `AGENTS.md` é **canônico**: o único lugar onde a doutrina de uma pasta é escrita, e o nome que a maioria das ferramentas lê. `CLAUDE.md` é **uma linha**, `@AGENTS.md` — o Claude Code procura esse nome, o ponteiro importa o canônico, e a regra existe uma vez só. `README.md` continua sendo para pessoas: o que a pasta é e como rodar, e ele *não* entra sozinho no contexto do agente.

Um `AGENTS.md` nasce numa pasta onde se trabalha uma tarefa inteira sem sair dela — um app, um pacote, uma experiência, a suíte de ponta a ponta — e sempre na RAIZ dela, nunca numa subpasta. Entra nele o que, se você não souber, você quebra: regra já violada e o que custou, contrato com quem consome, zona congelada, armadilha que passa no `typecheck`. Não entra o que o código já diz nem o que o `README.md` ao lado já explica. Onde o `README.md` já é a doutrina — `scripts/e2e`, `documentacao-latex` — o `AGENTS.md` é curto e serve de *carregador*: as poucas regras invioláveis e a ordem de abrir o README. Alterar a regra e o arquivo é o mesmo *commit*, e isso entra na *Definition of Done*.

Decisão irmã, do mesmo movimento: **varredura de vulnerabilidade pertence à esteira**, com o segredo e a política de prazos do repositório (**ADR-39**), não à sessão do agente. O agente não dispara `snyk` nem as ferramentas do MCP correspondente; as regras estão em `.claude/settings.json` (escopo do projeto) e no equivalente do usuário. Varredura na sessão gasta cota do plano, produz apontamento sem o portão que o cumpre, responde por uma árvore de trabalho local que não é o que vai a produção — e recicla falsos positivos já analisados e registrados, como o *code injection* do `setInterval` da *bridge*. Quem precisar de uma varredura avulsa a roda na mão.

Alternativas descartadas.

- *Um arquivo só, na raiz*: já foi assim, e o arquivo cresceu até o ponto em que a regra da api ocupava contexto de quem mexia numa experiência. A divisão por pasta existe porque o carregamento é por caminho: cada sessão recebe o que é dela.

- *`CLAUDE.md` com conteúdo próprio ao lado do `AGENTS.md`*: duas fontes de verdade sobre a mesma pasta. Elas divergem, sempre na direção de quem parou de ler a outra — a mesma armadilha do `expires_at` que saiu do *schema* e ficou no registro.
- *Só `README.md`, sem `AGENTS.md`*: o README não entra no contexto sozinho. Foi o que fez a doutrina da Vila do Lume ficar invisível para quem ela tentava avisar.
- *Gerar os `AGENTS.md` a partir do código*: o que vale ali é justamente o que o código não diz — por que a medida é 0,6 e não 0,5, o que custou descobrir. Gerador nenhum sabe disso.

Consequências. *Ruim*: são vinte pares de arquivos a manter, e cada um pode envelhecer sozinho — o custo é real e recai sobre a mesma pessoa. Mitigado por duas coisas: a proibição explícita de repetir conteúdo entre eles (cópia é o que ninguém atualiza junto) e o item novo na *Definition of Done*. *Ruim também*: bloquear o MCP de varredura no escopo do usuário vale para todos os projetos da máquina, não só para este.

Gatilho de reavaliação. Um `AGENTS.md` que ninguém consultou ao mudar a pasta que ele governa — sinal de que ele descreve o que o código já diz, e deve encolher ou desaparecer. Do lado da varredura: a esteira deixar de cobrir uma classe de vulnerabilidade que só a análise local pega.

18.57 ADR-56 — O fluxo por ambiente é imposto por *hooks* versionados, não por *branch protection*

> **Contexto**
>
> A **ADR-53** definiu o fluxo — *branch* de trabalho → `dev` → `hml` → `main` — e o *deploy* passou a depender dele: o *merge* do PR é o que sobe código para a escola. A regra existia em prosa no `AGENTS.md` e nada a sustentava: `git push origin dev` funcionava, e funcionaria de madrugada, no fim de uma sessão longa, sem portão e sem PR.
> A resposta natural seria `branch protection` ou `rulesets`. As duas APIs respondem **403 — "Upgrade to GitHub Pro or make this repository public"**: o repositório é privado numa conta de plano *Free*. Tornar público não é opção (é código de plataforma com dado de menor no desenho), e pagar por assento para uma pessoa, hoje, não se justifica — mas a regra precisa valer **agora**, porque `hml` já é um ambiente real.
> Faltavam também as duas pontas que só aparecem quando produção existe: como se corrige produção sem furar a fila, e como se diz *qual* versão está lá. Não havia uma *tag* no repositório.

Decisão. A barreira é local e versionada: `.githooks/`, ativado por `core.hooksPath` no `prepare` do `package.json` — ou seja, `pnpm install` instala. `pre-push` recusa *push* direto em `dev`, `hml` e `main` (inclusive deleção) e nome de *branch* fora do padrão; `commit-msg` valida *Conventional Commits* com os escopos deste monorepo. Os escapes são **nomeados** (`PERMITIR_PUSH_DIRETO=1`,

`PERMITIR_BRANCH_FORA_DO_PADRAO=1`), e não a *flag* genérica `--no-verify`: emergência acontece, e é melhor que ela deixe rastro numa variável que diz o que se está furando.

Junto com isso, duas regras que o fluxo não tinha. **Promoção para `main` leva *tag* semver e *release***, criadas depois do *merge*: MAJOR só em quebra de contrato com quem está fora do repositório (`PROTOCOL_VERSION` da ponte, manifesto, telemetria, contrato do Terraform), MINOR no caso comum, PATCH em correção. ***Hotfix* sai de `main`, entra por PR em `main` e volta para `hml` e `dev` por retro-merge** no mesmo dia. O fluxo operacional inteiro vive em `CONTRIBUTING.md` — a raiz do repositório, onde o GitHub o exibe ao abrir um PR.

Alternativas descartadas.

- *Deixar só a prosa do `AGENTS.md`*: é o estado que produziu o risco. Regra que depende de lembrar dela, às 2h da manhã, não é regra — e o custo de esquecer aqui é *deploy* para escola sem portão.
- *Tornar o repositório público para liberar `rulesets`*: trocar a superfície de exposição do produto inteiro por um recurso de processo. Desproporcional.
- *Husky + commitlint + lint-staged*: três dependências, uma cadeia de configuração e um *hook* gerado, para o que são dois arquivos de *shell* que o repositório inteiro consegue ler. A regra da raiz vale aqui ("simplicidade > sofisticação"), e `core.hooksPath` é nativo do git desde a 2.9.
- *Um workflow de release que cria a tag sozinho no push para `main`*: automação que decide o número da versão precisa de uma fonte para ele, e a única honesta seria um arquivo que alguém edita — o mesmo trabalho manual, com uma engrenagem a mais para falhar. Enquanto a promoção for um ato deliberado de uma pessoa, o comando documentado basta.
- *Trunk-based puro, com feature flags no lugar de `hml`*: exige disciplina de *flag* e um custo de teste que uma pessoa não paga; e `hml` existe porque a rede cliente precisa de um lugar para homologar antes da escola.

Consequências. *Ruim*: *hook* é conselho, não muralha — quem clonar e não rodar `pnpm install` não tem nenhum, e `--no-verify` fura. A barreira protege contra descuido, não contra intenção, e isso é aceitável enquanto o time é de uma pessoa; deixa de ser no primeiro colaborador externo. *Ruim também*: o `prepare` roda em CI e em contêiner, onde o *hook* não serve para nada — é inofensivo (`git config` num *checkout* que existe), mas é ruído.

Gatilho de reavaliação. O primeiro colaborador com *commit* no repositório, ou a conta virar *Pro* / organização: nos dois casos as mesmas regras viram `ruleset` no servidor, e os *hooks* passam a ser conveniência local (retorno rápido) em vez de barreira única.

18.58 ADR-57 — Observabilidade nativa do Google Cloud, com OpenTelemetry como fronteira; nenhuma *stack* auto-hospedada

Estado: aceita (2026-09-02; proposta em 2026-09-01 a partir do estudo do Capítulo 17). Aceita com a primeira peça no código: log JSON com *redaction* na API e no hub (fase 1) e *traces* OpenTelemetry com o exportador direto do Cloud Trace nos dois processos (fase 2, *traces*); as métricas da aplicação e o erro de cliente seguem o roteiro do capítulo.

> **Contexto**
>
> A plataforma é diagnosticável e não é observável: log em texto sem correlação, nenhuma métrica de aplicação, nenhum alerta, nenhuma sonda configurada no Cloud Run, erro dentro do `iframe` sem rastro. O risco `R-05` exige alerta antes de novembro de 2026. Uma pessoa opera; um contrato é um projeto GCP; haverá vários contratos. A regra da raiz proíbe serviço novo sem necessidade demonstrada, e a franquia gratuita do Google Cloud Observability (50 GiB de log, 150 MiB de métrica, 2,5 M de *spans*, 1 M de *uptime checks* por mês) cobre um contrato em regime com folga, desde que o log de requisição seja amostrado na origem.

Decisão. Os sinais operacionais vão para o Cloud Logging, Cloud Monitoring, Cloud Trace e Error Reporting **do projeto do contrato**. A API e o hub emitem log JSON pelo *stdout* (`pino`, com *redaction* mecânica e `severity`/`trace` nos campos que o Cloud Logging reconhece) e *traces* e métricas pelo SDK OpenTelemetry com os exportadores do Google, **direto do processo, sem *sidecar***. Alertas são poucos e por sintoma (SLO com *burn rate*), em duas severidades. Tudo o que é recurso — canais, políticas, *checks*, painel, exclusões, *bucket* de auditoria, *dead letter*, sondas do serviço — é Terraform, e passa a constar do contrato em `docs/INFRA.md`. A visão entre contratos é um *metrics scope* num projeto de observação sem recursos; log não atravessa projeto.

Alternativas descartadas.

- *Grafana + Loki + Tempo + Prometheus auto-hospedados*: uma VM central (ou uma por contrato), com retenção, *backup*, atualização e autenticação por conta própria, recebendo dado de N projetos por rede e IAM próprios — e custando, no mínimo, mais que o nativo no pior caso. Para uma pessoa, é a peça que quebra às 2h da manhã. Volta se uma rede exigir que os sinais fiquem fora do Google Cloud, ou se uma consulta demonstrada não couber no Logs Explorer e no Log Analytics.
- *SigNoz / OpenObserve / HyperDX* (tudo-em-um OpenTelemetry): mesma objeção operacional, com a vantagem de um só binário. Gatilho igual ao anterior.
- *Grafana Cloud Free* como destino (10 k séries, 50 GB de log, 14 dias): o dado sairia do projeto do contrato para uma conta de terceiro sem DPA da rede. Aceitável apenas como **painel** de leitura sobre a fonte de dados Cloud Monitoring, se a ergonomia do Grafana compensar — é leitura, não coleta.

- *Datadog / New Relic / Dynatrace*: centenas de dólares por mês por contrato, para um time de operação que não existe. A instrumentação OpenTelemetry deixa a porta aberta sem pagar por ela agora.
- *OpenTelemetry Collector como sidecar* (a recomendação do Google): justifica-se com vários destinos ou *tail sampling*; hoje há um destino e amostragem de cabeça basta. É um contêiner a mais em cada serviço de cada contrato, com versão e falha próprias.
- *Log de requisição na aplicação* (`morgan`, interceptor): duplicaria o log que o Cloud Run já grava, dobrando o sinal mais caro para acrescentar nada.
- *Continuar como está, com o checklist semanal do Apêndice F*: é o estado que produziu o `R-05`. Funciona em `dev` com uma pessoa olhando; não funciona com cem escolas às 8h.

Consequências. *Boa*: zero peças novas para operar; custo estimado em cerca de US$ 1 por contrato por mês hoje e US$ 7 quando a cobrança de política de alerta entrar; um contrato novo herda tudo do Terraform. *Ruim*: dependência do fornecedor nos **destinos** (painel, política, consulta são em formato do Google); mitigada porque a instrumentação é OpenTelemetry e o log é JSON — trocar de destino é trocar exportador e reescrever painéis, não reinstrumentar. *Ruim também*: dois custos crescem com o tráfego e precisam de teto na origem — log de requisição (por isso a exclusão e a amostragem são recursos do Terraform) e cardinalidade de métrica (por isso `escola_id` não é rótulo). E a cobrança anunciada de política de alerta muda a economia de "uma política por curiosidade": a lista fica curta também por dinheiro.

Gatilho de reavaliação. Uma rede exigir sinais fora do Google Cloud; um segundo operador (plantão, rodízio); um segundo destino (SIEM, Grafana) ou a necessidade de *tail sampling* — aí o *sidecar* entra; ou a franquia estourar em dois meses seguidos apesar da exclusão e da amostragem.

Adendo (2026-09-02): métricas por OTLP, não pelo exportador do Google. O texto acima diz "exportadores do Google". Vale para os *traces* (`@google-cloud/opentelemetry-cloud-trace-exporter`, gRPC, leve). Para as métricas, o `-cloud-monitoring-exporter` depende de `googleapis` — **202 MB** medidos em `node_modules` — para chamar um método da API de monitoramento; entraria inteiro na imagem da API. A decisão é exportar métricas em OTLP/JSON direto para `telemetry.googleapis.com/v1/metrics`, com um cliente de 60 linhas que pede o *token* ao `google-auth-library` a cada envio (`observabilidade/otlp-google.ts`) — o mesmo cliente por onde a API reexporta os *spans* do navegador. Alternativa descartada: o exportador OTLP/HTTP do próprio OpenTelemetry, que recebe cabeçalhos estáticos e não acompanha a expiração do *token* OAuth.

O risco foi exercido em `dev` no mesmo dia, e a decisão se sustentou — ao custo de duas descobertas que valem mais que a decisão:

1. **O *resource* precisa de `gcp.project_id`**, e o `gcpDetector` *não* o preenche (ele traz `cloud.account.id`, que a Telemetry API não aceita no lugar). Sem o atributo, TODA exportação volta `HTTP 400 Resource is missing required attribute`, a cada 60 s — com log e *trace* funcionando,

o que torna a falha invisível para quem olha o painel e visível só no `diag` do SDK. Corrigido em `observabilidade/recurso.ts`, com teste de regressão.

2. **A ingestão OTLP grava no domínio `prometheus.googleapis.com`**, não em `workload.googleapis.com` — onde o exportador legado gravava, e onde este documento mandava procurar. A série sai como `prometheus.googleapis.com/unooria.telemetria.descartados/counter`, no recurso monitorado `prometheus_target` (`job` = `service.name`, `instance` = a instância do Cloud Run, `location` = a região), com `otel_scope_name` nos rótulos. Por algumas horas isso fez parecer que *nenhuma* métrica estava sendo exportada, quando estavam todas. É consequência declarada da migração, não defeito: a nota oficial avisa que a mudança de domínio "quebra painéis existentes". Aqui saiu de graça, porque painel ainda não existe — e é o tipo de custo que se paga barato *antes* da produção.

O caminho de volta (exportador do Google, 202 MB) segue registrado e não foi preciso.

Segundo adendo (2026-09-02): os exportadores do Google para Node estão DEPRECIADOS — os PACOTES, não os produtos. O SDK avisa no *stdout*: `DEP_GCP_OTEL_TRACE_EXPORTER --- deprecated and will be archived after October 30th, 2026`. A nota oficial (`GoogleCloudPlatform/opentelemetry-operations-js`, `MIGRATION.md`) delimita o escopo: "*all packages in this repository are deprecated*" e "*the underlying Google Cloud services (Cloud Trace, Cloud Monitoring) remain operational*". **O Cloud Trace não vai a lugar nenhum**: muda o caminhão de entrega (OTLP pela Telemetry API), não o destino.

A migração foi feita no mesmo dia (`feat/obs-traces-otlp`), e o prazo — 30 de outubro de 2026, *anterior* à entrada em produção — deixou de ser um risco carregado. Nenhum pacote `@google-cloud/opentelemetry-*` sobrou na api nem no hub.

O desenho mudou numa coisa em relação ao primeiro adendo, e para melhor: os sinais do PRÓPRIO processo passam pelos exportadores **oficiais** do OpenTelemetry (`@opentelemetry/exporter-trace-otlp-http` e `-metrics-otlp-http`), não por cliente escrito à mão. O que tornou isso possível foi a opção `headers` aceitar uma *função assíncrona* — é ela que resolve o problema que motivou o cliente próprio (o *token* da ADC expira em uma hora, e cabeçalho estático pararia de exportar sozinho). A fábrica está em `observabilidade/otlp-google.ts` (`cabecalhosParaTelemetryApi`); o exportador de métricas escrito à mão foi apagado, e com ele um defeito que ele tinha e a biblioteca não tem: ele tratava qualquer 2xx como sucesso, ignorando o `partialSuccess` que o OTLP devolve quando aceita o lote mas descarta pontos.

O `ClienteOtlpGoogle` continua existindo para o que a biblioteca não sabe fazer: reencaminhar um corpo OTLP *já pronto*, vindo do navegador (`POST /observabilidade/v1/traces`).

Saldo de dependências: saíram `@google-cloud/opentelemetry-cloud-trace-exporter` e o `@grpc/grpc-js` que ela arrastava; entraram os dois exportadores oficiais, que compartilham `otlp-transformer` e `otlp-exporter-base` com o que já estava instalado.

18.59 ADR-58 — Erro de cliente entra pela API; nenhum SDK de terceiro no *bundle* da experiência

Estado: aceita (2026-09-02; proposta em 2026-09-01). O lado do servidor está no código: `POST /observabilidade/erros-cliente`, `/vitals` e `/v1/traces` (este último não estava no texto original: com o OpenTelemetry Web dentro da SDK, os *spans* do navegador também entram pela API, que os valida, reescreve o `resource` e reexporta com a própria credencial — o mesmo argumento, estendido). O lado do cliente também está no código (`feat/obs-sdk`, 2026-09-02): captura na SDK com dedupe e teto de 5 por sessão, `exp:erro` na ponte, *sourcemaps* `hidden` publicados em *bucket* privado à parte — e não na pasta do *bundle* com ACL, como o texto original supunha: o *bucket* público tem acesso uniforme, e ACL por objeto não existe lá.

> **Contexto**
>
> A classe de falha mais cega da plataforma é a que acontece no navegador do estudante, dentro de um `iframe` de origem diferente: não há `window.onerror`, `unhandledrejection` nem mensagem `exp:erro` na ponte, e os *bundles* não têm *source-map*. O caminho padrão da indústria é um SDK de rastreio de erro (Sentry e semelhantes) dentro do *bundle*, apontando para um *DSN*. A regra 2 da raiz proíbe segredo e URL interna em *bundle*; a regra 6 proíbe PII fora de controle. Um *DSN* não é segredo, mas é um destino: o SDK manda *user agent*, URL, *stack*, *breadcrumbs* e o que mais o erro arrastar para um processador fora do contrato — para um erro que aconteceu na tela de uma criança de uma rede pública, coberta por um DPA que não menciona esse processador.

Decisão. A `@edu/sdk` captura `window.onerror` e `unhandledrejection` e envia para `POST /observabilidade` da **própria API**, com o *token* que já tem: *schema* Zod fechado (`expId`, `versao`, `mensagem`, `stack`, `url`, `userAgent`), 8 KB no máximo, *dedupe* por *hash* e no máximo cinco por sessão no cliente, cota por `(escola, usuario)` na API. A API redige, acrescenta `escola_id` do *token* e emite **uma** linha `severity: ERROR` com `@type` de `ReportedErrorEvent`; o Error Reporting agrupa por `(expId, versao)`. O *build* das experiências gera *sourcemaps* ocultos, publicados com ACL privada ao lado do *bundle*, na mesma pasta versionada. A ponte ganha `exp:erro`, opcional e compatível com hub antigo, só para o hub trocar a tela. O hub faz o equivalente com `error.tsx` e `global-error.tsx`. Web Vitals seguem o mesmo caminho, amostrados e sem identificador de pessoa.

Alternativas descartadas.

- *SDK do Sentry (SaaS) no bundle*: melhor produto da categoria (*session replay*, *breadcrumbs*, *release health*); plano Developer grátis para uma pessoa. Descartado pelo destino, não pela ferramenta: dado de menor saindo do contrato sem DPA.
- *GlitchTip ou Sentry auto-hospedados no projeto do contrato*: resolve o destino, mas é uma peça a mais (Postgres, Redis, workers) por contrato — ou uma central com o mesmo problema de N projetos. Fica como saída se o agrupamento do Error Reporting se mostrar insuficiente para 48 experiências.

- *Só `exp:erro` na ponte, e o hub registra*: cria dependência do hub para um sinal que a experiência já consegue mandar sozinha (tem *token* e base da API), e perde o erro que acontece antes do *handshake* — justamente o mais comum. A ponte fica com o papel de interface, não de transporte.
- *Sourcemaps públicos no bundle*: expõem o fonte da experiência — e, na exp08ef1b, a lógica da revelação que o estudante não deve ver antes da hora.
- *Não capturar erro de cliente*: é o estado atual, e é a maior cegueira da Figura 17.1.

Consequências. *Boa*: nenhum terceiro no *bundle*; a regra 6 vale por construção, porque a *redaction* é a da API; a versão da experiência já está no caminho, então a *stack* é mapeável sem *release* à parte. *Ruim*: sem *session replay* nem *breadcrumbs* — o que se tem é mensagem, *stack* e contexto; um laço de erro numa experiência é contido pela cota, mas o que passa da cota se perde de propósito. *Ruim também*: mais uma rota pública autenticada na API, com Zod e cota, para manter.

Gatilho de reavaliação. O Error Reporting agrupar mal os erros das experiências (assinaturas instáveis por minificação), ou o diagnóstico exigir *replay*: nos dois casos a saída é um Sentry-compatível **dentro do projeto**, nunca o SaaS.

18.60 ADR-59 — Cadastro unitário existe, e ele DELEGA ao importador; `admin-contrato` tem rota própria

Estado: aceita (2026-09-01). Aciona o gatilho de reavaliação da **ADR-54**, que segue válida em tudo o mais.

> **Contexto**
>
> A **ADR-54** previu este dia por escrito: "se a operação pedir cadastro unitário com frequência, a saída é um formulário que monta a MESMA linha de CSV e chama o MESMO importador". A operação pediu. O atrito que a própria ADR registrou como consequência ruim — "o gestor que percebe que esqueceu de incluir a Ana precisa montar um arquivo para uma linha" — é diário no balcão da escola, e a tela só aceitava *upload*.
> O que NÃO mudou é o motivo da decisão original: uma pessoa não é uma linha em `users`. É lotação, matrícula em turma, período letivo aberto e senha provisória, nascidos na mesma transação, com o CPF decidindo se aquilo é gente nova ou transferência (**ADR-28**, **ADR-32**). Duas portas de escrita sobre esse conjunto divergem na primeira regra nova.

Decisão. Existe `POST /usuarios`, e ele **não escreve em `users`**: valida a forma do corpo com Zod, pergunta se a identidade já existe e, não existindo, serializa os campos como UMA linha da planilha (`planilhaDeUmaPessoa`, na mesma constante `COLUNAS_MODELO` que o leitor consome) e chama `importarPlanilha`. Toda a regra de domínio — CPF com dígito verificador, série e turno

normalizados, login gerado do nome, turma criada quando falta, lotação, matrícula — continua existindo em UM lugar só.

Duas diferenças em relação a importar um arquivo de uma linha, e as duas são deliberadas:

- **Criar é criar.** A importação é idempotente de propósito: reimportar a planilha do ano não pode duplicar ninguém. Num formulário chamado "Novo usuário" a mesma virtude vira armadilha — o gestor digita a Ana de novo, a linha antiga é atualizada e a tela diz "criado". Por isso o cadastro unitário **pergunta antes** (CPF na rede; matrícula e login na escola) e responde **409 sem escrever**, apontando o caminho certo: editar, transferir ou importar.
- **CPF de outra escola não revela nada.** O 409 desse caso sai sem id, sem nome e sem dizer qual unidade — é dado de outro *tenant*. A transferência continua sendo pela importação, que reconhece o CPF e move a matrícula junto.

E `admin-contrato` ganha rota separada (`POST /usuarios/admin-contrato`, `@Roles("admin-contrato")`): ele não passa pelo importador porque não cabe numa planilha DA ESCOLA — `escola_id` nulo, sem lotação, sem turma, e `PAPEIS_IMPORTAVEIS` sempre o excluiu. O login dele é único na REDE inteira, não na escola: o índice `(escola_id, username)` não cobre linha com `escola_id` nulo, e a autenticação procura o *username* na rede toda — dois donos do mesmo login viram recusa no login dos dois.

Alternativas descartadas.

- *Escrever direto em `users` a partir do formulário*: é exatamente a segunda porta que a **ADR-54** recusou. Na primeira regra nova — um papel que normaliza, um período letivo que precisa estar aberto — uma das duas aprenderia e a outra não, em silêncio.
- *Extrair do importador um método que receba objetos em vez de CSV*: mais limpo à primeira vista, e pior no que importa. A validação por campo vive no leitor da planilha; um caminho que a contorne precisa reescrevê-la em Zod, e aí são duas validações para manter — a divergência de novo, um nível abaixo. O custo aceito é a conversão JSON → CSV → pessoa, invisível em latência para uma linha.
- *Montar o CSV no NAVEGADOR e postar em `/rostering/importar`*: põe a regra de escape RFC 4180 no cliente e faz a api receber um arquivo que ela não sabe que veio de um formulário — sem chance de dar 409 no lugar de atualizar.
- *`admin-contrato` pela MESMA rota, com `papel` no corpo*: metade dos campos ficaria inerte, a autorização viraria um `if` escondido no *service* em vez do `@Roles` do método, e a matriz papel x rota perderia a linha que hoje se lê sozinha.

Consequências. *Ruim*: são duas rotas para "criar pessoa", e quem lê o apêndice C precisa entender por que a segunda existe. A conversão em três saltos é indireta o bastante para assustar quem depura pela primeira vez — daí o comentário no serializador. E o 409 tem uma janela de corrida de milissegundos: se alguém criar a mesma pessoa entre a pergunta e a escrita, o importador atualiza (corretamente) e a rota responde 409 `JA_EXISTIA` em vez de "criado", que é

a resposta honesta. *Bom*: a regra de domínio segue em um lugar, o formulário ganha de graça tudo o que a importação aprendeu, e o gestor deixa de abrir o Excel para incluir uma criança.

Gatilho de reavaliação. Se o cadastro unitário precisar de um campo que a planilha não tem — e que portanto não caiba numa coluna de `COLUNAS_MODELO` —, a delegação deixou de servir: a saída é acrescentar a coluna ao modelo (e ao leitor), não abrir o caminho direto de escrita.

18.61 ADR-60 — Microsoft Clarity carrega no hub e dentro do *bundle* da experiência

> **Contexto**
>
> A plataforma media comportamento por um caminho só: a telemetria própria da `@edu/sdk` (`POST /telemetry/batch`), que registra os *verbos* que cada experiência declara no manifesto — `attempted`, `interacted`, `progressed`, `scored`, `completed`. Ela responde "o que a pessoa fez", e responde bem, porque é desenhada com o domínio dentro. Ela não responde "o que a pessoa *tentou* fazer e não conseguiu": o clique repetido num elemento que não é botão, o *scroll* que sobe e desce numa tela que não explica, a tela abandonada antes do primeiro evento. Nenhum *verbo* nasce de uma frustração — só de uma ação que o código previu.
>
> Há uma decisão anterior em sentido contrário, e ela é explícita: a **ADR-58** tirou o SDK de erro de terceiro do *bundle* da experiência, com o argumento de que um *bundle* público manda dado "para um processador fora do contrato e do DPA da rede — para um erro que aconteceu na tela de uma criança". Esta ADR **abre exceção** a ela, para analytics de produto, e o faz de olhos abertos: o argumento da **ADR-58** continua verdadeiro, e as consequências abaixo o repetem.
>
> O pedido foi cobertura total: hub e missões. A restrição técnica que decide o desenho é que o `iframe` da experiência é **cross-origin** (o hub sai do Cloud Run, os *bundles* do *bucket*). Um *script* de gravação carregado no hub não enxerga nada dentro dele: grava um retângulo vazio exatamente onde o estudante passa a maior parte da sessão. Cobrir a missão exige carregar o *script* **dentro** do *bundle*.

Decisão. O *snippet* oficial do Microsoft Clarity carrega nas duas superfícies, a partir de **uma** fonte: `packages/sdk/src/clarity.ts`, que exporta o corpo do *snippet* e as chamadas tipadas (`identify`, `set`, `event`, `upgrade`), todas *no-op* quando `window.clarity` não existe — bloqueador de anúncio e rede de escola barrando domínio são o caso comum, não a exceção.

Superfície	Como carrega
Hub	`components/clarity-hub.tsx` monta o *snippet* com `next/script` (`afterInteractive`) no `app/layout.tsx` **raiz** — o único que cobre a *landing* pública, o login, as quatro áreas de papel, `/exp/**` e `/dev-embed`.

Identificação	`components/clarity-identidade.tsx`, montado na `casca-plataforma.tsx` — o único componente por onde `(plataforma)/**` e `/exp/**` passam os dois. Re-dispara a cada mudança de rota: o App Router navega sem recarregar a página, e o Clarity pede `identify` por **página**.
Experiência	`createExperiencia()` chama `carregarClarity` depois do *handshake*, com o id que chega no `config` do `hub:init` — campo aditivo, sem mexer no `PROTOCOL_VERSION`.

O id vai por **uma** variável, `NEXT_PUBLIC_CLARITY_PROJECT_ID`, e **ausente significa desligado** nas duas superfícies. Ela é inlinada no *bundle* do *client* durante o `next build`, então entra por `--build-arg` no `apps/hub/Dockerfile`, alimentado por `vars.CLARITY_PROJECT_ID` do *environment* no `deploy.yml`.

Duas escolhas de conteúdo fecham a decisão. O `identify` manda o **UUID opaco** do estudante e nada mais — o `friendlyName`, que levaria o nome, fica de fora, e escola e turma vão por `id`. E o `track()` da `@edu/sdk` passa a alimentar os dois destinos: a telemetria própria com a carga útil inteira, o Clarity só com `verbo:objeto`. A *resposta* do estudante não sai do contrato.

Alternativas descartadas.

- *Só nas superfícies adultas* (professor, gestor, `admin-contrato`): respeita o aviso do próprio Clarity sobre menores e não abre exceção nenhuma. Descartada porque deixa de fora o público principal — é no estudante que a dúvida de interface custa aprendizagem.
- *Só no hub, missões pela telemetria própria*: manteria terceiro fora do *bundle* público. Descartada pelo motivo técnico do contexto: o `iframe` é cross-origin, e "cobrir o hub" deixaria a maior parte da sessão do estudante como um retângulo vazio na gravação.
- *Pacote `@microsoft/clarity` (npm)*: 7,6 KB, zero dependências, com tipos. É CJS puro — sem campo `module` — e o Vite das experiências já custou tempo neste repositório com ESM / CJS. Internamente ele só injeta o mesmo *script* que está no `clarity.ts`; pagar uma dependência por isso não se justifica.
- *`VITE_CLARITY_PROJECT_ID` no bundle de cada experiência*: seria o precedente do `VITE_VIDEOS_BASE_URL`. Descartada porque `import.meta.env` dentro da `@edu/sdk` quebra o *build* do hub (que transpila o mesmo arquivo), e porque desligar passaria a exigir republicar as seis experiências.
- *Um projeto Clarity separado para o `iframe`*: dois painéis para uma sessão só. O mesmo id com a *tag* `superficie` separa o que precisa ser separado e mantém a costura pelo `identify`.

Consequências. *Ruim, e o principal*: a documentação do Clarity declara que ele **não deveria ser usado em sites ou aplicações destinadas a menores de 18 anos**, e a plataforma vai do 1º ano do Fundamental ao 3º do Médio. Isto é decisão de produto assumida, não descuido, e precisa constar de qualquer conversa de DPA com uma rede — ver `R-16` no Capítulo 19.

Ruim: a gravação de sessão sai do contrato e do projeto GCP da rede. Ao contrário do log e da métrica, que ficam no projeto do contrato, o *replay* vai para um processador de terceiro, sob os termos dele.

Ruim: as experiências disfarçadas (`exp08ef1b`, `exp04ef1b`, `exp02em1b`) carregam no `index.html` o aviso de que nada ali pode denunciar o exercício. Requisições para `clarity.ms` aparecem no DevTools, e um estudante que abra o inspetor vê um domínio que o clone imitado não teria.

Ruim: mexer na `@edu/sdk` republica as seis experiências (o filtro de mudanças do `deploy.yml` inclui o pacote), então toda alteração aqui custa um *bump* de versão em seis manifestos.

Bom: o `track()` existente virou instrumentação de funil sem tocar em nenhuma experiência, e o kill-switch é a ausência de uma variável — desligar não exige alterar código, e a máquina de quem desenvolve e a suíte e2e já nascem fora do painel.

Gatilho de reavaliação. Uma rede pública recusar o processamento em contrato, um parecer jurídico sobre o aviso de menores, ou a gravação deixar de responder pergunta que a telemetria própria não responda — nos três casos o caminho de volta é apagar uma variável de *environment*, e o de frente é um *replay* auto-hospedado dentro do projeto do contrato.

18.62 ADR-61 — Trabalho paralelo acontece em *worktree*, com a base escrita à mão

Complementada pela ADR-71 (2026-09-05), que responde o que esta não respondeu: *worktree* separa a árvore de trabalho, não o TRABALHO — e `apps/api` e `apps/hub` são comuns a todas as frentes. Lá está a regra da fatia vertical, a lista de costuras e o `pnpm worktree criar`, que transforma o passo a passo abaixo em um comando (e escreve o `origin/dev` que este parágrafo pede que se escreva à mão).

> **Contexto**
>
> O fluxo da **ADR-53** e da **ADR-56** descreve *branches*, não diretórios: pressupõe uma pessoa, uma cópia do repositório, uma tarefa por vez. Passou a haver mais de um agente trabalhando ao mesmo tempo no mesmo repositório, e alternar de *branch* na mesma cópia não serve para isso — há uma árvore de trabalho só, e ela carrega as alterações não commitadas de quem estava antes. Duas tarefas em paralelo na mesma cópia produzem um estado misturado, e é dele que sai o *commit*.

Decisão. Cada frente de trabalho paralela ganha um *worktree* em `.claude/worktrees/<nome>`, criado com a base **escrita explicitamente** — e a base é `origin/dev`.

Um clone antigo deste repositório carrega `origin/HEAD` apontando para `origin/main` (a *branch* padrão no GitHub já é `dev`; o ponteiro local não acompanha sozinho), então qualquer ferramenta que "parta da *branch* padrão" abre a *branch* sobre produção, e a **ADR-53** diz que trabalho nasce de `dev`. O erro é silencioso: a *branch* compila, os testes passam, e o PR mostra um diff cheio de *commits* que já estavam em `dev`. A raiz se conserta uma vez por clone (`git remote set-head origin --auto`); escrever a base no comando é o que evita depender do *default*, consertado ou não.

O paralelismo é de edição e teste unitário; o *runtime* é *singleton*. Portas fixas com `strictPort`, *compose* de nome fixo e um banco com um *schema*: dois *worktrees* não rodam a *stack* ao mesmo tempo, e isso é deliberado — é o que faz `pnpm dev` falhar alto em vez de servir a experiência errada. No *worktree*: editar, `typecheck`, `lint`, os testes unitários e de contrato, o *build*, e o Vite de **uma** experiência em porta alternativa, embedado pelo hub que já está no ar (`/dev-embed?src=`). Na cópia principal, uma frente por vez: a *stack* inteira, a suíte de ponta a ponta e as migrações. A *Definition of Done* exige a suíte do papel afetado para toda tela — numa frente em *worktree* isso significa trazer a *branch* para a cópia principal e rodá-la de lá, um agente por vez.

`.gitignore` já cobre o caminho (`.claude/*`), então o *worktree* não aparece no estado da cópia principal. O ciclo de *commit*, PR e promoção não muda em nada — o *worktree* é onde o trabalho acontece, não um fluxo novo. O passo a passo está em `CONTRIBUTING.md`, e desde a **ADR-71** ele é um comando: `pnpm worktree criar`.

Alternativas descartadas.

- *Alternar de branch na mesma cópia*: é o estado que produziu o problema. Serializa o trabalho e mistura alteração não commitada de tarefas diferentes.
- *Clones separados do repositório*: funciona, e custa uma cópia inteira do histórico e um `pnpm install` do zero por frente. O *worktree* compartilha o diretório de metadados e só duplica o que precisa ser duplicado.
- *Worktree fora de `.claude/`*: exigiria uma entrada nova no `.gitignore`, e um esquecimento ali significa *worktree* inteiro entrando num `add -A` — o mesmo acidente que já custou 250 MB de *cache* de *webpack* no histórico.

Consequências. *Ruim*: `node_modules` e o *client* do Prisma são por *worktree*, então cada frente paga um `pnpm install` (e um `prisma generate`) na criação. *Ruim*: `pnpm docker:dev` de dentro de um *worktree* recria os contêineres da *stack* principal — o nome do projeto do compose vem do diretório, e os dois disputam as mesmas portas. Quem precisa da *stack* inteira usa a cópia principal. *Ruim*: quatro arquivos que toda frente toca brigam no *merge* — a numeração de ADR (dois agentes já escreveram a mesma "ADR-N" ao mesmo tempo: reserve o número conferindo `origin/dev`), o `main.pdf` binário (resolve-se regerando, nunca escolhendo um lado), o `AGENTS.md` da raiz e o `pnpm-lock.yaml`. *Ruim*: cada *merge* em `dev` dispara um *deploy* que não cancela o anterior; PRs em rajada viram *deploys* em fila. *Bom*: a árvore principal fica intocada, com o que estiver em andamento nela preservado.

Gatilho de reavaliação. O trabalho paralelo deixar de acontecer, ou o repositório ganhar CI que rode as frentes em contêiner — nos dois casos o *worktree* vira conveniência local em vez de regra. E o inverso: a fila da suíte de ponta a ponta na cópia principal virar gargalo **medido** (duas frentes esperando uma pela outra, mais de uma vez) é o que justifica parametrizar porta e nome do *compose* por *worktree* e um banco por *branch* — não antes.

18.63 ADR-62 — O WAF não inspeciona o corpo de rota autenticada nem de texto livre de estudante

> **Contexto**
>
> O Cloud Armor da **ADR-57** entrou com sete regras pré-configuradas do OWASP CRS 3.3 em `sensitivity 1`, e em `dev` já saiu de *preview* — bloqueando de verdade. Nos primeiros dias, **242 requisições** `POST /telemetry/ingest` responderam 403 com `body_denied_by_security_policy`, todas do agente `APIs-Google`: é o *push* do Pub/Sub, negado pela assinatura `owasp-crs-v030301-id921150-protocolattack` — "HTTP Header Injection via payload (CR/LF detectado)".
>
> O envelope do *push* é um JSON com a mensagem em *base64*, e a 921150 casa qualquer corpo que carregue `\r\n`. O prejuízo é silencioso duas vezes: a *subscription* reentrega para sempre (não há DLQ), e a tela de resultados do professor fica vazia sem erro em lugar nenhum — a requisição morre no *load balancer*, antes da api, então não há log de aplicação a consultar.
>
> A mesma assinatura ameaça a exp02em1b: a pergunta do estudante é texto livre, e *Shift+Enter* ou colar de outra aba produzem quebra de linha legítima. O 403 do WAF não carrega cabeçalho de CORS, então o navegador reporta falha de rede: o estudante lê "A conexão falhou" e não há nada no servidor apontando para o WAF.

Decisão. Duas exceções cirúrgicas na política, e nenhuma regra desligada.

1. Uma regra `allow` em **priority 900** — antes das WAF, que começam em 1000 — casando `request.path.startsWith('/telemetry/ingest')`. Abrir o caminho não abre a rota: ela exige o *token* OIDC do Pub/Sub (`TELEMETRIA_PUSH_SA` e `TELEMETRIA_PUSH_AUDIENCE`) e responde 401 sem ele. O WAF ali era redundância que só custava dado.
2. Um `preconfigured_waf_config` na regra `protocolattack` (1006) excluindo **apenas** o `target_rule_id 921150` nos prefixos `/unoogpt/` e `/ai/` — as rotas que recebem texto livre de estudante. As outras seis assinaturas de protocolo continuam valendo nesses caminhos, e a 921150 continua valendo em todo o resto da api.

Alternativas descartadas.

- *Desligar a regra `protocolattack` inteira*: resolveria os dois sintomas com uma linha a menos, e removeria *smuggling* e *splitting* da superfície inteira da api por causa de duas rotas.
- *Voltar a política para preview*: transforma um bloqueio visível em bloqueio nenhum, e junto some a proteção contra os varredores que já batem no IP do *load balancer* todos os dias.
- *Fazer a api aceitar o push por outro caminho* (endpoint interno, fora do LB): exigiria expor a api sem o LB ou montar rede nova para um único endpoint — muito mais superfície do que a exceção.

- *Sanitizar o CR/LF antes de enviar* (no SDK e no publicador da telemetria): trata o sintoma no emissor errado. A quebra de linha na pergunta do estudante é conteúdo legítimo, e removê-la mudaria o dado que a missão mede.

Consequências ruins. `/telemetry/ingest` passa a não ter WAF nenhum — se o *guard* de autenticação daquela rota regredir, não há segunda camada. E as exclusões são por prefixo de caminho: uma rota nova sob `/ai/` herda a exclusão sem que ninguém decida isso.

Gatilho de reavaliação. A telemetria deixar de entrar por *push* no LB (um *worker* separado, como o desenho original previa) devolve o caso 1 ao WAF. E o Cloud Armor publicar exclusão por *método* além de caminho permitiria estreitar o caso 2 para os `POST` que de fato carregam texto do estudante.

18.64 ADR-63 — O CI não publica a mídia pesada, mas se recusa a publicar sem ela

> **Contexto**
>
> Mídia acima de 2 MB não entra no git nem no *bundle*: os 21 *mp4* da exp04ef1b (52 MB) vivem no *bucket* de *assets* do contrato, e o *build* recebe a base pública em `VITE_VIDEOS_BASE_URL`. O *upload* sempre foi manual, uma vez por contrato, e isso estava registrado como lacuna conhecida em `docs/INFRA.md`.
> Em 2026-09-01 a lacuna cobrou. A exp04ef1b foi publicada no contrato `entre-redes-dev` com o *bucket* de *assets* vazio: os 21 arquivos responderam 404, e como o `ShortsPlayer` do *design system* deriva `waiting` só de `waiting`/`playing` — sem escutar `error` — o evento `playing` nunca chegou e o feed ficou em "Carregando o vídeo…" para sempre. Salvar e Curtir são desabilitados enquanto o vídeo carrega (para não salvar sem assistir), então a missão ficou **impossível de concluir**. Nada falhou: nem o *deploy*, nem o log da api. Quem descobriu foi um estudante, na frente da tela.

Decisão. O CI continua **não** publicando a mídia — o *runner* faz *checkout* do git, e os arquivos não estão lá — e passa a **conferi-la**. Uma experiência com mídia externa declara um `midia-externa.json` versionado ao lado do `package.json`, com o prefixo no *bucket* e a lista de arquivos esperados. Antes de subir o *bundle* daquela experiência, o *job* `publicar-exps` lista o prefixo no *bucket* de *assets* e **falha o deploy** se faltar algum, imprimindo o que falta e o comando de *upload*.

Isso exige a var `ASSETS_BUCKET` no *environment* do GitHub, escrita por `scripts/configurar-github-ci.sh` a partir do *output* `bucket_assets` do Terraform. Sem a var, o passo apenas avisa — um *environment* ainda não reconfigurado não deve travar *deploy* por uma verificação que ele não tem como fazer.

Em paralelo, e independente disto, a experiência passou a tratar o `error` do `<video>` por conta própria: vídeo que falha sai do estado de carregamento, as ações voltam e a missão continua

concluível. As duas coisas respondem à mesma falha em camadas diferentes — uma impede publicar quebrado, a outra impede que o estudante fique preso quando algo quebrar mesmo assim.

Alternativas descartadas.

- *Versionar os mp4 no git*: 52 MB por experiência, num repositório que já teve 250 MB de *cache* de *webpack* entrando por um `add -A` distraído. Para 48 experiências, o histórico ficaria impraticável.
- *Git LFS*: resolveria o peso e acrescentaria um serviço com cota, um passo de *checkout* novo e uma forma nova de o *deploy* falhar — por mídia que muda uma vez por contrato.
- *O CI baixar a mídia de uma origem canônica e republicar*: exige escolher e operar essa origem (outro *bucket*, *release assets*), e transfere o problema em vez de resolvê-lo — alguém continua tendo que colocar o arquivo lá na primeira vez.
- *Só documentar melhor o passo manual*: já estava documentado. A lacuna estava escrita em `docs/INFRA.md` e mesmo assim o contrato subiu sem a mídia.

Consequências ruins. O *deploy* de uma experiência com mídia passa a depender de um recurso que o *job* não controla: *bucket* renomeado ou permissão de leitura retirada da *service account* do CI quebra a publicação de um *bundle* que estava correto. E a lista de arquivos no `midia-externa.json` é mantida à mão — acrescentar um vídeo sem acrescentar a linha faz a conferência passar sem conferir.

Gatilho de reavaliação. A terceira experiência com mídia externa: com uma, a lista à mão é honesta; com três, o `midia-externa.json` deve passar a ser gerado do catálogo da api, que já sabe o nome de cada arquivo.

18.65 ADR-64 — O marco da exp02em1b carrega o relógio do instante; a faixa da decisão é gravada, não recomputada

> **Contexto**
>
> O pedido do produto para o relatório do professor da Missão Tempestade foi sobre *tempo*: quanto demorou até a primeira fala, com quanto cronômetro cada fonte foi carregada, quantas vezes mudou o destino antes de confirmar, com quanto relógio confirmou (ou se estourou), e se apertou Recomeçar — e se o professor "tinha liberado". Ao levantar o que já existia, duas imprecisões de gravação apareceram. A primeira: `sessoes.minutos_gastos` só era atualizado pelo relógio dentro do turno do assistente; as rotas diretas (`alocar`, `carregarConjuntos`, `simular`) e as ferramentas chamadas de dentro do turno gravavam o marco com o minuto do **último turno** — e um turno leva de 10 a 30 s, dois a seis minutos da cidade. Pior: a hora de *saída* das equipes vinha do mesmo retrato, então quem alocava pelo mapa e enviava sem falar de novo saía no minuto de um

> turno atrás, e chegava a bairros que já não alcançaria. A segunda: a fala do estudante era gravada depois do `await` do provedor, datada pelo fim do turno, não pelo envio. Havia ainda o que o relatório precisaria e não tinha como reconstruir: o cronômetro de parede que a criança via num dado marco. `criado_em` existe, mas "+2 min" altera `duracao_seg` sem histórico e "zerar a turma" apaga o relógio — a conta feita depois mentiria em toda aula que teve extensão. E a faixa da decisão (200 corridas de `esperadoDe`) era recomputada a cada leitura: determinística hoje, mas uma mudança em `PARAMETROS` reescreveria em silêncio o relatório de toda partida antiga.

Decisão. Cinco movimentos, uma migração só de `ALTER`, sem tocar RLS nem `escola_id`:

1. `gravarMarco` recebe o `EstadoDoRelogio` **recalculado no instante** (`relogioAgora`), e grava `minuto`, `segundos_restantes` e `motivo_do_fim` dele. As rotas diretas passam a atualizar `minutos_gastos` pelo mesmo estado, e `aoConfirmar` usa esse minuto para a saída das equipes.
2. A fala do estudante leva `criadoEm: agora` — a hora em que apertou enviar.
3. Dois tipos de marco: `inicio` (entrou na Investigação, uma vez por tentativa — é de onde se mede o tempo até a primeira fala) e `recomeco`, gravado na tentativa que *fecha*, com `origem` (qual botão: a lixeira da Investigação ou o "Tentar de novo" do Resultado — a única parte que só o cliente sabe, e vai no corpo do `POST /unoogpt/reiniciar`), `fase` e `modo` do relógio da turma.
4. `exp02em1b_simulacoes.esperado`: a faixa e o percentil do resgate real, gravados no confirmar. Precedente: `app_70_sessoes.leitura_mensagens`. O `envio` passa a carregar também as justificativas como o professor lê, porque `reiniciar` zera as da sessão.
5. O resto é **derivado na leitura**, sem coluna: mudanças de destino (pares `(alfa, bravo)` consecutivos distintos antes do envio — contar linhas mentiria, porque o cartão de duas opções manda um POST por equipe e justificar passa pela mesma rota), tempo até a primeira fala (da largada e do Começar, os dois, porque antes da largada o campo está bloqueado e contar do Começar mediria o professor), % do risco coberto e posição na faixa. Mesmo princípio de `leitura.ts`: o servidor recalcula do bruto.

"Se o professor tinha liberado" recebe a resposta honesta: não existe liberação de recomeço — o que o professor controla é o relógio, e é o relógio que fica gravado ao lado do clique. Criar a liberação seria decisão de produto contra o desenho registrado ("o professor controla quando dá para falar, não o que a criança decide") e fica para uma ADR própria, se o dado gravado mostrar que faz falta.

Alternativas descartadas.

- *Telemetria como fonte*: os eventos já existem (`interacted:alocacao`, `attempted:recomeco`). Mas o canal é preenchido pelo cliente, assíncrono (Pub/Sub, desligado em `dev`), perde lote no fechar da aba, e nenhum código do repositório lê `telemetria_eventos` — além da regra escrita de que o que o professor lê nunca vem do canal que o cliente preenche.

- *Reconstruir o cronômetro de `criado_em` + relógio da turma*: falha em toda aula com "+2 min" (a duração muda sem histórico) e em toda turma zerada (o relógio some). Gravar no instante custa duas colunas e não mente.
- *Sessão por tentativa* (uma linha de `exp02em1b_sessoes` por rodada, preservando justificativas e conjuntos): resolveria a perda no `reiniciar` de forma geral, mas mudaria a chave primária de quatro tabelas e toda consulta do módulo. A cópia das justificativas no marco de `envio` resolve o que o relatório precisa, por uma fração do custo.
- *Persistir a contagem de mudanças e a cobertura*: contrariaria o princípio do módulo (o servidor recalcula do bruto, e a versão da leitura sobe quando a hipótese muda). Só se grava o que **não** dá para recalcular — o cronômetro e a faixa.

Consequências. *Ruim*: a correção do minuto muda o desfecho de quem confirma sem turno novo — é correção, mas partidas antigas e novas não são comparáveis nesse detalhe, e o produto foi avisado antes de qualquer teste controlado. *Ruim*: marcos e simulações anteriores à migração têm as colunas novas nulas; a tela diz "sem registro" e a leitura recomputa a faixa. *Ruim*: o cliente da exp02em1b sobe de versão pela `origem` no corpo — cliente antigo continua aceito (corpo vazio é válido). *Bom*: nenhum evento de telemetria novo, nenhuma tabela nova, RLS intocada; o relatório do professor sai inteiro de `GET /unoogpt/professor/estudantes/:id`, e o gestor continua sem rota que aceite `estudanteId`.

Gatilho de reavaliação. O produto pedir a comparação de *justificativas* ou *conjuntos* entre tentativas além do que o marco de envio guarda — aí a sessão por tentativa volta à mesa. Ou o dado de `recomeco` mostrar que crianças recomeçam depois de o professor encerrar, em número que justifique a liberação explícita.

18.66 ADR-65 — O guia da aula é dado renderizado por um componente único, com blocos vivos para o que tem fonte de verdade em outro lugar

Contexto

Só o Instanooria tinha guia da aula. Ele era uma página (`.../[ano]/[unidade]/guia/page.tsx`), 447 linhas de JSX com `notFound()` para qualquer outro `expId`: conteúdo, ordem e marcação na mesma camada. As outras quatro missões publicadas — UnooTube, Missão Tempestade, O Eco no Grupo e a Vila do Lume — chegavam ao professor com a ficha do catálogo (competência, atos, preparo) e nada sobre como conduzir a aula: nenhuma pergunta de debate, nenhuma mediação, nenhuma rubrica. A Vila tinha um guia *do debate*, excelente e parcial: cobria o pós-jogo e não o preparo nem o fechamento.

E o problema não era só de cobertura. Duas partes do guia do Instanooria já vinham de fonte viva por decisão anterior — a tabela dos momentos de `CONTROLE_DO_PROFESSOR` e

> as telas da revelação, então servidas por um pacote de *workspace* (desfeito em 2026-09-09; hoje são duas cópias gêmeas) — porque a versão escrita à mão tinha divergido: o guia falava em "os seis botões" e o painel tinha sete. A Vila serve os cartões da api por turma pela mesma razão. Qualquer solução para as outras missões precisava carregar esses casos junto.

Decisão. O guia vira **dado** (`apps/hub/lib/entre-redes/guias/<exp>.ts`, tipo `GuiaDaAula`) renderizado por um **componente único** (`apps/hub/components/guia/`), com a mesma ordem de seções em toda missão: objetivos e BNCC Computação, antes da aula (técnico, pedagógico, o que não dizer), roteiro cronometrado com a coluna "o que você faz e fala", banco de perguntas em quatro níveis, folha de observação, mediação, depois da aula (síntese, registro, rubrica, desdobramentos), cuidados, adaptações e dúvidas frequentes.

O que tem fonte de verdade em outro lugar entra por **bloco vivo**: um `ReactNode` que a página da missão ancora ao fim de uma seção. É por aí que entram os momentos e a revelação do Instanooria, os cartões da Vila (por turma) e a tabela dos oito vieses da Missão Tempestade — esta última por uma rota nova, `GET /unoogpt/professor/fontes`, com o viés **derivado** da instrução do agente em vez de escrito duas vezes.

Uma folha de impressão acompanha o renderizador: o professor conduz a aula com o guia na mão enquanto o computador projeta a tela da turma.

Alternativas descartadas.

- *Uma página por missão, como a do Instanooria*: cinco arquivos de cerca de 450 linhas, sem nada que os obrigue a ter as mesmas seções. A seção que alguém esquecesse de copiar não existiria para aquela turma, sem sinal nenhum — e a segunda missão que o professor rodasse teria a informação em outro lugar da página.
- *Guia em Markdown ou MDX*: resolveria a escrita e perderia as duas coisas que importam — o tipo que cobra seção faltando e os blocos vivos, que precisam de componente.
- *Guia no catálogo* (`lib/entre-redes/catalogo.ts`): o catálogo é a grade, lido por meia dúzia de telas; o guia é material de aula, pesado, e só a rota do guia precisa dele.
- *Copiar a tabela dos vieses para o hub*, evitando a rota nova: era a opção mais barata e é exatamente a que a doutrina do repositório proíbe. Recalibrado o cenário, o professor contaria à turma um viés que o jogo já não produz — o mesmo erro que os cartões da Vila vindos da api existem para evitar.
- *Guia por turma* (com `?turmaId=`): só a Vila precisa disso, e ela já resolve com seletor próprio dentro da revelação. Genéricos, todos os guias passariam a exigir turma para abrir — e o professor lê o guia na véspera, quando ainda não escolheu a sala.

Consequências. *Bom*: as cinco missões publicadas passam a ter guia com a mesma estrutura; `guias/index.test.ts` recusa missão publicada sem guia, com uma lista de exceções que exige razão escrita. *Bom*: `lib/rotas-da-missao.ts`, que já era o único tradutor `expId` → rota, passa a responder também se a missão tem guia — então a sala de aula mostra o *link* em toda

turma × toda missão sem inventar URL. *Ruim*: o conteúdo pedagógico das quatro missões novas é **autoral**, derivado do código e da *copy* congelada, e não passou por playtest em sala — está escrito para ser corrigido por quem der a primeira aula. *Ruim*: o alinhamento com a BNCC Computação é proposta da plataforma, não classificação oficial, e a tela precisa dizer isso. *Ruim*: uma rota a mais na api para uma tela de leitura, com a documentação e o teste que ela obriga.

Gatilho de reavaliação. Se o guia passar a variar por turma em mais de uma missão — aí o parâmetro deixa de ser exceção da Vila e vira propriedade do renderizador. Ou se a rede pedir o guia como PDF/impresso oficial, com marca e paginação de documento: aí a folha de impressão deixa de bastar e o caminho é gerar o documento, não estilizar a tela.

18.67 ADR-66 — A exp07ef1b escreve com IA na tela do estudante, e o gabarito continua fora do modelo

> **Contexto**
>
> A missão `exp07ef1b` chegou dos gates de produto com uma decisão explícita: "NÃO HÁ USO DE IA GENERATIVA nesta missão. Todas as mensagens de chat, reações da cascata, áudios e falas de personagens são 100% roteirizados e deterministicamente controlados." As cinco bancas aprovaram sobre essa premissa, e a banca de dados chegou a registrar como ponto forte a ausência de campo de texto livre de menor (`DAD-08`).
> O produto reabriu a decisão: quer interação por IA na missão. Isso não é um detalhe de implementação — muda o que o instrumento mede, o que a criança escreve e quanto a aula custa.

Decisão. A IA entra em **três superfícies**, e em nenhuma delas ela decide resultado.

1. *As reações do bairro* (Etapa 4). O desfecho — se o grupo se acalma, entra em pânico ou azeda — sai de `Opcao.desfecho` no catálogo do servidor, antes de qualquer modelo ser chamado. O que a IA escreve é **como isso soa**, reagindo ao que a criança de fato postou.
2. *Amara no privado*, e só DEPOIS da decisão no grupo. O silêncio dela durante a investigação é não-negociável de ofício: se ela apontasse a mensagem falsa, o reconhecimento de padrão que a missão mede deixaria de acontecer. O portão é do servidor (`amara-em-silencio`), não uma condição de tela.
3. *A devolutiva do "Meu percurso"*, um parágrafo sobre as escolhas. Não é nota: o `score` sai de `regras/pontuacao.ts` sobre o que aconteceu no grupo, e o formulário reflexivo não entra nele.

As três falham para o lado seguro. Sem saldo de IA, sem provider ou com a saída reprovada pelo sanitizador, a missão fecha inteira: as reações caem no roteiro autoral do catálogo, Amara "fica sem sinal" e a devolutiva não aparece. Nenhuma aula depende de o Vertex estar de pé

— e é isso que `tests/o-eco-no-grupo-postgres.spec.ts` exercita, rodando a suíte com o `MockProvider`, que falha fechado.

O texto livre de menor volta, com cinco travas. A conversa com Amara é o único campo aberto da missão: 600 caracteres, teto de 3 turnos, gravado em `exp07ef1b_mensagens` sob RLS, fora de log e fora da telemetria (só a contagem de turnos sai), e sem nenhuma rota de gestor que o alcance. É a mesma postura do `exp02em1b_mensagens`, com um teto menor.

Alternativas descartadas.

- *Manter tudo roteirizado*: é o que as bancas aprovaram, e continua sendo o piso do produto (o roteiro existe e é exercido). O que ele não dá é a reação ao que *aquela* criança escreveu — e o pedido do produto era exatamente esse.
- *IA decidindo o desfecho*: um modelo com um dia ruim mudaria a lição da aula. O desfecho é gabarito, e gabarito é servidor (regra 3).
- *Amara conversando durante a investigação*: destruiria o não-negociável do silêncio cognitivo. A conversa foi movida para depois da decisão, onde consolida em vez de entregar.
- *Prompts no bundle*: proibido pela regra 2, e pior que publicar o gabarito — o prompt descreve o mecanismo inteiro da missão em português.

Consequências. *Ruim*: o pacote de insumos aprovado passa a divergir do que está em produção em dois pontos (IA em runtime e texto livre), e as bancas C e D precisam revisitar `PED-14` e `DAD-08`. *Ruim*: a missão passa a consumir quota de IA de estudante — estimados 30 mil tokens por criança contra os 300 mil diários. *Ruim*: um sanitizador a mais para manter (`regras/sanitizador.ts`), e ele já barrou uma saída real: o mock ecoava o próprio prompt para dentro da tela. *Bom*: o desfecho, a pontuação e a fixação pelo administrador continuam determinísticos e testáveis sem rede.

Gatilho de reavaliação. Um playtest mostrar que a reação escrita ao vivo confunde mais que o roteiro autoral — ou que crianças usam a conversa com Amara para escrever o que não deveriam. Nos dois casos, o roteiro do catálogo já está no código e volta a ser o caminho único trocando uma linha.

18.68 ADR-67 — A mídia da exp07ef1b é gerada por IA, preparada pelo professor e cacheada por contrato

> **Contexto**
>
> O brief de assets da exp07ef1b pede oito peças de conteúdo autoral: duas gravações de voz, um podcast masterizado de 3m30 com dois locutores e três fotografias estilizadas de um portal de notícias fictício. Nenhuma delas existe, e nenhuma sai de código — são voz humana e design gráfico.

> Sem elas, três telas da missão ficam mudas: o áudio de Amara, o áudio da moradora que espalha o boato e o episódio que destrava a Aula 2.

Decisão. A mídia é **gerada pelo Vertex**, publicada no bucket de assets e servida como arquivo estático. Cinco movimentos:

1. `AiProvider` ganha `generateSpeech` (MP3, via Cloud Text-to-Speech) e `generateImage` ganha proporção opcional. Os dois são aditivos: nada fora de `src/ai` conhece o vendor, e `AiTimbre` é neutro (`jovem-feminino`, `idoso-feminino`) para o roteiro descrever *quem* fala, não o nome de catálogo de uma voz do Google.
2. Quem prepara é o **professor**, no painel da turma. A quota diária dele é 2 milhões de tokens contra 300 mil do estudante, gerar leva dezenas de segundos, e o preparo é o momento em que ele confere se a missão está pronta.
3. A mídia é do **contrato**, não da escola: `experiencias/{expId}/{arquivo}`, sem `escola_id` no caminho. Toda a rede ouve a mesma Dona Lúcia — duas turmas discutindo vozes diferentes do mesmo boato não estariam falando da mesma coisa.
4. **O print viral é um recorte da foto da reportagem**, não uma segunda geração. Mesma cena, mesma luz, mesmo instante: o corte visual espelha o corte do texto que o estudante vai descobrir. Duas gerações independentes dariam duas fotos parecidas, e o par casado que o brief exige seria mentira.
5. **A página do portal é HTML, não imagem.** Modelo de imagem erra texto longo, e o parágrafo suprimido precisa ser legível, selecionável por leitor de tela e com contraste medido. A IA faz a foto; o layout é código.

Alternativas descartadas.

- *Gerar sob demanda para cada estudante*: cada criança ouviria uma Dona Lúcia diferente, o `intervention_latency` mediria a latência do modelo, e o custo se multiplicaria por dez mil.
- *Script one-shot com os arquivos no git*: funciona, mas cada contrato novo dependeria de alguém rodar o script — e o precedente é caro (a exp04ef1b subiu com o bucket vazio e a missão ficou impossível de concluir, sem nada falhar no deploy nem no log). O script existe (`apps/api/scripts/smoke-midia-exp07ef1b.ts`), como ferramenta de conferência.
- *Print inteiro gerado por IA*: manchete e parágrafos distorcidos, o trecho omitido ilegível, e o texto invisível para leitor de tela.
- *WAV ou PCM*: não há ffmpeg neste repositório, e um WAV de 3m30 passaria de 10 MB. O `audioEncoding: MP3` resolve na origem.

Consequências. *Ruim*: a duração do podcast não bate com o brief — o roteiro aprovado rende 1 min 58 s, não 3 min 30 s, e chegar lá exigiria uma cadência de 11 caracteres por segundo, que não é ritmo de podcast. O cronograma da Aula 2 ganha folga; a copy é que teria de mudar. *Ruim*: com ADC de usuário a Cloud TTS exige `x-goog-user-project`, e o `aiplatform` não — no mesmo preparo, imagem funciona e voz responde 403 com uma mensagem que não fala de voz.

Ruim: uma dependência a mais para habilitar por projeto (`texttospeech.googleapis.com`). *Bom*: sem mídia preparada a missão roda igual — o áudio cai na transcrição, que já está sempre na tela por acessibilidade, e a foto na ilustração procedural.

Gatilho de reavaliação. A rede contratar locução humana para a trilha, ou um playtest mostrar que a voz sintética atrapalha a compreensão de quem tem 12 anos. Nos dois casos, o caminho é substituir os arquivos no bucket — o código da tela não muda.

18.69 ADR-68 — O clone de WhatsApp da exp07ef1b é escuro, e as quatro peças que faltam são compensação marcada

> **Contexto**
>
> Os wireframes e o sheet de copy da exp07ef1b descrevem a missão em **modo claro**: balões verdes e brancos sobre fundo bege, o WhatsApp que a maioria das pessoas usa. A seção `whatsappmobile` do `@unooria/design-system` 0.1.17 publica **exclusivamente o tema escuro** — `WaTheme = 'dark'`, e o CSS traz `:root, [data-wa-theme=dark]` e nada mais.
> Além do tema, o pacote não publica quatro componentes que a missão pede, e três deles sustentam não-negociáveis de ofício: o selo de encaminhada, o banner do alfinete, o balão de voz com transcrição e o menu de resposta citada.

Decisão. O clone é escuro, fiel ao que o pacote publica. Derivar uma paleta clara aqui é proibido pela mesma regra do `unitube`: a fidelidade é ao que o design system entrega, e uma paleta inventada seria um WhatsApp que não existe em lugar nenhum. O contraste do escuro (`#D3DAE0` sobre `#101D25`, cerca de 11:1) passa o AA que o `RNF-ACES-001` exige com folga.

Das quatro peças, **três saem de composição** com o que o pacote já tem: o selo é `Icon name="forward"` com `Badge`; o banner do alfinete usa a superfície de appbar; o menu é `Radio` com o texto no `label`. A quarta — o balão de voz — é a única compensação de verdade, marcada `(b) COMPENSAÇÃO` no CSS, com pedido registrado ao `storybook-unooria`.

Duas notas de mecânica que custaram tempo: o `PhoneFrame` do pacote **não entra** na experiência (quem desenha o aparelho é o hub, pelo `viewport` do manifesto — empilhar os dois daria um celular dentro de outro); e o `ListItem` **não serve** para as três opções de resposta, porque o título dele tem `white-space: nowrap` — truncar o texto faria a criança escolher sem ler, que é exatamente o comportamento que a missão existe para desfazer.

Alternativas descartadas.

- *Esperar o tema claro no `storybook-unooria`*: bloquearia o front inteiro da missão por uma diferença que o RNF de acessibilidade não cobra — o escuro passa o AA melhor que o claro.
- *Derivar a paleta clara localmente*: proibido, e é a porta para cada experiência ter a sua versão do mesmo clone.

- *Publicar os quatro componentes antes de começar*: bloquearia as etapas 2 a 4 da missão por trabalho em outro repositório, e três deles nem seriam componentes novos — são composições do que já existe.
- *Desfazer o `nowrap` do `ListItem` com dois seletores*: é o caminho que a doutrina permite, mas usar o componente errado e depois desfazer o CSS dele é pior que usar o certo.

Consequências. *Ruim*: o pacote de design aprovado descreve telas claras, e a missão em produção é escura — os wireframes precisam de uma nota, e o QA de design revisitaria `DES-06`. *Ruim*: uma compensação local aberta, que sai quando o pacote publicar o balão de voz. *Bom*: a ponte `@edu/ui/ds-whatsapp` existe e a seção deixa de ser "sem consumidor"; a próxima experiência de mensageria começa do que esta deixou pronto.

Gatilho de reavaliação. O `storybook-unooria` publicar o tema claro da seção `whatsappmobile` — aí a decisão volta à mesa, e a troca é de uma folha de estilo, não de código de tela.

18.70 ADR-69 — A missão vive DENTRO da turma: a sala de aula por liberação

> **Contexto**
>
> O enturmamento estava modelado desde a **ADR-28** — `periodos_letivos`, `turmas` com série, `matriculas` datadas com papel — e o escopo do professor já era a matrícula. Faltava a camada de cima: **nada ligava `turmas` a `experiencias`**.
> O efeito era visível nas três telas que o produto tem:
>
> - o hub do estudante listava as missões de uma **constante compilada no *bundle*** (`apps/hub/lib/missoes-do-estudante.ts`), igual para toda criança de toda escola — sem olhar série, turma nem escola. E `/exp/:expId` não conferia nada: qualquer estudante logado abria qualquer missão digitando a URL;
> - o professor navegava por ANO. Existe uma página por "ano/unidade", então as duas turmas da 2ª série do EM dividiam a mesma ficha, o mesmo guia e a mesma tela de resultados — a turma só reaparecia num `<select>` no fim;
> - "liberar" existia só DENTRO de três das cinco experiências, com um mecanismo diferente em cada uma: o `status` da atividade da exp08ef1b, o relógio da turma da exp02em1b (**ADR-25**), os *gates* da exp07ef1b. Todos respondem "a aula começou"; nenhum responde "esta missão pertence a esta sala".
>
> É a camada que todo LMS tem entre a turma e o conteúdo — *resources* ligados a uma *class* no OneRoster, *deep linking* num contexto no LTI. A **ADR-28** descartou o LTI porque esta plataforma não vive dentro de um LMS; a consequência que faltava tirar é a inversa: então ela precisa ter a própria camada de atribuição, porque é ela quem faz esse papel.

DECISÃO DE PROJETO

Uma tabela de plataforma, `liberacoes`: turma × experiência × estado. Sem linha, a missão não abre. `liberada` abre; `encerrada` fecha a porta e mantém tudo o que a criança fez.

Quem libera é **quem tem matrícula de professor na turma** — gestor lê a sala e não decide a aula. Não há prefixo `exp_NN_` porque a tabela serve a todas as experiências: ela é contrato entre a plataforma e cada uma.

A porta é do SERVIDOR, em `comum/liberacao.ts`: `exigirMissaoLiberada` é chamada na entrada de estudante de cada experiência — `POST /runs` e `GET /runs/boot`, e o funil de sessão do UnooTube, do unooGPT, da Vila e do Eco. Esconder o *card* no hub é UX; a regra que vale é esta, pela mesma razão da regra inviolável nº 3 (correção no servidor).

Ela é PRÉ-CONDIÇÃO dos relógios e *gates*, não substituta. Eles continuam dizendo "a aula começou" e continuam morando na experiência. Sem liberação, a pergunta do relógio nem chega a ser feita.

Três escolhas de tela, e nenhuma é neutra.

- **Missão bloqueada aparece com CADEADO**, não some. Sumir era mais simples e foi recusado: a grade do ano é a promessa do que vem depois, e uma lista que muda de tamanho sozinha não deixa a criança entender que a próxima unidade existe. `bloqueada` não é estado do banco — é a ausência de linha.
- **Missão encerrada continua VISÍVEL**, com o rótulo. O que a criança fez não pode evaporar da tela quando o professor fecha a aula.
- **Professor com uma turma só entra nela clicando**, como quem tem quatro. Havia a opção de redirecionar direto, e ela foi recusada: dois fluxos diferentes para o mesmo papel é o tipo de atalho que faz a tela de lista parecer quebrada para metade da rede.

Alternativas descartadas.

- *Filtrar só por série* (`experiencias.series`, que já existe e ninguém lia): resolve a criança do 4º ano ver a missão do 8º, e não resolve nada do resto — as duas turmas do mesmo ano continuariam idênticas, e o professor continuaria sem dizer quando a missão entra.
- *Continuar liberando dentro de cada experiência*: é o que existe, e o custo está medido — três mecanismos, e a criança só descobre que a missão não abriu DEPOIS de entrar nela, num 409 ou numa tela de espera.
- *Esconder no hub e não checar na API*: o *bundle* é público e `/exp/:expId` é endereço adivinhável.
- *Guardar "bloqueada" como linha*: obrigaria a criar uma por turma × experiência para o mundo nascer fechado. Ele nasce fechado de graça.
- *FK de `exp_id` para `experiencias`*: aquele catálogo é escrito pelo CI ao publicar, e em desenvolvimento está vazio — a FK faria liberar depender de um *deploy* que não aconteceu. É a mesma escolha de `progresso.exp_id`.

Consequências, inclusive as ruins.

- **Estudante sem turma não vê missão nenhuma**, e isto é mudança de comportamento. A exp02em1b tinha decisão registrada de que a criança solta via o relógio "aguardando" em vez de um cronômetro só dela; agora ela nem entra. Sem sala não há missão — e quem explica isso é a tela do hub, porque um erro dentro do aparelho não teria como dizer "peça para a coordenação te matricular" sem sair do disfarce.
- **O professor ganhou um passo antes da aula.** Missão que ninguém liberou não abre, e a sala precisa dizer isso com todas as letras — é o risco novo do capítulo 19.
- **O *seed* passou a escrever `experiencias`.** Em produção quem escreve é o CI ao publicar; sem catálogo não há grade, e o professor abriria a sala sem nada para liberar.
- A checagem custa três consultas indexadas na entrada de cada experiência. Ficam na CRIAÇÃO da sessão, não em toda rota: quem já entrou fica, e encerrar fecha a entrada em vez de expulsar a criança no meio da aula.

Gatilho de reavaliação. Uma rede querer liberar em lote pela coordenação (hoje gestor não libera: é uma linha no `@Roles` e uma frase na matriz do capítulo 7), ou pedir **janela de datas** — "abre segunda, fecha sexta" —, que hoje é um clique manual e viraria duas colunas e uma rotina que as cumpra. Enquanto não houver quem cumpra, a coluna não nasce: é a armadilha do `expires_at`.

18.71 ADR-70 — As páginas legais descrevem o sistema que existe, e a rede é a controladora

> **Contexto**
>
> O rodapé da landing oferecia "Privacidade" e "Termos" com `href="#"` — dois links que não levavam a lugar nenhum. Escrever as duas páginas expôs uma pergunta que não é de redação: **o que elas podem afirmar?**
> Uma auditoria do repositório inventariou o tratamento de dados a partir do código, não do desejo. Ela encontrou vinte afirmações que uma política de privacidade padrão faria e que **seriam falsas aqui**: não existe entidade, coluna ou tela de responsável legal; não existe rota de exclusão de pessoa (desligar alguém é `ativo = false`); não existe **nenhuma** rotina de retenção no repositório; a stack está em `us-east1` e a IA no endpoint `global` do Vertex; `users.cpf` é `TEXT` em claro (ADR-33); a telemetria é pseudonimizada, não anônima; e não há log de acesso individual — ninguém sabe hoje qual gestor abriu o CPF de qual criança.
> A pesquisa jurídica acrescentou o ponto que decide a forma do documento: o cliente é uma **rede pública de ensino**, e o Guia do Poder Público da ANPD usa como exemplo literal a matrícula de estudante para concluir que o consentimento obtido em relação compulsória é **nulo** (Enunciado CD/ANPD nº 1/2023).

Decisão. As páginas descrevem o sistema que existe. Toda frase tem lastro em tabela, rota ou envio verificado, e o que hoje não existe é dito como não existindo — inclusive quando isso é desconfortável. Onde faltava número (prazo de retenção), o texto atribui a definição à rede em vez de inventar meses; onde faltava processo (revisão humana de decisão automatizada), o texto não promete o processo.

Não há pedido de consentimento. A base é o art. 7º, III da LGPD — política pública —, e a página tem uma seção inteira explicando *por que* não existe botão de aceitar, para que a ausência não seja lida como descuido. O único consentimento legítimo do produto é o do professor no DASS-21 (art. 11, I), e ele está isolado ali. **A rede é a controladora; a Unooria é operadora** — com uma exceção declarada: nos registros de acesso do art. 15 do Marco Civil, a controladora é a Unooria.

Os Termos não elegem foro (Lei 14.133/2021, art. 92, parágrafo 1º manda o foro da sede da Administração no contrato, e cláusula de foro contra titular menor é abusiva) **e não limitam responsabilidade de forma ampla** (art. 51 do CDC).

Duas decisões de forma, que são de conteúdo disfarçado. **Zero `<table>`**: texto jurídico numa tabela de quatro colunas produz uma coluna de três palavras e outra de quarenta, célula altíssima e rolagem lateral no celular — que é onde a família abre o link. O mesmo dado vira `legal-card`, que cresce com o conteúdo e reflui sozinho. E **nenhum `[A DEFINIR]` espalhado pelo texto**: o que depende do contrato mora num bloco `IDENTIFICACAO` no topo de cada arquivo, e o que ainda não foi decidido tem uma seção final própria. Razão social e CNPJ ficam **vazios** até virem do cadastro — um CNPJ de exemplo numa página pública é declaração falsa que passa na revisão justamente por parecer preenchida.

Alternativas descartadas.

- *Adaptar um modelo de política de mercado*: todo modelo pressupõe consentimento do titular e autocadastro. Aqui os dois são falsos, e o consentimento seria nulo — o modelo erra justamente no que é específico.
- *Prometer o estado desejado e implementar depois*: promessa vincula (LGPD art. 6º, X). Seria a mesma armadilha do `expires_at` que saiu do schema e ficou no registro: a regra sai do código, o texto fica, e quem lê acredita nele.
- *Publicar só a Política e deixar os Termos para depois*: quem lê uma procura a outra, e metade dos pontos (simulações, IA, conduta) não cabe na Política.
- *Marcar `[A DEFINIR]` em cada lacuna do texto*: foi tentado e rejeitado — o documento fica ilegível e a lacuna deixa de ser vista de tanto se repetir.
- *Uma casca visual própria para as páginas legais*: elas são a mesma superfície pública da landing, alcançadas pelo rodapé dela. Casca própria faria a página parecer de outro produto na hora em que a confiança está sendo decidida.

Consequências. *Ruim*: as páginas declaram em público limitações reais do produto (sem exclusão por titular, sem retenção, sem log de acesso, processamento de IA fora do Brasil) — é material que uma rede pode usar em negociação, e o incentivo a fechar essas lacunas passa

a ser externo. *Bom, por tabela*: escrever a página encontrou duas contradições que o texto não podia encobrir. A do `docs/INFRA.md` **foi corrigida aqui** — ele afirmava que o Vertex fica em `southamerica-east1` e, mais abaixo, que a localização é `global`; o código concorda com a segunda em quatro lugares (`config/env.ts`, os dois *compose* e o `LOCALIZACAO_PADRAO` do provider), e o `3.6-flash` responde 404 na região brasileira. Era a linha que uma política copiaria para afirmar que os dados ficam no Brasil. *Ruim*: a outra é de produto e segue aberta — a app-91 exibe "Só você lê o que escreve aqui" sob a caixa cujo conteúdo vai ao Vertex para gerar o acolhimento. *Bom*: o inventário existe, e com ele o RIPD que a Resolução CD/ANPD nº 2/2022 torna exigível deixa de ser trabalho do zero.

Gatilho de reavaliação. Qualquer uma das lacunas ser fechada no código — rotina de retenção, rota de exclusão por titular, log de acesso individual, região brasileira para o Vertex —, o que muda o que a página pode afirmar; ou a primeira rede assinar, quando a controladora passa a ter nome e CNPJ.

18.72 ADR-71 — O trabalho paralelo se reparte por fatia vertical, e o runtime continua singleton

> **Contexto**
>
> A **ADR-61** resolveu a *árvore de trabalho*: cada frente paralela ganha um `git worktree`, e o `git status` de uma não contamina o commit da outra. Ela não resolveu a pergunta seguinte, que aparece no minuto em que existe mais de um terminal aberto: **`apps/api` e `apps/hub` são comuns às 48 experiências**. Worktree separa arquivos não commitados; não separa duas frentes que precisam editar o mesmo arquivo.
> A divisão que o instinto sugere é por camada — uma frente no *backend*, outra no *frontend*. Ela é a pior possível aqui: as duas passam a morar nos mesmos arquivos por motivos diferentes, e o conflito não é textual (duas linhas vizinhas), é semântico (dois desenhos do mesmo *service*). Nenhuma ferramenta de merge ajuda com isso.
> O segundo eixo é o tempo. O runtime deste repositório é **singleton** por decisão: portas com `strictPort`, um `docker-compose` de nome fixo (`unooria-plataforma`), um Postgres com um schema. Isso não é descuido — é o que faz `pnpm dev` falhar alto em vez de servir a experiência errada dentro do *iframe*, que é uma falha silenciosa e cara de diagnosticar.

Decisão. Reparta por fatia vertical. Cada frente leva o módulo da api, a rota do hub e a experiência *juntos* — `apps/api/src/unootube/` com a rota do hub e `apps/experiencias/exp04ef1b/`, e não "a api" de um lado e "o hub" do outro. Com isso api e hub deixam de ser recurso disputado: a fatia de cada frente dentro deles é uma **pasta própria**. Não é sorte — é o desenho de um módulo por domínio (`catalogo.ts` + `regras.ts` + service + controllers por papel) sendo usado para o que ele serve. **Duas frentes no mesmo módulo não se paralelizam: serializam.**

O que sobra compartilhado é linha de REGISTRO, não lógica, e vai em UM commit no fim da frente, depois de `git merge origin/dev`: um `import` e uma linha no array de `app.module.ts`; `model` novo **no fim** do `schema.prisma` (duas inserções no fim conflitam limpo, duas no meio não); a migração; a entrada em `middleware.ts` / `experiencias-dev.ts`. A porta de dev se reserva como o número da ADR: conferindo `origin/dev`.

O runtime continua singleton, e o trabalho se divide em duas fases. A *paralela*, N frentes: editar, `typecheck lint test`, `build --filter`, e o Vite de UMA experiência em porta `52xx` embedada por `/dev-embed?src=`. A *serial*, uma por vez na cópia principal: `pnpm dev`/`docker:dev`, migração, e a suíte de ponta a ponta.

E a fatia é BARREIRA, não conselho. `scripts/worktree.mjs` (`pnpm worktree criar/listar/remover`) faz o caminho inteiro — worktree sobre `origin/dev`, os seis `.env` copiados (todos ignorados pelo git, nenhum viaja), `pnpm install` e `prisma generate` (`node_modules` e o client do Prisma são por worktree), porta derivada de um *hash* do slug — e **resolve o `--fatia` contra a árvore** (`api/unootube` vira `apps/api/src/unootube`; `hub/professor` acha o grupo de rota do App Router) antes de gravar a lista em `<git-dir>/fatia`. Num worktree esse caminho é `.git/worktrees/<slug>/`: privado daquela frente, fora do versionamento, e portanto sem conflito possível entre frentes.

O `.githooks/pre-commit` lê essa lista e **recusa** o que estiver fora dela, com o escape nomeado `PERMITIR_FORA_DA_FATIA=1` — o mesmo desenho dos outros hooks. Sem o arquivo, o hook é inerte, e a cópia principal segue livre. A costura passa, mas passa *avisando*: o lugar dela é um commit separado no fim. O efeito colateral que vale mais que o hook: a lista de costuras deixa de existir só em prosa e passa a existir como código, num lugar onde ficar desatualizada quebra alguém em vez de enganar em silêncio.

Alternativas descartadas.

- *Repartir por camada (uma frente na api, outra no hub)*: é a origem do problema, não a solução. Some com a fronteira que os módulos por domínio construíram e transforma todo merge em releitura de desenho.
- *Stack por worktree — offset de portas, `name` do compose por slot, um banco por frente*: tornaria a fase serial paralela também, e foi a alternativa mais séria. Descartada por três razões. O `strictPort` existe para **falhar alto**: com portas móveis, o *iframe* volta a poder apontar para a experiência errada em silêncio, que é exatamente o bug que a porta fixa previne. N bancos divergem — a suíte e2e é a etapa que mais depende de um estado conhecido, e depurar "passou no meu banco" custa mais que a fila. E o custo de manutenção (compose parametrizado, envs por slot, `experiencias-dev.ts` dinâmico, a suíte e2e ciente do slot) cai sobre **uma pessoa**, para destravar uma fila que hoje tem dois ou três itens.
- *Banco por frente, mantendo as portas fixas*: metade do custo acima e menos da metade do ganho — continua sem poder rodar duas stacks, que é o que a fila realmente pede.
- *Deixar a mecânica só como prosa no `CONTRIBUTING.md`*: era o estado anterior. O passo que morde é o **primeiro**, e é o que a prosa não consegue impor — `git worktree add` sem `origin/dev` escrito abre a branch sobre produção em silêncio (**ADR-61**).

- *Barrar na ESCRITA, pelas permissões do agente* (um `.claude/settings.local.json` por worktree negando `Edit`/`Write` fora da fatia): pegaria o erro antes do trabalho ser feito, e não no commit. Descartada por cobertura e por forma. Cobre **um** agente — outra ferramenta, ou a edição à mão, passa direto —, enquanto o hook cobre qualquer coisa que produza um commit. E *glob* não expressa "tudo menos X": negar o vizinho exigiria listar os trinta e um módulos da api um a um, regerando a lista toda vez que nasce um módulo. Fica como camada extra possível no dia em que o custo do trabalho jogado fora superar o do commit recusado.
- *Registro de portas num arquivo versionado*: estado a mais para manter sincronizado, e conflito num arquivo que TODA frente tocaria. O *hash* do slug dá uma porta estável sem guardar nada — e estável importa: o `src=` do `/dev-embed` fica na barra do navegador de quem está trabalhando, e não pode mudar porque um vizinho abriu uma frente.

Consequências. *Ruim*: o teto útil é de duas a três frentes — toda frente que mexe em tela precisa da suíte e2e, que é serial, e além disso a fila come o ganho. *Ruim*: o `pre-commit` pega no **commit**, não na escrita — o agente ainda gasta trabalho num arquivo que não devia tocar, e descobre ao commitar — e a alternativa que pegaria antes, descartada acima, cobre um agente só. *Ruim*: hook é conselho para quem clona e não roda `pnpm install` — sem `core.hooksPath`, não há barreira nenhuma, e isso vale para os três hooks. *Ruim*: a fatia não reprova caminho inexistente, porque a frente pode estar criando o módulo; um erro de digitação vira uma fatia que protege menos do que parece — por isso o script *diz* qual caminho não existe. *Bom*: a classe de erro da **ADR-61** morre na origem, porque o `origin/dev` passa a estar escrito no script em vez de na memória de quem digita. *Bom*: o script recusa nome de branch que o `pre-push` recusaria, e a recusa acontece no minuto zero em vez de depois de horas de trabalho numa branch mal nomeada.

Gatilho de reavaliação. Passar a existir mais de uma pessoa desenvolvendo ao mesmo tempo (hoje é uma), quando a fila da fase serial deixa de ser aceitável; ou a suíte de ponta a ponta virar o gargalo medido — aí a stack por worktree volta à mesa com o custo justificado por número, não por intuição.

18.73 ADR-72 — A fronteira do disfarce é a revelação, e a Etapa 4 da exp06ef2b fica fora dele

Data. 2026-09-06 **Status.** Aceita

Contexto. A exp06ef2b termina com duas telas analíticas: um gráfico comparando a velocidade do boato com a da nota oficial, e um mapa da topologia de rede que desenha o alcance da decisão do estudante. Os wireframes e o *handoff* da missão colocam as duas **dentro do clone**, descritas como "painel de métricas do grupo" — diegéticas, faladas pela seção `whatsappmobile`.

O problema é que o aplicativo real não tem nada parecido. Ele mostra "Encaminhada com frequência" num balão; não mostra grafo de propagação, não conta conexões e não desenha nós. Uma tela dessas dentro do disfarce não é verossimilhança seletiva — que admite **ausências** (sem teclado, sem chamadas, sem Stories) — é **invenção**: uma funcionalidade que o app imitado não possui, apresentada como se possuísse.

Decisão. O disfarce vale **até o fim da Etapa 3**. Da pausa entre as aulas em diante quem fala é a escola: o card de espera, o gráfico, o mapa, o construtor do ciclo e o "Meu percurso" usam `@edu/ui/ds` e os tokens `--unooria-*`.

O critério que decide, e que já valia no repositório, é **"quem está falando?"** — não "é depois no tempo?". A exp04ef1b já o havia aplicado: feed, cromo e playlists falam de dentro da ficção; o ranking e os diálogos da missão falam pela escola, ainda que apareçam no meio dela.

Alternativas descartadas.

- *Seguir os wireframes e desenhar o painel dentro do clone*: era o caminho especificado, e o QA de design o aprovou. Descartada porque quebra o próprio teste de fidelidade que aquele QA aplicou ("o frame passaria por uma captura do app real?"): um grafo de topologia não passa, porque o app não tem grafo de topologia. E a fidelidade do clone é o instrumento — é ela que faz o comportamento medido ser espontâneo.
- *Levar o pedido ao `storybook-unooria`*, publicando um componente de visualização de rede na seção `whatsappmobile`: pioraria o problema em vez de resolvê-lo. O pacote passaria a publicar, como parte do clone, uma tela que o original não tem — e a próxima missão que usasse a seção herdaria a invenção.
- *Manter as duas telas no clone e apenas trocar os tokens*: é o pior dos dois mundos. A tela continuaria dentro do aparelho, com o cromo do mensageiro em volta, e a marca institucional apareceria no meio do disfarce — a mistura de seções que a regra de ouro proíbe.
- *Tirar as telas analíticas da missão* e deixar a revelação apenas para o debate mediado: mataria o beat memorável. A descoberta central da missão é a diferença entre 1.200 e 850 conexões, e ela precisa ser **vista**, não contada — é isso que a criança leva.

Consequências. *Ruim*: a missão troca de identidade visual no meio, e a transição precisa ser justificada ao estudante pelo próprio roteiro — o card de pausa é o que faz esse trabalho ("sua investigação está salva"), e sem ele a troca pareceria um erro de carregamento. *Ruim*: o insumo aprovado e o código divergem neste ponto, e a divergência tem de viver escrita em três lugares (aqui, no `AGENTS.md` da experiência e no `catalogo.ts` do hub) para não ser "corrigida" de volta por quem chegar depois lendo só os wireframes. *Bom*: o teste do app sem legenda continua valendo para todas as telas que se apresentam como o mensageiro. *Bom*: os tokens da Etapa 4 passam a ser os da plataforma, e isso corrigiu um defeito real — com os tokens do clone, calibrados para fundo escuro, os nós do mapa saíam quase invisíveis sobre o branco e o beat memorável não chegava.

Gatilho de reavaliação. O dia em que o aplicativo imitado publicar de fato uma visualização de propagação (o "encaminhada com frequência" já foi nessa direção). Aí a tela deixa de ser invenção, e a fronteira pode voltar para depois dela.

18.74 ADR-73 — O planejamento de ensino é dirigido pela BNCC extraída do PDF homologado, e o componente da rede não é o componente da Base

Contexto

A app-90 gera plano anual a partir do **currículo da rede importado como texto** (ADR-47): um documento em prosa entra no *prompt*, e o modelo infere dele a progressão do ano. O arranjo serve a quem tem documento curricular próprio e material didático adotado — e é inútil para quem não tem, que é a maioria das redes pequenas. Ele também não desce da intenção anual para a aula de quinta-feira, que é onde o trabalho do professor realmente acontece.

Havia um segundo limite, mais estreito: o produto nasceu para a **BNCC Computação**. Mas a Resolução CNE/CEB nº 1/2022 não cria componente isolado — ela **delega às redes** definir "parâmetros e abordagens pedagógicas" (art. 2º) —, e a Lei 14.533/2023 fez da educação digital componente do Fundamental e do Médio pela LDB (art. 26, parágrafo 11). Computação é camada transversal, não disciplina; um planejador que só a atenda serve a uma fração da carga horária de um professor.

Três achados do levantamento decidiram a forma da solução, e nenhum deles é óbvio:

Primeiro: no Ensino Médio, componente da matriz e componente da Base NÃO coincidem. A matriz de qualquer rede tem Biologia, Física e Química; a BNCC do Médio desce apenas até a **área**, e as três compartilham as 26 habilidades de Ciências da Natureza e suas Tecnologias (`EM13CNT`). O mesmo vale para Filosofia, Geografia, História e Sociologia, que dividem as de Ciências Humanas. As duas exceções são Língua Portuguesa e Matemática, os únicos componentes que a Base do Médio nomeia.

Segundo: a Base não é uniforme no organizador acima do objeto de conhecimento. Sete componentes usam unidade temática; Língua Portuguesa usa prática de linguagem dentro de campo de atuação; Língua Inglesa usa eixo; o Médio agrupa por competência específica de área.

Terceiro: o PDF que o MEC serve na página do Ensino Médio é o RASCUNHO de 11/05/2018, com 179 habilidades contra as 183 do homologado e **textos divergentes** — a `EM13MAT101` do rascunho perdeu "criticamente" e "fatos relativos". O documento consolidado de dezembro de 2018, distribuído sob o nome `BNCC_EI_EF_110518`, é o que traz as três etapas.

Decisão. Nasce a app-92, ao lado da app-90 e sem substituí-la, com o currículo como DADO ESTRUTURADO extraído do PDF homologado, cobrindo todos os componentes do 1º ano do

Fundamental à 3ª série do Médio, em dois níveis: plano **anual** (macro, por período de avaliação) e plano de **período** (granular, aula a aula, semanal ou quinzenal à escolha da rede)[1].

A fonte é o PDF homologado, extraído por coordenadas. As tabelas da Base são um *spread* de duas páginas — unidade temática e objeto de conhecimento na par, habilidades na ímpar — e no texto corrido do `pdftotext` as três colunas viram três blocos sequenciais: a ligação habilidade ↔ objeto de conhecimento se perde, e atribuí-la seria chute. O que salva é o alinhamento do InDesign: o objeto "Estratégia de leitura" sai em `yMin=260,5` e a `EF15LP02` em `yMin=260,4`, na página seguinte. A associação é geométrica e determinística (`scripts/extrair-bncc.mjs`), e rende **1.304** habilidades do Fundamental — o número oficial —, **183** do Médio e **129** de Computação, das quais 98 % com objeto de conhecimento.

O componente da rede é modelado separado do componente da Base (`componenteBncc` em `curriculo/componentes.ts`), e a tela AVISA quando as habilidades são da área. Sem isso, o professor de Biologia do 2º ano — o caso mais comum do Médio — receberia "nenhuma habilidade encontrada".

A distribuição de habilidades pelos períodos é DECISÃO HUMANA. Ela expressa a leitura que o professor faz da própria turma, e é a única coisa aqui que nenhum modelo deveria escolher por ele; a IA entra depois, para desenvolver pedagogicamente o que ele decidiu. Pelo mesmo motivo, o **esqueleto das aulas não vem do modelo**: número, data e horário saem do calendário e da grade real. Deixar o modelo decidir quantas aulas cabem na semana produzia cinco — o número redondo — para um professor de Arte que tem uma.

Há três barreiras contra código de habilidade inventado, e nenhuma sozinha basta: o *prompt* proíbe, a rota de distribuição recusa código fora do recorte, e a costura filtra o que o modelo devolveu (`planejamento/costura.ts`). Um plano que cite `EF67LP42` chega à coordenação como erro do professor, e destrói a confiança no produto inteiro.

A articulação com Computação é rotulada como PROPOSTA. Não existe *crosswalk* normativo entre habilidade de Computação e habilidade de componente — a norma delega a abordagem à rede —, e vender inferência como norma é risco de auditoria numa venda a poder público. Em contrapartida, a extração aproveita as colunas "Explicação da habilidade" e "Exemplos" do anexo do Parecer CNE/CEB nº 2/2022: texto pedagógico oficial que nenhum *dataset* público da Base empacota.

Alternativas descartadas.

Consumir um *dataset* público de terceiro (o `bncc-dev/bncc-dados`, CC BY 4.0, com DOI e proveniência auditável) foi seriamente considerado e é tecnicamente bom. Recusado por dois motivos: num contrato B2G a pergunta "de onde veio este texto?" precisa terminar no documento oficial, não num repositório de terceiro; e o mantenedor é uma *edtech*, o que faz da base curricular do produto uma dependência de um concorrente potencial. Ele continua útil como conferência de completude no CI.

[1] A escolha entre semanal e quinzenal foi retirada na ADR-76: o passo é sempre semanal, e entregar de quinze em quinze dias virou recorte de exportação.

Ingerir o Mapa de Foco do Instituto Reúna (priorização de habilidades por ano, pedagogicamente excelente) foi **vedado por licença**: os Termos de Uso proíbem alteração e comercialização sem autorização expressa. Se interessar, entra por contrato, não por clone de repositório.

Um sistema de RAG sobre os PDFs foi descartado por ser a ferramenta errada, não por custo. A pergunta que o produto faz é sempre "quais habilidades de *componente* no *ano*" — consulta exata sobre dois campos estruturados. Busca vetorial responderia com aproximação onde existe resposta certa, e um plano anual que "esqueceu" a `EF67LP08` porque ela ficou fora do *top-k* é um plano que descumpre o currículo.

Substituir a app-90 foi descartado: são ferramentas para redes diferentes, e migrar os planos já salvos custaria mais do que manter as duas.

Consequências ruins, declaradas.

Ficam duas telas de planejamento no menu do professor, e a diferença entre elas não é autoevidente para quem chega. A mitigação é a descrição de cada card no hub; se a confusão aparecer em campo, uma delas some.

A matriz de carga horária é mediana de dez redes, não norma — e a mediana esconde uma bimodalidade. A BNCC não define grade horária (LDB art. 23 e 26; Parecer CNE/CEB nº 8/2004), e o total semanal do Fundamental tem duas famílias sem nada no meio: seis redes em 25 aulas (833 h) e quatro em 28–30 (890–1.000 h, entre elas as duas maiores do país). Uma média entre as duas descreveria rede nenhuma, então o produto pré-preenche o subtotal da base comum e deixa o professor ajustar. Nos anos iniciais há um agravante: em várias redes o modelo nem é disciplinar — São Paulo e o município do Rio agregam cinco componentes num bloco de professor polivalente, e para elas a matriz sugerida não descreve a realidade.

Vinte e seis habilidades do Fundamental (2 %) ficaram sem objeto de conhecimento na extração, e aparecem num grupo "Sem classificação". Preferimos mostrá-las a escondê-las: habilidade escondida é currículo não cumprido.

A extração depende de `pdftotext` e de um PDF que o MEC pode substituir. Se o layout mudar, o extrator quebra — e quebra em silêncio, o que é pior. Contra isso há testes que travam o DADO, não só o código: os totais oficiais de cada documento, e o texto de `EM13MAT101` com as palavras que só existem na versão homologada.

Gatilho de reavaliação. Uma revisão da BNCC (há discussão pública sobre a do Ensino Médio desde a Lei 14.945/2024); a publicação de uma exportação oficial máquina-legível estável pelo MEC; ou o primeiro contrato cujo currículo próprio precise ser exibido AO LADO da Base — caso em que a fronteira entre app-90 e app-92 deixa de fazer sentido e as duas convergem.

Recorte posterior. Para o agente de IA do professor, e só para ele, o dono do produto decidiu buscar as habilidades num *data store* e avaliar (**ADR-127**); a exatidão foi preservada pela forma do dado (uma linha por habilidade, filtro exato e paginação até o fim), e a volta para a consulta sobre `src/curriculo` é o gatilho daquela decisão. Aqui nada muda.

18.75 ADR-74 — `generateJson` informa o que o vendor cobrou por um *callback*, e não pelo retorno

Data. 2026-09-06 **Status.** Aceita

Contexto. A regra 5 do projeto é literal: nenhuma chamada de IA sem `QuotaService` — `temSaldo` antes, `consumir` depois. Mas dos três métodos de texto do `AiProvider`, dois devolvem o que foi gasto (`generate` e `conversar`, por `AiResult.tokensUsados`) e `generateJson` devolvia só `T`. Sem número, não há o que debitar — e cada consumidor inventou a própria saída:

- a **app-92** (plano de aulas) chamava `temSaldo` e nunca `consumir`: o professor gerava as aulas de todos os bimestres e o contador não se movia. É a operação mais repetida da ferramenta — 4 bimestres $\times \sim 10$ semanas —, então o teto diário perdia sentido justamente onde mais importa;
- a **exp07ef1b** debitava `1_500` fixo, chute que não acompanha tamanho de *prompt* nem troca de modelo;
- o **bem-estar** desviou do problema: usa `generate` em vez de `generateJson` "porque só ele devolve `tokensUsados` para a quota" — o comentário está no código desde então;
- o módulo `geracao` (motor da exp08ef1b) não contabiliza nada.

Quatro consumidores, quatro respostas diferentes para o mesmo buraco. Não é descuido de quem escreveu cada um: é o contrato que não oferecia o número.

Decisão. `AiJsonRequest` ganha `aoUsarTokens?: (tokens: number) => void`, chamado **uma vez por tentativa** — inclusive pelas que `validar` rejeitou, porque foram cobradas igual. Quem chama `temSaldo` passa a ser obrigado a passá-lo. A app-92 debita no `finally` (token gasto em geração que falhou no meio também foi gasto) e grava o total em `app_92_planos_de_periodo.toke` a exp07ef1b troca o chute pelo número real.

Alternativas descartadas.

Mudar o retorno para `{ dados, tokensUsados }`, que era a forma mais explícita e a primeira escolha. Foi descartada ao olhar os pontos de chamada: três dos seis fazem `return provider.generateJson(...)` *direto* — em `geracao/prompts/`{`destilacao`, `personas`, `posts`} —, então a mudança não pararia neles: subiria pelo tipo de retorno dessas funções e por quem as chama, atravessando o motor da exp08ef1b, que é a zona mais delicada do repositório. Tudo isso para entregar um número a quatro consumidores que hoje não têm quota. O custo caía sobre o código mais arriscado e o benefício, sobre nenhum deles.

Um método irmão `generateJsonComUso()`. Aditivo e explícito, mas seriam dois métodos fazendo a mesma coisa — e o segundo, o que ninguém lembra de procurar.

O *provider* consumir a quota sozinho. Resolveria de vez o esquecimento, e foi descartada por quebrar a fronteira que a **ADR-23** sustenta: `src/ai` não conhece escola nem usuário, e passar a conhecer o faria depender do domínio para debitar.

Consequências ruins, aceitas. Continua sendo possível esquecer: o campo é opcional, e um consumidor novo que chame `temSaldo` sem `aoUsarTokens` volta a não debitar — exatamente o defeito que esta ADR corrige, agora com ferramenta disponível em vez de ausente. O compilador não pega; o que pega é a revisão e o texto do contrato, que diz a obrigação onde ela é lida. **O número é do vendor**: `usageMetadata.totalTokenCount` é o que o Google reporta, e uma mudança no que ele conta muda o que debitamos sem aviso. E **a exp07ef1b passa a debitar mais do que antes** nas respostas longas — o chute de 1.500 era otimista —, o que aproxima a quota da realidade mas encurta o dia de quem usava muito.

Gatilho de reavaliação. O primeiro consumidor de `generateJson` que esquecer o `aoUsarTokens` tendo `temSaldo` — aí o custo do retorno explícito passa a valer o risco de mexer no motor. Ou a decisão de dar quota ao módulo `geracao`, que traria três dos seis pontos de chamada para o lado de quem precisa do número.

18.76 ADR-75 — A rolagem do hub é o `ScrollBar` do *design system*

Data. 2026-09-07 **Status.** Aceita

Contexto. A barra de rolagem vertical devia aparecer enquanto a pessoa rola e sumir quando ela para. Na lista de estudantes de uma missão isso já acontecia, e a leitura natural foi "o resto do produto está errado". A medição mostrou outra coisa: ali ninguém implementou nada — a `.tabela-rolavel` não estilizava a barra, então ela caía na barra *overlay* NATIVA do macOS, que faz esse desaparecimento sozinha. Essa barra depende de três coisas fora do produto: a preferência `AppleShowScrollBars` em automático, o apontador ser *trackpad* (com mouse conectado o próprio macOS volta à barra permanente — medido, 15px ocupando *layout*) e o sistema ser macOS. No Windows e no ChromeOS — o laboratório da escola — ela é permanente sempre.

A varredura que respondeu a isso encontrou o problema de verdade, e ele não era a barra: **o *design system* publica um `ScrollBar`** (vertical, horizontal ou ambos, com `thickness`, `theme` e `autoHide`) desde a 0.1.x, e o hub **não o usava em lugar nenhum**. As 24 tabelas rolavam em `<div>` com `overflow` escrito à mão, mais o sumário das páginas legais e 5 tabelas largas do guia. Não houve decisão de não usar: o componente nunca foi procurado. É a regra de ouro do projeto — todo elemento de interface vem do pacote — falhando por ausência de PROCEDIMENTO, não por desacordo.

A mesma varredura (38 telas, os quatro papéis) encontrou um defeito à parte: três rotas do professor — plano de aula, plano de ensino e bem-estar — mostravam uma barra PARADA na tela, visível sem ter o que rolar. Elas subtraíam a altura do cabeçalho à mão (`calc(100dvh - 64px)`), e o cabeçalho mede 65px: o `h-16` são 64 e a `border-b` põe 1. A página transbordava 1px.

Decisão. Toda área rolável do hub passa a ser um `<ScrollBar>` do DS, com `autoHide`. Nenhuma tela do hub declara `overflow` para rolar. E a página que NÃO rola deixa de estimar

altura de irmão: a casca do professor virou coluna *flex* com o `main` em `flex-1`, e a tela cheia usa a classe `.tela-cheia`. São os dois lados da mesma régua — área que rola tem barra do DS, área que não rola não tem barra nenhuma.

A `.tabela-rolavel` não morre: ela **encolhe** para o que o pacote ainda não publica — a `max-height` que provoca a rolagem e o `<th>` preso (a lacuna de `stickyHeader` já registrada). O `overflow` sai dela, senão haveria dois contêineres de rolagem, um dentro do outro.

E a regra de ouro ganha o procedimento que faltava, no `AGENTS.md` da raiz: procurar no pacote ANTES de escrever interface; usar o que existe mesmo que dê mais trabalho; e, se não existe, abrir pedido no `storybook-unooria` com compensação local MARCADA — nunca um componente local silencioso.

Alternativas descartadas.

Uma camada própria de *auto-hide*, global, dirigida por rolagem. Foi implementada e medida antes desta decisão (*branch* `feat/auto-hide-da-rolagem`): um módulo na `@edu/sdk` marcava `data-rolando` no elemento que rolava e o CSS reagia. Entregava o comportamento literal pedido — aparecer ao rolar, em qualquer sistema operacional — e cobria o que o componente não alcança: a barra da própria página, as que nascem dentro de `Modal` e `Selection`, e as nove experiências. Foi descartada por uma razão de doutrina que vale mais que o ganho: era uma barra ESCRITA AQUI, com `!important` sobre as regras do pacote, para um componente que o pacote já publica. O projeto tem 48 experiências pela frente e uma pessoa desenvolvendo; a única forma de isso escalar é o pacote ser a fonte, mesmo quando ele entrega menos.

Trocar só a aparência, mantendo o `<div>` — aplicar as classes utilitárias `.unooria-scrollbar` sem o componente. Menor diff e nenhuma mudança de marcação. Descartada porque a utilitária declara `overflow-x: hidden`, que cortaria a coluna da direita das tabelas largas, e porque "usar as classes de um componente sem usar o componente" é o atalho que faz a próxima migração de versão do pacote quebrar em silêncio.

Consequências ruins, aceitas. O `autoHide` do DS é *hover*, não rolagem. O `ScrollBar.types.ts` documenta "hover ou rolagem", mas o CSS só tem regras `:hover`: a barra aparece ao passar o mouse por cima e continua invisível quando a rolagem vem da roda ou do teclado. É menos do que o pedido original, e é o preço de manter a fonte no pacote — com pedido aberto no `storybook-unooria` e a prop já no lugar para o dia em que ele publicar. **Fora do hub nada mudou**: as nove experiências seguem com a barra do sistema, porque o `ScrollBar` é da seção `unooria` e não pode entrar num clone. **Uma compensação a mais**, medida: `.tabela-rolavel .unooria-table-container { overflow: visible }` — a `Table` do DS traz o próprio contêiner com `overflow-x: auto`, e é nele que o `<th>` sticky se prende, um contêiner que não rola na vertical; sem essa regra o cabeçalho deixa de grudar. E **no papel o `ScrollBar` sai da frente** (`@media print` devolve `overflow: visible`), senão a tabela do guia sairia recortada na largura da coluna. **Uma segunda classe local**, a `.tela-cheia`: `height: 100%` não resolveria no lugar do *flex* — a altura de um item que cresceu por `flex-grow` dentro de um contêiner de `min-height` não é *definite*, o percentual cai para `auto` e o `<iframe>` volta aos 150px de altura

padrão dele — e o `main` só vira contêiner *flex* por `:has(> .tela-cheia)`, para as outras 16 páginas do professor não mudarem de contexto de *layout* por causa de três telas.

Gatilho de reavaliação. O `storybook-unooria` publicar `autoHide` dirigido por rolagem — aí o comportamento pedido chega sem código nosso. Ou a decisão de dar *auto-hide* às nove experiências, que exigiria o componente nas seções de clone ou uma camada neutra, e traria de volta a alternativa descartada aqui.

18.77 ADR-76 — O planejamento é semanal, e a quinzena é recorte de exportação; o `iframe` ganha `allow-downloads` por declaração

Data. 2026-09-07 **Status.** Aceita

Contexto. Três defeitos relatados no mesmo dia sobre a app-92, e os três tinham a mesma raiz: uma pergunta de *entrega* tratada como configuração de *produto*.

1. **Não havia como apagar um planejamento.** A regência nascia e ficava. `DELETE /planejamento/planos-a` existia desde a ADR-73, mas o que a lista do professor mostra é a **regência** (componente × ano), e para ela não havia rota nem botão.
2. **"Baixar .doc" não baixava nada**, sem erro na tela e sem linha no *console*. Duas causas somadas, e a principal não estava na experiência: num `iframe` com `sandbox`, o navegador **bloqueia todo *download*** sem o token `allow-downloads`, e o bloqueio é MUDO — o Chrome o registra no painel *Issues*, que ninguém abre, e o `a.click()` do *bundle* volta como se tivesse funcionado. MEDIDO: dois `iframe` com o mesmo *blob* e o mesmo código, um com o token e outro sem; só o primeiro escreveu arquivo. Somava-se a isso o `<a>` não anexado ao DOM (Chrome tolera, Firefox e Safari ignoram) e o `revokeObjectURL` na linha seguinte ao clique.
3. **"Existe semanal e quinzenal e eu não entendi como usar."** A regência pedia o "passo do planejamento", e ao fim o produto só oferecia detalhar aula a aula — a escolha não mudava a tela, mudava a aritmética interna do calendário.

Decisão.

(a) O passo do planejamento é SEMPRE semanal. A coluna `app_92_configuracoes.periodicidade` sai (migration `20260907120000_exp_92_planejamento_sempre_semanal`), com ela o campo do formulário e o parâmetro de `janelasDePlanejamento`, que passa a se chamar `semanasDoAnoLetivo`. **A quinzena vira recorte de exportação**: o professor planeja semana a semana e o botão de baixar junta duas semanas, ou o período de avaliação inteiro, num só documento — uma semana por página.

(b) `allow-downloads` entra no `sandbox`, por declaração. `downloads` passa a valer no `permissoes` do manifesto (Zod da api, CHECK de `experiencias.permissoes` e `ExpFrame`), e só a app-90 e a app-92 o declaram. **E as três rotas de professor passaram a repassar `permissoes` ao `ExpFrame`** — elas nunca repassaram, então a permissão sozinha não teria efeito nenhum ali.

(c) `DELETE /planejamento/configuracoes/:id` apaga a regência, o plano anual e todas as semanas. A cascata é do banco; o *object storage* não tem cascata, então os prefixos de cada plano anual são varridos ANTES do `delete` — depois dele não haveria mais como saber quais eram.

Alternativas descartadas.

Manter a periodicidade e corrigir só as datas. A `ordem` de `app_92_planos_de_periodo` muda de significado com o passo — a 3ª quinzena não é a 3ª semana —, então um plano gravado em modo quinzenal e lido em modo semanal casa pela ordem e devolve as datas de um recorte com as aulas de outro. Corrigir início e fim no `upsert` (o que foi feito, e era um defeito real: o `update` não os recalculava) resolve a troca de *calendário*, não a troca de *significado*.

`allow-downloads` para todas as experiências. Entregar arquivo ao computador de quem está usando é capacidade de quem declarou, como `pointer-lock` e `forms`. Nenhuma experiência de estudante exporta nada, e o *bundle* é público.

DOCX de verdade. Continua fora: o `.doc` é HTML com os *namespaces* do Office, zero dependência no *bundle*, e o Word, o Google Docs e o LibreOffice abrem como documento. Trocar somaria dependência para resolver problema que ninguém tem — o problema era o `sandbox`.

Consequências ruins. A migration **apaga** os planos de período das regências que estavam em quinzenal — é a única parte destrutiva, e a alternativa era mantê-los mentindo sobre as próprias datas. Regência, plano anual e distribuição de habilidades ficam; o que se perde é o detalhamento aula a aula, que o professor regera pelo botão da semana (gastando quota de IA de novo). O "Baixar o bimestre" faz uma chamada que traz as aulas de até dez semanas — aceito: são 40 linhas no ano inteiro e o filtro é no banco.

Gatilho de reavaliação. Uma rede pedir planejamento cuja unidade *pedagógica* não seja a semana (aula avulsa, ou projeto de duração variável) — aí volta a pergunta de qual é o passo, e a resposta não é reintroduzir uma coluna de enum. Ou o `storybook-unooria` publicar um componente de exportação com recortes, que substituiria os três botões.

18.78 ADR-77 — A trilha é do *layout* da área, nunca da página

Data. 2026-09-07 **Status.** Aceita

Contexto. O *breadcrumb* do hub estava em 22 telas e faltava em 15. Não houve decisão por trás disso: a trilha era responsabilidade da PÁGINA, cada uma declarava a sua, e nada obrigava. As ausências não eram aleatórias — eram as 7 telas de `/admin`, as 2 de `/analise`, o *hub* do estudante e cinco da própria área do professor. A barra lateral de `/admin` é a razão de ninguém notar por tanto tempo: ela cobre a navegação de primeiro nível e esconde a falta do segundo.

Decisão. Quem desenha a trilha é o *layout* da área (`TrilhaDaRota`, em `components/trilha.tsx`), e os itens saem de um REGISTRO DE ROTAS — `NOS`, em `lib/trilha.ts`, com o padrão da rota como chave. Rota nova é uma linha no registro e nada na página. A página só pode PUBLICAR

nome próprio (`RotuloDaTrilha`): o título da atividade, o nome da turma, o nome do estudante — o que uma tabela de rotas não tem como saber.

A assimetria é a decisão, não um detalhe dela: **a página consegue melhorar a trilha e não consegue apagá-la**. É isso que tira a ausência por esquecimento da mesa. Sem publicação, o item de um segmento dinâmico fica no rótulo de domínio do registro ("Atividade", "Turma", "Estudante"), nunca no id do endereço; rota que ninguém registrou cai num rótulo humanizado do segmento. Trilha aproximada é defeito que se vê e se corrige; trilha ausente é o defeito que ninguém nota até o dono do produto notar.

Alternativas descartadas.

- *Por página, como estava*: é o desenho que produziu o defeito. O custo já foi pago e medido — 15 telas sem trilha, nenhuma por decisão.
- *Só do layout, derivada da URL, sem contribuição da página*: o *layout* conhece o segmento, não o nome do que vive atrás dele. A trilha diria "Turmas / clx8f9a2b0001". Pior que trilha nenhuma.
- *A página publica a trilha INTEIRA, e o layout desenha o que receber*: devolve à página a autoridade sobre a ESTRUTURA — e era a estrutura que estava divergindo, não os rótulos. Volta ao primeiro caso por outro caminho.

Consequências. Página nova nasce com trilha sem escrever uma linha, e a largura da coluna subiu para o *layout* nas duas áreas onde cada página trazia a sua (1080 e 1180px numa, 5xl e 6xl na outra, sem razão registrada em nenhum dos casos) — com a trilha no *layout*, largura por página faria o *breadcrumb* nascer 50–64px à esquerda do texto.

Ruim: um quadro de rótulo genérico em cinco rotas. `RotuloDaTrilha` publica num *effect*, então o HTML do servidor traz "Atividade" e a hidratação troca por "Debate sobre X". É inevitável na direção do dado — o *layout* renderiza ANTES da página e não tem como perguntar nada a ela — e é o preço de a trilha ter UM autor.

Ruim: `rotuloDaUnidade` ESPELHA `rotuloDaMissao` de `lib/entre-redes/catalogo.ts` em vez de importá-lo, porque o módulo da trilha é *client* e o catálogo carrega as fichas e os guias das missões inteiras. Mudar o formato do *slug* do ano faz a trilha cair no genérico ("Missão") **em silêncio**, em doze telas. É por isso que a lógica foi para `lib/`: o *vitest* do hub só varre os testes de lá, e `lib/trilha.test.ts` prende os dois formatos (`8-ano` e `2-serie`).

Ruim: o `ProvedorDaTrilha` não pode renderizar DOM nenhum. Em `/professor/**` a altura das três telas de *embed* vem de `main:has(> .tela-cheia)`, e um elemento a mais quebra o seletor de filho direto — de volta à barra de rolagem parada que a **ADR-75** registra.

Duas exceções declaradas. `aPaginaDesenha: true` nas rotas de guia da aula, porque quem renderiza a trilha delas é o componente do guia, por dentro, e duas trilhas na mesma tela é pior que nenhuma; e a home de cada papel, onde uma trilha de um item só, que é a própria página, é ruído. Tela sem sessão, aparelho em tela cheia com `voltar` no *header* e superfície pública ficam fora porque não estão dentro de *layout* de área nenhum — o que significa que **área nova**

precisa de *layout* novo, e *layout* novo desenha a trilha. Sem isso, a área nasce como `/admin` nasceu.

Gatilho de reavaliação. O App Router publicar um caminho de a página contribuir dado ao *layout* no servidor — aí o quadro de hidratação desaparece sem mudar o desenho. Ou a terceira área que precise de trilha fora de *layout* de área, que obrigaria a rever a fronteira.

18.79 ADR-78 — O *logoff* mora no menu do avatar, não na barra

Data. 2026-09-07 **Status.** Aceita

Contexto. O `HeaderPlataforma` terminava em duas peças soltas: um `Avatar` decorativo e um `Button variant="ghost"` escrito "Sair". A ação mais destrutiva do *chrome* ficava a um clique de distância, ao lado do nome da pessoa, com o mesmo peso visual da marca — e o avatar não fazia nada.

Decisão. O avatar passa a ser o GATILHO de um menu, e o "Sair" mora dentro dele. O componente é o `HeaderMenu` da seção `unooria` do *design system*, publicado na 0.1.19 a partir do conjunto `Menu Desk` do Figma: uma tira que expande da direita para a esquerda, com o item de saída colado no gatilho.

O componente foi escrito no `storybook-unooria`, não aqui. É a aplicação direta da **ADR-19**: um menu tem estado e aparência de controle, então é componente, e componente nasce no pacote. A verificação que precede a decisão também está registrada — o DS 0.1.17 não publicava nada equivalente: não há `Dropdown`, `Menu` nem `Popover` na seção, o `Avatar` aceita `onClick` mas não tem prop de menu, e o único menu suspenso do pacote é o `TableDropdownMenu`, com gatilho fixo em três pontos e sem aceitar *children*. Publicado como PATCH de propósito: `^0.1.x` não sobe *minor*, e uma 0.2.0 exigiria editar o *range* à mão em dez `package.json`.

Alternativas descartadas.

- *Compensação local no hub*: destravava no mesmo dia sem tocar outro repositório, e é exatamente o que a **ADR-19** proíbe — o componente serviria uma superfície e as outras seguiriam com o botão solto.
- *Manter o botão e só tornar o avatar clicável*: duas affordances para a mesma ação, e nenhuma resolve o peso visual.
- *Trazer o header inteiro do Figma*: o nó publicado é um *header* completo, e adotá-lo exigiria migrar marca, seletor de escola e identidade de uma vez. O menor componente que resolve o pedido é o menu com gatilho plugável.

Consequências. *Ruim, e é a consequência que custou trabalho*: o nome e a linha de apoio tiveram de sair do `HeaderPlataforma` e entrar no mesmo componente do menu. A tira do `HeaderMenu` é um *overlay* ancorado à esquerda do gatilho, e o componente assume que ali há espaço vazio — verdade num *header* que só tem o avatar à direita, falsa neste, onde ali fica a identidade. Integrado ingenuamente, o item "Sair" abre DESENHADO EM CIMA das letras. Isso

passa no *typecheck* e no *build*, e só aparece abrindo o menu na tela: foi assim que apareceu. A compensação é a identidade ceder o espaço enquanto a tira está aberta — `invisible`, não desmontada, senão o gatilho escorrega embaixo do cursor — e ela é comportamento de página, não CSS, então NÃO tem entrada em `plataforma.css`: uma classe faria o próximo a integrar o componente achar que era decoração.

Ruim: o menu nasce com UM item. O componente aceita vários e a tentação é preencher com "Meu perfil" e "Configurações", telas que não existem. O candidato real, "Trocar senha", também ficou fora por um motivo verificado no `middleware.ts`: quem já trocou a senha é devolvido para a home do papel, então o item seria um link morto. Item de menu que não leva a lugar nenhum é pior que menu curto.

Uma ausência de produto que a decisão expôs. Não existe caminho voluntário para trocar a senha depois do primeiro acesso: a rota é forçada pelo *middleware* na largada e fechada em seguida. O menu da conta é o lugar óbvio desse item no dia em que ele existir.

Gatilho de reavaliação. O `HeaderMenu` publicar *reflow* ou âncora à direita — aí a compensação da identidade morre e o nome pode voltar para o `HeaderPlataforma`. Ou o produto ganhar duas telas de conta, que é quando o menu de um item passa a ser um menu de verdade.

18.80 ADR-79 — Toda escola pertence a uma regional, e a rede nunca fica sem nenhuma

Data. 2026-09-08 **Status.** Aceita

Contexto. A **ADR-34** fez o nível REGIONAL existir de verdade, mas deixou `pai_id` opcional — a unidade podia ficar órfã. O custo apareceu na tela: o filtro "Regional" não PARTICIONAVA a lista, porque havia escolas fora de toda regional, e a importação `orgs.csv` entrava com `pai_id` nulo de propósito quando o `parentSourcedId` era desconhecido ("melhor unidade sem regional que unidade pendurada em nada"). Somando os dois, "as escolas desta diretoria" nunca respondia pela rede inteira, e a lotação em lote — o único consumidor do agrupamento — cobria menos do que parecia cobrir.

O relato que originou a decisão é de usabilidade, e é o mesmo problema visto de fora: `/admin/escolas` listava os TRÊS níveis na mesma tabela, com o tipo numa etiqueta, travessões nas colunas de turma e estudante das linhas de regional, um resumo que somava níveis diferentes ("3 registros · 2 unidades") e um filtro de "Tipo" cuja única função era desfazer a mistura. O rótulo "Sem regional" do filtro de regional MENTIA: o valor vazio nunca filtrou órfãs — ele limpava o filtro e mostrava a rede toda.

Decisão. Três coisas, juntas porque uma sem a outra não fecha:

1. *Vínculo obrigatório*. `tipo = 'escola'` exige `pai_id` (CHECK `escolas_escola_tem_regional_check`), e o pai tem de ser `regional` — não mais `rede`. Escola pendurada na raiz era um segundo desenho de hierarquia que nenhuma consulta usava.

2. *Regional padrão*. A coluna `regional_padrao` marca a regional que a plataforma cria sozinha, para a rede sem divisão administrativa. Índice único PARCIAL garante no máximo uma. Ela nasce chamada `Rede` e o nome é EDITÁVEL.
3. *Telas separadas*. `/admin/regionais` e `/admin/escolas`, nessa ordem no menu, cada uma listando um nível só. O filtro de "Tipo" deixou de existir — separar as telas REMOVEU um controle.

Por que o nome nasce `Rede` e não o nome da rede. O nome real do contrato NÃO existe no banco: o contrato é a stack inteira, não uma linha, e não há entidade `rede` — o `tipo = 'rede'` é aceito pelo CHECK e nada no código o cria. A única fonte do nome hoje é `CONTRATO_NOME`, uma variável de ambiente DO HUB, opcional, usada apenas na emissão do perfil portátil, e uma migração não tem como lê-la. Quem sabe o nome é quem administra: a tela de Regionais tem "Editar".

Por que a exclusão de regional entrou. A recomendação técnica era NÃO criar exclusão: o produto não apaga nada (desativa, encerra vínculo, preserva histórico) e não havia `DELETE` de escola em rota nenhuma. A decisão de produto foi outra, e é razoável — a rede que se reorganiza em diretorias novas precisa apagar as antigas, e a regional é o único nível que não guarda dado próprio. `DELETE /escolas/:id` aceita só `tipo = 'regional'`; ESCOLA continua sem exclusão em lugar nenhum, porque ela é *tenant*, com turma, dado de menor e histórico atrás. Duas recusas, as duas de produto: a rede nunca fica sem regional (409 `regional-unica`) e regional com escolas só sai dizendo para onde elas vão (409 `regional-com-escolas`, com `escolas` e `destinos` no corpo — é o que a tela usa para decidir entre abrir o diálogo de transferência e explicar que não há destino). A transferência e a exclusão são UMA transação: em duas chamadas existiria a janela em que as escolas já mudaram de regional e a antiga continua na lista.

Alternativas descartadas.

- *Manter `pai_id` nulável e só não oferecer "sem regional" na tela*: dava 90% do benefício sem migração. Não elimina o estado, esconde — e é exatamente a armadilha que este projeto já registrou três vezes (o `expires_at` sem rotina, o `turmas.consentimento_em` do aplicativo de origem, as próprias colunas `tipo`/`pai_id` da **ADR-34**): a regra sai do código, o registro fica, e quem lê acredita nele.
- *Reconhecer a padrão por convenção de id* (`id = 'rede'`) em vez de coluna: morreria no primeiro `PATCH` de nome, e renomear é justamente o que a decisão de produto permite. Três coisas dependem de reconhecê-la: o rótulo na tela, a não-duplicação numa reexecução e o texto de apoio.
- *Criar a padrão SEMPRE na migração*: numa rede já organizada em diretorias, com todas as escolas vinculadas, ela seria uma regional vazia que não agrupa ninguém — ruído numa tela cujo propósito é parar de misturar níveis. Ela nasce em dois casos: rede sem regional nenhuma (contrato recém-provisionado) ou rede com escola órfã para adotar.

- *Fazer o CHECK recusar todo `INSERT` sem regional*: era a leitura literal da regra, e quebraria TRINTA suítes que criam uma escola de mentira para testar outra coisa — em módulos que nada têm a ver com esta frente (Vila, unooGPT, bem-estar, salas). Emendá-las seria editar o módulo do vizinho, que é o que a **ADR-71** identifica como a causa de conflito de merge insolúvel. O trigger ADOTA pela padrão em vez de recusar.
- *Somar turmas e estudantes por regional na tela nova*: seria útil e é tecnicamente permitido (`admin-contrato` é a exceção cross-escola), mas aproxima a tela da visão somada de diretoria que a **ADR-34** recusa, e custaria uma agregação sobre cinco mil escolas. A regional mostra um número só: quantas escolas agrupa.

Consequências. *Ruim, e é a que muda comportamento observável*: a importação `orgs.csv` de quem JÁ USA passa a falhar em linhas que antes entravam. Com uma regional na rede, o vínculo é automático e sai um aviso; com várias, a linha sem `parentSourcedId` válido é ERRO, com o número dela no relatório. É o que o card de produto pede, e é a única saída coerente: uma órfã ali reprovaria no CHECK como exceção crua de Postgres no meio de um arquivo de mil linhas.

Ruim: o trigger que adota pela padrão é comportamento silencioso, e este projeto desconfia de silêncio. A mitigação é que ele NÃO é o caminho do produto — o formulário e a importação exigem a escolha explícita quando há mais de uma regional — e que sem padrão na rede ele não inventa destino: `pai_id` continua nulo e o CHECK recusa.

Bom, e não estava no pedido: a separação expôs um buraco de tenancy. `lotarEmLote` filtrava `tipo: 'escola'` desde o início, mas `POST /escolas/:id/lotacoes` não filtrava nada — por ela dava para lotar alguém numa REGIONAL, produzindo no seletor do cabeçalho a "escola fantasma" (sem turma, sem estudante, sem dado) que o lote existia para evitar. Corrigido junto, com teste.

Gatilho de reavaliação. O produto ganhar uma entidade `rede` de verdade — aí a regional padrão passa a poder herdar o nome dela e a variável `CONTRATO_NOME` sai do hub para o banco. Ou a rede que administra mais de um contrato numa stack, que é quando "a rede tem ao menos uma regional" deixa de ser um invariante global e passa a ser por contrato.

18.81 ADR-80 — Seis papéis, com cadeia de atribuição, e o regional que não mexe na RLS

Data. 2026-09-08 **Status.** Aceita

Contexto. O vocabulário de papéis eram quatro nomes — `estudante`, `professor`, `gestor`, `admin-contrato` — e a **ADR-34** havia decidido, com razão escrita, que o supervisor de uma diretoria **não** seria papel novo: ele é um `gestor` lotado nas escolas dela, porque "autorizar por hierarquia seria uma segunda fonte de verdade sobre acesso, divergente da lotação no dia em que alguém corrigisse um pai".

A operação pediu outra coisa. Uma rede estadual tem gente que trabalha no nível da diretoria e não tem lotação em unidade nenhuma, e o vocabulário não dizia de quem era a decisão de conceder cada nível de acesso.

Decisão (de produto, contra a recomendação técnica registrada). Seis papéis:

`estudante` · `professor` · `gestor_escola` · `gestor_regional` · `gestor_contrato` · `admin`

Dois são renome sem mudança de semântica (`gestor` → `gestor_escola`; `admin-contrato` → `gestor_contrato` — o hífen sai porque era o único do vocabulário, e um separador diferente dos outros é a diferença que ninguém lembra na hora de escrever a comparação). Dois são novos: `gestor_regional` cobre as escolas de uma regional sem lotação em nenhuma, e `admin` é a conta da Unooria.

A CADEIA DE ATRIBUIÇÃO. Cada nível atribui o de baixo:

`admin` → `gestor_contrato` → `gestor_regional` → `gestor_escola` → `professor`, `estudante`

Escrita como uma tabela de "o que EU atribuo diretamente", com o alcance real saindo do **fechamento transitivo** (`papeisQueCria`). As duas coisas separadas de propósito: a cadeia é a regra de negócio, que se lê numa olhada; o fechamento é a consequência. Escrever a lista já expandida em cada papel daria quatro cópias da mesma regra, e a divergência que ninguém percebe é a que dá acesso a mais.

O fechamento e não a cadeia estrita porque quem atribui um `gestor_regional` atribui, por definição, alguém que atribui `gestor_escola`: recusar o atalho não protege nada — ele criaria o regional e pediria a ele — e transformaria toda operação de balcão numa corrente de três telefonemas.

O que a decisão APERTOU, e não é regressão. Antes, um `gestor` criava outro `gestor` e um `admin-contrato` criava outro `admin-contrato`. Os dois deixaram de valer: um papel que se autoatribui não tem quem o conceda, e o alcance do segundo é a rede inteira de um contrato. Duas suítes e dois testes afirmavam o comportamento antigo e foram reescritos para afirmar a cadeia.

O `gestor_regional` NÃO toca a RLS, e é isso que o torna viável. Ele trabalha **uma escola por vez**, como o `gestor_escola`: o *token* continua carregando um `escola_id`, as 31 tabelas com `escola_id` e a política de RLS ficam intocadas, e a regra inviolável nº 1 continua literalmente verdadeira. O que a regional dele determina é a **lista de escolas que ele pode ativar** — e essa diferença mora em UM lugar, `escolasDaPessoa`, por onde o login, o seletor do cabeçalho e a validação da troca de escola já passam. Espalhada, faltaria em um dos três, e o sintoma seria um supervisor sem escola nenhuma para escolher.

A regional dele é `users.regional_id` — coluna, e não uma lotação apontando para a linha da regional. A **ADR-79** proibiu lotação em regional, e a razão continua valendo: ela apareceria no seletor do cabeçalho como uma escola sem turma, sem estudante e sem dado.

Alternativas descartadas.

- *Estender a política de RLS para aceitar um CONJUNTO de escolas* (`escola_id = ANY(...)` com as escolas da regional na sessão): é a leitura literal de "gestor regional", e faria toda tabela de dado de estudante depender de um segundo caminho de autorização — exatamente a superfície que a regra nº 1 existe para não ter. Se um dia o produto quiser a visão SOMADA da diretoria, é esta a alternativa, e ela é uma ADR própria.
- *Manter a **ADR-34** e resolver por lotação em lote*: era a recomendação técnica, e ela continua funcionando — o que ela não resolve é quem responde pela concessão. Decisão de produto foi ter o papel.
- *Dar ao `admin` capacidades próprias*: ele tem as mesmas de `gestor_contrato`. O que o distingue é de QUEM é a conta, não o que ela pode — e inventar capacidade para justificar o nome seria criar acesso sem ninguém pedir.
- *Escrever os cinco nomes em cada `@Roles`*: são quarenta decoradores. A rota esquecida continua compilando e continua respondendo 403, e não existe teste que aponte para isso — só uma tela que não abre. Daí os grupos `GESTORES_DA_ESCOLA` e `GESTORES_DO_CONTRATO` e o `ehDoContrato`, que substituiu as ∼20 comparações `perfil === "admin-contrato"` espalhadas pela API.

Consequências. *Ruim, e é a maior*: o renome atravessa TODO módulo que tem `@Roles` — 101 arquivos, e o *hook* de fatia acusou 31 deles em módulos que nada têm a ver com a frente (Vila, unooGPT, unooTube, Eco, Rastro, análise, salas, portabilidade). Vocabulário de papel não É fatia vertical, e é o tipo de mudança que a **ADR-71** diz para não paralelizar: com quatro *branches* abertas, esta conflita com todas.

Ruim: a rota `POST /usuarios/admin-contrato` virou `POST /usuarios/sem-escola`, e o corpo passou a levar `papel`. É mudança observável, e o motivo é a cadeia: a rota serve agora dois papéis (`gestor_contrato` e `gestor_regional`), e um caminho com o nome de um deles mentiria sobre o outro.

Ruim, e admitido: `apps/hub/lib/papeis.ts` passou a ter uma segunda cópia da cadeia, porque o *middleware* roda no *edge* e não importa da API. O que segura a divergência é o `Record` exaustivo de rótulos — papel novo sem rótulo não compila — e o fato de quem decide ser sempre a API, que responde 403 mesmo se a lista da tela mentir.

Bom, e não estava no pedido: a cadeia fechou um caminho real de "alterar o perfil silenciosamente". A chave única de `lotacoes` é (escola, pessoa, PAPEL), e `lotarEmLote` aceitava um `papel` divergente de `users.role` — criando uma SEGUNDA linha para a mesma pessoa na mesma escola, que a contagem da tela somaria nas duas colunas. Agora divergência é recusa. É também um candidato à divergência de contagem de professores relatada em separado.

Gatilho de reavaliação. O produto pedir a visão SOMADA das escolas de uma regional — aí a alternativa da RLS por conjunto volta à mesa, com ADR própria. Ou a Unooria precisar de capacidade que `gestor_contrato` não tem, que é quando `admin` deixa de ser um rótulo e passa a ser um papel.

18.82 ADR-81 — O panorama da diretoria SOMA, e isso não custou RLS

Data. 2026-09-08 **Status.** Aceita

Contexto. A **ADR-34** decidiu que o panorama de quem administra muitas escolas "LISTA, não soma — uma linha por unidade", porque somar escolas numa métrica só apresentaria a diretoria como unidade de análise, e naquele momento a hierarquia não autorizava nada.

Duas coisas mudaram. A **ADR-80** fez a hierarquia SER o escopo de um papel (`gestor_regional`), então a premissa da recusa deixou de valer para ele. E uma inspeção mostrou que a decisão já estava parcialmente vencida na prática: a TELA `/admin/minhas-escolas` sempre imprimiu uma linha de resumo somando turmas, estudantes e pendências — a rota devolve lista, mas a tela soma.

Uma avaliação de custo estava ERRADA, e vale registrar. Ao levantar a possibilidade, este projeto estimou que a visão somada exigiria estender a política de RLS "em 31 tabelas". Falso, e a verificação é simples: os números do panorama saem de `lotacoes`, `turmas` e `users` — as três tabelas que estão FORA da RLS por desenho (são lidas no login, antes de haver escola), consultadas com `escola_id IN (...)` explícito. Somar o que a API já computou é aritmética, não caminho de acesso novo. O custo de RLS existiria apenas para dado PEDAGÓGICO (`progresso`, `runs`, tabelas `exp_*`), que está sob RLS e é território do `/analise`.

Decisão. O panorama soma, e se intitula pela diretoria quando quem olha é um `gestor_regional`. O que ele soma:

- *turmas*, *estudantes* e *sem primeiro acesso* — somam LIMPO: o estudante tem UMA lotação ativa (índice único parcial), e o primeiro acesso é contado por `users.escola_id`, que é uma escola por pessoa.
- *vínculos de professores*, e não "professores": por escola o número é de PESSOAS, mas o total conta duas vezes quem dá aula em duas. É a mesma distinção que a tela de Escolas faz desde a **ADR-79**.

Para quem chega por LOTAÇÃO (um `gestor_escola` em várias escolas) a tela segue como era: o escopo dele é um conjunto de escolas que por acaso é plural, não uma diretoria, e ali a distinção da **ADR-34** continua fazendo sentido.

Um defeito que a decisão expôs. `resumoDasEscolasDaPessoa` lia `lotacoes` DIRETO, então o panorama de um `gestor_regional` — que não tem lotação nenhuma — voltava VAZIO, com o item de menu aparecendo, porque o menu conta as escolas da sessão (que vêm da regional). Tela em branco, sem erro nenhum, sem linha de log: o sintoma que ninguém sabe diagnosticar. As escolas passam a sair de `escolasDaPessoa`, que é onde a diferença entre lotação e regional já mora.

Alternativas descartadas.

- *Estender a RLS para um CONJUNTO de escolas*: era o que a avaliação errada supunha necessário. Continua sendo o caminho para a visão somada de dado PEDAGÓGICO, e continua sendo uma ADR própria — ela faz toda tabela de dado de estudante depender de um segundo caminho de autorização.
- *Somar na API, em vez de na tela*: a tela já tem os números por escola; somá-los de novo no servidor seria uma segunda fonte da mesma conta, e a divergência apareceria como dois totais diferentes na mesma página.
- *Mostrar a soma também para o `gestor_escola` de várias escolas*: a tela já fazia isso, e foi mantido — mas sem o título de diretoria, que ali seria falso.

Consequências. *Bom*: o papel criado pela **ADR-80** passou a ter uma tela que justifica a existência dele, e ela custou um `escolasDaPessoa` em vez de uma migração.

Ruim: o panorama agora tem dois modos — por diretoria e por lotação — e a diferença entre eles é um `papel === "gestor_regional"` na tela. É pouco código e é a única forma honesta: a mesma tela descrevendo dois escopos diferentes precisaria mentir em um dos dois.

Ruim, e não tem a ver com esta decisão: ao renomear o vocabulário, o array de papéis do `scripts/e2e/rodar.mjs` passou a apontar para pastas que não existiam (`gestor/` e `admin-contrato/`), e o `existsSync` do runner devolve lista vazia em silêncio — a suíte caiu de 29 roteiros para 19 sem nenhum aviso. As pastas foram renomeadas junto. Runner que pula suíte calado é pior que runner que quebra.

Gatilho de reavaliação. O produto pedir número PEDAGÓGICO por diretoria (missões concluídas, perfis, bem-estar) — aí a RLS por conjunto volta à mesa, e com ela a pergunta que a **ADR-34** fez primeiro: a diretoria é unidade de análise de dado de criança, ou só de operação?

18.83 ADR-82 — O relatório da diretoria atravessa escolas com um LAÇO, não afrouxando a RLS

Data. 2026-09-08 **Status.** Aceita

Contexto. A **ADR-81** deu ao `gestor_regional` a soma OPERACIONAL da diretoria (turmas, estudantes, primeiro acesso), e ela saiu de graça porque essas contagens vêm de `lotacoes`, `turmas` e `users` — tabelas que estão fora da RLS por desenho. O pedido seguinte foi o número PEDAGÓGICO: quantas crianças da diretoria têm registro em cada missão, e em quantas turmas a missão foi liberada e encerrada.

Esses dados vivem em `liberacoes` e `progresso`, que estão SOB a política por escola. Cada consulta vê uma escola.

Decisão. O serviço abre a tranca **uma escola por vez** e soma na aplicação, em lotes de 8 (`aosPares`). A política de RLS não é tocada.

Não é padrão novo: é o mesmo que `lotarEmLote` faz (uma transação por escola, porque cinquenta numa só seguraria conexão por segundos) e o que `transferirEstudante` faz com `comEscolaNaTransacao`. A diferença é o propósito — ali é escrita, aqui é leitura agregada.

A autorização fica em UM lugar, e ele é um método. `regionalDoRelatorio` responde "qual regional este pedido pode ler": para `gestor_regional` vem de `users.regional_id` e o parâmetro `?regionalId=` é **IGNORADO**. Ignorado, e não validado contra o dele — o efeito é o mesmo, mas por um caminho em que esquecer a comparação não vira "leia a diretoria do vizinho por *querystring*". Nenhuma escola entra no laço sem ter saído do `pai_id` da regional autorizada.

As DUAS medidas, lado a lado. `estudantes` é a CRIANÇA com registro na missão; `turmasLiberadas`/`turmasE` é a AULA. Foi pedido explicitamente que as duas aparecessem, e a razão é que a diferença entre elas é a informação: turma encerrada com poucos registros é aula que passou sem a sala entrar; muitos registros com turma aberta é sala no meio do caminho. Uma coluna só apagaria os dois casos.

Alternativas descartadas.

- *Estender a política para aceitar um conjunto* (`escola_id = ANY(...)`): é UMA linha, em laço sobre as ∼29 tabelas — mecanicamente barato, e foi por isso que a estimativa anterior deste projeto errou nas duas direções (disse "31 tabelas" como se fosse trabalho manual, e tratou o custo como se fosse de digitação). O custo real não é escrever: é que a política É a garantia de que a escola A nunca vê a criança da escola B, e passar a aceitar lista significa que todo dado de estudante depende de o *caller* montar a lista certa. O laço mantém a garantia literal.
- *Agregado materializado* (tabela ou *view* escrita por rotina): resolve o teto de escala, e é o caminho para quando o laço ficar lento. Não entrou agora porque agregado tem que ser invalidado, e rotina de invalidação é a peça que este projeto já viu ficar sem dono (`expires_at`).
- *Somar na API o que a tela já tem*: a soma OPERACIONAL fica na tela (**ADR-81**) porque os números por escola já estavam lá. A pedagógica não estava, então ela nasce somada no servidor — somar nos dois lugares seria duas fontes da mesma conta.

O que ficou de fora, e não é omissão.

- *Bem-estar*: dado de saúde, sem rota de gestor nenhum (nem do contrato) por decisão registrada. Não é "falta esforço", é uma linha que o produto não atravessa.
- *O retrato de valores da Vila agregado por diretoria*: recomendado NÃO, e a recomendação foi aceita. A listagem de turma da Vila já não mostra valor de Schwartz nenhum, porque listagem ordenável vira ranking de crianças; o agregado por diretoria é o mesmo risco com outro nome.
- *Nome de criança*: o relatório é CONTAGEM. Para ver uma criança, o supervisor abre a escola dela. Uma coluna de nome aqui seria a lista ordenável de crianças de várias escolas que a **ADR-49** e a **ADR-34** recusam.

Consequências. *Ruim, e é o teto*: uma diretoria de 200 escolas são 200 transações. Em lotes de 8, contra Cloud SQL, isso é ordem de segundos — e o número 8 é o mesmo que o *seed* usa por túnel IAP, baixo de propósito para não trocar espera de rede por espera de conexão. Ficar lento é o gatilho do agregado materializado, NÃO de afrouxar a política.

Ruim: o relatório só sabe o que `liberacoes` e `progresso` sabem — que a criança TEM registro na missão, não o que ela fez nela. Profundidade por experiência (o retrato da Vila, a transcrição do unooGPT) não agrega para a diretoria sem uma decisão por experiência, e cada uma tem regra própria.

Bom: como as duas tabelas são da PLATAFORMA, o relatório cobre as nove experiências sem uma linha de código por experiência — e a décima entra de graça.

Gatilho de reavaliação. O laço passar de poucos segundos numa diretoria real (aí: agregado materializado), ou o produto pedir profundidade por experiência no nível da diretoria (aí: uma decisão por experiência, começando pela pergunta que a **ADR-34** fez primeiro — a diretoria é unidade de análise de dado de criança, ou só de operação?).

18.84 ADR-83 — O header ancora na JANELA, não na coluna de conteúdo

Data. 2026-09-08 **Status.** Aceita

Contexto. O `HeaderPlataforma` é um só para todos os papéis (**ADR-78**) — mesma altura, mesmos elementos, da tela do estudante à área do contrato. Mas ele nascera com um parâmetro `largura` (`5xl` | `6xl` | `cheia`) cujo comentário dizia, com todas as letras, que os valores "casavam com o `max-width` do `<main>` de cada área". Como o miolo é centralizado (`mx-auto`), a coluna da área virava a POSIÇÃO da marca.

O dono do produto relatou o sintoma sem a causa à vista: "a marca desliza para a direita em algumas páginas". A medição, feita com Playwright e logada em cada papel, mostrou **quatro** posições diferentes:

superfície	@1440px	@1920px
`/admin/**`, `/exp/:expId`, `/dev-embed`	20px	20px
`/analise`	164px	404px
`/professor/**`, `/estudante`	228px	468px

208px de salto num monitor de 1440px; **448px** em 1920px. E um segundo defeito, embutido: em `/analise` o header usava `px-5` e o `<main>` `px-6`, então a marca ficava 4px à esquerda do título da própria página — na única área onde a largura supostamente casava.

Decisão. O miolo do header é **full-bleed**: sem `max-width`, sem `mx-auto`. O parâmetro `largura` deixou de existir. O gutter é **24px** (`px-6`) no chrome E na coluna de conteúdo de todas as áreas — uma constante `GUTTER` no header, e o mesmo `px-6` nos layouts.

O princípio: **o header pertence à janela, não à página**. Ele é chrome do hub, e chrome que se move ao navegar é ruído que o usuário sente antes de conseguir nomear.

Alternativas descartadas.

- *Unificar a coluna de todas as áreas e manter o header casado com ela*: não converge, e não é questão de esforço. `/admin` tem navegação lateral e `/exp` é aparelho em tela cheia — essas áreas jamais terão a coluna de `/professor`. O resultado seria trocar quatro posições por duas, mexendo na largura de ∼30 telas para não resolver o problema.
- *Manter o parâmetro e só corrigir os 4px de `/analise`*: apaga o sintoma menor e deixa o salto de 448px, que é o que foi relatado.
- *Gutter de 20px (`px-5`, o do header antigo)*: 24px já era o de `/admin`, de `/analise` e da nav pública — escolher o valor da minoria obrigaria a mexer em mais lugares para o mesmo efeito.

O que ficou de fora, e é decisão, não omissão. A nav da superfície PÚBLICA (`/`, `/privacidade`, `/termos`) mantém miolo de 1440px centralizado e `Logo size="md"`: em 1920px a marca dela fica em 264px, contra os 24px do app. É outra superfície, com outro público e outra linguagem, e mexer nela é decisão de marketing. Quem reabrir: a mudança é trocar o `max-width` do `.landing-nav__miolo` por `none`.

Consequências. *Ruim, e é o preço declarado*: em telas largas a marca não alinha mais com o título da página nas áreas de coluna estreita — 24px contra 472px em `/professor` num monitor de 1920px. É o arranjo de *app shell* que `/admin` e `/exp` já usavam; a alternativa era o salto.

Ruim: a marca ainda muda de posição na transição público → logado, enquanto a nav pública ficar como está.

Bom: a marca e o menu da conta ficam no MESMO X em todas as superfícies logadas — medido em 1920px, 1440px e 900px, nas cinco áreas mais o `/dev-embed`. Em telas estreitas header e conteúdo passam a começar no mesmo X (24/24 em 900px), que antes só acontecia por coincidência em duas áreas.

Bom: área nova nasce alinhada sem decidir nada. Não há mais parâmetro para escolher errado — e o parâmetro era a única peça que exigia que quem cria um layout soubesse desta regra.

Gatilho de reavaliação. A superfície pública ser alinhada ao app (aí o gutter passa a ser um só do primeiro clique ao logout), ou o design system publicar um componente de barra de aplicação — que é onde o `GUTTER` deveria morar, em vez de numa constante do hub.

18.85 ADR-84 — Sinal do navegador sem sessão, e o aparelho como rótulo fechado

Contexto. A **ADR-58** pôs o erro de cliente entrando pela própria API, com `OidcGuard`, e isso resolveu a experiência dentro do *iframe*. Ficaram de fora as telas por onde **todo mundo passa**: a *landing*, o *login* e a troca de senha. Sem sessão, o BFF do hub responde 401, e o erro morria no console de quem não tinha conseguido entrar — "ninguém está conseguindo entrar" era, literalmente, o defeito mais invisível da plataforma. A suíte de ponta a ponta chegou a declarar o 401 do Web Vital dessa tela como *pendência conhecida*, o que é a forma educada de conviver com um sinal descartado.

Em 8 de setembro de 2026 a investigação de "por que não vejo nenhum erro no Cloud Logging" mediu o resto do estrago, e ele era maior: **nenhum erro de cliente jamais foi registrado**, em nenhum ambiente. O `NEXT_PUBLIC_HUB_VERSAO` é o `github.sha` — 40 caracteres — e o campo `versao` do schema aceitava 32, então *todo* erro do hub voltava 400. O mesmo valor é o `service.version` do lote OTLP, que o *proxy* valida com `{1,32}`: 425 *spans* do hub foram descartados em `hml` entre 5 e 7 de setembro **com a rota respondendo 204**. Nada disso tinha contador: o único vestígio era uma linha `corpo-invalido` que ninguém tinha motivo para procurar. Local, o *default* do *Dockerfile* é `dev`, com três caracteres — e é por isso que *typecheck*, teste e suíte e2e passavam.

Decisão. Três partes.

Primeira: existe um par **público** das rotas de sinal — `POST /observabilidade/publico/erros-cliente` e `POST /observabilidade/publico/vitals` —, sem `OidcGuard`. Quem escolhe o destino é o **BFF**, não o cliente: com sessão vai para a rota autenticada, e o log ganha o id do usuário; sem sessão vai para a pública, sem identidade. O cliente posta sempre no mesmo endereço e nunca vê um 401. A rota pública força `expId = hub` (antes do `hub:init` nenhuma experiência conhece a API), não aceita id de pessoa, e é limitada por **IP** — cinco erros por minuto — **e** por um teto global de instância.

Segunda: o campo `versao` passa a aceitar 64 caracteres nos dois pontos, e o CI manda o **SHA curto** (12). São as duas metades do mesmo conserto: o SHA curto arruma hoje, o teto maior impede que o tamanho de uma versão volte a decidir se um erro existe.

Terceira: erro e vital passam a carregar o que o navegador sabe do aparelho — viewport, *device pixel ratio*, `effectiveType` da conexão, memória e núcleos —, e a API deriva do *user-agent* três rótulos **fechados** (`navegador`, `so`, `tipo`). Só `tipo` vira rótulo de métrica; `navegador` e `so` ficam no log.

Alternativas descartadas.

- *Só corrigir o SHA e parar por aí.* É uma linha e resolve o erro autenticado. Descartada porque deixa a *landing* e o *login* tão cegos quanto antes — e é lá que um erro impede o uso do produto inteiro.

- *O cliente tentar a rota autenticada e, no 401, repetir na pública*. Não exige o BFF decidir nada. Descartada porque o 401 continua acontecendo: continuaria sujando o console, e a *pendência conhecida* da suíte teria de continuar existindo para tolerá-lo.
- *Cota por IP em tudo*. O `comum/rate-limit.ts` tem a lição oposta escrita nele: a chave certa é o usuário, porque uma turma inteira sai pelo mesmo NAT da escola. Aqui ela não se aplica — não há usuário, é o que define a rota, e estourar a cota não derruba ninguém: descarta amostra de telemetria. Cinco erros por minuto de uma escola bastam para saber que a tela quebrou.
- *`navegador` e `so` como rótulo de métrica também*. Seria a resposta mais direta a "trava no Chromebook?". Descartada por cardinalidade: 2 superfícies $\times$ $\sim$20 rotas $\times$ 5 perfis $\times$ 6 navegadores $\times$ 3 tipos são 3.600 combinações por histograma, e histograma multiplica por *bucket*. Com só o tipo são 600, e o navegador continua no log, onde não custa série.
- *Uma biblioteca de user-agent (`ua-parser-js`)*. Descartada porque a saída dela é aberta — voltaríamos ao problema da cardinalidade — e porque distingue aparelhos que não existem numa escola pública brasileira.

Consequências. *Ruim, e é o principal*: existe agora uma rota da API **sem autenticação** que escreve no log. A contenção é toda estrutural — Zod `.strict()`, corpo pequeno, `expId` forçado, *redaction*, cota por IP e teto global, e nenhuma linha de log por requisição recusada — mas a superfície de abuso deixou de ser zero, e isso precisa constar de qualquer revisão de segurança. *Ruim*: a combinação de memória, núcleos e tela aproxima-se de impressão digital de aparelho; a plataforma atende menores, então a coleta entra na conversa de DPA com a rede, e a lista é fechada de propósito — ela não cresce sem uma ADR nova. *Ruim*: o erro anterior ao *handshake* continua sem rastro, porque ali não há token nem `apiUrl` — o que existe é a ausência de `exp:ready`, e o hub ainda não a observa. *Bom*: a falha de *boot* de uma experiência passou a reportar (`exp?.reportarErro` no `catch` dos seis `bootstrap.tsx`), e era ela que o estudante via como "não deu para abrir agora" sem deixar sinal. *Bom*: todo caminho de recusa agora **conta** — inclusive o corpo inválido, cuja ausência de contador é a razão pela qual seis dias se passaram sem ninguém notar.

Gatilho de reavaliação. A rota pública aparecer em varredura de abuso, ou o teto global passar a ser atingido em operação normal — nos dois casos a saída é exigir um token de baixa confiança emitido pela própria *landing*, que hoje não existe e não se justifica. E, do lado da coleta, qualquer rede pedir para restringir o que se mede do aparelho: a lista encolhe sem quebrar nada, porque todo campo é opcional.

18.86 ADR-85 — Trocar de escola remonta o miolo da área

Contexto. Quem trabalha em mais de uma unidade — professor com duas cadeiras, coordenação de duas escolas, supervisor de uma diretoria inteira — escolhe a **escola ativa** no cabeçalho. A troca grava no servidor (`POST /api/escola-ativa`), que valida contra a API e reemite o co-

okie de sessão; a tela então chamava `router.refresh()`, e o comentário do seletor explicava por quê: "as telas são Server Components e leem a escola da sessão".

Essa frase era verdadeira quando foi escrita, e deixou de ser sem que ninguém percebesse. O `router.refresh()` re-renderiza **apenas** os Server Components. Ele não remonta um Client Component que já está montado, não re-executa os `useEffect` dele (as dependências não mudaram), não reinicializa um `useState` alimentado por *prop* do servidor e não recarrega um `<iframe>`. Toda a área administrativa é client, e as telas de produto do professor são *embed*.

Medido em 9 de setembro de 2026, com uma conta lotada em duas unidades: em `/admin/usuarios` o cabeçalho passava a dizer o nome da escola nova enquanto a tabela seguia com as 49 pessoas da anterior — as 1004 da nova só apareciam com F5; `/admin/turmas` ficava nas 27 turmas da outra unidade; e em `/professor/plano-de-aula` o hub já emitia *token* de `escola-demo-2` com o *iframe* ainda segurando o de `escola-demo`.

Os dois primeiros são tela desatualizada, e a RLS contém o estrago: a ação disparada de uma linha velha viaja com o *token* novo, a API não encontra a linha e a pessoa recebe um erro genérico. O terceiro é de outra natureza. Numa tela de *embed* o *token* viaja **uma** vez, no `hub:init` que responde ao `exp:ready`: sem remontar o aparelho, a experiência continua gravando na escola de onde a pessoa acabou de sair, com um *token* legítimo, que a RLS aceita sem reclamar. Não é leitura errada — é **escrita** na escola errada.

O defeito sobreviveu porque o teste tinha a mesma cegueira do produto: a suíte `gestor_escola/02-quem-tra` trocava a escola e navegava em seguida, e navegar é o próprio F5 que mascara o sintoma. Ela provava que o *token* mudava, e nunca olhava a tela onde a troca acontecera.

Decisão. O layout de cada área envolve o miolo em `<EscopoDaEscola>` (`apps/hub/components/escopo-da-e` um `Fragment` cuja `key` é a escola ativa da sessão. Quando a escola muda, o React desmonta a subárvore inteira e monta outra — estado zerado, efeitos refeitos, *iframe* recarregado, *handshake* refeito com o *token* certo.

Duas escolhas dentro da decisão. *É do layout, nunca da página*: mesma razão da trilha da **ADR-77** — página nova nasce escopada sem escrever uma linha, e as quatro telas quebradas eram quatro esquecimentos independentes do mesmo detalhe. *Envolve o miolo, não o cabeçalho*: o cabeçalho hospeda o próprio seletor, e remontá-lo tiraria o foco do teclado de quem acabou de trocar de escola pelo campo.

Alternativas descartadas.

- *`key` por página, nas quatro telas quebradas*. É o conserto mínimo e resolve o que foi medido. Descartada porque devolve a responsabilidade a quem escreve a próxima página — exatamente o desenho que produziu as quatro. A **ADR-77** já pagou esse preço com o *breadcrumb*: 22 telas com trilha e 15 sem, nenhuma por decisão.
- *Trocar o `router.refresh()` por `location.reload()`*. Uma linha, cobre tudo, inclusive as telas que ainda não existem. Descartada por ser um recarregamento completo a cada troca — perde a navegação sem recarga em toda a plataforma para resolver um problema de quatro

telas —, e por apagar o rastro do que a pessoa estava fazendo. Fica registrada como plano B se a fronteira por `key` se mostrar insuficiente em alguma área.

- *Reenviar `hub:token` ao iframe quando a escola muda, sem remontá-lo.* Trata o caso grave sem recarregar a experiência. Descartada porque o estado **dentro** da experiência continua sendo o da escola anterior — a lista de planos, o rascunho aberto, o que estiver em memória — e o resultado seria uma tela mostrando dados de uma escola com um *token* de outra: uma inconsistência mais difícil de perceber do que a que se estava corrigindo.
- *Escopar a escola pela URL, como faz `/analise`.* Torna o estado visível e compartilhável. Descartada porque a escola ativa é **do token**, não da tela: pô-la na URL criaria uma segunda fonte de verdade sobre a fronteira de *tenancy*, e as duas divergiriam no primeiro *link* colado de uma sessão para outra.

Consequências. *Ruim*: trocar de escola agora descarta o estado de tela — filtro digitado, página da paginação, formulário pela metade. É deliberado, e é o comportamento correto: um filtro de nome continua fazendo sentido, mas uma página 7 de uma lista de outra escola não. *Ruim*: numa tela de *embed* a troca custa o recarregamento da experiência, com o tempo de *boot* dela. *Ruim*: a fronteira é invisível no código da página — quem lê uma tela de `/admin` não vê nada que explique por que ela recarrega sozinha; o que amarra isso é este registro e a seção correspondente do `apps/hub/AGENTS.md`. *Bom*: área nova nasce correta, e a suíte de ponta a ponta passou a provar as duas metades na **mesma** tela — a lista rebuscada sem F5, e o aparelho da experiência remontado (marca-se o `<iframe>` e confere-se se a marca sobreviveu à troca).

Gatilho de reavaliação. Uma área precisar preservar estado de tela através da troca de escola — um formulário longo, um rascunho não salvo —, caso em que a fronteira desce daquele layout para dentro dele e a exceção se escreve aqui. E o surgimento de uma tela que precise comparar duas escolas lado a lado: ela não caberia neste desenho e exigiria a discussão que a **ADR-34** recusou.

18.87 ADR-86 — O guia da aula mostra o conteúdo da missão, não o descreve

Data: 2026-09-09 **Estado:** Aceita

Contexto. O guia da aula (**ADR-65**) é o material com que o professor conduz uma missão do Entre Redes. Três das seis missões publicadas já injetavam **blocos vivos** — conteúdo que o guia não guarda como texto, porque tem fonte de verdade em outro lugar: os momentos e as telas da revelação do Instanooria, os cartões da Vila do Lume por turma, a tabela dos vieses do unooGPT. As outras três — UnooTube, O Rastro do Boato e O Eco no Grupo — **descreviam** em prosa o que a turma lê na tela.

Descrever parecia inofensivo, e não é. Uma varredura pelo texto exato do catálogo de cada missão dentro do guia correspondente devolveu **zero** ocorrências nas três: nenhuma das opções que o estudante escolhe, nenhum título de vídeo, nenhuma reação do grupo aparecia com as

palavras que a criança lê. O que havia era paráfrase — *"a opção que respondeu no grupo em vez de sair dele"*, *"a opção que funciona cita a matéria sem atacar ninguém"* — e ela falha de dois modos distintos.

Primeiro, a paráfrase diverge. O guia do UnooTube dizia *"dos 21 vídeos, metade cita fonte verificável … a outra metade é sensacionalista"*. A divisão real do catálogo é **11 e 10**, porque 21 é ímpar. Ninguém escreveu isso errado de propósito: a frase foi escrita quando era verdadeira o bastante, o catálogo mudou, e nada obrigava as duas coisas a concordarem. É a mesma armadilha do `expires_at` sem rotina que a **ADR-13** registra — a regra sai do código, o registro fica, e quem lê acredita nele. Os números da pontuação (`+1000`, `+50`, `3` salvos) estavam copiados à mão pelo mesmo caminho.

Segundo, e mais grave, a paráfrase apaga o que sustenta a lição. No Rastro do Boato as opções A e C produzem a **mesma** reação do grupo — encaminhar amplifica ativamente, apagar para si mesmo deixa a corrente correr sem você, e as duas terminam com o Pedro anunciando que já recebeu em mais três grupos. É a peça central da missão: a inação alimenta a mesma curva. Lado a lado, a coincidência salta aos olhos; descrita, simplesmente não existe. No UnooTube, o professor não tinha como reconhecer o vídeo que a criança de nove anos citava — e ela cita pelo apelido do canal, não pelo tema.

Decisão. Toda missão publicada tem bloco vivo, e o conteúdo que a turma lê na tela chega ao guia **como é**, de uma fonte só.

Onde essa fonte mora depende de onde o conteúdo já vive, e as duas formas coexistem sem conflito:

- *Copy que roda dentro do bundle da experiência* — as telas da revelação do Instanooria — vive em **duas cópias gêmeas** de um arquivo de TypeScript puro, uma na experiência e outra no hub, cada uma compilada para o seu bundle. Existe assim porque a revelação abre no meio da aula: texto que depende de rede é texto que pode não aparecer. Foi um pacote de *workspace* (`@edu/conteudo-exp08ef1b`) até 2026-09-09, e a duplicação é decisão do dono do produto — a copy está fechada, e o pacote cobrava o custo de existir sem ter o que proteger. **O que segura a igualdade hoje é o cabeçalho dos dois arquivos, e nada mais.**
- *Conteúdo com gabarito, que já vive na api* — e é o caso das três missões desta decisão, porque a regra inviolável nº 3 mantém resposta certa, correção e pontuação no servidor — vem de uma rota **professor-only** do módulo da missão: `GET /unootube/professor/videos`, `GET /rastro-boato/professor/material` e `GET /o-eco-no-grupo/professor/material`, no molde de `GET /unoogpt/professor/fontes`, que já era assim.

As três rotas são catálogo, não dado de turma: sem `escola_id`, sem *tenancy* a aplicar e sem `?escolaId=` para o `gestor_contrato` informar. O que a guarda protege é o **papel** — por isso vivem em *controller* separado do de turmas, com `@Roles("professor", ...GESTORES)`. E devolvem justamente os campos que `semGabarito` retira antes de a opção chegar ao estudante: o que muda entre as duas superfícies é o leitor.

Duas escolhas dentro da decisão. *Número que o guia mostra é calculado do catálogo*: a rota do UnooTube devolve `resumo` (total, factuais, sensacionalistas) e `pontuacao` contados na origem, e a faixa de alerta no topo do guia perdeu os dígitos que repetia. *O guia abre inteiro com a api fora do ar*: cada bloco tem o próprio cartão de "não carregou", porque as outras nove seções continuam servindo — é o mesmo tratamento que o bloco do unooGPT já tinha.

Alternativas descartadas.

- *Copiar o conteúdo para dentro do guia, como texto*. Resolve a leitura e é o caminho mais curto. Descartada por ser exatamente o defeito que se está corrigindo, com um passo a mais: duas cópias literais divergem mais depressa que uma paráfrase, porque quem revisa a copy da experiência não tem motivo nenhum para abrir o guia.
- *Um pacote de workspace por missão, como o que a revelação do Instanooria usou até 2026-09-09*. Uniformizaria o desenho. Descartada porque o conteúdo destas três **não pode** ir para um pacote: ele carrega gabarito, e pacote de workspace é lido em tempo de *build* pelos dois consumidores — o bundle da experiência entre eles. A resposta certa chegaria ao JavaScript público, violando a regra inviolável nº 3. O pacote servia à revelação do Instanooria justamente porque ali não há gabarito nenhum: é copy que a criança vai ler de qualquer jeito.
- *Reaproveitar as rotas de estudante que a experiência já consome*. Nenhuma rota nova. Descartada porque elas passam por `semGabarito` por construção — é o desenho que impede o vazamento — e devolveriam ao professor exatamente a tela que a criança vê, sem a única informação que ele precisa ter e ela não.
- *Manter a prosa e prendê-la ao catálogo com um teste*. Barato, e pegaria a divergência dos números. Descartada porque não pega a segunda falha, que é a que importa: nenhum teste teria detectado que a paráfrase apagava a coincidência entre A e C. Ela não estava errada — estava incompleta.
- *Servir o conteúdo pelo bundle da experiência, deixando o hub buscá-lo do CDN*. Descartada de saída: o bundle publicado é JavaScript minificado num caminho versionado do *bucket*, e extrair texto de dentro dele seria criar uma dependência entre o hub e a saída de um compilador.

Consequências. *Ruim*: o guia da aula passou a depender da api em cinco das seis missões — com ela fora, o professor lê nove seções e um cartão de erro no lugar da décima. Aceito: ele lê o guia no preparo, sentado, e não no meio da aula com a turma esperando; a única superfície que não pode depender de rede é a revelação, e é por isso que ela continua num pacote. *Ruim*: três rotas novas para manter, documentar e cobrir — o custo de qualquer superfície nova. *Ruim*: o professor agora tem o gabarito à mão numa tela que ele pode projetar por engano, e o texto de cada bloco precisa dizer "não projete" com todas as letras, como o do unooGPT já dizia. *Bom*: a divergência que existia foi corrigida e não pode voltar por esquecimento — os números são contados na origem. *Bom*: o professor do UnooTube reconhece o vídeo que a criança cita, e o do Rastro do Boato vê que A e C terminam iguais antes de entrar na sala.

Gatilho de reavaliação. Uma missão cujo conteúdo de preparo seja grande demais para uma requisição de página — caso em que ele desce para o *bundle* do hub por geração estática, e não

para prosa escrita à mão. E a primeira experiência que precise mostrar ao professor conteúdo **gerado por IA** para aquela turma: ali o bloco vivo deixa de ser catálogo e vira dado de turma, com *tenancy*, e a decisão de onde ele é gerado e por quanto tempo vale precisa ser tomada à parte.

18.88 ADR-87 — Disponibilidade: um terceiro eixo, com nome próprio, e o produto como teto

Data: 2026-09-09 **Estado:** Aceita

Contexto. Duas coisas decidiam se uma missão abria para uma criança: o **catálogo** (`experiencias`: série, unidade, público — que forma a experiência tem) e a **liberação por turma** (`liberacoes`, **ADR-69** — se o professor a abriu naquela sala). Nenhuma das duas responde à pergunta que um contrato B2G faz: *este produto existe nesta rede?*

A resposta vivia em duas constantes compiladas no *bundle* do hub: o *array* `PRODUTOS` de `apps/hub/lib/produ` e o campo `status: 'disponivel' | 'em-breve'` do catálogo do Entre Redes. Ou seja: desligar o jogo numa rede era editar TypeScript e fazer *deploy* — e fazê-lo para UMA rede era impossível, porque a constante é a mesma para todas as *stacks* que rodam a mesma imagem. Duas necessidades concretas não tinham como ser atendidas: a rede que não contratou um produto o vê no cartão do professor, e a experiência que quebra num *deploy* só sai do ar com outro *deploy*.

Decisão. Existe um TERCEIRO eixo, chamado **disponibilidade**, gravado em `disponibilidades` e operado pela Unooria:

Eixo	Pergunta	Quem decide
catálogo	que forma a experiência tem?	o manifesto / o CI
liberação (**ADR-69**)	está aberta NESTA TURMA?	o professor
disponibilidade	existe NESTA REDE?	a Unooria (`admin`)

Disponibilidade é **pré-condição** de liberação, nunca o contrário. O nome é próprio de propósito: os três eixos respondem "a missão abre?" e confundi-los é o defeito caro — um produto desligado tem o sintoma de um bug de enturmamento.

Uma linha responde por um **recurso** num **escopo**. O recurso é `produto:<slug>` ou `experiencia:<expId>`; o escopo é a escola, a regional ou o contrato inteiro (`escola_id` nulo). Escola e regional são a mesma tabela `escolas` distinguida por `tipo`, então uma coluna nulável cobre os três níveis sem hierarquia nova.

Duas regras de resolução, e as duas são decisão de produto:

1. **O escopo mais específico ganha**: escola → regional → contrato → padrão do código.

2. **O produto é um TETO**: produto desligado no escopo que vale ⇒ nada dentro dele liga, mesmo com linha própria dizendo `ligado`.

E três escolhas menores que o desenho depende de fixar:

O padrão é LIGADO, o contrário de `liberacoes`. Lá a ausência de linha é o bloqueio ("o mundo nasce fechado de graça"); aqui a ausência é o mundo normal, porque este interruptor é mecanismo de **exceção** — não foi contratado, ou quebrou — e não de liberação. Nascer desligado exigiria uma linha por recurso a cada contrato provisionado, e toda experiência nova ficaria invisível até alguém lembrar de ligá-la.

A linha inteira é a decisão: `ligado` e `versao_fixa` não se resolvem em cadeias separadas. Resolver campo a campo daria mais flexibilidade e produziria estados que ninguém explica olhando a tela — a linha que o operador gravou deixaria de ser o que está valendo.

A composição produto → experiências é DERIVADA do manifesto, por uma coluna `experiencias.produto` que o `public/experiencia.json` declara. É isto que faz o painel se auto-estender: experiência nova aparece nele sem ninguém editar lista nenhuma. Quem cobra o campo é um teste que varre todos os manifestos do monorepo, e não o Zod da rota — um 400 no `PUT /experiencias/{expId}` mataria o *job* `ativar-exps` DEPOIS de os *bundles* já terem subido, com a `versao_ativa` na versão antiga.

Alternativas descartadas.

- *Reusar `liberacoes` com um estado novo* (`indisponivel`). Seria uma tabela a menos. Descartada porque `liberacoes` é por TURMA e o dono dela é o professor: desligar um produto na rede exigiria escrever uma linha por turma de cada escola, e a tela do professor passaria a mostrar, no mesmo lugar, uma decisão que não é dele e que ele não pode desfazer.
- *Deixar a experiência sobrepor o produto* (exceção por experiência vencendo o teto). Mais expressivo. Descartada porque faz "desliguei o Entre Redes para a rede" não significar o que diz: o operador teria de caçar exceções antes de confiar na tela. Religar o produto no escopo menor continua sendo o caminho, e é explicável.
- *Um campo booleano em `experiencias`* (`ativo`). O mais simples de todos. Descartada porque `experiencias` é escrita pelo CI a cada *publish*: o *deploy* seguinte sobrescreveria o desligamento feito à mão — e o pior momento para isso é justamente o *deploy* de correção. Além de não ter onde pôr o escopo nem o produto.
- *Flags num serviço de terceiro* (LaunchDarkly e similares). Descartada pela mesma razão que Redis está fora sem gargalo medido: um contrato é uma *stack* isolada, o dado cabe em dezenas de linhas, e um provedor externo põe uma dependência de rede no caminho de toda página.

Consequências. *Bom*: desligar um produto numa rede deixou de ser *deploy*, e *rollback* de experiência deixou de ser CLI. *Bom*: a tabela `experiencias_versoes` que nasceu com esta decisão conserta um defeito que ninguém havia notado — `publicar-manifesto.mjs --versao x.y.z` manda o número antigo com o manifesto ATUAL do repositório, então voltar a versão não vol-

tava `viewport` nem `permissoes`: a app-70 reabria sem `allow-pointer-lock`, a criança entrava na Vila e a câmera não girava, sem erro em lugar nenhum. Guardando o manifesto inteiro por versão, o *rollback* passa a ser fiel.

Ruim: nasceu uma alavanca de incidente. Um clique no escopo-contrato desliga um produto para os ~10 mil estudantes de uma rede, e o sintoma que ele produz — a missão desapareceu da grade — é **indistinguível** de um defeito. As mitigações são modal de confirmação, `motivo` obrigatório ao desligar (CHECK no banco, não só Zod) e o registro estruturado `recurso-alterado`; o sintoma entra no apêndice de operação. *Ruim*: um terceiro eixo é um terceiro lugar onde procurar quando algo não abre. *Ruim*: `disponibilidades` fica FORA da RLS, contra o hábito da regra inviolável nº 1 — é configuração da rede, não dado de estudante, e a linha de escopo-contrato tem `escola_id` nulo por desenho, de modo que uma política por `escola_id` esconderia justamente a resposta. A exceção é provada por teste, para que a próxima migração não a "conserte": ligar RLS ali faz TODO produto parecer ligado.

Aceito com nome: desligar não interrompe quem já está dentro. O *bundle* carregou, a api começa a recusar, e a experiência cai no estado de erro dela — a mesma degradação que a liberação `encerrada` já tem. E a telemetria de sessão em voo continua sendo ACEITA: recusá-la perderia o registro do que já aconteceu.

Gatilho de reavaliação. O primeiro pedido de "ligar para 10% das turmas" ou de janela de validade ("desligado até sexta"): ali disponibilidade deixa de ser um interruptor por escopo e vira segmentação, e segmentação com data é outro desenho. E o dia em que a pergunta "quem desligou isto, e quando?" precisar de TELA — hoje ela tem resposta no registro e nas colunas da linha, que é sobrescrita a cada mudança, e a resposta em tela pede uma tabela de auditoria *append-only*.

18.89 ADR-88 — O painel da Unooria é só do `admin`: a primeira assimetria com o `gestor_contrato`

Data: 2026-09-09 **Estado:** Aceita

Contexto. A **ADR-80** criou seis papéis e disse, com todas as letras, que `admin` e `gestor_contrato` têm **as mesmas capacidades** — "o que os distingue é de quem é a conta, não o que ela pode fazer". Os dois dividem a área `/admin` do hub, e quem os separa item a item é o `RolesGuard` da api.

O painel de disponibilidade (**ADR-87**) rompe isso pela primeira vez. Ligar e desligar produto não é operação de quem administra o contrato **do lado do cliente**: é decisão comercial e operacional da fornecedora. Um `gestor_contrato` que pode desligar o Entre Redes pode derrubar a rede inteira por engano; e um que pode LIGAR passa a ter acesso a produto que a rede não contratou.

Decisão. O painel vive numa área própria, `/unooria/*`, restrita ao papel `admin` — e esta é a primeira capacidade que `admin` tem e `gestor_contrato` não. A afirmação de igualdade da **ADR-80** passa a valer para **tudo menos** a disponibilidade de recursos.

Área própria, e não um item escondido dentro de `/admin`, por três razões. *Honestidade*: o painel da fornecedora ao lado das telas da escola convida ao clique errado. *Trilha*: pela regra do hub, área nova exige *layout* novo, e é o *layout* que desenha a trilha — foi por não ter *layout* próprio que `/admin` nasceu sem trilha nenhuma (**ADR-77**). *Crescimento*: o que a Unooria opera sobre uma rede não termina no interruptor, e um item solto viraria a segunda área por acidente.

Alternativas descartadas.

- *Manter a igualdade e dar o painel aos dois papéis*. Preserva a **ADR-80** intacta e é o caminho mais curto. Descartada porque entrega ao cliente o interruptor do que ele compra — inclusive o de ligar o que não comprou.
- *Item escondido dentro de `/admin`, visível só para `admin`*. Reaproveita casca, *nav* e trilha. Descartada pelas três razões acima; a economia é de umas poucas dezenas de linhas.
- *Um sétimo papel* (`operador_unooria`), deixando `admin` como era. Descartada por multiplicar a matriz de papéis para descrever a conta que já existe: `admin` JÁ é a conta da Unooria, e o problema não é falta de papel, é a afirmação de igualdade ter sido escrita antes de existir a primeira exceção.

Consequências. *Ruim*: a matriz de papéis do capítulo de *tenancy* deixa de poder dizer "`admin` ≡ `gestor_contrato`" e passa a carregar uma exceção — exatamente o tipo de detalhe que a próxima sessão "conserta" sem saber, e é por isso que esta ADR existe. *Ruim*: uma área a mais no `middleware`, e com ela uma armadilha que passa no *typecheck* — o *redirect* entre áreas compara UMA *string* (`areaDoPathname` contra `areaDoPapel`), então o `admin` precisa poder entrar em DUAS áreas ou ele é expulso de uma delas, com o sintoma "clico em Recursos e caio em Usuários". *Bom*: existe um lugar que é da fornecedora, e ele pode crescer sem se misturar com o da escola.

Gatilho de reavaliação. Um cliente que contratualmente exija operar o próprio catálogo — caso em que a resposta provável não é abrir o painel ao `gestor_contrato`, e sim recortar dele o que é seguro delegar (desligar sim, ligar não), o que já é uma decisão diferente desta.

18.90 ADR-89 — O histórico é tabela, não log — e a REDE lê o dela

Data: 2026-09-10 **Estado:** Aceita

Contexto. A **ADR-87** registrou, como consequência ruim aceita, que o histórico de quem mexeu no painel vivia só no registro estruturado (`codigo: recurso-alterado`): a linha de `disponibilidades` é SOBRESCRITA a cada mudança, então ela responde "o que vale agora" e não "quem desligou isto, e quando". O gatilho declarado ali era o dia em que a pergunta precisasse de TELA.

Ele chegou por dois caminhos ao mesmo tempo. Primeiro, a pergunta é feita às 8h da manhã por quem está com a escola ao telefone — não por quem tem acesso ao Cloud Logging e sabe escrever um filtro. Segundo, e mais importante: quem sofre a mudança é a REDE, e ela não tinha como saber que a perdeu. O produto desaparecia do painel do professor e o sintoma é

indistinguível de um defeito (está no Apêndice F por isso) — o gestor do contrato descobriria ligando para perguntar.

Decisão. O histórico é uma tabela **append-only** (`recursos_auditoria`), escrita por toda mudança do painel, e é lida por **DOIS** painéis: o da Unooria (`/unooria/recursos`, quem faz) e o da rede (`/admin/recursos`, quem sofre).

A assimetria da ADR-88 fica intacta. `GET /recursos/auditoria` é a única rota de `/recursos/*` que `gestor_contrato` alcança; `GET /recursos/painel` e as escritas continuam respondendo 403 para ele. Ler por que o produto mudou de estado é informação do cliente; decidir o estado não é. Verificado no navegador: a leitura responde 200 e o `PUT` responde 403 na mesma sessão.

Quatro escolhas dentro da decisão.

`autor_nome` é CONGELADO na linha, e não um *JOIN* com `users`. Um *JOIN* mostraria o nome de HOJE — ou nada, se a conta saiu da rede — e o registro deixaria de dizer quem era aquela pessoa NAQUELE dia. É a mesma razão de `exp08ef1b_runs.nome_no_app` guardar o apelido em vez de o resolver. Por isso também **não há FK** em `autor_id`: com `RESTRICT`, excluir qualquer conta que já tenha mexido no painel ficaria impossível para sempre, porque histórico não se apaga.

`UPDATE` é recusado por TRIGGER; `DELETE` continua permitido. Reescrever uma linha é o único jeito de esta tabela mentir, e é o que o *trigger* fecha — "append-only" garantido pelo código seria a promessa que a próxima rotina de correção quebra. O `DELETE` fica por duas razões concretas: retenção (nenhuma tabela deste produto tem prazo hoje, e quem cumprir o prazo precisará apagar) e ISOLAMENTO DE TESTE — tabela que não se limpa faz cada suíte deixar linha para trás, e esta mesma sessão gastou três diagnósticos nesse tipo de vazamento.

O registro NUNCA derruba a operação que o chamou. Desligar um produto num incidente não pode falhar porque o histórico não pôde ser escrito. Mas o erro não some: vai ao log com `codigo: recurso-auditoria-nao-gravada`, que é o que uma métrica de log alcança.

O histórico de uma ESCOLA inclui as decisões de CONTRATO. Sem elas, a tela da escola diria "nada aconteceu aqui" sobre um produto que a rede desligou — exatamente a pergunta do gestor.

Alternativas descartadas.

- *Ficar só no log e construir uma métrica ou painel no Cloud Logging.* Custo zero de código. Descartada porque a pergunta é de PRODUTO e quem a faz não tem console de nuvem — e porque o cliente nunca veria.
- *Versionar `disponibilidades` em vez de tabela nova* (uma linha por estado, a mais recente valendo). Uniformizaria as duas coisas. Descartada por transformar toda leitura do resolvedor — que roda no caminho de TODA entrada em experiência — numa consulta com `ORDER BY` e `DISTINCT ON`. O estado atual é lido milhares de vezes por manhã; o histórico, algumas por semana.

- *Dar a escrita ao* `gestor_contrato`, já que ele passa a ver o histórico. Descartada: é a decisão da **ADR-88**, e o recorte "desligar sim, ligar não" continua sendo o gatilho de reavaliação dela, não desta.
- *Abrir o histórico a* `gestor_escola` *e* `gestor_regional`. Descartada porque o histórico é do CONTRATO (a linha de escopo-contrato não tem escola), e recortá-lo por unidade daria a uma escola a notícia de que a rede perdeu um produto antes de a coordenação da rede saber.

Consequências. *Bom*: a pergunta operacional passou a ter resposta em tela, para os dois lados, e o cliente deixou de depender de um telefonema. *Bom*: o *trigger* torna a palavra "append-only" verdadeira em vez de aspiracional. *Ruim*: uma tabela que só cresce e não tem prazo — o mesmo risco `R-02` que `telemetria_eventos` e `vila_sessoes` já carregam, e pela mesma razão (prazo sem rotina que o cumpra é pior que prazo nenhum). *Ruim*: o *motivo* escrito pela Unooria passa a ser lido pelo cliente — o que é a intenção, e ao mesmo tempo significa que ele é texto de CONTRATO, não recado interno. Está dito no rótulo do campo, no diálogo.

Gatilho de reavaliação. O primeiro pedido de exportação (CSV do histórico para anexar a um relatório contratual) ou de retenção com prazo — o segundo é o que faz a tabela precisar de rotina, e aí ela entra na mesma conversa de `telemetria_eventos`.

18.91 ADR-90 — A suíte da API roda em UM *worker*

Data: 2026-09-10 **Estado:** Aceita

Contexto. A suíte de testes da API reprovava de forma intermitente — uma falha a cada 60 execuções completas, sempre em suítes que passavam sozinhas. A caça encontrou e consertou QUATRO defeitos reais de isolamento, e todos tinham a mesma raiz: **as tabelas de catálogo e de contrato estão fora da RLS por desenho** (`experiencias`, `experiencias_versoes`, `disponibilidades`, `periodos_letivos`, `recursos_auditoria`), então uma *fixture* escrita ali é ESTADO GLOBAL, não estado da suíte. Os quatro: uma linha de disponibilidade em escopo-contrato desligando um produto para as outras suítes; uma experiência de teste com `publico: "estudante"` entrando na grade de toda turma; o `GRANT` de preparação da *role* de *runtime* rodando em paralelo consigo mesmo (`XX000 tuple concurrently updated` — e o comentário do arquivo AFIRMAVA ser seguro em paralelo); e uma suíte apagando o período letivo compartilhado.

Consertados os quatro, a cauda continuou: 1 falha em 60 corridas paralelas contra 0 em 25 seriais. A conclusão é que o que resta não é um defeito localizável, é a CLASSE do problema — um banco, um *schema*, tabelas compartilhadas sem RLS e 40+ arquivos de teste escrevendo nelas ao mesmo tempo.

Decisão. `maxWorkers: 1` no `apps/api/jest.config.cjs`, com o número medido e o gatilho de reavaliação no comentário ao lado. **A troca é explícita**: 5,4 s → 17,6 s na suíte inteira, em

troca de zero intermitência. Trinta corridas completas depois da mudança (55 contando as seriais da medição) fecharam sem uma falha.

Alternativas descartadas.

- *Continuar caçando teste por teste.* Descartada porque o custo não converge: cada arquivo novo pode reintroduzir o problema, e a falha aparece uma vez em sessenta — o pior regime possível para quem lê um CI vermelho e tem de decidir se aquilo é a mudança dele.
- *Um banco por worker.* É o conserto de verdade, e continua sendo o caminho quando o tempo doer: ela custa *template* de banco, `DATABASE_URL` por *worker* e migração aplicada N vezes no *setup*. Cara demais para 17 s.
- *Ligar RLS nas tabelas de catálogo*, para que o isolamento viesse do banco. Descartada com força: a linha de escopo-contrato de `disponibilidades` tem `escola_id` nulo POR DESENHO, e a política a esconderia — todo produto passaria a parecer ligado, que é o sintoma oposto a um erro de permissão. Está escrito no dicionário de dados e provado por teste, para que a próxima migração não "conserte" isso.
- *Marcar as suítes de Postgres como serial e deixar o resto em paralelo.* Descartada por não pagar: as suítes de Postgres são a maioria do tempo da suíte, então o ganho seria de segundos, ao custo de uma regra que cada arquivo novo precisa saber e obedecer.

Consequências. *Bom*: o CI vermelho voltou a significar "a sua mudança quebrou algo". *Bom*: a caça deixou quatro *fixtures* realmente isoladas, e o comentário de cada uma diz por quê. *Ruim*: 12 segundos a mais em cada execução local e de CI. *Ruim*: serializar ESCONDE o problema de classe — um teste novo que dependa de ordem agora passa, e só vai doer no dia em que alguém reativar o paralelismo.

A face de LEITURA do mesmo problema, descoberta horas depois e registrada aqui porque é a mesma decisão: se *fixture* escrita em tabela de contrato é estado global, **leitura** dela é dependência invisível. Três suítes LIAM `experiencias` sem nunca a escrever — o teto do produto, a composição do painel, a grade da sala — e passavam porque o banco de desenvolvimento está SEMEADO. No Postgres descartável do CI a tabela nasce vazia, e o vermelho saía como `expect(codigo).toBe('recurso-indisponivel')` recebendo `null`: lê-se "a guarda não funciona" quando o que faltava era o dado, e não reproduz na máquina de ninguém. O conserto é declarar a pré-condição: `globalSetup` semeia o catálogo REAL a partir dos manifestos, uma vez, antes de qualquer suíte — a mesma fonte que o CI publica e que o *seed* lê, nunca uma lista de teste à mão. `update` mínimo, para que rodar os testes não desfaça um *rollback* deliberado em *dev*; e Postgres inalcançável avisa e segue, porque 75 das 95 suítes não precisam de banco.

Gatilho de reavaliação. A suíte passar de 60 s. Aí o banco por *worker* passa a valer o preço, e o comentário do `jest.config.cjs` está escrito para ser lido nesse dia.

18.92 ADR-91 — O idioma da interface tem TRÊS zonas, e o clone não se traduz

Data. 2026-09-10. **Status.** Aceita.

Contexto. A plataforma atende redes públicas, e o texto dela é lido por professor, gestor e criança do 1º ano EF ao 3º EM. Anglicismo em rótulo não é questão de gosto: é pressuposto de repertório que numa rede pública não se pode fazer. A varredura de 2026-09-10 achou **427 trechos de texto realmente visível** com termo em inglês, em 112 arquivos — e o achado que motivou a tarefa foi o mais claro de todos: a trilha da exp02em1b dizia `Briefing` ao lado de `Orientação`, `Investigação` e `Resultado`, ou seja, a palavra estava fora do padrão dos próprios vizinhos.

Mas uma regra única — "traduzir tudo" — faz a plataforma melhorar e as experiências piorarem. Dentro do clone, a fidelidade ao aplicativo real é o INSTRUMENTO de medida: é ela que faz a criança agir como age fora da aula, e é isso que a missão afere. `Shorts`, `Stories`, `Feed` e `Playlists` são os nomes que o YouTube e o Instagram usam nas versões brasileiras deles; traduzi-los quebraria o disfarce que a experiência existe para manter.

Decisão. Três zonas, com regras distintas:

1. **Plataforma** (hub, experiências do professor): pt-BR sem exceção.
2. **Superfície disfarçada** (dentro do clone): o nome que o aplicativo real usa em português. Não se traduz.
3. **O professor falando SOBRE o clone** (guia da aula, catálogo, painéis): o critério é se o termo **nomeia um objeto que está na tela** — `feed`, `Shorts`, `ranking` ficam, porque traduzir faria o guia dizer uma palavra e a tela mostrar outra, e o professor perderia o referente no meio da aula — ou se é **jargão de implementação vazado** — `card de pausa`, `o modal`, `num popup` saem, porque não nomeiam nada que a criança veja e existe palavra portuguesa exata.

As trocas aplicadas: `Briefing` → `A missão`; `card de pausa` → `tela de pausa`; `cards de sistematização` → `quadros de sistematização`; `modal` e `popup` → `janela`; `Login` → `Usuário` (o campo interno já se chamava `usuario` — o rótulo divergia do próprio dado); `Insight` → `Dica`; `Prompt` → `Como pedir`; `check-in` → `acompanhamento` para o produto e `Responder hoje` / `Você já respondeu hoje` para o ato.

Identificador não é texto, e mudá-lo é outra tarefa com outro custo: chave de objeto, rota, coluna, verbo/objeto de telemetria, nome de glifo do Material Symbols (`checklist`, `psychology`, `schedule`), variante de componente (`warning`, `success`), severidade de log e método HTTP ficam. O rótulo `abertura: 'A missão'` mudou; o `'abertura'` do banco não.

Uma reformulação além da troca de termo, no produto de bem-estar: "Faltam 3 check-ins para o retrato da semana" virou "Faltam 3 dias". Não é sinônimo — troca contagem de eventos técnicos por contagem de dias, que é como a pessoa efetivamente pensa, e é mais exato, porque é um por dia.

Anglicismos que FICAM, por decisão e não por esquecimento. Esta lista é curta de propósito, e está registrada para que ninguém a "corrija":

- `fake news` — consagrado no Brasil, inclusive em política pública e imprensa. E nos guias da exp06ef2b e da exp07ef1b a frase é "não anuncie que é uma aula sobre fake news": o que o professor NÃO deve dizer é exatamente a expressão que a criança reconhece, então traduzir mudaria o sentido da instrução.
- `cyberbullying` — é o termo da BNCC (EF07CO09) e da Lei 13.185/2015. Alterar desalinha do documento oficial que o próprio guia cita.
- `print` — é a palavra que a criança brasileira usa, e a missão da exp06ef2b e da exp07ef1b gira em torno desse objeto. "Captura de tela" tornaria o guia irreconhecível diante da fala da sala.
- `chat` e `on-line` — o primeiro incorporado ao português e nome da coisa no WhatsApp; o segundo é citação verbatim de habilidade da BNCC.

Alternativas descartadas.

- *Traduzir tudo, clone incluído.* Descartada porque destrói o instrumento: um "Vídeos curtos" onde o aplicativo real diz "Shorts" avisa a criança de que aquilo é software de escola, e a missão passa a medir a reação dela à escola em vez do comportamento que interessa.
- *Não mexer em nada, tratando anglicismo como português técnico consagrado.* Descartada porque a maior parte dos achados não era jargão consagrado — era jargão NOSSO, de implementação, vazado para o texto do professor. "Modal" não é palavra que professor de rede pública use.
- *Traduzir também `post` → `publicação`* (72 ocorrências). É o único caso em que traduzir AUMENTARIA a fidelidade — o Instagram em português diz "Publicações", não "posts" —, e ficou de fora desta ADR por volume, não por mérito: são 72 trechos, a maioria em texto denso de guia. Fica registrado como próximo passo.
- *Um arquivo de i18n com chaves.* Descartada: a plataforma é monolíngue por contrato (rede pública brasileira), e uma camada de internacionalização para um idioma só custa indireção em toda tela sem resolver nada do problema real, que era terminologia, não tradução.

Consequências. *Bom*: o rótulo passou a concordar com o dado em `Login` → `Usuário` — a coluna dizia uma coisa e a chave `usuario` outra. *Bom*: a doutrina das três zonas está no `AGENTS.md` da raiz e no de `apps/experiencias`, com a lista de exceções, o que impede a próxima "correção" de reintroduzir jargão ou apagar `Shorts`. *Ruim*: **a suíte e2e casa por texto exato** (`getByText`, `getByRole({name})`) — trocar copy sem atualizar a suíte a derruba com *timeout*, e o *timeout* reporta o seletor que faltou, nunca a causa; foram 36 linhas mexidas em `professor/04-bem-estar.mjs` só por isso. *Ruim*: `ROTULO_PASSO` da exp02em1b é espelho declarado de `apps/api/src/unoogpt/regras/passos.ts`, e mudar um lado só faria a trilha do estudante divergir do que a api devolve, sem o *typecheck* ver nada.

O que a execução pegou e o levantamento não previra: a versão da experiência vive em TRÊS lugares, não dois — manifesto, `package.json` e o `EXP_VERSION` do código. O `versao.test.ts` de cada experiência cobra os três, e foi ele que pegou o esquecimento; a ADR-84 conta o que o terceiro lugar esquecido custou (erro de cliente agrupado numa versão que nunca existiu, e `mapear-stack.mjs` procurando *sourcemap* em caminho vazio).

18.92.1 Emenda de 2026-09-10: acessível não é só *traduzido*

Um pedido de professores mostrou que a regra das três zonas resolve o inglês e NÃO resolve o outro caso de linguagem inacessível: a palavra em português que a criança lê sem saber o que significa. Na tela de resultado da exp02em1b, o número em vermelho era rotulado "afetadas" — e os estudantes não sabiam se aquilo contava morte, desabrigamento ou outro dano. O termo virou "atingidas pela enchente" ("Atingidas" no cabeçalho da tabela, onde a coluna precisa ser curta).

> **REGRA INVIOLÁVEL**
>
> **O termo novo não pode mudar o que o número CONTA.** O pedido original propunha "moradores que precisam de ajuda", e essa troca foi **recusada**: as três categorias da tela somam as pessoas em risco (resgatadas + atingidas + sobreviveram sem ajuda), e o campo interno se chama `vitimas` — é dano JÁ ocorrido, não necessidade futura. Com o rótulo proposto, aquele número passaria a descrever a mesma coisa que "sobreviveram sem ajuda" ao lado dele, e a gravidade que a tela existe para o estudante encarar desapareceria. Trocar palavra de interface exige ler o que ela nomeia; aceitar a sugestão à letra teria corrigido o vocabulário e quebrado o sentido.

E a causa não era só a palavra. A simulação não distingue o TIPO de dano, e era essa indefinição que produzia o mau sentimento relatado. Por isso a tela ganhou a frase que fecha a lacuna: "a simulação conta quantas pessoas a enchente atingiu, sem dizer que tipo de perda cada uma teve". Trocar o rótulo sem dizer isso deixaria o estudante com uma palavra mais clara sobre uma quantidade que continuava indefinida.

Gatilho de reavaliação. A plataforma precisar de um segundo idioma (rede indígena com material bilíngue, ou contrato fora do Brasil). Aí a decisão de não ter camada de i18n volta à mesa, e o custo dela deixa de ser indireção gratuita.

18.93 ADR-92 — Voltar um momento REABRE a sessão; a trava é a revelação, não o relatório

Data. 2026-09-10. **Status.** Aceita.

Contexto. O relógio da aula do Instanooria tem dois ponteiros: a `etapa` da TURMA, que o professor move, e a `fase` da RUN, que é a tela daquela criança. `faseAoVoltar` (`apps/api/src/comum/insta/etapas`

reconcilia os dois quando a turma anda para TRÁS, e o comentário dela declara, desde a origem, o motivo de existir: sem ela, "voltar um momento" mudaria o rótulo no painel e **nada** na tela dos estudantes.

Era exatamente o que acontecia. Um chamado de campo relatou que, encerrada a investigação, voltar para ela deixava a turma inteira congelada na tela da resposta. Reproduzido de ponta a ponta com as duas pontas vivas (professor no relógio, estudante no aparelho): a API responde `{etapa: "investigacao", voltou: true, movidos: 0}` — voltou a etapa e moveu ZERO telas —, e o painel fica autocontraditório no mesmo cartão, dizendo "agora Investigação — a turma explora o feed" acima de "1 respondendo".

A causa são duas peças corretas isoladamente. `moverEtapa` congela toda *run* que entrou no aparelho ao SAIR da investigação (`encerrarRun` grava o dossiê e `finished_at`) — sem isso o relatório abria vazio. E `faseAoVoltar` tinha o teto `sessaoCongelada ? conclusao : feed`. Como a primeira torna a condição da segunda sempre verdadeira, **o ramo do `feed` era inalcançável pelo caminho do painel**: código morto que o *typecheck* não vê, e um comentário descrevendo uma intenção que nunca valeu.

A justificativa registrada para a trava era "sessão congelada não reabre, senão o relatório mentiria". Ela não se sustenta: o dossiê é **recomputável** a partir dos eventos, que continuam todos gravados.

Decisão. Duas mudanças, que só funcionam em par:

1. **A trava passa a ser a REVELAÇÃO**, não o congelamento. `jaViuARevelacao(fase)` — verdadeira de `explicabilidade` em diante, incluindo as fases legadas via `normalizarFase`. Quem ainda não viu volta ao `feed`; quem já viu para na resposta, que dá para refazer. O motivo é o INSTRUMENTO: quem já sabe que o feed era enviesado não pode ser medido por ele de novo. O parâmetro `sessaoCongelada` SAIU da assinatura — ele era derivável do estado errado.
2. **Voltar para a investigação REABRE a sessão**: `moverEtapa` apaga o dossiê e zera `finished_at` de quem volta ao `feed`. Apagar o dossiê é o que o torna recomputável — `encerrarRun` é idempotente pelo dossiê, então zerar só o `finished_at` deixaria o relatório do próximo encerramento sem nada do que a criança fizesse dali em diante, que é precisamente o "relatório mentindo" de que a regra antiga tinha medo. A resposta ganha `reabertas`, e o painel diz ao professor que o relatório daquelas sessões será refeito.

O que o estudante PRODUZIU não é tocado: eventos, posts salvos, comentários, a resposta e a reflexão continuam. Quem apaga isso é `POST /runs/:id/reset`, e ele é outra intenção — "esta sessão não presta, comece do zero", não "a aula voltou um momento".

Alternativas descartadas.

- *Não congelar ao sair da investigação.* Resolveria o sintoma e reabriria o defeito que o congelamento existe para fechar: o relatório da turma abrindo vazio porque ninguém chamou `finish`.

- *Deixar como estava e documentar que voltar não devolve o feed.* Descartada porque o produto oferece o botão "voltar um momento", e um botão que muda só o rótulo é pior que botão nenhum — o professor conclui que o sistema travou no meio da aula.
- *Mandar o professor usar "Zerar" para reabrir.* É o que sobrava na prática, e custa o trabalho inteiro da criança: o `reset` apaga eventos, posts salvos, comentários, conclusão e reflexão. Corrigir um clique errado do adulto destruindo a produção de trinta estudantes é desproporcional.
- *Preservar o dossiê e recomputá-lo por versão.* Descartada por complexidade sem demanda: exigiria versionar dossiê por passagem, e ninguém pediu o relatório de "como estava antes de o professor voltar".

Consequências. *Bom*: o retorno funciona nas duas pontas, e a regra que o código já declarava passou a valer. *Bom*: o botão "voltar um momento" saiu de dentro do bloco do botão principal no painel — ele só era renderizado quando havia PRÓXIMA etapa, então na etapa `fim` não havia como voltar pela interface, e a aula que terminou cedo demais é justamente quando voltar importa. *Ruim*: o relatório daquela passagem é DESCARTADO ao voltar, e isso é irreversível — o painel avisa, mas quem voltar por engano perde o dossiê computado até ali (os eventos ficam, então o próximo encerramento reconstrói). *Ruim*: `POST /atividades/:id/etapa` ganhou um campo na resposta, e todo consumidor do contrato precisa regerar o OpenAPI.

O que a execução expôs: `comum/insta/etapas.ts` não tinha **nenhum teste** — o módulo que reconcilia dois relógios e decide o que 30 telas mostram ao mesmo tempo. Foi por isso que o ramo morto passou. Entraram duas suítes: `tests/insta-etapas.spec.ts` (as regras puras, 19 casos) e `tests/insta-voltar-etapa-postgres.spec.ts` (o ciclo congelar → voltar → reabrir → recomputar contra o Postgres). A segunda existe porque o defeito vivia no PAR: cada metade, lida sozinha, estava certa.

Limitação conhecida, e deliberada: `jaViuARevelacao` lê a fase ATUAL da *run*, não o histórico dela — e a fase é regressiva. Voltando um momento de cada vez, a *run* desce até `conclusao` antes de a turma chegar à investigação, e daquele degrau a proteção já não se aplica. O resultado depende do CAMINHO: o pulo direto de `explicabilidade` para `investigacao` para na resposta; os cinco cliques passo a passo, cada um com aviso na tela, terminam no *feed*. Fechar isso exigiria PERSISTIR o marco ("esta criança já viu a revelação"), porque a fase não o guarda — coluna nova, migração e retrocompatibilidade para um caminho de cinco cliques deliberados. Ficou de fora por proporção, não por descuido, e os dois casos estão fixados em teste para que a escolha seja vista e não descoberta.

Gatilho de reavaliação. Dois. (1) Alguém pedir o histórico das passagens — o relatório de como a turma estava antes de o professor voltar; aí o dossiê deixa de ser sobrescrito e passa a ser versionado por passagem. (2) O caminho passo a passo até a investigação aparecer em uso real com turma que já viu a revelação; aí o marco vira coluna e a proteção passa a valer pelos dois caminhos.

18.94 ADR-93 — Sincronização de sala por *polling* HTTP curto, não por WebSocket

Data. 2026-09-10. **Status.** Aceita.

Contexto. A exp08ef4b (*O Manifesto da IA*, 8º ano, unidade 4) tem um ponto de sincronização coletiva: ao fim da Etapa 3, o professor congela as telas da turma para um debate oral de dez minutos — o "Checkpoint A" do insumo. O *handoff* de engenharia especificou o mecanismo como um *broadcast* `hub:pause` por WebSocket, despachado pela rota `POST /manifesto-ia/turmas/:turmaId/pau` para todos os clientes conectados daquela turma.

Esse WebSocket não existe. A plataforma roda em Cloud Run, e não há infraestrutura de WebSocket de aplicação em nenhum dos ambientes: nem o *load balancer* está configurado para conexão persistente, nem a API mantém estado de conexão — ela é *stateless* por desenho, com escalonamento horizontal e instâncias que somem sem aviso. Uma conexão persistente por estudante criaria estado de sessão na instância, que é justamente o que o modelo de escala do produto recusa. A banca técnica registrou a ressalva **TEC-06** e apontou a alternativa que o próprio protocolo de negociação do *handoff* previa.

Decisão. O estado coletivo de sala é uma **linha em tabela**, e o cliente a lê por ***polling* HTTP curto de 5 segundos** numa rota dedicada e leve.

1. **A verdade é uma linha por turma.** `exp08ef4b_gates_turma` guarda `etapa` ∈ {`missao`, `debate`}, com RLS por `escola_id` como toda tabela de dado de aula. O clique do professor é um `upsert` idempotente.
2. **A rota do *polling* é deliberadamente magra.** `GET /manifesto-ia/gate` devolve três campos — a etapa da turma, o passo efetivo reconciliado e o instante do SERVIDOR — e não toca no catálogo, na conversa nem no percurso. Ela é ordens de grandeza mais barata que a rota de sessão.
3. **O *polling* só roda onde o congelamento tem efeito.** `deveObservarOGate` (`src/lib/maquina-estados.ts`) o liga de `decisao` a `percurso`. Ligado durante a missão inteira, uma turma de 35 custaria 7 req/s pelos 50 minutos; assim, custa isso durante a Etapa 4.
4. **A reconciliação é função PURA e compartilhada.** `passoEfetivo(passo, etapa)` mora em `regras/passos.ts` na API e é espelhada no cliente, porque os dois precisam chegar à mesma resposta — duas implementações divergentes seriam a tela discordando do servidor.

O gate mexe no TETO, nunca no piso, e isto é desenho, não detalhe. Na exp06ef2b a etapa da turma é uma porta que o professor ABRE, e por isso ela eleva o piso. Aqui o congelamento é uma interrupção OPCIONAL — a especificação admite a sessão individual assíncrona, e sem clique nenhum a missão corre inteira. Se `debate` também subisse o piso, apertar "Pausar para Debate" empurraria para a frente quem ainda estava auditando a máquina, e são justamente essas crianças que o debate tenta alcançar.

O custo, declarado: até 5 segundos de latência. O congelamento não é simultâneo; ele chega em até um ciclo de *polling*. É aceitável porque o que ele serve é um debate de dez minutos conduzido por uma pessoa falando — não um jogo em tempo real. O texto do painel diz isso ao professor ("a pausa leva até 5 segundos para chegar nas telas") em vez de deixá-lo descobrir olhando a sala.

Alternativas descartadas.

- *WebSocket de aplicação*, como o *handoff* pedia. Exige infraestrutura que não existe (conexão persistente no *load balancer*) e estado de conexão numa API *stateless* que escala horizontalmente. Seria a primeira peça do produto a não sobreviver a uma instância morrendo.
- *Server-Sent Events (SSE)*. Mais barato que WebSocket e unidirecional, que é o formato do problema — mas ainda é conexão longa, e Cloud Run tem teto de *request timeout*: a conexão cairia e voltaria em ciclo, o que é *polling* com mais partes móveis. Para um dado de dois valores lido durante dez minutos, não paga.
- *Pub/Sub para o navegador.* Já existe Pub/Sub no produto (a telemetria), mas ele é de saída e assíncrono; entregá-lo ao cliente exigiria um *gateway* de *fan-out*, que é o serviço novo que a doutrina do projeto recusa sem gargalo medido.
- *Canal de comandos do hub pela ponte da SDK.* Seria elegante — o hub já fala com o iframe por `postMessage` — mas o hub não sabe quando a turma foi congelada: ele também precisaria consultar a API. O *polling* apenas mudaria de lugar, ganhando um salto.

Consequências ruins. (1) Há uma janela de até 5 s em que dois estudantes da mesma turma estão em estados diferentes — um congelado, o outro não. (2) O *polling* continua rodando com a aba em segundo plano, e o navegador o desacelera de formas que variam por sistema: uma tela minimizada pode demorar mais que 5 s para congelar. (3) A rota de `gate` é a mais chamada da experiência, então uma regressão de desempenho nela aparece como "a aula ficou lenta" sem nada apontar para ela — por isso ela não faz JOIN e não lê catálogo.

Gatilho de reavaliação. Dois. (1) Uma experiência pedir sincronização com granularidade abaixo de um segundo (um jogo coletivo, um cronômetro compartilhado visível); aí o problema muda de classe e SSE volta à mesa, medindo o teto de *timeout* do Cloud Run. (2) A soma dos *pollings* de todas as experiências aparecer no custo ou na latência do banco; o caminho então é um cache de leitura por turma na frente da rota de `gate`, não trocar o mecanismo.

18.95 ADR-94 — O latch de turma anda por POLLING curto, nunca por canal persistente

Data. 2026-09-10. **Status.** Aceita.

Contexto. A exp01ef1b (O Que a Foto Conta, 1º ano EF) é a missão mais síncrona do catálogo: na Etapa 5 *todos* os tablets da turma congelam numa tela sem nenhum botão (a Roda de Conversa) e só o professor os solta, de uma vez, pelo `LIBERAR REFLEXÃO FINAL`. O insumo da

missão pedia “WebSocket/SSE” para esse destravamento, e a banca já havia rebaixado o pedido a ressalva (TEC-06): priorizar SSE ou *polling* HTTP curto, com o teto de 4 segundos entre o clique do professor e a reação do tablet.

Três fatos do repositório decidem a questão antes de qualquer preferência:

1. **Não existe canal persistente de aplicação nesta infraestrutura.** Nenhum WebSocket, nenhum SSE — e o Cloud Run cobra por instância ocupada, o que faz conexão pendurada de 35 tablets durante uma roda de conversa de 15 minutos custar exatamente o que o *polling* evita.
2. **O problema já foi resolvido três vezes, sempre por *polling* curto**: o *gate* da aula do o-eco-no-grupo (`GET /o-eco-no-grupo/gate`, ∼5s, com professor movendo a turma de etapa — o irmão exato deste latch), os momentos coletivos do motor (`/runs`) e o progresso de geração (`/geracao`, 2s).
3. **O “O que NÃO fazer” da raiz proíbe literalmente** o caminho contrário: “não adicionar Redis/filas/libs de estado sem gargalo medido”. SSE aqui seria tecnologia nova sem gargalo nenhum.

Decisão. O latch da exp01ef1b anda por *polling* HTTP de **3 segundos** na rota `GET /foto-conta/latch` — leve de propósito (sem pacote e sem catálogo: `{liberado, etapa, agora}`), com *rate limit* por usuário. A liberação é um `UPDATE` idempotente por erro (`409 latch-ja-liberado`) em `exp01ef1b_turma_config.latch_liberado`, sem *lock* de linha: quem “recebe o *broadcast*” é o *polling* dos tablets. O painel do professor acompanha a turma no mesmo desenho (∼5s, como o o-eco-no-grupo). O `agora` da resposta é o instante do SERVIDOR: o relógio do tablet da escola atrasa, e não é ele que decide o momento da aula.

Alternativas descartadas.

- **WebSocket persistente** (o pedido original do insumo): exige um serviço com conexões de longa duração que a plataforma não tem, quebra o modelo de escala do Cloud Run e cria a primeira dependência de estado de conexão do produto — para ganhar, no melhor caso, 3 segundos de latência numa transição que acontece UMA vez por aula.
- **SSE**: mais barato que WebSocket, mas ainda é conexão pendurada por tablet, ainda é tecnologia nova no repositório, e não há precedente — introduzi-la para uma missão de 1º ano criaria o segundo padrão de tempo real que o próximo módulo teria de escolher.

Consequências ruins, aceitas. A latência média do destravamento é metade do intervalo (∼1,5s, teto 3s + rede) — dentro do teto de 4s da banca, mas não instantânea. E uma turma de 35 tablets a 3s produz ∼12 req/s numa rota de leitura trivial durante a missão — carga desprezível hoje, e é exatamente ela o gatilho abaixo.

Gatilho de reavaliação. Se alguma rede real medir o *polling* de latch como fonte relevante de carga (ordem de grandeza: várias dezenas de turmas SIMULTÂNEAS na mesma stack, $>$200 req/s só de latch) ou se um novo requisito exigir latência sub-segundo numa transição de turma,

reabrir esta decisão — e aí a resposta provável é UM canal de eventos de turma para a plataforma inteira, nunca um WebSocket por experiência.

18.96 ADR-95 — A sessão do hub dura 2h de INATIVIDADE, e vale igual para todo papel

Data. 2026-09-10. **Status.** Aceita.

Contexto. A sessão do hub durava 12h (`apps/hub/lib/auth.ts`), com o comentário "cobre um dia de aula". O combinado original era **2h para o estudante e 4h para o professor**, com o racional escrito: 2h porque é o tempo de duas aulas seguidas, e 4h para o professor não deslogar várias vezes ao longo do turno. O próprio registro da combinação já previa o plano B — "na impossibilidade ajustamos para *logout* geral após 2h" — e é ele que o dono do produto pediu.

Um número de TTL não é uma decisão, e é por isso que esta ADR existe: "2 horas" pode querer dizer duas coisas incompatíveis — 2h **absolutas** desde o *login*, ou 2h de **inatividade** — e a diferença numa aula é enorme. Absoluta desloga a criança no meio da segunda aula; por inatividade, não.

O que o código já entregava, e ninguém tinha declarado. Lido no `@auth/core` 0.41.3 instalado: com `strategy: "jwt"`, o ramo `jwt` de `lib/actions/session.js` **re-assina** o token a cada leitura de sessão (o `jwt.encode` grava `exp = agora + maxAge`) e o `handleAuth` do `next-auth` devolve o `Set-Cookie` ao navegador. O `updateAge` (*default* de 24h) **não é consultado nesse ramo** — ele só existe na estratégia de banco. Ou seja: as 12h **nunca foram absolutas**; eram 12h de inatividade, e quem faz a janela deslizar é o *middleware*, porque é ele que roda em toda navegação e devolve o cabeçalho.

Decisão. Três partes:

1. **2h para TODOS os papéis** — o plano B, sem distinção por perfil.
2. **De INATIVIDADE**, preservando a semântica que o código já tinha. O comentário da linha passa a dizer isso, com o mecanismo, porque o antigo ("cobre um dia de aula") descrevia um número que deixou de existir.
3. **O valor fica FIXO no código**, não em variável de ambiente.

Alternativas descartadas.

- *2h para estudante e 4h para professor* (o plano A). Não é impossível — e o registro da combinação supôs que fosse —, mas também não é limpo. `session.maxAge` é um número global, e o `encode` do `@auth/core` faz `.setExpirationTime(now() + maxAge)` **sobrescrevendo** qualquer `exp` que o *callback* `jwt` tenha posto no token: não sai de dez linhas no *callback*. Sobram dois caminhos, e os dois cobram o mesmo preço. (a) Config como FUNÇÃO (`NextAuth((req) => …)`, que o v5 aceita) exigiria decifrar o *cookie* de sessão à mão, no

edge, a cada requisição, para saber o papel antes de montar a config que fornece a chave de decifração. (b) `maxAge` no maior valor (4h) mais uma segunda expiração nossa, com *claim* próprio de último uso conferido no *middleware*. As duas criam uma **segunda fonte de verdade** sobre quando a sessão morre, ao lado da da biblioteca — e sessão que expira por dois mecanismos diferentes é a divergência que só aparece em campo. Fica registrado que o plano A custa isto, e não a linha que se imaginou.

- *Sessão absoluta de 2h.* Desloga o estudante no meio da segunda aula — justamente o intervalo que as 2h existem para cobrir.
- *TTL em variável de ambiente.* Descartada por uma razão específica desta plataforma: a página `/privacidade` afirma o prazo **em texto, para a família**, e a regra do hub é que o texto legal só afirme o que o código faz (**ADR-70**). Com o número em *env*, ou ele sai da página legal — piorando o que a família lê — ou passa a poder divergir por rede sem ninguém perceber. Número fixo com comentário é auditável dos dois lados, e ajustar por rede não era pedido de ninguém.

Consequências, inclusive as ruins.

1. **O professor volta a logar no meio do turno.** É o custo aceito do plano B: ele perde as 4h combinadas, e duas horas sem abrir tela é plausível entre uma janela de aulas e o intervalo.
2. **Missão longa pode perder a sessão com a criança trabalhando.** `auth()` sem argumentos — o caminho de *React Server Component* e de *route handler* — LÊ a sessão e **descarta** o `Set-Cookie`, porque nesses contextos não se escreve *cookie*. Só o *middleware* renova. Quem entra numa missão não navega mais no hub, e o `exp:token-expirando` de 10 em 10 minutos cai em `GET /api/exp-token`, que está FORA do `matcher` e portanto não conta como atividade. Com 12h isso era inalcançável; com 2h passa a ser alcançável por uma missão que comece meia hora depois do *login*.
3. **E quando morre, morre em SILÊNCIO.** `apps/hub/components/ExpFrame.tsx` trata a falha de renovação com um `console.error` e nada mais: a experiência segue com o token velho até ele vencer (TTL de 15 min) e, daí em diante, toda chamada à API responde 401. O que a criança já enviou está salvo — as experiências gravam incrementalmente no servidor —, mas o que ela fizer depois **não é gravado**, e nada na tela pede para entrar de novo. Este defeito não nasce aqui: esta decisão apenas o torna alcançável, e é o principal motivo de o gatilho abaixo existir.
4. A página `/privacidade` mudou no mesmo *commit*, porque a frase dela afirmava as 12 horas.

Gatilho de reavaliação. Qualquer um dos três:

- **Um relato de criança deslogada no meio de missão.** A correção tem duas metades que só valem juntas: fazer a renovação do token CONTAR como atividade (o caminho é o *route handler* re-emitir o *cookie* — o `unstable_update` do `next-auth` escreve *cookie*, ao contrário de `auth()`) e dar ao `ExpFrame` uma tela que peça *login* em vez do `console.error`. É tam-

bém quando a variável de ambiente descartada acima passa a ter razão de existir: sem TTL injetável, a suíte de ponta a ponta só prova isso esperando 2h de relógio.

- **Reclamação de professor sobre relogar.** Aí o plano A volta à mesa, com o custo acima já medido.
- **O WSO2 assumindo a autenticação.** O TTL passa a ser dele, e este número deixa de valer.

18.97 ADR-96 — Ficar na missão CONTA como uso, e a sessão que morre tem tela

Data. 2026-09-10. **Status.** Aceita.

Contexto. A **ADR-95** pôs a sessão em 2h de inatividade e declarou, como consequência ruim, que a janela podia morrer com a criança DENTRO da missão. Duas peças produzem isso, e nenhuma delas é visível no *typecheck*:

1. `auth()` **sem argumentos** — o caminho de *React Server Component* e de *route handler* — LÊ a sessão e **descarta** o `Set-Cookie`, porque nesses contextos não se escreve *cookie*. Só o *middleware* renova.
2. `GET /api/exp-token`, que a SDK pede de 10 em 10 minutos, está FORA do `matcher` do *middleware*.

Quem abre uma missão não navega mais no hub, então o prazo envelhece como se a pessoa estivesse parada. E quando morria, morria **em silêncio**: `apps/hub/components/ExpFrame.tsx` tratava a falha de renovação com um `console.error` e nada mais, a experiência seguia com o token velho até o TTL de 15 minutos vencer e, daí em diante, **toda** chamada à API respondia 401. O que a criança já tinha enviado estava salvo; o que ela fizesse a partir dali, não — e nada na tela pedia para entrar de novo.

Com as 12h anteriores isso era inalcançável. Com 2h, uma missão que comece meia hora depois do *login* alcança — e as 2h foram para o ambiente de *dev*, então deixou de ser hipótese.

Decisão. Três partes:

1. **O pedido de passe conta como atividade.** `GET /api/exp-token` chama `unstable_update({})` depois de assinar o token. O `unstable_update` do `next-auth` escreve o *cookie* via `cookies().set()`; o `auth()` não. O objeto VAZIO é de propósito: o *callback* `jwt` só mescla quando recebe `session.usuario`, então `{}` renova o prazo sem tocar no conteúdo da sessão. E vem DEPOIS de assinar, engolindo o próprio erro: falhar em deslizar custa, no pior caso, um *logoff* mais cedo; falhar em emitir o passe custa a aula.
2. **A morte da sessão tem TELA.** O `ExpFrame` trata 401 e 403 como SESSÃO, não como rede: manda `hub:pause`, PARA a missão e mostra o `AlertBox` “Sua sessão expirou” com “Entrar de novo”. Fica no `ExpFrame` e não no `ExpHost` pelo mesmo motivo do `clarityProjectId`: é o único ponto por onde TODO *embed* passa — no `ExpHost`, as três telas de professor

ficariam de fora, e um professor dentro do gerador de plano por duas horas cai no mesmo buraco. E **substitui** o *iframe* em vez de cobri-lo: o *iframe* ocupa `height: 100%` em quatro *layouts* diferentes, e um `<div>` a mais na cadeia do *flex* é exatamente onde o percentual deixa de ser *definite* e ele volta aos 150px padrão.

3. **O TTL virou injetável, mas NÃO em produção.** `SESSAO_TTL_SEGUNDOS` (validada com Zod, piso de 60s, teto de 12h) existe para a suíte e é IGNORADA em produção, com aviso no *log*. O valor de produção é constante em `apps/hub/lib/sessao.ts`.

A pergunta que a ADR-95 deixou aberta, respondida. Ela descartou a variável de ambiente porque a `/privacidade` afirma o prazo **em texto, para a família**, e texto legal só pode afirmar o que o código faz (**ADR-70**). A resposta não é escolher um dos dois lados: é **pinar a produção e amarrar os dois por teste**. `apps/hub/lib/sessao.test.ts` afirma que o prazo de produção é de 2h e que a página contém a frase com esse número — mudar o valor sem mudar a frase passa a ser vermelho, em vez de uma página mentindo em silêncio. Nada no *typecheck* liga os dois.

Alternativas descartadas.

- **Pôr `/api/exp-token` no `matcher` do *middleware*.** É o atalho óbvio, e não funciona por DOIS motivos independentes. Com sessão válida, `podeEntrarNaArea(papel, "/api/exp-token")` é falso para todo papel: o *middleware* redirecionaria a própria rota para a *home* e cortaria a renovação de quem está usando o produto. Sem sessão, o *redirect* vai para `/login`, que o `fetch` SEGUE e devolve HTML com **status 200** — o `r.ok` passaria e o 401 honesto viraria erro de *parse*. Trocar um 401 legível por um JSON inválido é piorar o diagnóstico do defeito que se quer consertar.
- **Não fazer nada e documentar.** Era o estado depois da **ADR-95**, e ele foi para *dev*. O custo é trabalho de criança perdido em silêncio — a pior forma de falha deste produto, porque ninguém reclama do que não vê.
- **Renovar ANTES de assinar, ou deixar a falha de renovação derrubar a rota.** Inverte a prioridade: o passe é o que mantém a aula de pé.
- **Avisar a experiência por mensagem nova da ponte.** Exigiria subir o `PROTOCOL_VERSION` e republicar as seis experiências, e a mensagem apareceria DENTRO do disfarce — "sua sessão expirou" no meio do Instanooria quebra a ficção que a experiência está medindo. A tela é chrome do HUB, desenhada acima do aparelho.
- **Derivar a frase de `/privacidade` da env.** Obrigaria a página pública mais visitada, hoje estática, a virar dinâmica só para imprimir um número.

Consequências, inclusive as ruins.

1. Missão EM USO não perde mais a sessão por inatividade — desde que a experiência esteja pedindo passe, o que toda experiência que usa a SDK faz.
2. **A janela deixou de ser só "navegação".** Um aparelho aberto e esquecido renova a sessão de 10 em 10 minutos indefinidamente: a criança que fechou a tampa com a aba aberta

continua logada. É o preço declarado de não deslogar quem está trabalhando, e o teto real passou a ser "a aba ficar aberta", não o tempo.

3. `unstable_update` é API de *beta*, e o nome o diz. Se ele mudar na subida do `next-auth`, isto quebra — e quebra no *typecheck*, que é o melhor lugar para quebrar.
4. **A tela de sessão expirada NÃO é exercitada pela suíte.** O gatilho dela é o *tick* de 10 minutos da SDK, e nenhum TTL curto encurta esse *tick*. O `data-testid="sessao-expirada"` está no lugar para quando um roteiro souber dirigir a ponte.
5. Substituir o *iframe* descarta o estado de cliente da experiência. Com a sessão morta ele já não podia ser salvo, mas a criança perde o que estava na tela sem ter enviado.

Gatilho de reavaliação.

- **Relato de conta ficando logada por aba esquecida** (consequência 2). O caminho seria um teto ABSOLUTO ao lado da inatividade — que a **ADR-95** recusou porque deslogava no meio da aula, mas a conversa muda quando o que sustenta a sessão é uma aba e não uma pessoa.
- `unstable_update` saindo do *beta* ou trocando de nome.
- **A suíte ganhando como dirigir a ponte** (pedir `exp:token-expirando` sob comando): aí a tela passa a ser exercitável, e o passo que falta entra.

18.98 ADR-97 — A tela diz POR QUE não entrou, e a suíte espera o desfecho

Data. 2026-09-10. **Status.** Aceita.

Contexto. A verificação "a recusa aparece na tela, e não só no console", de `scripts/e2e/publico/02-login-` reprovou. Atrás de um vermelho estavam DOIS defeitos diferentes, e o primeiro escondia o segundo.

O vermelho em si era da SUÍTE. O passo esperava 3s fixos depois do clique e então lia a tela; naquela corrida o `POST /api/auth/callback/credentials` levou **4,5s**, porque era a primeira vez que o `next dev` compilava `/api/auth/[...nextauth]`, e a foto pegou o botão ainda em "Entrando…". A recusa aparece — 333ms depois do clique, com a rota quente, medido no navegador. O relatório acusou o produto de um modo de falhar MUDO que ele não tinha, e a mensagem do relatório é o pior tipo de erro de diagnóstico: ela não diz "não sei", ela aponta um culpado errado.

Mas procurar o defeito acusado encontrou um de verdade, na vizinhança exata que a verificação nomeia. O `authorize` de `apps/hub/lib/auth.ts` chamava `POST /auth/login` da API **sem prazo**, e só sabe recusar devolvendo `null` — e todo `null` chega ao cliente como a MESMA `CredentialsSignin` da senha errada. Daí dois comportamentos:

- **API pendurada** (não fora do ar — pendurada, que é como aparecem um Cloud Run em partida a frio e um *pool* de conexões esgotado): o `fetch` nunca volta, o `signIn` nunca resolve, e a

pessoa fica em “Entrando…” para sempre, sem mensagem nenhuma. Este É o modo de falhar mudo, e ele estava lá.

- **429 da API** (8 tentativas por minuto com o mesmo *login*): a tela dizia “Usuário ou senha incorretos”, culpando a pessoa por um bloqueio nosso e mandando-a insistir justamente quando insistir estende o bloqueio.

Decisão. Três partes.

1. **A espera pela API tem prazo.** `AbortSignal.timeout(8_000)` no `fetch` do `authorize`. 8s é folgado para o *bcrypt* do `/auth/login` (100ms medidos em *dev*) mais uma partida a frio.
2. **O MOTIVO viaja até a tela, e nenhuma mensagem diz qual metade errou.** O `code` de uma subclasse de `CredentialsSignin` é o único campo que o `@auth/core` carrega até o cliente: ele o escreve no `?code=` da URL de erro e o `signIn` o devolve em `SignInResponse.code` (verificado em execução, não só no código: `code=api-indisponivel` e `code=muitas-tentativas` chegam, e `authorize` devolvendo `null` continua chegando como `code=credentials`). As três mensagens — credencial, indisponibilidade e limite — e os dois códigos vivem num lugar SÓ, `apps/hub/lib/motivos-de-login.ts`, porque quem escreve o motivo (servidor) e quem o lê (cliente) estão em arquivos diferentes: uma cópia da string em cada ponta faria um erro de digitação cair calado na recusa de credencial — de novo a tela culpando a pessoa. Como o `code` vai para a URL, ele só pode dizer de que LADO foi a falha, nunca qual metade da credencial errou (**ADR-70** aplicada a interface: o texto não mente, e também não entrega a lista de *logins* da rede).
3. **A suíte espera o DESFECHO, não o relógio.** O passo corre o `AlertBox` de erro contra a saída do `/login` e deixa cada verificação com o seu nome — credencial inválida que ENTRA falha na hora, em vez de pagar o prazo inteiro e reportar o seletor que faltou.

Alternativas descartadas.

- **Reaproveitar “Usuário ou senha incorretos” para todo motivo** — era o que existia. É a tela mentindo em dois casos, e mentindo contra a pessoa: ela conferiria a senha que estava certa.
- **Dizer qual metade errou.** Nunca: transforma a tela de *login* num verificador de *logins* válidos da rede. A verificação que cobra isso é a mais antiga deste passo e continua verde.
- **Pôr o prazo no CLIENTE** (um `Promise.race` em volta do `signIn`). Cobriria também “o hub não respondeu”, que o prazo do servidor não cobre, mas cria uma mentira NOVA: se a resposta chegar depois de desistirmos, a pessoa está dentro e a tela diz que falhou. A lacuna fica declarada abaixo em vez de trocada por uma afirmação errada.
- **O prazo como variável de ambiente.** Mais um botão para algo que ninguém vai ajustar por rede, e um valor por ambiente que só some da memória de quem opera.
- **Aumentar a espera cega da suíte para 10s.** Mantém a espera cega e só troca o número — e nenhum número que a gente escolha limita uma compilação a frio.

Consequências ruins, aceitas.

- **A distinção é visível a qualquer pessoa.** Quem tenta entrar passa a conseguir separar "o sistema está fora" de "credencial errada". Não revela se um usuário existe — que é o que a regra protege — mas revela disponibilidade. Aceito: o custo de esconder isso é a tela mentindo para a professora que digitou tudo certo.
- **O cliente continua sem prazo.** Se o HUB não responder, o botão fica em "Entrando…" indefinidamente. O prazo de 8s cobre a API, que é o salto que se mede; o outro não tem tela.
- **8s é um número sem medição de produção.** Em produção, uma partida a frio da API mais lenta que isso vai mostrar "não foi possível entrar agora" a quem tinha a senha certa — honesto, e resolvido por tentar de novo, mas é uma recusa que não precisava existir.
- **Os dois caminhos novos não têm passo de e2e.** A suíte não sabe pendurar a API nem esgotar o limite por minuto; a tela não mudou de FORMA (mesmo `AlertBox`, texto diferente), então não há passo novo. `apps/hub/lib/motivos-de-login.test.ts` cobre a propriedade que importa — nenhuma das três mensagens diz qual metade errou, e motivo desconhecido cai na recusa de credencial — e amarra o `auth.ts` aos códigos desta tabela, mas a ligação ponta a ponta foi provada por sonda, não por teste do repositório.

Gatilho de reavaliação. Um *login* legítimo recusado com `api-indisponivel` em produção (o *log* do hub registra `[login] a api não respondeu: TimeoutError`) — aí o número de 8s está apertado, ou a partida a frio da API é o problema a resolver. E se o hub passar a responder o *callback* em mais de alguns segundos com a rota QUENTE, a espera de 20s da suíte deixa de ser folga e volta a ser um prazo a discutir.

18.99 ADR-98 — A plataforma assume UM fuso, numa constante só

Data. 2026-09-11. **Status.** Aceita.

Contexto. A suíte *e2e* da exp08ef4b morreu com `page.click: Timeout 30000ms exceeded` na tela da professora, e o console trazia `Hydration failed because the server rendered text didn't match the client`. O texto que divergia era uma **data**: o HTML do servidor dizia `11/09/2026` e a tela do navegador, `10/09/2026` — medido com a página aberta, logado, nos dois lados.

A causa é uma linha que existia em **onze** lugares, escrita do mesmo jeito em cada um:

```
new Date(iso).toLocaleDateString("pt-BR", { day: "2-digit", month: "2-digit" })
```

Sem `timeZone`, `Intl` usa o fuso de **quem executa**. No hub a mesma linha roda nos dois lados: o contêiner do Cloud Run está em **UTC** e o navegador da professora em `America/Sao_Paulo` (UTC-3). Um registro criado às 22h de Brasília é dia 10 para ela e dia 11 para o servidor. O React acusa, **regenera a árvore**, e o clique dela cai no vazio enquanto isso acontece.

Três coisas tornam esse defeito caro além do normal:

- **É intermitente por desenho.** Só diverge entre 21h e a meia-noite de Brasília — três horas em vinte e quatro. Toda revisão feita de manhã passa.
- **O sintoma fica a três saltos da causa.** Quem lê o relatório da suíte vê "o clique não funcionou", não "a data mudou de dia".
- **Não é só tela.** O "dia" do *check-in* do bem-estar é chave: `toISOString().slice(0, 10)` às 22h grava o dia seguinte, e o de amanhã seria recusado com "você já respondeu hoje". A API já fazia isso certo — e é justamente o ponto: a constante existia, em `src/bemestar/catalogo.ts`, onde ninguém fora do bem-estar a acharia.

Decisão. A plataforma assume UM fuso, declarado em uma constante única, e nenhuma tela formata data na mão.

- `FUSO_DA_PLATAFORMA` vive em `packages/sdk/src/fuso.ts`, com as seis funções que formatam data para gente ler — `data`, `dataCurta`, `dataHora`, `dataCurtaHora`, `hora`, `horaComSegundos` — mais `diaDaPlataforma`, que devolve `YYYY-MM-DD` para comparar e agrupar. Hub e experiências importam de lá.
- A API carrega a **mesma** constante em `apps/api/src/comum/fuso.ts`, porque ela não consome pacote de *workspace* (a regra e a exceção do *logger* estão em `packages/AGENTS.md`). `src/bemestar/catalogo.ts` passou a **reexportar** dela em vez de declarar a sua.
- **O valor inicial é `America/Sao_Paulo`**, e o que importa não é o valor: é existir UM lugar que o define.
- Número não tem fuso. `n.toLocaleString("pt-BR")` para separar milhar continua livre, e é por isso que `toLocaleString` sozinho não serve de sinal — só conta sobre uma data.

Alternativas descartadas.

- **Fixar `TZ=America/Sao_Paulo` no contêiner.** Resolveria a hidratação com uma linha de *Terraform* e nada de código. Descartada por duas razões: ela conserta o **servidor** e deixa o navegador decidir — um tablet da escola com o relógio errado, ou uma criança em viagem, continuam vendo outro dia —, e ela é invisível no repositório. O dia em que alguém mexer no *Terraform* sem saber disso, o defeito volta sem uma linha de código mudar.
- **Formatar tudo no servidor e mandar a *string* pronta.** Mata a divergência na origem, porque o navegador deixa de formatar. Custa caro: toda rota da API e todo *Server Component* passariam a carregar texto de apresentação, o formato deixaria de ser decisão da tela, e as experiências — que são bundles estáticos e às vezes não falam com a API naquele passo — ficariam sem resposta.
- **Um fuso POR ESCOLA, vindo do banco.** É o que o produto vai querer um dia, e é a razão de a constante existir: quando a primeira rede fora de Brasília entrar, ela vira uma leitura, não onze edições. Descartada **agora** porque não há contrato fora de São Paulo, porque o campo teria de subir por token, por *props* e pela ponte `postMessage` até dentro do *iframe*, e porque nenhuma tela hoje sabe de que escola é a data que está mostrando.

Consequências ruins, declaradas.

- **Uma rede em Rio Branco verá a hora de Brasília**, errada em duas horas; em Manaus, uma. Isso não é efeito colateral: é a decisão. O teste `packages/sdk/src/fuso.test.ts` fixa esse comportamento com um caso nomeado ("um relógio no Acre vê a hora de Brasília — é a decisão, não um bug"), para que ninguém o "conserte" sem reabrir esta ADR.
- **Duas cópias da constante** (SDK e API). É o preço de a API não consumir *workspace*. Diferente da cópia da *copy* da exp08ef1b — que é guardada por um comentário e nada mais —, esta é amarrada por `apps/api/tests/fuso-da-plataforma.spec.ts`, que lê o arquivo do SDK e reprova se os valores divergirem.
- **A trava contra regressão cobre o hub, não as experiências.** `apps/hub/lib/formatacao-de-data.test.ts` varre `app/`, `components/` e `lib/` e reprova data formatada sem fuso, apontando arquivo e linha. As experiências não têm a varredura equivalente: elas não têm SSR, então lá o defeito não quebra a hidratação — mostra a data do aparelho, que é mais silencioso e menos grave.
- **Um formato de tela mudou.** O *perfil portátil* recebido em `admin/turmas/[id]` usava `dateStyle: "short"` (ano de dois dígitos) e passou a `dataHora` (quatro). Dois formatos de data e hora na mesma plataforma era inconsistência sem razão registrada; a suíte *e2e* casa com "emitido em", não com a data, e não foi afetada.

Gatilho de reavaliação. A primeira rede fora do fuso de Brasília. Quando um contrato for assinado em Rondônia, no Acre, no Amazonas ou em Mato Grosso, a constante vira um campo — provavelmente em `escolas`, e o caminho até o *iframe* é o trabalho de verdade. Antes disso, qualquer mudança é só trocar a string de um lugar. Um segundo gatilho, menor: se alguma tela precisar mostrar a data **no fuso de quem lê** (um relatório exportado para outro país), o formatador ganha um parâmetro em vez de a constante mudar.

18.100 ADR-99 — O fuso vem de uma variável de ambiente, em três superfícies

Data. 2026-09-11. **Status.** Aceita. **Emenda** à **ADR-98**.

Contexto. A ADR-98 decidiu que a plataforma assume UM fuso, num lugar só, e cravou `America/Sao_Paulo` no código — registrando, no gatilho de reavaliação, que a primeira rede fora de Brasília reabriria a questão. Ela reabriu antes de existir: o dono do produto leu a decisão e afirmou que **vai haver contrato fora de Brasília**, e que o fuso precisa ser configurável por quem opera a *stack*, sem alterar código.

O que muda não é o valor — ele continua sendo `America/Sao_Paulo` por omissão. O que muda é **de onde ele vem**. E isso não é trocar uma linha, porque o valor tem de chegar a **três superfícies com naturezas diferentes**:

- a **API** (NestJS), um processo Node que lê o ambiente;

- o **hub** (Next.js), que roda a MESMA linha no servidor e no navegador — e o navegador não lê `process.env`;
- as **experiências**, *bundles* Vite estáticos, construídos uma vez, servidos de um *bucket* e que **não conhecem contrato**.

Decisão. Uma variável de ambiente, `FUSO_DA_PLATAFORMA`, com o MESMO nome nos dois serviços; ausente vale `America/Sao_Paulo`; e cada superfície a recebe pelo canal que ela já tem.

- **Env, e não coluna no banco. 1 contrato = 1 *stack* de infra** — a granularidade da configuração é a do **deploy**, não a da escola. Coluna permitiria duas escolas do mesmo contrato divergirem (o que ninguém pediu) e custaria migração, tela de administração e a viagem do valor por *token*, *props* e `postMessage`.
- **API**: lida do ambiente e **validada com Zod no BOOT** (`apps/api/src/config/env.ts`). Um *typo* como `America/SaoPaulo` não tem sintoma nenhum até alguém renderizar uma data, dias depois do *deploy* — e aí sai uma exceção do `Intl` no meio de uma requisição. Na subida ele é barato: a revisão nova não pega tráfego.
- **Hub**: lido no servidor e **entregue ao navegador por um `<meta name="unooria-fuso">`** escrito pelo `generateMetadata` do *layout* raiz — o mesmo caminho que `unooria-obs-ratio` já usa (cap. 17). Servidor e navegador leem o valor da **mesma resposta HTTP**, e por isso não têm como divergir.
- **Experiências**: o valor viaja no `config` do `hub:init`, na chave `fuso`, e `createExperiencia` o aplica **antes da primeira tela**. Vai em `config` e não como campo de topo porque o contrato manda: campo novo em `config` é aditivo, mudar a FORMA do `hub:init` obriga a subir `PROTOCOL_VERSION` — e *bundle* já publicado passaria a recusar o *handshake*, o que é tela de erro para a criança.
- **As seis funções não mudam de assinatura.** `data(iso)` continua `data(iso)`: quem sabe o fuso é o MÓDULO, resolvendo por cascata — *runtime* (`hub:init`) → `process.env` → `<meta>` → padrão. Passar o fuso em cada chamada espalharia pelos onze sítios o problema que a ADR-98 juntou num lugar só.

Alternativas descartadas.

- **`NEXT_PUBLIC_FUSO_DA_PLATAFORMA` no hub.** É a forma idiomática do Next de expor configuração ao navegador, e seria uma linha. Descartada porque `NEXT_PUBLIC_*` é **assado no build**: a mesma imagem pode servir mais de um contrato, e nesse dia o valor fica errado **sem nada acusar** — exatamente o modo de falhar mudo que a ADR-98 existe para fechar. O `<meta>` de *runtime* custa cinco linhas e funciona nos dois casos.
- **Um *provider* React descendo o valor por contexto.** É o desenho que um revisor esperaria, e foi o primeiro escrito aqui. Descartado porque o valor passaria a valer **a partir do momento em que o React executa o *provider***, e a ordem entre componentes irmãos é detalhe do renderizador. O `<meta>` está no `<head>`, resolvido antes de a hidratação começar. Consertar

uma falha de hidratação com algo que a hidratação possa quebrar é trocar um defeito por outro mais difícil de ver.

- **Coluna `fuso` em `escolas`, resolvida por requisição.** É o que a ADR-98 apontava como o futuro, e continua sendo o desenho certo no dia em que **um mesmo contrato** tiver escolas em fusos diferentes. Descartada agora porque nenhuma rede pediu isso: uma rede estadual do Acre está inteira no Acre. Ela custaria migração, tela de administração e o caminho do valor até dentro do *iframe* — e o problema que o dono do produto trouxe ("vai haver cliente fora de Brasília") é resolvido por inteiro pela env, que é uma linha de *Terraform* por contrato.

Consequências ruins, declaradas.

- **O mesmo valor é declarado em dois serviços**, e nada no código impede alguém de configurar a API e esquecer o hub. O sintoma seria o defeito da ADR-98 de volta: entre as 21h e a meia-noite, API e hub falando de dias diferentes. O que reduz o risco é fora deste repositório — `docs/INFRA.md` pede um `variable` ÚNICO no *Terraform*, consumido pelos dois serviços. **Não há teste que cubra isso**, porque a configuração não mora aqui.
- **A falha é diferente em cada superfície, de propósito.** A API recusa subir; o hub e as experiências caem no padrão com `console.error`. A assimetria é a decisão — derrubar a subida de um processo é barato, deixar a rede sem tela não é —, mas ela significa que um *typo* só no hub passa **silencioso para quem não lê o console**.
- **Um *offset* fixo (`-04:00`) é aceito.** O `Intl` o aceita e a validação não o recusa — recusar o que o `Intl` aceita seria recusar configuração que funciona. Mas ele **congela o deslocamento**: se a rede voltar a ter horário de verão, a plataforma fica uma hora atrasada sem nada acusar. O `.env.example`, o apêndice B e o `docs/INFRA.md` pedem o nome IANA.
- **O fuso resolvido é memorizado** no módulo do SDK, porque as funções são chamadas por linha de tabela. Trocar a variável **exige reiniciar o processo** — e é isso que um *deploy* faz. Em teste, `esquecerFusoDaPlataforma()` desfaz a memória; em produção ninguém o chama.
- **A experiência roda no padrão até o `hub:init` chegar.** São milissegundos e nenhuma tela de missão mostra data nesse intervalo, mas a janela existe — é a mesma aceita para o Clarity e para o OpenTelemetry Web, pela mesma razão: a alternativa é cravar o valor em cada *bundle*.

Gatilho de reavaliação. Um contrato com escolas em fusos diferentes — uma rede municipal partida por divisa de fuso, ou um contrato estadual que cubra os dois lados de uma. Aí a env deixa de bastar e a coluna `fuso` em `escolas` passa a ser o desenho certo, com o valor viajando no *token* e no `hub:init` por escola. Um segundo gatilho, menor: se a plataforma passar a servir mais de um contrato pela MESMA revisão de Cloud Run, a env deixa de ser por contrato e a decisão inteira precisa ser refeita.

18.101 ADR-100 — Da sala para a missão há UM caminho, e ele leva à ficha

Data. 2026-09-15. **Status.** Aceita.

Contexto. A sala de aula (`/professor/turmas/:id`) listava, por missão liberada, **três** links: "Como funciona" (a ficha), "Guia da aula" e "Conduzir com esta turma". O terceiro apontava direto para `.../resultados`.

Essa rota é um **despachante por `expId`** que termina em `notFound()`. O Instanooria não tem ramo nela — o que se acompanha nessa missão é a **atividade** que o professor montou, não uma tela de resultados por turma. O professor do 8º ano clicava em "Conduzir com esta turma" e recebia **404**; e a tela de erro padrão do Next é texto claro sobre fundo claro, então nem se lia o que era. Relatado pelo dono do produto em 2026-09-14.

O defeito de fundo não é o link errado: é que a **sala precisava saber o que cada missão oferece**, e ela não tem como saber. O Instanooria não tem tela de resultados; o Rastro do Boato não tem guia; a próxima missão publicada terá outra combinação. Toda missão nova obrigaria a mexer na sala, e esquecer disso não quebra nada visível — produz link para 404, ou link que falta.

Duas correções foram tentadas antes desta, e as duas estavam erradas, em direções opostas:

1. **Esconder o link de quem não tem a tela.** Acabou com o 404, mas tirou da sala o caminho da aula daquela missão. Trocou um defeito barulhento por um mudo: **ninguém abre chamado por um link que não existe**.
2. **Fazer o link resolver a atividade da turma** (consultando a API e escolhendo entre ativa, em preparo e encerrada). Levava ao lugar certo, mas mantinha a sala decidindo o que a missão oferece — e escondia as demais atividades daquela turma atrás de uma heurística de desempate.

Decisão. A sala mostra UM link por missão — "Conduzir com esta turma" — e ele leva à FICHA da missão, com `?turmaId=`. "Como funciona" e "Guia da aula" saem da sala.

- **A sala faz uma pergunta só:** "onde fica esta missão?". Quem responde o que ela oferece é o **corpo da ficha**, que já existe por missão (sete corpos mais a Vila do Lume) e já sabe se há guia, se há resultados, se há atividades a configurar.
- **A ficha abre NA TURMA.** O despachante lê `?turmaId=`, resolve a turma com `turmaDaConducao` e a repassa aos corpos. Com turma: uma faixa de contexto diz qual é; `comTurma()` repõe o parâmetro nos links de dentro (resultados, guia da Vila, nova atividade); e a lista de atividades do Instanooria vem **filtrada pela API** (`GET /atividades?turmaId=`). Sem turma, tudo abre como sempre abriu.
- **O filtro é da API, não um `filter` na tela.** A resposta já não traz as atividades das outras turmas. Elas não seriam ruído apenas: seriam o convite a abrir o painel da turma errada no meio da aula.

- **`turmaDaConducao` resolve pela lista de turmas DO PROFESSOR**, e não por `GET /turmas/:id`: um id de turma alheia não está nela, então a ficha não tem como nomear na tela uma turma que não é dele.
- **A faixa de contexto mora no despachante**, não nos corpos — mesma razão da trilha (**ADR-77**) e do `EscopoDaEscola` (**ADR-85**): são sete corpos, e sete cópias divergem.

Alternativas descartadas.

- **Manter os três links e consertar só o destino do terceiro.** É o menor *diff*, e foi a segunda tentativa. Descartada porque deixa de pé a causa: a sala continua tendo de saber o que cada missão oferece, e a missão nova volta a exigir manutenção lá — com o mesmo modo de falhar mudo.
- **Dar ao Instanooria uma tela `.../resultados` própria**, para que a regra "o link leva aos resultados" valesse para todos. É o desenho mais uniforme. Descartada porque construiria uma tela **que duplica o painel da atividade** (monitor, progresso, perfis, revisão), e as duas divergiriam — e porque a uniformidade seria falsa: a missão continua sendo conduzida pela atividade, que é onde o professor configura o conteúdo.
- **Uma tela de lista de atividades por turma**, destino do link quando há mais de uma. Descartada porque a ficha **já é** essa lista, com a explicação da missão em volta — e uma rota nova só para listar repetiria o conteúdo dela.
- **Deixar o link ir sempre ao guia da aula.** É o que o professor abre na hora da aula, e seria útil em quatro das oito missões. Descartada porque o Rastro do Boato não tem guia e o Instanooria precisa da atividade antes: o link acertaria em algumas missões e erraria nas outras, que é exatamente o problema atual com outro destino.

Consequências ruins, declaradas.

- **Um clique a mais para quem ia direto aos resultados.** Quem conduzia o UnooTube abria a tela de resultados a partir da sala; agora passa pela ficha. O ganho é a sala não ter mais como mandar ninguém para um `notFound()`; o custo é real e cai sobre as missões que *funcionavam*.
- **O guia saiu da sala.** Ele é o que o professor abre na véspera e na hora, e agora está um nível abaixo. A ficha o oferece em destaque, mas quem tinha o caminho na memória vai procurá-lo onde não está mais.
- **A ficha passou a ter dois comportamentos** — com e sem turma — e a única coisa que os distingue na tela é a faixa de contexto. Se ela for removida por ser "ruído visual", o professor perde o que diz de qual turma são as atividades que está vendo.
- **`turmaDaConducao` custa uma chamada a mais** por abertura de ficha vinda da sala (`GET /turmas/minhas`). É a lista pequena que as telas do professor já pedem, mas é tráfego que antes não havia.

Gatilho de reavaliação. A primeira missão cuja condução não caiba na ficha — por exemplo, uma que precise de uma tela ao vivo aberta em projetor, na qual passar pela ficha atrapalhe. Aí o caminho único deixa de bastar, e a saída provável é a ficha redirecionar sozinha nesse caso, e não a sala voltar a decidir.

18.102 ADR-101 — Quatro ambientes, e a frente nasce e volta em `dev-local`

Data. 2026-09-15. **Status.** Aceita. **Emenda** à **ADR-53** (um ambiente por branch) e à **ADR-56** (o fluxo, imposto sem *branch protection*).

Contexto. A ADR-53 desenhou um ambiente por *branch* e a ADR-56 escreveu o caminho: trabalho → `dev` → `hml` → `main`. Desde então passou a existir uma quarta camada que nenhum documento registrou — `dev-local`, a *branch* que corresponde ao ambiente **Docker da máquina de quem desenvolve** (`pnpm docker:dev`). Ela está no repositório desde 2026-09, recebe *merge* de *branch* de trabalho e já promoveu para `dev` por PR. Mas nenhum `AGENTS.md`, nenhum *hook* e nenhum *script* a conhecia: a raiz mandava partir de `origin/dev`, o `scripts/worktree.mjs` tinha `origin/dev` ESCRITO com um parágrafo de justificativa, e o `.githooks/pre-push` protegia `dev hml main` deixando `dev-local` aberta a *push* direto.

O custo foi medido três vezes, e nenhuma delas se parece com a causa:

- **Frente nascendo atrás.** `dev-local` recebe o trabalho primeiro e por isso costuma estar **à frente** de `dev`. A ADR-79 nasceu sobre `dev-local`, que já tinha a 78, enquanto `dev` parava na 71 — quem partisse de `dev` reservaria um número já gasto.
- **A camada pulada.** Os PRs #49 e #50 foram de *branch* de trabalho direto para `dev`, e o histórico mostra a prática misturada: PRs `dev-local` → `dev` (#42, #43) convivendo com *merges* locais e com PRs que saltavam a camada. O que se perde ao saltar é o único portão em que **alguém olha a tela**: o CI roda *typecheck*, *lint*, testes e *build*, e não abre navegador.
- **O agente obedecendo o que está escrito.** A regra real vivia apenas na memória automática da ferramenta de codificação — que é local à máquina, não viaja no repositório e **não é carregada em subagente nenhum**. Contra ela, um `AGENTS.md` de 727 linhas dizendo o contrário, com justificativa. Instrução contraditória é resolvida arbitrariamente, e foi.

Decisão. Quatro ambientes, quatro *branches*, e `dev-local` é a BASE e o ALVO de toda *branch* de trabalho.

Branch	Ambiente	Quem testa	O merge dispara
`dev-local`	Docker na máquina	quem desenvolve	nada: a stack local sobe à mão
`dev`	*stack* GCP de dev	o time	*deploy* de dev
`hml`	*stack* GCP de hml	o cliente	*deploy* de hml
`main`	produção	as escolas	*deploy* de produção

- **`dev-local` é *branch* de AMBIENTE**, ainda que o ambiente seja um *container* e não uma *stack* em nuvem. O `.githooks/pre-push` a recusa a *push* direto pela mesma razão que recusa em `dev`: o que entra ali entra por PR, com registro. Ela não aparece no `deploy.yml` — não há *deploy* a disparar.
- **TODA tarefa começa num *worktree***, paralela ou não. A cópia principal é onde a *stack* RODA (`docker:dev`, a suíte e2e, as migrações); editar nela mistura, no mesmo `git status`, o que já subiu com o que ainda está sendo escrito.
- **A base vai ESCRITA**: `origin/dev-local`, literal, no `scripts/worktree.mjs`. Nenhuma ferramenta adivinha — a `default_branch` do GitHub é `dev`, um clone antigo tem `origin/HEAD` em `main`, e a criação automática de *worktree* do agente segue a padrão. Todas erram, e erram em silêncio.
- **`--fatia` passa a ser obrigatória** (ADR-71). Se o *worktree* é o começo de toda tarefa, a cerca de caminhos deixa de ser instrumento de paralelismo e vira declaração de escopo. Frente que não toca módulo de domínio declara a pasta da ferramenta: `--fatia scripts,.githooks`.
- **O *hotfix* desce a escada inteira**, até `dev-local`. Parar em `dev` deixa a base local sem a correção, e a frente seguinte nasce sem ela.

Alternativas descartadas.

- **Deixar como estava**: `dev-local` como conveniência pessoal não registrada. Foi o estado até aqui, e é exatamente o que produziu as três falhas do contexto. Uma regra que existe na cabeça de quem desenvolve e não no repositório é, para qualquer agente e para qualquer pessoa nova, equivalente a não existir.
- **`dev-local` como base, mas PR direto para `dev`**. Economiza um PR por frente. Perde o registro do que entrou no ambiente local e, com ele, a resposta para "isto chegou a rodar na máquina antes de custar um *deploy*?".
- **Escrever a regra só no `AGENTS.md`**. É o que já havia para o fluxo antigo, e não bastou: instrução é contexto, não portão. A ADR-56 já tinha decidido que a regra que o servidor não impõe, o *hook* impõe — esta ADR apenas estende a lista de *branches* protegidas.
- **Renomear `dev-local` para algo como `integracao`**. O nome atual diz a verdade: é o ambiente local. Renomear custaria reescrever referências e não resolveria nada que a ausência de registro causava.

Consequências ruins, declaradas.

- **Um PR a mais por frente.** Quem antes ia de *branch* de trabalho a `dev` agora passa por `dev-local`. Numa equipe de uma pessoa, é cerimônia que ela cumpre sozinha — e cerimônia que se cumpre sozinha é a primeira a ser pulada num dia ruim.
- **`dev-local` e `dev` divergem por construção.** A escada ganhou um degrau, e com ele mais uma *branch* que pode ficar para trás. O *retro-merge* do *hotfix* ficou um passo mais longo.

- ***Worktree* para toda tarefa custa** `pnpm install` mais `prisma generate` por frente — minutos e disco, mesmo para corrigir uma linha. O *flag* `--sem-instalar` existe para a frente que só edita documentação, mas é uma decisão a mais na hora de criar.
- **A fatia obrigatória cobra declaração de quem não precisa dela.** Frente que muda um *typo* em três arquivos de pastas diferentes vai declarar três caminhos, ou usar `PERMITIR_FORA_DA_FATIA`. O risco conhecido é treinar o reflexo de desligar o *hook*.
- **O ambiente local não é reproduzível para mais ninguém.** O portão de `dev-local` é uma pessoa olhando uma tela na máquina dela. Não há artefato, *log* nem evidência de que o olhar aconteceu — ao contrário dos outros três, que deixam execução de CI.

Gatilho de reavaliação. Uma segunda pessoa desenvolvendo. `dev-local` é, por construção, o ambiente de UMA máquina: duas pessoas com Docker local diferente empurrando para a mesma *branch* transformam o degrau em fonte de conflito, em vez de portão. Nesse dia a saída provável é `dev-local` por pessoa (`dev-local-lucas`), ou a camada desaparecer em favor de *previews* efêmeros por PR.

18.103 ADR-102 — A doutrina é uma fonte só, lida por quatro agentes

Data. 2026-09-15. **Status.** Aceita. **Emenda** à **ADR-55** (`AGENTS.md` canônico, `CLAUDE.md` ponteiro).

Contexto. A ADR-55 resolveu o problema de **duas fontes** — a doutrina se escreve uma vez, em `AGENTS.md`, e o `CLAUDE.md` é um ponteiro de uma linha. Dois anos de uso depois, apareceram dois problemas que ela não previa.

O primeiro é tamanho. O `AGENTS.md` da raiz chegou a 757 linhas e 51 KB, o de `apps/api` a 1.407 linhas e 93 KB, o de `apps/hub` a 910 e 53 KB. Os três entram em contexto *inteiros*: quem ia corrigir o check-in de bem-estar recebia a Vila do Lume, a telemetria e as três armadilhas de `RENAME` junto. E a documentação do Claude Code é explícita sobre o custo — "*longer files consume more context and reduce adherence*", com alvo de **menos de 200 linhas**. Não é um corte: é degradação silenciosa, em que a regra certa compete com outras setecentas linhas.

O segundo é portabilidade. `AGENTS.md` virou um **padrão aberto** adotado por cerca de 23 agentes de codificação — Codex, Cursor, Copilot, Zed, Warp, Jules, Devin, Windsurf. O Claude Code é a exceção que lê `CLAUDE.md`, e é só por isso que o ponteiro existe. Mas duas ferramentas que passaram a ser usadas aqui não leem `AGENTS.md` sozinhas: o **Antigravity** lê `.agents/rules/` do *workspace* e corta cada arquivo de regra em **12.000 caracteres**; o **Gemini CLI** procura `GEMINI.md` e, sem configuração, não acha nada. Servir cada uma com a sua cópia recriaria exatamente o problema que a ADR-55 resolveu.

Decisão. Uma fonte por assunto, em `docs/regras/`, com *symlinks* nas pastas que cada ferramenta lê — e nenhum arquivo acima de 12.000 caracteres.

- **Por PASTA continua valendo**, e ganhou profundidade: o `AGENTS.md` de `apps/api` virou um índice de 149 linhas mais 17 `AGENTS.md` de módulo, e o de `apps/hub`, 131 linhas mais sete de área. O mecanismo é o mesmo nas três ferramentas — **o mais próximo vence, e carrega sozinho quando a sessão abre a pasta**.
- **Por ASSUNTO é novo**: a regra que atravessa pastas (*design system*, idioma, convenções, comandos, git) vive em `docs/regras/`, uma por arquivo. É também onde mora todo **contrato entre consumidores** — o `postMessage` e o manifesto não pertencem à pasta de nenhum dos lados.
- **As pastas de regra são *symlinks* para `docs/regras/`**: `.claude/rules/` (que o Claude Code carrega sob demanda, pelo `paths:` no *frontmatter* da fonte) e `.agents/rules/` (que o Antigravity lê inteiro). Editar em `docs/regras/` muda o que todas leem, no mesmo instante.
- **O teto de 12.000 caracteres vale para tudo** — não porque o Antigravity seja o leitor mais importante, mas porque um corte silencioso é o pior modo de falhar: o que some é o *fim* do arquivo, onde está a consequência.
- `.gitignore`: `.agents/` deixa de ser ignorada inteira (`skills/` continua fora — é instalação de cada máquina) e `.claude/rules/` entra ao lado do `settings.json`.

Alternativas descartadas.

- **Um arquivo por ferramenta, com o mesmo conteúdo.** É a solução óbvia e é exatamente o que a ADR-55 proíbe: três cópias divergem sempre, na direção de quem parou de ler as outras.
- **Só `AGENTS.md`, e cada pessoa configura a ferramenta dela.** O Antigravity tem um *setting* para carregar `AGENTS.md` aninhados, e o Gemini CLI aceita `context.fileName`. Mas configuração por máquina não viaja no repositório: quem clonar amanhã tem a doutrina invisível, e não há sintoma — o agente simplesmente não sabe a regra.
- **Cópias geradas por *script*, com um *hook* de verificação.** Resolve a divergência, ao custo de um gerador a manter e de um *diff* de arquivos derivados em todo PR. O *symlink* faz o mesmo trabalho com zero código.
- **Deixar como estava.** Foi a opção até aqui, e o custo já era medido: a regra de `dev-local` vivia num arquivo de 727 linhas e foi ignorada em favor de uma contraditória (**ADR-101**).

Consequências ruins, declaradas.

- ***Symlink* em repositório é frágil fora do Unix.** No Windows sem Modo de Desenvolvedor, o *clone* escreve arquivos de texto com o caminho dentro, e as regras somem sem erro. Hoje só há uma máquina, macOS; no dia em que houver outra, isto é a primeira coisa a conferir.
- **A regra transversal deixou de carregar sozinha no Codex e no Cursor.** Eles leem `AGENTS.md` e *abrem os links* — o que depende do agente decidir abrir. Antes, tudo chegava sem decisão nenhuma; o preço da economia de contexto é este.
- **Quarenta e sete `AGENTS.md` são quarenta e sete arquivos para manter.** A regra de atualizar no mesmo *commit* fica mais cara, e a chance de um ficar para trás sobe com a contagem.

- **Um `CLAUDE.md` de uma linha por pasta nova** — 24 arquivos que existem só porque uma ferramenta procura outro nome. Esquecer um não dá erro: a doutrina daquela pasta simplesmente não chega.
- **Três arquivos continuam acima do teto** (`packages/ui`, `app-92` e a raiz, todos a menos de 20%), porque a fronteira de assunto ainda não é óbvia neles. Estão registrados em `docs/regras/arquivos-de-instrucao.md` para não passarem por decisão.

Gatilho de reavaliação. Uma segunda máquina, ou uma máquina Windows. Os *symlinks* são o ponto único de falha deste desenho, e o modo de falhar é mudo. Se isso acontecer, a saída provável é gerar as duas pastas de regra por *script* no `prepare` do `pnpm install` — a alternativa descartada acima, que volta a fazer sentido quando o *symlink* deixa de funcionar.

18.104 ADR-103 — A frente mergeada se enterra sozinha

Data. 2026-09-15. **Status.** Aceita. **Emenda** à **ADR-61** (a frente paralela é um *worktree*) e à **ADR-101** (quatro ambientes, com `dev-local` como base).

Contexto. A **ADR-61** deu à frente de trabalho um comando de abertura — `pnpm worktree criar` — e deixou o fechamento com quem lembrasse: `git worktree remove`, `git branch -d`, `git push origin --delete`, três comandos em três lugares. Na prática ninguém lembra, e o motivo é estrutural: **o *merge* do PR acontece no GitHub, e nenhum *hook* local dispara nele.** A máquina que abriu a frente não tem como saber que ela acabou.

O custo não é desordem. O *worktree* órfão **segura a *branch*** contra **`git branch -d`**, o *slug* continua reservando uma porta de Vite entre 5201 e 5299, e o `pnpm worktree listar` deixa de responder "quais frentes estão abertas" — que é a única pergunta que ele existe para responder. Em 2026-09-15 o dono do produto apagou três frentes à mão, rodando os comandos que um agente tinha ditado, e pediu que isso parasse de ser trabalho dele.

Decisão. Um comando de poda, e um *hook* que o dispara no único instante em que a máquina fica sabendo do *merge*: o `git pull` em `dev-local`.

- `pnpm worktree limpar` remove, de cada frente já contida em `origin/dev-local`, o *worktree*, a *branch* local e a remota. `--seco` mostra o que sairia sem remover nada.
- `.githooks/post-merge` o chama quando o `git pull` de `dev-local` traz o *merge* do PR. É o gatilho automático, e ele **nunca derruba o `pull`**: qualquer erro sai como aviso e o `exit` é 0. Escape: `PULAR_PODA_DE_WORKTREE`.
- `pnpm worktree criar` também poda antes de abrir — quem começa uma frente já fechou a anterior, e ali a poda não interrompe nada.
- **Três recusas deliberadas.** *Worktree* com alteração não commitada é PULADO, e o `git status` dele é impresso; a frente de onde o comando está rodando é pulada; e ***branch* sem *commit* próprio não conta como mergeada**.

A terceira recusa é a que quase passou. "Estar contida na base" é o critério óbvio, e ele está **errado** para uma frente recém-aberta: ela aponta para o MESMO *commit* da base e é trivialmente ancestral dela. Com a poda rodando dentro do `criar`, abrir uma frente apagaria a que o vizinho tinha aberto minutos antes — e sem sintoma, porque o *worktree* estaria limpo. O critério correto tem duas partes: contida na base **e** com a ponta DIFERENTE da ponta dela. Depois de um *merge* de verdade a base fica à frente (o repositório usa *merge commit* ou *squash*, com *rebase* desabilitado), então a diferença existe; antes do primeiro *commit*, não existe.

Alternativas descartadas.

- **Deixar como estava, documentando melhor.** A receita já estava escrita no `CONTRIBUTING.md` e mesmo assim as frentes ficavam de pé. Uma instrução que depende de alguém lembrar, no fim de uma tarefa que já terminou, é uma instrução que não vai ser seguida.
- **`delete_branch_on_merge` do GitHub.** Resolve UM dos três restos — a *branch* remota — e deixa o *worktree* e a *branch* local, que são os que atrapalham de verdade. Vale ligar assim mesmo, e a poda já tolera a remota ausente.
- **Podar no `prepare` do `pnpm install`.** Roda com muito mais frequência que o *merge*, e num momento em que quem instalou está esperando outra coisa. Remover *worktree* como efeito colateral de instalar dependência é surpresa.
- **Um *workflow* do GitHub Actions apagando a *branch*.** Não alcança a máquina: o *worktree* e a *branch* local são estado de um disco que a esteira não vê.

Consequências ruins, declaradas.

- **Um `git pull` passou a apagar coisas.** É seguro pelas três recusas, mas é um efeito colateral que ninguém espera de um `pull` — e quem não leu esta ADR vai ver *worktrees* sumirem sem ter pedido. O *hook* imprime o que está fazendo e o nome do escape, e é só isso que separa a poda de parecer um defeito.
- **A base está cravada em `origin/dev-local`.** Uma frente que saia de outra base (um PR empilhado, por exemplo) só é podada depois que a pilha inteira chega a `dev-local`. É o comportamento conservador certo, mas quem empilha vê a frente de baixo sobreviver à própria *merge*.
- **Sem rede, a poda subestima o que foi mergeado.** O `fetch` falha em silêncio e o critério passa a valer sobre o que já estava em `.git`: nada é apagado por engano, mas nada é podado também.
- **`git push origin --delete` é irreversível na prática.** A *branch* remota some, e recuperá-la exige o SHA. Ele só roda sobre *branch* já contida na base, então o conteúdo está preservado — mas o nome, não.

Gatilho de reavaliação. A primeira frente que precise sobreviver ao *merge*. Uma *branch* de experimento longo que recebe *merge* da base e continua, por exemplo. Hoje o desenho pressupõe que frente mergeada é frente terminada. Nesse dia a saída provável é uma marca explícita na frente (um arquivo no *git-dir* dela, como a `fatia` já é) que a poda respeite.

18.105 ADR-104 — A base se realinha sozinha depois da promoção

Data. 2026-09-15. **Status.** Aceita. **Emenda** à **ADR-101** (quatro ambientes, com `dev-local` como base de toda frente).

Contexto. A **ADR-101** fez de `dev-local` a base de toda frente e o primeiro degrau da escada. O que ela não previu é o que acontece **depois** da promoção: o *merge* do PR **nasce no destino**. Ao mergear `dev-local` → `dev`, o GitHub não move `dev` para onde `dev-local` está — ele cria um *merge commit* novo em `dev`, com dois pais. `dev` passa a ter um *commit* que `dev-local` não tem, **ainda que o conteúdo seja idêntico**: o `git diff` entre as duas dá vazio, e o `git rev-list` diz "`dev-local` tem 0, `dev` tem 1".

Uma vez é cosmético. O defeito é que **acumula**: a cada promoção mais um, e toda frente nova nasce de uma base cada vez mais atrás do que está publicado. E há um ponto em que deixa de ser cosmético — **o primeiro *hotfix***. Ele sai de `main` e desce por *retro-merge* até `dev`; se `dev-local` está atrás, a correção chega aqui depois, e uma frente aberta no meio nasce sem ela. É exatamente o defeito que a ADR-101 existe para evitar, voltando pela porta dos fundos.

O sintoma apareceu na primeira promoção real, em 2026-09-15, e foi lido como "*dev-local* está atrasado" — que é a descrição certa e a causa errada: não havia trabalho faltando, havia um ponteiro parado.

Decisão. O `pnpm worktree criar` realinha `origin/dev-local` com `origin/dev` antes de abrir a frente, por *fast-forward*, sem perguntar.

- **No `criar`, e não num comando próprio**, porque é ali que a base importa: a frente está prestes a nascer dela. Um comando separado dependeria de alguém lembrar — e foi por depender disso que os *worktrees* ficavam de pé (**ADR-103**).
- **Só quando é *fast-forward* puro.** Se `dev-local` está à frente (o estado normal, com trabalho esperando promoção), não faz nada. Se as duas **divergiram** — cada uma com *commit* que a outra não tem — ele **RECUSA** e manda resolver à mão: mover a base por cima do que só existe nela perderia trabalho, e isso nenhum *script* decide.
- **Empurra `origin/dev` direto para a *branch* remota** (`git push origin origin/dev:refs/heads/dev-local`) em vez de fazer *checkout*. A cópia principal pode estar com `dev-local` aberta e suja, e realinhar não pode depender do estado de quem não pediu nada.
- **O escape do `pre-push` é usado por dentro, e DITO na saída.** `dev-local` é *branch* de ambiente e o *hook* recusa *push* direto nela; aqui o `PERMITIR_PUSH_DIRETO` é deliberado, porque um *fast-forward* não traz conteúdo — só atualiza o ponteiro.

Alternativas descartadas.

- **Recusar e mandar realinhar à mão**, como o `criar` já faz com a `--fatia` e com o nome de *branch*. É a opção coerente com o resto do desenho e foi a recomendada — e foi descartada por quem opera: dois comandos copiados a cada promoção, para consertar algo que não tem decisão nenhuma dentro (o *fast-forward* ou é possível, ou não é).

- **Um comando próprio, `pnpm realinhar`.** Mais explícito, e depende de lembrar. O mesmo argumento que a **ADR-103** já gastou.
- **Só documentar, como item do *template* de promoção.** É o que existia para a remoção do *worktree*, e não funcionou.
- **Promover por *fast-forward*, sem *merge commit*.** Elimina a causa, e não pode: o *job* `mudancas` compara `HEAD^` — o primeiro pai, que num *merge commit* é a ponta antiga da *branch* de ambiente. Sem ele, o *diff* do *deploy* enxerga a coisa errada (**ADR-56**).

Consequências ruins, declaradas.

- **Um *push* direto em *branch* de ambiente passou a acontecer por dentro de um *script*.** A barreira do `pre-push` continua de pé para gente, mas a ferramenta a fura sozinha — e quem não leu esta ADR não tem como saber. O que separa isto de um defeito é uma linha na saída, e nada mais.
- **`criar` passou a ter efeito FORA da frente que cria.** Ele move uma *branch* de ambiente no remoto. É um comando que parecia local e não é mais.
- **Sem rede, não realinha e não avisa direito**: o `fetch` falha em silêncio e a comparação vale sobre *refs* velhas. A frente nasce atrasada sem que nada acuse.
- **A divergência de verdade vira uma parada dura.** O `criar` morre e não abre a frente, no meio do trabalho de alguém. É o comportamento certo — mas é uma interrupção, e ela chega sem aviso prévio.

Gatilho de reavaliação. Uma segunda pessoa desenvolvendo, ou o dia em que `dev-local` deixar de ser de uma máquina só. O realinhamento automático pressupõe que quem roda o `criar` é dono da base: com duas pessoas, mover a *branch* de ambiente do outro no meio do trabalho dele é exatamente o tipo de surpresa que esta decisão aceita hoje porque não há outro.

18.106 ADR-105 — A quota de IA é DADO, editável pela Unooria, e não variável de ambiente

Data. 2026-09-15. **Status.** Aceita. **Complementa** a **ADR-87** (o interruptor da rede) e a **ADR-88** (o painel é só do `admin`); **revê** a decisão do Capítulo 11 de o teto viver em `AI_DAILY_TOKEN_QUOTA_PER_*`.

Contexto. A regra inviolável nº 5 exige quota diária de IA por usuário *e por perfil*, e o `QuotaService` a aplica em onze pontos da api — o tutor da exp08ef1b, a exp02em1b, a exp07ef1b (estudante), e exp01ef1b, exp07ef1b, exp08ef4b, app-90, app-91 e app-92 (professor). O **contador** já era persistido (`ia_uso_diario`); o **limite** não: eram duas variáveis de ambiente, `AI_DAILY_TOKEN_QUOTA_PER_ESTUDANT` e `AI_DAILY_TOKEN_QUOTA_PER_PROFESSOR`, lidas a cada chamada.

Mudar o teto de UMA rede era, portanto, editar o Terraform de outro repositório e fazer *deploy* da api — para uma decisão que não é de código: quanto de IA aquela rede contratou, ou quanto se quer fechar durante um consumo anormal. É o mesmo defeito que a **ADR-87** consertou

para “este produto existe nesta rede?”: configuração comercial da rede vivendo em constante de *deploy*. O pedido veio como cartão (“quota configurável de uso de IA para alunos e professores”), com a direção dada por quem opera: semente por *migration*, alteração só pela Unooria, diretamente no papel do estudante e do professor.

Decisão. O limite vira uma linha por perfil na tabela `ia_quotas`, escopo do contrato inteiro, semeada pela *migration* com os valores que valiam por *env* (300.000 e 2.000.000) para que nenhuma rede mude de comportamento no *deploy*. O `QuotaService` lê a linha a cada `temSaldo`; o `admin` a altera em `/unooria/uso-de-ia` (`PUT /recursos/quotas-de-ia/{perfil}`), com **motivo obrigatório sempre**, e a mudança entra no histórico do painel (`recursos_auditoria`, ação `quota-alterada`, `detalhe = {de, para}`).

- **O banco vence, a *env* morre.** As duas variáveis saem do `env.ts`, dos dois *compose*, do `.env.example` e do Terraform. Manter a *env* como *fallback* seria a segunda fonte de verdade — a tela diria uma coisa enquanto o processo aplica outra, e a primeira pessoa a subir a *env* para “resolver rápido” produziria exatamente esse estado.
- **Vocabulário FECHADO, dois perfis** (`estudante`, `professor`; CHECK espelhando `PERFIS_COM_QUOTA`). Só esses dois chamam IA hoje; gestores e `admin` não têm rota de IA. Todo perfil que não é `estudante` lê a linha `professor` — é o que já valia com as *envs*, agora escrito em vez de implícito num `if`.
- **Sem linha, vale a constante do código e sai `ia-quota-sem-linha`** no registro. Falhar fechado derrubaria toda a IA de uma *stack* recém-provisionada por um dado ausente, com um 429 que não explica nada.
- **Sem *cache***, e é decisão: uma leitura por chave primária numa tabela de duas linhas, antes de uma chamada que dura segundos. A regra da casa é não acrescentar estado sem gargalo medido.
- **Zero e negativo são recusados** (Zod e CHECK), e há um teto de sanidade de 100.000.000 por dia. Fechar a IA de um produto é o interruptor da **ADR-87**, não a quota — e um zero a mais no campo não pode virar quota infinita.
- **Só o `admin`**, pela mesma razão da **ADR-88**: um `gestor_contrato` que pudesse SUBIR a quota passaria a decidir o custo variável da Unooria.

Alternativas descartadas.

- **Manter a *env* e só documentar onde mudar.** É o estado anterior, e o custo dele é um *deploy* por rede por ajuste.
- **Escopo por escola ou regional**, como `disponibilidades`. Nenhuma rede pediu, e dobraria o resolvedor (escola → regional → contrato → padrão) para um número que hoje é um por perfil. Fica como gatilho abaixo.
- **Quota por experiência.** A exp02em1b já tem o controle certo para custo de partida — o teto por SESSÃO (150.000, medido) —, e é por sessão porque a sessão tem começo e fim. A quota diária é o fusível da plataforma, não o orçamento de cada missão.

- **Tabela de histórico própria (`ia_quotas_historico`).** Seriam duas telas de "quem mudou o quê" com colunas quase iguais, e a segunda é a que ninguém abre. O custo de reaproveitar `recursos_auditoria` foi uma ação nova e uma coluna `detalhe` JSONB com CHECK bicondicional — o mesmo desenho de `versao`.
- ***Cache* da linha no processo.** Pouparia uma consulta por chamada de IA, e faria a alteração do `admin` valer "em até N minutos" — com `min-instances: 3` no Cloud Run, em momentos diferentes em cada instância. A promessa da tela ("vale na próxima chamada") só é verdadeira sem *cache*.

Consequências ruins, declaradas.

- **Uma consulta a mais por chamada de IA.** Por chave primária, em tabela de duas linhas; ainda assim, é uma ida ao banco que não existia.
- **A alteração vale para a REDE INTEIRA na chamada seguinte**, e quem já passou do teto novo hoje fica sem IA até amanhã. O diálogo diz isso com todas as letras; é a mesma alavanca de incidente da **ADR-87**, e o mesmo tipo de mitigação.
- **Uma suíte de Postgres passa a ESCREVER numa tabela de contrato** (`quotas-de-ia-postgres.spec.ts` e a e2e `admin/02`). Ela só SOBE o teto e devolve o valor no fim, então uma corrida abortada não recusa ninguém; mas é uma exceção à regra "suíte nunca escreve linha de escopo-contrato", e está escrita nas duas.
- **O motor da exp08ef1b (`geracao/`) e as reações da Vila do Lume continuam FORA do `QuotaService`** — já apontados na **ADR-74** e não corrigidos aqui. Uma quota configurável só vale para quem passa pelo funil; esses dois pontos são cartão à parte, e o da Vila é chamada de estudante sem fusível nenhum.
- **O "dia" da quota é o dia UTC** (`hoje()` do `QuotaService`): o contador zera às 21h de Brasília. Não é escopo desta decisão; a tela diz isso para quem opera.

Gatilho de reavaliação. A primeira rede que pedir teto diferente para UMA escola ou regional, ou o dia em que gestor ou `admin` ganhar rota de IA (aí o perfil entra na tupla, no CHECK e na semente — nunca só num dos três). E o volume: se a leitura de `ia_quotas` aparecer na latência de alguma rota de IA, o *cache* volta à mesa com a medida na mão.

18.107 ADR-106 — O `_template` nasce com a fiação inteira e sem componente de interface

Data. 2026-09-16. **Status.** Aceita. **Complementa** a **ADR-84** (a versão em três lugares), a **ADR-87** (`produto` no manifesto) e a **ADR-69** (`unidade` e liberação por turma); **mantém** a regra de ouro do *design system* (Capítulo 14): nenhum componente local.

Contexto. `pnpm new:exp` copia `apps/experiencias/_template` para começar uma experiência nova. Até aqui o template tinha cinco arquivos de fonte (`App.tsx`, `main.tsx`, `index.html`, `vite.config.ts` e o manifesto), dependia só de React e do SDK, e o *script* de teste era `echo`

`'sem testes ainda'`. O manifesto não declarava `unidade`, `publico` nem `permissoes`. Cada missão nova refazia à mão o que as anteriores já tinham feito: acrescentar o *design system* e a ponte `@edu/ui`, escrever o *bootstrap* do *handshake*, o *singleton* da api, o vocabulário de telemetria, o teste de versão em três lugares e o `vitest`. A exp01ef1b nasceu em React 18 porque o template estava em 18 (**ADR-84** conta o custo); as exp06ef2b, exp07ef1b e exp08ef4b convergiram para a mesma estrutura de `src/lib/` sem que o template a registrasse.

Decisão. O template passa a trazer a fiação que TODA experiência precisa — e continua sem componente de interface:

- **Manifesto completo**: `id`, `nome`, `versao`, `produto`, `series`, `unidade`, `publico`, `capabilities`, `permissoes`, `verbos`.
- **Dependências** iguais às de uma missão real: `@unooria/design-system^0.2.2`, `@edu/ui`, `vitest`; *scripts* idênticos aos das outras experiências.
- **`src/bootstrap.tsx`**: `createExperiencia`, *singleton* do *handle*, telemetria ligada, primeira leitura de estado e erro de abertura reportado pelo SDK (**ADR-84**).
- **`src/lib/`**: `tipos.ts` (`EXP_ID`, `EXP_VERSION`), `api.ts` (toda chamada pelo SDK), `telemetria.ts` (o vocabulário `verbo:objeto` num lugar só, com `resultado` e `contexto` como objetos) e `textos.ts` (o que a tela escreve, em pt-BR).
- **`src/main.tsx`** com as folhas da seção `unooria` (componente + fonte), e o lugar marcado para a seção do clone.
- **`src/versao.test.ts`**: versão nos três lugares, `id` no código e no pacote, e o manifesto dentro do vocabulário fechado (`produto`, `publico`, `unidade`, `permissoes`, `capabilities`).
- **`README.md`** e `AGENTS.md` com a tabela "o que trocar primeiro". O `new:exp` passa a renomear também `src/lib/tipos.ts` e o `README.md`.
- **Interface: só `Button` de `@edu/ui/ds`** na tela de exemplo. Nenhum componente local, nenhuma cor escrita à mão.

Alternativas descartadas.

- **Deixar o template cru e documentar a lista do que falta.** É o estado anterior; o gerador de missões (PRD) passou a listar essa lacuna, e listar não impede a próxima experiência de nascer sem teste.
- **Linkar em vez de copiar** (um pacote `@edu/exp-base` com o *bootstrap*). Faria a fiação valer para todas de uma vez, mas a experiência deixaria de ser um *bundle* autocontido e cada mudança no pacote republicaria 48 caminhos imutáveis. Fiação que precisa valer para todas continua sendo mudança no `@edu/sdk`.
- **Template rico em componentes** (cartão, cabeçalho, botão local). Seria a primeira violação da regra de interface, replicada 48 vezes.

Consequências ruins, declaradas.

- **O template pode envelhecer de novo.** Ele segue a exp08ef4b; quando uma missão consolidar um padrão melhor, ele volta para cá no mesmo PR — senão a próxima nasce velha. Gatilho de reavaliação: duas missões seguidas alterando o mesmo arquivo copiado do template.
- **Mais dependências no *scaffold***: quem cria uma ferramenta de professor sem clone ainda recebe o DS e a ponte — que ela usa de qualquer forma na seção `unooria`.
- **A porta `5173` do `dev` continua sendo a da exp08ef1b**; o template não tem como saber a próxima livre. O `new:exp` imprime o lembrete, e o `strictPort` falha alto se a troca for esquecida.

18.108 ADR-107 — O site público entra no hub como PORTE, e o cadastro de interesse passa pela api

Data. 2026-09-16. **Status.** Aceita. **Substitui** a **ADR-40** na raiz (`/`); **complementa** a **ADR-88** (o painel da Unooria) e a **ADR-70** (as páginas legais). **Abre exceção** à regra inviolável nº 1 (tenancy) e à regra de ouro do *design system* (Capítulo 14), nas fronteiras que esta ADR delimita.

Contexto. O site institucional do ENTRE REDES foi produzido fora deste repositório (`Unooria_Entre_Redes_s`: HTML, uma folha de 24 KB, cinco arquivos de JS de *canvas* e 1,7 MB de imagens e fontes, servidos por um *Worker* que também expunha `POST /api/cadastro` gravando num SQLite (Cloudflare D1) e uma página `/cadastros` protegida por um *token* no fragmento da URL. Ele está aprovado, funciona em celular e em desktop, e é a peça comercial do produto.

A plataforma, enquanto isso, tinha na raiz a landing da **ADR-40** — outra proposta, feita com o *design system*. E o formulário do site vivia numa arquitetura paralela: um banco, um *runtime* e um modelo de autorização que não aparecem em nenhum diagrama deste documento, com dados pessoais dentro.

Decisão. O site vira a raiz do hub, portado 1:1 para o Next; o formulário passa pela api; e o `admin` lê os cadastros no painel da Unooria. Em quatro partes:

- **Porte, não releitura.** `components/site/` traz o `site.css` copiado inteiro (uma única regra acrescentada, marcada no arquivo) e a marcação 1:1 — mesma árvore, mesmas classes, mesmos ids. Os cinco *scripts* viraram `efeitos/*.ts`, corpo literal, com o cancelamento preso a um `AbortController`.
- **O DS não vale ali, e a fronteira é a porta do login.** O site tem tipografia, paleta e movimento próprios. Da porta para dentro, a regra de ouro continua sem exceção.
- **O cadastro passa pela api**: `POST /api/cadastro-de-interesse` (hub, sem sessão) → `POST /prospeccao/publico/cadastros` (api, sem *guard*) → `prospeccao_leads` (Postgres). A defesa é Zod, cota por IP, campo-armadilha e *dedupe* de uma hora.
- **A leitura é do `admin`**, em `/unooria/prospeccao`, com exportação em CSV gerada pela api.

A landing anterior não foi apagada: responde em `/landing-anterior` com `noindex`, e `components/landing/` continua no lugar porque `/privacidade` e `/termos` montam o fundo e a folha dela (**ADR-70**).

Ajuste de 2026-09-17, a pedido do dono do produto. O formulário deixou de ficar aberto no fim da página e passou a abrir numa **janela** (`<dialog>`), pelos três convites que agora dizem "Quero conhecer a solução" — o do cabeçalho, o do fechamento e o da seção de cadastro. Enviar fecha a janela e deixa a confirmação na página, no lugar onde o convite estava; Esc, o `×` do topo e o "Cancelar" também fecham. Nada mudou no contrato com a api — mesmas rotas, mesmos campos, mesma frase de consentimento. A janela reaproveita o desenho que o material já tinha para os outros dois diálogos do site (medidas do índice, moldura, `.button.outline`), e as armadilhas que o ajuste produziu estão em `apps/hub/components/site/AGENTS.md`.

Revisão 19, de 2026-09-18. O material voltou revisado, e o pacote traz o formato que os próximos devem seguir: um `alteracoes.json` com o mapa `before`/`after` das trocas de texto, uma PRÉVIA em HTML (que é uma exportação deste próprio site, com o CSS novo embutido) e a arte trocada. Além dos 19 ajustes de texto, ela centralizou o hero (que passou a ocupar a tela) e trocou os dois campos de partículas do material por UM só, fixo atrás da página inteira. **O mapa não cobria a parte visual**: o texto batia 100% e a tela estava diferente — por isso a conferência de um pacote de revisão compara TRÊS coisas (texto, CSS computado e medidas), e o procedimento está em `apps/hub/components/site/AGENTS.md`.

Alternativas descartadas.

- **Servir o HTML como arquivo estático em `public/`**, com *rewrite* da raiz. É o caminho de menor risco de divergência visual — e deixa o site fora do *build*: sem *typecheck*, sem *lint*, sem as fontes e os assets sob o mesmo pipeline, e com o formulário precisando de um caminho próprio até a api. O porte paga uma vez o custo de conferência e devolve o site ao regime de qualidade do repositório.
- **Reescrever o site com o *design system***. Seria trocar uma peça aprovada por outra parecida, com a fidelidade decidida arquivo a arquivo. O material é de marketing: o DS existe para a plataforma.
- **Manter o *Worker* e o D1 só para o formulário.** Dado pessoal de adulto, com base legal de consentimento, num banco que não tem *backup* conhecido, não aparece no modelo de dados e cuja leitura é protegida por um *token* que viaja no fragmento da URL — e que, uma vez compartilhado, não expira.
- **Guardar o *lead* com `escola_id`** de uma escola fictícia, para não abrir exceção na regra nº 1. Seria inventar um tenant para caber numa política, e a primeira consulta cruzada acharia contatos comerciais dentro de uma rede real.
- **Exigir sessão na escrita** (ampliar a exceção do BFF da **ADR-84**). O visitante não tem conta; é o cadastro que vem ANTES de qualquer conta existir.

Consequências ruins, declaradas.

- **Duas superfícies públicas com propostas parecidas** (`/` e `/landing-anterior`), e o `noindex` é tudo que separa uma da outra no buscador. Gatilho de reavaliação: quando `/privacidade` e `/termos` forem realinhadas ao site novo, `components/landing/` sai inteiro.
- **A fidelidade do porte não tem rede automática.** *Typecheck* e *lint* não veem layout. A conferência é por MEDIDA, e está escrita em `components/site/AGENTS.md`: altura do documento, topo de cada seção e contagem de `.reveal`/`img`/`section`, em 390 e 1.440. No dia do porte, as treze seções caíam no mesmo pixel.
- **Uma rota de escrita sem *guard*** passa a existir na api. É superfície nova de abuso, mitigada por cota e armadilha, não eliminada. Se o volume de lixo passar do que a cota segura, o próximo passo é desafio (*captcha*) — não fechar a rota, que é o formulário do site.
- **A cota por IP conta por instância** do Cloud Run: com N instâncias, o teto real é N vezes maior. Aceitável — o teto existe contra enxurrada, não contra fraude.
- **Não há rotina de retenção nem rota de exclusão** para `prospeccao_leads`. O `GRANT` tem `DELETE` e o pedido do titular é atendido à mão, no banco. A tela diz isso; inventar prazo ou botão seria a mentira mais fácil de escrever e a mais fácil de provar.
- **A frase do consentimento existe em dois lugares** — o rótulo da caixa no site e a constante da api. Mudar um sem o outro faz o banco registrar um aceite que ninguém deu; os dois *AGENTS.md* dizem isso.

18.109 ADR-108 — A missão de sala nasce VAZIA, e a revelação é do relógio da turma

Data. 2026-09-17. **Status.** Aceita. A missão que a aplicava (a v1 de Quem mandou isso?) saiu da trilha na **ADR-114**, e a v2.2 retoma o padrão (**ADR-118**): nasce vazia e a revelação é do relógio da turma, agora com três botões e sem empurrar a triagem. **Aplica** à exp05ef1b a **ADR-69** (a liberação por turma), a **ADR-93** (*polling* HTTP, não WebSocket) e a **ADR-94** (o *latch* de turma). **Rompe**, num ponto declarado abaixo, o padrão que a exp08ef4b e a exp01ef1b estabeleceram: aqui a turma sem preparo **não** cai no banco curado.

> **Contexto**
>
> O PRD da exp05ef1b especifica uma aula síncrona: trinta crianças de dez anos no mesmo grupo simulado, um telão, e um momento em que a sala vê junta o que encaminhou. Para sustentar isso ele pede quatro coisas que esta plataforma não tem, ou tem de outro jeito:
>
> - um **PIN de turma**, digitado pela professora e pelos estudantes, para formar a sala;
> - **WebSocket** para o *broadcast* da revelação coletiva;
> - **AES-256** na reflexão escrita pela criança;
> - **um gate por etapa** — revelar, depois explicar, depois liberar a reflexão —, ou seja, três botões no painel.

> E há uma quinta pergunta que o PRD não faz e que o repositório já respondeu duas vezes: **o que a criança vê quando a professora não preparou nada.** A exp08ef4b e a exp01ef1b caem no banco curado da Unooria e a aula corre.

Decisão. Em cinco partes:

- **Quem entra na sala é a liberação por turma** (**ADR-69**), não um PIN. `exigirMissaoLiberada` já responde "esta missão está aberta nesta turma?", e o `turma_id` do *token* já diz de qual sala a criança é. A guarda entra no funil das rotas de estudante da exp05ef1b, não só na de leitura.
- **A revelação é uma LINHA** em `exp05ef1b_gates_turma`, que a tela lê por *polling* HTTP de 5 segundos em `GET /quem-mandou-isso/gate` (**ADR-93**).
- **A missão nasce vazia.** Sem `aprovado_por_professor` para aquela turma, o estudante vê **tela de espera** — não o banco curado.
- **Três etapas, dois botões e o inverso deles.** O card de síntese entra JUNTO da revelação, ao fim do Mural.
- **A reflexão fica em claro no banco**, protegida por RLS e pelo recorte da rota do professor da turma.
- **O TEXTO da reflexão é de quem LECIONA a turma, e o campo vem `null` para todo o resto.** A rota do painel continua respondendo a gestor, diretor e `admin` — eles precisam do progresso da turma —, mas o parágrafo em si sai só para quem tem matrícula de professor naquela turma. O que todos continuam vendo é `entregouReflexao`: acompanhar a turma não exige ler o que cada criança escreveu.

Alternativas descartadas.

- **PIN de turma próprio.** Seria um segundo mecanismo de pertencimento a uma sala, ao lado do enturmamento e da liberação — com tela de digitação para crianças de dez anos no início da aula, e com a pergunta "quem entra?" passando a ter duas respostas que podem divergir. O PIN também não é autorização: quem sabe o número entra, inclusive de outra turma, e a RLS não teria como saber disso. A liberação por turma já é a resposta, e ela é a mesma de todas as outras missões.
- **WebSocket para o *broadcast*.** Não há WebSocket de aplicação no Cloud Run desta plataforma, e a **ADR-93** já gastou as quatro alternativas. O que se ganharia é latência abaixo de um segundo numa transição que a professora anuncia em voz alta antes de clicar.
- **Criptografia em nível de aplicação na reflexão.** Não existe nenhuma no repositório, e esta seria a primeira: uma chave para gerar, guardar, rodar e restaurar, mantida por uma pessoa, para um campo de 500 caracteres. O Cloud SQL já cifra em repouso, o dado tem RLS, e a rota que o lê é a do professor da turma. Cifrar só aqui também tiraria a reflexão do painel — que é a única razão de ela existir.
- **Queda para o banco curado sem a professora**, como na exp08ef4b. É a alternativa que mais custou a descartar, porque o resto do repositório faz assim. O que a separa é o con-

teúdo: o pacote da exp08ef4b é sobre governança de IA, igual em qualquer escola; o lote da exp05ef1b é escrito EM CIMA do acontecimento que a professora escolheu. Cair no lote padrão no meio da aula dela poria na tela oito mensagens sobre uma mudança no intervalo que não houve naquela escola — e a mecânica que a missão mede, desconfiar de uma urgência que soa familiar, desmonta na primeira leitura. Esperar é pior para o software e melhor para a aula.

- **Reescrever a frase em vez de apertar o código**, no recorte da reflexão. Cinco lugares deste módulo afirmavam que "quem lê é o professor da turma dela", e a api não fazia isso: `professorDoToken` devolve `undefined` para quem não é professor, e `professorLeciona` responde `true` a `undefined`, porque gestor enxerga a escola inteira. Bastaria trocar as cinco frases por "quem lê é o professor ou o gestor", e o repositório passaria a descrever o que faz. Foi descartado porque a frase era a decisão certa e o código é que a contradizia: a caixa livre de menor de idade (LGPD art. 14) não tem por que alcançar quem não dá aula àquela criança, e a alternativa transformaria uma regra em relato. **A exp06ef2b e a exp08ef4b continuam como estavam** — entregam o texto a qualquer gestor, com a mesma frase no comentário. A divergência é conhecida e está registrada no `AGENTS.md` do módulo: apertar as duas muda o que a tela de gestor delas mostra hoje, e isso é frente própria, com a conversa de produto que ela merece.
- **Um terceiro gate para o card de síntese.** O PRD separa "revelado" de "explicação" por mais um avanço docente. Seria um botão a mais no painel que a professora aperta um segundo depois do anterior, todas as vezes — controle morto, e mais uma coisa para esquecer de clicar com trinta crianças esperando. A exp06ef2b e a exp08ef4b já tinham feito a mesma leitura.

Consequências ruins, declaradas.

- **A turma não entra se a professora esquecer de aprovar.** É a consequência direta da missão nascer vazia, e ela custa a aula inteira, não um recurso. A mitigação é de produto e não de código: a tela do professor abre no preparo, o painel diz em letra grande que a turma está esperando, e o banco curado é aprovável com um clique — mas nada disso impede o esquecimento. *Gatilho de reavaliação:* a primeira aula perdida por esse motivo em rede real.
- **Até 5 segundos de atraso na revelação**, e as telas da sala não acendem juntas — elas acendem dentro da mesma janela de cinco segundos. Numa sala em que a professora fala antes de clicar, o efeito é o de um aplauso desencontrado, não o de um defeito. É o preço da **ADR-93**, e ele é conhecido.
- **Quem não confirmou fica fora do histograma**, e o número da sala é MENOR que o número de crianças na turma. "Encaminhada por 18 de 24" não diz "de 24 estudantes", diz "de 24 que confirmaram" — e a tela precisa dizer isso, porque a leitura errada é a natural. Contar a curadoria em andamento seria pior: mostraria à turma uma escolha que a criança ainda podia desfazer.
- **A reflexão fica em claro no banco.** Quem tem acesso ao PostgreSQL de produção lê texto escrito por criança de dez anos. A proteção é RLS mais o recorte da rota, e **não** é criptografia.

Está declarado aqui em vez de descrito como “protegido” em algum lugar, porque a palavra sem o mecanismo é o que engana numa auditoria.

- **Não há rotina de retenção** para `exp05ef1b_reflexoes` (**ADR-13**), como não há para as outras caixas de texto livre do catálogo. Inventar um prazo aqui seria escrever uma promessa que nenhum *scheduler* cumpre.

Gatilho. Uma missão de sala cujo conteúdo NÃO dependa do acontecimento local — aí a queda para o banco curado volta à mesa, e a parte 3 desta decisão deixa de valer para ela. E, para a parte 2, latência medida acima de cinco segundos em rede de escola: o problema passaria a ser o intervalo do *polling*, não o transporte.

18.110 ADR-109 — O rótulo da tabela da BNCC de Computação é CENTRADO, e a atribuição é uma PARTIÇÃO

Data. 2026-09-17. **Status.** Aceita. **Corrige** a extração do anexo do Parecer CNE/CEB nº 2/2022 descrita em “O currículo como dado”.

> **Contexto**
>
> `scripts/extrair-bncc.mjs` associa cada habilidade ao rótulo da linha dela por COORDENADA, e a regra que ele usa veio da BNCC regular: *o último rótulo cujo topo está na altura da habilidade ou acima dela*. Ali a regra é correta — o InDesign alinha a célula pelo topo, e um rótulo que cobre três habilidades aparece na altura da primeira.
> **O anexo de Computação usa outro layout**: o rótulo é CENTRADO na célula. Um eixo que cobre quatro habilidades aparece na altura da terceira — então a regra do topo dava às duas primeiras o eixo ANTERIOR, e à primeira de cada ano um `null`. Somando eixo e objeto de conhecimento, **80 das 130 linhas estavam erradas**.
> Nada disso aparecia. O tipo é `string | null`, o `tsc` não tem o que dizer, e a tela do plano de ensino agrupa por eixo e lista o objeto: o resultado era um agrupamento plausível e inteiramente falso, que a professora leria como currículo. Dois efeitos colaterais vieram no mesmo pacote: onze linhas com o texto “OBJETO DE CONHECIMENTO” gravado como o objeto delas (a lista de ruído do extrator só trazia o plural da BNCC regular), e a `EF05CO011` inteiramente ausente — a Base a publica com TRÊS dígitos, num erro de digitação dela, e a expressão regular do extrator só reconhecia dois. O teste do dataset registrava “129 (Computação, de 130 no anexo)”, isto é, documentava a perda em vez de recusá-la.

Decisão.

- **A atribuição é uma PARTIÇÃO, não um vizinho.** Os rótulos ladrilham a coluna sem buraco nem sobreposição, então as habilidades de cada um formam um bloco CONSECUTIVO, e o centro do bloco é o centro da célula — que é onde o rótulo está. Escolhe-se a divisão em blocos consecutivos que minimiza a distância entre o centro de cada rótulo e o centro do bloco dele. A busca é exaustiva e cabe: uma página tem no máximo oito habilidades.

- **O código fica como a Base o publica**, `EF05CO011` inclusive.
- **A correção é do GERADOR**, nunca do arquivo gerado.
- **A prova é ESTRUTURAL, não amostral** — ver abaixo.

Alternativas descartadas.

- **O rótulo de centro mais próximo.** Resolve a maioria e erra na fronteira entre células de tamanhos diferentes: na página do 5º ano o PENSAMENTO COMPUTACIONAL cobre UMA habilidade e o MUNDO DIGITAL cobre três, e a `EF05CO05` fica mais perto do centro da célula pequena. Foram cinco linhas erradas — menos que 80, e do mesmo tipo.
- **Normalizar `EF05CO011` para `EF05CO11`.** Seria inventar um código que nenhuma rede vai procurar e que não existe em documento nenhum. O erro de digitação é da Base; consertá-lo aqui é criar uma segunda verdade.
- **Editar o arquivo gerado à mão.** Eram 80 linhas, e a próxima execução do extrator as desfaria — em silêncio, que é o modo de falhar que o cabeçalho "NÃO edite à mão" existe para evitar.
- **Conferir por amostragem** (alguns códigos escritos no teste). Foi o que existia, e foi o que deixou 80 linhas passarem: uma amostra confirma o que ela contém e não diz nada sobre o resto.

Como isto se prova, e por que a prova basta.

O teste em `apps/api/src/curriculo/curriculo.spec.ts` não compara com uma amostra: ele cobra a ESTRUTURA que o documento tem.

- **Todo ano traz os três eixos, uma vez cada, na ordem** PENSAMENTO COMPUTACIONAL → MUNDO DIGITAL → CULTURA DIGITAL. Onze anos, e nenhum deles saía assim com a regra antiga.
- **Nenhum objeto de conhecimento reaparece** depois de outro dentro do mesmo ano — a célula da tabela não se parte em duas.
- **Nenhum cabeçalho** ("EIXO", "OBJETO DE CONHECIMENTO", "HABILIDADE") entra como valor.
- **Toda habilidade do Fundamental tem eixo, e nenhuma do Médio tem** — no Médio a coluna é COMPETÊNCIA ESPECÍFICA, e inventar um eixo ali seria pior que a ausência.
- **O total é 130**, e não mais "129 de 130".

A conferência independente que produziu esta ADR foi um segundo extrator, escrito do zero e por LINHA em vez de por bloco: os dois concordam nas 130 descrições, caractere a caractere, e as cinco divergências de eixo entre eles foram decididas a favor deste pela coluna do objeto de conhecimento — `EF15CO07` tem "Sistema Operacional" por objeto, que é MUNDO DIGITAL em todos os outros anos.

Consequências ruins, declaradas.

- **A busca exaustiva é exponencial no número de rótulos.** Cabe porque uma página da Base tem no máximo oito habilidades e três eixos. Um documento com dezenas de linhas por página pediria programação dinâmica — e ninguém vai notar isso até tentar.
- **A regra do topo continua valendo para a BNCC regular**, e as duas convivem no mesmo arquivo. Quem escrever a extração do próximo documento tem de PERGUNTAR qual layout ele usa, e a resposta não está no PDF: está em olhar uma página.
- **Os 45 % sem exemplos continuam sem exemplos.** O casamento das colunas de comentário é por ORDEM, e ele se recusa quando as contagens divergem — o que é a decisão certa (atribuir o exemplo de uma habilidade a outra seria pior), mas não é cobertura.

Gatilho. Um terceiro layout de tabela na Base, ou o dia em que o MEC republicar o anexo com a `EF05CO011` corrigida: aí o código muda no documento, e o dataset muda junto.

18.111 ADR-110 — O código da experiência diz onde ela mora na trilha

Data. 2026-09-23. **Status.** Aceita. **Revê** o prefixo de tabela da **ADR-20**; o resto dela continua valendo.

> **Contexto**
>
> O identificador de toda experiência era `exp-NN-slug`, e o `NN` era a **ordem de criação**: a `exp-01` era o Instanooria, do 8º ano; a `exp-04` era o Foto Conta, do 1º. O número PARECIA dizer a série e não dizia, e o sintoma é o mais caro que um identificador pode ter — quem lê "exp-04" numa conversa sobre o 4º ano está falando de outra coisa, e nada avisa.
> O formato nunca foi decidido. Ele existia implícito em seis expressões regulares copiadas na api e no hub, com três variações (uma aceitava `exp-` seguido de qualquer coisa, outra exigia dois dígitos), e no *script* de *scaffolding*. As faixas "70 = transversal" e "9x = ferramenta do professor" eram costume citado em documento, sem registro. A **ADR-20** USAVA o número — para o prefixo de tabela — sem dizer de onde ele vinha.
> A trilha do Entre Redes tem uma forma fixa: 12 anos (1º ao 9º do Fundamental, 1ª à 3ª série do Médio), 4 unidades por ano, uma missão por unidade. O Lucas padronizou a nomenclatura do produto a partir dela.

Decisão.

- **Missão da trilha:** `exp` + ano com dois dígitos + etapa (`ef` ou `em`) + unidade (1 a 4) + `b`. `exp08ef1b` é o 8º ano do Fundamental, unidade 1; `exp02em1b`, a 2ª série do Médio, unidade 1. O `b` é fixo: é o nome que o produto dá à unidade (o bimestre), e não carrega informação que o resto do código não tenha.
- **O que não é missão** (ferramenta do professor, o transversal): `app-NN-nome`, com o número e o nome que já tinha — `app-70-vila-do-lume`, `app-90-plano-de-aula`. Não mora numa unidade, então não tem código de grade para ter.

- **O código é o id em TODO lugar**: manifesto, pasta, pacote `@edu/*`, serviço do *compose*, prefixo de tabela (`exp08ef1b_`, `app_70_`), *model* do Prisma, caminho no *bucket*, telemetria, `service.name`, nome de *branch* e escopo de *commit*. Módulo da api e arquivo do hub continuam pelo NOME (`src/quem-mandou-isso/`, `missao-rastro-boato.tsx`): nome é conteúdo, código é lugar.
- **O código de missão é DERIVADO, nunca escolhido.** `codigoDaMissao(series[0], unidade)` o monta a partir do manifesto; `apps/api/tests/codigo-da-experiencia.spec.ts` reprova o manifesto de missão cujo `id` não é esse, a pasta cujo nome não é o `id` e a experiência sem `unidade` fora do formato `app-`. O `pnpm new:exp` recebe o código e escreve `series` e `unidade` a partir dele.
- **O formato mora em um lugar por lado**: `apps/api/src/comum/codigo-da-experiencia.ts` e `packages/sdk/src/codigo-da-experiencia.ts`, amarrados pelo mesmo teste, que lê o arquivo do SDK — o desenho do fuso (**ADR-99**). As seis cópias saem.
- **A migração dos dados** (`20260923120000_codigo_das_experiencias`): `RENAME` por OID de tabela, *constraint*, índice, política e *sequence*, e `UPDATE` do id em `experiencias`, `experiencias_versoes`, `liberacoes`, `progresso`, `telemetria_eventos`, `disponibilidades` e `recursos_auditoria`. O `FORCE ROW LEVEL SECURITY` sai e volta dentro da transação — com ele o `UPDATE` casaria zero linhas em silêncio — e a asserção roda antes de ele voltar.

Antes	Série	Código
`exp-04-o-que-a-foto-conta`	1EF, U1	`exp01ef1b`
`exp-02-unootube`	4EF, U1	`exp04ef1b`
`exp-05-quem-mandou-isso`	5EF, U1	`exp05ef1b`
`exp-06-rastro-boato`	6EF, U2	`exp06ef2b`
`exp-07-o-eco-no-grupo`	7EF, U1	`exp07ef1b`
`exp-01-instanooria`	8EF, U1	`exp08ef1b`
`exp-08-manifesto-ia`	8EF, U4	`exp08ef4b`
`exp-03-unoogpt`	2EM, U1	`exp02em1b`
`exp-70-vila-do-lume`	—	`app-70-vila-do-lume`
`exp-90-plano-de-aula`	—	`app-90-plano-de-aula`
`exp-91-bem-estar`	—	`app-91-bem-estar`
`exp-92-plano-de-ensino`	—	`app-92-plano-de-ensino`

Alternativas descartadas.

- **Manter `exp-NN-slug` e documentar a faixa.** Registra o costume e deixa o defeito: o número continua parecendo a série. E o slug no id não acrescenta nada que o `nome` do manifesto não diga.

- **Trocar só o id como DADO** (manifesto, banco, *bucket*) e manter pasta, pacote e tabelas. Seria o pior dos dois: as tabelas `exp_04_` passariam a pertencer à `exp01ef1b`, e cada leitura do código pediria o de-para de cabeça.
- **Código com o slug** (`exp08ef1b-instanooria`). Carrega o nome duas vezes, e o slug muda quando a missão é renomeada — o código é o lugar na grade, e o lugar não muda com o título.
- **Dar código de grade também às ferramentas**, com série e unidade fictícias. Seria o mesmo defeito ao contrário: um número que diz uma série que a ferramenta não tem.
- **Deixar `recursos_auditoria` com o id antigo**, respeitando o *append-only* da **ADR-89** à risca. O histórico de cada experiência SUMIRIA da tela dela, que filtra pelo id novo; a outra saída, traduzir na leitura, deixaria um de-para vivendo no código para sempre. Trocar o id não muda o que a linha afirma — quem, quando, em que recurso —, só o nome do recurso. É a única escrita que o *trigger* deixa passar, e só dentro desta migração.
- **Uma cópia só do formato**, com a api importando o SDK. A api não consome pacote de *workspace* (`packages/AGENTS.md`), e arrastar o SDK — *handshake*, OpenTelemetry de navegador — para dentro do NestJS por causa de uma expressão regular é pior que a cópia amarrada.

Consequências ruins, declaradas.

- **O de-para embaralha os números.** A `exp01ef1b` é a antiga `exp-04`, não a antiga `exp-01`. Ler *commit*, PR e PDF anteriores a 2026-09-23 pede a tabela acima. O que o formato garante é que a referência antiga não RESOLVE para outra coisa: `exp-01` deixou de ser um id válido, ao contrário do que renumerar ADRs faria (ver a abertura deste capítulo).
- **Uma missão por unidade, uma série por missão.** O código não tem lugar para duas missões na mesma unidade, e o teste exige `series` com um elemento. É a forma do produto hoje (48 missões = 12 anos × 4 unidades); se ela mudar, o formato cresce.
- **O que está fora do repositório carrega o id antigo até ser trocado à mão**, uma vez por contrato: as versões anteriores dos *bundles*, os vídeos da `exp04ef1b` e a mídia que a api GEROU para a `exp07ef1b` (`experiencias/{expId}/` no *bucket* de *assets*: sem a cópia, a api gera tudo de novo no Vertex, e a primeira geração passa dos 30 s que o proxy do hub espera) (o comando está em `docs/INFRA.md`; o *deploy* REPROVA até os vídeos estarem no prefixo novo, de propósito), e, no Terraform, o *uptime check* da versão ativa, os painéis por `service.name` e as métricas de log por `codigo` (`exp07.*` virou `exp07ef1b.*`). Série temporal que cruza a troca aparece partida em duas.
- **Perfil portátil assinado antes da troca** (**ADR-35**) carrega `exp-70-vila-do-lume`. É documento assinado, e não se reescreve: ele continua sendo importado (o `expId` ali é texto livre), só não acha mais o nome da experiência no catálogo.
- **Os serviços do *compose* mudaram de nome** (`exp-01` → `exp08ef1b`): os *containers* antigos ficam órfãos até um `--remove-orphans`.
- **O nome de índice cresce até 3 caracteres**, e um deles passou de 63: o Postgres corta o FIM em silêncio e o Prisma corta o MEIO. A migração aplica o corte do Prisma e aborta se

aparecer nome longo sem sufixo conhecido — sem isso, `prisma migrate diff` acusaria *drift* num nome que ninguém escreveu.

Como isto se prova. A migração rodou num Postgres descartável sobre o estado anterior, com linhas dos doze ids em todas as tabelas afetadas e uma de id desconhecido: os doze trocaram, o desconhecido ficou, o `FORCE` e o *trigger* voltaram, o catálogo (tabelas, RLS, políticas) saiu idêntico a menos dos nomes, e o `prisma migrate diff` contra o *schema* novo devolveu exatamente o *drift* que já existia antes da troca — nenhum a mais.

Gatilho. Uma segunda missão na mesma unidade; uma missão que valha para mais de uma série; ou o produto passar a dividir o ano em outra coisa que não bimestre — aí o `b` passa a mentir.

18.112 ADR-111 — Equipe é quem divide um computador, e o painel conta por computador

Data. 2026-09-25. **Status.** Aceita. **Estende** a sala da **ADR-69** sem mexer na porta dela: a liberação continua por turma, e o caminho da sala para a missão continua sendo a ficha (**ADR-100**).

> **Contexto**
>
> A escola pública tem menos computadores que crianças. Na sala real, dois ou três estudantes sentam diante da mesma tela e fazem a missão juntos, com o login de UM deles. A plataforma não sabia disso: o painel do professor contava a sala como se cada criança tivesse o próprio computador, e dizia "12 não começaram" quando os 12 estavam sentados ao lado de quem começou. Os totais do UnooTube e do unooGPT caíam na mesma proporção, e o professor aprendia a desconfiar do painel justamente na aula em que ele mais precisava dele.
> Nada no modelo dizia QUEM ESTÁ COM QUEM. A liberação (**ADR-69**) é por turma; o dado da experiência é por estudante. Faltava a camada do meio, e duas restrições da sala decidiram a forma dela:
>
> - **Nem toda missão sobrevive a uma tela dividida.** A Vila do Lume (app-70) mede um retrato de valores INDIVIDUAL, que alimenta a largada do Instanooria e o CSV da análise: uma equipe gravaria as escolhas do grupo como o perfil de uma criança. E o Instanooria divide a turma em dois lados do *feed* (A e B): uma equipe com gente dos dois lados diante da mesma tela desmancha a missão.
> - **Quem chega atrasado não faz login.** Na sala de computador compartilhado, a criança que entra depois senta num computador já logado. Nada que dependa do login dela vai acontecer.

Decisão (épico E13, "Equipes na missão").

- **Equipe é quem DIVIDE UM COMPUTADOR numa missão**, e não grupo pedagógico. Ela existe por turma × missão, em três tabelas de PLATAFORMA — sem prefixo de experiência, como `liberacoes`, com `escola_id` e RLS forçada —, no módulo `apps/api/src/equipes/`: `equipes_montagens` (a decisão da aula: `modo`, `computadores`, `ausentes` e a `versao`), `equipes` (cada computador, com nome e ordem) e `equipes_membros` (quem senta diante de cada um, com `turma_id` e `exp_id` repetidos para o banco garantir **uma equipe por criança por missão**).
- **O professor diz quantos computadores há e marca quem faltou, e a plataforma SEMPRE pergunta como montar**, sem opção marcada: "Sortear as equipes" ou "Eu separo" (nascem N equipes VAZIAS, e o professor põe cada estudante numa). Quando a turma tem equipes salvas em OUTRA missão, aparece a terceira: "Manter as equipes da última missão", reprojetada sobre a turma de hoje — quem saiu da turma cai (e é contado), quem chegou depois fica sem equipe, e os ausentes de lá não vêm, porque falta é da aula, não da criança.
- **O tamanho é rígido**: os tamanhos diferem em no máximo 1, os maiores primeiro (25 presentes em 10 computadores dão cinco equipes de 3 e cinco de 2). `computadores` ≥ presentes quer dizer cada um no seu computador, e é legítimo: é a sala com uma máquina por criança.
- **Só três missões aceitam equipe nesta onda**, e a lista mora no CÓDIGO: `EXPERIENCIAS_COM_EQUIPE`, em `equipes/regras.ts` — `exp08ef1b` (Instanooria), `exp02em1b` (unooGPT, a Missão Tempestade) e `exp04ef1b` (UnooTube), conferidas uma a uma. Ficam de fora a `app-70-vila-do-lume`, pelo retrato individual acima; a `exp07ef1b` (O Eco no Grupo), que vai ser substituída por A Seleção, e esta nasce com equipe; e `exp01ef1b`, `exp05ef1b`, `exp06ef2b` e `exp08ef4b`, que o dono do produto retira da trilha logo depois desta frente.
- **Quem MONTA é só o professor que leciona a turma**, como quem libera (**ADR-69**): quem não leciona recebe **404**, nunca 403 — um "sem permissão" já contaria que a turma existe. Gestores e `admin` só LEEM.
- **Concorrência otimista pela `versao`.** O `PUT` diz em que versão a tela se baseou (`0` sem montagem), e versão velha é **409** com `versaoAtual`: o computador da sala e o celular do professor abrem a mesma montagem, e o segundo a salvar não apaga em silêncio o que o primeiro arrastou. A troca é INTEIRA (as equipes antigas saem, as novas entram), e a sugestão e a "última" nunca gravam.
- **As equipes são SEMPRE editáveis**, inclusive com a missão encerrada — é por ali que se corrige "ela faltou". E **não existe encaixe automático de quem chega atrasado**: o gatilho seria o login dele, que não acontece.
- **O estudante vê a própria equipe** numa faixa do hub, fora do `iframe`, acima da missão: "Sua equipe: Equipe 3 · com Ana e Bruno", só com o PRIMEIRO nome dos colegas (regra 6). `GET /equipes/minha` responde **sempre 200**, com `equipe: null` quando não há: um 404 viraria `api-recusou` no log do hub a cada abertura de missão, para a imensa maioria das crianças, que não tem equipe. A sala mostra uma `Tag` "N equipes prontas", que NÃO é link (**ADR-100**): montar e editar é na ficha.
- **Os painéis do UnooTube e do unooGPT contam por COMPUTADOR.** Uma linha por equipe, representada pelo membro que entrou (`colapsarEmUnidades`, em `equipes/regras.ts`: a

linha de quem iniciou; com dois logins no mesmo computador, a do mais recente), com `equipe` preenchida na linha; os totais contam essas unidades, e o resumo diz `porComputador: true`. Sem montagem, o painel sai exatamente como era.

- **O Instanooria é a PRÓXIMA frente** (`feat/equipes-instanooria`), e o gancho já existe: `EquipesService.registrarFonteDeGrupo(expId, fonte)`, pelo qual o módulo da missão diz o lado de cada criança sem que `equipes/` importe nada dele. Com fonte registrada, o sorteio evita juntar os dois lados na mesma tela, usa os NEUTROS (quem não tem perfil da Vila) como enchimento e CONTA as equipes mistas quando a aritmética não fecha. O planejado: equipes do MESMO lado do *feed*, e o lado da equipe resolvido na LARGADA do *run* — pelo *run* de um colega de equipe, ou pela maioria —, com uma origem de lado nova, `'equipe'`. Hoje nenhum módulo registra fonte, e o `agrupamento` da sugestão volta `null`.
- **A evidência pedagógica é outro plano** (épico E14: a replicação do resultado da equipe por habilidade da BNCC). Esta frente não a calcula; guarda a composição que ela vai ler. E a falta marcada na montagem **não é frequência**: tira do sorteio, e só.

Alternativas descartadas.

- **Equipes fixas por turma, para o bimestre.** Quebra na primeira missão que pede outro tamanho, e na aula em que o laboratório tem menos máquinas — o número de computadores muda de aula para aula. O reúso que ela prometia é o que "Manter as equipes da última missão" já dá, sem congelar nada.
- **Um passo "confirmar quem participou" no fim da aula.** Trabalho a mais em TODA aula para corrigir a exceção, na hora em que a sala já está saindo. As equipes sempre editáveis cobrem a correção quando ela é necessária, e só então.
- **Encaixar automaticamente quem chega atrasado.** Nunca dispararia: na sala de computador compartilhado, o atrasado senta num computador já logado e não faz login. Um gatilho que depende do login dele seria código sem evento.
- **Um "modo projeção" do professor dentro de cada missão.** Tocaria todas as missões, uma a uma, e cada missão nova herdaria a obrigação. Projetar a missão é um estudante logado no computador do professor; o que a plataforma projeta é a lista das equipes ("Mostrar equipes"), para cada criança achar o seu computador.
- **Uma coluna ou *flag* no catálogo** em vez da lista no código. "Funciona com duas crianças numa tela" é propriedade do DESENHO da missão — quem clica, quem lê, o que o painel conta —, não configuração da rede; e o catálogo `experiencias` é escrito pelo CI e está vazio em desenvolvimento. Missão que entrasse por *flag*, sem conferência, abriria para equipe e continuaria contando por criança no painel.
- **Uma tabela à parte com o lado de cada equipe, para o Instanooria.** O `PUT` troca a montagem inteira, e as equipes antigas saem com tudo o que pende delas, por cascata: o lado seria perdido a cada edição, em silêncio. O lado é resolvido na largada do *run*, a partir do *run* de um colega ou da maioria — dado que já existe e que a edição não apaga.

Consequências ruins, declaradas.

- **Missão nova com equipe passa por *deploy*.** A lista no código é de propósito (a conferência é do desenho), mas A Seleção e as próximas entram por PR, nunca por um painel.
- **No unooGPT, a equipe divide a conta de UM estudante.** As 3 perguntas e a cota diária de IA são de quem entrou, e o que for escrito fica no nome dele. O guia da aula da `exp02em1b` diz isso ao professor; o da `exp04ef1b` diz que os 3 vídeos salvos passam a ser da equipe.
- **O dado da experiência continua por criança.** Quem não entrou não tem linha em `exp04ef1b_sessoes` nem em `exp02em1b_sessoes`; quem conta por computador é a LEITURA. Relatório novo que não passe por `colapsarEmUnidades` volta a contar por criança — e o mapa de equipes tem de ser lido DENTRO de `comTenancy`: fora dele a RLS das três tabelas devolve vazio, e o painel volta a contar por criança sem erro nenhum.
- **A criança que chegou atrasada só entra na equipe pela mão do professor.** Até lá, no painel, ela é a própria linha — e aparece como "não iniciou", que é o sintoma que esta decisão existia para tirar.
- **A falta marcada pode ser lida como chamada**, e não é: não vai a diário nem a relatório. Quem a usar como frequência vai ler errado.
- **Criança que muda de turma deixa a linha em `equipes_membros`.** A leitura a filtra por `users.turma_id` (`equipes/consulta.ts`), então a equipe aparece menor e o próximo salvamento limpa; o "Manter as equipes da última missão" a conta em `saidas`, para a tela dizer por que uma equipe encolheu.
- **A faixa do estudante ocupa uma linha** de uma coluna de altura travada; o aparelho da experiência recalcula a escala e fica um pouco menor.

Como isto se prova. `apps/api/tests/equipes-regras.spec.ts` cobre as regras puras: os tamanhos (nunca diferem em mais de 1 e somam os presentes), o sorteio com e sem lados sob semente fixa (12 A + 8 B + 5 neutros em 10 computadores sem nenhuma equipe mista; 13 A + 15 B em 7 computadores de 4 com UMA mista, contada), o modo manual, a validação que devolve a lista INTEIRA de problemas e o `colapsarEmUnidades`. `apps/api/tests/equipes-postgres.spec.ts` cobra o banco: RLS habilitada E forçada nas três tabelas, isolamento de verdade entre escolas, a *role* sem *ownership* montando pelo serviço, o `UNIQUE` de uma equipe por criança por missão segurando mesmo com o código errado, os *CHECKs* iguais a `MODOS`/`NOME_MAX`, o 409 de versão, a troca inteira, o 404 para quem não leciona, a `minha` sempre 200 com primeiros nomes, a contagem na sala e a "última" reprojetada. Os dois painéis têm os seus casos em `unootube-postgres.spec.ts` e `unoogpt-postgres.spec.ts` (equipe de 3 com um login vira UMA linha; dois logins, vale o último), e a suíte `scripts/e2e/professor/10-equipes-na-missao.mjs` percorre a ficha, a `Tag` da sala, "Mostrar equipes" e a faixa do estudante.

Gatilho. Uma missão com equipe cujo dado precise ser da EQUIPE, e não de quem entrou — a primeira candidata é A Seleção —; a evidência do E14 pedindo uma composição que a troca inteira não preserva (histórico de quem esteve em qual equipe em cada aula); ou uma rede em que a sala tenha um computador por criança de forma estável, em que a montagem vira trabalho sem retorno.

18.113 ADR-112 — A evidência é por atividade e rodada, e a melhor rodada vale por habilidade

Data. 2026-09-25. **Status.** Aceita. **Lê** a composição das equipes da **ADR-111** sem que `equipes/` saiba que a evidência existe, passa pela porta da sala da **ADR-69** nas rotas do estudante e guarda o gabarito no servidor, como a **ADR-09**. Responde ao gatilho da **ADR-111** sobre "quem esteve em qual equipe em cada aula": a cópia guarda a equipe que teve.

> **Contexto**
>
> A missão já via o que a criança fazia — tirou o post do ar, respondeu a pergunta — e isso morria dentro da experiência. No fim do bimestre, o professor não tinha como dizer "a Ana atingiu a EF08CO05 e o Caio ainda está em processo", que é a frase que a rede pede e que a BNCC organiza. Faltava uma camada de PLATAFORMA que transformasse o que a missão observou num nível por habilidade da Base, que o professor lê, corrige e leva para a avaliação dele.
> Quatro fatos da sala decidiram a forma dela:
>
> - **A atividade se refaz.** O professor reinicia a atividade para a turma tentar de novo, e a segunda tentativa não pode apagar o que a criança já tinha demonstrado na primeira.
> - **O computador é dividido** (**ADR-111**). O que a missão vê numa tela foi feito por dois ou três, com o login de UM deles.
> - **Às vezes a tela é uma só.** O professor projeta a missão e a turma inteira responde junto; a criança no fundo da sala não demonstrou o que a turma respondeu no telão.
> - **O *bundle* é público** (regra inviolável nº 3): o gabarito de uma pergunta não pode estar nele, nem no manifesto que o acompanha.
>
> E uma restrição de produto: em 2026-09-25 só o Instanooria tem ATIVIDADES (a atividade preparada pelo professor, clonada por turma, **ADR-49**); o unooGPT e o UnooTube serão refeitos com atividades numa próxima onda.

Decisão (épico E14, "Evidência pedagógica"; a política foi fixada pelo dono do produto em 2026-09-25).

- **A unidade é ATIVIDADE + RODADA, e a atividade é DA MISSÃO.** A plataforma não tem conceito próprio de atividade: `atividade_id` é um texto opaco que a missão dá (`ATIVIDADE_ID_RE`: 1 a 100 caracteres entre letras, dígitos e `_ . : -`, porque ele vai para o endereço da rota do painel), e é a missão quem conta os reinícios. Reiniciar abre a rodada seguinte; a anterior fica.
- **A régua é uma FICHA PEDAGÓGICA por missão, no código da api**: `evidencias/fichas/<expId>.ts`, no registro estático `FICHAS`. Ela diz quais habilidades da BNCC a missão mede, por quais MARCADORES (uma `acao`, que a missão observa e gradua, ou uma pergunta de `multipla_escolha`, graduada pelo gabarito), a pergunta e o nível que cada alternativa vale; e carrega a `versao`, que vai gravada em cada marca. `validarFicha` recusa código que não

existe no documento homologado (com a grafia exata), nível fora da escala, pergunta sem marcador e marcador sem pergunta. **Nesta frente o registro está VAZIO**: é só a plataforma, nenhuma missão foi adaptada, e nada aparece para o professor até a primeira missão entrar com a sua ficha.

- **A escala é `atingiu` > `em_processo` > `nao_atingiu`, e N/A é a AUSÊNCIA de linha** — "Não avaliado", que nunca conta como não atingiu. `nao_participou` é o N/A EXPLÍCITO que só o professor põe: barra a cópia da equipe e, no perfil, conta como N/A.
- **Dentro de uma rodada, dois marcadores da mesma habilidade: vale o MELHOR** (`statusProprio`) — demonstrar uma vez é demonstrar. E **o resultado da atividade é a MELHOR RODADA POR HABILIDADE** (`melhorRodadaPorHabilidade`), não a melhor rodada inteira: quem atingiu X na rodada 1 e Y na rodada 2 atingiu as duas.
- **Precedência: professor > a própria criança > a cópia.** A marca própria vence a cópia mesmo sendo pior — o que a criança fez com o próprio login é dela —, e nada vence o ajuste.
- **A cópia para a equipe leva só o NÍVEL, nunca texto.** Quando o resultado próprio sai, os colegas de computador (`mapaDeEquipes`, de `equipes/consulta.ts`) o recebem como origem `equipe` (`decidirCopia`). Colega com marca própria daquela habilidade na rodada, ou com ajuste do professor, não é sobrescrito; entre duas fontes da mesma equipe, fica a melhor.
- **A PROJEÇÃO chega "a revisar".** Montagem com `computadores = 1`, ou uma equipe com a turma inteira (`ehProjecao`), copia como `projecao`/`a_revisar`, e essa cópia **não conta** — nem no resultado, nem no perfil — até "Confirmar para a turma", que passa as da rodada a `valida`. Depois o professor ainda ajusta um a um.
- **A equipe mudou: recalcula SÓ a rodada em andamento**, a (atividade, rodada) da marca mais recente da turma × missão. O recálculo entra pelo gancho `EquipesService.aoMudarEquipes`, registrado no `onModuleInit`, e roda DENTRO da transação do `PUT` e do `DELETE` das equipes: um erro nele desfaz a montagem. Rodada passada fica com a equipe que teve — a cópia guarda `equipe_id` e `fonte_estudante_id` como carimbo, sem FK — e o que estiver errado nela se corrige no ajuste. A cópia que não mudou de nível nem de origem fica com a confirmação que tinha (`diferencaDasCopias`).
- **Múltipla escolha: o gabarito é do servidor.** `GET /evidencias/:expId/perguntas/:perguntaId` devolve só o enunciado e as alternativas; `POST .../resposta` leva só a alternativa, e a atividade e a rodada saem do resolvedor que a missão registra (`registrarContexto`) — nunca do cliente, senão a criança escolheria em que rodada a resposta conta. **Vale a PRIMEIRA resposta**; a segunda é aceita e ignorada, e as duas respondem o mesmo `{ enviada: true }`: a criança lê "resposta enviada" e não descobre, pela resposta, se acertou nem se já tinha respondido.
- **O professor que leciona ajusta a qualquer momento**, mesmo com a missão encerrada, um estudante ou vários (até 200 por `PATCH`), e cada mudança vai para `evidencias_eventos` com de/para, quem e quando. O histórico é append-only NO BANCO, no molde de `recursos_auditoria` (**ADR-89**): um *trigger* recusa `UPDATE` para qualquer *role*, e a de *runtime* só tem `SELECT, INSERT`.

- **Gestores leem; o estudante nunca vê o próprio resultado; não há ranking.** Os quatro papéis de gestão, `admin` incluído, leem o painel e o perfil e recebem 403 no confirmar e no ajustar — administrar a escola não é dar a aula. O que eles leem é NÍVEL por habilidade, nunca texto: a evidência não guarda nada que a criança escreveu. Não existe rota em que o estudante leia a própria evidência. A ordem dos estudantes é a alfabética, e nenhum campo soma níveis numa nota: o nível é de uma habilidade, não da criança.
- **O perfil do estudante é por missão e por atividade, sem juntar missões**: a mesma habilidade medida em duas missões aparece nas duas. Juntar é assunto de um painel futuro, com regra pedagógica própria.
- **Três tabelas de PLATAFORMA**, sem prefixo de experiência, com `escola_id` e RLS forçada, no módulo `apps/api/src/evidencias/`: `evidencias_marcas` (o dado bruto, que só a missão escreve), `evidencias` (o nível por habilidade, com origem e situação) e `evidencias_eventos` (o histórico). As missões entram por UMA porta, `EvidenciasService.marcar(tx, …)`, chamada da transação delas: a marca cai junto se a jogada cair. No hub, um painel genérico "Habilidades da BNCC" na página da atividade (hoje, só a do Instanooria), desenhado só quando a resposta diz `temFicha: true`, e uma página nova de perfil do estudante, aberta pela aba Estudantes da turma.

Alternativas descartadas.

- **Replicação automática para toda a equipe, projeção incluída** — o pedido do documento do dono do produto. Na equipe de dois ou três ela vale: quem estava ao lado fez junto. Na projeção, a "equipe" é a turma inteira, e a cópia automática gravaria trinta "atingiu" pelo acerto de uma criança no telão: a evidência afirmaria o que ninguém observou. A cópia da projeção nasce a revisar, e um clique do professor a confirma.
- **Recalcular tudo quando a equipe muda.** A aula de ontem mudaria de resultado porque hoje a sala tem menos computadores: a montagem de ontem já não existe (a **ADR-111** troca a montagem INTEIRA), e a cópia seria refeita com a composição de hoje. Recalcula-se só a rodada em andamento, e o passado se corrige no ajuste, com autor no histórico.
- **Três estados, sem N/A.** "Não avaliado" viraria "não atingiu", e a criança que faltou, ou que a missão não chegou a observar, entraria no boletim como quem tentou e não conseguiu — a ausência de dado lida como afirmação, que a plataforma já se recusa a fazer nos relatórios. São quatro: os três níveis e a ausência de linha, com o N/A explícito do professor ao lado.
- **A atividade como conceito de plataforma já agora** (uma tabela `atividades` genérica, com ciclo de vida). Só uma missão tem atividade hoje, e cada uma que vier terá a sua forma: um modelo comum desenhado a partir de um exemplo erraria nos outros. A plataforma recebe um id opaco e conta rodadas; quando o unooGPT e o UnooTube voltarem com atividades, o conceito comum, se houver, sai de três exemplos.
- **A ficha no manifesto ou no *bundle*.** O gabarito chegaria ao aparelho do estudante antes da resposta (regra inviolável nº 3), e o manifesto é escrito pelo CI, sem conferência pedagógica. No código da api, a ficha é versionada com o desenho da missão, revisada em PR e conferida por teste — os códigos na BNCC, o `expId` nos manifestos, o gabarito na escala.

- **"A tentativa nova substitui a anterior".** Reiniciar para treinar apagaria o que a criança já tinha demonstrado, e o professor que reinicia para a turma inteira puniria quem tinha ido bem. Rodadas com histórico, e o resultado é a melhor rodada por habilidade.
- **Juntar as habilidades de missões diferentes no perfil.** A mesma habilidade medida por duas réguas diferentes não tem soma evidente — vale a melhor? a mais recente? a da missão mais longa? — e escolher agora seria uma regra pedagógica escrita pela engenharia. O perfil mostra cada missão e cada atividade; juntar é de um painel futuro, com a regra definida pelo pedagógico.

Consequências ruins, declaradas.

- **Nada aparece para o professor até a primeira ficha.** A plataforma está pronta e invisível: o painel responde `temFicha: false`, e o hub não desenha o bloco. Quem abrir a tela sem ficha não vê nada, e pode achar que quebrou.
- **Missão nova com evidência passa por *deploy*, e por três trabalhos no mesmo PR**: a ficha, o resolvedor de contexto e a chamada a `marcar`. Sem o resolvedor, a pergunta responde 409 `sem-atividade-em-andamento` para todo mundo; sem subir a `versao` da ficha ao mudar um gabarito, as marcas antigas e as novas ficam indistinguíveis.
- **A cópia da equipe pode atribuir a quem estava distraído.** É o preço de a evidência valer para quem divide o computador; o professor corrige com o ajuste ou com "não participou", e a projeção, onde o erro seria em massa, espera confirmação.
- **A rodada passada guarda a equipe que teve — inclusive a errada.** Quem montou mal as equipes ontem corrige ontem à mão, estudante por estudante; trocar as equipes hoje não conserta.
- **A rodada em andamento é INFERIDA** da marca mais recente da turma × missão. Duas atividades da mesma missão correndo em paralelo na mesma turma fariam o recálculo das equipes tocar só a mais recente.
- **A evidência não é produto e não tem interruptor próprio** no painel da Unooria (regra inviolável nº 8): segue o da missão. A pergunta e a resposta passam por `exigirMissaoLiberada`, então missão desligada na rede ou fechada na turma responde 403; o painel e o perfil do professor continuam lendo o que foi registrado.
- **A gestão passa a ler o nível de UMA criança.** O perfil aceita `estudanteId` para os quatro papéis de gestão, ao contrário do agregado do unooGPT (que não tem rota onde um `estudanteId` caiba) e da linha da Vila (só do professor). É leitura de nível por habilidade — nunca texto, nunca o relatório da missão —, e foi decisão do dono do produto; revê-la é tirar `GESTORES` do `@Roles` das duas leituras.
- **O perfil repete a habilidade em cada missão que a mede**, o que pode parecer duplicado a quem lê. E o histórico cresce sem prazo: o `DELETE` fica fora do *trigger*, para a retenção e para a cascata da turma, mas o prazo de guarda do dado de criança ainda não está definido.
- **O vocabulário mora em dois lugares**: as constantes de `evidencias/regras.ts` e os *CHECK*s da *migration*. É o par que já custou um *deploy* em `permissoes`, e por isso o teste de banco compara os dois.

Como isto se prova. `apps/api/tests/evidencias-regras.spec.ts` cobre as regras puras, sem banco: a escala e o `melhor`, a melhor rodada por habilidade (a projeção a revisar fora, o `nao_participou` como N/A), o `decidirCopia` com cada precedência, a projeção pelos dois caminhos, o `copiasDesejadas` e o `diferencaDasCopias` (a confirmação que sobrevive, o de/-para de quem entrou e saiu) e o `validarFicha` recusando cada defeito. `apps/api/tests/evidencias-postgres.spec.ts` cobra o banco e o serviço: RLS habilitada E forçada nas três tabelas, isolamento de verdade entre escolas, a *role* sem *ownership* marcando e ajustando pelo serviço, os *CHECKs* iguais ao vocabulário e os de coerência, o `UPDATE` recusado no histórico até para a dona da tabela, a conta apagada levando marca, evidência e histórico; a cópia só do nível, o recálculo que toca só a rodada em andamento, a falha no recálculo desfazendo a montagem, a projeção fora do perfil até a confirmação (e a confirmação sobrevivendo a salvar as equipes de novo), o ajuste com histórico e com a missão encerrada, a pergunta sem gabarito, a primeira resposta valendo, o 409 sem contexto, o 403 da missão não liberada, o painel sem ficha em 200, o 404 para quem não leciona e o `RolesGuard` de cada rota. A suíte registra em tempo de execução uma ficha da exp08ef1b (`registrarFicha`, só para teste). `apps/api/tests/evidencias-fichas.spec.ts` confere cada ficha do registro — e, com ele vazio, prova que a trava reprova de verdade uma ficha com missão sem manifesto e código fora da Base.

Gatilho. A primeira missão com ficha mostrar que o nível não cabe em três graus (uma régua que precise de “superou”, ou de nota); o unooGPT e o UnooTube voltarem com atividades e o id opaco pedir um ciclo de vida comum — aí a atividade vira conceito de plataforma; o painel que junta missões ganhar a regra pedagógica; ou uma turma rodar duas atividades da mesma missão em paralelo, que a rodada em andamento inferida não distingue.

18.114 ADR-113 — A Seleção substitui o Eco no Grupo, e a limpeza vem antes

Data. 2026-09-25. **Status.** Aceita. **Consequência** da **ADR-110**: o código da missão é DERIVADO da série e da unidade, e o formato não tem lugar para duas missões na mesma unidade. Tira de circulação o que a **ADR-66** descrevia (a IA no *runtime* das conversas do Eco) e deixa a **ADR-67** como molde de mídia gerada pelo professor, sem missão que a use até a próxima.

> **Contexto**
>
> O documento de requisitos de **A Seleção** (7º ano, 1º bimestre, clone de rede social de fotos) chegou em 2026-09-19 apontando para o código `exp07ef1b` — o mesmo de **O Eco no Grupo**, publicado desde 2026-09-03 (7º ano, unidade 1, clone de mensageiro). As duas cobrem a mesma unidade do bimestre, e a de fotos é a que acompanha o paradidático dele.
>
> O código não é escolhido: sai de `codigoDaMissao(serie, unidade)`, e um teste reprova manifesto cujo `id` não bata com a série e a unidade. Duas missões na unidade 1 do 7º ano

> são, para a plataforma, a mesma pasta, o mesmo prefixo de tabela, o mesmo caminho no *bucket* e a mesma linha em `liberacoes`.

Decisão.

- **A Seleção substitui o Eco por completo**, e o Eco sai da trilha ANTES de a missão nova começar — numa frente própria, com PR próprio. A frente da missão nova não começa sobre um código ocupado.
- **Sai tudo o que é do Eco**: a experiência, o módulo da api e as rotas `/o-eco-no-grupo/*`, os quatro *models* e as tabelas `exp07ef1b_*`, as telas do professor (missão, resultados, painel da aula, zerar a turma, guia), a entrada no catálogo (o 7º ano, unidade 1, fica "em breve"), a suíte do estudante, o serviço do *compose* e as menções nos Termos de Uso e no painel da quota.
- **Saem também as linhas de PLATAFORMA que dizem `exp07ef1b`**, porque o código vai passar a significar outra missão: `experiencias` e `experiencias_versoes` (senão a sala ofereceria o Eco para liberar e o *rollback* ofereceria o *bundle* 0.1.5 no lugar da missão nova), `liberacoes` (a turma que liberou o Eco abriria a outra sem ninguém ter liberado), `progresso` (o SDK da missão nova leria o JSON salvo pelo Eco, com a conclusão dele) e `disponibilidades` (inclusive a `versao_fixa` que prende uma escola a um *bundle*). A migração tira o FORCE da RLS de `liberacoes` e `progresso`, apaga, confere que não sobrou nada e devolve o FORCE — o molde da **ADR-110**.
- **Ficam**: `telemetria_eventos`, porque o `exp_version` de cada linha já separa o Eco (0.1.x) da missão nova (1.0.0 em diante), e o `DELETE` varreria todas as partições; `recursos_auditoria`, porque é o histórico só de inserção do painel da Unooria (**ADR-89**); a ponte `@edu/ui/ds-whatsapp`, que é uma das cinco que o `packages/ui` promete; e a capacidade de voz do *provider* (`generateSpeech`), agora sem missão que a use. A porta 5179 fica reservada para a missão nova.
- **A missão nova começa acima da 0.1.5**: o caminho `gs://{contrato}-exp/exp07ef1b/{versao}/` é tratado como imutável, e ele guarda os *bundles* do Eco. A mídia gerada do Eco, em `experiencias/exp07e` no *bucket* de *assets*, é apagada fora do repositório; a missão nova grava a dela num subprefixo próprio.

Alternativas descartadas.

- **A Seleção em outra unidade do 7º ano** (`exp07ef2b` a `exp07ef4b`, livres). Manteria o Eco no ar, mas descolaria a missão nova do bimestre cujo paradidático ela reencena — o documento dela nasce amarrado à unidade 1.
- **Mudar o formato do código** para caber duas missões por unidade. É o gatilho que a própria **ADR-110** previu, mas o catálogo tem uma missão por unidade por desenho (48 missões, 12 anos × 4 unidades), e abrir o formato para um caso que o produto resolveu por substituição espalharia a mudança por pasta, tabela, *bucket*, catálogo do hub e `buscarMissao`.

- **Reaproveitar o Eco por dentro** (trocar o conteúdo e manter tabelas e rotas). A mecânica, o clone e os dados são outros; o que sobraria do Eco seria só o nome das tabelas, e com ele o risco de ler dado velho como se fosse da missão nova.
- **Remover o Eco na mesma frente da missão nova.** Um PR só misturaria apagar e construir, e o *diff* deixaria de mostrar o que saiu. O dono do produto pediu a limpeza primeiro.

Consequências ruins, declaradas.

- **O 7º ano fica sem missão entre as duas frentes.** A unidade 1 aparece "em breve" até a missão nova entrar.
- **O que o Eco já produziu some das telas**: as sessões, as conversas com a Amara e as reflexões são apagadas com as tabelas, e o progresso das crianças que o concluíram deixa de aparecer. A telemetria fica, mas nenhum painel a lê por missão retirada.
- **Parte da limpeza é fora do repositório**: a mídia do Eco no *bucket*, as métricas de log `exp07ef1b.*` e a sonda de disponibilidade no Terraform. Esquecer a mídia faz o *bucket* guardar arquivos que nenhum código lê; esquecer as métricas faz um painel olhar para uma missão que mudou de natureza.
- **Comentários nos módulos das missões que vão sair da trilha** (Foto Conta, Quem mandou isso?, Rastro do Boato, Manifesto da IA) ainda citam o Eco como precedente. Ficam: esses módulos saem em seguida, e corrigi-los agora é trabalho descartado.

Verificação. A migração foi aplicada sobre um Postgres descartável já semeado, com linhas do Eco em `progresso`, `liberacoes`, `disponibilidades`, `experiencias` e `experiencias_versoes`: todas sumiram, as de outras missões ficaram, as quatro tabelas saíram e o FORCE da RLS voltou. A suíte da api (111 suítes) e as dos demais pacotes passam sem o Eco.

Gatilho. Outra missão nova cair numa unidade já ocupada — aí a pergunta "substituir ou mudar o formato" volta, e a resposta desta ADR não vale sozinha para a próxima; ou o produto pedir o Eco de volta, que volta por outra unidade ou por outro código.

18.115 ADR-114 — Quem mandou isso? v1 é retirada; a v2.2 entra com o mesmo código

Data. 2026-09-25. **Status.** Aceita. **Consequência** da **ADR-110**: o código da missão é DERIVADO da série e da unidade, e a versão nova da missão do 5º ano, unidade 1, cai no mesmo `exp05ef1b` da primeira. Segue o caminho que a retirada do Eco no Grupo abriu na **ADR-113**, que corre em paralelo: limpar antes, numa frente própria. Deixa a **ADR-108** como precedente, sem missão que a aplique.

> **Contexto**
>
> A primeira versão de **Quem mandou isso?** (`exp05ef1b`, 0.1.1, 5º ano, unidade 1, clone de mensageiro) entrou em 2026-09-17 e **nunca saiu de `dev-local`**: nenhuma stack em nuvem aplicou a migração dela, e nenhum *bundle* dela foi publicado. O documento de requisitos da versão 2.2 chegou com uma mecânica quase inteira diferente:
>
> - as mensagens se classificam numa **matriz 2×2** — verdadeira ou falsa, emocional ou neutra —, e não em verdadeira, falsa e ambígua;
> - o estudante encaminha **exatamente três**, e não "pelo menos três";
> - entram uma **enquete de motivo** (por que encaminhou) e a **reflexão Se/Então**, no lugar do texto livre sobre o áudio do tio;
> - a missão passa a marcar **evidência pedagógica** pela régua da ADR-112 e a aceitar **equipes** (ADR-111) — a v1 não fazia nenhuma das duas.
>
> O dono do produto fixou a regra para esse caso: "se já houver missão da mesma série e bimestre, primeiro vamos limpar e excluir tudo da antiga".

Decisão.

- **A v1 sai da trilha por completo ANTES de a v2.2 começar**, numa frente própria, e a v2.2 é reconstruída do zero com o mesmo código.
- **Sai tudo o que é da v1**: a experiência, o módulo da api e as rotas `/quem-mandou-isso/*`, os cinco *models* e as tabelas `exp05ef1b_*`, as telas do professor (missão, resultados, painel da aula, preparo do lote, zerar a turma, bloco do guia), o guia da aula, a entrada no catálogo (o 5º ano, unidade 1, fica "em breve"), o prefixo do BFF, a porta do mapa de *dev*, a suíte do estudante, o serviço do *compose* e a liberação do *seed*.
- **Saem também as linhas de PLATAFORMA que dizem `exp05ef1b`** — `experiencias`, `experiencias_versoe` `liberacoes`, `progresso` e `disponibilidades` —, pela razão da ADR-113: o código vai significar outra coisa, e a turma que liberou a v1 abriria a v2.2 sem ninguém ter liberado. A migração é o molde da ADR-110: tira o FORCE da RLS de `liberacoes` e `progresso`, apaga, confere e devolve.
- **Ficam**: `telemetria_eventos` e `recursos_auditoria` (mesma razão da ADR-113); a ponte `@edu/ui/ds-whatsapp`, que outras missões usam; as chaves `veracidade`, `tom` e `explicacao` na lista de redação do *log*, agora sem consumidor — a v2.2 classifica as mensagens do mesmo jeito, e chave que sai da lista só afrouxa a rede; a porta 5184 no `CORS_ORIGINS` da api, reservada; e o teste do dado da habilidade EF05CO08, que prova a correção do extrator e não depende de missão nenhuma.
- **A v2.2 começa na 0.2.0**, e não na 0.1.x: o caminho `gs://{contrato}-exp/exp05ef1b/{versao}/` é tratado como imutável, e a telemetria separa as duas versões pelo `exp_version`.

Alternativas descartadas.

- **Evoluir a v1 no lugar.** As tabelas, os passos e o contrato mudam quase todos: a veracidade de três valores vira uma matriz, o piso de encaminhamentos vira número exato, a reflexão muda de forma, e entram enquete, evidência e equipes. O que sobraria da v1 seria o nome das tabelas, e com ele o risco de ler dado velho como se fosse da v2.2. E a evolução pagaria migração de dado sem dado nenhum para migrar: a v1 nunca saiu de `dev-local`.
- **Retirar a v1 na mesma frente da v2.2.** Um PR só misturaria apagar e construir, e o *diff* deixaria de mostrar o que saiu — a mesma razão da **ADR-113**.
- **Manter a v1 no ar até a v2.2 ficar pronta.** Duas missões com o mesmo código não coexistem na plataforma (pasta, tabelas, *bucket* e a linha em `liberacoes` são as mesmas), e a v1 continuaria acumulando dado local que seria apagado de qualquer jeito.

Consequências ruins, declaradas.

- **O 5º ano fica sem missão até a v2.2 entrar**: a unidade 1 aparece "em breve", como os anos que ainda não têm nenhuma.
- **O que a v1 produziu no banco local some**: sessões, mensagens encaminhadas e reflexões saem com as tabelas. Não há dado de estudante real — a v1 só rodou em `dev-local` —, mas a demonstração local do 5º ano deixa de existir.
- **A cópia principal precisa de passos depois do merge**: aplicar a migração no banco do *compose* e remover o contêiner e o volume de `node_modules` da experiência (`--remove-orphans`). Sem a migração, a grade da sala continua oferecendo a v1 para liberar.
- **O padrão da ADR-108** (a missão de sala que nasce vazia e a revelação pelo relógio da turma) fica sem missão que o aplique. A v2.2 decide se o retoma; até lá ele é precedente, não código.
- **A v2.2 precisa escrever a própria suíte de ponta a ponta**: a `estudante/10-quem-mandou-isso.mjs` saiu com a v1, e a cobertura do 5º ano na suíte do estudante fica vazia até lá.

Verificação. A migração foi aplicada sobre um Postgres descartável semeado, com linhas da v1 nas cinco tabelas dela e em `progresso`, `liberacoes`, `disponibilidades`, `experiencias` e `experiencias_versoes`, e com linhas de controle de outra missão nas mesmas tabelas de plataforma: as da v1 sumiram, as de controle ficaram, as cinco tabelas, a sequência e as políticas saíram, e o FORCE da RLS voltou. Num banco virgem, a cadeia inteira de migrações aplica, e o *diff* entre as migrações e o *schema* não acusa nada sobre a missão. A suíte da api (112 suítes) e as dos demais pacotes passam sem ela.

Gatilho. A v2.2 atrasar além do bimestre dela — aí a pergunta "deixar o 5º ano vazio ou trazer a v1 de volta" precisa de resposta explícita, e a v1 volta por esta ADR revertida, não por *cherry-pick*; ou outra missão nova cair numa unidade já ocupada, que repete a escolha desta ADR e da **ADR-113**.

18.116 ADR-115 — A Seleção: o par do Mural é do computador, e a IA gradua a legenda

Data. 2026-09-25. **Status.** Aceita. **Responde** ao gatilho da **ADR-111** (“uma missão com equipe cujo dado precise ser da EQUIPE”), é a segunda missão com ficha da **ADR-112** (a primeira foi Quem mandou isso? v2.2, **ADR-118**) e ocupa o código que a **ADR-113** liberou.

> **Contexto**
>
> A Seleção (7º ano, unidade 1) mede se o estudante percebe que publicar é recortar. O instrumento é social: a tríade dele ao lado da de um colega que escolheu outra coisa do MESMO feed. Quatro fatos da plataforma decidiram a forma: a sala divide computador (**ADR-111**), a evidência é por atividade e rodada (**ADR-112**), a revelação é do relógio da turma (**ADR-108**) e o *bundle* é público (regra inviolável nº 3). E um pedido do produto: o nível de EF07CO11 depende de a legenda explicitar um critério editorial — coisa que regra fixa não sabe ler.

Decisão.

- **O par do Mural é do COMPUTADOR.** O pareamento roda na passagem para o Mural, numa transação, sobre `colapsarEmUnidades`: distância máxima entre as matrizes de enquadramento, nunca dentro da mesma unidade. O par é carimbado pela *sessão* do representante, com o rótulo congelado (o primeiro nome ou o nome da equipe), e **nunca** pelo id da equipe — o `PUT` das equipes recria os ids. Quem sobra, quem publica depois do Mural e a aula de um computador só pareiam com um colega virtual do catálogo.
- **UM botão percorre quatro etapas** (`exploracao`, `mural`, `fichas`, `explicacao`), com o destino explícito na requisição. Não repete o “controle morto” que a **ADR-108** recusou: cada clique abre um momento de aula que existe — o Mural em silêncio, o debate, as fichas, o debate das lentes, a explicação. O destino explícito faz o clique duplo não pular etapa.
- **A IA gradua a legenda para EF07CO11** (o Canal A), exceção declarada ao princípio de que a IA não decide desfecho (a **ADR-66** o firmava). As travas: temperatura 0; uma tentativa e teto de cerca de 6 s; saída num *schema* com enum e validada; rubrica fixa no servidor e a legenda delimitada como dado; a quota do estudante verificada antes e debitada em `aoUsarTokens` (inclusive a chamada que perde a corrida); limite de três chamadas em dez minutos. **Qualquer falha não marca nada** — fica “Não avaliado” —, nenhuma justificativa é gravada, a criança não vê o nível, e o professor o confere e ajusta. A enquete `p1` conta junto, e vale o melhor (**ADR-112**).
- **Atividade = cobertura; rodada = zerar.** Zerar abre a rodada seguinte e guarda a anterior, ao contrário do `DELETE` das missões antigas; homologar um tema novo abre uma atividade nova.
- **As alternativas das enquetes saem embaralhadas a cada leitura**, pelo embaralhamento que a **ADR-118** pôs na plataforma para TODA ficha. Esta frente nasceu com um campo opcio-

nal na ficha para isso; quando as duas se encontraram, o campo saiu, para não haver dois caminhos para a mesma regra.

Alternativas descartadas.

- **Três botões** no painel. O mesmo resultado com mais coisa para esquecer; um botão que diz a próxima etapa ensina a ordem.
- **Parear por estudante.** Com computador dividido, só quem entrou tem sessão: juntaria linhas vazias, e dois logins na mesma máquina pareariam a equipe com ela mesma.
- **Carimbar o par pelo id da equipe.** Some no primeiro "Salvar equipes".
- **IA com piso "em processo"** para quem publicou. Esconderia a legenda desconexa, e a regra "vale o melhor" anularia o "não atingiu" da IA. O dono do produto escolheu os três níveis.
- **Ficha sem IA** (só as enquetes e o professor). Barata, mas deixaria o produto que a missão pede (a legenda) sem leitura nenhuma.
- **Zerar apagando.** Contradiz a rodada com histórico da **ADR-112**.

Consequências ruins, declaradas.

- **Em dev e na e2e a IA é o mock, que sempre falha**: o Canal A nunca marca, e a EF07CO11 só aparece pela enquete. A leitura da legenda só se prova com o Vertex.
- **A IA pode errar**, e o erro é um rótulo sobre uma criança — visível só ao professor, que o corrige; a quota do estudante é consumida por algo que ele não vê.
- **O colega vê a tríade, a legenda e o primeiro nome.** É o instrumento da missão, e os Termos de Uso e a Política de Privacidade dizem isso. O sanitizador barra link, telefone, e-mail, menção e palavrão, mas não moderação semântica.
- **O par não muda quando as equipes mudam, mas a evidência copiada da rodada em andamento, sim** — os dois comportamentos são os corretos, e diferem.

Verificação. `a-selecao-regras.spec.ts` e `a-selecao-postgres.spec.ts` (RLS nas quatro tabelas, latch concorrente, rodada guardada, par por computador que sobrevive ao `PUT` das equipes, recorte do gestor, Canal A com provedor falso nos casos válido, inválido, sem saldo e atrasado), e a suíte `scripts/e2e/estudante/11-a-selecao.mjs`.

Gatilho. O Vertex rotular com viés sistemático (a calibragem fica visível no ajuste do professor); uma missão nova que precise de dado da equipe com histórico de composição; ou o produto pedir que a criança veja o próprio nível.

18.117 ADR-116 — A experiência que perde o código é arquivada, não apagada

Data. 2026-09-25. **Status.** Aceita. **Consequência** da **ADR-110**: o código da missão é DERIVADO da série e da unidade, e o Jornal da Escola, a missão nova do 4º ano, unidade 1, cai no `exp04ef1b`

do UnooTube. As duas vezes anteriores apagaram a antiga (**ADR-113**, o Eco no Grupo; **ADR-114**, a v1 de Quem mandou isso?). Esta abre o outro caminho, e deixa as duas como a escolha certa para o caso delas.

> **Contexto**
>
> O UnooTube foi a primeira missão do 4º ano e a única que **esteve em nuvem**: está publicado em `dev` e `hml` como `exp-02-unootube` (1.2.7), e em `dev-local` como `exp04ef1b` (1.2.8). O Eco e a v1 do 5º ano nunca tinham dado a guardar; o UnooTube tem sessões, liberações, progresso, equipes montadas e o histórico do interruptor.
> O dono do produto decidiu, ao receber o Jornal: "mude tudo para prefixo-backup e crie uma área nova de arquivados na plataforma" — e, entre guardar o código numa pasta fora do monorepo ou no lugar, escolheu **no lugar, só renomeado**. A área de arquivados **só lista**: ninguém joga uma experiência arquivada.

Decisão.

- **Arquivar é dar o prefixo `backup-`**, e o arquivamento é DERIVADO do id, como o código é derivado do manifesto: não há campo "arquivada" no manifesto para ficar dessincronizado do nome da pasta. O id arquivado é `backup-` mais um código válido (`comum/arquivadas.ts`), e **não passa no `EXP_ID_RE`**: rota, catálogo, grade da sala e painel de recursos aceitam só experiência viva, e a arquivada some de todos sem um filtro a mais.
- **O código fica no lugar, renomeado, e continua compilando e testando**: a experiência vira `apps/experiencias/backup-exp04ef1b`, o módulo da api vira `apps/api/src/backup-unootube` (FORA do `app.module`: nenhuma rota é servida), a tabela vira `backup_exp04ef1b_sessoes` e as *specs* continuam rodando contra ela. No hub, os arquivos ganham o prefixo e saem só da fiação — os três despachantes, o registro dos guias e `MISSOES_PUBLICADAS` (o 4º ano, unidade 1, fica "em breve" até o Jornal entrar). A suíte de ponta a ponta vai para `scripts/e2e/arquivados`, que o *runner* não percorre.
- **A linha do catálogo SAI de `experiencias` e vai para `experiencias_arquivadas`**, com a **fotografia** do arquivamento: quantas turmas tinham liberado e quantos estudantes tinham progresso. A migração conta enquanto o FORCE da RLS está fora — o único momento em que contar atravessa escolas sem pedir nada à RLS — e a tela só lê.
- **O código como VALOR vira `backup-exp04ef1b`** em `experiencias_versoes`, `liberacoes`, `progresso`, nas três tabelas de equipes, nas três de evidência e, com o prefixo `experiencia:`, em `disponibilidades` e `recursos_auditoria`. Sem isso o Jornal nasceria com as liberações, o progresso, as equipes e o histórico do interruptor do UnooTube. As travas afrouxadas dentro da transação são as da **ADR-110**: o FORCE da RLS e o *trigger* append-only da auditoria. `telemetria_eventos` fica: o `exp_version` separa o UnooTube (1.x) do Jornal (2.x).
- **A área `/unooria/arquivados`** (só `admin`, **ADR-88**) lista o que está arquivado por `GET /recursos/arquivad` nome, código de origem, onde ficava na grade, última versão, a fotografia, o motivo e a data. O documento de portabilidade também lê `experiencias_arquivadas`, para a participação da criança continuar nomeada.

- **O *deploy* pula `backup-*`** nos dois laços (subir e ativar), o *compose* de desenvolvimento perde o serviço do UnooTube, e a porta 5174 fica para o Jornal. O *seed* escreve a arquivada a partir das pastas `backup-*`, porque num banco novo a migração não tem linha para mover.
- **`exp04ef1b` sai de `EXPERIENCIAS_COM_EQUIPE`**: a lista é por código, e o código mudou de dono. O Jornal volta a entrar quando nascer com equipe.

Alternativas descartadas.

- **Apagar, como a ADR-113 e a ADR-114.** Certo para quem não tinha o que guardar; o UnooTube esteve em nuvem, e o dono pediu para guardar.
- **Mover o código para uma pasta fora do monorepo**, sem compilar nem testar. Congelaria a missão como estava, e apodreceria sem aviso: no dia de trazer de volta, nada garantiria que ainda compila. O dono escolheu o custo contrário.
- **Uma coluna `arquivada_em` em `experiencias`.** Seis leitores listam aquela tabela (a grade, o catálogo, o painel de recursos, a evidência, a portabilidade, o próprio catálogo de rotas), e cada um teria de lembrar de filtrar; o que esquecesse mostraria o UnooTube de novo.
- **Recontar turmas e estudantes em tempo de uso.** Seria uma rota do `admin` contando `liberacoes` e `progresso` de todas as escolas, isto é, atravessando a RLS — uma capacidade que nenhuma rota tem hoje, e que não vale para uma tela que só lista.

Consequências ruins, declaradas.

- **Código arquivado custa manutenção**: continua no *typecheck* e nos testes, e quebra quando o DS, o SDK ou o *schema* mudarem. Quem quebrar conserta ali — a pasta tem um `AGENTS.md` que diz isso.
- **O 4º ano fica sem missão até o Jornal entrar**, como a ADR-114 deixou o 5º.
- **A fotografia não se atualiza**, e a tela diz isso. Os dados guardados não mudam, então ela só mente se alguém escrever nas tabelas arquivadas à mão.
- **O *bucket* continua com o UnooTube**: os *bundles* em `exp-02-unootube/*` e os vídeos em `exp04ef1b/videos`, sem uso. A cópia manual de `exp-02-unootube/*` para `exp04ef1b/` que o `docs/INFRA.md` descrevia **não deve mais rodar**: poria o UnooTube 1.x no caminho do Jornal.
- **Duas suítes mudaram de missão**: a `professor/10` (equipes) passou para o unooGPT, e a `estudante/01` usa o unooGPT como a missão "de outra série".
- **Os Termos de uso citam o UnooTube** como exemplo de ambiente imitado; a frase fica para o dono do produto revisar, com a do Eco.

Verificação. A migração foi aplicada sobre um Postgres descartável semeado com dados do UnooTube em duas escolas — sessão, liberação, progresso, montagem, equipe, membros, interruptor e auditoria — e com linhas de controle de outra missão nas mesmas tabelas. A primeira corrida **abortou na asserção**: `equipes_membros` guarda o código como cópia, e a FK dela aponta para `equipes(id)`, não para a montagem; o UPDATE próprio entrou. Depois disso: tudo

renomeado, a fotografia com 2 turmas e 3 estudantes, o controle intacto, o FORCE da RLS de volta e o *trigger* religado. Num banco virgem, a cadeia inteira aplica e a suíte da api passa (112 suítes, 1614 testes); o *seed* num banco virgem escreve a arquivada e não libera nada do 4º ano.

Gatilho. A segunda experiência a perder o código (é ela que prova que o molde é o molde); alguém precisar ABRIR uma arquivada, e não só listá-la (aí entra um modo de demonstração, que hoje não existe); ou o custo de manter o código arquivado compilando passar do valor de guardá-lo no lugar.

18.118 ADR-117 — Jornal da Escola: a redação é da Unooria, o professor abre o resultado e a revisão, e a IA só escolhe o layout

Data. 2026-09-27. **Status.** Aceita. **Ocupa** o código `exp04ef1b` que a **ADR-116** liberou ao arquivar o UnooTube; **aplica** a **ADR-111** (equipe) e a **ADR-112** (evidência por atividade e rodada) e **estende** a fronteira da **ADR-72** para uma tela que não é clone de nada.

> **Contexto**
>
> O Jornal da Escola (4º ano, unidade 1) põe a criança como editora: oito notícias de oito meios, três vagas na capa, uma publicação, o impacto nos leitores e uma segunda edição depois de aprender a checar fonte, data e autoria. Chegaram dois documentos que divergem: o PRD da esteira, que pôs a redação dentro do clone do Instagram por uma regra dos prompts dela (os "cinco ambientes", que não existe neste repositório), com a pergunta de motivo antes de publicar e o resultado sem portão; e a proposta revisada em PDF, que o dono do produto mandou seguir à risca: uma redação PRÓPRIA da Unooria, sem imitar rede social, em que a IA monta a página e o professor libera o resultado. A missão também mistura na mesma tela conteúdo que vem de plataformas diferentes — um post, uma mensagem encaminhada, um recorte de jornal —, o que nenhuma experiência tinha feito.

Decisão.

- **A redação é da Unooria, e o conteúdo de cada plataforma usa a seção dela.** Mesa, página, Bento, leitores e Chave usam a seção `unooria`; na gaveta, a matéria aparece no formato original: o post de rede social com o `PostCard` do `instanooria`, a mensagem encaminhada com o `MessageBubble` do `whatsappmobile`, e os meios que não imitam app nenhum (jornal, rádio, telejornal, blog, portal, mural) com o `NewsArticle` da própria `unooria`. É uma EXCEÇÃO nova ao "não misturar seções": a peça imitada fala na voz da plataforma que ela imita, mesmo dentro de uma tela da Unooria. As oito peças da redação nasceram no DS 0.2.8.
- **Dois portões, UM botão.** `exploracao` → `resultado` → `revisao`: o professor libera a leitura da 1ª edição e, depois da mediação oral, a revisão. A etapa só conduz quem publicou (a publicação atrasada é caminho previsto) e só até a Chave; dali em diante a criança anda sozinha.

- **A pergunta de motivo vem DEPOIS do latch**, e antes de a criança ver o resultado. Perguntar no modal de publicar anteciparia a descoberta (PDF, seção 4); perguntar depois do resultado contaminaria a resposta.
- **A IA diagrama a página em sala, e só escolhe um layout.** A saída é um valor de um enum de três (`classic`, `hero-image`, `columns`), validado, sem nenhum texto: a IA não escolhe notícia, não reordena as vagas e não pode reescrever uma pista (PDF, seção 5). As travas são as da **ADR-115**: temperatura 0, uma tentativa, teto de cerca de 6 s por `Promise.race`, a quota do *estudante* verificada antes e debitada em `aoUsarTokens`, e um cache por pacote e combinação de vagas, que limita o custo às combinações usadas. Qualquer falha cai no modelo automático, uma regra determinística pelo tamanho do texto e pela imagem da manchete, e a tela não fica sabendo.
- **As pistas descem pelo servidor.** O texto de fonte, data e autoria não vem na sessão: vem na requisição que abre a pista, que é o registro da EF15CO04. Uma pista lida sem registro seria evidência que a plataforma não vê.
- **A Chave do Editor tem iscas.** Seis blocos, três deles pilares e três iscas (“foto”, “urgência”, “compartilhada”); a isca volta para a bandeja, e o nível sai do número de tentativas. Sem isca, a EF15CO02 daria “atingiu” para toda criança que tocasse três blocos.
- **Atividade = pacote; rodada = zerar**, como na **ADR-115** e na **ADR-118**. Sem homologação, a turma roda o banco padrão (“Feira de Ciências e Notícias da Cidade”), criado na primeira entrada — a exceção à **ADR-108** que o PDF pede (seção 10: dá para aplicar sem configurar).
- **A ficha mede três habilidades**: EF15CO04 pelas pistas abertas antes de publicar, EF15CO02 pela Chave, e EF04CO08 pela 2ª edição e pelas perguntas P1 e P2, valendo o melhor (**ADR-112**). A síntese final é UMA só: a resposta às perguntas não diz à criança se ela acertou, e uma síntese por nível diria.
- **A versão nasce em 2.0.0.** O caminho `exp04ef1b/` do bucket já teve o UnooTube 1.x; uma versão 1.x do Jornal poderia cair num caminho tratado como imutável.

Alternativas descartadas.

- **A redação no clone do Instagram**, como o PRD pôs. Contradiz o documento que o dono do produto mandou seguir e faria a criança ler “jornal” como “rede social”.
- **Tudo na seção `unooria`**, redesenhando post e mensagem. Um post que não parece post não ensina a reconhecer post.
- **O resultado sem portão.** A mediação oral do PDF (seção 7) precisa da sala parada no mesmo lugar.
- **A IA escrevendo a página** (títulos e textos). Pode alterar a pista que a missão pede para a criança ler — é exatamente o que o PDF proíbe.
- **Nenhuma IA em sala** (só o modelo). Deixaria de cumprir o “a IA monta a edição” do PDF; o modelo fica como reserva.
- **A Chave aceitando qualquer toque** (sem isca). Não separa quem entendeu.
- **As pistas no *bundle* ou na sessão.** A tela as abriria sem registro.

Consequências ruins, declaradas.

- **Em dev e na e2e a IA é o mock**, que sempre cai no modelo: a diagramação por IA só se prova com o Vertex.
- **A quota do estudante paga uma decisão cosmética**, limitada pelo cache e pelo teto baixo de *tokens*.
- **Abrir uma pista precisa de rede.** Sem conexão a pista não abre, e a tela diz isso na voz da redação.
- **"Vale o melhor" afrouxa o "E" do PRD** na EF04CO08: a P2 sozinha pode dar "atingiu".
- **O critério do livro não foi conferido** (PDF, seção 8): fonte, data e autoria são a referência provisória. O dono do produto revisa depois.
- **O texto das oito notícias do banco padrão é de IA** e não passou por aprovação pedagógica (decisão do dono do produto); as imagens precisam de revisão humana antes da primeira rede.

Verificação. `jornal-escola-regras.spec.ts` e `jornal-escola-postgres.spec.ts` (RLS nas quatro tabelas, latch concorrente das duas edições, motivo uma vez, resultado bloqueado antes do portão, rodada guardada, colapso por computador, recorte do gestor, diagramação com provedor falso nos casos válido, inválido, sem saldo e atrasado), os testes da experiência (fluxo inteiro com api falsa, e o *bundle* sem gabarito) e os do hub (o espelho do contrato, lido da api como texto, e as regras puras do painel e do preparo). Na contagem da frente, a api tem 116 suítes e 1.903 testes, o hub 267 testes e a experiência 69. A suíte de ponta a ponta (`scripts/e2e/estudante/12-jornal-da-escola.mjs`) roda verde (2026-09-27, três execuções seguidas, com o segundo estudante pela api e a turma zerada no começo), e o Jornal desliga no painel da Unooria.

Gatilho. O livro do 4º ano trazer outro critério ou uma ordem obrigatória (muda a Chave, as pistas e a ficha); o Vertex diagramar mal com frequência (fica só o modelo); ou uma segunda missão misturar seções na mesma tela, e a exceção virar regra geral.

18.119 ADR-118 — Quem mandou isso? v2.2: atividade é o pacote, rodada é a reaplicação, e a régua mede pela seleção e pelas perguntas do PO

Data. 2026-09-25. **Status.** Aceita. **Consequência** da **ADR-114**: a v1 saiu, e a v2.2 volta com o mesmo `exp05ef1b`, a partir da 0.2.0. **Aplica** a **ADR-112** — é a PRIMEIRA missão com ficha pedagógica — e a **ADR-111** (nasce com equipes). **Retoma** o padrão da **ADR-108**, que ficara sem missão: a missão nasce vazia e a revelação é do relógio da turma, por *polling* (**ADR-93**). Segue a **ADR-69** (a porta da sala) e o critério de fronteira da **ADR-72** ("quem fala").

Contexto

O documento de requisitos da v2.2 chegou com as **perguntas de evidência** do PO (o "Canal B"). O briefing travou só três coisas: o contrato de evidência da BNCC, os textos literais das seis etapas e as quantidades — oito mensagens na matriz 2×2 (verdadeira ou falsa, emocional ou neutra), **exatamente** três encaminhamentos ao Mural, o "Então" com pelo menos 15 caracteres e quatro opções de motivo. O resto — rotas, tabelas, nomes, a ordem das telas — segue o repositório, e não o DDL nem as rotas do documento.
Três fatos do repositório pesaram mais que o documento:

- **A evidência (ADR-112) é por atividade e rodada**, e quem diz qual é a atividade é a missão. Nenhuma missão tinha ficha, e esta seria a primeira a responder o que é "atividade" e o que é "rodada" numa aula que não cria atividade nenhuma.
- **A escola divide computador** (ADR-111): a v1 fora deixada de fora das equipes porque ia sair; a v2.2 não tem esse motivo.
- **Não há arquivo de mídia**: o documento pede comunicados e prévias de link, e o pacote de cada turma é escrito pela IA no preparo — que escreve texto, não imagem.

Decisão. Os itens abaixo são os que tinham alternativa; o dono do produto fechou os quatro primeiros em 2026-09-25, e os três últimos são os *defaults* da especificação, aplicados sem contestação.

- **A atividade é o PACOTE aprovado; a rodada é a reaplicação.** O `pacoteId` é o `atividadeId` da evidência. "Zerar Turma para Nova Aplicação" soma 1 à rodada do mesmo pacote e **não apaga nada**: as sessões, as regras e as marcas da rodada anterior ficam, e o painel de habilidades mostra as rodadas e fica com a melhor de cada habilidade. Aprovar um pacote DIFERENTE abre atividade nova (o anterior fica inativo, com o histórico); aprovar o MESMO é idempotente — a comparação é por uma assinatura do conteúdo, porque o JSONB reordena chaves. Os dois voltam a turma à triagem.
- **A régua mede pela seleção e pelas perguntas, e o Se/Então não é corrigido.** A EF05CO08 sai das três mensagens do Mural — três verdadeiras, ou duas e uma falsa *emocional*, é `atingiu`; duas e uma falsa *neutra*, `em_processo`; uma verdadeira ou nenhuma, `nao_atingiu` —, marcada NA MESMA transação do *latch* do Mural. A EF05CO03 vem da P1, e a EF05CO04 **só** da P2. Da regra Se/Então o servidor recusa apenas o que não é resposta (curto, longo, texto sem sentido); o professor a lê e ajusta a EF05CO04 pelo painel. *Atualização de 2026-09-26*: esta régua foi substituída pela **ADR-120** — a faixa da EF05CO08, as duas perguntas e a rubrica do Se/Então. O resto desta decisão continua valendo.
- **As perguntas do PO ficam no chat da Nina, DEPOIS do Se/Então, com a mesma enquete da Etapa 3.** A ordem é casca (a regra) → clone (P1, P2 e a despedida da Nina) → casca ("Missão Concluída"): a fronteira do disfarce cruza duas vezes, e cada travessia segue o critério de quem fala. As alternativas descem **embaralhadas no servidor a cada leitura** e sem o nível, e o recibo é sempre `{ enviada: true }` — o embaralhamento é mudança de PLATAFORMA

(`evidencias/`), e vale para toda missão que tiver pergunta. Quem sai antes delas fica "Não avaliado", nunca "não atingiu".

- **Nasce com equipes.** A `exp05ef1b` entra na lista de missões com equipe do módulo `equipes/`, e o painel e o Termômetro contam por computador — o mapa das equipes lido dentro do `comTenancy`, e as linhas colapsadas em unidades, como no UnooTube e no unooGPT.
- **Mídia desenhada, sem arquivo.** O comunicado é o texto do pacote desenhado como papel (`MessageBubble.Media` sem `src`, com o texto no `alt`); o link é a prévia montada da legenda ("Nome do site: manchete"); o termômetro e a síntese entram no chat como imagem desenhada. Sem áudio assistivo na V1 (entra quando houver gravação), sem *benchmarking* e sem CSV.
- **Imagem e link com uma verdadeira e uma falsa cada.** Nos artefatos, os dois links do pacote curado ("O Mistério do Passeio ao Museu") eram falsos, e a turma aprenderia "link é mentira" — a heurística que a missão existe para desmontar. A régua do pacote passa a exigir o equilíbrio de formato, para o curado e para o gerado; no curado a `msg_5` (verdadeira) virou link e a `msg_7` (falsa) virou texto, sem mudar os textos. A versão original reprova, e há teste que prende isso.
- **Gestor só lê.** Pacote, painel e material respondem a professor e às coordenações; gerar, aprovar, os três botões e o zerar são só do professor. O "Então" é texto livre de menor: vem preenchido só para quem leciona na turma, e a coordenação recebe `null`.

Do padrão da **ADR-108**, a v2.2 retoma o **nascer vazia** (sem pacote aprovado a lista de conversas fica vazia, com uma linha de espera, e não há queda para o banco curado) e o **relógio da turma** — mas com TRÊS botões (revelação, síntese, reflexão), só para a frente, e sem empurrar a triagem: quem ainda não confirmou quando a revelação abre continua escolhendo e vê a própria revelação depois do voto. O passo do estudante é derivado dos carimbos e da fase (`passoEfetivo`), nunca coluna.

Alternativas descartadas.

- **A atividade ser a TURMA** (uma por turma, e cada reaplicação uma rodada). Um pacote novo — outro tema, outras oito mensagens — cairia na mesma atividade, e a "melhor rodada" compararia instrumentos diferentes como se fossem o mesmo.
- **Zerar apagando**, como a v1 e a exp08ef4b fazem. A política da **ADR-112** é justamente ver se houve melhora entre as tentativas; apagar a rodada anterior apagaria a pergunta.
- **Corrigir o Se/Então para a EF05CO04**, por regra ou por IA. É o único texto livre da missão, escrito por uma criança de 10 anos: correção por IA põe um modelo julgando texto de menor, e correção por regra de palavra-chave premia quem copia a fórmula. O PO desenhou uma pergunta para medir a habilidade — a P2 —, e o professor continua podendo ajustar.
- **As perguntas ANTES do Se/Então, ou na casca.** Antes, a P2 (qual regra funciona melhor) entregaria o modelo pronto à criança que vai escrever a própria; na casca, elas perderiam a voz da Nina que as apresenta e a enquete que a turma já conhece da Etapa 3.

- **Mandar as alternativas na ordem da ficha**, ou embaralhar no cliente. Na ordem da ficha a certa é sempre a primeira, e a posição vira a pista; embaralhar no cliente deixaria a ordem canônica na resposta da rede, legível no DevTools.
- **Nascer sem equipes** e entrar depois, como as missões que o E13 deixou de fora. Não havia a razão delas (sair da trilha), e o painel por computador já nasce no desenho do Termômetro.
- **Mídia com arquivo** — foto gerada ou imagem de banco. Exigiria geração de imagem por turma, *bucket* e revisão de cada imagem pelo professor, para o que é, no fundo, um papel com um texto que ele já aprovou.
- **O pacote curado como veio**, com os dois links falsos.

Consequências ruins, declaradas.

- **Só Computação é medida.** Os códigos de Língua Portuguesa do *one-pager* (EF15LP01, EF05LP15) ficaram fora da régua.
- **Textos escritos por esta frente esperam a revisão do PO**: a instrução de "Apresentar Síntese", a confirmação da regra, a despedida da Nina, a prévia do link da `msg_5`, os fatos do recorte do pacote curado e o vocabulário de componentes curriculares.
- **O equilíbrio de formato do pacote curado é *default***, à espera da resposta do dono do produto (a decisão em aberto está no mapa de produto).
- **Rodada é contagem que só cresce**: cada "Zerar" deixa uma rodada no histórico, inclusive a vazia de uma aula desistida no começo.
- **Duas compensações sobre o DS 0.2.4**: a seta esgotada em 3/3 continua respondendo (`aria-disabled` em vez de `disabled`, que engoliria o toque do aviso de limite) e o toque no comunicado desenhado é um `<button>` sem aparência (o `Media` não publica `onClick`). As duas são pedidos ao `storybook-unooria`; até lá, a suíte de ponta a ponta clica a seta esgotada com `force`. *Atualização de 2026-09-26*: os dois pedidos foram publicados na 0.2.6 (`exhausted` no `ForwardButton`, `onClick` e `actionLabel` no `.Media`) e as compensações saíram na exp05ef1b 0.2.1. O `force` da suíte fica: quem recusa `aria-disabled` é o Playwright.
- **O embaralhamento mudou a rota de plataforma**: toda missão futura com pergunta o herda, e a tela dela precisa responder pelo `id` e carregar a pergunta uma vez por aba — cada leitura troca a ordem.
- **A cópia principal precisa de passos depois do merge**: aplicar a migração `20260926130000` no banco do *compose* (nasceu `20260926120000` e foi renomeada no *merge*, porque a **ADR-116** chegou antes com o mesmo carimbo), subir o serviço da experiência na 5184 (`docker:dev`), reiniciar o hub (o prefixo `quem-mandou-isso` voltou à *allowlist* do BFF) e a api, e rodar a `estudante/10-quem-mandou-isso.mjs` e a `professor/10`. A suíte foi escrita nesta frente e só rodou depois do *merge*. *Atualização de 2026-09-26*: as duas rodaram verdes na cópia principal (21 passos e 69 verificações; 9 passos e 33 verificações), e o mapa de produto trocou para "pronto" as histórias cuja única pendência era o teste.

Verificação. No *worktree*, contra um Postgres descartável migrado do zero, `typecheck`, `lint` e `test` do repositório inteiro, pelo `turbo`, verdes nas 53 tarefas: a api com 114 suítes e 1.762 testes — a régua da EF05CO08 conferida nas 56 combinações de três entre oito, os *latches* sob toques simultâneos, a marca na transação do Mural, a RLS com a *role* sem `BYPASSRLS`, o "Então" só para quem leciona, a rodada nova e o pacote novo —, o hub com 205 testes (as regras da tela, puras) e a experiência com 89 (as regras de tela, a versão nos três lugares e a varredura do `src/` atrás do gabarito, repetida no `dist/`); e o *build* de produção do hub. A suíte de ponta a ponta tem `node --check` e *lint*, e não rodou.

Gatilho. A régua mudar — a EF05CO04 só pela P2, ou o peso da falsa neutra na EF05CO08 — e a ficha subir de versão (disparado em 2026-09-26: a **ADR-120**); o PO pedir que o Se/Então conte para a evidência; uma segunda missão precisar de "atividade" sem ter um pacote, que é o gatilho da **ADR-112** para a atividade virar conceito de plataforma; ou o DS publicar a seta esgotada e a mídia acionável, e as duas compensações saírem.

18.120 ADR-119 — O podcast da A Seleção é do professor, e a IA reescreve só as três falas que dependem do tema

Data. 2026-09-26. **Status.** Aceita. **Completa** a **ADR-115** (A Seleção) com o "Podcast de Recapitulação" que o documento de requisitos pedia e a primeira versão deixou de fora sem registrar. **Usa** a voz que a plataforma já tinha (`generateSpeech`, o TTS dos modelos Gemini pela Cloud Text-to-Speech, que o Eco usava antes da **ADR-113**) e o molde de mídia gerada pelo professor da **ADR-67**.

> **Contexto**
>
> O documento de requisitos pede "3m45s de áudio estéreo (Lucas e Clara) disponível no Hub pós-aula", com roteiro fechado no sheet de copy: cinco blocos, vinte e uma falas, cerca de 3.200 bytes. Três fatos decidiram a forma:
>
> - **O texto fala com o estudante** ("se você acabou de sair da missão…"), mas quem decide a aula é o professor. O dono do produto decidiu que o podcast é **do professor**, que o toca à turma se quiser.
> - **O Bloco 2 descreve os posts da Praça Central** — as maquetes da Folha Oficial, a padaria da Voz Coletiva, o orçamento por metro quadrado da Métrica Urbana. Com um tema gerado por IA (**ADR-115**), esse trecho contaria à turma uma cobertura que ela não leu.
> - **A síntese de voz tem teto por chamada** (4.000 bytes de texto, cerca de 655 s de áudio) e o BFF do hub corta a chamada em 30 s; o episódio inteiro numa chamada só não cabe no segundo.

Decisão.

- **O podcast mora na aula da turma**, em "A aula e os resultados", com o `AudioPlayer` do DS (0.2.7) e a transcrição sempre na tela. Não mora no guia: o guia é igual para todas as turmas, e o Bloco 2 é do tema de cada uma. Nenhuma rota de estudante.
- **Só três falas mudam com o tema**: as três do Bloco 2 que dizem o que cada veículo mostrou. A primeira e a última fala do Bloco 2 e os Blocos 1, 3, 4 e 5 inteiros servem a qualquer tema e ficam como o sheet aprovou. Num tema da IA, as três falas saem do **mesmo lote** do preparo, sob a mesma régua da aprovação (12 a 60 palavras, até 400 bytes, sem link, e-mail nem "@"), e a professora as revisa junto com os posts. Moram na cobertura (`exp07ef1b_coberturas.podcast`); nulo lê as do sheet.
- **Um trecho por chamada, e a emenda no fim.** Cada bloco é uma chamada de DIÁLOGO (Lucas e Clara numa chamada, com a mesma direção de atuação), e com os cinco prontos a mesma chamada emenda o MP3 — tirando de cada trecho as etiquetas e o quadro de duração, que diriam ao navegador a duração de um trecho só. Sem ffmpeg, numa função pura.
- **O arquivo de cada trecho é o hash do texto e do pedido de voz**: os blocos fixos saem uma vez por contrato, e um tema novo paga só o Bloco 2. O custo vai à quota do professor, estimado por trecho.

Alternativas descartadas.

- **Podcast para o estudante**, na tela final da missão — o que o texto sugere. Descartada pelo dono do produto: o professor decide quando a turma ouve.
- **Podcast só no tema curado**, sumindo com um tema da IA. Deixaria sem recapitulação justamente a turma que personalizou a aula.
- **Um Bloco 2 genérico**, sem citar os posts. Desviaria do texto aprovado e tiraria do podcast o que ele recapitula.
- **A IA reescrevendo o roteiro inteiro por tema.** Cinco blocos gerados por turma, a revisar antes de cada aula, para mudar três falas.
- **O episódio numa chamada só**, ou uma lista de cinco arquivos tocados em sequência no player. A primeira não cabe nos 30 s; a segunda faria a barra de posição e a duração mentirem entre um trecho e outro.

Consequências ruins, declaradas.

- **O áudio é mono e comprimido pela síntese**, não o MP3 estéreo de 128 kbps de estúdio que o brief previa.
- **Entre um bloco e outro pode haver uma pausa** que a gravação contínua não teria: cada trecho é uma interpretação à parte, amarrada só pela mesma direção e pelos mesmos timbres.
- **Em dev e na e2e o áudio não sai**: o mock falha fechado, e o caminho que roda é o da transcrição com o player desabilitado.
- **O texto fixo cita marcas** ("Instagram simulado", "parque da Disney"), porque o sheet aprovado as cita.

Gatilho. O PO pedir o podcast também ao estudante; a voz sintetizada não passar na revisão de conteúdo e for preciso gravar em estúdio (entra no lugar do episódio, com o mesmo roteiro); ou a síntese passar a devolver o episódio inteiro dentro de uma chamada, e a emenda deixar de ser necessária.

18.121 ADR-120 — A régua v2 de Quem mandou isso?: cada habilidade pelo verbo dela, e refazer o mesmo pacote não mede memória

Data. 2026-09-26. **Status.** Aceita. **Substitui** a régua da **ADR-118** (a faixa da EF05CO08 e as duas perguntas); o resto dela — atividade é o pacote, rodada é a reaplicação, as perguntas no chat da Nina — continua valendo. **Usa** a porta das missões e a rota de ajustes da **ADR-112**.

> **Contexto**
>
> A régua de 25/09 foi relida contra o texto das três habilidades na BNCC, e cinco coisas não se sustentavam:
>
> - **A EF05CO08 era generosa.** Duas verdadeiras e uma falsa *emocional* davam `atingiu`: a criança repassou uma mentira e "distinguiu o confiável". Das 56 combinações de três entre oito, 16 atingiam — 29% de acerto por sorteio.
> - **A EF05CO03 é lógica** ("negação, conjunção e disjunção sobre sentenças … e valores verdadeiro e falso"), e a P1 perguntava se o tom diz a verdade: parente, mas não exige a operação.
> - **A EF05CO04 tem dois verbos**, criar e simular, e a P2 pedia para *escolher* a melhor regra. O criar — a regra que a criança escreve — não contava para nada.
> - **Nas duas perguntas, a alternativa certa era a mais longa** e a mais cheia de nuance: a criança aprende a escolher a maior, e embaralhar a ordem não esconde o tamanho.
> - **A rodada nova do mesmo pacote contava.** Depois da revelação e da roda de conversa a criança já sabe quais eram falsas; com a melhor rodada valendo, reaplicar podia inflar o nível com memória.

Decisão.

- **EF05CO08: só três verdadeiras atingem.** Duas verdadeiras e uma falsa, de qualquer tipo, é `em_processo`; duas falsas ou mais, `nao_atingiu`. O tipo da falsa vira diagnóstico do professor — a folha mostra o quadrante de cada escolhida e o Termômetro conta a falsa neutra —, não nível. Por sorteio atingem 4 das 56 (7%).
- **A EF05CO08 é medida uma vez por criança e pacote.** Antes de marcar, a missão pergunta à plataforma se a criança já foi avaliada nessa habilidade numa rodada anterior da mesma atividade (`EvidenciasService.jaAvaliadaAntes`, na mesma transação do *latch*): vale o nível próprio, o da equipe, o da projeção e o do professor, e só `nao_participou` não conta. Já avaliada, o Mural fecha sem marca nova. Quem faltou é medido na primeira vez que joga. Para medir a turma de novo, aprova-se um pacote novo, e a tela do "Zerar" diz isso.

- **EF05CO03 pela P1 nova, que exige a operação.** Diante de uma mensagem "FALSA e EMOCIONAL", a criança escolhe a frase certa entre três com "não", "e" e "ou": a verdadeira atinge; a que acerta só uma das duas partes está em processo; a que erra as duas não atinge. O exemplo é da pergunta, não do pacote, e serve ao pacote pronto e ao gerado.
- **EF05CO04 pelos dois verbos.** A P2 nova mede o *simular*: seguir uma regra pronta no áudio do tio. O *criar* é a regra Se/Então que a criança escreveu, e quem a classifica é o professor, com um clique ao lado dela na folha da turma: condição e um gesto de conferir antes de repassar é `atingiu`; condição com ação vaga, `em_processo`; nada conferido, ou repasse, `nao_atingiu`. O clique é um ajuste da evidência (a rota de ajustes da **ADR-112**) e vale sobre a pergunta; sem clique, fica o nível da P2.
- **A certa nunca é a mais longa.** Nas duas perguntas as três alternativas têm o mesmo tamanho e a mesma forma, e há teste que cobra.
- **A ficha sobe para a versão 2.** Cada registro guarda a versão que valia quando foi feito, e os da versão 1 ficam como estão.

Alternativas descartadas.

- **Manter a régua de 25/09.** Mantém os cinco problemas.
- **Contar a P1 antiga também para a EF05CO08**, para ter duas observações. Na E14 vale o melhor dos momentos da rodada: uma pergunta acertada no chute (33%) apagaria a escolha real do Mural.
- **Corrigir o Se/Então com IA.** É texto livre de menor, e a missão só usa IA no preparo do professor.
- **Só a rodada 1 do pacote mede.** Mais simples, mas a criança que faltou na primeira aula nunca seria medida naquele pacote.
- **Uma terceira pergunta para a EF05CO03, mantendo a P1 antiga.** Alonga uma missão de 50 minutos; trocar a P1 mantém duas perguntas.

Consequências ruins, declaradas.

- **Os textos da P1 e da P2 são da equipe** e substituem os do documento do PO (Canal B): entram na revisão do PO, junto com as falas da Nina que as antecedem.
- **Uma medida por habilidade continua fraca**: chutar uma pergunta de três alternativas acerta em 33%. O contrapeso da rubrica existe só na EF05CO04, e só quando o professor clica.
- **A rodada nova mostra "Não avaliado" na EF05CO08** para quem já jogou o pacote. É a regra, e a tela do "Zerar" avisa, mas no painel pode parecer falta.
- **Réguas diferentes no mesmo histórico**: cada registro guarda a versão, mas o painel não separa as rodadas medidas pela v1 das medidas pela v2.

Verificação. Na api, a régua da EF05CO08 nas 56 combinações (4, 24 e 28), a ficha v2 (marcadores, níveis, a certa que não é a mais longa, a P1 com não, e e ou) e, no banco, a Ana e o Caio (este pela equipe) jogando a rodada 2 sem marca nova, e a Gabi — que entrou na turma depois

— medida na 2. No hub, a rubrica acende só o clique do professor, e só da rodada da aula. De ponta a ponta, contra a api e o hub desta frente (Postgres descartável, portas alternativas): na primeira corrida, a EF05CO08 pela régua v2 e a regra classificada na EF05CO04 como ajuste docente (22 passos, 70 verificações); na seguinte, com a turma zerada no mesmo pacote, a EF05CO08 não medida de novo (71 verificações); e a das equipes verde.

Gatilho. O PO reescrever a P1 ou a P2; uma segunda missão medir uma ação que a revelação entrega, e a regra da rodada virar regra da E14; os professores não usarem a rubrica (medir quantas regras recebem clique); ou o painel precisar separar as rodadas por versão da ficha.

18.122 ADR-121 — O CI verifica só os PRs para dev, hml e main

Data. 2026-09-26. **Status.** Aceita. **Muda** o gatilho do `ci.yml` (Listagem 15.2, no Capítulo 15); **não muda** o portão do *deploy*, que continua chamando o `ci.yml` por `workflow_call` antes de publicar em `dev` e `hml`.

> **Contexto**
>
> - **O `ci.yml` rodava em todo PR**, inclusive nos de *branch* de trabalho para `dev-local` (**ADR-101**), que são quase todos.
> - **Em setembro de 2026 a conta passou dos 2.000 minutos do plano Free do GitHub Actions**, que todos os repositórios privados da conta dividem, e o GitHub parou de iniciar os *jobs*. O *deploy* de `dev` e `hml` parou junto, porque também roda no Actions. Neste repositório, cerca de 550 dos 1.580 minutos do mês foram do CI de PR.
> - **Num PR para `dev-local`, o CI repetia o que a *Definition of Done* já manda rodar** no *worktree* (`pnpm turbo typecheck lint test`), e a promoção `dev-local` → `dev` já pede o mesmo na raiz.

Decisão.

- **O gatilho `pull_request` passa a ter `branches: [dev, hml, main]`.** PR para `dev-local`, ou para outra *branch* de trabalho (frente empilhada), não dispara o CI.
- **No PR para `dev-local`, o portão é a verificação local da *Definition of Done*.** O GitHub não mostra *check* nenhum ali, e isso é o normal.
- **A promoção `dev-local` → `dev` verifica tudo o que acumulou**, no CI e na raiz, como o *template* de promoção já pedia.

Alternativas descartadas.

- **Tirar o CI de todos os PRs**, deixando só o portão do *deploy*. O vermelho só apareceria depois do *merge* em `dev` ou `hml`, com o *deploy* já barrado e a *branch* de ambiente quebrada para todo mundo.

- **Manter o CI em todo PR e cancelar a execução anterior** a cada *push* no mesmo PR (`concurrency` com `cancel-in-progress`). Economiza, mas continua gastando a cota com a verificação que a *Definition of Done* já faz.
- **Pular os PRs só de documentação** (`paths-ignore`). Corta pouco dos PRs para `dev-local`, que são os que mais gastam.

Consequências ruins, declaradas.

- **Um defeito que só aparece em ambiente limpo** (arquivo não commitado, Prisma Client gerado só na máquina, cache do `turbo`) passa pelo PR para `dev-local` e só aparece na promoção, misturado com o que as outras frentes trouxeram.
- **Frente empilhada também fica sem CI**: a base dela é outra *branch* de trabalho, e a regra é a mesma.

Gatilho. Um defeito de ambiente passar pela verificação local e só aparecer na promoção; ou a conta passar a ter minutos que não acabam (plano pago ou repositório público).

18.123 ADR-122 — A modelagem UML entra no Capítulo 5, com os catorze tipos da UML 2.5.1 e par Mermaid mesmo onde o Mermaid não tem o tipo

Data. 2026-09-27. **Status.** Aceita. **Muda** o Capítulo 5, que ganha a Seção 5.9, a biblioteca `preambulo/diagramas.tex`, que ganha os elementos UML que faltavam, e o Apêndice A. **Não muda** o C4, que continua sendo a notação da arquitetura.

> **Contexto**
>
> - **O documento descrevia a arquitetura em C4, ER e notação de infraestrutura**, e a UML aparecia em cinco tipos — casos de uso, sequência, estados, atividades e um diagrama de classes no nível 4 do C4. Não havia notação para interface fornecida e requerida, artefato implantado, instância, prazo nem perfil, e cliente e edital pedem os diagramas UML de uso corrente na indústria.
> - **Número de capítulo é citado fora do documento**: "cap. 17" aparece dezenas de vezes em comentário de código da observabilidade. Um capítulo novo depois do 5 renumeraria do 6 ao 21, e a citação passaria a apontar para outro capítulo sem quebrar nada — a mesma armadilha que impede renumerar ADR.
> - **Todo diagrama TikZ exige par Mermaid** (Apêndice A), e o Mermaid não tem nove dos catorze tipos da UML: objetos, pacotes, componentes com interfaces, estrutura composta, implantação, perfil, comunicação, tempo e visão geral de interação.

Decisão.

- **Uma seção nova no Capítulo 5, em arquivo próprio** (`secoes/05b-modelagem-uml.tex`, sem `\chapter`), com ao menos um exemplar de cada um dos catorze tipos, distribuídos pelas visões 4+1 que o capítulo já usava. Onde o documento já tinha o tipo, o catálogo aponta para a figura existente em vez de repeti-la.
- **Cada exemplar vai onde o tipo mostra o que nenhum outro mostra**: o tempo mostra a janela de 401 da renovação do *token*; os componentes mostram que o JWKS é fornecido pelo hub e requerido pela API; os objetos mostram as três regras da disponibilidade combinadas.
- **Os nomes nos diagramas são os do código**, e cada figura cita os arquivos de onde saiu. A seção foi conferida contra o código na data desta ADR, e o que a conferência achou de divergente no próprio documento (a SDK importada pela API no grafo do *monorepo*, a quota "de todos" na camada de IA, os quatro papéis do Capítulo 4) foi corrigido no mesmo *commit*.
- **Tudo em TikZ, com par Mermaid para todo diagrama.** Onde o Mermaid não tem o tipo, o par é uma aproximação (`flowchart`, `gantt`), e a primeira linha dele diz o que se perde.
- **Um perfil UML da plataforma** (Figura 5.10) dá nome às regras invioláveis como estereótipos — «TabelaDeTenant», «RotaDeEstudante», «ChamadaDeIA» —, e os outros diagramas os usam.

Alternativas descartadas.

- **Um capítulo próprio logo depois do 5.** É onde a modelagem se lê melhor, mas renumera os capítulos 6 a 21, e a citação "cap. 17" do código passaria a apontar para outro capítulo, em silêncio.
- **Um capítulo no fim, ou um apêndice.** Não renumera nada, mas põe a arquitetura depois da conclusão, como material de consulta, e separa a visão lógica do C4 que ela complementa.
- **PlantUML como fonte dos diagramas.** Tem mais tipos UML que o Mermaid, mas exige Java e Graphviz fora do Tectonic, e o GitHub não o renderiza no *pull request* — que é o lugar para o qual o par Mermaid existe.
- **Desenhar só os tipos que o Mermaid tem.** O par ficaria fiel, mas a seção perderia justamente os tipos que dizem algo novo: implantação com artefatos, componentes com interface e tempo.
- **Trocar o C4 pela UML.** A UML não tem notação melhor para contexto e contêiner, e o C4 é o que o time e as ADRs anteriores já leem.

Consequências ruins, declaradas.

- **O Capítulo 5 fica duas vezes maior**, e quem procura a modelagem pelo sumário a encontra dentro da arquitetura de *software*, não num capítulo com o próprio nome.
- **Nove pares Mermaid são aproximações.** Quem editar o diagrama pelo Mermaid perde notação: o soquete vira nó, a linha de tempo vira `gantt`, a porta vira nó.
- **Mais figuras para manter.** Mudar a ponte, a quota, a disponibilidade, a sessão ou a implantação passa a tocar também uma figura UML, e diagrama que diverge do código engana com a autoridade de quem o desenhou.

Gatilho. O Mermaid passar a ter os tipos que faltam, ou o GitHub renderizar PlantUML; uma figura da Seção 5.9 ser encontrada divergente do código numa revisão — sinal de que o custo de manter passou do valor — ou uma segunda pessoa passar a manter os diagramas e preferir outra fonte.

18.124 ADR-123 — O plano anual tem uma estrutura só, definida pelo *prompt*, e o template é exemplo de formato

Data. 2026-09-30. **Status.** Aceita. **Muda** o *prompt* de sistema do gerador de plano anual (`planos/prompt-plano-de-aula.ts`), o template de exemplo (`planos/template-plano.ts`), as duas mensagens que o serviço monta para o modelo e o texto do *provider mock*. **Não muda** a geração assíncrona, o armazenamento, a quota nem a app-92.

> **Contexto**
>
> - **O modelo recebia duas definições da estrutura.** O *prompt* pedia nove blocos (fundamentação, objetivos, cinco seções por bimestre, metodologia, avaliação, duas tabelas e referências); o template mostrava seis; e a mensagem ao modelo mandava "seguir rigorosamente a estrutura do template". A forma do plano dependia de a qual delas o modelo obedecia em cada geração.
> - **O template ensinava o que o *prompt* proíbe.** O *prompt* diz para nunca inventar código de habilidade, e o exemplo trazia a EF04HI03 com o texto da EF04HI04 e uma EF04HI06 cujo texto não consta na BNCC do repositório. Avaliava também por participação e por produção de cartaz, quando o *prompt* afirma que isso não basta.
> - **O arco do ano só aparecia no fim.** Duas tabelas depois dos bimestres (produtos e progressão) repetiam o que cada bimestre já dizia, e o professor só via a trajetória do ano depois de ler todo o detalhe.

Decisão.

- **A estrutura é definida num lugar só: a seção "Estrutura obrigatória do *output*" do *prompt*.** O template passa a ser um exemplo de formato, tom e nível de detalhe, de outra disciplina, e as duas mensagens ao modelo dizem isso: a estrutura é a das instruções e o conteúdo do exemplo não se reaproveita.
- **Uma seção "Percurso do ano" no início**, em uma tabela (período, questão norteadora, aprendizagem central, produto de síntese) e uma frase de progressão, no lugar das duas tabelas do fim. O professor vê o ano inteiro antes do detalhe, e o modelo planeja a progressão antes de escrevê-la.
- **Cada bimestre traz questão norteadora, conteúdos, habilidades, estratégias, formas de avaliação e, quando houver, produto de síntese.**

- **A avaliação é por evidência**: cada item diz o que o estudante mostra que aprendeu e se liga a uma habilidade do bimestre. Os instrumentos formais que o currículo da rede exigir (prova bimestral, recuperação) continuam, dizendo o que verificam.
- **Habilidade entra com o código e o texto exatos do documento de origem**, sem paráfrase.
- **Os títulos e os limites de itens vivem em `planos/estrutura-do-plano.ts`**; o *prompt* interpola os limites, e `tests/planos-estrutura.spec.ts` confere *prompt*, exemplo e *mock* contra ele, e cada habilidade do exemplo contra a BNCC do repositório. Verificado que o teste reprova o template antigo e o texto trocado da EF04HI03.

Alternativas descartadas.

- **O template manda, e o *prompt* só descreve o conteúdo de cada seção.** Mantém o formato que a rede já conhecia, mas perde a questão norteadora, o produto de síntese e o percurso, que são o que dá progressão ao plano, e faz de um exemplo de uma disciplina a forma de todas.
- **Manter as duas tabelas no fim e acrescentar as seções ao template.** Conserva o *prompt* como estava, mas diz a mesma coisa três vezes (bimestre, produtos, progressão) e mantém o arco do ano escondido no fim.
- **Montar o documento por seção, com uma chamada de IA para cada uma.** Garantiria a forma, mas custa uma chamada por seção e tira do modelo a coerência entre os bimestres, que é justamente a progressão.
- **Passar ao *prompt* as habilidades da BNCC do repositório em vez de confiar nos documentos da rede.** É o que a app-92 faz, mas o Gerador de Plano de Aula existe para a rede que já tem currículo e material próprios.

Consequências ruins, declaradas.

- **O efeito no plano gerado não foi medido com uma geração real do Vertex.** O teste garante o que está escrito no *prompt*, no exemplo e no *mock*; não garante como o modelo os segue.
- **Planos novos saem com outra forma.** Os já salvos não mudam (`gerado.md` é imutável), e quem conhecia as tabelas do fim passa a encontrá-las unidas no início.
- **O exemplo ficou cerca de cinco mil caracteres maior**, em cada geração, diante de um *prompt* de cerca de 190.000 *tokens*.
- **A redação pedagógica do exemplo é uma escolha de quem desenvolveu**, e ainda não passou por quem ensina.

Gatilho. Uma geração real em que o modelo ignore a estrutura ou copie o conteúdo do exemplo; uma rede pedir outro formato de plano; a revisão pedagógica do exemplo apontar uma estrutura melhor; ou o Gerador de Plano de Aula passar a receber as habilidades como dado, como a app-92, caso em que a origem das habilidades muda.

18.125 ADR-124 — O `CLAUDE.md` admite o bloco que uma ferramenta escreve, e só ele

Data. 2026-09-30. **Status.** Aceita. **Emenda** à **ADR-55** (`AGENTS.md` canônico, `CLAUDE.md` ponteiro de uma linha) e à **ADR-102**.

> **Contexto**
>
> - **A regra era absoluta.** A **ADR-55** fixou que o `CLAUDE.md` é `@AGENTS.md` e nada mais, porque um `CLAUDE.md` com texto próprio é uma segunda fonte de verdade sobre a mesma pasta, e duas fontes divergem.
> - **A ferramenta não a conhecia.** O `graphify claude install` escreveu no `CLAUDE.md` da raiz um bloco `## graphify` de dez linhas, que manda consultar o grafo do código antes de responder e atualizá-lo depois de mexer. A regra continuou dizendo "uma linha", e o repositório passou a descumpri-la sem que ninguém tivesse decidido isso.
> - **O bloco precisa ficar.** A decisão do dono do produto é usar o graphify, e a ferramenta reescreve o bloco sempre que roda: apagá-lo só recria uma alteração não commitada, o mesmo que o bloco do Next em `apps/hub/AGENTS.md` já ensinou.

Decisão.

- **O `CLAUDE.md` é `@AGENTS.md` e, no máximo, o bloco que uma ferramenta escreve e reescreve** — hoje, o do graphify. Conteúdo do projeto continua só no `AGENTS.md`.
- **É a única exceção**, e vale para bloco de ferramenta, não para texto de pessoa. A regra está escrita onde ela é lida: `docs/regras/arquivos-de-instrucao.md`, `docs/regras/o-que-nao-fazer.md` e o `AGENTS.md` da raiz.
- **Um teste confere a forma**: toda pasta com `AGENTS.md` tem um `CLAUDE.md` que é só o ponteiro, e o da raiz é o ponteiro mais, no máximo, o bloco do graphify (`apps/api/tests/regras-de-agentes.`

Alternativas descartadas.

- **Mover o bloco para o `AGENTS.md`.** Serviria também a Codex e Cursor, mas o `graphify install` o recria no `CLAUDE.md`, e é o Claude Code que o precisa ali.
- **Deixar de usar o graphify.** Resolveria o conflito apagando a ferramenta, o contrário do que o dono do produto decidiu.
- **Manter a regra e tolerar o bloco sem registrar.** Uma regra que o próprio repositório descumpre deixa de ser regra, e o próximo agente a "corrige" apagando o bloco.

Consequências ruins, declaradas.

- **O `CLAUDE.md` da raiz deixa de ser uma linha**, e isso abre precedente: a próxima ferramenta que escrever no arquivo vai pedir a mesma exceção.

- **O bloco descreve um grafo que não existe num clone limpo**: `graphify-out/` é ignorado pelo *git*, e só a primeira regra do bloco é condicional a ele. Cada máquina, e cada *worktree*, constrói o seu.
- **O bloco está em inglês**, num repositório em pt-BR. Não é interface, e a ferramenta o reescreveria.

Gatilho. A ferramenta passar a escrever mais de um bloco, ou fora do `CLAUDE.md`; uma segunda ferramenta pedir o mesmo; ou o graphify deixar de ser usado — e o bloco sair junto.

18.126 ADR-125 — O grafo do código nasce copiado em cada *worktree* e se atualiza sozinho no *pull*, e fica fora do *git*

Data. 2026-09-30. **Status.** Aceita. **Completa** a **ADR-124**, cuja segunda consequência ruim era justamente um grafo que "cada máquina, e cada *worktree*, constrói o seu".

> **Contexto**
>
> - **Toda tarefa começa num *worktree*** (**ADR-101**), e o `graphify-out/` é ignorado pelo *git*: o *worktree* nascia sem grafo, e o bloco do `CLAUDE.md` mandava consultá-lo mesmo assim.
> - **A cópia principal também envelhecia.** O grafo só mudava quando alguém lembrava de rodar `graphify update .`, e um símbolo entregue por um PR mergeado não aparecia nas consultas seguintes.
> - **O ganho é de leitura, não de pergunta.** Medido neste repositório, em *tokens*: `explain` de um nome conhecido custa de 275 a 875; `path`, cerca de 30; `query` com pergunta em linguagem natural, cerca de 1 600 **fixos**; um `grep` preciso, de 110 a 1 500; **ler os arquivos que o `grep` encontra, de 9 500 a 139 000**. O grafo paga quando troca a leitura inteira por um subgrafo, e **não** paga quando já se sabe o nome e basta um `grep`.

Decisão.

- **`pnpm worktree criar` copia o `graph.json`** da cópia principal para o *worktree* novo, avisa a idade do grafo e recomenda `graphify update .` quando ele tem dois dias ou mais. Sem grafo na principal, o passo só avisa.
- **O `.githooks/post-merge` atualiza o grafo em segundo plano** quando o `git pull` de `dev-local` traz o merge do PR. É o `graphify update .`, só AST e sem custo de IA, com trava de quinze minutos em `graphify-out/.atualizando` e log em `graphify-out/atualizacao.log`. Nunca derruba o *pull*, não cria grafo onde nunca existiu e tem escape (`PULAR_GRAFO=1`).
- **O grafo *não* entra no *git*.** Ele é um artefato de cada máquina, regenerável, e a instalação dos *hooks* do próprio graphify continua desligada.

- **O uso está escrito onde é lido**: o `AGENTS.md` da raiz manda `explain`/`path` para "quem chama quem", e os custos medidos ficam em `docs/regras/comandos.md`. O `query` fica para a área que não se sabe nomear.

Alternativas descartadas.

- **Versionar o `graph.json`.** Resolveria o *worktree* cego, mas o arquivo tem cerca de 18 MB e muda a cada `update`: todo PR o alteraria, dois PRs em paralelo conflitariam nele (como o `main.pdf`, mas sem ninguém que o regenere), e o histórico engordaria 18 MB por alteração, contra a regra de não commitar asset acima de 2 MB.
- **Instalar os *hooks* do graphify.** Ele escreve em `.githooks/` quando o `core.hooksPath` está definido, e passaria a rodar a cada commit e troca de *branch*, em todas as máquinas, sem que o fluxo de git do repositório o tivesse decidido.
- **Cada *worktree* reconstruir o seu.** Gasta a reconstrução inteira toda vez que uma frente abre, e o grafo novo nasce da mesma base que a cópia já tem.
- **Gerar no CI e baixar.** O CI não roda nos PRs para `dev-local` (**ADR-121**), e um artefato de rede para uma consulta local troca um problema simples por um de credencial.

Consequências ruins, declaradas.

- **A cópia é um retrato da principal, não do *worktree*.** Símbolo criado na frente só aparece depois de `graphify update .` nela, e o que outras frentes abertas escrevem não está lá.
- **O *post-merge* só dispara no `pull` da cópia principal.** Quem nunca puxa por ali, ou cria o *worktree* enquanto a atualização ainda roda, copia um grafo anterior.
- **O grafo cobre código, não instrução nem documentação.** `AGENTS.md` e LaTeX estão fora; a camada de documentos exige `/graphify --update`, que custa IA, e não foi adotada.
- **O bloco do graphify no `CLAUDE.md` aconselha `query` primeiro**, e a medição diz que `explain` e `path` saem mais baratos. A ferramenta reescreve o bloco, então a correção vive no `AGENTS.md`, que vence em conflito.
- **Cada *worktree* carrega 18 MB a mais** em disco.

Gatilho. A medição passar a mostrar que `explain`/`path` não poupam leitura neste código; o grafo crescer a ponto de a cópia ou a atualização atrapalharem o `git pull`; o graphify deixar de atualizar sem IA; ou mais de uma pessoa passar a depender do grafo, caso em que vale reavaliar um artefato compartilhado.

18.127 ADR-126 — O agente de IA do professor roda no Vertex AI Agent Engine, atrás da api

Data. 2026-09-30. **Status.** Aceita. **Revisa** a **ADR-07** (IA atrás de um *gateway* na própria API) **para este produto**, e abre a única exceção a duas proibições da raiz: "não chamar provedo-

res de IA fora de `apps/api/src/ai`" e, no espírito, "não criar serviço novo sem necessidade demonstrada". Recurso de IA novo **não** herda a exceção.

> **Contexto**
>
> - **O produto.** Um chat flutuante no layout de `/professor`: o professor pergunta sobre a BNCC e sobre o acervo de livros didáticos da Unooria, recebe a resposta em pedaços, com as fontes (título e página do PDF), e as conversas ficam guardadas, com título, até ele apagá-las.
> - **O que a ADR-07 descartou.** "Serviço de IA separado: um Cloud Run a mais para operar, sem ganho", com o gatilho "volume que justifique isolar o consumo, ou GPU própria". **Nenhum dos dois gatilhos disparou**: a revisão é decisão do dono do produto, não medida, e é por isso que ela fica escrita aqui com as consequências inteiras.
> - **O que este produto pede e as outras rotas de IA não têm**: conversa de vários turnos com memória (sessão), ferramentas que buscam em índice (os dois *data stores* da **ADR-127**) e resposta transmitida (**ADR-128**).
> - **A escolha do dono do produto**: Google ADK, em Python, servido pelo Vertex AI Agent Engine — que **gerencia a sessão** (eventos, estado, expiração).

Decisão.

- **O agente é um pacote Python do *monorepo***: `apps/agente-professor`, Python 3.12, `google-adk~=2.10.0` fixado no `uv.lock`, servido por uma imagem de *container* que segue o contrato de *runtime* do Agent Engine (porta 8080, `/api/reasoning_engine` e `/api/stream_reasoning_engine`). O *prompt*, as ferramentas e os modelos moram nele: a conversa em `AGENTE_MODELO`, a guarda de escopo e o título em `AGENTE_MODELO_ESCOPO`, ambos em `global` com reserva em `us` pelo `client_kwargs` do `Gemini`. **`GOOGLE_CLOUD_LOCATION` é reservada no engine** e vale a região do *deploy*: configurar o modelo por ela o mandaria para `us-east1`, onde ele pode nem existir.
- **A api é a ÚNICA porta.** O navegador nunca fala com o engine nem com o *data store*. Na api: `@Roles("professor")` e só ele (a coordenação ficou fora por decisão do dono); `DisponibilidadeGuard` com `@Produto("agente-professor")` na classe — o primeiro produto sem experiência (**ADR-87**); *rate limit* de 10 mensagens por minuto por professor; e o `QuotaService` do perfil `professor`, com `temSaldo` **antes** de abrir o *stream* e `consumir` num `finally` com a **soma dos *tokens* dos eventos NÃO parciais** do turno — o que inclui a guarda de escopo e o título, que o agente soma ao `usage_metadata` do evento seguinte.
- **Só `src/ai/agent-engine.client.ts` conhece o engine**; o módulo `src/agente-professor` fala com a `AgenteProfessorPort`. Porta própria, e não um método a mais no `AiProvider`, porque modelo, ferramentas e instrução moram no agente: a api só conduz a sessão.
- **A posse mora no Postgres**: `agente_professor_conversas` (`escola_id`, `usuario_id`, `sessao_id`, `titulo`), com RLS. Os *endpoints* de sessão do engine pedidos por nome de recurso **não conferem o usuário**, e a SA da api enxerga todas as sessões do engine: sem a linha, quem soubesse um `sessao_id` leria a conversa de outro professor. A checagem do ADK ("does

not belong to user") é defesa em profundidade, não a tranca. Toda rota confere `usuario_id` e `escola_id`; outro dono é 404.

- **Identidade só do *token***: o `user_id` no engine é o `users.id`; o `nome_professor` do estado é o primeiro nome de `users.display_name`, lido do banco.
- **O título é do agente**: o LLM dele escreve `state.titulo_conversa` depois do primeiro turno, e a api o espelha na coluna `titulo` — que é *cache* do estado, não fonte.
- **Histórico sem limite, com exclusão.** A sessão nasce com `expire_time` distante (dez anos pedidos) e é renovada por `PATCH …/sessions/{id}?updateMask=expireTime` na API nativa a cada mensagem entregue; apagar é `async_delete_session` mais a linha, e o `DELETE` é a única rota com `@SemDisponibilidade`.
- **Região**: `us-east1` em `dev` e `hml`; produção fica a decidir. Na api, `AGENTE_PROFESSOR_MODO` ausente vale `desligado`, em qualquer `NODE_ENV`: o engine nasce por contrato em momentos diferentes, e derrubar a api por falta dele tiraria todos os produtos do ar para proteger um.

Alternativas descartadas.

- **Um `conversar()` na própria api, com a ADR-07 de pé.** Era o caminho de menor custo operacional: o *function calling* já está no `AiProvider` (**ADR-23**) e o `unoogpt/` já tem agente com ferramentas. O dono do produto escolheu o ADK pelo que ele entrega pronto e a api teria de construir: a sessão gerenciada (eventos, estado e expiração, sem tabela de mensagens própria nem formato de histórico a versionar), o laço de ferramentas transmitido, os *plugins* (guarda de escopo, título, poda de contexto) e a avaliação por `adk eval`.
- **Um Cloud Run próprio com `adk api_server`.** Os *endpoints* são outros (a api teria dois clientes, um por ambiente), a sessão iria para um banco pelo `DatabaseSessionService` e seria o serviço Cloud Run novo que a doutrina recusa sem gargalo medido.
- **`DatabaseSessionService` no Cloud SQL**, com ou sem o engine. O ADK cria as próprias tabelas: sem `escola_id`, sem RLS, fora do Prisma e das migrações — a regra inviolável nº 1 quebrada por um esquema que o projeto não controla, numa segunda conexão ao banco vinda de outro processo.

Consequências ruins, declaradas.

- **Um segundo *runtime***: Python e `uv` no CI (`astral-sh/setup-uv`), uma segunda imagem e um segundo *job* de *deploy*. Sem `uv` na máquina, o `pnpm turbo` da raiz falha de propósito.
- **Um segundo lugar onde moram *prompt* e modelo** (`instrucoes.py`, `modelos.py`), fora de `src/ai/prompts.ts`. O `MedidorDeProvider` não mede o agente; a métrica dele é a linha de log `agente-professor.turno`.
- **As quotas do Agent Engine**: 90 `streamQuery` por minuto e 300 eventos de sessão por minuto, por projeto e região. Uma mensagem gasta um `streamQuery` e vários eventos. O aumento se pede no console e leva dias: **tem de ser pedido antes do *go-live***.
- ***Cold start***: o primeiro turno depois de o engine ficar ocioso paga a subida do *container*.

- **O conteúdo da conversa mora fora do Postgres.** O professor pode escrever o nome de um estudante, e esse texto fica no engine, fora do PITR e das exportações do banco. Apagar exige o engine no ar: com ele fora, o `DELETE` responde 503 e a linha **fica** — apagar só a linha deixaria a conversa guardada sem dono alcançável. E apagar o engine apaga o histórico de todos (o `deletion_policy` padrão recusa engine com sub-recursos; não se troca para `FORCE`).
- **O modelo processa em `global`**, sem garantia de residência no Brasil — como já era na api (`docs/INFRA.md`); consta do DPA.
- **O máximo do `expire_time` não está documentado** (só o mínimo de 24 h e o padrão de 365 dias). Se o engine recusar o pedido, a criação repete sem ele, e é a renovação a cada mensagem que mantém a conversa viva: a que passar o TTL sem mensagem some da lista no próximo `GET`.
- **O papel custom da SA da api não tranca nada hoje**: ela já tem `roles/aiplatform.user` no projeto por causa do Gemini. Ele existe para o dia em que o papel amplo for estreitado.

Gatilho de reavaliação. O volume do chat bater nas quotas do Agent Engine mesmo depois do aumento; o ADK ou o Agent Engine mudarem de contrato a ponto de o cliente da api pedir manutenção recorrente; produção exigir região que o Agent Engine não tem; ou o segundo recurso de IA pedir a mesma exceção — aí a pergunta é se o engine vira a camada de agentes da plataforma (e a ADR-07 é revista de vez) ou se o agente volta para dentro da api.

18.128 ADR-127 — Só *data store*, inclusive para as habilidades da BNCC

Data. 2026-09-30. **Status.** Aceita, em avaliação. **Recorta** a **ADR-73 para este produto**: a recusa de RAG para as habilidades continua valendo para o planejamento de ensino (app-92).

> **Contexto**
>
> - **A ADR-73 recusou busca sobre os PDFs para as habilidades**: "busca vetorial responderia com aproximação onde existe resposta certa". As habilidades viraram dado extraído (`src/curriculo`), com código, ano, componente e texto oficial.
> - **O agente precisa de duas coisas diferentes**: a lista *exata* ("todas as habilidades de Matemática do 6º ano", o texto oficial de um código) e *trechos* da BNCC completa, do anexo de Computação e dos livros didáticos, com a página para citar.
> - **A escolha do dono do produto**: usar só *data store* (Vertex AI Search, o "Agent Search" da documentação de 2026) para as duas, e avaliar.
> - **A lição que pesa contra**: em 22/09 o `gerador-missoes` inventou eixo e texto para uma habilidade real. Modelo citando código é o lugar onde a aproximação vira erro de currículo.

Decisão. A resposta exata é preservada **pela forma do dado e da consulta**, não pela confiança no modelo:

- **Habilidades num *data store* ESTRUTURADO** (`agente-professor-habilidades`), **uma linha por habilidade**, com esquema próprio (`dynamic = false`). O JSONL sai de `src/curriculo/dados` e o `exportar_habilidades` **aborta** se os totais não forem 1.304 EF + 183 EM + 130 Computação — o *import* usa `reconciliationMode: FULL`, que apaga o que falta.
- **Consulta exata**: `buscar_habilidades` manda *query* vazia com filtro exato (`codigo`, `anos`, `componente_slug`, `etapa`) e **pagina até o fim**; com `texto`, faz busca semântica sob o mesmo filtro. Um dicionário só (`curriculo.py`) normaliza componente, ano e etapa para quem escreve o *data store* e para quem consulta.
- **Os PDFs num OUTRO *data store*** (`agente-professor-acervo`), não estruturado, com *parser* de *layout* e *chunking* de 500 *tokens* com títulos ancestrais; o modo `CHUNKS` devolve o `pageSpan`, convertido para a página do PDF original quando o arquivo foi dividido. São dois porque um *engine* de busca com *data store* estruturado é forçado a `DOCUMENTS` e perde os trechos. Não há *engine* (app) de busca: responde o `servingConfigs/default_search` do próprio *data store*.
- **Ferramentas próprias sobre o REST do Discovery Engine, nunca `VertexAiSearchTool`**: ela tem teto de 10 resultados, não pagina e, com outra ferramenta no agente, é trocada em silêncio por um substituto que descarta a página.
- **Conferência determinística na api.** Todo código citado no texto do turno é conferido contra `habilidadePorCodigo` de `src/curriculo`; o que não existe vira o evento `aviso` (`habilidade-inexistente`) no fim do turno. É aviso, não censura: o texto já está na tela, e o aviso diz para conferir antes de pôr num plano.
- **Escopo fechado**: só BNCC e acervo. A guarda de escopo, com o modelo leve, recusa pelo nome antes de chamar o modelo principal.

Alternativas descartadas.

- **Ferramenta exata sobre `src/curriculo`** — a ADR-73 aplicada ao agente. Garante a resposta certa por construção; descartada pelo dono para avaliar o *data store*, e é a volta prevista no gatilho. Custaria levar o dado ao agente Python (hoje são `.ts` gerados) ou uma ferramenta que chama a api.
- **Um *data store* só, com tudo.** O estruturado perde os trechos; o não estruturado perde o filtro exato.
- **`VertexAiSearchTool`**, pelos três defeitos acima: "as habilidades de Matemática do 6º ano" voltando com dez é um plano que descumpre o currículo, e ninguém vê.
- **Índice vetorial próprio no Postgres**: extensão nova, *pipeline* de *embeddings* a manter, e a mesma aproximação.

Consequências ruins, declaradas.

- **A exatidão depende de o filtro casar com o dado.** Um *slug* divergente entre a ingestão e a consulta não dá erro: dá "nenhuma habilidade encontrada". O caso que mais custa é o EM (Biologia, Física e Química viram `ciencias-da-natureza`).

- **Ainda não avaliado.** O relatório de `ingestao/avaliar_habilidades` (*recall* por componente × ano) não rodou contra o *data store* real até esta data.
- **Dois *data stores* a manter e reimportar** quando um PDF muda; o *chunking* nasce certo ou não nasce (a configuração de processamento é imutável: trocá-la é recriar e reimportar tudo).
- **Contexto caro**: a Língua Portuguesa do 8º ano inteira tem cerca de 58.000 caracteres, e a poda de `buscar_habilidades` foi levantada para 120.000 para não cortar o plano.
- **A página citada é a do leitor de PDF**, não a impressa (na BNCC completa, impressa = PDF − 2).
- **O aviso chega depois do texto**: o professor já leu o código inexistente quando o aviso aparece.

Gatilho de reavaliação. O relatório de `apps/agente-professor/ingestao/avaliar_habilidades`: se o *recall* não for 100% em todo componente × ano, volta a ferramenta exata sobre `src/curriculo`. E o primeiro `aviso` de `habilidade-inexistente` visto em uso real conta como evidência a favor da volta.

18.129 ADR-128 — SSE só no chat do agente

Data. 2026-09-30. **Status.** Aceita. **Não revê** a ADR-93: a sala continua em *polling* curto.

> **Contexto**
>
> - **A ADR-93 descartou SSE para sincronizar a sala**: conexão longa num Cloud Run com teto de *request timeout*, para um dado de dois valores lido durante dez minutos.
> - **Aqui o problema é outro.** Um turno do agente leva de segundos a minutos (guarda, busca, geração), e o texto nasce em pedaços. Sem transmissão, o professor olha um indicador de espera por um minuto; com o padrão 202 + *polling* das gerações (ADR-07), o texto parcial teria de ser guardado em algum lugar e reenviado inteiro a cada consulta.

Decisão.

- **SSE de ponta a ponta**: o engine (`:streamQuery?alt=sse`, que responde ora linhas `data:`, ora NDJSON — o cliente da api decide por linha, não pelo `content-type`) → a api (`text/event-stream`) → o hub → o navegador, que lê com `fetch` POST e o *stream* do corpo: o `EventSource` só faz GET, e a pergunta iria na URL.
- **Rota de *stream* própria no hub** (`/api/plataforma/agente-professor/conversas/[id]/mensagens`), exceção ao BFF genérico, que tem teto de 180 s e devolve o corpo sem os cabeçalhos de *stream*. Ela usa a mesma sessão e o mesmo *token* curto do BFF (`lib/bff.ts`), tem teto de 250 s (acima dos 240 s da api, que cortam antes com o evento `erro`) e responde `cache-control: no-cache, no-transform` — sem o `no-transform`, a compressão do servidor do Next segura os pedaços e o texto chega todo no fim.

- **Eventos fechados**: `delta`, `fontes`, `titulo`, `aviso`, `fim` e `erro`, com `: ping` a cada 15 s; o *stream* termina em `fim` **ou** em `erro`, nunca nos dois.
- **Toda recusa em JSON ANTES de abrir o *stream***: corpo (400), posse (404), `RATE_LIMIT` e `QUOTA` (429). Aberto o `text/event-stream`, o *status* já foi 200.
- **Tetos**: 240 s na api (`TEMPO_ESGOTADO`); o Cloud Run do hub corta em 300 s e o da api em 900 s; o *load balancer* com NEG *serverless* tem 60 minutos fixos. O elo mais curto é o da própria api, de propósito.
- **Cancelamento em cada salto**: "Parar", trocar de conversa ou fechar a aba abortam o `fetch` do navegador, o hub repassa o `request.signal`, a api ouve o `close` da RESPOSTA e aborta a chamada ao Vertex. **Fechar o painel NÃO aborta** (decisão do dono do produto): o painel fica montado e escondido, o turno segue, e a resposta que termina com ele fechado acende o selo do botão — abortar ali seria cobrar a quota e jogar a resposta fora. Os *tokens* gerados até um abandono são consumidos assim mesmo.
- **Não vale para nenhuma outra rota.** A segunda rota que pedir *stream* pede uma ADR nova.

Alternativas descartadas.

- **202 + *polling***: o parcial teria de morar no banco ou na memória de uma instância, e cada consulta reenviaria o texto inteiro.
- **WebSocket**: bidirecional sem necessidade e sem infraestrutura, pela mesma razão da **ADR-93**.
- **`EventSource`**: só GET, sem corpo — a pergunta do professor (até 4.000 caracteres, possivelmente com nome de estudante) iria na URL e nos registros do *load balancer*.
- **Repassar pelo BFF genérico**: o teto de 180 s cortaria o turno longo, e a resposta sairia sem os cabeçalhos de *stream* — comprimida, chegando de uma vez no fim.

Consequências ruins, declaradas.

- **Conexão longa no Cloud Run**: cada turno ocupa uma requisição concorrente na api e outra no hub por até 240 s.
- **O alerta de p95 da api (10 s) fica inflado pelos *streams***, e a métrica `run.googleapis.com/request_latencies` não tem rótulo de caminho. A proposta — uma métrica de log de distribuição sobre o *request log* do Cloud Run da api, excluindo o caminho do agente — está registrada no RUNBOOK do repositório de fundação e **não foi implementada**.
- **O `ContratoInterceptor` não confere o SSE** (a rota escreve na resposta crua). Quem confere cada `data:` é o teste, contra `DADOS_POR_EVENTO`; evento novo entra lá.
- **O ADK em SSE repete o texto** no evento final agregado; a api deduplica por segmento, e mudar isso duplica a resposta na tela.

Gatilho de reavaliação. A segunda rota que pedir transmissão; o alerta de p95 da api ficar inútil antes de a métrica filtrada existir; ou o Cloud Run, o *load balancer* ou o Agent Engine mudarem os tetos de que este desenho depende.

18.130 ADR-129 — O *markdown* do agente vira HTML no navegador

Data. 2026-09-30. **Status.** Aceita. **Excepciona** o precedente de `planos/` (app-90), em que o `marked` e o `sanitize-html` rodam na api e "o cliente não converte nada".

> **Contexto**
>
> O agente responde em *markdown* (listas, tabelas de habilidades, negrito), e o texto chega em pedaços pelo SSE da **ADR-128**. No `planos/` o documento chega inteiro, uma vez, e converter na api é o mais seguro: o navegador só exibe HTML já saneado. Com transmissão, o HTML de um pedaço não existe sozinho — uma tabela só fecha várias linhas depois.

Decisão. O hub converte, **no navegador**, com `react-markdown` + `remark-gfm`, e a segurança vem do que fica desligado:

- **HTML cru desligado**, sem `rehype-raw`: *tag* escrita pelo modelo vira TEXTO escapado, nunca elemento — e não some (`skipHtml` apagaria o trecho, e a resposta a "como explico a *tag* `<p>`?" perderia o assunto).
- **Links só absolutos `http`, `https` e `mailto`**, conferidos pelo `URL` do navegador e não por expressão regular, abertos em nova guia com `rel="noopener noreferrer"`; `javascript:` e `data:` não viram link. Imagem vira o texto alternativo.
- **Nada de `dangerouslySetInnerHTML`.**
- **A tipografia é do DS**: `RichContent` e `RichContentTable` da seção `unooria` (0.2.10).
- **O painel leva `data-clarity-mask`**: o professor pode escrever o nome de um estudante, e o Clarity grava a tela (**ADR-60**).

Alternativas descartadas.

- **Converter na api a cada `delta`**: o HTML do turno inteiro teria de ser reconvertido e reenviado a cada pedaço, e o evento deixaria de ser incremental.
- **Converter só no fim**: o professor leria *markdown* cru (asteriscos, barras de tabela) durante toda a transmissão, e a tela pularia de forma no último evento.

Consequências ruins, declaradas.

- **Dois conversores de *markdown* na plataforma**, com regras diferentes (`marked` na api, `react-markdown` no hub), e duas dependências a acompanhar.
- **A segurança passa a depender de configuração no cliente**: ligar `rehype-raw` ou aceitar outro esquema de link reabre o vetor, e nenhum saneador na api o pega depois.
- **O histórico é reconvertido a cada abertura** da conversa; o custo é do navegador do professor.

Gatilho de reavaliação. A resposta do agente precisar de elemento que o *markdown* não expressa (e a tentação de ligar HTML cru); ou um segundo produto querer o mesmo componente, caso em que ele sobe para o DS com a configuração travada.

18.131 ADR-130 — A atividade é do professor, e a equipe é da atividade

Data. 2026-10-01. **Status.** Aceita. **Supera em parte** a **ADR-111** (a montagem das equipes deixa de ser por turma × missão) e muda o nascimento do conteúdo da A Seleção (exp07ef1b) e do Jornal da Escola (exp04ef1b), que nasciam com o banco curado criado sozinho.

> **Contexto**
>
> O Instanooria (exp08ef1b) tinha, desde o começo, uma lista "Suas atividades nesta missão": cada atividade é uma configuração da missão para um tema do professor. As outras missões com ficha pedagógica já tinham a mesma coisa POR BAIXO — a cobertura da A Seleção, o pacote do Jornal e o do Quem Mandou Isso? (exp05ef1b) eram, na api, a atividade da evidência (**ADR-112**) —, mas a tela escondia isso em "A aula e os resultados", onde aprovar um tema novo TROCAVA a aula em andamento. E as equipes eram uma seção solta no alto da ficha, uma montagem por turma × missão: duas atividades da mesma missão na mesma turma (uma por conteúdo, em dias diferentes, com o laboratório dividido de outro jeito) dividiam, em silêncio, as mesmas equipes. O dono do produto pediu as quatro missões como o Instanooria, com a equipe DENTRO da construção da atividade.

Decisão.

- **A missão é o esqueleto narrativo; a atividade é a missão que a turma faz de fato, e a plataforma nunca a cria.** Acabou a cobertura `padrao` criada na primeira entrada da A Seleção e o pacote `padrao` do Jornal: sem atividade em andamento, a api responde 403 `atividade-nao-preparada` — a mesma porta fechada que a experiência já mostra para a liberação (**ADR-69**), e por isso nenhum *bundle* muda. O banco curado vira ponto de partida editável da criação.
- **Criar não põe no ar.** A atividade nasce PRONTA, e o professor escolhe, na lista, qual está EM ANDAMENTO ("Usar esta atividade agora"); uma por turma. Pôr no ar abre a rodada SEGUINTE à última que rodou, na etapa inicial: retomar a mesma depois do Mural seria o "voltar" que o botão único proíbe. No Instanooria, "Ativar" passou de "uma ativa por professor" a "uma ativa por turma".
- **A criação é um assistente de quatro passos** no hub (turma, conteúdo, equipes, revisar e criar), o mesmo para as missões; o passo do conteúdo é de cada uma. O Instanooria mantém a criação dele (a geração dos posts é longa e assíncrona) e as equipes entram na página da atividade, antes do "Ativar".
- **A equipe é da atividade**: `atividade_id` (texto opaco, o alfabeto de `evidencias_marcas`) nas três tabelas de equipes e na chave; a missão sem atividade (o unooGPT, até ser refeito)

usa o id fixo `turma`. A atividade em andamento de cada missão é lida num `Record` exaustivo (`equipes/atividades.ts`), e é ela que vale para a faixa do estudante e para a sala.
- **Terceiro modo, "um aluno por computador"** (`individual`): grava a montagem, sem equipes e com `computadores` nulo — o painel volta a contar por criança.

Alternativas descartadas.

- **Uma tabela geral de atividades na plataforma**: obrigaria a migrar quatro modelos que já funcionam (e já são a atividade da evidência) para um quinto; o `atividade_id` opaco faz o mesmo papel nas equipes, como já fazia na evidência.
- **Manter a equipe por turma × missão e só mover a tela**: resolvia a tela e mantinha o defeito — a segunda atividade herdando as equipes da primeira.
- **Manter o conteúdo padrão criado sozinho**: a lista teria uma atividade que o professor não criou, e "a plataforma não cria nada" deixaria de valer justamente nas duas missões em que ela criava.
- **Criar já em andamento** (a regra de antes): preparar a próxima aula tiraria do ar a de agora.

Consequências ruins, declaradas.

- **A missão só abre com uma atividade em andamento.** Professor que libera a missão na sala e não cria atividade vê a turma diante da porta fechada; a lista da ficha diz isso, mas é um passo a mais do que antes.
- **Criar e montar equipes não são uma transação**: são dois módulos. Se a equipe falhar depois de a atividade nascer, a atividade fica, e a página dela mostra o que faltou — nunca se apaga o conteúdo por causa da equipe.
- **Quem faltou só se sabe no dia**: o sorteio da criação é sobre a turma inteira, e a página da atividade é onde se marca a falta e se refaz.
- **As montagens que já existiam** foram para a atividade em andamento da turma; a que não tinha atividade em andamento saiu.

Gatilho de reavaliação. O unooGPT ser refeito com atividades (o id fixo `turma` sai); uma quinta missão com atividade (entra no `Record` de `equipes/atividades.ts` e em `MISSOES_COM_ATIVIDADE` no hub); ou a necessidade de reabrir a MESMA rodada de uma atividade, que pediria guardar a etapa por atividade, e não só no gate da turma.

19

Riscos e evolução

19.1 Registro de riscos

Probabilidade e impacto em escala de três (baixo, médio, alto). A ordem é por severidade, e a severidade considera o prazo de novembro de 2026.

ID	Risco	Prob	Imp	Mitigação e estado
R-01	**Carga não verificada.** O comportamento no pico de 4.000 simultâneos é projetado, não medido.	Média	**Alto**	Teste de carga com k6 — 4.000 *logins* e aberturas de experiência em dez minutos — **obrigatório antes de novembro de 2026**. **Não executado.** É o maior risco técnico aberto.
R-02	**Retenção indefinida de dado de menor.** Não há prazo declarado para `app_70_sessoes` nem para `telemetria_eventos`.	Alta	**Alto**	Decisão consciente (ADR-13) em troca de reprocessamento. Com a tabela particionada, a rotina de expurgo passou a ser trivial — o que remove o último argumento técnico. **Pendente de decisão de produto e contrato**, e é da plataforma inteira.
R-17	**Latência de IA numa sala inteira.** O turno da exp02em1b é síncrono e leva de 10 a 30 segundos, e a largada é simultânea: 30 estudantes perguntam ao mesmo tempo.	Média	**Alto**	O desenho aceita a latência (a tela cobre com indicador de "consultando", e a SDK espera 45 s). O que **não** foi medido é a concorrência: 30 turnos simultâneos por turma, com até seis voltas de ferramenta cada. Entra no mesmo teste de carga de R-01. Saída conhecida se reprovar: `streamGenerateContent` no provider — mudança dentro de `src/ai`, sem tocar o módulo da experiência.

ID	Risco	Prob	Imp	Mitigação e estado
R-18	**Custo de IA da exp02em1b.** Uma partida custa duas ordens de grandeza mais que um turno de tutor.	Certa	Médio	**MEDIDO** em 2026-08-27: 71.397 tokens numa partida completa de 8 turnos (**ADR-24**). Os tetos foram fixados sobre esse número, e a medição achou de passagem que a quota diária de 20.000 recusava o quarto turno — corrigida para 300.000. **Fechado**, com um resíduo: a medição é de UMA partida, e o custo varia com o número de voltas que o modelo escolhe dar.
R-19	**Guia de condução do debate da exp02em1b não existe.** A missão deliberadamente não revela a Lacuna de nenhuma fonte por software: a lição acontece no debate.	Certa	**Alto**	O guia do projeto de origem está vazio. A ficha publicada (competência, quatro atos, preparo) foi redigida a partir das ADRs e dos dois *playtests*, e está marcada como ponto de partida. **Pendente de revisão pedagógica** — é a lacuna mais séria desta experiência, e não é técnica.
R-03	**Localidade de processamento de IA.** O tráfego de IA sai de São Paulo, inclusive o do estudante.	Certa	Médio	Consequência aceita e documentada (**ADR-16**). **Precisa constar do contrato e do DPA de cada rede.** Voltar à região exige trocar modelo e localização juntos.
R-24	**Alavanca de incidente no painel da Unooria.** Um clique no escopo-contrato desliga um produto para os ~10 mil estudantes de uma rede, e o sintoma que ele produz — a missão desapareceu da grade — é **indistinguível** de um defeito de enturmamento ou de catálogo (**ADR-87**).	Possível	**Alto**	Mitigado em quatro camadas, e nenhuma delas é permissão: diálogo de confirmação que declara o alcance, `motivo` obrigatório ao desligar (CHECK no banco, não só Zod), registro estruturado `recurso-alterado` em toda escrita, e o sintoma no apêndice de operação. **Nada é destruído**: a liberação por turma do professor não é tocada, e religar devolve a sala como ela estava. O que NÃO está mitigado é a auditoria em tela — a linha é sobrescrita a cada mudança, e o histórico de quem mexeu vive só no registro.

ID	Risco	Prob	Imp	Mitigação e estado
R-26	**Histórico do painel sem prazo.** `recursos_auditoria` só cresce, e nenhuma tabela deste produto tem prazo declarado hoje (**ADR-89**).	Certa	Baixo	Mesma decisão consciente de `R-02`: prazo sem rotina que o cumpra é pior que prazo nenhum. O volume aqui é ordens de grandeza menor que `telemetria_eventos` — são algumas mudanças por semana, não evento por criança —, e nenhuma linha carrega dado de pessoa além do NOME de quem operou o painel (conta da Unooria ou da coordenação, nunca de estudante). Quando o prazo vier, ele vem para as duas tabelas na mesma conversa.
R-25	**Versão fixada por escopo divergindo em silêncio.** Uma escola pode ficar meses numa versão antiga porque alguém a fixou como canário e não a soltou — e nada avisa.	Possível	Médio	O painel mostra a fixação por escopo, e o *rollback* do contrato tem aviso próprio (`emRollback`). A fixação por escopo **não** tem: ela sobrevive ao *deploy* de propósito (o CI não toca `disponibilidades`), e é justamente isso que a torna esquecível. Gatilho para agir: a primeira fixação que passar de um bimestre — aí ela pede prazo de validade, que é o mesmo mecanismo do gatilho de reavaliação da **ADR-87**.
R-04	**Fator humano único.** Uma pessoa desenvolve e opera.	Certa	**Alto**	Mitigado *por arquitetura*: menos peças, reversão por ponteiro, decisões registradas em ADR e este documento. Não é mitigável por engenharia sozinha.
R-05	**Sem *alerting*.** Há log e métrica nativa; não há alerta nem painel, e nenhuma sonda chama `/readyz`. Erro no navegador do estudante não deixa rastro.	Alta	Médio	**Estudado e planejado** (Capítulo 17, **ADR-57**, **ADR-58**): a fase 0 — canais, cinco *uptime checks*, quatorze políticas por sintoma, SLOs, painel, exclusões e *dead letter*, tudo em Terraform — fecha o risco e **continua obrigatória antes de novembro de 2026**. Custo estimado por contrato: cerca de US$ 1 por mês. **Não implementado.**
R-06	**Sem revisão de segurança externa.** Nenhum teste de intrusão.	Alta	**Alto**	Item de contratação, não de engenharia. Para plataforma que trata dado de menor em rede pública, é exigência prática de edital.
R-07	**Acessibilidade não verificada.** Nenhuma checagem de contraste, foco ou leitor de tela.	Alta	Médio	Rede pública tem exigência legal e prática. O *design system* é o lugar certo para resolver de uma vez — o que torna isto um pedido ao `storybook-unooria`, não 48 correções.

ID	Risco	Prob	Imp	Mitigação e estado
R-08	**Partição de telemetria sem *job*.** As faixas vão até 2028-12 e não há rotina que crie novas.	Baixa	Médio	A partição `DEFAULT` é rede de segurança (evento não é recusado). Verificar a contagem dela é item do *checklist* de operação. Encher = faltou criar partição.
R-09	**Mídia pesada publicada à mão.** O CI publica `dist/`; os 52 MB de MP4 da exp04ef1b vão por `gsutil` manual.	Alta	Baixo	Aceitável para uma experiência; **não escala para 48**. O passo `gsutil rsync` de mídia precisa entrar no CI.
R-10	**Geração órfã sem retomada no *boot*.** Se todas as instâncias morrerem e ninguém abrir o painel, a geração fica parada.	Média	Baixo	Aceito (**ADR-17**): o *polling* do painel é justamente quando alguém se importa com aquela geração. Gatilho: primeira reclamação real.
R-11	**Dependência do *design system* externo.** Lacuna do pacote bloqueia a experiência até a próxima publicação.	Média	Baixo	Compensação local marcada no CSS, decrescente por política, com pedido correspondente. Cinco lacunas abertas, verificadas na 0.1.14.
R-12	**Catálogo de modelos de IA instável.** Dois modelos *default* já saíram do catálogo sem aviso.	Alta	Médio	*Provider* plugável (**ADR-07**), modelo em variável de ambiente, e `finishReason` tratado como erro. O sintoma da última vez foi silencioso: toda geração de imagem caindo no *fallback* procedural.
R-13	**WSO2 ainda não é o emissor.** A plataforma roda com o hub assinando.	Média	Baixo	**ADR-15** é transitória por desenho: trocar são duas variáveis. O risco real é o `iss` *tenant-qualified* do WSO2 ser montado por concatenação em vez de copiado do *discovery*. Ficou **mais barato** com o enturmamento: `users.fonte_id` existe para casar o `sub` do IdP com a pessoa, sem depender do `username`.
R-14	**Virada do ano letivo.** Em fevereiro de 2027 — três meses depois da entrada em produção — o 8A vira 9A em toda a rede.	Alta	**Alto**	**Mitigado** por **ADR-28**: com `periodos_letivos` e matrícula datada, o "8A de 2026" e o "8A de 2027" são turmas diferentes, e o acumulado do ano anterior continua apontando para a turma certa. Antes disso, `users.turma_id` era coluna: todo relatório histórico passaria a ser lido pela turma nova, **em silêncio**. O que **falta** é a operação da virada — criar o período de 2027 e importar o enturmamento novo — e ela é manual hoje.

ID	Risco	Prob	Imp	Mitigação e estado
R-15	**Senha compartilhada até o primeiro acesso.** Importação com uma senha para o lote inteiro deixa qualquer estudante entrar na conta de outro antes do primeiro acesso dele.	Média	Médio	Aceito e **dito na própria tela** (ADR-29), com o modo "uma senha por pessoa" ao lado, recomendado para turma grande. Fecha de vez quando o WSO2 for o emissor.
R-16	***Session replay* de terceiro sobre tela de menor.** O Microsoft Clarity grava a sessão no hub e dentro do `iframe` da missão (ADR-60); a documentação do próprio Clarity diz que ele **não deveria ser usado em aplicações destinadas a menores de 18 anos**, e a plataforma vai do 1º EF ao 3º EM.	Alta	**Alto**	**Aceito como decisão de produto**, não mitigado. O que existe hoje: o `identify` manda só o UUID opaco (nunca nome, nunca e-mail), o `track()` manda `verbo:objeto` sem a resposta do estudante, e o Clarity mascara por *default* campo de entrada, número e e-mail. O que **falta** é a ponta contratual: a gravação sai do projeto GCP do contrato para um processador de terceiro, e isso precisa estar no DPA da rede **antes** do primeiro contrato em produção. Desligar é apagar `CLARITY_PROJECT_ID` do *environment* — sem *deploy* de código, mas com *rebuild* da imagem.
R-21	**Texto livre de menor persistido.** O Eco no Grupo (retirado, ADR-113) reabriu um canal que o produto tinha fechado por desenho, e as missões seguintes o mantêm aberto (exp06ef2b, exp08ef4b, exp05ef1b).	Certa	Médio	Aceito com contenção: teto de caracteres, RLS, nenhum caminho para *log* ou telemetria, e o texto alcançável só pelo professor da turma — a exp08ef4b ainda recusa dado pessoal antes de gravar. Entra no mesmo regime de retenção de R-02, e é mais um argumento para fechá-lo.

ID	Risco	Prob	Imp	Mitigação e estado
R-23	**Turma sem missão liberada no dia da aula.** Desde a sala de aula (**ADR-69**) a missão só abre onde o professor liberou — e um passo novo antes da aula é um passo que se esquece com a sala cheia.	Alta	Médio	**Mitigado na tela, não no código**: a sala mostra a grade inteira com o estado de cada missão, e a criança vê o **cadeado** em vez de uma lista vazia — "a missão sumiu" e "ainda não abriu" deixam de ser a mesma coisa. O 403 carrega `codigo: missao-nao-liberada`, então o sintoma aponta para a causa em vez de virar chamado. O que **falta** é janela de datas ("abre segunda, fecha sexta"), e ela não nasce sem rotina que a cumpra: é a armadilha do `expires_at`.
R-27	**Quota do Agent Engine.** O padrão é 90 `streamQuery` e 300 eventos de sessão por minuto, por projeto e região, e uma mensagem gasta um `streamQuery` e vários eventos (**ADR-126**).	Alta	**Alto**	O *rate limit* da api (10 mensagens por minuto por professor) protege o professor, não o projeto: a rede inteira às 8h passa de 90 por minuto. O aumento se pede no console e leva dias. **Pendente: pedir antes do *go-live***, e medir no mesmo teste de carga de `R-01`.
R-28	**O ADK muda a cada duas semanas.** O contrato que a api lê do *stream* (parciais, final agregado repetido, título no `state_delta`, *tokens* no `usage_metadata`) é comportamento medido, não documentado.	Alta	Médio	Versão fixada (`google-adk~=2.10.0` no `uv.lock`) e as medições em `tests/test_e2e.py` do agente. Subir de versão é refazê-las; o `DeduplicadorDeTexto` e a soma de *tokens* são os dois pontos que quebram em silêncio (resposta duplicada; quota errada).
R-29	***Stream* truncado depois de ferramenta.** Se a conexão com o engine terminar limpa logo depois do retorno de uma ferramenta, sem o texto final e sem evento de erro, a api toma o fim do corpo como fim do turno.	Média	Médio	As duas formas de falha medidas (evento com `error_code`; conexão cortada) viram `AGENTE_INDISPONIVEL`. O corte limpo, não: o professor vê uma resposta vazia ou pela metade terminando em `fim`. **Aberto**; a saída é exigir texto do agente depois da última resposta de ferramenta antes de emitir `fim`.

ID	Risco	Prob	Imp	Mitigação e estado
R-30	**Residência do agente.** A sessão (o histórico, que pode trazer nome de estudante) fica no Agent Engine em `us-east1`; o modelo e os *data stores*, em `global`.	Certa	Médio	Mesma consequência de `R-03`, agora com dado guardado, não só processado. A região de produção está **a decidir**, e precisa constar do DPA. Apagar exige o engine no ar (o `DELETE` responde 503 e a linha fica).
R-31	**O máximo do `expire_time` não é documentado.** A api pede dez anos e renova a cada mensagem; a documentação só diz o mínimo (24 h) e o padrão (365 dias).	Média	Médio	Se o engine recusar o pedido, a criação repete sem ele e a sessão nasce com 365 dias; a renovação a cada mensagem a mantém viva, mas a conversa sem mensagem por mais que o TTL some da lista. **Confirmar em `dev`** antes de prometer "histórico sem limite".
R-32	***Data store* de habilidades não avaliado.** A resposta exata depende de o filtro casar com o dado (**ADR-127**).	Média	**Alto**	O relatório de recall por componente × ano (`ingestao/avaliar_habilidades`) **não rodou contra o data store real**. A conferência de códigos da api pega código INVENTADO, não habilidade OMITIDA — e a omissão é o plano que descumpre o currículo. Recall abaixo de 100% devolve o produto à ferramenta exata sobre `src/curriculo`.



Figura 19.1 – Matriz de riscos. A maioria dos riscos da zona crítica **não é resolvida por código**: retenção é decisão de produto, segurança externa é contratação, o *session replay* sobre tela de menor é cláusula de DPA, o guia de condução é revisão pedagógica, e o fator humano único é estrutural.

19.2 Estado de implementação

Fase	Escopo	Estado	Observação
M0	Contratos congelados (*claims*, ponte, telemetria)	✓ Concluído	`packages/sdk`, `src/auth/claims.ts`
M1	*Tenancy*, RLS, papéis, importação de usuários	✓ Concluído	Teste com *role* sem *ownership*
M2	exp08ef1b completa: motor, geração, dossiê	✓ Concluído	Migração do aplicativo de origem
M3	exp04ef1b, domínio por experiência	✓ Concluído	**ADR-09**
M4	Ferramentas de professor: app-90, app-91	✓ Concluído	Armazenamento privado, *guardrails*
M5	app-70 e o Ato 1 integrado	✓ Concluído	`exp08ef1b_app_70_perfis_importados`
M6	Hub do estudante, *header* único, casca compartilhada	✓ Concluído	`/estudante`, `CascaPlataforma`
M7	exp02em1b e exp07ef1b: IA em *runtime* dentro da missão	✓ Concluído	**ADR-23**, **ADR-66**, **ADR-67**; a exp07ef1b (O Eco no Grupo) foi retirada depois (**ADR-113**)
—	*Stack* `dev` em GCP operacional	✓ Concluído	CI/CD completo com WIF
—	**Teste de carga**	– Pendente	`R-01`
—	**Observabilidade (fases 0–2)**	– Pendente	`R-05`; estudo e roteiro no Capítulo 17
—	**WSO2 como emissor**	– Pendente	**ADR-15**
—	Agente de IA do professor	✓ Código; – engine	**ADR-126**: api, agente e ingestão prontos; o engine nasce no segundo *apply*; `R-27`, `R-32`
—	41 experiências restantes	– Em aberto	Gabarito estabelecido

19.3 Evolução planejada

19.3.1 Antes de novembro de 2026 (bloqueante)

1. **Teste de carga com k6**: 4.000 *logins* e aberturas de experiência em dez minutos, contra a *stack* `dev`. Verificar saturação de conexões, latência do `/runs/:id/feed` sob *lock*, e o caminho de ingestão de telemetria.
2. **Observabilidade, fase 0** (Capítulo 17): canais de notificação, *uptime checks* do JWKS e de `/readyz`, SLO de disponibilidade com *burn rate*, falha do Cloud Run Job de migração, saturação do Cloud SQL, *dead letter* da telemetria e *smoke* pós-*deploy* — tudo em Terraform e no `deploy.yml`, sem tocar em código de aplicação. A fase 1 (log estruturado com *redaction* e `ExceptionFilter`) entra logo depois, porque é o que torna o alerta diagnosticável.
3. **Decisão de retenção**, com a rotina que a cumpre. Se a resposta for "sem prazo", ela precisa estar no contrato — não só neste documento.
4. **Revisão de segurança externa** e teste de intrusão.
5. **DPA por rede** contemplando a localidade de processamento de IA.
6. **WSO2 como emissor**, com o `iss` *tenant-qualified* copiado literalmente do *discovery*.
7. **Agente de IA do professor**: pedir o aumento de quota do Agent Engine (`R-27`), rodar o relatório de recall das habilidades (`R-32`), decidir a região de produção (`R-30`) e confirmar o máximo do `expire_time` (`R-31`).

19.3.2 Depois, por ordem de retorno

Evolução	Por quê, e o que ela exige
Rsync de mídia no CI	`R-09`. Com 48 experiências, publicação manual de mídia deixa de ser aceitável.
Filtro por série vindo do manifesto	Hoje o hub do estudante lista missões de um mapa estático. Quando o manifesto virar a fonte, `series: []` tem de **continuar** querendo dizer "todas" — o contrário faz a experiência transversal desaparecer da tela sozinha.
Acessibilidade no *design system*	`R-07`. Resolver no pacote é resolver uma vez; resolver nas experiências é resolver 48 vezes.
Telemetria offline real (IndexedDB)	Hoje o *buffer* é em memória, e há um `TODO` explícito na SDK. Rede pública tem escola com conectividade ruim, e é lá que o dado se perde.
Métricas de aplicação	Hoje só métrica nativa de infraestrutura. Nada mede "quantos estudantes concluíram a missão hoje".
Reprocessamento do mapeamento	`VERSAO_MAPEAMENTO` vai subir, e o *payload* está guardado justamente para isso (**ADR-13**). Falta a rotina que reprocessa e o relatório que compara duas gerações.
Visão agregada de bem-estar	Só com DPA e *k*-anonimato explícitos (**ADR-10**). **Não sai de um `GROUP BY`.**

Worker dedicado de telemetria	Se o volume justificar isolar o consumo (**ADR-21**).
BigQuery	Só se um relatório efetivamente precisar de análise em escala sobre `telemetria_eventos` (**ADR-14**).

19.4 O que continuará fora de escopo

Descartado	Condição para reavaliar
Kubernetes (GKE)	*Workload* de longa duração ou topologia de rede que o Cloud Run não expresse.
Microsserviço por experiência	Nenhuma. Contraria **ADR-01** na raiz.
Multi-região	Exigência contratual de continuidade regional.
Redis	Gargalo medido que o Pub/Sub e o `INSERT` em lote não resolvam.
Módulo genérico de avaliação	Três experiências consecutivas com mecânica idêntica.
Banco ou *schema* por experiência	Necessidade de fronteira de `GRANT` por experiência.

> **NOTA**
>
> A lista acima é tão importante quanto a de evolução. Num projeto com uma pessoa e um prazo, **o que se recusa a construir é o que permite terminar o resto** — e recusar sem registrar a condição de reavaliação transforma a recusa em teimosia.

20

Conclusão

20.1 O que foi construído

A Plataforma Unooria é um *monorepo* com dois serviços em execução — um hub Next.js e uma API NestJS — servindo um catálogo crescente de experiências imersivas entregues como *bundles* estáticos versionados em CDN. Em 27 de agosto de 2026, o estado é:

Dimensão	Estado
Serviços em execução	2 (hub, API) + 2 gerenciados (PostgreSQL, Pub/Sub)
Experiências publicadas	5 de 48, cobrindo os quatro casos estruturais: estudante com IA, estudante sem IA, jogo transversal e ferramenta de professor
Módulos da API	22, com um módulo de domínio por experiência
Tabelas	27 em `public` + partições mensais em `particoes`
Migrações	9, todas com RLS preservada
Arquivos de teste	34 (regra pura, integração, contrato, *smoke*)
Walkthroughs visuais	7
Decisões registradas	75 ADRs
Milestones concluídos	M0–M6

20.2 As decisões que mais definiram o resultado

Se este documento tivesse de ser reduzido a cinco frases, seriam estas.

1. **Experiência é artefato, não serviço** (ADR-01). É a decisão que torna 48 experiências viáveis para uma pessoa: o custo marginal de publicar a próxima é copiar arquivos, e a reversão de emergência é um `UPDATE` de uma coluna.
2. **Isolamento físico entre contratos, lógico entre escolas** (ADR-04, ADR-08). Uma *stack* por rede satisfaz o edital; RLS forçada com uma porta única de banco satisfaz a engenharia. E as duas trancas — política no banco e filtro na aplicação — são independentes de propósito.

3. **Regra que pode ser expressa em tipo é expressa em tipo** (ADR-12). `comTenancy` *lança* sem escola em vez de consultar sem filtro; a leitura de valores não *tem* campo numérico onde não há medida. Comentário avisa; tipo impede. É a diferença entre uma regra documentada e uma regra que vale.
4. **O que se recusa a construir é o que permite terminar** (Capítulo 19). Sem GKE, sem microsserviço por experiência, sem Redis, sem BigQuery, sem *worker* de telemetria, sem módulo genérico de avaliação — cada um com a condição de reavaliação escrita, para que a recusa não vire teimosia.
5. **Medir antes de otimizar, e medir antes de estimar** (ADR-14, ADR-16). O gargalo da telemetria não era o transporte, era um laço de `INSERT`. O armazenamento da app-70 não era 2,4 GB, era 30 MB. O modelo de IA não existia na região que parecia óbvia. Nenhuma dessas três coisas era previsível — e todas as três mudaram uma decisão.

20.3 O que este projeto ensinou sobre documentação

Vale registrar, porque muda como este documento deve ser mantido.

> **NOTA**
>
> As decisões mais finas deste projeto não estavam em documento nenhum: estavam em **comentários no código**, ao lado da linha que elas explicam — por que o terceiro argumento do `set_config` é `true`, por que `exp:ready` é reenviado, por que a tabela de contrato carrega dois números, por que `cartoesServidos` não é "apareceram".
> Isso é um bom padrão e deve continuar. O que este documento acrescenta não é substituir aqueles comentários — é dar-lhes um *lugar*: a estrutura em que uma pessoa que nunca viu o repositório consegue entender por que ele é assim, e a matriz que liga cada decisão ao requisito que a exige e ao teste que a verifica.

> **ATENÇÃO**
>
> O risco desta documentação é o mesmo do `expires_at` que foi removido do *schema* (ADR-13): **documento que ninguém atualiza é pior que documento nenhum**, porque quem o lê acredita nele.
> Por isso a atualização deste documento é item explícito da *Definition of Done* (Capítulo 16) e está registrada em `AGENTS.md`, não apenas aqui. Uma implementação nova que mude arquitetura, contrato, modelo de dados, decisão ou risco **tem de** atualizar o capítulo correspondente e regerar o PDF no mesmo *commit*.

20.4 Estado de prontidão para produção

Dimensão	Estado	Falta

Funcionalidade	✓ Pronta	As 43 experiências restantes são *conteúdo*, não plataforma.
Isolamento e segurança de *tenancy*	✓ Pronta e verificada	Revisão externa (`R-06`).
Entrega e reversão	✓ Pronta	*Rsync* de mídia no CI (`R-09`).
Identidade	✓ Funcional	WSO2 como emissor (**ADR-15**).
Desempenho sob pico	**– Não verificado**	**Teste de carga** (`R-01`). É o item que separa "funciona" de "funciona às oito da manhã".
Operabilidade	– Parcial	**Observabilidade, fase 0** (`R-05`; Capítulo 17).
Conformidade	– Parcial	**Retenção decidida** (`R-02`) e **DPA com a localidade de IA** (`R-03`).
Acessibilidade	– Não verificada	Checagem no *design system* (`R-07`).

LIMITAÇÃO CONHECIDA

A plataforma está **funcionalmente pronta** e **operacionalmente incompleta**. As quatro lacunas em negrito acima são bloqueantes para novembro de 2026, e três delas não são resolvidas por código: retenção é decisão de produto, DPA é jurídico, revisão de segurança é contratação. Reconhecer isso explicitamente é mais útil que declarar prontidão.

20.5 Encerramento

O desenho descrito aqui é conservador por escolha. Não há tecnologia interessante nele: há PostgreSQL com RLS, contêineres sem orquestrador, arquivos estáticos numa CDN e uma fila onde a fila era inevitável. A parte difícil não foi escolher as peças — foi recusar as que não cabiam, e escrever por que.

O que este documento tenta preservar não é o *estado* do sistema, que o código descreve melhor. É o *porquê*: as vinte e uma decisões, as medições que as sustentam, as armadilhas que já custaram tempo e as consequências ruins que foram aceitas de olhos abertos. Um sistema pode ser lido a partir do código. Um projeto, não.

21

Pendências identificadas e roadmap de evolução

Este capítulo consolida as pendências arquiteturais, lacunas de automação, débitos técnicos documentados e pontos de atenção operacional identificados durante a revisão técnica integral multiagente da Plataforma Unooria. As análises cobrem os contratos de nuvem, banco de dados, identidade, governança hierárquica, integridade de documentação e observabilidade.

O objetivo deste levantamento não é apenas listar tarefas pendentes, mas qualificar os riscos de cada uma, documentar a dependência com as decisões arquiteturais (**ADRs**) e registrar o gatilho de resolução para que a plataforma atinja maturidade plena para a operação de produção em novembro de 2026.

21.1 Quadro resumo de pendências

ID	Área	Descrição da pendência	Criticidade	ADR / Ref.
PEND-01	Identidade	Validação E2E da transição do emissor JWT do Hub para WSO2 IS em homologação	Alta	**ADR-03**, **ADR-15**
PEND-02	Testes E2E	Cobertura de testes automatizados para fluxos de `gestor_regional`	Alta	**ADR-80**, **ADR-81**
PEND-03	Monitoramento	Alarmes de exaustão de quota de tokens de IA por escola e perfil	Média	**ADR-24**, **ADR-74**
PEND-04	CI / OpenAPI	Portão de CI que valida sincronismo obrigatório do `docs/openapi.json`	Média	**ADR-37**
PEND-05	Banco de Dados	Automação periódica de checagem de integridade de RLS nas migrações	Alta	**ADR-08**, **ADR-20**

PEND-06	Frontend / Hub	Auditoria contínua de componentes locais versus pacote do Design System	Baixa	ADR-19, ADR-75
PEND-07	Telemetria	Validação de DLQ e reprocessamento assíncrono em picos matinais	Média	ADR-06, ADR-62
PEND-08	Catálogo Exp	Esteira de empacotamento estático automatizado para bundles 08 a 69	Baixa	ADR-01

21.2 Detalhamento técnico das pendências

21.2.1 PEND-01 — Transição do emissor de tokens OIDC para WSO2 IS

> **Contexto e Débito Atual**
>
> Conforme deliberado na ADR-15, o Hub temporariamente atua como emissor de tokens JWT RS256 para as experiências e para a API, expondo o JWKS em `/.well-known/jwks.json`. Com a evolução da governança para seis papéis (ADR-80) e a inclusão do atributo obrigatório `regional_id`, o contrato de identidade em `docs/INFRA-IDENTITY.md` prevê o WSO2 Identity Server 7.3.0 como o provedor central multi-contrato.

Ações necessárias:

1. Configurar no WSO2 IS o mapeamento de *claims* customizados: `turma_id`, `escola_id`, `perfil` e `regional_id`.
2. Conduzir teste de carga e autenticação em ambiente de homologação apontando `OIDC_ISSUER` e `OIDC_JWKS_URI` para a instância do WSO2.
3. Validar o comportamento do middleware ao expirar a chave no rodízio a frio previsto na ADR-33.

21.2.2 PEND-02 — Testes E2E de autorização regional e troca de contexto

> **Contexto e Débito Atual**
>
> A ADR-80 introduziu o papel `gestor_regional`, com permissão de leitura consolidada no Panorama da Diretoria (ADR-81) e Relatório da Regional (ADR-82). A ADR-85 padronizou o componente `<EscopoDaEscola>` para forçar a remontagem de estado em trocas de escola ativa.

Ações necessárias:

1. Criar suíte em `scripts/e2e/` com Playwright simulando login de `gestor_regional`.

2. Testar a tentativa de acesso a uma escola pertencente à sua regional (esperado: 200 OK) e a uma escola fora da sua regional (esperado: 403 Forbidden).
3. Testar a troca de escola na UI do Hub verificando o cancelamento de requisições pendentes e a purga dos seletores de turmas da escola anterior.

21.2.3 PEND-03 — Monitoramento de quotas de IA e telemetria de custos

> **Contexto e Débito Atual**
>
> O consumo de tokens de inteligência artificial é rigidamente controlado via `QuotaService` (**ADR-24** e **ADR-74**). Um plano de ensino na app-92 ou uma sessão iterativa na exp02em1b consomem centenas de milhares de tokens por execução.

Ações necessárias:

1. Configurar métrica baseada em logs no Google Cloud Logging para monitorar a frequência de erros `429 Quota Exceeded`.
2. Estabelecer política de alerta no Cloud Monitoring disparando notificação quando uma escola atingir 80% da cota diária antes do meio-dia.
3. Avaliar agregação semanal em tabela analítica para previsão orçamentária por contrato.

21.2.4 PEND-04 — Portão de validação de integridade do OpenAPI no CI

> **Contexto e Débito Atual**
>
> O arquivo `docs/openapi.json` reflete a especificação pública da API derivada diretamente dos esquemas Zod dos controladores. Caso um desenvolvedor adicione rotas ou modifique esquemas sem rodar `pnpm run api:openapi`, a especificação fica defasada em relação ao código real.

Ações necessárias:

1. Adicionar ao `.github/workflows/ci.yml` um passo que execute o script de geração e compare o diff contra o repositório (`git diff --exit-code docs/openapi.json`).
2. Bloquear o merge caso haja alteração de contrato não registrada no JSON.

21.2.5 PEND-05 — Verificação estrutural de Row-Level Security em migrações

> **Contexto e Débito Atual**
>
> A regra de ouro da arquitetura Unooria estabelece que toda tabela com dados de estudante e escola deve ter `escola_id NOT NULL` e RLS forçada (`FORCE ROW LEVEL SECURITY`). Atualmente, essa garantia depende do cuidado manual do desenvolvedor ao escrever a migração SQL do Prisma.

Ações necessárias:

1. Implementar teste estrutural automatizado em Jest que consulte o catálogo do PostgreSQL (`pg_tables` e `pg_policies`) em ambiente de teste local.
2. Falhar o teste caso qualquer tabela criada fora da lista de isenções explícitas careça de política de isolamento por `escola_id`.

21.3 Cronograma de mitigação e alinhamento com lançamento

O plano de resolução das pendências acima respeita a esteira de entrega do projeto, cuja entrada em operação assistida com redes piloto está planejada para o último trimestre do ciclo de engenharia:

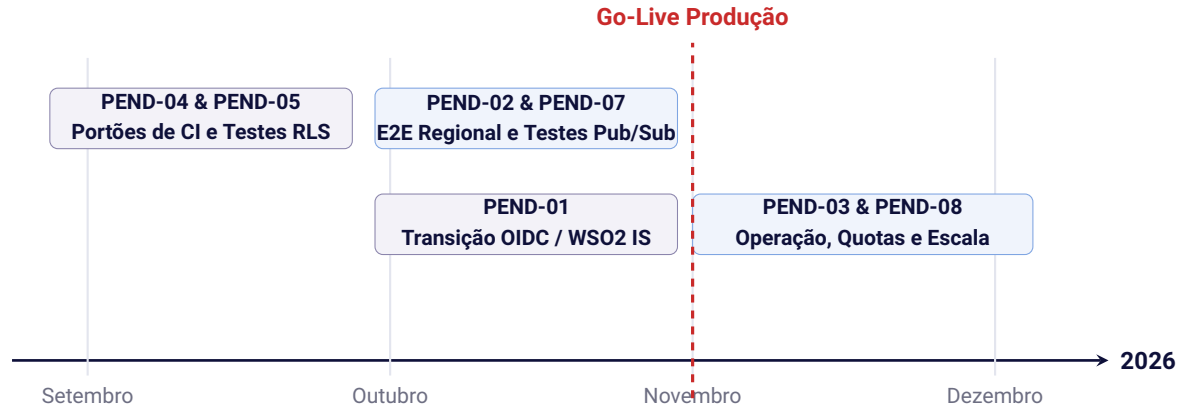


Figura 21.1 – Cronograma de resolução das pendências identificadas até a entrada em produção.

> A consolidação contínua destas pendências garante que os débitos técnicos sejam resolvidos preventivamente, assegurando que o monorepo da Plataforma Unooria chegue à produção com aderência estrita às regras de segurança, privacidade e desempenho.

Apêndice A

Fontes Mermaid dos diagramas

Todo diagrama deste documento é vetorial, desenhado nativamente em TikZ — o que garante tipografia única, nitidez em qualquer *zoom* e ausência de dependência externa de renderização. Este apêndice traz a fonte **Mermaid equivalente** de cada figura, para quem precisa editar, revisar ou versionar o diagrama fora do LaTeX (num *pull request*, num Storybook, num quadro).

> **REGRA INVIOLÁVEL**
>
> **Ao acrescentar um diagrama TikZ, inclua aqui a fonte Mermaid equivalente**, referenciando o mesmo número de figura. Diagrama sem par Mermaid é diagrama que só uma pessoa consegue alterar.

> **NOTA**
>
> As fontes abaixo usam `C4Context`, `C4Container` e `C4Component` onde o diagrama é C4, `sequenceDiagram` onde é sequência, `erDiagram` onde é entidade-relacionamento, `stateDiagram-v2` onde é máquina de estados, `classDiagram` onde é de classes (e também no perfil e na taxonomia da UML), `gantt` no diagrama de tempo e `flowchart` nos demais. Não são idênticas ao TikZ: o Mermaid não expressa posicionamento fino nem notas ancoradas, e as notas longas dos diagramas TikZ aparecem aqui como `note` ou comentário. Onde o Mermaid não tem o tipo da UML (objetos, pacotes, componentes, estrutura composta, perfil, comunicação, tempo, visão geral de interação e implantação), a primeira linha da fonte é um comentário que diz a aproximação e o que ela perde.

A.1 Figura 1.1 — Visão geral da plataforma

```
C4Container
  title Visao geral da Plataforma Unooria

  Person(estudante, "Estudante", "1o EF ao 3o EM")

  Container_Boundary(plat, "Plataforma Unooria") {
    Container(hub, "Hub", "Next.js", "Autentica, roteia por papel, embeda")
    Container(exp, "Experiencia", "Vite estatico + CDN", "Roda no iframe do navegador")
    Container(api, "API", "NestJS", "Regra, gabarito, tenancy")
```

```
    ContainerDb(pg, "PostgreSQL", "Cloud SQL", "RLS por escola")
    ContainerQueue(ps, "Pub/Sub", "", "Telemetria analitica")
  }

  System_Ext(idp, "WSO2 Identity Server", "Identidade, futuro")
  System_Ext(vertex, "Vertex AI", "Gemini")

  Rel(estudante, hub, "HTTPS")
  BiRel(hub, exp, "postMessage v1", "token, contexto")
  Rel_L(hub, idp, "OIDC")
  Rel(hub, api, "BFF")
  Rel(exp, api, "SDK: /telemetry, /ai, dominio")
  BiRel(api, pg, "SQL + RLS")
  Rel(api, ps, "publica")
  Rel(ps, api, "push POST /telemetry/ingest")
  Rel(api, vertex, "quota por perfil")
```

A.2 Figura 4.1 — Casos de uso

```
flowchart LR
  estudante([Estudante]); prof([Professor]); gestor([Gestor de escola]); admin([Gestor do contrato])
  idp[[WSO2 IS]]; vertex[[Vertex AI]]; pubsub[[Pub/Sub]]; ci[[CI/CD]]

  subgraph SIS["Plataforma Unooria"]
    UC01(("UC-01 Autenticar-se"))
    UC02(("UC-02 Escolher missao no hub"))
    UC03(("UC-03 Realizar experiencia"))
    UC04(("UC-04 Registrar telemetria"))
    UC05(("UC-05 Renovar token da sessao"))
    UC06(("UC-06 Ver o proprio resultado"))
    UC10(("UC-10 Preparar atividade"))
    UC11(("UC-11 Gerar conteudo com IA"))
    UC12(("UC-12 Conduzir o momento coletivo"))
    UC13(("UC-13 Ler resultados da turma"))
    UC14(("UC-14 Gerar plano de aula"))
    UC15(("UC-15 Check-in de bem-estar"))
    UC16(("UC-16 Zerar dados da turma"))
    UC20(("UC-20 Importar usuarios por CSV"))
    UC21(("UC-21 Editar pessoa e redefinir senha"))
    UC22(("UC-22 Ler agregado da rede"))
    UC23(("UC-23 Publicar manifesto"))
    UC24(("UC-24 Ativar ou reverter versao"))
  end

  estudante --- UC01 & UC02 & UC03 & UC06
  prof  --- UC01 & UC10 & UC12 & UC13 & UC14 & UC15 & UC16
  gestor --- UC20 & UC21
  admin --- UC22 & UC23

  UC03 -.->|include| UC04
  UC03 -.->|include| UC05
  UC10 -.->|include| UC11
  UC23 -.->|extend| UC24

  UC01 --- idp
  UC11 --- vertex
```

```
  UC14 --- vertex
  UC15 --- vertex
  UC04 --- pubsub
  UC23 --- ci
```

A.3 Figura 5.1 — C4 nível 1: contexto

```
C4Context
  title C4 nivel 1 - Contexto do sistema

  Person(estudante, "Estudante", "1o EF ao 3o EM")
  Person(prof, "Professor", "Conduz a aula")
  Person(gestor, "Gestor escolar", "Mantem usuarios")
  Person(admin, "Admin do contrato", "Rede")

  System(sis, "Plataforma Unooria", "Hub multi-tenant que autentica por papel, embeda experiencias imersivas versionadas, corrige e pontua no servidor, coleta telemetria e produz leitura de turma - com isolamento logico por escola")

  System_Ext(idp, "WSO2 Identity Server", "Identidade central, varios contratos")
  System_Ext(vertex, "Vertex AI - Gemini", "Texto e imagem generativos")
  System_Ext(ci, "GitHub Actions", "Publica bundles e manifestos")
  System_Ext(tf, "Terraform (repo separado)", "Provisiona a stack do contrato")

  Rel(estudante, sis, "realiza missoes")
  Rel(prof, sis, "prepara, conduz, le")
  Rel(gestor, sis, "administra")
  Rel(admin, sis, "le o agregado")
  Rel(idp, sis, "OIDC")
  BiRel(sis, vertex, "HTTPS + ADC")
  Rel(ci, sis, "publica")
  Rel(tf, sis, "provisiona")
```

A.4 Figura 5.2 — C4 nível 2: contêineres

```
C4Container
  title C4 nivel 2 - Conteineres (stack de um contrato)

  Person(estudante, "Estudante")
  Person(prof, "Professor")
  Person(adm, "Gestor / Admin")

  Container_Boundary(stack, "Plataforma Unooria - stack de um contrato") {
    Container(hub, "Hub", "Next.js App Router", "Sessao, roteamento por papel, chrome institucional, montagem do embed, BFF, JWKS e assinatura de token")
    Container(exp, "Experiencia", "Vite + React, ESTATICO", "Uma por missao. Roda no navegador; nunca e um processo do servidor")
    Component(sdk, "@edu/sdk", "TypeScript", "Ponte, telemetria, API, progresso")
    Container(api, "API", "NestJS + Prisma", "Autorizacao por papel, tenancy, regra de dominio, gabarito, correcao, gateway de IA, ingestao de telemetria")
```

```
    ContainerDb(pg, "PostgreSQL", "Cloud SQL HA + PITR", "Unico banco, unico schema; RLS por escola")
    ContainerQueue(ps, "Pub/Sub", "", "Canal analitico de telemetria")
    Container(buckets, "Object Storage", "Cloud Storage", "bundles, assets publicos, planos privados ")
    Container(engine, "Agente do professor", "Python + ADK, Vertex AI Agent Engine", "Sessao, ferramentas, guarda de escopo e titulo; so a API o chama (ADR-126)")
    ContainerDb(ds, "Vertex AI Search", "dois data stores, global", "habilidades (estruturado) e acervo (PDFs) - ADR-127")
  }

  System_Ext(vertex, "Vertex AI")
  Container_Ext(idp, "WSO2 IS", "", "Externo a stack")

  Rel(estudante, hub, "HTTPS")
  Rel(prof, hub, "HTTPS")
  Rel(adm, hub, "HTTPS")
  BiRel(hub, exp, "postMessage v1 - targetOrigin explicito")
  Rel(exp, sdk, "usa")
  Rel(sdk, api, "HTTPS + Bearer")
  Rel(hub, api, "BFF")
  BiRel(api, pg, "SQL + RLS")
  Rel(api, ps, "publica")
  Rel(ps, api, "push /telemetry/ingest")
  Rel(api, buckets, "assets, planos")
  Rel(api, vertex, "HTTPS + ADC")
  Rel(api, engine, ":query / :streamQuery, ADC")
  Rel(engine, vertex, "Gemini")
  Rel(engine, ds, "busca")
  Rel(hub, idp, "OIDC")
  Rel(buckets, exp, "CDN serve o bundle")
```

A.5 Figura 5.3 — C4 nível 3: componentes da API

```
C4Component
  title C4 nivel 3 - Componentes da API (NestJS)

  Container_Boundary(api, "API") {
    %% --- transversal: roda em toda rota
    Component(guard,  "OidcGuard", "", "Valida JWT por JWKS")
    Component(roles,  "RolesGuard", "", "Restringe por papel")
    Component(tenint, "TenancyInterceptor", "", "Preenche escola/perfil no AsyncLocalStorage aberto pelo middleware")
    Component(env,    "validarEnv", "Zod", "Valida env no boot")
    Component(obs,    "observabilidade/", "pino + OpenTelemetry", "Contexto da requisicao ( middleware), logger com redaction, filtro global, sampler por rota; otel.ts e a primeira importacao do main.ts")

    %% --- infraestrutura: as portas unicas
    Component(prisma, "PrismaService", "Prisma", "comTenancy: transacao + RLS")
    Component(ai,     "ai/", "", "AiProvider, QuotaService")
    Component(store,  "object-storage/", "", "local | GCS")
    Component(assets, "assets/", "", "Imagem gerada")

    %% --- plataforma: serve a todas as experiencias
    Component(auth,   "auth/", "", "TokenService")
```

```
    Component(usu,    "usuarios/", "", "CSV, senha")
    Component(expmod, "experiencias/", "", "Manifesto")
    Component(tel,    "telemetry/", "", "batch, beacon, ingest")
    Component(prog,   "progress/", "", "Progresso generico")
    Component(health, "health/", "", "healthz, readyz")
    Component(analise,"analise/", "", "Turma e rede, CSV")
    Component(comum,  "comum/", "", "Series, rate limit, guardas de liberacao e disponibilidade")
    Component(rost,   "rostering/", "", "Pessoas, escolas, turmas")
    Component(port,   "portabilidade/", "", "Perfil portatil")
    Component(salas,  "salas/", "", "A sala: grade e liberacao por turma (ADR-69)")
    Component(equipes,"equipes/", "", "Quem divide um computador na missao (ADR-111); consulta.ts e a porta dos paineis")
    Component(recs,   "recursos/", "", "O interruptor da rede e a versao (ADR-87); escrita so admin")
    Component(prosp,  "prospeccao/", "", "Cadastro de interesse do site publico, sem tenant (ADR-107)")
    Component(evid,   "evidencias/", "", "Habilidade da BNCC por atividade e rodada (ADR-112); a porta marcar das missoes, que a exp05ef1b chama")

    %% --- dominio por experiencia (nenhum conhece outro)
    Component(motor,  "motor/ geracao/ dossie/ atividades/ perfis/", "", "exp08ef1b")
    Component(utube,  "unootube/", "", "exp04ef1b - painel conta por computador")
    Component(vila,   "vila/", "", "app-70")
    Component(planos, "planos/", "", "app-90")
    Component(bem,    "bemestar/", "", "app-91")
    Component(ugpt,   "unoogpt/", "", "exp02em1b - motor probabilistico, agente com ferramentas, relogio da turma; painel conta por computador")
    Component(outros, "rastro-boato/ manifesto-ia/ quem-mandou-isso/ foto-conta/ planejamento/", "", "exp06ef2b, exp08ef4b, exp05ef1b, exp01ef1b, app-92 - sem seta na figura")

    %% --- plataforma, fora da figura TikZ (2026-09-30, ADR-126)
    Component(agente, "agente-professor/", "", "Agente de IA do professor: posse da conversa, SSE, conferencia de codigos; @Produto('agente-professor')")
    Component(porta, "AgenteProfessorPort", "ai/agent-engine.client.ts", "A segunda porta de ai/: so ela conhece o Vertex AI Agent Engine")
  }

  Rel(guard, roles, "antes de")
  Rel(tenint, prisma, "escola do token")
  Rel(salas, equipes, "contagem na sala")
  Rel(salas, prisma, "comTenancy")
  Rel(equipes, prisma, "comTenancy")
  Rel(utube, equipes, "consulta.ts")
  Rel(ugpt, equipes, "consulta.ts")
  Rel(evid, equipes, "gancho aoMudarEquipes + consulta.ts")
  Rel(evid, prisma, "comTenancy")
  Rel(motor, prisma, "comTenancy")
  Rel(vila, prisma, "comTenancy")
  Rel(ugpt, prisma, "comTenancy")
  Rel(ugpt, ai, "IA com ferramentas + quota")
  Rel(planos, ai, "IA + quota")
  Rel(bem, ai, "IA + quota")
  Rel(planos, store, "bucket privado")
  %% quem-mandou-isso/ (exp05ef1b, ADR-118) mora na caixa "outros" da figura, sem
  %% seta; as relacoes dela ficam aqui: a primeira missao que chama marcar.
  Rel(outros, evid, "exp05ef1b: marcar na transacao do Mural + registrarContexto")
  Rel(outros, equipes, "exp05ef1b: consulta.ts (painel por computador)")
  Rel(outros, ai, "exp05ef1b: IA + quota no preparo")
  Rel(agente, prisma, "comTenancy: agente_professor_conversas")
  Rel(agente, ai, "QuotaService: temSaldo antes, consumir no finally")
```

```
  Rel(agente, porta, "criar/ler/apagar sessao, enviar, renovar")

  %% A figura TikZ desenha a infraestrutura ENTRE a plataforma e o dominio, com
  %% a coluna 2 vazia: e por ela que unootube/ e unoogpt/ sobem ate equipes/.
  %% evidencias/ chega a equipes/ pelo vao entre a 2a e a 3a linha da
  %% plataforma, por cima de recursos/ e prospeccao/.
```

A.6 Figura 5.4 — C4 nível 3: componentes do hub

```
C4Component
  title C4 nivel 3 - Componentes do Hub (Next.js)

  Container_Boundary(hub, "Hub") {
    Component(mw,      "middleware.ts", "", "Sessao + area do papel")
    Component(papeis,  "lib/papeis.ts", "", "edge-safe: papel -> area")
    Component(authlib, "lib/auth.ts", "NextAuth", "Sessao")
    Component(jwks,    "/.well-known/jwks.json", "", "Chave publica")

    Component(embed,  "lib/embed-experiencia.ts", "", "Token + manifesto + origem, UMA vez")
    Component(chave,  "lib/chave-token.ts", "", "Assina RS256, TTL 15 min")
    Component(devsrv, "lib/experiencias-dev.ts", "", "Porta do dev server")
    Component(apiserv,"lib/api-servidor.ts", "", "BFF")

    Component(casca,  "CascaPlataforma", "", "CSS do DS + fontes")
    Component(header, "HeaderPlataforma", "", "Um so para todos os papeis")
    Component(disp,   "DispositivoFrame", "", "Casca de aparelho")
    Component(frame,  "ExpFrame", "", "Sandbox + protocolo v1")

    Component(landing, "components/landing", "", "Rota / sem sessao: landing publica (ADR-40)")
  }

  Rel(mw, papeis, "usa")
  Rel(landing, casca, "usa a casca do DS")
  Rel(embed, chave, "assina")
  Rel(embed, devsrv, "porta em dev")
  Rel(disp, frame, "envolve")
```

A.7 Figura 5.5 — C4 nível 4: tipos que carregam a regra

```
classDiagram
  class connectToHub {
    +hubOrigin: string
    +timeoutMs = 8000
    +READY_RETRY_MS = 400
    valida ev.origin, senao descarta
    reenvia exp:ready ate responder
    compara protocolVersion MAIOR
    valida pos-init com Zod
  }

  class SdkTypes {
```

```
    +PROTOCOL_VERSION = 2
    +HubInitSchema
    +HubParaExpSchema
    +ExpParaHubSchema
    unioes discriminadas por type
  }

  class PrismaService {
    +comTenancy(fn, escolaId?, opts?)
    abre $transaction
    set_config('app.escola_id', ..., true)
    sem escola: LANCA
  }

  class TenancyContext {
    +AsyncLocalStorage~string~
    +comEscolaNoContexto(id, fn)
    +escolaDoContexto(): string?
  }

  class LinhaDeLeitura {
    <<union>>
    scored:      numero: number
    indicative:  numero: null
    zero:        numero: null
    none:        numero: null
  }

  connectToHub ..> SdkTypes : usa
  PrismaService ..> TenancyContext : le
  note for LinhaDeLeitura "Comentario avisa; TIPO impede.\nO hub nunca recebe numero onde nao ha
↪ medida."
```

A.8 Figura 5.6 — Grafo de dependências do *monorepo*

```
flowchart TB
  ds[["@unooria/design-system (npm publico)"]]
  sdk["packages/sdk"]
  ui["packages/ui"]
  hub["apps/hub"]; api["apps/api"]
  e01["exp08ef1b"]; e02["exp04ef1b"]; e70["app-70"]; e90["app-90"]; e91["app-91"]
  tpl["_template"]

  ds -->|versao publicada| ui
  sdk --> hub & e01 & e02 & e70 & e90 & e91 & tpl
  %% apps/api NAO importa pacote do workspace: valida o que recebe com os proprios schemas
  ui --> hub & e01 & e02 & e90 & e91
  %% app-70 NAO depende de packages/ui: canvas 3D, sem DOM de UI (ADR-18)
```

A.9 Figura 5.7 — Curadoria de um bloco de *feed*

```
sequenceDiagram
  autonumber
  participant E as Experiencia (exp08ef1b no iframe)
  participant C as runs.controller (NestJS)
  participant S as motor/servico
  participant P as PostgreSQL

  E->>C: POST /runs/:id/eventos (dwell, curtidas)
  C->>S: aplicarEventos
  S->>P: comTenancy: set_config('app.escola_id')
  S->>P: pg_advisory_xact_lock(runId)
  Note over S,P: Sem o lock, dois pedidos simultaneos de bloco<br/>se sobrescrevem e um bloco inteiro desaparece.<br/>Transacao com timeout de 20s.
  P-->>S: estado da run
  S->>S: curarBloco (puro e deterministico)
  S->>P: UPDATE engine_state + posts entregues
  P-->>S: COMMIT (libera o lock)
  S-->>C: bloco + modulos + diagnostico
  C->>C: paraFeedPost: remove lado, perfil, intensidade
  Note over C: Esses campos sao a variavel independente<br/>do experimento. No cliente, o estudante saberia<br/>de que lado o algoritmo o colocou.
  C-->>E: GET /runs/:id/feed (sanitizado)
```

A.10 Figura 5.8 — Estados de uma geração assíncrona

```
stateDiagram-v2
  [*] --> pendente : POST (202)
  pendente --> gerando : claim atomico (updateMany por status)
  gerando --> pronto : gravou o artefato
  gerando --> erro : falha do provider OU finishReason=MAX_TOKENS
  gerando --> pendente : orfa (>12 min sem worker), ate 2 retomadas
  pronto --> [*]

  note right of pendente
    O claim atomico substitui uma fila externa:
    duas instancias podem tentar, so uma ganha a linha.
    Menor peca possivel que resolve o problema (ADR-17).
  end note
```

A.11 Figura 5.9 — Os catorze tipos de diagrama da UML 2.5.1

```
classDiagram
  class Diagrama {
    <<abstract>>
  }
  class Estrutura["Diagrama de estrutura"] {
    <<abstract>>
  }
  class Comportamento["Diagrama de comportamento"] {
    <<abstract>>
  }
```

```
  class Interacao["Diagrama de interacao"] {
    <<abstract>>
  }
  Diagrama <|-- Estrutura
  Diagrama <|-- Comportamento

  %% estrutura: sete folhas
  class Classes {
    fig:uml-classes-dominio, fig:uml-classes-ia
    antes: fig:c4l4, fig:graus
  }
  class Objetos {
    fig:uml-objetos
  }
  class Pacotes {
    fig:uml-pacotes
    antes: fig:monorepo (grafo)
  }
  class Componentes {
    fig:uml-componentes
  }
  class EstruturaComposta["Estrutura composta"] {
    fig:uml-estrutura
  }
  class Implantacao["Implantacao"] {
    fig:uml-implantacao
    antes: fig:deploy-contrato (infra)
  }
  class Perfil {
    fig:uml-perfil
  }
  Estrutura <|-- Classes
  Estrutura <|-- Objetos
  Estrutura <|-- Pacotes
  Estrutura <|-- Componentes
  Estrutura <|-- EstruturaComposta
  Estrutura <|-- Implantacao
  Estrutura <|-- Perfil

  %% comportamento: tres folhas e o ramo abstrato da interacao
  class CasosDeUso["Casos de uso"] {
    fig:uml-casos-gestao
    antes: fig:casos-de-uso
  }
  class Atividades {
    fig:uml-atividade-disp
    antes: fig:fluxo-embed, fig:guardrails-bemestar, fig:app92-caminho
  }
  class MaquinaDeEstados["Maquina de estados"] {
    fig:uml-estados-ponte
    antes: fig:estados-geracao
  }
  Comportamento <|-- CasosDeUso
  Comportamento <|-- Atividades
  Comportamento <|-- MaquinaDeEstados
  Comportamento <|-- Interacao

  class Sequencia["Sequencia"] {
    fig:uml-seq-entrada, fig:uml-seq-ia
    antes: fig:seq-feed, fig:seq-handshake, fig:seq-erro-cliente
```

```
  }
  class Comunicacao["Comunicacao"] {
    fig:uml-comunicacao
  }
  class Tempo {
    fig:uml-tempo
  }
  class VisaoGeral["Visao geral de interacao"] {
    fig:uml-visao-geral
  }
  Interacao <|-- Sequencia
  Interacao <|-- Comunicacao
  Interacao <|-- Tempo
  Interacao <|-- VisaoGeral

  note "Como ler: abstract e classificador abstrato - nao existe diagrama<br/>'de estrutura' que nao seja de um tipo concreto. 'antes' aponta para<br/>a figura do mesmo tipo que o documento ja tinha ; (grafo) e (infra)<br/>marcam as que cumprem o papel com outra notacao."
```

A.12 Figura 5.10 — Perfil UML da plataforma

```
%% aproximacao: o Mermaid nao tem diagrama de perfil; a extensao de metaclasse (seta de ponta cheia) vira associacao rotulada "estende", e as restricoes entre chaves vao para notas.
classDiagram
  %% [x] no corpo da classe = {x} da figura: chave fecha o corpo no Mermaid.
  namespace PerfilUnooria {
    class TabelaDeTenant {
      <<estereotipo>>
      prefixo: String [0..1]
      codigo da experiencia, vazio na plataforma
    }
    class Experiencia {
      <<estereotipo>>
      id: String, o expId
      versao: SemVer
      produto: Produto
      publico: Publico = estudante
      permissoes: String [*]
      viewport: String [0..1]
    }
    class Bundle {
      <<estereotipo>>
      caminho = gs://[contrato]-exp/[expId]/[versao]/
    }
    class RotaDeEstudante {
      <<estereotipo>>
      EXP_ID: String
    }
    class ChamadaDeIA {
      <<estereotipo>>
      perfil: estudante | professor
    }
    class Produto {
      <<enumeracao>>
      entre-redes
      plano-de-aula
```

```
      plano-de-ensino
      bem-estar
      agente-professor
    }
    class Publico {
      <<enumeracao>>
      estudante
      professor
    }
  }

  %% metaclasses da UML, fora do perfil
  class MClass["Class"] {
    <<metaclasse>>
  }
  class MComponent["Component"] {
    <<metaclasse>>
  }
  class MArtifact["Artifact"] {
    <<metaclasse>>
  }
  class MOperation["Operation"] {
    <<metaclasse>>
  }

  TabelaDeTenant --> MClass : estende
  Experiencia --> MComponent : estende
  Bundle --> MArtifact : estende
  RotaDeEstudante --> MOperation : estende
  ChamadaDeIA --> MOperation : estende
  Bundle ..> Experiencia : manifesta
  Experiencia ..> Produto

  note for TabelaDeTenant "{escola_id NOT NULL}<br/>{ENABLE e FORCE ROW LEVEL SECURITY}<br/>{toda query dentro de comTenancy}"
  note for Experiencia "{gabarito e correcao so na API}<br/>{token so em memoria}"
  note for Bundle "{imutavel: mudar o conteudo e subir a versao}<br/>{versao igual ao version do package.json}"
  note for RotaDeEstudante "{chama exigirMissaoLiberada no funil}<br/>{403 sem linha em liberacoes}"
  note for ChamadaDeIA "{QuotaService.temSaldo antes}<br/>{QuotaService.consumir depois}<br/>{provider so por AI_PROVIDER}"
  note "Como ler: um elemento marcado TabelaDeTenant em outro diagrama<br/>carrega as tres restricoes sem repeti-las. A seta cheia e extensao<br/>de metaclasse; nao confundir com a heranca, de seta aberta."
  note "Onde o perfil ainda nao e lei: geracao/ e as reacoes de motor/<br/>chamam o provider sem temSaldo - as excecoes registradas na ADR-74<br/>e em ai/AGENTS.md. Uma ChamadaDeIA nova nao herda a excecao."
```

A.13 Figura 5.11 — Casos de uso da administração da rede

```
flowchart LR
  gesc([Gestor de escola]); greg([Gestor regional]); gctr([Gestor do contrato]); gadm(["Admin (Unooria)"])
  prof([Professor]); ci[[CI/CD]]; visit([Visitante do site])

  %% generalizacao de atores (triangulo aberto na figura): o filho faz tudo o que o pai faz
```

```
  greg ==>|generalizacao| gesc
  gctr ==>|generalizacao| greg
  gadm ==>|generalizacao| gctr

  subgraph SIS["Plataforma Unooria - administracao"]
    UC20(("UC-20 Importar pessoas e enturmar"))
    UC29(("UC-29 Cadastrar pessoa no balcao"))
    UC21(("UC-21 Editar pessoa e redefinir senha"))
    UC30(("UC-30 Conferir a cadeia de atribuicao"))
    UC32(("UC-32 Trocar a escola ativa"))
    UC33(("UC-33 Ver o panorama da diretoria"))
    UC34(("UC-34 Gerar o relatorio da diretoria"))
    UC31(("UC-31 Criar papel sem escola"))
    UC35(("UC-35 Manter escolas e regionais"))
    UC22(("UC-22 Ler o agregado da rede"))
    UC36(("UC-36 Ler o historico de recursos"))
    UC37(("UC-37 Ligar ou desligar recurso"))
    UC39(("UC-39 Editar a quota de IA"))
    UC40(("UC-40 Ler cadastros de interesse"))
    UC24(("UC-24 Ativar ou reverter versao"))
    UC38(("UC-38 Fixar versao por escopo"))
    UC42(("UC-42 Liberar ou encerrar missao na turma"))
    UC43(("UC-43 Conferir a disponibilidade"))
    UC44(("UC-44 Montar as equipes da missao"))
    UC23(("UC-23 Publicar manifesto"))
    UC41(("UC-41 Cadastrar interesse"))
  end

  gesc --- UC20 & UC29 & UC21
  greg --- UC32 & UC33 & UC34
  gctr --- UC31 & UC35 & UC22 & UC36
  gadm --- UC37 & UC39 & UC40
  prof --- UC42 & UC44
  ci --- UC23
  visit --- UC41

  UC29 -.->|include| UC30
  UC29 -.->|include| UC20
  UC31 -.->|include| UC30
  UC42 -.->|include| UC43
  UC24 -.->|extend| UC37
  UC38 -.->|extend| UC37
  UC24 -.->|extend| UC23

  %% caso de uso de suporte (incluido ou extensao)
  classDef suporte stroke-dasharray: 4 3
  class UC30,UC24,UC38,UC43 suporte
```

A.14 Figura 5.12 — Classes: o modelo de domínio da plataforma

```
classDiagram
  %% [x] no corpo da classe = {x} da figura: chave fecha o corpo no Mermaid.
  %% As restricoes entre chaves de cada classe estao nas notas.
  class Escola {
    nome: String
    codigoInep: String [0..1]
```

```
    tipo: TipoDeEscola
    municipio, uf: String [0..1]
  }
  class Turma {
    nome: String
    serie: String
    tipo: regular | componente
    turno: String [0..1]
  }
  class PeriodoLetivo {
    tipo, titulo: String
    ano: Int
    inicio, fim: Date
  }
  class Lotacao {
    papel: estudante | professor | gestor_escola
    principal: Boolean
    inicio: Date
    fim: Date [0..1]
  }
  class Matricula {
    <<TabelaDeTenant>>
    papel: estudante | professor
    principal: Boolean
    inicio: Date
    fim: Date [0..1]
  }
  class Liberacao {
    <<TabelaDeTenant>>
    estado: liberada | encerrada
    liberadaEm: DateTime
    encerradaEm [0..1]
  }
  class Usuario {
    username: String
    displayName: String
    role: Papel
    ativo: Boolean
    senhaProvisoria: Boolean
    /escola: Escola [0..1]
    /turma: Turma [0..1]
  }
  class Disponibilidade {
    ligado: Boolean
    versaoFixa: String [0..1]
    motivo: String [0..1]
    definidoEm: DateTime
  }
  class Experiencia {
    expId: String
    versaoAtiva: String
    series: String [*]
    publico: Publico
    permissoes: String [*]
  }
  class Papel {
    <<enumeracao>>
    estudante
    professor
    gestor_escola
```

```
    gestor_regional
    gestor_contrato
    admin
  }
  class Recurso {
    <<abstract>>
    /recursoId: String
    produto:[slug] ou experiencia:[expId]
  }
  class Produto {
    slug: Produto
    vocabulario em recursos/catalogo.ts
  }

  Escola "1" -- "*" Turma
  Turma "*" -- "1" PeriodoLetivo
  Escola "pai 0..1" -- "filhas *" Escola
  Escola "1" -- "*" Lotacao
  Lotacao "*" -- "1" Usuario
  Turma "1" -- "*" Matricula
  Matricula "*" -- "1" Usuario
  Turma "1" -- "*" Liberacao
  Liberacao "*" -- "1" Experiencia : por expId, sem FK
  Usuario -- "0..1" Escola : regional
  Disponibilidade "*" -- "0..1" Escola : escopo
  Disponibilidade "*" -- "1" Recurso
  Recurso <|-- Produto
  Recurso <|-- Experiencia
  Produto "0..1" -- "*" Experiencia : teto
  Usuario ..> Papel

  note for Escola "{regional e Escola com tipo = regional}<br/>{o contrato nao e linha: e a stack}"
  note for Turma "{unica por escola, periodo e nome}<br/>{fora da RLS: o filtro e a tranca}"
  note for PeriodoLetivo "{do contrato, sem escola}"
  note for Lotacao "{unica por escola, usuario e papel}"
  note for Matricula "{no maximo uma principal ativa por estudante}"
  note for Liberacao "{sem linha = bloqueada}<br/>{unica por turma e expId}"
  note for Usuario "{escola obrigatoria, salvo papeis sem escola}<br/>{role = gestor_regional se e
↪ so se regional nao nula}"
  note for Disponibilidade "{sem linha = ligado}<br/>{unica por recurso e escopo}<br/>{escopo nulo =
↪  o contrato}"
```

A.15 Figura 5.13 — Classes: a porta da IA

```
classDiagram
  %% [x] no corpo da classe = {x} da figura: chave fecha o corpo no Mermaid.
  %% Linha sem parenteses cai no compartimento de atributos: as anotacoes
  %% cinza que a figura poe sob as operacoes aparecem aqui entre os atributos.
  class AiController {
    <<controller>>
    -provider: AiProvider
    -quota: QuotaService
    +gerar(corpo) [texto]
    POST /ai/gerar, so OidcGuard
  }
  class PlanosService {
```

```
    <<service>>
    e mais oito modulos de dominio
    +gerar(...)
    temSaldo -> provider -> consumir
  }
  class AiResult {
    <<tipo de dado>>
    texto: string
    tokensUsados: number
    tokensEntrada?, tokensSaida?
    finishReason?: string
    voltas?, tentativas?
  }
  class QuotaService {
    -prisma: PrismaService
    +limite(perfil) Promise~number~
    +usadoHoje(escolaId, usuarioId) Promise~number~
    +temSaldo(escolaId, usuarioId, perfil = "estudante") Promise~boolean~
    +consumir(escolaId, usuarioId, tokens) Promise~number~
  }
  class AiProvider {
    <<interface>>
    injetado pelo token AI_PROVIDER
    +generate(req: AiRequest) Promise~AiResult~
    +generateJson~T~(req) Promise~T~
    +generateImage(prompt, opcoes?) Promise~Buffer~
    +generateSpeech(req) Promise~Buffer~
    +conversar(req) Promise~AiResult~
  }
  class AiModule {
    <<modulo @Global>>
    exports: [AI_PROVIDER, QuotaService]
    +useFactory(config, metricas) AiProvider
  }
  class VertexProvider {
    -auth: GoogleAuth
    ADC, sem chave em env
    as cinco operacoes, por :generateContent
  }
  class MockProvider {
    deterministico
    generate, conversar: respondem
    os outros tres: lancam
  }
  class MedidorDeProvider {
    <<decorador>>
    -real: AiProvider
    -metricas: Metricas
    -modelos: ModelosDoMedidor
    as cinco operacoes, cada uma por medir(op, modelo, fn)
  }
  class ErroDeGeracao {
    extends Error
  }
  class ia_quotas {
    perfil PK
    tokens_por_dia
    sem escola: do contrato
  }
  class ia_uso_diario {
```

```
    <<TabelaDeTenant>>
    escola_id, usuario_id, dia PK
    tokens_usados
  }

  VertexProvider ..|> AiProvider
  MockProvider ..|> AiProvider
  MedidorDeProvider ..|> AiProvider
  MedidorDeProvider o-- "1" AiProvider : real
  MockProvider ..> ErroDeGeracao : lanca
  AiProvider ..> AiResult : usa
  AiController ..> QuotaService : injeta
  AiController ..> AiProvider
  PlanosService ..> AiProvider : @Inject(AI_PROVIDER)
  PlanosService ..> QuotaService
  AiModule ..> MedidorDeProvider : cria
  QuotaService ..> ia_quotas : le
  QuotaService ..> ia_uso_diario : upsert

  note for AiModule "A fabrica, no codigo: new MedidorDeProvider(env === 'mock'<br/>? new
↪ MockProvider() : new VertexProvider(), metricas, modelos).<br/>Trocar de fornecedor e um arquivo
↪ novo em src/ai e uma linha aqui."
  note for QuotaService "O que o tipo nao garante: nenhum consumidor e obrigado a chamar<br/>
↪ temSaldo. Entre o temSaldo e o consumir nao ha trava, entao duas<br/>chamadas simultaneas do mesmo
↪  usuario passam as duas e podem<br/>ultrapassar o teto em ate uma resposta cada."
```

A.16 Figura 5.14 — Objetos: um instantâneo da disponibilidade

```
%% aproximacao: o Mermaid nao tem diagrama de objetos; cada instancia vira um no com os valores dos
↪ slots, o nome sublinhado vira a tag u do HTML, e cada link vira aresta rotulada com o papel.
flowchart TB
  norte["<u>norte : Escola</u><br/>tipo = 'regional'<br/>nome = 'Regional Norte'"]
  ef["<u>ef : Escola</u><br/>tipo = 'escola'<br/>nome = 'EE Cora Coralina'"]
  ee["<u>ee : Escola</u><br/>tipo = 'escola'<br/>nome = 'EE Monteiro Lobato'"]

  d3["<u>d3 : Disponibilidade</u><br/>recursoId =<br/>'experiencia:exp06ef2b'<br/>ligado = false<br/>
↪ motivo = 'adiar'"]
  d4["<u>d4 : Disponibilidade</u><br/>recursoId =<br/>'produto:plano-de-aula'<br/>ligado = false<br/>
↪ motivo = 'nao contratado'"]
  d1["<u>d1 : Disponibilidade</u><br/>recursoId =<br/>'experiencia:exp06ef2b'<br/>escola = nulo (
↪ contrato)<br/>ligado = false"]
  d2["<u>d2 : Disponibilidade</u><br/>recursoId =<br/>'experiencia:exp06ef2b'<br/>ligado = true<br/>
↪ motivo = 'piloto'"]

  t6a["<u>t6a : Turma</u><br/>nome = '6A'<br/>serie = '6EF'"]
  paula["<u>aula : Produto</u><br/>slug = 'plano-de-aula'"]
  e1["<u>e1 : Experiencia</u><br/>expId = 'exp06ef2b'<br/>produto = 'entre-redes'<br/>series = ['6EF
↪ ']"]
  l1["<u>l1 : Liberacao</u><br/>estado = 'liberada'"]
  e2["<u>e2 : Experiencia</u><br/>expId =<br/>'app-90-plano-de-aula'<br/>publico = 'professor'"]

  ef ---|pai| norte
  ee ---|pai| norte
  d3 ---|escopo| ef
  d4 ---|escopo| ee
  d2 ---|escopo| norte
```

```
  t6a ---|escola| ee
  l1 ---|turma| t6a
  l1 ---|expId| e1
  d1 ---|recurso| e1
  d2 ---|recurso| e1
  d3 ---|recurso| e1
  d4 ---|recurso| paula
  e2 ---|"produto (teto)"| paula

  %% d1 nao tem link de escopo: escola nula e o contrato inteiro.
```

A.17 Figura 5.15 — Pacotes: quem pode importar quem

```
%% aproximacao: o Mermaid nao tem diagrama de pacotes; cada pacote vira subgraph, e a dependencia de codigo (import/use) vira seta tracejada rotulada.
flowchart TB
  subgraph MONO["monorepo - plataforma-unooria"]
    subgraph APPS["apps"]
      subgraph HUB["hub"]
        hub["@edu/hub<br/>Next.js, App Router<br/>rotas por papel, BFF, JWKS,<br/>montagem do embed"]
      end
      subgraph EXPS["experiencias"]
        exps["_template<br/>exp01ef1b ... exp08ef4b<br/>app-70, app-90,<br/>app-91, app-92<br/>13 bundles Vite; backup-*<br/>fora do deploy"]
      end
      subgraph API["api - @edu/api - NestJS + Prisma"]
        subgraph LTR["transversal"]
          ltr["auth/ tenancy/ observabilidade/ config/"]
        end
        subgraph LPL["plataforma"]
          lpl["usuarios/ rostering/ salas/ equipes/ evidencias/ recursos/<br/>experiencias/ telemetry / analise/ progress/ comum/ ..."]
        end
        subgraph LIN["infraestrutura"]
          lin["prisma/ ai/ object-storage/ assets/"]
        end
        subgraph LDO["dominio {nenhum modulo importa outro}"]
          ldo["motor/ geracao/ vila/ unoogpt/ quem-mandou-isso/ a-selecao/ ..."]
        end
      end
    end
    subgraph PKGS["packages"]
      subgraph SDK["sdk"]
        sdk["@edu/sdk<br/>TypeScript + Zod<br/>ponte postMessage v2, cliente<br/>HTTP, telemetria em lote"]
      end
      subgraph UI["ui"]
        ui["@edu/ui<br/>as cinco pontes do DS:<br/>'use client', nomes sem<br/>colisao, tree-shaking"]
      end
    end
    subgraph DS["externo - npm"]
      ds["@unooria/design-system<br/>repo storybook-unooria<br/>componentes + CSS de secao"]
    end
  end
```

```
38
39    %% dentro da api: toda faixa usa a transversal
40    ldo -.->|import| lpl
41    ldo -.->|import| lin
42    lpl -.->|import| lin
43    lpl -.->|"import - toda faixa usa a transversal"| ltr
44
45    %% entre pacotes do workspace
46    exps -.->|import| sdk
47    exps -.->|import componentes| ui
48    exps -.->|use: so CSS da secao e fontes| ds
49    hub -.->|import| sdk
50    hub -.->|import componentes| ui
51    hub -.->|use: so CSS da secao e fontes| ds
52    ui -.->|import| ds
53
54    %% A api NAO importa nada do workspace (nem @edu/sdk): o contrato da ponte e
55    %% duplicado de proposito, com Zod de cada lado. Hub e api nao se importam:
56    %% falam por HTTP, pelo BFF. A app-70 importa so a SDK (jogo em canvas, sem
57    %% DOM de interface). No dominio, duas experiencias conversam por tabela de
58    %% contrato ou por endpoint de fronteira, nunca por import.
```

A.18 Figura 5.16 — Componentes e interfaces

```
1  %% aproximacao: o Mermaid nao tem diagrama de componentes; cada interface vira um no circular, a linha
   ↪ cheia sai de quem a FORNECE (a bola) e a seta tracejada sai de quem a REQUER (o soquete).
2  flowchart TB
3    hub["componente<br/>Hub<br/>[Next.js]<br/>sessao, papel, embed,<br/>assinatura do token"]
4    subgraph EXP["componente - Experiencia [Vite + React, bundle]"]
5      sdk["biblioteca<br/>@edu/sdk - ponte, HTTP, telemetria"]
6    end
7    api["componente<br/>API<br/>[NestJS + Prisma]<br/>autorizacao por papel, tenancy, regra de dominio,
   ↪  gabarito, porta da IA"]
8    pg["externo<br/>PostgreSQL<br/>[Cloud SQL]"]
9    ps["externo<br/>Pub/Sub"]
10   vx["externo<br/>Vertex AI"]
11   gcs["externo<br/>Cloud Storage"]
12
13   %% hub e experiencia: a ponte (o hub fornece, a experiencia requer)
14   iPonte(("Ponte postMessage v2<br/>hub:init, hub:token,<br/>hub:pause/resume"))
15   hub --- iPonte
16   EXP -.-> iPonte
17
18   %% hub e api
19   iLogin(("POST /auth/login<br/>Credentials"))
20   iBff(("REST pelo BFF<br/>/api/plataforma/*"))
21   iJwks(("JWKS<br/>/.well-known/jwks.json"))
22   api --- iLogin
23   hub -.-> iLogin
24   api --- iBff
25   hub -.-> iBff
26   hub --- iJwks
27   api -.-> iJwks
28
29   %% experiencia (pela sdk) e api
30   iRest(("REST com Bearer<br/>rotas de dominio"))
```

```
  iTel(("Telemetria<br/>/telemetry/batch<br/>/telemetry/beacon"))
  api --- iRest
  sdk -.-> iRest
  api --- iTel
  sdk -.-> iTel

  %% api e externos
  iSql(("SQL + RLS<br/>comTenancy"))
  iPub(("Publicacao<br/>{contrato}-telemetria"))
  iIngest(("Ingestao push<br/>/telemetry/ingest"))
  iGen(("Geracao<br/>ADC, :generateContent"))
  iObj(("Objetos<br/>assets, planos"))
  pg --- iSql
  api -.-> iSql
  ps --- iPub
  api -.-> iPub
  api --- iIngest
  ps -.-> iIngest
  vx --- iGen
  api -.-> iGen
  gcs --- iObj
  api -.-> iObj

  %% O Cloud Storage tambem serve o bundle ao navegador pela CDN: e artefato,
  %% nao interface, e esta no diagrama de implantacao (fig:uml-implantacao).
```

A.19 Figura 5.17 — Estrutura composta: a página da missão

```
%% aproximacao: o Mermaid nao tem diagrama de estrutura composta; as partes viram subgraphs aninhados e nos, as portas viram nos pequenos dentro da parte a que pertencem, e a fronteira do sandbox e o subgraph do iframe.
flowchart LR
  page["server component<br/>app/exp/[expId]/page.tsx<br/>montarEmbed(): token,<br/>src, expOrigin,<br/>permissoes, viewport"]
  cdn["CDN<br/>{cdn}/{expId}/{versao}/<br/>index.html?hubOrigin=..."]
  rprog["BFF<br/>POST /api/plataforma/<br/>progresso/:expId/<br/>concluir"]
  rtok["route handler<br/>GET /api/exp-token<br/>401/403 -> hub:pause<br/>e o aviso"]
  rlog["route handler<br/>POST /api/ponte<br/>uma linha de log; 204"]

  subgraph EXPHOST["ExpHost - client component"]
    pEmbed["porta"]
    pProg["porta"]
    subgraph DISP["dispositivo : DispositivoFrame [0..1] - so quando o manifesto declara viewport"]
      subgraph FRAME["frame : ExpFrame [1]"]
        pTok["porta"]
        pLog["porta"]
        proto["ponte : protocolo v2<br/>postMessage;<br/>ev.origin + Zod"]
        aviso["aviso : AlertBox [0..1]<br/>'Sua sessao expirou'; substitui o iframe"]
        subgraph IFRAME["iframe [1] - fronteira do sandbox"]
          pSrc["porta src"]
          pPonte["porta ponte"]
          sbx["um HTMLIFrameElement com<br/>sandbox = 'allow-scripts<br/>allow-same-origin allow-modals'<br/>+ pointer-lock, forms, downloads<br/>se o manifesto pedir<br/>allow = 'autoplay; fullscreen;<br/>clipboard-write'"]
          bexp["exp : Experiencia (estereotipo)<br/>o bundle, servido pela CDN"]
          bsdk["sdk : @edu/sdk"]
```

```
          btok["token : string<br/>{so em memoria}"]
        end
      end
    end
  end

  %% conectores; === marca os dois unicos que atravessam o sandbox
  page ---|embed| pEmbed
  pProg ---|progresso| rprog
  pTok ---|token| rtok
  pLog ---|sinal| rlog
  pPonte === proto
  proto --- pTok
  proto --- pLog
  cdn === pSrc
  pPonte --- bexp
  bexp --- bsdk
  bsdk --- btok

  classDef borda stroke:#e8740c,stroke-width:2px
  class pPonte,pSrc borda

  %% targetOrigin e sempre expOrigin, nunca *: o frame so envia para a origem
  %% que montarEmbed calculou, e so aceita mensagem cujo ev.origin seja ela.
  %% O ExpHost tem uma segunda sobreposicao, fora do desenho: 'A missao
  %% travou', com o hash do erro, quando a experiencia manda exp:erro. E dele,
  %% nao do frame: sobrevive a remontagem do frame ao recarregar.
```

A.20 Figura 5.18 — Atividades: o resolvedor de disponibilidade

```
flowchart TB
  subgraph ESQ["comum/disponibilidade.ts - com banco"]
    ini([inicio])
    a1["buscar experiencias[expId]<br/>produto, publico; pode nao existir"]
    a2["cadeiaDaEscola<br/>[escola] ou [escola, regional]"]
    a3["linhasDoEscopo<br/>escola nula OU na cadeia; so ids legiveis"]
    a4["produtoConhecido<br/>fora do vocabulario -> null"]
    d4{"efetivo.ligado?"}
    fim(((fim)))
    a7["lanca 403 recurso-indisponivel"]
    ff((("X - fim de fluxo")))
  end

  subgraph DIR["recursos/regras.ts - funcao pura"]
    d1{"produto?"}
    a5["maisEspecifica(<br/>'produto:{slug}')"]
    d2{"linha do produto?"}
    r1["desligado,<br/>tetoDoProduto"]
    m1{" "}
    a6["maisEspecifica(<br/>'experiencia:{expId}')"]
    d3{"linha da experiencia?"}
    r2["ligado,<br/>versaoFixa = null"]
    r3["ligado e versaoFixa<br/>da linha"]
    m2{" "}
  end

```

```
  ini --> a1 --> a2 --> a3 --> a4
  a4 -->|resolverExperiencia| d1
  d1 -->|sim| a5
  a5 --> d2
  d2 -->|desligada| r1
  d2 -->|nenhuma ou ligada| m1
  d1 -->|nao| m1
  m1 --> a6 --> d3
  d3 -->|nenhuma| r2
  d3 -->|uma linha| r3
  r2 --> m2
  r3 --> m2
  r1 --> m2
  m2 --> d4
  d4 -->|sim| fim
  d4 -->|nao| a7
  a7 --> ff

  %% m1 e m2 sao nos de juncao. O X e o fim de fluxo pela excecao: a transacao
  %% de quem chamou desfaz o que tiver feito.
  %% maisEspecifica percorre a cadeia na ordem - escola, depois regional - e
  %% devolve a primeira linha do recurso; sem nenhuma, a de escola nula (o
  %% contrato); sem ela, null. Sobe so um nivel: a rede nao tem arvore mais funda.
  %% A linha inteira e a decisao: ligado e versaoFixa nao se resolvem em cadeias
  %% separadas. A tela mostra exatamente a linha que vale.
  %% Sem linha em experiencias (stack nova, antes do primeiro publish) NAO tranca:
  %% sem produto, vale so o interruptor da propria experiencia.
  %% A mensagem depende do publico: 'Este recurso nao esta disponivel para a sua
  %% rede.' para professor; 'Esta missao nao esta aberta agora.' para estudante.
```

A.21 Figura 5.19 — Sequência: a entrada do estudante na missão

```
sequenceDiagram
  participant N as Navegador (hub no cliente)
  participant H as Hub (servidor)
  participant A as API (NestJS)
  participant P as PostgreSQL (Cloud SQL)

  N->>H: GET /exp/:expId (cookie)
  H->>A: GET /salas/minhas
  A->>P: turmasDoEstudante + liberacoes
  P-->>A: matriculas e linhas
  A-->>H: grade: estado por missao, sem as indisponiveis

  alt missao nao esta liberada na grade
    H-->>N: 404 (notFound)
  else liberada
    H->>H: montarEmbed: assina o token (RS256, 15 min)
    H->>A: GET /experiencias/:expId e /recursos/efetivos
    A-->>H: manifesto, versao fixada da escola
    H-->>N: HTML com ExpFrame
  end

  %% a linha de vida da experiencia comeca na mensagem create: antes dela, o iframe nao existe
  create participant E as Experiencia (iframe)
  N-->>E: create - bundle da CDN
```

```
  Note over N,E: ref: handshake exp:ready / hub:init - fig:seq-handshake

  E->>A: 1a rota de estudante (Bearer)
  A->>A: funil: garantirSessao ou estudanteDaEscola
  A->>P: exigirRecursoDisponivel (fig:uml-atividade-disp)
  A->>P: missoesLiberadasPara
  P-->>A: linhas de disponibilidade e de liberacao

  alt recurso desligado no escopo que vale
    A-->>E: 403 recurso-indisponivel
  else sem liberacao liberada nas turmas do estudante
    A-->>E: 403 missao-nao-liberada
  else as duas passam
    A-->>E: 200 - a rota segue
  end

  %% O primeiro alt (o filtro do hub) e UX; as travas estao no segundo, na API,
  %% e disparam em serie: disponibilidade antes de liberacao.
```

A.22 Figura 5.20 — Sequência: uma chamada de IA com quota

```
sequenceDiagram
  participant E as Experiencia (pela SDK)
  participant G as OidcGuard (auth/)
  participant C as AiController (ai/)
  participant Q as QuotaService (ai/)
  participant P as PostgreSQL (Cloud SQL)
  participant M as provider (MedidorDeProvider)
  participant V as Vertex AI (Gemini)

  E->>G: POST /ai/gerar (Bearer)
  G->>G: TokenService.verificar: JWKS do hub
  break token ausente ou invalido
    G-->>E: 401
  end
  G->>C: req.auth = {usuarioId, escolaId, perfil}
  C->>C: GerarSchema.parse (Zod, 400 se invalido)
  C->>Q: temSaldo(escola, usuario, perfil)
  Q->>P: limite: ia_quotas, fora do comTenancy
  P-->>Q: tokens_por_dia
  Q->>P: usadoHoje: ia_uso_diario, em comTenancy
  P-->>Q: tokens_usados
  Q-->>C: usado < limite
  break sem saldo
    C-->>E: 429 'Quota diaria de IA atingida'
  end
  C->>C: promptDoTutor(expId)
  C->>P: serieDoPedido: turma.findFirst
  P-->>C: serie -> '6o ano'
  C->>M: generate({system, user})
  M->>M: medir: VertexProvider
  M->>V: :generateContent (ADC)
  V-->>M: texto e uso
  M-->>C: AiResult {texto, tokensUsados}
  C->>Q: consumir(..., tokensUsados)
```

```
  Q->>P: upsert em comTenancy
  P-->>Q: tokens_usados
  C-->>E: 200 {texto}

  %% break e a saida antecipada: quando a guarda vale, o operando roda e o resto
  %% da interacao NAO acontece. O consumo e gravado DEPOIS da resposta do modelo,
  %% com os tokens que ele declarou; entre o temSaldo e o consumir nao ha trava.
```

A.23 Figura 5.21 — Sequência: um turno do agente de IA do professor

```
sequenceDiagram
  participant N as Navegador (painel do chat)
  participant H as Hub (rota de stream)
  participant A as AgenteProfessor (controller + service)
  participant P as PostgreSQL (Cloud SQL)
  participant E as Agente (ADK no Agent Engine)
  participant G as Gemini (global -> us)
  participant D as Data store (Vertex AI Search)

  N->>H: POST {texto}
  H->>A: POST /agente-professor/conversas/:id/mensagens (token curto)
  A->>A: OidcGuard, RolesGuard (professor), @Produto
  A->>P: daPosse: usuario_id e escola_id
  P-->>A: sessao_id
  A->>P: temSaldo: ia_quotas, ia_uso_diario
  P-->>A: usado < limite
  break corpo invalido, outro dono, mais de 10/min, sem saldo
    A-->>N: 400 | 404 | 429 em JSON, antes do stream
  end
  A-->>H: 200 text/event-stream
  H-->>N: repassa o corpo
  A->>E: :streamQuery?alt=sse {user_id, session_id, message}
  E->>E: guarda de escopo (modelo leve)
  E->>G: gerar, em stream
  G-->>E: function_call
  E->>D: buscar_habilidades: filtro exato, paginando
  D-->>E: habilidades
  E->>G: com o retorno
  G-->>E: texto em pedacos
  loop cada evento do agente
    E-->>A: evento: parcial, final ou state_delta
    A-->>N: event: delta (sem a repeticao do final agregado)
  end
  A->>A: fontes, titulo, conferencia dos codigos (habilidadePorCodigo)
  A-->>N: fontes, titulo, aviso, fim
  A->>P: finally: consumir(soma dos nao parciais)
  A->>P: atualizada_em, titulo
  A->>E: PATCH expireTime (renovar a sessao)

  %% Durante o loop a api escreve ": ping" a cada 15 s. "Parar" (ou fechar a
  %% aba) aborta cada salto ate o Vertex; fechar o PAINEL nao aborta; o teto de 240 s vira event: erro
↪ TEMPO_ESGOTADO. O
  %% consumo roda no finally tambem no abandono e no erro; a renovacao so depois
  %% de fim, e a falha dela e so log.
```

A.24 Figura 5.22 — Comunicação: o caminho de um lote de telemetria

```
%% aproximacao: o Mermaid nao tem diagrama de comunicacao; cada link vira aresta, a mensagem numerada vira o rotulo dela e a seta da a direcao (o detalhe de cada numero esta na tabela que segue a figura ).
flowchart LR
  exp["<u>:Experiencia</u><br/>codigo da missao"]
  tel["<u>t : TelemetryClient</u><br/>@edu/sdk"]
  tok["<u>:TokenService</u><br/>auth/"]
  ctrl["<u>:TelemetryController</u><br/>telemetry/"]
  ps["<u>:Topic</u><br/>{contrato}-telemetria"]
  guard["<u>:PubSubPushGuard</u><br/>ID token do Google"]
  pg["<u>:telemetria_eventos</u><br/>TabelaDeTenant, por mes"]

  exp -->|"1: track()"| tel
  tel -->|"1.1: POST /telemetry/batch<br/>3: sendBeacon /beacon"| ctrl
  ctrl -->|"1.1.1: identidade do token"| ctrl
  ctrl -->|"3.1: verificar()"| tok
  ctrl -->|"1.1.2, 3.2: publishMessage()"| ps
  ps -->|"2: push /ingest"| guard
  guard -->|"2.1: ingest()"| ctrl
  ctrl -->|"2.1.1: INSERT ... DO NOTHING"| pg

  %% Tres gatilhos, tres numeros de topo: 1 e o relogio da SDK, 3 e a saida da
  %% pagina e 2 e o Pub/Sub, quando ele quiser. Nenhum espera o outro.
  %% Sem TELEMETRIA_TOPIC (compose local, CI), 1.1.2 vira uma linha de debug e
  %% nada e gravado.
```

A.25 Figura 5.23 — Tempo: *token*, renovação e sessão

```
%% aproximacao: o Mermaid nao tem diagrama de tempo; cada faixa vira uma secao do gantt e cada estado um intervalo no eixo de minutos, e as mensagens entre faixas ficam em comentario.
%%{init: {"gantt": {"displayMode": "compact"}}}%%
gantt
  title Missao aberta - o pedido de token dos 20 min falha por rede
  dateFormat mm
  axisFormat %M
  tickInterval 5minute

  section Chamadas da experiencia a API
  200                              :c1, 00, 25m
  401 {5 min de 401}               :crit, c2, 25, 5m
  200                              :c3, 30, 10m

  section Token vigente na experiencia
  A - vale ate 15                  :t1, 00, 10m
  B - vale ate 25 {15 min}         :t2, 10, 15m
  B vencido                        :crit, t2v, 25, 5m
  D - vale ate 45                  :t3, 30, 10m

  section Pedido de token do hub
  pedindo - ok {10 min}            :p1, 10, 36s
  pedindo - falhou (rede)          :crit, p2, 20, 36s
  pedindo - ok                     :p3, 30, 36s
```

```
  pedindo                          :p4, 40, 24s

  section Sessao do hub
  ativa {expira 2 h depois do ultimo ponto}  :s1, 00, 40m
  renova o prazo                   :milestone, r0, 00, 0m
  renova o prazo                   :milestone, r1, 10, 0m
  renova o prazo                   :milestone, r2, 30, 0m

  %% Fora dos pulsos, o pedido de token esta ocioso.
  %% Mensagens entre faixas: exp:token-expirando aos 10, 20 e 30 (experiencia
  %% para o hub); hub:token de volta aos 10,6 e aos 30,6 (so os pedidos ok).
  %% O token B (emitido aos 10) vence aos 25; o proximo pedido e aos 30; entre
  %% os dois, toda chamada da experiencia a API recebe 401.
```

A.26 Figura 5.24 — Máquina de estados de protocolo: a ponte

```
stateDiagram-v2
  [*] --> AguardandoInit : connectToHub(hubOrigin)

  AguardandoInit --> Conectada : hub:init [trunc(v) = 2] / limpa os temporizadores, resolve
  AguardandoInit --> Incompativel : hub:init [trunc(v) != 2]
  AguardandoInit --> TempoEsgotado : after(8 s)

  state Conectada {
    [*] --> Ativa
    Ativa --> Pausada : hub:pause
    Pausada --> Ativa : hub:resume
  }

  Incompativel : Incompativel
  Incompativel : reject - 'Protocolo incompativel'
  TempoEsgotado : TempoEsgotado
  TempoEsgotado : reject - 'Timeout no handshake'
  Incompativel --> [*]
  TempoEsgotado --> [*]

  %% transicoes internas (as que nao mudam de estado): evento [guarda] / efeito
  note right of AguardandoInit
    entry / normalizarHubOrigin, envia exp:ready
    do / reenvia exp:ready a cada 400 ms
    message [origin != hubOrigin] / descarta
    message [nao casa HubInitSchema] / ignora
  end note
  note right of Conectada
    hub:token / token := msg.token
    after(10 min) / envia exp:token-expirando
    hub:init / ignora (resposta a um exp:ready reenviado)
    message [nao casa HubParaExpSchema] / avisa e descarta
  end note

  %% A SDK nao guarda Ativa nem Pausada: repassa as duas mensagens aos handlers
  %% onPause/onResume, e a experiencia para os proprios cronometros. O hub manda
  %% hub:pause em visibilitychange, em blur (salvo quando o foco foi para o
  %% proprio iframe) e antes do aviso de sessao expirada.
  %% Na pratica, o incompativel termina em tempo esgotado: o ExpFrame confere a
  %% versao do exp:ready e, se o maior diverge, registra protocolo-incompativel e
```

```
  %% NAO responde. A SDK nunca recebe o hub:init que a levaria a Incompativel:
  %% cai no after(8 s). O estado existe para o dia em que um hub responder com
  %% outra versao.
```

A.27 Figura 5.25 — Visão geral de interação: a jornada do estudante

```
%% aproximacao: o Mermaid nao tem diagrama de visao geral de interacao; cada quadro ref vira um no de sub-rotina com a figura em que a interacao esta desenhada, e as barras de bifurcacao e juncao viram nos estreitos.
flowchart TB
  ini([inicio])
  login[["ref: login (UC-01)<br/>signIn('credentials') -> POST /auth/login"]]
  dsenha{"senha provisoria?"}
  senha[["ref: trocar a senha<br/>/trocar-senha (UC-25)"]]
  m1{" "}
  grade[["ref: a grade do estudante<br/>GET /salas/minhas, sem as indisponiveis"]]
  dlib{"o card esta liberado?"}
  cad["cadeado: 'Ainda nao abriu' ou 'Esta voce ja fechou'"]
  ff1(("X - fim de fluxo"))
  entra[["ref: entrada na missao<br/>as duas travas (fig:uml-seq-entrada)"]]
  hand[["ref: handshake<br/>exp:ready/hub:init (fig:seq-handshake)"]]
  fork["bifurcacao"]
  dom[["ref: rotas de dominio<br/>e IA com quota (fig:uml-seq-ia)"]]
  tel[["ref: telemetria em lote<br/>(fig:uml-comunicacao)"]]
  ren[["ref: renovacao do token<br/>a cada 10 min (fig:uml-tempo)"]]
  join["juncao"]
  dfim{"como a missao termina?"}
  conc[["ref: concluir<br/>POST .../progresso/:expId/concluir"]]
  exp["aviso 'Sua sessao expirou'; hub:pause"]
  trav["'A missao travou', com o hash do erro"]
  f1(((fim))); f2(((fim))); f3(((fim)))

  ini --> login --> dsenha
  dsenha -->|sim| senha
  senha --> m1
  dsenha -->|nao| m1
  m1 --> grade --> dlib
  dlib -->|nao| cad --> ff1
  dlib -->|sim| entra --> hand --> fork
  fork --> dom & tel & ren
  dom & tel & ren --> join
  join --> dfim
  dfim -->|exp:concluir| conc --> f1
  dfim -->|"/api/exp-token da 401 ou 403"| exp --> f2
  dfim -->|exp:erro| trav --> f3

  classDef barra fill:#1f2a44,stroke:#1f2a44,color:#ffffff
  class fork,join barra

  %% As tres interacoes do meio correm juntas enquanto a missao esta aberta. A
  %% saida pelo cadeado termina em fim de fluxo: o estudante continua na home.
```

A.28 Figura 5.26 — Implantação: que artefato roda em que nó

```
%% aproximacao: o Mermaid nao tem diagrama de implantacao; cada no (e ambiente de execucao) vira subgraph, cada artefato vira um no dentro dele, o caminho de comunicacao vira linha e a dependencia implanta vira seta grossa.
flowchart LR
  subgraph NAV["device - Navegador da escola"]
    subgraph NAVENV["browser - aba do hub"]
      apag["artefato<br/>pagina do hub"]
    end
    subgraph IFR["sandbox - iframe"]
      bnav["Bundle<br/>{expId}/{versao}/<br/>baixado da CDN; token so em memoria"]
    end
  end

  subgraph WSO2["externo - WSO2 IS"]
    iwso2["projeto proprio; futuro emissor - hoje e o hub"]
  end
  subgraph GHA["no - GitHub Actions"]
    igha["deploy.yml; WIF, sem chave JSON"]
  end

  subgraph GCP["projeto GCP - uma stack por contrato"]
    subgraph LB["no - Load Balancer global + Cloud CDN"]
      ilb["um IP, host rules:<br/>hub, api, exp, assets"]
    end
    subgraph RUNHUB["Cloud Run - {contrato}-frontend"]
      ahub["imagem hub:{sha}<br/>node apps/hub/server.js"]
    end
    subgraph RUNAPI["Cloud Run - {contrato}-api"]
      aapi["imagem api:{sha}<br/>node dist/src/main.js"]
    end
    subgraph JOB["Cloud Run Job - {contrato}-migrate"]
      ajob["imagem api:{sha}<br/>prisma migrate deploy"]
    end
    subgraph SM["no - Secret Manager"]
      ism["sete segredos: database-url, token-signing-key,<br/>auth-secret e mais quatro"]
    end
    subgraph GCS["no - Cloud Storage"]
      igcs["-exp: publico, CDN<br/>-assets: publico, CDN<br/>-planos: PRIVADO<br/>-mapas: sourcemaps, privado"]
      bgcs["Bundle<br/>{expId}/{versao}/<br/>index.html com max-age=60"]
    end
    subgraph SQL["no - Cloud SQL {contrato}-pg"]
      isql["PostgreSQL, HA + PITR,<br/>so IP privado; role app<br/>sem BYPASSRLS"]
    end
    subgraph PS["no - Pub/Sub"]
      ips["topic {contrato}-telemetria<br/>push -telemetria-ingest"]
    end
    subgraph AR["no - Artifact Registry"]
      iar["hub:{sha}, api:{sha},<br/>agente-professor:{sha};<br/>push direto do CI"]
    end
    subgraph VX["servico gerenciado - Vertex AI"]
      ivx["Gemini; ADC da conta {contrato}-api"]
    end
    subgraph ENG["Agent Engine - us-east1 (nasce no apply 2)"]
      aeng["imagem agente-professor:{sha}<br/>SA {prefixo}-agente"]
    end
  end
```

```
%% caminhos de comunicacao
NAV ---|HTTPS| LB
LB ---|hub| RUNHUB
LB ---|api| RUNAPI
LB ---|CDN| GCS
RUNHUB ---|"HTTPS, API_INTERNAL_URL"| RUNAPI
RUNAPI ---|"assets, planos"| GCS
RUNAPI ---|"publica; push /telemetry/ingest"| PS
RUNAPI ---|VPC| SQL
JOB ---|VPC| SQL
RUNAPI --- VX
RUNAPI ---|SSE| ENG
SM -.-|env| RUNHUB

%% o CI implanta cada artefato
GHA ==>|implanta| AR
GHA ==>|rsync| GCS
bgcs ==>|implanta| bnav
GHA ==>|"PATCH imageUri"| aeng

%% A mesma imagem api:{sha} esta implantada em dois nos, e o bundle esta
%% armazenado num e executado em outro. O CI ainda executa o job de migracao e
%% publica o manifesto (PUT /experiencias/:expId), o que o diagrama deixa no
%% texto. Escala, rede e certificados sao do Terraform, em outro repositorio.
```

A.29 Figura 6.1 — Implantação de um contrato

```
C4Deployment
  title Implantacao de um contrato

  Deployment_Node(internet, "Internet", "escola") {
    Container(browser, "Navegador do estudante", "", "")
  }

  Deployment_Node(gcp, "Google Cloud - projeto do contrato", "GCP") {
    Deployment_Node(borda, "Borda global", "") {
      Container(lb,  "Cloud Load Balancing", "", "1 IP, host rules")
      Container(cdn, "Cloud CDN", "", "")
    }
    Deployment_Node(regiao, "Regiao de execucao (us-east1)", "") {
      Container(hub, "Cloud Run {contrato}-frontend", "hub", "min>=1")
      Container(api, "Cloud Run {contrato}-api", "", "CPU sempre alocada, conc. 80")
      Deployment_Node(vpc, "VPC - rede privada", "") {
        ContainerDb(sql, "Cloud SQL {contrato}-pg", "PostgreSQL HA, PITR, pooler", "")
      }
    }
    Deployment_Node(gcs, "Cloud Storage", "") {
      Container(bexp, "{contrato}-exp", "publico, imutavel, CDN", "")
      Container(bass, "{contrato}-assets", "publico, CDN", "")
      Container(bpla, "{contrato}-planos", "PRIVADO, versionado", "")
    }
    ContainerQueue(ps, "Pub/Sub {contrato}-telemetria", "", "")
  }

  System_Ext(idp, "WSO2 IS", "projeto proprio")
```

```
  System_Ext(vertex, "Vertex AI", "endpoint global")

  Rel(browser, lb, "443")
  Rel(lb, cdn, "bundles e assets")
  Rel(cdn, bexp, "serve")
  Rel(lb, hub, "HTTPS")
  Rel(lb, api, "HTTPS")
  Rel(api, sql, "5432 - IP PRIVADO")
  Rel(api, bpla, "so pela API")
  BiRel(api, ps, "publica / recebe push")
  Rel(hub, idp, "OIDC")
  Rel(api, vertex, "HTTPS + ADC")
```

A.30 Figura 6.2 — Implantação detalhada

```
flowchart TB
  browser([Navegador do estudante]); gh[[GitHub Actions]]; wso2[[WSO2 IS]]; vertex[[Vertex AI - global]]

  subgraph GCP["Google Cloud - projeto do contrato"]
    subgraph BORDA["Borda global"]
      lb["Load Balancer (1 IP, host rules)"]; cdn["Cloud CDN"]; cert["Certificados gerenciados (um por dominio)"]
    end
    subgraph EXEC["Regiao de execucao (us-east1)"]
      hub["Cloud Run -frontend (min>=1)"]
      api["Cloud Run -api (CPU sempre alocada, conc. 80)"]
      job["Cloud Run Job -migrate (na VPC)"]
      subgraph VPC["VPC - rede privada"]
        sql[("Cloud SQL -pg (HA, PITR, pooler)")]
        conn["Serverless VPC Access (conector)"]
      end
    end
    subgraph STORAGE["Cloud Storage"]
      bexp["{contrato}-exp - publico, imutavel, CDN"]
      bass["{contrato}-assets - publico, CDN"]
      bpla["{contrato}-planos - PRIVADO, sem CDN, versionado"]
    end
    subgraph MSG["Mensageria"]
      topic["topic -telemetria"]; sub["push sub -telemetria-ingest (oidc_token)"]
    end
    subgraph GOV["Identidade, segredos e artefatos"]
      sm["Secret Manager (6 segredos)"]; sa["Service Accounts (-api, -run, -push)"]
      wif["WIF pool github (sem chave JSON)"]; ar["Artifact Registry"]
      log["Cloud Logging / Monitoring"]
    end
  end

  browser -->|443| lb
  lb --> cdn --> bexp & bass
  lb --> hub & api
  hub -->|BFF| api
  api -->|5432 privado| conn --> sql
  job --> conn
  api -->|so pela API| bpla
  api --> bass
```

```
  api --> topic --> sub -->|POST /telemetry/ingest| api
  gh -->|OIDC do GitHub| wif
  gh -->|push da imagem| ar
  wso2 -->|OIDC| hub
  api -->|ADC| vertex
  sm --> api & hub
  sa --> api & hub
```

A.31 Figura 6.3 — Identidade de carga de trabalho

```
flowchart LR
  subgraph CI["CI: Workload Identity Federation"]
    gh["GitHub Actions (id-token: write)"] --> tok["token OIDC do GitHub (repo, ref, workflow)"]
    tok --> wif["WIF pool github (valida e troca)"]
    wif --> sadeploy["access token da DEPLOY_SA (curta duracao)"]
  end

  subgraph APP["Aplicacao: Application Default Credentials"]
    run["Cloud Run (API) - sem env de credencial"] --> md["metadata server (ADC nivel 3)"]
    md --> saapi["SA {contrato}-api (roles/aiplatform.user)"]
    saapi --> vx["Vertex AI - autorizado"]
  end

  %% A mesma cadeia nos tres ambientes:
  %%   1. GOOGLE_APPLICATION_CREDENTIALS
  %%   2. arquivo de `gcloud auth application-default login`
  %%   3. metadata server (Cloud Run)
  %% Antes havia SO o nivel 3 — e por isso nenhuma chamada de IA funcionava
  %% fora do Cloud Run. NUNCA chave de API.
```

A.32 Figura 7.1 — Os três níveis de isolamento

```
flowchart TB
  subgraph A["Contrato A - stack GCP"]
    apiA["API do contrato A"] --> pgA[("PostgreSQL do contrato A")]
    subgraph ESC["Dentro de UMA stack: isolamento LOGICO"]
      e1["escola sp-001"]; e2["escola sp-002"]; e3["... 100+"]
    end
    tab["Toda tabela de dado de estudante:<br/>escola_id TEXT NOT NULL<br/>ENABLE + FORCE ROW LEVEL SECURITY<br/>POLICY USING (escola_id = current_setting('app.escola_id', true))"]
    e1 -.-> tab; e2 -.-> tab; e3 -.-> tab
  end
  subgraph B["Contrato B - OUTRA stack"]
    apiB["API do contrato B"] --> pgB[("PostgreSQL do contrato B")]
  end
```

A.33 Figura 7.2 — Propagação da escola

```
flowchart LR
  req["Requisicao HTTP (Authorization: Bearer)"] --> guard["OidcGuard (JWKS; injeta req.auth)"]
  guard --> roles["RolesGuard (le req.auth.perfil)"]
  roles --> int["TenancyInterceptor (definirTenancyNoContexto)"]
  int --> ctrl["controller"] --> svc["service: comTenancy(fn)"]
  svc --> tx["Transacao: set_config + query"]
  svc -->|sem escola no contexto| fail["LANCA"]

  %% Guards rodam ANTES de interceptors, entao req.auth ja existe.
  %% Rota sem OidcGuard nao abre contexto — e comTenancy sem escola falha alto.
  %% A RLS e rede de seguranca, nao a unica tranca: os services continuam
  %% filtrando escola_id explicitamente em toda query.
```

A.34 Figura 8.1 — Modelo de dados da plataforma

```
erDiagram
  escolas ||--o{ turmas : tem
  escolas ||--o{ users  : tem
  turmas  ||--o{ users  : agrupa
  experiencias ||..o{ progresso : "exp_id"
  telemetria_eventos ||..o{ particoes : "ATTACH por mes"

  escolas {
    text id PK "MESMO valor do claim escola_id e do set_config"
    text nome
    timestamptz created_at
  }
  turmas {
    uuid id PK
    text escola_id FK
    text nome "UNIQUE (escola_id, nome)"
    text serie "1EF..9EF | 1EM..3EM"
  }
  users {
    uuid id PK
    text role "estudante|professor|gestor|admin-contrato"
    text escola_id FK "NULL SO p/ admin-contrato (CHECK)"
    uuid turma_id FK
    text username "UNIQUE (escola_id, username)"
    text display_name
    text password_hash
  }
  experiencias {
    text exp_id PK
    text nome
    text versao_ativa "rollback = UPDATE deste campo"
    text_array series "VAZIO = TODAS as series"
    text_array capabilities
    jsonb viewport "presente => casca de aparelho"
    text publico "estudante|professor"
    text_array permissoes "hoje so pointer-lock"
  }
  progresso {
    text escola_id PK
    text estudante_id PK
```

```
    text exp_id PK
    jsonb dados
  }
  ia_uso_diario {
    text escola_id PK
    text usuario_id PK
    text dia PK "YYYY-MM-DD"
    int tokens_usados
  }
  telemetria_eventos {
    uuid event_id PK "PK COMPOSTA: o Postgres exige a chave de particao"
    timestamptz ts PK
    text escola_id
    text estudante_id
    text exp_id
    text exp_version
    uuid sessao_id
    text verbo
    text objeto
    jsonb resultado
    jsonb contexto
  }
```

A.35 Figura 8.2 — Enturmamento e sala de aula

```
erDiagram
  periodos_letivos ||--o{ turmas     : "a turma vive dentro de um ano letivo"
  turmas           ||--o{ matriculas : enturma
  users            ||--o{ matriculas : "pessoa x turma, datada"
  turmas           ||--o{ liberacoes : "a missao DENTRO da sala"
  users            ||--o{ liberacoes : "quem liberou"

  periodos_letivos {
    uuid id PK
    text tipo "ano_letivo|semestre|bimestre|trimestre"
    text titulo "UNIQUE (tipo, titulo)"
    smallint ano
    date inicio
    date fim
    uuid pai_id FK "ano letivo -> bimestre; NULL = raiz"
  }
  turmas {
    uuid id PK
    text escola_id FK
    uuid periodo_letivo_id FK "NOT NULL: turma sem periodo e users.turma_id de novo"
    text nome "UNIQUE (escola_id, periodo_letivo_id, nome)"
    text serie "1EF..9EF | 1EM..3EM"
    text tipo "regular|componente (classType do OneRoster)"
    text turno
  }
  matriculas {
    uuid id PK "RLS por escola_id"
    text escola_id FK
    uuid turma_id FK
    uuid usuario_id FK
    text papel "estudante|professor — gestor NAO entra"
```

```
    boolean principal "alimenta o cache users.turma_id"
    date inicio
    date fim "sair e fim, nunca DELETE"
  }
  liberacoes {
    uuid id PK "RLS por escola_id"
    text escola_id FK
    uuid turma_id FK
    text exp_id "SEM FK: o catalogo experiencias e escrito pelo CI"
    text estado "liberada|encerrada — nao existe bloqueada"
    uuid liberada_por FK
    timestamptz liberada_em
    timestamptz encerrada_em "CHECK: casa com estado"
  }
  users {
    uuid id PK
    text matricula_externa "o RA da rede; indice unico parcial por escola"
    uuid turma_id FK "CACHE da matricula principal — a verdade e matriculas"
    boolean ativo
    boolean senha_provisoria
  }

  %% A ausencia de linha em liberacoes E o bloqueio: guardar bloqueio como linha
  %% obrigaria a criar uma por turma x experiencia so para o mundo nascer fechado.
  %% liberacoes e PRE-CONDICAO do relogio da exp02em1b e dos gates da exp06ef2b, que
  %% dizem "a aula comecou" e continuam morando na tabela DA experiencia.
```

A.36 Figuras 8.3 e 8.4 — Modelo da exp08ef1b

```
erDiagram
  exp08ef1b_activities ||--o{ exp08ef1b_authors : tem
  exp08ef1b_authors    ||--o{ exp08ef1b_posts   : escreve
  exp08ef1b_posts      ||--o{ exp08ef1b_post_comments : tem
  exp08ef1b_activities ||--o{ exp08ef1b_generation_jobs : enfileira
  exp08ef1b_activities ||--o{ exp08ef1b_runs : hospeda
  exp08ef1b_runs ||--o{ exp08ef1b_events : registra
  exp08ef1b_runs ||--o{ exp08ef1b_student_posts : publica
  exp08ef1b_runs ||--|| exp08ef1b_conclusoes : conclui
  exp08ef1b_runs ||--|| exp08ef1b_reflexoes : reflete
  exp08ef1b_runs ||--|| exp08ef1b_dossiers : gera
  exp08ef1b_runs ||--o{ exp08ef1b_comentarios_da_sessao : ancora
  exp08ef1b_activities ||--o{ exp08ef1b_app_70_perfis_importados : "largada"

  exp08ef1b_activities {
    uuid id PK
    text escola_id FK
    uuid professor_id FK
    text pergunta_chave
    text pergunta_origem "ia|professor - auditoria de autoria"
    text etapa "relogio COLETIVO da turma"
    boolean ia_ao_vivo "desligavel sem deploy"
    jsonb antagonismo
    jsonb engine_config
    text status
  }
  exp08ef1b_posts {
```

```
    uuid id PK
    text lado "VARIAVEL INDEPENDENTE - sanitizada antes de sair"
    text perfil "idem"
    smallint intensidade "idem"
    text image_status "pendente|ok|fallback"
    boolean aprovado
  }
  exp08ef1b_runs {
    uuid id PK
    text lado_inicial
    text lado_origem "perfil|sorteio|redistribuido - auditoria"
    jsonb engine_state "estado do motor; unidade de LOCK"
    text nome_no_app "a conta e do APARELHO, nao da crianca"
  }
  exp08ef1b_events {
    bigint id PK
    text tipo "canal OPERACIONAL: sincrono e transacional"
    jsonb payload "{dwellMs, dwellNorm}"
  }
  exp08ef1b_comentarios_da_sessao {
    uuid post_id FK "XOR com student_post_id (CHECK na migration)"
    uuid student_post_id FK
    boolean de_ia "para o professor distinguir na auditoria"
  }
  exp08ef1b_app_70_perfis_importados {
    jsonb escores "4 quadrantes, 0-1"
    text dominante
    text origem "jogo|professor"
    text lado_forcado "PRESERVADO no upsert"
  }
```

A.37 Figuras 8.5 e 8.6 — Demais experiências

```
erDiagram
  app_70_sessoes ||--o{ app_70_perfis : "recalculado pelo servidor"
  app_70_perfis  ||..|| app_70_bolhas : "mesmo estudante"
  app_90_planos  ||..o{ app_90_referencias : "monta o prompt"
  app_91_ciclos  ||--o{ app_91_checkins : "7 check-ins"

  exp04ef1b_sessoes {
    text escola_id PK
    uuid estudante_id PK
    text_array salvos "CHECK cardinality <= 3"
    boolean ranking_desbloqueado "LATCH: nao volta"
    boolean credibilidade_adicionada "congela os salvos"
  }
  app_70_sessoes {
    uuid id PK "EMITIDO pela API - o cliente colidia"
    uuid turma_id FK "desnormalizado: turma NO MOMENTO da partida"
    text bolha "A|B|C - do SERVIDOR"
    smallint cartoes_servidos "!= apareceram"
    timestamptz aberta_em "relogio do SERVIDOR"
    jsonb payload "bruto - unica fonte do reprocessamento"
    jsonb leitura_mensagens
  }
  app_70_perfis {
```

```
    smallint versao_mapeamento "sobe => nasce linha nova"
    jsonb valores "null NAO e zero"
    jsonb graus "scored|indicative|zero|none"
    float abertura_vs_conservacao "NULL se polo nao observado"
    boolean amostra_suficiente "falso => o painel avisa que NAO SABE"
  }
  app_70_bolhas {
    text escola_id PK
    uuid estudante_id PK "por ESTUDANTE, nao por partida"
    text bolha
  }
  app_90_planos {
    text status "pendente|gerando|pronto|erro"
    text caminho_gerado_md "IMUTAVEL, bucket privado"
    text caminho_html "o que o editor salvou, sanitizado"
  }
  app_90_referencias {
    text disciplina PK
    text serie PK "NULL = todas"
    text nome PK
    text texto "catalogo do CONTRATO - sem escola_id, FORA da RLS"
  }
  app_91_ciclos {
    uuid id PK
    int_array ordem_itens "permutacao de 1..21"
    int depressao "somas x2, ao concluir - DADO SENSIVEL"
  }
  app_91_checkins {
    smallint numero "1..7"
    text dia "UNIQUE (escola_id, professor_id, dia)"
    text texto_livre "NUNCA em log nem telemetria"
    text origem_texto "ia|fallback|seguranca"
    boolean sinal_de_apoio
  }
```

A.38 Figura 9.1 — *Handshake* hub-experiência

```
sequenceDiagram
  autonumber
  participant E as Experiencia (@edu/sdk no iframe)
  participant F as ExpFrame (React, no hub)
  participant R as Rota do hub (servidor)
  participant A as API (NestJS)

  R->>F: monta o embed: assina token RS256 (15 min)
  R->>A: GET /experiencias/:expId (manifesto)
  A-->>R: { versaoAtiva, viewport, permissoes }
  F->>E: renderiza iframe src = URL versionada
  E->>E: connectToHub({ hubOrigin })
  loop a cada 400ms, ate o hub responder
    E->>F: exp:ready { protocolVersion: 2 }
  end
  F->>F: valida ev.origin e a versao MAIOR
  F-->>E: hub:init { token, estudante, apiUrl, config }
  E->>E: valida com Zod; resolve a Promise
  E->>A: chamadas de dominio, /telemetry/batch, /ai/gerar
```

```
  E->>F: exp:token-expirando
  F->>R: GET /api/exp-token
  R-->>F: { token }
  F-->>E: hub:token { token }

  Note over E,F: Com assets em cache (F5), a experiencia fica pronta ANTES<br/>de o ExpFrame hidratar . Um disparo unico se perderia.<br/>hub:init repetido e evento normal, nao ruido.
```

A.39 Figura 9.2 — Resolução da URL do *embed*

```
flowchart TB
  ini([inicio]) --> q{tem linha em experiencias?}
  q -->|sim| com["{cdn}/{expId}/{versaoAtiva}/index.html?hubOrigin=..."]
  q -->|nao| sem["{baseDev}/?hubOrigin=... (porta de lib/experiencias-dev.ts<br/>ou o src explicito de /dev-embed)"]
  com --> vp1["viewport e permissoes vem do MANIFESTO"]
  sem --> vp2["le {baseInterna}/experiencia.json<br/>baseInterna = EXP_INTERNAL_HOST aplicado a base "]
  vp1 --> frame["ExpFrame: sandbox + allow,<br/>dentro de DispositivoFrame se houver viewport"]
  vp2 --> frame
  frame --> fim([fim])

  %% A leitura do experiencia.json NAO e so de dev: stack com a tabela vazia
  %% servia a exp08ef1b esticada na janela em vez de dentro do aparelho.
```

A.40 Figura 9.3 — Contrato entre duas experiências

```
flowchart LR
  v["app-70 (Vila do Lume)"] -->|payload bruto| mv["src/vila/ - quadrantes.ts: colapso 10 -> 4"]
  mv -->|"POST /atividades/:id/perfis/da-vila (professor)"| t["exp08ef1b_app_70_perfis_importados"]
  t -->|le| mi["src/perfis/largada.ts - decide o lado do feed"]
  mi --> i["exp08ef1b (Instanooria)"]

  %% Duas travas que CUSTAM estudantes, de proposito:
  %%  1. valor nao observado (indicative|none) fica FORA da media — nunca zero
  %%  2. quadrante sem nenhum membro observado INVALIDA o perfil inteiro
  %% Sem perfil, o Instanooria cai no sorteio balanceado, que e honesto sobre
  %% nao saber. E `lado_forcado` e PRESERVADO no upsert.
```

A.41 Figura 10.1 — Canal analítico

```
flowchart LR
  exp["Experiencia: track(verbo, objeto, dados)"] --> sdk["TelemetryClient<br/>buffer em memoria; lote de 50 ou 10s"]
  sdk -->|"<= 50 eventos"| batch["POST /telemetry/batch<br/>OidcGuard; identidade vem do TOKEN"]
  batch --> topic[["Pub/Sub topic -telemetria"]]
```

```
  sdk -->|sendBeacon no pagehide| beacon["POST /telemetry/beacon<br/>token NO CORPO; responde 204 SEMPRE"]
  beacon --> topic
  topic -->|push autenticado| ingest["POST /telemetry/ingest<br/>PubSubPushGuard: ID token do Google, aud, email"]
  ingest --> pg[("telemetria_eventos<br/>INSERT multi-linha<br/>ON CONFLICT (event_id, ts) DO NOTHING<br/>particionada por mes")]

  %% Onde ha perda possivel, e SO ali: entre o track() e o flush.
  %% Do Pub/Sub em diante a entrega e DURAVEL — e por isso um buffer em Redis
  %% nao o substituiria: reintroduziria a perda que ele eliminou.
```

A.42 Figura 11.1 — Camada de IA

```
C4Component
  title Camada de IA - src/ai e a UNICA porta

  Container_Boundary(api, "API") {
    Component(g, "geracao/", "", "exp08ef1b")
    Component(m, "motor/", "", "exp08ef1b")
    Component(p, "planos/", "", "app-90")
    Component(b, "bemestar/", "", "app-91")
    Component(a, "POST /ai/gerar", "", "SDK")

    Container_Boundary(ai, "src/ai/ - a UNICA porta") {
      Component(q,  "QuotaService", "", "temSaldo(escola, usuario, perfil) ANTES; consumir DEPOIS")
      Component(pr, "AiProvider", "", "generate, generateJson, generateImage")
      Component(pp, "prompts.ts", "", "prompt de sistema; NUNCA sai daqui")
      Component(vx, "vertex.provider.ts", "", "default; ADC + REST")
      Component(mk, "mock.provider.ts", "", "testes e dev sem GCP")
    }
    ComponentDb(ia, "ia_uso_diario", "", "quota persistida por (escola, usuario, dia)")
  }
  System_Ext(vertex, "Vertex AI", "endpoint global")

  Rel(g, pr, "sem quota (ADR-74)")
  Rel(m, pr, "sem quota (ADR-74)")
  Rel(p, q, "1")
  Rel(b, q, "1")
  Rel(a, q, "1")
  BiRel(q, ia, "incremento atomico")
  Rel(q, pr, "SO com saldo")
  Rel(pr, pp, "usa")
  Rel(pr, vx, "AI_PROVIDER=vertex")
  Rel(pr, mk, "AI_PROVIDER=mock")
  Rel(vx, vertex, "HTTPS + ADC")
```

A.43 Figura 11.2 — *Guardrails* do bem-estar

```
flowchart TB
  ini([inicio]) --> abre["GET /bemestar/hoje - 3 itens da permutacao do ciclo"]
```

```
  abre --> sinal{"sinal de apoio?<br/>item 17 ou 21 >= 2, ou termo de risco"}
  sinal -->|sim| fixo["texto FIXO de apoio (CVV 188, SAMU 192, CAPS/UBS)<br/>a IA NAO e chamada<br/>origem_texto = 'seguranca'"]
  sinal -->|nao| saldo{tem saldo e provider?}
  saldo -->|nao| fall["texto FIXO do catalogo<br/>origem_texto = 'fallback'"]
  saldo -->|sim| ia["IA gera abertura, pergunta aberta e acolhimento<br/>recebe SO: dia da semana, mes, no do check-in,<br/>texto livre delimitado"]
  ia --> val{validarTextoDaIa?}
  val -->|barrado| fall
  val -->|passou| ok["origem_texto = 'ia'"]
  fixo --> fim([fim]); fall --> fim; ok --> fim

  %% Os 21 itens do DASS-21 sao FIXOS e VERBATIM. A IA nunca reescreve um item:
  %% reformular quebra a validacao psicometrica e invalida as faixas de corte.
  %% O check-in NUNCA deixa de abrir.
```

A.44 Figura 12.1 — Os quatro graus de medida

```
classDiagram
  class scored {
    >= 2 sinais independentes
    numero: number
    entra na media
    E PUBLICADO com numero
  }
  class indicative {
    exatamente 1 sinal
    numero: null
    descrito em TEXTO
    fora dos eixos e das medias
  }
  class zero {
    teve a chance e NAO foi
    numero: null
    ENTRA na media, como zero de verdade
  }
  class none {
    a oportunidade NAO chegou
    numero: null
    SAI da conta - nao e zero
  }
  note "A distincao mais importante do modelo e entre zero e none.\nTratar 'nao teve chance' como 'escolheu nao' faz uma crianca\nparecer ter recusado algo que nunca lhe foi oferecido."
```

A.45 Figura 12.2 — app-92: o caminho do planejamento

```
flowchart TB
  subgraph fora["Fora do produto, uma vez"]
    pdf[PDFs homologados do MEC] --> ext[scripts/extrair-bncc.mjs]
    ext --> dados[("api/src/curriculo/dados/*.ts<br/>1.304 EF - 183 EM - 129 Computacao")]
  end
```

```
  ini((inicio))
  ini --> reg["1. Regencia<br/>componente, serie, ano letivo, grade de horarios,<br/>minutos por aula, divisao do ano<br/>POST /planejamento/configuracoes"]
  reg --> dist["2. Distribuicao<br/>o professor poe cada habilidade num periodo - DECISAO HUMANA<br/>PUT .../planos-anuais/:id/distribuicao"]
  dist --> anual["3. Plano anual (IA)<br/>a prosa pedagogica do ano, gerada e depois reescrita pelo professor<br/>POST .../planos-anuais/:id/gerar"]
  anual --> aulas["4. Aulas, semana a semana (IA)<br/>40 semanas, uma por vez. O esqueleto e do calendario<br/>POST .../periodos/:ordem/gerar"]
  aulas --> exp["Exportacao<br/>montada no NAVEGADOR, sem rota e sem modelo:<br/>.doc (HTML com os namespaces do Office) e impressao"]
  exp --> fim(((fim)))

  dados --> dist

  reg --> cfg[("app_92_configuracoes<br/>e, se a rede ainda nao tinha, periodos_letivos - catalogo do CONTRATO")]
  dist --> jd[("app_92_planos_anuais.distribuicao (JSONB)<br/>o insumo do gerador, e o que sobrevive a reescrever o documento")]
  anual --> doc[("bucket privado: gerado.md (imutavel) + atual.html (sanitizado)")]
  aulas --> ja[("app_92_planos_de_periodo.aulas (JSONB)<br/>dado estruturado, nao prosa")]
  exp --> rec["tres recortes, so das semanas pronto:<br/>esta semana - esta e a proxima -<br/>o periodo de avaliacao inteiro (bimestre, trimestre ou semestre)"]

  %% Onde a IA NAO entra: a extracao da Base; a consulta de habilidades
  %% (exata por componente e ano, sem busca vetorial); a distribuicao; o
  %% esqueleto das aulas; a costura; cumprida e observacaoDoProfessor; e a
  %% exportacao.
  %% Os dois pontos de IA passam por temSaldo ANTES e consumir depois - e o
  %% consumo conta mesmo quando a geracao e rejeitada: o token foi gasto.
```

A.46 Figura 12.3 — app-92: como uma semana nasce

```
flowchart TB
  cal["Calendario (calendario.ts)<br/>numero, data, dia da semana e horario das aulas DESTA semana,<br/>da grade real do professor. Feriado REMOVE a aula, nao a desloca"]
  rec["O recorte da distribuicao<br/>so os codigos do periodo de avaliacao em que esta semana cai"]
  reg["A regencia<br/>componente, serie e minutos por aula"]

  cal --> brief
  rec --> brief
  reg --> brief

  brief["montarBriefingDePeriodo<br/>um briefing com as aulas que EXISTEM e as habilidades que PODEM entrar"]
  brief --> gen["provider.generateJson (IA)<br/>schema de resposta, modelo longo, temperatura 0.5<br/>devolve SO o conteudo pedagogico das aulas"]
  gen --> cost["costurarAulas - a conciliacao entre o que o modelo devolveu<br/>e as aulas que existem de verdade"]
  cost --> out[("app_92_planos_de_periodo.aulas<br/>JSONB estruturado, status = pronto")]
  out --> edita["O professor edita - PUT /planejamento/periodos/:id/aulas<br/>cumprida e a observacao da aula sao registro DELE, nunca do modelo"]

  %% As tres regras da costura:
  %% 1. O esqueleto e do CALENDARIO. Oito aulas para uma semana de tres: a
```

```
  %%    sobra e descartada. Uma para tres: as outras duas nascem em branco.
  %% 2. Codigo fora do recorte e REMOVIDO, nao corrigido.
  %% 3. Campo ausente vira string VAZIA, nunca undefined.
  %% As tres barreiras contra um codigo inventado: o prompt (instrucao, nao
  %% garantia), o PUT .../distribuicao (valida o que o PROFESSOR escolheu) e a
  %% costura (remove o que o MODELO escreveu depois). Nenhuma sozinha basta.
```

A.47 Figuras 13.1 e 13.2 — Mapa de rotas da API

```
%% Figura 13.1 - plataforma e exp08ef1b
flowchart TB
  subgraph PLAT["Plataforma - serve a todos os papeis"]
    au["/auth/login - sem papel"]; he["/healthz, /readyz - PUBLICO"]
    ex["/experiencias - GET qualquer; PUT admin"]
    pg["/progresso/:expId - estudante"]; te["/telemetry/* - ver capitulo 10"]
    us["/usuarios/* - gestor, admin"]; ai["/ai/gerar - autenticado + quota"]
    an["/analise/* - ADMIN-CONTRATO"]
    ro["/rostering/* - gestor, admin"]
    tu["/turmas/* - prof. (as dele, por MATRICULA), gestor"]
    mt["/matriculas/:id - gestor, admin"]
    pll["/periodos-letivos - GET prof.; POST gestor"]
    pp["/perfis-recebidos - gestor, admin"]
    rec["/recursos/* - o interruptor da REDE (ADR-87)"]
    sal["/salas/* - prof. libera, gestor le; /minhas: estudante (ADR-69)"]
    eq["/equipes/* - prof. monta, gestor le; /minha: estudante (ADR-111)"]
    ev["/evidencias/* - prof. confirma e ajusta, gestor le; estudante: so a pergunta e a resposta (
↪ ADR-112)"]
    agp["/agente-professor/* - SO PROFESSOR, posse; a mensagem responde SSE (ADR-126)"]
  end
  subgraph E01["exp08ef1b - Instanooria"]
    atv["/atividades/* - professor"]; rn["/runs/* - estudante (/eventos: professor tb.)"]
    po["/posts/:id/comentarios - estudante"]; pf["/perfis/importar - chave de servico"]
  end
```

```
%% Figura 13.2 - demais experiencias: uma linha por experiencia,
%% colunas = estudante | professor e gestao | guia, agregado e servico
flowchart TB
  subgraph UT["backup-exp04ef1b - UnooTube (arquivada, ADR-116)"]
    ut["sem rotas: fora do app.module"]; utt["era /unootube/turmas/*"]
    utp["era /unootube/professor/*"]
  end
  subgraph VI["app-70 - Vila do Lume"]
    vi["/vila/sessoes - estudante"]; vp["/vila/professor/* - PROFESSOR"]
    va["/vila/agregado/* - prof, gestor"]
  end
  subgraph PL["app-90 - Plano de aula"]
    pl["/planos/* - professor"]; pref["/planos/referencias - chave de servico"]
  end
  subgraph BE["app-91 - Bem-estar"]
    be["/bemestar/* - SO PROFESSOR, com posse"]
  end
  subgraph UG["exp02em1b - unooGPT"]
    ug["/unoogpt/* - estudante"]
    ugp["/unoogpt/professor/* - SO PROFESSOR, transcricao"]
```

```
    uga["/unoogpt/agregado/* - gestor, admin - sem estudanteId"]
  end
  subgraph RB["exp06ef2b - O Rastro do Boato"]
    rb["/rastro-boato/* - estudante"]
    rbt["/rastro-boato/turmas/* - prof, gestor, admin - o gate da aula"]
    rbp["/rastro-boato/professor/* - prof, gestor - material do guia"]
  end
  subgraph MIA["exp08ef4b - O Manifesto da IA"]
    mia["/manifesto-ia/* - estudante"]
    miat["/manifesto-ia/turmas/* - prof, gestor - o debate E o conteudo da turma"]
  end
  subgraph FC["exp01ef1b - O Que a Foto Conta"]
    fc["/foto-conta/* - estudante"]
    fct["/foto-conta/turmas/* - prof, gestor - o latch da Roda"]
  end
  subgraph JE["exp04ef1b - Jornal da Escola (ADR-117)"]
    je["/jornal-escola/* - estudante - e as perguntas em /evidencias"]
    jet["/jornal-escola/turmas/* - prof, gestor - o pacote, os dois portoes e o painel"]
    jep["/jornal-escola/professor/* - prof, gestor - material do guia"]
  end
  subgraph QM["exp05ef1b - Quem mandou isso?"]
    qm["/quem-mandou-isso/* - estudante - e as perguntas em /evidencias"]
    qmt["/quem-mandou-isso/turmas/* - prof, gestor - o pacote E o relogio da turma"]
    qmp["/quem-mandou-isso/professor/* - prof, gestor - material do guia"]
  end

  %% Duas areas por AUSENCIA deliberada: bem-estar nao tem rota de gestor nem
  %% de agregado; e no unoogpt o controller do gestor nao tem NENHUMA rota que
  %% aceite estudanteId, o que torna estrutural (nao disciplinar) o limite do que
  %% ele alcanca.
```

A.48 Figura 14.1 — A linha da revelação

```
flowchart LR
  subgraph CLONE["Dentro do iframe, ANTES da revelacao - @edu/ui/ds-unootube"]
    feed["feed de videos"]; cromo["cromo, busca, menu"]
    play["playlists"]; canal["canal do estudante"]; toast["toast: 'Video salvo!'"]
  end
  subgraph PLATF["DEPOIS da revelacao, no MESMO iframe - @edu/ui/ds"]
    rank["ranking"]; d1["dialogo 1 da missao"]; d2["dialogo 2 da missao"]; fim["fechamento"]
  end
  CLONE ==>|REVELACAO| PLATF

  %% A linha e a REVELACAO, nao o arquivo. O toast NAO cruza: ele fala de
  %% dentro da ficcao. O criterio nao e "e depois no tempo?", e "quem fala?".
  %%
  %% CONTRAEXEMPLO, exp02em1b (unooGPT): nao ha revelacao no software. O vies de
  %% cada fonte e o material do debate do professor, e nenhuma tela o entrega.
  %% Logo a exp02em1b e 100% da secao do clone (@edu/ui/ds-chatgpt) e nunca troca
  %% para a da plataforma. A marca institucional entra so como chrome do HUB,
  %% desenhada ACIMA do iframe.
```

A.49 Figura 15.1 — *Pipeline* de entrega

```
flowchart TB
  push["push para main"] --> ver["PORTAO: ci.yml (typecheck, lint, test, build)"]
  push --> diff["job mudancas: git diff HEAD^ HEAD -> api | hub | exps"]

  subgraph api["job deploy-api (paralelo)"]
    img["imagem da api (camada de deps do pnpm fetch + cache GHA)"]
    img --> mig["Cloud Run Job -migrate (na VPC, --wait)"]
    mig --> dep["gcloud run deploy api (revisao nova)"]
  end

  subgraph hub["job deploy-hub (paralelo)"]
    imgh["imagem do hub"] --> deph["gcloud run deploy hub"]
  end

  subgraph exps["job publicar-exps — DEPOIS do hub e da api"]
    build["turbo build (filtrado, ou tudo se 'forcar')"]
    build --> rsync["gcloud storage rsync -> gs://{bucket}/{exp}/{ver}/"]
    rsync --> cc["Cache-Control: max-age=60 SO no index.html"]
    cc --> man["publicar-manifesto.mjs -> PUT /experiencias/:expId (token de 5 min)"]
  end

  ver --> api
  ver --> hub
  diff --> api
  diff --> hub
  dep --> exps
  deph --> exps

  %% ORDEM 1: migracao ANTES do deploy — o schema novo tem de estar no banco
  %%          antes de a revisao nova receber trafego. --wait faz falhar.
  %% ORDEM 2: experiencias DEPOIS do hub — o PUT do manifesto e validado contra
  %%          o JWKS que o HUB serve. Deploys PULADOS nao bloqueiam
  %%          (!cancelled() + result != 'failure').
```

A.50 Figura 16.1 — Estratégia de testes

```
flowchart TB
  v["PONTA A PONTA - Playwright, navegador real<br/>scripts/e2e por papel: publico, estudante,<br/>professor, gestor, admin-contrato + probe-pointerlock"]
  c["CONTRATO EXTERNO<br/>telemetria-push (guard do Pub/Sub), vertex-provider (cadeia ADC)"]
  i["INTEGRACAO COM POSTGRES<br/>tenancy, planos-tenancy, motor-concorrencia, motor-postgres,<br/>unootube-postgres, vila-postgres, vila-ato1, usuarios, planos, bemestar"]
  u["REGRA PURA - Vitest/Jest, milissegundos, sem banco<br/>vila-regras, unootube-regras, bemestar-regras, motor-pesos,<br/>motor-bolha, motor-sugestoes, motor-reversibilidade, bridge.test"]
  u --- i --- c --- v

  %% A base e larga de proposito: regras.ts puro em todo modulo de dominio.
  %% E o TOPO nao e "E2E de fluxo": e verificacao de que a INTERFACE apareceu.
  %% Icone virando texto, iframe vazio, cor inexistente, fonte removida pelo
  %% Next — tudo INVISIVEL para o compilador.
```

A.51 Figura 17.1 — Cadeia de uma requisição e o que cada elo revela

```
flowchart LR
  nav[Navegador - iframe da exp] --> lb[LB + CDN] --> hub[Cloud Run -frontend] --> api[Cloud Run - api]
  api --> sql[(Cloud SQL)]
  api <--> ps[Pub/Sub]
  api --> vx[Vertex AI]

  nav -.- v0["ve-se: nada<br/>cego: crash do bundle, ponte sem hub:init, token nao renovado, lote perdido"]
  lb  -.- v1["ve-se: contagem e latencia por host<br/>cego: quem foi afetado (sem correlacao)"]
  hub -.- v2["ve-se: log de requisicao, instancias, CPU/mem<br/>cego: null do apiDoServidor, redirect em laco, erro de render"]
  api -.- v3["ve-se: idem + /readyz (se alguem abrir)<br/>cego: causa do 500, latencia da IA, descarte de telemetria, fila em erro"]
  sql -.- v4["ve-se: conexoes, CPU, disco, backlog<br/>cego: query lenta por rota; dead letter (nao existe)"]

  %% Nenhum elo propaga identificador de correlacao.
```

A.52 Figura 17.2 — Erro de cliente entra pela API

```
sequenceDiagram
  participant E as Experiencia (@edu/sdk)
  participant H as Hub (ExpFrame)
  participant A as API (/observabilidade)
  participant L as Cloud Logging / Error Reporting

  E->>E: window.onerror / unhandledrejection
  Note over E: dedupe por hash, max. 5 por sessao, 8 KB
  E->>A: POST /observabilidade/erros-cliente (Bearer)
  Note over A: Zod: expId, versao, mensagem, stack, url
  A->>A: redaction + escola_id do token
  A->>L: stdout JSON, severity ERROR, @type ReportedErrorEvent
  L->>L: agrupa por assinatura; "novo na versao X"
  A-->>E: 204 (ou 429: cota da sessao)
  E-->>H: exp:erro (opcional, targetOrigin explicito)
  Note over H: mostra "algo deu errado" e oferece recarregar
```

A.53 Figura 19.1 — Matriz de riscos

```
quadrantChart
  title Matriz de riscos - probabilidade x impacto
  x-axis "Probabilidade baixa" --> "Probabilidade alta"
  y-axis "Impacto baixo" --> "Impacto alto"
  quadrant-1 "CRITICO - resolver antes de nov/2026"
  quadrant-2 "Impacto alto, prob. baixa"
  quadrant-3 "Monitorar"
  quadrant-4 "Prob. alta, impacto baixo"
```

```
  "R-01 carga nao verificada": [0.55, 0.92]
  "R-02 retencao indefinida":  [0.90, 0.90]
  "R-04 fator humano unico":   [0.98, 0.88]
  "R-06 sem revisao externa":  [0.85, 0.86]
  "R-05 sem alerting":         [0.85, 0.58]
  "R-07 acessibilidade":       [0.85, 0.55]
  "R-12 catalogo de modelos":  [0.82, 0.50]
  "R-03 localidade de IA":     [0.98, 0.55]
  "R-08 particao sem job":     [0.20, 0.50]
  "R-09 midia manual":         [0.85, 0.22]
  "R-10 orfa sem retomada":    [0.50, 0.20]
  "R-11 dependencia do DS":    [0.50, 0.22]
  "R-13 WSO2 nao e o emissor": [0.50, 0.18]
  "R-14 virada do ano letivo": [0.85, 0.90]
  "R-15 senha compartilhada":  [0.52, 0.52]
  "R-16 replay sobre menor":   [0.88, 0.94]
  "R-17 latencia de IA":       [0.55, 0.86]
  "R-18 custo de IA exp02em1b":   [0.96, 0.60]
  "R-19 guia da exp02em1b":       [0.96, 0.84]
  "R-21 texto livre de menor": [0.96, 0.52]
```

Apêndice B

Variáveis de ambiente

Catálogo completo, derivado de `.env.example` e de `apps/api/src/config/env.ts`. Toda variável da API é validada com Zod **no *boot***: valor errado derruba o processo na subida, não no meio de uma requisição de estudante.

REGRA INVIOLÁVEL

Variável marcada **fecha** significa: *ausente* ⇒ a funcionalidade responde 401 ou 503, **nunca** fica aberta. Essa é a convenção de *fail closed* da plataforma (**RNF-09**).

DECISÃO DE PROJETO

Identidade, enturmamento e importação não têm variável nenhuma (ADR-28, ADR-29). É afirmação, não omissão deste apêndice: o período letivo vigente é derivado das *datas* em `periodos_letivos` — sem coluna `ativo` e sem `ANO_LETIVO` —, a senha inicial do lote é escolhida na tela de importação e não fica em configuração, e o teto do CSV está no código (`main.ts` e `rostering.service.ts`). Configuração é lugar de endereço e segredo; regra de negócio em variável de ambiente é regra que ninguém encontra quando ela sai do esperado.
Houve por um dia uma `CPF_PEPPER`, quando o CPF era guardado em *hash*; ela saiu junto com a decisão (ADR-33).

B.1 API — `apps/api/.env`

B.1.1 Banco e identidade

Variável	Default	Papel
`DATABASE_URL`	— (obrigatória)	URL do PostgreSQL. Validada com `startsWith("postgresql://")`. Em produção vem do Secret Manager.

`OIDC_ISSUER`	— (obrigatória)	Emissor aceito. Hoje é o hub; amanhã o WSO2. **Não é segredo** — é URL pública, e vai em texto plano derivada de `hub_domain`, o que garante que `iss` e esperado não possam divergir.
`OIDC_JWKS_URI`	— (obrigatória)	JWKS do emissor.
`OIDC_AUDIENCE`	`plataforma-ap`	Audiência exigida no *token*.
`CORS_ORIGINS`	hub + *dev servers*	Lista explícita, separada por vírgula. **Nunca** `*`. **Toda porta de experiência nova entra aqui** — e no *fallback* do `main.ts`, que já ficou duas versões atrás da realidade: a experiência abre e morre em "`Failed to fetch`", como se a API estivesse fora do ar, e o erro fala de CORS em vez da porta que faltou.
`PORT`	`3001`	Porta HTTP.

ATENÇÃO

O `issuer` do WSO2 é *tenant-qualified* (`.../t/{contrato}/oauth2/token`) e deve ser copiado **literalmente** do *discovery*, nunca montado por concatenação. Um `iss` montado à mão que difere por uma barra faz toda validação falhar com "*issuer* inválido" — e a mensagem não diz qual dos dois lados está errado.

B.1.2 Inteligência artificial

Variável	Default	Papel
`AI_PROVIDER`	`vertex`	`vertex` ou `mock`. Com `mock` a plataforma sobe sem nenhuma credencial de GCP.
`VERTEX_PROJECT_ID`	—	**Obrigatória com `AI_PROVIDER=vertex`**: o *boot* recusa sem ela. Sem projeto, a URL viraria `.../projects/undefined/...` e o erro só apareceria na primeira geração real.
`VERTEX_LOCATION`	`global`	O `3.6-flash` **só** existe em `global` (medido: 404 em `southamerica-east1`, `us-east1`, `us-central1`). Trocar por uma região exige trocar *também* o modelo.
`VERTEX_MODEL`	`gemini-3.6-fla`	Modelo do dia a dia.
`VERTEX_MODEL_LONGO`	`gemini-3.6-flash`	Modelo de saída longa (planos anuais). A variável separada existe para trocar um sem tocar no outro.

`VERTEX_IMAGE_MODEL`	`gemini-3.1-fla`	Arte da exp08ef1b. O antigo `gemini-2.0-flash-exp` **não existe mais** — e por isso toda geração de imagem caía no *fallback* procedural. Quem trocar precisa conferir a proporção devolvida.
`VERTEX_TTS_MODEL`	`gemini-2.5-flash-tt`	Voz (`generateSpeech`). **Nenhuma missão sintetiza voz hoje**: quem usava era o Eco no Grupo (retirado, ADR-113), e a variável ficou com a operação. **Não é o *aiplatform***: a síntese vem da Cloud Text-to-Speech, com os mesmos modelos por trás e o mesmo ADC, e a API precisa estar habilitada no projeto. Com ADC de *usuário* ela exige `x-goog-user-project` — o *provider* manda; sem isso, imagem funciona e voz responde 403 (ADR-67).
`VERTEX_THINKING_BUDGET`	`0`	Tokens de *raciocínio* saem do `maxOutputTokens`. `0` desligado (paridade com o aplicativo de origem); `-1` dinâmico; > 0 teto. Com pensamento livre, o plano anual voltava **vazio**.
`AI_DAILY_TOKEN_QUOTA_PER_*`	**removidas**	A quota diária por perfil **não é *env* desde a ADR-105**: é a tabela `ia_quotas`, semeada pela *migration* (300.000 estudante, 2.000.000 professor) e editada pelo `admin` em `/unooria/uso-de-ia`. Uma *env* aqui seria a segunda fonte de verdade.

> **NOTA**
>
> **Não existe variável para chave de API de IA** em lugar nenhum do projeto — nem aqui, nem em `.env.example`. A credencial é sempre ADC (ADR-16).

B.1.3 Agente de IA do professor (ADR-126)

Variável	Default	Papel
`AGENTE_PROFESSOR_MODO`	`desligado`	`agent-engine` (Vertex AI Agent Engine, ADC) | `local` (o *shim* de `apps/agente-professor`) | `mock` (turno enlatado, o da suíte e2e) | `desligado`. **Fecha**: ausente = `desligado` em QUALQUER `NODE_ENV`, e o chat responde `AGENTE_INDISPONIVEL` (503) — ao contrário do `AI_PROVIDER`, porque o engine nasce por contrato em momentos diferentes e derrubar a api por falta dele tiraria todos os produtos do ar. `mock` e `local` são **recusados em produção** pelo Zod.

AGENTE_PROFESSOR_ENGINE	—	**Obrigatória com `agent-engine`**: o *resource name* COMPLETO (`projects/<p>/locations/<r>/reasoningEngines/<id>`). A região sai dele — uma variável a menos para divergir. O Terraform a preenche no segundo *apply*.
AGENTE_PROFESSOR_URL_LOCAL	—	**Obrigatória com `local`**: a base do *shim* (`http://localhost:8090`; no `docker:dev`, `http://host.docker.internal:8090`).

B.1.4 Armazenamento

Variável	Default	Papel
ASSETS_DRIVER	`local`	`local` (a API serve em `/assets`) ou `gcs` (*bucket* público atrás da CDN).
ASSETS_BUCKET	—	`{contrato}-assets`.
ASSETS_PUBLIC_URL	—	Base pública da CDN de *assets*.
ASSETS_LOCAL_DIR	—	Diretório local em *dev*.
CDN_EXP_URL	—	De onde os *bundles* das experiências são servidos, visto pela API — o mesmo valor do `NEXT_PUBLIC_CDN_EXP_URL` do hub. Serve a UMA coisa: conferir com um HEAD se a versão que o painel está ativando existe no *bucket* antes de gravá-la (**ADR-87**). Apontar a rede para uma versão que não subiu é tela branca para todo mundo, e o erro só apareceria quando a primeira criança abrisse a missão. **Ausente = a conferência é PULADA e a resposta diz isso** (`conferida: false`) — é o caso de *dev*, onde o hub serve do *dev server* do Vite e não há caminho versionado a conferir.
PLANOS_DRIVER	`local`	`local` grava em `storage-privado/`; `gcs` usa o *bucket* **privado**.
PLANOS_BUCKET	—	`{contrato}-planos`. **Privado**, sem CDN, com versionamento de objetos.
PLANOS_LOCAL_DIR	—	Diretório local em *dev*.

B.1.5 Telemetria e chaves de serviço

Variável	Ausente	Papel
TELEMETRIA_TOPIC	console	Vazio em *dev* = eventos apenas *logados*. Não há Pub/Sub local.
TELEMETRIA_PUSH_SA	**fecha (401)**	*E-mail* da conta de serviço que assina o *push* do Pub/Sub.
TELEMETRIA_PUSH_AUDIENCE	**fecha (401)**	Audiência configurada na *subscription*. Sem as duas, `/telemetry/ingest` rejeita tudo.

`PERFIL_IMPORT_TOKEN`	**fecha (503)**	Cabeçalho `x-perfil-token` da ponte externa do Ato 1.
`PLANOS_IMPORT_TOKEN`	**fecha (503)**	Cabeçalho `x-import-token` do *script* de importação de referências curriculares.
`PORTABILIDADE_EMISSOR`	degrada	De quem a api aceita conferir um perfil portátil. O `emissor.hub` vem de DENTRO do arquivo, ou seja, de um desconhecido: sem esta lista a api sairia buscando a URL que um arquivo mandou (SSRF). O documento **seleciona** entre estas origens; nunca introduz uma. Cada entrada é `origem` ou `origem=urlDoJwks` — a segunda porque o `emissor.hub` é o endereço do **navegador** (mesma distinção de `EXP_INTERNAL_HOST`): de dentro do contêiner, `localhost:3000` é a própria api. JWKS por `http` só fora de produção, com aviso no *log*. Vazia = nenhum documento é verificado e todos entram `nao-verificada` — não é fechar a rota, porque isso recusaria um direito do titular (ADR-35).
`SWAGGER_USUARIO`	**some (404)**	Usuário da documentação OpenAPI em `/docs`.
`SWAGGER_SENHA`	**some (404)**	Senha, mínimo 16 caracteres. **Sem as duas, `/docs` e `/docs-json` não são montadas** — quem responde é o 404 do roteador, não um 401. As rotas de serviço acima devolvem 401/503 porque a existência delas já é pública; aqui o ativo protegido É o mapa das rotas, e um 401 confirmaria que ele está atrás daquela porta. **Meia credencial derruba a subida**: uma porta que não abre com alguém achando que configurou a documentação é pior que nenhuma (ADR-36). Em *dev* os dois *compose* trazem um default (`docs` / `unooria-docs-local`) — sem ele o `/docs` só funcionaria para quem descobrisse que precisa exportar variável, isto é, para ninguém. Em produção vêm do Secret Manager.
`SEED_SENHA`	**o seed falha**	Senha das contas de demonstração criadas por `prisma/seed.ts` (mínimo 8 caracteres). **Não tem *default* no fonte**: o seed é idempotente e roda contra o `DATABASE_URL` que estiver no ambiente, e uma constante ali criaria 61 contas com senha conhecida — inclusive o `admin-contrato`, o único papel que atravessa escolas — se ele fosse apontado por engano ao banco de um contrato. O *default* de *dev* vive nos dois *compose*, como o do Swagger acima; no *host*, no `apps/api/.env` (o Prisma o carrega) ou na linha de comando. O seed não imprime mais a senha (ADR-47). Só *dev*: nenhum *pipeline* de entrega roda o seed.

B.1.6 Fuso da plataforma (ADR-99)

Variável	Default	Papel
FUSO_DA_PLATAFORMA	America/Sao_Paulo	O fuso em que a plataforma inteira mostra data e hora, e em que decide o "dia" que grava. Nome **IANA**; validada no *boot* construindo um Intl.DateTimeFormat — valor que o Intl recuse **derruba a subida**. A **mesma** variável, com o **mesmo valor**, vai para o hub (§B, seção do Hub): é ele que a repassa ao navegador e às experiências.

> **ATENÇÃO**
>
> Um *offset* fixo (-04:00) **passa** na validação, porque o Intl o aceita — e é a única forma de configurar errado sem nada acusar: ele congela o deslocamento, e se a rede voltar a ter horário de verão a plataforma fica uma hora atrasada. Use America/Manaus, America/Rio_Branco, America/Belem, America/Noronha — o nome, nunca o deslocamento.
> A outra forma de errar é **configurar a API e esquecer o hub**. O sintoma é o defeito da **ADR-98** de volta: entre as 21h e a meia-noite os dois falam de dias diferentes, o React acusa hydration failed e o clique da professora cai no vazio. docs/INFRA.md pede um variable ÚNICO no Terraform, consumido pelos dois serviços — não duas entradas.

B.1.7 Observabilidade (Capítulo 17)

Validadas pelo Zod de config/env.ts desde 2 de setembro de 2026 (fase 1 da API). Nenhuma é segredo: entram como *env* em texto plano no Cloud Run, injetadas pelo Terraform. O logger raiz lê LOG_LEVEL, GCP_PROJECT_ID e NODE_ENV **do ambiente cru**, antes do Nest — ele nasce na importação, e é usado pelo main.ts e pelo *bootstrap* do OpenTelemetry.

Variável	Default	Papel
LOG_LEVEL	info	Nível mínimo do pino (debug liga o log de *query* do Prisma). Em produção, info; nunca debug por mais de um incidente.
GCP_PROJECT_ID	— (vazio)	Identificador do projeto do contrato, para montar logging.googleapis.com/trace como projects/{id}/traces/{traceId}. Vazio = log sem correlação com o *trace* (dev). Injetada pelo Terraform; também lida pelo hub.
OBS_TRACE_RATIO	0	Fração de requisições com *spans* exportados ao Cloud Trace (0.02 em produção; 0 desliga o exportador, que é o *default* de dev). Rotas de IA e o *feed* usam dez vezes este valor, até 1.
OBS_DEBUG	0	1 imprime *spans* e métricas no console local, sem GCP. Só para desenvolvimento; nunca no Terraform.

`LOG_FORMATO`	— (vazio)	Fora de produção o log sai em `pino-pretty`; `json` força o formato de produção no terminal, para conferir uma linha como o Cloud Logging a verá.

B.2 Hub — `apps/hub/.env.local`

Variável	Escopo	Papel
`NEXT_PUBLIC_API_URL`	**cliente**	Endereço da API que o **navegador** usa. Vai ao `hub:init` como `apiUrl`.
`API_INTERNAL_URL`	servidor	Endereço da API que o *hub* usa (*authorize*, BFF, leitura de manifesto).
`NEXT_PUBLIC_CDN_EXP_URL`	**cliente**	Base dos *bundles*. Em produção, a CDN; em *dev* sem manifesto, cada experiência usa a porta de `lib/experiencias-dev.ts`.
`NEXT_PUBLIC_CLARITY_PR`	**cliente**	Id do projeto Microsoft Clarity (ADR-60). **Ausente = Clarity desligado**, no hub *e* dentro das missões — o hub não renderiza a *tag* e não manda o id no `config` do `hub:init`. É a única `NEXT_PUBLIC_*` que chega ao *bundle* do **cliente**, e por isso é inlinada no `next build`: entra por `--build-arg` no `Dockerfile`, não como env de *runtime* do Cloud Run. Trocar o id exige *rebuild* da imagem.
`OBS_TRACE_RATIO`	servidor	Fração de requisições do PRÓPRIO hub com *spans* exportados ao Cloud Trace (Capítulo 17). `0` desliga. Mesmo gate da API: exige `GCP_PROJECT_ID` ou `OBS_DEBUG=1`.
`OBS_TRACE_RATIO_CLIENT`	servidor	Fração de sessões das EXPERIÊNCIAS com OpenTelemetry Web. Vai no `config.obs.traceRatio` do `hub:init`: env de *runtime*, então mudar a amostragem não republica bundle nenhum. `0` desliga (a SDK nem baixa os pacotes).
`SESSAO_TTL_SEGUNDOS`	servidor	Prazo de INATIVIDADE da sessão, em segundos — **só fora de produção** (ADR-96). Existe para a suíte não esperar duas horas de relógio; piso de 60s, teto de 12h, validada com Zod. Em produção ela é IGNORADA com aviso no *log* e o prazo é a constante de `lib/sessao.ts`: a `/privacidade` afirma esse número em texto para a família, e `lib/sessao.test.ts` prende os dois lados.
`NEXT_PUBLIC_HUB_VERSAO`	**cliente**	O SHA do *build*, inlinado no *bundle* (`ARG` do Dockerfile, como a do Clarity): a "versão" do hub nos erros de CLIENTE do Error Reporting. Ausente (*build* local) fica `dev`.

`EXP_INTERNAL_URL`	servidor	De onde o hub **lê** o `experiencia.json`. Endereço **fixo** — serve quando há *uma* origem de *bundles* (a CDN, em produção).
`EXP_INTERNAL_HOST`	servidor	Troca **só o host** e **preserva a porta**. É o que serve em *dev* e em contêiner, onde cada experiência tem um *dev server* numa porta diferente.
`HUB_ORIGIN`	servidor	*Origin* do hub, passada à experiência por *query string* e usada como `targetOrigin`.
`AUTH_SECRET`	servidor	Segredo de sessão do NextAuth.
`TOKEN_SIGNING_KEY_PEM`	servidor	Par RS256 que assina os *tokens* de experiência. Gerar com `node apps/hub/scripts/gerar-chave-token.mjs`. **Obrigatória em produção**; vem do Secret Manager.
`AUTH_WSO2_ISSUER`	servidor	A **presença** desta variável registra o *provider* OIDC no NextAuth.
`AUTH_WSO2_CLIENT_ID`	servidor	Cliente OIDC do hub.
`AUTH_WSO2_CLIENT_SECRE`	servidor	Segredo do cliente OIDC.
`CONTRATO_ID`	servidor	Identifica o EMISSOR dentro do perfil portátil. Ausente: a emissão responde 503 — documento cujo emissor não se identifica não vale nada na rede de destino, que precisa dele (e do `HUB_ORIGIN`) para achar o JWKS.
`CONTRATO_NOME`	servidor	Nome legível da rede, mostrado a quem receber o documento. Opcional.
`FUSO_DA_PLATAFORMA`	servidor ⇒ **cliente**	O **mesmo** valor da API (**ADR-99**). Lido no servidor e escrito pelo `generateMetadata` do *layout* raiz num `<meta name="unooria-fuso">` — é assim que chega ao navegador, que não lê `process.env`, e de lá ao `config` do `hub:init` de toda experiência. **Não** é `NEXT_PUBLIC_`: aquilo é assado no *build*, e a mesma imagem pode servir mais de um contrato. Valor inválido **não** derruba o hub — cai no *default* com `console.error`, ao contrário da API.

> **ATENÇÃO**
>
> `EXP_INTERNAL_URL` *versus* `EXP_INTERNAL_HOST` é a confusão de configuração mais custosa do projeto, porque o sintoma não aponta para a causa: com um `EXP_INTERNAL_URL` fixo em *dev*, as seis experiências leem o manifesto de *uma só* — e o UnooTube, que não declara `viewport`, aparece **dentro da casca de celular da Instanooria**.

B.3 Experiências — variáveis de *build* do Vite

Variável	Papel
VITE_VIDEOS_BASE_URL	exp04ef1b: base dos 21 arquivos MP4 (52 MB), que ficam fora do *git* e fora do *bundle*. Vazio = `./videos` (`public/videos/`, *gitignorado*, em *dev*). Em produção: `https://<cdn-assets>/exp04ef1b/videos`.

> **NOTA**
>
> **A app-70 não tem nenhuma variável de ambiente.** Ela não usa IA, não usa *bucket* e não tem prazo de retenção declarado — o que é registrado explicitamente em `.env.example`, para que a ausência seja lida como decisão e não como esquecimento.

B.4 Agente de IA do professor — `apps/agente-professor`

Lidas pelo pacote (`agente_professor/config.py`, `servidor.py` e `shim.py`), no Agent Engine — pela `spec` que o Terraform escreve — e no *shim* local.

Variável	Default	Papel
AGENTE_PROJETO	GOOGLE_CLOUD_PROJECT	Projeto dos *data stores* e do Gemini.
AGENTE_MODELO	gemini-3.7-flash	Modelo da conversa.
AGENTE_MODELO_LOCATION	global	Location do modelo.
AGENTE_MODELO_LOCATION_RESERVA	us	Para onde vai quando `global` responde 404, 429 ou 5xx — só antes do primeiro pedaço.
AGENTE_MODELO_ESCOPO	gemini-3.5-flash-lite	Guarda de escopo e título.
AGENTE_BUSCA_LOCATION	global	Location dos *data stores*.
AGENTE_DATASTORE_HABILIDADES	agente-professor-habilidades	
AGENTE_DATASTORE_ACERVO	agente-professor-acervo	
AGENTE_LOG	INFO	Nível de log do servidor.
AGENTE_SHIM_PORTA	8090	Porta do *shim* local.
AGENTE_PROFESSOR_PULAR_SEM_UV	—	`1` pula o pacote no `pnpm turbo` quando a máquina não tem `uv`, com aviso em destaque. Sem ela, a falta do `uv` **falha** o *build*.

> **ATENÇÃO**
>
> **Nunca defina `GOOGLE_CLOUD_LOCATION`, `GOOGLE_CLOUD_PROJECT` nem `PORT` no Agent Engine**: são reservadas, e o engine as preenche (a location vale a região do *deploy*, `us-east1`). O modelo em `global` se configura por `AGENTE_MODELO_LOCATION`; trocar pela reservada manda o modelo para a região do engine.

B.5 `vars` do repositório (CI/CD)

O *workflow* de *deploy* não conhece o nome de nenhum recurso: tudo entra por `vars` do repositório, cujos valores saem de `terraform output`. Há um *script* que os configura (`scripts/configurar-github-`

	Conteúdo
`GCP_PROJECT`	Projeto do contrato.
`REGION`	Região do Cloud Run e do Artifact Registry.
`AR_REPO`	Repositório do Artifact Registry.
`API_SERVICE`, `HUB_SERVICE`	Nomes dos serviços do Cloud Run.
`MIGRATE_JOB`	Nome do Cloud Run Job de migração.
`EXP_BUCKET`	*Bucket* dos *bundles*.
`ASSETS_BASE_URL`	Base pública de *assets* (entra no *build* da exp04ef1b).
`CLARITY_PROJECT_ID`	Id do projeto Microsoft Clarity, por *environment* (`dev`, `hml`, `prd`). O `deploy.yml` o passa como `--build-arg` ao construir a imagem do hub. **Não** sai de `terraform output` — é criada à mão nas *Variables* do *environment*, e ausente significa hub com o Clarity desligado.
`API_URL`, `HUB_ORIGIN`	Endereços públicos, usados pelo *script* de manifesto.
`WIF_PROVIDER`, `DEPLOY_SA`	Identidade federada do CI.
`AGENTE_PROFESSOR_ENGINE`	*Resource name* do Agent Engine, por *environment*. **Vazia** = o engine ainda não existe: o *job* `deploy-agente-professor` só publica a imagem e avisa; preenchida, ele faz o `PATCH` da `imageUri`. Sai de `terraform output agente_professor_engine` depois do segundo *apply*.

Secret do CI	Conteúdo
`TOKEN_SIGNING_KEY_PEM`	A mesma chave RS256 do hub (`{contrato}-token-signing-key` no Secret Manager). O CI a usa para assinar o *token* de cinco minutos do *upsert* de manifesto.

B.6 Configuração mínima para subir sem GCP

Listing B.1 – A combinação que faz **RNF-14** ser verdade

```
# API
DATABASE_URL=postgresql://postgres:pass@localhost:5432/unooria
OIDC_ISSUER=http://localhost:3000
OIDC_JWKS_URI=http://localhost:3000/.well-known/jwks.json
AI_PROVIDER=mock          # sem GCP nenhum; devolve texto de exemplo
AGENTE_PROFESSOR_MODO=mock  # o chat do agente com turno enlatado (ausente: desligado)
ASSETS_DRIVER=local       # a API serve /assets
PLANOS_DRIVER=local       # grava em storage-privado/
TELEMETRIA_TOPIC=         # vazio: eventos so no console
# TELEMETRIA_PUSH_* vazias  => /telemetry/ingest responde 401 (correto em dev)

# HUB
NEXT_PUBLIC_API_URL=http://localhost:3001
API_INTERNAL_URL=http://localhost:3001
NEXT_PUBLIC_CDN_EXP_URL=http://localhost:5173
# NEXT_PUBLIC_CLARITY_PROJECT_ID=  # ausente: Clarity desligado (o normal em dev)
HUB_ORIGIN=http://localhost:3000
AUTH_SECRET=troque-me
TOKEN_SIGNING_KEY_PEM=    # node apps/hub/scripts/gerar-chave-token.mjs --env
```

Apêndice C

Catálogo de *endpoints*

Todas as rotas da API, com o papel autorizado. **Este apêndice é gerado** a partir do `@Contrato` de cada rota (**ADR-36**): a coluna de autorização vem do `@Roles` real, e a semântica, do resumo e das notas declarados no *controller*. Convenções transversais e justificativas estão no Capítulo 13.

Marca	Significado
`papel`	`@Roles(...)` restringe aos papéis listados.
(qualquer)	`OidcGuard` apenas: qualquer papel autenticado.
público	Sem *guard*, deliberadamente.
serviço	Chave em cabeçalho; **503** se a variável estiver ausente.
push	`PubSubPushGuard`; **401** se as variáveis estiverem ausentes.
token no corpo	O *token* vai no corpo, não no cabeçalho — `sendBeacon` não manda cabeçalho.

C.1 a-selecao

Rota	Autorização	Semântica
`POST /a-selecao/concluir`	`estudante`	"Missão concluída!" — idempotente. Leva a sessão a `fim` e carimba `concluidaEm`, no mesmo update. As enquetes não são exigidas (a enquete que falha pula para o fim); a reflexão, sim. **Restrições:** • Antes da reflexão responde **409** `reflexao-pendente`. Concluir de novo devolve o mesmo estado.

Rota	Autorização	Semântica
`POST /a-selecao/decisao`	`estudante`	"Confirmar publicação" — os 3 posts e a legenda nos Stories (latch). O gesto que a missão mede, e ele é IRREVERSÍVEL. A api confere a tríade contra a cobertura da turma, calcula a matriz de enquadramento AQUI (o cliente nunca a manda) e grava carimbo e passo juntos. A legenda passa pelo **sanitizador** antes: outro estudante vai lê-la no Mural. Link, telefone, e-mail, `@` de fora da ficção e palavrão respondem **422** com o código do que tirar — `legenda-com-link`, `legenda-com-telefone`, `legenda-com-email`, `legenda-com-mencao`, `legenda-com-palavrao`. As contas da ficção (`@folhaoficial_`, `@voz_coletiva`, `@metrica_urbana`, `@estudante_gremio`) podem ser citadas. Depois do latch, fora da transação dele, a legenda é graduada para a evidência pedagógica (Canal A). A resposta não diz o resultado: **o estudante não vê o nível**. **Restrições:** • `selecao`: exatamente 3 ids diferentes, de `post_a_1` a `post_c_4`. `legenda`: 10 a 140 caracteres contados em code points, depois de juntar espaços e quebras de linha. (Os `.refine()` somem do OpenAPI; daí esta nota.) • Segunda publicação — inclusive duas simultâneas — responde **409** `latch-ja-confirmado`. Id fora da cobertura da turma responde **400** `selecao-invalida`. • Com o Mural já aberto, quem publica recebe o par do computador dele, ou o colega virtual mais distante (Tomás ou Malu). • A graduação por IA tem teto de ~6 s. Sem saldo, fora do tempo ou com saída inválida, nada é marcado: fica "Não avaliado", e o professor decide.
`GET /a-selecao/gate`	`estudante`	Em que etapa a turma está (polling de ~5 s). Magra de propósito **ADR-93**: sem posts, sem contagem da sala, sem JOIN. É o que deixa a professora conduzir 30 telas — o Mural abre para todos ao mesmo tempo. **Polling HTTP, nunca WebSocket**: a infraestrutura Cloud Run da plataforma não tem WebSocket de aplicação. `agora` é o instante do SERVIDOR. **Restrições:** • Passa pelo MESMO funil de liberação das outras rotas **ADR-69** e **ADR-87**: turma sem a missão liberada recebe 403 em vez de descobrir a etapa. • Não escreve nada. `rodada` e `coberturaId` são `null` só antes da primeira entrada da turma; quando mudam, a tela recarrega a sessão (a professora zerou ou trocou o tema).

Rota	Autorização	Semântica
`GET /a-selecao/mural`	`estudante`	O Mural da Turma: a própria tríade ao lado da do par. Antes das fichas, só imagem e legenda — a tela nua da Fase 1 do PRD. `fichas` e `equilibrio` são `null` e o gabarito NÃO está na resposta. Com a etapa `fichas` (ou depois), cada post ganha fonte ouvida, ângulo da imagem e lente, e cada tríade o equilíbrio editorial em percentuais. O par é o que o pareamento CONGELOU na abertura do Mural: um colega da sala (ou a equipe dele), com o rótulo que a tela escreve, ou o colega virtual. **Restrições:** • Sem publicação responde **409** `stories-nao-publicados`; antes da etapa `mural`, **409** `mural-nao-liberado`. • `par.rotulo` é o primeiro nome (desempatado quando repete na sala), o nome da equipe, ou Tomás/Malu — nunca o nome completo.
`POST /a-selecao/passo`	`estudante`	Registra o passo que a própria criança deu: feed, salvos ou reflexão. Existe para o passo andar no SERVIDOR: um F5 nos salvos não pode devolver a criança à DM da Amara. `feed` vale do desafio ou de volta dos salvos, e só antes do latch. `salvos` só de quem está no feed. `reflexao` só de quem publicou e com a explicação liberada — pedir de novo, ou depois de já ter enviado, devolve o passo em que ela está. **Restrições:** • Depois do latch, `feed` e `salvos` respondem **409** `latch-ja-confirmado`. `salvos` saindo do desafio responde **409** `passo-invalido`. • `reflexao` sem publicação responde **409** `stories-nao-publicados`; antes da etapa `explicacao`, **409** `reflexao-nao-liberada`. • `tempoFeedS` (0 a 3600) é o tempo de feed que a tela mediu. Indicativo, nunca mérito, e só gravado antes do latch.

Rota	Autorização	Semântica
`GET /a-selecao/professor/material`	`professor`, `gestor_escola`, `gestor_regiona` `gestor_contrat` `admin`	O banco curado com o gabarito, o roteiro da Etapa 4 e os colegas virtuais. **Professor only.** Alimenta o bloco vivo do guia da aula. É o BANCO CURADO da Unooria (a Praça Central), não a cobertura da turma: se a professora homologou outro tema, ele está em `GET /a-selecao/turmas/:turmaId/coberturas`. Os dois coexistem de propósito — o guia é escrito sobre o banco curado, e uma turma com tema próprio não reescreve o guia. **Restrições:** • Nada aqui é escopado por escola: é o catálogo da missão, igual para todo contrato. • `roteiroDaEtapa4.aviso` é a proteção do gap: as perguntas não podem ser lidas em voz alta antes de o Mural abrir. • `gatilhos` está chaveado pela etapa de DESTINO do botão — a mesma que o `POST :turmaId/gate` recebe.
`POST /a-selecao/reflexao`	`estudante`	A reflexão individual da Etapa 6 (latch) — leva às enquetes. Texto livre de 100 a 500 caracteres, contados em code points. É caixa escrita por menor de idade: não vai a log nem à telemetria, e só o professor que leciona a turma o lê no painel (LGPD art. 14; recorte da **ADR-108**. Depois dela vêm as duas enquetes da DM da Amara, pelas rotas de `/evidencias/exp07ef1b/perguntas`. **Restrições:** • O `trim` é anterior à contagem: cem espaços são uma reflexão em branco. • Sem publicação responde **409** `stories-nao-publicados`; antes da etapa `explicacao`, **409** `reflexao-nao-liberada`; a segunda, **409** `reflexao-ja-enviada`.

Rota	Autorização	Semântica
`GET /a-selecao/sessao`	`estudante`	O estado da missão e os 12 posts do feed, sem o gabarito. Boot da experiência e o F5. Os posts saem **na ordem do feed** e **sem 'enquadramento', 'tagFonte' e 'tagAngulo'** — o post público é uma allowlist no servidor, então o gabarito não tem campo onde caber. Ele só desce no `GET /a-selecao/mural`, com as fichas acesas. **A missão não nasce vazia**: sem homologação, a turma roda o banco curado da Praça Central, e a cobertura `padrao` nasce na primeira entrada. `imagem.url` é `null` enquanto a professora não gerou as imagens — a tela desenha o card com o `alt`. **Restrições:** • `passo` já vem reconciliado com a etapa da turma; `passoDoEstudante` é o gravado. A etapa só conduz quem PUBLICOU: antes do latch, a criança segue no feed ou nos salvos com o Mural aberto ou não. • `coberturaId` é a ATIVIDADE e `rodada` é a rodada dela: mudam quando a professora troca o tema ou zera a turma, e a sessão é outra. • Missão não liberada na turma responde **403** `missao-nao-liberada` (ou `recurso-indisponivel`). Estudante sem turma responde **409** `sem-turma`.
`GET /a-selecao/turmas`	`professor`, `gestor_escola`, `gestor_regional`, `gestor_contrato`, `admin`	Turmas para o seletor do painel. Filtradas por MATRÍCULA para o professor; o gestor vê a escola. `gestor_contrato` precisa dizer `?escolaId=`.
`GET /a-selecao/turmas/{turmaId}/coberturas`	`professor`, `gestor_escola`, `gestor_regiona` `gestor_contrat` `admin`	As atividades da turma: a em andamento, COM o gabarito, e a lista de todas. **Professor only**: é aqui que `enquadramento`, `tagFonte` e `tagAngulo` de cada post aparecem — são o roteiro de quem conduz. `historico` lista todas as coberturas (a em andamento inclusa, com `ativa: true`): cada uma é uma ATIVIDADE, e o `id` dela é o `atividadeId` das equipes e do painel de habilidades **ADR-130**. `ativa` é `null` quando nenhuma está em andamento: a plataforma não cria atividade. **Restrições:** • `rodadas` é a maior rodada com sessão daquela cobertura; 0 = nunca rodou.

Rota	Autorização	Semântica
`GET /a-selecao/turmas/{turmaId}/coberturas/{coberturaId}`	`professor`, `gestor_escola`, `gestor_regional`, `gestor_contrato`, `admin`	UMA atividade, inteira e COM o gabarito — em andamento ou pronta. A página da atividade. `rodada` e `etapa` só vêm quando ela está em andamento; `rodadas` é a maior rodada com sessão (0 = nunca rodou). **Restrições:** • Cobertura de outra turma, ou que não existe, responde **404** `atividade-nao-encontrada`.
`POST /a-selecao/turmas/{turmaId}/coberturas/{coberturaId}/em-andamento`	`professor`, `gestor_escola`, `gestor_regiona`, `gestor_contrat`, `admin`	“Usar esta atividade agora” — a professora escolhe o que a turma faz **ADR-130**. A escolhida passa a ser a ATIVIDADE EM ANDAMENTO da turma; a que estava volta a ser pronta — fica, com as sessões e as rodadas. A etapa volta à exploração. A RODADA: a atividade que nunca rodou começa na 1; a que já rodou abre a SEGUINTE à última — retomar a mesma rodada depois do Mural seria o “voltar” que o botão único proíbe. As rodadas anteriores continuam no painel de habilidades. **Restrições:** • **Idempotente**: pôr no ar a que já está no ar devolve `mudou: false` e não muda nada. • Trocar com a aula correndo leva quem está jogando para a atividade nova: a experiência nota o `coberturaId` novo no polling do gate e recarrega. A tela pede confirmação antes. • Cobertura de outra turma responde **404** `atividade-nao-encontrada`.

Rota	Autorização	Semântica
`POST /a-selecao/turmas/{turmaId}/gate`	`professor`, `gestor_escola`, `gestor_regional`, `gestor_contrato`, `admin`	O botão único da aula: “Liberar Mural”, “Revelar fichas”, “Liberar conclusão”. O corpo diz a etapa de DESTINO. Anda uma etapa por vez — `exploracao → mural → fichas → explicacao` —, nunca pula e nunca volta; destino igual ao atual é aceito e não muda nada (o segundo clique). Abrir o **Mural** é o momento do PAREAMENTO: na mesma transação, cada computador que publicou recebe o par de matriz mais distante; quem sobra, quem não tem par divergente e a sala de um computador só recebem o colega virtual (Tomás ou Malu). `pareadas` diz quantas sessões receberam par nesta chamada. **Restrições:** • Destino fora de ordem responde **409** `transicao-de-etapa-invalida`, com `etapaAtual` e `proximaEtapa` no corpo. • É a linha que o polling de 5 s do estudante lê (`GET /a-selecao/gate`): não há WebSocket de aplicação **ADR-93**. • Sem atividade em andamento na turma, **403** `atividade-nao-preparada`: a plataforma não cria atividade — a professora cria e põe em andamento (`POST :turmaId/coberturas/:coberturaId/em-andamento`).

Rota	Autorização	Semântica
`POST /a-selecao/turmas/{turmaId}/gerar-conteudo`	`professor`, `gestor_escola`, `gestor_regiona` `gestor_contrat` `admin`	Gera um RASCUNHO dos 12 posts com IA, sobre o tema da professora (pré-aula). **Não persiste nada**: o que volta é rascunho para a tela editar, e publicar é o `POST homologar`. Sem saldo de IA, com o provedor fora do ar ou com a saída fora do molde, volta o BANCO CURADO com `usouIa: false` e `origem: "padrao"` — e a tela diz que o tema padrão foi carregado, em vez de fingir que a IA respondeu. **Restrições:** • Quota do PERFIL do professor **ADR-105**: `temSaldo` antes, e o que o vendor cobrou é debitado por tentativa, inclusive as recusadas. • O lote gerado é recusado — e regerado uma vez — fora do 4:4:4, com legenda fora de 20 a 45 palavras ou com termo que antecipe a revelação (“viés”, “enquadramento”, “lado da história”…). • `podcast` traz as três falas do Bloco 2 do podcast de recapitulação — o que a Folha Oficial, a Voz Coletiva e a Métrica Urbana mostraram NESTE tema —, escritas junto com o lote e sob a mesma régua (12 a 60 palavras, até 400 bytes, sem link, e-mail nem @). No banco curado, são as do sheet. • Os tetos de `tema` (5 a 100) e `foco` (até 100) são a primeira barreira contra injeção de instrução no gerador.

Rota	Autorização	Semântica
`POST /a-selecao/turmas/{turmaId}/gerar-midia`	`professor`, `gestor_escola`, `gestor_regional` `gestor_contrato` `admin`	Gera as próximas imagens dos posts de uma atividade — incremental. Doze imagens não cabem nos 30 s que o BFF dá a uma chamada: cada chamada gera ATÉ DUAS imagens novas, grava as URLs na cobertura e devolve o progresso. **Repita enquanto 'completo' for 'false' e a chamada anterior tiver gerado alguma** — `falharam` diz o que não saiu, e esses posts seguem com o card da descrição. Idempotente: o que já tem URL é pulado, e a imagem de mesma descrição que já está no bucket entra sem custo. Os posts de métricas do banco curado não pedem imagem: a experiência desenha o gráfico. **Restrições:** • `?coberturaId=` diz a atividade (a pronta, preparada antes de ir ao ar); sem ele, a em andamento — e sem nenhuma, **403** `atividade-nao-preparada`. • Quota do professor, com custo ESTIMADO por imagem (a geração não devolve contagem de tokens). Sem saldo responde **429** `sem-saldo-de-ia` — e a missão roda igual, com os cards. • Duas chamadas simultâneas para a mesma cobertura recebem o mesmo trabalho (lock de processo).

Rota	Autorização	Semântica
`POST /a-selecao/turmas/{turmaId}/gerar-podcast`	`professor`, `gestor_escola`, `gestor_regiona` `gestor_contrat` `admin`	Sintetiza o próximo trecho do podcast e, com os cinco prontos, emenda o episódio — incremental. Os cinco blocos não cabem numa chamada de 30 s: cada chamada sintetiza UM trecho (a voz é o TTS do Gemini pelo provider da api, `generateSpeech`, Lucas e Clara numa chamada de diálogo) e, quando os cinco existem, emenda o episódio num MP3 só. **Repita enquanto 'completo' for 'false' e a chamada anterior tiver gerado um trecho** — `falhou` diz qual bloco não saiu, e a transcrição continua na tela. Idempotente: o trecho que já está no bucket é pulado. O nome de cada trecho é o hash do texto e do pedido de voz, então os blocos fixos saem UMA vez por contrato e só o Bloco 2 de um tema da IA custa síntese nova. **Restrições:** • `?coberturaId=` diz a atividade; sem ele, a em andamento. • Quota do professor, com custo ESTIMADO por trecho (a síntese é cobrada por caractere e não devolve contagem). Sem saldo responde **429** `sem-saldo-de-ia`. • Duas chamadas simultâneas para o mesmo episódio recebem o mesmo trabalho (lock de processo).

Rota	Autorização	Semântica
`POST /a-selecao/turmas/{turmaId}/homologar`	`professor`, `gestor_escola`, `gestor_regional`, `gestor_contrato`, `admin`	"Criar atividade" — a curadoria humana **ADR-66**/67). Grava os 12 posts como a professora os deixou, numa ATIVIDADE NOVA que nasce **pronta, não no ar ADR-130**: a em andamento continua em andamento, e quem escolhe o que a turma faz é a professora (`POST :turmaId/coberturas/:coberturaId/em-andamento`). O corpo carrega os posts INTEIROS porque curadoria é editar: a professora pode reescrever qualquer legenda antes de aprovar. **Restrições:** • O lote é recusado com **400** `lote-invalido` e a LISTA de problemas em `erros`, em frases para a professora: 12 posts; ids de `post_a_1` a `post_c_4`, cada um uma vez; bloco `a` = `@folhaoficial_`/institucional, `b` = `@voz_coletiva`/comunitário, `c` = `@metrica_urbana`/estatístico (4:4:4); legenda de 20 a 45 palavras; nenhum termo que antecipe a revelação na legenda, na descrição da imagem ou nos comentários. (`.superRefine()` some do OpenAPI; daí esta nota.) • As imagens de post com a MESMA descrição de uma cobertura recente da turma são herdadas — o arquivo no bucket é o hash da descrição. • `podcast` é obrigatório: as três falas do Bloco 2 que a professora revisou (no tema curado, as do sheet). Fora de 12 a 60 palavras, acima de 400 bytes, ou com link, e-mail ou @, o 400 `lote-invalido` diz qual fala.

Rota	Autorização	Semântica
`GET /a-selecao/turmas/{turmaId}/painel`	`professor`, `gestor_escola`, `gestor_regiona` `gestor_contrat` `admin`	O painel da aula: o botão da próxima etapa e quem publicou, POR COMPUTADOR. Uma linha por computador quando a turma montou equipes para esta missão (`colapsarEmUnidades`), uma por criança quando não — `resumo.porComputador` diz qual. `resumo.computadores` é o "X de Y computadores publicaram": o Y sai da montagem das equipes, porque o colapso conta também quem faltou. `proximaEtapa` é o rótulo do botão único (`null` depois da explicação). **Sem nível, nota ou acerto por criança** (`PROIBIDO_NO_PAINEL`, com teste). O nível da EF07CO11 mora no painel de habilidades, em `/evidencias/turma/:turmaId/missao/exp07ef1b/atividade/:coberturaId`. **Restrições:** • `legenda` e `reflexaoTexto` são texto livre de menor e só vêm preenchidos para quem LECIONA a turma. Gestor, gestor de contrato e admin recebem `null` e leem `publicou` e `entregouReflexao` (recorte da **ADR-108**. • Sem atividade em andamento na turma, **403** `atividade-nao-preparada`: a plataforma não cria atividade — a professora cria e põe em andamento (`POST :turmaId/coberturas/:coberturaId/em-andamento`).
`GET /a-selecao/turmas/{turmaId}/podcast`	`professor`, `gestor_escola`, `gestor_regional` `gestor_contrato` `admin`	O podcast de recapitulação de uma atividade: o roteiro e o estado do áudio. O podcast é do PROFESSOR, para tocar à turma depois da aula se quiser. `roteiro` é o texto inteiro (a transcrição, que fica na tela junto do player): os cinco blocos do sheet aprovado, com as três falas do Bloco 2 que a cobertura trouxe. `url` é o episódio, quando os cinco trechos já foram sintetizados e emendados; `null` enquanto não. **Não gera nada** — é o `POST gerar-podcast`. **Restrições:** • `?coberturaId=` diz a atividade; sem ele, a em andamento — e sem nenhuma, **403** `atividade-nao-preparada`.

Rota	Autorização	Semântica
`POST /a-selecao/turmas/{turmaId}/zerar`	`professor`, `gestor_escola`, `gestor_regiona` `gestor_contrat` `admin`	"Zerar a turma" — abre a próxima rodada, sem apagar nada. Sobe a rodada da atividade e volta a etapa para a exploração. **Nada é apagado**: as sessões, as tríades, os pares e as reflexões da rodada anterior ficam, e são elas que o painel de habilidades mostra como a rodada passada **ADR-112**. A cobertura também fica — o preparo custou tempo da professora. **Restrições:** • Repetir o mesmo tema com a mesma turma mede memória, não leitura (PED-13): a tela avisa antes, e o guia recomenda criar um tema novo, que é OUTRA atividade. • Sem atividade em andamento na turma, **403** `atividade-nao-preparada`: a plataforma não cria atividade — a professora cria e põe em andamento (`POST :turmaId/coberturas/:coberturaId/em-andamento`).

C.2 agente-professor

Rota	Autorização	Semântica
`GET /agente-professor/conversas`	`professor`	As conversas do professor na escola ativa, a mais recente primeiro. Sai do Postgres (`agente_professor_conversas`), que é a fonte da POSSE — o Agent Engine não confere o usuário numa sessão pedida por nome. Trocar de escola troca a lista: a conversa pertence a (professor, escola ativa). **Restrições:** • `titulo` é `null` até o fim do 1º turno: quem escreve o título é o agente (`titulo_conversa` no estado da sessão).

Rota	Autorização	Semântica
`POST /agente-professor/conversas`	`professor`	Abre uma conversa nova com o agente de IA do professor. Cria a sessão no Agent Engine com o estado `nome_professor` (primeiro nome de `users.display_name`, lido do banco) e expiração distante, e grava a linha da posse. Não chama o modelo — não consome quota. **Restrições:** • **503** `{ codigo: "AGENTE_INDISPONIVEL" }` quando o agente não responde (ou `AGENTE_PROFESSOR_MODO=desligado`). • **403** `{ codigo: "recurso-indisponivel" }` com o produto `agente-professor` desligado para a escola.
`GET /agente-professor/conversas/{id}`	`professor`	Uma conversa: título e mensagens, montados da sessão no agente. Confere a posse (professor E escola) antes de tocar a sessão. As mensagens são o texto do professor e o texto FINAL do agente, sem os parciais do stream; `fontes` vem dos retornos de `buscar_acervo` daquele turno. **Restrições:** • Conversa de OUTRO professor, ou de outra escola, é **404** — nunca 403, que confirmaria que o id existe. • Se a sessão sumiu do agente (expirou), a linha é apagada e a resposta é 404.
`DELETE /agente-professor/conversas/{id}`	`professor`	Apaga a conversa: a sessão no agente e a linha da posse. Escapa do interruptor (`@SemDisponibilidade`): com o produto desligado, o professor continua podendo apagar o que é dele. Sessão que já não existe no agente não é erro. **Restrições:** • Agente fora do ar: **503** e a linha FICA — apagar só a linha deixaria a conversa guardada no agente sem ninguém capaz de alcançá-la. • Conversa de outro professor ou de outra escola: **404**.

Rota	Autorização	Semântica
`POST /agente-professor/conversas/{id}/mensagens`	`professor`	Envia uma mensagem e recebe a resposta do agente em SSE. Responde `text/event-stream` com os eventos `delta` / `fontes` / `titulo` / `aviso` / `fim` / `erro` de `docs/regras/agente-professor.md`, e `: ping` a cada 15 s. O stream termina em `fim` OU em `erro`. Teto de 240 s (`TEMPO_ESGOTADO`); fechar a conexão aborta a chamada ao agente. **Restrições:** • `texto`: 1 a 4000 caracteres depois do `trim` — fora disso, **400** JSON. • **429 JSON ANTES do stream**: `{ codigo: "RATE_LIMIT" }` acima de 10 mensagens por minuto, `{ codigo: "QUOTA" }` sem saldo diário de IA (perfil `professor`, `ia_quotas`). • Os tokens consumidos são a soma dos eventos NÃO parciais do turno, contados mesmo se o professor fechar o painel. • Conversa de outro professor ou de outra escola: **404** JSON, antes do stream.

C.3 ai

Rota	Autorização	Semântica
`POST /ai/gerar`	*(qualquer)*	Conversa com o tutor de IA da experiência. Única porta de IA para o cliente. O prompt de sistema é escolhido no servidor pelo `expId` e **nunca sai daqui** — segredo em bundle de experiência é regra 2. **Restrições:** • Quota diária POR PERFIL (o teto é a linha do perfil em `ia_quotas`, editável pelo `admin` em `/unooria/uso-de-ia` — **ADR-105**, contador persistido em `ia_uso_diario`: `temSaldo` ANTES, `consumir` depois. Sem saldo, **429**. • A série do estudante vai ao prompt RESOLVIDA da turma do token (`turmas.serie` → "4º ano"), nunca vinda do corpo. Sem turma, a linha some do prompt.

C.4 analise

Rota	Autorização	Semântica
`GET /analise/atividades/{id}`	`gestor_contrato` `admin`	O detalhe de uma atividade, com a eficácia da escrita por perfil. `eficaciaDaEscrita` é **a única métrica do sistema capaz de refutar a premissa do produto**: o dwell dos posts de um perfil entre estudantes daquele perfil, menos o dwell dos MESMOS posts entre estudantes de outro — sempre com a exposição no denominador. Sem o denominador, mediria o próprio viés do feed (que serve mais posts do perfil do estudante) e "confirmaria" a premissa por construção. Se o delta for zero, as fichas de linguagem não funcionam — e a gente descobre com número em vez de achismo.
`GET /analise/csv`	`gestor_contrat` `admin`	Export do nível de produto — por run ou por evento. Dois níveis porque as perguntas são diferentes: **por run** responde "como as turmas se comparam"; **por evento** é a matéria-prima para quem quer recalcular tudo fora daqui, inclusive contestando as definições. Um sistema de análise que não deixa auditar o próprio cálculo não deveria ser levado a sério. **Restrições:** • Teto de 200.000 eventos no nível `evento`.
`GET /analise/resumo`	`gestor_contrato` `admin`	Uma linha por atividade — o nível de produto. Guarda de papel própria: `gestor_contrato` não é professor nem estudante, e as guardas existentes recusariam os dois. **O professor NÃO alcança esta rota.**

C.5 atividades

Rota	Autorização	Semântica
`GET /atividades`	`professor`	As atividades DESTE professor. Posse por `professor_id` + `escola_id`: atividade é autoral. `?turmaId=` estreita para as daquela turma — é como a SALA DE AULA resolve para onde o link "Conduzir com esta turma" leva **ADR-100**.
`POST /atividades`	`professor`	Cria a atividade em rascunho, com os defaults do motor. **Restrições:** • `qtdPostsTese` precisa ser múltiplo de 8 (2 lados × 4 perfis) para a matriz fechar sem célula quebrada — **recusar é melhor que arredondar em silêncio**. É um `.refine()`, invisível no esquema.

Rota	Autorização	Semântica
`GET /atividades/{id}`	`professor`	O detalhe de uma atividade. `_count` espelha o `select` do Prisma da página de origem — o porte dos painéis consome exatamente esse shape —, e os agregados nomeados (`estudantes`, `postsAprovados`) ficam junto.
`DELETE /atividades/{id}`	`professor`	Exclui a atividade. Devolve o título e quantas sessões foram junto — "pronto" sem número deixa o professor sem saber se apagou a atividade certa.
`GET /atividades/{id}/analise`	`professor`	A análise pedagógica da turma — identificada. É a turma DELE, e o professor precisa saber com quem falar. Só o professor DONO da atividade lê. A peça central é `conclusaoPorLadoSemeado`: se o lado em que o feed nasceu prevê a conclusão que o estudante escreveu, o resultado experimental da turma está nessa grade — e é ela que o professor projeta no momento-espelho. Tudo o mais na tela existe para explicar essa grade. **Restrições:** • `imersao.total` nunca aparece sem os componentes — regra do doc, e vale para o relatório, o monitor e a análise. • `ociosoHaMs` é `null` quando a run acabou: ninguém precisa ser cutucado por estar "ocioso" num relatório. • `colecao` conta o ÚLTIMO evento por post (salvar e dessalvar se anulam); `interacoes.salvos` conta eventos.
`GET /atividades/{id}/analise/csv`	`professor`	A mesma análise da turma, em planilha. O nome do arquivo sai no `content-disposition`. O caminho **não** termina em `.csv` porque no app de origem segmento de rota com ponto quebrava a validação de tipos do App Router — mantido igual, porque é o path que as telas portadas chamam. **Restrições:** • BOM UTF-8 **e** `charset=utf-8`: um sem o outro ainda quebra acento em alguma planilha.
`PATCH /atividades/{id}/antagonismo`	`professor`	O professor aprova (ou reescreve) os dois lados e os neutros. Nada é gerado antes desta aprovação: os dois lados e as justificativas de eixo são o que decide de que lado cada estudante começa, e isso não sai da IA direto para a turma. **Restrições:** • `forcar: true` passa por cima do medidor de equilíbrio, de propósito e explicitamente.

Rota	Autorização	Semântica
`POST /atividades/{id}/ativar`	`professor`	Ativa a atividade para os estudantes entrarem. Devolve a MATRIZ 2 lados × 4 perfis: é ela que diz se a geração fechou. Célula vazia significa que um perfil de um lado não tem post — e o feed daquele estudante nasceria torto. **Restrições:** • `forcar: true` ativa com a matriz incompleta, de propósito — e a resposta devolve `forcado: true` para a tela dizer isso.
`POST /atividades/{id}/destilar`	`professor`	Destila a pergunta-chave e os dois lados a partir do recorte. O professor pode mandar a `perguntaChave` pronta — e aí `perguntaOrigem` sai como `professor`. Sem ela, a IA destila e a origem é `ia`.
`GET /atividades/{id}/distribuicao`	`professor`	Onde cada estudante vai cair — antes de começar. Existe para o professor **ver antes**. A distribuição de perfis varia por sala, e aí os lados nascem desequilibrados: não é defeito do sistema, é a turma — mas o professor precisa saber antes, não descobrir no meio da aula. **Restrições:** • `semLado` são os que caem no SORTEIO (perfil ausente ou `misto`): não entram na projeção porque o lado só é decidido quando a run nasce. • `precisaRedistribuir` é `desequilibrio > 0.15`.
`GET /atividades/{id}/estudantes/{runId}`	`professor`	Tudo o que UM estudante fez, durante a atividade. As interações vêm com os **salvos primeiro**: é a coleção que o estudante vai defender no debate. **Restrições:** • `doLadoSemeado` é a definição de COERÊNCIA: o post é do lado em que o feed dele nasceu? • `nomeNoApp` é apelido dentro da ficção do Instanooria — nunca identidade em tela de professor ou relatório.
`GET /atividades/{id}/etapa`	`professor`	Etapa atual e quantos estudantes em cada tela. É com isto que o painel decide se dá para avançar a turma.

Rota	Autorização	Semântica
`POST /atividades/{id}/etapa`	`professor`	O RELÓGIO DA AULA: move a turma inteira de momento. Escreve a etapa na atividade e leva as runs junto. `voltou: true` quando a turma RETROCEDE de etapa. **A etapa é PISO indo para frente e TETO voltando** — o oposto de propósito: o piso existe para não punir quem leu mais rápido dentro do momento liberado; voltar é outra intenção, e com a regra do piso o retorno mudaria o rótulo do painel e nada na tela dos estudantes. **Restrições:** • `foraDoAparelho` conta quem ficou no onboarding: não entrou e não pode ser movido. • Sair da investigação CONGELA quem entrou no aparelho (computa o dossiê e grava `finished_at`) — sem isso o relatório abre vazio. • **Voltar para a investigação REABRE** quem volta ao feed: apaga o dossiê e zera `finished_at`, e `reabertas` diz quantos. O relatório é recomputado no encerramento seguinte, a partir dos eventos, que não são tocados. Ver **ADR-92**. • Quem JÁ VIU a revelação (`explicabilidade` em diante) não volta ao feed: para na resposta, que dá para refazer. O motivo é o instrumento — quem sabe do viés não pode ser medido por ele de novo. • O que o estudante produziu (eventos, posts salvos, comentários, conclusão, reflexão) NUNCA é apagado aqui. Quem apaga é `POST /runs/:id/reset`.
`GET /atividades/{id}/geracao/progresso`	`professor`	O progresso da geração — o alvo do polling da tela. As imagens são contadas em três estados porque `fallback` **não é erro**: a arte procedural entra no lugar e o feed nunca fica com buraco.
`POST /atividades/{id}/gerar`	`professor`	Enfileira a geração dos posts. **Duas respostas, e a diferença importa**: `retomado: true` quando havia uma geração parada no meio (nada novo foi enfileirado), e `lotes: N` quando ela começou agora.
`PATCH /atividades/{id}/ia`	`professor`	Liga ou desliga a IA ao vivo durante a aula.
`POST /atividades/{id}/limpar`	`professor`	Apaga as sessões dos estudantes, mantendo a atividade e os posts. É o botão de rodar a mesma atividade com outra turma.

Rota	Autorização	Semântica
`GET /atividades/{id}/monitor`	`professor`	O painel da turma ao vivo. **'ociosoHaMs' é o único campo que pede AÇÃO** — é o número que faz o professor levantar da cadeira. Tudo o mais é retrato. **Restrições:** • `ladoDominante: "indefinido"` sai como `neutro`: o painel não tem coluna para "ainda não dá para dizer". • A imersão nunca vem só com o total — os componentes vão junto, e essa é a regra em toda tela.
`GET /atividades/{id}/perfis`	`professor`	O estado do perfil de cada estudante, para conferência. **Restrições:** • `origem: "ausente"` = a criança ainda não tem perfil; ela cai no sorteio.
`POST /atividades/{id}/perfis`	`professor`	Importa perfis à mão — o fallback do professor. Aceita **duas formas**: escores completos (colados de planilha) ou só o `dominante` (seleção manual na lista). A segunda existe porque, na prática, o professor às vezes tem só o resultado final do jogo impresso. **Restrições:** • `username` que não existe nesta escola volta em `desconhecidos` — dito, nunca engolido.
`POST /atividades/{id}/perfis/da-vila`	`professor`	Traz os perfis da Vila do Lume para a largada do Instanooria. Colapsa o perfil Schwartz da app-70 nos quatro quadrantes e grava com `origem: "jogo"` — é o que decide de que lado o feed nasce. O disparo é do PROFESSOR: importar sozinho desfaria em silêncio uma redistribuição feita antes da aula. **Restrições:** • O colapso tem duas travas que CUSTAM estudantes, de propósito: valor não observado (`indicative`/`none`) fica **fora da média**, nunca entra como zero; e quadrante sem nenhum membro observado **invalida o perfil inteiro**. • Sem perfil, o Instanooria cai no sorteio balanceado — que é honesto sobre não saber. • Exige `amostraSuficiente`: o `dominante` vira coluna no CSV do `/analise` ao lado da identidade da criança. • `ladoForcado` é PRESERVADO no upsert.

Rota	Autorização	Semântica
`GET /atividades/{id}/posts`	`professor`	A grade de revisão do professor, paginada e filtrável. A ordenação é `(created_at, id)`, e o `id` **não é enfeite**: os posts entram por `createMany` numa transação, então o lote inteiro compartilha o mesmo `created_at`. Com a chave empatada, `LIMIT/OFFSET` não tem ordem definida — a mesma linha vem em duas páginas e outra nunca vem (medido no app de origem: 207 de 208 visíveis, de forma reprodutível). **Restrições:** • A `matriz` 2×4 existe para o professor ver de relance se alguma linguagem ficou fraca: os lados podem estar 50/50 e um perfil inteiro estar vazio. • 24 posts por página.
`POST /atividades/{id}/redistribuir`	`professor`	Rebalanceia a turma quando ela cai quase toda num lado. Duas regras definem o comportamento: 1. **Move os de afinidade mais fraca primeiro** — `misto` antes de todos, depois por margem crescente. Preserva a pedagogia onde o sinal do perfil é forte. 2. **Só age em quem ainda não começou.** Trocar o lado de quem já está rolando o feed invalidaria a sessão: os scores acumulados passariam a contar para o lado errado. **Restrições:** • `bloqueados` conta quem já começou e por isso ficou — o rebalanceamento pode não ter chegado ao alvo, e o professor precisa saber. • Pergunta e lados não aprovados ⇒ **409**.

C.6 auth

Rota	Autorização	Semântica
`POST /auth/login`	**público**	Autentica uma credencial da rede. Consumida pelo `authorize()` do NextAuth no hub — **nunca pelo navegador**. A api é a única dona do banco; o hub não conhece hash nenhum. Devolve as escolas em que a pessoa tem lotação ativa: professor de rede estadual tem mais de uma, e o token carrega UMA. **Restrições:** • Conta com `ativo = false` é recusada com a MESMA mensagem genérica de senha errada — dizer "conta desativada" confirmaria que o login existe. • `username` é único por ESCOLA, não por contrato: a senha desambigua, e empate entre escolas é recusado. • 8 tentativas por minuto por usuário (429). Não há limite por IP: a api só enxerga o IP do hub.
`POST /auth/trocar-senha`	**público**	Primeiro acesso: troca a senha provisória. A senha do lote (ou a temporária de um reset) vale uma vez. Exige a senha ATUAL — é o mesmo nível de prova do login, e por isso a rota não pede token. Limpa `senha_provisoria` **ADR-29**. O hub refaz a sessão com a senha nova depois disto: o cookie antigo ainda diz "precisa trocar". **Restrições:** • A senha nova precisa ser diferente da atual (400). • 8 tentativas por minuto por usuário (429).

C.7 bemestar

Rota	Autorização	Semântica
`POST /bemestar/checkins/{id}/respostas`	`professor`	Responde as três frases do dia. No **7º** check-in a api soma por subescala (×2) e classifica pelas faixas publicadas — correção server-side (regra 3) —, e o ciclo fecha com `resultado` preenchido. **Restrições:** • **Sinal de apoio**: item 17 ou 21 ≥ 2, ou termo de risco no texto ⇒ mensagem FIXA (CVV 188, SAMU 192, CAPS/UBS) e **a IA não é chamada**. • Responder duas vezes no mesmo dia é **409** com `codigo: JA_RESPONDIDO`. • O texto livre nunca vai a log nem à telemetria — só contagens.

Rota	Autorização	Semântica
`DELETE /bemestar/dados`	`professor`	Apaga TUDO do bem-estar do professor logado. Direito de eliminação (LGPD). Apaga de verdade — ciclos e check-ins —, e a resposta diz quanto sumiu.
`GET /bemestar/historico`	`professor`	Os ciclos concluídos e as anotações — só do próprio professor. Posse por `professor_id` em toda query, com RLS por cima. **Não existe rota de gestor nem agregado** em lugar nenhum: DASS-21 é dado sensível de saúde (LGPD art. 11), e só o professor lê o dele.
`GET /bemestar/hoje`	`professor`	O check-in de hoje — o já respondido, o pendente, ou um novo. **3 frases por dia; 7 check-ins fecham os 21 itens** do DASS-21. Cria o pendente se ainda não houver, e nesse caminho chama a IA para o entorno. Os 21 itens são FIXOS e *verbatim*: a IA nunca reescreve um item — reformular quebra a validação psicométrica e as faixas de corte. O que ela gera é a abertura do dia, a pergunta aberta e o acolhimento. **Restrições:** • Todo texto de IA passa por `validarTextoDaIa` (sem vocabulário clínico, ordens, promessas, gestão, links). Barrado, sem saldo ou sem provider ⇒ texto fixo do catálogo: **o check-in nunca deixa de abrir**. • A IA nunca recebe nome, escola, pontuação ou faixa — só dia da semana, mês, nº do check-in e o texto livre, delimitado como conteúdo. • 1 por dia, em `America/Sao_Paulo`. O ciclo expira em 21 dias sem fechar.
`GET /bemestar/resumo`	`professor`	Estado leve do check-in, para o card do hub. **Não abre ciclo e não chama IA** — é o que o card do hub lê a cada carregamento, e um card não pode custar uma geração.

C.8 equipes

Rota	Autorização	Semântica
`GET /equipes/minha`	`estudante`	A equipe do estudante nesta missão: o nome dela e o primeiro nome dos colegas. É o que o hub mostra antes de abrir a missão — "você está na Equipe 3, com a Bia e o Caio" — para a criança saber de qual computador ela é. **Sempre 200**: sem equipe, `equipe: null`. Um 404 aqui viraria `api-recusou` no log do hub a cada abertura de missão, para a imensa maioria das crianças, que não tem equipe. **Restrições:** • `colegas` leva SÓ o primeiro nome dos outros membros — o nome completo de outra criança não tem o que fazer nesta tela (regra 6). Quando o primeiro nome repete dentro da equipe (o próprio estudante incluso), sobe um degrau por vez só até desempatar: "Gabriela C.", depois "Gabriela Cardoso". • `expId` ausente ou fora do formato responde **400**; missão que não aceita equipe responde `equipe: null`. • A turma sai da matrícula e do cache (`turmasDoEstudante`), a mesma da porta da sala.
`GET /equipes/turma/{turmaId}/missao/{expId}/atividade/{atividadeId}`	`professor`, `gestor_escola`, `gestor_regiona` `gestor_contrat` `admin`	As equipes de uma atividade nesta turma: a montagem, as equipes e quem está sem equipe. Equipe é quem DIVIDE UM COMPUTADOR na missão. `semEquipe` são os estudantes da turma fora de qualquer equipe — os ausentes da montagem inclusive. Missão que não aceita equipe responde **200** com `permitida: false` e listas vazias, e não 400: é por esta resposta que o hub decide se desenha o bloco de equipes. **Restrições:** • Turma em que o professor não tem matrícula responde **404**, não 403: um "sem permissão" já contaria que a turma existe. • Gestor e admin leem a escola inteira, de propósito — e não montam. • Membro que saiu da turma não aparece: a fonte dos estudantes é `users.turma_id`, a mesma dos painéis de missão. • A equipe é da ATIVIDADE **ADR-130**: `atividadeId` é o id que a missão dá (a cobertura, o pacote, a atividade do Instanooria). Atividade que não é desta turma responde **404** `atividade-nao-encontrada`; a missão sem atividade (o unooGPT) usa o id fixo `turma`.

Rota	Autorização	Semântica
`PUT /equipes/turma/{turmaId}/missao/{expId}/atividade/{atividadeId}`	`professor`	Grava a montagem inteira das equipes desta atividade, com concorrência otimista. **Só quem tem matrícula de professor na turma.** Troca a montagem INTEIRA: as equipes anteriores saem e as do corpo entram, na ordem em que vieram. `versao` é a versão em que a tela se baseou (`0` quando não havia montagem). Versão velha responde **409** com `versaoAtual` — o computador da sala e o celular do professor salvando ao mesmo tempo não se sobrescrevem em silêncio. **Restrições:** • Montagem impossível responde **400** `montagem-invalida` com `erros`: a lista INTEIRA, em frases para o professor — membro ou ausente de fora da turma, estudante em duas equipes, ausente dentro de equipe, nome vazio, repetido (sem distinguir maiúsculas) ou com mais de 30 caracteres, equipe sem ninguém, nenhuma equipe. • Missão que não aceita equipe responde **400** `missao-sem-equipes`; fora do catálogo, **404**; ferramenta de professor, **403**; recurso desligado na rede, **403** `recurso-indisponivel`. • `individual` (um aluno por computador) grava a montagem SEM equipes e com `computadores` nulo; `sorteio` e `manual` exigem `computadores` e ao menos uma equipe. • A equipe é da ATIVIDADE **ADR-130**: `atividadeId` é o id que a missão dá (a cobertura, o pacote, a atividade do Instanooria). Atividade que não é desta turma responde **404** `atividade-nao-encontrada`; a missão sem atividade (o unooGPT) usa o id fixo `turma`.
`DELETE /equipes/turma/{turmaId}/missao/{expId}/atividade/{atividadeId}`	`professor`	Desfaz as equipes desta atividade na turma. A montagem sai, e as equipes e os membros caem junto. **Idempotente**: apagar o que não existe devolve o mesmo `ok`. **Restrições:** • Sem checagem de catálogo, de disponibilidade nem de que a atividade ainda exista: desfazer é sempre seguro, e missão desligada na rede não pode deixar equipes presas.

Rota	Autorização	Semântica
`POST /equipes/turma/{turmaId}/missao/{expId}/atividade/{atividadeId}/sugestao`	`professor`	Uma proposta de equipes (sorteio ou N equipes vazias) — nunca grava. O `individual` não tem proposta. `sorteio` distribui os presentes (a turma menos `ausentes`) por `computadores` equipes de tamanhos que diferem em no máximo 1. `manual` devolve N equipes VAZIAS e todos os presentes em `semEquipe`, para o professor arrastar. Se a missão tem lados (o Instanooria), o sorteio evita juntar os dois lados na mesma tela e conta em `agrupamento.equipesMistas` quando a aritmética não fecha. Sem fonte de lado, `agrupamento` volta `null`. **Restrições:** • Missão que não aceita equipe responde **400** `missao-sem-equipes`. • Com mais computadores que presentes, cada criança fica numa equipe só dela — é a sala com um computador por estudante. • Ausente que não é da turma é ignorado aqui; o `PUT` o recusa. • A equipe é da ATIVIDADE **ADR-130**: `atividadeId` é o id que a missão dá (a cobertura, o pacote, a atividade do Instanooria). Atividade que não é desta turma responde **404** `atividade-nao-encontrada`; a missão sem atividade (o unooGPT) usa o id fixo `turma`.
`GET /equipes/turma/{turmaId}/missao/{expId}/atividade/{atividadeId}/ultima`	`professor`	As equipes da atividade anterior desta turma, reprojetadas sobre a turma de hoje. A montagem mais recente da MESMA turma em QUALQUER OUTRA atividade (desta missão ou de outra) que tenha equipes — a `individual` não conta. Quem saiu da turma cai (e é contado em `saidas`), equipe que ficou vazia some, e quem chegou depois fica em `semEquipe`. Os ausentes de lá não vêm: falta é da aula. Sem montagem anterior, `origem` volta `null` e todos os estudantes em `semEquipe`. Nada é gravado. **Restrições:** • Missão que não aceita equipe responde **400** `missao-sem-equipes`. • A equipe é da ATIVIDADE **ADR-130**: `atividadeId` é o id que a missão dá (a cobertura, o pacote, a atividade do Instanooria). Atividade que não é desta turma responde **404** `atividade-nao-encontrada`; a missão sem atividade (o unooGPT) usa o id fixo `turma`.

C.9 escolas

Rota	Autorização	Semântica
`GET /escolas`	`gestor_contrato`, `admin`	Todas as escolas do contrato, com contagem. Só `gestor_contrato` chega aqui. O filtro `paiId` é o insumo da lotação em lote: ele resolve "as unidades desta diretoria" em LISTA, que é o que a tela mostra para o admin confirmar.
`POST /escolas`	`gestor_contrat` `admin`	Cria uma escola. **'gestor_contrato' e ninguém mais**, inclusive no `orgs.csv`: escola é *tenant*, e abrir um inquilino com RLS e dados de menores por trás não é operação de quem administra uma unidade. **Restrições:** • Id já existente é **400**. • **'tipo: escola' exige 'paiId'** de uma regional existente **ADR-79**: sem ele o 400 traz o texto do produto. `regional` entra sem pai. • `pai_id` liga unidade a regional, e um trigger no banco recusa pai que seja `escola`, `rede` ou a própria linha.
`PATCH /escolas/{id}`	`gestor_contrato` `admin`	Edita os dados da escola. `id` e `tipo` não se editam: os dois circulam em token e em RLS. É por aqui que a regional padrão ganha o nome da rede — ela nasce `Rede` porque o nome real do contrato não existe no banco, e quem sabe qual é é quem administra. **Restrições:** • **Escola não volta a ficar sem regional**: `paiId` vazio numa `escola` é 400. Mudar de regional é permitido — é assim que uma unidade troca de diretoria.

Rota	Autorização	Semântica
`DELETE /escolas/{id}`	`gestor_contrat` `admin`	Exclui uma regional. **Só regional.** Escola é *tenant* e não se exclui por rota nenhuma — a saída de uma unidade é assunto de contrato, não de um botão numa tabela. Duas recusas, as duas de produto ADR-79: a rede **nunca fica sem regional** (409 `regional-unica`), e regional com escolas só sai dizendo para onde elas vão (409 `regional-com-escolas`, com `escolas` e `destinos` no corpo — é o que a tela usa para abrir o diálogo de transferência ou para explicar que não há destino). **Restrições:** • A regional PADRÃO não tem proteção própria: depois que a rede se organizou em diretorias de verdade, ela é a sobra que se quer remover. O que a segura é a mesma regra da última regional. • A transferência e a exclusão são UMA transação: falhar no meio não deixa escola apontando para regional apagada.
`POST /escolas/{id}/ lotacoes`	`gestor_escola`, `gestor_regional` `gestor_contrato` `admin`	Lota alguém nesta escola. Para professor e gestor é **ACRÉSCIMO**: a lotação que a pessoa tem em outra escola da rede continua valendo — é o caso que a rede estadual tem aos milhares, e a razão de `lotacoes` existir. **Restrições:** • `gestor_contrato` não recebe lotação: ele é a exceção cross-escola.
`GET /escolas/{id}/ pessoas`	`gestor_escola`, `gestor_regiona` `gestor_contrat` `admin`	Quem está lotado nesta escola. Gestor vê a própria; `gestor_contrato`, qualquer uma. `papel` diferente de `estudante`/`gestor_escola` cai em `professor`. **Restrições:** • O CPF vem FORMATADO e visível: o gestor confere identidade contra o sistema da rede e atende a família no balcão ADR-33.

Rota	Autorização	Semântica
`POST /escolas/lotacoes-em-lote`	`gestor_contrato` `admin`	Lota uma pessoa em MUITAS escolas de uma vez. É assim que o supervisor de uma Diretoria de Ensino nasce: um `gestor_escola` lotado nas unidades dela, **sem papel novo e sem tocar na RLS** — o token continua carregando UMA escola. Idempotente: reaplicar reabre o que estava encerrado e não duplica. **Restrições:** • Recebe a LISTA de escolas, nunca o critério (“todas da regional X”): autorizar por hierarquia seria uma segunda fonte de verdade sobre acesso, divergente da lotação no dia em que alguém corrigisse um pai. • `regional` e `rede` entram em `ignoradas`: não têm turma nem dado, e virariam escola fantasma no seletor do cabeçalho. • Uma transação POR escola — cinquenta numa só seguraria conexão por segundos, e falhar na décima não é motivo para desfazer as nove.
`GET /escolas/minhas`	`estudante`, `professor`, `gestor_escola`, `gestor_regiona` `gestor_contrat` `admin`	As escolas de QUEM ESTÁ PEDINDO. Declarada antes de `:id` de propósito: no Nest a primeira rota que casa vence. É com ela que o hub monta o seletor de escola do professor de duas unidades — e **valida a troca** antes de assinar um token com a escola nova. A validação é sempre contra esta rota, nunca contra a lista que o cookie carrega: senão bastaria pedir uma escola qualquer. **Restrições:** • Barata de propósito: está no caminho do LOGIN. Contagens ficam em `/escolas/minhas/resumo`. • Professor e gestor ACUMULAM lotação; estudante tem UMA ativa, garantida por índice único parcial no banco.

Rota	Autorização	Semântica
`GET /escolas/minhas/relatorio`	`gestor_regional`, `gestor_contrato`, `admin`	Relatório da diretoria: consolidado e por escola. As DUAS medidas lado a lado, porque medem coisas diferentes: `estudantes` é a CRIANÇA com registro na missão; `turmasLiberadas`/`turmasEncerradas` é a AULA. Turma encerrada com poucos registros é aula que passou sem a sala entrar; muitos registros com turma aberta é sala no meio do caminho. **Atravessa escolas SEM afrouxar a RLS**: `liberacoes` e `progresso` estão sob a política por escola, e o serviço abre a tranca uma escola por vez, somando na aplicação — o mesmo padrão de `lotacoes-em-lote` e da transferência. **Restrições:** • `gestor_regional` lê só a própria regional: o `?regionalId=` dele é IGNORADO, não validado — aceitar o parâmetro seria dar a diretoria do vizinho por querystring. • **Nome de criança nunca aparece**: o relatório é contagem. Para ver uma criança, abre-se a escola dela. • **Bem-estar não entra**, e não é omissão: é dado de saúde, sem rota de gestor nenhum (nem do contrato) por decisão registrada. • O teto é uma transação por escola, em lotes de 8. Diretoria muito grande ficar lenta é gatilho para agregado materializado — não para afrouxar a política.
`GET /escolas/minhas/resumo`	`professor`, `gestor_escola`, `gestor_regiona` `gestor_contrat` `admin`	O panorama de quem tem muitas escolas. A tela do supervisor de diretoria. **LISTA, não soma**: uma linha por unidade, com turmas, estudantes e quantos ainda não fizeram o primeiro acesso. Somar escolas numa métrica só seria autorizar por hierarquia — exatamente o que a **ADR-34** recusa. O supervisor não é papel novo: é um `gestor_escola` lotado nas unidades da diretoria.

Rota	Autorização	Semântica
`GET /escolas/profissionais`	`gestor_contrato` `admin`	Busca professor e coordenação na REDE inteira. Por nome, login, matrícula ou CPF (com ou sem pontuação), **sem exigir a escola de origem**. Existe por um DEFEITO, não por conforto: a busca anterior era `GET /usuarios?escolaId=`, que filtra `users.escola_id` — o *cache* da lotação PRINCIPAL. Quem trabalha em duas escolas só era encontrado pela principal; procurando pela outra, a lista voltava vazia e quem administra concluía que a pessoa não existe. Era justamente a população que a lotação em lote existe para administrar. **Restrições:** • Devolve as escolas em que a pessoa JÁ está — é com elas que a tela marca "já vinculado a esta escola" e deixa esse destino fora da criação. • **Não devolve CPF.** A busca o aceita, mas o número numa lista que varre a rede inteira é superfície nova — login e escolas já distinguem homônimos **ADR-33** restringe o CPF visível ao balcão da escola). • Mínimo de 2 caracteres na busca, e 5 dígitos para casar CPF: sem piso, o primeiro clique no campo viraria varredura de `users`. • Sem `comTenancy`, e não é exceção nova: `users` e `lotacoes` já estão fora da RLS, e quem pergunta é o único papel que atravessa escolas.

C.10 estudantes

Rota	Autorização	Semântica
`GET /estudantes/{id}/historico`	`professor`, `gestor_escola`, `gestor_regional` `gestor_contrato` `admin`	O histórico do estudante ATRAVESSANDO escolas. A única leitura fora de `gestor_contrato` que atravessa escolas — e por isso é uma porta só. Alcança apenas quem tem lotação na escola **onde o estudante está HOJE**: é a unidade responsável por ele. **Restrições:** • Devolve PARTICIPAÇÕES, **nunca o conteúdo dos instrumentos**: o retrato de valores da Vila e a transcrição do unooGPT continuam atrás das telas de quem tem a posse pedagógica na escola onde aquilo aconteceu. • A escola de HOJE é a PRIMEIRA da lista `escolas` — a tela imprime nessa ordem, e a primeira linha é a que o adulto lê como "a escola dela".

Rota	Autorização	Semântica
`GET /estudantes/{id}/perfil-portatil`	`gestor_escola`, `gestor_regiona` `gestor_contrat` `admin`	Monta o documento SEM assinatura, para o hub assinar. A api monta; **o hub assina**, porque é ele que tem a chave e publica o JWKS. Os campos `emissor` e `emitidoEm` são preenchidos na assinatura, e por isso não vêm daqui. Transferir é dentro do contrato; o perfil portátil é para FORA dele — a criança que vai da rede estadual para a municipal sai da instalação, e não há garantia de que o destino comprou o produto ADR-35. **Restrições:** • Mesma tranca de `GET /estudantes/:id/historico`: quem pede precisa de lotação na escola onde a criança está HOJE. • **Sem CPF não há perfil portátil** (400): numa rede de destino nenhum id daqui existe, e o CPF é o que reconhece a criança. • Bem-estar (app-91) **nunca** entra: documento que o gestor emite não carrega o que nem o gestor lê. • Sem `CONTRATO_ID` no hub, a emissão é **503**.
`GET /estudantes/{id}/perfis-recebidos`	`gestor_escola`, `gestor_regional` `gestor_contrato` `admin`	Os documentos que esta escola já recebeu sobre esta criança. Contagens, não conteúdo: a tela diz o que veio sem despejar o documento inteiro. **Restrições:** • O que vem de fora vive SÓ em `perfis_portateis`: não vira matrícula, progresso nem perfil da Vila, e não entra em média. É a afirmação de outra instalação.
`POST /estudantes/{id}/transferir`	`gestor_escola`, `gestor_regiona` `gestor_contrat` `admin`	Transfere o estudante entre escolas da rede. A escola de DESTINO é a de quem pede: a unidade que recebe a criança que a família matriculou nela. `gestor_contrato` pode mover para qualquer uma. **O dado que o estudante produziu NÃO se move** — continua com o `escola_id` de onde aconteceu, que é verdade histórica. O que segue o estudante é a identidade ADR-32. **Restrições:** • É a única operação que troca o `app.escola_id` no MEIO de uma transação: sem isso o update na origem não veria linha nenhuma (a RLS o esconderia, sem erro) e a criança ficaria nas duas. • `jaEstava: true` quando o destino já era a escola atual — nada foi movido.

C.11 evidencias

Rota	Autorização	Semântica
`GET /evidencias/{expId}/perguntas/{perguntaId}`	`estudante`	Uma pergunta de múltipla escolha da ficha da missão — sem o gabarito. O enunciado e as alternativas (`id` e `texto`). O nível que cada alternativa vale fica no servidor (regra 3): quem corrige é a api, na resposta. As alternativas saem EMBARALHADAS, numa ordem nova a cada chamada: a ficha as guarda na ordem da régua, e servida assim "a primeira é a certa" vira a pista. A tela mostra na ordem que recebeu e responde pelo `id` — nunca pela posição. **Restrições:** • Pergunta que não existe na ficha da missão responde **404** `pergunta-nao-encontrada`. • Missão não liberada na turma do estudante responde **403** `missao-nao-liberada` (ou `recurso-indisponivel`, se a rede não tem o recurso).
`POST /evidencias/{expId}/perguntas/{perguntaId}/resposta`	`estudante`	A resposta do estudante a uma pergunta — a api corrige, e só devolve o recibo. O corpo leva SÓ a alternativa. A atividade e a rodada saem do contexto da missão, no servidor — nunca do cliente. Vale a PRIMEIRA resposta; as seguintes são aceitas e ignoradas, e as duas respondem o mesmo `{ enviada: true }`: a criança não descobre, pela resposta, se acertou nem se já tinha respondido. **Restrições:** • Sem atividade em andamento na missão responde **409** `sem-atividade-em-andamento`. • Missão não liberada na turma do estudante responde **403** `missao-nao-liberada`. • Pergunta fora da ficha responde **404** `pergunta-nao-encontrada`; alternativa que não é dela, **400** `alternativa-desconhecida`.
`GET /evidencias/estudante/{estudanteId}`	`professor`, `gestor_escola`, `gestor_regional`, `gestor_contrato`, `admin`	O perfil de evidência de um estudante: cada missão, cada atividade, o nível de cada habilidade. Em cada atividade, a MELHOR rodada por habilidade — só com o que vale: sem a projeção que o professor ainda não confirmou, sem N/A e sem `nao_participou`. Sem origem: quem lê o perfil quer saber o que a criança demonstrou, não de que computador veio. O estudante NÃO tem rota para ver o próprio resultado. **Restrições:** • Estudante que o professor não ensina (nenhuma turma dele) responde **404**, não 403. • Gestor e admin leem qualquer estudante da escola.

Rota	Autorização	Semântica
`GET /evidencias/turma/{turmaId}/missao/{expId}/atividade/{atividadeId}`	`professor`, `gestor_escola`, `gestor_regiona` `gestor_contrat` `admin`	O painel da evidência de uma atividade: cada estudante, cada rodada, cada habilidade da BNCC. Para cada estudante da turma: a equipe dele nesta missão, a evidência de cada habilidade em cada rodada (`porRodada`, com `status`, `origem` e `situacao`) e o `resultado` — a MELHOR rodada por habilidade, só com o que vale. Habilidade ausente é **N/A** (“Não avaliado”). `pendentes` conta, por rodada, as cópias da projeção que esperam o professor confirmar. Missão sem ficha pedagógica responde **200** com `temFicha: false` e tudo vazio, e não 400: é por esta resposta que o hub decide se desenha o bloco. **Restrições:** • Turma em que o professor não tem matrícula responde **404**, não 403: um “sem permissão” já contaria que a turma existe. • Gestor e admin leem a escola inteira, de propósito — e não confirmam nem ajustam. • Sem ranking: a ordem dos estudantes é a alfabética, e nenhum campo soma níveis.
`PATCH /evidencias/turma/{turmaId}/missao/{expId}/atividade/{atividadeId}/rodadas/{rodada}/ajustes`	`professor`	Ajusta a evidência de um ou vários estudantes numa rodada. **Só quem tem matrícula de professor na turma**, a qualquer momento — mesmo com a missão encerrada. O ajuste é a origem `professor`: nada o sobrescreve depois (nem a marca da criança, nem a cópia da equipe). `nao_participou` é o N/A explícito: barra a cópia da equipe e não conta no perfil. Toda mudança vai para o histórico com de/para, quem e quando; ajuste que repete o que já estava é ignorado. Devolve o painel já atualizado. **Restrições:** • Turma em que o professor não tem matrícula responde **404**, não 403: um “sem permissão” já contaria que a turma existe. • Estudante de fora da turma, habilidade fora da ficha ou o mesmo par duas vezes respondem **400** `ajuste-invalido` com `erros`: a lista INTEIRA, em frases para o professor. • Rodada depois da última que a atividade teve responde **400** `rodada-inexistente` (a rodada 1 sempre existe). • Missão sem ficha pedagógica responde **400** `missao-sem-ficha`.

Rota	Autorização	Semântica
`POST /evidencias/turma/{turmaId}/missao/{expId}/atividade/{atividadeId}/rodadas/{rodada}/confirmar`	`professor`	Confirma para a turma as cópias da projeção desta rodada. Com a turma inteira diante de um computador só (a projeção), o que a missão viu é copiado para todos como `a_revisar` — e NÃO conta até o professor confirmar. Este botão passa todas as desta rodada a `valida`. Depois, o professor ainda pode ajustar cada criança. Devolve o painel já atualizado. Repetir não muda nada. **Restrições:** • Turma em que o professor não tem matrícula responde **404**, não 403: um "sem permissão" já contaria que a turma existe. • Missão sem ficha pedagógica responde **400** `missao-sem-ficha`.

C.12 experiencias

Rota	Autorização	Semântica
`GET /experiencias`	*(qualquer)*	O catálogo de experiências do contrato. Tabela GLOBAL, sem RLS: manifesto de versões não é dado de estudante. O hub monta a URL do iframe com `versaoAtiva`, e filtra por série na interface. **Restrições:** • `series: []` quer dizer TODAS as séries, não nenhuma — é como uma experiência transversal se declara (app-70).
`GET /experiencias/{expId}`	*(qualquer)*	Uma linha do manifesto. Devolve `experiencia: null` quando não há linha — e isso não é erro: o hub cai no `experiencia.json` servido ao lado do bundle. É o caso de dev e o de uma stack nova, antes do primeiro publish do CI.

Rota	Autorização	Semântica
`PUT /experiencias/{expId}`	`gestor_contrato` `admin`	Publica ou faz rollback de uma versão. Chamada pelo CI ao publicar `gs://{contrato}-exp/{expId}/{versao}/`, com o conteúdo do `public/experiencia.json` da experiência. **Rollback é trocar 'versaoAtiva' para a anterior** — não há deploy a desfazer, porque o bundle antigo continua no bucket. **Restrições:** • `permissoes` é POR EXPERIÊNCIA: prender o ponteiro do estudante é capacidade de quem declarou, nunca default do frame. • `produto` é o que põe a experiência no painel da Unooria **ADR-87** e o que lhe dá o TETO do produto. Ausente = ela responde só pelo próprio interruptor; quem cobra o campo é o teste dos manifestos, no CI, e não esta rota — 400 aqui mataria a ativação DEPOIS de os bundles subirem. • Cada PUT grava também uma linha em `experiencias_versoes` com o MANIFESTO INTEIRO. É o que faz o rollback pela tela devolver `viewport` e `permissoes` daquela versão, e não os de hoje. • `viewport` ausente ⇒ o hub NÃO embeda dentro da casca de aparelho (a exp04ef1b foi desenhada para desktop).

C.13 foto-conta

Rota	Autorização	Semântica
`POST /foto-conta/feed`	`estudante`	Etapa 2: registra a exploração do feed e abre a decisão. Curtidas e permanência são grau `none` (cosmético/motor): familiarização com o clone, sem peso avaliativo — o dashboard mostra a média da turma e nada mais.
`GET /foto-conta/galeria`	`estudante`	O mural da turma (pós-publicação): as fotos dos colegas, chegando. Estritamente passiva (irreversibilidade pós-missão do handoff): sem curtir, sem editar, sem republicar. Quem assina cada post é a MARCA (coruja/estrela/sol) — nome de criança não existe nesta resposta. **Restrições:** • O cliente atualiza por polling lento (~10s); a galeria é vitrine, não canal.

Rota	Autorização	Semântica
`GET /foto-conta/latch`	`estudante`	O polling do latch (3s): a turma já foi liberada? **ADR-94** Leve de propósito, como o gate do o-eco-no-grupo: sem pacote e sem catálogo — é o que deixa 35 tablets perguntarem a cada 3 segundos sem pesar. `agora` é o instante do SERVIDOR: o relógio do tablet da escola atrasa, e não é ele que decide o momento da aula. **Restrições:** • Polling HTTP curto, e não WebSocket/SSE: decisão da **ADR-94** — não existe canal persistente de aplicação nesta infraestrutura, e o teto de 4s de reação docente da banca é atendido com folga.
`POST /foto-conta/marca`	`estudante`	Etapa 1: grava a marca escolhida e abre o mural. **Restrições:** • Seleção unária: a marca nova substitui a anterior só ANTES de confirmar — depois de entrar no mural ela é a identidade da ficção e não muda mais.
`POST /foto-conta/publicar`	`estudante`	Etapa 3 (o PICO): publica o par foto+legenda e devolve o eco dos colegas. A escolha é validada CONTRA O PACOTE da turma (id fora do pacote é payload forjado, não clique) e os comentários voltam **deterministicamente** — o mesmo par devolve sempre o mesmo Leo e a mesma Sofia, e um F5 não muda a cena (não-negociável 1 do handoff). Repetir a chamada com o MESMO par devolve a publicação existente (toque duplo e retry de rede não criam segundo post); par DIFERENTE responde `409 publicacao-ja-feita` — publicar é irreversível por desenho. **Restrições:** • `latenciaMs` é cronometrada pelo CLIENTE: o dado é o tempo de decisão da criança, não a latência da api. Teto de 2h (aba esquecida). • A cadência do eco (Leo a 1,5s, Sofia a 3,0s) é do CLIENTE, fixada em constante — o servidor entrega os textos de uma vez.

Rota	Autorização	Semântica
`POST /foto-conta/reflexao`	`estudante`	Etapa 6: a síntese metacognitiva — o servidor decide, sem nota. O cliente manda `lupa` ou `globo` e NUNCA sabe de antemão qual conclui (regra 3): `conclui: true` celebra e congela a run; `false` é o retry formativo — o card pisca sem vermelho punitivo, Nina acolhe e a criança escolhe de novo, sem teto de tentativas e sem penalidade. **Restrições:** • Exige o latch liberado (`409 roda-em-andamento` antes disso): responder antes da Roda quebraria a mediação presencial que a missão existe para provocar.
`POST /foto-conta/roda`	`estudante`	Etapa 4 → 5: o eco terminou; a run entra no latch da Roda de Conversa. **Restrições:** • Daqui em diante o tablet é porta-retrato: a tela não tem botão nenhum e quem anda é o professor, liberando a turma inteira pelo painel.
`GET /foto-conta/sessao`	`estudante`	O boot da missão: estado da run + o pacote da aula, sem a matriz do Aha. O `pacote` sai **sem 'comentarios'** — o tipo do servidor é um `omit` do schema completo, então a surpresa do Aha é impossível de vazar aqui, não só ausente por descuido. Os comentários chegam um par por vez, na resposta de `POST /foto-conta/publicar`. A run da tentativa corrente é criada no primeiro toque; um F5 devolve exatamente o estado gravado (reentrada determinística, bloco 4 do handoff). **Restrições:** • Estudante sem turma responde 409 `sem-turma`: a missão é síncrona por turma e não há aula para entrar. Quem explica a resolução é a tela do HUB, fora do disfarce. • `latchLiberado` já vem no boot para o F5 no meio da roda renderizar direto a tela de espera — ou de reflexão, se o professor liberou durante a queda.
`GET /foto-conta/turmas`	`professor`, `gestor_escola`, `gestor_regional`, `gestor_contrato`, `admin`	As turmas de quem pergunta, para o seletor do painel. Filtradas por MATRÍCULA para o professor; o gestor vê a escola. Perfis de contrato precisam dizer `?escolaId=`.

Rota	Autorização	Semântica
`GET /foto-conta/turmas/{turmaId}`	`professor`, `gestor_escola`, `gestor_regiona` `gestor_contrat` `admin`	O dashboard ao vivo: distribuição A/B, latch e a lista de hesitação. É o alvo do polling do painel durante a aula (~5s). `hesitando` traz id E nome — o nome é hidratado AQUI, da matrícula autenticada da turma (DAD-07): o modelo da run, a telemetria e o canal de status só conhecem o id opaco. Sem ranking, sem nota, sem ordem de velocidade: as métricas proibidas do handoff não existem neste payload por desenho. **Restrições:** • “Hesitação” é run parada na etapa de decisão há mais de 60s — a lista de mediação de leitura, não um alerta de desempenho.
`POST /foto-conta/turmas/{turmaId}/aprovar-pacote`	`professor`, `gestor_escola`, `gestor_regional` `gestor_contrato` `admin`	Aprova o pacote conferido no preview — o `cfg_turma_ready` da spec. A api REVALIDA o pacote inteiro com o MESMO schema da geração: aprovação é o último portão antes da turma, não um carimbo. Sem aprovação até a aula, o banco curado padrão assume sozinho (fallback do bloco 9 — a aula nunca fica sem material).
`POST /foto-conta/turmas/{turmaId}/configurar`	`professor`, `gestor_escola`, `gestor_regiona` `gestor_contrat` `admin`	Grava tema e nível de leitura da turma (o passo anterior à geração).
`POST /foto-conta/turmas/{turmaId}/destravar-latch`	`professor`, `gestor_escola`, `gestor_regional` `gestor_contrato` `admin`	LIBERAR REFLEXÃO FINAL: destrava os tablets da turma inteira. UPDATE simples e sem lock de linha (DAD-11): quem “recebe o broadcast” é o polling de 3s dos tablets. Liberar de novo responde `409 latch-ja-liberado` — idempotência por erro, e a mensagem diz que está tudo bem: a turma JÁ está liberada. **Restrições:** • `tabletsNoLatch` é a contagem no instante da liberação — a confirmação visual do painel (“N tablets receberam o sinal” chega pelo polling seguinte do dashboard).

Rota	Autorização	Semântica
`POST /foto-conta/turmas/{turmaId}/gerar-conteudo`	`professor`, `gestor_escola`, `gestor_regiona` `gestor_contrat` `admin`	Gera o pacote da aula com IA (Vertex), para o preview do professor. A IA escreve os TEXTOS (legendas e comentários) adaptados ao tema e ao nível de leitura; as fotos são sempre do banco curado, e a estrutura (3 posts, 2 legendas, 4 combinações) é fixa do servidor. `daIa: false` = a geração não deu (sem saldo, provider fora, saída reprovada) e o pacote devolvido é o banco curado padrão — a tela diz isso com todas as letras. Nada disso chega à turma sem o `aprovar-pacote`: curadoria docente é obrigatória (bloco 14 do one-pager). **Restrições:** • Passa pelo QuotaService por usuário/dia e POR PERFIL (regra inviolável 5); a recusa por saldo cai no mesmo `daIa: false`. • Os comentários aparecem AQUI (preview do professor) — são o material da mediação dele; do estudante, continuam escondidos até a publicação.
`GET /foto-conta/turmas/{turmaId}/projecao`	`professor`, `gestor_escola`, `gestor_regional` `gestor_contrato` `admin`	A projeção da lousa (E5): o par lado a lado, os rótulos e as 3 perguntas. O material do debate vem da API, nunca de cópia no hub: uma vírgula divergente entre o que os tablets mostraram e o que a lousa projeta põe a sala discutindo uma foto que ninguém viu. `noLatch`/`conectados` são o termômetro de “a turma toda chegou”.
`POST /foto-conta/turmas/{turmaId}/reset`	`professor`, `gestor_escola`, `gestor_regiona` `gestor_contrat` `admin`	Zerar e Reiniciar Turma: arquiva a tentativa e rearma o latch. Transação serializada (DAD-11): a `tentativa` incrementa e as runs anteriores ficam ARQUIVADAS na tentativa velha, imutáveis (DAD-06) — nada é apagado, e o histórico analítico não é tocado. Os tablets veem o número novo no boot seguinte e recomeçam do crachá. O guia do professor recomenda TROCAR o tema na reaplicação no mesmo ano (PED-13): repetir o pacote mede memória, não leitura de imagem — recomendação, não automatismo.

C.14 healthz

Rota	Autorização	Semântica
`GET /healthz`	**público**	O processo está vivo. Não toca no banco de propósito: é a sonda que decide reiniciar o container, e um Postgres lento derrubaria um processo saudável.

C.15 jornal-escola

Rota	Autorização	Semântica
`POST /jornal-escola/chave`	`estudante`	Uma tentativa da Chave do Editor: três blocos, em qualquer ordem. Cada envio é uma TENTATIVA — a isca também conta. Sucesso é o conjunto dos três pilares, corrigido no servidor (o conjunto certo nunca desce ao bundle). `devolvidos` são as iscas enviadas, que voltam à bandeja. No sucesso, o passo vai a `revisao` e a EF15CO02 é marcada pelo número de tentativas (1 = atingiu, 2–3 = em processo, mais = não atingiu). Com a Chave já formada, responde `sucesso: true` sem contar outra tentativa. **Restrições:** • `blocos`: exatamente 3 ids diferentes dos seis da bandeja. (Os `.refine()` somem do OpenAPI; daí esta nota.) • `tentativa` (opcional, 1 a 100) é o número que a tela acha que está enviando. Número já contado é REENVIO: devolve o resultado gravado, sem somar tentativa nem marcar de novo — com a Chave formada, `sucesso: true`; senão, `sucesso: false` com os `devolvidos` recalculados sobre os blocos enviados. Sem o número, todo envio conta. • Antes de a revisão abrir para esta criança responde **409** `revisao-nao-liberada`. • Missão não liberada na turma responde **403** `missao-nao-liberada` (ou `recurso-indisponivel`). Estudante sem turma responde **409** `sem-turma`. Corpo fora do formato responde **400** `payload-invalido`.

Rota	Autorização	Semântica
`GET /jornal-escola/comparativo`	`estudante`	O antes e o depois: as duas edições, com o que cada uma causou. "1ª Edição: 1 de 3 verificadas → 2ª Edição: 3 de 3" (PDF §9). `vagas` diz o pilar que faltava em cada notícia das DUAS edições — a devolutiva final explica quais evidências sustentaram o resultado. **Restrições:** • Antes da 2ª publicação responde **409** `edicao-nao-publicada`. • Missão não liberada na turma responde **403** `missao-nao-liberada` (ou `recurso-indisponivel`). Estudante sem turma responde **409** `sem-turma`. Corpo fora do formato responde **400** `payload-invalido`.
`POST /jornal-escola/concluir`	`estudante`	"Concluir Missão" — idempotente. Leva a sessão de `transferencia` a `fim` e carimba `concluidaEm`, no mesmo update. P1 e P2 do Desafio Final vão pelas rotas genéricas de `/evidencias/exp04ef1b/perguntas`, e não são exigidas para concluir. **Restrições:** • Antes do Desafio Final responde **409** `passo-invalido`. Concluir de novo devolve o mesmo estado. • Missão não liberada na turma responde **403** `missao-nao-liberada` (ou `recurso-indisponivel`). Estudante sem turma responde **409** `sem-turma`. Corpo fora do formato responde **400** `payload-invalido`.

Rota	Autorização	Semântica
`POST /jornal-escola/diagramar`	`estudante`	Diagrama a primeira página com as 3 vagas de agora — a IA só escolhe o layout. A IA atua na diagramação, não na decisão editorial (PDF §5): recebe das três notícias só a manchete, o tamanho do resumo e se há imagem, como DADO, e escolhe UM dos layouts do DS (`classic`, `hero-image`, `columns`). Não escreve texto, não troca a ordem das vagas e nunca vê o gabarito. **Nunca falha por causa da IA**: cache da mesma página → IA (teto de ~6 s, quota do estudante, rate limit) → o modelo automático, determinístico, com `origem: "modelo"`. Em dev e na e2e é sempre o modelo. **Restrições:** • `slots`: exatamente 3 ids diferentes, de `n1` a `n8`, na ordem das vagas (a primeira é a manchete principal). (Os `.refine()` somem do OpenAPI; daí esta nota.) • Fora da mesa e da revisão responde **409** `passo-invalido`. • Missão não liberada na turma responde **403** `missao-nao-liberada` (ou `recurso-indisponivel`). Estudante sem turma responde **409** `sem-turma`. Corpo fora do formato responde **400** `payload-invalido`.
`GET /jornal-escola/gate`	`estudante`	Em que etapa a turma está (polling de ~5 s). Magra de propósito **ADR-93**: sem notícias, sem contagem da sala, sem JOIN. **Polling HTTP, nunca WebSocket.** `agora` é o instante do SERVIDOR. **Restrições:** • Passa pelo MESMO funil de liberação das outras rotas **ADR-69** e **ADR-87**: turma sem a missão liberada recebe 403 em vez de descobrir a etapa. • Não escreve nada. `rodada` e `pacoteId` são `null` só antes da primeira entrada da turma; quando mudam, a tela recarrega a sessão.

Rota	Autorização	Semântica
`POST /jornal-escola/motivo`	`estudante`	Como a escolha foi feita — a pergunta de depois do latch, uma vez só. `visual` (“as fotos mais chamativas e os avisos mais urgentes”) ou `checagem` (“quem postou, quando aconteceu e quem assina”). Vem DEPOIS da publicação (perguntar antes anteciparia a descoberta, PDF §4) e ANTES do resultado, com qualquer etapa: a resposta não pode sair contaminada pela consequência. **Restrições:** • Antes da 1ª publicação responde **409** `edicao-nao-publicada`; a segunda resposta, **409** `motivo-ja-respondido`. • Missão não liberada na turma responde **403** `missao-nao-liberada` (ou `recurso-indisponivel`). Estudante sem turma responde **409** `sem-turma`. Corpo fora do formato responde **400** `payload-invalido`.
`POST /jornal-escola/passo`	`estudante`	Registra o passo que a própria criança deu: a mesa, ou o Desafio Final. `mesa` vale do desafio (“Explorar Mesa de Notícias”); pedir de novo, ou depois da 1ª publicação, devolve o passo em que ela está — a escada é de sentido único. `transferencia` é o “Avançar para Desafio Final”, só do comparativo. **Restrições:** • `transferencia` antes do comparativo responde **409** `passo-invalido`. • Missão não liberada na turma responde **403** `missao-nao-liberada` (ou `recurso-indisponivel`). Estudante sem turma responde **409** `sem-turma`. Corpo fora do formato responde **400** `payload-invalido`.

Rota	Autorização	Semântica
`POST /jornal-escola/pista`	`estudante`	Abre uma pista (fonte, data ou autoria) de uma notícia — o texto vem do servidor. O texto da pista desce AQUI, e não na sessão: o registro de quem abriu o quê é o dado da EF15CO04 (decompor "posso confiar?" em três perguntas), e uma pista lida sem registro seria evidência que a plataforma não vê. Abrir um campo não prova compreensão (PDF §11) — o registro apoia a observação do professor. Só na mesa da 1ª edição (fase 1) e na mesa de revisão (fase 2). Idempotente por par na mesma fase: abrir de novo devolve o mesmo texto e não conta. **Restrições:** • Fora da mesa e da revisão responde **409** `passo-invalido`. • `fase` diz onde a pista foi aberta (1 = mesa, 2 = revisão); `abertas` conta os pares (notícia, tipo) distintos das duas fases. No `pistasAbertas` da sessão, o par aberto nas duas fases aparece duas vezes. • Missão não liberada na turma responde **403** `missao-nao-liberada` (ou `recurso-indisponivel`). Estudante sem turma responde **409** `sem-turma`. Corpo fora do formato responde **400** `payload-invalido`.
`GET /jornal-escola/professor/material`	`professor`, `gestor_escola`, `gestor_regional`, `gestor_contrato`, `admin`	O banco padrão com o gabarito, os blocos da Chave e o roteiro da mediação. **Professor e gestores.** É o "Usar Banco Padrão: Feira de Ciências e Notícias da Cidade" do preparo — o hub o manda de volta no `POST :turmaId/homologar` com `origem: "padrao"` — e o bloco vivo do guia: com um computador só, o professor projeta as notícias e conduz a turma (PDF §11). É o BANCO PADRÃO da Unooria, não o pacote da turma: se o professor homologou outro, ele está em `GET /jornal-escola/turmas/:turmaId/pacotes`. **Restrições:** • Nada aqui é escopado por escola: é o catálogo da missão, igual para todo contrato. • `blocos[].pilar` diz quais três formam a Chave — é gabarito, e só existe nesta rota de professor. • `roteiro` é o da copy revisada (§4b): o gatilho do resultado, as três perguntas do debate e o gatilho da revisão.

Rota	Autorização	Semântica
`POST /jornal-escola/publicar`	`estudante`	"Distribuir Jornal Agora" — a 1ª edição (latch). O gesto que a missão mede, e ele é IRREVERSÍVEL. A api calcula as verificadas AQUI, contra o gabarito do pacote (o cliente só manda os três ids), e grava vagas, layout, carimbo e o passo `motivo` juntos. **A resposta não traz o resultado**: ele só desce no `GET /jornal-escola/resultado`, depois do motivo e com o portão do professor aberto. Marca a EF15CO04 (`pistas`) pelas pistas que a criança abriu antes de publicar. **Restrições:** • `slots`: exatamente 3 ids diferentes, de `n1` a `n8`. `layout` e `origemDoLayout` são o que a tela desenhou (o `POST diagramar`): cosméticos, não valem na correção. • Segunda publicação — inclusive duas simultâneas — responde **409** `edicao-ja-publicada`. Do desafio, **409** `passo-invalido`. Com a rodada trocada no meio (o professor zerou), **409** `rodada-mudou`. • Missão não liberada na turma responde **403** `missao-nao-liberada` (ou `recurso-indisponivel`). Estudante sem turma responde **409** `sem-turma`. Corpo fora do formato responde **400** `payload-invalido`.
`POST /jornal-escola/republicar`	`estudante`	"Publicar 2ª Edição Revisada" — a 2ª edição (latch). O mesmo corpo da 1ª. Pode repetir as três vagas da 1ª edição: quem acertou de primeira confirma, sem trocar à força (PDF §9). Marca a EF04CO08 (`edicao-2`) pelas pistas das duas fases e as verificadas da 2ª. **Restrições:** • Segunda republicação — inclusive duas simultâneas — responde **409** `edicao-ja-publicada`; antes da Chave, **409** `chave-nao-formada`; com a rodada trocada no meio, **409** `rodada-mudou`. • Missão não liberada na turma responde **403** `missao-nao-liberada` (ou `recurso-indisponivel`). Estudante sem turma responde **409** `sem-turma`. Corpo fora do formato responde **400** `payload-invalido`.

Rota	Autorização	Semântica
`GET /jornal-escola/resultado`	`estudante`	O que a 1ª edição causou: as verificadas e a reação de um leitor por vaga. "X de 3 notícias verificadas nesta edição (Y%)" — 0, 33, 67 ou 100, descrevendo a SELEÇÃO, não a criança (PDF §6). As reações são só das notícias efetivamente escolhidas (PDF §7), uma por vaga, com o humor do balão derivado de a notícia ser publicável. Não diz quais notícias trocar: isso é da revisão. **Restrições:** • Sem publicação responde **409** `edicao-nao-publicada`; sem o motivo, **409** `motivo-pendente`; antes do portão do professor (etapa `exploracao`), **409** `resultado-nao-liberado`. • Missão não liberada na turma responde **403** `missao-nao-liberada` (ou `recurso-indisponivel`). Estudante sem turma responde **409** `sem-turma`. Corpo fora do formato responde **400** `payload-invalido`.
`GET /jornal-escola/revisao`	`estudante`	A mesa de revisão: o marcador que a Chave pôs em cada vaga da 1ª edição. `pilarEmFalta` de cada vaga — `fonte`, `data` ou `autoria` —, ou `null` quando a notícia daquela vaga está verificada. A essa altura a criança já viu o resultado e montou a Chave. **Restrições:** • Antes de a Chave se formar responde **409** `chave-nao-formada`. • Missão não liberada na turma responde **403** `missao-nao-liberada` (ou `recurso-indisponivel`). Estudante sem turma responde **409** `sem-turma`. Corpo fora do formato responde **400** `payload-invalido`.

Rota	Autorização	Semântica
`GET /jornal-escola/sessao`	`estudante`	O estado da missão e as 8 notícias da mesa, sem o gabarito. Boot da experiência e o F5. As notícias saem **na ordem da mesa** e **sem 'gabarito', 'pistas' e 'consequencia'** — a notícia pública é uma allowlist no servidor. O texto das pistas desce no `POST /jornal-escola/pista` (é o registro que mede a EF15CO04), e o F5 devolve as já abertas em `pistasAbertas`. **A missão não nasce vazia**: sem homologação, a turma roda o banco padrão ("Feira de Ciências e Notícias da Cidade"), e o pacote `padrao` nasce na primeira entrada. `imagem.url` é `null` enquanto o professor não gerou as imagens — a tela desenha o cartão pelo meio. `blocos` (os seis da Chave do Editor, na ordem da bandeja DESTA sessão, estável num F5) é `null` até a revisão abrir para esta criança. Nenhum bloco diz se é pilar ou isca. **Restrições:** • `passo` já vem reconciliado com a etapa da turma; `passoDoEstudante` é o gravado. A etapa só conduz quem publicou a 1ª edição e respondeu o motivo. • `pacoteId` é a ATIVIDADE e `rodada` é a rodada dela: mudam quando o professor homologa outro pacote ou zera a turma, e a sessão é outra. • Missão não liberada na turma responde **403** `missao-nao-liberada` (ou `recurso-indisponivel`). Estudante sem turma responde **409** `sem-turma`. Corpo fora do formato responde **400** `payload-invalido`.
`GET /jornal-escola/turmas`	`professor`, `gestor_escola`, `gestor_regional`, `gestor_contrato`, `admin`	Turmas para o seletor do painel. Filtradas por MATRÍCULA para o professor; o gestor vê a escola. `gestor_contrato` precisa dizer `?escolaId=`.

Rota	Autorização	Semântica
`POST /jornal-escola/turmas/{turmaId}/gate`	`professor`, `gestor_escola`, `gestor_regiona` `gestor_contrat` `admin`	O botão único da aula: "Liberar resultado", depois "Liberar revisão". O corpo diz a etapa de DESTINO. Anda uma etapa por vez — `exploracao` → `resultado` → `revisao` —, nunca pula e nunca volta; destino igual ao atual é aceito e não muda nada (o segundo clique). Grava quando cada portão abriu (EVT-P05). Quem ainda não publicou continua montando a página com o portão aberto: a publicação atrasada é caminho previsto. **Restrições:** • Destino fora de ordem responde **409** `transicao-de-etapa-invalida`, com `etapaAtual` e `proximaEtapa` no corpo. • É a linha que o polling de 5 s do estudante lê (`GET /jornal-escola/gate`): não há WebSocket de aplicação **ADR-93**. • Sem atividade em andamento na turma, **403** `atividade-nao-preparada`: a plataforma não cria atividade — o professor cria e põe em andamento (`POST :turmaId/pacotes/:pacoteId/em-andamento`).
`POST /jornal-escola/turmas/{turmaId}/gerar-conteudo`	`professor`, `gestor_escola`, `gestor_regional` `gestor_contrato` `admin`	Gera um RASCUNHO das 8 notícias com IA, sobre o tema do professor (pré-aula). **Não persiste nada**: o que volta é rascunho para a tela editar, e publicar é o `POST homologar`. A estrutura (que notícia é de que meio e que papel tem no gabarito) é sorteada no servidor; a IA escreve o conteúdo e a saída que foge do molde ou das regras da missão é regerada uma vez. **Sem saldo ou com a IA fora do ar, a resposta é o ERRO** — e o hub oferece o banco padrão em um toque (`GET /jornal-escola/professor/material`). **Restrições:** • Quota do PERFIL do professor **ADR-105**: `temSaldo` antes, e o que o vendor cobrou é debitado por tentativa, inclusive as recusadas. Sem saldo responde **429** `sem-saldo-de-ia`; com o provedor fora do ar ou a saída fora das regras, **503** `ia-indisponivel`. • Cada geração que chamou a IA conta no EVT-P02 (`geracoes`, `tokens_usados`) do pacote em sala. • Os tetos de `disciplina` (2 a 60), `tema` (5 a 60) e `topicos` (até 600) são a primeira barreira contra injeção de instrução no gerador.

Rota	Autorização	Semântica
`POST /jornal-escola/turmas/{turmaId}/gerar-midia`	`professor`, `gestor_escola`, `gestor_regiona` `gestor_contrat` `admin`	Gera as próximas imagens das notícias de uma atividade — incremental. As imagens não cabem nos 30 s que o BFF dá a uma chamada: cada chamada gera ATÉ DUAS novas, grava as URLs no pacote e devolve o progresso. **Repita enquanto 'completo' for 'false' e a chamada anterior tiver gerado alguma** — `falharam` diz o que não saiu, e essas notícias seguem com o cartão desenhado pelo meio. Idempotente: o que já tem URL é pulado, e a imagem de mesma descrição que já está no bucket entra sem custo. **Restrições:** • `?pacoteId=` diz a atividade (a pronta, preparada antes de ir ao ar); sem ele, a em andamento — e sem nenhuma, **403** `atividade-nao-preparada`. • Quota do professor, com custo ESTIMADO por imagem (a geração não devolve contagem de tokens). Sem saldo responde **429** `sem-saldo-de-ia` — e a missão roda igual, com os cartões pelo meio. • Duas chamadas simultâneas para o mesmo pacote recebem o mesmo trabalho (lock de processo).

Rota	Autorização	Semântica
`POST /jornal-escola/turmas/{turmaId}/homologar`	`professor`, `gestor_escola`, `gestor_regional`, `gestor_contrato`, `admin`	"Criar atividade" — a curadoria humana **ADR-66**/67). Grava as 8 notícias como o professor as deixou, numa ATIVIDADE NOVA que nasce **pronta, não no ar** **ADR-130**: o pacote em andamento continua em andamento, e quem escolhe o que a turma faz é o professor (`POST :turmaId/pacotes/:pacoteId/em-andamento`). O mesmo conteúdo do pacote mais recente não cria outro: é o segundo clique. O corpo carrega as notícias INTEIRAS porque curadoria é editar — e editar uma notícia edita também o gabarito e a consequência dela (PDF §10). O banco padrão também passa por aqui (`origem: "padrao"`) quando o professor o aprova em um toque. **Restrições:** • O pacote é recusado com **400** `pacote-invalido` e a LISTA de problemas em `erros`, em frases para o professor: 8 notícias com os ids `n1` a `n8`, nesta ordem; os oito meios, um por notícia; pelo menos 3 publicáveis, 2 ou 3 antigas apresentadas como atuais e 2 ou 3 sem confirmação ou sem autoria; publicáveis e não publicáveis misturando meios tradicionais e digitais, e com imagem; pelo menos 4 notícias com imagem, e sempre na TV e na rede social; locutor no rádio e perfil com @ na rede social; manchete de 3 a 10 palavras, resumo de 25 a 35, texto de 15 a 80; e nenhum termo de carimbo ("falsa", "fake news", "você errou"...) no que a criança lê. (`.superRefine()` some do OpenAPI; daí esta nota.) • As imagens de notícia com a MESMA descrição de um pacote recente da turma são herdadas — o arquivo no bucket é o hash da descrição.
`GET /jornal-escola/turmas/{turmaId}/pacotes`	`professor`, `gestor_escola`, `gestor_regiona`, `gestor_contrat`, `admin`	As atividades da turma: o pacote em andamento, COM o gabarito, e a lista de todos. **Professor e gestores**: é aqui que o gabarito, as pistas e a consequência de cada notícia aparecem — são o roteiro de quem conduz e a tela do preparo. `historico` lista todos os pacotes (o em andamento incluso, com `ativo: true`): cada um é uma ATIVIDADE, e o `id` dele é o `atividadeId` das equipes e do painel de habilidades **ADR-130**. `ativo` é `null` quando nenhum está em andamento: a plataforma não cria atividade. **Restrições:** • `rodadas` é a maior rodada com sessão daquele pacote; 0 = nunca rodou.

Rota	Autorização	Semântica
`GET /jornal-escola/turmas/{turmaId}/pacotes/{pacoteId}`	`professor`, `gestor_escola`, `gestor_regional`, `gestor_contrato`, `admin`	UMA atividade, inteira e COM o gabarito — em andamento ou pronta. A página da atividade. `rodada` e `etapa` só vêm quando ela está em andamento; `rodadas` é a maior rodada com sessão (0 = nunca rodou). **Restrições:** • Pacote de outra turma, ou que não existe, responde **404** `atividade-nao-encontrada`.
`POST /jornal-escola/turmas/{turmaId}/pacotes/{pacoteId}/em-andamento`	`professor`, `gestor_escola`, `gestor_regiona`, `gestor_contrat`, `admin`	“Usar esta atividade agora” — o professor escolhe o que a turma faz **ADR-130**. O escolhido passa a ser a ATIVIDADE EM ANDAMENTO da turma; o que estava volta a ser pronto — fica, com as sessões e as rodadas. A etapa volta à exploração, com os portões fechados. A RODADA: o pacote que nunca rodou começa na 1; o que já rodou abre a SEGUINTE à última — retomar a mesma depois do resultado liberado seria o “voltar” que o botão único proíbe. As rodadas anteriores continuam no painel de habilidades. **Restrições:** • **Idempotente**: pôr no ar o que já está no ar devolve `mudou: false` e não muda nada. • Trocar com a aula correndo leva quem está jogando para a atividade nova: a experiência nota o `pacoteId` novo no polling do gate e recarrega. A tela pede confirmação antes. • Pacote de outra turma responde **404** `atividade-nao-encontrada`.

Rota	Autorização	Semântica
`GET /jornal-escola/turmas/{turmaId}/painel`	`professor`, `gestor_escola`, `gestor_regional`, `gestor_contrato`, `admin`	O painel da aula: o botão da próxima etapa e o progresso, POR COMPUTADOR. Uma linha por computador quando a turma montou equipes para esta missão (`colapsarEmUnidades`), uma por criança quando não — `resumo.porComputador` diz qual. `resumo.computadores` é o "X de Y computadores publicaram": o Y sai da montagem das equipes, porque o colapso conta também quem faltou. **Só progresso por linha.** Quantas verificadas cada computador teve NÃO está na linha: está no histograma anônimo (`resumo.histograma`, [0, 1, 2, 3 verificadas]). O radar é do OBJETO editorial — em quantas capas cada notícia foi —, e o nível por habilidade mora no painel de habilidades (`/evidencias/turma/:turmaId/missao/exp04ef1b/atividade/:pacoteId`). **Restrições:** • Sem atividade em andamento na turma, **403** `atividade-nao-preparada`: a plataforma não cria atividade — o professor cria e põe em andamento (`POST :turmaId/pacotes/:pacoteId/em-andamento`). • A leitura do PROFESSOR conta o EVT-P04 (`painel_visto_em`, `painel_vistas`); a do gestor não. • Turma que o professor não leciona responde **404** `turma-nao-encontrada`.
`POST /jornal-escola/turmas/{turmaId}/regenerar-noticia`	`professor`, `gestor_escola`, `gestor_regional`, `gestor_contrato`, `admin`	Regenera UMA notícia do rascunho, mantendo o id, o meio e o papel no gabarito. "Regenerar Notícia" (E0-PROF-04). A nova mantém o id, o MEIO e o papel no gabarito (publicável, antiga, sem fonte ou sem autoria) da que sai — senão a regeneração de uma desmontaria a composição das oito —, vem com imagem em todo meio menos a mensagem, e não repete a manchete de outra. Não persiste. **Restrições:** • `noticias` leva, de cada uma das oito, só `id`, `meio`, `manchete` e o papel no gabarito — sem régua de tamanho: um campo vazio em outra notícia não trava esta. • A mesma quota do `gerar-conteudo`: sem saldo responde **429** `sem-saldo-de-ia`. Com a IA fora do ar ou a saída fora das regras, responde **200** com `usouIa: false` e a notícia COMO ESTAVA (a do pacote ativo da turma, ou a do banco padrão, pelo id).

Rota	Autorização	Semântica
`POST /jornal-escola/turmas/{turmaId}/zerar`	`professor`, `gestor_escola`, `gestor_regional`, `gestor_contrato`, `admin`	"Zerar a turma" — abre a próxima rodada, sem apagar nada. Sobe a rodada da atividade e volta a etapa para a exploração, com os portões fechados. **Nada é apagado**: as sessões, as edições e as pistas da rodada anterior ficam, e são elas que o painel de habilidades mostra como a rodada passada **ADR-112**. O pacote também fica. **Restrições:** • Na rodada nova do MESMO pacote, quem já foi avaliado numa anterior refaz sem marca nova nas ações **ADR-120**: já viu o resultado e a Chave. O guia recomenda criar outro pacote, que é OUTRA atividade. • Sem atividade em andamento na turma, **403** `atividade-nao-preparada`: a plataforma não cria atividade — o professor cria e põe em andamento (`POST :turmaId/pacotes/:pacoteId/em-andamento`).

C.16 lotacoes

Rota	Autorização	Semântica
`DELETE /lotacoes/{id}`	`gestor_escola`, `gestor_regional`, `gestor_contrato`, `admin`	Encerra o vínculo da pessoa com a escola. **Não apaga**: grava `fim`. É a lotação que diz que a pessoa esteve ali. Fecha junto as matrículas de turma daquela escola (sem escola, não há turma), e se era a principal, a próxima lotação ativa assume — senão `users.escola_id` apontaria para onde a pessoa não está mais. **Restrições:** • A checagem de escola vem DEPOIS de achar a lotação, porque é ela que diz de qual escola é — e um gestor não encerra vínculo de outra unidade (403).

C.17 manifesto-ia

Rota	Autorização	Semântica
`POST /manifesto-ia/abrir-conversa`	`estudante`	Sai do briefing de Amara e abre o chat do EduPredict IA. Idempotente. Existe como rota para o `passo` andar no SERVIDOR: um F5 na Etapa 2 não pode devolver a criança ao alerta de Amara que ela já ouviu.
`POST /manifesto-ia/clausula`	`estudante`	Fixa a salvaguarda prioritária — irrevogável (latch). O PICO da missão. A api confere a cota de auditoria ANTES de aceitar (409 `auditoria-insuficiente`) e **compila o manifesto no servidor**, porque o documento é a evidência do percurso: o que a professora lê no painel e o que o estudante copiou têm de ser o mesmo texto. Segunda chamada responde 409 `clausula-ja-submetida`: a escolha é definitiva, e é isso que faz o estudante assumir o peso dela. **Nada é corrigido aqui.** As quatro cláusulas são salvaguardas válidas, e calcular acerto entre elas está na lista de métricas proibidas do handoff. **Restrições:** • `delayMs` é o delay diegético de 2000ms (não-negociável 2). O timer roda no CLIENTE porque é ritmo de cena, e o número vem do servidor para a documentação e a tela lerem o mesmo valor. • `confirmouProvocacao` registra que a criança clicou em menos de 5s e passou pela réplica provocativa do bot. É `indicative` e alimenta a hipótese de exploração mínima — nunca avalia ninguém. • A guarda da cota vale no servidor mesmo com o botão desabilitado na tela: `/manifesto-ia/clausula` é endereço alcançável.
`POST /manifesto-ia/concluir`	`estudante`	Finaliza o percurso e concede o selo — latch definitivo. **Não recebe 'score'**, e a ausência é a regra 3 aplicada a uma missão sem nota: o `exp:concluir` da bridge tem o campo, e aceitar um número dali deixaria a tela pontuar uma atividade cuja premissa é que as quatro escolhas valem igual. O progresso da plataforma lê `concluido_em`. Exige o percurso registrado (409 `sem-clausula`). Segunda chamada responde 409 `missao-concluida`.

Rota	Autorização	Semântica
`POST /manifesto-ia/disparar`	`estudante`	Envia um questionamento e recebe a defesa da máquina. **A 'replica' só existe nesta resposta.** Ela não está no bundle nem em `GET /sessao` antes do disparo: ver a defesa antes de fazer a pergunta desmontaria a cena da auditoria. Quando o SEGUNDO questionamento distinto entra, o `passo` anda para `decisao` aqui, no servidor. Deixar o cliente pedir a transição seria deixá-lo escolher quando o pico da missão começa — e uma tela que decide isso é uma tela que pode pular o pico. **Restrições:** • Reenviar o mesmo id responde 409 `prompt-ja-enviado`: o card já disparado fica desabilitado no seletor, então um segundo disparo só chega por clique duplo ou aba repetida — e a conversa não pode ganhar a mesma pergunta duas vezes. • `latenciaSeg` é calibragem de fluxo, NUNCA medida de mérito: ninguém é avaliado por ler rápido. • O terceiro questionamento é opcional e permitido: dois é o mínimo que abre a Etapa 3, não um teto.
`GET /manifesto-ia/gate`	`estudante`	Em que etapa a turma está — o polling de 5s do Checkpoint A. Leve de propósito: sem catálogo e sem conversa. **É a alternativa ao WebSocket** ADR-93: a infraestrutura Cloud Run da plataforma não tem WebSocket de aplicação, e o congelamento da turma para o debate oral é uma linha em `exp08ef4b_gates_turma` que a tela do estudante lê a cada 5 segundos durante a Etapa 4. `agora` é o instante do SERVIDOR: o relógio da máquina da escola atrasa, e não é ele que decide o momento da aula. **Restrições:** • Estudante sem turma não tem gate: a missão corre no ritmo dele. Prendê-lo num congelamento seria um beco sem saída, porque não há quem o solte. • O limite é mais alto que o das outras rotas justamente porque este é o polling — 240/min por usuário cobre 5s com folga para retry.

Rota	Autorização	Semântica
`POST /manifesto-ia/percurso`	`estudante`	O “Meu Percurso” — 2 palavras-chave e o parágrafo autoral (latch). Texto livre de 20 a 800 caracteres. É o **único** campo livre da missão e o dado mais sensível dela: caixa escrita por menor de idade. Não vai a log, não vai à telemetria analítica, e a única rota que o devolve é o painel do professor DA TURMA dela (LGPD art. 14). **Dado pessoal é RECUSADO com 422 ‘dado-pessoal-no-texto‘ e NÃO gravado.** As alternativas (gravar com flag, ou ofuscar e salvar) foram descartadas: a primeira deixaria o telefone da criança na coluna à espera de alguém, a segunda devolveria à professora um parágrafo mutilado. Recusar também ensina — a dica diz o que tirar, no momento em que ela ia entregar o dado. Durante o Checkpoint A responde 409 `debate-em-andamento`: a assinatura individual abre depois da discussão coletiva. **Restrições:** • O `trim` é anterior ao `min`: vinte espaços passariam o piso de 20 caracteres e contariam como entregue no painel. • As duas palavras-chave têm de ser DIFERENTES: repetir deixa um slot logicamente vazio dentro de um array cheio. • O filtro NÃO tenta adivinhar nome próprio, de propósito: recusar ’meu tio Silva foi reprovado’ cortaria justamente o argumento situado que esta etapa mede. O que ele pega é padrão estruturado — telefone, CPF, e-mail, CEP, cartão, @perfil.

Rota	Autorização	Semântica
`GET /manifesto-ia/sessao`	`estudante`	O estado da missão, o pacote da turma e a conversa reconstituída. Os 3 questionamentos saem **sem 'replicala'** e as 4 cláusulas **sem 'justificativaSugerida'** — os tipos do servidor são `Omit<...>`, então nem a defesa da máquina antes da pergunta nem o material de mediação do professor podem vazar aqui. `falas` é a conversa até agora, e é o que faz o F5 devolver a tela exata em vez de um chat vazio: a réplica de uma pergunta JÁ FEITA entra, porque a criança já a leu. O conteúdo vem do pacote HOMOLOGADO da turma; sem homologação, do banco curado da Unooria (`origemDoConteudo` diz qual). **Restrições:** • `passo` já vem reconciliado com a etapa da turma; `passoDoEstudante` é o cru. Divergentes com `aguardandoATurma: true` = o Checkpoint A está ativo e a tela mostra o overlay de pausa. • `manifestoMarkdown` é `null` até a cláusula ser fixada. Devolver um esqueleto com o lugar da cláusula em branco entregaria a cena da Etapa 4. • Rascunho de IA NÃO conta como pacote da turma: sem `aprovado_por_professor`, a missão lê o curado (exigência 8.2).
`GET /manifesto-ia/turmas`	`professor`, `gestor_escola`, `gestor_regiona` `gestor_contrat` `admin`	Turmas para o seletor do painel. Filtradas por MATRÍCULA para o professor; o gestor vê a escola. `gestor_contrato` precisa dizer `?escolaId=`.
`DELETE /manifesto-ia/turmas/{turmaId}`	`professor`, `gestor_escola`, `gestor_regional` `gestor_contrato` `admin`	Zerar a turma — a segunda aplicação. Apaga sessões, auditorias e percursos desta turma (CASCADE da sessão) e devolve o gate a `missao`, senão a turma nova nasceria congelada num debate que já aconteceu. **Preserva 'telemetria_eventos'**, que é a trilha analítica imutável, **e preserva 'exp08ef4b_conteudo_turma'**: a curadoria da professora não é execução de estudante. O percurso vai junto, e isso é deliberado: refazer a missão com o parágrafo da turma do ano passado na tela seria um portfólio que mistura duas turmas.

Rota	Autorização	Semântica
`GET /manifesto-ia/turmas/{turmaId}/conteudo`	`professor`, `gestor_escola`, `gestor_regiona` `gestor_contrat` `admin`	O pacote da turma, com o material de mediação. **Professor only**, e é aqui que a `replicaIa` de cada questionamento e a `justificativaSugerida` de cada cláusula aparecem — as duas coisas que a rota do estudante recorta. A professora as lê ANTES da aula, de propósito: são o roteiro dela. Sem pacote homologado devolve o curado da Unooria com `aprovadoPorProfessor: false`. Não é erro: o banco default é homologado pela Unooria, e a missão roda inteira com ele.
`POST /manifesto-ia/turmas/{turmaId}/gate`	`professor`, `gestor_escola`, `gestor_regional` `gestor_contrato` `admin`	O botão mestre: Pausar para Debate / Retomar Missão. O Checkpoint A. É o que congela 30 telas ao mesmo tempo para o debate oral de 10 minutos — e é a rota que o polling de 5s do estudante lê (`GET /manifesto-ia/gate`), porque **não há WebSocket de aplicação no Cloud Run** ADR-93. **Idempotente de propósito**: apertar duas vezes é exatamente o clique que se dá duas vezes, e a segunda não pode derrubar nada. O gate mexe só no TETO do passo, nunca no piso: pausar não empurra para a frente quem ainda estava auditando a máquina — e são justamente essas crianças que o debate tenta alcançar. **Restrições:** • Quem já assinou o percurso fica onde está: `pausar-para-debate` não o traz de volta. O latch é do estudante, não da turma. • A missão corre inteira sem ninguém apertar nada: o congelamento é opcional, porque a spec admite a sessão assíncrona individual.

Rota	Autorização	Semântica
`POST /manifesto-ia/turmas/{turmaId}/gerar-conteudo`	`professor`, `gestor_escola`, `gestor_regiona` `gestor_contrat` `admin`	Gera um RASCUNHO do pacote da turma com IA (pré-aula). A única chamada de IA desta experiência, e ela roda **antes** da aula, com o professor olhando. Em sala o motor é 100% determinístico: nenhum token de estudante é gasto e nenhuma aula depende de o Vertex estar de pé. **Não persiste nada.** O que volta é rascunho para a tela editar; publicar é o `POST /homologar` (exigência 8.2 / PED-04). Falha de vendor, quota esgotada ou saída fora do formato devolvem o pacote CURADO com `usouIa: false` — e a tela diz 'Utilizando pacote curado padrão da Unooria' em vez de fingir que a IA respondeu. O único jeito de esta rota estragar a aula seria bloquear a professora. **Restrições:** • A quota é a do PERFIL **ADR-74**: `temSaldo` antes, e o que o vendor cobrou é debitado por TENTATIVA — inclusive as que a validação rejeitou, porque foram pagas igual. • A validação rejeita balão acima de 60 palavras e termo proibido na tela, e rejeitar faz o modelo REGERAR: vale mais que aceitar e contar com a revisão humana de um texto que a professora talvez só passe o olho. • Os ids das cláusulas e dos questionamentos são fixados pela ordem do catálogo, nunca pelo que o modelo devolveu: id inventado produziria uma cláusula que a tela mostra e o servidor não reconhece.

Rota	Autorização	Semântica
`POST /manifesto-ia/turmas/{turmaId}/homologar`	`professor`, `gestor_escola`, `gestor_regional`, `gestor_contrato`, `admin`	O clique em "Aprovar Conteúdo da Turma" — o portão da exigência 8.2. Grava o pacote como a professora o deixou e marca `aprovado_por_professor: true`. **É o único lugar que liga esse booleano**, e é ele que a rota do estudante consulta: sem homologação, a missão lê o banco curado da Unooria. O corpo carrega o pacote INTEIRO em vez de um `{ aprovado: true }` porque a curadoria é editar: a professora pode reescrever qualquer réplica ou cláusula antes de aprovar. Aprovar sem editar manda de volta o que a tela recebeu — o que também é curadoria, porque passou pelos olhos dela. **Restrições:** • Idempotente: homologar duas vezes é exatamente o clique que se dá duas vezes. • Sobrevive ao 'Zerar turma'. O preparo custou tempo da professora, e apagá-lo junto obrigaria a refazer a curadoria para aplicar a missão na turma seguinte no mesmo dia.
`GET /manifesto-ia/turmas/{turmaId}/painel`	`professor`, `gestor_escola`, `gestor_regiona`, `gestor_contrat`, `admin`	O painel de mediação ao vivo (Checkpoint A). Uma linha por estudante mais o resumo agregado. **Sem coluna de pontos, sem nível e sem taxa de acerto** (PED-07 / D6b e `PROIBIDO_NO_PAINEL`): nesta missão a tentação é específica — com quatro cláusulas e uma 'mais votada', calcular percentual de acerto é o erro natural de quem chegar depois, e as quatro são salvaguardas válidas. `distribuicao` conta os votos por cláusula: é o material das três perguntas do debate. `justificativaTexto` sai NESTA rota e só nela: é caixa livre de menor de idade, e quem a lê é o professor da turma dela (LGPD art. 14). **Restrições:** • `ociosoMin` alimenta o alerta de inatividade (> 3 min). Serve para o professor ir até a carteira, não para punir. • `auditouOMinimo` é o indicador central da missão: quantos contraditaram a máquina ao menos duas vezes antes de fixar a regra. • `tags` no resumo é a nuvem agregada da sala. No nível do estudante ela é nominal, porque a professora precisa saber com quem conversar.

C.18 matriculas

Rota	Autorização	Semântica
`PATCH /matriculas/{id}`	`gestor_escola`, `gestor_regional`, `gestor_contrato`, `admin`	Move de turma, ou torna esta a turma de referência. Mover é UM passo de propósito: à mão, o gestor tira da antiga e esquece de pôr na nova, e o estudante fica sem turma — invisível para o professor, sem nada na tela dizendo o que faltou. Nenhuma das duas operações apaga linha. **Restrições:** • Informe `turmaDestinoId` OU `principal`, nunca os dois nem nenhum — é um `.refine()`, que o JSON Schema não sabe expressar.
`DELETE /matriculas/{id}`	`gestor_escola`, `gestor_regiona` `gestor_contrat` `admin`	Encerra a matrícula. **Não apaga linha**: grava `fim`. O dado da Vila, do UnooTube e do unooGPT continua pendurado nela, e apagar deixaria esse dado órfão de turma. Sair da REDE é outra coisa — `ativo = false` na pessoa.

C.19 observabilidade

Rota	Autorização	Semântica
`POST /observabilidade/erros-cliente`	*(qualquer)*	Registra um erro não capturado do navegador (experiência ou hub). A SDK captura `window.onerror/unhandledrejection` (dedupe por hash, no máximo 5 por sessão) e manda para cá. A api redige `mensagem` e `stack`, acrescenta `escola_id` e o id opaco do estudante a partir do TOKEN e emite UMA linha `ERROR` no formato que o Error Reporting agrupa, com `serviceContext.service = expId` e `version = versao` — é assim que um erro vira "novo na versão X". Nenhum SDK de terceiro entra no bundle **ADR-58**. **Restrições:** • Cota de 5 por minuto por (escola, usuário): acima, **429** `{ codigo: "cota-excedida" }` e um contador, sem linha de log — um laço de erro no cliente não pode virar inundação. • `url` sem query string nem fragmento (`.refine`): é neles que mora id, token e texto. • `traceId/spanId` opcionais ligam o erro ao trace do navegador quando o OpenTelemetry Web está ligado.

Rota	Autorização	Semântica
`POST /observabilidade/publico/erros-cliente`	*(qualquer)*	Registra um erro de navegador ocorrido ANTES do login (landing, tela de login, troca de senha). Mesma captura da SDK, sem token: é a única rota de observabilidade sem `OidcGuard` **ADR-84**. A api força `expId = hub`, redige `mensagem`/`stack`, classifica o user-agent em rótulos fechados e emite UMA linha `ERROR` no formato do Error Reporting, com `autenticado: false` e SEM id de pessoa — quem ainda não entrou não tem identidade a registrar. O que correlaciona os eventos de uma mesma aba é o `sessaoId`. **Restrições:** • Cota de 5 por minuto por IP e 300 por minuto no total da instância: acima, **429**, contado e sem linha de log. • `expId` do corpo é IGNORADO (sempre `hub`): antes do `hub:init` nenhuma experiência conhece a api. • O IP serve de chave de cota e não é registrado em lugar nenhum.
`POST /observabilidade/publico/vitals`	*(qualquer)*	Recebe um Web Vital medido antes do login. A tela de login é a primeira de toda sessão, e o vital dela é o único que mede a primeira impressão de desempenho da plataforma. Registrado no mesmo histograma das telas autenticadas, com `perfil: anonimo` — sem token não há perfil a declarar. **Restrições:** • Cota de 20 por minuto por IP, mais o teto global da instância.
`POST /observabilidade/v1/traces`	*(qualquer)*	Ingestão OTLP/JSON dos spans que a SDK exporta do navegador. O OpenTelemetry Web da experiência exporta para cá, com o token da plataforma; o navegador nunca fala com o Cloud Trace. A api valida o lote (Zod fechado: ≤ 64 spans, atributos por allowlist, URLs sem query), SUBSTITUI o `resource` pelo `service.name`/`service.version` validados e reexporta para a Telemetry API do Google com a credencial da própria api. Sem `GCP_PROJECT_ID` (dev) o lote é contado e descartado. **Restrições:** • Cota de 30 lotes por minuto por (escola, usuário) e teto de 256 KB por corpo (**413**); acima da cota, **429**. • Lote com 0 ou mais de 64 spans é descartado (contado em `spans_descartados{motivo=tamanho}`). • Chaves fora do subconjunto (links, traceState, flags) são removidas, não recusadas.

Rota	Autorização	Semântica
`POST /observabilidade/vitals`	*(qualquer)*	Recebe um Web Vital (LCP, INP, CLS, TTFB) do hub ou de uma experiência. Amostrado a 10% no cliente, sem identificador de pessoa no corpo. A api normaliza `rota` para um conjunto fechado (`/professor/:secao`, `exp04ef1b`…, senão `outra`) e registra num histograma OpenTelemetry por `superficie`, `rota` e `perfil` (do token). É o único sinal de que "a plataforma está lenta na escola X" que não depende de alguém reclamar — e nunca vira log. **Restrições:** • Cota de 60 por minuto por (escola, usuário): acima, **429**. • `valor` acima de 10 minutos é recusado (relógio, não lentidão).

C.20 perfis

Rota	Autorização	Semântica
`POST /perfis/importar`	**serviço**	Ponte externa do Ato 1 — compatibilidade. Chamada pelo JOGO, que roda fora do navegador e não tem sessão: autenticação por chave de serviço no cabeçalho, não por cookie. Continua viva como compatibilidade; hoje o caminho normal é `POST /atividades/:id/perfis/da-vila`, disparado pelo professor. **Restrições:** • `escolaId` vem no CORPO porque esta rota não tem token: sem ele não há escola no contexto, e `comTenancy` falha alto. O `activityId` é conferido DENTRO dessa escola, então um `escolaId` errado não alcança atividade de outra rede. • Sem `PERFIL_IMPORT_TOKEN` a rota responde **503** — desligada, nunca aberta.

C.21 perfis-recebidos

Rota	Autorização	Semântica
`POST /perfis-recebidos`	`gestor_escola`, `gestor_regional`, `gestor_contrato`, `admin`	Recebe um documento de outra rede. Receber é **ato da escola de destino** — ninguém empurra perfil para dentro de uma rede; alguém de lá importa o arquivo, olhando para ele. A api verifica porque verificar é o passo que **não pode confiar em quem chama**. **Restrições:** • **Assinatura inválida é RECUSA (400)** — isso é adulteração. Emissor inalcançável **não** é: entra marcado `nao-verificada`, e a tela sai em âmbar em vez de fingir que conferiu. • A classificação é pelo `code` da jose, NUNCA pelo texto: "signature verification failed" contém "failed", e uma versão anterior classificava documento FORJADO como "não deu para conferir". • `PORTABILIDADE_EMISSORES` é a decisão de confiança: o `emissor.hub` vem de dentro do arquivo, então o documento SELECIONA entre origens que o operador escreveu e nunca introduz uma (sem isso, SSRF). Vazia = tudo entra `nao-verificada`. • **Importar não cria estudante**: liga pelo CPF a quem já está aqui. Matricular é ato da escola.
`GET /perfis-recebidos/{id}`	`gestor_escola`, `gestor_regiona` `gestor_contrat` `admin`	O documento recebido inteiro, como veio. Guardado *verbatim*: alterar qualquer byte destruiria a assinatura. É a **única exceção** à contenção do CPF em `users.cpf` — o número é o conteúdo do documento **ADR-33**.
`DELETE /perfis-recebidos/{id}`	`gestor_escola`, `gestor_regional`, `gestor_contrato`, `admin`	Apaga um perfil recebido. Existe porque o documento é de FORA: importado na criança errada, ou emitido por engano, ele tem de sair — e apagá-lo não apaga nada que esta rede tenha medido.

C.22 periodos-letivos

Rota	Autorização	Semântica
`GET /periodos-letivos`	`professor`, `gestor_escola`, `gestor_regional` `gestor_contrato` `admin`	O calendário do contrato, com quantas turmas cada período tem. Catálogo do CONTRATO: sem `escola_id` e sem RLS, por isso controller separado. A contagem de turmas é o que impede apagar um período por engano. **Restrições:** • `vigente` é DERIVADO das datas, nunca coluna — estado que ninguém atualiza mente mais do que ajuda.
`POST /periodos-letivos`	`gestor_escola`, `gestor_regiona` `gestor_contrat` `admin`	Cria um período letivo — a primeira etapa da virada do ano. A partir da data de início ele passa a ser o vigente, e toda turma nova nasce nele. **A segunda etapa é importar o enturmamento**, e nenhuma das duas é automática: as turmas do ano anterior ficam onde estão, e é isso que preserva o histórico. **Restrições:** • `fim` antes de `inicio` é **400**.

C.23 planejamento

Rota	Autorização	Semântica
`GET /planejamento/configuracoes`	`professor`	As regências configuradas por ESTE professor. Cada uma é um par componente × ano com carga horária e grade de horários. `dimensionamento` traz o total de aulas no ano e a conversão para horas-relógio (hora-aula NÃO é hora-relógio: Parecer CNE/CEB 8/2004), além do aviso `gradeCoerente: false` quando os horários marcados não somam as aulas semanais declaradas.
`POST /planejamento/configuracoes`	`professor`	Cria ou atualiza a regência de um componente × ano. Idempotente na chave (professor, ano letivo, série, componente): reenviar atualiza. Recusa componente que a rede não oferta naquele ano e componente sem habilidade da BNCC — os dois produziriam um plano vazio de que o professor levaria a culpa. `inicioDoAnoLetivo` alimenta `periodos_letivos`, que é catálogo do CONTRATO (a rede inteira compartilha o calendário).

Rota	Autorização	Semântica
`DELETE /planejamento/configuracoes/{id}`	`professor`	Apaga o planejamento inteiro daquele componente × ano. A regência, o plano anual dela e todas as semanas planejadas. O banco faz a cascata; o object storage não tem cascata, então os prefixos de cada plano anual são varridos ANTES do delete — depois dele não haveria mais como saber quais eram. Irreversível, e é a tela que confirma.
`GET /planejamento/configuracoes/{id}/calendario`	`professor`	Os períodos de avaliação e as semanas de planejamento do ano. As 40 semanas que o professor vai planejar, cada uma com quantas aulas ela tem de fato (da grade real) e o plano já gerado, se houver. O passo é sempre semanal **ADR-76**: entregar de duas em duas semanas, ou o período inteiro, é recorte de EXPORTAÇÃO e não de planejamento. Como os períodos são divididos em semanas fechadas, nenhuma semana atravessa a virada do bimestre.
`POST /planejamento/configuracoes/{id}/plano-anual`	`professor`	Cria o plano anual de uma regência. Nasce com a distribuição VAZIA: escolher que habilidade cai em que período é decisão pedagógica do professor, não do gerador. A geração só é liberada depois de `PUT /planos-anuais/:id/distribuicao`.
`GET /planejamento/curriculo/componentes`	`professor`, `gestor_escola`, `gestor_regiona` `gestor_contrat` `admin`	Os componentes curriculares que a rede oferta naquele ano. Com o total de habilidades da BNCC de cada um e as aulas semanais da matriz mediana de referência. `habilidadesPorArea: true` avisa que as habilidades vêm da ÁREA e não do componente — o caso de Biologia, Física e Química no Ensino Médio, que compartilham as habilidades de Ciências da Natureza. A BNCC não define grade horária: `aulasSemanaisDeReferencia` é a mediana observada em nove redes públicas e serve de ponto de partida editável, nunca de obrigação legal.

Rota	Autorização	Semântica
`GET /planejamento/curriculo/habilidades`	`professor`, `gestor_escola`, `gestor_regional`, `gestor_contrato`, `admin`	As habilidades da BNCC daquele componente naquele ano, agrupadas. Agrupadas pelo organizador que a BNCC usa NAQUELE componente — a Base não é uniforme: sete componentes usam unidade temática, Língua Portuguesa usa prática de linguagem, Língua Inglesa usa eixo, e o Ensino Médio agrupa por competência específica da área. `rotuloDoOrganizador` diz como nomear o agrupador na tela. `computacao` traz a camada transversal do ano (Res. CNE/CEB 1/2022) separada, com a explicação e os exemplos oficiais do anexo — a articulação entre uma habilidade de Computação e uma do componente é PROPOSTA pedagógica, não dado normativo: não existe mapeamento oficial.
`GET /planejamento/periodos/{id}`	`professor`	O plano de uma semana, aula a aula.
`PUT /planejamento/periodos/{id}/aulas`	`professor`	Salva as aulas editadas — e o registro do que foi cumprido. As aulas são dado estruturado, não prosa: o professor edita a aula de quinta sem tocar na de terça. `cumprida` e `observacaoDoProfessor` são preenchidos DEPOIS da aula e não vêm da IA — é o registro do que aconteceu, e é ele que faz o plano deixar de ser intenção.
`GET /planejamento/planos-anuais`	`professor`	Os planos anuais DESTE professor.
`GET /planejamento/planos-anuais/{id}`	`professor`	O plano anual com o conteúdo. `html` é `null` enquanto o status não for `pronto` — é assim que a tela sabe que deve continuar o polling. Vem com a configuração e os períodos de avaliação, para a tela não precisar de três chamadas.
`DELETE /planejamento/planos-anuais/{id}`	`professor`	Apaga o plano anual e os planos de período dele. O conteúdo no object storage vai junto (`apagarPrefixo`).
`PUT /planejamento/planos-anuais/{id}/conteudo`	`professor`	Salva o plano anual editado pelo professor. O HTML passa por allowlist antes de ir ao storage: ele volta ao navegador de outro usuário (o próprio professor depois, ou quem herdar a turma), então é XSS armazenado se passar cru. O Markdown original fica intocado em `gerado.md`.

Rota	Autorização	Semântica
`PUT /planejamento/planos-anuais/{id}/distribuicao`	`professor`	Salva a distribuição de habilidades pelos períodos. É a decisão pedagógica central do produto, e o insumo do gerador. Recusa código que não pertence ao recorte (componente × ano): um `EF67LP42` colado à mão chegaria ao prompt e sairia no documento como se fosse currículo, e o professor levaria à coordenação um plano citando habilidade inexistente.
`POST /planejamento/planos-anuais/{id}/gerar`	`professor`	Dispara a geração do plano anual. 202 porque um plano anual leva minutos. O claim é atômico (`updateMany` condicionado ao status), então duas instâncias não geram o mesmo plano. Quota de IA conferida ANTES (regra 5 do AGENTS.md) e consumida depois — inclusive quando o resultado é rejeitado, porque o token foi gasto.
`GET /planejamento/planos-anuais/{id}/periodos`	`professor`	As semanas JÁ PLANEJADAS, com as aulas — para exportar várias juntas. Só as `pronto`, em ordem: é a fonte do documento de mais de uma semana (duas, para a escola que recolhe de quinze em quinze dias, ou o período de avaliação inteiro). Um documento que a coordenação recebe não leva semana que falhou na geração nem semana que ninguém planejou. Com `periodoDeAvaliacao` o filtro é no BANCO — baixar o 1º bimestre não tem por que carregar as aulas dos outros três.
`POST /planejamento/planos-anuais/{id}/periodos/{ordem}/gerar`	`professor`	Gera o plano aula a aula de uma semana. O esqueleto das aulas (quantas, em que data, em que horário) NÃO vem do modelo: sai do calendário e da grade real do professor. O modelo preenche o conteúdo pedagógico das aulas que existem — deixá-lo decidir quantas caberiam produzia cinco, o número redondo, para um professor de Arte que tem uma. `feriados` REMOVE a aula em vez de deslocá-la: repor é decisão pedagógica, não conta do sistema. Reenviar REGERA a semana, e o `update` recalcula início, fim e período de avaliação: a `ordem` é a única chave, e o que ela endereça muda se a regência corrigir o início do ano letivo.

C.24 planos

Rota	Autorização	Semântica
`GET /planos`	`professor`	Os planos DESTE professor. Posse por `professor_id` em toda query — plano de aula é autoral.
`POST /planos`	`professor`	Pede a geração de um plano de aula. **202**: a geração corre em background (claim atômico na linha), e o cliente acompanha por `GET /planos/:id` até o status virar `pronto` ou `erro`. **Restrições:** • 3 gerações por hora por professor: cada uma custa ~220k tokens só de prompt (currículo da rede + as 4 unidades do ano). Acima disso, **429** com `codigo: RATE_LIMIT`. • O teto de saída é 64k tokens, não 32k: com o material real o gemini-2.5-flash era cortado e o professor recebia "o plano ficou grande demais" sem plano nenhum. • Passa pelo `QuotaService` por PERFIL (a linha `professor` de `ia_quotas`, **ADR-105**.
`GET /planos/{id}`	`professor`	Um plano, com o conteúdo quando estiver pronto. É o alvo do polling que segue o 202 de `POST /planos`. **Restrições:** • `html: null` enquanto o status não for `pronto` — é assim que o cliente sabe que ainda deve continuar o polling.
`PATCH /planos/{id}`	`professor`	Renomeia o plano. **Restrições:** • Título entre 3 e 160 caracteres; fora disso, **400**.
`DELETE /planos/{id}`	`professor`	Exclui o plano.
`PUT /planos/{id}/conteudo`	`professor`	Salva o conteúdo editado pelo professor. Sobrescreve o `atual.html` no object storage PRIVADO. O `gerado.md` é imutável e continua lá — é o que `POST /planos/:id/restaurar` devolve. **Restrições:** • O HTML é sanitizado com allowlist antes de gravar. • Teto de 1 MB: um plano anual tem 30–150 KB. Acima disso, **400**.
`POST /planos/{id}/restaurar`	`professor`	Descarta a edição e volta ao plano gerado. O `gerado.md` é IMUTÁVEL de propósito: é o que torna esta rota possível. Sem ele, salvar por cima seria irreversível.

Rota	Autorização	Semântica
`GET /planos/referencias`	**serviço**	O inventário das referências curriculares do contrato. **Sem o texto**: existe para o script de importação ser idempotente e para auditoria — o texto de um currículo inteiro não cabe numa listagem. **Restrições:** • Catálogo GLOBAL do contrato: sem `escola_id`, fora da RLS.
`POST /planos/referencias`	**serviço**	Importa (ou sobrescreve) uma referência curricular. Chamada pelo `scripts/importar-referencias-plano.mjs`, que extrai o texto dos PDF/DOCX **localmente** e manda só texto. Upsert por `(disciplina, serie, nome)`. **Restrições:** • `serie: null` = vale para todas as séries daquela disciplina. • O upsert é find + update/create, e não `upsert`: `serie` nulo não entra em `where` de unique composto no Prisma, e o Postgres trata NULLs como distintos no índice. • Sem `PLANOS_IMPORT_TOKEN` a rota responde **503** — desligada, nunca aberta.
`DELETE /planos/referencias/{id}`	**serviço**	Remove uma referência do catálogo.
`GET /planos/referencias/disponiveis`	`professor`	Disciplinas e séries que têm material curricular importado. Alimenta os seletores da app-90. Vem com os rótulos prontos: a tela imprime o que o servidor mandou, em vez de manter um segundo dicionário que divergiria na primeira disciplina nova.

C.25 posts

Rota	Autorização	Semântica
`PATCH /posts/{id}`	`professor`	Edita o post: legenda, hashtags, comentários e aprovação.
`DELETE /posts/{id}`	`professor`	Exclui um post da grade.

Rota	Autorização	Semântica
`GET /posts/{id}/comentarios`	*(qualquer)*	Os comentários de um post do feed. Na ordem de uma CONVERSA: o comentário do estudante no topo (como o Instagram faz com o seu), as reações da IA logo depois, e os do roteiro por último. **Restrições:** • `doEstudante` distingue o que a criança escreveu do que veio do roteiro — só o estudante da própria run vê os dele.
`POST /posts/{id}/regenerar`	`professor`	Regenera o texto ou a imagem de um post.

C.26 progresso

Rota	Autorização	Semântica
`GET /progresso/{expId}`	*(qualquer)*	O progresso do estudante autenticado nesta experiência. Escopado por `escola_id` + `estudante_id` do token — nunca por parâmetro. O estudante da requisição é o único progresso que esta rota alcança. **Restrições:** • `dados: null` = a criança ainda não começou; não é erro.
`POST /progresso/{expId}`	*(qualquer)*	Salva o progresso da experiência. Substitui `dados` inteiro. É a experiência que decide o que cabe ali.
`POST /progresso/{expId}/concluir`	*(qualquer)*	Registra a conclusão da experiência. O `exp:concluir` do protocolo da bridge, gravado pelo hub. **Merge** no jsonb: nunca sobrescreve o que a experiência salvou. **Restrições:** • Isto é progresso (“terminou”), NÃO nota. Pontuação que vale nota é recalculada server-side pelo módulo de avaliação (regra 3).

C.27 prospeccao

Rota	Autorização	Semântica
`GET /prospeccao/cadastros`	`admin`	Os cadastros de interesse recebidos pelo site público. Do mais novo para o mais antigo. Alimenta `/unooria/prospeccao` **ADR-107**. A data sai em **ISO 8601**, e quem a formata é a tela, no fuso da plataforma **ADR-98**/99). **Restrições:** • Só `admin` **ADR-88**: é o funil comercial da Unooria, não da rede. • A lista vem limitada a 5000 linhas; `total` diz quantas existem de verdade — se os dois divergirem, a tela avisa em vez de mentir que aquilo é tudo. • O `id` da linha não sai: nenhuma rota o consome, e um identificador na resposta convida a uma rota de edição que não deve existir.
`GET /prospeccao/cadastros/exportacao`	`admin`	Os mesmos cadastros, em CSV. O arquivo que o `admin` baixa do painel. Separador `;` e BOM UTF-8: é o par que faz o Excel em português abrir o arquivo com as colunas separadas e os acentos certos — com vírgula, a planilha chega numa coluna só. **Restrições:** • Só `admin` **ADR-88**. • O caminho **não** termina em `.csv`, como em `/analise/exportar/:nivel`: segmento de rota com ponto quebra a validação de tipos do App Router no hub. O nome do arquivo sai no `content-disposition`. • A data sai formatada no fuso da plataforma (`FUSO_DA_PLATAFORMA`), e não em ISO: este arquivo é lido por gente, numa planilha.

Rota	Autorização	Semântica
`POST /prospeccao/publico/cadastros`	**público**	Registra um cadastro de interesse feito no site público. A rota do formulário do ENTRE REDES (`/`, **ADR-107**. Sem sessão de propósito: quem preenche é um visitante, antes de existir contrato. Grava em `prospeccao_leads` — a única tabela sem `escola_id` e sem RLS, pelas razões da migration. O aceite é gravado com a FRASE que a pessoa marcou, não como um booleano solto. **Restrições:** • Cota de 5 por hora por IP e 200 por hora na instância: acima, **429**. O IP é só chave de cota e não é registrado. • `consentimento` precisa ser exatamente `true`: sem aceite não há base legal, e o corpo é recusado com 400. • `website` é campo-armadilha. Preenchido, a resposta é **201 e nada é gravado** — recusar com erro ensinaria o robô. • Mesmo e-mail de novo dentro de 60 minutos devolve 201 sem gravar linha nova. • A resposta NUNCA diz se o e-mail já estava na base: isso transformaria a rota num verificador de quem se cadastrou.

C.28 quem-mandou-isso

Rota	Autorização	Semântica
`POST /quem-mandou-isso/concluir`	`estudante`	Encerra a missão, depois das perguntas de evidência. Não exige as respostas de P1 e P2: quem sai antes delas fica **Não avaliado** em EF05CO03/EF05CO04 (a ausência de marca), nunca "não atingiu". Depois disto as perguntas não aceitam mais resposta. **Restrições:** • A porta da sala **ADR-69** e a disponibilidade **ADR-87** valem aqui: turma sem a missão liberada responde **403** `missao-nao-liberada` (ou `recurso-indisponivel`). • **409** `reflexao-nao-enviada` antes do latch 3. Idempotente.

Rota	Autorização	Semântica
`POST /quem-mandou-isso/enquete`	`estudante`	O voto na enquete de motivo — o latch 2. Um de quatro: `urgencia`, `confianca_remetente`, `medo_indignacao`, `imagem_link_real`. Depois dele o estudante espera a revelação da turma. **Restrições:** • A porta da sala **ADR-69** e a disponibilidade **ADR-87** valem aqui: turma sem a missão liberada responde **403** `missao-nao-liberada` (ou `recurso-indisponivel`). • **409** `mural-nao-enviado` antes do latch 1. O mesmo voto de novo é idempotente; outro é **409** `enquete-ja-votada`.
`POST /quem-mandou-isso/entrar`	`estudante`	Toca "Entrar no grupo": abre a sessão desta rodada. Idempotente. Carimba `entrou_em` UMA vez — um F5 não reinicia o relógio do `tempo_feed_segundos` (entrar → confirmar o Mural). **Restrições:** • A porta da sala **ADR-69** e a disponibilidade **ADR-87** valem aqui: turma sem a missão liberada responde **403** `missao-nao-liberada` (ou `recurso-indisponivel`). • Sem pacote aprovado pelo professor responde **409** `grupo-em-preparo` — espera, não bloqueio.
`GET /quem-mandou-isso/gate`	`estudante`	A fase da turma e o passo do estudante (polling de 3 s). Leve de propósito: sem pacote e sem contas. **Polling HTTP, nunca WebSocket** **ADR-93**: a fase é uma linha em `exp05ef1b_gates_turma` que o professor move pelo painel. `agora` é o instante do SERVIDOR. **Restrições:** • A porta da sala **ADR-69** e a disponibilidade **ADR-87** valem aqui: turma sem a missão liberada responde **403** `missao-nao-liberada` (ou `recurso-indisponivel`). • `preparada: false` é a espera do preparo, não um erro.

Rota	Autorização	Semântica
`POST /quem-mandou-isso/mural`	`estudante`	"Confirmar envio" no Mural da Turma — o latch 1 (exatamente 3). O corpo leva as três que o estudante está VENDO no Mural, e são elas que fecham. Irreversível: a partir daqui a seleção não muda. Na MESMA transação, a marca da **EF05CO08** (evidência E14): 3 verdadeiras → atingiu; 2 verdadeiras e a falsa é emocional → atingiu; 2 verdadeiras e a falsa é neutra → em processo; ≤ 1 verdadeira → não atingiu. Se a marca falhar, o Mural não fecha. **Restrições:** • A porta da sala **ADR-69** e a disponibilidade **ADR-87** valem aqui: turma sem a missão liberada responde **403** `missao-nao-liberada` (ou `recurso-indisponivel`). • Sem pacote aprovado pelo professor responde **409** `grupo-em-preparo` — espera, não bloqueio. • **400** `selecao-incompleta` com menos de 3 (e os mesmos códigos do `PUT /selecao`). • Idempotente para o duplo toque: a MESMA seleção depois do latch devolve o estado sem marcar de novo; uma seleção diferente é **409** `mural-ja-enviado`.
`GET /quem-mandou-isso/professor/material`	`professor`, `gestor_escola`, `gestor_regiona` `gestor_contrat` `admin`	O guia da aula: o banco curado com gabarito, o roteiro e as perguntas de evidência. **Quem dá aula e quem administra.** O hub não guarda cópia: em paráfrase, o que sustenta a lição — a falsa NÃO parece malfeita, a verdadeira NÃO parece profissional — desaparece. É o BANCO CURADO, não o pacote da turma (esse está em `GET /quem-mandou-isso/turmas/:turmaId/pacote`). As perguntas de evidência vêm COM o nível de cada alternativa — é o professor quem lê; o estudante nunca o recebe. **Restrições:** • Nada aqui é escopado por escola: é o catálogo da missão, igual para todo contrato.

Rota	Autorização	Semântica
`POST /quem-mandou-isso/reflexao`	`estudante`	A regra Se/Então da Etapa 6 — o latch 3. `se` é uma das quatro condições; `entao` é o **único texto livre da missão**, escrito por menor de idade: vai para a tabela confinada `exp05ef1b_reflexoes`, não vai a log nem a telemetria, e só o professor que leciona na turma o lê no painel (LGPD art. 14). A regra NÃO é corrigida: a EF05CO04 vem da pergunta P2. **Restrições:** • A porta da sala **ADR-69** e a disponibilidade **ADR-87** valem aqui: turma sem a missão liberada responde **403** `missao-nao-liberada` (ou `recurso-indisponivel`). • `entao` de 15 a 240 caracteres DEPOIS do `trim`: **400** `entao-curto`, `entao-longo`, ou `entao-sem-sentido` (um caractere repetido, sequência de dígitos, pouca variedade de letras, teclado amassado). • **409** `enquete-nao-votada` antes do latch 2 e `reflexao-nao-liberada` antes de o professor liberar a reflexão. A mesma regra de novo é idempotente; outra é **409** `reflexao-ja-enviada`.
`PUT /quem-mandou-isso/selecao`	`estudante`	Guarda a triagem em andamento: de 0 a 3 mensagens para o Mural. Substitui a seleção inteira (é um `PUT`). O servidor guarda para o F5 restaurar; nada conta até o `POST /mural`. **Restrições:** • A porta da sala **ADR-69** e a disponibilidade **ADR-87** valem aqui: turma sem a missão liberada responde **403** `missao-nao-liberada` (ou `recurso-indisponivel`). • Sem pacote aprovado pelo professor responde **409** `grupo-em-preparo` — espera, não bloqueio. • **400** `selecao-acima-do-limite` com mais de 3, `selecao-repetida` com id duplicado, `mensagem-desconhecida` com id fora do pacote da turma. **409** `grupo-nao-aberto` antes de entrar e `mural-ja-enviado` depois do latch.

Rota	Autorização	Semântica
`GET /quem-mandou-isso/sessao`	`estudante`	O estado da missão e as 8 mensagens do grupo, sem o gabarito. As mensagens saem **sem 'veracidade' e 'tom'** — o tipo do servidor é um `Omit`. O gabarito só aparece em `revelacao`, e `revelacao` é `null` até o estudante VOTAR na enquete e a turma abrir a revelação; ela traz só as TRÊS mensagens dele. `pacote` é `null` enquanto o professor não aprovou um pacote para a turma: **a missão nasce vazia** e a tela é de espera. `passo` é DERIVADO no servidor (os latches da sessão × a fase da turma): `abertura` → `grupo` → `enquete` → `espera` → `revelacao` → `sintese` → `reflexao` → `perguntas` → `concluida`. `grupo` cobre o grupo E o Mural (as duas vistas da mesma seleção); `selecao` restaura um F5. **Restrições:** • A porta da sala **ADR-69** e a disponibilidade **ADR-87** valem aqui: turma sem a missão liberada responde **403** `missao-nao-liberada` (ou `recurso-indisponivel`). • Quem ainda não confirmou o Mural quando a turma abre a revelação CONTINUA no grupo; depois de votar, recebe `revelacao` junto com o passo da turma (que pode já ser `sintese` ou `reflexao`) — a tela mostra os crachás antes do passo corrente. • `textos.perguntas` são as falas da Nina que antecedem as perguntas de evidência (`p1`, `p2`). O enunciado e as alternativas vêm de `GET /evidencias/exp05ef1b/perguntas/:perguntaId` — embaralhadas, sem o nível —, e a resposta vai a `POST /evidencias/exp05ef1b/perguntas/:perguntaId/resposta`, que só aceita no passo `perguntas`. • `reflexao` é sempre a do PRÓPRIO estudante.

Rota	Autorização	Semântica
`POST / quem-mandou-isso/ turmas/{turmaId}/ aprovar`	`professor`	"Criar atividade" — aprova o pacote como uma atividade PRONTA da turma. O corpo carrega o pacote INTEIRO como o professor o deixou: a curadoria é editar qualquer texto antes de aprovar. **Aprovar um pacote novo é uma ATIVIDADE nova** da evidência (E14), e ela nasce **pronta, não no ar** **ADR-130**: o pacote em andamento continua em andamento, e a fase da turma não muda. Quem escolhe o que a turma joga é o professor (`POST :turmaId/pacotes/ :pacoteId/em-andamento`). Aprovar de novo EXATAMENTE o pacote aprovado mais recente é idempotente (mesmo id, sem atividade nova). **Restrições:** • Turma em que o professor não tem matrícula responde **404** `turma-nao-encontrada`, não 403: um "sem permissão" já contaria que a turma existe. • A régua do pacote é cobrada por regra (e some do documento, por ser `.superRefine()`): exatamente 8 mensagens (`msg_1` a `msg_8`, uma vez cada); exatamente 2 verdadeiras emocionais, 2 verdadeiras neutras, 2 falsas emocionais e 2 falsas neutras; exatamente 4 textos, 2 imagens e 2 links; a imagem e o link com UMA verdadeira e UMA falsa cada (senão o formato entrega a resposta); `legenda` obrigatória na imagem e no link e ausente no texto; texto até 45 palavras e legenda até 25; remetente de 2 a 30 caracteres; nenhum "atividade", "gabarito", "nota" nem "exercício" no que a criança lê; e o mesmo recorte do `POST /gerar`. Fora disso: **400** `pacote-invalido` com `erros`.
`POST / quem-mandou-isso/ turmas/{turmaId}/gate`	`professor`	Os três botões do console: revelação, síntese e reflexão. `liberar_revelacao` → `revelacao`, `apresentar_ sintese` → `explicar`, `liberar_reflexao` → `reflexao`. É a linha que o polling de 3 s do estudante lê **ADR-93**. A fase só anda para a FRENTE: o clique repetido ou fora de ordem não muda nada, e pular a síntese é permitido. **Restrições:** • Turma em que o professor não tem matrícula responde **404** `turma-nao-encontrada`, não 403: um "sem permissão" já contaria que a turma existe. • Sem pacote aprovado: **409** `pacote-nao-aprovado`.

Rota	Autorização	Semântica
`POST /quem-mandou-isso/turmas/{turmaId}/gerar`	`professor`	Gera com IA o RASCUNHO das 8 mensagens (pré-aula). A única chamada de IA desta missão, e ela roda **antes** da aula: em sala tudo é determinístico. A quota é a do PERFIL (regra 5): `temSaldo` ANTES, e o que o vendor cobrou — por tentativa, inclusive as rejeitadas — é debitado depois. A estrutura (o quadrante e o formato de cada posição) é SORTEADA no servidor e o modelo só escreve; a saída passa pela mesma régua da aprovação, e sair dela faz o modelo regerar. Sem saldo ou com o vendor fora, o rascunho sai do BANCO CURADO (`usouIa: false`, `motivo` `sem-saldo` ou `indisponivel`, com a disciplina do recorte). O rascunho é GRAVADO (um por turma, substituído a cada geração): o preparo custou saldo, e um F5 não pode jogá-lo fora. **Não aprova**: isso é o `POST /aprovar`. **Restrições:** • Turma em que o professor não tem matrícula responde **404** `turma-nao-encontrada`, não 403: um "sem permissão" já contaria que a turma existe. A turma é conferida ANTES de gastar token. • O recorte é cobrado por regra (e some do documento, por ser `.superRefine()`): `tema` de 3 a 60 caracteres e até 8 palavras, sem "atividade", "gabarito", "nota" nem "exercício" (ele entra na fala da Nina, dentro do clone); `assuntos` de 3 a 6 fatos, cada um de 2 a 120 caracteres, até 600 somados; `contexto` opcional, até 300. Fora disso: **400** `preparo-invalido` com `erros` (a lista inteira).
`POST /quem-mandou-isso/turmas/{turmaId}/nova-rodada`	`professor`	"Zerar Turma para Nova Aplicação" — abre a rodada seguinte, guardando o histórico. Nada é apagado: as sessões e a evidência da rodada anterior ficam (o `PainelDeHabilidades` mostra as rodadas, e o resultado é a melhor por habilidade). A turma recomeça numa sessão nova, do mesmo pacote, e a fase volta à triagem. `rodada` é a que o professor VIU no painel: o segundo clique (a atual já é a seguinte) devolve sem avançar de novo. **Restrições:** • Turma em que o professor não tem matrícula responde **404** `turma-nao-encontrada`, não 403: um "sem permissão" já contaria que a turma existe. • Sem pacote aprovado: **409** `pacote-nao-aprovado`. Rodada que não é nem a atual nem a anterior: **409** `rodada-desatualizada`.

Rota	Autorização	Semântica
`GET /quem-mandou-isso/turmas/{turmaId}/pacote`	`professor`, `gestor_escola`, `gestor_regiona` `gestor_contrat` `admin`	O pacote da turma: o ativo, o rascunho e o banco curado — COM o gabarito. **Rota de quem dá aula** — é aqui que `veracidade` e `tom` de cada mensagem aparecem, e são o roteiro do professor. `ativo` é o pacote aprovado que a turma usa (a ATIVIDADE da evidência); `rascunho` é o último gerado e ainda não aprovado; `padrao` é o banco curado "O Mistério do Passeio ao Museu", sempre à mão para "Usar Preset Curado Padrão" (a tela o carrega e aprova pelo `POST /aprovar`). **Restrições:** • Turma em que o professor não tem matrícula responde **404** `turma-nao-encontrada`, não 403: um "sem permissão" já contaria que a turma existe.
`GET /quem-mandou-isso/turmas/{turmaId}/pacotes`	`professor`, `gestor_escola`, `gestor_regional` `gestor_contrato` `admin`	As atividades da turma: os pacotes aprovados, com o em andamento marcado. A lista de atividades da ficha **ADR-130** e o seletor do painel de habilidades: cada pacote APROVADO é uma ATIVIDADE, e o `id` dele é o `atividadeId` das equipes e da evidência. O rascunho não entra — ainda não é atividade. `emAndamento` é `null` quando o professor ainda não pôs nenhum no ar: a plataforma não cria atividade. **Restrições:** • Turma em que o professor não tem matrícula responde **404** `turma-nao-encontrada`, não 403: um "sem permissão" já contaria que a turma existe. • `rodadas` é a maior rodada com sessão daquele pacote; 0 = nunca rodou.
`GET /quem-mandou-isso/turmas/{turmaId}/pacotes/{pacoteId}`	`professor`, `gestor_escola`, `gestor_regiona` `gestor_contrat` `admin`	UMA atividade, inteira e COM o gabarito — em andamento ou pronta. A página da atividade. `ativo` diz se ela está em andamento; `rodadas` é a maior rodada com sessão (0 = nunca rodou). **Restrições:** • Turma em que o professor não tem matrícula responde **404** `turma-nao-encontrada`, não 403: um "sem permissão" já contaria que a turma existe. • O rascunho, ou o pacote de outra turma, responde **404** `atividade-nao-encontrada`.

Rota	Autorização	Semântica
`POST /quem-mandou-isso/turmas/{turmaId}/pacotes/{pacoteId}/em-andamento`	`professor`	"Usar esta atividade agora" — o professor escolhe o que a turma joga **ADR-130**. O escolhido passa a ser a ATIVIDADE EM ANDAMENTO da turma; o que estava volta a ser pronto — fica, com as sessões e a evidência. A fase volta à triagem. A RODADA: o pacote que nunca rodou fica na 1; o que já rodou abre a SEGUINTE à última com sessão — retomar a mesma depois da revelação seria o "voltar" que os botões proíbem. As anteriores continuam no painel de habilidades. **Restrições:** • Turma em que o professor não tem matrícula responde **404** `turma-nao-encontrada`, não 403: um "sem permissão" já contaria que a turma existe. • **Idempotente**: pôr no ar o que já está no ar devolve `mudou: false` e não muda nada. • O rascunho, ou o pacote de outra turma, responde **404** `atividade-nao-encontrada`.
`GET /quem-mandou-isso/turmas/{turmaId}/painel`	`professor`, `gestor_escola`, `gestor_regiona` `gestor_contrat` `admin`	O painel ao vivo: o progresso, o Termômetro da Turma e a folha nominal. **Por COMPUTADOR quando há equipe** (`porComputador: true`): cada linha é uma unidade — a de quem jogou, com a equipe junto —, e os números contam unidades. Antes da revelação, SÓ progresso: `termometro` é `null`. Depois: % das unidades que encaminharam ≥ 1 falsa neutra; % das verdadeiras emocionais NÃO encaminhadas, (2•N - VE encaminhadas)/(2•N); e o cruzamento motivo × "encaminhou falsa neutra". **Nenhum nome no Termômetro.** `atividade` é o que o `PainelDeHabilidades` (E14) recebe: `GET /evidencias/turma/:turmaId/missao/exp05ef1b/atividade/:pacoteId`. **Restrições:** • Turma em que o professor não tem matrícula responde **404** `turma-nao-encontrada`, não 403: um "sem permissão" já contaria que a turma existe. • `reflexao.entao` é TEXTO LIVRE DE MENOR: preenchido só para o professor que leciona na turma. Gestor e admin recebem a mesma linha com `entao: null` (LGPD art. 14). • Sem coluna de acerto por criança: o nível por habilidade é da evidência (E14), onde o professor também o ajusta.

C.29 rastro-boato

Rota	Autorização	Semântica
`POST /rastro-boato/auditoria`	`estudante`	Registra a abertura da janela “Dados da mensagem”. A cota de 3 é sobre mensagens **DISTINTAS**, e a dedupe é do servidor — um contador incrementado no cliente seria um número que o estudante controla. Reabrir o mesmo balão devolve 200 com `novo: false`: reler é o comportamento que a missão quer, e um 409 ali viraria a releitura em erro na tela. **Restrições:** • Quando a terceira mensagem distinta fecha a cota, o `passo` anda para `decisao` AQUI, no servidor. Deixar o cliente pedir a transição seria deixá-lo escolher quando o pico da missão começa. • `tempoLeituraS` é calibragem de fluxo, não medida de mérito: ninguém é avaliado por ler rápido.
`POST /rastro-boato/decisao`	`estudante`	A intervenção no grupo — irrevogável (latch). O pico da missão. A api confere a cota de auditoria ANTES de aceitar (409 `auditoria-insuficiente`), corrige a escolha contra o catálogo e devolve o desfecho, a reação do grupo e a topologia da rede já calculada. Segunda chamada responde 409 `decisao-ja-registrada`: a escolha é definitiva, e é isso que faz o estudante assumir o peso dela antes de ver o mapa. **Restrições:** • `latenciaSeg` é cronometrado pelo CLIENTE — a latência da api não é o dado. O que se mede é o tempo entre a crise aparecer e o toque. • `houveInacao90s` alimenta a hipótese H3 do playtest e o painel; nunca entra no nível de intervenção. • A topologia vem AGRUPADA em anéis (`nos` real, `desenhar` quantos círculos): 1.200 elementos SVG travariam o Chromebook da escola.
`POST /rastro-boato/entrar`	`estudante`	Sai da tela de contexto e abre a lista de conversas. Idempotente. Existe como rota para o `passo` andar no SERVIDOR: um F5 na Etapa 2 não pode devolver a criança à abertura de Tomás que ela já leu.

Rota	Autorização	Semântica
`GET /rastro-boato/gate`	`estudante`	Em que etapa a turma está (polling de ~5s). Leve de propósito: sem catálogo. É o que deixa a docente conduzir 30 telas — a Aula 2 abre para todos ao mesmo tempo. `agora` é o instante do SERVIDOR: o relógio da máquina da escola atrasa, e não é ele que decide o momento da aula. **Restrições:** • Estudante sem turma não tem gate: a missão corre no ritmo dele. Prendê-lo no breakpoint seria um beco sem saída, porque não há quem dê a largada.
`POST /rastro-boato/modelagem`	`estudante`	O modelo do ciclo da desinformação — 4 blocos, sem nota. A api confere a sequência contra a ordem do catálogo e devolve `coerente`. **Não há penalidade**: sequência incoerente grava igual e a missão segue (RF-007). O que recusa é modelo INCOMPLETO — 400 `blocos-incompletos` para quantidade errada, bloco desconhecido ou repetido (repetir deixa um slot logicamente vazio). **Restrições:** • A ordem correta nunca sai do servidor: o cliente recebe os blocos e devolve a sequência que montou. • Exige decisão enviada (409 `sem-decisao`): o construtor sistematiza o que aconteceu no grupo, e sem a cena não há o que sistematizar.
`GET /rastro-boato/professor/material`	`professor`, `gestor_escola`, `gestor_regiona` `gestor_contrat` `admin`	As três intervenções com o gabarito e o ciclo ordenado, para o guia da aula. **Professor only.** Alimenta o guia da aula em `/professor/entre-redes/6-ano/unidade-2/guia`. `desfecho` é o mesmo gabarito que a tela do estudante só conhece do PRÓPRIO caso, depois de decidir; `ciclo` chega ordenado aqui e embaralhado lá, porque montar a ordem é a tarefa dela. **Restrições:** • Nada aqui é escopado por escola: é o catálogo da missão, igual para todo contrato.

Rota	Autorização	Semântica
`POST /rastro-boato/reflexao`	`estudante`	O “Meu percurso” — encerra a missão (latch). Texto livre de até 280 caracteres. É o **único** campo livre da missão e o dado mais sensível dela: caixa escrita por menor de idade. Não vai a log, não vai à telemetria analítica, e a única rota que o devolve é o painel do professor DA TURMA dela (LGPD art. 14). Grava `concluido_em` e leva o `passo` a `fim`. Segunda chamada responde 409 `reflexao-ja-enviada`. **Restrições:** • O `trim` é anterior ao `min(1)`: um campo com três espaços é um percurso em branco, e ele contaria como entregue no painel. • Exige o modelo do ciclo confirmado (400 `blocos-incompletos`).
`GET /rastro-boato/sessao`	`estudante`	O estado da missão, as 4 conversas e os metadados de cada mensagem. As 3 opções de intervenção saem **sem ‘desfecho‘ e sem ‘reacao‘**, e os 4 blocos do ciclo saem **sem a ordem certa** — os tipos do servidor são `Omit<...>`, então o gabarito é impossível de vazar aqui, não só ausente por descuido. O feed vem da api, e não do bundle, para que corrigir uma palavra da copy não exija republicar um caminho imutável no bucket. **Restrições:** • `passo` já vem reconciliado com a etapa da turma; `passoDoEstudante` é o cru. Quando os dois divergem e `aguardandoATurma` é `true`, o estudante está preso no breakpoint esperando a Aula 2. • `topologia` e `desfecho` são `null` até a intervenção ser enviada. Devolvê-los antes entregaria qual opção quebra a corrente. • `metadados.saltos` é `null` numa mensagem escrita ali mesmo — ausência de encaminhamento não é zero encaminhamentos, e a tela mostra as duas coisas de formas diferentes.
`GET /rastro-boato/turmas`	`professor`, `gestor_escola`, `gestor_regional`, `gestor_contrato`, `admin`	Turmas para o seletor do painel. Filtradas por MATRÍCULA para o professor; o gestor vê a escola. `gestor_contrato` precisa dizer `?escolaId=`.

Rota	Autorização	Semântica
`DELETE /rastro-boato/turmas/{turmaId}`	`professor`, `gestor_escola`, `gestor_regiona` `gestor_contrat` `admin`	Zerar a turma — a segunda aplicação. Apaga sessões, auditorias, percursos e o gate desta turma para a missão poder ser refeita no ano seguinte (PED-13). **Preserva 'telemetria_eventos'**, que é a trilha analítica imutável. O percurso vai junto, e isso é deliberado: refazer a missão com a reflexão da turma do ano passado na tela seria um portfólio que mistura duas turmas.
`POST /rastro-boato/turmas/{turmaId}/gate`	`professor`, `gestor_escola`, `gestor_regional`, `gestor_contrato`, `admin`	Abre a Aula 2, ou pausa a simulação de volta na Aula 1. O relógio da AULA. É o que permite fechar a Aula 1 em 30 telas ao mesmo tempo e fazer a turma chegar junta ao mapa da rede. **Idempotente de propósito**: `liberar-aula2` duas vezes é exatamente o clique que se dá duas vezes, e a segunda não pode derrubar nada. `pausar-simulacao` volta para `aula1` SEM apagar nada — é para quem abriu a Aula 2 antes da hora. **Restrições:** • A etapa é PISO e TETO ao mesmo tempo: quem está adiantado não é puxado de volta (seria punir quem leu rápido), mas ninguém passa do breakpoint antes da Aula 2. • Quem já entregou o percurso fica em `fim`: `pausar-simulacao` não o traz de volta. O latch é do estudante, não da turma.
`GET /rastro-boato/turmas/{turmaId}/painel`	`professor`, `gestor_escola`, `gestor_regiona` `gestor_contrat` `admin`	O que a turma está fazendo na missão, ao vivo. Uma linha por estudante mais o resumo agregado. **Sem coluna de pontos e sem nível** (PED-07 e `PROIBIDO_NO_PAINEL`): a leitura é de PROGRESSO. Uma coluna de pontos ordenaria crianças na leitura ainda que a tela não ordenasse. `distribuicao` conta A/B/C entre quem respondeu — é o material do debate, nunca 'quem errou'. `reflexaoTexto` sai NESTA rota e só nela: é caixa livre de menor de idade, e quem a lê é o professor da turma dela (LGPD art. 14). **Restrições:** • `ociosoMin` alimenta o alerta de inatividade (> 3 min). Serve para o professor ir até a carteira, não para punir. • `estouraramOAlerta` conta quem deixou o timer de 90s aparecer no pico. É o sinal da hipótese H3 do playtest, e um número alto pede intervenção oral — não uma anotação sobre a criança. • `auditaramTudo` é o KPI-01 da missão: quantos inspecionaram 3 ou mais metadados antes de intervir.

C.30 readyz

Rota	Autorização	Semântica
`GET /readyz`	**público**	As dependências estão prontas (banco). Banco fora do ar responde **503**, não `ok: false` — o balanceador tira a instância do rodízio pelo código, não pelo corpo.

C.31 recursos

Rota	Autorização	Semântica
`PUT /recursos/{recursoId}`	`admin`	Liga ou desliga um recurso num escopo. Grava a linha de `disponibilidades` daquele (recurso, escopo). Idempotente por `upsert` — o índice único PARCIAL `WHERE escola_id IS NULL` é o que dá idempotência ao escopo-contrato, onde o único composto não bastaria. **Restrições:** • **Desligar exige 'motivo'** (Zod aqui, CHECK no banco). `Desligado sem motivo` é o estado que ninguém sabe desfazer meses depois. • `escolaId: null` no corpo é o CONTRATO inteiro — a rede toda. Ele é explícito de propósito: numa querystring, ausente e vazio são indistinguíveis, e desligar a rede não pode acontecer por URL mal montada. • `versao_fixa` não se escreve por aqui: produto não tem versão, e a fixação por escopo é outra rota (fase 3).
`DELETE /recursos/{recursoId}`	`admin`	Remove a exceção de um escopo (volta a herdar). Apaga a linha daquele (recurso, escopo). É o terceiro estado do interruptor: **herda** não é `ligado` — é a ausência de decisão neste nível, e sem esta rota uma exceção criada por engano seria permanente. **Restrições:** • Sem `?escolaId=`, remove a linha do CONTRATO. • Remover o que não existe devolve `removida: false` e **não é erro**: o estado final pedido — não haver linha aqui — é o que já valia.

Rota	Autorização	Semântica
`GET /recursos/arquivadas`	`admin`	As experiências arquivadas: tiradas do produto sem serem apagadas **ADR-116**. Sai de `experiencias_arquivadas`. Uma experiência é arquivada quando uma missão nova toma o código dela (o código é derivado da série e da unidade, **ADR-110**: tudo o que era dela ganha o prefixo `backup-` e some da grade, do catálogo e deste painel, com os dados guardados. **Só lista** — ninguém joga uma arquivada, e desarquivar é uma migração. **Restrições:** • `turmasGuardadas` e `estudantesGuardados` são a FOTOGRAFIA tirada pela migração que arquivou, não uma recontagem: contar `liberacoes` e `progresso` de todas as escolas atravessaria a RLS. • A ordem é a do arquivamento, da mais recente para a mais antiga.
`GET /recursos/auditoria`	`gestor_contrat admin`	Quem ligou ou desligou o quê, e quando. O registro **append-only** das mudanças de disponibilidade e de versão. Existe porque a linha de `disponibilidades` é SOBRESCRITA a cada mudança: ela responde o estado atual, não o histórico. `UPDATE` é recusado por trigger no banco. **Restrições:** • Aberta a `admin` E `gestor_contrato` — a única de `/recursos/*` que a rede alcança. Escrever continua sendo só do `admin` **ADR-88**. • `autorNome` vem CONGELADO da linha, não de um JOIN: o histórico tem de dizer quem era aquela pessoa NAQUELE dia, e não o nome de hoje (ou nada, se a conta saiu). • Sem `?escolaId=` traz o contrato inteiro, incluindo as mudanças de escopo-contrato (`escolaId: null`). COM ele, traz as daquela escola **e** as do contrato, porque foram elas que decidiram o que vale ali. • Teto de 200 linhas, dito na resposta (`truncado`) — mesmo desenho de `GET /rostering/contas-de-assento`. • `?prefixo=quota:` recorta as mudanças de quota de IA **ADR-105**, que a tela `/unooria/uso-de-ia` mostra sozinhas. `detalhe` (`{ de, para }`) só vem nessas linhas.

Rota	Autorização	Semântica
`GET /recursos/efetivos`	*(qualquer)*	O que está disponível para a escola do token. Resolve a cadeia escola → regional → contrato e aplica o TETO do produto. É daqui que o hub monta os cards do professor e decide o 404 das rotas de produto — **esconder o card é UX**; a porta é `exigirMissaoLiberada` / `DisponibilidadeGuard`, no servidor. **Restrições:** • `produtos` traz SEMPRE todos os produtos do vocabulário (hoje cinco), ligados ou não: um mapa que só lista o ligado obriga quem lê a saber a lista completa de cor. • `agente-professor` é produto SEM experiência: o hub lê `produtos["agente-professor"].ligado` para montar (ou não) o botão flutuante do layout do professor. • Perfil de contrato (`gestor_contrato`, `admin`) não tem escola no token, então a resposta é a do CONTRATO — a cadeia fica vazia e só as linhas de escopo-contrato valem. • Não devolve `motivo` nem quem desligou: isso é conversa entre a Unooria e a rede, e vive no painel.
`POST /recursos/experiencias/{expId}/ativar`	`admin`	Ativa uma versão para o CONTRATO (o rollback). Troca `experiencias.versao_ativa` e **restaura o manifesto daquela versão** — `viewport`, `permissoes`, `series`, `unidade`, `produto`. Antes disto, rollback era `publicar-manifesto.mjs --versao x.y.z`, que mandava o número antigo com o manifesto ATUAL do repositório: voltar a versão não voltava as permissões de iframe, e a app-70 reabria sem `allow-pointer-lock` — a criança entrava na Vila e a câmera não girava, sem erro em lugar nenhum. **Restrições:** • **Confere no CDN antes de gravar** (um HEAD no `index.html` da versão). Apontar a rede para uma versão que não subiu é tela branca para todo mundo, e o erro só apareceria quando a primeira criança abrisse a missão. • `conferida: false` = `CDN_EXP_URL` ausente (desenvolvimento). A tela precisa dizer isso: 'ativei sem conferir' e 'ativei e conferi' são garantias diferentes. • Vale para o CONTRATO inteiro. Fixar versão numa escola é `PUT /recursos/experiencias/:expId/versao`.

Rota	Autorização	Semântica
`PUT /recursos/experiencias/{expId}/versao`	`admin`	Fixa (ou solta) a versão de uma experiência num escopo. Grava `disponibilidades.versao_fixa`. É o canário: uma escola roda a versão nova enquanto a rede segue na anterior. `versao: null` solta a fixação e volta a seguir a `versao_ativa`. **Restrições:** • Fixar num escopo que não tinha linha CRIA a linha como `ligado`. É decisão explícita: quem fixa versão para uma escola está dizendo que a experiência vale ali. • Confere no CDN, como a ativação — e pelo mesmo motivo. • O CHECK do banco recusa `versao_fixa` em linha de `produto:*`: produto não tem bundle, logo não tem versão.
`GET /recursos/experiencias/{expId}/versoes`	`admin`	As versões publicadas de uma experiência. Sai de `experiencias_versoes`, que o `PUT /experiencias/:expId` do CI escreve a cada publish (e o seed em desenvolvimento). É a lista do **rollback pela tela** — antes dela, voltar uma versão exigia saber o número anterior de cabeça. **Restrições:** • `emRollback` é `true` quando a ativa não é a última publicada. É o estado que a tela tem de dizer: sem ele, o próximo deploy 'conserta' o rollback sem ninguém perceber. • O `manifesto` de cada versão NÃO vem na resposta: ele é insumo da ativação, não informação de tela.
`GET /recursos/painel`	`admin`	O painel: produtos, experiências e o que vale em cada escopo. Resolve a cadeia escola → regional → contrato e devolve, por recurso, **o que o interruptor daquele escopo mostra** (`estadoAqui`, três estados) e **o que efetivamente vale** (`efetivo`, com a origem). Sem `?escolaId=`, o escopo é o CONTRATO inteiro. **Restrições:** • `estadoAqui: herda` é a AUSÊNCIA de linha neste escopo — é o terceiro estado, e sem ele desfazer uma exceção seria impossível. • `tetoDoProduto` marca a experiência que está desligada pelo PRODUTO, não por linha própria: a tela desabilita o interruptor dela e escreve o motivo ao lado, em vez de mostrá-lo ligado. • `versaoAtiva` é a que o hub monta hoje; `versaoFixa` é a fixada NESTE escopo (`null` = segue a ativa).

Rota	Autorização	Semântica
`GET /recursos/quotas-de-ia`	`admin`	O teto diário de IA de cada perfil. Uma linha por perfil de `ia_quotas` (`estudante`, `professor`), escopo do contrato inteiro. É o número que o `QuotaService` consulta ANTES de toda chamada de IA (regra 5). **Restrições:** • Só `admin` **ADR-88**/105). O contador do dia (`ia_uso_diario`) NÃO sai aqui: é dado por pessoa. • Todo perfil que não é `estudante` cai na linha `professor` — é o que já valia com as envs, agora escrito.
`PUT /recursos/quotas-de-ia/{perfil}`	`admin`	Define o teto diário de IA de um perfil. Sobrescreve a linha do perfil em `ia_quotas`. **Motivo obrigatório sempre**: aqui toda mudança é exceção ao padrão da plataforma, e é o motivo que, meses depois, diz se foi decisão comercial ou incidente. A mudança entra no histórico do painel (`recursos_auditoria`, ação `quota-alterada`, com `{ de, para }`). **Restrições:** • Zero e negativo são recusados (Zod e CHECK): fechar a IA de um produto é o interruptor da **ADR-87**, não a quota. • Teto de sanidade de 100 milhões de tokens/dia — um zero a mais no campo não vira quota infinita. • Valor igual ao atual devolve `alterada: false` e não grava nem historia: nada mudou. • Vale para a REDE INTEIRA na próxima chamada de IA: o `QuotaService` lê a linha a cada `temSaldo`, sem cache.

C.32 rostering

Rota	Autorização	Semântica
`GET /rostering/contas-de-assento`	`gestor_escola`, `gestor_regional`, `gestor_contrato`, `admin`	As contas sem matrícula da rede — as que podem ser de assento. As experiências vieram de projetos onde a conta era do APARELHO: outra turma reusava o mesmo login. Esta lista mostra quanto cada conta acumulou de dado, para o gestor decidir o que desativar. **Restrições:** • **Não existe fundir contas**: juntar duas move o que uma criança fez para outra pessoa, e depois ninguém audita de quem era. • Teto de 200 linhas, dito na resposta em `truncado` em vez de cortado em silêncio.
`POST /rostering/importar`	`gestor_escola`, `gestor_regiona` `gestor_contrat` `admin`	Importa a planilha de pessoas. Corpo é o **CSV cru** (`Content-Type: text/csv`), até 30MB. Uma linha por pessoa, cabeçalho lido por NOME (ordem livre): `matricula,nome,usuario,email,papel,turma,serie,turno`. Só `nome` e `papel` são obrigatórios — estudante também exige `turma` e `serie`. Idempotente: a chave é a MATRÍCULA da pessoa na rede e, na falta dela, o login. **Restrições:** • **O CPF é a primeira chave**: achado em outra escola da rede, a linha vira transferência (estudante) ou lotação a mais (professor). • Reimportar NÃO reseta a senha de quem já entrou — seria trancar a escola inteira para fora. • Coluna desconhecida é ERRO, não coluna ignorada. • 5 importações por minuto por usuário (429); acima de 30MB ou 50k linhas, 400 — o teto cabe debaixo dos 32 MiB do Cloud Run, para o 413 do ambiente nunca ser o que o gestor vê.
`GET /rostering/modelos/exemplo.csv`	`gestor_escola`, `gestor_regional`, `gestor_contrato`, `admin`	O mesmo modelo, preenchido com um exemplo.
`GET /rostering/modelos/oneroster/{arquivo}`	`gestor_escola`, `gestor_regiona` `gestor_contrat` `admin`	O modelo em branco de um arquivo do OneRoster 1.2. Só o cabeçalho, na grafia do padrão. É um SUBCONJUNTO das colunas que a api sabe ler — as que ela ignora ficam de fora, porque modelo com 24 colunas não é modelo, é a especificação. **Restrições:** • Modelo e exemplo são arquivos SEPARADOS: linha de exemplo dentro do modelo vira pessoa fantasma importada no primeiro uso.

Rota	Autorização	Semântica
`GET /rostering/modelos/oneroster/{arquivo}/exemplo`	`gestor_escola`, `gestor_regional`, `gestor_contrato`, `admin`	O mesmo arquivo do OneRoster, preenchido com um exemplo. Gerado com a escola de VERDADE no `sourcedId` do `orgs.csv` e no `schoolSourcedId` das turmas: um id fictício faria o lote voltar como "escola não é a desta importação" para o gestor e, para o `gestor_contrato` (que PODE criar escola por CSV), abriria um tenant de mentira.
`GET /rostering/modelos/usuarios.csv`	`gestor_escola`, `gestor_regiona` `gestor_contrat` `admin`	O modelo em branco da planilha de pessoas. Gerado da MESMA constante que o leitor consome — um modelo escrito à mão divergiria do parser no primeiro campo novo, e o sintoma seria "importei o modelo de vocês e deu erro de cabeçalho". **Restrições:** • Sai com BOM e `charset=utf-8`: um sem o outro ainda quebra acento em alguma planilha.
`POST /rostering/oneroster/{arquivo}`	`gestor_escola`, `gestor_regional`, `gestor_contrato`, `admin`	Binding CSV do OneRoster 1.2 — UM arquivo por chamada. Ordem: `manifest`, `orgs`, `academicSessions`, `classes`, `users`, `enrollments`, `roles`. **Fora de ordem não corrompe nada**: a linha que aponta para um `sourcedId` inexistente volta no relatório dizendo QUAL arquivo falta. O padrão vive só nesta fronteira: nada do domínio conhece `sourcedId`. **Restrições:** • `orgs.csv` só APRENDE o código INEP desta escola: escola é *tenant*, não nasce de upload. Criar escola por CSV é só `gestor_contrato`. • Sem a coluna `role` (que saiu de `users.csv` na 1.2) a pessoa entra como **estudante**, o menor privilégio; o `roles.csv` corrige. • A coluna `password` do padrão é IGNORADA, e a contagem vai ao relatório. • Em `bulk`, ausência de linha **não desativa ninguém** — é contada em `candidatosAInativacao`: lote que chegou pela metade não pode apagar uma rede por omissão. • `status = tobedeleted` desativa (pessoa) ou encerra (matrícula), nunca apaga.

C.33 runs

Rota	Autorização	Semântica
`POST /runs`	`estudante`	Cria a run do estudante na atividade ativa. A LARGADA: é aqui que o lado do feed é decidido — pelo perfil do Ato 1, por redistribuição do professor, ou por sorteio balanceado quando não há perfil.
`GET /runs/{id}/busca`	`estudante`	Busca no acervo da atividade. Usar a busca é o sinal de INVESTIGAÇÃO que o relatório conta — por isso ela é uma rota, e não um filtro no cliente.
`POST /runs/{id}/conclusao`	`estudante`	A escolha de um lado no fim da missão. Enviar duas vezes é **409**, com linguagem de app real: "isso já foi enviado", nada de "fase" ou "atividade".
`GET /runs/{id}/dossie`	`estudante`, `professor`	O dossiê da sessão — o relatório da revelação. Aqui o disfarce JÁ CAIU, e por isso o `lado` de cada post pode aparecer. A seção mais importante é `explicacaoDoLado`: **por que o feed começou daquele lado**. **Restrições:** • `conclusao` é anexada na LEITURA, não no congelamento: o dossiê é computado no `finish`, antes de o estudante escrever. `undefined` quando a sessão foi interrompida — e o relatório precisa funcionar assim. • Os rótulos de perfil vêm prontos no payload em vez de o cliente importar as FICHAS: elas carregam o vocabulário dos prompts, que não tem por que viajar até o navegador. • `imersao.total` nunca vem sem os componentes.
`POST /runs/{id}/entrar`	`estudante`	O estudante entra no aparelho e vai ao feed. `nome` é APELIDO dentro da ficção do Instanooria — nunca identidade em tela de professor ou relatório.
`POST /runs/{id}/eventos`	`estudante`	Lote de eventos do cliente — o canal OPERACIONAL. Síncrono e transacional, lido em tempo quase real para curar o próximo bloco. **'dwell' nunca passa por Pub/Sub**: latência e falta de ordenação quebrariam a adaptação e o lock por run. **Restrições:** • Idempotência por `(client_ts, tipo, postId)`: `descartados` conta o que já existia — reenvio é rotina no cliente, não erro. • Cota por SESSÃO, não por IP: a turma toda compartilha a rede da escola.
`GET /runs/{id}/fase`	`estudante`	A tela que o estudante deve estar vendo agora. Já reconciliada com a etapa da TURMA — o cliente faz polling porque os momentos coletivos são do professor, não do estudante.

Rota	Autorização	Semântica
`GET /runs/{id}/feed`	`estudante`	O próximo bloco do feed. O post sai **sem 'lado', sem 'perfil' e sem 'intensidade'** — o disfarce depende disso: um bundle que soubesse o lado de cada post entregaria o experimento antes da revelação. O `diagnostico` do motor (domínio, lado dominante, bloco) fica no SERVIDOR. **Restrições:** • Lock consultivo por run: duas abas pedindo bloco ao mesmo tempo se sobrescreviam, e um bloco se perdia.
`POST /runs/{id}/finish`	`estudante`	Congela a run e computa o dossiê. Encerrar antes da hora é **409** — e a mensagem é de app real ("sua sessão ainda está rolando"), não de sistema.
`GET /runs/{id}/perdidos`	`estudante`	O que o estudante PERDEU — posts do lado oposto que nunca foram servidos. **Só das etapas 'perdidos' em diante**: antes disso derrubaria o disfarce, que é o que a atividade inteira depende de manter até a revelação.
`POST /runs/{id}/reacoes`	`estudante`	Reações das personas ao que o estudante escreveu. Nunca falha para o estudante: sem IA, sem cota ou com erro, devolve `reacoes: []` e diz por quê. **Restrições:** • `desligada: true` = o professor desligou a IA ao vivo. • `limite: true` = teto de 12 reações por sessão — estudante comentando sem parar não queima a cota da turma.
`GET /runs/{id}/reflexao`	`estudante`	O texto da reflexão que o estudante já escreveu. Lê a reflexão desta run — a dele, e só a dele: o `where` casa `estudanteId` do token junto com o id. **Restrições:** • `texto: ""` quando ainda não escreveu — o campo abre vazio, não com erro.
`POST /runs/{id}/reflexao`	`estudante`	O fechamento escrito da atividade. É sobre a ATIVIDADE (letramento digital), não sobre a pergunta-chave — a conclusão já foi a escolha de um lado. **Restrições:** • 60 caracteres é o PISO: passa "o algoritmo me mostrou mais de um lado" e barra "gostei". Não é medida de qualidade, é filtro de vazio.

Rota	Autorização	Semântica
`POST /runs/{id}/reset`	`professor`	Reabre a run de um estudante, sorteando a largada de novo. Rota de PROFESSOR: é o socorro para a sessão que travou no meio da aula.
`GET /runs/{id}/salvos`	`estudante`	A coleção do estudante — o que ele salvou. O ÚLTIMO evento por post decide: salvar e dessalvar se anulam. É a coleção que ele vai defender no debate.
`POST /runs/{id}/student-post`	`estudante`	O estudante publica no app, e as personas reagem. **Falha fechada**: IA desligada, erro de rede ou reação recusada ⇒ lista vazia, e o post é publicado do mesmo jeito — como num app real quando ninguém responde. A chamada de IA fica FORA da transação, de propósito.
`GET /runs/boot`	`estudante`	O bootstrap do aparelho — o que a tela de entrada precisa saber. Só os RÓTULOS dos lados — os únicos campos do antagonismo que o cliente pode ver antes da revelação. Zero lado/intensidade dos posts.

C.34 salas

Rota	Autorização	Semântica
`GET /salas/minhas`	`estudante`	As salas do estudante: as turmas dele e a grade de cada uma. Devolve a grade INTEIRA da série da turma, não só o que está aberto: missão bloqueada aparece com **cadeado** e a encerrada aparece com o rótulo. Quem decide o que abre é o `estado`. **Restrições:** • `bloqueada` **não é um estado do banco** — é a ausência de linha em `liberacoes`. • Sem turma, `salas` volta VAZIO: sem sala não há missão, e é a tela que explica que falta enturmar. • Esconder o card é UX. A porta é `exigirMissaoLiberada`, na entrada de cada experiência — `/exp/:expId` é endereço adivinhável.

Rota	Autorização	Semântica
`GET /salas/turma/{turmaId}`	`professor`, `gestor_escola`, `gestor_regiona` `gestor_contrat` `admin`	A sala: a turma, a grade de missões e quem liberou cada uma. A grade é a das experiências da SÉRIE da turma mais as transversais (`series: []`). `?todas=1` acrescenta as de outras séries, marcadas com `deOutraSerie` — turma multisseriada e reforço existem, mas trazê-las por padrão afogaria a grade do ano. **Restrições:** • Turma em que o professor não tem matrícula responde **404**, não 403: um "sem permissão" já contaria que a turma existe. • Gestor e admin-contrato leem a escola inteira, de propósito.
`PUT /salas/turma/{turmaId}/missoes/{expId}`	`professor`	Libera ou encerra uma missão nesta turma. **Só quem tem matrícula de professor na turma.** Gestor não libera: ele vê a sala e não decide a aula — a opção não aparece na tela dele e a rota responde 404 para turma que não é dele. Idempotente: repetir não muda nada além do carimbo. **Restrições:** • Não existe estado `bloqueada` no corpo: bloquear é a ausência de linha, e o caminho de volta é `encerrada` — a criança precisa continuar vendo o que fez. • Recusa experiência de `publico: professor` (app-90, app-91): liberar uma delas seria um botão que não abre nada, porque `/exp/*` responde 404 para ferramenta de professor.

C.35 telemetry

Rota	Autorização	Semântica
`POST /telemetry/batch`	*(qualquer)*	Recebe um lote do canal ANALÍTICO da SDK. Publica no Pub/Sub, que entrega em `POST /telemetry/ingest`. É o canal analítico — **não** o operacional: `dwell` e o feed adaptativo do motor passam por `/runs/:id/eventos`, que é síncrono e ordenado. **Restrições:** • `estudanteId` e `escolaId` NÃO vêm do corpo: são acrescentados a partir do token. Aceitá-los do cliente deixaria um bundle gravar evento no nome de outra criança. • `recebidos` conta o que foi PUBLICADO, não o que foi gravado — a gravação acontece do outro lado do Pub/Sub.
`POST /telemetry/beacon`	**token no corpo**	Último lote de uma sessão que está fechando. Chamada no `pagehide` com `navigator.sendBeacon`. **Responde 204 SEMPRE**: a página já foi descartada e ninguém lê o corpo — erro aqui só poluiria o log do cliente. Corpo inválido ou token que não confere viram descarte silencioso, com aviso no servidor.
`POST /telemetry/ingest`	**push**	Consome a push subscription do Pub/Sub e grava os eventos. Push, e não um worker próprio, é uma peça a menos para operar. Grava em `telemetria_eventos` (particionada por mês, schema `particoes`). **Restrições:** • `ON CONFLICT (event_id, ts) DO NOTHING`: o Pub/Sub entrega *at least once*, então reentrega é rotina, não exceção. • UM INSERT multi-linha por escola, não um por evento — no pico da manhã o laço custava ~800 INSERTs/s. • Sem `TELEMETRIA_PUSH_SA`/`TELEMETRIA_PUSH_AUDIENCE` a rota responde 401, nunca aberta: ela escreve dado de estudante.

C.36 turmas

Rota	Autorização	Semântica
`GET /turmas`	`gestor_escola`, `gestor_regional`, `gestor_contrato`, `admin`	As turmas da escola. Filtra por período letivo quando `periodoLetivoId` vem. Sem ele, TODAS as turmas de todos os anos — é isso que preserva o histórico depois da virada.

Rota	Autorização	Semântica
`POST /turmas`	`gestor_escola`, `gestor_regiona` `gestor_contrat` `admin`	Cria uma turma. Sem `periodoLetivoId`, nasce no período VIGENTE — derivado das datas. Sem nenhum período cadastrado, **400**: turma sem período é a coluna `users.turma_id` de novo, com outro nome. **Restrições:** • `UNIQUE (escola_id, periodo_letivo_id, nome)`: o “8A de 2026” e o “8A de 2027” são turmas DIFERENTES.
`GET /turmas/{id}`	`professor`, `gestor_escola`, `gestor_regional` `gestor_contrato` `admin`	Os matriculados da turma — a lista nominal. Para o PROFESSOR a checagem é de MATRÍCULA, não de escola: o professor do 8A pedindo o 9B leva **403**. Antes disto, qualquer professor da escola lia qualquer turma dela — a única prova exigida era “mesma escola”, que é a regra de tenancy e não a de quem dá aula ali. **Restrições:** • `matriculaId` é da MATRÍCULA e `id` é da PESSOA — mover e encerrar operam sobre o primeiro. • `senhaProvisoria: true` = ainda não fez o primeiro acesso.
`PATCH /turmas/{id}`	`gestor_escola`, `gestor_regiona` `gestor_contrat` `admin`	Edita nome, série ou turno da turma. Não move de período: isso seria reescrever o histórico do ano.
`POST /turmas/{id}/ matriculas`	`gestor_escola`, `gestor_regional` `gestor_contrato` `admin`	Matricula uma pessoa na turma. `papel` só admite `estudante` e `professor`. Gestor é papel da ESCOLA, não matrícula de turma — e é por isso que ele continua vendo a escola inteira.
`GET /turmas/minhas`	`professor`, `gestor_escola`, `gestor_regiona` `gestor_contrat` `admin`	As turmas DO PROFESSOR — as que ele tem matrícula, e só elas. Declarada antes de `:id` de propósito: no Nest a primeira rota que casa vence, e `/turmas/minhas` bateria em `/turmas/:id` com o id “minhas”. Gestor e admin-contrato caem nas turmas da escola inteira — é a diferença entre administrar a escola e dar aula numa turma.

C.37 unoogpt

Rota	Autorização	Semântica
`GET /unoogpt/agregado/escola`	`gestor_escola`, `gestor_regional`, `gestor_contrato`, `admin`	O agregado da escola — uma linha por turma. Controller separado do professor de propósito. A alternativa — um `@Roles` misto com exceção por método — é como uma rota individual acaba aberta para gestor sem ninguém notar. **Nada aqui identifica uma criança.** **Restrições:** • `gestor_contrato` precisa dizer `?escolaId=`; gestor só vê a própria escola (403).
`POST /unoogpt/alocacao`	`estudante`	Manda uma equipe para um bairro. `bairro: null` tira a equipe do destino. A justificativa é **portão de servidor**: só vale fonte que o estudante carregou, ou "foi intuição" — que está sempre disponível, porque sem essa saída a pergunta vira pegadinha e o estudante aprende a mentir para o formulário.
`PUT /unoogpt/conjuntos`	`estudante`	Carrega fontes no modelo pelo painel. `PUT` porque é atribuição de conjunto. As camadas ACUMULAM.
`POST /unoogpt/contrafactual`	`estudante`	"E se eu tivesse mandado para…" — calculado e descartado. **Não grava e não debita**: é a pergunta que o estudante faz ao VER o resultado, e transformá-la em tentativa nova apagaria a diferença entre a decisão que ele tomou e a que ele imaginou depois.
`POST /unoogpt/passo`	`estudante`	Move a sessão para outro passo da missão.
`POST /unoogpt/pergunta`	`estudante`	O turno do Cientista — a rota que fala com o modelo. Rate limit apertado (**12/min**) porque cada chamada custa tokens de verdade: o teto por sessão é o controle de custo, e este é o freio contra o laço. **Restrições:** • Falha do PROVEDOR **não** chega ao estudante como 500: vira `cientista-indisponivel` (**503**), que a tela traduz em "tente falar de novo". E **não debita** — o débito acontece só depois de resposta válida. • O log fica com o motivo técnico, **sem o texto do estudante**: é caixa livre escrita por menor de idade. • O número que um botão de opção mostra sai do MOTOR, nunca da LLM.

Rota	Autorização	Semântica
`GET /unoogpt/professor/estudantes/{estudanteId}`	`professor`, `gestor_contrato`, `admin`	A linha de UMA criança: leitura em graus e a Transcrição. **Professor only.** O relatório abre pelo que a missão MEDIU, em graus. **Restrições:** • Número só com `grau: "scored"`. E `zero` (“teve a chance e não foi”) nunca se confunde com `none` (“a oportunidade não chegou”) — é a distinção mais importante do modelo, e ela está no TIPO. • `resumo` nunca afirma o que o estudante É, e partida curta some com o resto: dois minutos de jogo não sustentam frase.
`GET /unoogpt/professor/fontes`	`professor`, `gestor_contrat` `admin`	As 8 fontes do cenário com o viés de cada uma, para o guia da aula. **Professor only.** Alimenta o guia da aula em `/professor/entre-redes/2-serie/unidade-1/guia`. O hub NÃO guarda cópia disto: a missão não revela o viés por software, e uma cópia desatualizada faria o professor contar à turma um viés que o cenário já não produz. **Restrições:** • Nada aqui é escopado por escola: é o catálogo do cenário, igual para todo contrato.
`GET /unoogpt/professor/turmas`	`professor`, `gestor_contrato` `admin`	Turmas para o seletor de resultados do unooGPT. Filtradas por MATRÍCULA para o professor. **Gestor não entra aqui** — ele tem `/unoogpt/agregado/escola`.
`GET /unoogpt/professor/turmas/{turmaId}`	`professor`, `gestor_contrat` `admin`	A folha da turma: relógio, retrato de conduta e uma linha por estudante. O retrato fala de **conduta, não de nota** — conta “decidiram no escuro” e “justificaram por intuição” em vez de somar acertos, porque a Missão não tem resposta certa.
`DELETE /unoogpt/professor/turmas/{turmaId}`	`professor`, `gestor_contrato` `admin`	Zera a turma: apaga as partidas E o relógio. É o caminho de exclusão que substitui o `expires_at` do app de origem — prazo real, apertado por quem tem a posse pedagógica, em vez de coluna que nenhuma rotina cumpre. O relógio vai junto porque rodar de novo com a aula “já começada” faria a segunda rodada nascer no meio da tempestade.

Rota	Autorização	Semântica
`POST /unoogpt/professor/turmas/{turmaId}/relogio`	`professor`, `gestor_contrat` `admin`	Dá a largada, soma tempo, encerra, troca o modo ou redefine. É o único lugar onde alguém que não é o estudante mexe no que ele vê — e mesmo aqui **não toca o Passo dele**: o que o professor controla é QUANDO dá para falar com o Cientista, não o que a criança decide.
`POST /unoogpt/reiniciar`	`estudante`	Recomeça — abre uma tentativa NOVA. **Não apaga a conversa anterior.** Cada tentativa fica no resultado com o desfecho e a faixa DELA: é essa comparação que ensina que a mesma cidade dá números diferentes. O clique fica gravado na tentativa que fecha (marco `recomeco`), com o relógio da turma naquele instante. `origem` diz qual botão — é a única parte que só o cliente sabe.
`GET /unoogpt/relogio`	`estudante`	O relógio da turma — polling de ~5s. Leve de propósito: sem IA e sem mapa. `agora` é o instante do SERVIDOR, senão o relógio da máquina da escola (que atrasa) decide quanto tempo a turma tem. **Restrições:** • 240 chamadas por minuto por estudante.
`GET /unoogpt/resultado`	`estudante`	O resultado da tentativa, com a faixa e o que ele nunca consultou. A **faixa é a manchete**: corridas da MESMA decisão, para "não repete igual" sair do texto e virar gráfico. Uma rodada é anedota — o estudante lê 1.345 e conclui que tirou uma nota. **Restrições:** • `sobrou` é um FATO com nome de bairro e número de pessoas, **não um veredito**: quem transforma fato em lição é o professor, no debate. • `naoConsultados` traz só `comoFalar/rotuloCurto/serve` — o nome técnico, o viés e a cobertura são o instrumento, e ficam com o professor.

Rota	Autorização	Semântica
`GET /unoogpt/sessao`	`estudante`	O bootstrap da Missão Tempestade. Cria a sessão na primeira visita. Repare no que NÃO existe nestas rotas: **id de sessão**. No app de origem toda rota era `/api/missao/:id/*` com uma guarda `donoDaSessao` para provar a posse; aqui a identidade é `(escola, estudante)` vinda do TOKEN, e não há como pedir a partida de outra pessoa. A guarda desapareceu junto com a classe de bug que ela protegia. **Restrições:** • A `projecao` é SERVIDA, nunca calculada no navegador: um bundle capaz de computá-la deixaria o estudante prever o desfecho antes de enviar. • O mapa começa vazio: só o que uma Pergunta revelou chega ao navegador — `estimativa: null` é o cartão em branco, e **branco é informação**. • A `ficha` sai por allowlist, sem os números por bairro: eles são a verdade do mundo, e mandá-los daria de graça a resposta que a missão existe para o estudante não ter.
`POST /unoogpt/simular`	`estudante`	Enfrenta a tempestade e fecha a tentativa. Exige justificativa de **cada equipe com destino** — sem isso, `falta-justificar`. Devolve o mesmo que `GET /unoogpt/resultado`.

C.38 usuarios

Rota	Autorização	Semântica
`GET /usuarios`	`gestor_escola`, `gestor_regional`, `gestor_contrato`, `admin`	As pessoas da escola, paginadas. A tela `/admin/usuarios`. Busca por nome, login, matrícula **e CPF — com ou sem pontuação**. **Restrições:** • `situacao: sem-primeiro-acesso` lista quem ainda não trocou a senha da importação. • A consulta roda dentro de `comTenancy` porque inclui MATRÍCULAS, que têm RLS: fora do contexto a lista voltaria com todo mundo “sem turma”, em silêncio.

Rota	Autorização	Semântica
`POST /usuarios`	`gestor_escola`, `gestor_regiona` `gestor_contrat` `admin`	Cadastra UMA pessoa da escola (estudante, professor ou gestor). O formulário de `/admin/usuarios`. Monta a mesma linha da planilha e delega ao importador **ADR-59** — por isso as regras de CPF, série, turno e login são idênticas às da importação. **Restrições:** • **Criar é criar**: CPF, matrícula ou login já existentes devolvem 409 e nada é escrito — o caminho de quem já existe é `PATCH /usuarios/:id`. • CPF achado em OUTRA escola da rede devolve 409 sem id, sem nome e sem dizer qual escola: transferir é pela importação, que move a matrícula junto. • A senha sai UMA vez no corpo da resposta e é sempre provisória. Em branco, é sorteada. • `gestor_contrato` NÃO entra aqui: ele não tem escola nem lotação, e tem rota própria. • Limite de 30 cadastros por minuto por usuário — é balcão, não carga.
`PATCH /usuarios/{id}`	`gestor_escola`, `gestor_regional` `gestor_contrato` `admin`	Edita nome, CPF, e-mail, papel ou situação da pessoa. Caminho de balcão: a planilha veio com o dígito trocado e a família traz o documento. **Restrições:** • `cpf: ""` **APAGA** o campo — é o que se faz quando o número era de outra pessoa. • Sair da REDE é `ativo: false`, não DELETE: há dado de estudante pendurado em oito tabelas.
`POST /usuarios/{id}/ reset-senha`	`gestor_escola`, `gestor_regiona` `gestor_contrat` `admin`	Gera uma senha temporária e a devolve UMA vez. Marca a conta como provisória: quem entrar com ela cai em `/trocar-senha` sem saída — trava tripla, no middleware, no BFF e no `/api/exp-token`. **Restrições:** • A senha **não fica em coluna nem em log**: se a tela perder, gera-se outra.
`POST /usuarios/ importar-csv`	`gestor_escola`, `gestor_regional` `gestor_contrato` `admin`	Compatibilidade: delega para `POST /rostering/ importar`. Continua respondendo porque scripts e documentação a citam. **Delega** — dois importadores lado a lado divergiriam na primeira regra nova.

Rota	Autorização	Semântica
`POST /usuarios/sem-escola`	`gestor_contrat` `admin`	Cadastra um papel SEM escola (`gestor_contrato` ou `gestor_regional`). Os papéis que não nascem por importação: não pertencem a escola nenhuma, e por isso não cabem numa linha de planilha da escola. Quem pode pedir qual sai da **cadeia de atribuição ADR-80**: `admin` atribui `gestor_contrato`; `gestor_contrato` atribui `gestor_regional`. Pedir acima do próprio alcance é 403 — e a checagem é `podeCriarPapel`, não o `@Roles`, porque o guard protege a ROTA e não o campo `papel` do corpo. **Restrições:** • Nasce com `escola_id` nulo e nenhuma lotação — é o mesmo formato que o seed cria. • `gestor_regional` EXIGE `regionalId`, e `gestor_contrato` o recusa: é o escopo de um e não do outro, com CHECK bicondicional no banco. • O login é único na REDE inteira, não na escola: o índice `(escola_id, username)` não cobre linha com `escola_id` nulo, e a autenticação procura o username na rede. • A senha sai UMA vez e é provisória: o primeiro acesso cai em `/trocar-senha`. • A rota se chamava `/usuarios/admin-contrato` e servia um papel só. O caminho mudou com o vocabulário **ADR-80**.

C.39 vila

Rota	Autorização	Semântica
`GET /vila/agregado/turmas`	`professor`, `gestor_escola`, `gestor_regional`	Turmas para o seletor da Vila. Sem filtro por série — a Vila é transversal —, mas **com** filtro por matrícula: o professor vê as turmas dele, o gestor vê a escola.
`GET /vila/agregado/turmas/{turmaId}`	`professor`, `gestor_escola`, `gestor_regiona`	O agregado da turma — contagens, nunca por criança. Abre para GESTOR também, e nenhum campo aqui identifica uma criança: porta a regra do app de origem (“pesquisador nunca vê a linha de uma criança; só agregados”). A distribuição de bolhas é o número que o professor precisa **antes** do debate: sem as três bolhas na sala, o momento em que a turma descobre que recebeu mensagens diferentes não acontece.

Rota	Autorização	Semântica
`GET /vila/professor/cartoes`	`professor`	TODOS os cartões, das três bolhas, COM o gabarito. É a revelação final da missão: o professor projeta as três versões lado a lado, marcando quais eram verdadeiras, e é a primeira vez que a turma vê as outras. Vem da api, e não de uma cópia no hub, porque **os textos SÃO o instrumento**: uma vírgula divergente entre o que o jogo serviu e o que o professor projeta põe a sala discutindo uma mensagem que ninguém recebeu. **Restrições:** • **COM 'turmaId', devolve só as bolhas que AQUELA sala recebeu.** Sem isso, o guia de uma turma de 3º ano projetava o golpe da bolha B — que a api nunca serviu ali — e mandava o professor esperar a reação de um grupo que não existe na sala. • `bolhasDaTurma: null` = nenhuma turma escolhida; a tela mostra as três e avisa.
`GET /vila/professor/estudantes/{estudanteId}`	`professor`	O relatório da última partida concluída de um estudante. **Professor only** — o gestor tem o agregado, nunca a linha de uma criança. São controllers separados de propósito: um `@Roles` misto com exceção por método é como uma rota individual acaba aberta para gestor sem ninguém notar. A seção das mensagens abre pelo **NÍVEL** que ela recebeu, antes do que ela fez: "caiu em 2 de 3" sem dizer que era o nível C faz o adulto concluir que a criança é ingênua — o rótulo que este projeto existe para não produzir. **Restrições:** • `relatorio: null` é resposta legítima: o estudante existe e não jogou. Um 404 faria a tela dizer "não encontrado" para uma criança da turma dele. • `grau` é invariante de API: só a variante `scored` tem número; as outras têm `null` POR TIPO, então o hub não consegue imprimir "Poder 100" a partir de um sinal só. • `mensagens: null` = nenhum cartão chegou a aparecer — **não** é "ela ignorou tudo".
`GET /vila/professor/turmas/{turmaId}`	`professor`	Uma linha por estudante da turma. Quantas partidas, a bolha, e se a última sustenta leitura. **Sem valor de Schwartz nenhum**: listagem é ordenável, e uma coluna "Poder" viraria ranking de crianças. Os valores vivem na tela individual, uma criança por vez, com o grau ao lado.

Rota	Autorização	Semântica
`DELETE /vila/professor/turmas/{turmaId}/dados`	`professor`	Zera os dados da Vila desta turma. `DELETE`, e não um `POST /limpar`: é remoção, e o verbo diz isso a quem lê o log. Devolve as contagens do que sumiu — "pronto" sem número deixa o professor sem saber se apagou a turma certa. **Restrições:** • As **bolhas** somem junto: rodar de novo com a mesma bolha serviria à criança o mesmo golpe — ela reconhece, e a segunda rodada não mede nada. • `perfis_importados` **não** é tocado: aquilo pertence à atividade do Instanooria, e apagar de lá trocaria os lados de uma aula em andamento.
`POST /vila/sessoes`	`estudante`	Abre a partida e entrega os cartões DESTA criança. **A bolha é do SERVIDOR.** Atribuída A/B/C de forma balanceada dentro da TURMA e persistida em `vila_bolhas` (estável entre partidas: a folha do professor diz "esse estava na C"). Era `estudanteSlot % 3` no navegador, o que punha no bundle a condição do próprio instrumento e deixava as três versões legíveis antes da aula. **Restrições:** • Os cartões vêm **sem gabarito** e só os desta bolha: a explicação final da missão depende de ser a primeira vez que a turma vê as outras versões. • Nos anos iniciais (1EF–5EF) a bolha B **não é servida** — "suporte" exige entender o que é suporte, e só a api sabe a série. • Há lock consultivo por turma: a sala clica junto, e sem ele os 25 caíam todos na bolha A. • Identidade sempre do token: o payload não traz estudante nem turma.

Rota	Autorização	Semântica
`POST /vila/sessoes/{id}/concluir`	`estudante`	Fecha a partida e grava o payload bruto. **Idempotente**: a criança pode recarregar, e o `pagehide` pode disparar duas vezes — daí `status: "jaEstava"`. O servidor RECALCULA o retrato; nunca aceita o perfil do cliente. É isso que permite reprocessar o histórico quando `VERSAO_MAPEAMENTO` subir. **Restrições:** • `CHAVES_DE_PERFIL` recusa payload que traga valores/graus/perfil calculados: o jogo sabe computar o retrato e deliberadamente não o envia. • `eventos[].dados` só aceita número, booleano ou string de um **vocabulário fechado** — o `texto` livre era o caminho por onde o nome de uma criança chegaria a um JSONB. • O gabarito da missão fica no servidor: o cliente manda qual opção escolheu, a api corrige (regra 3).

Apêndice D

Payloads e esquemas de validação

Exemplos reais dos *payloads* que atravessam as fronteiras da plataforma, com os esquemas Zod que os validam. Todo *input* externo é validado; nenhum esquema aqui é decorativo.

D.1 Protocolo `postMessage` (v1)

Listing D.1 – `packages/sdk/src/types.ts` — os esquemas do protocolo

```
export const PROTOCOL_VERSION = 2;

export const EstudanteSchema = z.object({
  id: z.string(), nome: z.string(),
  serie: z.string(),          // "6EF", "2EM"
  turmaId: z.string(), escolaId: z.string(),
});

export const HubInitSchema = z.object({
  type: z.literal("hub:init"),
  protocolVersion: z.number(),
  token: z.string(),
  estudante: EstudanteSchema,
  apiUrl: z.string().url(),
  config: z.record(z.unknown()).default({}),
});

/** Mensagens POS-init, validadas com Zod nos DOIS lados. Unioes
 *  discriminadas — mensagem que nao casa e DESCARTADA com aviso, nunca
 *  despachada crua. Tudo aqui e ADITIVO ao v1: hub antigo ignora mensagem
 *  que nao conhece; experiencia antiga nunca recebe as novas sem pedir. */
export const HubParaExpSchema = z.discriminatedUnion("type", [
  z.object({ type: z.literal("hub:token"), token: z.string() }),
  z.object({ type: z.literal("hub:pause") }),
  z.object({ type: z.literal("hub:resume") }),
]);

export const ExpParaHubSchema = z.discriminatedUnion("type", [
  z.object({ type: z.literal("exp:ready"), protocolVersion: z.number() }),
  z.object({ type: z.literal("exp:concluir"),
             payload: ExpConcluirPayloadSchema.default({}) }),
  z.object({ type: z.literal("exp:token-expirando") }),
  z.object({ type: z.literal("exp:estado"), estado: z.record(z.unknown()) }),
  // a exp quebrou; o hub troca a tela. O registro vai SDK -> api (ADR-58)
```

```
  z.object({ type: z.literal("exp:erro"),
             mensagem: z.string().max(256), hash: z.string().max(16) }),
]);
```

Listing D.2 – `hub:init` — o que a experiência recebe

```
{
  "type": "hub:init",
  "protocolVersion": 2,
  "token": "eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9...",
  "estudante": {
    "id": "8f2c1a44-...", "nome": "Ana",
    "serie": "8EF",
    "turmaId": "b17e...", "escolaId": "sp-001"
  },
  "apiUrl": "https://api-entreredes.unooria.com",
  "config": {
    "clarityProjectId": "ybo0odt6cn",
    "expId": "exp08ef1b",
    "obs": { "traceRatio": 0.02 },
    "fuso": "America/Sao_Paulo"
  }
}
```

`config.obs.traceRatio` (2026-09-02) é a fração de sessões desta experiência com OpenTelemetry Web ligado — vem de `OBS_TRACE_RATIO_CLIENTE` do hub, env de *runtime*, então mudar a amostragem não republica bundle nenhum; `0` faz a SDK nem baixar os pacotes.

`config.fuso` (2026-09-11, **ADR-99**) é o fuso da plataforma, vindo de `FUSO_DA_PLATAFORMA` do hub. `createExperiencia` o aplica **antes da primeira tela**, e a partir daí `data(iso)` e as outras cinco funções de formatação do `@edu/sdk` respondem nele — sem mudar de assinatura. Ausente (hub anterior à ADR-99) vale `America/Sao_Paulo`. Ele vai em `config`, e não como campo de topo, pela regra da linha acima: o *bundle* já publicado ignora o que não conhece, enquanto mudar a FORMA do `hub:init` obrigaria a subir `PROTOCOL_VERSION` — e aí o *handshake* seria recusado, com tela de erro para a criança.

D.2 *Token* de experiência

Listing D.3 – *Claims* do JWT — contrato congelado

```
{
  "sub":       "8f2c1a44-...",       // usuarioId (nao "estudanteId": professor
                                     // passa pelo MESMO guard)
  "escola_id": "sp-001",             // vazio APENAS para admin-contrato
  "turma_id":  "b17e...",            // opcional
  "perfil":    "estudante",              // estudante | professor | gestor | admin-contrato
  "aud":       "plataforma-api",
  "iss":       "https://entreredes.unooria.com",
  "exp":       1756290000            // TTL de 15 minutos
}
```

> **NOTA**
>
> `sub` vira `usuarioId` nas *claims* tipadas da API (`src/auth/claims.ts`) — e não `estudanteId`, porque professor, gestor e admin passam pelo mesmo *guard*. O nome do campo é documentação: um `estudanteId` que às vezes contém um professor é uma armadilha para quem lê a *query* seis meses depois.

D.3 Telemetria — canal analítico

Listing D.4 – `src/telemetry/telemetry.controller.ts` — os esquemas

```ts
const EventoSchema = z.object({
  eventId:    z.string().uuid(),
  ts:         z.string(),
  expId:      z.string(),
  expVersion: z.string(),
  sessaoId:   z.string(),
  verbo:      z.string(),
  objeto:     z.string(),
  resultado:  z.record(z.unknown()).optional(),
  contexto:   z.record(z.unknown()).optional(),
});
// estudanteId e escolaId NAO estao no esquema de entrada: sao SOBRESCRITOS a
// partir do token. O cliente nao PODE colocar PII no lugar do id.

const BatchSchema  = z.object({ eventos: z.array(EventoSchema).max(50) });

/** O beacon do pagehide NAO envia headers, entao o token vem no CORPO. */
const BeaconSchema = BatchSchema.extend({
  beacon: z.literal(true).optional(),
  token:  z.string().min(1),
});

/** Envelope de push subscription do Pub/Sub. */
const PushSchema = z.object({
  message: z.object({ data: z.string().optional(), messageId: z.string().optional() }),
  subscription: z.string().optional(),
});
```

Listing D.5 – `POST /telemetry/batch` — corpo

```json
{
  "eventos": [
    {
      "eventId":    "3f9a2c10-8b41-4d2e-9c77-1a5e6b0d4f22",
      "ts":         "2026-08-27T11:14:03.512Z",
      "expId":      "exp04ef1b",
      "expVersion": "1.1.0",
      "sessaoId":   "c81b5e77-2d94-4a10-b3f6-77c2e9a4d001",
      "verbo":      "interacted",
      "objeto":     "video/v07",
      "resultado":  { "acao": "salvou", "salvos": 2 },
      "contexto":   { "tela": "feed", "ordenacao": "relevancia" }
    }
```

```
  ]
}
```

D.4 Sinais de observabilidade do navegador

Os três corpos que a SDK (ou o hub) manda para `/observabilidade/*` (Capítulo 17). Todos `.strict()`: campo a mais é recusado, não ignorado — é a fronteira entre um *bundle* público e um projeto do Google.

Listing D.6 – `POST /observabilidade/erros-cliente`

```
{
  "expId": "exp04ef1b",
  "versao": "0.3.1",
  "mensagem": "Cannot read properties of undefined (reading 'video')",
  "stack": "TypeError: ...\n    at feed.tsx:42:11",
  "url": "https://exp-entreredes.unooria.com/exp04ef1b/0.3.1/index.html",
  "userAgent": "Mozilla/5.0 ...",
  "sessaoId": "6f1a1d1e-9c2b-4a4e-8f1e-1234567890ab",
  "traceId": "4bf92f3577b34da6a3ce929d0e0e4736",
  "spanId": "00f067aa0ba902b7"
}
```

Tetos: `mensagem` 2 KB, `stack` 8 KB, `url` 512 sem *query* nem fragmento; `traceId`/`spanId` opcionais (do *span* ativo no navegador). O que a API emite é uma linha de log com `serviceContext.service = expId`, `version = versao`, `stack_trace`, `estudanteId` (opaco, do *token*) e `escolaId` (do contexto) — nunca o *username*.

Listing D.7 – `POST /observabilidade/vitals`

```
{ "nome": "LCP", "valor": 1832, "rota": "/professor/turmas/9f3a", "superficie": "hub" }
```

`nome` ∈ {LCP, INP, CLS, TTFB}; `valor` em ms (CLS em unidade), teto de 600 000; `rota` é o `pathname` (hub) ou o `expId` (experiência), normalizados pela API para um conjunto fechado. O `perfil` não vem no corpo: vem do *token*.

Listing D.8 – `POST /observabilidade/v1/traces` — subconjunto OTLP/JSON

```
{
  "resourceSpans": [{
    "resource": { "attributes": [
      { "key": "service.name",    "value": { "stringValue": "exp04ef1b" } },
      { "key": "service.version", "value": { "stringValue": "0.3.1" } }
    ]},
    "scopeSpans": [{
      "scope": { "name": "@opentelemetry/instrumentation-fetch", "version": "0.222.0" },
      "spans": [{
        "traceId": "4bf92f3577b34da6a3ce929d0e0e4736",
        "spanId": "00f067aa0ba902b7",
        "parentSpanId": "0000000000000001",
```

```
        "name": "HTTP GET",
        "kind": 3,
        "startTimeUnixNano": "1756771200000000000",
        "endTimeUnixNano":   "1756771200500000000",
        "attributes": [
          { "key": "http.method",      "value": { "stringValue": "GET" } },
          { "key": "http.url",         "value": { "stringValue": "https://api/unootube/sessao" } },
          { "key": "http.status_code", "value": { "intValue": 200 } }
        ],
        "status": { "code": 1 }
      }]
    }]
  }]
}
```

Um `resourceSpans`, até 4 `scopeSpans`, até 64 *spans*, até 24 atributos por *span* (valores de até 256 caracteres); `links`, `traceState` e chaves desconhecidas são removidos. O `resource` do cliente é lido só para `service.name` (o código da experiência, **ADR-110**, ou `hub`) e `service.version`, e SUBSTITUÍDO; atributos passam por *allowlist* e `http.url`/`url.full` perdem *query* e fragmento antes de sair.

D.5 app-70 — o *payload* de uma partida

Este é o *payload* mais elaborado da plataforma, e o que carrega mais decisões de projeto por campo.

Listing D.9 – `POST /vila/sessoes/:id/concluir` — corpo (abreviado)

```
{
  "v": 1,
  "bundleVersion": "1.0.0",
  "duracaoSeg": 1042,          // do "ENTRAR NA VILA" ao fim: tempo de JOGO
  "duracaoTotalSeg": 1387,     // desde a pagina abrir, incluindo a intro
  "status": "completa",

  "resumo": {
    "fogueirasAcesas": 8, "pedrasQuebradas": 3, "aldeoesAjudados": 2,
    "lampioesAcesos": 4, "tochasAcesas": 6,
    "dilemasVistos": 3, "dilemasResolvidos": 2, "distanciaM": 1840
  },

  "medidas": {
    "cobertura": { "celulas": 62, "total": 144, "pisoObrigatorio": 31 },
    "movimento": { "movendoSeg": 712, "escuroSeg": 128, "luzSeg": 584,
                   "atalhos": 2, "desvioM": 340, "raioMax": 58 },
    "lume":      { "conversas": 4, "perguntasCaminho": 1, "lore": 2,
                   "ajuda": 1, "sugestaoSeguida": null },
    "tocha":     { "aldeao": 2, "lampiao": 3, "pedra": 1,
                   "passarela": 0, "retomadas": 1 },
    "pedra":     { "borda": true, "destaque": false,
                   "nomesApagados": 0, "nomesLidos": 5 },
    "fumaca":    { "avistadas": 3, "seguidas": 2 },
    "brinquedos": 4,
    "resgates":  [ { "at": 512, "firesLit": 5 } ]
```

```
  },

  "oportunidades": { "lampiao": 6, "aldeao": 4, "pedra": 1 },

  "dilemas": [ { "pairId": 0, "escolha": "aldeao", "emMs": 214000 } ],

  "eventos": [
    { "tipo": "chama_acesa",  "emMs": 61000,  "dados": { "zona": "praca" } },
    { "tipo": "tocha_gasta",  "emMs": 188000, "dados": { "destino": "aldeao" } },
    { "tipo": "pedra_gravou", "emMs": 402000, "dados": { "acao": "borda" } }
  ],

  "seguranca": {
    "apareceram": 4, "respondidas": 3, "ignoradas": 1,
    "cartoes": [
      { "id": "c1", "emMs": 240000, "viuOpcoes": true,
        "escolha": 2, "decisaoMs": 9400 },
      { "id": "c2", "emMs": 431000, "viuOpcoes": true,
        "escolha": 1, "decisaoMs": 1800 },
      { "id": "c3", "emMs": 655000, "viuOpcoes": false,
        "escolha": null, "decisaoMs": null }
    ]
  }
}
```

D.5.1 As decisões que cada campo carrega

Campo	Decisão registrada
Ausência de `sessaoLocalId`, `destino`, `iniciadaEm`	Identidade e turma vêm da *sessão autenticada*; repeti-las no *payload* criaria uma segunda fonte de verdade para o servidor conferir contra a primeira. O *id* da partida é emitido pela API (é a chave de idempotência) e o instante de início é cravado por ela.
`seguranca.cartoes[]` sem classificação	**Regra 3.** O aplicativo de origem recebia `seguras`, `risco`, `golpeAceito`, `legitimaRecusada` e `bolha` por cartão — o *cliente* classificando as próprias respostas. Agora ele manda o que *fez*, e a API corrige contra `catalogo.ts`. Um *bundle* adulterado já não pode se reportar como tendo recusado tudo.
`viuOpcoes`	Separa “deixei passar” de “nem vi”. As duas caíam em `ignoradas`, e são crianças diferentes.
`decisaoMs`	O campo mais informativo do conjunto: cair em dois segundos e cair depois de dez são duas crianças diferentes — a primeira não avaliou, a segunda avaliou e foi convencida.
`oportunidades` — **a ausência de uma chave**	Significa “não observado”, que é *diferente* de zero. Zero é “teve a chance e não foi” e entra na média do polo; não observado **sai da conta**.
`pedra.borda` *versus* `pedra.destaque`	Gravou onde ainda cabia, sem apagar ninguém (Realização) *versus* apagou um nome tendo a borda livre (Poder). **A distinção é o ponto.**

`lume.sugestaoSeguida: null`	= a sugestão não chegou a ser testada. **Não** é "não seguiu".
`cobertura.pisoObriga`	Quantas células a rota obrigatória já toca. Sem isso, cobertura mediria *cumprimento do objetivo* em vez de iniciativa.
`eventos[].emMs`	Tempo **relativo** ao início. Nunca relógio de parede.
`eventos[].dados`	Número, booleano ou *string* de **vocabulário fechado** (ids das 8 zonas + Fogo Sagrado, destinos da Tocha, as duas ações da Pedra, ids dos cartões). Nome de criança não cabe num vocabulário fechado.
`bundleVersion`	Fica na linha para o *skew* de *publish* ser **visível**: quem estava com a aba aberta durante uma publicação termina a partida com o *bundle* anterior, e uma partida dura 15–25 minutos.

D.5.2 As duas travas estruturais

Listing D.10 – Por que `.strict()` e por que a inspeção acontece no BRUTO

```
/**
 * ATENCAO AO `.strict()` E A ORDEM DE `validarPayload`.
 *
 * A primeira versao disto no app de origem tinha as recusas mais importantes
 * escritas como `.refine()`, e elas NAO FUNCIONAVAM: o zod remove chaves
 * desconhecidas ANTES de rodar refine, entao `{ nome: "Lara" }` era descartado
 * em silencio e o refine recebia um objeto ja limpo. O payload passava e o nome
 * simplesmente sumia sem ninguem saber que tinha chegado. Por isso:
 *
 *   1. `.strict()` – chave desconhecida e ERRO, nunca descarte silencioso;
 *   2. `validarPayload()` inspeciona o dado BRUTO antes do parse, porque so o
 *      bruto ainda contem o que nao deveria ter vindo.
 */
```

Trava	Como funciona
`CHAVES_PROIBIDAS` (por *substring*)	`nome`, `name`, `apelido`, `nick`, `estudante`, `crianca`, `jogador`, `player`, `cpf`, `matricula`, `email`, `telefone`, `responsavel`, `professor`, `endereco`, `nascimento`. Por *substring*, porque `nomeCompleto`, `nome_do_estudante` e `playerName` são a mesma coisa para quem for reidentificar a criança.
`CHAVES_PROIBIDAS_EXATA`	`rg`, `ra`, `id_estudante`, `doc` — **só por igualdade**. `rg` casaria dentro de `perguntasCaminho`, e um falso positivo recusa a partida inteira de uma criança.
`CHAVES_DO_CONTRATO`	`nomesApagados` e `nomesLidos` são chaves legítimas que contêm um termo proibido como *substring* (contam nomes na Pedra — números, nunca texto). A lista é **fechada**: campo novo que colida tem de ser adicionado aqui *de forma consciente*.
`CHAVES_DE_PERFIL`	Recusa *payload* que traga o perfil calculado. O jogo sabe computá-lo e deliberadamente não o envia — é isso que permite reprocessar o histórico quando `VERSAO_MAPEAMENTO` subir.

> **NOTA**
>
> A varredura por nome de criança importa *mais* aqui do que no aplicativo de origem: lá o *bundle* não tinha nome nenhum em memória; aqui o *shim* entrega o primeiro nome da criança ao jogo, para o Lume chamá-la e para a Pedra dos Nomes. **Existe um nome a vazar agora** — e rejeitar é melhor que ignorar em silêncio, porque um campo de nome aparecendo é *bug do jogo*.

D.5.3 O cap de eventos, e por que ele caiu de 2000 para 200

```
.max(200)
/**
 * 200, e o teto REAL do jogo e ~22.
 *
 * MEDIDO no bundle: o jogo tem 7 pontos que gravam evento, e todos sao atos
 * discretos com guarda propria — 8 Chamas Sagradas, 4 aldeoes, 1 Borda + 3
 * Destaques, 4 cartoes da missao, 1 primeira conversa com o Lume. Nada por
 * quadro, nada por metro andado. `restart()` zera o log, entao replay nao
 * acumula.
 *
 * Herdamos 2000 do app de origem. Cabe baixar: o cap e a UNICA coisa que
 * limita o tamanho da linha se um cliente adulterado mandar lixo, e
 * 2000 x ~80 bytes = 160KB por partida (1,6GB num contrato de 10 mil estudantes)
 * contra ~2KB reais. Com 200 ha 9x de folga para eventos novos.
 */
```

D.6 Agente de IA do professor — o *stream* e o engine

O contrato das quatro pontas está na Seção 9.5 e em `docs/regras/agente-professor.md`; os *schemas* das respostas JSON e de cada `data:`, no `contrato-de-rotas.ts` do módulo `src/agente-professor` da api.

Listing D.11 – `POST /agente-professor/conversas/:id/mensagens` — corpo

```
{ "texto": "Quais habilidades de Matematica do 6o ano tratam de fracoes?" }
// 1 a 4000 caracteres depois do trim; .strict(): campo a mais e 400
```

Listing D.12 – A resposta: `text/event-stream` (um turno com fonte, título e aviso)

```
event: delta
data: {"texto":"No 6o ano, as habilidades de fracoes sao "}

: ping

event: delta
data: {"texto":"EF06MA07, EF06MA08 e EF06MA09."}

event: fontes
data: {"fontes":[{"titulo":"BNCC - Ensino Fundamental","paginaInicio":300,"paginaFim":301}]}

```

```
event: titulo
data: {"titulo":"Fracoes no 6o ano"}

event: aviso
data: {"codigo":"habilidade-inexistente","mensagem":"O codigo EF99ZZ99 citado na resposta nao existe na BNCC. Confira antes de usa-lo."}

event: fim
data: {}
```

Listing D.13 – Recusa ANTES do *stream* — 429 em JSON

```
{ "codigo": "RATE_LIMIT", "mensagem": "Muitas mensagens seguidas. ...", "retryAfterSeconds": 42 }
{ "codigo": "QUOTA",      "mensagem": "Voce usou toda a cota de IA de hoje. ..." }
// 404 { "codigo": "conversa-inexistente" } para conversa de outro dono
// 403 { "codigo": "recurso-indisponivel" } com o produto desligado na escola
```

Listing D.14 – A api → o Agent Engine: `:query` e `:streamQuery?alt=sse`

```
// POST https://us-east1-aiplatform.googleapis.com/v1/{engine}:query
{ "class_method": "async_create_session",
  "input": { "user_id": "8f2c1a44-...",              // users.id, do token
             "state": { "nome_professor": "Ana" },
             "expire_time": "2036-09-30T12:00:00.000Z" } }

// POST .../v1/{engine}:streamQuery?alt=sse
{ "class_method": "async_stream_query",
  "input": { "user_id": "8f2c1a44-...", "session_id": "<sessao_id>",
             "message": "<texto do professor>",
             "run_config": { "streaming_mode": "sse" } } }

// PATCH .../v1beta1/{engine}/sessions/{sessao_id}?updateMask=expireTime
{ "expireTime": "2036-09-30T12:05:00.000Z" }
```

Listing D.15 – Um evento do ADK no *stream* (abreviado), e o que a api lê dele

```
{ "author": "agente_professor",
  "partial": false,                                 // so os NAO parciais contam tokens
  "content": { "parts": [ { "text": "..." } ] },    // texto final agregado: REPETE os parciais
  "usage_metadata": { "total_token_count": 1834 },
  "actions": { "state_delta": { "titulo_conversa": "Fracoes no 6o ano" } },
  "custom_metadata": { "fora_do_escopo": true,     // so na recusa de escopo
                       "tokens_auxiliares": 41 } } // guarda + titulo, ja somados
// retorno de buscar_acervo, de onde sai o evento "fontes":
{ "trechos": [ { "titulo": "...", "pagina_inicio": 300, "pagina_fim": 301,
                 "trecho": "...", "fonte_id": "..." } ] }
```

NOTA

O corpo do `:streamQuery` chega em linhas `data: {...}` **ou** em NDJSON puro, no mesmo serviço; o cliente decide por linha, não pelo `content-type`. O ADK aceita *snake_case* e o REST das sessões devolve *camelCase*: o cliente normaliza os dois em `EventoAgente`.

D.7 Manifesto de experiência

Listing D.16 – `public/experiencia.json` — exp08ef1b, com `viewport`

```
{
  "id": "exp08ef1b",
  "nome": "Instanooria – O Conteúdo que não chegou",
  "versao": "1.1.4",
  "produto": "entre-redes",
  "series": ["8EF"],
  "capabilities": ["telemetria", "progresso", "ia"],
  "viewport": { "largura": 375, "altura": 812 },
  "verbos": {
    "attempted":  "entrou no aparelho",
    "completed":  "concluiu a missão (dossiê visto)",
    "progressed": "mudou de fase"
  }
}
```

Listing D.17 – `public/experiencia.json` — app-90, ferramenta de professor

```
{
  "id": "app-90-plano-de-aula",
  "nome": "Gerador de Plano de Aula",
  "versao": "1.2.1",
  "produto": "plano-de-aula",
  "series": [],
  "capabilities": ["telemetria", "ia"],
  "publico": "professor",
  "verbos": {
    "attempted":  "pediu a geração de um plano anual (disciplina + série)",
    "completed":  "salvou o plano pela primeira vez",
    "interacted": "exportou .doc, restaurou o original ou excluiu um plano"
  }
}
```

Listing D.18 – `public/experiencia.json` — exp05ef1b, com `permissoes`

```
{
  "id": "exp05ef1b",
  "nome": "Quem mandou isso?",
  "versao": "0.1.1",
  "produto": "entre-redes",
  "series": ["5EF"],
  "unidade": 1,
  "capabilities": ["telemetria", "progresso", "avaliacao"],
  "viewport": { "largura": 375, "altura": 812 },
  "permissoes": ["forms"],
  "publico": "estudante",
  "verbos": {
    "attempted":  "entrou no grupo da turma e confirmou o envio para o Mural",
    "interacted": "encaminhou mensagens, abriu a transcrição do áudio e o card do link",
    "progressed": "leu o grupo até o fim e esperou a revelação coletiva da turma",
    "completed":  "respondeu ao dilema do áudio do tio"
  }
}
```

NOTA

`series: []` na app-90 significa "não se aplica" porque `publico: "professor"`; na app-70 significa **todas as séries**, porque é experiência de estudante *transversal*. O mesmo valor, duas leituras — e o que as distingue é `publico`. Quando o filtro por série voltar a valer, vazio tem de continuar querendo dizer "todas" para o público `estudante`.

D.8 Validação de variáveis de ambiente

Listing D.19 – `src/config/env.ts` — o *refine* que evita um incidente

```
}).superRefine((env, ctx) => {
  if (env.AI_PROVIDER === "vertex" && !env.VERTEX_PROJECT_ID) {
    ctx.addIssue({
      code: z.ZodIssueCode.custom,
      path: ["VERTEX_PROJECT_ID"],
      message:
        "obrigatorio com AI_PROVIDER=vertex (em dev, use AI_PROVIDER=mock " +
        "para subir sem GCP)",
    });
  }
});

export function validarEnv(config: Record<string, unknown>): Env {
  const parsed = EnvSchema.safeParse(config);
  if (!parsed.success) {
    const erros = parsed.error.issues
      .map((i) => `  ${i.path.join(".")}: ${i.message}`)
      .join("\n");
    throw new Error(`Variaveis de ambiente invalidas:\n${erros}`);
  }
  return parsed.data;
}
```

NOTA

Sem esse `superRefine`, a URL do Vertex viraria `.../projects/undefined/...` e o erro só apareceria na *primeira geração real* — provavelmente de um professor, em aula. A mensagem diz o que fazer, não só o que está errado.

D.9 Importação de usuários por CSV

Listing D.20 – `POST /usuarios/importar-csv` — corpo `text/csv` cru

```
nome,usuario,senha,papel,turma,serie
Ana Souza,ana.souza,trocar123,estudante,8A,8EF
Bruno Lima,bruno.lima,trocar123,estudante,8A,8EF
Carla Dias,carla.dias,trocar123,professor,,
```

> **NOTA**
>
> O corpo é `text/csv` **cru**, com teto de 2 MB — por isso `src/main.ts` registra um *body parser* explícito de texto além do JSON. `username` é único **por escola**: uma rede tem muitos `joao.silva`.

Apêndice E

Dicionário de dados

Catálogo das 78 tabelas de `public`, mais o *schema* `particoes`. O número é o de `model` em `apps/api/prisma/schema.prisma` em 2026-09-30 (todos com `@@map`; a `_prisma_migrations` é do Prisma e fica fora), e o índice abaixo tem uma linha por `model`: tabela nova sem linha aqui é o sinal de que este apêndice ficou para trás. Os diagramas entidade-relacionamento estão no Capítulo 8.

Regime	Significado
Tenant	Tem `escola_id NOT NULL` + `ENABLE/FORCE ROW LEVEL SECURITY` + política por escola. Acesso só por `comTenancy`.
Tenant+posse	O acima, *mais* filtro por `professor_id` em toda *query*: dois professores da mesma escola não se leem.
Catálogo	Sem `escola_id`, **fora** da RLS. Pertence ao contrato, não à escola.

E.1 Índice de tabelas

Tabela	Regime	Conteúdo
`escolas`	Tenant raiz	Unidades escolares do contrato
`periodos_letivos`	**Catálogo**	Ano letivo e seus recortes
`turmas`	Tenant	Turmas por escola, período e série
`matriculas`	Tenant	**O enturmamento** (estudante e professor)
`lotacoes`	**Fora da RLS**	**A lotação**: pessoa × escola, datada — lida no *login*, antes de haver escola (Seção 8.5)
`liberacoes`	Tenant	**A missão dentro da turma** (ADR-69)
`equipes_montagens`	Tenant	**Quem divide um computador**: a montagem por turma × missão × atividade (ADR-111, ADR-130)
`equipes`	Tenant	Um computador da montagem, com nome e ordem

Tabela	Regime	Conteúdo
`equipes_membros`	Tenant	Quem senta diante de cada computador — uma equipe por criança por atividade
`evidencias_marcas`	Tenant	**A evidência pedagógica** (ADR-112): o que a missão viu, por marcador da ficha, atividade e rodada
`evidencias`	Tenant	O nível de um estudante numa habilidade da BNCC, por atividade e rodada — N/A é a ausência de linha
`evidencias_eventos`	Tenant	O histórico da evidência — append-only no banco
`users`	Tenant	Pessoas e papéis
`perfis_portateis`	Tenant	O perfil que veio de OUTRA instalação, assinado — nunca misturado ao dado local (ADR-35)
`experiencias`	**Catálogo**	Manifesto de versões
`experiencias_versoes`	**Catálogo**	Uma linha por versão publicada, com o manifesto inteiro (ADR-87)
`experiencias_arquivadas`	**Catálogo**	A experiência arquivada, com a fotografia do arquivamento (ADR-116)
`disponibilidades`	**Catálogo**	**O interruptor da rede** (ADR-87)
`prospeccao_leads`	**Sem tenant**	Cadastro de interesse do site público (ADR-107) — a ÚNICA tabela de dado pessoal sem `escola_id` e sem RLS
`recursos_auditoria`	**Catálogo**	**Quem mexeu no interruptor** — append-only (ADR-89)
`progresso`	Tenant	Progresso genérico da SDK
`ia_uso_diario`	Tenant	Contador de quota de IA
`ia_quotas`	**Catálogo**	**O teto diário de IA por perfil** (ADR-105)
`agente_professor_conversa`	Tenant	**A posse** das conversas do agente de IA do professor; o conteúdo mora no Agent Engine (ADR-126)
`telemetria_eventos`	Tenant, particionada	Canal analítico
`exp08ef1b_activities`	Tenant	Atividade preparada pelo professor
`exp08ef1b_authors`	Tenant	Personas fictícias
`exp08ef1b_posts`	Tenant	Posts gerados
`exp08ef1b_post_comments`	Tenant	Comentários dos posts
`exp08ef1b_generation_jobs`	Tenant	Fila de geração
`exp08ef1b_runs`	Tenant	Sessão de um estudante
`exp08ef1b_events`	Tenant	Canal **operacional** de telemetria
`exp08ef1b_student_posts`	Tenant	Posts publicados pelo estudante

Tabela	Regime	Conteúdo
exp08ef1b_conclusoes	Tenant	Inclinação escolhida — a ÚLTIMA em inclinacao e a PRIMEIRA em inclinacao_inicial, com trocas. A tela convida a mudar de ideia (“vale a última”) e o upsert apagava a primeira; atualizado_em existe porque created_at congelava no primeiro envio e passava a mentir (ADR-43)
exp08ef1b_reflexoes	Tenant	Texto de fechamento
exp08ef1b_dossiers	Tenant	Dossiê calculado
exp08ef1b_comentarios_da_sessao	Tenant	Comentários da sessão
exp08ef1b_app_70_perfis_	Tenant	**Contrato entre duas experiências**
backup_exp04ef1b_sessoes	Tenant	Estado da missão do UnooTube (arquivada, **ADR-116**)
exp04ef1b_pacotes	Tenant	A ATIVIDADE do Jornal da Escola: as 8 notícias com o gabarito e a consequência, e as URLs das imagens
exp04ef1b_gates_turma	Tenant	A etapa da aula (os dois portões), o pacote e a rodada, por turma
exp04ef1b_sessoes	Tenant	Uma sessão por estudante, pacote e rodada: as pistas abertas, as duas edições, o motivo e a Chave — sem texto livre
exp04ef1b_diagramacoes	Tenant	O *cache* do layout escolhido pela IA, por pacote e combinação de vagas
exp02em1b_sessoes	Tenant	Estado da partida da Missão Tempestade
exp02em1b_mensagens	Tenant	A Transcrição (texto livre do estudante)
exp02em1b_marcos	Tenant	Trilha da decisão
exp02em1b_simulacoes	Tenant	Desfecho por tentativa
exp02em1b_relogio_turma	Tenant	O Relógio da Aula, por turma
exp01ef1b_turma_config	Tenant	A aula da exp01ef1b por turma: pacote aprovado, latch e tentativa
exp01ef1b_runs	Tenant	Execução por estudante e tentativa (reset arquiva, não apaga)
exp01ef1b_posts_turma	Tenant	O mural coletivo, assinado pela marca — sem nome de criança
exp06ef2b_sessoes	Tenant	Estado da missão de O Rastro do Boato: passo, opção e latência da decisão
exp06ef2b_auditorias	Tenant	Cada abertura de “Dados da mensagem” — a evidência que a missão mede
exp06ef2b_reflexoes	Tenant	O “Meu percurso” (texto livre do estudante)
exp06ef2b_gates_turma	Tenant	A etapa da aula, por turma

Tabela	Regime	Conteúdo
`exp07ef1b_coberturas`	Tenant	A ATIVIDADE da A Seleção: o tema homologado, os 12 posts com o gabarito, as URLs da mídia e as três falas do podcast
`exp07ef1b_gates_turma`	Tenant	A etapa da aula, a cobertura e a rodada, por turma
`exp07ef1b_sessoes`	Tenant	Uma sessão por estudante, cobertura e rodada: a tríade, a legenda, a matriz e o par
`exp07ef1b_reflexoes`	Tenant	A reflexão (texto livre do estudante, só do professor)
`exp08ef4b_conteudo_turma`	Tenant	O pacote curado da turma, homologado pela professora
`exp08ef4b_sessoes`	Tenant	Estado da missão de O Manifesto da IA
`exp08ef4b_auditorias`	Tenant	Cada questionamento disparado contra a máquina
`exp08ef4b_respostas_percurso`	Tenant	O “Meu percurso” (texto livre do estudante)
`exp08ef4b_gates_turma`	Tenant	O *checkpoint* do debate, por turma
`exp05ef1b_pacotes`	Tenant	O pacote da turma — as 8 mensagens com o gabarito; é a atividade da evidência
`exp05ef1b_gates_turma`	Tenant	O relógio da turma: triagem, revelação, síntese, reflexão
`exp05ef1b_sessoes`	Tenant	A sessão por rodada, com os três *latches* (Mural, voto, regra)
`exp05ef1b_reflexoes`	Tenant	A regra Se/Então (texto livre do estudante)
`app_70_sessoes`	Tenant	Uma partida da Vila do Lume
`app_70_perfis`	Tenant	Retrato Schwartz recalculado
`app_70_bolhas`	Tenant	Bolha por estudante
`app_90_planos`	Tenant+posse	Metadados do plano de aula
`app_90_referencias`	**Catálogo**	Referências curriculares
`app_92_configuracoes`	Tenant+posse	A regência: componente, ano, carga e grade
`app_92_planos_anuais`	Tenant+posse	Plano anual e a distribuição por período
`app_92_planos_de_periodo`	Tenant+posse	As aulas da semana ou quinzena
`app_91_ciclos`	Tenant+posse	Ciclo do DASS-21 (**sensível**)
`app_91_checkins`	Tenant+posse	*Check-in* diário (**sensível**)

E.2 Tabelas da plataforma

E.2.1 `escolas`

Coluna	Tipo	Nota
id	`TEXT` PK	**Texto, não UUID.** É o *mesmo* valor que circula no *claim* `escola_id` do JWT e no `set_config` da RLS — um id legível (`sp-001`) vale mais que um UUID quando o valor aparece em três lugares de um diagnóstico.
`nome`	`TEXT`	
`codigo_inep`	`TEXT?`	O `orgs.identifier` do OneRoster. É por onde a escola daqui se reconhece na base da rede.
`tipo`	`TEXT`	`escola` | `regional` | `rede` — *CHECK* na migração. Só a `escola` é *tenant*: regional e rede não têm turma, estudante nem lotação. **Agrupa, não autoriza** (ADR-34).
pai_id	`TEXT?` FK	Auto-relacionamento. **Obrigatório quando `tipo = 'escola'`** (*CHECK* `escolas_escola_tem_regional_check`) e o pai tem de ser `regional` — nunca `rede`. Nulo só na própria regional. O *trigger* `escolas_pai_agrupa` recusa pai que seja `tipo = 'escola'` ou a própria linha, e ADOTA pela regional padrão a escola que chegue sem regional — ADR-79.
`regional_padrao`	`BOOLEAN`	A regional que a plataforma cria sozinha, para a rede sem divisão administrativa. **No máximo uma** — índice único PARCIAL `escolas_regional_padrao_unica`. Nasce chamada `Rede` e o nome é EDITÁVEL: o nome real do contrato não existe no banco, porque o contrato é a stack e não uma linha — ADR-79.
`fonte_id`	`TEXT?`	`sourcedId` desta organização no sistema de origem. Chave de reconciliação da importação — nunca é o id daqui.
`created_at`	`TIMESTAMPTZ(6)`	
`atualizado_em`	`TIMESTAMPTZ(6)`	

E.2.2 `perfis_portateis`

O documento assinado que veio de OUTRA instalação da plataforma (ADR-35). Ele NÃO se mistura com o dado local: não vira matrícula, progresso nem perfil da Vila, e não entra em média nenhuma.

Coluna	Tipo	Nota
id	`UUID` PK	
escola_id	`TEXT` FK	RLS por escola. O documento pertence à unidade que o recebeu.
estudante_id	`UUID` FK	Ligado pelo CPF na importação — nunca cria a pessoa: matricular é ato da escola, e só depois o perfil se anexa.

`emissor_contrato`	`TEXT`	Quem emitiu. Pode ser uma rede que esta instalação nunca viu.
`emissor_hub`	`TEXT`	Origem `https` onde o JWKS é procurado.
`emissor_nome`	`TEXT?`	Nome legível da rede de origem.
`emitido_em`	`TIMESTAMPTZ(6)`	Do documento, não da importação.
`documento`	`JSONB`	O documento como veio — fonte para reprocessar. **Única exceção à contenção do CPF** (ADR-33): o número é o conteúdo, e alterá-lo destruiria a assinatura. A exceção está numa lista explícita dentro do teste estrutural.
`assinatura`	`TEXT`	JWS compacto; o *payload* é `{ hash }` do documento canônico.
`verificacao`	`TEXT`	`verificada` | `nao-verificada` — *CHECK*. Assinatura INVÁLIDA nunca chega aqui: é recusa.
importado_por	`UUID` FK	Quem importou. `ON DELETE RESTRICT`.
`importado_em`	`TIMESTAMPTZ(6)`	

> **DECISÃO DE PROJETO**
>
> `UNIQUE (escola_id, estudante_id, emissor_contrato, emitido_em)`: a escola recebe o arquivo por e-mail e alguém importa duas vezes. Duas cópias do mesmo fato pareceriam duas medidas. Reimportar reconfere a assinatura e atualiza só o veredito — a rede pode ter voltado, e "não consegui falar com o emissor" deixar de valer não é fato novo sobre a criança.

E.2.3 `periodos_letivos` (catálogo)

Coluna	Tipo	Nota
id	`UUID` PK	
`tipo`	`TEXT`	`ano_letivo` | `semestre` | `bimestre` | `trimestre` — *CHECK* na migração.
`titulo`	`TEXT`	`UNIQUE (tipo, titulo)`.
`ano`	`SMALLINT`	
`inicio`, `fim`	`DATE`	*CHECK*: `fim >= inicio`. O período "de hoje" é derivado destas datas — sem coluna `ativo`, que é estado que ninguém atualiza.
pai_id	`UUID?` FK	Hierarquia do padrão: ano letivo → bimestre.
`fonte_id`	`TEXT?`	

Sem `escola_id` e sem RLS, de propósito: o calendário é do **contrato**, mesmo precedente de `app_90_referencias`. Uma rede compartilha o ano letivo, e duplicá-lo por escola só criaria cem linhas divergentes.

E.2.4 `turmas`

Coluna	Tipo	Nota
id	`UUID` PK	
escola_id	`TEXT` FK	
periodo_letivo_id	`UUID` FK	**NOT NULL** de propósito: turma sem período é a coluna `users.turma_id` de novo, com outro nome — e é exatamente o que a virada do ano quebra.
`nome`	`TEXT`	`UNIQUE (escola_id, periodo_letivo_id, nome)`: duas escolas podem ter "8A", e o "8A" de 2026 e o de 2027 são turmas diferentes.
`serie`	`TEXT`	`1EF–9EF` | `1EM–3EM`, validado com Zod na aplicação.
`tipo`	`TEXT`	`regular` | `componente` — o `classType` do padrão (`homeroom`|`scheduled`). *CHECK* na migração.
`turno`	`TEXT?`	`manha` | `tarde` | `noite` | `integral`.
`codigo`, `fonte_id`	`TEXT?`	Código da turma na rede e o `sourcedId` dela no sistema de origem.

E.2.5 `matriculas`

Coluna	Tipo	Nota
id	`UUID` PK	
escola_id	`TEXT`	**RLS** por esta coluna. Consulta fora de `comTenancy` volta **vazia**, sem erro — o sintoma é lista de pessoas "sem turma".
turma_id	`UUID` FK	`ON DELETE CASCADE`.
usuario_id	`UUID` FK	`ON DELETE CASCADE`.
`papel`	`TEXT`	`estudante` | `professor` — *CHECK* na migração. **Gestor não entra**: o papel dele é da escola, não da turma.
`principal`	`BOOLEAN`	A turma de referência da pessoa. É ela que alimenta o cache `users.turma_id` e, por ele, o *claim* `turma_id` do *token*.
`inicio`	`DATE`	
`fim`	`DATE?`	Preenchido em vez de `DELETE` quando a pessoa sai. *CHECK*: `fim >= inicio`.
`fonte_id`	`TEXT?`	

`UNIQUE (escola_id, turma_id, usuario_id, papel)` mais índices por `(escola_id, usuario_id)` — "as turmas do professor" — e por `(escola_id, turma_id)` — "quem está nesta turma".

E.2.6 `liberacoes`

A missão DENTRO da turma (ADR-69). Sem linha, a experiência não abre para o estudante daquela sala. Não confundir com o relógio da exp02em1b nem com os *gates* da exp06ef2b, que respondem "a aula já começou" e são tabelas DA experiência: esta é a pré-condição de todos eles.

Coluna	Tipo	Nota
id	`UUID` PK	
escola_id	`TEXT`	**RLS** por esta coluna.
turma_id	`UUID` FK	`ON DELETE CASCADE`.
`exp_id`	`TEXT`	O *slug* da experiência, **sem FK** para `experiencias`: aquele catálogo é escrito pelo CI ao publicar, e em desenvolvimento está vazio. Mesma escolha de `progresso.exp_id`.
`estado`	`TEXT`	`liberada` | `encerrada` — *CHECK* na migração. **Não há `bloqueada`**: bloqueio é a ausência da linha.
liberada_por	`UUID` FK	Quem apertou. É a pergunta que a sala faz quando a missão aparece sem ninguém saber por quê — turma com dois professores é comum.
`liberada_em`	`TIMESTAMPTZ`	
`encerrada_em`	`TIMESTAMPTZ?`	*CHECK*: `(estado = 'encerrada') = (encerrada_em IS NOT NULL)`. Data sem regra que a cumpra é a armadilha do `expires_at`; reliberar LIMPA a data.

`UNIQUE (escola_id, turma_id, exp_id)` — uma linha por missão por sala, que é o que torna liberar idempotente — e índice por `(escola_id, turma_id)`, que é a leitura da grade.

E.2.7 `equipes_montagens`

As equipes na missão (ADR-111): a decisão da aula, uma linha por turma × missão × ATIVIDADE (ADR-130) — duas atividades da mesma missão na turma são duas aulas, com o laboratório dividido de jeitos diferentes. Equipe é quem DIVIDE UM COMPUTADOR, e não agrupamento pedagógico. Não confundir com as equipes Alfa e Bravo da Missão Tempestade, que são personagens da cidade: esta é gente sentada diante da mesma máquina.

Coluna	Tipo	Nota

(escola_id, turma_id, exp_id, atividade_id)	PK composta	Uma montagem por atividade por sala. `escola_id` é a coluna da **RLS**.
`atividade_id`	`TEXT`	O id que a MISSÃO dá à atividade (a cobertura, o pacote, a `activity` do Instanooria): opaco, *CHECK* `^[A-Za-z0-9_.:-]{1,100}$` — o alfabeto de `evidencias_marcas.atividade_id`. Sem FK: são quatro tabelas donas. A missão sem atividade (o unooGPT) usa o fixo `turma`.
turma_id	`UUID` FK	Para `turmas`, `ON DELETE CASCADE`.
`exp_id`	`TEXT`	**Sem FK** para `experiencias`, pela razão de `liberacoes.exp_id`. Quem restringe as missões que aceitam equipe é `EXPERIENCIAS_COM_EQUIPE`, no código.
`modo`	`TEXT`	`sorteio` | `manual` | `individual` — *CHECK* `equipes_montagens_modo_check`, o MESMO vocabulário de `MODOS` (`equipes/regras.ts`). `manual` nasce com N equipes VAZIAS; `individual` ("um aluno por computador") grava a montagem sem equipe nenhuma.
`computadores`	`INTEGER`	Nulo SÓ no `individual`, e `>= 1` nos outros — *CHECK* com `IS NOT NULL` explícito, porque um *CHECK* que dá NULL passa. Zero não é aula com equipes: quem desfaz apaga a montagem.
`ausentes`	`UUID[]`	Padrão vazio. Quem faltou na hora de montar: fica fora do sorteio. **Não é frequência**, e não é matrícula — amanhã a criança volta e entra na próxima.
`versao`	`INTEGER`	Padrão 1, *CHECK* `>= 1`. A concorrência otimista: o `PUT` diz a versão em que se baseou (`0` sem montagem), e versão velha é 409.
`atualizado_por`	`UUID`	Quem salvou por último. **Sem FK**, de propósito: é carimbo, e uma FK para `users` faria a saída de um professor da rede apagar ou travar a montagem da turma.
`atualizado_em`	`TIMESTAMPTZ`	É por ele que "Usar as equipes da atividade anterior" escolhe a montagem mais recente da turma.

Índice por `(escola_id, turma_id)`. RLS habilitada e forçada, com a política `equipes_montagens_por_escola`; *grant* da *role* de *runtime*: `SELECT, INSERT, UPDATE, DELETE`, nas três tabelas.

E.2.8 `equipes`

Um computador da montagem.

Coluna	Tipo	Nota

id	`UUID` PK	`gen_random_uuid()`.
(escola_id, turma_id, exp_id, atividade_id)	FK composta	Para `equipes_montagens`, `ON DELETE CASCADE`: desfazer a montagem leva as equipes junto, e salvar de novo APAGA as equipes e grava as novas. **RLS** por `escola_id`.
`nome`	`TEXT`	O que a criança lê na tela (“Equipe 3”). *CHECK* `char_length(btrim(nome)) BETWEEN 1 AND 30` — o Postgres o devolve desmontado em `>= 1 AND <= 30` no `pg_get_constraintdef`. O teto é `NOME_MAX` no código.
`ordem`	`INTEGER`	*CHECK* `>= 0`. A ordem em que o professor montou, que é a da tela: pelo nome, “Equipe 10” viria antes de “Equipe 2”.

`UNIQUE (escola_id, turma_id, exp_id, atividade_id, nome)` — duas “Equipe 3” no mesmo painel são a dúvida que ninguém resolve no meio da aula; a repetição que só difere em maiúsculas é recusada antes, em `validarMontagem` — e índice por `(escola_id, turma_id, exp_id, atividade_id)`.

E.2.9 `equipes_membros`

Quem senta diante de cada computador. É a única das três com id de criança.

Coluna	Tipo	Nota
(equipe_id, estudante_id)	PK composta	
escola_id	`TEXT`	**RLS** por esta coluna.
equipe_id	`UUID` FK	Para `equipes`, `ON DELETE CASCADE`.
`turma_id`, `exp_id`, `atividade_id`	`UUID`, `TEXT`, `TEXT`	REPETIDOS da equipe, e sem FK própria: existem para o `UNIQUE` abaixo. Sem elas aqui, a regra de uma equipe por criança por atividade só existiria no código.
estudante_id	`UUID` FK	Para `users`, `ON DELETE CASCADE`. A linha NÃO some quando a criança muda de turma: a leitura filtra por `users.turma_id` (`equipes/consulta.ts`).

`UNIQUE (escola_id, turma_id, exp_id, atividade_id, estudante_id)` — **uma criança, uma equipe por atividade**, garantida pelo banco mesmo com dois `PUT` em paralelo — e índice por `(escola_id, estudante_id)`, que é a leitura de `GET /equipes/minha`.

E.2.10 `evidencias_marcas`

A evidência pedagógica (**ADR-112**), o DADO BRUTO: o que a missão viu o estudante fazer numa atividade, numa rodada — uma AÇÃO (“tirou o post do ar”) ou a resposta de uma PERGUNTA de

múltipla escolha. É a única das três que a missão escreve, pela porta `EvidenciasService.marcar`; a evidência que o professor lê é derivada daqui.

Coluna	Tipo	Nota
(escola_id, exp_id, atividade_id, rodada, estudante_id, marcador)	PK composta	**Sem** `turma_id`: a marca é da criança naquela atividade e rodada. É o conflito nesta chave que faz valer a PRIMEIRA resposta. `escola_id` é a coluna da **RLS**.
turma_id	`UUID` FK	Para `turmas`, `ON DELETE CASCADE`. O contexto em que a marca foi registrada.
`exp_id`	`TEXT`	**Sem FK** para `experiencias`, pela razão de `liberacoes.exp_id`.
`atividade_id`	`TEXT`	O id da ATIVIDADE, dado pela missão e opaco para a plataforma. *CHECK* com o mesmo *regex* de `ATIVIDADE_ID_RE`: 1 a 100 caracteres entre letras, dígitos e `_ . : -`, porque o id vai para o endereço da rota do painel.
`rodada`	`INTEGER`	*CHECK* `>= 1`. Reiniciar a atividade abre a rodada seguinte; a anterior fica.
estudante_id	`UUID` FK	Para `users`, `ON DELETE CASCADE` — a trava da identidade única, e a marca sai com a criança.
`habilidade`	`TEXT`	O código da BNCC que o marcador mede, copiado da ficha no registro (`EF08CO05`).
`marcador`	`TEXT`	O id do marcador na ficha.
`status`	`TEXT`	`atingiu` | `em_processo` | `nao_atingiu` — *CHECK*. Só os três níveis: "não participou" é do professor, nunca o que a missão viu. Na ação, fica a MELHOR marca.
`tipo`	`TEXT`	`acao` | `multipla_escolha` — *CHECK*. Na ação vale a melhor; na pergunta, a primeira.
`versao_ficha`	`INTEGER`	*CHECK* `>= 1`. A versão da ficha em vigor quando a marca entrou.
`registrada_em`	`TIMESTAMPTZ`	É por ela que se acha a RODADA EM ANDAMENTO da turma × missão — a da marca mais recente.

Índice por `(escola_id, turma_id, exp_id, registrada_em)`, que é a busca da rodada em andamento. RLS habilitada e forçada, com a política `evidencias_marcas_por_escola`; *grant* da *role* de *runtime*: `SELECT, INSERT, UPDATE, DELETE`.

E.2.11 `evidencias`

A EVIDÊNCIA de uma habilidade: o nível de UM estudante numa habilidade da BNCC, numa atividade, numa rodada. É o que o painel do professor lê. **"Não avaliado" é a ausência de linha.**

Coluna	Tipo	Nota
id	`UUID` PK	`gen_random_uuid()`.
`escola_id`, turma_id, `exp_id`, `atividade_id`, `rodada`		Como em `evidencias_marcas`, com os mesmos *CHECKs*; `turma_id` com FK para `turmas` em cascata. **RLS** por `escola_id`.
estudante_id	`UUID` FK	Para `users`, `ON DELETE CASCADE`.
`habilidade`	`TEXT`	O código da BNCC.
`status`	`TEXT`	`atingiu` | `em_processo` | `nao_atingiu` | `nao_participou` — *CHECK*. `nao_participou` é o N/A EXPLÍCITO e só aceita a origem `professor` (`evidencias_nao_participou_check`).
`origem`	`TEXT`	`propria` | `equipe` | `projecao` | `professor` — *CHECK*. A precedência (professor > própria > cópia) é regra de `evidencias/regras.ts`.
`situacao`	`TEXT`	`valida` | `a_revisar` — *CHECK*. `a_revisar` SÓ na projeção (`evidencias_a_revisar_check`), e não conta até o professor confirmar.
`fonte_estudante_i`	`UUID`	De quem a cópia veio. Preenchida se e só se a origem for `equipe` ou `projecao` (`evidencias_fonte_check`). **Sem FK**: é carimbo.
`equipe_id`	`UUID`	A equipe da cópia no momento em que ela foi feita. **Sem FK**, de propósito: as equipes são trocadas INTEIRAS a cada `PUT` (**ADR-111**), e a cópia de uma rodada passada guarda a equipe que teve.
`editado_por`	`UUID`	Quem ajustou. Preenchida se e só se a origem for `professor` (`evidencias_editado_por_check`). **Sem FK**: carimbo não trava a saída de um professor da rede.
`atualizado_em`	`TIMESTAMPTZ`	`@updatedAt`.

`UNIQUE (escola_id, exp_id, atividade_id, rodada, estudante_id, habilidade)` — uma evidência por habilidade, por estudante, por rodada — e índices por `(escola_id, turma_id, exp_id, atividade_id, rodada)`, a leitura do painel, e por `(escola_id, estudante_id)`, a do perfil. RLS habilitada e forçada (`evidencias_por_escola`); *grant*: `SELECT, INSERT, UPDATE, DELETE`.

E.2.12 `evidencias_eventos`

O HISTÓRICO da evidência: toda mudança, com de/para, quem e quando. **Append-only no banco**, no desenho de `recursos_auditoria`.

Coluna	Tipo	Nota
id	`UUID` PK	`gen_random_uuid()`.
`escola_id`, turma_id, `exp_id`, `atividade_id`, `rodada`		Como nas outras duas. **RLS** por `escola_id`.
estudante_id	`UUID` FK	Para `users`, `ON DELETE CASCADE` — e não `SET NULL`: anular seria um `UPDATE`, que o *trigger* recusa. NULA no evento da TURMA (a confirmação da projeção vale para todos).
`habilidade`	`TEXT`	NULA no evento da turma.
`tipo`	`TEXT`	`marca` | `copia` | `recalculo` | `confirmacao` | `ajuste` — *CHECK*.
`de`, `para`	`JSONB`	O estado antes (`null` = não havia linha) e depois (`null` = a linha saiu).
`autor_id`	`UUID`	O estudante na marca, o professor no ajuste e na confirmação, `null` no que a plataforma recalculou sozinha. **Sem FK**.
`criado_em`	`TIMESTAMPTZ`	

O *trigger* `evidencias_eventos_sem_update` (`BEFORE UPDATE`) chama `evidencias_eventos_nao_reescreve`, que recusa o `UPDATE` para qualquer *role*. O `DELETE` fica fora do *trigger* (retenção, isolamento de teste, cascata da turma), e a *role* de *runtime* recebe só `SELECT, INSERT`. Índices por `(escola_id, turma_id, exp_id, atividade_id, rodada)` e por `(escola_id, estudante_id)`; RLS habilitada e forçada (`evidencias_eventos_por_escola`).

E.2.13 `users`

Coluna	Tipo	Nota
id	`UUID` PK	
`role`	`TEXT`	`estudante` | `professor` | `gestor_escola` | `gestor_regional` | `gestor_contrato` | `admin` — *CHECK* na migração, e o MESMO vocabulário de `PERFIS` (`auth/claims.ts`) e do *claim* `perfil` do *token*. Foram quatro até a **ADR-80**. Quem atribui qual sai da **cadeia** (`podeCriarPapel`), não do `@Roles`.
escola_id	`TEXT?` FK	***CHECK* na migração:** `NOT NULL` *exceto* para `gestor_contrato` e `admin` (que enxergam a rede inteira) e `gestor_regional` (cuja escola ativa é escolhida por sessão, entre as da regional dele — não há lotação a cachear). Nenhuma outra combinação é aceita pelo banco.

regional_id	`TEXT?` FK	A regional que um `gestor_regional` cobre, e SÓ ele: *CHECK* **bicondicional** (`((role = 'gestor_regional') = (regional_id IS NOT NULL))`), mais *trigger* `users_regional_agrupa` exigindo que a linha apontada seja `tipo = 'regional'` — *CHECK* não aceita subconsulta no Postgres. É daqui que sai a lista de escolas que ele pode ativar, no lugar da lotação; ele continua trabalhando UMA escola por vez, e a RLS não muda (**ADR-80**).
turma_id	`UUID?` FK	
`username`	`TEXT`	`UNIQUE (escola_id, username)` — único **por escola**, não globalmente: uma rede tem muitos `joao.silva`.
`display_name`	`TEXT`	
`password_hash`	`TEXT`	
`avatar`	`TEXT?`	
`matricula_externa`	`TEXT?`	O RA da pessoa na rede. É a chave de reconciliação da importação: com ela, corrigir nome ou login continua sendo a mesma pessoa; sem ela o *upsert* cai no `username` e um login corrigido vira um segundo estudante, com o histórico preso no primeiro. Índice único **parcial** por escola — vive na migração, porque o Prisma não modela índice parcial.
`cpf`	`TEXT?`	Os 11 dígitos, **sem pontuação** (*CHECK*) — a formatação é da tela, e guardar as duas formas faria "529.982.247-25" e "52998224725" coexistirem como duas pessoas. É a identidade IMUTÁVEL na rede: sobrevive a troca de nome, *login*, matrícula e ESCOLA, e é por ela que uma transferência é reconhecida como a mesma criança. Único por **contrato** (índice parcial), não por escola. **O gestor vê o número** — é como ele confere identidade contra o sistema da rede. Esta é a ÚNICA coluna do banco onde o CPF existe (**ADR-33**).
`fonte_id`	`TEXT?`	`sourcedId` da pessoa no sistema de origem. **Distinto** do `sub` do provedor de identidade: sistema de gestão e IdP são coisas diferentes, e as duas reconciliações não são a mesma.
`email`	`TEXT?`	
`ativo`	`BOOLEAN`	Saiu da rede → `false`. Nunca `DELETE`: há dado de estudante pendurado. `ativo = false` recusa *login* com a mesma mensagem genérica de credencial inválida.
`senha_provisoria`	`BOOLEAN`	A senha veio da importação ou de um reajuste e ainda não foi trocada. Enquanto verdadeira, o hub prende em `/trocar-senha` e não emite *token* de experiência (**ADR-29**).
`senha_definida_em`	`TIMESTAMPTZ?`	
`atualizado_em`	`TIMESTAMPTZ(6)`	

`turma_id` permanece, com novo significado: **cache** da matrícula principal. O *login* e o *claim* `turma_id` do *token* — contrato congelado — o leem antes de haver escola no contexto, onde `matriculas` não é alcançável.

E.2.14 `experiencias` (catálogo)

Coluna	Tipo	Nota
exp_id	`TEXT` PK	Slug da experiência.
`nome`	`TEXT`	
`versao_ativa`	`TEXT`	***Rollback* = `UPDATE` deste campo.** O hub monta a URL do `iframe` com ele.
`series`	`TEXT[]`	**Vazio = TODAS as séries**, não nenhuma. É assim que uma experiência transversal se declara. Desde a sala de aula (**ADR-69**) este campo é LIDO: é ele que decide se a missão entra na grade de uma turma.
`unidade`	`SMALLINT?`	A unidade do ano (1 a 4), com *CHECK*. `NULL` na transversal e na ferramenta de professor — elas não moram numa unidade. É o que ORDENA a grade da sala. Desde a **ADR-110** o `exp_id` de missão já segue a grade (`exp06ef2b` antes de `exp08ef1b`), mas quem ordena continua sendo a unidade: a transversal e a ferramenta não têm uma, e a ordem da grade não pode depender da forma de um identificador.
`produto`	`TEXT?`	`entre-redes` | `plano-de-aula` | `plano-de-ensino` | `bem-estar` | `agente-professor`, com *CHECK* espelhando `PRODUTOS` do código — o último não tem experiência nenhuma apontando para ele (**ADR-126**), mas o *CHECK* espelha a tupla inteira. Declarado no `public/experiencia.json`. É o que põe a experiência no painel da Unooria e o que lhe dá o TETO do produto (**ADR-87**); `NULL` ⇒ ela responde só pelo próprio interruptor. Quem cobra o campo é um teste que varre os manifestos, não o Zod da rota.
`capabilities`	`TEXT[]`	`telemetria`, `progresso`, `ia`, `avaliacao`, `tempo-real`.
`viewport`	`JSONB?`	`{largura, altura}` em pixels lógicos. Presente ⇒ o hub embeda dentro do `DispositivoFrame`.
`publico`	`TEXT`	`estudante` (default) | `professor`, com *CHECK*. `professor` ⇒ `/exp/*` responde 404.
`permissoes`	`TEXT[]`	Permissões de `iframe`; hoje só `pointer-lock`, com *CHECK* do vocabulário. **`fullscreen` NÃO entra aqui**: o `allow` do *frame* já o delega a todas, e campo de manifesto que não muda nada é pior que campo nenhum.

E.2.15 `experiencias_versoes` (catálogo)

Uma linha por VERSÃO publicada de cada experiência — a lista que o painel da Unooria oferece no *rollback* (**ADR-87**). PK composta `(exp_id, versao)`; índice `(exp_id, publicada_em DESC)`.

Coluna	Tipo	Nota
exp_id	`TEXT` PK_1	**Sem FK** para `experiencias`, pela razão de `liberacoes` e `progresso`: aquele catálogo é escrito pelo CI no *publish*, e numa *stack* nova ele está vazio.
versao	`TEXT` PK_2	*Semver* `x.y.z`.
`manifesto`	`JSONB`	**O manifesto INTEIRO daquela versão**, e não só o número. É o que torna o *rollback* fiel: hoje `publicar-manifesto.mjs --versao x.y.z` manda o número antigo com o `experiencia.json` ATUAL do repositório, então voltar a versão não voltava `viewport` nem `permissoes` — a app-70 reabria sem `allow-pointer-lock`, a criança entrava na Vila e a câmera não girava, sem erro em lugar nenhum.
`publicada_em`	`TIMESTAMPTZ(6)`	
`publicada_por`	`TEXT`	**Texto, sem FK** — ao contrário de `disponibilidades.definido_por`. Quem publica quase sempre é o CI, cujo *token* tem `sub = 'ci-deploy'`: não é UUID e não existe em `users`. Uma FK aqui mataria o *job* `ativar-exps` DEPOIS de os *bundles* já terem subido.

E.2.16 `experiencias_arquivadas` (catálogo)

A experiência **arquivada**: tirada do produto sem ser apagada (**ADR-116**). A linha SAI de `experiencias` e vem para cá — uma tabela à parte, e não uma coluna, para nenhum dos leitores do catálogo precisar lembrar de filtrar. Sem RLS, como `experiencias`: é catálogo, não dado de estudante. Quem lê é a área `/unooria/arquivados` (só `admin`) e o documento de portabilidade.

Coluna	Tipo	Nota
exp_id	`TEXT` PK	`backup-` mais o código de origem. *CHECK*: `exp_id = 'backup-' || codigo_de_origem`.
`codigo_de_origem`	`TEXT`	O código que ela tinha — e que outra missão passou a usar.
`nome`, `versao`	`TEXT`	O nome e a versão que estavam no ar.
`series`, `unidade`, `produto`		Onde ela ficava na grade.
`motivo`	`TEXT`	Por que foi arquivada, em uma frase.

`turmas_guardadas`, `estudantes_guardado`	`INTEGER`	A FOTOGRAFIA: contadas pela migração que arquivou, com o `FORCE` da RLS fora — o único momento em que contar atravessa escolas sem pedir nada à RLS. Não são recontadas.
`arquivada_em`	`TIMESTAMPTZ(6)`	

E.2.17 `disponibilidades` (catálogo)

O interruptor da rede (ADR-87): uma linha responde "este recurso existe neste escopo?". Terceiro eixo, ao lado do catálogo (que forma a experiência tem) e da liberação por turma (ADR-69, se o professor a abriu) — e **pré-condição** dos dois outros, nunca o contrário.

Ausência de linha = LIGADO, o contrário de `liberacoes`. Lá a ausência é o bloqueio; aqui é o mundo normal, porque este interruptor é mecanismo de EXCEÇÃO — não contratado, ou quebrado no *deploy*. Nascer desligado exigiria uma linha por recurso a cada contrato provisionado, e toda experiência nova ficaria invisível até alguém ligá-la.

Coluna	Tipo	Nota
id	`UUID` PK	
`recurso_id`	`TEXT`	`produto:<slug>` ou `experiencia:<expId>`, com *CHECK* do padrão espelhando `RECURSO_ID_RE` do código. Texto com prefixo, e **não** duas colunas (`tipo` + `alvo`): a chave da decisão é o par (recurso, escopo), e com duas o único abaixo teria três campos com um deles nulável — de novo o desenho em que o PostgreSQL não garante unicidade.
`escola_id`	`TEXT?`	**`NULL` = o CONTRATO inteiro.** Preenchido, é uma linha de `escolas` — que pode ser `tipo=escola` OU `tipo=regional`, porque a hierarquia já vive em `escolas.pai_id`. FK com `ON DELETE CASCADE`: escola apagada leva as exceções dela.
`ligado`	`BOOLEAN`	
`versao_fixa`	`TEXT?`	Fixa a versão NESTE escopo; `NULL` = segue `experiencias.versao_ativa`. *CHECK* exige `recurso_id LIKE 'experiencia:%'` — produto não tem *bundle*, logo não tem versão, e campo que não muda nada é pior que campo nenhum (a razão de `fullscreen` ter ficado fora de `permissoes`).
`motivo`	`TEXT?`	**Obrigatório ao desligar** (*CHECK* `ligado OR motivo IS NOT NULL`). "Desligado sem motivo" é o estado que ninguém sabe desfazer: meses depois não há como saber se é decisão comercial ou incidente não revertido. No banco e não só no Zod porque um acerto em SQL no meio de um incidente é o caso real.

`definido_por`	`UUID`	FK para `users`, `ON DELETE RESTRICT`, pela razão de `liberacoes.liberada_por`: "quem desligou o jogo da rede?" é a pergunta que se faz às 8h da manhã, e ela some do painel se o vínculo virar texto solto.
`definido_em`, `atualizado_em`	`TIMESTAMPTZ(6)`	A linha é SOBRESCRITA a cada mudança: o histórico de quem mexeu vive no registro estruturado, e a resposta em TELA é o gatilho de reavaliação da ADR-87.

Objeto	Nota
`disponibilidades_recurso_id_escola_id_key`	Único `(recurso_id, escola_id)`. **Não basta**, e é a armadilha central desta tabela: no PostgreSQL `NULL` é distinto de `NULL` num índice único, então `('produto:bem-estar', NULL)` cabe quantas vezes se quiser.
`disponibilidades_recurso_no_contrato`	**Único PARCIAL** `(recurso_id) WHERE escola_id IS NULL` — é ele que fecha o furo acima. Sem ele, dois cliques em "desligar para a rede" criam duas linhas e o resolvedor passa a depender da ordem do `findMany`: sem erro, sem sintoma, até as duas discordarem. Mesmo desenho de `escolas_regional_padrao_unica`, e é ele que dá idempotência de graça ao `upsert` do painel.

Sem RLS, nas duas tabelas, e a exceção é declarada — toda tabela nova deste repositório entra na política da regra inviolável nº 1. Duas razões: não são dado de estudante (é configuração comercial e operacional da rede, do mesmo naipe de `experiencias`, que também está fora); e a linha de escopo-contrato tem `escola_id` nulo POR DESENHO, de modo que `escola_id = current_setting('app.escola_id')` esconderia justamente a resposta que a tabela existe para dar. É o motivo de `users`, `turmas` e `lotacoes` ficarem fora: são lidas ANTES de haver escola no contexto. A ausência é provada por teste, para que a próxima migração não a "conserte": ligar RLS ali faz TODO produto parecer ligado, porque a linha que o desligava fica invisível — o sintoma oposto a um erro de permissão.

E.2.18 `recursos_auditoria` (catálogo)

Quem desligou isto, e quando? (ADR-89). A ADR-87 deixou esse gatilho aberto de propósito: a linha de `disponibilidades` é SOBRESCRITA, então o histórico vivia só no registro estruturado (`codigo=recurso-alterado`). Registro responde a pergunta para quem tem acesso ao Cloud Logging; não responde na TELA — e é na tela que ela é feita, às 8h da manhã, por quem está com a escola ao telefone.

Dois painéis leem esta tabela: o da Unooria (`/unooria/recursos`, quem opera) e o da REDE (`/admin/recursos`, quem sofre). A rede não escreve nada — ligar e desligar continua só do `admin` (ADR-88) —, mas ver por que o produto dela mudou de estado é informação do cliente.

Coluna	Tipo	Nota

id	`UUID` PK	
`recurso_id`	`TEXT`	Mesmo formato de `disponibilidades.recurso_id`, e **sem FK para ela**: a decisão registrada sobrevive à linha que a aplicava — "herdado" apaga a linha de `disponibilidades` e é justamente o evento que precisa constar.
`escola_id`	`TEXT?`	`NULL` = decisão de escopo-contrato. Também sem FK, pela razão acima.
`acao`	`TEXT`	Vocabulário FECHADO por *CHECK* (`ck_recursos_auditoria_acao`), espelhando `ACOES_DE_AUDITORIA` do código: `ligado`, `desligado`, `herdado`, `versao-ativada`, `versao-fixada`, `versao-soltada`, `quota-alterada` (**ADR-105**). O teste prende os dois lados, porque a divergência só apareceria como 500 no log.
`versao`	`TEXT?`	*CHECK* BICONDICIONAL: preenchida exatamente nas duas ações de versão, nula em todas as outras. Campo cheio onde não significa nada é pior que campo vazio — quem lê `desligado` com uma versão ao lado conclui que a versão foi parte da decisão.
`motivo`	`TEXT?`	O texto que o painel exige ao desligar, copiado para cá.
`detalhe`	`JSONB?`	`{de, para}` da mudança de quota (**ADR-105**) — e SÓ em `quota-alterada`, por *CHECK* bicondicional (`ck_recursos_auditoria_detalhe`), pela mesma razão de `versao`: campo preenchido onde não significa nada faz quem lê procurar um número que não existiu. Em `quota-alterada`, `recurso_id` é `quota:<perfil>`.
`autor_id`	`UUID`	**Sem FK**, ao contrário de `disponibilidades.definido_por`: uma FK com `RESTRICT` travaria para sempre a exclusão de qualquer conta que já tenha mexido no painel, porque histórico não se apaga.
`autor_nome`	`TEXT`	**CONGELADO na linha.** Um *JOIN* mostraria o nome de hoje — ou nada, se a conta saiu da rede —, e o registro deixaria de dizer quem era aquela pessoa NAQUELE dia. Mesma razão de `exp08ef1b_runs.nome_no_app` guardar o apelido em vez de o resolver.
`quando`	`TIMESTAMPTZ(6)`	

`UPDATE` é recusado pelo BANCO, por *trigger* (`recursos_auditoria_sem_update`). Sem ele, "append-only" seria promessa do código — e a promessa que o banco não cobra é a que a próxima rotina de correção quebra; reescrever uma linha é o único jeito de esta tabela mentir. **`DELETE` continua permitido, e isso é decisão**: retenção (nenhuma tabela deste produto tem prazo hoje, e quem cumprir o prazo precisará apagar) e isolamento de teste — tabela que não se limpa faz cada suíte deixar linha para trás.

Sem RLS, pelo mesmo par de razões de `disponibilidades` (não é dado de estudante; a linha de escopo-contrato tem `escola_id` nulo por desenho, e a política a esconderia). Quem protege a leitura é o `RolesGuard` (`GESTORES_DO_CONTRATO`) — e o *grant* é `SELECT, INSERT, DELETE`: sem `UPDATE`, a role de runtime não alcança nem o *trigger*.

E.2.19 `ia_quotas` (catálogo)

O teto diário de IA por perfil (ADR-105) — o número que o `QuotaService` compara com `ia_uso_diario` ANTES de toda chamada de IA. Uma linha por perfil, escopo do CONTRATO inteiro (1 contrato = 1 *stack*), semeada pela *migration* com os valores que valiam por *env* (300.000 e 2.000.000) e sobrescrita pelo `admin` em `/unooria/uso-de-ia`. O histórico de quem mudou vive em `recursos_auditoria` (ação `quota-alterada`).

Coluna	Tipo	Nota
perfil	`TEXT` PK	Vocabulário FECHADO por *CHECK* (`ck_ia_quotas_perfil`): `estudante`, `professor`, espelhando `PERFIS_COM_QUOTA` do código. Só esses dois chamam IA; todo outro perfil lê a linha `professor`. Perfil novo entra na tupla, no CHECK e na semente.
`tokens_por_dia`	`INTEGER`	*CHECK* > 0: zero e negativo não são quota, são o interruptor da ADR-87 entrando pela porta errada. O teto de sanidade (100.000.000) é do Zod.
`motivo`	`TEXT?`	Obrigatório na rota (Zod), sempre: aqui toda mudança é exceção ao padrão da plataforma.
`definido_por`	`UUID?`	**Sem FK**, como `recursos_auditoria.autor_id`: a linha é SOBRESCRITA e o nome congelado de quem mexeu vive no histórico. `NULL` = veio da *migration*.
`definido_em`, `atualizado_em`	`TIMESTAMPTZ(6)`	

Sem RLS, pelo par de razões de `disponibilidades`: não é dado de estudante, e não há `escola_id` — a linha vale para o contrato inteiro. *Grant* da role de runtime: `SELECT, INSERT, UPDATE`, sem `DELETE` — a linha de um perfil não se apaga; sem ela, vale a constante do código e o registro acusa `ia-quota-sem-linha`.

E.2.20 `agente_professor_conversas`

A posse das conversas do agente de IA do professor (ADR-126). O conteúdo da conversa — mensagens, estado, expiração — é a sessão do Vertex AI Agent Engine; esta linha diz **de quem ela é**, porque o engine não confere o usuário numa sessão pedida por nome e a SA da api enxerga todas. Toda rota de `/agente-professor/*` confere `usuario_id` e `escola_id` aqui antes de tocar a sessão.

Coluna	Tipo	Nota
id	`UUID` PK	`gen_random_uuid()`. É o id que o hub conhece — nunca o `sessao_id`.
escola_id	`TEXT`	`NOT NULL`, `ON DELETE CASCADE`; a escola ATIVA de quem abriu. Trocar de escola troca a lista.
usuario_id	`UUID`	`NOT NULL`, FK para `users`, `ON DELETE CASCADE`. É também o `user_id` da sessão no engine.
`sessao_id`	`TEXT`	**Único**: uma sessão tem um dono só. *CHECK* de 1 a 200 caracteres.
`titulo`	`TEXT?`	**CACHE** do `titulo_conversa` que o AGENTE escreve no estado da sessão depois do primeiro turno; nulo até lá. *CHECK* de 1 a 120 caracteres (espelha `TAMANHO_MAXIMO_DO_TITULO`).
`criada_em`, `atualizada_em`	`TIMESTAMPTZ(6)`	`atualizada_em` sobe a cada turno entregue e ordena a lista.

RLS `ENABLE` + `FORCE`, política `agente_professor_conversas_por_escola`: o título sai da pergunta, e a pergunta pode trazer o nome de um estudante. Índice `agente_professor_conversas_lista_idx` em `(escola_id, usuario_id, atualizada_em DESC)`. *Grant* da role de runtime: `SELECT, INSERT, UPDATE, DELETE` — apagar a conversa apaga a linha, depois da sessão. Migração `20260930120000_agente_professor`.

E.2.21 `progresso`

PK composta `(escola_id, estudante_id, exp_id)`; `dados JSONB DEFAULT '{}'`; `atualizado_em`. É o progresso *genérico* da SDK — estado rico de experiência vai para a tabela da experiência.

E.2.22 `ia_uso_diario`

PK composta `(escola_id, usuario_id, dia)`, com `dia` no formato `YYYY-MM-DD`; `tokens_usados INT`. Incremento atômico por `ON CONFLICT DO UPDATE ... RETURNING`, para que requisições simultâneas não se sobrescrevam.

E.2.23 `telemetria_eventos` (particionada)

Coluna	Tipo	Nota
event_id	`UUID` PK_1	Chave de *dedupe*, gerada pela SDK.

ts	`TIMESTAMPTZ(6)` PK_2	**Chave de partição.** A PK é composta porque o PostgreSQL exige a chave de partição em todo índice único — e a *dedupe* segue correta, porque a reentrega da mesma mensagem traz o *mesmo* `ts`.
`escola_id`, `estudante_id`	`TEXT`	`estudante_id` é **opaco**: nunca nome nem *e-mail*. Derivado do *token* no servidor.
`exp_id`, `exp_version`	`TEXT`	
`sessao_id`	`UUID`	
`verbo`, `objeto`	`TEXT`	Vocabulário xAPI simplificado.
`resultado`, `contexto`	`JSONB?`	Livres, por experiência.

Objeto	Nota
`idx_tel_escola_exp_ts`	Índice `(escola_id, exp_id, ts)`.
`particoes.telemetria_eventos_YYYY_MM`	Uma por mês, faixas declaradas até 2028-12. Vivem no *schema* `particoes` — em `public` cada uma viraria *drift* de *schema*.
`particoes.…_default`	**Rede de segurança.** Sem ela, evento com `ts` fora das faixas seria **recusado** — e telemetria perdida em silêncio é pior que telemetria mal guardada. Encher = faltou criar partição.

E.2.24 `prospeccao_leads` (sem tenant)

Quem pediu contato pelo formulário do site público (ADR-107), antes de existir contrato. Substituiu um SQLite (Cloudflare D1) servido por um *Worker* fora da plataforma, com listagem protegida por um *token* no fragmento da URL — um banco, um *runtime* e um modelo de autorização a mais na arquitetura, nenhum deles visível para quem opera o produto.

Sem `escola_id` e sem RLS, a exceção da regra nº 1 (Capítulo 7): não há tenant a isolar. Quem protege a leitura é `@Roles("admin")`; a escrita é pública, e a defesa dela é o Zod, a cota por IP, o campo-armadilha e o *dedupe*.

Coluna	Tipo	Nota
id	`UUID` PK	`gen_random_uuid()`. Não sai na resposta da api: nenhuma rota o consome, e um identificador no *payload* convida a uma rota de edição que não deve existir.
`nome`, `telefone`, `organizacao`, `cargo`	`TEXT`	*CHECK* de não-vazio nos cinco campos mais o aceite. O Zod da rota já recusa; o *CHECK* impede que qualquer outro caminho (semente, correção manual, importação futura) escreva a linha que a tela mostraria como uma fileira de células em branco.

email	TEXT	**Sempre em minúsculas** (*CHECK* email = lower(email)): é a chave do *dedupe*, e a mesma pessoa escreve o próprio endereço de três jeitos ao longo de um ano.
consentimento_texto	TEXT	**A FRASE que a pessoa marcou**, não um booleano. Booleano prova que alguém clicou; ele não prova a QUE se consentiu, que é o que a LGPD cobra (art. 8º, §2º). Mudou o texto no site, muda a constante da api no mesmo *commit* — as linhas antigas continuam dizendo o que valia no dia delas.
criado_em	TIMESTAMPTZ(6)	

Objeto	Nota
prospeccao_leads_email_criado_em_idx	O *dedupe* da rota pública: mesmo e-mail dentro de uma hora não vira linha nova.
prospeccao_leads_criado_em_idx	DESC — a ordem em que o painel lê.
GRANT	SELECT, INSERT, DELETE. **Com DELETE** (o titular pode revogar o consentimento e pedir a eliminação, art. 18, VI) e **sem UPDATE** (cadastro não se edita: se a pessoa reenviar com o dado certo, a linha nova é o registro certo, e a antiga continua sendo verdade sobre o que foi enviado naquele dia).

E.3 Tabelas da exp08ef1b

E.3.1 exp08ef1b_activities

Coluna	Tipo	Nota
id	UUID PK	
professor_id	UUID FK	
turma_id	UUID? FK	Para qual TURMA é esta atividade (**ADR-49**). NULL = atividade anterior à coluna, que vale para a escola inteira — o comportamento antigo, preservado sem *backfill*. Com turma, o painel de perfis, a importação da Vila e a entrada do estudante em POST /runs falam todos da MESMA sala. ON DELETE RESTRICT.
titulo, disciplina, tema_geral, recorte	TEXT	
contexto_aula	TEXT?	

`pergunta_chave`	`TEXT`	Gerada pela IA a partir do tema, validada pelo professor.
`pergunta_origem`	`TEXT`	`ia` | `professor` — **auditoria de autoria pedagógica**, e custa uma coluna.
`etapa`	`TEXT`	`investigacao` | `conclusao` | `debate` | `explicabilidade` | `perdidos` | `reflexao` | `fim`. **Relógio COLETIVO da turma:** o professor abre e fecha, e a tela do estudante trava conforme.
`ia_ao_vivo`	`BOOLEAN`	Personas reagem ao vivo. Desligável no meio da aula, **sem *deploy***.
`antagonismo`	`JSONB?`	Destilação **aprovada** pelo professor.
`categorias_neutras` `neutros_no_tema`	`TEXT[]`	Categorias dos neutros fora do tema; rótulos dos neutros dentro dele.
`estilo_visual`	`TEXT`	
`qtd_posts_tese`	`INT`	Posts que carregam o antagonismo; divisível por 8 (2 lados × 4 perfis).
`proporcoes`, `engine_config`	`JSONB`	Pesos e mixagem do motor.
`status`	`TEXT`	`rascunho` | `antagonismo` | `gerando` | `revisao` | `ativa` | `encerrada`.

E.3.2 `exp08ef1b_posts`

Coluna	Tipo	Nota
`lado`	`TEXT`	`A` | `B` | `neutro`.
`perfil`	`TEXT`	`abertura` | `autotranscendencia` | `autopromocao` | `conservacao`.
`intensidade`	`SMALLINT`	1, 2 ou 3.

> **REGRA INVIOLÁVEL**
>
> `lado`, `perfil` e `intensidade` são **a variável independente do experimento**. Eles existem no banco e são **removidos pelo sanitizador** antes de o *feed* sair da API. Se chegassem ao cliente, a experiência poderia exibi-los — e o estudante saberia de que lado o algoritmo o colocou antes da revelação.

Coluna	Tipo	Nota
`formato`	`TEXT`	`feed` | `story` | `enquete`.
`caption`, `hashtags`, `emotional_hook`	`TEXT`/`TEXT[]`	Gerados pela IA.

`image_brief`	`TEXT`	*Prompt* da arte.
`image_url`	`TEXT?`	URL absoluta da CDN de *assets*.
`image_status`	`TEXT`	`pendente` | `ok` | `fallback`. `fallback` = arte procedural, porque a geração falhou — e a atividade não trava.
`likes_count`, `timestamp_label`	`INT`/`TEXT`	Ficção visual do clone.
`aprovado`	`BOOLEAN`	O professor revisa antes de ativar.

Índice de curadoria: `(activity_id, lado, perfil, intensidade, aprovado)` — é exatamente a consulta que o motor faz a cada bloco.

E.3.3 `exp08ef1b_runs`

Coluna	Tipo	Nota
`fase`	`TEXT`	`onboarding` | `feed` | `encerramento` | `conclusao` | `revelacao` | `dossie` | `comparacao`.
`lado_inicial`	`TEXT`	`A` | `B`. Vem da afinidade do perfil; **nunca muda**.
`lado_origem`	`TEXT`	`perfil` | `sorteio` | `redistribuido` — **auditoria de por que** aquele estudante caiu naquele lado. Sem isso, uma reclamação de professor não tem resposta.
`perfil_jogo`	`JSONB?`	Perfil do Ato 1, **congelado** na criação da `run`.
`perfil_origem`	`TEXT`	`jogo` | `professor` | `ausente`.
`engine_state`	`JSONB`	Estado do motor. Lido e escrito *inteiro*, sempre dentro do mesmo *lock* — normalizá-lo daria *joins* sem ganho de consulta.
`nome_no_app`	`TEXT?`	O nome que a criança digitou. **Fica na `run`, não em `users`**, porque a conta é **do aparelho**: outra turma reusa o mesmo *login*.
`avatar_escolhido`	`TEXT?`	Mesma razão.

`UNIQUE (activity_id, estudante_id)`: uma sessão por estudante por atividade.

E.3.4 `exp08ef1b_events` — o canal operacional

Coluna	Tipo	Nota
id	`BIGSERIAL` PK	Ordenação por inserção é significativa aqui — diferente do canal analítico.
`tipo`	`TEXT`	`visto` | `curtiu` | `descurtiu` | `salvou` | `abriu_comentarios` | `comentou` | `compartilhou` | `seguiu` | `deixou_seguir` | `nao_interessa` | `denunciou` | `enquete` | `rever` | `abriu_perfil`.

payload	JSONB?	{dwellMs, dwellNorm}, {texto}, {opcao}, {authorId}…
client_ts	BIGINT	Relógio do cliente, guardado como *dado*, não usado como verdade.

> **REGRA INVIOLÁVEL**
>
> ***Dwell* nunca passa pelo Pub/Sub.** A latência e a falta de ordenação quebrariam a adaptação do *feed* e o *lock* por run. Esta tabela é o canal *operacional*; telemetria_eventos é o analítico (Capítulo 10).

E.3.5 exp08ef1b_app_70_perfis_importados

Coluna	Tipo	Nota
escores	JSONB	Os quatro quadrantes normalizados 0–1.
dominante	TEXT	Um dos quatro, ou misto.
origem	TEXT	jogo | professor.
lado_forcado	TEXT?	A | B quando o professor redistribuiu a turma; NULL = segue a afinidade. **PRESERVADO no *upsert*** — importar sozinho, ou levar o objeto inteiro no update, desfaria em silêncio uma redistribuição feita antes da aula.
vila_sessao_id	UUID?	De QUE partida da Vila esta medida saiu. **SEM chave estrangeira**, de propósito (ADR-50): zerar a turma na Vila apaga sessões, e uma cascata — ou um RESTRICT — levaria junto o perfil que está decidindo o lado de uma aula em andamento. É RASTRO da medida, não integridade. NULL = não veio da Vila desta instalação.
versao_mapeament	SMALLINT?	A VERSAO_MAPEAMENTO sob a qual o perfil foi medido. É o que permite o painel dizer que o mapeamento mudou desde a importação — sem ela, subir a constante deixava as cópias velhas decidindo o feed em silêncio.
medido_em	TIMESTAMPTZ	

E.4 Tabelas das demais experiências

E.4.1 backup_exp04ef1b_sessoes

O UnooTube **arquivado** (ADR-116): a tabela ficou com os dados, sem rota que a sirva. Era exp04ef1b_sessoes.

Coluna	Tipo	Nota

(escola_id, estudante_id)	PK composta	Uma linha por estudante.
`salvos`	`TEXT[]`	Ids do catálogo. *CHECK* `cardinality(salvos) <= 3` na migração.
`curtidos`	`TEXT[]`	Segue livre mesmo após a conclusão.
`ranking_desbloqueado`	`BOOLEAN`	***Latch***: remover um salvo depois *não* tranca o ranking de novo. Estado que anda para trás confunde a criança e o professor.
`credibilidade_adicionada`	`BOOLEAN`	A conclusão pedagógica. **Congela os salvos**: o *score* já foi reportado, e nota e estado não podem divergir.
`concluido_em`	`TIMESTAMPTZ?`	*CHECK*: conclusão implica `concluido_em`.
`reinicios`	`SMALLINT`	Reiniciar descongela.

REGRA INVIOLÁVEL

O gabarito (`factual`) **não está aqui** e não é devolvido em nenhuma rota de estudante: vive em `src/backup-unootube/catalogo.ts`. O que *sai* de lá é `VideoPublico`, isto é, tudo menos `factual` — e há teste que enumera os campos, para que um campo novo não escape por distração. Desde a 1.1.0 são também públicos `visualizacoesRotulo` e `curtidasRotulo`: rótulos **cosméticos** de popularidade, sem peso em conta nenhuma, mas escolhidos para que o vídeo *fake* seja o dos números grandes — o contraste que o *ranking* desmonta depois.

E.4.2 `exp04ef1b_sessoes`

O Jornal da Escola (**ADR-117**). A descrição das quatro tabelas e das decisões de esquema está na Seção 8.12; aqui ficam as colunas da sessão, que é onde os *latches* moram.

Coluna	Tipo	Nota
id	`UUID`	`UNIQUE` em (escola, turma, estudante, pacote, rodada): uma sessão por rodada.
`passo`	`TEXT`	O passo GRAVADO, no vocabulário de `regras/passos.ts`. Nunca `resultado` nem `chave`: a etapa da turma os impõe.
`pistas`	`JSONB`	*Array* de `{noticiaId, tipo, fase, em}`: um registro por par e FASE (1 = mesa, 2 = revisão). É o dado da EF15CO04.
`slots_ed1`, `slots_ed2`	`TEXT[]`	Vazio ou três ids DIFERENTES entre `n1` e `n8`, na ordem das vagas.
`layout_ed*`, `origem_layout_ed*`	`TEXT?`	`classic` | `hero-image` | `columns`, e `ia` | `modelo`. Cosméticos.
`verificadas_ed*`	`INTEGER?`	0 a 3, calculadas no servidor no *latch*.
`motivo_ed1`	`TEXT?`	`visual` | `checagem`, depois do *latch* da 1ª.

`chave_tentativas`, `chave_formada_em`	`INTEGER`, `TIMESTAMPTZ?`	A base da EF15CO02. Formada exige ao menos uma tentativa.
`publicada1_em`, `publicada2_em`, `concluida_em`	`TIMESTAMPTZ?`	Os *latches*, cada um EQUIVALENTE ao passo gravado.
`versao_instrumento`	`TEXT`	`1.0.0`.

REGRA INVIOLÁVEL

O gabarito, as pistas e a consequência **não estão aqui**: moram no `JSONB` de `exp04ef1b_pacotes`, que só rota de professor devolve inteiro. O estudante recebe cada parte num portão (Seção 12.6), e as chaves `gabarito`, `consequencia` e `pilarEmFalta` estão na redação do *log*.

E.5 Tabelas da exp02em1b

E.5.1 `exp02em1b_sessoes`

Coluna	Tipo	Nota
(escola_id, estudante_id)	PK composta	Uma linha por estudante. **Não há id de sessão**: a identidade vem do *token*.
`turma_id`	`UUID?`	Denormalizado: o painel do professor consulta por turma o tempo todo, e resolver por *join* a cada leitura era o caminho para esquecer o filtro numa consulta nova.
`passo`	`TEXT`	*CHECK* nos quatro Passos. Derivado do que aconteceu na conversa, não de um clique de avançar.
`tentativa`	`SMALLINT`	Recomeçar abre a seguinte. *CHECK* ≥ 1.
`minutos_gastos`	`INT`	Minuto narrativo desde as 17h00. *Snapshot* do que o relógio da turma já produziu — a fonte da verdade é `exp02em1b_relogio_turma`. É também o minuto de **saída** das equipes quando a tempestade é enfrentada. *CHECK* 0–60.
`conjuntos`	`TEXT[]`	Fontes carregadas, em camadas. *CHECK* `cardinality <= 8`.
`perguntas_gastas`	`INT`	Conta o **gasto**, não o resto: mudar o teto passa a valer para partida em curso. *CHECK* 0–3.
`alocacao_alfa`, `alocacao_bravo`	`TEXT?`	Slug do bairro de destino. O agente **não escolhe** — registra a escolha do estudante.

`justificativa_alfa`, `justificativa_bravo`	`TEXT?`	Slug de um conjunto que o estudante carregou, ou `'intuicao'`. *CHECK*: justificativa sem destino é estado impossível; destino sem justificativa é legítimo (o portão exige antes de *enviar*).
`tokens_gastos`, `chamadas_ia`	`INT`	O teto por sessão (**ADR-24**). Somam voltas de ferramenta e tentativas rejeitadas.

E.5.2 `exp02em1b_mensagens`

Coluna	Tipo	Nota
id	`BIGSERIAL`	Paginação por id é estável; por *timestamp* não.
`papel`	`TEXT`	*CHECK* `IN ('estudante','agente')`.
`texto`	`TEXT`	**Caixa livre escrita por menor de idade**. Nunca vai a log nem à telemetria; nenhuma rota de gestor a alcança (**ADR-27**).
`anexo`	`JSONB?`	O cartão que acompanha a fala. É o que faz a conversa voltar inteira depois de um F5.
`debitou`	`BOOLEAN`	Se esta fala custou uma Pergunta — a trilha de auditoria do orçamento. *CHECK*: só fala do estudante pode debitar.

E.5.3 `exp02em1b_marcos`

Coluna	Tipo	Nota
id	`BIGSERIAL`	*UNIQUE* `(escola_id, estudante_id, tentativa, ordem)`.
`tipo`	`TEXT`	*CHECK* `IN ('fontes','leitura','alocacao','envio','inicio','recomeco')`. `inicio` = entrou na Investigação; `recomeco` = clicou em Recomeçar, gravado na tentativa que fecha, com `dados.origem` (`investigacao` | `resultado`), `fase` e `modo` do relógio da turma.
`minuto`	`INT`	Minuto do relógio em que aconteceu, recalculado do relógio da turma no instante — nunca copiado de `sessoes.minutos_gastos` (**ADR-64**). *CHECK* 0–60.
`segundos_restantes`	`INT NULL`	O cronômetro de parede que o estudante via. `NULL` = modo livre, sem relógio, ou marco anterior à coluna. Não se reconstrói depois: "+2 min" muda a duração sem histórico e "zerar" apaga o relógio. *CHECK* ≥ 0.
`motivo_do_fim`	`TEXT NULL`	`tempo` | `professor` quando o relógio já tinha fechado no instante do marco: é o "estourou" do relatório, sem inferência.

`dados`	`JSONB`	*Payload* por tipo. **Só dado público**: este JSON chega ao navegador. O `envio` carrega também as `justificativas` como o professor lê — `reiniciar` zera as da sessão, e sem a cópia a da tentativa 1 sumia quando a 2 começava.

E.5.4 `exp02em1b_simulacoes`

Coluna	Tipo	Nota
id	`UUID`	*UNIQUE* `(escola_id, estudante_id, ordem)`.
`alocacao`, `desfecho`	`JSONB`	A decisão e o que ela rendeu.
`minuto_da_saida`	`INT`	O minuto em que as equipes saíram: é o que decidiu se cada uma chegou. Guardado junto do desfecho para a linha ser autoexplicativa.
`versao_prompt`	`INT`	Sem ela não se sabe qual *prompt* produziu o que está em produção.
`esperado`	`JSONB NULL`	`{v, media, faixa, vitimasMedia, vitimasFaixa, amostras, percentil}`: a faixa da MESMA decisão (200 corridas) e o percentil do resgate real, gravados no confirmar. Antes era recomputada a cada leitura, e uma mudança de parâmetro reescrevia em silêncio o relatório de partidas antigas (**ADR-64**). `NULL` = anterior à coluna; a leitura recomputa.

E.5.5 `exp02em1b_relogio_turma`

Coluna	Tipo	Nota
(escola_id, turma_id)	PK composta	O relógio é da **turma** (**ADR-25**).
`modo`	`TEXT`	*CHECK* `IN ('cronometrado','livre')`.
`iniciado_em`	`TIMESTAMPTZ?`	**Nulo = a missão ainda não começou**: o estudante vê o cronômetro parado e não fala com o agente.
`duracao_seg`	`INT`	Segundos de parede. “+2 min” soma 120. *CHECK* > 0 — zero dividiria por zero no minuto narrativo.
`encerrado_em`	`TIMESTAMPTZ?`	*CHECK*: encerrar o que nunca começou é estado impossível.
`pivo_seg`, `pivo_min`	`INT`	O **pivô** da extensão: onde o relógio da cidade estava quando a duração mudou. É o que impede o minuto narrativo de rebobinar — aos 4:00 de 5:00 o estudante está no minuto 48, e recalcular com 7:00 daria 34.

> **REGRA INVIOLÁVEL**
>
> O gabarito da exp02em1b não está em tabela nenhuma: os oito conjuntos, com o viés (`instrucao`) e a cobertura de cada um, vivem em `src/unoogpt/modelo/conjuntos.ts`, e os parâmetros ocultos do mundo em `modelo/parametros.ts`. Nada disso é persistido, nada disso é servido, e há conferência por `grep` no `dist/` da experiência.

E.5.6 `app_70_sessoes`

Coluna	Tipo	Nota
id	`UUID` PK	**Emitido pela API.** No aplicativo de origem o cliente escolhia a própria chave de idempotência, e duas crianças na mesma máquina colidiam — a segunda partida voltava "duplicada" e desaparecia.
turma_id	`UUID?` FK	**Desnormalizado:** turma *no momento da partida*. O estudante troca de turma entre anos letivos, e o resultado precisa continuar na turma em que foi medido.
`exp_version`	`TEXT`	Versão do *bundle* que jogou — torna o *skew* de *publish* visível em vez de silencioso.
`schema_version`	`SMALLINT`	Versão do contrato de *payload*.
`bolha`	`TEXT`	`A` | `B` | `C`. Atribuída pela **API**, nunca pelo cliente, e copiada aqui porque **o relatório abre pelo NÍVEL**, antes do que ela fez.
`cartoes_servidos`	`SMALLINT`	Quantos a API *entregou* — $\neq$ **quantos apareceram**. Zero $\Rightarrow$ a missão não aconteceu, e o relatório precisa dizer isso em vez de "ela ignorou tudo".
`aberta_em`, `concluida_em`	`TIMESTAMPTZ`	Cravados pelo **servidor**. O aplicativo de origem derivava do relógio do cliente, e ele mesmo documentava que a hora numa máquina de escola costuma estar errada.
`duracao_seg`	`INT?`	Tempo de **jogo** — denominador dos ritmos.
`duracao_total_seg`	`INT?`	Desde a página abrir, incluindo a intro. Uma criança de 6 anos leva vários minutos ali, e sem o total o professor cronometra a atividade inteira e vê um número bem menor no painel.
`status`	`TEXT`	`aberta` | `completa` | `abandonada`. `aberta` que nunca fecha é partida perdida — e saber quantas são é *um dado sobre o jogo*.
`fogueiras_acesas` … `distancia_m`	`SMALLINT/INT`	Resumo **tipado**: o que o painel consulta e ordena. O detalhe fica no *payload*.
`payload`	`JSONB?`	O corpo bruto, íntegro. **Única fonte para recomputar** o perfil quando `VERSAO_MAPEAMENTO` subir.

`leitura_mensagens`	`JSONB?`	A leitura da missão, **persistida**: o relatório sobrevive a qualquer expurgo futuro do *payload* e não precisa recomputar a cada visita.

> **DECISÃO DE PROJETO**
>
> **Não há `expires_at`** (ADR-13). Houve, de 90 dias, e nenhuma rotina que o cumprisse. Coluna de prazo que ninguém lê é pior que prazo nenhum.

E.5.7 `app_70_perfis`

Coluna	Tipo	Nota
`versao_mapeamento`	`SMALLINT`	Hoje **3**, herdada do piloto para os dados serem comparáveis. Sobe a cada mudança de peso, *probe* ou fórmula; **nunca** reescreve uma versão publicada.
`valores`	`JSONB`	Os dez valores em 0–100, e **`null` NÃO É ZERO**: é valor que não recebeu número, porque tem uma fonte de medida só ou porque a oportunidade não chegou.
`graus`	`JSONB`	`scored` | `indicative` | `zero` | `none` por valor. **A distinção mais importante do modelo.**
`abertura_vs_conservac`	`FLOAT?`	Eixo do circumplexo. **`NULL` quando um dos polos não foi observado** — eixo com polo não observado não é zero, é indefinido.
`transcendencia_vs_autopromocao`	`FLOAT?`	Idem.
`amostra_suficiente`	`BOOLEAN`	Falso ⇒ o painel abre avisando que **não sabe**. Não é rigor estatístico: é impedir que uma partida sem escolha vire uma frase sobre quem a criança é.

`UNIQUE (sessao_id, versao_mapeamento)`: quando a hipótese mudar, nasce uma linha nova e as antigas ficam — porque comparar as duas é o único jeito de saber o que a mudança fez.

E.5.8 `app_70_bolhas`

PK composta `(escola_id, estudante_id)`; `bolha`, `atribuida_em`.

> **DECISÃO DE PROJETO**
>
> **Por estudante, não por partida.** Repetir a partida não pode trocar o nível: a folha do professor diz "esse estava na C", e o relatório abre pelo nível. A atribuição é balanceada **dentro da turma** — o aplicativo de origem fazia `nº do estudante mod 3`, que dava terços de graça porque os números eram contíguos de 1 a 40; *hash* de UUID dá terços na média

> da rede e pode dar 14/6/5 numa sala de 25 — e aí o debate da missão não acontece, porque ele depende de as três bolhas estarem na mesma sala.

E.5.9 `app_90_planos` e `app_90_referencias`

Coluna	Tipo	Nota
`status`	`TEXT`	`pendente` | `gerando` | `pronto` | `erro`, com *CHECK*.
`tentativas`	`SMALLINT`	Retomadas de geração *interrompida* (*restart*, *deploy*), não de falha da IA. Máximo 2.
`iniciado_em`	`TIMESTAMPTZ?`	`gerando` há mais de 12 minutos sem *worker* nesta instância = órfã.
`tokens_usados`	`INT`	
`caminho_gerado_md`	`TEXT?`	`planos/{escolaId}/{id}/gerado.md` — Markdown da IA, **imutável**.
`caminho_html`	`TEXT?`	`atual.html` — o que o editor salvou, sanitizado por *allowlist*.

`app_90_referencias` é **catálogo do contrato**: sem `escola_id`, fora da RLS, com `serie NULL` significando "vale para todas as séries da disciplina" (ex.: o currículo da rede). Escrita só pelo *script* de importação.

E.5.10 `app_92_configuracoes`

A **regência**: o que o professor leciona, para quem e com quanta aula. É a entrada de todo o planejamento — sem ela o plano anual distribuiria habilidades em "quatro bimestres" sem saber se cabem 200 aulas de Matemática ou 40 de Arte, e a diferença entre esses dois números é a diferença entre um plano executável e uma lista de intenções.

Coluna	Tipo	Nota
`escola_id`	`TEXT`	RLS forçada
`professor_id`	`UUID`	Posse: o plano é autoral
`ano_letivo`	`SMALLINT`	`CHECK` 2025–2100
`serie`	`TEXT`	`1EF`–`9EF`, `1EM`–`3EM`
`componente`	`TEXT`	*Slug* do componente **da rede** (`biologia` é válido; as habilidades dele vêm da área)
`turma_id`	`UUID?`	Opcional de propósito: em novembro o professor planeja o ano seguinte e a turma ainda não existe
`divisao_do_ano`	`TEXT`	`bimestral` | `trimestral` | `semestral` (`CHECK`)

`minutos_por_aula`	`SMALLINT`	`CHECK` 30–60. **Não** é hora-relógio: é desta coluna que sai a conversão
`aulas_semanais`	`SMALLINT`	`CHECK` 1–20 (o topo real é 10, em Língua Portuguesa dos anos iniciais de São Paulo)
`grade`	`JSONB`	`[{diaDaSemana, horarios[]}]`. JSON porque é configuração de UMA linha, lida inteira e nunca consultada por dentro
`incluir_computacao`	`BOOLEAN`	Sobrepõe a camada transversal (Res. CNE/CEB nº 1/2022). A articulação é **inferência do produto**
`inicio_do_ano_letivo`	`DATE?`	A volta às aulas DESTA regência. Ver a nota abaixo

Restrição	Por quê
`UNIQUE(escola_id, professor_id, ano_letivo, serie, componente)`	Duas turmas do mesmo 7º ano de Matemática **compartilham** o planejamento — é o que o professor faz na prática, e forçar uma cópia por turma multiplicaria o trabalho dele sem acrescentar informação.

ATENÇÃO

`inicio_do_ano_letivo` existe por um bug. `periodos_letivos` é catálogo do CONTRATO e o *seed* o preenche como ano **civil** (01/01–31/12); como o calendário do plano alinha cada período na segunda-feira, o professor que declarava 02/02/2026 recebia um 1º bimestre começando em **29/12/2025** — e a tela pedia a data e a descartava em silêncio. `NULL` significa "usa a sugestão do catálogo", para as regências criadas antes da coluna.

NOTA

Houve uma coluna `periodicidade` (`semanal` | `quinzenal`), o passo do planejamento granular, e ela foi retirada na `20260907120000_exp_92_planejamento_sempre_semanal` (ADR-76). O passo é sempre semanal; entregar de quinze em quinze dias, ou o período inteiro, é recorte de **exportação**. A razão de não ser configurável está em `app_92_planos_de_periodo`: a `ordem` **muda de significado** com o passo — a 3ª quinzena não é a 3ª semana —, então um plano gravado num modo e lido no outro casa pela ordem e devolve as datas de um recorte com as aulas de outro. A migration **apaga** os planos de período das regências que estavam em quinzenal, por isso.

E.5.11 `app_92_planos_anuais`

Coluna	Tipo	Nota
`configuracao_id`	`UUID`	`ON DELETE CASCADE`

`status`	`TEXT`	`pendente` | `gerando` | `pronto` | `erro` (`CHECK`). Mesma máquina de estados da app-90; o *claim* atômico impede duas instâncias de gerar o mesmo plano
`distribuicao`	`JSONB`	`{"1": ["EF67LP08"], ...}` — a **decisão humana** de que habilidade cai em que período de avaliação, e o insumo do gerador
`distribuicao_computac`	`JSONB`	Separada porque é **sugestão**, não currículo do componente: não existe *crosswalk* normativo
`caminho_gerado_md`	`TEXT?`	*Bucket* privado, imutável
`caminho_html`	`TEXT?`	*Bucket* privado, sanitizado
`tentativas`	`SMALLINT`	Retomada de geração interrompida por *restart*, não de falha da IA

> **DECISÃO DE PROJETO**
>
> Por que a distribuição fica no banco e o documento no bucket São coisas de naturezas diferentes. A distribuição é **dado estruturado** e é consultada (o plano de período pergunta "quais habilidades do 2º bimestre?"); o documento é **prosa** de 40–200 KB que só se lê inteira. A separação também protege o professor: ele pode reescrever o texto todo sem que o sistema perca a distribuição — e é ela que o plano de período precisa para ser gerado.

E.5.12 `app_92_planos_de_periodo`

O plano granular de uma semana ou quinzena. Aqui as aulas são **dado estruturado**, e é essa escolha que faz o produto valer: o professor edita a aula de quinta sem tocar na de terça, o sistema sabe quantas aulas foram planejadas contra quantas existem, e o documento impresso nasce do dado em vez de pedir à IA que formate uma tabela.

Coluna	Tipo	Nota
`plano_anual_id`	`UUID`	`ON DELETE CASCADE`
`ordem`	`SMALLINT`	1 = primeira semana/quinzena do ano letivo
`inicio`, `fim`	`DATE`	`CHECK fim >= inicio` — a inversão que um fuso mal convertido produz
`periodo_de_avaliacao`	`SMALLINT`	`CHECK` 1–4. Liga o granular ao macro: é por aqui que a janela herda as habilidades do anual
`aulas`	`JSONB`	Ver a estrutura abaixo

Cada item de `aulas`:

Campo	Origem e nota
`numero`, `data`, `diaDaSemana`, `horario`	**Do calendário**, nunca do modelo
`tema`, `objetivo`, `conteudo`	Da IA, editáveis
`habilidades[]`	Da IA, **filtradas** contra o recorte e contra o catálogo — a terceira barreira contra código inventado
`momentos`	`retomada`, `desenvolvimento`, `sistematizacao`, `fechamento`, cada um com `o_que` e `minutos`. **Não é enfeite**: é o que separa um plano de aula de uma lista de conteúdos, e o que permite a um substituto assumir a turma
`recursos[]`, `avaliacao`, `tarefaDeCasa`, `adaptacoes`	Da IA, editáveis. `avaliacao` é o que o professor OBSERVA nesta aula
`cumprida`, `observacaoDoProfess`	**Nunca da IA**: preenchidos DEPOIS da aula. É o registro do que aconteceu, e é ele que faz o plano deixar de ser intenção

> **NOTA**
>
> `UNIQUE(escola_id, plano_anual_id, ordem)`: uma janela tem um plano. A regeração sobrescreve em vez de acumular, porque duas versões da mesma semana não têm como conviver no caderno do professor.

E.5.13 `exp08ef4b_conteudo_turma`

Coluna	Tipo	Nota
(escola_id, turma_id)	PK composta	O pacote curado é **por turma**: cada professor contextualiza os dilemas com o caso da escola dele.
`caso_contextual`, `foco_prioritario`	`TEXT`	O que o professor escreveu no formulário. Guardado para ele reencontrar o que pediu — não é insumo de nenhuma regra.
`contexto`	`JSONB`	Escola, título do caso e a premissa autoritária com que a máquina abre. *CHECK* `jsonb_typeof = 'object'`.
`prompts_json`	`JSONB`	Os 3 questionamentos **com as réplicas**. *CHECK* `jsonb_array_length = 3`.
`clausulas_json`	`JSONB`	As 4 cláusulas **com os argumentos de mediação**. *CHECK* `= 4`. `JSONB` e não tabelas-filhas: o pacote é lido e escrito SEMPRE inteiro — o professor aprova o conjunto, não um prompt de cada vez.

`aprovado_por_professor`	`BOOLEAN`	**O portão da exigência 8.2.** Falso = rascunho, e rascunho **não chega ao estudante**: a experiência lê o banco curado da Unooria. É o único booleano do produto que separa curadoria de publicação automática.
`usou_banco_default`	`BOOLEAN`	O professor manteve o pacote da Unooria em vez de gerar com IA. A resposta "sim" não é problema: o banco é homologado.

REGRA INVIOLÁVEL

Esta é a única tabela da exp08ef4b que sobrevive ao "Zerar turma". A curadoria custou tempo do professor, e apagá-la junto obrigaria a refazer o preparo para aplicar a missão na turma seguinte no mesmo dia.

E.5.14 `exp08ef4b_sessoes`

Coluna	Tipo	Nota
(escola_id, estudante_id)	PK composta	Uma sessão por estudante e por escola.
`turma_id`	`UUID?`	Denormalizado da identidade: o painel e o gate consultam por turma o tempo todo. Reescrito em toda mutação — uma linha com a turma antiga desapareceria da tela do professor novo.
`passo`	`TEXT`	*CHECK* `IN ('briefing','chat','decisao', 'manifesto','percurso','podcast','fim')`. A ordem é `regras/passos.ts`; *CHECK* e não `enum` porque um `ALTER TYPE` amarraria toda mudança de roteiro a uma migração.
`clausula_id`	`TEXT?`	A salvaguarda fixada — *latch*. *CHECK* contra os quatro ids do catálogo. **Não é gabarito**: as quatro são válidas, e o *CHECK* existe para o id inventado, que produziria uma tela mostrando uma cláusula que o servidor não reconhece.
`latencia_clausula_seg`	`INT?`	Segundos entre a concessão da máquina e a submissão. Cronometrado pelo **cliente**: a latência da API não é o dado. *CHECK* 0–7200 — acima disso é aba esquecida aberta.
`confirmou_provocacao`	`BOOLEAN`	A criança clicou em menos de 5 s e passou pela réplica provocativa. É *indicative* e alimenta a hipótese de exploração mínima — **nunca** avalia ninguém.

`manifesto_markdown`	`TEXT?`	O documento promulgado, compilado no **servidor**. Gravado porque é a EVIDÊNCIA do percurso: o que o professor lê e o que a criança copiou têm de ser o mesmo texto, e recompilar com a *copy* revisada mudaria um documento já assinado. *CHECK*: `(clausula_id IS NULL) = (manifesto_markdown IS NULL)`.
`versao_instrumento`	`TEXT`	`'1.0.0'`. Na linha, para permitir reprocessamento quando o catálogo mudar.
`concluido_em`	`TIMESTAMPTZ?`	*CHECK*: `(passo = 'fim') = (concluido_em IS NOT NULL)`. Uma tela dizendo "missão cumprida" sobre uma linha sem carimbo é um relatório que mente.

REGRA INVIOLÁVEL

Não há coluna `score`, e é a única tabela de sessão do catálogo sem uma. A exp06ef2b tem porque o `exp:concluir` da ponte carrega o campo; aqui a missão **não pontua** — as quatro cláusulas valem igual, e um número na linha seria uma nota que nenhuma tela mostra e que alguém um dia usaria para ordenar crianças.

E.5.15 `exp08ef4b_auditorias`

Coluna	Tipo	Nota
id	`BIGSERIAL`	Exige `GRANT USAGE` na **sequência** para a *role* de runtime — a quarta aparição da armadilha.
`prompt_id`	`TEXT`	Id do catálogo. `UNIQUE (escola_id, estudante_id, prompt_id)`: reenviar o mesmo questionamento não é auditoria nova, e o índice faz a dedupe valer mesmo se um caminho novo esquecer de checar.
`iteracao`	`INT`	A ordem do disparo na conversa — 1 e 2 são os obrigatórios, 3 é o extra. É por ela que a conversa é reconstituída depois de um F5. *CHECK* ≥ 1: zero seria uma fala fora da linha do tempo.
`latencia_seg`	`INT`	Calibragem de fluxo, **não** medida de mérito: ninguém é avaliado por ler rápido. *CHECK* 0–3600.

E.5.16 `exp08ef4b_respostas_percurso`

Coluna	Tipo	Nota
(escola_id, estudante_id)	PK composta	Um percurso por estudante.

`palavras_chave_tags`	`TEXT[]`	Exatamente 2, **e distintas**: *CHECK* `cardinality = 2`, `tags[1] <> tags[2]` e `<@ ARRAY['autonomia','transparencia','justica','privacidade']`. Repetir deixaria um *slot* logicamente vazio dentro de um *array* cheio, e a nuvem da turma contaria a mesma criança duas vezes na mesma palavra. São ids sem acento — identificador, não texto de tela (ADR-91).
`justificativa_texto`	`TEXT`	**O parágrafo autoral: caixa livre escrita por menor de idade.** *CHECK* `char_length(btrim(...)) BETWEEN 20 AND 800` — o `btrim` cobre o piso, que o tipo não cobre: vinte espaços contariam como percurso entregue no painel. Está em `CHAVES_REDIGIDAS` nos dois nomes (`justificativaTexto` e `justificativa_texto`), porque a comparação do *logger* é por nome INTEIRO e `texto` não os cobria.
`chars_count`	`INT`	O tamanho, para a leitura agregada não precisar tocar o texto. *CHECK* 20–800.

REGRA INVIOLÁVEL

Sem coluna de moderação, e a ausência é a decisão: o texto com dado pessoal é recusado com **422** e não chega a esta tabela. Não há registro moderado a marcar — e um dado que não foi gravado não vaza em *backup*, em *dump* de diagnóstico nem em *log*. **Retenção:** vinculada à vigência contratual escolar, **sem rotina ativa de expurgo programado** (ADR-13). A frase é literal e a escolha é deliberada: não existe *scheduler* de expurgo na plataforma, e declarar "um ano letivo" aqui seria escrever um prazo que ninguém cumpre.

E.5.17 `exp08ef4b_gates_turma`

Coluna	Tipo	Nota
(escola_id, turma_id)	PK composta	O Checkpoint A é da **turma**.
`etapa`	`TEXT`	*CHECK* `IN ('missao','debate')`. É **esta linha** que o *polling* de 5 s lê, porque não há WebSocket de aplicação no Cloud Run (ADR-93).

NOTA

Duas colunas, e o gate é **opcional**: a missão corre inteira sem ninguém apertar nada. Por isso `debate` mexe só no TETO do passo do estudante, nunca no piso — subir o piso empurraria para a frente quem ainda estava auditando a máquina, e são justamente essas crianças que o debate tenta alcançar.

E.5.18 exp05ef1b_pacotes

Coluna	Tipo	Nota
id	UUID PK	**É a atividade da evidência** (ADR-118): o atividade_id de evidencias_marcas. Alvo, com escola_id, da FK composta das sessões (UNIQUE (escola_id, id)).
turma_id	UUID	O pacote é **por turma**: a professora escreve a missão em cima do assunto da escola dela. FK em cascata.
disciplina	TEXT	*CHECK* com os dez componentes curriculares de catalogo.ts#DISCIPLINAS (letramento-digital …ensino-religioso).
tema	TEXT	*CHECK*: btrim entre 3 e 60 caracteres e até 8 palavras. **É a frase da Nina** ("Rolou uma confusão sobre {tema}"), e o Zod ainda recusa "atividade", "gabarito", "nota" e "exercício".
assuntos	TEXT[]	Os fatos balizadores, de 3 a 6. *CHECK* assuntos IS NOT NULL AND cardinality BETWEEN 3 AND 6 — o IS NOT NULL é explícito porque cardinality(NULL) é NULL, que um *CHECK* deixa passar.
contexto	TEXT?	Opcional, até 300.
mensagens_json	JSONB	As 8 mensagens **com** veracidade e tom — é aqui que mora o gabarito. *CHECK* jsonb_typeof = 'array' e jsonb_array_length = 8; a matriz e o equilíbrio de formato são do Zod.
origem	TEXT	*CHECK* IN ('ia', 'padrao'). padrao é o banco curado, e não falha registrada.
ativo	BOOLEAN	O pacote que a turma usa. *CHECK*: só com aprovado_em. Índice único parcial (escola_id, turma_id) WHERE ativo.
rodada_atual	SMALLINT	A reaplicação: "Zerar" soma 1, e as sessões das rodadas anteriores ficam. *CHECK* >= 1, e rascunho sempre em 1.
aprovado_em, aprovado_por	TIMESTAMPTZ?, UUID?	Nulos juntos (*CHECK*) — nulo é rascunho, e há um rascunho por turma (índice único parcial WHERE aprovado_em IS NULL). O autor é carimbo, sem FK.

E.5.19 exp05ef1b_gates_turma

Coluna	Tipo	Nota
(escola_id, turma_id)	PK composta	A revelação é da **turma**.

`fase`	`TEXT`	*CHECK* `IN ('triagem', 'revelacao', 'explicar', 'reflexao')`. Só anda para a frente (`avancarFase`), e volta a `triagem` ao aprovar um pacote e ao zerar. É **esta linha** que o *polling* de 3 s do estudante lê (ADR-93).
`atualizado_em`	`TIMESTAMPTZ`	Quando a professora moveu a turma.

E.5.20 `exp05ef1b_sessoes`

Coluna	Tipo	Nota
(escola_id, estudante_id, pacote_id, rodada)	PK composta	Uma sessão por estudante, por pacote e por **rodada**: zerar a turma abre outra linha, e esta fica.
`turma_id`	`UUID`	Desnormalizado do pacote: o painel lê por turma o tempo todo.
`entrou_em`	`TIMESTAMPTZ`	O toque em "Entrar no grupo" — a linha nasce aqui, e não ao abrir a missão.
`selecao`	`TEXT[]`	De 0 a 3 ids do pacote. *CHECK*: não nulo, no máximo três, e `<@ ARRAY['msg_1' …'msg_8']`. Antes do *latch* é rascunho; depois, o Mural.
`confirmado_em`	`TIMESTAMPTZ?`	**O latch 1**, e o gesto que a EF05CO08 mede. *CHECK*: exige `cardinality(selecao) = 3`.
`tempo_feed_segundos`	`INT?`	Segundos entre entrar e confirmar. Existe exatamente quando o Mural fechou, e nunca é negativo.
`motivo`, `votou_em`	`TEXT?`, `TIMESTAMPTZ?`	**O latch 2**. *CHECK* `IN ('urgencia', 'confianca_remetente', 'medo_indignacao', 'imagem_link_real')`; os dois nulos juntos, e o voto exige `confirmado_em`.
`reflexao_em`	`TIMESTAMPTZ?`	**O latch 3**: a regra foi enviada (o texto está em `exp05ef1b_reflexoes`). Exige `votou_em`.
`concluido_em`	`TIMESTAMPTZ?`	Exige `reflexao_em` — e não as respostas às perguntas: quem sai antes delas fica "Não avaliado".
`versao_instrumento`	`TEXT`	`'2.2.0'`. Na linha, para reprocessar o histórico se o instrumento mudar.

> **REGRA INVIOLÁVEL**
>
> **Não há coluna de passo, nem de pontuação.** O passo é derivado dos carimbos e da fase da turma (`passoEfetivo`); uma coluna seria a terceira fonte de uma verdade que os carimbos já dizem. E o nível da criança não é coluna daqui: é marca da evidência pedagógica (Seção 8.6.2), lida pelo professor no painel de habilidades.

E.5.21 `exp05ef1b_reflexoes`

Coluna	Tipo	Nota
(escola_id, estudante_id, pacote_id, rodada)	PK composta	1:1 com a sessão (FK composta em cascata).
`condicional_se`	`TEXT`	O "Se": *CHECK* `IN ('urgente_alarmante', 'parente_proximo', 'link_nao_checado', 'promessa_incrivel')`.
`acao_entao`	`VARCHAR(240)`	**Texto livre escrito por menor de idade.** *CHECK* `char_length(btrim(acao_entao)) BETWEEN 15 AND 240`. Fora de *log* (redação por nome de chave), fora da telemetria, e preenchido no painel só para quem leciona na turma. **Não é corrigido**: a EF05CO04 vem da pergunta P2.
`chars_count`	`INT`	O tamanho, para a leitura agregada não tocar o texto. *CHECK* 15–240.

REGRA INVIOLÁVEL

Sem criptografia em nível de aplicação, como nas outras missões: o Cloud SQL cifra em repouso, e a proteção é a RLS mais o recorte do *service*. **Retenção:** a da vigência contratual, sem rotina de expurgo (**ADR-13**).

E.5.22 `app_91_ciclos` e `app_91_checkins`

Coluna	Tipo	Nota
`ordem_itens`	`INT[]`	Permutação dos itens 1..21 sorteada na abertura do ciclo. Cada *check-in* leva 3 itens, um de cada subescala.
`expira_em`	`TIMESTAMPTZ`	21 dias. Ciclo não fechado vira `expirado`, sem pontuação.
`depressao`, `ansiedade`, `estresse`	`INT?`	Já $\times 2$ (escala do DASS-42), preenchidas **ao concluir** o sétimo *check-in*, no servidor.
`numero`	`SMALLINT`	1..7 dentro do ciclo.
`dia`	`TEXT`	`YYYY-MM-DD` no fuso `America/Sao_Paulo`. `UNIQUE (escola_id, professor_id, dia)`: um por dia.
`itens`	`INT[]`	Os 3 itens deste *check-in*.
`respostas`	`JSONB?`	`[{ item, valor }]` $\times 3$, valor 0..3.
`texto_livre`	`TEXT?`	**Nunca vai a log nem à telemetria.** Só contagens.
`abertura`, `pergunta`, `acolhimento`	`TEXT`	Os textos de entorno que o professor *viu* — guardados, não regerados.

`origem_texto`	`TEXT`	`ia` | `fallback` (IA indisponível ou barrada pelo *guardrail*) | `seguranca` (sinal de apoio — texto fixo, a IA nem é chamada).
`sinal_de_apoio`	`BOOLEAN`	Item 17 ou 21 ≥ 2, ou termo de risco no texto.
`tokens_ia`	`INT`	

REGRA INVIOLÁVEL

Dado sensível de saúde (LGPD art. 11). RLS por `escola_id` **mais** posse por `professor_id` em toda *query*; **nenhuma** rota de gestor ou agregado; e o próprio titular apaga tudo com `DELETE /bemestar/dados`.

E.6 O que o Prisma não modela

ATENÇÃO

Ler apenas `schema.prisma` dá uma visão **incompleta** do banco. Vivem *só* na migração:

- políticas de **RLS** (`ENABLE`, `FORCE`, `CREATE POLICY`);
- ***CHECK* constraints** (papel *versus* `escola_id`, cardinalidade dos salvos, vocabulário de `permissoes`, coerência de conclusão, XOR de ancoragem de comentário);
- **partições** e o *schema* `particoes`;
- **`GRANT`** para a *role* de aplicação.

Os campos correspondentes são `TEXT`/`SMALLINT` no *schema*, e a validação de aplicação é feita com Zod.

Apêndice F

Operação

F.1 Comandos de desenvolvimento

Listing F.1 – Subir a plataforma

```
# --- Tudo em container, sincronizado com o editor (RECOMENDADO)
pnpm docker:dev          # pg + api + hub + as 6 experiencias, cada um com watcher
pnpm docker:dev:logs     # acompanha a recompilacao
pnpm docker:dev:deps     # DEPOIS de mexer em dependencia; e reiniciar
pnpm docker:dev:down     # desce

# --- So o banco em container, o resto no host
docker compose up -d pg
cp .env.example apps/api/.env && cp .env.example apps/hub/.env.local   # e editar
node scripts/gerar-chave-token.mjs --env      # chave RS256 do hub
pnpm --filter @edu/api migrate:dev            # migrations (Prisma + RLS)
pnpm --filter @edu/api seed                   # escola-demo, prof/gestor/estudante01-40
pnpm --filter @edu/api seed:demo              # atividade DEMO com 92 posts, sem IA
pnpm dev                                      # hub :3000, api :3001, exp :5173

# --- Zerar o banco e recomecar do zero absoluto (DEV)
pnpm --filter @edu/api exec prisma migrate reset --force --skip-seed
SEED_MINIMO=1 pnpm --filter @edu/api seed      # escola + gestor.demo + admin, nada mais

# --- Imagens congeladas (mais proximo da producao)
docker compose up -d --build                  # migrations e seed no 1o up
```

> **ATENÇÃO**
>
> As portas são as mesmas nos três modos, então **não dá para rodar `pnpm docker:dev` e `pnpm dev` ao mesmo tempo**. Se salvar um arquivo e nada recompilar: `DEV_POLLING=1 pnpm docker:dev`.

Listing F.2 – Experiências, uma a uma

```
pnpm --filter @edu/exp08ef1b    dev   # :5173
pnpm --filter @edu/exp04ef1b       dev   # :5174 (videos em public/videos)
pnpm --filter @edu/app-90-plano-de-aula  dev   # :5175 -> /professor/plano-de-aula
pnpm --filter @edu/app-91-bem-estar      dev   # :5176 -> /professor/bem-estar
pnpm --filter @edu/app-70-vila-do-lume   dev   # :5177 -> /exp/app-70-vila-do-lume

```

```
pnpm new:exp exp06ef3b                      # scaffolda do _template (ADR-110)
```

Listing F.3 – O IdP sobe à parte

```
docker compose -f wso2/docker-compose.yml up -d    # IS :9443, pg dele :5433
docker compose -f wso2/docker-compose.yml watch    # olha conf/ e bootstrap/
docker compose -f wso2/docker-compose.yml logs bootstrap   # imprime as envs OIDC
```

Listing F.4 – Verificação e build

```
pnpm turbo typecheck lint test              # o portao da Definition of Done
pnpm turbo build --filter=@edu/exp08ef1b       # 1 experiencia (+ deps)
pnpm turbo build --filter='...[HEAD^]'      # so o que mudou (o CI usa isso)
pnpm build:isolado                          # build de PRODUCAO do hub
                                            # (HUB_DIST_DIR=.next-prod)
pnpm --filter @edu/api test
```

ATENÇÃO

Validar *build* de produção **sempre** com `pnpm build:isolado`. Um `next build` comum apaga o `.next` do *dev*, e o sintoma é "tela branca" ou "CSS bugado" depois — que se confunde com bug de aplicação. E `apps/hub/next-env.d.ts` alterna entre `./.next` e `./.next-prod`: modificado depois de um `build:isolado` = **descartar, não *commitar***.

F.2 Testes de ponta a ponta

```
node scripts/e2e/rodar.mjs              # tudo, na ordem canonica
node scripts/e2e/rodar.mjs professor    # so um papel
node scripts/e2e/rodar.mjs estudante/02 # so uma suite
node scripts/e2e/rodar.mjs --deck       # tudo + deck (.pptx) + videos (.webm)
node scripts/probe-pointerlock.mjs
```

A suíte vive em `scripts/e2e/`, organizada por PAPEL — `publico`, `estudante`, `professor`, `gestor` e `admin-contrato` —, e as evidências saem em `testes/<papel>/<suíte>/`: os *prints*, um `passos.json` e um `relatorio.md`.

NOTA — a ordem dos papéis não é alfabética

O `publico` não depende de nada e falha rápido se o *hub* estiver fora — é o canário mais barato. O `estudante` PRODUZ o estado (*runs*, partidas, sessões) que as telas do `professor` leem: invertê-los deixaria as telas de resultado vazias e as asserções de conteúdo sem objeto. E `gestor`/`admin-contrato` vêm por último porque importam pessoas e trocam senhas — efeitos que atrapalhariam as contas usadas pelos roteiros de cima.
Cada suíte roda em processo PRÓPRIO. É mais lento que um processo só, e é deliberado: uma suíte que trava o navegador não leva as outras junto.

NOTA — o manifesto é a fonte do deck e dos vídeos

Cada `registrar()` escreve, junto com o *print*, o que a tela MOSTRA e a regra do projeto que ela PROVA. É desse `passos.json` que nascem os slides e as legendas dos vídeos. Antes existia um roteiro de 66 KB mantido à mão, separado dos scripts, e ele divergiu: descrevia telas que já não eram fotografadas e omitia passos novos. A descrição passou a morar onde o passo mora.

NOTA — os roteiros de DOIS contextos

As missões conduzidas pelo professor (`estudante/02-instanooria.mjs` e `estudante/04-unoogpt.mjs`) sobem DOIS navegadores: um com o docente no relógio da turma, outro com o estudante no aparelho. Nada avança sem o botão do professor — um contexto só não chega ao fim, e passaria com o relógio quebrado.
O da exp08ef1b mede a `boundingBox` do *iframe* antes e depois do "Ver meu relatório" ($375 \rightarrow 1280 \rightarrow 349$px), escreve uma marca dentro do *frame* para provar que ele NÃO foi remontado, e lê o `backgroundColor` COMPUTADO — `toBeVisible()` não pega nenhuma das três.
Duas armadilhas medidas ao escrevê-lo. O *frame* precisa vir do próprio elemento (`elementHandle().contentFrame()`): procurar por `"exp08ef1b"` na lista de *frames* devolve o frame DO HUB, cuja rota é `/exp/exp08ef1b` — e aí a verificação de "não recarregou" fica verde por construção, medindo a página errada. E a medição precisa esperar um elemento do FIM do relatório: o conteúdo vem de `React.lazy`, e medir logo após o título pega a tela montando, o que sai como fundo transparente e zero gráfico.

NOTA — erro provocado não é erro descoberto

Quando um roteiro exercita uma trava de propósito — ativar uma atividade com conteúdo de menos (422), subir um perfil portátil adulterado (400), abrir a análise de produto com o papel errado (403) — o erro que aparece é a PROVA de que a trava existe. Ele é declarado com `r.provocado(padrão, motivo)` e sai numa seção própria do relatório, em vez de pintar de vermelho a corrida que estava justamente demonstrando o comportamento.

Pré-requisito	Detalhe
Browsers do Playwright	`pnpm exec playwright install chromium`, uma vez por máquina — o pacote está nas `devDependencies` desde a fundação do monorepo, mas os navegadores ficam FORA dele, num cache do usuário. **Não** `npx playwright install`: o `npx` resolve o *playwright* mais NOVO do npm e baixa a *build* de navegador DELE, deixando `node_modules` e cache em versões diferentes.
Plataforma no ar	Contêineres **ou** `pnpm dev` — um *ou* outro, nunca os dois na mesma porta: o navegador e o servidor passam a ler *dev servers* diferentes.
Chrome do **sistema**	Obrigatório para a suíte da app-70 e para o *probe* de *pointer lock*: o Chromium do Playwright não concede *pointer lock* nem no topo da página.

Duas escolas no *seed*	`escola-demo` (8A, 8B, 4A, 2A) e `escola-demo-2` (5A), com `prof.demo` lotado nas DUAS — é o que faz aparecer o seletor de escola no cabeçalho dele. `gestor.vilanova` administra a segunda.
Logins do *seed*	`gestor.demo` → `/admin/usuarios`; `prof.demo` → `/professor`; `estudante01`–`estudante40` → `/estudante`; `estudante4a01` para a exp04ef1b. Senha de todos: `unooria123`. Eles nascem com `senha_provisoria = false` de propósito: prendê-los na troca do primeiro acesso pararia as suítes de estudante na mesma tela. `prof.demo` é matriculado nas quatro turmas — sem isso, as telas dele abrem vazias.
Bancada de *embed*	`/dev-embed?src=http://localhost:5174`. Sanidade do *design system*: `/ds-smoke` (só em *dev*).

F.3 *Design system*

```
npm view @unooria/design-system version         # qual e a ultima publicada
pnpm up -r @unooria/design-system --latest      # sobe em ui/hub/exps de uma vez
                                                # (^0.1.x NAO sobe minor)
cd ../storybook-unooria && npm run storybook     # :6007, o catalogo das 6 secoes
```

ATENÇÃO

Commitar o *lockfile* **e** os `package.json` juntos. E lembrar: **quem importa, declara** — todo aplicativo que escreve `@unooria/design-system` no código tem a dependência no próprio `package.json`, hub incluído.

F.4 Publicação e reversão

Listing F.5 – Reversão — as três formas, por velocidade

```
# 1. EXPERIENCIA (o mais rapido: sem build, sem imagem, sem deploy)
node apps/hub/scripts/publicar-manifesto.mjs exp04ef1b --versao 0.9.0
#    Requer: TOKEN_SIGNING_KEY_PEM, API_URL, HUB_ORIGIN

# 2. HUB ou API (revisao do Cloud Run)
gcloud run services update-traffic "$HUB_SERVICE" \
  --region "$REGION" --project "$GCP_PROJECT" --to-revisions "<revisao>=100"

# 3. CONTEUDO DE PROFESSOR (versionamento de objetos do bucket de planos)
gcloud storage ls --all-versions \
  "gs://$PLANOS_BUCKET/planos/$ESCOLA/$PLANO/atual.html"
```

Mudou uma experiência? Suba a `versao` do `public/experiencia.json`. O caminho no bucket é `{expId}/{versao}/` e o `rsync` usa `--delete-unmatched-destination-objects`: publicar sem subir a versão sobrescreve um caminho que o próprio *pipeline* trata como imutável

(`Cache-Control` longo em tudo menos o `index.html`). Quem está no meio de uma sessão tem os *assets* daquela versão em cache e vai pedir arquivos que acabaram de ser apagados. Suba a versão e o problema não existe: caminho novo, nada a sobrescrever, e a versão anterior continua no bucket — que é o que faz a reversão nº 1 acima ser possível.

Listing F.6 – Erro de cliente: da linha de log ao fonte

```
# 1. A linha ERROR do Error Reporting (servico = expId, versao = a do manifesto):
gcloud logging read 'jsonPayload.codigo="erro-cliente"' --project "$GCP_PROJECT" \
  --limit 1 --format='value(jsonPayload.expId,jsonPayload.versao,jsonPayload.stack_trace)'
# 2. A stack e minificada. O mapa da versao exata esta no bucket PRIVADO:
EXP_MAPAS_BUCKET=$(terraform -chdir="$INFRA_DIR" output -raw bucket_exp_mapas) \
  node scripts/mapear-stack.mjs exp04ef1b 1.2.3 < stack.txt
```

Listing F.7 – Publicar tudo (primeiro *deploy* de um contrato)

```
# GitHub Actions -> workflow "deploy" -> Run workflow -> forcar: true
# Ignora o diff e publica hub, api e TODAS as experiencias.
```

F.5 Provisionamento de um contrato novo

1. No repositório de Terraform: instanciar o *module* `contrato` com o novo prefixo. **Nada é criado por *script* deste repositório.**
2. `terraform output` → configurar as `vars` do repositório (`scripts/configurar-github-ci.sh`).
3. Gerar o par RS256 e gravar em `{contrato}-token-signing-key` no Secret Manager; espelhar no *secret* `TOKEN_SIGNING_KEY_PEM` do GitHub.
4. Conceder `roles/aiplatform.user` à conta de serviço `{contrato}-api`. **Sem esse papel, toda geração falha com 403 do Vertex** — e o 403 vem do Vertex, o que se confunde com quota esgotada.
5. Verificar que a *role* de banco **não** tem `BYPASSRLS` nem `SUPERUSER` (SQL de verificação no *runbook* do repositório de infra).
6. Executar o *deploy* com `forcar: true`.
7. Importar usuários por CSV (`POST /usuarios/importar-csv`).
8. Importar as referências curriculares: `node scripts/importar-referencias-plano.mjs` com `PLANOS_IMPORT_TOKEN`.
9. **Manual, e pendente de automação:** `gsutil rsync` da mídia pesada da exp04ef1b para o *bucket* de *assets* (`R-09`).
10. Rodar `node scripts/e2e/rodar.mjs` contra o ambiente novo.

F.6 Diagnóstico: sintoma → causa

Esta tabela existe porque, nesta plataforma, o sintoma quase nunca aponta para a causa.

Sintoma	Causa provável
A ficha de uma missão mostra “Nova atividade” e não tem “Abrir a aula e os resultados”	A missão não tem ramo no DESPACHANTE de `app/(plataforma)/professor/entre-redes/[ano]/[unidade]/page.tsx`, e caiu no `return` final — que é o corpo do **Instanooria**. O modo de falhar é MUDO: a tela abre, e mostra a ficha de OUTRA missão. Duas consequências caras: a professora não acha o painel da turma (o único caminho para congelar o debate e ver os resultados), e o botão “Nova atividade” a leva a configurar uma linha em `exp08ef1b_activities` que a missão dela nunca lê. Aconteceu com a exp06ef2b e de novo com a exp08ef4b (esta em 2026-09-11), com o aviso já escrito no arquivo ao lado. A rede é `apps/hub/lib/entre-redes/despachante-da-ficha.test.ts`, que reprova missão publicada sem ramo.
O botão do painel da professora não faz nada, e a rede mostra 404 numa rota que existe na api	O prefixo da experiência não está em `PREFIXOS_PERMITIDOS` do BFF (`apps/hub/app/api/plataforma/[...path]/route.ts`). Toda tela *client-side* do professor passa por ali, e prefixo fora da lista devolve **404** — não 403, não 405 — com uma linha `codigo: bff-prefixo-recusado` no log do hub. O sintoma engana porque a rota EXISTE na api e responde bem por `apiDoServidor`: a tela de resultados carrega (é *Server Component*) e só os botões morrem. Método que falta no BFF, por contraste, dá **405**. Visto na exp08ef4b em 2026-09-11: a preparação, os dois botões do debate e o “zerar turma” respondiam 404 em silêncio.
Um *rollback* de versão “desfez-se” sozinho depois de um *deploy*	Ele é temporário por natureza: o *job* `ativar-exps` faz `PUT /experiencias/{expId}` a cada publicação, e aquele PUT grava a versão do repositório. O painel avisa (`emRollback`) justamente porque esse estado tem prazo. Se o *rollback* precisa sobreviver ao *deploy*, a resposta não é reativá-lo depois: é a experiência não entrar no filtro do `ativar-exps` (não mexer na pasta dela) ou fixar a versão POR ESCOPO, que vive em `disponibilidades` e o CI não toca (**ADR-87**).
Fixei a versão de uma escola e ela continua abrindo a versão da rede	Confira em ordem: a linha (`SELECT recurso_id, escola_id, versao_fixa FROM disponibilidades`), o escopo (a fixação vale para a escola e o que está ABAIXO dela, não para a irmã), e `CDN_EXP_URL` — em desenvolvimento o hub serve do *dev server* do Vite e NÃO existe caminho versionado, então a fixação não tem efeito visível ali por desenho. Em produção, o `montarEmbed` lê `GET /recursos/efetivos` para montar a URL: se aquela leitura falhar, ele cai na `versao_ativa` de propósito — deixar de montar a experiência porque uma leitura acessória caiu seria pior.

Sintoma	Causa provável
A missão desapareceu da grade da turma, e ninguém liberou nem encerrou nada	O RECURSO foi desligado para esta rede (ADR-87). O sintoma é **indistinguível** de um defeito de enturmamento ou de catálogo, e é por isso que ele está nesta tabela. A missão indisponível SAI da grade (não fica com cadeado), e a linha em `liberacoes` continua intacta — religar devolve tudo. Confira em ordem: `SELECT * FROM disponibilidades` (linha com `escola_id` nulo vale para a rede INTEIRA), depois `experiencias.produto` da missão (nulo = ela responde só pelo próprio interruptor, sem teto de produto). No log, o 403 sai com `codigo: recurso-indisponivel` — se o código for `missao-nao-liberada`, o problema é da SALA e não do painel.
Card de produto sumiu da home do professor, ou a rota dele dá 404	Mesmo eixo do item acima, um nível acima: o PRODUTO está desligado no escopo que vale (escola, regional ou contrato). A tela some por decisão — ela não explica que o produto existe e foi desligado, porque isso contaria ao cliente o que a Unooria vendeu a quem. Confira `disponibilidades` com `recurso_id = 'produto:<slug>'`. Se a linha NÃO existir, é a api que não respondeu: a leitura falha ABERTA, e nesse caso a home mostra tudo — card faltando com a tabela vazia é outra coisa.
Barra de rolagem parada na tela, sem ter o que rolar	A página transborda por 1–2px. Quase sempre é uma altura calculada a partir da altura de um irmão: a tela cheia do professor subtraía `64px` do `100dvh`, e o `HeaderPlataforma` mede **65** — o `h-16` são 64 e a `border-b` põe 1. Medir com `documentElement.scrollHeight - clientHeight`: 1px é subtração errada, não conteúdo. A saída é a classe `.tela-cheia`, que deixa o *flex* distribuir em vez de estimar (ADR-75). E **não conclua pelo seu Mac**: com a preferência do sistema em automático e *trackpad*, a barra *overlay* nativa desaparece sozinha em qualquer contêiner não estilizado — e esconde o que Windows e Chromebook mostram.
O cabeçalho da tabela deixou de grudar ao rolar	A `Table` do DS traz o próprio `.unooria-table-container` com `overflow-x: auto`. Dentro do `ScrollBar` ele vira um SEGUNDO contêiner de rolagem, e é nele que o `<th>` *sticky* se prende — um contêiner que não rola na vertical. A regra `.tabela-rolavel .unooria-table-container { overflow: visible }` devolve o papel ao `ScrollBar` (ADR-75). Conferir com `getComputedStyle(th).position`: ele responde `sticky` nos dois casos — o que muda é QUEM é o contêiner.
Ícone virando texto (“addInvestigação”)	O Next removeu o `@import` externo do CSS do *design system*, e `Material Symbols Rounded` é fonte de ligadura. Verificar `FontesDaPlataforma` em `casca-plataforma.tsx`. O `getComputedStyle` *responde* “Open Sans”, então não confie nele.
O professor não vê a turma dele	Falta **matrícula**, não falta permissão. Desde o enturmamento (ADR-28) “as turmas do professor” são as turmas em que ele tem linha em `matriculas`; o caminho é `/admin/turmas/:id` → “Adicionar professor”, ou a coluna `turma` na linha dele da planilha.

Sintoma	Causa provável
A lista de pessoas mostra todo mundo "sem turma"	Consulta que lê `matriculas` **fora** de `comTenancy`: a RLS devolve zero linhas, sem erro nenhum. É a armadilha registrada na migração do enturmamento.
A pessoa troca a senha e volta para a tela de trocar a senha	Sessão antiga: o *cookie* ainda diz `precisaTrocarSenha`. O fluxo **refaz** o *login* com a senha nova justamente por isso (**ADR-29**); se ele falhar, a saída é `/login` — a senha JÁ foi trocada.
Importei e ninguém consegue entrar	A senha do lote não foi anotada. Ela é devolvida **uma vez**, na folha da tela de importação. O caminho de volta é `POST /usuarios/:id/reset-senha`, uma pessoa por vez.
Reimportei a planilha e nasceu gente duplicada	A coluna `matricula` estava vazia. Sem o RA, a chave de reconciliação cai no `usuario`, e um *login* corrigido vira uma segunda pessoa — com o histórico preso na primeira.
"Sem credencial do Google Cloud" — e o token funciona quando testado no container	O `GoogleAuth` é resolvido no **import** do módulo (`ai/vertex.provider.ts`), uma vez por processo. Se o arquivo do ADC estava sendo reescrito naquele instante (o `gcloud` renova o *refresh token* sozinho), a api fica presa num cliente quebrado até o fim da vida do processo, e um teste manual no mesmo contêiner passa. Conserto: reiniciar a api.
"Unique constraint failed on `cpf`" na importação	Comportamento correto: o índice único de CPF é da REDE, não da escola. Duas pessoas com o mesmo CPF não podem existir no contrato — e se a intenção era transferência, o caminho é importar na escola de DESTINO, que o reconhece. Em paralelo (duas importações simultâneas) a linha volta como erro dela, sem derrubar o lote.
Toda criança transferida virou cadastro novo	A planilha veio **sem** a coluna `cpf`. Sem ela, a chave de reconciliação cai na matrícula da rede — que muda de escola para escola — e a mesma criança nasce duas vezes.
O professor não vê a segunda escola no cabeçalho	Falta **lotação** nela. O seletor só aparece com duas ou mais: com uma, seria um campo de uma opção. O caminho é `/admin/escolas` → a escola → lotar, ou a planilha com o CPF dele importada na segunda unidade.
Lotei a supervisora na diretoria e ela continua sem ver as escolas	Estar numa regional não dá acesso a nada — a hierarquia agrupa, quem autoriza é a lotação (**ADR-34**). O caminho é `/admin/escolas` → filtrar pela regional → marcar as unidades → "Lotar pessoa nestas escolas".
Lotei em lote e o relatório diz menos escolas do que marquei	Havia `regional` ou `rede` na seleção, e elas não recebem lotação: não têm turma, estudante nem dado, e virariam escola fantasma no seletor do cabeçalho. A barra de seleção da tela já avisa quantas serão ignoradas.
O `orgs.csv` entrou mas as escolas ficaram sem regional	`parentSourcedId` apontou para uma organização que não veio no arquivo, ou veio DEPOIS. Isso é *aviso*, não erro: a escola entra sem regional, porque perdê-la por causa do agrupamento seria trocar o essencial pelo acessório. Reimportar com o pai presente corrige.

Sintoma	Causa provável
“Sem CPF não há perfil portátil”	É a regra, não um erro: numa rede de destino nenhum id daqui existe, e o CPF é a única chave que reconhece a criança. Preencha o CPF em `/admin/usuarios` e emita de novo.
Todo perfil importado entra “não verificado”	`PORTABILIDADE_EMISSORES` vazia, ou a origem do documento não está nela. Sem emissores configurados a api não busca JWKS nenhum, e marcar visivelmente é melhor que recusar. **Em contêiner, a entrada precisa apontar o JWKS**: `http://localhost:3000=http://hub:3000/.well-known/jwks.json` — o `emissor.hub` do documento é o endereço do navegador, e de dentro da api `localhost:3000` é ela mesma. É o *default* do `docker-compose.dev.yml`.
“em produção o JWKS só é buscado por https”	A entrada da allowlist aponta para `http`. Fora de produção isso é aceito (com aviso no *log*); em produção, não — chave pública por canal reescrevível não prova nada.
A emissão responde 503	Falta `CONTRATO_ID` no hub. Documento cujo emissor não se identifica não vale nada no destino, que precisa dele para achar o JWKS e para dizer de quem veio.
“Não há estudante com este CPF nesta rede”	O documento não cria pessoa. Matricule a criança primeiro (planilha ou OneRoster) e importe o perfil depois — a matrícula é ato da escola.
O item “Minhas escolas” não aparece na navegação	A pessoa tem UMA lotação. O panorama é a tela de quem administra várias unidades; com uma escola ele seria uma linha só, e item de menu que não leva a nada é pior que item nenhum.
O admin do contrato abre uma tela de escola e ela vem vazia	Falta `?escolaId=` na URL. O *token* dele não tem escola (é a exceção cross-escola), então a tela precisa dizer qual — e o caminho que já leva o parâmetro é `/admin/escolas`, pelos botões “Turmas” e “Pessoas”.
“Nenhum período letivo cadastrado” ao importar ou criar turma	Banco novo, ou virada de ano sem o período criado. Turma vive DENTRO de um período letivo: o caminho é `/admin/periodos-letivos`. O *seed* cria o do ano corrente; um ambiente novo sem *seed* não tem nenhum.
Turma nova nasceu no ano errado	`vigente` é derivado das DATAS. Se nenhum período contém hoje, a plataforma cai no mais recente para não travar — e a tela do ano letivo avisa isso em faixa. Corrija as datas em vez de criar outro período.
O OneRoster recusa a linha dizendo que a turma é desconhecida	Ordem dos arquivos. `enrollments.csv` referencia `classSourcedId`, que vem de `classes.csv`, que referencia `termSourcedIds`, que vêm de `academicSessions.csv`. O relatório nomeia o arquivo que falta.
Todo mundo importado pelo OneRoster virou estudante	Na 1.2 o papel não está em `users.csv` — foi para `roles.csv`. Sem a coluna `role`, a pessoa nasce com o menor privilégio de propósito; aplique `roles.csv` para corrigir.
Mandei `bulk` e ninguém foi desativado	É o comportamento. Ausência de linha num lote completo não é ordem de exclusão: o relatório conta e lista os que não vieram, e a desativação é ato do gestor.

Sintoma	Causa provável
Elemento branco no branco	Classe de cor que não existe em `plataforma.css`. Não é erro em lugar nenhum. Checar o `backgroundColor` **computado**.
Experiência esticada na janela em vez de dentro do aparelho	`viewport` não lido: tabela `experiencias` vazia e `EXP_INTERNAL_HOST` ausente ou errado.
O UnooTube aparece dentro da casca de celular da Instanooria	`EXP_INTERNAL_URL` fixo em *dev*: as experiências estão lendo o manifesto de *uma só*. Usar `EXP_INTERNAL_HOST`, que preserva a porta.
`iframe` vazio	*Handshake* expirado, *dev server* fora do ar, ou *origin* divergente. Ver o console: a SDK avisa o descarte por esquema.
"Baixar .doc" não baixa nada, sem erro na tela e sem linha no *console*	`allow-downloads` ausente no *sandbox* do `iframe`. Confira em ORDEM: (1) o `permissoes` do `public/experiencia.json` tem `downloads`; (2) a **linha** de `experiencias` tem também — em *dev* o hub lê a tabela quando ela existe, não o manifesto do *bundle*; (3) a rota do hub passa `permissoes` ao `ExpFrame` (as três de professor não passavam). O bloqueio é do navegador e é MUDO: o Chrome o registra no painel *Issues*, não no *console*. Verificação de um comando, no console do hub: `document.querySelector('iframe').getAttribute('sandbox')`.
Câmera do jogo não anda *desde o começo*	`allow-pointer-lock` ausente: `permissoes` não chegou (manifesto vazio e `experiencia.json` não lido). `requestPointerLock()` morre com `SecurityError`. Desde 2026-08-27 o hub **avisa** no log (`embed-experiencia.ts`) nomeando URL, causa e consequência — antes o `catch {}` engolia, e uma configuração errada virava bug de jogo. O *sandbox* do `iframe` é a evidência: sem o *token*, o suíte da app-70 falha no primeiro passo.
A câmera congela *depois* de já ter andado — ao apertar Esc ou trocar de aba — **e o aviso "clique pra voltar" não aparece**	O `pointerlockchange` não foi registrado. No jogo de origem os `<script>` ficavam no fim do `index.html` e o `load` disparava depois deles; no porte o `carregador.ts` injeta os *scripts* depois do *handshake* com o hub, quando o `load` já passou. Como o `pausedHint` é o **único** `controls.lock()` fora dos diálogos, a partida ficava sem caminho de volta. Corrigido em `16-boot.js` (2026-08-27); coberto pelo passo *pausa-e-volta* da suíte da app-70.
Dois *dev servers* da mesma experiência na mesma porta	Um no *host* e um no *compose*. O navegador e o servidor podem alcançar **servidores diferentes** (`localhost` resolve para um, `host.docker.internal` para o outro), e o mais antigo responde com o `vite.config.ts` que ele tinha ao subir — daí um `403 Blocked request` no `allowedHosts` que *está* no arquivo. `pnpm docker:dev` e `pnpm dev` são um **ou** outro.
Vídeo não toca (`play()` rejeitado em silêncio)	`autoplay` não delegado no `allow` do *frame*: a *Permissions-Policy* tem *default* `'self'`.

Sintoma	Causa provável
O vídeo para no primeiro toque *dentro* da experiência e não volta — e a suíte acusa "autoplay BLOQUEADO"	**Não é o `allow` do *frame***, e é aí que o sintoma engana: o `play()` manual funciona e o `readyState` é 4. Clicar dentro do `iframe` tira o foco da janela do hub, o `ExpFrame.tsx` traduzia esse `blur` em `hub:pause` e o `focus` correspondente nunca voltava enquanto o estudante continuasse ali. A evidência é `visivel: false` em `lib/visibilidade.ts`. Corrigido em 2026-08-27: `blur` com `document.activeElement` igual ao próprio `iframe` não é perda de visibilidade nenhuma.
`confirm()` da experiência não aparece	`allow-modals` ausente no *sandbox*: o navegador **ignora** e devolve `false`.
Toda geração de imagem sai como *fallback* procedural	`VERTEX_IMAGE_MODEL` aponta para um modelo que saiu do catálogo. Verificar no Vertex.
Arte cortada pela metade	Proporção não pedida no `imageConfig`: o modelo devolve 1408×768 e o `cover` de 1080×1350 descarta a largura. Pedir `4:5`.
Plano de aula volta vazio com `finishReason: MAX_TOKENS`	`VERTEX_THINKING_BUDGET` diferente de `0`: os *tokens* de pensamento saem do `maxOutputTokens`.
Toda chamada de IA falha com 403	A conta de serviço não tem `roles/aiplatform.user`. O 403 vem do *Vertex*, não da API — e se confunde com quota esgotada.
IA não funciona no *host* nem no contêiner	ADC ausente. `gcloud auth application-default login`; os *compose* montam o arquivo em `/adc`.
API não sobe, reclamando de `VERTEX_PROJECT_ID`	Correto: com `AI_PROVIDER=vertex` a variável é obrigatória. Em *dev*, use `AI_PROVIDER=mock`.
`/telemetry/ingest` responde 401	Em *dev*, correto (`TELEMETRIA_PUSH_*` vazias = rota fechada). Em produção, conferir SA e *audience* contra a *subscription*.
Validação de *token* falha com "*issuer* inválido"	`iss` montado por concatenação. O do WSO2 é *tenant-qualified* e deve ser copiado **literalmente** do *discovery*.
Um bloco de *feed* desapareceu; posts repetidos	*Lock* consultivo não tomado. Verificar `pg_advisory_xact_lock(runId)` e o `timeout` de 20 s.
`comTenancy sem escola`	Rota sem `OidcGuard`/`TenancyInterceptor`, ou chamada fora de requisição sem `escolaId` explícito. **É o comportamento correto** — a alternativa era consultar sem filtro.
A sala inteira caiu na bolha A	*Lock* consultivo por turma não tomado em `POST /vila/sessoes`. A sala clica junto.
Partida da app-70 recusada	`.strict()` pegou chave desconhecida, ou a varredura achou chave de nome, ou o *payload* trouxe perfil. **Recusa é o comportamento correto**; chave nova legítima precisa entrar no contrato *de forma consciente*.
Dependência nova não aparece no contêiner	`node_modules` do contêiner é volume próprio. `pnpm docker:dev:deps` + reiniciar.

Sintoma	Causa provável
Vite recusa a conexão do hub em contêiner	`allowedHosts` sem `host.docker.internal`. O Vite 5.4+ recusa `Host` que não conheça.
Vite subiu numa porta diferente	Segunda instância da mesma experiência. O hub passa a ler o manifesto da experiência errada.
***Link* sublinhado, marcador de lista de volta**	*Preflight* do Tailwind desligado. O pacote estiliza *componentes*, não elementos.
Sobrescrita de classe do *design system* não pega	Peso igual: o CSS do pacote é injetado *depois*. Usar **dois** seletores (`.app-shell > .x`).
`exp08ef1b_exp08ef1b_...` depois de uma migração de renomeação	`FOR ... IN SELECT` sobre o catálogo releu o que o laço alterou, ou `RENAME CONSTRAINT` renomeou também o índice. Ver a migração de referência.
Na exp02em1b, o estudante digita a pergunta, aperta Enter e NADA acontece — a conversa parece viva, porque a fala de abertura do agente está na tela	Falta `allow-forms` no *sandbox* do `iframe`: o `Composer` do *design system* é um `<form>`. O manifesto tem de declarar `"permissoes": ["forms"]`. O único rastro é um aviso de console sobre "*form's frame is sandboxed*" — e *typecheck* e *lint* passam.
Tela BRANCA dentro do `iframe`, sem erro de console	Uma rota de mutação devolveu a sessão *crua* em vez do envelope `{ sessao }`; o cliente leu `undefined` e derrubou a árvore do React. Toda rota de mutação da exp02em1b devolve o envelope, e há teste fixando isso.
Na exp02em1b, "o Cientista não conseguiu responder" em toda tentativa	Credencial do Vertex ausente ou `AI_PROVIDER=vertex` sem ADC montado. A API traduz falha do provedor em `cientista-indisponivel` (503) de propósito — em desenvolvimento, use `AI_PROVIDER=mock`. Confira também `tokens_gastos` da sessão: teto estourado dá a mesma cara para o estudante, com outra frase.
Na exp02em1b, a turma inteira vê o cronômetro parado e ninguém consegue falar	**A largada não foi dada.** O relógio é da turma (**ADR-25**), e sai de `/professor/entre-redes/2-serie/unidade-1/resultados`. É o motivo nº 1 de "a missão não abre", e por isso a ficha do professor abre com a faixa avisando.
"Sem credencial do Google Cloud para falar com o Vertex", com o ADC montado e legível	A credencial de usuário precisa de **reautenticação periódica** (política da organização). O erro real, visível só ao pedir o token à mão, é `invalid_grant` com `error_subtype: invalid_rapt` — *RAPT* é *Reauth Proof Token*. **A janela é de 1 hora** (MEDIDO: login 22h20, expirado 23h31), ou seja, ela vence NO MEIO de uma sessão de trabalho e a suíte de ponta a ponta perde dois roteiros sem que uma linha de código tenha mudado. Antes de investigar "regressão" nos caminhos de IA, peça o token. Resolve com `gcloud auth application-default login` no HOST. Não é permissão nem *mount*: confira nessa ordem (existe? legível? token?), porque as três falham com a mesma mensagem na API. O `gcloud` reescreve o arquivo *no lugar*, então o *bind mount* do container acompanha sem recriar — mas confirme comparando o `inode`.

Sintoma	Causa provável
Teste ou script ignora o valor que você passou ao `ConfigService`	O `ConfigService` do Nest lê `process.env` com **prioridade** sobre o objeto do construtor. Num container com a variável definida, o valor do código não vale. Foi o que fez uma medição de custo bater na quota de 20.000 com 5.000.000 declarados no script.
Dependência nova não chega no container; a API não sobe depois de `prisma migrate`	O `node_modules` **e o Prisma Client** dos containers são próprios. `pnpm docker:dev:deps`, ou `prisma generate` dentro do container, e **recriar** o serviço — `restart` não relê o `compose`, então variável de ambiente nova (como `CORS_ORIGINS`) não entra.
Snyk verde no PR, `SNYK-0005` `Authentication error` só no *push* para `main`	Workflow **chamado** não herda *secrets*: no PR o `ci.yml` roda direto e recebe `SNYK_TOKEN`; chamado pelo `deploy.yml`, precisa de `secrets: inherit` no *job* `verificar`. O passo falha alto de propósito — token ausente nunca vira portão aberto.
`vite build` morre com "Transforming destructuring … is not supported yet" em todas as experiências	`esbuild` fora da faixa que o Vite pina (um *override* de segurança, tipicamente): o 0.28 não faz mais o *lowering* para os *targets default* do Vite 6 (`chrome87/es2020`). Reverter o *override*; a vulnerabilidade fica em `.snyk` com prazo até o Vite pinar o esbuild corrigido.
Build da imagem morre em `corepack: not found` (exit 127) depois de subir a base `node:NN-slim`	A partir do Node 25 a imagem oficial **não traz mais o corepack**. Os três *Dockerfiles* instalam o pnpm pelo npm, pinado na versão do `packageManager`. O sintoma engana porque o `docker compose up` segue usando a imagem **antiga** do cache sem avisar — os containers continuam no Node velho e só o `node --version` lá dentro denuncia.
Dois números diferentes para os posts que o algoritmo escondeu — a criança lê "32 posts que nunca chegaram até você" e o relatório do professor, sobre ela, diz "escondeu 30"	Dossiê **congelado** numa versão anterior à **ADR-42**. As duas contas já vêm de `dossie/exposicao.ts`, mas o dossiê é gravado no `finish` e só é recomputado na leitura quando `versaoDoRelatorio` está atrás de `VERSAO_DO_RELATORIO`. Se os números seguirem divergindo com a versão em dia, o caso é outro: alguém desaprovou posts entre o encerramento da investigação e a tela dos perdidos — a matriz é viva.
O envolvimento da turma cai sozinho conforme a aula avança (52,7 → 42,8 → 38,7 → 35,1 no painel ao vivo)	**Não é a turma desengajando.** O *buffer* de eventos da experiência vive enquanto a *run* vive, e cada `hub:pause` durante o debate, a explicação ou a reflexão ainda registra `perdeu_foco`. Se a janela do tempo não terminar no último `visto` (`fimDoFeedDe`, **ADR-44**), `feedMs` cresce depois de o feed fechar e `ativoMs / feedMs` derrete. Conferir a versão do relatório antes de olhar para o estudante.
O aparelho do estudante recarrega e ele volta para a revelação	Alguém tirou o `children` de dentro do `DispositivoFrame` em vez de ligar a *prop* `expandido`. Trocar a árvore desmonta o `<iframe>`; o modo expandido é ESTILO, e só (**ADR-41**).

Sintoma	Causa provável
TODA chamada de IA morre com `fetch failed` — e o `getAccessToken()` à mão devolve token	**Não é credencial**: é o `Agent` do pacote `undici` passado ao `fetch` GLOBAL, que usa o undici EMBUTIDO na *runtime* — outro pacote, outra versão (Node 26.7 traz a 8.x; a api declarava `^6`). A v6 não satisfaz mais a interface de *handler* da v8. A razão real, `invalid onError method`, vive só em `error.cause`, e o `catch` logava só `message` — por isso passou dias parecendo ADC vencido, que era um problema DE VERDADE por cima. Conferir com `node -p "process.versions.undici"` contra `require("undici/package.json").version`. Resolvido pela **ADR-45**: `Agent` e `fetch` do MESMO pacote.
Reautenticou o ADC, o token novo funciona, e a api continua dizendo "Sem credencial"	O processo cacheia a credencial: `new GoogleAuth()` é criado UMA vez, no escopo do módulo (`vertex.provider.ts`), e segura o *refresh token* lido na subida. `gcloud auth application-default login` reescreve o arquivo, mas quem já subiu não relê. **Reautenticar não basta: reinicie a api.** Comparar `docker inspect -f '{{.State.StartedAt}}'` com o `mtime` do ADC responde na hora.
A suíte acusa "turma não está no seletor (rode o seed)" — e a turma ESTÁ no banco, com o professor matriculado nela	O roteiro pegava `locator("select").first()`, e quem tem mais de uma lotação (`prof.demo` tem duas) ganhou um seletor de ESCOLA no cabeçalho, que vem ANTES no DOM. Ele procurava "2A" entre nomes de escola. **A mensagem culpava o banco por uma mudança de *header*** — e o seed é o primeiro lugar onde ninguém acha nada. Usar `select#turma`, como os outros roteiros já faziam. Corrigido em 2026-08-29; era o único `select` cru em `scripts/`.
Toda suíte morre em `Executable doesn't exist` — com o cache de navegadores CHEIO	O executável que falta é o `chromium_headless_shell-1234`, e alguém rodou `npx playwright install` em vez de `pnpm exec playwright install`. O `npx` ignora o `node_modules` do repo, resolve o *playwright* mais novo do npm e baixa a *build* de navegador DELE (a 1237); o repo fixa a 1.62.1, que procura a 1234. **O sintoma mente duas vezes**: o cache tem navegador de sobra, e o banner do próprio Playwright manda rodar exatamente o `npx` que causou o descasamento. Conferir com `pnpm exec playwright --version` e reinstalar por ali.
O relatório do estudante abre em tela cheia no meio do FEED	Alguém inverteu a condição do `ExpHost` para expandir por *default*. O `IntersectionObserver` do *dwell* se refere ao *iframe*: numa área larga o sinal central do motor infla em silêncio, e nada na tela denuncia. A condição é positiva — `fase === "explicabilidade"` E `passo === "relatorio"` —, tudo o mais é aparelho.
O seed morre com "SEED_SENHA não definida"	Não é regressão: a senha das contas de demonstração saiu do fonte (**ADR-47**). Nos dois *compose* ela já vem; no *host*, ponha `SEED_SENHA` no `apps/api/.env` (o Prisma carrega esse arquivo) ou prefixe o comando.
A experiência lança "hubOrigin não é quem embedou"	O `?hubOrigin=` da URL não bate com `location.ancestorOrigins[0]` (**ADR-46**). Em desenvolvimento, quase sempre é `HUB_ORIGIN` do hub apontando para outro endereço — `host.docker.internal` onde o navegador vê `localhost`, por exemplo. **Antes desta trava o sintoma era outro**: *handshake* expirando em 8 s sem dizer por quê.

Sintoma	Causa provável
O CSV grande é recusado — e a mensagem diz QUAL dos três tetos foi	São três, em ordem (**ADR-48**). "CSV acima de 30MB — divida o arquivo": o nosso, e é o esperado. `PayloadTooLarge` do Express: o *bodyParser* de `main.ts` ficou menor que o `TETO_DO_ARQUIVO` da rota — os dois saem da mesma constante, e o `mb` do *parser* é MEBIbyte justamente para ficar acima. 413 SEM mensagem nossa e sem linha nenhuma no log: é o Cloud Run, que recusa acima de 32 MiB em HTTP/1 antes do contêiner — só acontece se alguém chamar a rota fora da tela. Caminho, nos três: dividir o `enrollments.csv` por escola ou por diretoria.
O *bootstrap* do WSO2 falha em "TLS com identity-server:9443"	Ele agora PINA o certificado do IS e falha se não conseguir lê-lo, onde antes desligava a verificação de TLS do processo e seguia (**ADR-47**). O IS ainda está subindo, ou a porta 9443 não responde — o *healthcheck* do *compose* deveria ter segurado; conferir `docker compose -f wso2/docker-compose.yml ps`.
O painel da atividade lista estudantes que não são meus	Se a atividade tem turma, é bug — abrir um chamado. Se ela NÃO tem (`exp08ef1b_activities.turma_id IS NULL`), é o comportamento das atividades anteriores à **ADR-49**: elas valem para a escola inteira, e o painel diz isso numa faixa azul. Criar a atividade de novo, agora escolhendo a turma. **Antes da ADR-49 o sintoma era pior e mudo**: a consulta rodava sem `escola_id` numa tabela FORA da RLS, então apareciam crianças de OUTRAS escolas do contrato.
Os perfis da Vila não mudam de lado depois de subir a `VERSAO_MAPEAMENTO`	Esperado: o que a constante nova recalcula é `app_70_perfis`, não a cópia em `exp08ef1b_app_70_perfis_importados`, que é a que decide o lado do feed. O painel do Ato 1 mostra quantos vieram de versão anterior; "Trazer perfis da Vila do Lume" atualiza, e NÃO desfaz quem o professor moveu de lado (**ADR-50**). Não há rotina automática, de propósito.
O estudante entra e vê "Nada rolando por aqui agora"	Com **ADR-49**, `POST /runs` só entrega atividade SEM turma ou da turma DELE — e a turma sai de `users.turma_id`, o cache da matrícula principal. Estudante sem `turma_id` (importado sem enturmar, ou movido pela metade) só alcança atividade sem turma. Conferir em `/admin/turmas`.
A planilha volta com "papel não reconhecido"	A coluna `papel` aceita `estudante`, `professor` e `gestor`, mais os sinônimos de `SINONIMOS_DE_PAPEL` (`discente`, `docente`, `coordenacao`, `diretor`… e o `student`/`teacher` do OneRoster). Fora disso a linha vira relatório, não erro mudo. O modelo para download (`GET /rostering/modelos/usuarios.csv`) sai da MESMA constante que o parser lê.
`Protocolo incompatível: hub vN, experiência vM` no console	O *bundle* servido tem `PROTOCOL_VERSION` de outro *major* que o hub. É a falha correta — alta e legível, em vez do *handshake* expirando em 8s — e o conserto é republicar a experiência (`deploy.yml`, trabalho `publicar-exps`). Apontar o hub para uma versão anterior no CDN NÃO resolve quando é justamente a versão anterior que está atrás do protocolo: *rollback* só funciona entre versões do mesmo *major*.

Sintoma	Causa provável
O *deploy* sobe os *bundles*, fica verde em `publicar-exps` e a experiência continua na versão antiga	Olhe o trabalho `ativar-exps`: é ele que escreve `versaoAtiva`, e o *bundle* no *bucket* é inerte até esse *PUT*. A falha que produz exatamente este quadro é `ERR_MODULE_NOT_FOUND: Cannot find package 'jose'` — o trabalho pulava o `pnpm install` sob a justificativa de que o `publicar-manifesto.mjs` era "Node puro", e ele assina o *token* de `admin-contrato` com `SignJWT` do `jose`. Apareceu no primeiro *deploy* completo de uma *stack* nova, porque antes o caminho nunca havia sido exercitado. O sintoma engana duas vezes: o trabalho que *parece* responsável (`publicar-exps`) está verde, e nada no ar explica a versão velha.
`P1013: invalid port number in database URL` no túnel, com a porta certa na string	A porta está certa; o que quebra o *parse* é a SENHA. O `db_password` do Terraform é aleatório e pode conter `@`, `:` ou `/` — e aí o `@` da senha vira o separador de *host*, e o resto da senha vira "porta". Não é bug do Prisma nem da instância: **percent-encode a senha** antes de montar a URL. Passa despercebido de ambiente para ambiente porque uma senha sorteada sem esses caracteres funciona — em `dev` funcionou, em `hml` não. O Cloud Run não sofre disso: lá a string é montada pelo Terraform, não à mão.
`ativar-exps` falha numa *stack* nova — de DUAS formas diferentes, e a segunda mente	Causa única nas duas: **certificado ainda `PROVISIONING`**. Este é o único trabalho que fala HTTPS por **hostname público** (o *PUT* do manifesto); os outros usam `gcloud`. Qual cara aparece depende de QUAL certificado falta: ■ Falta o da **api** → o TLS quebra no *runner*: `fetch failed` → "`socket disconnected before secure TLS`". Aponta para a causa. ■ Falta o do **hub** → `401 {"message":"Token inválido"}`. **Este engana**: parece chave errada ou *secret* trocado, e não é — a api valida o *token* contra o JWKS que o HUB serve, e é ELA que toma o `ECONNRESET` ao buscá-lo. A prova está no *log* da api, não no do CI: `[TokenService] Token rejeitado: ECONNRESET ...`. Confirmar com `gcloud certificate-manager certificates list`; se os CNAMEs `_acme-challenge` já batem com `dns-authorizations list`, **não há nada a consertar**: é tempo de emissão. Repetir o *dispatch* com `forcar` quando virarem `ACTIVE`. O estado intermediário é seguro por desenho — os *bundles* já estão no *bucket* e são inertes até a ativação.

Sintoma	Causa provável
Feed da exp04ef1b em "Carregando o vídeo…" para sempre, e Salvar/Curtir desabilitados junto	**O *bucket* de *assets* do contrato está sem os 21 *mp4*** — o *upload* da mídia pesada é deliberado e não acontece no CI (**ADR-63**). Confirmar com `gcloud storage ls gs://{contrato}-assets/exp04ef1b/videos/` e no *log* do *load balancer*, onde os `GET` do `/exp04ef1b/videos/vNN.mp4` aparecem em 404. **Não é o *bundle* nem a variável**: se a URL pedida traz o domínio de *assets* e o caminho certo, a configuração está correta e o que falta é o arquivo. O sintoma engana porque o `ShortsPlayer` não escuta `error` — o estado de carregamento nunca é desfeito, e a criança lê como internet lenta. Se a URL pedida NÃO traz o domínio de *assets*, aí sim é a var `ASSETS_BASE_URL` vazia no *environment*: interpolada vazia ela produz um caminho relativo válido, e o *fallback* de dev nunca dispara.
Controle dentro de um *modal* "sai fora" — o `<select>` fecha sozinho, o clique parece cancelado	**Handler instável passado ao `Modal` do *design system***, combinado com algo que renderiza o pai periodicamente. O `Modal` das seções `chatgpt` e `unootube` mantém `onClose` nas dependências do *mesmo* `useEffect` que faz o foco: identidade nova ⇒ *cleanup* devolve o foco para fora do *modal* e o efeito o joga no primeiro focável, que é o botão de fechar. Perder o foco fecha um `<select>` nativo aberto. Não há rastro no servidor — as requisições que saem voltam 201. Procurar arrow *inline* em `onClose`/`onFechar` e *polling* ou cronômetro que renderize mais de uma vez por segundo. O `Modal` da seção `unooria` já separa os dois efeitos e é o desenho a copiar.
Tela de resultados do professor vazia, sem erro em lugar nenhum	**`POST /telemetry/ingest` bloqueado pelo Cloud Armor** antes de chegar à api (**ADR-62**) — por isso não há *log* de aplicação. Confirmar no *log* do *load balancer*: `jsonPayload.statusDetails="body_denied_by_security_policy"`, com `enforcedSecurityPolicy.preconfiguredExprIds` nomeando a assinatura. O Pub/Sub reentrega para sempre (não há DLQ), então o volume de 403 cresce sozinho e não indica ataque nenhum: o agente é `APIs-Google`.
A coluna diz "1 professor" e a lista da mesma escola diz "Ninguém por aqui ainda"	Dois lados perguntando coisas diferentes com o mesmo rótulo. A coluna conta LOTAÇÕES naquela escola; a lista filtrava `users.escola_id`, que é o CACHE da lotação PRINCIPAL. Quem trabalha em duas escolas é contado nas duas e aparecia só na principal. **"Quem trabalha nesta escola" é sempre `lotacoes`** — e a regressão a proteger é a IGUALDADE entre contagem e lista, não a *query* de um dos lados. Corrigido em `usuarios.service.ts`; o mesmo cache já havia produzido o defeito da busca de profissionais (**ADR-80**).
O item de menu aparece e a tela abre VAZIA, sem erro e sem linha de log	A tela e o menu leem fontes diferentes. Aconteceu com o panorama do `gestor_regional`: o menu conta as escolas da SESSÃO (que vêm da regional) e o painel lia `lotacoes` direto — e ele não tem lotação nenhuma. Quando uma tela é habilitada por uma fonte e alimentada por outra, o estado impossível é "habilitada e vazia". As duas passam por `escolasDaPessoa` (**ADR-81**).

Sintoma	Causa provável
A suíte de ponta a ponta roda "tudo" e o número de roteiros caiu	O `rodar.mjs` monta o caminho a partir do array `PAPEIS` e usa `existsSync`: pasta que não existe devolve lista VAZIA, e ele PULA aquele papel em silêncio. Renomear o vocabulário sem renomear as pastas derrubou a suíte de 29 para 19 roteiros sem um aviso (**ADR-80**). **Confira sempre a contagem do cabeçalho** (`== suíte … · N roteiro(s) ==`): é o único lugar onde a ausência aparece.
Toda tela do hub responde 401 no BFF, mas o login funciona	O login é chamada direta à API (usuário e senha); as telas passam pelo BFF, que assina um *token* com a chave do HUB e a API o valida contra `OIDC_ISSUER`. Se o hub subiu numa porta diferente da que o `OIDC_ISSUER` aponta, a API busca o JWKS de OUTRO hub e recusa tudo. E o sintoma engana duas vezes: o login passa (não usa *token*), e um processo antigo que sobreviveu ao `pkill` continua servindo a porta com a configuração velha — `lsof -nP -iTCP:<porta> -sTCP:LISTEN` diz QUEM está lá, e o `EADDRINUSE` no *log* do processo novo é a confirmação.
A imagem gerada sai, e a voz responde `403 PERMISSION_DENIED`	ADC de **usuário** sem projeto de cota. A síntese (`generateSpeech`, que nenhuma missão chama hoje) não vai ao `aiplatform`: vai à Cloud Text-to-Speech, que exige o cabeçalho `x-goog-user-project`. O *provider* o manda quando `VERTEX_PROJECT_ID` está definida — confira a variável antes de suspeitar de papel IAM. Com *service account* (produção) o sintoma não existe. Se o 403 falar em API desabilitada, é outra coisa: `texttospeech.googleapis.com` precisa estar habilitada no projeto.
O roteiro de ponta a ponta clica no `Radio` e nada é marcado	O `<input>` das seções do *design system* é `sr-only` de 1×1 px: o Playwright acha o elemento, clica na coordenada dele e o navegador entrega o clique a quem está por cima. Clique no `<label>` (`.unooria-radio`, `.unooria-checkbox`), com `scrollIntoViewIfNeeded` antes. O sintoma engana porque não há erro: o passo seguinte é que reprova, "botão desabilitado".
O professor relata um erro e manda um código	É o `requestId` do corpo do 500 (`{codigo: "erro-interno", requestId}`), igual ao *header* `x-request-id` de toda resposta e ao `traceId` do *load balancer*. No Logs Explorer: `jsonPayload.requestId="<código>"` traz a linha `ERROR` com `stack_trace`, escola e perfil; o *request log* do Cloud Run com o mesmo `trace` vem agrupado ao lado. Sem código na tela (rota antiga, 4xx), filtre por `jsonPayload.codigo` — todo `logger.*` da api carrega um.

Sintoma	Causa provável
500 no *boot* de uma experiência, e o `stack_trace` diz `25P02 current transaction is aborted`	Alguma coisa tentou se recuperar de um erro **dentro da transação em que o erro aconteceu**. No Postgres o primeiro erro aborta a transação inteira: todo comando seguinte responde `25P02`, e o que aparece no log é a consulta de recuperação, não a que quebrou. O caso real foi o `bemestar.service.ts`: o `create` do check-in do dia batia no `UNIQUE (escola, professor, dia)` numa corrida de *boot*, e o `catch` relia a linha da outra requisição pela MESMA `comTenancy`. O caminho escrito para salvar a corrida era o que a virava 500. A releitura depois de conflito de chave única vai numa transação NOVA. Procure por `catch` que consulta usando o mesmo `tx`.
Uma suíte do Jest reprova com "Test suite failed to run" e `TypeError: Cannot read properties of undefined (reading '$disconnect')`	**O erro reportado não é a causa**: é o *teardown* substituindo o erro do `beforeAll` no relatório. O cliente do Prisma era criado no FIM do `beforeAll` e desconectado no `afterAll`; falhando o *setup* antes daquela linha, a variável fica `undefined` e o `afterAll` estoura — e é o `TypeError` dele que o Jest imprime, no lugar do que quebrou de verdade. O sintoma é intermitente (só aparece quando o *setup* falha) e aponta para o arquivo certo pela razão errada. **Cliente de banco nasce no MÓDULO, com `const`**, e a limpeza do `afterAll` vai *best-effort* (`.catch(() => undefined)`): falha de limpeza também substitui o erro do teste. Para conferir que o mascaramento saiu, sem esperar a intermitência: ponha um `throw` na primeira linha do `beforeAll` e veja se o relatório mostra a SUA mensagem.
Uma suíte do Jest passa sozinha e reprova junto com as outras	**Procure primeiro a *fixture* que escreve em tabela SEM RLS** — `experiencias`, `experiencias_versoes`, `disponibilidades`, `periodos_letivos`, `recursos_auditoria`. Ali a linha não é estado da suíte, é estado GLOBAL, e o efeito atravessa arquivos: uma disponibilidade em escopo-contrato desliga o produto para todo mundo; uma experiência de teste com `publico: "estudante"` entra na grade de toda turma; `upsert` com `update` completo REESCREVE o catálogo que outra suíte esperava. Regra que sai disso: *fixture* de catálogo escreve o MÍNIMO no `update`, usa id próprio (`exp-99-...`) e nunca deleta linha compartilhada. Depois de consertados os casos localizáveis, a cauda que sobra é a CLASSE do problema (um banco, tabelas compartilhadas, 40+ arquivos escrevendo juntos): a suíte da API roda com `maxWorkers: 1` por decisão medida (**ADR-90**), e **reprovação intermitente que volte com o serial ligado é defeito NOVO**, não a cauda antiga. Para medir em vez de adivinhar: rode a suíte 30 vezes num laço, redirecionando com `> arquivo 2>&1` — o Jest escreve em `stderr`, e `2>&1 > arquivo` manda o relatório para o terminal e grava um arquivo vazio (esta captura errada já fez uma caça parecer limpa).

Sintoma	Causa provável
A suíte passa na MÁQUINA e reprova no CI	**Procure a tabela de contrato que a suíte LÊ e não escreve** — `experiencias` antes de todas. O banco de desenvolvimento está semeado; o do CI é descartável e nasce só com o schema, então "o produto desligado fecha a porta" reprova por não haver experiência declarando pertencer ao produto. O sintoma MENTE: `expect(codigo)` recebendo `null` lê-se como guarda quebrada, e a guarda está certa. É o espelho do sintoma acima — lá a *fixture* escrita vaza, aqui a leitura depende de quem semeou — e as duas raízes são a mesma: tabela de contrato não isola por suíte (**ADR-90**). O catálogo real virou pré-condição declarada (`globalSetup` em `apps/api/jest.config.cjs`); **para reproduzir antes de empurrar**, suba um Postgres descartável (`docker run -e POSTGRES_PASSWORD=pass -e POSTGRES_DB=unooria -p 55433:5432 postgres:16`), rode `migrate:deploy` contra ele e a suíte inteira — é exatamente o que o CI faz, e é o único jeito de ver este vermelho sem esperar o *push*.
O painel de recursos mudou o estado e o histórico não mostra a linha	**Isto é por desenho, e o log diz**: procure `codigo=recurso-auditoria-nao-gravada` no Cloud Logging. O registro do histórico NUNCA derruba a operação que o chamou (**ADR-89**) — o interruptor da rede falhando porque o histórico dele falhou seria pior de todos os jeitos —, então a mudança vale e o histórico fica incompleto, em silêncio na tela. Aquele contador é o alarme. Se o log não tem a linha, a mudança que você procura foi em OUTRO escopo: o histórico de uma escola inclui as decisões de escopo-contrato, mas o do contrato não desce até as das escolas.
O histórico da escola mostra uma mudança que ninguém fez naquela escola	**Também por desenho.** Filtrar por escola inclui as linhas de escopo-contrato (`escola_id IS NULL`): foram elas que decidiram o que vale ali. Sem isso, o histórico de uma escola apareceria VAZIO no dia em que a rede inteira mudou — e é justamente o dia em que alguém vai olhar. A coluna "escopo" diz "a rede inteira" nessas linhas.
A suíte e2e reprova a geração, e o BANCO diz que ela ficou `pronto` — na mesma corrida	A espera estava vigiando o alerta ANTIGO. Quando o botão que dispara a ação fica DENTRO do alerta de erro anterior (o "Tentar de novo" da app-92 é o caso), o `esperarOuFalharCedo` seguinte vê aquele alerta e aborta antes de a ação começar. Confirme pelo banco: `SELECT ordem, status, tentativas, atualizado_em FROM app_92_planos_de_periodo ORDER BY atualizado_em DESC` — `tentativas` incrementado e `status = pronto` com carimbo DA corrida significa que o produto funcionou e o roteiro mentiu. É o pior falso negativo que existe, porque manda investigar o que está certo. O conserto é esperar o estado `gerando` com `waitFor` puro antes de armar a vigilância.

Sintoma	Causa provável
A suíte falha ao clicar num item de lista, e o *call log* termina em `<div id="unooria-collapse-…">`	O item está dentro de um `Collapse` FECHADO. O corpo fechado fica no DOM (o pacote esconde por altura e `opacity`, não por `display`), então o *locator* acha o elemento, o Playwright o considera visível e o clique morre interceptado pelo contêiner. Abra os *collapses* antes de procurar — o cabeçalho tem `role="button"` e `aria-expanded`. O mesmo desenho é a lacuna de acessibilidade da Seção 14: aqueles controles também recebem foco do teclado.
Voltei a turma do Instanooria para a investigação e a tela dos estudantes não mudou	Confira primeiro o `movidos` da resposta de `POST /atividades/{id}/etapa`: `voltou: true` com `movidos: 0` é o retrato exato deste problema, e o painel fica autocontraditório no mesmo cartão — "agora Investigação" acima de "N respondendo". **Duas causas, e só uma é defeito.** Se as *runs* estão em `explicabilidade` ou adiante, é POR DESENHO: quem já viu a revelação não volta ao *feed*, para na resposta, porque quem sabe que o *feed* era enviesado não pode ser medido por ele de novo (**ADR-92**). Fora disso, a versão da API é anterior à **ADR-92**: até 2026-09-10 sair da investigação congelava a sessão e voltar NUNCA reabria, o que tornava o retorno um *no-op* para a turma inteira. Confirme com `SELECT fase, finished_at FROM exp08ef1b_runs WHERE activity_id = ...`: fase `conclusao` com `finished_at` preenchido e a atividade em `investigacao` é a assinatura do defeito.
Apertei "Pausar para Debate" e a tela do estudante não congelou	**Espere 5 segundos antes de investigar.** O congelamento da exp08ef4b não é *push*: é uma linha em `exp08ef4b_gates_turma` que a tela lê por *polling* de 5 s (**ADR-93**), e não há WebSocket de aplicação no Cloud Run. Passados dois ciclos, as causas em ordem: (1) o estudante está **antes** da decisão — o *polling* só roda de `decisao` a `percurso`, e o gate mexe no TETO do passo, então quem ainda audita a máquina não é tocado **de propósito**; (2) o estudante não tem turma (sem `turma_id` não há gate, senão ele ficaria preso sem quem o solte); (3) a aba está em segundo plano e o navegador desacelerou o *timer*. Confirme com `SELECT etapa FROM exp08ef4b_gates_turma WHERE turma_id = ...`: se está `debate` e a tela não congelou, é uma das três — não é o botão.
O estudante da exp08ef4b diz que "não deixa escrever" a justificativa	Dois casos, e o código HTTP separa. **422 `dado-pessoal-no-texto`**: o filtro achou telefone, CPF, e-mail, CEP, cartão ou `@perfil` e recusou **sem gravar** — é comportamento, não defeito, e a dica na tela diz o que tirar. **400**: o texto tem menos de 20 caracteres depois do `trim`, ou as palavras-chave não são exatamente duas distintas. O log leva o *nome* do padrão que casou (`padroes: ["telefone"]`) e o **tamanho** do texto, nunca o trecho — procurar o conteúdo no log é procurar o que a regra existe para não guardar.

Sintoma	Causa provável
A turma toda da exp08ef4b escolheu a mesma cláusula — o painel está quebrado?	Não. **As quatro cláusulas são válidas** e a distribuição é contagem, não acerto: unanimidade é um resultado possível, e o guia da aula trata dela como o cenário em que o debate quase não acontece (a mediação é pedir o CUSTO da cláusula escolhida). Se você está procurando percentual de acerto no painel, ele não existe por decisão — `PROIBIDO_NO_PAINEL` tem o teste que cobra a ausência de `acerto` e `taxa`.
O professor gerou o conteúdo com IA e veio o texto padrão da Unooria	É o *fallback* funcionando, e a tela diz qual pacote está em uso ("Utilizando pacote curado padrão da Unooria"). Três causas: quota de IA do perfil professor esgotada no dia, provedor indisponível, ou a saída reprovada pela validação (balão acima de 60 palavras, termo proibido em tela, cardinalidade errada) — e reprovar faz o modelo regerar até esgotar as tentativas. Procure `codigo: exp08ef4b-preparo-falhou` ou `exp08ef4b-preparo-sem-saldo` no log. **Nada disso bloqueia a aula**: o banco curado é homologado, e o professor pode aprová-lo como está.
A suíte diz que a recusa de *login* não aparece na tela — e ela aparece	A espera era CEGA. O passo esperava 3s fixos depois do clique e então lia a tela; em 2026-09-10 o `POST /api/auth/callback/credentials` levou **4,5s**, porque era a primeira vez que o `next dev` compilava `/api/auth/[...nextauth]`, e a foto pegou o botão ainda em "Entrando…". O relatório então acusou o PRODUTO — "a recusa não aparece, e não só no console" — quando o defeito era da espera. Confirme no log do hub: `POST /api/auth/callback/credentials? 200 in 4.5s`; com a rota quente a recusa aparece em 333ms (medido no mesmo dia). O conserto é o da espera que corre contra o erro: espere o DESFECHO — o `AlertBox` de erro OU a saída do `/login` — e nunca o relógio. Note que aqui a espera cega era curta DEMAIS, o inverso do caso comum em que ela paga o prazo inteiro: os dois reportam a causa errada, e este reporta uma causa que culpa o produto.
***Spans* descartados com `motivo=sem-projeto` — e `GCP_PROJECT_ID` ESTÁ no serviço**	A *env* está certa: o que falta é o `ClienteOtlpGoogle` chegar ao construtor do `ProxyOtlpService`. Ele era importado com `import type`, o identificador sai do escopo de valor, o `emitDecoratorMetadata` grava `Function` no `design:paramtypes`, o Nest não acha provider para esse *token* — e o `@Optional()` injeta `undefined` sem erro nenhum. O ramo comentado como "dev sem projeto" vira o ÚNICO que roda, a rota responde **204**, e o Cloud Trace nunca recebe span de navegador. Vigorou de 2026-09-02 a 2026-09-14, em *dev* e *hml* ao mesmo tempo. Verificação de um comando: procure `design:paramtypes` no `otlp-proxy.service.js` de `dist` — o segundo tipo tem de ser `ClienteOtlpGoogle`, nunca `Function`. A regra que fica: em NestJS, classe usada como *token* de injeção NUNCA entra por `import type`, e `@Inject` explícito tira o *token* das mãos do compilador. Rede: `apps/api/tests/observabilidade-injecao.spec.ts` — e note que as suítes antigas não pegavam, porque faziam `new ProxyOtlpService(...)`, que não passa pelo *container*.

Sintoma	Causa provável
***Spans* com `motivo=upstream` logo após consertar o `sem-projeto`**	Leia como PROGRESSO antes de ler como defeito: `upstream` só é contado quando o cliente EXISTE e a chamada ao Google falhou — ou seja, a injeção voltou a funcionar. A causa medida em 2026-09-15 foi `400 Resource is missing required attribute`, sobre `gcp.project_id`: o `reescreverLote` monta o resource do ZERO (o do navegador é descartado inteiro, por ser entrada não confiável) e não repetia o atributo que `recurso.ts` já punha nos sinais do processo. **Um defeito escondia o outro** — enquanto o cliente nunca era injetado, o lote não chegava a ser enviado e o 400 não tinha como aparecer. Hoje o projeto vem do próprio cliente (`ClienteOtlpGoogle.projeto`), que é a mesma fonte do cabeçalho de cota. Quem responde é `codigo: otlp.upstream-recusou`, com `status` e `detalhe`, e ela sai **uma vez por minuto** — não uma por lote, para o upstream fora do ar não virar inundação de log cobrada por GiB.
Rajada de `motivo=sem-identidade`, em UM ambiente só	O `service.version` que o navegador manda não passa no formato que a api daquele ambiente aceita, e o lote inteiro é recusado antes de qualquer *span* ser contado (o contador soma 1 por LOTE, não por *span*). O caso medido é de ambiente ATRASADO: até `cf9489f` (2026-09-08) a api validava a versão com `{1,32}` enquanto o CI injetava `NEXT_PUBLIC_HUB_VERSAO` com o `github.sha` INTEIRO, de 40 caracteres — todo lote do hub caía, com a rota respondendo **204**. Em `hml`, parada em código de 05/09, isso rendeu 23 lotes descartados em 2026-09-14. Antes de procurar defeito, confira em que *commit* o ambiente está (`gcloud run revisions list`) e compare com `origin/dev`: o conserto de um ambiente atrasado é a PROMOÇÃO, não código.
O *deploy* de um ambiente falhou — e a pergunta certa é o que já SUBIU	Os passos são independentes, então a falha de um deixa os outros aplicados: em 2026-09-15 o `deploy-hub` de `hml` morreu e o ambiente ficou dez minutos com **banco migrado e api nova servindo um hub VELHO** — numa promoção que renomeava papéis, e o hub carrega a própria cópia do vocabulário. `ativar-exps` e `smoke` foram PULADOS, então nada avisou. Antes de qualquer coisa: `gh run view <id> --json jobs`. Depois decida pela CAUSA: falha de infraestrutura (credencial, rede, *timeout* do *runner*) se resolve com `gh run rerun <id> --failed`, que refaz também os pulados; falha de código, teste ou *migration* exige correção em `dev` e promoção nova — re-rodar só repete. Se nem isso resolver, o *rollback* é por CAMADA: revisão anterior no Cloud Run e `versao_ativa` das experiências pelo painel da Unooria.

Sintoma	Causa provável
`error getting credentials` no *push* da imagem	Parece permissão e NÃO é: é o *credential helper* do Docker falhando sozinho. O CI autenticava com `gcloud auth configure-docker`, que registra o `gcloud` como *helper* e o faz rodar a cada *push*; em 2026-09-15 ele saiu com `exit status 1` e saída VAZIA depois de 22s subindo camadas, enquanto o `deploy-api`, com a mesma configuração e no mesmo *run*, passou. Desde então os dois *jobs* de imagem usam `token_format: access_token` no `auth@v2` e entregam o *token* ao `docker/login-action` — um processo a menos no caminho do *push*. Se voltar a acontecer, é transitório: re-rode o *job*.
O chat do agente de IA do professor responde "O agente de IA do professor não respondeu agora" a toda mensagem, e abrir conversa nova dá 503	Antes de procurar defeito: `AGENTE_PROFESSOR_MODO=desligado`. É o PADRÃO da api (ausente = `desligado`, em qualquer ambiente) e é o que o Terraform põe enquanto o Agent Engine não existe — ele só nasce no segundo *apply*, depois de o CI publicar a imagem (Capítulo 15). Com o modo certo, a linha `agente-professor.indisponivel` traz o *status*: 403 é a SA da api sem o papel no engine; sem *status*, o ADC (em *dev*, `gcloud auth application-default login` e reiniciar a api).
A pergunta fora do escopo não mostra resposta nenhuma: o balão do agente fica vazio e o turno termina	A recusa NÃO vem em pedaços: é um evento final único, sem parcial antes (não houve modelo principal), com `custom_metadata.fora_do_escopo`. A api tem de emiti-la como `delta` — o `DeduplicadorDeTexto` passa o final inteiro quando não houve parcial. Quem "simplifica" o dedupe para repassar só os parciais apaga a recusa da tela, sem erro em lugar nenhum. O sintoma inverso, a resposta aparecendo DUAS vezes, é o mesmo arquivo: o final agregado do ADK repete o texto dos parciais.
O professor cola um trecho de SQL, de HTML ou de código na pergunta e a mensagem falha como "falha de rede", sem linha no log da api	Cloud Armor. A exclusão da regra 1106 tira só a 921150 (CR/LF) dos caminhos do chat; os conjuntos de SQLi, XSS e os demais continuam valendo, e o 403 do LB não carrega cabeçalho de CORS — o navegador reporta falha de rede, e a requisição morre antes do Cloud Run. Confira no log do LB (`enforcedSecurityPolicy`) qual assinatura casou. Antes da 1106, bastava um Shift+Enter para dar o mesmo sintoma.
Uma conversa some da lista do professor sozinha	A sessão expirou no engine: o `GET` da conversa recebe "não existe", apaga a linha de `agente_professor_conversas` e responde 404 (a linha `agente-professor.sessao-sumiu`). Acontece se o engine recusou o `expire_time` pedido e a conversa ficou mais que o TTL padrão sem mensagem, ou se a renovação vem falhando (`agente-professor.renovacao-falhou`: a SA da api sem `sessions.update`).
Apagar uma conversa do agente dá erro e ela continua na lista	De propósito: com o engine fora do ar, o `DELETE` responde 503 e a linha FICA, porque apagar só a linha deixaria a conversa guardada no engine sem ninguém capaz de alcançá-la. Volte quando o engine responder.

F.7 *Checklist* de operação

Frequência	Item
Contínuo (fase 0 do Capítulo 17, **pendente**)	Os quatorze alertas por sintoma e os cinco *uptime checks* tornam as linhas “semanal” e “mensal” abaixo confirmação, não descoberta. Até lá, elas são o único mecanismo.
Após cada *deploy*	Conferir que a migração rodou (`--wait` no *job*); abrir `/readyz`; abrir uma experiência de estudante e uma ferramenta de professor. Com a fase 0: o *smoke* do `deploy.yml` faz os três e notifica se falhar.
Antes da primeira aula de cada contrato	Conferir `ASSETS_BUCKET` e o `ASSETS_DRIVER=gcs` do *environment*. Nada falha se estiverem errados, e o *deploy* fica verde: o precedente é a exp04ef1b, que subiu com o *bucket* vazio e ficou impossível de concluir.
Semanal, e após *deploy*	Error Reporting: erro **novo** na revisão que acabou de receber tráfego, e erro de cliente novo por `(expId, versao)` — reverter pelo manifesto antes de investigar.
Semanal	Taxa de 5xx do Cloud Run; conexões ativas do Cloud SQL; contagem de `status = 'aberta'` em `app_70_sessoes` (partidas perdidas); planos em `status = 'erro'`.
Mensal	**Contagem da partição `DEFAULT` de telemetria** — encher significa que faltou criar partição (`R-08`); volume de `telemetria_eventos` por mês; consumo de quota de IA por professor.
Trimestral	`pnpm up -r @unooria/design-system --latest` e rodar os suítes de ponta a ponta; revisar as lacunas do *design system* ainda compensadas localmente; conferir se os modelos de IA em uso continuam no catálogo (`R-12`).
Anual, ou por *release*	Revisar as ADRs cujo gatilho de reavaliação pode ter sido acionado (Capítulo 18).
Sob suspeita de que o catálogo de uma *stack* tenha sido escrito por engano	`apps/api/scripts/conferir-catalogo-remoto.mjs` — **só leitura**, com a receita do *job* efêmero no próprio cabeçalho (o Cloud SQL só tem IP privado). Ele procura as impressões digitais concretas de *fixture* de teste (`nome = exp_id`, `exp-99-%`, `escola LIKE 'teste-%'`) e imprime o catálogo, as disponibilidades e o histórico do painel. **Não existe migração de conserto aqui, de propósito**: o valor certo mora no `public/experiencia.json`, e o *deploy* já o reaplica — uma migração que “restaurasse” o catálogo gravaria valores inventados por quem a escreveu, e o sintoma disso é pior que o do problema (uma experiência sai da grade de uma série sem ninguém ter mudado nada).

Listing F.8 – As consultas do *checklist*

```sql
-- A particao DEFAULT esta enchendo? (deveria ser sempre 0)
SELECT count(*) FROM particoes.telemetria_eventos_default;

-- Partidas perdidas (aba fechada no meio) por turma
SELECT turma_id, count(*) FROM app_70_sessoes
 WHERE status = 'aberta' AND aberta_em < now() - interval '1 day'
 GROUP BY turma_id ORDER BY 2 DESC;
```

```
-- Planos que falharam
SELECT id, serie, disciplina, tentativas, erro FROM app_90_planos
 WHERE status = 'erro' ORDER BY updated_at DESC;

-- Consumo de IA no dia
SELECT usuario_id, tokens_usados FROM ia_uso_diario
 WHERE dia = to_char(now(), 'YYYY-MM-DD') ORDER BY 2 DESC LIMIT 20;

-- Volume de telemetria por mes (nome da particao ja diz o mes)
SELECT relname, n_live_tup FROM pg_stat_user_tables
 WHERE schemaname = 'particoes' ORDER BY relname;
```

ATENÇÃO

Toda consulta manual acima roda **fora** de `comTenancy`, ou seja, com a *role* de aplicação a RLS a filtraria para nada. Diagnóstico em produção usa conexão administrativa — e é por isso que a *role* de aplicação **não pode** ter `BYPASSRLS`: a separação entre "a aplicação" e "um humano diagnosticando" tem de ser real.

F.8 Compilar esta documentação

```
cd documentacao-latex
./compilar.sh              # build completo (Tectonic: baixa o que falta e roda o BibTeX)
./compilar.sh --rapido     # uma passagem, para iterar na escrita
./compilar.sh --limpar     # remove os auxiliares

# Com uma distribuicao de TeX completa instalada, alternativamente:
latexmk -pdf main.tex
```

Requisito	Detalhe
Motor	**Tectonic** (recomendado: baixa os pacotes sob demanda e chama o BibTeX sozinho) *ou* qualquer distribuição com `pdflatex` + `bibtex`.
Fontes	`roboto` e `roboto-mono`.
Diagramas	`tikz` e `pgfplots` — todos vetoriais, sem renderização externa.
Bibliografia	`abntex2cite` (estilo `alf`) processado por **BibTeX** — *não* biblatex/biber, para que o Tectonic feche o ciclo sem ferramenta externa.

REGRA INVIOLÁVEL

Atualizar esta documentação faz parte da *Definition of Done* (Capítulo 16). Implementação que mude arquitetura, contrato, modelo de dados, decisão ou risco atualiza o capítulo correspondente **e regera o `main.pdf` no mesmo *commit***. A regra está registrada em `AGENTS.md`, não apenas aqui.

Referências Bibliográficas

Advanced Distributed Learning Initiative. *Experience API (xAPI) Specification, version 2.0*. 2023. <https://github.com/adlnet/xAPI-Spec>. Acesso em: 27 ago. 2026.

Autoridade Nacional de Proteção de Dados. *Guia orientativo: tratamento de dados pessoais de crianças e adolescentes*. 2024. ANPD, Brasília. Acesso em: 27 ago. 2026.

BEYER, B. et al. *Site Reliability Engineering: How Google Runs Production Systems*. Sebastopol: O'Reilly Media, 2016.

BEYER, B. et al. *The Site Reliability Workbook: Practical Ways to Implement SRE*. Sebastopol: O'Reilly Media, 2018.

Brasil. *Lei n. 13.709, de 14 de agosto de 2018 — Lei Geral de Proteção de Dados Pessoais (LGPD)*. 2018. Diário Oficial da União, Brasília.

BRIKMAN, Y. *Terraform: Up & Running — Writing Infrastructure as Code*. 3. ed. Sebastopol: O'Reilly Media, 2022.

BROWN, S. *Software Architecture for Developers: Volume 2 — Visualise, document and explore your software architecture*. Victoria: Leanpub, 2024.

BROWN, S. *The C4 model for visualising software architecture*. 2026. <https://c4model.com>. Acesso em: 27 ago. 2026.

DöRING, A. K. et al. Assessing values at an early age: The picture-based value survey for children (pbvs-c). *Journal of Personality Assessment*, v. 92, n. 5, p. 439–448, 2010.

Google Cloud. *Deploy the Google-Built OpenTelemetry Collector on Cloud Run*. 2026. <https://cloud.google.com/stackdriver/docs/instrumentation/opentelemetry-collector-cloud-run>. Acesso em: 1 set. 2026.

Google Cloud. *Google Cloud Observability Documentation: Cloud Logging structured logs, Cloud Monitoring alerting and SLOs, Cloud Trace, Error Reporting*. 2026. <https://cloud.google.com/stackdriver/docs>. Acesso em: 1 set. 2026.

Google Cloud. *Google Cloud Observability pricing*. 2026. <https://cloud.google.com/stackdriver/pricing>. Acesso em: 1 set. 2026. Valores de referência: Cloud Logging 50 GiB/projeto/mês gratuitos e US$ 0,50/GiB; métricas 150 MiB gratuitos e US$ 0,258/MiB; Cloud Trace 2,5 milhões de *spans*/mês gratuitos e US$ 0,20/milhão; *uptime checks* 1 milhão de execuções/projeto/mês gratuitas; políticas de alerta US$ 0,35 por referência de métrica por mês, com cobrança anunciada para não antes de setembro de 2026.

International Organization for Standardization. *ISO/IEC/IEEE 42010: Systems and software engineering — Architecture description*. Genebra, 2022.

International Organization for Standardization. *ISO/IEC 25010: Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model*. Genebra, 2023.

KRUCHTEN, P. The 4+1 view model of architecture. *IEEE Software*, v. 12, n. 6, p. 42–50, 1995.

LOVIBOND, P. F.; LOVIBOND, S. H. The structure of negative emotional states: Comparison of the depression anxiety stress scales (dass) with the beck depression and anxiety inventories. *Behaviour Research and Therapy*, v. 33, n. 3, p. 335–343, 1995.

MAJORS, C.; FONG-JONES, L.; MIRANDA, G. *Observability Engineering: Achieving Production Excellence*. Sebastopol: O'Reilly Media, 2022.

MCDONNELL, C. *Zod: TypeScript-first schema validation with static type inference*. 2026. <https://zod.dev>. Acesso em: 27 ago. 2026.

Mozilla Developer Network. *The iframe sandbox attribute — HTML*. 2026. <https://developer.mozilla.org/en-US/docs/Web/HTML/Element/iframe>. Acesso em: 27 ago. 2026.

Mozilla Developer Network. *Window.postMessage() — Web APIs*. 2026. <https://developer.mozilla.org/en-US/docs/Web/API/Window/postMessage>. Acesso em: 27 ago. 2026.

NestJS. *NestJS Documentation: Guards, Interceptors and Modules*. 2026. <https://docs.nestjs.com>. Acesso em: 27 ago. 2026.

NYGARD, M. *Documenting Architecture Decisions*. 2011. <https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions>. Acesso em: 27 ago. 2026.

Object Management Group. *OMG Unified Modeling Language (OMG UML), Version 2.5.1*. Needham, MA, 2017. Documento formal/17-12-05. Disponível em: <https://www.omg.org/spec/UML/2.5.1>. Acesso em: 27 set. 2026.

OpenTelemetry Authors. *OpenTelemetry Documentation: signals, SDKs, sampling and the Collector*. 2026. <https://opentelemetry.io/docs/>. Acesso em: 1 set. 2026.

OWASP Foundation. *OWASP Top 10: 2021 — The Ten Most Critical Web Application Security Risks*. 2021. <https://owasp.org/Top10/>. Acesso em: 27 ago. 2026.

SCHWARTZ, S. H. An overview of the schwartz theory of basic values. *Online Readings in Psychology and Culture*, v. 2, n. 1, 2012.

The PostgreSQL Global Development Group. *PostgreSQL Documentation: Row Security Policies*. 2026. <https://www.postgresql.org/docs/current/ddl-rowsecurity.html>. Acesso em: 27 ago. 2026.

The PostgreSQL Global Development Group. *PostgreSQL Documentation: Table Partitioning*. 2026. <https://www.postgresql.org/docs/current/ddl-partitioning.html>. Acesso em: 27 ago. 2026.

Vercel. *Next.js Documentation: App Router, Server Components and Middleware*. 2026. <https://nextjs.org/docs>. Acesso em: 27 ago. 2026.

Vercel. *Turborepo Documentation: task pipelines, filtering and remote caching*. 2026. <https://turborepo.com/docs>. Acesso em: 27 ago. 2026.

VIGNOLA, R. C. B.; TUCCI, A. M. Adaptation and validation of the depression, anxiety and stress scale (dass) to brazilian portuguese. *Journal of Affective Disorders*, v. 155, p. 104–109, 2014.

W3C. *Trace Context — W3C Recommendation*. 2021. <https://www.w3.org/TR/trace-context/>. Acesso em: 1 set. 2026.